

Northstar Retail Complete Manual Build Workbook

Learner: Kirolos Boules

Required platform path: Track A local-first batch architecture

Repository: <https://github.com/Losboules/SQL-Data-Warehouse-Project>

Workbook status: Instructional source of truth. No project execution is claimed.

Generated for use on: Windows

Current-source access date: August 16, 2026

How to use this workbook

This single Markdown file is the course, construction manual, exercise book, reference-answer book, validation guide, troubleshooting guide, Git guide, learning journal, progress tracker, and portfolio/interview guide. The project folders and files are created manually by you in your repository while following it. Code shown here is teaching material or an optional reference answer; it is not evidence that you typed, ran, tested, committed, or verified it.

Use one status only:

- ☐ Not started
- ☐ In progress
- ☒ Verified by me
- ☐ Needs troubleshooting

Also use these ownership labels:

- **PLANNED:** The workbook specifies the work.
- **CREATED BY USER:** You created the file/object but have not necessarily run or validated it.
- **RUN BY USER:** You executed it and preserved the actual result.
- **VERIFIED BY USER:** You independently inspected the required result and passed the gate.
- **NEEDS TROUBLESHOOTING:** The result differs or a gate failed.

Never change a status to verified because the reference answer looks correct. Complete the **MY TASK** section before opening the optional answer. Use hints in order: concept, pseudocode, then near-complete pattern.

Master table of contents

- [How to use this workbook](#)
- [Master progress tracker](#)
- [Major-concept three-level primer](#)
- [Reference-answer navigation](#)
- [Project architecture and ownership](#)
- [Official source register](#)
- [Windows path primer](#)
- [Phase 0 — Project orientation and the end-to-end mental model](#)
 - [Lesson 0.1](#)
 - [Lesson 0.2](#)
 - [Lesson 0.3](#)
 - [Lesson 0.4](#)
 - [Lesson 0.5](#)
 - [Optional Phase 0 reference answer](#)
- [Phase 1 — Repository inspection, preservation, branching, and preparation](#)
 - [Lesson 1.1](#)
 - [Lesson 1.2](#)
 - [Lesson 1.3](#)
 - [Lesson 1.4](#)
 - [Lesson 1.5](#)
 - [Optional Phase 1 reference answer](#)
- [Phase 2 — Install and verify the Windows toolchain](#)
 - [Lesson 2.1](#)
 - [Lesson 2.2](#)
 - [Lesson 2.3](#)
 - [Lesson 2.4](#)
 - [Lesson 2.5](#)
 - [Optional Phase 2 reference answer](#)
- [Phase 3 — Write business requirements and acceptance criteria](#)
 - [Lesson 3.1](#)
 - [Lesson 3.2](#)
 - [Lesson 3.3](#)
 - [Lesson 3.4](#)
 - [Lesson 3.5](#)
 - [Optional Phase 3 reference answer](#)
- [Phase 4 — Design and create the SQL Server and PostgreSQL source systems](#)
 - [Lesson 4.1](#)
 - [Lesson 4.2](#)
 - [Lesson 4.3](#)
 - [Lesson 4.4](#)
 - [Lesson 4.5](#)
 - [Optional Phase 4 reference answer](#)
- [Phase 5 — Build and validate the synthetic-data generator](#)
 - [Lesson 5.1](#)
 - [Lesson 5.2](#)
 - [Lesson 5.3](#)
 - [Lesson 5.4](#)
 - [Lesson 5.5](#)
 - [Optional Phase 5 reference answer](#)

- Phase 6 — Load and verify the local source databases
 - Lesson 6.1
 - Lesson 6.2
 - Lesson 6.3
 - Lesson 6.4
 - Lesson 6.5
 - Optional Phase 6 reference answer
- Phase 7 — Build full and incremental local extraction
 - Lesson 7.1
 - Lesson 7.2
 - Lesson 7.3
 - Lesson 7.4
 - Lesson 7.5
 - Optional Phase 7 reference answer
- Phase 8 — Configure Databricks, governed storage, notebooks, and compute
 - Lesson 8.1
 - Lesson 8.2
 - Lesson 8.3
 - Lesson 8.4
 - Lesson 8.5
 - Optional Phase 8 reference answer
- Phase 9 — Build the Bronze ingestion layer
 - Lesson 9.1
 - Lesson 9.2
 - Lesson 9.3
 - Lesson 9.4
 - Lesson 9.5
 - Optional Phase 9 reference answer
- Phase 10 — Build the Silver cleaning and conformance layer
 - Lesson 10.1
 - Lesson 10.2
 - Lesson 10.3
 - Lesson 10.4
 - Lesson 10.5
 - Optional Phase 10 reference answer
- Phase 11 — Build Gold dimensions, surrogate keys, and SCD behavior
 - Lesson 11.1
 - Lesson 11.2
 - Lesson 11.3
 - Lesson 11.4
 - Lesson 11.5
 - Optional Phase 11 reference answer
- Phase 12 — Build Gold fact tables and reconciled business measures
 - Lesson 12.1
 - Lesson 12.2
 - Lesson 12.3
 - Lesson 12.4
 - Lesson 12.5
 - Optional Phase 12 reference answer
- Phase 13 — Build data-quality, reconciliation, and failure gates
 - Lesson 13.1

- Lesson 13.2
- Lesson 13.3
- Lesson 13.4
- Lesson 13.5
- Optional Phase 13 reference answer
- Phase 14 — Create and operate a multi-task Databricks Job
 - Lesson 14.1
 - Lesson 14.2
 - Lesson 14.3
 - Lesson 14.4
 - Lesson 14.5
 - Optional Phase 14 reference answer
- Phase 15 — Publish Gold data into the SQL Server dimensional warehouse
 - Lesson 15.1
 - Lesson 15.2
 - Lesson 15.3
 - Lesson 15.4
 - Lesson 15.5
 - Optional Phase 15 reference answer
- Phase 16 — Build and validate the Power BI semantic model
 - Lesson 16.1
 - Lesson 16.2
 - Lesson 16.3
 - Lesson 16.4
 - Lesson 16.5
 - Optional Phase 16 reference answer
- Phase 17 — Build and validate all Power BI report pages
 - Lesson 17.1
 - Lesson 17.2
 - Lesson 17.3
 - Lesson 17.4
 - Lesson 17.5
 - Optional Phase 17 reference answer
- Phase 18 — Secure secrets, control cost, and review operational settings
 - Lesson 18.1
 - Lesson 18.2
 - Lesson 18.3
 - Lesson 18.4
 - Lesson 18.5
 - Optional Phase 18 reference answer
- Phase 19 — Build GitHub Actions continuous integration
 - Lesson 19.1
 - Lesson 19.2
 - Lesson 19.3
 - Lesson 19.4
 - Lesson 19.5
 - Optional Phase 19 reference answer
- Phase 20 — Build the README, evidence package, and portfolio presentation
 - Lesson 20.1
 - Lesson 20.2
 - Lesson 20.3

- [Lesson 20.4](#)
- [Lesson 20.5](#)
- [Optional Phase 20 reference answer](#)
- [Phase 21 — Prepare interview explanations and defend every design choice](#)
 - [Lesson 21.1](#)
 - [Lesson 21.2](#)
 - [Lesson 21.3](#)
 - [Lesson 21.4](#)
 - [Lesson 21.5](#)
 - [Optional Phase 21 reference answer](#)
- [Phase 22 — Evaluate optional extensions without disrupting the required path](#)
 - [Lesson 22.1](#)
 - [Lesson 22.2](#)
 - [Lesson 22.3](#)
 - [Lesson 22.4](#)
 - [Lesson 22.5](#)
 - [Optional Phase 22 reference answer](#)
- [Complete glossary](#)
- [Troubleshooting index](#)
- [Reference-code and file responsibility index](#)
- [Final definition of done](#)
- [Final portfolio and interview audit](#)

Master progress tracker

Phase	Milestone	Status	Evidence path/link	Git commit	Teach-back complete?
0	Project orientation and the end-to-end mental model	[] Not started	[ENTER]	[ENTER]	[]
1	Repository inspection, preservation, branching, and preparation	[] Not started	[ENTER]	[ENTER]	[]
2	Install and verify the Windows toolchain	[] Not started	[ENTER]	[ENTER]	[]
3	Write business requirements and acceptance criteria	[] Not started	[ENTER]	[ENTER]	[]
4	Design and create the SQL Server and PostgreSQL source systems	[] Not started	[ENTER]	[ENTER]	[]
5	Build and validate the synthetic-data generator	[] Not started	[ENTER]	[ENTER]	[]
6	Load and verify the local source databases	[] Not started	[ENTER]	[ENTER]	[]
7	Build full and incremental local extraction	[] Not started	[ENTER]	[ENTER]	[]
8	Configure Databricks, governed storage, notebooks, and compute	[] Not started	[ENTER]	[ENTER]	[]
9	Build the Bronze ingestion layer	[] Not started	[ENTER]	[ENTER]	[]
10	Build the Silver cleaning and conformance layer	[] Not started	[ENTER]	[ENTER]	[]
11	Build Gold dimensions, surrogate keys, and SCD behavior	[] Not started	[ENTER]	[ENTER]	[]

Phase	Milestone	Status	Evidence path/link	Git commit	Teach-back complete?
12	Build Gold fact tables and reconciled business measures	[] Not started	[ENTER]	[ENTER]	[]
13	Build data-quality, reconciliation, and failure gates	[] Not started	[ENTER]	[ENTER]	[]
14	Create and operate a multi-task Databricks Job	[] Not started	[ENTER]	[ENTER]	[]
15	Publish Gold data into the SQL Server dimensional warehouse	[] Not started	[ENTER]	[ENTER]	[]
16	Build and validate the Power BI semantic model	[] Not started	[ENTER]	[ENTER]	[]
17	Build and validate all Power BI report pages	[] Not started	[ENTER]	[ENTER]	[]
18	Secure secrets, control cost, and review operational settings	[] Not started	[ENTER]	[ENTER]	[]
19	Build GitHub Actions continuous integration	[] Not started	[ENTER]	[ENTER]	[]
20	Build the README, evidence package, and portfolio presentation	[] Not started	[ENTER]	[ENTER]	[]
21	Prepare interview explanations and defend every design choice	[] Not started	[ENTER]	[ENTER]	[]
22	Evaluate optional extensions without disrupting the required path	[] Not started	[ENTER]	[ENTER]	[]

Course-wide evidence rules

1. A screenshot proves only the visible state and filter/context shown.
2. A green test proves only the checks that actually ran against the data/configuration they actually read.
3. A Git commit proves the repository files existed at that snapshot; it does not prove a database or cloud object existed.
4. A database row count proves the selected table and filter returned that count; it does not prove upstream reconciliation.
5. A successful push proves commits reached the remote branch; it does not prove review, CI, or merge.
6. Never place passwords, tokens, connection strings, personal data, or account IDs in this workbook, screenshots, Git, logs, notebook output, or Power BI pages.

Major-concept three-level primer

Read this primer before the architecture walkthrough. Each concept is defined in plain language first, then tied to Northstar Retail, then stated in professional interview language. The first sentence under each concept is its definition; the everyday analogy follows immediately.

Source systems

Level 1 — Child-simple explanation

A **source system** is the place where information is first recorded while real work happens. It is like the original notebook where a cashier writes a sale before anyone makes a report about it.

Level 2 — Northstar Retail project explanation

Northstar uses SQL Server for ERP/store operations, PostgreSQL for web and marketing activity, and CSV/JSON files for partner-style feeds. These sources preserve operational business keys and events before the data is copied into analytical layers.

Level 3 — Professional explanation

Source systems are authoritative operational producers with their own schemas, transaction patterns, ownership, latency, and data-quality limits. The pipeline treats each source as a separate contract and does not redesign it merely for reporting convenience.

ETL and ELT

Level 1 — Child-simple explanation

ETL means extract, transform, then load; **ELT** means extract, load, then transform. ETL is like washing vegetables before placing them in the refrigerator; ELT is like storing them first and preparing them in the kitchen later.

Level 2 — Northstar Retail project explanation

Python extracts SQL Server, PostgreSQL, CSV, and JSON data into traceable batches. Databricks first lands raw evidence in Bronze and then transforms it into Silver and Gold, so the lakehouse portion follows an ELT-style pattern while local export preparation also performs controlled extraction work.

Level 3 — Professional explanation

ETL and ELT describe where transformation occurs relative to loading. Northstar uses a hybrid batch design: source-aware extraction and validation locally, immutable raw landing, and scalable transformation inside Databricks with auditable layer boundaries.

Medallion architecture

Level 1 — Child-simple explanation

A **medallion architecture** separates data into Bronze, Silver, and Gold levels. It is like keeping the original receipt, making a corrected copy, and then making a clean summary for the manager.

Level 2 — Northstar Retail project explanation

Bronze keeps source values and ingestion metadata, Silver applies cleaning and conformance rules while retaining rejected rows, and Gold creates dimensions, facts, and business-ready measures. Each layer has its own checks and rerun behavior.

Level 3 — Professional explanation

Medallion architecture is a layered lakehouse pattern that progressively increases trust and semantic structure. It separates raw lineage, conformed detail, and consumption-ready models so failures, schema changes, and reconciliations can be isolated at explicit contracts.

Data warehousing

Level 1 — Child-simple explanation

A **data warehouse** is a place arranged for answering questions about history rather than running the store minute by minute. It is like a well-organized record room instead of the cash register currently serving customers.

Level 2 — Northstar Retail project explanation

NorthstarRetail_DW receives verified Gold exports and stores staging tables, dimensions, facts, audit records, and analyst-facing views. Power BI reads this warehouse instead of joining the operational databases directly.

Level 3 — Professional explanation

Data warehousing creates a stable analytical boundary with historical integration, conformed definitions, dimensional structures, controlled loading, and repeatable reconciliation. It decouples reporting workloads and semantics from source-system transaction design.

Dimensional modeling

Level 1 — Child-simple explanation

Dimensional modeling puts measurable events in fact tables and descriptive labels in dimension tables. It is like keeping receipts in one box and customer, product, store, and calendar cards in labeled boxes around it.

Level 2 — Northstar Retail project explanation

Northstar creates nine dimensions and six facts. Sales, returns, inventory snapshots, shipments, web activity, and marketing spend each have a declared row meaning and connect to descriptive dimensions through warehouse-controlled keys.

Level 3 — Professional explanation

Dimensional modeling organizes analytical data around declared fact grains, conformed dimensions, surrogate-key relationships, additive behavior, and query-friendly star schemas. It makes business definitions explicit and supports predictable BI filtering.

Table grain

Level 1 — Child-simple explanation

Grain means exactly what one row represents. Saying “one row is one order item” is like saying every line on a receipt represents one purchased product, not the whole shopping trip.

Level 2 — Northstar Retail project explanation

fact_sales has one row per valid order item, while fact_inventory_snapshot has one row per product-store-snapshot date. Keys, duplicate tests, and measures are correct only when they match that declared grain.

Level 3 — Professional explanation

Table grain is the atomic business meaning of a row and must be declared before columns, keys, aggregations, or joins are designed. Grain drives uniqueness constraints, additive behavior, reconciliation logic, and late-arriving-key handling.

Surrogate keys

Level 1 — Child-simple explanation

A **surrogate key** is a warehouse-made identifier with no business meaning. It is like giving every library card a new internal number even when two outside systems use different customer numbers.

Level 2 — Northstar Retail project explanation

Gold dimensions assign integer keys to customer, product, store, employee, supplier, promotion, channel, campaign, and date rows. Facts store those keys so source identifiers can change or overlap without breaking warehouse relationships.

Level 3 — Professional explanation

Surrogate keys decouple analytical identity from mutable or heterogeneous natural keys. They support slowly changing history, compact fact relationships, unknown-member handling, and consistent integration across source systems.

SCD Type 2

Level 1 — Child-simple explanation

Slowly Changing Dimension Type 2 keeps an old version and adds a new version when a history-worthy description changes. It is like keeping both an old and a new address card with dates showing when each was valid.

Level 2 — Northstar Retail project explanation

Northstar uses SCD Type 2 for selected customer and product attributes. A changed record expires the prior current row, inserts a new effective-dated row, and lets facts resolve the version valid at the event time.

Level 3 — Professional explanation

SCD Type 2 preserves historical attribute truth through multiple surrogate-keyed versions per natural key, effective start/end timestamps, and a current-row indicator. Correct implementation requires change detection, non-overlapping validity windows, and idempotent reruns.

Incremental loading

Level 1 — Child-simple explanation

An **incremental load** processes only new or changed information after an accepted point. It is like reading only the pages added to a diary since the last bookmark instead of rereading the whole diary.

Level 2 — Northstar Retail project explanation

Northstar records a high-water mark and extracts rows after the last successful boundary, with a tie-breaking key where timestamps can repeat. The watermark advances only after the entire batch is accepted and audited.

Level 3 — Professional explanation

Incremental loading limits processing to a deterministic change range using source positions such as timestamps plus keys, change tracking, or CDC. It requires atomic watermark advancement, replay safety, late-data policy, and source-to-target reconciliation.

Idempotency

Level 1 — Child-simple explanation

Idempotency means repeating the same request does not create extra copies or a different final result. It is like pressing an elevator button twice without ordering two elevators.

Level 2 — Northstar Retail project explanation

A repeated Northstar batch ID is recognized through audit records, unique keys, Delta MERGE logic, or controlled replacement. The same accepted batch must not duplicate Bronze rows, dimension versions, facts, or warehouse loads.

Level 3 — Professional explanation

Idempotency is an operational guarantee that replaying the same logical input converges on the intended state. It depends on stable batch/event identifiers, deterministic transformations, transactional audit state, uniqueness rules, and explicit correction semantics.

Data-quality gates

Level 1 — Child-simple explanation

A **data-quality gate** is a check that must pass before data moves forward. It is like a door that stays locked when a package has the wrong label, missing pieces, or an unexplained count.

Level 2 — Northstar Retail project explanation

Northstar tests schema, nulls, keys, accepted/rejected counts, referential integrity, dimensions, fact grain, amounts, freshness, and source-to-target reconciliation. A blocking failure prevents Gold publication or downstream loading.

Level 3 — Professional explanation

Data-quality gates are executable acceptance criteria tied to business and technical risk. They record expected and actual values, severity, batch context, and evidence; blocking rules fail orchestration rather than merely producing a warning nobody reviews.

Orchestration

Level 1 — Child-simple explanation

Orchestration controls which jobs run first, which wait, and what happens after a failure. It is like a conductor telling different musicians when to start and stop.

Level 2 — Northstar Retail project explanation

A Databricks Lakeflow Job passes parameters, runs Bronze tasks, waits for dependencies, builds Silver and Gold, executes quality gates, and publishes only after required checks pass. Retry, timeout, repair, and notification behavior is documented.

Level 3 — Professional explanation

Orchestration coordinates task dependencies, execution context, parameters, retries, timeouts, schedules, observability, and failure propagation. A sound DAG makes data dependencies explicit and keeps repair runs consistent with idempotency rules.

Semantic models

Level 1 — Child-simple explanation

A **semantic model** gives data shared meanings, relationships, and calculations for reports. It is like a labeled map that tells everyone which roads connect and how distances are measured.

Level 2 — Northstar Retail project explanation

Power BI imports the warehouse tables, creates one-to-many relationships, uses a marked date table, hides technical keys, and defines measures such as Total Revenue, Gross Profit, Conversion Rate, and Return on Ad Spend.

Level 3 — Professional explanation

A semantic model is the governed analytical layer of entities, relationships, measures, formats, hierarchies, descriptions, and security consumed by reports. It centralizes business logic and filter behavior so visuals do not redefine metrics independently.

CI/CD

Level 1 — Child-simple explanation

CI/CD means automatically checking changes and, when appropriate, delivering approved changes through a repeatable process. It is like inspecting every part on a conveyor belt before allowing it into the next room.

Level 2 — Northstar Retail project explanation

GitHub Actions checks required files, Python tests, Markdown links, notebooks, SQL dialect expectations, and secret safety without pretending to run the learner's local databases or cloud workspace. Deployment remains limited to what is honestly automated.

Level 3 — Professional explanation

Continuous integration validates each proposed change in a clean runner, while continuous delivery/deployment promotes validated artifacts through controlled environments. Northstar's portfolio scope implements infrastructure-free CI and documents the boundary between repository validation and live-platform deployment.

[Back to the master table of contents](#)

Optional reference-answer navigation

Attempt each **MY TASK** and use the three hints before opening its answer. These links point to in-workbook phase answers; no separate answer-key or appendix file exists.

- [Phase 0 reference answer](#)
- [Phase 1 reference answer](#)
- [Phase 2 reference answer](#)
- [Phase 3 reference answer](#)
- [Phase 4 reference answer](#)
- [Phase 5 reference answer](#)
- [Phase 6 reference answer](#)
- [Phase 7 reference answer](#)
- [Phase 8 reference answer](#)
- [Phase 9 reference answer](#)
- [Phase 10 reference answer](#)
- [Phase 11 reference answer](#)
- [Phase 12 reference answer](#)
- [Phase 13 reference answer](#)
- [Phase 14 reference answer](#)
- [Phase 15 reference answer](#)
- [Phase 16 reference answer](#)
- [Phase 17 reference answer](#)
- [Phase 18 reference answer](#)
- [Phase 19 reference answer](#)
- [Phase 20 reference answer](#)
- [Phase 21 reference answer](#)
- [Phase 22 reference answer](#)

[Back to the master table of contents](#)

Project architecture and ownership

Level 1 — Child-simple architecture

Northstar Retail has information in different boxes. SQL Server holds store/ERP records, PostgreSQL holds web/marketing records, and files bring returns, promotions, costs, and tracking events. Python copies each batch into labeled files. Databricks stores the raw copy in Bronze, cleans it in Silver, and arranges it for reporting in Gold. Verified Gold files are brought back to a local SQL Server warehouse. Power BI reads that warehouse and shows measures and report pages. GitHub stores the instructions and code, not the running databases.

Level 2 — One fictional order through every layer

A fictional customer places an online order. `erp.orders` records the order header and `erp.order_items` records each product line. PostgreSQL web tables record the session, campaign touchpoint, and conversion. Python extracts both sources into one batch with a batch ID, manifest, counts, and checksums. A human uploads that batch to a governed Databricks volume in Track A. Bronze preserves source values and file lineage. Silver parses dates, validates quantities/statuses, deduplicates business keys, and quarantines invalid records. Gold resolves dimension keys and writes one `fact_sales` row per valid order item. Quality gates reconcile counts and amounts. After a successful Job, verified Gold exports are downloaded and loaded into NorthstarRetail_DW. Power BI Import creates relationships and measures such as Total Revenue, Gross Profit, Conversion Rate, and Return on Ad Spend. GitHub records the code, tests, decisions, screenshots, and limitations.

Level 3 — Professional architecture walkthrough

The platform is a heterogeneous, batch-oriented, local-first portfolio implementation. SQL Server and PostgreSQL model normalized operational domains; structured file feeds model partner/operational handoffs. Deterministic Python workflows generate and load synthetic data, then create full and incremental extract batches with watermarks, manifests, checksums, and audit logs. Databricks implements a governed medallion architecture using Delta tables, parameterized notebooks, quality/reconciliation gates, and Lakeflow Jobs. Gold dimensions use surrogate keys and selected SCD Type 1/2 policies; Gold facts enforce declared grains and unknown-key behavior. Track A uses manual governed transfers because a remote workspace cannot automatically connect to a laptop's localhost. A local SQL Server dimensional warehouse provides a stable consumption boundary. Power BI uses Import, a star-schema semantic model, documented DAX, and SQL validation. GitHub Actions performs infrastructure-free repository checks, while live platform evidence remains separately verified.

What exists where

Location	Code/configuration	Data/objects	What it does not contain
Local repository	Python, SQL, YAML, Markdown, tests, workflow, Power BI requirements	Small fictional fixtures and verified screenshots only when appropriate	Running databases or Databricks tables
SQL Server source	T-SQL definitions are in Git	NorthstarRetail_ERP, erp tables, source audit data	PostgreSQL digital tables or Gold lakehouse tables
PostgreSQL source	PostgreSQL SQL definitions are in Git	northstar_digital, digital tables	SQL Server ERP tables
Local extraction folders	Python in Git controls the process	Batch files, manifest, checksum, watermark, logs	A cloud-accessible localhost endpoint
Databricks	Source notebooks/files can be Git-backed	Volume files, Bronze/Silver/Gold/quality Delta tables, Job runs	Automatic access to laptop loopback without networking
SQL Server warehouse	Warehouse DDL/loaders are in Git	NorthstarRetail_DW, staging, dimensions, facts, audits, views	Raw source-system behavior
Power BI Desktop	DAX and model requirements documented in Git	Local semantic model/report file and imported data	Source-of-truth raw data
GitHub	Versioned files and Actions logs	Commits, branches, PRs, issues, evidence images	Secrets, local databases, or unverified claims

Track A manual boundaries

1. Local extraction batch → manual upload to Databricks governed storage.
2. Verified Databricks Gold export → manual download to the local computer.
3. Local Python/SQL loader → SQL Server warehouse.
4. Power BI Import/refresh → local semantic model.

The manual steps are limitations to document, not gaps to hide.

Official and current source register

Installation screens, interface labels, account tiers, and product capabilities can change. The workbook uses the following official sources, accessed on **August 16, 2026**. Before clicking an installer or paying for a service, reopen the relevant official page and record any interface difference.

Official source	Used for	Accessed
Git for Windows	Installation and current Git package path.	August 16, 2026
Git switch	Branch creation and switching syntax.	August 16, 2026
GitHub: cloning a repository	HTTPS clone workflow.	August 16, 2026

Official source	Used for	Accessed
GitHub flow	Lightweight branch, commit, push, and pull-request workflow.	August 16, 2026
GitHub: ignoring files	.gitignore behavior.	August 16, 2026
GitHub: sensitive data	Credential-removal warning and recovery.	August 16, 2026
GitHub Actions quickstart	Actions UI and run-log navigation.	August 16, 2026
GitHub Actions concepts	Workflow, event, job, step, and runner definitions.	August 16, 2026
VS Code on Windows	User installer, PATH behavior, and code ..	August 16, 2026
VS Code Python tutorial	Interpreter selection and Python workflow.	August 16, 2026
Python on Windows	Official Windows installation guidance.	August 16, 2026
Python virtual environments	.venv creation and activation behavior.	August 16, 2026
SQL Server downloads	SQL Server 2025 Developer and Express download choices.	August 16, 2026
Install SSMS	SSMS 22 installer and Visual Studio Installer workflow.	August 16, 2026
Connect with SSMS	Connect dialog and query-window basics.	August 16, 2026
Microsoft ODBC Driver for SQL Server	Current ODBC driver downloads.	August 16, 2026
PostgreSQL Windows installers	Stable PostgreSQL installer, pgAdmin, and Stack-Builders.	August 16, 2026
PostgreSQL psql	Terminal client connection arguments and file execution.	August 16, 2026
pgAdmin connect to server	Server-tree connection behavior.	August 16, 2026
pgAdmin Query Tool	Query editor and output panes.	August 16, 2026
Databricks Free Edition limitations	Current serverless-only, quota, Jobs concurrency, language, networking, and account limitations.	August 16, 2026
Databricks notebooks	Current notebook experience.	August 16, 2026
Manage Databricks notebooks	Current create/open/clone paths.	August 16, 2026
Unity Catalog volume files	Catalog Explorer and notebook file-upload paths.	August 16, 2026
Configure Lakeflow Jobs	Jobs & Pipelines configuration.	August 16, 2026
Repair failed job runs	Repair-run workflow.	August 16, 2026
Delta Lake MERGE	Merge semantics used for idempotent updates.	August 16, 2026
Databricks secrets	Secret scopes and protected values.	August 16, 2026
Power BI Desktop download	64-bit Power BI Desktop and monthly updates.	August 16, 2026
Power Query SQL Server connector	Import versus DirectQuery and connection fields.	August 16, 2026
Power BI star schema	Fact and dimension modeling guidance.	August 16, 2026
Power BI relationships	Cardinality, filter direction, active/inactive relationships.	August 16, 2026
Power BI date tables	Date-table requirements.	August 16, 2026
Power BI drillthrough	Drill-through page behavior.	August 16, 2026
Power BI report tooltips	Tooltip configuration.	August 16, 2026

Current-version notes used in this workbook

- Microsoft's official download page offered SQL Server 2025 Developer for non-production development/testing and SQL Server 2025 Express on the access date.
- Microsoft's SSMS installation page described SSMS 22 through the Visual Studio Installer; the SSMS release notes identified 22.9.0, released August 11, 2026, as the current release on the access date.
- PostgreSQL's Windows page announced PostgreSQL 18.6 as a current stable release and PostgreSQL 19 Beta 3 as a development release on August 13, 2026; the beginner path uses a stable supported release rather than beta.
- pgAdmin documentation identified the current documentation stream as pgAdmin 4 version 9.17 on the access date.
- Python's current Windows documentation, published in the Python 3.14.7 documentation stream, emphasizes the Python Install Manager. Northstar deliberately pins the latest patched Python 3.13 runtime for project compatibility; record the exact patch you install.
- Power BI Desktop is 64-bit and updated monthly. Microsoft's official download page says only the latest version is supported, while the official archive's latest visible monthly section on the access date was June 2026; exact ribbon placement can therefore differ from examples.

- Databricks Free Edition documentation dated July 20, 2026 described serverless-only compute, quota limits, and a maximum of five concurrent Job tasks per account. Databricks notebook, volume, and Jobs interfaces still vary by cloud, workspace tier, permissions, and rollout, so the workbook gives a current primary path and a safe object-based fallback.
- The Microsoft ODBC Driver version available can change; record the exact installed driver name with Python/ODBC inspection and use that exact string.

Public repository observation before any write

On August 16, 2026, the public Losboules/SQL-Data-Warehouse-Project main-branch page displayed datasets/, docs/, scripts/, LICENSE, and README.md, with eight commits shown. Treat this only as a time-stamped observation. Phase 1 requires you to inspect the current remote and local clone yourself, preserve existing work, and avoid deleting or overwriting anything merely because this workbook saw a different state.

Windows path primer used throughout the workbook

The example repository path is C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. User is a placeholder and can differ on another computer. Never type a placeholder without finding your actual value.

- An **absolute path** begins with a drive such as C:\ and identifies a location from the drive root.
- A **relative path** is interpreted from the current folder. . means the current folder; .. means its parent; .\scripts means the scripts child of the current folder.
- A backslash separates Windows folder names. A space after C: is wrong because C: \Users is not C:\Users.
- Quotation marks are required when a path contains spaces and are safe around paths that do not contain spaces.
- \$HOME in PowerShell is your Windows home folder. Verify it with Write-Output \$HOME; do not assume it equals another person's path.
- Get-Location shows the current shell folder. git rev-parse --show-toplevel shows the repository root. These paths should agree when the action requires the root.
- Test-Path checks whether a path exists. Get-ChildItem shows what is inside it.
- SQL-Data-Warehouse-Project and SQL_Data_Warehouse_Project are different names. Missing or added hyphens do not refer to the same folder.

Path verification action

Action 1.P.1 — Verify the current Windows folder

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell or VS Code PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
Get-Location
```

Where to enter it:

Windows PowerShell or VS Code PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** Get-Location — Get-Location prints the current PowerShell folder

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.P.2 — Verify the Git repository root

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell or VS Code PowerShell terminal inside the clone

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

Windows PowerShell or VS Code PowerShell terminal inside the clone. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Phase 0 — Project orientation and the end-to-end mental model

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: Nothing. This is the shared mental model.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 1 uses the architecture map to distinguish repository files from running systems.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 0 lesson index

1. [Lesson 0.1 — Understand the phase and starting screen](#)
2. [Lesson 0.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 0.3 — Create and understand the phase files](#)
4. [Lesson 0.4 — Build and validate required objects](#)
5. [Lesson 0.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 0.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** Web browser, File Explorer, VS Code, and a paper or digital journal.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 0 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 0 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 0 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

Web browser, File Explorer, VS Code, and a paper or digital journal. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 0 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 0 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 0 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 0.1 completion checklist

☐

I read why Phase 0 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 0.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 0 lesson index](#)

Lesson 0.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 0 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

A mental map of the whole platform before you install or code anything. You will connect each unfamiliar word to one Northstar Retail example and write two architecture explanations: one simple enough to teach a child and one suitable for an interview.

2. Why it matters

Beginners often copy commands without knowing which system owns the data or why one step precedes another. Requirements precede source design; sources precede ingestion; raw data precedes cleaning; facts and dimensions precede semantic measures; tests precede publication. This phase gives every later action a place in that sequence.

3. Words to know

Term	Plain-language definition
Database	An organized system for storing and querying related data; SQL Server and PostgreSQL are separate database systems.
Data warehouse	A database designed for historical analysis, stable business definitions, and reporting.
Data lake	File-oriented storage that can retain raw and varied data.
Lakehouse	A lake with table reliability and warehouse-style management; this project uses Delta tables in Databricks.
ETL / ELT	Extract-transform-load versus extract-load-transform; the order tells you where transformation happens.
Pipeline	A repeatable path that moves and changes data through controlled steps.
Ingestion	Bringing source data into the platform.
Transformation	Changing type, structure, quality, or meaning under a documented rule.
Orchestration	Coordinating task order, parameters, retries, schedules, and failure handling.
Bronze / Silver / Gold	Raw evidence, trusted/conformed detail, and business-ready dimensions/facts.
Semantic layer	Shared relationships, measures, formats, and definitions used by reports.
Consumption layer	Where people query or view the data, such as Power BI or SQL.
Fact / dimension	A measurable event or snapshot, and the descriptive context around it.
Grain	Exactly what one row represents.
Primary / foreign key	A row identifier and a reference to another table's identifier.
Surrogate key	A warehouse-controlled key independent of a source business identifier.
Incremental loading	Processing new or changed records rather than rebuilding everything.
Data quality	Evidence that data is fit for a stated use.
Lineage	Evidence showing where a number came from and how it changed.

4. What must already be completed

Nothing. Begin with this workbook and inspect only repository files that already exist. Do not import, copy, or extract a prebuilt project.

5. Applications to open

Application	How to open it	Why it is open
Web browser	Open your normal browser from the Windows Start menu.	Read the public repository and official references.
File Explorer	Press Windows key + E.	Inspect the current repository and its visible folders without changing them.
VS Code or a text editor	Open it if already installed; otherwise use Notepad for this phase.	Read Markdown and write your first journal entry.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run Get-Location; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
Get-Date	Prints the current date and time.	Your journal and evidence need timestamps.
Get-Location	Shows the exact folder PowerShell currently considers active.	Folder awareness is the first defense against creating or deleting files in the wrong place.
Get-ChildItem README.md, docs\architecture.md	Confirms the three orientation files exist.	You should not proceed from an incomplete copy.

One-page architecture in plain language

The store system, website system, and files are separate boxes. Python makes safe practice data and copies one batch out. Databricks keeps the untouched batch in Bronze, makes a cleaned version in Silver, and organizes reporting tables in Gold. Tests act like a locked gate. After the gate opens, the local warehouse and Power BI use the trusted copy.

Interview version

Track A is a manifest-driven batch ELT architecture with heterogeneous local sources, a manual governed volume boundary, Delta medallion layers, SCD-aware dimensional Gold, a blocking quality gate, a SQL Server serving mirror, and a Power BI semantic model. The manual boundary is a deliberate low-cost networking constraint, not hidden automation.

MY TASK

Draw the platform as nine labeled boxes on paper or in your journal. Under each box, write who/what owns it, its input, its output, and one failure you would test.

STARTER SKELETON

TODO(USER) Simple explanation:
Northstar Retail starts with _____. Python then _____. Bronze keeps _____. Silver changes _____. Gold represents _____. The quality gate prevents _____. SQL Server DW serves _____. Power BI defines _____.

TODO(USER) Interview explanation:
This project uses a _____ batch architecture because _____. The declared grain of the main sales fact is _____. Trust is established with _____.

Acceptance criteria: Both explanations name the manual Track A boundary, distinguish Bronze/Silver/Gold, and explain why tests come before publication.

HINT 1 — CONCEPT

Think of Bronze as a sealed evidence bag, Silver as a cleaned workbench, and Gold as labeled reporting shelves.

HINT 2 — PSEUDOCODE

For each arrow, write: input → action → output → check.

HINT 3 — NEAR-COMPLETE PATTERN

Use the two sample explanations in docs/architecture.md only after attempting your own.

13. Expected files or output

- A journal entry dated today.
- Your own simple and professional explanations.
- A hand-drawn or text architecture with all required layers.
- No claim that any database or Databricks job has run.

14. How to verify

- You can state fact_sales grain without reading: one valid order item.
- You can explain why Power BI does not read the raw source directly in this design.
- You can explain why cloud Databricks cannot automatically reach your PC's localhost.

15. Automated tests

- Self-test: point to every architecture arrow and name the file/table produced.
- Quiz answers are checked only after writing them.
- No automated platform test is required in Phase 0.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
I keep mixing database, warehouse, and lakehouse	The words are new and overlap in normal conversation.	Open the architecture and identify where rows live at each stage.	Use one Northstar example per term; do not memorize dictionary definitions alone.	Explain each term using orders, not abstract words.
I think Bronze means bad data	Bronze means preserved source evidence, not worthless data.	Read the layer contract in docs/architecture.md.	Say 'raw with metadata' rather than 'bad'.	You can name why a raw copy is useful during a dispute.
I want to skip requirements and start SQL	Visible code feels like progress.	Open docs/business_requirements.md and choose one KPI.	Trace the KPI formula back to required source columns before coding.	You can state what source data would make the KPI impossible.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why does a pipeline need both a technical sequence and a business definition?
2. What does one row of fact_sales mean?

3. What is the difference between a source database and the serving warehouse?
4. What evidence would you need before saying a dashboard value is trusted?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. Which layer should preserve raw source values and source-file metadata?
2. Why is inventory snapshot quantity not safely additive across dates?
3. What is the purpose of a surrogate key?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture the overall architecture open in your browser beside your dated journal entry. This proves orientation work, not pipeline execution.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
docs: complete phase 00 architecture teach-back
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

I read both architecture explanations — [STATUS]

☐

I defined all required words using Northstar examples — [STATUS]

☐

I wrote my own two explanations — [STATUS]

☐

I understand the Track A manual boundary — [STATUS]

☐

I recorded no unverified result — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-0. The quiz answer is in [the answer-key section immediately after the quiz in this workbook#phase-0] (the answer-key section immediately after the quiz in this workbook#phase-0).

Next phase: Phase 1 safely inspects, clones, backs up, and expands the existing GitHub repository.

Required action cards

Action 0.2.0 — Inspect the workbook and any existing repository without importing a prebuilt project

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Inspect the workbook and any existing repository without importing a prebuilt project. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open this Markdown workbook in VS Code or a trusted Markdown viewer.
2. LOOK FOR: Confirm the filename is `Northstar_Retail_Complete_Manual_Build_Workbook.md`.
3. CLICK: Open File Explorer with Windows key+E.
4. LOOK FOR: Inspect whether a `SQL-Data-Warehouse-Project` folder already exists; do not create, extract, copy, delete, or overwrite anything yet.
5. If the folder exists, single-click it and record its full path from the address bar.
6. If the folder does not exist, record repository not cloned yet; Phase 1 teaches the clone action.
7. LOOK FOR: Keep this workbook as the only instructional source; old workbooks, exported notebooks, archives, or generated kits are not inputs to the manual-build path.
8. DO NOT CONTINUE UNTIL: You can state whether the repository already exists and where it is, without having changed it.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open this Markdown workbook in VS Code or a trusted Markdown viewer.
2. LOOK FOR: Confirm the filename is `Northstar_Retail_Complete_Manual_Build_Workbook.md`.
3. CLICK: Open File Explorer with Windows key+E.
4. LOOK FOR: Inspect whether a `SQL-Data-Warehouse-Project` folder already exists; do not create, extract, copy, delete, or overwrite anything yet.
5. If the folder exists, single-click it and record its full path from the address bar.
6. If the folder does not exist, record repository not cloned yet; Phase 1 teaches the clone action.
7. LOOK FOR: Keep this workbook as the only instructional source; old workbooks, exported notebooks, archives, or generated kits are not inputs to the manual-build path.
8. DO NOT CONTINUE UNTIL: You can state whether the repository already exists and where it is, without having changed it.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 0 lesson index](#)

Action 0.2.1 — Open README.md and locate the business problem, architecture, and Track A boundary.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Open README.md and locate the business problem, architecture, and Track A boundary. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open README.md and locate the business problem, architecture, and Track A boundary.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 2

Action

Open README.md and locate the business problem, architecture, and Track A boundary.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 0 lesson index](#)

Action 0.2.2 — Open docs/architecture.md. Read the child-simple explanation first, then the professional explanation.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Open docs/architecture.md. Read the child-simple explanation first, then the professional explanation. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open docs/architecture.md. Read the child-simple explanation first, then the professional explanation.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 3****Action**

Open docs/architecture.md. Read the child-simple explanation first, then the professional explanation.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location

with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 0 lesson index](#)

Action 0.2.3 — Open docs/images/overall_architecture.svg in the browser. Trace one order from SQL Server to Power BI with y

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Open docs/images/overall_architecture.svg in the browser. Trace one order from SQL Server to Power BI with y. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open docs/images/overall_architecture.svg in the browser. Trace one order from SQL Server to Power BI with your finger or cursor.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 4

Action

Open docs/images/overall_architecture.svg in the browser. Trace one order from SQL Server to Power BI with your finger or cursor.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.



I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 0 lesson index](#)

Action 0.2.4 — Open the complete glossary near the end of this workbook. Highlight only the terms that remain unclear; do n

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Open the complete glossary near the end of this workbook. Highlight only the terms that remain unclear; do n. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.

3. Perform only this immediate task: Open the complete glossary near the end of this workbook. Highlight only the terms that remain unclear; do not memorize everything at once.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 5****Action**

Open the complete glossary near the end of this workbook. Highlight only the terms that remain unclear; do not memorize everything at once.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 0 lesson index](#)

Action 0.2.5 — Create your first entry in the learning-journal fields in this phase and write the pipeline in your own word

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Create your first entry in the learning-journal fields in this phase and write the pipeline in your own word. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Create your first entry in the learning-journal fields in this phase and write the pipeline in your own words.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 6

Action

Create your first entry in the learning-journal fields in this phase and write the pipeline in your own words.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 0 lesson index](#)

Action 0.2.6 — Write down the sequence Requirements → Sources → Extract → Bronze → Silver → Gold → Quality Gate → Warehouse →

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Write down the sequence Requirements → Sources → Extract → Bronze → Silver → Gold → Quality Gate → Warehouse →. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Write down the sequence Requirements → Sources → Extract → Bronze → Silver → Gold → Quality Gate → Warehouse → Semantic Model → Dashboard.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 7****Action**

Write down the sequence Requirements → Sources → Extract → Bronze → Silver → Gold → Quality Gate → Warehouse → Semantic Model → Dashboard.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 0 lesson index](#)

Action 0.2.C1 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The orientation material in this workbook; no repository change is required yet. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Get-Date
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** Get-Date — Get-Date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 0 lesson index](#)

Action 0.2.C2 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The orientation material in this workbook; no repository change is required yet. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Get-Location
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** Get-Location — Get-Location prints the current PowerShell folder

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 0 lesson index](#)

Action 0.2.C3 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The orientation material in this workbook; no repository change is required yet. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Get-ChildItem README.md, docs\architecture.md
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `Get-ChildItem README.md, docs\architecture.md` — `Get-ChildItem` lists files and folders; `README.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `architecture.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 0 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 0.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 0 starting-screen checklist and re-verify the active context.

Lesson 0.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 0 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — docs/

Exact folder name: docs

Exact parent: repository root

Why it is needed now: Phase 0 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.

3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: docs.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 0.D1.1 — Create the folder docs

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The orientation material in this workbook; no repository change is required yet. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\docs" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 0 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\docs" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\docs" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\docs" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — docs/architecture.md

Exact filename: architecture.md

Filename extension: .md. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: docs relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 0.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in docs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: architecture.md.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled architecture.md and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 0.F1.1 — Create the file docs/architecture.md from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The orientation material in this workbook; no repository change is required yet. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.

6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\architecture.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\docs\architecture.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs\architecture.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 0 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\docs\architecture.md" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/architecture.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under docs.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `docs/architecture.md`.
3. In PowerShell at the repository root, run `Test-Path ".\docs\architecture.md"` and require True.
4. Run `git status --short -- "docs/architecture.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for docs/architecture.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: docs/architecture.md

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 0.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Architecture

## Child-simple explanation

Northstar Retail has information in three places: a store-and-order database, a website database, and files. A Python helper makes fictional practice data and later copies data out of the local databases. You manually place one sealed batch folder in Databricks because a cloud Databricks workspace cannot magically reach a database on your own computer. Databricks stores the original copy in Bronze, cleans it in Silver, and organizes it for reporting in Gold. After tests pass, you download Gold files and load them into a local SQL Server warehouse. Power BI reads that warehouse and shows definitions everyone can share.

![Overall architecture](images/overall_architecture.svg)

## Professional interview explanation

The default implementation is a batch ELT lakehouse pattern with three heterogeneous sources, deterministic synthetic fixtures, manifest-driven file handoff, Delta medallion layers, orchestrated validation gates, a dimensional Gold model, a mirrored SQL Server serving warehouse, and a Power BI semantic model. Track A intentionally uses manual upload/download boundaries to avoid false claims about private-network connectivity. Batch IDs, checksums, audit metadata, quarantine tables, effective-date joins, and idempotent replacement/merge patterns support traceability and safe reruns.

## Track A – local-first data flow

![Local-first flow](images/local_first_flow.svg)
```

Manual steps are honest parts of this learning track:

1. Upload the local extraction batch to a Unity Catalog volume.
2. Start the Lakeflow Job with that exact batch ID.
3. Download the validated Gold export.
4. Run the local SQL Server publication loader.
5. Refresh the imported Power BI model.

Track B – optional cloud-connected flow

![Cloud-connected flow](images/cloud_connected_flow.svg)

Track B replaces manual boundaries with network-accessible managed databases and cloud object storage. It requires real firewall/private networking decisions, supported JDBC drivers, secret management, least-privilege identities, and cost monitoring. It is not required to complete the beginner project.

Layer contract

Layer	Contract	Allowed changes	Rerun behavior
---	---	---	---
Bronze	Raw values plus source/batch/file/hash metadata	Parsing only enough to read; no business correction	Delete and replace same batch, or merge on batch/hash
Silver	Typed, standardized, deduplicated, conformed, valid and quarantined	Business-quality rules are explicit and tested	Batch replacement or Delta MERGE on business key/change hash
Gold	Dimensional facts/dimensions and governed aggregates	Surrogate keys, SCD, measures, unknown members	Deterministic keys and grain uniqueness prevent duplicates
Serving DW	Relational mirror for analysts/Power BI	Constraints, indexes, semantic views	Transactional stage-and-publish with audit records
Semantic/consumption	Shared definitions and report behavior	Measures, relationships, formatting, descriptions	Refresh only after validated publication

Source ERDs

![SQL Server ERP ERD](images/sqlserver_erd.svg)

![PostgreSQL digital ERD](images/postgres_erd.svg)

Gold star schema

![Gold star schema](images/gold_star_schema.svg)

Job dependency

![Jobs dependency](images/jobs_dependency.svg)

Lineage example

![Lineage](images/lineage.svg)

SCD Type 2 example

![SCD2 timeline](images/scd2_timeline.svg)

Line-by-line explanation

- **Line 1:** # Architecture is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** ## Child-simple explanation is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** Northstar Retail has information in three places: a store-and-order database, a website database, and files. A Python helper makes fictional practice data and later copies data out of the local databases. You manually place one sealed batch folder in Databricks because a cloud Databricks workspace cannot magically reach a database on your own computer. Databricks stores the original copy in Bronze, cleans it in Silver, and organizes it for reporting in Gold. After tests pass, you download Gold files and load them into a local SQL Server warehouse. Power BI reads that warehouse and shows definitions everyone can share. — Northstar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Retail is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; has is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; information is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; three is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; places is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding

explanation; store-and-order is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; database is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** `![Overall architecture](images/overall_architecture.svg)` — `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Overall` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `architecture` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `images` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `overall_architecture.svg` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** `## Professional interview` explanation is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 10:** blank line used only to separate logical sections for readability.
- **Line 11:** The default implementation is a batch ELT lakehouse pattern with three heterogeneous sources, deterministic synthetic fixtures, manifest-driven file handoff, Delta medallion layers, orchestrated validation gates, a dimensional Gold model, a mirrored SQL Server serving warehouse, and a Power BI semantic model. Track A intentionally uses manual upload/download boundaries to avoid false claims about private-network connectivity. Batch IDs, checksums, audit metadata, quarantine tables, effective-date joins, and idempotent replacement/merge patterns support traceability and safe reruns. — The is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `implementation` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ELT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `lakehouse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pattern` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `three` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `heterogeneous` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** `## Track A – local-first data flow` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 14:** blank line used only to separate logical sections for readability.
- **Line 15:** `![Local-first flow](images/local_first_flow.svg)` — `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Local-first` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `flow` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `images` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `local_first_flow.svg` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** blank line used only to separate logical sections for readability.
- **Line 17:** Manual steps are honest parts of this learning track: — `Manual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `steps` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `are` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `honest`

is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; parts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; of is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; this is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; learning is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; track is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** 1. Upload the local extraction batch to a Unity Catalog volume. — 1 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Upload is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; local is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; extraction is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; to is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Unity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Catalog is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; volume is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 20:** 2. Start the Lakeflow Job with that exact batch ID. — 2 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Start is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Lakeflow is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Job is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; with is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; that is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; exact is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 21:** 3. Download the validated Gold export. — 3 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Download is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; validated is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; export is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 22:** 4. Run the local SQL Server publication loader. — 4 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; local is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; publication is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; loader is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 23:** 5. Refresh the imported Power BI model. — 5 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Refresh is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

planation; imported is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Power is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BI is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; model is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 24:** blank line used only to separate logical sections for readability.
- **Line 25:** ## Track B – optional cloud-connected flow is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 26:** blank line used only to separate logical sections for readability.
- **Line 27:** ![Cloud-connected flow](images/cloud_connected_flow.svg) — [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Cloud-connected is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; flow is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; images is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; cloud_connected_flow.svg is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** blank line used only to separate logical sections for readability.
- **Line 29:** Track B replaces manual boundaries with network-accessible managed databases and cloud object storage. It requires real firewall/private networking decisions, supported JDBC drivers, secret management, least-privilege identities, and cost monitoring. It is not required to complete the beginner project. — Track is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; B is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; replaces is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; manual is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; boundaries is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; with is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; network-accessible is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; managed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; databases is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cloud is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; object is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 30:** blank line used only to separate logical sections for readability.
- **Line 31:** ## Layer contract is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 32:** blank line used only to separate logical sections for readability.
- **Line 33:** | Layer | Contract | Allowed changes | Rerun behavior | — Layer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Contract is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Allowed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; changes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Rerun is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; behavior is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** | --- | --- | --- | --- | — --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command
- **Line 35:** | Bronze | Raw values plus source/batch/file/hash metadata | Parsing only enough to read; no business correction | Delete and replace same batch, or merge on batch/hash | — Bronze is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Raw is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; values

is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; plus is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; hash is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; metadata is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 36:** | Silver | Typed, standardized, deduplicated, conformed, valid and quarantined | Business-quality rules are explicit and tested | Batch replacement or Delta MERGE on business key/change hash | — Silver is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Typed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; standardized is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; deduplicated is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; conformed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quarantined is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 37:** | Gold | Dimensional facts/dimensions and governed aggregates | Surrogate keys, SCD, measures, unknown members | Deterministic keys and grain uniqueness prevent duplicates | — Gold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Dimensional is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; facts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; dimensions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; governed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; aggregates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Surrogate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; keys is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SCD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 38:** | Serving DW | Relational mirror for analysts/Power BI | Constraints, indexes, semantic views | Transactional stage-and-publish with audit records | — Serving is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DW is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Relational is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mirror is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; analysts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; Power is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BI is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Constraints is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; indexes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 39:** | Semantic/consumption | Shared definitions and report behavior | Measures, relationships, formatting, descriptions | Refresh only after validated publication | — Semantic is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; consumption is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Shared is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; definitions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; report is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; behavior is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Measures is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; relationships is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 40:** blank line used only to separate logical sections for readability.
- **Line 41:** ## Source ERDs is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 42:** blank line used only to separate logical sections for readability.
- **Line 43:** ![SQL Server ERP ERD](images/sqlserver_erd.svg) — [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ERP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ERD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; images is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; sqlserver_erd.svg is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 44:** blank line used only to separate logical sections for readability.
- **Line 45:** ![PostgreSQL digital ERD](images/postgres_erd.svg) — [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PostgreSQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; digital is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ERD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; images is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; postgres_erd.svg is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** blank line used only to separate logical sections for readability.
- **Line 47:** ## Gold star schema is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 48:** blank line used only to separate logical sections for readability.
- **Line 49:** ![Gold star schema](images/gold_star_schema.svg) — [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; star is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; images is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; gold_star_schema.svg is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 50:** blank line used only to separate logical sections for readability.
- **Line 51:** `## Job dependency` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 52:** blank line used only to separate logical sections for readability.
- **Line 53:** `![Jobs dependency](images/jobs_dependency.svg)` — `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Jobs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dependency` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `images` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `jobs_dependency.svg` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 54:** blank line used only to separate logical sections for readability.
- **Line 55:** `## Lineage example` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 56:** blank line used only to separate logical sections for readability.
- **Line 57:** `![Lineage](images/lineage.svg)` — `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Lineage` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `images` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `lineage.svg` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 58:** blank line used only to separate logical sections for readability.
- **Line 59:** `## SCD Type 2 example` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 60:** blank line used only to separate logical sections for readability.
- **Line 61:** `![SCD2 timeline](images/scd2_timeline.svg)` — `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCD2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timeline` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `images` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `scd2_timeline.svg` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 0 lesson index](#)

What your screen should look like now

- Every Phase 0 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 0.3 completion checklist



`docs/architecture.md` was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 0.4](#) and recheck the root and branch.

Lesson 0.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Action 0.4.G1 — Inspect the current state without changing it

Application:

Web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Inspect the current state without changing it. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 0 lesson index](#)

Action 0.4.G2 — Create the smallest required object or decision

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Create the smallest required object or decision. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 0 lesson index](#)

Action 0.4.G3 — Run the phase-specific verification

Application:

Web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Run the phase-specific verification. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 0 lesson index](#)

Action 0.4.G4 — Demonstrate one safe failure or alternate-result branch

Application:

Web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Demonstrate one safe failure or alternate-result branch. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 0 lesson index](#)

Action 0.4.G5 — Record evidence and prepare the downstream handoff

Application:

Web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Record evidence and prepare the downstream handoff. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 0 lesson index](#)

What your screen should look like now

- The new Phase 0 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 0.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 0.5](#).

Lesson 0.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 0 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Local-first learning architecture

Question:

How should a beginner connect local databases, Databricks, a warehouse, and Power BI without inventing network connectivity?

Options considered:

- Track A with manual governed handoffs.
- Track B with cloud-accessible sources and automated connectivity.
- Skip Databricks and build only a local warehouse.

Beginner path selected:

Track A is required; Track B is optional.

Why this path was selected:

It exposes every boundary and lets the learner validate files, manifests, checksums, and tables without paying for a production network.

Tradeoff:

Manual upload/download adds latency and operational effort.

What a company might do instead:

Use private networking, cloud object storage, managed identities, service principals, and automated deployment.

How to explain this in an interview:

I chose a local-first batch path to make each boundary observable and reproducible; I documented the manual handoffs and did not claim that cloud Databricks could reach laptop localhost.

Design decision — Local versus cloud databases

Question:

Should the required beginner path run its source databases locally or pay for managed cloud databases?

Options considered:

- Local SQL Server and PostgreSQL with explicit manual cloud handoffs.
- Managed cloud source databases reachable by Databricks.
- A single all-cloud platform that removes the heterogeneous local boundary.

Beginner path selected:

Run the operational source databases locally and document the manual Track A transfer boundary.

Why this path was selected:

Local databases minimize cost, expose installation and connectivity fundamentals, and prevent the workbook from assuming cloud networking or paid services.

Tradeoff:

The laptop must be running for local work, collaboration is harder, and Databricks cannot directly reach loopback without real networking.

What a company might do instead:

Use managed databases, private networking, cloud object storage, managed identities, backups, monitoring, and infrastructure as code.

How to explain this in an interview:

I chose local source engines for cost-controlled learning and made the cloud handoff explicit; I would move to managed databases when availability, collaboration, security, and automated connectivity justify it.

If you see Result A

Your diagram names every system and manual boundary. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

One or more boxes/arrows are missing or ownership is unclear. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: Web browser, File Explorer, VS Code, and a paper or digital journal.

Window/panel: Show the architecture map and dated learning-journal fields.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase00_project-orientation-and-the-end-to-end-mental-model_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 0's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 0 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 0.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The orientation material in this workbook; no repository change is required yet. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 0 lesson index](#)

Action 0.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The orientation material in this workbook; no repository change is required yet. or the more specific location stated below.

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 0 lesson index](#)

Action 0.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The orientation material in this workbook; no repository change is required yet. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 0 lesson index](#)

Interpret `git status --short`

- `??` means untracked: Git sees a new path but it is not staged.
- `M` means modified in the working tree but not staged.
- `M` means the staged version differs from the last commit.
- `MM` means a staged version exists and the file has additional unstaged edits.
- `A` means staged as a new file.
- `D` means deletion; stop and investigate unless deletion is deliberate.
- `R` means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 0.5.G4 — Inspect the unstaged diff for `docs/architecture.md`

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The orientation material in this workbook; no repository change is required yet. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "docs/architecture.md"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "docs/architecture.md"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"docs/architecture.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 0 lesson index](#)

Action 0.5.G5 — Stage only the reviewed file docs/architecture.md

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The orientation material in this workbook; no repository change is required yet. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "docs/architecture.md"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "docs/architecture.md"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"docs/architecture.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 0 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 0.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The orientation material in this workbook; no repository change is required yet. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 0 lesson index](#)

Action 0.5.G7 — Create the Phase 0 commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The orientation material in this workbook; no repository change is required yet. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "docs: record phase 00 architecture teach-back"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "docs: record phase 00 architecture teach-back"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"docs: record phase 00 architecture teach-back"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 0 lesson index](#)

Action 0.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The orientation material in this workbook; no repository change is required yet. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 0 lesson index](#)

Action 0.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The orientation material in this workbook; no repository change is required yet.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The orientation material in this workbook; no repository change is required yet. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 0 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message `docs: record phase 00 architecture teach-back` and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 0.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.

6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 0 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

“Phase 0 created the verified boundary described by **Project orientation and the end-to-end mental model**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed.”

Mini quiz

1. Multiple choice: Why does Phase 0 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

My Phase 0 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 0.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 1 uses the architecture map to distinguish repository files from running systems.

Phase 0 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 1 — Repository inspection, preservation, branching, and preparation

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: Phase 0 architecture and ownership rules.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 2 installs only the tools the inspected repository and planned architecture require.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 1 lesson index

1. [Lesson 1.1 — Understand the phase and starting screen](#)
2. [Lesson 1.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 1.3 — Create and understand the phase files](#)
4. [Lesson 1.4 — Build and validate required objects](#)
5. [Lesson 1.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 1.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** Web browser, File Explorer, Windows PowerShell, VS Code, and GitHub.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 1 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 0 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 1 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

Web browser, File Explorer, Windows PowerShell, VS Code, and GitHub. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 1 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 1 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 1 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 1.1 completion checklist

☐

I read why Phase 1 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 1.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 1 lesson index](#)

Lesson 1.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 1 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

A safe local clone and feature branch that preserves the public repository's existing README, license, placeholder folders, and any local scripts/init_database.sql content before any manually created project structure is added.

2. Why it matters

Git gives you recoverable history, reviewable changes, and evidence of your own progress. The most dangerous beginner mistake here is not a complex merge—it is working in the wrong folder or overwriting existing work because two similar folder names are treated as identical.

3. Words to know

Term	Plain-language definition
Repository	A project folder whose changes Git tracks.
Clone	A local copy connected to a remote repository.
Remote	The GitHub copy, commonly named <code>origin</code> .
Branch	An independent line of changes.
Commit	A named snapshot of staged changes.
Stage	Select changes for the next commit.
Push	Send local commits to the remote.
Pull/rebase	Bring remote changes into local history.
Diff	A line-by-line view of changes.
Repository root	The top folder containing <code>.git</code> and README.
Explorer panel	VS Code's left-side file tree.
Source Control panel	VS Code view for Git changes and diffs.

4. What must already be completed

Phase 0 is complete. Git and VS Code may not be installed yet; if either command is missing, record the result and complete installation in Phase 2 before retrying.

5. Applications to open

Application	How to open it	Why it is open
Web browser	Open Start menu and your browser; navigate to the repository URL.	Inspect existing files and copy the clone URL.
Windows PowerShell	Right-click Start > Terminal/PowerShell, not an administrator window.	Navigate, clone, branch, and verify Git.
VS Code	Type <code>code .</code> after entering the clone, or open from Start.	Review Explorer and Source Control safely.

Application	How to open it	Why it is open
File Explorer	Press Windows key + E.	Cross-check actual folder names.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run Get-Location; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
Get-Location	Prints your current folder.	Every following relative path depends on this folder.
Get-ChildItem -Directory Select-Object Name	Lists exact child directory names only.	It prevents the earlier mistake of creating norths-tarretail-data-platform but entering northstar-retail-data-platform.
git clone https://github.com/Losboules/SQL-Data-Warehouse-Project.git	Creates a connected local folder named SQL-Data-Warehouse-Project.	The existing GitHub repository is the source baseline.
Set-Location .\SQL-Data-Warehouse-Project	Moves PowerShell into the cloned repository.	Git commands must target this project.
git rev-parse --show-toplevel	Asks Git for the true repository root.	Folder name alone does not prove you are inside a Git repository.
git status	Shows branch and changed/untracked files.	You need a clean baseline before migration.
git switch -c feature/northstar-platform	Creates and enters a new feature branch.	The manual project change should be reviewable and reversible without changing main immediately.
New-Item -ItemType Directory -Path scripts\legacy -ErrorAction Stop	Creates a safe legacy folder without deleting anything.	A surfaced legacy SQL file must be preserved before new SQL paths are added.
if (Test-Path scripts\init_database.sql) { Copy-Item scripts\init_database.sql scripts\legacy\init_database.original.sql -Force }	Copies the legacy SQL file only if it exists locally.	The repository inspection could not retrieve its content and therefore must not assume it is disposable.
code .	Opens this exact folder in VS Code.	Opening VS Code from the verified root avoids editing a look-alike folder.

Folder and file map before creation

Path	Purpose	First phase used
config/	non-secret scale, logging, and environment templates	5
datasets/	quick public sample; ignored raw batch folders	5-7
docs/	workbook, contracts, diagrams, evidence templates	all
notebooks/	GitHub showcase notebook and HTML	20
scripts/	Python/PowerShell generation, movement, validation	1 onward
sql/sqlserver/	T-SQL source, warehouse, validation, analytics	4, 15
sql/postgres/	PostgreSQL source and validation SQL	4
sql/databricks/	Spark SQL teaching patterns	9-13

Path	Purpose	First phase used
databricks/	runnable source notebooks and Job map	8-14
powerbi/	semantic model, DAX, page specification	16-17
tests/	unit/integration/quality/reconciliation tests	5 onward
.github/	CI and collaboration templates	19

the file-responsibility index in this workbook later names every individual file and its phase.

MY TASK

Clone the correct repository, create the feature branch, preserve any local legacy SQL file, create only the current phase files manually while preserving existing work, and write a one-paragraph migration note describing what existed before.

STARTER SKELETON

```
TODO(USER) Repository root: _____
TODO(USER) Existing branch before change: _____
TODO(USER) New branch: _____
TODO(USER) Files preserved: _____
TODO(USER) Files manually created or preserved and reviewed: _____
TODO(USER) Validation command/result: _____
```

Acceptance criteria: `git status` runs from the true root, the feature branch is active, existing work is preserved, and the first migration commit contains only understood changes.

HINT 1 — CONCEPT

A path is exact text: `northstarretail` is not `northstar-retail`.

HINT 2 — PSEUDOCODE

Ask Git for the root instead of trusting the prompt: `git rev-parse --show-toplevel`.

HINT 3 — NEAR-COMPLETE PATTERN

Use VS Code Source Control to open every changed README/SQL/config file before staging.

13. Expected files or output

- A local folder `SQL-Data-Warehouse-Project` connected to origin.
- Branch `feature/northstar-platform`.
- Preserved legacy copy if the file exists.
- The Phase 1 files you manually created or preserved are visible in Explorer.
- A clean validation result before commit.

14. How to verify

- `git remote -v` points to the correct public repository.
- `git branch --show-current` prints the feature branch.
- `git status --short` shows only intended migration changes before commit.
- `python -m scripts.validation.check_required_files` passes after Python setup in Phase 2.

15. Automated tests

- Manual root check with `git rev-parse`.
- File hash comparison for any preserved legacy SQL.
- Required-file CI script after dependencies are installed.
- GitHub diff review after push.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
Set-Location cannot find path	The typed child name differs from the one created or current parent is wrong.	Get-Location; Get-ChildItem -Directory \ Select Name.	Copy the exact returned name and use Set-Location '.\exact-name'.	Get-Location shows intended path.
fatal: not a git repository	PowerShell is outside the clone or inside a non-Git look-alike.	Get-ChildItem -Force; search for .git only in known project folders.	Return to the parent and enter the clone; do not run git init inside the wrong folder.	git rev-parse --show-top-level succeeds.
code is not recognized	VS Code PATH integration is missing or VS Code is not installed.	Get-Command code -Error-Action SilentlyContinue.	Complete Phase 2 VS Code install, reopen PowerShell, or use VS Code File > Open Folder.	VS Code opens the verified root.
Push rejected	Remote changed or wrong branch/upstream.	git branch --show-current; git fetch origin; git log --all --graph --oneline -15.	Review and rebase/pull the intended branch; never force-push main.	Remote feature branch shows commit.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. What is the difference between the GitHub repository name and any parent folder you create?
2. Why preserve scripts/init_database.sql even though the later project design introduces additional SQL files?
3. What does staging do that saving a file does not?
4. Why is the migration commit separated from later implementation work?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. Which command proves the true repository root?
2. Why should a beginner avoid working directly on main?
3. What should you do before overwriting an existing file?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture VS Code with the Explorer root, active feature branch, and Source Control changes visible. Do not include unrelated personal folders.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
chore: add northstar platform manual project structure while preserving baseline
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

Correct repository URL inspected — [STATUS]

☐

Exact local folder verified — [STATUS]

☐

Feature branch active — [STATUS]

☐

Existing content backed up — [STATUS]

☐

Manual file diffs reviewed — [STATUS]

☐

Migration commit pushed — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-1. The quiz answer is in [the answer-key section immediately after the quiz in this workbook#phase-1] (the answer-key section immediately after the quiz in this workbook#phase-1).

Next phase: Phase 2 installs and verifies every required tool using current official guidance.

Required action cards

Action 1.2.0 — Inspect whether an earlier prebuilt archive was already imported without copying or deleting anything

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Inspect whether an earlier prebuilt archive was already imported without copying or deleting anything. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the repository root in VS Code only after `git rev-parse --show-toplevel` confirms the path.
2. CLICK: Open the Source Control panel in the left Activity Bar.
3. LOOK FOR: Record every changed and untracked path before editing.
4. CLICK: Open Explorer and inspect whether folders such as `config`, `databricks`, `powerbi`, `sql`, or `tests` already exist.
5. TYPE THIS: `git status --short` in the VS Code PowerShell terminal.
6. PRESS: Enter.
7. TYPE THIS: `git ls-files`.
8. PRESS: Enter.
9. LOOK FOR: Distinguish tracked files from untracked files and compare them with the public repository baseline described in this phase.
10. DO NOT: Extract, copy, bulk-import, delete, or overwrite any prior archive contents.
11. Record whether an earlier starter archive appears to have been imported and list the evidence.
12. DO NOT CONTINUE UNTIL: The current state is preserved in `docs/repository_current_state.md` and no unexplained change has been made.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the repository root in VS Code only after `git rev-parse --show-toplevel` confirms the path.
2. CLICK: Open the Source Control panel in the left Activity Bar.
3. LOOK FOR: Record every changed and untracked path before editing.
4. CLICK: Open Explorer and inspect whether folders such as `config`, `databricks`, `powerbi`, `sql`, or `tests` already exist.
5. TYPE THIS: `git status --short` in the VS Code PowerShell terminal.
6. PRESS: Enter.
7. TYPE THIS: `git ls-files`.
8. PRESS: Enter.
9. LOOK FOR: Distinguish tracked files from untracked files and compare them with the public repository baseline described in this phase.
10. DO NOT: Extract, copy, bulk-import, delete, or overwrite any prior archive contents.
11. Record whether an earlier starter archive appears to have been imported and list the evidence.
12. DO NOT CONTINUE UNTIL: The current state is preserved in `docs/repository_current_state.md` and no unexplained change has been made.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.0A — Adopt the manual-build rule for every new folder and file

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Adopt the manual-build rule for every new folder and file. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. LOOK FOR: Keep the Phase 1 Explorer tree visible.
2. Read the folder and file tutorials in Lesson 1.3.
3. Create only the folder needed for the file currently being taught.
4. Create only one file at a time by the VS Code or PowerShell path in its file introduction.
5. Save the file.
6. Inspect its targeted Git diff.
7. Verify the file before creating the next one.
8. DO NOT: Use an archive, bulk copy, robocopy, Copy-Item from a prebuilt project, or a command that creates the whole future repository tree.
9. DO NOT CONTINUE UNTIL: You understand that the repository grows phase by phase through your own work.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. LOOK FOR: Keep the Phase 1 Explorer tree visible.
2. Read the folder and file tutorials in Lesson 1.3.
3. Create only the folder needed for the file currently being taught.
4. Create only one file at a time by the VS Code or PowerShell path in its file introduction.
5. Save the file.
6. Inspect its targeted Git diff.
7. Verify the file before creating the next one.
8. DO NOT: Use an archive, bulk copy, robocopy, Copy-Item from a prebuilt project, or a command that creates the whole future repository tree.
9. DO NOT CONTINUE UNTIL: You understand that the repository grows phase by phase through your own work.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.1 — Open the correct GitHub page

Application:

Web browser on GitHub

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open the correct GitHub page. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Click the browser address bar.
2. Enter your repository address and press **Enter**.
3. Read the browser address carefully.
4. Confirm it contains both Losboules and SQL-Data-Warehouse-Project.
5. Scroll below the file list and read the current README.md before changing anything.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Web browser on GitHub. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Expanded Step 1 — Open the correct GitHub page

Application: Chrome, Edge, or another web browser.

1. Click the browser address bar.
2. Enter your repository address and press **Enter**.
3. Read the browser address carefully.
4. Confirm it contains both Losboules and SQL-Data-Warehouse-Project.
5. Scroll below the file list and read the current README.md before changing anything.

Why: You are confirming that you are looking at your repository and learning what already exists.

Do not continue until: The address bar shows the correct owner and repository name.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.2 — Copy the HTTPS clone address

Application:

Web browser on GitHub

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Copy the HTTPS clone address. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Find the green **Code** button above the file list.
2. Click **Code**.
3. Make sure the **Local** tab is selected.
4. Select **HTTPS**.

5. Click the copy icon beside the displayed address.

Command or code:

```
https://github.com/Losboulos/SQL-Data-Warehouse-Project.git
```

Where to enter it:

Web browser on GitHub. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Expanded Step 2 — Copy the HTTPS clone address**

Application: Your web browser, still on the repository page.

1. Find the green **Code** button above the file list.
2. Click **Code**.
3. Make sure the **Local** tab is selected.
4. Select **HTTPS**.
5. Click the copy icon beside the displayed address.

The copied value should be:

Why: `git clone` needs the exact remote address.

Verification: Paste the address temporarily into Notepad, confirm it ends with `.git`, and then close Notepad without saving.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.3 — Open PowerShell and identify your current folder

Application:

Windows File Explorer

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open PowerShell and identify your current folder. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open PowerShell and identify your current folder
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

```
Get-Location
```

Where to enter it:

Windows File Explorer. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Expanded Step 3 — Open PowerShell and identify your current folder**

Application: Windows PowerShell. Do not use an administrator window.

Run:

Get-Location asks PowerShell, "Which folder am I currently inside?" The output may be C:\Users\User, C:\Users\User\Desktop, or another normal location.

Now move to the parent folder:

Verify:

Expected: The path ends with Northstar_Retail.

Common error: Cannot find path ... because it does not exist means one part of the path is spelled differently. Use File Explorer or `Get-ChildItem -Directory` to read the exact folder name instead of guessing.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.4 — Read the exact folder names before cloning

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Read the exact folder names before cloning. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Read the exact folder names before cloning
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

```
Get-ChildItem -Directory
```

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Expanded Step 4 — Read the exact folder names before cloning

Application: Windows PowerShell.

Current folder: C:\Users\User\Desktop\Northstar_Retail.

Run:

Read each name exactly. You may see the older folder you created, such as:

Leave that folder alone for now. The real cloned repository will be named:

Check whether it already exists:

- False means it is not present and you may clone.
- True means do not clone again; inspect the existing folder first.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.5 — Clone only when the repository folder does not exist

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Clone only when the repository folder does not exist. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Clone only when the repository folder does not exist
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

```
git clone https://github.com/Losboules/SQL-Data-Warehouse-Project.git
```

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Expanded Step 5 — Clone only when the repository folder does not exist**

Application: Windows PowerShell.

Current folder: C:\Users\User\Desktop\Northstar_Retail.

When Test-Path returned False, run:

What it does: Downloads the repository, its commit history, and the connection to GitHub. Git automatically creates SQL-Data-Warehouse-Project.

Expected: Messages beginning with Cloning into and ending without a red fatal error.

Common error:

This means the folder already exists. Do not delete it automatically. Enter it and verify whether it is the correct Git clone.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.6 — Enter the clone and prove that it is the repository root

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Enter the clone and prove that it is the repository root. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Enter the clone and prove that it is the repository root
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

```
Set-Location ".\SQL-Data-Warehouse-Project"
```

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Expanded Step 6 — Enter the clone and prove that it is the repository root

Run:

Check the Windows location:

Ask Git for the repository root:

Confirm the GitHub remote:

Expected: Both location commands point to SQL-Data-Warehouse-Project, and the remote contains Losboules/SQL-Data-Warehouse-Project.git.

Common error: fatal: not a git repository means you are standing in the parent folder or another unrelated folder.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.7 — Confirm a clean starting point and create a feature branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Confirm a clean starting point and create a feature branch. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Confirm a clean starting point and create a feature branch
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

```
git status
```

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Expanded Step 7 — Confirm a clean starting point and create a feature branch

Run:

Ideally, Git says:

Create and enter your work branch:

Verify:

Expected: feature/northstar-platform.

If Git says the branch already exists, use:

Why: main remains the stable original line while your large manual project change stays reviewable and reversible.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.8 — Open the correct folder in VS Code

Application:

Windows File Explorer

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open the correct folder in VS Code. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Open VS Code from the Start menu.
2. Select **File > Open Folder**.
3. Browse to C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project.
4. Click **Select Folder**.

Command or code:

```
code .
```

Where to enter it:

Windows File Explorer. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Expanded Step 8 — Open the correct folder in VS Code**

While PowerShell is inside the repository root, run:

The dot means “open the folder I am currently inside.”

If code is not recognized:

1. Open VS Code from the Start menu.
2. Select **File > Open Folder**.
3. Browse to C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project.
4. Click **Select Folder**.

In the **Explorer** panel, locate the existing README.md, LICENSE, datasets, docs, and scripts. In **Source Control**, make sure there are no unexplained changes before importing anything.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.9 — Preserve the existing SQL script

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Preserve the existing SQL script. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Preserve the existing SQL script
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

```
New-Item -ItemType Directory -Path ".\scripts\legacy" -ErrorAction Stop
```

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Expanded Step 9 — Preserve the existing SQL script

Application: VS Code integrated PowerShell terminal.

Current folder: Repository root.

Create the backup folder:

Confirm the original exists:

When the result is True, copy it:

Verify:

Why: New work should not silently erase an earlier file. The backup gives you a migration path and a clear history.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.10 — Open the public repository and read the current README before clicking anything. Confirm the URL contains `Los`

Application:

Windows File Explorer

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open the public repository and read the current README before clicking anything. Confirm the URL contains `Los`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open the public repository and read the current README before clicking anything. Confirm the URL contains Losboules/SQL-Data-Warehouse-Project.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows File Explorer. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 1

Action

Open the public repository and read the current README before clicking anything. Confirm the URL contains Losboulès/SQL-Data-Warehouse-Project.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.11 — Click the green button, select , and copy the displayed clone URL.

Application:

Windows File Explorer

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Click the green button, select , and copy the displayed clone URL. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Click the green **Code** button, select **Local > HTTPS**, and copy the displayed clone URL.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows File Explorer. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 2****Action**

Click the green **Code** button, select **Local > HTTPS**, and copy the displayed clone URL.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.12 — Open PowerShell. Run Get-Location; choose a normal parent such as C:\Users\User\Desktop\Northstar_Retail.

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open PowerShell. Run Get-Location; choose a normal parent such as C:\Users\User\Desktop\Northstar_Retail. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open PowerShell. Run Get-Location; choose a normal parent such as C:\Users\User\Desktop\Northstar_Retail.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 3

Action

Open PowerShell. Run Get-Location; choose a normal parent such as C:\Users\User\Desktop\Northstar_Retail.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.13 — Run Get-ChildItem -Directory and read the exact folder names. A hyphen and a missing hyphen create different

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run Get-ChildItem -Directory and read the exact folder names. A hyphen and a missing hyphen create different. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run `Get-ChildItem -Directory` and read the exact folder names. A hyphen and a missing hyphen create different paths.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 4****Action**

Run `Get-ChildItem -Directory` and read the exact folder names. A hyphen and a missing hyphen create different paths.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.14 — Clone the repository. Do not first create another folder with a competing project name unless you intend it as

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Clone the repository. Do not first create another folder with a competing project name unless you intend it as. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Clone the repository. Do not first create another folder with a competing project name unless you intend it as the parent.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 5

Action

Clone the repository. Do not first create another folder with a competing project name unless you intend it as the parent.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.15 — Enter SQL-Data-Warehouse-Project, run `git rev-parse --show-toplevel`, and compare the returned path to ``Get`

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Enter `SQL-Data-Warehouse-Project`, run `git rev-parse --show-toplevel`, and compare the returned path to ``Get`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Enter `SQL-Data-Warehouse-Project`, run `git rev-parse --show-toplevel`, and compare the returned path to `Get-Location`.
4. Pause while the application finishes the operation; do not click the same control twice.

5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 6****Action**

Enter SQL-Data-Warehouse-Project, run `git rev-parse --show-toplevel`, and compare the returned path to Get-Location.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.16 — Create feature/northstar-platform; do not work directly on main.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Create feature/northstar-platform; do not work directly on main. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Create feature/northstar-platform; do not work directly on main.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 7

Action

Create feature/northstar-platform; do not work directly on main.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.17 — Open VS Code with code .. In Explorer, locate README, LICENSE, datasets, docs, and scripts. In Source Contro

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open VS Code with code .. In Explorer, locate README, LICENSE, datasets, docs, and scripts. In Source Contro. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open VS Code with code .. In Explorer, locate README, LICENSE, datasets, docs, and scripts. In Source Control, confirm there are no unreviewed changes yet.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 8

Action

Open VS Code with code .. In Explorer, locate README, LICENSE, datasets, docs, and scripts. In Source Control, confirm there are no unreviewed changes yet.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.18 — Before creating any new project file, create scripts/legacy; if scripts/init_database.sql exists, copy it

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Before creating any new project file, create scripts/legacy; if scripts/init_database.sql exists, copy it . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Before creating any new project file, create scripts/legacy; if scripts/init_database.sql exists, copy it there as init_database.original.sql.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 9****Action**

Before creating any new project file, create scripts/legacy; if scripts/init_database.sql exists, copy it there as init_database.original.sql.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#)

Action 1.2.C1 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Get-Location
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** Get-Location — Get-Location prints the current PowerShell folder

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.2.C2 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Get-ChildItem -Directory | Select-Object Name
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `Get-ChildItem -Directory | Select-Object Name` — `Get-ChildItem` lists files and folders; `-Directory` is a command option or flag; its exact behavior is defined beside the command; `Select-Object` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.2.C3 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
git clone https://github.com/Losboules/SQL-Data-Warehouse-Project.git
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git clone https://github.com/Losboules/SQL-Data-Warehouse-Project.git` — `git` calls the Git version-control program; `clone` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `https` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `://` is a path, relative location, branch-style name, or file argument; verify every character before running it; `github.com` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Losboules` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `SQL-Data-Warehouse-Project.git` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.2.C4 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Set-Location .\SQL-Data-Warehouse-Project
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** Set-Location .\SQL-Data-Warehouse-Project — Set-Location changes the current PowerShell folder; . is a path, relative location, branch-style name, or file argument; verify every character before running it; SQL-Data-Warehouse-Project is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.2.C5 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
git rev-parse --show-toplevel
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.2.C6 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
git status
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.2.C7 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
git switch -c feature/northstar-platform
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git switch -c feature/northstar-platform` — `git` calls the Git version-control program; `switch` changes the active branch; `-c` creates the named branch before switching; `feature` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `northstar-platform` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.2.C8 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
New-Item -ItemType Directory -Path scripts\legacy -ErrorAction Stop
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path scripts\legacy -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; scripts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; legacy is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.2.C9 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
if (Test-Path scripts\init_database.sql) { Copy-Item scripts\init_database.sql scripts\legacy\init_database.original.sql -Force }
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `if (Test-Path scripts\init_database.sql) { Copy-Item scripts\init_database.sql scripts\legacy\init_database.original.sql -Force }` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Test-Path` returns True or False for path existence; `scripts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `init_database.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Copy-Item` copies a selected file or folder; `scripts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `init_database.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scripts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `legacy` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use

git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.2.C10 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
code .
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `code` . — `code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 1.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]



I distinguished typed text from observed output — [STATUS]



I recorded differences caused by versions/tier — [STATUS]



I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 1 starting-screen checklist and re-verify the active context.

Lesson 1.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 1 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — docs/

Exact folder name: docs

Exact parent: repository root

Why it is needed now: Phase 1 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: docs.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 1.D1.1 — Create the folder docs

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\docs" -PathType Container` and require True.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\docs" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\docs" -PathType Container`.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — README.md

Exact filename: README.md

Filename extension: .md. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: . relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 1.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in . one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.

6. CLICK: Select **New File**.
7. TYPE THIS: README.md.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled README.md and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 1.F1.1 — Create the file README.md from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\README.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType File -Path ".\README.md" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; File is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\README.md" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\README.md" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- README.md`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.

5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use TODO(USER) where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under ..
2. Right-click the editor tab and choose **Copy Path**; compare the ending with README.md.
3. In PowerShell at the repository root, run Test-Path ".\README.md" and require True.
4. Run git status --short -- "README.md" to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for README.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: README.md

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 1.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Northstar Retail End-to-End Data Platform

> A beginner-owned, evidence-driven portfolio project for learning SQL Server, PostgreSQL, Python, Databricks, Delta Lake, dimensional modeling, Lakeflow Jobs, SQL Server warehousing, and Power BI.

**Original repository:** [https://github.com/Losboules/SQL-Data-Warehouse-Project](https://github.com/Losboules/SQL-Data-Warehouse-Project)
**Reference design assembled:** 2026-08-16
**Status:** This is an instructional reference. Kirolos must manually create, run, test, and verify each component before claiming it as completed.

## Business problem

Northstar Retail is a fictional omnichannel retailer. Leadership receives inconsistent answers because ERP sales/inventory, website behavior, marketing, shipments, returns, and supplier updates live in separate systems. This project creates a governed daily path from those sources to trusted dimensions, facts, KPIs, and a Power BI report.

## Architecture

(docs/images/overall_architecture.svg)

### Track A – default local-first path

1. Generate fictional source files with Python.
2. Load ERP data into local SQL Server and digital data into local PostgreSQL.
3. Extract timestamped Parquet/file batches with manifests and checksums.
4. Manually upload one batch folder to a Databricks Unity Catalog volume.
5. Run parameterized Bronze, Silver, Gold, quality, and publication tasks.
6. Download the validated Gold export.
7. Load it transactionally into local `NorthstarRetail_DW`.
8. Import the star schema into Power BI and validate every measure against SQL.

Databricks does not connect to the local machine's `localhost` in Track A. Manual upload/download steps are explicit learning boundaries.

### Track B – optional cloud-connected path

Use network-accessible managed SQL Server/Azure SQL and PostgreSQL, supported JDBC connections, controlled firewall/private networking, secret-managed identities, and cloud storage. This can incur database, compute, storage, egress, networking, and BI costs.

## Technology stack

| Area | Technology |
| --- | --- |
| Version control | Git, GitHub, GitHub Actions |
| Local development | Windows PowerShell, VS Code, Python 3.13.x |
| ERP source and serving warehouse | SQL Server + SSMS |
| Digital source | PostgreSQL + pgAdmin/psql |
| Lakehouse | Databricks, PySpark, Spark SQL, Delta Lake, Unity Catalog volumes |
| Orchestration | Databricks Lakeflow Jobs |
| Modeling | Bronze/Silver/Gold, star schema, SCD Type 1/2, unknown members |
| Semantic/consumption | Power BI and Databricks SQL views |
| Trust | pytest, SQL assertions, quality tables, manifests, checksums, reconciliation |

## Data sources

- SQL Server ERP: customers, addresses, products, categories, suppliers, stores, employees, orders, items, payments, shipments, inventory movements and snapshots.
- PostgreSQL digital: users, sessions, events, campaigns, touchpoints, and marketing spend.
- Files: returns CSV, supplier cost updates CSV, promotion calendar CSV, nested shipment tracking JSONL.
```

See [``docs/data_dictionary.md``](docs/data_dictionary.md) and [``docs/source_to_target_mapping.md``](docs/source_to_target_mapping.md).

Pipeline flow

`SQL Server ERP + PostgreSQL digital + CSV/JSON feeds → timestamped local extraction batch → Databricks Bronze → Silver → Gold → blocking quality gate → Gold export → NorthstarRetail\_DW → Power BI / analyst SQL.`

Track A deliberately uses manual upload and download boundaries; Track B replaces those boundaries with network-accessible sources and secured cloud storage.

Medallion layers

- **Bronze:** raw values, source file/system, ingestion timestamp, batch ID, and record hash.
- **Silver:** typed, trimmed, standardized, deduplicated, conformed, quality-flagged, and quarantined records.
- **Gold:** nine dimensions, six facts, SCD2 history for customer/product, unknown members, and governed aggregates/views.

Data model

![Gold star schema](docs/images/gold_star_schema.svg)

Every fact's grain is documented in the workbook and data dictionary. ``fact_sales`` is one valid order item. Inventory is a periodic snapshot and must not be summed across dates without a deliberate calculation.

Data-quality strategy

The generator intentionally creates controlled defects. Silver handles them with explicit rules and quarantine. Required tests cover not-null, uniqueness, accepted values, referential integrity, date/numeric ranges, freshness, schema drift, duplicate batches, source/layer reconciliation, SCD behavior, and revenue arithmetic. Publication depends on a passing quality gate.

Orchestration

![Lakeflow Jobs dependency](docs/images/jobs_dependency.svg)

The version-controlled example is [``databricks/jobs/job_definition.example.yml``](databricks/jobs/job_definition.example.yml). Every task path exists in ``databricks/notebooks/``.

Semantic model and dashboard

Power BI uses single-direction dimension-to-fact relationships, a marked date table, explicit measures, display folders, descriptions, and validation SQL. Report pages cover executive, sales, products, stores, customers, marketing, inventory, shipping/returns, and data quality. No ``pbix`` or result screenshot is fabricated.

How to run the project safely

1. Clone the existing repository

Run in **Windows PowerShell**, from the parent folder where you want the clone:

```
``powershell
git clone https://github.com/Losboules/SQL-Data-Warehouse-Project.git
```

```
Set-Location .\SQL-Data-Warehouse-Project
```

```
git switch -c feature/northstar-platform
```

2. Create Python environment

Run in **Windows PowerShell**, from the repository root:

```
py -3.13 -m venv .venv
```

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

```
.\.venv\Scripts\Activate.ps1
```

```
python -m pip install --upgrade pip
```

```
python -m pip install -r requirements-dev.txt
```

3. Generate and validate quick data

```
python -m scripts.data_generation.generate_northstar_data --scale quick --seed 20260815 --output-dir datasets\sample
```

```
python -m scripts.validation.validate_generated_data --input-dir datasets\sample
```


4. Run local checks

```
python -m scripts.validation.check_required_files
python -m scripts.validation.validate_sql_dialects
python -m scripts.validation.validate_notebooks
python -m scripts.validation.scan_secrets
python -m scripts.validation.validate_markdown_links
python -m pytest -q
```

Follow the single manual-build workbook during construction. Keep this README limited to repository facts and evidence you have personally verified.

Repository structure

```
SQL-Data-Warehouse-Project/
├─ README.md, LICENSE, configuration and dependency files
├─ datasets/           # small sample, ignored raw batches, issue fixtures
├─ docs/              # workbook, phase chapters, diagrams, contracts, portfolio docs
├─ notebooks/         # GitHub showcase notebook and HTML
├─ scripts/           # setup, generation, extraction, loading, validation, utilities
├─ sql/              # dialect-separated SQL Server, PostgreSQL, Databricks SQL
├─ databricks/        # source notebooks and job definition
├─ powerbi/           # model, DAX, page requirements, screenshot checklist
├─ tests/             # unit, integration, data-quality, reconciliation
└─ .github/           # Actions, issue, and pull-request templates
```

During construction, use the file-responsibility index inside the manual-build workbook; do not create a separate file-manifest document.

Screenshots

Replace these placeholders with your own verified evidence:

- [ADD YOUR VERIFIED ROW COUNT]
- [ADD PHASE 4 SOURCE DATABASE SCREENSHOT]
- [ADD PHASE 14 JOB RUN SCREENSHOT]
- [ADD PHASE 15 WAREHOUSE RECONCILIATION SCREENSHOT]
- [ADD PHASE 16 POWER BI MODEL SCREENSHOT]
- [ADD PHASE 17 DASHBOARD SCREENSHOT]

Limitations

- Track A has manual cloud handoffs and daily batch latency.
- The sample and quick generator are reference material; record your own local test result before claiming verification. External-platform execution is not claimed.
- Beginner reporting assumes USD and simplified converting-session attribution.
- The Gold fact notebook reference includes a fact_sales teaching pattern and TODO(USER) ownership work for the remaining facts; optional answers appear only inside the manual-build workbook.
- The project is educational, not production certification.

Future improvements

See [docs/optional_extensions.md](#). Choose one only after the required checklist is personally verified.

What I learned

Replace this section after completion. Suggested prompts: How did grain prevent a mistake? Which failure did the quality gate catch? Why was SCD2 needed? What made a rerun safe? How did SQL and DAX validation differ?

[WRITE YOUR OWN VERIFIED REFLECTION]

Contact

Kirolos Boules

LinkedIn: [ADD LINK]

Email or portfolio: [ADD SAFE PUBLIC CONTACT]

License

MIT. All sample identities and business entities are fictional.

Line-by-line explanation

```
- **Line 1:** `# Northstar Retail End-to-End Data Platform` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** `> A beginner-owned, evidence-driven portfolio project for learning SQL Server, PostgreSQL, Python, Databricks, Delta Lake, dimensional modeling, Lakeflow Jobs, SQL Server warehousing, and Power BI.` - `>` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `A` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `beginner-owned` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `project` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `evidence-driven` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `portfolio` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `learning` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Server` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** `**Original repository:** [https://github.com/Losboules/SQL-Data-Warehouse-Project](https://github.com/Losboules/SQL-Data-Warehouse-Project)` - `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Original` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `repository` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `https` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `://` is a path, relative location, branch-style name, or file argument; verify every character before running it; `github.com` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Losboules` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `SQL-Data-Warehouse-Project` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 6:** `**Reference design assembled:** 2026-08-16` - `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Reference` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `design` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `assembled` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2026-08-16` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.
- **Line 7:** `**Status:** This is an instructional reference. Kirolos must manually create, run, test, and verify each component before claiming it as completed.` - `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Status` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `This` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `an` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `instructional` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `reference` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Kirolos` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `must` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manually` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** `## Business problem` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 10:** blank line used only to separate logical sections for readability.
- **Line 11:** `Northstar Retail is a fictional omnichannel retailer. Leadership receives inconsistent answers because ERP sales/inventory, website behavior, marketing, shipments, returns, and supplier updates live in separate systems. This project creates a governed daily path from those sources to trusted dimensions, facts, KPIs, and a Power BI report.` - `Northstar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Retail` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fictional` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `omnichannel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `retailer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Leadership` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `receives` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `inconsistent` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `answers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
```

- **Line 12:** blank line used only to separate logical sections for readability.

- **Line 13:** `## Architecture` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 14:** blank line used only to separate logical sections for readability.

- **Line 15:** `[Northstar Retail architecture](docs/images/overall architecture.svg)` - `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Northstar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Retail` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `architecture` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `images` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `overall architecture.svg` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.

- **Line 16:** blank line used only to separate logical sections for readability.

- **Line 17:** `### Track A - default local-first path` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 18:** blank line used only to separate logical sections for readability.

- **Line 19:** `1. Generate fictional source files with Python.` - `1` is a numeric literal used by the command or expression;`.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Generate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fictional` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `files` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Python` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;`.` is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 20:** `2. Load ERP data into local SQL Server and digital data into local PostgreSQL.` - `2` is a numeric literal used by the command or expression;`.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Load` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ERP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `into` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `local` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Server` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `digital` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 21:** `3. Extract timestamped Parquet/file batches with manifests and checksums.` - `3` is a numeric literal used by the command or expression;`.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Extract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestamped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Parquet` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batches` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manifests` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `checksums` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 22:** `4. Manually upload one batch folder to a Databricks Unity Catalog volume.` - `4` is a numeric literal used by the command or expression;`.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Manually` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `upload` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `folder` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `to` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Databricks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Unity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Catalog` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 23:** `5. Run parameterized Bronze, Silver, Gold, quality, and publication tasks.` - `5` is a numeric literal used by the command or expression;`.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Run` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parameterized` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Bronze` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;`,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Silver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;`,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Gold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;`,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quality` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;`,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 24:** `6. Download the validated Gold export.` - `6` is a numeric literal used by the command or expression;`.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Download` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `validated` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Gold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `export` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;`.` is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 25:** `7. Load it transactionally into local \NorthstarRetail\_DW\.` - `7` is a numeric literal used by the command or expression;`.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Load` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `it` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `transactionally` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; `into` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `local` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NorthstarRetail\_DW` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 26:** `8. Import the star schema into Power BI and validate every measure against SQL.` - `8` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `star` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `into` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Power` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BI` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `validate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `every` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 27:** blank line used only to separate logical sections for readability.

- **Line 28:** `Databricks does \*\*not\*\* connect to the local machine's `localhost` in Track A. Manual upload/download steps are explicit learning boundaries.` - `Databricks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `does` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\*\*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\*\*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connect` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `to` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `local` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `machine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `s` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `localhost` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 29:** blank line used only to separate logical sections for readability.

- **Line 30:** `### Track B - optional cloud-connected path` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 31:** blank line used only to separate logical sections for readability.

- **Line 32:** `Use network-accessible managed SQL Server/Azure SQL and PostgreSQL, supported JDBC connections, controlled firewall/private networking, secret-managed identities, and cloud storage. This can incur database, compute, storage, egress, networking, and BI costs.` - `Use` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `network-accessible` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `managed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Server` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Azure` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PostgreSQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `supported` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 33:** blank line used only to separate logical sections for readability.

- **Line 34:** `## Technology stack` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 35:** blank line used only to separate logical sections for readability.

- **Line 36:** `| Area | Technology |` - `Area` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Technology` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 37:** `[---]` - `---` is a command option or flag; its exact behavior is defined beside the command; `---` is a command option or flag; its exact behavior is defined beside the command

- **Line 38:** `| Version control | Git, GitHub, GitHub Actions |` - `Version` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `control` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Git` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `GitHub` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Actions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 39:** `| Local development | Windows PowerShell, VS Code, Python 3.13.x |` - `Local` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `development` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Windows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PowerShell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Python` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3.13.x` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 40:** `| ERP source and serving warehouse | SQL Server + SSMS |` - `ERP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `serving` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `warehouse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Server` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SSMS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding

```

explanation
- **Line 41:** `| Digital source | PostgreSQL + pgAdmin/psql |` - `Digital` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PostgreSQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pgAdmin` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `psql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** `| Lakehouse | Databricks, PySpark, Spark SQL, Delta Lake, Unity Catalog volumes |` - `Lakehouse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Databricks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PySpark` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Spark` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Delta` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Lake` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Unity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 43:** `| Orchestration | Databricks Lakeflow Jobs |` - `Orchestration` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Databricks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Lakeflow` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Jobs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 44:** `| Modeling | Bronze/Silver/Gold, star schema, SCD Type 1/2, unknown members |` - `Modeling` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Bronze` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Silver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Gold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `star` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Type` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 45:** `| Semantic/consumption | Power BI and Databricks SQL views |` - `Semantic` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `consumption` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Power` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BI` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Databricks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `views` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** `| Trust | pytest, SQL assertions, quality tables, manifests, checksums, reconciliation |` - `Trust` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pytest` is the Python test runner; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `assertions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quality` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tables` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manifests` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `checksums` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 47:** blank line used only to separate logical sections for readability.
- **Line 48:** `## Data sources` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 49:** blank line used only to separate logical sections for readability.
- **Line 50:** `SQL Server ERP: customers, addresses, products, categories, suppliers, stores, employees, orders, items, payments, shipments, inventory movements and snapshots.` - `.` is a command option or flag; its exact behavior is defined beside the command; `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Server` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ERP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `addresses` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `products` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `categories` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 51:** `PostgreSQL digital: users, sessions, events, campaigns, touchpoints, and marketing spend.` - `.` is a command option or flag; its exact behavior is defined beside the command; `PostgreSQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `digital` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `users` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `events` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

```

```

tation;`,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;`campaigns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;`,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 52:**`- Files: returns CSV, supplier cost updates CSV, promotion calendar CSV, nested shipment tracking JSONL.` - `` is a command option or flag; its exact behavior is defined beside the command; `Files` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `returns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CSV` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `supplier` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cost` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `updates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CSV` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `promotion` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 53:** blank line used only to separate logical sections for readability.
- **Line 54:**`See [\docs\data_dictionary.md](docs/data_dictionary.md) and [\docs/source_to_target_mapping.md](docs/source_to_target_mapping.md).` - `See` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a path, relative location, branch-style name, or file argument; verify every character before running it; `data_dictionary.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a path, relative location, branch-style name, or file argument; verify every character before running it; `data_dictionary.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 55:** blank line used only to separate logical sections for readability.
- **Line 56:**`## Pipeline flow` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 57:** blank line used only to separate logical sections for readability.
- **Line 58:**`SQL Server ERP + PostgreSQL digital + CSV/JSON feeds -> timestamped local extraction batch -> Databricks Bronze -> Silver -> Gold -> Blocking quality gate -> Gold export -> NorthstarRetail_DW -> Power BI / analyst SQL.` - `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Server` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ERP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PostgreSQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `digital` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CSV` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a path, relative location, branch-style name, or file argument; verify every character before running it; `JSON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `feeds` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestamped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 59:** blank line used only to separate logical sections for readability.
- **Line 60:**`Track A deliberately uses manual upload and download boundaries; Track B replaces those boundaries with network-accessible sources and secured cloud storage.` - `Track` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `A` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `deliberately` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `uses` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `upload` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `download` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `boundaries` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Track` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `B` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `replaces` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 61:** blank line used only to separate logical sections for readability.
- **Line 62:**`## Medallion layers` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 63:** blank line used only to separate logical sections for readability.
- **Line 64:**`*Bronze:* raw values, source file/system, ingestion timestamp, batch ID, and record hash.` - `` is a command option or flag; its exact behavior is defined beside the command; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Bronze` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `values` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a path, relative location, branch-style name, or file argument; verify every character before running it; `system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 65:**`*Silver:* typed, trimmed, standardized, deduplicated, conformed, quality-flagged, and quarantined records.` - `` is a command option or flag; its exact behavior is defined beside the command; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Silver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `typed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `trimmed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `deduplicated` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `conformed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quality-flagged` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quarantined` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

```

```

- 'is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'deduplicated' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 66:** - *Gold:** nine dimensions, six facts, SCD2 history for customer/product, unknown members, and governed aggregates/views.' - ' is a command option or flag; its exact behavior is defined beside the command; '**' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Gold' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ':**' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'nine' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dimensions' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'six' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'facts' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'SCD2' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'history' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 67:** blank line used only to separate logical sections for readability.
- **Line 68:** '## Data model' is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 69:** blank line used only to separate logical sections for readability.
- **Line 70:** '!Gold star schema(docs/images/gold_star_schema.svg)' - ' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Gold' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'star' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'schema' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '()' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'docs' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'images' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'gold_star_schema.svg' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ')' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 71:** blank line used only to separate logical sections for readability.
- **Line 72:** 'Every fact's grain is documented in the workbook and data dictionary. \fact_sales\' is one valid order item. Inventory is a periodic snapshot and must not be summed across dates without a deliberate calculation.' - 'Every' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'fact' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 's' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'grain' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'is' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'documented' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'in' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'the' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'workbook' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'and' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'data' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dictionary' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 73:** blank line used only to separate logical sections for readability.
- **Line 74:** '## Data-quality strategy' is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 75:** blank line used only to separate logical sections for readability.
- **Line 76:** 'The generator intentionally creates controlled defects. Silver handles them with explicit rules and quarantine. Required tests cover not-null, uniqueness, accepted values, referential integrity, date/numeric ranges, freshness, schema drift, duplicate batches, source/layer reconciliation, SCD behavior, and revenue arithmetic. Publication depends on a passing quality gate.' - 'The' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'generator' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'intentionally' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'creates' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'controlled' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'defects' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'Silver' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'handles' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'them' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'with' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'explicit' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 77:** blank line used only to separate logical sections for readability.
- **Line 78:** '## Orchestration' is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 79:** blank line used only to separate logical sections for readability.
- **Line 80:** '!Lakeflow Jobs dependency(docs/images/jobs_dependency.svg)' - ' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Lakeflow' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Jobs' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dependency' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '()' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'docs' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'images' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'jobs_dependency.svg' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ')' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 81:** blank line used only to separate logical sections for readability.
- **Line 82:** 'The version-controlled example is [\'databricks/jobs/job_definition.example.yml\'](databricks/jobs/job_definition.example.yml). Every task path exists in \'databricks/notebooks/\'. - 'The' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'version-controlled' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'example' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'is' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '[' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'databricks' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'jobs' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'job_definition' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'example' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ']' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'databricks/notebooks/' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'databricks' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'jobs' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'job_definition' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'example' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

```

role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'jobs' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'job_definition.example.yml' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ']' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'databricks' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 83:** blank line used only to separate logical sections for readability.
- **Line 84:** `## Semantic model and dashboard` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 85:** blank line used only to separate logical sections for readability.
- **Line 86:** `Power BI uses single-direction dimension-to-fact relationships, a marked date table, explicit measures, display folders, descriptions, and validation SQL. Report pages cover executive, sales, products, stores, customers, marketing, inventory, shipping/returns, and data quality. No '\.pbix\' or result screenshot is fabricated.` - `Power` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BI` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `uses` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `single-direction` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dimension-to-fact` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `relationships` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `marked` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 87:** blank line used only to separate logical sections for readability.
- **Line 88:** `## How to run the project safely` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 89:** blank line used only to separate logical sections for readability.
- **Line 90:** `### 1. Clone the existing repository` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 91:** blank line used only to separate logical sections for readability.
- **Line 92:** `Run in \*\*Windows PowerShell\*\*, from the parent folder where you want the clone:` - `Run` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\*\*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Windows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PowerShell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\*\*`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parent` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `folder` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `where` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `you` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 93:** blank line used only to separate logical sections for readability.
- **Line 94:** `\\.\powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 95:** `git clone https://github.com/Losboules/SQL-Data-Warehouse-Project.git` - `git` calls the Git version-control program; `clone` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `https` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `://` is a path, relative location, branch-style name, or file argument; verify every character before running it; `github.com` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Losboules` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `SQL-Data-Warehouse-Project.git` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 96:** `\\.\` -
- **Line 97:** blank line used only to separate logical sections for readability.
- **Line 98:** `\\.\powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 99:** `Set-Location .\SQL-Data-Warehouse-Project` - `Set-Location` changes the current PowerShell folder; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `SQL-Data-Warehouse-Project` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 100:** `\\.\` -
- **Line 101:** blank line used only to separate logical sections for readability.
- **Line 102:** `\\.\powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 103:** `git switch -c feature/northstar-platform` - `git` calls the Git version-control program; `switch` changes the active branch; `-c` creates the named branch before switching; `feature` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `northstar-platform` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 104:** `\\.\` -
- **Line 105:** blank line used only to separate logical sections for readability.
- **Line 106:** `### 2. Create Python environment` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 107:** blank line used only to separate logical sections for readability.
- **Line 108:** `Run in \*\*Windows PowerShell\*\*, from the repository root:` - `Run` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\*\*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Windows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PowerShell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\*\*`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `repository` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `root` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;


```

tion; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 109:** blank line used only to separate logical sections for readability.
- **Line 110:** ``powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is de-
scribed in the surrounding explanation
- **Line 111:** `py -3.13 -m venv .venv` - `py` invokes the Windows Python launcher or Python Install Manager command shim; `-3` is a command
option or flag; its exact behavior is defined beside the command; `.` is a path, relative location, branch-style name, or file argument; veri-
fy every character before running it; `13` is a numeric literal used by the command or expression; `-m` asks Python to run an installed mod-
ule; `venv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.`
is a path, relative location, branch-style name, or file argument; verify every character before running it; `venv` is a name, keyword, argu-
ment, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 112:** `` -
- **Line 113:** blank line used only to separate logical sections for readability.
- **Line 114:** ``powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is de-
scribed in the surrounding explanation
- **Line 115:** `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` - `Set-ExecutionPolicy` is a name, keyword, argument, identifier,
or literal whose project-specific role is described in the surrounding explanation; `-Scope` is a command option or flag; its exact behavior
is defined beside the command; `Process` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the
surrounding explanation; `-ExecutionPolicy` is a command option or flag; its exact behavior is defined beside the command; `Bypass` is a name,
keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 116:** `` -
- **Line 117:** blank line used only to separate logical sections for readability.
- **Line 118:** ``powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is de-
scribed in the surrounding explanation
- **Line 119:** `.\venv\Scripts\Activate.ps1` - `.` is a path, relative location, branch-style name, or file argument; verify every character
before running it; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `venv` is
a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Scripts` is a
name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Activate.ps1` is a
name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 120:** `` -
- **Line 121:** blank line used only to separate logical sections for readability.
- **Line 122:** ``powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is de-
scribed in the surrounding explanation
- **Line 123:** `python -m pip install --upgrade pip` - `python` invokes Python when that command name is registered; `-m` asks Python to run
an installed module; `pip` the Python package installer module; `install` is a name, keyword, argument, identifier, or literal whose project-
specific role is described in the surrounding explanation; `--upgrade` is a command option or flag; its exact behavior is defined beside the
command; `pip` the Python package installer module
- **Line 124:** `` -
- **Line 125:** blank line used only to separate logical sections for readability.
- **Line 126:** ``powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is de-
scribed in the surrounding explanation
- **Line 127:** `python -m pip install -r requirements-dev.txt` - `python` invokes Python when that command name is registered; `-m` asks Py-
thon to run an installed module; `pip` the Python package installer module; `install` is a name, keyword, argument, identifier, or literal
whose project-specific role is described in the surrounding explanation; `-r` is a command option or flag; its exact behavior is defined be-
side the command; `requirements-dev.txt` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the
surrounding explanation
- **Line 128:** `` -
- **Line 129:** blank line used only to separate logical sections for readability.
- **Line 130:** `### 3. Generate and validate quick data` is a comment for a human reader; it normally does not change data or execute busi-
ness logic.
- **Line 131:** blank line used only to separate logical sections for readability.
- **Line 132:** ``powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is de-
scribed in the surrounding explanation
- **Line 133:** `python -m scripts.data_generation.generate_northstar_data --scale quick --seed 20260815 --output-dir datasets\sample` - `py-
thon` invokes Python when that command name is registered; `-m` asks Python to run an installed module;
`scripts.data_generation.generate_northstar_data` is a name, keyword, argument, identifier, or literal whose project-specific role is de-
scribed in the surrounding explanation; `--scale` is a command option or flag; its exact behavior is defined beside the command; `quick` is a
name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--seed` is a command
option or flag; its exact behavior is defined beside the command; `20260815` is a numeric literal used by the command or expression; `--out-
put` is a command option or flag; its exact behavior is defined beside the command; `-dir` is a command option or flag; its exact behavior is
defined beside the command; `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the
surrounding explanation; `sample` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the sur-
rounding explanation
- **Line 134:** `` -
- **Line 135:** blank line used only to separate logical sections for readability.
- **Line 136:** ``powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is de-
scribed in the surrounding explanation
- **Line 137:** `python -m scripts.validation.validate_generated_data --input-dir datasets\sample` - `python` invokes Python when that command
name is registered; `-m` asks Python to run an installed module; `scripts.validation.validate_generated_data` is a name, keyword, argument,
identifier, or literal whose project-specific role is described in the surrounding explanation; `--input` is a command option or flag; its ex-
act behavior is defined beside the command; `-dir` is a command option or flag; its exact behavior is defined beside the command; `datasets`
is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sample` is a
name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 138:** `` -
- **Line 139:** blank line used only to separate logical sections for readability.
- **Line 140:** `### 4. Run local checks` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 141:** blank line used only to separate logical sections for readability.
- **Line 142:** ``powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is de-
scribed in the surrounding explanation
- **Line 143:** `python -m scripts.validation.check_required_files` - `python` invokes Python when that command name is registered; `-m` asks
Python to run an installed module; `scripts.validation.check_required_files` is a name, keyword, argument, identifier, or literal whose pro-
ject-specific role is described in the surrounding explanation
- **Line 144:** `python -m scripts.validation.validate_sql_dialects` - `python` invokes Python when that command name is registered; `-m` asks
Python to run an installed module; `scripts.validation.validate_sql_dialects` is a name, keyword, argument, identifier, or literal whose pro-
ject-specific role is described in the surrounding explanation
- **Line 145:** `python -m scripts.validation.validate_notebooks` - `python` invokes Python when that command name is registered; `-m` asks
Python to run an installed module; `scripts.validation.validate_notebooks` is a name, keyword, argument, identifier, or literal whose project-
specific role is described in the surrounding explanation

```

```

- **Line 146:** `python -m scripts.validation.scan_secrets` - `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.scan_secrets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 147:** `python -m scripts.validation.validate_markdown_links` - `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.validate_markdown_links` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 148:** `python -m pytest -q` - `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `pytest` the Python test runner; `-q` is a command option or flag; its exact behavior is defined beside the command
- **Line 149:** `` -
- **Line 150:** blank line used only to separate logical sections for readability.
- **Line 151:** `Follow the single manual-build workbook during construction. Keep this README limited to repository facts and evidence you have personally verified.` - `Follow` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `single` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manual-build` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `workbook` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `during` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `construction` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Keep` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `this` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `README` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `limited` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 152:** blank line used only to separate logical sections for readability.
- **Line 153:** `## Repository structure` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 154:** blank line used only to separate logical sections for readability.
- **Line 155:** ``text` - `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 156:** `SQL-Data-Warehouse-Project/` - `SQL-Data-Warehouse-Project` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 157:** `| README.md, LICENSE, configuration and dependency files` - `README.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `LICENSE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `configuration` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dependency` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `files` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 158:** `| datasets/` # small sample, ignored raw batches, issue fixtures` - `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `small` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sample` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ignored` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batches` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `issue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fixtures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 159:** `| docs/` # workbook, phase chapters, diagrams, contracts, portfolio docs` - `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `workbook` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `phase` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `chapters` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `diagrams` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `contracts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `portfolio` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 160:** `| notebooks/` # GitHub showcase notebook and HTML` - `notebooks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `GitHub` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `showcase` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `notebook` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `HTML` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 161:** `| scripts/` # setup, generation, extraction, loading, validation, utilities` - `scripts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `setup` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `generation` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `extraction` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `loading` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `validation` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 162:** `| sql/` # dialect-separated SQL Server, PostgreSQL, Databricks SQL` - `sql` is a name, keyword, argument,

```

identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'dialect-separated' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'SQL' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Server' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ',' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'PostgreSQL' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '.' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Databricks' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'SQL' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 163:** |— databricks/ # source notebooks and job definition - 'databricks' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'source' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'notebooks' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'and' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'job' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'definition' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 164:** |— powerbi/ # model, DAX, page requirements, screenshot checklist - 'powerbi' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'model' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ',' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'DAX' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'page' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'requirements' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ',' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'screenshot' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'checklist' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 165:** |— tests/ # unit, integration, data-quality, reconciliation - 'tests' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'unit' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ',' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'integration' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'data-quality' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ',' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'reconciliation' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 166:** |— .github/ # Actions, issue, and pull-request templates - '.' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'github' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'Actions' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ',' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'issue' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ',' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'and' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'pull-request' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'templates' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 167:** |— |— |— |— |—

- **Line 168:** blank line used only to separate logical sections for readability.

- **Line 169:** 'During construction, use the file-responsibility index inside the manual-build workbook; do not create a separate file-manifest document.' - 'During' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'construction' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ',' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'use' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'the' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'file-responsibility' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'index' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'inside' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'the' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'manual-build' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'workbook' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'do' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 170:** blank line used only to separate logical sections for readability.

- **Line 171:** '## Screenshots' is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 172:** blank line used only to separate logical sections for readability.

- **Line 173:** 'Replace these placeholders with your own verified evidence:' - 'Replace' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'these' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'placeholders' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'with' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'your' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'own' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'verified' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'evidence' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ':' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 174:** blank line used only to separate logical sections for readability.

- **Line 175:** '- [ADD YOUR VERIFIED ROW COUNT]' - '-' is a command option or flag; its exact behavior is defined beside the command; '[' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'ADD' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'YOUR' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'VERIFIED' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'ROW' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'COUNT' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ']' is a name, keyword, argument, identifier, or

literal whose project-specific role is described in the surrounding explanation

- **Line 176:** blank line used only to separate logical sections for readability.

- **Line 177:** ` ` - ` \[ADD PHASE 4 SOURCE DATABASE SCREENSHOT]\` - ` ` is a command option or flag; its exact behavior is defined beside the command; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ADD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PHASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `4` is a numeric literal used by the command or expression; `SOURCE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATABASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCREENSHOT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 178:** ` ` - ` \[ADD PHASE 14 JOB RUN SCREENSHOT]\` - ` ` is a command option or flag; its exact behavior is defined beside the command; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ADD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PHASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `14` is a numeric literal used by the command or expression; `JOB` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `RUN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCREENSHOT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 179:** ` ` - ` \[ADD PHASE 15 WAREHOUSE RECONCILIATION SCREENSHOT]\` - ` ` is a command option or flag; its exact behavior is defined beside the command; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ADD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PHASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `15` is a numeric literal used by the command or expression; `WAREHOUSE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `RECONCILIATION` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCREENSHOT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 180:** ` ` - ` \[ADD PHASE 16 POWER BI MODEL SCREENSHOT]\` - ` ` is a command option or flag; its exact behavior is defined beside the command; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ADD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PHASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `16` is a numeric literal used by the command or expression; `POWER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BI` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `MODEL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCREENSHOT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 181:** ` ` - ` \[ADD PHASE 17 DASHBOARD SCREENSHOT]\` - ` ` is a command option or flag; its exact behavior is defined beside the command; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ADD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PHASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `17` is a numeric literal used by the command or expression; `DASHBOARD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCREENSHOT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 182:** blank line used only to separate logical sections for readability.

- **Line 183:** `## Limitations` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 184:** blank line used only to separate logical sections for readability.

- **Line 185:** ` ` - Track A has manual cloud handoffs and daily batch latency. ` ` - ` ` is a command option or flag; its exact behavior is defined beside the command; `Track` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `A` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `has` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cloud` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `handoffs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `daily` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `latency` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 186:** ` ` - The sample and quick generator are reference material; record your own local test result before claiming verification. External-platform execution is not claimed. ` ` - ` ` is a command option or flag; its exact behavior is defined beside the command; `The` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sample` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quick` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `generator` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `are` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `reference` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `material` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `record` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `your` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `own` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 187:** ` ` - Beginner reporting assumes USD and simplified converting-session attribution. ` ` - ` ` is a command option or flag; its exact behavior is defined beside the command; `Beginner` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `reporting` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `assumes` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `USD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `simplified` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `converting-session` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `attribution` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 188:** ` ` - The Gold fact notebook reference includes a `fact\_sales\` teaching pattern and `TODO(USER)\` ownership work for the remaining facts; optional answers appear only inside the manual-build workbook. ` ` - ` ` is a command option or flag; its exact behavior is defined beside the command; `The` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

rounding explanation; `Gold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `notebook` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `reference` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `includes` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `teaching` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pattern` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 189:** - The project is educational, not production certification. - - is a command option or flag; its exact behavior is defined beside the command; `The` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `project` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `educational` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `production` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `certification` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 190:** blank line used only to separate logical sections for readability.

- **Line 191:** `## Future improvements` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 192:** blank line used only to separate logical sections for readability.

- **Line 193:** `See [\\docs\\optional\_extensions.md\\](docs/optional\_extensions.md). Choose one only after the required checklist is personally verified.` - `See` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\\` is a path, relative location, branch-style name, or file argument; verify every character before running it; `optional\_extensions.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `optional\_extensions.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Choose` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 194:** blank line used only to separate logical sections for readability.

- **Line 195:** `## What I learned` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 196:** blank line used only to separate logical sections for readability.

- **Line 197:** `> Replace this section after completion. Suggested prompts: How did grain prevent a mistake? Which failure did the quality gate catch? Why was SCD2 needed? What made a rerun safe? How did SQL and DAX validation differ?` - `>` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Replace` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `this` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `section` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `after` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `completion` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Suggested` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `prompts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `How` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `did` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 198:** blank line used only to separate logical sections for readability.

- **Line 199:** `[\\WRITE YOUR OWN VERIFIED REFLECTION]\\` - `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `WRITE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `YOUR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `OWN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VERIFIED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `REFLECTION` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\\` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 200:** blank line used only to separate logical sections for readability.

- **Line 201:** `## Contact` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 202:** blank line used only to separate logical sections for readability.

- **Line 203:** `\*\*Kirolos Boules\*\*` - `\*\*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Kirolos` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Boules` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\*\*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 204:** `LinkedIn: \\[ADD LINK]\\` - `LinkedIn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\\` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ADD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `LINK` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\\` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 205:** `Email or portfolio: \\[ADD SAFE PUBLIC CONTACT]\\` - `Email` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `or` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `portfolio` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ADD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SAFE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PUBLIC` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CONTACT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\\` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

```
- **Line 206:** blank line used only to separate logical sections for readability.
- **Line 207:** `## License` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 208:** blank line used only to separate logical sections for readability.
- **Line 209:** `MIT`. All sample identities and business entities are fictional.` – `MIT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;`.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `All` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sample` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `identities` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `business` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `entities` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `are` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fictional` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;`.` is a path, relative location, branch-style name, or file argument; verify every character before running it
```

[Back to the Phase 1 lesson index](#phase-1-lesson-index)

```
<a id="phase-1-file-gitignore"></a>
### File introduction – `.gitignore`
```

```
**Exact filename:** `.gitignore`
**Filename extension:** `(no extension)`. In this project it identifies project text or configuration and helps the correct application choose syntax highlighting or execution behavior.
**Exact parent folder:** `.` relative to the repository root.
**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.
**What the file will contain:** project text or configuration for Phase 1.
**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.
**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.
**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.
**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.
```

VS Code click path

```
1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `.` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `.gitignore`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `.gitignore` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.
```

PowerShell path

```
<a id="action-1-f2-1"></a>
## Action 1.F2.1 – Create the file `.gitignore` from PowerShell
```

```
**Application:**
Windows PowerShell or the VS Code PowerShell terminal
```

```
**Window or panel:**
VS Code integrated PowerShell terminal opened at the repository root
```

```
**Starting condition:**
The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.
```

```
**Current folder or location:**
The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.
```

```
**What you are about to do:**
Run one explained PowerShell instruction and inspect its result before any next command is introduced.
```

```
**Why this action comes now:**
The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.
```

```
**Exact physical actions:**
```

```
1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches `The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.` or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.
```

```
**Command or code (PowerShell):**
```

```
```powershell  
New-Item -ItemType File -Path ".\.\.gitignore" -ErrorAction Stop
```

### Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\.\.gitignore" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\.\.gitignore"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

### How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

## File-creation decision branches

### If the command creates the file

Run `Test-Path -LiteralPath ".\.\.gitignore" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

### If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- ".gitignore"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

### If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

## File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

## File verification

1. In the VS Code Explorer, confirm the file appears under `..`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `.gitignore`.
3. In PowerShell at the repository root, run `Test-Path ".\.\.gitignore"` and require `True`.
4. Run `git status --short -- ".gitignore"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

## Optional reference answer for .gitignore

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** text

**Exact file:** `.gitignore`

**Purpose:** project text or configuration.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 1.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
Python
.venv/
__pycache__/
*.py[cod]
.pytest_cache/
.ruff_cache/
.coverage
htmlcov/

Secrets and local configuration
.env
.env.*
!.env.example
config/dev.yml
*.pem
*.key
*.pfx
*.p12

Generated data: keep only the intentionally small sample folder
/datasets/raw/**
!/datasets/raw/**/.gitkeep
/datasets/generated/**
/datasets/tmp/**

Logs and manifests created during local runs
/logs/**
!/logs/.gitkeep
/run_artifacts/**
```



```
Power BI and database artifacts
*.pbix
*.pbit
*.bak
*.bacpac
*.mdf
*.ldf

Databricks and local tooling
.databricks/
.vscode/settings.json
.idea/

OS
Thumbs.db
.DS_Store
```

## Line-by-line explanation

- **Line 1:** `#` Python is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `.venv/` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `venv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 3:** `__pycache__/` — `__pycache__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 4:** `*.py[cod]` — `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `py` invokes the Windows Python launcher or Python Install Manager command shim; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cod` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.pytest_cache/` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `pytest_cache` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 6:** `.ruff_cache/` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `ruff_cache` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 7:** `.coverage` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `coverage` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `htmlcov/` — `htmlcov` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 9:** blank line used only to separate logical sections for readability.
- **Line 10:** `# Secrets and local configuration` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 11:** `.env` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `.env.*` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 13:** `!.env.example` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `env.example` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** `config/dev.yml` — `config` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `dev.yml` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 15:** \*.pem — \*. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pem is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** \*.key — \*. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** \*.pfx — \*. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pfx is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** \*.p12 — \*. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p12 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** blank line used only to separate logical sections for readability.
- **Line 20:** # Generated data: keep only the intentionally small sample folder is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 21:** /datasets/raw/\*\* — / is a path, relative location, branch-style name, or file argument; verify every character before running it; datasets is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; raw is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; /\*\* is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 22:** !/datasets/raw/\*\*/.gitkeep — / is a path, relative location, branch-style name, or file argument; verify every character before running it; datasets is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; raw is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; /\*\*/. is a path, relative location, branch-style name, or file argument; verify every character before running it; gitkeep is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** /datasets/generated/\*\* — / is a path, relative location, branch-style name, or file argument; verify every character before running it; datasets is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; generated is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; /\*\* is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 24:** /datasets/tmp/\*\* — / is a path, relative location, branch-style name, or file argument; verify every character before running it; datasets is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; tmp is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; /\*\* is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 25:** blank line used only to separate logical sections for readability.
- **Line 26:** # Logs and manifests created during local runs is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 27:** /logs/\*\* — / is a path, relative location, branch-style name, or file argument; verify every character before running it; logs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; /\*\* is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 28:** !/logs/.gitkeep — / is a path, relative location, branch-style name, or file argument; verify every character before running it; logs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; /. is a path, relative location, branch-style name, or file argument; verify every character before running it; gitkeep is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** /run\_artifacts/\*\* — / is a path, relative location, branch-style name, or file argument; verify every character before running it; run\_artifacts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; /\*\* is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 30:** blank line used only to separate logical sections for readability.
- **Line 31:** # Power BI and database artifacts is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 32:** \*.pbix — \*. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pbix is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** \*.pbix — \*. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pbix is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** \*.bak — \*. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bak is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** \*.bacpac — \*. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bacpac is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 36:** \*.mdf — \*. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mdf is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 37:** \*.ldf — \*. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ldf is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 38:** blank line used only to separate logical sections for readability.
- **Line 39:** # Databricks and local tooling is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 40:** .databricks/ — . is a path, relative location, branch-style name, or file argument; verify every character before running it; databricks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 41:** .vscode/settings.json — . is a path, relative location, branch-style name, or file argument; verify every character before running it; vscode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; settings.json is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** .idea/ — . is a path, relative location, branch-style name, or file argument; verify every character before running it; idea is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 43:** blank line used only to separate logical sections for readability.
- **Line 44:** # OS is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 45:** Thumbs.db — Thumbs.db is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** .DS\_Store — . is a path, relative location, branch-style name, or file argument; verify every character before running it; DS\_Store is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 1 lesson index](#)

## File introduction — docs/repository\_current\_state.md

**Exact filename:** repository\_current\_state.md

**Filename extension:** .md. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

**Exact parent folder:** docs relative to the repository root.

**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

**What the file will contain:** project documentation for Phase 1.

**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.

**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

## VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in docs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: repository\_current\_state.md.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled repository\_current\_state.md and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

## PowerShell path

### Action 1.F3.1 — Create the file docs/repository\_current\_state.md from PowerShell

---

#### Application:

Windows PowerShell or the VS Code PowerShell terminal

#### Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

#### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

#### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

#### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

#### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

#### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\repository_current_state.md" -ErrorAction Stop
```

#### Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

## What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\docs\repository_current_state.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs\repository_current_state.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

## Does it change anything?

This can change files, Git state, packages, database objects, or data.

## Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

## What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

## Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

## What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

## How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

## Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

## Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

## Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

## What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

## How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

## File-creation decision branches

### If the command creates the file

Run `Test-Path -LiteralPath ".\docs\repository_current_state.md" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

### If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/repository_current_state.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

## If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

## File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

## File verification

1. In the VS Code Explorer, confirm the file appears under docs.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `docs/repository_current_state.md`.
3. In PowerShell at the repository root, run `Test-Path ".\docs\repository_current_state.md"` and require `True`.
4. Run `git status --short -- "docs/repository_current_state.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

## Optional reference answer for docs/repository\_current\_state.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** Markdown

**Exact file:** `docs/repository_current_state.md`

**Purpose:** project documentation.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 1.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
Inspected Repository State

Repository: https://github.com/Losboules/SQL-Data-Warehouse-Project
Reference inspection date: 2026-08-16

The public repository was inspected while this workbook was authored. Treat the observations below as a time-stamped reference only. You must inspect the current remote and your local clone yourself before changing anything.

Visible reference baseline

- Default branch displayed: `main`.
- Eight commits were visible at reference-inspection time.
- Root files displayed: `README.md` and an MIT `LICENSE`.
- Root folders displayed: `datasets/`, `docs/`, and `scripts/`.
- `datasets/` and `docs/` visibly contained placeholders.
- `scripts/` visibly listed `init_database.sql` and a placeholder.
- The visible README described a SQL Server data warehouse project.

Retrieval limitation

A directory listing can show that `scripts/init_database.sql` exists without proving its current contents. This workbook therefore does
not claim to know or replace that file. Inspect your clone first. When the file exists, preserve a reviewed copy as `scripts/legacy/
init_database.original.sql` before editing or relocating it.

Safe decision

Create new Northstar paths manually only when their phase introduces them. Use clearly separated paths such as `sql/sqlserver/source/` and
`sql/sqlserver/warehouse/`; do not overwrite a root-level legacy script. Begin with a feature branch, a written state record, and a targeted
backup of any file that will actually change.
```

## Line-by-line explanation

- **Line 1:** `# Inspected Repository State` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** `**Repository:** [https://github.com/Losboules/SQL-Data-Warehouse-Project](https://github.com/Losboules/SQL-Data-Warehouse-Project)` — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Repository` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[https://github.com/Losboules/SQL-Data-Warehouse-Project]` is a name, keyword, argument,

identifier, or literal whose project-specific role is described in the surrounding explanation; https is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :// is a path, relative location, branch-style name, or file argument; verify every character before running it; github.com is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; Losboules is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; SQL-Data-Warehouse-Project is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ] ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 4:** `**Reference inspection date:** 2026-08-16 — **` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Reference` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `inspection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2026-08-16` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** `The public repository was inspected while this workbook was authored. Treat the observations below as a time-stamped reference only. You must inspect the current remote and your local clone yourself before changing anything. — The` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `public` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `repository` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `was` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `inspected` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `while` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `this` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `workbook` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `was` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `authored` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Treat` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `## Visible reference baseline` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 9:** blank line used only to separate logical sections for readability.
- **Line 10:** `- Default branch displayed: \main\.` is a command option or flag; its exact behavior is defined beside the command; `Default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `displayed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ``` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 11:** `- Eight commits were visible at reference-inspection time. —` is a command option or flag; its exact behavior is defined beside the command; `Eight` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `commits` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `were` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `visible` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `reference-inspection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `time` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 12:** - Root files displayed: \README.md` and an MIT `LICENSE`.—is a command option or flag; its exact behavior is defined beside the command;Rootis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;filesis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;displayedis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;;is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;README.md is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;andis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;anis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;MITis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;LICENSEis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;.` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 13:** - Root folders displayed: \datasets/, `docs/`, and `scripts/`.—is a command option or flag; its exact behavior is defined beside the command;Rootis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;foldersis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;displayedis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;;is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;datasetsis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;/is a path, relative location, branch-style name, or file argument; verify every character before running it;is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;docsis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;/is a path, relative location, branch-style name, or file argument; verify every character before running it;is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 14:** - \datasets/` and `docs/` visibly contained placeholders.—is a command option or flag; its exact behavior is defined beside the command;datasetsis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;/is a path, relative location, branch-style name, or file argument; verify every character before running it;andis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;docsis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;/is a path, relative location, branch-style name, or file argument; verify every character before running it;visiblyis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;containedis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;placeholdersis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;.` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 15:** - \scripts/` visibly listed `init\_database.sql` and a placeholder.—is a command option or flag; its exact behavior is defined beside the command;scriptsis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;/is a path, relative location, branch-style name, or file argument; verify every character before running it;visiblyis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;listedis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;init\_database.sqlis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;andis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;ais a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;placeholderis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;.` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 16:** - The visible README described a SQL Server data warehouse project. — - is a command option or flag; its exact behavior is defined beside the command; The is a name, keyword, argument, identifier, or literal whose pro-



ject-specific role is described in the surrounding explanation; visible is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; README is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; described is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; warehouse is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; project is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 17:** blank line used only to separate logical sections for readability.
- **Line 18:** ## Retrieval limitation is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 19:** blank line used only to separate logical sections for readability.
- **Line 20:** A directory listing can show that \scripts/init\_database.sql exists without proving its current contents. This workbook therefore does **not** claim to know or replace that file. Inspect your clone first. When the file exists, preserve a reviewed copy as \scripts/legacy/init\_database.original.sql before editing or relocating it.–A is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; listing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; can is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; show is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; that is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; scripts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; init\_database.sql is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; exists is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; without is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; proving is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 21:** blank line used only to separate logical sections for readability.
- **Line 22:** ## Safe decision is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 23:** blank line used only to separate logical sections for readability.
- **Line 24:** Create new Northstar paths manually only when their phase introduces them. Use clearly separated paths such as \sql/sqlserver/source/ and \sql/sqlserver/warehouse/; do not overwrite a root-level legacy script. Begin with a feature branch, a written state record, and a targeted backup of any file that will actually change.–Create is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; new is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Northstar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; manually is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; only is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; when is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; their is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; phase is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; introduces is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; them is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ` is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.

## File introduction — docs/repository\_migration\_plan.md

**Exact filename:** repository\_migration\_plan.md

**Filename extension:** .md. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

**Exact parent folder:** docs relative to the repository root.

**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

**What the file will contain:** project documentation for Phase 1.

**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.

**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

### VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in docs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: repository\_migration\_plan.md.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled repository\_migration\_plan.md and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

### PowerShell path

## Action 1.F4.1 — Create the file docs/repository\_migration\_plan.md from PowerShell

---

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.

4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\repository_migration_plan.md" -ErrorAction Stop
```

### Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\docs\repository_migration_plan.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs\repository_migration_plan.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

## What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

## How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

## File-creation decision branches

### If the command creates the file

Run `Test-Path -LiteralPath ".\docs\repository_migration_plan.md" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

### If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/repository_migration_plan.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

### If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

## File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

## File verification

1. In the VS Code Explorer, confirm the file appears under docs.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `docs/repository_migration_plan.md`.
3. In PowerShell at the repository root, run `Test-Path ".\docs\repository_migration_plan.md"` and require True.
4. Run `git status --short -- "docs/repository_migration_plan.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

## Optional reference answer for docs/repository\_migration\_plan.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** Markdown

**Exact file:** `docs/repository_migration_plan.md`

**Purpose:** project documentation.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 1.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
Safe Manual Preparation Plan for the Existing Repository
```

```
This plan protects the existing public and local repository while you build the project manually. It does not import, copy, or extract a prebuilt project.
```

1. Inspect the current GitHub repository in a browser and record the visible branch, folders, files, and commit count.
2. Locate an existing local clone or clone the repository into a normal parent folder such as ``C:\Users\User\Desktop\Northstar_Retail``.
3. Confirm the true repository root with ``git rev-parse --show-toplevel``.
4. Record ``git status``, ``git branch --show-current``, ``git remote -v``, and the latest commit before editing.
5. If uncommitted work exists, preserve it by committing understood changes on the correct branch or by creating a named patch/stash only after reviewing the diff.
6. Create or switch to ``feature/northstar-platform`` without deleting any existing branch.
7. If ``scripts/init_database.sql`` exists and later work may affect it, create ``scripts/legacy/`` manually and copy only that reviewed file to ``scripts/legacy/init_database.original.sql``.
8. Create ``docs/repository_current_state.md`` and ``docs/repository_migration_plan.md`` manually from the workbook lessons.
9. Create future folders and files only when their phase introduces them; never bulk-create the complete tree.
10. Review targeted diffs, stage only the intended Phase 1 files, commit the preparation work, and verify the branch on GitHub.

```
Suggested commit after you actually complete and verify these steps:
```

```
```text
docs: record repository baseline and manual build plan
```

If README.md needs later revision, preserve its original meaning in Git history and use VS Code diff before editing. Git history is the primary recovery mechanism; a backup copy is added only when the workbook explicitly teaches one for a file at risk.

Line-by-line explanation

```
- **Line 1:** `# Safe Manual Preparation Plan for the Existing Repository` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** `This plan protects the existing public and local repository while you build the project manually. It does not import, copy, or extract a prebuilt project.` – `This` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `plan` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `protects` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `existing` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `public` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `local` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `repository` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `while` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `you` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `build` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** `1. Inspect the current GitHub repository in a browser and record the visible branch, folders, files, and commit count.` – `1` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Inspect` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `current` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `GitHub` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `repository` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `browser` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `record` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 6:** `2. Locate an existing local clone or clone the repository into a normal parent folder such as \C:\Users\User\Desktop\Northstar_Retail\.` – `2` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Locate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `an` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `existing` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `local` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `clone` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `or` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `clone` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `repository` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `into` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 7:** `3. Confirm the true repository root with \git rev-parse --show-toplevel\.` – `3` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Confirm` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `true` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `repository` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `root` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `--toplevel` is a command option or flag; its exact behavior is defined beside the command; remaining punctuation and identifiers continue the same statement.
- **Line 8:** `4. Record \git status\, \git branch --show-current\, \git remote -v\, and the latest commit before editing.` – `4` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Record` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `--current` is a command option or flag; its exact behavior is defined beside the command; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `git` calls the Git version-control program; remaining punctuation and identifiers continue the same statement.
- **Line 9:** `5. If uncommitted work exists, preserve it by committing understood changes on the correct branch or by creating a named patch/stash only after reviewing the diff.` – `5` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `If` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `uncommitted` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `work` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `preserve` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `it` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `by` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `committing` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;
```

rounding explanation; 'understood' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 10:** '6. Create or switch to '\feature/northstar-platform\' without deleting any existing branch.' - '6' is a numeric literal used by the command or expression; '.' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'Create' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'or' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'switch' changes the active branch; 'to' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'feature' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'northstar-platform' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'without' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'deleting' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'any' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 11:** '7. If '\scripts/init database.sql\' exists and later work may affect it, create '\scripts/legacy/' manually and copy only that reviewed file to '\scripts/legacy/init database.original.sql\'.' - '7' is a numeric literal used by the command or expression; '.' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'If' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'scripts' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'init database.sql' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'exists' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'and' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'later' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'work' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'may' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'affect' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 12:** '8. Create '\docs/repository_current_state.md\' and '\docs/repository_migration_plan.md\' manually from the workbook lessons.' - '8' is a numeric literal used by the command or expression; '.' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'Create' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'docs' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'repository_current_state.md' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'and' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'docs' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'repository_migration_plan.md' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'manually' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'from' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 13:** '9. Create future folders and files only when their phase introduces them; never bulk-create the complete tree.' - '9' is a numeric literal used by the command or expression; '.' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'Create' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'future' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'folders' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'and' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'files' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'only' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'when' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'their' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'phase' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'introduces' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 14:** '10. Review targeted diffs, stage only the intended Phase 1 files, commit the preparation work, and verify the branch on GitHub.' - '10' is a numeric literal used by the command or expression; '.' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'Review' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'targeted' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'diffs' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '.' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stage' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'only' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'the' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'intended' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Phase' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '1' is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.

- **Line 15:** blank line used only to separate logical sections for readability.

- **Line 16:** 'Suggested commit after you actually complete and verify these steps:' - 'Suggested' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'commit' records the staged snapshot; 'after' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'you' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'actually' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'complete' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'and' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'verify' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'these' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'steps' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '.' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.

- **Line 17:** blank line used only to separate logical sections for readability.

- **Line 18:** '\text' - 'text' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.

- **Line 19:** 'docs: record repository baseline and manual build plan' - 'docs' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ':' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'record' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'repository' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'baseline' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'and' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'manual' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.

in the surrounding explanation; `build` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `plan` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 20:** ```` -
- **Line 21:** blank line used only to separate logical sections for readability.
- **Line 22:** `If` \README.md\ needs later revision, preserve its original meaning in Git history and use VS Code diff before editing. Git history is the primary recovery mechanism; a backup copy is added only when the workbook explicitly teaches one for a file at risk.` - `If` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `README.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `needs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `later` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `revision` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `preserve` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `its` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `original` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `meaning` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Git` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

[Back to the Phase 1 lesson index](#phase-1-lesson-index)

What your screen should look like now

- Every Phase 1 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 1.3 completion checklist

- [] `README.md` was created by me, saved, located correctly, and inspected – [STATUS]
- [] `.gitignore` was created by me, saved, located correctly, and inspected – [STATUS]
- [] `docs/repository\_current\_state.md` was created by me, saved, located correctly, and inspected – [STATUS]
- [] `docs/repository\_migration\_plan.md` was created by me, saved, located correctly, and inspected – [STATUS]

Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 1.4] (#phase-1-lesson-4) and recheck the root and branch.

Lesson 1.4 – Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Action 1.4.G1 – Inspect the current state without changing it

Application:
Web browser

Window or panel:
Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:
The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:
The repository root `C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.

What you are about to do:
Inspect the current state without changing it. This is one bounded construction or inspection task.

Why this action comes now:
It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.

2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

****Command or code:****

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

****Where to enter it:****
Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

****What each part means:****

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

****What should happen:****
A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

****Expected example:****

> Your exact values may differ. The expected result is the same **type** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

****What may look different:****
Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

****How to verify success:****
Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

****Do not continue unless:****
You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

****Common mistake:****
Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

****Exact recovery:****
Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

****What you just accomplished:****
You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

****How this connects to the project:****
The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#phase-1-lesson-index)

Action 1.4.G2 – Create the smallest required object or decision

****Application:****
Visual Studio Code

****Window or panel:****
Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

****Starting condition:****
The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

****Current folder or location:****
The repository root `C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.

****What you are about to do:****
Create the smallest required object or decision. This is one bounded construction or inspection task.

****Why this action comes now:****
It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

****Exact physical actions:****

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

****Command or code:****

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

****Where to enter it:****

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

****What each part means:****

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

****What should happen:****

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

****Expected example:****

> Your exact values may differ. The expected result is the same **type** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

****What may look different:****

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

****How to verify success:****

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

****Do not continue unless:****

You can point to the visible result and explain what it proves. Mark the action ``VERIFIED BY USER`` only after that check.

****Common mistake:****

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

****Exact recovery:****

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

****What you just accomplished:****

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

****How this connects to the project:****

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#phase-1-lesson-index)

Action 1.4.G3 – Run the phase-specific verification

****Application:****

Web browser

****Window or panel:****

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

****Starting condition:****

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

****Current folder or location:****

The repository root ``C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project``. The ``User`` segment is an example; verify your real Windows username.

****What you are about to do:****

Run the phase-specific verification. This is one bounded construction or inspection task.

****Why this action comes now:****

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

****Exact physical actions:****

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

****Command or code:****

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

****Where to enter it:****

Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

****What each part means:****

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

****What should happen:****

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

****Expected example:****

> Your exact values may differ. The expected result is the same **type** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

****What may look different:****

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

****How to verify success:****

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

****Do not continue unless:****

You can point to the visible result and explain what it proves. Mark the action ``VERIFIED BY USER`` only after that check.

****Common mistake:****

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

****Exact recovery:****

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

****What you just accomplished:****

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

****How this connects to the project:****

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#phase-1-lesson-index)

Action 1.4.G4 – Demonstrate one safe failure or alternate-result branch

****Application:****

Web browser

****Window or panel:****

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

****Starting condition:****

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

****Current folder or location:****

The repository root ``C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project``. The ``User`` segment is an example; verify your real Windows username.

****What you are about to do:****

Demonstrate one safe failure or alternate-result branch. This is one bounded construction or inspection task.

****Why this action comes now:****

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

****Exact physical actions:****

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

****Command or code:****

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

****Where to enter it:****

Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

****What each part means:****

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

****What should happen:****
A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

****Expected example:****

> Your exact values may differ. The expected result is the same **type** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

****What may look different:****
Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

****How to verify success:****
Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

****Do not continue unless:****
You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

****Common mistake:****
Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

****Exact recovery:****
Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

****What you just accomplished:****
You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

****How this connects to the project:****
The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#phase-1-lesson-index)

Action 1.4.G5 – Record evidence and prepare the downstream handoff

****Application:****
Web browser

****Window or panel:****
Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

****Starting condition:****
The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

****Current folder or location:****
The repository root `C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.

****What you are about to do:****
Record evidence and prepare the downstream handoff. This is one bounded construction or inspection task.

****Why this action comes now:****
It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

****Exact physical actions:****

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

****Command or code:****

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

****Where to enter it:****
Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

****What each part means:****

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

****What should happen:****
A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

****Expected example:****

> Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

****What may look different:****
Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

****How to verify success:****
Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

****Do not continue unless:****
You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

****Common mistake:****
Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

****Exact recovery:****
Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

****What you just accomplished:****
You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

****How this connects to the project:****
The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 1 lesson index](#phase-1-lesson-index)

What your screen should look like now

- The new Phase 1 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 1.4 completion checklist

- [] Every required object was built separately – [STATUS]
- [] Every object has a declared responsibility/grain – [STATUS]
- [] Inputs and outputs were inspected – [STATUS]
- [] A rerun or alternate-result branch was tested where applicable – [STATUS]
- [] Counts, keys, measures, relationships, or settings were independently validated – [STATUS]
- [] I can explain one failure and its safest recovery – [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 1.5](#phase-1-lesson-5).

Lesson 1.5 – Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 1 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision – Git branching strategy

****Question:****
How should project changes be isolated from the default branch?

****Options considered:****

- Commit directly to main.
- Use one feature branch for the guided build.
- Use many short-lived issue branches.

****Beginner path selected:****
Use a descriptive feature branch and targeted commits.

****Why this path was selected:****
It teaches branch isolation and review without imposing a complex team model on one learner.

****Tradeoff:****
A long-lived branch can drift from main.

****What a company might do instead:****
Teams often use short-lived branches, protected main, required reviews, and automated checks.

****How to explain this in an interview:****
I used a feature branch so my existing repository state remained recoverable, reviewed diffs before staging, and used targeted commits.

If you see Result A

`git status --short` is empty or shows only changes you intentionally documented. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

The repository contains existing modified or untracked files that must be preserved and explained. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

- [] Required files/objects exist in the exact location.
- [] I personally created or configured them.
- [] I ran the phase commands in the documented context.
- [] I inspected the output rather than relying on a green icon alone.
- [] Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.
- [] Same-batch/rerun behavior is known where applicable.
- [] A controlled failure or realistic alternate-result branch was tested safely where applicable.
- [] No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.
- [] Actual verified result: `[ENTER VERIFIED RESULT]`.

Point-of-failure troubleshooting record

Exact or representative error message:
`[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]`

What it means in plain language:
`[WRITE YOUR INTERPRETATION]`

Most likely cause:
`[WRITE ONE TESTABLE HYPOTHESIS]`

What not to do:
Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:
`[ENTER THE SMALLEST OBSERVATIONAL CHECK]`

How to read the diagnostic result:
`[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:
`[ENTER THE NARROW FIX]`

Alternative fix when applicable:
`[ENTER OR N/A]`

How to verify the fix:
`[ENTER THE INDEPENDENT RECHECK]`

How to return to the main tutorial:
Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: Web browser, File Explorer, Windows PowerShell, VS Code, and GitHub.

Window/panel: Show GitHub file list, local branch/status, and preserved baseline evidence.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: `docs/images/phase01\_repository-inspection-preservation-branching-and-prepar\_verified.png`.

Recommended repository folder: `docs/images/`.

README placement: The section describing Phase 1's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

`[ADD YOUR VERIFIED SCREENSHOT HERE]`

Git checkpoint after the verified Phase 1 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use ****Save All****, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 1.5.G1 – Verify the repository root before Git operations

****Application:****

Windows PowerShell or the VS Code PowerShell terminal

****Window or panel:****

VS Code integrated PowerShell terminal

****Starting condition:****

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

****Current folder or location:****

The repository root `C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.

****What you are about to do:****

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

****Why this action comes now:****

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

****Exact physical actions:****

1. CLICK: Bring ****Windows PowerShell or the VS Code PowerShell terminal**** to the front.
2. LOOK FOR: Confirm the current location matches `The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.` or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press ****Enter**** for a one-line terminal command, ****F5/Execute**** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

****Command or code (PowerShell):****

```
``powershell
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Interpret `git status --short`

- `??` means untracked: Git sees a new path but it is not staged.
- `M` means modified in the working tree but not staged.
- `M` means the staged version differs from the last commit.
- `MM` means a staged version exists and the file has additional unstaged edits.
- `A` means staged as a new file.
- `D` means deletion; stop and investigate unless deletion is deliberate.
- `R` means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 1.5.G4 — Inspect the unstaged diff for `README.md`

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "README.md"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "README.md"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"README.md"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.5.G5 — Stage only the reviewed file README.md

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "README.md"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "README.md"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"README.md"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 1.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.5.G7 — Create the Phase 1 commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "chore: preserve baseline and prepare feature branch"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "chore: preserve baseline and prepare feature branch"` — git calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"chore: preserve baseline and prepare feature branch"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Action 1.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 1 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message chore: preserve baseline and prepare feature branch and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 1.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 1 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

“Phase 1 created the verified boundary described by **Repository inspection, preservation, branching, and preparation**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed.”

Mini quiz

1. Multiple choice: Why does Phase 1 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.

5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** PLANNED, CREATED BY USER, and RUN BY USER describe earlier states; only inspected evidence supports verification.

My Phase 1 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 1.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 2 installs only the tools the inspected repository and planned architecture require.

Phase 1 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 2 — Install and verify the Windows toolchain

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: A protected repository state from Phase 1.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 3 uses the verified editor and Git workflow to create requirements safely.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 2 lesson index

1. [Lesson 2.1 — Understand the phase and starting screen](#)
2. [Lesson 2.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 2.3 — Create and understand the phase files](#)
4. [Lesson 2.4 — Build and validate required objects](#)
5. [Lesson 2.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 2.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** Web browser, Windows Settings, installers, PowerShell, VS Code, SSMS, pgAdmin, Databricks, and Power BI Desktop.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 2 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 1 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 2 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

Web browser, Windows Settings, installers, PowerShell, VS Code, SSMS, pgAdmin, Databricks, and Power BI Desktop. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 2 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 2 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 2 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 2.1 completion checklist

☐

I read why Phase 2 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 2.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

Lesson 2.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 2 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

A documented Windows development environment with Git/GitHub authentication, VS Code, Python 3.13 virtual environment, SQL Server and SSMS, PostgreSQL and pgAdmin/psql, ODBC driver, Power BI Desktop, and an available Databricks workspace option. Docker remains optional.

2. Why it matters

Tool installation is part of engineering because versions, drivers, authentication, services, and interpreters determine whether code can run. Recording the installed version and verification result makes later errors diagnosable instead of mysterious.

3. Words to know

Term	Plain-language definition
Installer	Program that adds software and often configures services or PATH.
PATH	Windows list of folders searched for commands.
Interpreter	The Python executable running your code.
Virtual environment	Project-specific Python packages isolated in <code>.venv</code> .
Database engine	Background service storing/querying data; SSMS/pgAdmin are client tools, not engines.
Instance	A named SQL Server engine installation, such as default or SQLEXPRESS.
Driver	Software allowing Python/BI/Spark to communicate with a database.
Authentication	How a system proves identity.
Extension	VS Code add-on for language/tool support.
Workspace	Databricks environment containing notebooks, data objects, compute, SQL, and Jobs.
Stable release	Supported production release, not preview/beta.
Quota	Usage limit that can stop or restrict free-tier work.

4. What must already be completed

Phase 1 clone/branch is prepared, and the verified repository state from Phase 1 is available. You have permission to install software on the Windows computer. If the computer is employer-managed, follow employer policy instead of bypassing controls.

5. Applications to open

Application	How to open it	Why it is open
Web browser	Open from Start.	Download only from official Git, GitHub, Microsoft, Python, PostgreSQL, pgAdmin, Databricks pages.
Windows PowerShell	Open a normal user session.	Run verification commands and create .venv.
Windows Settings / installers	Open downloaded signed installers.	Install local tools.
VS Code	Open after install.	Add extensions and select interpreter.
SSMS / pgAdmin / Power BI	Open each from Start after installation.	Verify first launch and record version.
Databricks browser workspace	Sign in to an available account/workspace.	Confirm tier-specific capabilities before relying on them.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run Get-Location; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
git --version	Prints the Git command version.	Confirms install and PATH.
code --version	Prints VS Code version/build information.	Confirms command-line launch is available.
py -3.13 --version	Uses the Windows Python launcher to select Python 3.13.	Avoids accidentally creating .venv with another interpreter.
py -3.13 -m venv .venv	Creates an isolated interpreter and package folder.	Project dependencies should not modify global Python.
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass	Allows scripts only in this PowerShell process.	Activation can be blocked by policy; process scope avoids a permanent machine change.
..\env\Scripts\Activate.ps1	Places .venv Python first for this terminal.	All package installs and scripts must use the project interpreter.
python -m pip install --upgrade pip	Updates pip inside .venv.	Reduces packaging issues before dependency install.
python -m pip install -r requirements-dev.txt	Installs runtime and development dependencies from version constraints.	The generator, tests, notebook export, and validation scripts need them.
psql --version	Prints PostgreSQL command-line client version.	Confirms CLI availability independently of pgAdmin.
python -c "import pyodbc; print(pyodbc.drivers())"	Lists installed ODBC driver display names.	.env must use the exact SQL Server driver name.
.\scripts\setup\verify_local_tools.ps1	Runs grouped non-destructive CLI checks.	Creates one final tool availability snapshot.

Current version guidance (verified 2026-08-16)

Tool	Guidance	Avoid
Git / VS Code	Current stable official Windows release	third-party download sites
Python	Latest patched 3.13.x for project compatibility	beta/preview or unsupported EOL release

Tool	Guidance	Avoid
SQL Server	SQL Server 2025 Developer (non-production) or Express	treating Developer as production license
SSMS	Current SSMS 22	assuming SSMS installs the engine
ODBC	Microsoft ODBC Driver 18 current patch	guessing driver string
PostgreSQL	Current stable PostgreSQL 18.x	PostgreSQL 19 beta for this beginner project
Power BI	Current Desktop from Microsoft Store/official source	multiple unmanaged installs
Databricks	Available workspace; verify Free Edition limits	assuming paid features or unlimited compute

Exact point releases can change. Record what you installed and keep links in the `official source` register near the beginning of this workbook.

MY TASK

Install/verify each required tool, record exact versions and SQL Server/PostgreSQL instance/port choices without secrets, create `.venv`, install dependencies, and select the correct VS Code interpreter.

STARTER SKELETON

```
TODO(USER) Tool inventory
Git: version _____ / verification _____
VS Code: version _____ / interpreter path _____
Python: version _____
SQL Server: edition _____ / instance _____ / auth _____
SSMS: version _____
ODBC driver exact name: _____
PostgreSQL: version _____ / port _____ / service _____
pgAdmin: version _____
Power BI Desktop: version _____
Databricks: workspace type/tier _____ / limitations reviewed _____
Docker (optional): not used / reason _____
```

Acceptance criteria: All required verification commands succeed, GUI tools launch, the selected interpreter is `.venv`, and no credential appears in tracked files or the journal.

HINT 1 — CONCEPT

The database engine is a service; SSMS and pgAdmin are windows used to talk to engines.

HINT 2 — PSEUDOCODE

When import fails, compare `python -m pip --version` and `sys.executable` before reinstalling.

HINT 3 — NEAR-COMPLETE PATTERN

Use the `official source` register near the beginning of this workbook as a checklist and record the installed version beside each source.

13. Expected files or output

- `.venv/` exists locally and is ignored by Git.
- Dependencies installed into `.venv`.
- Tool/version inventory in the journal.
- SQL Server instance name and PostgreSQL port recorded without passwords.
- Databricks tier/limitations recorded.
- No duplicate Python or Power BI install confusion left unresolved.

14. How to verify

- `git --version`, `code --version`, `py -3.13 --version`, and `psql --version` work.
- `Python import` command succeeds.

- SSMS opens and shows Connect dialog.
- pgAdmin opens and prompts for master password/server connection as applicable.
- Power BI Desktop opens to Home.
- Databricks workspace opens and current capability/limit pages have been read.

15. Automated tests

- Run `python -m pytest -q tests/unit/test_generator.py` after sample generation later; dependency import is enough now.
- Run secret scan to ensure setup notes contain no credential: `python -m scripts.validation.scan_secrets`.
- Run `git check-ignore .venv .env`; both should be ignored once `.env` exists.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
Command not recognized	Installer did not add PATH or current terminal predates install.	<code>Get-Command <name> -ErrorAction SilentlyContinue</code> .	Restart terminal; verify official install options; use full path temporarily rather than random PATH edits.	Version command succeeds.
Activation is disabled	PowerShell policy blocks scripts.	<code>Get-ExecutionPolicy -List</code> .	Use process-scope Bypass only, then activate.	Prompt shows <code>.venv</code> .
Wrong Python in VS Code	Global interpreter selected.	VS Code terminal: <code>python -c "import sys; print(sys.executable)"</code> .	Select Interpreter and reopen terminal.	Path ends <code>.venv\Scripts\python.exe</code> .
ODBC driver absent	Driver installer not run or architecture mismatch.	Python pyodbc driver list.	Install current official 64-bit Microsoft ODBC driver and restart apps.	Exact driver appears.
Databricks menu differs	Workspace cloud, tier, rollout, or UI changed.	Check workspace/account type and current official page.	Use the equivalent current object; record variance; do not claim unavailable feature.	Environment check later uses accessible resources.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why is SSMS not the SQL Server database engine?
2. Why use `.venv` and `python -m pip`?
3. What is the difference between an instance name and a database name?
4. Why must Databricks feature/tier assumptions be verified from current documentation?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. Which Python version line should create this project's `.venv`?
2. What command lists exact installed ODBC driver names?
3. Why is process-scope execution policy safer than changing machine scope for this task?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture a four-panel evidence set: PowerShell version checks, VS Code interpreter path, SSMS Connect dialog, and pgAdmin server tree. Never expose passwords.

Save the image under `docs/images/` with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

Official sources used — [STATUS]

☐

Git/GitHub auth verified — [STATUS]

☐

VS Code/extensions/interpreter verified — [STATUS]

☐

Python .venv and packages verified — [STATUS]

☐

SQL Server + SSMS + ODBC verified — [STATUS]

☐

PostgreSQL + pgAdmin/psql verified — [STATUS]

☐

Power BI verified — [STATUS]

☐

Databricks option and limitations verified — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-2. The quiz answer is in the answer-key section immediately after the quiz in this workbook#phase-2.

Next phase: Phase 3 turns the business problem into testable requirements before source schemas are built.

Required action cards

Action 2.2.1 — Open the official source register near the beginning of this workbook. Use only its official domains as the

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open the official source register near the beginning of this workbook. Use only its official domains as the . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.

2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open the official source register near the beginning of this workbook. Use only its official domains as the primary installation authority. The access date is 2026-08-16; verify the current stable release shown when you install.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 1****Action**

Open the official source register near the beginning of this workbook. Use only its official domains as the primary installation authority. The access date is 2026-08-16; verify the current stable release shown when you install.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.2 — Install Git for Windows. Keep the recommended Git Credential Manager option and normal HTTPS behavior. Avoid c

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Install Git for Windows. Keep the recommended Git Credential Manager option and normal HTTPS behavior. Avoid c. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Install Git for Windows. Keep the recommended Git Credential Manager option and normal HTTPS behavior. Avoid changing line-ending or shell options you do not understand; record choices.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 2

Action

Install Git for Windows. Keep the recommended Git Credential Manager option and normal HTTPS behavior. Avoid changing line-ending or shell options you do not understand; record choices.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.3 — Authenticate to GitHub through the browser/device flow prompted by Git Credential Manager on the first protect

Application:

Web browser on GitHub

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Authenticate to GitHub through the browser/device flow prompted by Git Credential Manager on the first protect. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Authenticate to GitHub through the browser/device flow prompted by Git Credential Manager on the first protected operation. GitHub account passwords are not used as raw Git passwords.
4. Pause while the application finishes the operation; do not click the same control twice.

5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Web browser on GitHub. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 3****Action**

Authenticate to GitHub through the browser/device flow prompted by Git Credential Manager on the first protected operation. GitHub account passwords are not used as raw Git passwords.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.4 — Install VS Code using the Windows user installer when appropriate. Select Explorer context-menu and PATH optio

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Install VS Code using the Windows user installer when appropriate. Select Explorer context-menu and PATH option. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Install VS Code using the Windows user installer when appropriate. Select Explorer context-menu and PATH options when offered. Reopen PowerShell after install.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 4

Action

Install VS Code using the Windows user installer when appropriate. Select Explorer context-menu and PATH options when offered. Reopen PowerShell after install.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.5 — In VS Code Extensions (Ctrl+Shift+X), install Microsoft's , , , , and a PostgreSQL client extension only if

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

In VS Code Extensions (Ctrl+Shift+X), install Microsoft's , , , , and a PostgreSQL client extension only if . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: In VS Code Extensions (Ctrl+Shift+X), install Microsoft's **Python**, **Pylance**, **Jupyter**, **SQL Server (mssql)**, and a PostgreSQL client extension only if you choose one. Avoid installing many overlapping SQL formatters initially.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 5

Action

In VS Code Extensions (Ctrl+Shift+X), install Microsoft's **Python, Pylance, Jupyter, SQL Server (mssql)**, and a PostgreSQL client extension only if you choose one. Avoid installing many overlapping SQL formatters initially.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.6 — Install 64-bit Python 3.13.x, the latest patched 3.13 release available. Include the Python launcher. Do not s

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Install 64-bit Python 3.13.x, the latest patched 3.13 release available. Include the Python launcher. Do not s. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Install 64-bit Python 3.13.x, the latest patched 3.13 release available. Include the Python launcher. Do not select a Python 19/preview-style beta; the current major release on the website may be newer, but this project pins compatibility to 3.13.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 6****Action**

Install 64-bit Python 3.13.x, the latest patched 3.13 release available. Include the Python launcher. Do not select a Python 19/preview-style beta; the current major release on the website may be newer, but this project pins compatibility to 3.13.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.7 — Install SQL Server 2025 Developer for full non-production learning features, or Express if your computer/resou

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Install SQL Server 2025 Developer for full non-production learning features, or Express if your computer/resou. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Install SQL Server 2025 Developer for full non-production learning features, or Express if your computer/resources require it. Record the exact instance name and authentication choice. Developer is not licensed for production use.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 7****Action**

Install SQL Server 2025 Developer for full non-production learning features, or Express if your computer/resources require it. Record the exact instance name and authentication choice. Developer is not licensed for production use.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.8 — Install current SSMS 22 from Microsoft. SSMS is the management client; it does not replace the SQL Server engi

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Install current SSMS 22 from Microsoft. SSMS is the management client; it does not replace the SQL Server engine. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Install current SSMS 22 from Microsoft. SSMS is the management client; it does not replace the SQL Server engine.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 8****Action**

Install current SSMS 22 from Microsoft. SSMS is the management client; it does not replace the SQL Server engine.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.9 — Install the current Microsoft ODBC Driver 18 for SQL Server. Record the exact driver name Python reports.

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Install the current Microsoft ODBC Driver 18 for SQL Server. Record the exact driver name Python reports. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Install the current Microsoft ODBC Driver 18 for SQL Server. Record the exact driver name Python reports.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 9

Action

Install the current Microsoft ODBC Driver 18 for SQL Server. Record the exact driver name Python reports.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.10 — Install the current stable PostgreSQL 18.x Windows package. Record the port (commonly 5432), superuser name, p

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Install the current stable PostgreSQL 18.x Windows package. Record the port (commonly 5432), superuser name, p. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Install the current stable PostgreSQL 18.x Windows package. Record the port (commonly 5432), superuser name, password location (not in Git), and service name. Include pgAdmin and command-line tools.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL `psql`, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 10****Action**

Install the current stable PostgreSQL 18.x Windows package. Record the port (commonly 5432), superuser name, password location (not in Git), and service name. Include pgAdmin and command-line tools.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.11 — Install Power BI Desktop from the Microsoft Store where allowed for update management; otherwise use Microsoft

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Install Power BI Desktop from the Microsoft Store where allowed for update management; otherwise use Microsoft. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Install Power BI Desktop from the Microsoft Store where allowed for update management; otherwise use Microsoft's official download center. Do not install both paths without a reason.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 11

Action

Install Power BI Desktop from the Microsoft Store where allowed for update management; otherwise use Microsoft's official download center. Do not install both paths without a reason.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.12 — Create or sign in to an available Databricks workspace. As of the access date, Free Edition supports beginner

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Create or sign in to an available Databricks workspace. As of the access date, Free Edition supports beginner . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Create or sign in to an available Databricks workspace. As of the access date, Free Edition supports beginner notebook/SQL/Jobs work but has quota and feature limits. Read the current Free Edition and limitations pages before selecting it.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 12****Action**

Create or sign in to an available Databricks workspace. As of the access date, Free Edition supports beginner notebook/SQL/ Jobs work but has quota and feature limits. Read the current Free Edition and limitations pages before selecting it.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.13 — From the repository root, create .venv, activate it, upgrade pip, and install requirements-dev.txt.

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

From the repository root, create .venv, activate it, upgrade pip, and install requirements-dev.txt. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: From the repository root, create .venv, activate it, upgrade pip, and install requirements-dev.txt.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 13

Action

From the repository root, create .venv, activate it, upgrade pip, and install requirements-dev.txt.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.14 — Select .venv in VS Code: Ctrl+Shift+P → → choose the path ending .venv\Scripts\python.exe.

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Select .venv in VS Code: Ctrl+Shift+P → → choose the path ending .venv\Scripts\python.exe. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Select `.venv` in VS Code: `Ctrl+Shift+P` → **Python: Select Interpreter** → choose the path ending `.venv\Scripts\python.exe`.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 14****Action**

Select `.venv` in VS Code: `Ctrl+Shift+P` → **Python: Select Interpreter** → choose the path ending `.venv\Scripts\python.exe`.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.15 — Run every verification command and paste the version outputs into your learning journal. Never paste passwords

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run every verification command and paste the version outputs into your learning journal. Never paste passwords. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run every verification command and paste the version outputs into your learning journal. Never paste passwords or tokens.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 15

Action

Run every verification command and paste the version outputs into your learning journal. Never paste passwords or tokens.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.16 — Optional Docker path: use only after the native path works or when you can explain ports, volumes, container p

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Optional Docker path: use only after the native path works or when you can explain ports, volumes, container p. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Optional Docker path: use only after the native path works or when you can explain ports, volumes, container persistence, licensing, and Windows virtualization. Docker is not required for Track A.
4. Pause while the application finishes the operation; do not click the same control twice.

5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 16****Action**

Optional Docker path: use only after the native path works or when you can explain ports, volumes, container persistence, licensing, and Windows virtualization. Docker is not required for Track A.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.2.C1 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
git --version
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git --version` — `git` calls the Git version-control program; `--version` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.2.C2 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
code --version
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `code --version` — `code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--version` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.2.C3 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
py -3.13 --version
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `py -3.13 --version` — `py` invokes the Windows Python launcher or Python Install Manager command shim; `-3` is a command option or flag; its exact behavior is defined beside the command; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `13` is a numeric literal used by the command or expression; `--version` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.2.C4 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
py -3.13 -m venv .venv
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `py -3.13 -m venv .venv` — `py` invokes the Windows Python launcher or Python Install Manager command shim; `-3` is a command option or flag; its exact behavior is defined beside the command; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `13` is a numeric literal used by the command or expression; `-m` asks Python to run an installed module; `venv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `venv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.2.C5 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass — Set-ExecutionPolicy is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Scope is a command option or flag; its exact behavior is defined beside the command; Process is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -ExecutionPolicy is a command option or flag; its exact behavior is defined beside the command; Bypass is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.2.C6 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.

4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
.\.env\Scripts\Activate.ps1
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `.\.env\Scripts\Activate.ps1` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Scripts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Activate.ps1` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.2.C7 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m pip install --upgrade pip
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m pip install --upgrade pip` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `pip` the Python package installer module; `install` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--upgrade` is a command option or flag; its exact behavior is defined beside the command; `pip` the Python package installer module

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.2.C8 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m pip install -r requirements-dev.txt
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m pip install -r requirements-dev.txt` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `pip` the Python package installer module; `install` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-r` is a command option or flag; its exact behavior is defined beside the command; `requirements-dev.txt` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.2.C9 — Windows PowerShell

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
psql --version
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** psql --version — psql is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; --version is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.2.C10 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -c "import pyodbc; print(pyodbc.drivers())"
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -c "import pyodbc; print(pyodbc.drivers())"` — python invokes Python when that command name is registered; `-c` creates the named branch before switching; `"import pyodbc; print(pyodbc.drivers())"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.2.C11 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
.\scripts\setup\verify_local_tools.ps1
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `.\scripts\setup\verify_local_tools.ps1` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `scripts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `setup` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `verify_local_tools.ps1` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 2.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 2 starting-screen checklist and re-verify the active context.

Lesson 2.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 2 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — config/

Exact folder name: config

Exact parent: repository root

Why it is needed now: Phase 2 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: config.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 2.D1.1 — Create the folder config

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\config" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\config" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\config"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\config" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\config" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\config" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — scripts/

Exact folder name: `scripts`

Exact parent: `repository root`

Why it is needed now: Phase 2 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `repository root` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `scripts`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 2.D2.1 — Create the folder scripts

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\scripts" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\scripts" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\scripts"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use

`git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\scripts" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\scripts" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — scripts/setup/

Exact folder name: `setup`

Exact parent: `scripts`

Why it is needed now: Phase 2 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `scripts` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `setup`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

Action 2.D3.1 — Create the folder scripts/setup

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\scripts\setup" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\scripts\setup" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\scripts\setup" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\scripts\setup" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts\setup" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\scripts\setup" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — requirements.txt

Exact filename: `requirements.txt`

Filename extension: `.txt`. In this project it identifies project text or configuration and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `.` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source

SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project text or configuration for Phase 2.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in . one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: requirements.txt.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled requirements.txt and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 2.F1.1 — Create the file requirements.txt from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\requirements.txt" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\requirements.txt" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\requirements.txt"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\requirements.txt" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "requirements.txt"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `..`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `requirements.txt`.
3. In PowerShell at the repository root, run `Test-Path ".\requirements.txt"` and require `True`.
4. Run `git status --short -- "requirements.txt"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for requirements.txt

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: text

Exact file: requirements.txt

Purpose: project text or configuration.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 2.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Runtime dependencies. Use Python 3.13.x for the local learning environment.
pandas>=2.2,<3.0
Faker>=30,<40
PyYAML>=6,<7
python-dotenv>=1,<2
pyarrow>=18,<23
SQLAlchemy>=2,<3
pyodbc>=5.2,<6
psycpg[binary]>=3.2,<4
tenacity>=9,<10
rich>=13,<15
```

Line-by-line explanation

- **Line 1:** `# Runtime dependencies. Use Python 3.13.x for the local learning environment.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `pandas>=2.2,<3.0` — `pandas` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2.2` is a numeric literal used by the command or expression; `,` `<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3.0` is a numeric literal used by the command or expression
- **Line 3:** `Faker>=30,<40` — `Faker` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30` is a numeric literal used by the command or expression; `,` `<` is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 40 is a numeric literal used by the command or expression

- **Line 4:** `PyYAML>=6,<7` — `PyYAML` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `6` is a numeric literal used by the command or expression; `,<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `7` is a numeric literal used by the command or expression
- **Line 5:** `python-dotenv>=1,<2` — `python-dotenv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `,<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression
- **Line 6:** `pyarrow>=18,<23` — `pyarrow` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `18` is a numeric literal used by the command or expression; `,<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `23` is a numeric literal used by the command or expression
- **Line 7:** `SQLAlchemy>=2,<3` — `SQLAlchemy` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression; `,<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression
- **Line 8:** `pyodbc>=5.2,<6` — `pyodbc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `5.2` is a numeric literal used by the command or expression; `,<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `6` is a numeric literal used by the command or expression
- **Line 9:** `psycopg[binary]>=3.2,<4` — `psycopg` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `binary` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3.2` is a numeric literal used by the command or expression; `,<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `4` is a numeric literal used by the command or expression
- **Line 10:** `tenacity>=9,<10` — `tenacity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `9` is a numeric literal used by the command or expression; `,<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `10` is a numeric literal used by the command or expression
- **Line 11:** `rich>=13,<15` — `rich` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `13` is a numeric literal used by the command or expression; `,<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `15` is a numeric literal used by the command or expression

[Back to the Phase 2 lesson index](#)

File introduction — requirements-dev.txt

Exact filename: requirements-dev.txt

Filename extension: .txt. In this project it identifies project text or configuration and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: . relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project text or configuration for Phase 2.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in . one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: requirements-dev.txt.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled requirements-dev.txt and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 2.F2.1 — Create the file requirements-dev.txt from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\requirements-dev.txt" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\requirements-dev.txt" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\requirements-dev.txt"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\requirements-dev.txt" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add -Force, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "requirements-dev.txt"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under ..
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `requirements-dev.txt`.
3. In PowerShell at the repository root, run `Test-Path ".\requirements-dev.txt"` and require True.
4. Run `git status --short -- "requirements-dev.txt"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for requirements-dev.txt

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: text

Exact file: `requirements-dev.txt`

Purpose: project text or configuration.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 2.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-r requirements.txt
pytest>=8,<10
pytest-cov>=5,<7
ruff>=0.9,<1.0
nbformat>=5,<6
nbconvert>=7,<8
jsonschema>=4,<5
```

Line-by-line explanation

- **Line 1:** `-r requirements.txt` — `-r` is a command option or flag; its exact behavior is defined beside the command; `requirements.txt` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `pytest>=8,<10` — `pytest` the Python test runner; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `8` is a numeric literal used by the command or expression; `<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `10` is a numeric literal used by the command or expression
- **Line 3:** `pytest-cov>=5,<7` — `pytest-cov` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `5` is a numeric literal used by the command or expression; `<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `7` is a numeric literal used by the command or expression
- **Line 4:** `ruff>=0.9,<1.0` — `ruff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0.9` is a numeric literal used by the command or expression; `<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1.0` is a numeric literal used by the command or expression
- **Line 5:** `nbformat>=5,<6` — `nbformat` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific

role is described in the surrounding explanation; 5 is a numeric literal used by the command or expression; , < is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 6 is a numeric literal used by the command or expression

- **Line 6:** nbconvert>=7, <8 — nbconvert is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; >= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 7 is a numeric literal used by the command or expression; , < is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 8 is a numeric literal used by the command or expression
- **Line 7:** jsonschema>=4, <5 — jsonschema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; >= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 4 is a numeric literal used by the command or expression; , < is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 5 is a numeric literal used by the command or expression

[Back to the Phase 2 lesson index](#)

File introduction — config/dev.example.yml

Exact filename: dev.example.yml

Filename extension: .yml. In this project it identifies human-readable configuration and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: config relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: human-readable configuration for Phase 2.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? Yes—review carefully before Git staging.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in config one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: dev.example.yml.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled dev.example.yml and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 2.F3.1 — Create the file config/dev.example.yml from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\config\dev.example.yml" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\config\dev.example.yml" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\config\dev.example.yml` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\config\dev.example.yml" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "config/dev.example.yml"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `config`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `config/dev.example.yml`.
3. In PowerShell at the repository root, run `Test-Path ".\config\dev.example.yml"` and require `True`.
4. Run `git status --short -- "config/dev.example.yml"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for config/dev.example.yml

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: YAML

Exact file: `config/dev.example.yml`

Purpose: human-readable configuration.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 2.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```

environment: dev
paths:
  generated_data: datasets/generated/quick
  extracts: datasets/raw
  logs: logs
sqlserver:
  server_env: SQLSERVER_SERVER
  source_database_env: SQLSERVER_SOURCE_DATABASE
  warehouse_database_env: SQLSERVER_WAREHOUSE_DATABASE
  driver_env: SQLSERVER_DRIVER
  trusted_connection_env: SQLSERVER_TRUSTED_CONNECTION
postgres:
  host_env: POSTGRES_HOST
  port_env: POSTGRES_PORT
  database_env: POSTGRES_DATABASE
  username_env: POSTGRES_USERNAME
  password_env: POSTGRES_PASSWORD
databricks:
  catalog: northstar
  bronze_schema: bronze
  silver_schema: silver
  gold_schema: gold
  landing_volume: /Volumes/northstar/landing/raw

```

Line-by-line explanation

- **Line 1:** `environment: dev` — `environment` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dev` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `paths:` — `paths` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `generated_data: datasets/generated/quick` — `generated_data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `generated` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `quick` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `extracts: datasets/raw` — `extracts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `logs: logs` — `logs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `logs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `sqlserver:` — `sqlserver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `server_env: SQLSERVER_SERVER` — `server_env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQLSERVER_SERVER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `source_database_env: SQLSERVER_SOURCE_DATABASE` — `source_database_env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQLSERVER_SOURCE_DATABASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `warehouse_database_env: SQLSERVER_WAREHOUSE_DATABASE` — `warehouse_database_env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

SQLSERVER_WAREHOUSE_DATABASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 10:** driver_env: SQLSERVER_DRIVER — driver_env is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQLSERVER_DRIVER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** trusted_connection_env: SQLSERVER_TRUSTED_CONNECTION — trusted_connection_env is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQLSERVER_TRUSTED_CONNECTION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** postgres: — postgres is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** host_env: POSTGRES_HOST — host_env is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; POSTGRES_HOST is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** port_env: POSTGRES_PORT — port_env is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; POSTGRES_PORT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** database_env: POSTGRES_DATABASE — database_env is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; POSTGRES_DATABASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** username_env: POSTGRES_USERNAME — username_env is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; POSTGRES_USERNAME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** password_env: POSTGRES_PASSWORD — password_env is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; POSTGRES_PASSWORD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** databricks: — databricks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** catalog: northstar — catalog is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; northstar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** bronze_schema: bronze — bronze_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** silver_schema: silver — silver_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; silver is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** gold_schema: gold — gold_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** landing_volume: /Volumes/northstar/landing/raw — landing_volume is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; Volumes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; northstar is a name, keyword,

argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; landing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; raw is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 2 lesson index](#)

File introduction — scripts/setup/verify_local_tools.ps1

Exact filename: verify_local_tools.ps1

Filename extension: .ps1. In this project it identifies Windows automation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: scripts/setup relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: Windows automation for Phase 2.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in scripts/setup one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: verify_local_tools.ps1.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled verify_local_tools.ps1 and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 2.F4.1 — Create the file scripts/setup/verify_local_tools.ps1 from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\scripts\setup\verify_local_tools.ps1" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType File -Path ".\scripts\setup\verify_local_tools.ps1" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; File is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\scripts\setup\verify_local_tools.ps1" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\scripts\setup\verify_local_tools.ps1" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "scripts/setup/verify_local_tools.ps1"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use TODO(USER) where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `scripts/setup`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `scripts/setup/verify_local_tools.ps1`.
3. In PowerShell at the repository root, run `Test-Path ".\scripts\setup\verify_local_tools.ps1"` and require True.
4. Run `git status --short -- "scripts/setup/verify_local_tools.ps1"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for scripts/setup/verify_local_tools.ps1

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: PowerShell

Exact file: `scripts/setup/verify_local_tools.ps1`

Purpose: Windows automation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 2.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Runs in: Windows PowerShell
# Purpose: show which required command-line tools Windows can find.
$ErrorActionPreference = "Continue"

$checks = @(
    @{ Name = "Git"; Command = "git --version" },
    @{ Name = "Python launcher"; Command = "py --version" },
    @{ Name = "Python"; Command = "python --version" },
    @{ Name = "VS Code"; Command = "code --version" },
    @{ Name = "PostgreSQL psql"; Command = "psql --version" }
```

```

)

foreach ($check in $checks) {
    Write-Host "`n=== $($check.Name) ==="
    try {
        Invoke-Expression $check.Command
    }
    catch {
        Write-Warning "$($check.Name) was not found on PATH. Read Phase 2 before changing PATH manually."
    }
}

Write-Host "`nSQL Server, SSMS, pgAdmin, Power BI Desktop, and Databricks are verified through their user interfaces."

```

Line-by-line explanation

- **Line 1:** # Runs in: Windows PowerShell is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** # Purpose: show which required command-line tools Windows can find. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** \$ErrorActionPreference = "Continue" — \$ErrorActionPreference is a variable or environment-value reference rather than literal output; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Continue" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** \$checks = @(— \$checks is a variable or environment-value reference rather than literal output; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** @{ Name = "Git"; Command = "git --version" }, — { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Git" is a quoted literal; the quotation marks preserve spaces or mark text; Command is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "git --version" is a quoted literal; the quotation marks preserve spaces or mark text; }, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** @{ Name = "Python launcher"; Command = "py --version" }, — { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Python launcher" is a quoted literal; the quotation marks preserve spaces or mark text; Command is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "py --version" is a quoted literal; the quotation marks preserve spaces or mark text; }, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** @{ Name = "Python"; Command = "python --version" }, — { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Python" is a quoted literal; the quotation marks preserve spaces or mark text; Command is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "python --version" is a quoted literal; the quotation marks preserve spaces or mark text; }, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** @{ Name = "VS Code"; Command = "code --version" }, — { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "VS Code" is a quoted literal; the quotation marks preserve spaces or mark text; Command is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "code --version" is a quoted literal; the quotation marks preserve spaces or mark text; }, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 10:** `@{ Name = "PostgreSQL psql"; Command = "psql --version" }` — `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PostgreSQL psql"` is a quoted literal; the quotation marks preserve spaces or mark text; `Command` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"psql --version"` is a quoted literal; the quotation marks preserve spaces or mark text; `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** `foreach ($check in $checks) {` — `foreach` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `$check` is a variable or environment-value reference rather than literal output; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `$checks` is a variable or environment-value reference rather than literal output; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** `Write-Host "`n=== $($check.Name) ==="` — `Write-Host` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"`n=== $($check.Name) ==="` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 15:** `try {` — `try` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `Invoke-Expression $check.Command` — `Invoke-Expression` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `$check.Command` is a variable or environment-value reference rather than literal output
- **Line 17:** `}` — `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** `catch {` — `catch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** `Write-Warning "$($check.Name) was not found on PATH. Read Phase 2 before changing PATH manually."` — `Write-Warning` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"$($check.Name) was not found on PATH. Read Phase 2 before changing PATH manually."` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 20:** `}` — `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** `}` — `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** blank line used only to separate logical sections for readability.
- **Line 23:** `Write-Host "`nSQL Server, SSMS, pgAdmin, Power BI Desktop, and Databricks are verified through their user interfaces."` — `Write-Host` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"`nSQL Server, SSMS, pgAdmin, Power BI Desktop, and Databricks are verified through their user interfaces."` is a quoted literal; the quotation marks preserve spaces or mark text

[Back to the Phase 2 lesson index](#)

What your screen should look like now

- Every Phase 2 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 2.3 completion checklist

☐

requirements.txt was created by me, saved, located correctly, and inspected — [STATUS]

☐

requirements-dev.txt was created by me, saved, located correctly, and inspected — [STATUS]

☐

config/dev.example.yml was created by me, saved, located correctly, and inspected — [STATUS]

☐

scripts/setup/verify_local_tools.ps1 was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 2.4](#) and recheck the root and branch.

Lesson 2.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Action 2.4.G1 — Inspect the current state without changing it

Application:

Web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Inspect the current state without changing it. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.

2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.4.G2 — Create the smallest required object or decision

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Create the smallest required object or decision. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.

4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.4.G3 — Run the phase-specific verification

Application:

Web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run the phase-specific verification. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.4.G4 — Demonstrate one safe failure or alternate-result branch

Application:

Web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Demonstrate one safe failure or alternate-result branch. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

Action 2.4.G5 — Record evidence and prepare the downstream handoff

Application:

Web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Record evidence and prepare the downstream handoff. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 2 lesson index](#)

What your screen should look like now

- The new Phase 2 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 2.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 2.5](#).

Lesson 2.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 2 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Tool editions and stable versions

Question:

Which product editions should support a no-production-use learning project?

Options considered:

- Latest stable free/developer editions.
- Preview/beta versions.
- Paid enterprise editions.

Beginner path selected:

Use current stable free or Developer editions and record exact versions.

Why this path was selected:

This maximizes learning value while avoiding production licensing claims and beta instability.

Tradeoff:

The interface can change monthly and free tiers have limits.

What a company might do instead:

A company standardizes supported versions, patch windows, licenses, and managed deployment.

How to explain this in an interview:

I verified the exact tool versions and chose stable, supported editions appropriate for non-production development.

Design decision — SQL Server edition

Question:

Which SQL Server edition is appropriate for this non-production portfolio project?

Options considered:

- Developer edition for full non-production features.
- Express edition for a smaller free footprint.
- A paid production edition.

Beginner path selected:

Prefer current SQL Server Developer for non-production learning; use Express only when machine constraints require its limits.

Why this path was selected:

Developer provides the broadest learning surface without a production-use claim, while Express is a practical fallback on constrained hardware.

Tradeoff:

Developer may use more resources, and neither choice authorizes production deployment.

What a company might do instead:

A company selects and licenses an edition from workload, availability, security, support, and cost requirements, often using managed Azure SQL services.

How to explain this in an interview:

I selected a current stable non-production edition, recorded the exact version, and documented licensing and resource limits rather than treating all SQL Server editions as interchangeable.

Design decision — PostgreSQL installation method

Question:

How should PostgreSQL be installed on a beginner Windows computer?

Options considered:

- Official Windows installer with the database service and pgAdmin.
- Docker container.
- Windows Subsystem for Linux package installation.

Beginner path selected:

Use the official Windows installer and verify the Windows service, port, superuser choice, and pgAdmin connection.

Why this path was selected:

It gives the most visible Windows-native service and GUI workflow and avoids requiring Docker or Linux knowledge before database fundamentals.

Tradeoff:

The installer adds a local service and requires careful password, port, PATH, and upgrade management.

What a company might do instead:

Teams often standardize containers, managed PostgreSQL, automated images, secrets, backups, and infrastructure as code.

How to explain this in an interview:

I used the official Windows installer for transparent beginner setup, while documenting containers and managed services as reproducibility and operations alternatives.

If you see Result A

The version command opens the expected stable tool and prints a version. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

The command is not recognized, resolves to a different executable, or the UI version differs. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: Web browser, Windows Settings, installers, PowerShell, VS Code, SSMS, pgAdmin, Databricks, and Power BI Desktop.

Window/panel: Show version commands, VS Code interpreter, SSMS connection, pgAdmin tree, Databricks workspace, and Power BI About dialog.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase02_install-and-verify-the-windows-toolchain_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 2's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 2 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 2.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Interpret git status --short

- ?? means untracked: Git sees a new path but it is not staged.
- M means modified in the working tree but not staged.
- M means the staged version differs from the last commit.
- MM means a staged version exists and the file has additional unstaged edits.
- A means staged as a new file.
- D means deletion; stop and investigate unless deletion is deliberate.
- R means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 2.5.G4 — Inspect the unstaged diff for requirements.txt

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "requirements.txt"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "requirements.txt"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"requirements.txt"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.5.G5 — Stage only the reviewed file `requirements.txt`

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "requirements.txt"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "requirements.txt"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"requirements.txt"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 2.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.5.G7 — Create the Phase 2 commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "docs: record verified local toolchain versions"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "docs: record verified local toolchain versions"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"docs: record verified local toolchain versions"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Action 2.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 2 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message docs: record verified local toolchain versions and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 2.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 2 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

"Phase 2 created the verified boundary described by **Install and verify the Windows toolchain**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed."

Mini quiz

1. Multiple choice: Why does Phase 2 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

My Phase 2 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 2.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 3 uses the verified editor and Git workflow to create requirements safely.

Phase 2 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 3 — Write business requirements and acceptance criteria

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: A working repository and toolchain.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 4 turns approved requirements, grains, KPIs, and quality expectations into source schemas.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 3 lesson index

1. [Lesson 3.1 — Understand the phase and starting screen](#)
2. [Lesson 3.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 3.3 — Create and understand the phase files](#)
4. [Lesson 3.4 — Build and validate required objects](#)
5. [Lesson 3.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 3.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** VS Code, Markdown Preview, Windows PowerShell, and Git.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 3 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 2 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 3 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

VS Code, Markdown Preview, Windows PowerShell, and Git. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 3 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.

- No new Phase 3 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 3 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 3.1 completion checklist

☐

I read why Phase 3 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 3.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 3 lesson index](#)

Lesson 3.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 3 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

A reviewed business requirements package containing company context, stakeholders, pain points, user stories, KPI contracts, freshness and quality targets, security assumptions, success criteria, out-of-scope items, risks, and assumptions.

2. Why it matters

Code can be technically correct and still answer the wrong business question. Requirements determine grain, date role, filters, currency, attribution, and quality thresholds. They also prevent scope creep by stating what the first project will not solve.

3. Words to know

Term	Plain-language definition
Stakeholder	Person or team making a decision with the data.
User story	Short statement of who needs what and why.
KPI	Governed measure tied to a decision.
Numerator / denominator	Top and bottom parts of a rate.

Term	Plain-language definition
Filter	Condition limiting which records count.
Date role	Which business date controls a metric, such as order or return date.
Freshness target	How recent data should be.
Success criterion	Observable condition proving the requirement is met.
Out of scope	Deliberate boundary for this version.
Assumption	Decision accepted without full evidence, recorded for review.
Risk	Uncertain event that could damage trust, schedule, cost, or security.
Acceptance criteria	Specific checks defining task completion.

4. What must already be completed

Phase 0 mental model and Phase 1 repository setup are complete. Tool installation is helpful but not required to edit Markdown.

5. Applications to open

Application	How to open it	Why it is open
VS Code	Open the verified repository root with code ..	Edit requirements and logs.
Web browser	Open only official references when a current platform constraint affects a requirement.	Validate tier/capability assumptions.
PowerShell	Open repository root.	Check files and Git diff.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run Get-Location; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
Get-Content .\docs\business_requirements.md -TotalCount 40	Prints the beginning of the requirement document.	Confirms you are editing the intended file and can inspect it outside VS Code.
Select-String -Path .\docs\kpi_definitions.md -Pattern '^### [0-9]+.'	Lists KPI section headings.	Confirms every required KPI definition exists.
git diff -- docs\business_requirements.md docs\assumption_log.md docs\risk_register.md	Shows only your requirement edits.	Requirements changes should be reviewed as carefully as code.

Why requirements come before code

Consider “return rate.” It could mean returned orders / orders, returned units / sold units, refund amount / revenue, purchase-date cohorts, or return-date activity. Those choices require different grains and date relationships. Writing SQL before choosing makes disagreement inevitable.

Vertical-slice decision summary

The first slice should prove one customer, product, order, order item, clean Silver rule, dimension lookup, sales fact, revenue calculation, and validation. Breadth comes after the smallest trustworthy path works.

MY TASK

Rewrite one KPI definition in your own words, document one assumption and one risk, and define the first vertical slice acceptance criteria without changing the required overall scope.

STARTER SKELETON

```
## TODO(USER) KPI decision
- Stakeholder and decision:
- Business meaning:
- Numerator:
- Denominator:
- Grain:
- Date role:
- Filters/exclusions:
- Currency:
- Source columns:
- SQL validation:
- Common misuse:

## TODO(USER) Vertical slice acceptance criteria
- Given _____
- When _____
- Then source count _____
- And Silver rejects _____
- And Gold grain _____
- And revenue equals _____
```

Acceptance criteria: One KPI contract is complete enough that another person can independently reproduce it; one assumption/risk is owned; first-slice criteria are measurable and do not claim execution.

HINT 1 — CONCEPT

A stakeholder needs a decision, not 'a dashboard'.

HINT 2 — PSEUDOCODE

Write the denominator first for rates; most interpretation errors hide there.

HINT 3 — NEAR-COMPLETE PATTERN

Use the KPI catalog fields as a near-complete checklist, then rewrite the wording yourself.

13. Expected files or output

- Edited assumption/risk logs.
- One personally rewritten KPI contract.
- A vertical-slice acceptance list.
- No SQL code added solely to avoid deciding definitions.

14. How to verify

- A reader can state one row's grain and the KPI's date role.
- Every required KPI maps to existing or planned columns.
- Freshness and success statements are targets until measured.
- Out-of-scope items include streaming/CDC/cloud production.

15. Automated tests

- Manual requirement trace: KPI → columns → source table → Silver rule → Gold fact → DAX/SQL.
- Link check later ensures requirement references resolve.

- No platform automation test is a substitute for stakeholder definition review.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
KPI has two possible denominators	Business definition is unresolved.	Write sample numbers and calculate each interpretation.	Choose one for Track A, name the other separately, record assumption.	A new reader gets one answer from the contract.
Freshness target is written as achieved	Target and evidence are being confused.	Search for words 'achieved', 'runs daily', or performance claims.	Change to target/desired until run history proves it.	README status remains honest.
Every idea became in scope	No prioritization boundary.	Compare against Phase 22 extensions.	Move non-required ideas to optional extensions with prerequisites/evidence.	Required checklist stays feasible.
KPI cannot map to columns	Source model lacks needed data or definition is too vague.	Use data dictionary filter by table/column.	Add a justified source column before DDL, or change/out-scope KPI.	Mapping is explicit.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why can return rate be correct mathematically but wrong for the business?
2. What is the difference between a freshness target and a verified service level?
3. How does declared grain constrain a KPI?
4. Why record an assumption instead of quietly treating it as fact?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. What four details must be aligned for a ratio across two facts?
2. Why is ROAS attribution an assumption rather than a simple join?
3. What is one success criterion that proves trust rather than file existence?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture the edited KPI contract and assumption log side by side. The image should show your own wording and no fabricated values.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
docs: define northstar business requirements and KPI contracts
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

Stakeholders/decisions reviewed — [STATUS]

☐

All required KPIs mapped — [STATUS]

☐

Freshness targets labeled honestly — [STATUS]

☐

Quality expectations mapped to tests — [STATUS]

☐

Security assumptions reviewed — [STATUS]

☐

Risk and assumption added — [STATUS]

☐

Vertical slice accepted — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-3. The quiz answer is in [the answer-key section immediately after the quiz in this workbook#phase-3] (the answer-key section immediately after the quiz in this workbook#phase-3).

Next phase: Phase 4 turns those requirements into normalized SQL Server and PostgreSQL source schemas.

Required action cards

Action 3.2.1 — Open docs/business_requirements.md and read the business problem aloud.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open docs/business_requirements.md and read the business problem aloud. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open docs/business_requirements.md and read the business problem aloud.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 1

Action

Open docs/business_requirements.md and read the business problem aloud.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 3 lesson index](#)

Action 3.2.2 — For each stakeholder, identify one decision—not merely one chart—they need to make.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

For each stakeholder, identify one decision—not merely one chart—they need to make. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: For each stakeholder, identify one decision—not merely one chart—they need to make.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 2****Action**

For each stakeholder, identify one decision—not merely one chart—they need to make.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location

with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 3 lesson index](#)

Action 3.2.3 — Open docs/kpi_definitions.md. Review meaning, formula, grain, tables, filters, validation, and common mistakes

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open docs/kpi_definitions.md. Review meaning, formula, grain, tables, filters, validation, and common mistakes. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open docs/kpi_definitions.md. Review meaning, formula, grain, tables, filters, validation, and common mistakes for every KPI.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 3

Action

Open docs/kpi_definitions.md. Review meaning, formula, grain, tables, filters, validation, and common mistakes for every KPI.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.



I can describe what changed on the screen or filesystem.



I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 3 lesson index](#)

Action 3.2.4 — Choose one ambiguous KPI such as return rate or ROAS. Write the alternative interpretations and record the sel

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Choose one ambiguous KPI such as return rate or ROAS. Write the alternative interpretations and record the sel. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Choose one ambiguous KPI such as return rate or ROAS. Write the alternative interpretations and record the selected beginner definition in docs/assumption_log.md.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 4****Action**

Choose one ambiguous KPI such as return rate or ROAS. Write the alternative interpretations and record the selected beginner definition in docs/assumption_log.md.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 3 lesson index](#)

Action 3.2.5 — Review freshness targets. Label them target, not achieved, until scheduled runs prove them.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Review freshness targets. Label them target, not achieved, until scheduled runs prove them. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Review freshness targets. Label them target, not achieved, until scheduled runs prove them.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 5

Action

Review freshness targets. Label them target, not achieved, until scheduled runs prove them.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 3 lesson index](#)

Action 3.2.6 — Review data-quality expectations and map each to a future automated test.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Review data-quality expectations and map each to a future automated test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Review data-quality expectations and map each to a future automated test.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.

6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 6

Action

Review data-quality expectations and map each to a future automated test.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 3 lesson index](#)

Action 3.2.7 — Review security assumptions even though data is synthetic; classify customer-like fields as synthetic PII.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Review security assumptions even though data is synthetic; classify customer-like fields as synthetic PII. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Review security assumptions even though data is synthetic; classify customer-like fields as synthetic PII.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 7

Action

Review security assumptions even though data is synthetic; classify customer-like fields as synthetic PII.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 3 lesson index](#)

Action 3.2.8 — Read the out-of-scope list and add one tempting extension you will postpone.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Read the out-of-scope list and add one tempting extension you will postpone. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Read the out-of-scope list and add one tempting extension you will postpone.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 8****Action**

Read the out-of-scope list and add one tempting extension you will postpone.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 3 lesson index](#)

Action 3.2.9 — Open docs/risk_register.md; assign yourself as owner where appropriate and add an early warning for your big

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open docs/risk_register.md; assign yourself as owner where appropriate and add an early warning for your big. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open docs/risk_register.md; assign yourself as owner where appropriate and add an early warning for your biggest local risk.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 9****Action**

Open docs/risk_register.md; assign yourself as owner where appropriate and add an early warning for your biggest local risk.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 3 lesson index](#)

Action 3.2.10 — Write acceptance criteria for the first vertical slice: customers + products + one valid sales line from source

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Write acceptance criteria for the first vertical slice: customers + products + one valid sales line from source. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Write acceptance criteria for the first vertical slice: customers + products + one valid sales line from source through a SQL validation query.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 10

Action

Write acceptance criteria for the first vertical slice: customers + products + one valid sales line from source through a SQL validation query.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 3 lesson index](#)

Action 3.2.C1 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Get-Content .\docs\business_requirements.md -TotalCount 40
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** Get-Content .\docs\business_requirements.md -TotalCount 40 — Get-Content is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; docs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; business_requirements.md is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -TotalCount is a command option or flag; its exact behavior is defined beside the command; 40 is a numeric literal used by the command or expression

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 3 lesson index](#)

Action 3.2.C2 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Select-String -Path .\docs\kpi_definitions.md -Pattern '^## [0-9]+\.'
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `Select-String -Path .\docs\kpi_definitions.md -Pattern '^## [0-9]+\.'` — `Select-String` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `kpi_definitions.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Pattern` is a command option or flag; its exact behavior is defined beside the command; `'^## [0-9]+\.'` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 3 lesson index](#)

Action 3.2.C3 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
git diff -- docs\business_requirements.md docs\assumption_log.md docs\risk_register.md
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- docs\business_requirements.md docs\assumption_log.md docs\risk_register.md` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `business_requirements.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `assumption_log.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `risk_register.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 3 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 3.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 3 starting-screen checklist and re-verify the active context.

Lesson 3.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 3 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — docs/

Exact folder name: docs

Exact parent: repository root

Why it is needed now: Phase 3 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: docs.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 3.D1.1 — Create the folder docs

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 3 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\docs" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\docs" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\docs" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — docs/business_requirements.md

Exact filename: `business_requirements.md`

Filename extension: `.md`. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `docs` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 3.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `docs` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `business_requirements.md`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `business_requirements.md` and an empty file under the correct parent folder.
10. PRESS: `Ctrl+S` after adding each explained section.

PowerShell path

Action 3.F1.1 — Create the file docs/business_requirements.md from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\business_requirements.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType File -Path ".\docs\business_requirements.md" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; File is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\docs\business_requirements.md" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 3 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\docs\business_requirements.md" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/business_requirements.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `docs`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `docs/business_requirements.md`.
3. In PowerShell at the repository root, run `Test-Path ".\docs\business_requirements.md"` and require `True`.
4. Run `git status --short -- "docs/business_requirements.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for docs/business_requirements.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: docs/business_requirements.md

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 3.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Business Requirements

## Company background

Northstar Retail is a fictional omnichannel retailer with physical stores and an e-commerce website. Its operational sales, customer, product, employee, shipment, and inventory data live in SQL Server. Website behavior and marketing activity live in PostgreSQL. Returns, supplier cost changes, promotions, and tracking events arrive as files.

## Business problem

Leadership has inconsistent reports because sales, customers, inventory, website behavior, marketing, shipments, and returns live in separate systems. The company needs a governed data platform that produces trusted daily metrics and supports executive, sales, operations, marketing, and customer analysis.

## Stakeholders

| Stakeholder | Decision | Needed information | Primary concern |
|---|---|---|---|
| Chief Executive Officer | Overall business direction | Revenue, profit, customer, operations health | One trusted summary |
| Finance leader | Profitability and reconciliation | Revenue, COGS, gross profit, discounts, refunds | Definitions and tie-outs |
| Sales leader | Channel/store performance | Orders, AOV, revenue, margin | Comparable stores and channels |
| Merchandising | Product and category choices | Units, margin, discount, return rate | Product-level grain |
| Marketing | Campaign allocation | Spend, sessions, conversion, attributed revenue | Attribution assumptions |
| Operations | Inventory and shipping | On hand, risk, delivery performance | Timeliness and exceptions |
| Customer experience | Returns and delays | Return reasons, late shipments | Traceable order detail |
| Data engineering | Reliability | Freshness, quality, lineage, rerun safety | Recoverable batches |
| Analysts | Self-service reporting | Star schema and semantic measures | Clear definitions |

## User stories

- As an executive, I need a daily overview so I can identify material changes without reconciling multiple reports.
- As a finance analyst, I need line-level revenue and cost logic so I can reproduce gross profit.
- As a store leader, I need store and region comparisons so I can investigate underperformance.
- As a marketer, I need spend, sessions, and converting orders joined under a documented attribution rule.
- As an inventory planner, I need latest store-product stock and reorder thresholds so I can prioritize replenishment.
- As a data engineer, I need batch manifests, tests, quarantine records, and idempotent tasks so failed loads can be repaired safely.

## Business questions

1. How much revenue and gross profit are we generating?
2. Which products, categories, stores, regions, and channels perform best?
3. What is average order value?
4. What is the return rate?
5. What percentage of delivered shipments arrive on time?
6. Which promotions create profitable sales?
7. How does website activity convert into orders?
8. How effective is marketing spend by channel and campaign?
9. Which products are at risk of stocking out?
10. How many new versus returning customers do we have?

Definitions and validation methods are in [`${kpi_definitions.md}`](kpi_definitions.md).

## Freshness targets

| Data domain | Target for this portfolio | Reason |
|---|---|---|
| Sales/orders/payments | Daily by 8:00 AM local reporting time | Executive and store reporting |
| Shipments/returns | Daily by 9:00 AM | Operations and customer experience |
| Inventory snapshot | Daily after source snapshot closes | Avoid summing partial snapshots |
| Website events/sessions | Daily | Batch learning scope |
| Marketing spend | Daily when feed is available | Campaign pacing |
| Power BI | Refresh only after warehouse publication succeeds | Prevent partial reports |

These are learning objectives, not measured service-level agreements until you schedule and observe several verified runs.

## Data-quality expectations

- Required keys are not null in trusted tables.
- Fact grain is unique.
- Dimension references resolve or intentionally use unknown key `0`.
- Sales arithmetic reconciles within one cent per row.
- Source manifests tie to Bronze accepted plus unreadable records.
- Bronze ties to Silver valid plus quarantined records.
- Silver ties to Gold under documented filtering/deduplication rules.
- Currency is USD in the beginner path; non-USD rows are quarantined until conversion is designed.
```

- Dates are parseable, event order is plausible, and timestamps are normalized to UTC.
- No required quality check is hidden or filtered out before publication.

Security assumptions

- All data is synthetic, but customer-like fields are treated as synthetic PII for practice.
- Local credentials belong only in `.env`, which is ignored by Git.
- Databricks credentials belong in secret management or supported identity mechanisms, never notebooks.
- Analysts receive read-only access to serving schemas/views.
- Screenshots must not expose passwords, tokens, connection strings, personal workspace email, or local paths unnecessarily.

Success criteria

The project is successful only when Kirolos has personally verified the gates in the final definition-of-done section in this workbook, can reproduce KPI values in SQL and Power BI, can explain every major design choice, and has screenshot/run evidence. Files existing in Git are not proof of execution.

Out of scope for the required path

- Real customer data, production operations, or legal compliance certification.
- Streaming, CDC, Kafka, production IaC, or automated cloud networking.
- Multi-currency conversion and financial accounting close.
- Causal campaign attribution.
- A fabricated `.pbix` file or fabricated Databricks run.
- Production-scale performance claims.

Assumption log template

ID	Assumption	Why	How to test or revisit	Status
A-001	USD is the reporting currency in Track A	Keeps beginner calculations interpretable	Add FX source and effective-rate rules in an extension	Accepted for Track A
A-002	One sales fact row equals one valid order item	Supports additive revenue and product analysis	Run grain uniqueness and source reconciliation	Must verify
A-003	A shipment is on time when actual date <= promised date	Common operational definition	Confirm business owner treatment of missing dates	Must verify
A-004	Converting session order ID provides simplified attribution	Source has a direct link for some sessions	Check duplicate order links; document last-session limitation	Must verify
A-005	Manual upload/download is acceptable for learning	Avoids unsupported localhost connectivity claims	Move to Track B only with network-accessible sources	Accepted for Track A

Line-by-line explanation

- **Line 1: #** Business Requirements is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3: ##** Company background is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** Northstar Retail is a fictional omnichannel retailer with physical stores and an e-commerce website. Its operational sales, customer, product, employee, shipment, and inventory data live in SQL Server. Website behavior and marketing activity live in PostgreSQL. Returns, supplier cost changes, promotions, and tracking events arrive as files. — Northstar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Retail is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fictional is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; omnichannel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; retailer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; with is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; physical is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stores is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; an is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7: ##** Business problem is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 8:** blank line used only to separate logical sections for readability.

- **Line 9:** Leadership has inconsistent reports because sales, customers, inventory, website behavior, marketing, shipments, and returns live in separate systems. The company needs a governed data platform that produces trusted daily metrics and supports executive, sales, operations, marketing, and customer analysis. — Leadership is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; has is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; inconsistent is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reports is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; because is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; inventory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; website is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 10:** blank line used only to separate logical sections for readability.
- **Line 11:** ## Stakeholders is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** | Stakeholder | Decision | Needed information | Primary concern | — Stakeholder is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Decision is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Needed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; information is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Primary is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concern is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** |---|---|---|---| — --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command
- **Line 15:** | Chief Executive Officer | Overall business direction | Revenue, profit, customer, operations health | One trusted summary | — Chief is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Executive is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Officer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Overall is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; business is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; direction is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 16:** | Finance leader | Profitability and reconciliation | Revenue, COGS, gross profit, discounts, refunds | Definitions and tie-outs | — Finance is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; leader is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Profitability is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reconciliation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COGS is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gross is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 17:** | Sales leader | Channel/store performance | Orders, AOV, revenue, margin | Comparable stores and channels | — Sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; leader is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Channel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; store is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; performance is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AOV is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 18:** | Merchandising | Product and category choices | Units, margin, discount, return rate | Product-level grain | — Merchandising is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; category is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; choices is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Units is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; margin is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; discount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; return is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 19:** | Marketing | Campaign allocation | Spend, sessions, conversion, attributed revenue | Attribution assumptions | — Marketing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Campaign is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; allocation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; conversion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; attributed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Attribution is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 20:** | Operations | Inventory and shipping | On hand, risk, delivery performance | Timeliness and exceptions | — Operations is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Inventory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; shipping is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; On is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; hand is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; risk is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; delivery is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; performance is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Timeliness is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 21:** | Customer experience | Returns and delays | Return reasons, late shipments | Traceable order detail | — Customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; experience is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Returns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; delays is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Return is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reasons is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; late is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Traceable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 22:** | Data engineering | Reliability | Freshness, quality, lineage, rerun safety | Recoverable batches | — Data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; engineering is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Reliability is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Freshness is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quality is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; lineage is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rerun is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; safety is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Recoverable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 23:** | Analysts | Self-service reporting | Star schema and semantic measures | Clear definitions | — Analysts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Self-service is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reporting is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Star is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; semantic is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; measures is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Clear is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; definitions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** blank line used only to separate logical sections for readability.
- **Line 25:** ## User stories is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 26:** blank line used only to separate logical sections for readability.
- **Line 27:** - As an executive, I need a daily overview so I can identify material changes without reconciling multiple reports. — - is a command option or flag; its exact behavior is defined beside the command; As is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; an is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

executive is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; I is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; need is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; daily is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; overview is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; so is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; I is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 28:** - As a finance analyst, I need line-level revenue and cost logic so I can reproduce gross profit. — - is a command option or flag; its exact behavior is defined beside the command; As is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; finance is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; analyst is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; I is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; need is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; line-level is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cost is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 29:** - As a store leader, I need store and region comparisons so I can investigate underperformance. — - is a command option or flag; its exact behavior is defined beside the command; As is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; store is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; leader is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; I is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; need is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; store is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; region is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; comparisons is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 30:** - As a marketer, I need spend, sessions, and converting orders joined under a documented attribution rule. — - is a command option or flag; its exact behavior is defined beside the command; As is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; marketer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; I is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; need is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 31:** - As an inventory planner, I need latest store-product stock and reorder thresholds so I can prioritize replenishment. — - is a command option or flag; its exact behavior is defined beside the command; As is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; an is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding

explanation; inventory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; planner is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; I is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; need is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; latest is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; store-product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stock is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 32:** - As a data engineer, I need batch manifests, tests, quarantine records, and idempotent tasks so failed loads can be repaired safely. — - is a command option or flag; its exact behavior is defined beside the command; As is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; engineer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; I is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; need is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; manifests is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tests is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 33:** blank line used only to separate logical sections for readability.
- **Line 34:** ## Business questions is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 35:** blank line used only to separate logical sections for readability.
- **Line 36:** 1. How much revenue and gross profit are we generating? — 1 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; How is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; much is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gross is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; we is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; generating is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 37:** 2. Which products, categories, stores, regions, and channels perform best? — 2 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Which is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; products is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; categories is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stores is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; regions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 38:** 3. What is average order value? — 3 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; What is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; average is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 39:** 4. What is the return rate? — 4 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; What is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; return is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 40:** 5. What percentage of delivered shipments arrive on time? — 5 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; What is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; percentage is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; of is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; delivered is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; arrive is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** 6. Which promotions create profitable sales? — 6 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Which is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; promotions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; create is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; profitable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** 7. How does website activity convert into orders? — 7 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; How is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; does is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; website is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; activity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; convert is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; into is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 43:** 8. How effective is marketing spend by channel and campaign? — 8 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; How is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; effective is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; marketing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; by is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; channel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; campaign is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 44:** 9. Which products are at risk of stocking out? — 9 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Which is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; products is a name, keyword, argument, identifier, or literal whose project-specific role is described in the

surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; risk is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; of is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stocking is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; out is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 45:** 10. How many new versus returning customers do we have? — 10 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; How is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; many is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; new is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; versus is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; returning is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; do is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; we is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; have is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** blank line used only to separate logical sections for readability.
- **Line 47:** Definitions and validation methods are in [\kpi_definitions.md](kpi_definitions.md).—Definitions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; validation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; methods is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; kpi_definitions.md is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; kpi_definitions.md is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 48:** blank line used only to separate logical sections for readability.
- **Line 49:** ## Freshness targets is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 50:** blank line used only to separate logical sections for readability.
- **Line 51:** | Data domain | Target for this portfolio | Reason | — Data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; domain is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Target is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; this is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; portfolio is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Reason is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 52:** | --- | --- | --- | — --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command
- **Line 53:** | Sales/orders/payments | Daily by 8:00 AM local reporting time | Executive and store reporting | — Sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; payments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Daily is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

surrounding explanation; by is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 8 is a numeric literal used by the command or expression; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 00 is a numeric literal used by the command or expression; AM is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; local is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 54:** | Shipments/returns | Daily by 9:00 AM | Operations and customer experience | — Shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; returns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Daily is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; by is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 9 is a numeric literal used by the command or expression; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 00 is a numeric literal used by the command or expression; AM is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Operations is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 55:** | Inventory snapshot | Daily after source snapshot closes | Avoid summing partial snapshots | — Inventory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; snapshot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Daily is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; after is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; snapshot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; closes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Avoid is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; summing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; partial is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; snapshots is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 56:** | Website events/sessions | Daily | Batch learning scope | — Website is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; events is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Daily is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; learning is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; scope is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 57:** | Marketing spend | Daily when feed is available | Campaign pacing | — Marketing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Daily is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; when is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; feed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; available is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Campaign is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pacing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 58:** | Power BI | Refresh only after warehouse publication succeeds | Prevent partial reports | — Power is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BI is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Refresh is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; only is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; after is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; warehouse is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; publication is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; succeeds is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Prevent is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; partial is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reports is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 59:** blank line used only to separate logical sections for readability.
- **Line 60:** These are learning objectives, not measured service-level agreements until you schedule and observe several verified runs. — These is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; learning is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; objectives is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; measured is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; service-level is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; agreements is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; until is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; you is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; schedule is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 61:** blank line used only to separate logical sections for readability.
- **Line 62:** ## Data-quality expectations is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 63:** blank line used only to separate logical sections for readability.
- **Line 64:** - Required keys are not null in trusted tables. — - is a command option or flag; its exact behavior is defined beside the command; Required is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; keys is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; null is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; trusted is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tables is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 65:** - Fact grain is unique. — - is a command option or flag; its exact behavior is defined beside the command; Fact is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; grain is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; unique is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 66:** - Dimension references resolve or intentionally use unknown key \0.—is a command option or flag; its exact behavior is defined beside the command; Dimension is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; references is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; resolve is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; intentionally is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; use is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; unknown is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

specific role is described in the surrounding explanation;key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;0 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 67:** - Sales arithmetic reconciles within one cent per row. — - is a command option or flag; its exact behavior is defined beside the command; Sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; arithmetic is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reconciles is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; within is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cent is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; per is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; row is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 68:** - Source manifests tie to Bronze accepted plus unreadable records. — - is a command option or flag; its exact behavior is defined beside the command; Source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; manifests is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tie is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; to is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Bronze is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; accepted is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; plus is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; unreadable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; records is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 69:** - Bronze ties to Silver valid plus quarantined records. — - is a command option or flag; its exact behavior is defined beside the command; Bronze is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ties is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; to is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Silver is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; plus is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quarantined is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; records is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 70:** - Silver ties to Gold under documented filtering/deduplication rules. — - is a command option or flag; its exact behavior is defined beside the command; Silver is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ties is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; to is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; under is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; documented is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; filtering is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; deduplication is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rules is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 71:** - Currency is USD in the beginner path; non-USD rows are quarantined until conversion is designed. — - is a command option or flag; its exact behavior is defined beside the command; Currency is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; USD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding ex-

planation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; beginner is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; non-USD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quarantined is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 72:** - Dates are parseable, event order is plausible, and timestamps are normalized to UTC. — - is a command option or flag; its exact behavior is defined beside the command; Dates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; parseable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; event is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; plausible is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timestamps is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 73:** - No required quality check is hidden or filtered out before publication. — - is a command option or flag; its exact behavior is defined beside the command; No is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; required is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quality is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; check is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; hidden is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; filtered is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; out is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; before is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; publication is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 74:** blank line used only to separate logical sections for readability.
- **Line 75:** ## Security assumptions is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 76:** blank line used only to separate logical sections for readability.
- **Line 77:** - All data is synthetic, but customer-like fields are treated as synthetic PII for practice. — - is a command option or flag; its exact behavior is defined beside the command; All is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; synthetic is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; but is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer-like is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fields is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; treated is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; as is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 78:** - Local credentials belong only in `\.env\`, which is ignored by Git.— is a command option or flag; its exact behavior is defined beside the command; Local is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; credential is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; belong is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; only is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is a path, relative location, branch-style name, or file argument; verify every character before running it; env is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; which is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ignored is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 79:** - Databricks credentials belong in secret management or supported identity mechanisms, never notebooks. — is a command option or flag; its exact behavior is defined beside the command; Databricks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; credential is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; belong is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; secret is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; management is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; supported is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; identity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mechanisms is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 80:** - Analysts receive read-only access to serving schemas/views. — is a command option or flag; its exact behavior is defined beside the command; Analysts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; receive is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; read-only is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; access is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; to is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; serving is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; schemas is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; views is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 81:** - Screenshots must not expose passwords, tokens, connection strings, personal workspace email, or local paths unnecessarily. — is a command option or flag; its exact behavior is defined beside the command; Screenshots is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; must is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; expose is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; passwords is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tokens is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; connection is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; strings is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 82:** blank line used only to separate logical sections for readability.

- **Line 83:** ## Success criteria is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 84:** blank line used only to separate logical sections for readability.
- **Line 85:** The project is successful only when Kirolos has personally verified the gates in the final definition-of-done section in this workbook, can reproduce KPI values in SQL and Power BI, can explain every major design choice, and has screenshot/run evidence. Files existing in Git are not proof of execution. — The is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; project is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; successful is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; only is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; when is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Kirolos is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; has is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; personally is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; verified is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 86:** blank line used only to separate logical sections for readability.
- **Line 87:** ## Out of scope for the required path is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 88:** blank line used only to separate logical sections for readability.
- **Line 89:** - Real customer data, production operations, or legal compliance certification. — - is a command option or flag; its exact behavior is defined beside the command; Real is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; production is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; operations is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; legal is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; compliance is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; certification is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 90:** - Streaming, CDC, Kafka, production IaC, or automated cloud networking. — - is a command option or flag; its exact behavior is defined beside the command; Streaming is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CDC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Kafka is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; production is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IaC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; automated is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 91:** - Multi-currency conversion and financial accounting close. — - is a command option or flag; its exact behavior is defined beside the command; Multi-currency is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; conversion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; financial is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; accounting is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; close is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 92:** - Causal campaign attribution. — - is a command option or flag; its exact behavior is defined beside the command; Causal is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; campaign is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; attribution is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 93:** - A fabricated \.pbix file or fabricated Databricks run.—is a command option or flag; its exact behavior is defined beside the command; A is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fabricated is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; pbix is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fabricated is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Databricks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 94:** - Production-scale performance claims. — - is a command option or flag; its exact behavior is defined beside the command; Production-scale is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; performance is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; claims is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 95:** blank line used only to separate logical sections for readability.
- **Line 96:** ## Assumption log template is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 97:** blank line used only to separate logical sections for readability.
- **Line 98:** | ID | Assumption | Why | How to test or revisit | Status | — ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Assumption is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Why is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; How is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; to is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; revisit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Status is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 99:** | --- | --- | --- | --- | --- | — --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command
- **Line 100:** | A-001 | USD is the reporting currency in Track A | Keeps beginner calculations interpretable | Add FX source and effective-rate rules in an extension | Accepted for Track A | — A-001 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; USD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reporting is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; currency is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; Track is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; A is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Keeps is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; beginner is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; calculations is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 101:** | A-002 | One sales fact row equals one valid order item | Supports additive revenue and product analysis | Run grain uniqueness and source reconciliation | Must verify | — A-002 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; One is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; row is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; equals is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; item is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Supports is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; additive is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 102:** | A-003 | A shipment is on time when actual date <= promised date | Common operational definition | Confirm business owner treatment of missing dates | Must verify | — A-003 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; A is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; shipment is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; when is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; actual is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; <= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; promised is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 103:** | A-004 | Converting session order ID provides simplified attribution | Source has a direct link for some sessions | Check duplicate order links; document last-session limitation | Must verify | — A-004 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Converting is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; session is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; provides is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; simplified is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; attribution is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; has is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; direct is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 104:** | A-005 | Manual upload/download is acceptable for learning | Avoids unsupported localhost connectivity claims | Move to Track B only with network-accessible sources | Accepted for Track A | — A-005 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Manual is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; upload is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; download is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; acceptable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; learning is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Avoids is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; unsupported is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; localhost is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

[Back to the Phase 3 lesson index](#)

File introduction — docs/risk_register.md

Exact filename: risk_register.md

Filename extension: .md. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: docs relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 3.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in docs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: risk_register.md.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled risk_register.md and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 3.F2.1 — Create the file docs/risk_register.md from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\risk_register.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\docs\risk_register.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\docs\risk_register.md` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use

`git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 3 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\docs\risk_register.md" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/risk_register.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under docs.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `docs/risk_register.md`.
3. In PowerShell at the repository root, run `Test-Path ".\docs\risk_register.md"` and require `True`.
4. Run `git status --short -- "docs/risk_register.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for docs/risk_register.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: `docs/risk_register.md`

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 3.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

Risk Register

```
| ID | Risk | Probability | Impact | Early warning | Mitigation | Owner | Status |
|---|---|---|---|---|---|---|---|
| R-001 | Credentials are committed | Medium | High | `.env` appears in `git status` | `.gitignore`, secret scan, rotate immediately if exposed | Kirolos | Open |
| R-002 | Databricks cannot see local source | High if misunderstood | Medium | JDBC timeout to localhost | Use Track A volume handoff; do not tunnel casually | Kirolos | Mitigated by design |
| R-003 | Duplicate rerun inflates facts | Medium | High | Same batch doubles row count | Batch deletion/merge, unique grain tests, max concurrent runs 1 | Data engineering | Open until verified |
| R-004 | SCD2 facts join to current instead of event-time row | Medium | High | Historical reports change after customer/product update | Effective-date join tests and worked example | Data engineering | Open |
| R-005 | Return rate mixes incompatible date roles | Medium | Medium | SQL and dashboard disagree by period | Label purchase-date vs return-date view; validation samples | Analytics | Open |
| R-006 | Free-tier quotas interrupt learning runs | Medium | Low | Compute unavailable or quota warning | Use quick/small scale, stop retries, preserve outputs | Kirolos | Monitor |
| R-007 | Power BI ambiguous relationships change totals | Medium | High | Warning icon or totals vary unexpectedly | Single-direction star schema, inactive role dates, SQL validation | Kirolos | Open |
| R-008 | Larger generated files enter Git | Medium | Medium | Git status shows datasets/raw or generated | `.gitignore`, publish quick sample only | Kirolos | Mitigated by config |
| R-009 | Reference TODO is mistaken for completed work | Medium | High | README claims verified results without evidence | Status labels, checklist, screenshot proof, replace placeholders only after verification | Kirolos | Open |
| R-010 | Schema changes break ingestion | Medium | Medium | read/merge error or missing column | Schema drift tests, quarantine, explicit column lists | Data engineering | Open |
```

Line-by-line explanation

- Line 1:** # Risk Register is a comment for a human reader; it normally does not change data or execute business logic.
- Line 2:** blank line used only to separate logical sections for readability.
- Line 3:** | ID | Risk | Probability | Impact | Early warning | Mitigation | Owner | Status | — ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Risk is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Probability is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Impact is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Early is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; warning is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Mitigation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Owner is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Status is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** |---|---|---|---|---|---|---| — --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; ---: is a command option or flag; its exact behavior is defined beside the command; ---: is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command
- Line 5:** | R-001 | Credentials are committed | Medium | High | `.env` appears in `git status` | `.gitignore`, secret scan, rotate immediately if exposed | Kirolos | Open | — R-001 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Credentials is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; committed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Medium is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; High is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is a path, relative location, branch-style name, or file argument; verify every character before running it; env is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; appears is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; git calls the Git version-control program; status shows tracked, staged, modified, and untracked work; remaining punctuation and identifiers continue the same statement.
- Line 6:** | R-002 | Databricks cannot see local source | High if misunderstood | Medium | JDBC timeout to localhost | Use Track A volume handoff; do not tunnel casually | Kirolos | Mitigated by design | — R-002 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

tion; Databricks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cannot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; see is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; local is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; High is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; misunderstood is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Medium is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JDBC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timeout is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 7:** | R-003 | Duplicate rerun inflates facts | Medium | High | Same batch doubles row count | Batch deletion/merge, unique grain tests, max concurrent runs 1 | Data engineering | Open until verified | — R-003 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Duplicate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rerun is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; inflates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; facts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Medium is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; High is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Same is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; doubles is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; row is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 8:** | R-004 | SCD2 facts join to current instead of event-time row | Medium | High | Historical reports change after customer/product update | Effective-date join tests and worked example | Data engineering | Open | — R-004 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SCD2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; facts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; join is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; to is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; current is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; instead is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; of is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; event-time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; row is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Medium is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; High is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 9:** | R-005 | Return rate mixes incompatible date roles | Medium | Medium | SQL and dashboard disagree by period | Label purchase-date vs return-date view; validation samples | Analytics | Open | — R-005 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Return is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mixes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; incompatible is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; roles is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Medium is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Medium is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is

a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dashboard is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 10:** | R-006 | Free-tier quotas interrupt learning runs | Medium | Low | Compute unavailable or quota warning | Use quick/small scale, stop retries, preserve outputs | Kirolos | Monitor | — R-006 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Free-tier is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quotas is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; interrupt is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; learning is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; runs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Medium is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Low is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Compute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; unavailable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quota is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 11:** | R-007 | Power BI ambiguous relationships change totals | Medium | High | Warning icon or totals vary unexpectedly | Single-direction star schema, inactive role dates, SQL validation | Kirolos | Open | — R-007 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Power is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BI is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ambiguous is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; relationships is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; change is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; totals is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Medium is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; High is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Warning is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; icon is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 12:** | R-008 | Larger generated files enter Git | Medium | Medium | Git status shows datasets/raw or generated | \.gitignore, publish quick sample only | Kirolos | Mitigated by config | —R-008 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Larger is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; generated is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; files is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; enter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Git is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Medium is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Medium is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Medium is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Git is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; status shows tracked, staged, modified, and untracked work; shows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 13:** | R-009 | Reference TODO is mistaken for completed work | Medium | High | README claims verified results without evidence | Status labels, checklist, screenshot proof, replace placeholders only after verification | Kirolos | Open | — R-009 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Reference is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; TODO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mistaken is a name, keyword, ar-

gument, identifier, or literal whose project-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; completed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; work is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Medium is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; High is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; README is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; claims is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 14:** | R-010 | Schema changes break ingestion | Medium | Medium | read/merge error or missing column | Schema drift tests, quarantine, explicit column lists | Data engineering | Open | — R-010 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; changes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; break is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ingestion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Medium is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Medium is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; read is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; merge is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; error is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

[Back to the Phase 3 lesson index](#)

File introduction — docs/assumption_log.md

Exact filename: assumption_log.md

Filename extension: .md. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: docs relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 3.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in docs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: assumption_log.md.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled assumption_log.md and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 3.F3.1 — Create the file docs/assumption_log.md from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\assumption_log.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType File -Path ".\docs\assumption_log.md" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; File is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\docs\assumption_log.md" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 3 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\docs\assumption_log.md" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/assumption_log.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under docs.

2. Right-click the editor tab and choose **Copy Path**; compare the ending with docs/assumption_log.md.
3. In PowerShell at the repository root, run `Test-Path ".\docs\assumption_log.md"` and require True.
4. Run `git status --short -- "docs/assumption_log.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for docs/assumption_log.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: docs/assumption_log.md

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 3.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Assumption Log

Update this file whenever you make a decision without complete evidence.

| ID | Date | Assumption or decision | Owner | Evidence | Risk if wrong | Review date | Status |
|----|----|-----|-----|-----|-----|-----|-----|
| A-001 | [DATE] | Track A reports in USD only. | Kirolos | Project scope | Non-USD rows are excluded rather than converted. | [DATE] | Accepted for Track A |
| A-002 | [DATE] | [ADD YOUR ASSUMPTION] | [OWNER] | [LINK/QUERY] | [RISK] | [DATE] | Open |
```

Line-by-line explanation

- **Line 1:** # Assumption Log is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** Update this file whenever you make a decision without complete evidence. — Update is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; this is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; whenever is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; you is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; make is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; decision is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; without is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; complete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; evidence is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** | ID | Date | Assumption or decision | Owner | Evidence | Risk if wrong | Review date | Status | — ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Assumption is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; decision is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Owner is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Evidence is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Risk is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; wrong is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Review is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 6:** ---|---|---|---|---|---|---|---| — --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command;

is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command

- **Line 7:** | A-001 | [DATE] | Track A reports in USD only. | Kirolos | Project scope | Non-USD rows are excluded rather than converted. | [DATE] | Accepted for Track A | — A-001 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Track is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; A is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reports is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; USD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; only is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Kirolos is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 8:** | A-002 | [DATE] | [ADD YOUR ASSUMPTION] | [OWNER] | [LINK/QUERY] | [RISK] | [DATE] | Open | — A-002 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ADD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; YOUR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ASSUMPTION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; OWNER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

[Back to the Phase 3 lesson index](#)

What your screen should look like now

- Every Phase 3 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 3.3 completion checklist

☐

docs/business_requirements.md was created by me, saved, located correctly, and inspected — [STATUS]

☐

docs/risk_register.md was created by me, saved, located correctly, and inspected — [STATUS]

☐

docs/assumption_log.md was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with `Ctrl+S`. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 3.4](#) and recheck the root and branch.

Lesson 3.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Action 3.4.G1 — Inspect the current state without changing it

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Inspect the current state without changing it. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 3 lesson index](#)

Action 3.4.G2 — Create the smallest required object or decision

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Create the smallest required object or decision. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 3 lesson index](#)

Action 3.4.G3 — Run the phase-specific verification

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run the phase-specific verification. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 3 lesson index](#)

Action 3.4.G4 — Demonstrate one safe failure or alternate-result branch

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Demonstrate one safe failure or alternate-result branch. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 3 lesson index](#)

Action 3.4.G5 — Record evidence and prepare the downstream handoff

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Record evidence and prepare the downstream handoff. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

What your screen should look like now

- The new Phase 3 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 3.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 3.5](#).

Lesson 3.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 3 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Verification before automation

Question:

Should the phase prioritize a visible manual verification or jump directly to automation?

Options considered:

- Manual verification first, then automation.
- Automation without a known-good baseline.
- Manual operation forever.

Beginner path selected:

Establish one verified manual baseline and then automate repeatable checks.

Why this path was selected:

A known-good example makes failures interpretable and prevents automation from hiding a wrong assumption.

Tradeoff:

The first run takes longer.

What a company might do instead:

Teams often pair infrastructure as code and automated tests with controlled bootstrap and acceptance procedures.

How to explain this in an interview:

I established a verified baseline, then encoded the same contract in tests and orchestration so automation preserved meaning rather than just speed.

If you see Result A

Every requirement maps to an owner, metric, source, acceptance criterion, and limitation. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

A requirement is vague, unmeasurable, unsupported by a source, or missing an acceptance test. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: VS Code, Markdown Preview, Windows PowerShell, and Git.

Window/panel: Show requirements section plus acceptance-criteria mapping.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase03_write-business-requirements-and-acceptance-criteria_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 3's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 3 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 3.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 3 lesson index](#)

Action 3.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 3 lesson index](#)

Action 3.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 3 lesson index](#)

Interpret git status --short

- ?? means untracked: Git sees a new path but it is not staged.
- M means modified in the working tree but not staged.
- M means the staged version differs from the last commit.
- MM means a staged version exists and the file has additional unstaged edits.
- A means staged as a new file.
- D means deletion; stop and investigate unless deletion is deliberate.
- R means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 3.5.G4 — Inspect the unstaged diff for docs/business_requirements.md

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):


```
git diff -- "docs/business_requirements.md"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "docs/business_requirements.md"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"docs/business_requirements.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 3 lesson index](#)

Action 3.5.G5 — Stage only the reviewed file `docs/business_requirements.md`

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "docs/business_requirements.md"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "docs/business_requirements.md"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"docs/business_requirements.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 3 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 3.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.

6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 3 lesson index](#)

Action 3.5.G7 — Create the Phase 3 commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "docs: define northstar business requirements"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "docs: define northstar business requirements"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"docs: define northstar business requirements"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 3 lesson index](#)

Action 3.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

Action 3.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 3 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message `docs: define northstar business requirements` and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 3.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.

9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 3 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

"Phase 3 created the verified boundary described by **Write business requirements and acceptance criteria**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed."

Mini quiz

1. Multiple choice: Why does Phase 3 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

My Phase 3 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 3.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 4 turns approved requirements, grains, KPIs, and quality expectations into source schemas.

Phase 3 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 4 — Design and create the SQL Server and PostgreSQL source systems

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: Written business requirements and source placement decisions.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 5 uses each column, constraint, and relationship as a generation requirement.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 4 lesson index

1. [Lesson 4.1 — Understand the phase and starting screen](#)
2. [Lesson 4.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 4.3 — Create and understand the phase files](#)
4. [Lesson 4.4 — Build and validate required objects](#)
5. [Lesson 4.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 4.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** VS Code, SSMS, pgAdmin, and PowerShell.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 4 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 3 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 4 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

VS Code, SSMS, pgAdmin, and PowerShell. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 4 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 4 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 4 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 4.1 completion checklist

☐

I read why Phase 4 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 4.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 4 lesson index](#)

Lesson 4.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 4 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

Two separate normalized operational databases: NorthstarRetail_ERP in Microsoft SQL Server, managed through SSMS, and northstar_digital in PostgreSQL, managed through pgAdmin or psql. Each has keys, constraints, indexes, audit columns, ERDs, safe idempotent setup, validation, and guarded development reset scripts.

2. Why it matters

Source systems optimize operational correctness and updates, not reporting convenience. Building them separately teaches dialect boundaries, client tools, service/connection concepts, normalization, and why the later warehouse model intentionally looks different.

3. Words to know

Term	Plain-language definition
T-SQL	SQL Server's SQL dialect.
PostgreSQL SQL	PostgreSQL's SQL dialect; similar but not interchangeable with T-SQL.
Normalized model	Entities separated to reduce update inconsistency and repetition.
Constraint	Database rule such as primary key, foreign key, unique, or check.
Index	Structure that speeds selected reads at storage/write cost.
Schema	Namespace inside a database, such as erp or digital.
DDL	Data Definition Language used to create/alter objects.
Idempotent DDL	Setup that can be rerun without blindly duplicating/replacing objects.
Object Explorer	SSMS tree for servers/databases/tables.
Query Tool	pgAdmin editor for PostgreSQL SQL.
Connection	Host/instance, port, database, authentication, and encryption settings.
Self-referencing key	Foreign key from a table to itself, such as employee manager.

4. What must already be completed

Phases 1–3 are complete. SQL Server engine, SSMS, PostgreSQL, pgAdmin, psql, and ODBC are installed/verified. You know the exact SQL Server service/instance and PostgreSQL port/password location.

5. Applications to open

Application	How to open it	Why it is open
SQL Server Configuration Manager or Windows Services	Search Start for the installed tool/service console.	Confirm SQL Server instance service name/running state.
SSMS	Start menu > SQL Server Management Studio.	Connect, run T-SQL, inspect ERP objects.
pgAdmin	Start menu > pgAdmin 4.	Connect/register server, create database, run PostgreSQL SQL.

Application	How to open it	Why it is open
Windows PowerShell	Repository root.	Diagnose services/ports and open files.
VS Code	Repository root.	Read DDL and ERDs before execution.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run `Get-Location`; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
Get-Service Where-Object { \$.Name -like 'MSSQL*' -or \$.DisplayName -like 'SQL Server (*)' } Format-Table Name, DisplayName, Status	Lists installed SQL Server engine services and state.	Server names must come from the actual instance, not a guess.
SELECT @@SERVERNAME AS server_name, SUSER_SNAME() AS login_name, DB_NAME() AS database_name;	Shows server, login, and current database.	DDL should never run against an assumed context.
-- Execute the entire opened 01_create_erp_database.sql file with the Execute button or F5.	Creates database and schemas only when missing.	Tables need a database/schema home.
-- Execute the entire opened 02_create_erp_tables.sql file.	Creates normalized ERP tables, constraints, and selected indexes.	The generator/loaders need real target structures.
Get-Service postgres Format-Table Name, Status, DisplayName	Lists PostgreSQL service state.	Connection refused often means the engine is stopped.
Test-NetConnection localhost -Port 5432	Tests whether a process accepts TCP connections on the PostgreSQL port.	Separates service/network issues from password/database issues.
-- Execute the entire opened 01_create_digital_schema.sql file using the Execute/Play button.	Creates digital/audit schemas, tables, constraints, and indexes.	Digital generated files need their own PostgreSQL source model.
psql -h localhost -p 5432 -U postgres -d northstar_digital	Starts an interactive PostgreSQL session; it prompts for password if required.	Provides an alternate tool and isolates pgAdmin-specific issues.

SQL dialect boundary

Concept	SQL Server / T-SQL	PostgreSQL SQL
Batch separator	GO client directive	none; use semicolons
Identity	IDENTITY(1,1)	generated identity/serial patterns
Current UTC	SYSUTCDATETIME()	now() returns timestampz
Top rows	SELECT TOP (10)	LIMIT 10
Conditional table	IF OBJECT_ID...	CREATE TABLE IF NOT EXISTS
Quoted identifier	[name] or ANSI quotes	"name"

Never paste `GO`, `TOP`, or `OBJECT_ID` into PostgreSQL. Never paste PostgreSQL `jsonb`, `timestampz`, or `DO $$` into T-SQL without translation.

MY TASK

Create both source databases, inspect every required table, explain two normalization choices, and add one justified non-unique index as a TODO(USER) exercise without constraining the deliberately dirty business key.

STARTER SKELETON

```
-- Dialect: SQL Server / T-SQL
-- TODO(USER): choose a real query pattern before adding an index.
USE NorthstarRetail_ERP;
GO
IF NOT EXISTS (
    SELECT 1 FROM sys.indexes
    WHERE object_id = OBJECT_ID(N'erp._____' )
    AND name = N'IX_____'
)
BEGIN
    CREATE INDEX IX_____
    ON erp._____(_____)
    INCLUDE (____);
END;
GO

-- TODO(USER) Explain the read benefit and write/storage cost.
```

Acceptance criteria: Both database contexts show all required tables; setup reruns without destructive errors; the added index is tied to a stated query pattern and does not block controlled quality issues.

HINT 1 — CONCEPT

Normalize around business entities and transactional update rules, not dashboard pages.

HINT 2 — PSEUDOCODE

Index columns used to filter/join/order frequently; INCLUDE columns can cover output without becoming key columns.

HINT 3 — NEAR-COMPLETE PATTERN

Model the existing IX_orders_updated_at pattern only after choosing a different table/query.

13. Expected files or output

- NorthstarRetail_ERP with schemas erp and audit.
- northstar_digital with schemas digital and audit.
- 13 ERP and 6 digital source tables.
- Object inventory query outputs.
- Journal records exact server/port/database context without passwords.

14. How to verify

- Run both source validation SQL files.
- Re-run creation scripts; no data/object replacement occurs.
- Inspect primary/foreign keys in SSMS and pgAdmin properties.
- Explain why customer_number is intentionally not unique in the source fixture.

15. Automated tests

- DDL object inventory.
- Database constraints prevent normal orphan records after load.
- Dialect header validation script checks every SQL file.
- Later loaders/reconciliation prove schema compatibility.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
SSMS cannot connect	Wrong instance/service/auth or certificate choice.	Service listing; SSMS exact error; try recorded instance only.	Start service and use actual server name/auth; do not enable broad insecure protocols blindly.	Context query succeeds.
Database already exists	Setup rerun or prior attempt.	SELECT DB_ID('Northstar-Retail_ERP');.	Use idempotent script; do not drop to silence.	Tables/inventory correct.
PostgreSQL connection refused	Service/port issue.	Service list and Test-NetConnection.	Start correct service/use recorded port.	pgAdmin/psql connects.
CREATE DATABASE cannot run in transaction	pgAdmin executed it in a transaction or database already exists.	Refresh Databases; inspect exact error.	Use pgAdmin Create Database UI; then connect to it before schema script.	Current database query returns northstar_digital.
Syntax error near GO/TOP/jsonb	Wrong dialect/tool.	Read first-line dialect header and current editor connection.	Run T-SQL in SSMS and PostgreSQL SQL in pgAdmin/psql.	Correct file executes.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why are SQL Server and PostgreSQL separate systems even though both use SQL?
2. Why does the operational source use normalized tables?
3. What tradeoff does an index introduce?
4. Why do guarded reset scripts still require judgment?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. Which application runs T-SQL in this project?
2. What does a foreign key protect?
3. Why was a unique constraint intentionally omitted from customer_number?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture SSMS Object Explorer expanded to NorthstarRetail_ERP > Tables and pgAdmin expanded to northstar_digital > Schemas > digital > Tables, plus context query outputs.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
feat: create normalized sql server and postgres source schemas
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

SQL Server instance/context verified — [STATUS]

☐

ERP database/tables created — [STATUS]

☐

PostgreSQL server/port/context verified — [STATUS]

☐

Digital database/tables created — [STATUS]

☐

DDL rerun safety verified — [STATUS]

☐

ERDs and normalization explained — [STATUS]

☐

Source validation queries run — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-4. The quiz answer is in [the answer-key section immediately after the quiz in this workbook#phase-4] (the answer-key section immediately after the quiz in this workbook#phase-4).

Next phase: Phase 5 generates reproducible fictional data, including controlled defects for Silver.

Required action cards

Action 4.2.1 — Open docs/images/sqlserver_erd.svg and postgres_erd.svg. For each table, state its primary key and one par

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Open docs/images/sqlserver_erd.svg and postgres_erd.svg. For each table, state its primary key and one par. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open docs/images/sqlserver_erd.svg and postgres_erd.svg. For each table, state its primary key and one parent/child relationship.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL psql, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 1

Action

Open docs/images/sqlserver_erd.svg and postgres_erd.svg. For each table, state its primary key and one parent/child relationship.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Action 4.2.2 — In PowerShell, list SQL Server services. A default instance service is commonly MSSQLSERVER; a named Express

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

In PowerShell, list SQL Server services. A default instance service is commonly MSSQLSERVER; a named Express. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: In PowerShell, list SQL Server services. A default instance service is commonly MSSQLSERVER; a named Express instance commonly contains SQLEXPRESS, but use the actual result.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 2****Action**

In PowerShell, list SQL Server services. A default instance service is commonly MSSQLSERVER; a named Express instance commonly contains SQLEXPRESS, but use the actual result.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Action 4.2.3 — Open SSMS. In , choose . Server name should match your verified instance, often localhost for default or `lo

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Open SSMS. In , choose . Server name should match your verified instance, often localhost for default or `lo. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open SSMS. In **Connect to Server**, choose **Database Engine**. Server name should match your verified instance, often localhost for default or localhost\SQLEXPRESS for Express. Choose Windows Authentication unless you intentionally configured SQL authentication. Check encryption/certificate options according to local official guidance.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 3

Action

Open SSMS. In **Connect to Server**, choose **Database Engine**. Server name should match your verified instance, often localhost for default or localhost\SQLEXPRESS for Express. Choose Windows Authentication unless you intentionally configured SQL authentication. Check encryption/certificate options according to local official guidance.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Action 4.2.4 — Click , then . Before running DDL, execute SELECT @@SERVERNAME, SUSER_SNAME() , DB_NAME() ; and confirm identi

Application:

Windows File Explorer

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Click , then . Before running DDL, execute `SELECT @@SERVERNAME, SUSER_SNAME(), DB_NAME();` and confirm identity. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Click **Connect**, then **New Query**. Before running DDL, execute `SELECT @@SERVERNAME, SUSER_SNAME(), DB_NAME();` and confirm identity/context.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows File Explorer. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 4****Action**

Click **Connect**, then **New Query**. Before running DDL, execute `SELECT @@SERVERNAME, SUSER_SNAME(), DB_NAME();` and confirm identity/context.

Before continuing
☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Action 4.2.5 — Open sql/sqlserver/source/01_create_erp_database.sql using , execute, read Messages, then open/run `02_creat

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Open sql/sqlserver/source/01_create_erp_database.sql using , execute, read Messages, then open/run `02_creat. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open sql/sqlserver/source/01_create_erp_database.sql using **File > Open > File**, execute, read Messages, then open/run 02_create_erp_tables.sql.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 5

Action

Open sql/sqlserver/source/01_create_erp_database.sql using **File > Open > File**, execute, read Messages, then open/run 02_create_erp_tables.sql.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Action 4.2.6 — In Object Explorer, right-click , expand NorthstarRetail_ERP > Tables, and inspect erp.* tables. Right-cli

Application:

Windows File Explorer

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

In Object Explorer, right-click , expand NorthstarRetail_ERP > Tables, and inspect erp.* tables. Right-click. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: In Object Explorer, right-click **Databases > Refresh**, expand NorthstarRetail_ERP > Tables, and inspect erp.* tables. Right-click one table > **Select Top 1000 Rows** only as inspection; do not edit rows there.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows File Explorer. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 6****Action**

In Object Explorer, right-click **Databases > Refresh**, expand NorthstarRetail_ERP > Tables, and inspect erp.* tables. Right-click one table > **Select Top 1000 Rows** only as inspection; do not edit rows there.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Action 4.2.7 — Run sql/sqlserver/validation/01_validate_erp_source.sql. Before data load, counts are zero and object checks

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run sql/sqlserver/validation/01_validate_erp_source.sql. Before data load, counts are zero and object checks. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run sql/sqlserver/validation/01_validate_erp_source.sql. Before data load, counts are zero and object checks should work.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 7

Action

Run sql/sqlserver/validation/01_validate_erp_source.sql. Before data load, counts are zero and object checks should work.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Action 4.2.8 — Open pgAdmin. If the installer-created local server is not present, right-click . Give it a descriptive name;

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Open pgAdmin. If the installer-created local server is not present, right-click . Give it a descriptive name; . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open pgAdmin. If the installer-created local server is not present, right-click **Servers > Register > Server**. Give it a descriptive name; Connection host localhost, verified port, maintenance database postgres, username postgres or your configured admin. Save password only if your local security choice allows it.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL psql, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 8****Action**

Open pgAdmin. If the installer-created local server is not present, right-click **Servers > Register > Server**. Give it a descriptive name; Connection host localhost, verified port, maintenance database postgres, username postgres or your configured admin. Save password only if your local security choice allows it.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Action 4.2.9 — Expand the connected server, right-click , name it northstar_digital, owner your local admin, Save. This avo

Application:

Windows File Explorer

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Expand the connected server, right-click , name it northstar_digital, owner your local admin, Save. This avo. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Expand the connected server, right-click **Databases > Create > Database**, name it northstar_digital, owner your local admin, Save. This avoids rerun issues with CREATE DATABASE in a transaction.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows File Explorer. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 9

Action

Expand the connected server, right-click **Databases > Create > Database**, name it northstar_digital, owner your local admin, Save. This avoids rerun issues with CREATE DATABASE in a transaction.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.



I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Action 4.2.10 — Select northstar_digital, choose , open sql/postgres/source/01_create_digital_schema.sql, execute, and ins

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Select northstar_digital, choose , open sql/postgres/source/01_create_digital_schema.sql, execute, and ins. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.

3. Perform only this immediate task: Select northstar_digital, choose **Tools > Query Tool**, open sql/postgres/source/01_create_digital_schema.sql, execute, and inspect the Data Output/Messages tabs.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL psql, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 10****Action**

Select northstar_digital, choose **Tools > Query Tool**, open sql/postgres/source/01_create_digital_schema.sql, execute, and inspect the Data Output/Messages tabs.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Action 4.2.11 — Refresh . Right-click a table > for inspection only.

Application:

Windows File Explorer

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Refresh . Right-click a table > for inspection only. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Refresh **Schemas > digital > Tables**. Right-click a table > **View/Edit Data > First 100 Rows** for inspection only.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows File Explorer. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 11

Action

Refresh **Schemas > digital > Tables**. Right-click a table > **View/Edit Data > First 100 Rows** for inspection only.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Action 4.2.12 — Use psql as an alternate: connect explicitly to host, port, user, and database; run \dt digital.* and a cont

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Use psql as an alternate: connect explicitly to host, port, user, and database; run \dt digital.* and a cont. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Use psql as an alternate: connect explicitly to host, port, user, and database; run \dt digital.* and a context query.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL psql, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 12

Action

Use psql as an alternate: connect explicitly to host, port, user, and database; run `\dt digital.*` and a context query.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Action 4.2.13 — Never run the `90_development_reset.sql` files unless the safety guard is deliberately changed in a disposable

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Never run the 90_development_reset.sql files unless the safety guard is deliberately changed in a disposable. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Never run the 90_development_reset.sql files unless the safety guard is deliberately changed in a disposable environment and a backup/migration decision is recorded.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 13****Action**

Never run the 90_development_reset.sql files unless the safety guard is deliberately changed in a disposable environment and a backup/migration decision is recorded.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Action 4.2.C1 — Windows PowerShell

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Get-Service | Where-Object { $_.Name -like 'MSSQL*' -or $_.DisplayName -like 'SQL Server (*)' } | Format-Table Name, DisplayName, Status
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `Get-Service | Where-Object { $_.Name -like 'MSSQL*' -or $_.DisplayName -like 'SQL Server (*)' } | Format-Table Name, DisplayName, Status` — `Get-Service` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Where-Object` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `$_` is a variable or environment-value reference rather than literal output; `-like` is a command option or flag; its exact behavior is defined beside the command; `'MSSQL*'` is a quoted literal; the quotation marks preserve spaces or mark text; `-or` is a command option or flag; its exact behavior is defined beside the command; `$_` is a variable or environment-value reference rather than literal output; `-like` is a command option or flag; its exact behavior is defined beside the command; `'SQL Server (*)'` is a quoted literal; the quotation marks preserve spaces or mark text; `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Format-Table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Action 4.2.C2 — SSMS query window

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SSMS query window

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT @@SERVERNAME AS server_name, SUSER_SNAME() AS login_name, DB_NAME() AS database_name;
```

Where to enter it:

SSMS query window. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** SELECT @@SERVERNAME AS server_name, SUSER_SNAME() AS login_name, DB_NAME() AS database_name; — SELECT requests data without changing it unless combined with a modifying statement; SERVERNAME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; server_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUSER_SNAME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; login_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DB_NAME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Action 4.2.C3 — SSMS query window

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SSMS query window

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.

2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
-- Execute the entire opened 01_create_erp_database.sql file with the Execute button or F5.
```

Where to enter it:

SSMS query window. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** -- Execute the entire opened 01_create_erp_database.sql file with the Execute button or F5. is a comment for a human reader; it normally does not change data or execute business logic.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Action 4.2.C4 — SSMS query window

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SSMS query window

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
-- Execute the entire opened 02_create_erp_tables.sql file.
```

Where to enter it:

SSMS query window. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** -- Execute the entire opened 02_create_erp_tables.sql file. is a comment for a human reader; it normally does not change data or execute business logic.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Action 4.2.C5 — Windows PowerShell

Application:

pgAdmin 4 or PostgreSQL `psql`, as named in the action

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.

2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Get-Service *postgres* | Format-Table Name, Status, DisplayName
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** Get-Service *postgres* | Format-Table Name, Status, DisplayName — Get-Service is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; * is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; postgres is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; * is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Format-Table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Status is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DisplayName is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Action 4.2.C6 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Test-NetConnection localhost -Port 5432
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `Test-NetConnection localhost -Port 5432` — `Test-NetConnection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `localhost` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Port` is a command option or flag; its exact behavior is defined beside the command; `5432` is a numeric literal used by the command or expression

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Action 4.2.C7 — pgAdmin Query Tool

Application:

pgAdmin 4 or PostgreSQL `psql`, as named in the action

Window or panel:

pgAdmin Query Tool

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
-- Execute the entire opened 01_create_digital_schema.sql file using the Execute/Play button.
```

Where to enter it:

pgAdmin Query Tool. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** -- Execute the entire opened 01_create_digital_schema.sql file using the Execute/Play button. is a comment for a human reader; it normally does not change data or execute business logic.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Action 4.2.C8 — Windows PowerShell

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
psql -h localhost -p 5432 -U postgres -d northstar_digital
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `psql -h localhost -p 5432 -U postgres -d northstar_digital` — `psql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-h` is a command option or flag; its exact behavior is defined beside the command; `localhost` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-p` is a command option or flag; its exact behavior is defined beside the command; `5432` is a numeric literal used by the command or expression; `-U` is a command option or flag; its exact behavior is defined beside the command; `postgres` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-d` is a command option or flag; its exact behavior is defined beside the command; `northstar_digital` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 4.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 4 starting-screen checklist and re-verify the active context.

Lesson 4.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 4 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — docs/

Exact folder name: docs

Exact parent: repository root

Why it is needed now: Phase 4 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: docs.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 4.D1.1 — Create the folder docs

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\docs" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\docs" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\docs" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\docs" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/

Exact folder name: `sql`

Exact parent: `repository root`

Why it is needed now: Phase 4 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `repository root` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `sql`.
7. PRESS: Enter.

8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 4.D2.1 — Create the folder sql

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/postgres/

Exact folder name: postgres

Exact parent: sql

Why it is needed now: Phase 4 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path sql one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: postgres.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 4.D3.1 — Create the folder sql/postgres

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\postgres" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\postgres" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\postgres"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\postgres" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\postgres" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run Test-Path ".\sql\postgres" -PathType Container.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/sqlserver/

Exact folder name: sqlserver

Exact parent: sql

Why it is needed now: Phase 4 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path sql one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: sqlserver.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 4.D4.1 — Create the folder sql/sqlserver

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.

3. **CLICK:** Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. **TYPE THIS:** Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. **PRESS:** Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. **LOOK FOR:** Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. **DO NOT CONTINUE UNTIL:** Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\sqlserver" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\sqlserver" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\sqlserver"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\sqlserver" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\sqlserver" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\sqlserver" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/postgres/source/

Exact folder name: `source`

Exact parent: `sql/postgres`

Why it is needed now: Phase 4 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `sql/postgres` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `source`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method**Action 4.D5.1 — Create the folder sql/postgres/source**

Application:

pgAdmin 4 or PostgreSQL `psql`, as named in the action

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\postgres\source" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\postgres\source" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\sql\postgres\source` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\postgres\source" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\postgres\source" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\postgres\source" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/sqlserver/source/

Exact folder name: `source`

Exact parent: `sql/sqlserver`

Why it is needed now: Phase 4 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `sql/sqlserver` one level at a time.

4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: source.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 4.D6.1 — Create the folder sql/sqlserver/source

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\sqlserver\source" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\sql\sqlserver\source" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\sql\sqlserver\source" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\sqlserver\source" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\sqlserver\source" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\sqlserver\source" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — sql/sqlserver/source/01_create_erp_database.sql

Exact filename: 01_create_erp_database.sql

Filename extension: .sql. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: sql/sqlserver/source relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 4.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in sql/sqlserver/source one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 01_create_erp_database.sql.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 01_create_erp_database.sql and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 4.F1.1 — Create the file sql/sqlserver/source/01_create_erp_database.sql from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.

4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\sqlserver\source\01_create_erp_database.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\sqlserver\source\01_create_erp_database.sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\sqlserver\source\01_create_erp_database.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\sql\sqlserver\source\01_create_erp_database.sql" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/sqlserver/source/01_create_erp_database.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `sql/sqlserver/source`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `sql/sqlserver/source/01_create_erp_database.sql`.
3. In PowerShell at the repository root, run `Test-Path ".\sql\sqlserver\source\01_create_erp_database.sql"` and require True.
4. Run `git status --short -- "sql/sqlserver/source/01_create_erp_database.sql"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `sql/sqlserver/source/01_create_erp_database.sql`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: `sql/sqlserver/source/01_create_erp_database.sql`

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 4.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: SQL Server / T-SQL
-- Purpose: Create the local operational ERP database and schema safely.
-- Run in: SSMS query window connected to your local SQL Server instance.
-- Safety: This file creates missing objects. It does not drop existing data.

USE master;
GO

IF DB_ID(N'NorthstarRetail_ERP') IS NULL
BEGIN
    PRINT 'Creating database NorthstarRetail_ERP.';
    CREATE DATABASE NorthstarRetail_ERP;
END
ELSE
BEGIN
    PRINT 'Database NorthstarRetail_ERP already exists; leaving it in place.';
END;
GO
```

```

USE NorthstarRetail_ERP;
GO

IF SCHEMA_ID(N'erp') IS NULL
    EXEC(N'CREATE SCHEMA erp AUTHORIZATION dbo;');
GO

IF SCHEMA_ID(N'audit') IS NULL
    EXEC(N'CREATE SCHEMA audit AUTHORIZATION dbo;');
GO

IF OBJECT_ID(N'audit.source_load_batch', N'U') IS NULL
BEGIN
    CREATE TABLE audit.source_load_batch
    (
        batch_id          VARCHAR(80) NOT NULL PRIMARY KEY,
        process_name       VARCHAR(100) NOT NULL,
        started_at_utc     DATETIME2(3) NOT NULL CONSTRAINT DF_source_load_started DEFAULT SYSUTCDATETIME(),
        finished_at_utc    DATETIME2(3) NULL,
        status             VARCHAR(20) NOT NULL,
        source_file_count  INT          NULL,
        loaded_row_count   BIGINT       NULL,
        error_message      NVARCHAR(2000) NULL
    );
END;
GO

SELECT DB_NAME() AS current_database, SCHEMA_ID(N'erp') AS erp_schema_id;
GO

```

Line-by-line explanation

- **Line 1:** -- Dialect: SQL Server / T-SQL is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** -- Purpose: Create the local operational ERP database and schema safely. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** -- Run in: SSMS query window connected to your local SQL Server instance. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** -- Safety: This file creates missing objects. It does not drop existing data. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** USE master; — USE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; master is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** IF DB_ID(N'NorthstarRetail_ERP') IS NULL — IF begins a conditional decision; DB_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'NorthstarRetail_ERP' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** PRINT 'Creating database NorthstarRetail_ERP.'; — PRINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Creating database NorthstarRetail_ERP.' is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 12:** CREATE DATABASE NorthstarRetail_ERP; — CREATE defines a new database object; DATABASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NorthstarRetail_ERP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** END — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 14:** ELSE — ELSE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** PRINT 'Database NorthstarRetail_ERP already exists; leaving it in place.'; — PRINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Database NorthstarRetail_ERP already exists; leaving it in place.' is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 17:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** G0 — G0 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** blank line used only to separate logical sections for readability.
- **Line 20:** USE NorthstarRetail_ERP; — USE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NorthstarRetail_ERP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** G0 — G0 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** blank line used only to separate logical sections for readability.
- **Line 23:** IF SCHEMA_ID(N'erp') IS NULL — IF begins a conditional decision; SCHEMA_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'erp' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** EXEC(N'CREATE SCHEMA erp AUTHORIZATION dbo; '); — EXEC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'CREATE SCHEMA erp AUTHORIZATION dbo;' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** G0 — G0 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** blank line used only to separate logical sections for readability.
- **Line 27:** IF SCHEMA_ID(N'audit') IS NULL — IF begins a conditional decision; SCHEMA_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'audit' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** EXEC(N'CREATE SCHEMA audit AUTHORIZATION dbo; '); — EXEC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'CREATE SCHEMA audit AUTHORIZATION dbo;' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** G0 — G0 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 30:** blank line used only to separate logical sections for readability.
- **Line 31:** IF OBJECT_ID(N'audit.source_load_batch', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'audit.source_load_batch' is a quoted literal; the quotation marks preserve spaces or

mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 32:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** CREATE TABLE audit.source_load_batch — CREATE defines a new database object; TABLE the database object that stores rows and columns; audit.source_load_batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** batch_id VARCHAR(80) NOT NULL PRIMARY KEY, — batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 80 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 36:** process_name VARCHAR(100) NOT NULL, — process_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 37:** started_at_utc DATETIME2(3) NOT NULL CONSTRAINT DF_source_load_started DEFAULT SYSUTCDATETIME(), — started_at_utc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DF_source_load_started is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 38:** finished_at_utc DATETIME2(3) NULL, — finished_at_utc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 39:** status VARCHAR(20) NOT NULL, — status shows tracked, staged, modified, and untracked work; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 20 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 40:** `source_file_count INT NULL`, — `source_file_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** `loaded_row_count BIGINT NULL`, — `loaded_row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** `error_message NVARCHAR(2000) NULL` — `error_message` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NVARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2000` is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 43:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 44:** `END`; — `END` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 45:** `G0` — `G0` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** blank line used only to separate logical sections for readability.
- **Line 47:** `SELECT DB_NAME() AS current_database, SCHEMA_ID(N'erp') AS erp_schema_id`; — `SELECT` requests data without changing it unless combined with a modifying statement; `DB_NAME` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `current_database` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCHEMA_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'erp'` is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 48:** `G0` — `G0` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 4 lesson index](#)

File introduction — `sql/sqlserver/source/02_create_erp_tables.sql`

Exact filename: `02_create_erp_tables.sql`

Filename extension: `.sql`. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `sql/sqlserver/source` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 4.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in sql/sqlserver/source one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 02_create_erp_tables.sql.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 02_create_erp_tables.sql and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 4.F2.1 — Create the file sql/sqlserver/source/02_create_erp_tables.sql from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\sqlserver\source\02_create_erp_tables.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\sqlserver\source\02_create_erp_tables.sql" -ErrorAction Stop`
— `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\sqlserver\source\02_create_erp_tables.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\sql\sqlserver\source\02_create_erp_tables.sql" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add -Force, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/sqlserver/source/02_create_erp_tables.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `sql/sqlserver/source`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `sql/sqlserver/source/02_create_erp_tables.sql`.
3. In PowerShell at the repository root, run `Test-Path ".\sql\sqlserver\source\02_create_erp_tables.sql"` and require True.
4. Run `git status --short -- "sql/sqlserver/source/02_create_erp_tables.sql"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `sql/sqlserver/source/02_create_erp_tables.sql`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: `sql/sqlserver/source/02_create_erp_tables.sql`

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 4.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: SQL Server / T-SQL
-- Purpose: Create the normalized ERP source tables.
-- Run in: SSMS query window connected to NorthstarRetail_ERP.
-- Idempotency: each table and index is created only when missing.

USE NorthstarRetail_ERP;
GO

IF OBJECT_ID(N'erp.product_categories', N'U') IS NULL
BEGIN
    CREATE TABLE erp.product_categories
    (
        category_id          INT NOT NULL CONSTRAINT PK_product_categories PRIMARY KEY,
        category_name        NVARCHAR(100) NOT NULL,
        parent_category_id   INT NULL,
        created_at            DATETIME2(3) NOT NULL,
        updated_at            DATETIME2(3) NOT NULL,
        CONSTRAINT FK_product_categories_parent FOREIGN KEY (parent_category_id)
            REFERENCES erp.product_categories(category_id)
    );
END;
GO

IF OBJECT_ID(N'erp.suppliers', N'U') IS NULL
BEGIN
    CREATE TABLE erp.suppliers
    (
        supplier_id          INT NOT NULL CONSTRAINT PK_suppliers PRIMARY KEY,
        supplier_code         VARCHAR(20) NOT NULL,
        supplier_name         NVARCHAR(150) NOT NULL,
        country_code          CHAR(2) NOT NULL,
        lead_time_days         INT NOT NULL,
        active_flag           BIT NOT NULL,
        created_at            DATETIME2(3) NOT NULL,
        updated_at            DATETIME2(3) NOT NULL,
    );
END;
GO
```

```

        CONSTRAINT UQ_suppliers_supplier_code UNIQUE (supplier_code),
        CONSTRAINT CK_suppliers_lead_time CHECK (lead_time_days >= 0)
    );
END;
GO

IF OBJECT_ID(N'erp.products', N'U') IS NULL
BEGIN
    CREATE TABLE erp.products
    (
        product_id          INT NOT NULL CONSTRAINT PK_products PRIMARY KEY,
        sku                 VARCHAR(30) NOT NULL,
        product_name        NVARCHAR(200) NOT NULL,
        category_id         INT NOT NULL,
        supplier_id         INT NOT NULL,
        brand               NVARCHAR(100) NOT NULL,
        unit_cost            DECIMAL(12,2) NOT NULL,
        list_price           DECIMAL(12,2) NOT NULL,
        currency_code       CHAR(3) NOT NULL,
        active_flag         BIT NOT NULL,
        created_at           DATETIME2(3) NOT NULL,
        updated_at          DATETIME2(3) NOT NULL,
        CONSTRAINT UQ_products_sku UNIQUE (sku),
        CONSTRAINT FK_products_category FOREIGN KEY (category_id) REFERENCES erp.product_categories(category_id),
        CONSTRAINT FK_products_supplier FOREIGN KEY (supplier_id) REFERENCES erp.suppliers(supplier_id),
        CONSTRAINT CK_products_amounts CHECK (unit_cost >= 0 AND list_price >= 0)
    );
END;
GO

IF OBJECT_ID(N'erp.stores', N'U') IS NULL
BEGIN
    CREATE TABLE erp.stores
    (
        store_id            INT NOT NULL CONSTRAINT PK_stores PRIMARY KEY,
        store_code          VARCHAR(20) NOT NULL,
        store_name          NVARCHAR(150) NOT NULL,
        region              VARCHAR(30) NOT NULL,
        state_code          CHAR(2) NOT NULL,
        city                NVARCHAR(100) NOT NULL,
        open_date           DATE NOT NULL,
        active_flag         BIT NOT NULL,
        created_at          DATETIME2(3) NOT NULL,
        updated_at          DATETIME2(3) NOT NULL,
        CONSTRAINT UQ_stores_store_code UNIQUE (store_code)
    );
END;
GO

IF OBJECT_ID(N'erp.employees', N'U') IS NULL
BEGIN
    CREATE TABLE erp.employees
    (
        employee_id         INT NOT NULL CONSTRAINT PK_employees PRIMARY KEY,
        employee_number     VARCHAR(20) NOT NULL,
        store_id            INT NULL,
        manager_employee_id INT NULL,
        first_name          NVARCHAR(80) NOT NULL,
        last_name           NVARCHAR(80) NOT NULL,
        job_title           NVARCHAR(100) NOT NULL,
        hire_date           DATE NOT NULL,
        active_flag         BIT NOT NULL,
        created_at          DATETIME2(3) NOT NULL,
        updated_at          DATETIME2(3) NOT NULL,
        CONSTRAINT UQ_employees_number UNIQUE (employee_number),
        CONSTRAINT FK_employees_store FOREIGN KEY (store_id) REFERENCES erp.stores(store_id),
        CONSTRAINT FK_employees_manager FOREIGN KEY (manager_employee_id) REFERENCES erp.employees(employee_id)
    );
END;
GO

IF OBJECT_ID(N'erp.customers', N'U') IS NULL
BEGIN
    CREATE TABLE erp.customers
    (
        customer_id         INT NOT NULL CONSTRAINT PK_customers PRIMARY KEY,
        customer_number     VARCHAR(25) NOT NULL,
        first_name          NVARCHAR(80) NOT NULL,
        last_name           NVARCHAR(80) NOT NULL,
        email               NVARCHAR(255) NULL,
        phone               VARCHAR(40) NULL,
        signup_date         DATE NOT NULL,
        loyalty_tier         VARCHAR(20) NOT NULL,
        marketing_opt_in     BIT NOT NULL,
        created_at          DATETIME2(3) NOT NULL,
        updated_at          DATETIME2(3) NOT NULL
        -- Deliberately no unique constraint on customer_number in the learning source:
    );
END;
GO

```

```

    -- controlled duplicate business keys must reach Silver and be handled there.
    );
END;
GO

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE object_id = OBJECT_ID(N'erp.customers') AND name = N'IX_customers_customer_number')
    CREATE INDEX IX_customers_customer_number ON erp.customers(customer_number);
GO

IF OBJECT_ID(N'erp.addresses', N'U') IS NULL
BEGIN
    CREATE TABLE erp.addresses
    (
        address_id          INT NOT NULL CONSTRAINT PK_addresses PRIMARY KEY,
        customer_id         INT NOT NULL,
        address_type        VARCHAR(20) NOT NULL,
        line1               NVARCHAR(200) NOT NULL,
        line2               NVARCHAR(200) NULL,
        city                NVARCHAR(100) NOT NULL,
        state_code          VARCHAR(10) NOT NULL,
        postal_code         VARCHAR(20) NOT NULL,
        country_code        VARCHAR(10) NOT NULL,
        is_default           BIT NOT NULL,
        created_at          DATETIME2(3) NOT NULL,
        updated_at          DATETIME2(3) NOT NULL,
        CONSTRAINT FK_addresses_customer FOREIGN KEY (customer_id) REFERENCES erp.customers(customer_id)
    );
END;
GO

IF OBJECT_ID(N'erp.orders', N'U') IS NULL
BEGIN
    CREATE TABLE erp.orders
    (
        order_id            BIGINT NOT NULL CONSTRAINT PK_orders PRIMARY KEY,
        order_number        VARCHAR(30) NOT NULL,
        customer_id         INT NOT NULL,
        store_id            INT NULL,
        employee_id         INT NULL,
        channel_code        VARCHAR(20) NOT NULL,
        order_timestamp     DATETIME2(3) NOT NULL,
        order_status        VARCHAR(20) NOT NULL,
        promotion_code      VARCHAR(30) NULL,
        currency_code       CHAR(3) NOT NULL,
        created_at          DATETIME2(3) NOT NULL,
        updated_at          DATETIME2(3) NOT NULL,
        CONSTRAINT UQ_orders_order_number UNIQUE (order_number),
        CONSTRAINT FK_orders_customer FOREIGN KEY (customer_id) REFERENCES erp.customers(customer_id),
        CONSTRAINT FK_orders_store FOREIGN KEY (store_id) REFERENCES erp.stores(store_id),
        CONSTRAINT FK_orders_employee FOREIGN KEY (employee_id) REFERENCES erp.employees(employee_id),
        CONSTRAINT CK_orders_channel CHECK (channel_code IN ('STORE', 'ONLINE'))
    );
END;
GO

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE object_id = OBJECT_ID(N'erp.orders') AND name = N'IX_orders_updated_at')
    CREATE INDEX IX_orders_updated_at ON erp.orders(updated_at) INCLUDE (order_id, customer_id, order_timestamp);
GO

IF OBJECT_ID(N'erp.order_items', N'U') IS NULL
BEGIN
    CREATE TABLE erp.order_items
    (
        order_item_id      BIGINT NOT NULL CONSTRAINT PK_order_items PRIMARY KEY,
        order_id           BIGINT NOT NULL,
        line_number        INT NOT NULL,
        product_id         INT NOT NULL,
        quantity           INT NOT NULL,
        unit_price         DECIMAL(12,2) NOT NULL,
        unit_cost          DECIMAL(12,2) NOT NULL,
        discount_amount    DECIMAL(12,2) NOT NULL,
        tax_amount         DECIMAL(12,2) NOT NULL,
        created_at         DATETIME2(3) NOT NULL,
        updated_at         DATETIME2(3) NOT NULL,
        CONSTRAINT UQ_order_items_line UNIQUE (order_id, line_number),
        CONSTRAINT FK_order_items_order FOREIGN KEY (order_id) REFERENCES erp.orders(order_id),
        CONSTRAINT FK_order_items_product FOREIGN KEY (product_id) REFERENCES erp.products(product_id)
        -- No positive-quantity check: controlled impossible quantities are a Silver lesson.
    );
END;
GO

IF OBJECT_ID(N'erp.payments', N'U') IS NULL
BEGIN
    CREATE TABLE erp.payments
    (
        payment_id         BIGINT NOT NULL CONSTRAINT PK_payments PRIMARY KEY,

```

```

        order_id            BIGINT NOT NULL,
        payment_timestamp   DATETIME2(3) NOT NULL,
        payment_method      VARCHAR(30) NOT NULL,
        payment_status      VARCHAR(30) NOT NULL,
        amount              DECIMAL(14,2) NOT NULL,
        currency_code       VARCHAR(3) NOT NULL,
        transaction_reference VARCHAR(80) NOT NULL,
        created_at          DATETIME2(3) NOT NULL,
        updated_at          DATETIME2(3) NOT NULL,
        CONSTRAINT UQ_payments_transaction UNIQUE (transaction_reference),
        CONSTRAINT FK_payments_order FOREIGN KEY (order_id) REFERENCES erp.orders(order_id)
    );
END;
GO

IF OBJECT_ID(N'erp.shipments', N'U') IS NULL
BEGIN
    CREATE TABLE erp.shipments
    (
        shipment_id         BIGINT NOT NULL CONSTRAINT PK_shipments PRIMARY KEY,
        order_id            BIGINT NOT NULL,
        carrier              NVARCHAR(100) NOT NULL,
        tracking_number      VARCHAR(100) NOT NULL,
        ship_date            DATE NOT NULL,
        promised_delivery_date DATE NOT NULL,
        actual_delivery_date DATE NULL,
        shipment_status     VARCHAR(30) NOT NULL,
        shipping_cost        DECIMAL(12,2) NOT NULL,
        created_at          DATETIME2(3) NOT NULL,
        updated_at          DATETIME2(3) NOT NULL,
        CONSTRAINT UQ_shipments_tracking UNIQUE (tracking_number),
        CONSTRAINT FK_shipments_order FOREIGN KEY (order_id) REFERENCES erp.orders(order_id)
    );
END;
GO

IF OBJECT_ID(N'erp.inventory_transactions', N'U') IS NULL
BEGIN
    CREATE TABLE erp.inventory_transactions
    (
        inventory_transaction_id BIGINT NOT NULL CONSTRAINT PK_inventory_transactions PRIMARY KEY,
        store_id                 INT NOT NULL,
        product_id               INT NOT NULL,
        transaction_timestamp    DATETIME2(3) NOT NULL,
        transaction_type         VARCHAR(30) NOT NULL,
        quantity_change          INT NOT NULL,
        reference_type           VARCHAR(30) NULL,
        reference_id             BIGINT NULL,
        created_at              DATETIME2(3) NOT NULL,
        CONSTRAINT FK_inventory_transactions_store FOREIGN KEY (store_id) REFERENCES erp.stores(store_id),
        CONSTRAINT FK_inventory_transactions_product FOREIGN KEY (product_id) REFERENCES erp.products(product_id)
    );
END;
GO

IF OBJECT_ID(N'erp.inventory_snapshots', N'U') IS NULL
BEGIN
    CREATE TABLE erp.inventory_snapshots
    (
        inventory_snapshot_id BIGINT NOT NULL CONSTRAINT PK_inventory_snapshots PRIMARY KEY,
        snapshot_date         DATE NOT NULL,
        store_id              INT NOT NULL,
        product_id            INT NOT NULL,
        quantity_on_hand      INT NOT NULL,
        quantity_reserved     INT NOT NULL,
        reorder_point         INT NOT NULL,
        created_at            DATETIME2(3) NOT NULL,
        CONSTRAINT UQ_inventory_snapshot_grain UNIQUE (snapshot_date, store_id, product_id),
        CONSTRAINT FK_inventory_snapshots_store FOREIGN KEY (store_id) REFERENCES erp.stores(store_id),
        CONSTRAINT FK_inventory_snapshots_product FOREIGN KEY (product_id) REFERENCES erp.products(product_id)
    );
END;
GO

SELECT s.name AS schema_name, t.name AS table_name
FROM sys.tables AS t
JOIN sys.schemas AS s ON s.schema_id = t.schema_id
WHERE s.name IN ('erp', 'audit')
ORDER BY s.name, t.name;
GO

```

Line-by-line explanation

- **Line 1:** -- Dialect: SQL Server / T-SQL is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 2:** -- Purpose: Create the normalized ERP source tables. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** -- Run in: SSMS query window connected to NorthstarRetail_ERP. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** -- Idempotency: each table and index is created only when missing. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** USE NorthstarRetail_ERP; — USE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NorthstarRetail_ERP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** IF OBJECT_ID(N'erp.product_categories', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'erp.product_categories' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** CREATE TABLE erp.product_categories — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.product_categories is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** category_id INT NOT NULL CONSTRAINT PK_product_categories PRIMARY KEY, — category_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PK_product_categories is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** category_name NVARCHAR(100) NOT NULL, — category_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NVARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** parent_category_id INT NULL, — parent_category_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** created_at DATETIME2(3) NOT NULL, — created_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used

by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 17:** updated_at DATETIME2(3) NOT NULL, — updated_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** CONSTRAINT FK_product_categories_parent FOREIGN KEY (parent_category_id) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FK_product_categories_parent is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FOREIGN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; parent_category_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** REFERENCES erp.product_categories(category_id) — REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.product_categories is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; category_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** blank line used only to separate logical sections for readability.
- **Line 24:** IF OBJECT_ID(N'erp.suppliers', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'erp.suppliers' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** CREATE TABLE erp.suppliers — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.suppliers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** supplier_id INT NOT NULL CONSTRAINT PK_suppliers PRIMARY KEY, — supplier_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PK_suppliers is a name, keyword, argument, identifier,

or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; supplier_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 37:** CONSTRAINT CK_suppliers_lead_time CHECK (lead_time_days >= 0) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CK_suppliers_lead_time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CHECK is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; lead_time_days is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; >= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 38:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 39:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 40:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** blank line used only to separate logical sections for readability.
- **Line 42:** IF OBJECT_ID(N'erp.products', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'erp.products' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 43:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 44:** CREATE TABLE erp.products — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.products is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 45:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** product_id INT NOT NULL CONSTRAINT PK_products PRIMARY KEY, — product_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PK_products is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 47:** sku VARCHAR(30) NOT NULL, — sku is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 48:** product_name NVARCHAR(200) NOT NULL, — product_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NVARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 200 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Line 58: CONSTRAINT UQ_products_sku UNIQUE (sku), — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UQ_products_sku is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UNIQUE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sku is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Line 59: CONSTRAINT FK_products_category FOREIGN KEY (category_id) REFERENCES erp.product_categories(category_id), — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FK_products_category is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FOREIGN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; category_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.product_categories is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; category_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Line 60: CONSTRAINT FK_products_supplier FOREIGN KEY (supplier_id) REFERENCES erp.suppliers(supplier_id),
— CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the
surrounding explanation; FK_products_supplier is a name, keyword, argument, identifier, or literal whose project-
specific role is described in the surrounding explanation; FOREIGN is a name, keyword, argument, identifier, or literal
whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier,
or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument,
identifier, or literal whose project-specific role is described in the surrounding explanation; supplier_id is a name,
keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a
name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;
REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the
surrounding explanation; erp.suppliers is a name, keyword, argument, identifier, or literal whose project-specific role
is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-
specific role is described in the surrounding explanation; supplier_id is a name, keyword, argument, identifier, or lit-
eral whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identi-
fier, or literal whose project-specific role is described in the surrounding explanation

Line 61: CONSTRAINT CK_products_amounts CHECK (unit_cost >= 0 AND list_price >= 0) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CK_products_amounts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CHECK is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; unit_cost is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; >= is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; AND is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; list_price is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; >= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 62:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 63:** END ; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 64:** G0 — G0 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 65:** blank line used only to separate logical sections for readability.
- **Line 66:** IF OBJECT_ID(N'erp.stores' , 'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'erp.stores' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 67:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 68:** CREATE TABLE erp.stores — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.stores is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 69:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 70:** store_id INT NOT NULL CONSTRAINT PK_stores PRIMARY KEY, — store_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PK_stores is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 71:** store_code VARCHAR(20) NOT NULL, — store_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 20 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 72:** store_name NVARCHAR(150) NOT NULL, — store_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NVARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 150 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 73:** region VARCHAR(30) NOT NULL, — region is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 82:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 83:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 84:** blank line used only to separate logical sections for readability.
- **Line 85:** IF OBJECT_ID('erp.employees', 'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'erp.employees' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 86:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 87:** CREATE TABLE erp.employees — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.employees is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 88:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 89:** employee_id INT NOT NULL CONSTRAINT PK_employees PRIMARY KEY, — employee_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PK_employees is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 90:** employee_number VARCHAR(20) NOT NULL, — employee_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 20 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 91:** store_id INT NULL, — store_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 92:** manager_employee_id INT NULL, — manager_employee_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 93:** first_name NVARCHAR(80) NOT NULL, — first_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NVARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 80 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 100:** CONSTRAINT UQ_employees_number UNIQUE (employee_number), — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UQ_employees_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UNIQUE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; employee_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `erp.stores` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `store_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 102:** `CONSTRAINT FK_employees_manager FOREIGN KEY (manager_employee_id) REFERENCES erp.employees(employee_id)` — `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FK_employees_manager` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FOREIGN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manager_employee_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `REFERENCES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `erp.employees` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `employee_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 103:** `); —` `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 104:** `END; —` `END` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 105:** `G0 —` `G0` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 106:** blank line used only to separate logical sections for readability.
- **Line 107:** `IF OBJECT_ID(N'erp.customers', N'U') IS NULL` — `IF` begins a conditional decision; `OBJECT_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'erp.customers'` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'U'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 108:** `BEGIN` — `BEGIN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 109:** `CREATE TABLE erp.customers` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `erp.customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 110:** `( —` `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 111:** `customer_id INT NOT NULL CONSTRAINT PK_customers PRIMARY KEY,` — `customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PK_customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 112:** `customer_number VARCHAR(25) NOT NULL,` — `customer_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; 25 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 121:** updated_at DATETIME2(3) NOT NULL — updated_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 122:** -- Deliberately no unique constraint on customer_number in the learning source: is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 123:** -- controlled duplicate business keys must reach Silver and be handled there. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 124:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 125:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 126:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 127:** blank line used only to separate logical sections for readability.
- **Line 128:** IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE object_id = OBJECT_ID(N'erp.customers') AND name = N'IX_customers_customer_number') — IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 1 is a numeric literal used by the command or expression; FROM identifies the source table or query; sys.indexes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; object_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 129:** CREATE INDEX IX_customers_customer_number ON erp.customers(customer_number); — CREATE defines a new database object; INDEX is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IX_customers_customer_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 130:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 131:** blank line used only to separate logical sections for readability.
- **Line 132:** IF OBJECT_ID(N'erp.addresses', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'erp.addresses' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 150:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 151:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 152:** blank line used only to separate logical sections for readability.
- **Line 153:** IF OBJECT_ID(N'erp.orders', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'erp.orders' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 154:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 155:** CREATE TABLE `erp.orders` — CREATE defines a new database object; TABLE the database object that stores rows and columns; `erp.orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 156:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 157:** `order_id` BIGINT NOT NULL CONSTRAINT PK_orders PRIMARY KEY, — `order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BIGINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PK_orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 158:** `order_number` VARCHAR(30) NOT NULL, — `order_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 159:** `customer_id` INT NOT NULL, — `customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 160:** `store_id` INT NULL, — `store_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 161:** `employee_id` INT NULL, — `employee_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 162:** `channel_code` VARCHAR(20) NOT NULL, — `channel_code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

fier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 20 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 174:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 175:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 176:** G0 — G0 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 177:** blank line used only to separate logical sections for readability.
- **Line 178:** IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE object_id = OBJECT_ID(N'erp.orders') AND name = N'IX_orders_updated_at') — IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 1 is a numeric literal used by the command or expression; FROM identifies the source table or query; sys.indexes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; object_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 179:** CREATE INDEX IX_orders_updated_at ON erp.orders(updated_at) INCLUDE (order_id, customer_id, order_timestamp); — CREATE defines a new database object; INDEX is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IX_orders_updated_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; updated_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INCLUDE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 180:** G0 — G0 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 181:** blank line used only to separate logical sections for readability.
- **Line 182:** IF OBJECT_ID(N'erp.order_items', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'erp.order_items' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 183:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 184:** CREATE TABLE erp.order_items — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.order_items is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 185:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 186:** order_item_id BIGINT NOT NULL CONSTRAINT PK_order_items PRIMARY KEY, — order_item_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BIGINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; PK_order_items is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 200:** -- No positive-quantity check: controlled impossible quantities are a Silver lesson. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 201:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 202:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 203:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 204:** blank line used only to separate logical sections for readability.
- **Line 205:** IF OBJECT_ID(N'erp.payments', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'erp.payments' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 206:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 207:** CREATE TABLE erp.payments — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.payments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 208:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 209:** payment_id BIGINT NOT NULL CONSTRAINT PK_payments PRIMARY KEY, — payment_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BIGINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PK_payments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 210:** order_id BIGINT NOT NULL, — order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BIGINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 211:** payment_timestamp DATETIME2(3) NOT NULL, — payment_timestamp is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 212:** payment_method VARCHAR(30) NOT NULL, — payment_method is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal

used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 220:** CONSTRAINT FK_payments_order FOREIGN KEY (order_id) REFERENCES erp.orders(order_id) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FK_payments_order is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FOREIGN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 221:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 222:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 223:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 224:** blank line used only to separate logical sections for readability.
- **Line 225:** IF OBJECT_ID(N'erp.shipments', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'erp.shipments' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 226:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 227:** CREATE TABLE erp.shipments — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 228:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 229:** shipment_id BIGINT NOT NULL CONSTRAINT PK_shipments PRIMARY KEY, — shipment_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BIGINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PK_shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 230:** order_id BIGINT NOT NULL, — order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BIGINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

- **Line 239:** updated_at DATETIME2(3) NOT NULL, — updated_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 240:** CONSTRAINT UQ_shipments_tracking UNIQUE (tracking_number), — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UQ_shipments_tracking is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UNIQUE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tracking_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 241:** CONSTRAINT FK_shipments_order FOREIGN KEY (order_id) REFERENCES erp.orders(order_id) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FK_shipments_order is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FOREIGN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 242:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 243:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 244:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 245:** blank line used only to separate logical sections for readability.
- **Line 246:** IF OBJECT_ID(N'erp.inventory_transactions', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'erp.inventory_transactions' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 247:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 248:** CREATE TABLE erp.inventory_transactions — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.inventory_transactions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 249:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 250:** inventory_transaction_id BIGINT NOT NULL CONSTRAINT PK_inventory_transactions PRIMARY KEY, — inventory_transaction_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in

the surrounding explanation; BIGINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PK_inventory_transactions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 259:** CONSTRAINT FK_inventory_transactions_store FOREIGN KEY (store_id) REFERENCES erp.stores(store_id), — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FK_inventory_transactions_store is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FOREIGN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; store_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.stores is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; store_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 260:** CONSTRAINT FK_inventory_transactions_product FOREIGN KEY (product_id) REFERENCES erp.products(product_id) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FK_inventory_transactions_product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FOREIGN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.products is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 261:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 262:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 263:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 264:** blank line used only to separate logical sections for readability.
- **Line 265:** IF OBJECT_ID(N'erp.inventory_snapshots', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'erp.inventory_snapshots' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 266:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 267:** CREATE TABLE erp.inventory_snapshots — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.inventory_snapshots is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 268:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; store_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 278:** CONSTRAINT FK_inventory_snapshots_store FOREIGN KEY (store_id) REFERENCES erp.stores(store_id), — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FK_inventory_snapshots_store is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FOREIGN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; store_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.stores is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; store_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 279:** CONSTRAINT FK_inventory_snapshots_product FOREIGN KEY (product_id) REFERENCES erp.products(product_id) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FK_inventory_snapshots_product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FOREIGN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.products is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 280:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 281:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 282:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 283:** blank line used only to separate logical sections for readability.
- **Line 284:** SELECT s.name AS schema_name, t.name AS table_name — SELECT requests data without changing it unless combined with a modifying statement; s.name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; schema_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; t.name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 285:** FROM sys.tables AS t — FROM identifies the source table or query; sys.tables is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; t is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 286:** JOIN sys.schemas AS s ON s.schema_id = t.schema_id — JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sys.schemas is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s.schema_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; t.schema_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 287:** WHERE s.name IN ('erp', 'audit') — WHERE filters rows to those meeting a condition; s.name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'erp' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'audit' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 288:** ORDER BY s.name, t.name; — ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s.name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; t.name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 289:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 4 lesson index](#)

File introduction — sql/postgres/source/00_create_database.sql

Exact filename: 00_create_database.sql

Filename extension: .sql. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: sql/postgres/source relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 4.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in sql/postgres/source one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 00_create_database.sql.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 00_create_database.sql and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 4.F3.1 — Create the file sql/postgres/source/00_create_database.sql from PowerShell

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\postgres\source\00_create_database.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\postgres\source\00_create_database.sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\postgres\source\00_create_database.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\sql\postgres\source\00_create_database.sql" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/postgres/source/00_create_database.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `sql/postgres/source`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `sql/postgres/source/00_create_database.sql`.
3. In PowerShell at the repository root, run `Test-Path ".\sql\postgres\source\00_create_database.sql"` and require True.
4. Run `git status --short -- "sql/postgres/source/00_create_database.sql"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `sql/postgres/source/00_create_database.sql`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: `sql/postgres/source/00_create_database.sql`

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 4.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: PostgreSQL SQL
-- Run in: pgAdmin Query Tool while connected to the default 'postgres' database.
-- PostgreSQL cannot run CREATE DATABASE inside a transaction block.
-- Execute this statement once. If the database already exists, do not run it again.
CREATE DATABASE northstar_digital;
```

Line-by-line explanation

- **Line 1:** `-- Dialect: PostgreSQL SQL` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `-- Run in: pgAdmin Query Tool while connected to the default 'postgres' database.` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 3:** -- PostgreSQL cannot run CREATE DATABASE inside a transaction block. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** -- Execute this statement once. If the database already exists, do not run it again. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** CREATE DATABASE northstar_digital; — CREATE defines a new database object; DATABASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; northstar_digital is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 4 lesson index](#)

File introduction — sql/postgres/source/01_create_digital_schema.sql

Exact filename: 01_create_digital_schema.sql

Filename extension: .sql. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: sql/postgres/source relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 4.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in sql/postgres/source one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 01_create_digital_schema.sql.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 01_create_digital_schema.sql and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 4.F4.1 — Create the file sql/postgres/source/01_create_digital_schema.sql from PowerShell

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\postgres\source\01_create_digital_schema.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\postgres\source\01_create_digital_schema.sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\postgres\source\01_create_digital_schema.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\sql\postgres\source\01_create_digital_schema.sql" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/postgres/source/01_create_digital_schema.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `sql/postgres/source`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `sql/postgres/source/01_create_digital_schema.sql`.
3. In PowerShell at the repository root, run `Test-Path ".\sql\postgres\source\01_create_digital_schema.sql"` and require True.
4. Run `git status --short -- "sql/postgres/source/01_create_digital_schema.sql"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `sql/postgres/source/01_create_digital_schema.sql`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: `sql/postgres/source/01_create_digital_schema.sql`

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 4.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: PostgreSQL SQL
-- Purpose: create the digital source schema and tables.
```

```

-- Run in: pgAdmin Query Tool connected to database northstar_digital.
-- Idempotency: CREATE IF NOT EXISTS avoids replacing existing objects.

CREATE SCHEMA IF NOT EXISTS digital;
CREATE SCHEMA IF NOT EXISTS audit;

CREATE TABLE IF NOT EXISTS audit.source_load_batch
(
    batch_id          varchar(80) PRIMARY KEY,
    process_name      varchar(100) NOT NULL,
    started_at_utc    timestampz NOT NULL DEFAULT now(),
    finished_at_utc   timestampz NULL,
    status            varchar(20) NOT NULL,
    source_file_count integer NULL,
    loaded_row_count  bigint NULL,
    error_message     text NULL
);

CREATE TABLE IF NOT EXISTS digital.web_users
(
    web_user_id       bigint PRIMARY KEY,
    customer_id       integer NULL,
    anonymous_cookie_id varchar(64) NOT NULL,
    email_hash        char(64) NULL,
    created_at        timestampz NOT NULL,
    updated_at        timestampz NOT NULL,
    CONSTRAINT uq_web_users_cookie UNIQUE (anonymous_cookie_id)
);

CREATE TABLE IF NOT EXISTS digital.campaigns
(
    campaign_id       integer PRIMARY KEY,
    campaign_code     varchar(30) NOT NULL UNIQUE,
    campaign_name     varchar(200) NOT NULL,
    channel           varchar(40) NOT NULL,
    start_date        date NOT NULL,
    end_date          date NOT NULL,
    budget_amount     numeric(14,2) NOT NULL,
    currency_code     varchar(3) NOT NULL,
    created_at        timestampz NOT NULL,
    updated_at        timestampz NOT NULL,
    CONSTRAINT ck_campaign_dates CHECK (end_date >= start_date)
);

CREATE TABLE IF NOT EXISTS digital.web_sessions
(
    session_id        bigint PRIMARY KEY,
    web_user_id       bigint NULL REFERENCES digital.web_users(web_user_id),
    session_start_utc timestampz NOT NULL,
    session_end_utc   timestampz NULL,
    source_channel     varchar(40) NOT NULL,
    campaign_id       integer NULL REFERENCES digital.campaigns(campaign_id),
    device_type       varchar(30) NOT NULL,
    country_code      varchar(10) NOT NULL,
    converted_order_id bigint NULL,
    created_at        timestampz NOT NULL,
    CONSTRAINT ck_session_end CHECK (session_end_utc IS NULL OR session_end_utc >= session_start_utc)
);

CREATE TABLE IF NOT EXISTS digital.web_events
(
    event_id          bigint PRIMARY KEY,
    session_id        bigint NOT NULL REFERENCES digital.web_sessions(session_id),
    event_timestamp   timestampz NOT NULL,
    event_type        varchar(50) NOT NULL,
    page_url          text NULL,
    product_id        integer NULL,
    order_id          bigint NULL,
    event_properties  jsonb NULL,
    created_at        timestampz NOT NULL
);

CREATE TABLE IF NOT EXISTS digital.campaign_touchpoints
(
    touchpoint_id     bigint PRIMARY KEY,
    web_user_id       bigint NULL REFERENCES digital.web_users(web_user_id),
    session_id        bigint NULL REFERENCES digital.web_sessions(session_id),
    campaign_id       integer NOT NULL REFERENCES digital.campaigns(campaign_id),
    touchpoint_timestamp timestampz NOT NULL,
    touchpoint_type   varchar(40) NOT NULL,
    attribution_weight numeric(8,6) NOT NULL,
    created_at        timestampz NOT NULL,
    CONSTRAINT ck_touchpoint_weight CHECK (attribution_weight >= 0 AND attribution_weight <= 1)
);

CREATE TABLE IF NOT EXISTS digital.marketing_spend
(

```

```

    spend_id          bigint PRIMARY KEY,
    spend_date        date NOT NULL,
    campaign_id       integer NOT NULL REFERENCES digital.campaigns(campaign_id),
    channel           varchar(40) NOT NULL,
    spend_amount       numeric(14,2) NOT NULL,
    impressions        bigint NOT NULL,
    clicks            bigint NOT NULL,
    currency_code      varchar(3) NOT NULL,
    created_at         timestampz NOT NULL,
    CONSTRAINT uq_marketing_spend_grain UNIQUE (spend_date, campaign_id, channel),
    CONSTRAINT ck_spend_nonnegative CHECK (spend_amount >= 0 AND impressions >= 0 AND clicks >= 0)
);

CREATE INDEX IF NOT EXISTS ix_web_sessions_start ON digital.web_sessions(session_start_utc);
CREATE INDEX IF NOT EXISTS ix_web_events_session_time ON digital.web_events(session_id, event_timestamp);
CREATE INDEX IF NOT EXISTS ix_marketing_spend_date ON digital.marketing_spend(spend_date);

SELECT table_schema, table_name
FROM information_schema.tables
WHERE table_schema IN ('digital', 'audit')
ORDER BY table_schema, table_name;

```

Line-by-line explanation

- **Line 1:** -- Dialect: PostgreSQL SQL is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** -- Purpose: create the digital source schema and tables. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** -- Run in: pgAdmin Query Tool connected to database northstar_digital. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** -- Idempotency: CREATE IF NOT EXISTS avoids replacing existing objects. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** CREATE SCHEMA IF NOT EXISTS digital; — CREATE defines a new database object; SCHEMA is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; digital is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** CREATE SCHEMA IF NOT EXISTS audit; — CREATE defines a new database object; SCHEMA is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; audit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** CREATE TABLE IF NOT EXISTS audit.source_load_batch — CREATE defines a new database object; TABLE the database object that stores rows and columns; IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; audit.source_load_batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** batch_id varchar(80) PRIMARY KEY, — batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; varchar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 80 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** process_name varchar(100) NOT NULL, — process_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; varchar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 13:** `started_at_utc timestampz NOT NULL DEFAULT now()`, — `started_at_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestampz` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DEFAULT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `now` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** `finished_at_utc timestampz NULL`, — `finished_at_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestampz` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** `status varchar(20) NOT NULL`, — `status` shows tracked, staged, modified, and untracked work; `varchar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `20` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `source_file_count integer NULL`, — `source_file_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `integer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** `loaded_row_count bigint NULL`, — `loaded_row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bigint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** `error_message text NULL` — `error_message` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** `) ; —` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** blank line used only to separate logical sections for readability.
- **Line 21:** `CREATE TABLE IF NOT EXISTS digital.web_users` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `IF` begins a conditional decision; `NOT` reverses a Boolean condition; `EXISTS` tests whether a matching object or row is present; `digital.web_users` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** `( — (` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** `web_user_id bigint PRIMARY KEY`, — `web_user_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bigint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** `customer_id integer NULL`, — `customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `integer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** `anonymous_cookie_id varchar(64) NOT NULL`, — `anonymous_cookie_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `varchar` is a name, keyword, ar-

- gument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 64 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** email_hash char(64) NULL, — email_hash is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; char is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 64 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 27:** created_at timestampz NOT NULL, — created_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timestampz is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 28:** updated_at timestampz NOT NULL, — updated_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timestampz is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 29:** CONSTRAINT uq_web_users_cookie UNIQUE (anonymous_cookie_id) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; uq_web_users_cookie is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UNIQUE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; anonymous_cookie_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 30:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 31:** blank line used only to separate logical sections for readability.
 - **Line 32:** CREATE TABLE IF NOT EXISTS digital.campaigns — CREATE defines a new database object; TABLE the database object that stores rows and columns; IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; digital.campaigns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 33:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 34:** campaign_id integer PRIMARY KEY, — campaign_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; integer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 35:** campaign_code varchar(30) NOT NULL UNIQUE, — campaign_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; varchar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UNIQUE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 44:** `CONSTRAINT ck_campaign_dates CHECK (end_date >= start_date)` — `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ck_campaign_dates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CHECK` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `end_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `start_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 45:** `);` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** blank line used only to separate logical sections for readability.
- **Line 47:** `CREATE TABLE IF NOT EXISTS digital.web_sessions` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `IF` begins a conditional decision; `NOT` reverses a Boolean condition; `EXISTS` tests whether a matching object or row is present; `digital.web_sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 48:** `(` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 49:** `session_id bigint PRIMARY KEY,` — `session_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bigint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 50:** `web_user_id bigint NULL REFERENCES digital.web_users(web_user_id),` — `web_user_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bigint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `REFERENCES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `digital.web_users` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `web_user_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 51:** `session_start_utc timestamptz NOT NULL,` — `session_start_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestamptz` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 52:** `session_end_utc timestamptz NULL,` — `session_end_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestamptz` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 53:** `source_channel varchar(40) NOT NULL,` — `source_channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `varchar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `40` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 54:** `campaign_id integer NULL REFERENCES digital.campaigns(campaign_id),` — `campaign_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `integer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `REFERENCES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

described in the surrounding explanation; `digital.campaigns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `campaign_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 55:** `device_type varchar(30) NOT NULL`, — `device_type` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `varchar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 56:** `country_code varchar(10) NOT NULL`, — `country_code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `varchar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `10` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 57:** `converted_order_id bigint NULL`, — `converted_order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bigint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 58:** `created_at timestamptz NOT NULL`, — `created_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestamptz` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 59:** `CONSTRAINT ck_session_end CHECK (session_end_utc IS NULL OR session_end_utc >= session_start_utc)` — `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ck_session_end` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CHECK` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `session_end_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `OR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `session_end_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `session_start_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 60:** `);` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 61:** blank line used only to separate logical sections for readability.
- **Line 62:** `CREATE TABLE IF NOT EXISTS digital.web_events` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `IF` begins a conditional decision; `NOT` reverses a Boolean condition; `EXISTS` tests whether a matching object or row is present; `digital.web_events` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 63:** `(` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 64:** `event_id bigint PRIMARY KEY`, — `event_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bigint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

fier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 65:** `session_id bigint NOT NULL REFERENCES digital.web_sessions(session_id)`, — `session_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bigint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `REFERENCES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `digital.web_sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `session_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 66:** `event_timestamp timestamptz NOT NULL`, — `event_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestamptz` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 67:** `event_type varchar(50) NOT NULL`, — `event_type` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `varchar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `50` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 68:** `page_url text NULL`, — `page_url` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 69:** `product_id integer NULL`, — `product_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `integer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 70:** `order_id bigint NULL`, — `order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bigint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 71:** `event_properties jsonb NULL`, — `event_properties` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `jsonb` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 72:** `created_at timestamptz NOT NULL` — `created_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestamptz` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 73:** `);` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 74:** blank line used only to separate logical sections for readability.
- **Line 75:** `CREATE TABLE IF NOT EXISTS digital.campaign_touchpoints` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `IF` begins a conditional decision; `NOT` reverses a Boolean con-

dition; EXISTS tests whether a matching object or row is present; digital.campaign_touchpoints is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 76:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 77:** touchpoint_id bigint PRIMARY KEY, — touchpoint_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bigint is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 78:** web_user_id bigint NULL REFERENCES digital.web_users(web_user_id), — web_user_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bigint is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; digital.web_users is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; web_user_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 79:** session_id bigint NULL REFERENCES digital.web_sessions(session_id), — session_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bigint is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; digital.web_sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; session_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 80:** campaign_id integer NOT NULL REFERENCES digital.campaigns(campaign_id), — campaign_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; integer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; digital.campaigns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; campaign_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 81:** touchpoint_timestamp timestamptz NOT NULL, — touchpoint_timestamp is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timestamptz is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 82:** touchpoint_type varchar(40) NOT NULL, — touchpoint_type is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; varchar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 40 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 83:** attribution_weight numeric(8,6) NOT NULL, — attribution_weight is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; numeric is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 8 is a

numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 6 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 84:** `created_at timestampz NOT NULL, — created_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestampz` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 85:** `CONSTRAINT ck_touchpoint_weight CHECK (attribution_weight >= 0 AND attribution_weight <= 1) —` CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ck_touchpoint_weight` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CHECK is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `attribution_weight` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; AND is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `attribution_weight` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 86:** `); —` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 87:** blank line used only to separate logical sections for readability.
- **Line 88:** `CREATE TABLE IF NOT EXISTS digital.marketing_spend —` CREATE defines a new database object; TABLE the database object that stores rows and columns; IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; `digital.marketing_spend` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 89:** `( — (` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 90:** `spend_id bigint PRIMARY KEY, — spend_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bigint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 91:** `spend_date date NOT NULL, — spend_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 92:** `campaign_id integer NOT NULL REFERENCES digital.campaigns(campaign_id), — campaign_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; integer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `digital.campaigns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `campaign_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spend_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; >= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; AND is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; impressions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; >= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; AND is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 101:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 102:** blank line used only to separate logical sections for readability.
- **Line 103:** CREATE INDEX IF NOT EXISTS ix_web_sessions_start ON digital.web_sessions(session_start_utc); — CREATE defines a new database object; INDEX is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; ix_web_sessions_start is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; digital.web_sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; session_start_utc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 104:** CREATE INDEX IF NOT EXISTS ix_web_events_session_time ON digital.web_events(session_id, event_timestamp); — CREATE defines a new database object; INDEX is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; ix_web_events_session_time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; digital.web_events is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; session_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; event_timestamp is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 105:** CREATE INDEX IF NOT EXISTS ix_marketing_spend_date ON digital.marketing_spend(spend_date); — CREATE defines a new database object; INDEX is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; ix_marketing_spend_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; digital.marketing_spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spend_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 106:** blank line used only to separate logical sections for readability.
- **Line 107:** SELECT table_schema, table_name — SELECT requests data without changing it unless combined with a modifying statement; table_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 108:** FROM information_schema.tables — FROM identifies the source table or query; information_schema.tables is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 109:** WHERE table_schema IN ('digital', 'audit') — WHERE filters rows to those meeting a condition; table_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the

surrounding explanation; IN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'digital' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'audit' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 110:** ORDER BY table_schema, table_name; — ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 4 lesson index](#)

File introduction — docs/data_dictionary.csv

Exact filename: data_dictionary.csv

Filename extension: .csv. In this project it identifies structured rows used as a definition, mapping, or fixture and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: docs relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: structured rows used as a definition, mapping, or fixture for Phase 4.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in docs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: data_dictionary.csv.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled data_dictionary.csv and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 4.F5.1 — Create the file docs/data_dictionary.csv from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\data_dictionary.csv" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\docs\data_dictionary.csv" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\docs\data_dictionary.csv` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use

`git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\docs\data_dictionary.csv" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/data_dictionary.csv"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `docs`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `docs/data_dictionary.csv`.
3. In PowerShell at the repository root, run `Test-Path ".\docs\data_dictionary.csv"` and require `True`.
4. Run `git status --short -- "docs/data_dictionary.csv"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

[Back to the Phase 4 lesson index](#)

File introduction — docs/source_to_target_mapping.csv

Exact filename: `source_to_target_mapping.csv`

Filename extension: `.csv`. In this project it identifies structured rows used as a definition, mapping, or fixture and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `docs` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: structured rows used as a definition, mapping, or fixture for Phase 4.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in docs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `source_to_target_mapping.csv`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `source_to_target_mapping.csv` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 4.F6.1 — Create the file docs/source_to_target_mapping.csv from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\source_to_target_mapping.csv" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\docs\source_to_target_mapping.csv" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs\source_to_target_mapping.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\docs\source_to_target_mapping.csv" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/source_to_target_mapping.csv"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `docs`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `docs/source_to_target_mapping.csv`.
3. In PowerShell at the repository root, run `Test-Path ".\docs\source_to_target_mapping.csv"` and require `True`.
4. Run `git status --short -- "docs/source_to_target_mapping.csv"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

[Back to the Phase 4 lesson index](#)

What your screen should look like now

- Every Phase 4 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 4.3 completion checklist

☐

`sql/sqlserver/source/01_create_erp_database.sql` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`sql/sqlserver/source/02_create_erp_tables.sql` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`sql/postgres/source/00_create_database.sql` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`sql/postgres/source/01_create_digital_schema.sql` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`docs/data_dictionary.csv` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`docs/source_to_target_mapping.csv` was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 4.4](#) and recheck the root and branch.

Lesson 4.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Source object 1 — `erp.product_categories`

Real-world object: `product_categories` in the SQL Server ERP domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to SQL Server ERP; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per category identifier. Verify that every column and example row describes that same unit.

Primary key: `category_id`

Foreign keys: `parent_category_id` → `product_categories.category_id`

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
<code>category_id</code>	INT	No	Yes	—	Category identifier	10	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
<code>category_name</code>	NVARCHAR(100)	No	No	—	Business category name	Home Office	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
<code>parent_category_id</code>	INT	Yes	No	<code>product_categories.category_id</code>	Optional parent category	NULL	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Nullable because the source can legitimately omit it
<code>created_at</code>	DATETIME2	No	No	—	Source creation timestamp	2026-01-02T10:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
updated_at	DATETIME2	No	No	—	Source update timestamp	2026-02-03T11:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- category_id = 1
- category_name = Electronics
- parent_category_id = ``
- created_at = 2024-01-01T00:00:00
- updated_at = 2026-08-15T12:00:00

Action 4.4.S1 — Create and inspect `erp.product_categories`

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Create and inspect `erp.product_categories`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is `NorthstarRetail_ERP` and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.product_categories` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with `Ctrl+S` before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows `NorthstarRetail_ERP`.
8. LOOK FOR: No error in the Messages pane and the table under `NorthstarRetail_ERP > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is NorthstarRetail_ERP and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.product_categories` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows NorthstarRetail_ERP.
8. LOOK FOR: No error in the Messages pane and the table under NorthstarRetail_ERP > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: T-SQL

Exact file: `sql/sqlserver/source/02_create_erp_tables.sql`

Execution location: SQL Server Management Studio (SSMS), database NorthstarRetail_ERP.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE erp.product_categories
(
    category_id          INT NOT NULL CONSTRAINT PK_product_categories PRIMARY KEY,
    category_name        NVARCHAR(100) NOT NULL,
    parent_category_id   INT NULL,
    created_at           DATETIME2(3) NOT NULL,
    updated_at           DATETIME2(3) NOT NULL,
    CONSTRAINT FK_product_categories_parent FOREIGN KEY (parent_category_id)
```

);

Line-by-line DDL explanation

- **Line 1:** `CREATE TABLE erp.product_categories` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `erp.product_categories` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `( — (` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `category_id INT NOT NULL CONSTRAINT PK_product_categories PRIMARY KEY, —` `category_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PK_product_categories` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `category_name NVARCHAR(100) NOT NULL, —` `category_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NVARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `100` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `parent_category_id INT NULL, —` `parent_category_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `created_at DATETIME2(3) NOT NULL, —` `created_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `updated_at DATETIME2(3) NOT NULL, —` `updated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `CONSTRAINT FK_product_categories_parent FOREIGN KEY (parent_category_id) —` `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FK_product_categories_parent` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FOREIGN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parent_category_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `REFERENCES erp.product_categories(category_id) —` `REFERENCES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `erp.product_categories` is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; category_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 10:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S1.V — Verify metadata and row count for `erp.product_categories`

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SQL Server Management Studio (SSMS) query window connected to `NorthstarRetail_ERP`

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM erp.product_categories;
```

Where to enter it:

SQL Server Management Studio (SSMS) query window connected to `NorthstarRetail_ERP`. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM erp.product_categories;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument,

identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; `erp.product_categories` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record `VERIFIED BY USER` and proceed without recreating it.
- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for `erp.product_categories`

Show SQL Server Management Studio (SSMS) with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: `docs/images/phase04_erp_product_categories_schema_verified.png`. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled `erp.product_categories` at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 2 — `erp.suppliers`

Real-world object: `suppliers` in the SQL Server ERP domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to SQL Server ERP; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per supplier identifier. Verify that every column and example row describes that same unit.

Primary key: `supplier_id`

Foreign keys: No foreign key; this object is a parent/reference table or independent event table.

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
<code>supplier_id</code>	INT	No	Yes	—	Supplier identifier	501	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
<code>supplier_code</code>	VARCHAR(20)	No	No	—	Supplier business key	SUP-0501	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
<code>supplier_name</code>	NVARCHAR(150)	No	No	—	Fictional supplier name	Blue Harbor Goods	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
<code>country_code</code>	CHAR(2)	No	No	—	ISO-like country code	US	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
<code>lead_time_days</code>	INT	No	No	—	Expected replenishment lead time	14	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
active_flag	BIT	No	No	—	Whether supplier is active	1	a true/false flag. Required because the business record is incomplete without it
created_at	DATETIME2	No	No	—	Source creation timestamp	2025-01-01T00:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
updated_at	DATETIME2	No	No	—	Source update timestamp	2026-02-01T00:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- supplier_id = 5001
- supplier_code = SUP-05001
- supplier_name = Johnson Ltd Supply
- country_code = US
- lead_time_days = 14
- active_flag = 1
- created_at = 2024-01-01T00:00:00
- updated_at = 2026-08-15T12:00:00

Action 4.4.S2 — Create and inspect erp.suppliers

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create and inspect erp.suppliers. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is NorthstarRetail_ERP and the query editor is not connected to master, postgres, or another database.

3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.suppliers` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows `NorthstarRetail_ERP`.
8. LOOK FOR: No error in the Messages pane and the table under `NorthstarRetail_ERP > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is `NorthstarRetail_ERP` and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.suppliers` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows `NorthstarRetail_ERP`.
8. LOOK FOR: No error in the Messages pane and the table under `NorthstarRetail_ERP > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

DDL section — build and explain it before execution

Language: T-SQL

Exact file: sql/sqlserver/source/02_create_erp_tables.sql

Execution location: SQL Server Management Studio (SSMS), database NorthstarRetail_ERP.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE erp.suppliers
(
    supplier_id      INT NOT NULL CONSTRAINT PK_suppliers PRIMARY KEY,
    supplier_code    VARCHAR(20) NOT NULL,
    supplier_name    NVARCHAR(150) NOT NULL,
    country_code     CHAR(2) NOT NULL,
    lead_time_days   INT NOT NULL,
    active_flag      BIT NOT NULL,
    created_at       DATETIME2(3) NOT NULL,
    updated_at       DATETIME2(3) NOT NULL,
    CONSTRAINT UQ_suppliers_supplier_code UNIQUE (supplier_code),
    CONSTRAINT CK_suppliers_lead_time CHECK (lead_time_days >= 0)
);
```

Line-by-line DDL explanation

- **Line 1:** CREATE TABLE erp.suppliers — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.suppliers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** supplier_id INT NOT NULL CONSTRAINT PK_suppliers PRIMARY KEY, — supplier_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PK_suppliers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** supplier_code VARCHAR(20) NOT NULL, — supplier_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 20 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** supplier_name NVARCHAR(150) NOT NULL, — supplier_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NVARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 150 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** country_code CHAR(2) NOT NULL, — country_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** `lead_time_days INT NOT NULL`, — `lead_time_days` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `active_flag BIT NOT NULL`, — `active_flag` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `created_at DATETIME2(3) NOT NULL`, — `created_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `updated_at DATETIME2(3) NOT NULL`, — `updated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** `CONSTRAINT UQ_suppliers_supplier_code UNIQUE (supplier_code)`, — `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UQ_suppliers_supplier_code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UNIQUE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `supplier_code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `CONSTRAINT CK_suppliers_lead_time CHECK (lead_time_days >= 0)` — `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CK_suppliers_lead_time` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CHECK` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `lead_time_days` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `);` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S2.V — Verify metadata and row count for `erp.suppliers`

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SQL Server Management Studio (SSMS) query window connected to `NorthstarRetail_ERP`

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM erp.suppliers;
```

Where to enter it:

SQL Server Management Studio (SSMS) query window connected to NorthstarRetail_ERP. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM erp.suppliers;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `erp.suppliers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record `VERIFIED BY USER` and proceed without recreating it.
- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for `erp.suppliers`

Show SQL Server Management Studio (SSMS) with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: `docs/images/phase04_erp_suppliers_schema_verified.png`. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled `erp.suppliers` at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 3 — `erp.products`

Real-world object: products in the SQL Server ERP domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to SQL Server ERP; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per product identifier. Verify that every column and example row describes that same unit.

Primary key: `product_id`

Foreign keys: `category_id` → `product_categories.category_id`; `supplier_id` → `suppliers.supplier_id`

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
product_id	INT	No	Yes	—	Product identifier	10001	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
sku	VARCHAR(30)	No	No	—	Stock keeping unit business key	NRT-ELE-10001	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
product_name	NVARCHAR(200)	No	No	—	Product display name	Northstar Wireless Mouse	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
category_id	INT	No	No	product_categories.category_id	Product category	10	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
supplier_id	INT	No	No	suppliers.supplier_id	Primary supplier	501	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
brand	NVARCHAR(100)	No	No	—	Fictional brand	Aurora Works	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
unit_cost	DECIMAL(12,2)	No	No	—	Current source cost	18.50	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
list_price	DECIMAL(12,2)	No	No	—	Standard selling price	39.99	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it
currency_code	CHAR(3)	No	No	—	Price currency	USD	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
active_flag	BIT	No	No	—	Whether product is sellable	1	a true/false flag. Required because the business record is incomplete without it
created_at	DATETIME2	No	No	—	Source creation timestamp	2025-01-01T00:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
updated_at	DATETIME2	No	No	—	Source update timestamp	2026-02-01T00:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- product_id = 10001
- sku = NRT-OUT-10001
- product_name = Aurora Works Mat 1
- category_id = 5
- supplier_id = 5008
- brand = Cedar & Finch
- unit_cost = 22.77
- list_price = 37.63
- currency_code = USD
- active_flag = 1
- created_at = 2024-01-01T00:00:00
- updated_at = 2026-08-15T12:00:00

Action 4.4.S3 — Create and inspect erp.products

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Create and inspect `erp.products`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is `NorthstarRetail_ERP` and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.products` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with `Ctrl+S` before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/`F5` only after the database selector shows `NorthstarRetail_ERP`.
8. LOOK FOR: No error in the Messages pane and the table under `NorthstarRetail_ERP > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is `NorthstarRetail_ERP` and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.products` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with `Ctrl+S` before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/`F5` only after the database selector shows `NorthstarRetail_ERP`.
8. LOOK FOR: No error in the Messages pane and the table under `NorthstarRetail_ERP > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: T-SQL

Exact file: sql/sqlserver/source/02_create_erp_tables.sql

Execution location: SQL Server Management Studio (SSMS), database NorthstarRetail_ERP.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE erp.products
(
    product_id          INT NOT NULL CONSTRAINT PK_products PRIMARY KEY,
    sku                 VARCHAR(30) NOT NULL,
    product_name        NVARCHAR(200) NOT NULL,
    category_id         INT NOT NULL,
    supplier_id         INT NOT NULL,
    brand               NVARCHAR(100) NOT NULL,
    unit_cost           DECIMAL(12,2) NOT NULL,
    list_price          DECIMAL(12,2) NOT NULL,
    currency_code       CHAR(3) NOT NULL,
    active_flag         BIT NOT NULL,
    created_at          DATETIME2(3) NOT NULL,
    updated_at          DATETIME2(3) NOT NULL,
    CONSTRAINT UQ_products_sku UNIQUE (sku),
    CONSTRAINT FK_products_category FOREIGN KEY (category_id) REFERENCES erp.product_categories(category_id),
    CONSTRAINT FK_products_supplier FOREIGN KEY (supplier_id) REFERENCES erp.suppliers(supplier_id),
    CONSTRAINT CK_products_amounts CHECK (unit_cost >= 0 AND list_price >= 0)
);
```

Line-by-line DDL explanation

- **Line 1:** CREATE TABLE erp.products — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.products is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** product_id INT NOT NULL CONSTRAINT PK_products PRIMARY KEY, — product_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PK_products is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

ifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

eral whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 18:** CONSTRAINT CK_products_amounts CHECK (unit_cost >= 0 AND list_price >= 0) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CK_products_amounts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CHECK is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; unit_cost is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; >= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; AND is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; list_price is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; >= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S3.V — Verify metadata and row count for erp.products

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SQL Server Management Studio (SSMS) query window connected to NorthstarRetail_ERP

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM erp.products;
```

Where to enter it:

SQL Server Management Studio (SSMS) query window connected to NorthstarRetail_ERP. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM erp.products;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `erp.products` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record `VERIFIED BY USER` and proceed without recreating it.

- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for `erp.products`

Show SQL Server Management Studio (SSMS) with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: `docs/images/phase04_erp_products_schema_verified.png`. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled `erp.products` at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 4 — `erp.stores`

Real-world object: `stores` in the SQL Server ERP domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to SQL Server ERP; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per store identifier. Verify that every column and example row describes that same unit.

Primary key: `store_id`

Foreign keys: No foreign key; this object is a parent/reference table or independent event table.

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
<code>store_id</code>	INT	No	Yes	—	Store identifier	201	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
<code>store_code</code>	VARCHAR(20)	No	No	—	Store business key	VA-ARL-01	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
<code>store_name</code>	NVARCHAR(150)	No	No	—	Store display name	Northstar Arlington	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
<code>region</code>	VARCHAR(30)	No	No	—	Reporting region	Mid-Atlantic	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
state_code	CHAR(2)	No	No	—	US state code	VA	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
city	NVARCHAR(100)	No	No	—	Store city	Arlington	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
open_date	DATE	No	No	—	Store opening date	2021-05-01	a calendar date with no time-of-day component. Required because the business record is incomplete without it
active_flag	BIT	No	No	—	Whether store is open	1	a true/false flag. Required because the business record is incomplete without it
created_at	DATETIME2	No	No	—	Source creation timestamp	2021-05-01T00:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
updated_at	DATETIME2	No	No	—	Source update timestamp	2026-02-01T00:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- store_id = 201
- store_code = NY-NEW-01
- store_name = Northstar New York 1
- region = Northeast
- state_code = NY
- city = New York
- open_date = 2023-09-01
- active_flag = 1
- created_at = 2024-01-01T00:00:00
- updated_at = 2026-08-15T12:00:00

Action 4.4.S4 — Create and inspect erp.stores

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Create and inspect `erp.stores`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is `NorthstarRetail_ERP` and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.stores` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with `Ctrl+S` before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/`F5` only after the database selector shows `NorthstarRetail_ERP`.
8. LOOK FOR: No error in the Messages pane and the table under `NorthstarRetail_ERP > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is `NorthstarRetail_ERP` and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.stores` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with `Ctrl+S` before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/`F5` only after the database selector shows `NorthstarRetail_ERP`.
8. LOOK FOR: No error in the Messages pane and the table under `NorthstarRetail_ERP > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: T-SQL

Exact file: sql/sqlserver/source/02_create_erp_tables.sql

Execution location: SQL Server Management Studio (SSMS), database NorthstarRetail_ERP.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE erp.stores
(
    store_id          INT NOT NULL CONSTRAINT PK_stores PRIMARY KEY,
    store_code        VARCHAR(20) NOT NULL,
    store_name        NVARCHAR(150) NOT NULL,
    region            VARCHAR(30) NOT NULL,
    state_code        CHAR(2) NOT NULL,
    city              NVARCHAR(100) NOT NULL,
    open_date         DATE NOT NULL,
    active_flag       BIT NOT NULL,
    created_at        DATETIME2(3) NOT NULL,
    updated_at        DATETIME2(3) NOT NULL,
    CONSTRAINT UQ_stores_store_code UNIQUE (store_code)
);
```

Line-by-line DDL explanation

- **Line 1:** CREATE TABLE erp.stores — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.stores is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** store_id INT NOT NULL CONSTRAINT PK_stores PRIMARY KEY, — store_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PK_stores is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

- **Line 12:** `updated_at DATETIME2(3) NOT NULL`, — `updated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `CONSTRAINT UQ_stores_store_code UNIQUE (store_code)` — `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UQ_stores_store_code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UNIQUE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `store_code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** `);` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S4.V — Verify metadata and row count for `erp.stores`

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SQL Server Management Studio (SSMS) query window connected to `NorthstarRetail_ERP`

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM erp.stores;
```

Where to enter it:

SQL Server Management Studio (SSMS) query window connected to `NorthstarRetail_ERP`. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM erp.stores;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `erp.stores` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record `VERIFIED BY USER` and proceed without recreating it.

- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for `erp.stores`

Show SQL Server Management Studio (SSMS) with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: `docs/images/phase04_erp_stores_schema_verified.png`. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled `erp.stores` at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 5 — `erp.employees`

Real-world object: `employees` in the SQL Server ERP domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to SQL Server ERP; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per employee identifier. Verify that every column and example row describes that same unit.

Primary key: `employee_id`

Foreign keys: `store_id` → `stores.store_id`; `manager_employee_id` → `employees.employee_id`

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
<code>employee_id</code>	INT	No	Yes	—	Employee identifier	301	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
<code>employee_number</code>	VARCHAR(20)	No	No	—	Employee business key	EMP-00301	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
<code>store_id</code>	INT	Yes	No	<code>stores.store_id</code>	Assigned store	201	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Nullable because the source can legitimately omit it
<code>manager_employee_id</code>	INT	Yes	No	<code>employees.employee_id</code>	Manager identifier	300	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Nullable because the source can legitimately omit it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
first_name	NVARCHAR(80)	No	No	—	Synthetic first name	Mina	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
last_name	NVARCHAR(80)	No	No	—	Synthetic last name	Nassar	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
job_title	NVARCHAR(100)	No	No	—	Job title	Sales Associate	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
hire_date	DATE	No	No	—	Hire date	2024-03-11	a calendar date with no time-of-day component. Required because the business record is incomplete without it
active_flag	BIT	No	No	—	Employment status	1	a true/false flag. Required because the business record is incomplete without it
created_at	DATETIME2	No	No	—	Source creation timestamp	2024-03-11T00:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
updated_at	DATETIME2	No	No	—	Source update timestamp	2026-02-01T00:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- employee_id = 301
- employee_number = EMP-00301
- store_id = 201
- manager_employee_id = ``
- first_name = Alexis
- last_name = Clark
- job_title = Store Manager

- hire_date = 2023-02-12
- active_flag = 1
- created_at = 2024-01-01T00:00:00
- updated_at = 2026-08-15T12:00:00

Action 4.4.S5 — Create and inspect `erp.employees`

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Create and inspect `erp.employees`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is `NorthstarRetail_ERP` and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.employees` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with `Ctrl+S` before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/`F5` only after the database selector shows `NorthstarRetail_ERP`.
8. LOOK FOR: No error in the Messages pane and the table under `NorthstarRetail_ERP > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is `NorthstarRetail_ERP` and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.employees` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with `Ctrl+S` before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/`F5` only after the database selector shows `NorthstarRetail_ERP`.
8. LOOK FOR: No error in the Messages pane and the table under `NorthstarRetail_ERP > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.

10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: T-SQL

Exact file: sql/sqlserver/source/02_create_erp_tables.sql

Execution location: SQL Server Management Studio (SSMS), database NorthstarRetail_ERP.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE erp.employees
(
    employee_id          INT NOT NULL CONSTRAINT PK_employees PRIMARY KEY,
    employee_number      VARCHAR(20) NOT NULL,
    store_id             INT NULL,
    manager_employee_id  INT NULL,
    first_name           NVARCHAR(80) NOT NULL,
    last_name            NVARCHAR(80) NOT NULL,
    job_title            NVARCHAR(100) NOT NULL,
    hire_date            DATE NOT NULL,
    active_flag          BIT NOT NULL,
    created_at           DATETIME2(3) NOT NULL,
    updated_at           DATETIME2(3) NOT NULL,
    CONSTRAINT UQ_employees_number UNIQUE (employee_number),
    CONSTRAINT FK_employees_store FOREIGN KEY (store_id) REFERENCES erp.stores(store_id),
    CONSTRAINT FK_employees_manager FOREIGN KEY (manager_employee_id) REFERENCES erp.employees(employee_id)
);
```

Line-by-line DDL explanation

- **Line 1:** CREATE TABLE erp.employees — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.employees is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

- **Line 17:** `) ; — )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S5.V — Verify metadata and row count for `erp.employees`

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SQL Server Management Studio (SSMS) query window connected to `NorthstarRetail_ERP`

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM erp.employees;
```

Where to enter it:

SQL Server Management Studio (SSMS) query window connected to `NorthstarRetail_ERP`. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM erp.employees;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `erp.employees` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record `VERIFIED BY USER` and proceed without recreating it.
- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for `erp.employees`

Show SQL Server Management Studio (SSMS) with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: `docs/images/phase04_erp_employees_schema_verified.png`. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled `erp.employees` at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 6 — `erp.customers`

Real-world object: customers in the SQL Server ERP domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to SQL Server ERP; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per customer identifier. Verify that every column and example row describes that same unit.

Primary key: `customer_id`

Foreign keys: No foreign key; this object is a parent/reference table or independent event table.

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
<code>customer_id</code>	INT	No	Yes	—	Customer identifier	100001	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
<code>customer_number</code>	VARCHAR(25)	No	No	—	Customer business key; dirty duplicates are intentional	CUS-100001	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
<code>first_name</code>	NVARCHAR(80)	No	No	—	Synthetic first name	Amanuel	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
<code>last_name</code>	NVARCHAR(80)	No	No	—	Synthetic last name	Gebre	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
<code>email</code>	NVARCHAR(255)	Yes	No	—	Synthetic email; may contain controlled issues	amanuel@example.test	text with either a stated maximum length or engine-managed variable length. Nullable because the source can legitimately omit it
<code>phone</code>	VARCHAR(40)	Yes	No	—	Synthetic phone	+1-555-0100	text with either a stated maximum length or engine-managed variable length. Nullable because the source can legitimately omit it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
signup_date	DATE	No	No	—	Customer signup date	2025-06-12	a calendar date with no time-of-day component. Required because the business record is incomplete without it
loyalty_tier	VARCHAR(20)	No	No	—	Loyalty segment	Silver	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
marketing_opt_in	BIT	No	No	—	Marketing consent flag	1	a true/false flag. Required because the business record is incomplete without it
created_at	DATETIME2	No	No	—	Source creation timestamp	2025-06-12T12:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
updated_at	DATETIME2	No	No	—	Source update timestamp	2026-01-05T12:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- customer_id = 100001
- customer_number = CUS-0100001
- first_name = Andrew
- last_name = Hayes
- email = ``
- phone = +1-555-7736
- signup_date = 2025-10-26
- loyalty_tier = Bronze
- marketing_opt_in = 1
- created_at = 2025-10-26T00:00:00
- updated_at = 2026-08-15T12:00:00

Action 4.4.S6 — Create and inspect erp.customers

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Create and inspect `erp.customers`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is `NorthstarRetail_ERP` and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.customers` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with `Ctrl+S` before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/`F5` only after the database selector shows `NorthstarRetail_ERP`.
8. LOOK FOR: No error in the Messages pane and the table under `NorthstarRetail_ERP > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is `NorthstarRetail_ERP` and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.customers` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with `Ctrl+S` before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/`F5` only after the database selector shows `NorthstarRetail_ERP`.
8. LOOK FOR: No error in the Messages pane and the table under `NorthstarRetail_ERP > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: T-SQL

Exact file: sql/sqlserver/source/02_create_erp_tables.sql

Execution location: SQL Server Management Studio (SSMS), database NorthstarRetail_ERP.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE erp.customers
(
    customer_id          INT NOT NULL CONSTRAINT PK_customers PRIMARY KEY,
    customer_number      VARCHAR(25) NOT NULL,
    first_name           NVARCHAR(80) NOT NULL,
    last_name            NVARCHAR(80) NOT NULL,
    email                NVARCHAR(255) NULL,
    phone                VARCHAR(40) NULL,
    signup_date          DATE NOT NULL,
    loyalty_tier          VARCHAR(20) NOT NULL,
    marketing_opt_in     BIT NOT NULL,
    created_at           DATETIME2(3) NOT NULL,
    updated_at           DATETIME2(3) NOT NULL
    -- Deliberately no unique constraint on customer_number in the learning source:
    -- controlled duplicate business keys must reach Silver and be handled there.
);
```

Line-by-line DDL explanation

- **Line 1:** CREATE TABLE erp.customers — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** customer_id INT NOT NULL CONSTRAINT PK_customers PRIMARY KEY, — customer_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PK_customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** customer_number VARCHAR(25) NOT NULL, — customer_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 25 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

ifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 13:** updated_at DATETIME2(3) NOT NULL — updated_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** -- Deliberately no unique constraint on customer_number in the learning source: is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 15:** -- controlled duplicate business keys must reach Silver and be handled there. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 16:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S6.V — Verify metadata and row count for erp.customers

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SQL Server Management Studio (SSMS) query window connected to NorthstarRetail_ERP

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM erp.customers;
```

Where to enter it:

SQL Server Management Studio (SSMS) query window connected to NorthstarRetail_ERP. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM erp.customers;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `erp.customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record `VERIFIED BY USER` and proceed without recreating it.

- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for `erp.customers`

Show SQL Server Management Studio (SSMS) with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: `docs/images/phase04_erp_customers_schema_verified.png`. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled `erp.customers` at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 7 — `erp.addresses`

Real-world object: addresses in the SQL Server ERP domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to SQL Server ERP; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per address identifier. Verify that every column and example row describes that same unit.

Primary key: `address_id`

Foreign keys: `customer_id` → `customers.customer_id`

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
<code>address_id</code>	INT	No	Yes	—	Address identifier	500001	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
<code>customer_id</code>	INT	No	No	<code>customers.customer_id</code>	Owning customer	100001	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
<code>address_type</code>	VARCHAR(20)	No	No	—	Billing or shipping	Shipping	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
<code>line1</code>	NVARCHAR(200)	No	No	—	Synthetic street line	123 Example Ave	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
line2	NVARCHAR(200)	Yes	No	—	Optional unit	Apt 4	text with either a stated maximum length or engine-managed variable length. Nullable because the source can legitimately omit it
city	NVARCHAR(100)	No	No	—	City	Richmond	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
state_code	VARCHAR(10)	No	No	—	State code; controlled invalid values included	VA	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
postal_code	VARCHAR(20)	No	No	—	Postal code	23220	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
country_code	VARCHAR(10)	No	No	—	Country code	US	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
is_default	BIT	No	No	—	Default address flag	1	a true/false flag. Required because the business record is incomplete without it
created_at	DATETIME2	No	No	—	Source creation timestamp	2025-06-12T12:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
updated_at	DATETIME2	No	No	—	Source update timestamp	2026-01-05T12:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- address_id = 500001
- customer_id = 100001
- address_type = Shipping
- line1 = 643 Hammond Shoals
- line2 = ``
- city = San Diego
- state_code = VIRGINIA
- postal_code = 51556
- country_code = US
- is_default = 1
- created_at = 2025-10-26T00:00:00
- updated_at = 2026-08-15T12:00:00

Action 4.4.S7 — Create and inspect erp.addresses

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create and inspect erp.addresses. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is NorthstarRetail_ERP and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open sql/sqlserver/source/02_create_erp_tables.sql in VS Code beside the database editor.
4. TYPE THIS: Build the erp.addresses definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows NorthstarRetail_ERP.
8. LOOK FOR: No error in the Messages pane and the table under NorthstarRetail_ERP > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.

2. LOOK FOR: Confirm the active connection is NorthstarRetail_ERP and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open sql/sqlserver/source/02_create_erp_tables.sql in VS Code beside the database editor.
4. TYPE THIS: Build the erp.addresses definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows NorthstarRetail_ERP.
8. LOOK FOR: No error in the Messages pane and the table under NorthstarRetail_ERP > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: T-SQL

Exact file: sql/sqlserver/source/02_create_erp_tables.sql

Execution location: SQL Server Management Studio (SSMS), database NorthstarRetail_ERP.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE erp.addresses
(
    address_id          INT NOT NULL CONSTRAINT PK_addresses PRIMARY KEY,
    customer_id         INT NOT NULL,
    address_type        VARCHAR(20) NOT NULL,
    line1               NVARCHAR(200) NOT NULL,
    line2               NVARCHAR(200) NULL,
    city                NVARCHAR(100) NOT NULL,
    state_code          VARCHAR(10) NOT NULL,
    postal_code         VARCHAR(20) NOT NULL,
    country_code        VARCHAR(10) NOT NULL,
    is_default          BIT NOT NULL,
    created_at          DATETIME2(3) NOT NULL,
    updated_at          DATETIME2(3) NOT NULL,
```

```
CONSTRAINT FK_addresses_customer FOREIGN KEY (customer_id) REFERENCES erp.customers(customer_id)
);
```

Line-by-line DDL explanation

- **Line 1:** `CREATE TABLE erp.addresses` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `erp.addresses` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `(` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `address_id INT NOT NULL CONSTRAINT PK_addresses PRIMARY KEY,` — `address_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PK_addresses` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `customer_id INT NOT NULL,` — `customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `address_type VARCHAR(20) NOT NULL,` — `address_type` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `20` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `line1 NVARCHAR(200) NOT NULL,` — `line1` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NVARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `200` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `line2 NVARCHAR(200) NULL,` — `line2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NVARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `200` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `city NVARCHAR(100) NOT NULL,` — `city` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NVARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `100` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `state_code VARCHAR(10) NOT NULL,` — `state_code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `10` is a numeric literal used by

the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

Verification query

Action 4.4.S7.V — Verify metadata and row count for erp.addresses

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SQL Server Management Studio (SSMS) query window connected to NorthstarRetail_ERP

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM erp.addresses;
```

Where to enter it:

SQL Server Management Studio (SSMS) query window connected to NorthstarRetail_ERP. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM erp.addresses;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `erp.addresses` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record `VERIFIED BY USER` and proceed without recreating it.
- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for `erp.addresses`

Show SQL Server Management Studio (SSMS) with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: `docs/images/phase04_erp_addresses_schema_verified.png`. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled `erp.addresses` at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 8 — `erp.orders`

Real-world object: orders in the SQL Server ERP domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to

SQL Server ERP; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per order identifier. Verify that every column and example row describes that same unit.

Primary key: order_id

Foreign keys: customer_id → customers.customer_id; store_id → stores.store_id; employee_id → employees.employee_id

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
order_id	BIGINT	No	Yes	—	Order identifier	9000001	a whole-number type used where identifiers or counts may exceed the normal integer range. Required because the business record is incomplete without it
order_number	VARCHAR(30)	No	No	—	Order business key	ORD-09000001	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
customer_id	INT	No	No	customers.customer_id	Ordering customer	100001	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
store_id	INT	Yes	No	stores.store_id	Store for store sales or fulfillment	201	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Nullable because the source can legitimately omit it
employee_id	INT	Yes	No	employees.employee_id	Sales employee when applicable	301	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Nullable because the source can legitimately omit it
channel_code	VARCHAR(20)	No	No	—	STORE or ONLINE	ONLINE	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
order_timestamp	DATETIME2	No	No	—	Order event time	2026-01-04T14:22:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
order_status	VARCHAR(20)	No	No	—	Operational order status	SHIPPED	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
promotion_code	VARCHAR(30)	Yes	No	—	Promotion business key	WINTER10	text with either a stated maximum length or engine-managed variable length. Nullable because the source can legitimately omit it
currency_code	CHAR(3)	No	No	—	Order currency	USD	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
subtotal_amount	DECIMAL(14,2)	No	No	—	Pre-tax and pre-shipping amount after item discounts	99.98	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it
tax_amount	DECIMAL(14,2)	No	No	—	Tax amount	6.00	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it
shipping_amount	DECIMAL(14,2)	No	No	—	Shipping charge	5.99	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
order_total_amount	DECIMAL(14,2)	No	No	—	Customer charged total	111.97	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it
created_at	DATETIME2	No	No	—	Source creation timestamp	2026-01-04T14:22:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
updated_at	DATETIME2	No	No	—	Source update timestamp	2026-01-05T10:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- order_id = 9000001
- order_number = ORD-009000001
- customer_id = 100050
- store_id = 204
- employee_id = 309.0
- channel_code = STORE
- order_timestamp = 2026-06-25T14:48:00
- order_status = COMPLETED
- promotion_code = WELCOME10
- currency_code = USD
- created_at = 2026-06-25T14:48:00
- updated_at = 2026-08-15T12:00:00

Action 4.4.S8 — Create and inspect erp.orders

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create and inspect `erp.orders`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is `NorthstarRetail_ERP` and the query editor is not connected to `master`, `postgres`, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.orders` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with `Ctrl+S` before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/`F5` only after the database selector shows `NorthstarRetail_ERP`.
8. LOOK FOR: No error in the Messages pane and the table under `NorthstarRetail_ERP > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is `NorthstarRetail_ERP` and the query editor is not connected to `master`, `postgres`, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.orders` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with `Ctrl+S` before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/`F5` only after the database selector shows `NorthstarRetail_ERP`.
8. LOOK FOR: No error in the Messages pane and the table under `NorthstarRetail_ERP > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: T-SQL

Exact file: sql/sqlserver/source/02_create_erp_tables.sql

Execution location: SQL Server Management Studio (SSMS), database NorthstarRetail_ERP.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE erp.orders
(
    order_id          BIGINT NOT NULL CONSTRAINT PK_orders PRIMARY KEY,
    order_number      VARCHAR(30) NOT NULL,
    customer_id       INT NOT NULL,
    store_id          INT NULL,
    employee_id       INT NULL,
    channel_code      VARCHAR(20) NOT NULL,
    order_timestamp   DATETIME2(3) NOT NULL,
    order_status      VARCHAR(20) NOT NULL,
    promotion_code    VARCHAR(30) NULL,
    currency_code     CHAR(3) NOT NULL,
    created_at        DATETIME2(3) NOT NULL,
    updated_at        DATETIME2(3) NOT NULL,
    CONSTRAINT UQ_orders_order_number UNIQUE (order_number),
    CONSTRAINT FK_orders_customer FOREIGN KEY (customer_id) REFERENCES erp.customers(customer_id),
    CONSTRAINT FK_orders_store FOREIGN KEY (store_id) REFERENCES erp.stores(store_id),
    CONSTRAINT FK_orders_employee FOREIGN KEY (employee_id) REFERENCES erp.employees(employee_id),
    CONSTRAINT CK_orders_channel CHECK (channel_code IN ('STORE', 'ONLINE'))
);
```

Line-by-line DDL explanation

- **Line 1:** CREATE TABLE erp.orders — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** order_id BIGINT NOT NULL CONSTRAINT PK_orders PRIMARY KEY, — order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BIGINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PK_orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** order_number VARCHAR(30) NOT NULL, — order_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

surrounding explanation; `erp.employees` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `employee_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)`, `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 19:** `CONSTRAINT CK_orders_channel CHECK (channel_code IN ('STORE', 'ONLINE'))` — `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CK_orders_channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CHECK` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `channel_code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'STORE'` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'ONLINE'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** `) ; —` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S8.V — Verify metadata and row count for `erp.orders`

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SQL Server Management Studio (SSMS) query window connected to `NorthstarRetail_ERP`

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM erp.orders;
```

Where to enter it:

SQL Server Management Studio (SSMS) query window connected to NorthstarRetail_ERP. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM erp.orders;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `erp.orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.

- **If the object matches exactly and was created by your verified earlier run:** Record VERIFIED BY USER and proceed without recreating it.
- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for `erp.orders`

Show SQL Server Management Studio (SSMS) with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: `docs/images/phase04_erp_orders_schema_verified.png`. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled `erp.orders` at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 9 — `erp.order_items`

Real-world object: `order_items` in the SQL Server ERP domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to SQL Server ERP; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per sales-line identifier. Verify that every column and example row describes that same unit.

Primary key: `order_item_id`

Foreign keys: `order_id` → `orders.order_id`; `product_id` → `products.product_id`

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
<code>order_item_id</code>	BIGINT	No	Yes	—	Sales-line identifier	9100001	a whole-number type used where identifiers or counts may exceed the normal integer range. Required because the business record is incomplete without it
<code>order_id</code>	BIGINT	No	No	<code>orders.order_id</code>	Parent order	9000001	a whole-number type used where identifiers or counts may exceed the normal integer range. Required because the business record is incomplete without it
<code>line_number</code>	INT	No	No	—	Line sequence within order	1	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
product_id	INT	No	No	products.product_id	Sold product	10001	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
quantity	INT	No	No	—	Units ordered; controlled impossible values included	2	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
unit_price	DECIMAL(12,2)	No	No	—	Price per unit before discount	39.99	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it
discount_amount	DECIMAL(12,2)	No	No	—	Line discount	4.00	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it
tax_amount	DECIMAL(12,2)	No	No	—	Allocated line tax	4.56	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it
unit_cost	DECIMAL(12,2)	No	No	—	Cost per unit at sale	18.50	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
line_total_amount	DECIMAL(14,2)	No	No	—	Quantity times unit price minus discount	75.98	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it
created_at	DATETIME2	No	No	—	Source creation timestamp	2026-01-04T14:22:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
updated_at	DATETIME2	No	No	—	Source update timestamp	2026-01-04T14:22:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- order_item_id = 10000001
- order_id = 9000001
- line_number = 1
- product_id = 10026
- quantity = -2
- unit_price = 17.69
- unit_cost = 11.26
- discount_amount = 1.77
- tax_amount = 1.68
- created_at = 2026-06-25T14:48:00
- updated_at = 2026-08-15T12:00:00

Action 4.4.S9 — Create and inspect erp.order_items

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create and inspect erp.order_items. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is NorthstarRetail_ERP and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.order_items` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows NorthstarRetail_ERP.
8. LOOK FOR: No error in the Messages pane and the table under NorthstarRetail_ERP > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is NorthstarRetail_ERP and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.order_items` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows NorthstarRetail_ERP.
8. LOOK FOR: No error in the Messages pane and the table under NorthstarRetail_ERP > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: T-SQL

Exact file: sql/sqlserver/source/02_create_erp_tables.sql

Execution location: SQL Server Management Studio (SSMS), database NorthstarRetail_ERP.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE erp.order_items
(
    order_item_id    BIGINT NOT NULL CONSTRAINT PK_order_items PRIMARY KEY,
    order_id         BIGINT NOT NULL,
    line_number      INT NOT NULL,
    product_id       INT NOT NULL,
    quantity         INT NOT NULL,
    unit_price       DECIMAL(12,2) NOT NULL,
    unit_cost        DECIMAL(12,2) NOT NULL,
    discount_amount  DECIMAL(12,2) NOT NULL,
    tax_amount       DECIMAL(12,2) NOT NULL,
    created_at       DATETIME2(3) NOT NULL,
    updated_at       DATETIME2(3) NOT NULL,
    CONSTRAINT UQ_order_items_line UNIQUE (order_id, line_number),
    CONSTRAINT FK_order_items_order FOREIGN KEY (order_id) REFERENCES erp.orders(order_id),
    CONSTRAINT FK_order_items_product FOREIGN KEY (product_id) REFERENCES erp.products(product_id)
    -- No positive-quantity check: controlled impossible quantities are a Silver lesson.
);
```

Line-by-line DDL explanation

- **Line 1:** `CREATE TABLE erp.order_items` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `erp.order_items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `(` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `order_item_id BIGINT NOT NULL CONSTRAINT PK_order_items PRIMARY KEY,` — `order_item_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PK_order_items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `order_id BIGINT NOT NULL,` — `order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `line_number INT NOT NULL,` — `line_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 14:** CONSTRAINT UQ_order_items_line UNIQUE (order_id, line_number), — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UQ_order_items_line is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UNIQUE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; line_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** CONSTRAINT FK_order_items_order FOREIGN KEY (order_id) REFERENCES erp.orders(order_id), — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FK_order_items_order is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FOREIGN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** CONSTRAINT FK_order_items_product FOREIGN KEY (product_id) REFERENCES erp.products(product_id) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FK_order_items_product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FOREIGN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.products is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** -- No positive-quantity check: controlled impossible quantities are a Silver lesson. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 18:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S9.V — Verify metadata and row count for erp.order_items

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SQL Server Management Studio (SSMS) query window connected to NorthstarRetail_ERP

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM erp.order_items;
```

Where to enter it:

SQL Server Management Studio (SSMS) query window connected to NorthstarRetail_ERP. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM erp.order_items;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `erp.order_items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record `VERIFIED BY USER` and proceed without recreating it.
- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for `erp.order_items`

Show SQL Server Management Studio (SSMS) with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: `docs/images/phase04_erp_order_items_schema_verified.png`. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled `erp.order_items` at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 10 — `erp.payments`

Real-world object: payments in the SQL Server ERP domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to SQL Server ERP; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per payment identifier. Verify that every column and example row describes that same unit.

Primary key: `payment_id`

Foreign keys: `order_id` → `orders.order_id`

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
payment_id	BIGINT	No	Yes	—	Payment identifier	9200001	a whole-number type used where identifiers or counts may exceed the normal integer range. Required because the business record is incomplete without it
order_id	BIGINT	No	No	orders.order_id	Paid order	9000001	a whole-number type used where identifiers or counts may exceed the normal integer range. Required because the business record is incomplete without it
payment_timestamp	DATETIME2	No	No	—	Payment event time	2026-01-04T14:23:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
payment_method	VARCHAR(30)	No	No	—	Card, wallet, gift card, or cash	CARD	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
payment_status	VARCHAR(20)	No	No	—	Payment status	CAPTURED	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
amount	DECIMAL(14,2)	No	No	—	Payment amount	111.97	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it
currency_code	CHAR(3)	No	No	—	Payment currency	USD	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
transaction_reference	VARCHAR(80)	No	No	—	Fictional processor reference	PAY-ABC123	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
created_at	DATETIME2	No	No	—	Source creation timestamp	2026-01-04T14:23:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
updated_at	DATETIME2	No	No	—	Source update timestamp	2026-01-04T14:23:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- payment_id = 20000001
- order_id = 90000001
- payment_timestamp = 2026-06-25T14:54:00
- payment_method = CARD
- payment_status = CAPTURED
- amount = 35.29
- currency_code = usd
- transaction_reference = PAY-5E01CF81A39CE7B9
- created_at = 2026-06-25T14:48:00
- updated_at = 2026-08-15T12:00:00

Action 4.4.S10 — Create and inspect erp.payments

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create and inspect erp.payments. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is NorthstarRetail_ERP and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.payments` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows NorthstarRetail_ERP.
8. LOOK FOR: No error in the Messages pane and the table under NorthstarRetail_ERP > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is NorthstarRetail_ERP and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.payments` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows NorthstarRetail_ERP.
8. LOOK FOR: No error in the Messages pane and the table under NorthstarRetail_ERP > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: T-SQL

Exact file: sql/sqlserver/source/02_create_erp_tables.sql

Execution location: SQL Server Management Studio (SSMS), database NorthstarRetail_ERP.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE erp.payments
(
    payment_id          BIGINT NOT NULL CONSTRAINT PK_payments PRIMARY KEY,
    order_id            BIGINT NOT NULL,
    payment_timestamp   DATETIME2(3) NOT NULL,
    payment_method      VARCHAR(30) NOT NULL,
    payment_status      VARCHAR(30) NOT NULL,
    amount              DECIMAL(14,2) NOT NULL,
    currency_code       VARCHAR(3) NOT NULL,
    transaction_reference VARCHAR(80) NOT NULL,
    created_at          DATETIME2(3) NOT NULL,
    updated_at          DATETIME2(3) NOT NULL,
    CONSTRAINT UQ_payments_transaction UNIQUE (transaction_reference),
    CONSTRAINT FK_payments_order FOREIGN KEY (order_id) REFERENCES erp.orders(order_id)
);
```

Line-by-line DDL explanation

- Line 1:** `CREATE TABLE erp.payments` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `erp.payments` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `(` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `payment_id BIGINT NOT NULL CONSTRAINT PK_payments PRIMARY KEY,` — `payment_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PK_payments` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `order_id BIGINT NOT NULL,` — `order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 5:** `payment_timestamp DATETIME2(3) NOT NULL,` — `payment_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 6:** `payment_method VARCHAR(30) NOT NULL,` — `payment_method` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 14:** CONSTRAINT FK_payments_order FOREIGN KEY (order_id) REFERENCES erp.orders(order_id) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FK_payments_order is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FOREIGN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S10.V — Verify metadata and row count for erp.payments

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SQL Server Management Studio (SSMS) query window connected to NorthstarRetail_ERP

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM erp.payments;
```

Where to enter it:

SQL Server Management Studio (SSMS) query window connected to NorthstarRetail_ERP. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM erp.payments;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `erp.payments` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.

- **If the object matches exactly and was created by your verified earlier run:** Record VERIFIED BY USER and proceed without recreating it.
- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for `erp.payments`

Show SQL Server Management Studio (SSMS) with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: `docs/images/phase04_erp_payments_schema_verified.png`. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled `erp.payments` at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 11 — `erp.shipments`

Real-world object: shipments in the SQL Server ERP domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to SQL Server ERP; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per shipment identifier. Verify that every column and example row describes that same unit.

Primary key: `shipment_id`

Foreign keys: `order_id` → `orders.order_id`; `shipping_address_id` → `addresses.address_id`

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
<code>shipment_id</code>	BIGINT	No	Yes	—	Shipment identifier	9300001	a whole-number type used where identifiers or counts may exceed the normal integer range. Required because the business record is incomplete without it
<code>order_id</code>	BIGINT	No	No	<code>orders.order_id</code>	Shipped order	9000001	a whole-number type used where identifiers or counts may exceed the normal integer range. Required because the business record is incomplete without it
<code>shipping_address_id</code>	INT	Yes	No	<code>addresses.address_id</code>	Destination address	500001	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Nullable because the source can legitimately omit it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
carrier_code	VARCHAR(30)	No	No	—	Fictional carrier code	NORTHPARCEL	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
tracking_number	VARCHAR(80)	No	No	—	Synthetic tracking number	NP00000001	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
ship_timestamp	DATETIME2	Yes	No	—	Shipment departure time	2026-01-05T10:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Nullable because the source can legitimately omit it
promised_delivery_date	DATE	Yes	No	—	Customer promise date	2026-01-08	a calendar date with no time-of-day component. Nullable because the source can legitimately omit it
delivery_timestamp	DATETIME2	Yes	No	—	Actual delivery time	2026-01-07T15:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Nullable because the source can legitimately omit it
shipment_status	VARCHAR(30)	No	No	—	Shipment status	DELIVERED	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
shipping_cost	DECIMAL(12,2)	No	No	—	Internal shipping cost	7.25	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
created_at	DATETIME2	No	No	—	Source creation timestamp	2026-01-05T10:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
updated_at	DATETIME2	No	No	—	Source update timestamp	2026-01-07T15:00:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- shipment_id = 30000001
- order_id = 9000002
- carrier = National Ground
- tracking_number = NST9000002665
- ship_date = 2024-02-12
- promised_delivery_date = 2024-02-16
- actual_delivery_date = 2024-02-13
- shipment_status = DELIVERED
- shipping_cost = 7.99
- created_at = 2024-02-12T00:00:00
- updated_at = 2026-08-15T12:00:00

Action 4.4.S11 — Create and inspect erp.shipments

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create and inspect erp.shipments. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is NorthstarRetail_ERP and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open sql/sqlserver/source/02_create_erp_tables.sql in VS Code beside the database editor.
4. TYPE THIS: Build the erp.shipments definition one column and constraint at a time from the reference block below.

5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows NorthstarRetail_ERP.
8. LOOK FOR: No error in the Messages pane and the table under NorthstarRetail_ERP > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is NorthstarRetail_ERP and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open sql/sqlserver/source/02_create_erp_tables.sql in VS Code beside the database editor.
4. TYPE THIS: Build the erp.shipments definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows NorthstarRetail_ERP.
8. LOOK FOR: No error in the Messages pane and the table under NorthstarRetail_ERP > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: T-SQL

Exact file: sql/sqlserver/source/02_create_erp_tables.sql

Execution location: SQL Server Management Studio (SSMS), database NorthstarRetail_ERP.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE erp.shipments
(
    shipment_id      BIGINT NOT NULL CONSTRAINT PK_shipments PRIMARY KEY,
    order_id         BIGINT NOT NULL,
    carrier          NVARCHAR(100) NOT NULL,
    tracking_number   VARCHAR(100) NOT NULL,
    ship_date        DATE NOT NULL,
    promised_delivery_date DATE NOT NULL,
    actual_delivery_date DATE NULL,
    shipment_status  VARCHAR(30) NOT NULL,
    shipping_cost     DECIMAL(12,2) NOT NULL,
    created_at       DATETIME2(3) NOT NULL,
    updated_at       DATETIME2(3) NOT NULL,
    CONSTRAINT UQ_shipments_tracking UNIQUE (tracking_number),
    CONSTRAINT FK_shipments_order FOREIGN KEY (order_id) REFERENCES erp.orders(order_id)
);
```

Line-by-line DDL explanation

- **Line 1:** `CREATE TABLE erp.shipments` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `erp.shipments` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `(` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `shipment_id BIGINT NOT NULL CONSTRAINT PK_shipments PRIMARY KEY,` — `shipment_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PK_shipments` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `order_id BIGINT NOT NULL,` — `order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `carrier NVARCHAR(100) NOT NULL,` — `carrier` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NVARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `100` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `tracking_number VARCHAR(100) NOT NULL,` — `tracking_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `100` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `ship_date DATE NOT NULL,` — `ship_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATE` is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

tion; FK_shipments_order is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FOREIGN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 16:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S11.V — Verify metadata and row count for erp.shipments

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SQL Server Management Studio (SSMS) query window connected to NorthstarRetail_ERP

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM erp.shipments;
```

Where to enter it:

SQL Server Management Studio (SSMS) query window connected to NorthstarRetail_ERP. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM erp.shipments;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `erp.shipments` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record `VERIFIED BY USER` and proceed without recreating it.

- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for `erp.shipments`

Show SQL Server Management Studio (SSMS) with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: `docs/images/phase04_erp_shipments_schema_verified.png`. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled `erp.shipments` at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 12 — `erp.inventory_transactions`

Real-world object: `inventory_transactions` in the SQL Server ERP domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to SQL Server ERP; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per inventory movement identifier. Verify that every column and example row describes that same unit.

Primary key: `inventory_transaction_id`

Foreign keys: `store_id` → `stores.store_id`; `product_id` → `products.product_id`

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
<code>inventory_transaction_id</code>	BIGINT	No	Yes	—	Inventory movement identifier	9400001	a whole-number type used where identifiers or counts may exceed the normal integer range. Required because the business record is incomplete without it
<code>store_id</code>	INT	No	No	<code>stores.store_id</code>	Inventory location	201	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
<code>product_id</code>	INT	No	No	<code>products.product_id</code>	Inventory product	10001	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
transaction_timestamp	DATETIME2	No	No	—	Movement timestamp	2026-01-04T14:22:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
transaction_type	VARCHAR(30)	No	No	—	RECEIPT, SALE, RETURN, ADJUSTMENT, or TRANSFER	SALE	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
quantity_change	INT	No	No	—	Signed quantity change	-2	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
reference_type	VARCHAR(30)	Yes	No	—	Related object type	ORDER	text with either a stated maximum length or engine-managed variable length. Nullable because the source can legitimately omit it
reference_id	VARCHAR(60)	Yes	No	—	Related identifier	9000001	text with either a stated maximum length or engine-managed variable length. Nullable because the source can legitimately omit it
unit_cost	DECIMAL(12,2)	Yes	No	—	Cost for valuation	18.50	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Nullable because the source can legitimately omit it
created_at	DATETIME2	No	No	—	Source creation timestamp	2026-01-04T14:22:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- inventory_transaction_id = 40000001
- store_id = 201

- `product_id = 10005`
- `transaction_timestamp = 2026-07-01T00:00:00`
- `transaction_type = OPENING`
- `quantity_change = 75`
- `reference_type = SYNTHETIC`
- `reference_id = ```
- `created_at = 2026-08-15T12:00:00`

Action 4.4.S12 — Create and inspect `erp.inventory_transactions`

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Create and inspect `erp.inventory_transactions`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is `NorthstarRetail_ERP` and the query editor is not connected to `master`, `postgres`, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.inventory_transactions` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with `Ctrl+S` before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/`F5` only after the database selector shows `NorthstarRetail_ERP`.
8. LOOK FOR: No error in the Messages pane and the table under `NorthstarRetail_ERP > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is `NorthstarRetail_ERP` and the query editor is not connected to `master`, `postgres`, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.inventory_transactions` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with `Ctrl+S` before execution.

6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows NorthstarRetail_ERP.
8. LOOK FOR: No error in the Messages pane and the table under NorthstarRetail_ERP > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: T-SQL

Exact file: sql/sqlserver/source/02_create_erp_tables.sql

Execution location: SQL Server Management Studio (SSMS), database NorthstarRetail_ERP.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE erp.inventory_transactions
(
    inventory_transaction_id BIGINT NOT NULL CONSTRAINT PK_inventory_transactions PRIMARY KEY,
    store_id INT NOT NULL,
    product_id INT NOT NULL,
    transaction_timestamp DATETIME2(3) NOT NULL,
    transaction_type VARCHAR(30) NOT NULL,
    quantity_change INT NOT NULL,
    reference_type VARCHAR(30) NULL,
    reference_id BIGINT NULL,
    created_at DATETIME2(3) NOT NULL,
    CONSTRAINT FK_inventory_transactions_store FOREIGN KEY (store_id) REFERENCES erp.stores(store_id),
    CONSTRAINT FK_inventory_transactions_product FOREIGN KEY (product_id) REFERENCES erp.products(product_id)
);
```

Line-by-line DDL explanation

- **Line 1:** CREATE TABLE erp.inventory_transactions — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.inventory_transactions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

- **Line 11:** `created_at DATETIME2(3) NOT NULL`, — `created_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `CONSTRAINT FK_inventory_transactions_store FOREIGN KEY (store_id) REFERENCES erp.stores(store_id)`, — `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FK_inventory_transactions_store` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FOREIGN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `store_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `REFERENCES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `erp.stores` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `store_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `CONSTRAINT FK_inventory_transactions_product FOREIGN KEY (product_id) REFERENCES erp.products(product_id)` — `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FK_inventory_transactions_product` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FOREIGN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `product_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `REFERENCES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `erp.products` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `product_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** `);` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S12.V — Verify metadata and row count for `erp.inventory_transactions`

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SQL Server Management Studio (SSMS) query window connected to `NorthstarRetail_ERP`

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM erp.inventory_transactions;
```

Where to enter it:

SQL Server Management Studio (SSMS) query window connected to NorthstarRetail_ERP. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM erp.inventory_transactions;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `erp.inventory_transactions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record `VERIFIED BY USER` and proceed without recreating it.
- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for `erp.inventory_transactions`

Show SQL Server Management Studio (SSMS) with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: `docs/images/phase04_erp_inventory_transactions_schema_verified.png`. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled `erp.inventory_transactions` at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 13 — `erp.inventory_snapshots`

Real-world object: `inventory_snapshots` in the SQL Server ERP domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to SQL Server ERP; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per snapshot row identifier. Verify that every column and example row describes that same unit.

Primary key: `inventory_snapshot_id`

Foreign keys: `store_id` → `stores.store_id`; `product_id` → `products.product_id`

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
inventory_snapshot_id	BIGINT	No	Yes	—	Snapshot row identifier	9500001	a whole-number type used where identifiers or counts may exceed the normal integer range. Required because the business record is incomplete without it
snapshot_date	DATE	No	No	—	Snapshot date	2026-01-31	a calendar date with no time-of-day component. Required because the business record is incomplete without it
store_id	INT	No	No	stores.store_id	Inventory location	201	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
product_id	INT	No	No	products.product_id	Inventory product	10001	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
on_hand_quantity	INT	No	No	—	Units physically on hand	42	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
reserved_quantity	INT	No	No	—	Units reserved for open orders	3	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it
reorder_point	INT	No	No	—	Threshold for replenishment	10	a whole-number type for identifiers, counts, flags represented as numbers, or day values. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
source_updated_at	DATETIME2	No	No	—	Source calculation time	2026-01-31T23:59:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
created_at	DATETIME2	No	No	—	Source creation timestamp	2026-02-01T00:05:00	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- inventory_snapshot_id = 50000001
- snapshot_date = 2026-07-31
- store_id = 201
- product_id = 10005
- quantity_on_hand = 88
- quantity_reserved = 8
- reorder_point = 13
- created_at = 2026-08-15T12:00:00

Action 4.4.S13 — Create and inspect `erp.inventory_snapshots`

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Create and inspect `erp.inventory_snapshots`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is `NorthstarRetail_ERP` and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.inventory_snapshots` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with `Ctrl+S` before execution.

6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows NorthstarRetail_ERP.
8. LOOK FOR: No error in the Messages pane and the table under NorthstarRetail_ERP > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **SQL Server Management Studio (SSMS)**.
2. LOOK FOR: Confirm the active connection is NorthstarRetail_ERP and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/sqlserver/source/02_create_erp_tables.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `erp.inventory_snapshots` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows NorthstarRetail_ERP.
8. LOOK FOR: No error in the Messages pane and the table under NorthstarRetail_ERP > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: T-SQL

Exact file: sql/sqlserver/source/02_create_erp_tables.sql

Execution location: SQL Server Management Studio (SSMS), database NorthstarRetail_ERP.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE erp.inventory_snapshots
(
    inventory_snapshot_id BIGINT NOT NULL CONSTRAINT PK_inventory_snapshots PRIMARY KEY,
    snapshot_date         DATE NOT NULL,
    store_id              INT NOT NULL,
    product_id            INT NOT NULL,
    quantity_on_hand      INT NOT NULL,
    quantity_reserved     INT NOT NULL,
    reorder_point         INT NOT NULL,
    created_at            DATETIME2(3) NOT NULL,
    CONSTRAINT UQ_inventory_snapshot_grain UNIQUE (snapshot_date, store_id, product_id),
    CONSTRAINT FK_inventory_snapshots_store FOREIGN KEY (store_id) REFERENCES erp.stores(store_id),
    CONSTRAINT FK_inventory_snapshots_product FOREIGN KEY (product_id) REFERENCES erp.products(product_id)
);
```

Line-by-line DDL explanation

- Line 1:** CREATE TABLE erp.inventory_snapshots — CREATE defines a new database object; TABLE the database object that stores rows and columns; erp.inventory_snapshots is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** inventory_snapshot_id BIGINT NOT NULL CONSTRAINT PK_inventory_snapshots PRIMARY KEY, — inventory_snapshot_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BIGINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PK_inventory_snapshots is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** snapshot_date DATE NOT NULL, — snapshot_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 5:** store_id INT NOT NULL, — store_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 6:** product_id INT NOT NULL, — product_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 7:** quantity_on_hand INT NOT NULL, — quantity_on_hand is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 8:** quantity_reserved INT NOT NULL, — quantity_reserved is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition;

NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

- **Line 14:** `) ; — )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S13.V — Verify metadata and row count for `erp.inventory_snapshots`

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SQL Server Management Studio (SSMS) query window connected to `NorthstarRetail_ERP`

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM erp.inventory_snapshots;
```

Where to enter it:

SQL Server Management Studio (SSMS) query window connected to `NorthstarRetail_ERP`. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM erp.inventory_snapshots;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `erp.inventory_snapshots` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record `VERIFIED BY USER` and proceed without recreating it.
- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for `erp.inventory_snapshots`

Show SQL Server Management Studio (SSMS) with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: `docs/images/phase04_erp_inventory_snapshots_schema_verified.png`. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled `erp.inventory_snapshots` at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 14 — digital.web_users

Real-world object: web_users in the PostgreSQL Digital domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to PostgreSQL Digital; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per digital user identifier. Verify that every column and example row describes that same unit.

Primary key: web_user_id

Foreign keys: No foreign key; this object is a parent/reference table or independent event table.

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
web_user_id	BIGINT	No	Yes	—	Digital user identifier	700001	a whole-number type used where identifiers or counts may exceed the normal integer range. Required because the business record is incomplete without it
web_user_key	VARCHAR(40)	No	No	—	Digital business key	WU-700001	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
customer_id	INTEGER	Yes	No	—	Loose ERP customer reference	100001	the storage type selected for the business meaning and source-system behavior of this column. Nullable because the source can legitimately omit it
email	VARCHAR(255)	Yes	No	—	Synthetic login email	visitor@example.test	text with either a stated maximum length or engine-managed variable length. Nullable because the source can legitimately omit it
registration_timestamp	TIMESTAMPTZ	No	No	—	Registration time	2026-01-01T12:00:00Z	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
country_code	VARCHAR(10)	No	No	—	Country code	US	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
marketing_consent	BOOLEAN	No	No	—	Marketing consent	true	a true/false flag. Required because the business record is incomplete without it
created_at	TIMESTAMPZ	No	No	—	Source creation time	2026-01-01T12:00:00Z	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
updated_at	TIMESTAMPZ	No	No	—	Source update time	2026-01-01T12:00:00Z	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- web_user_id = 700001
- customer_id = 100051.0
- anonymous_cookie_id = f5d63f1e27baff0434521fc83e64e93b
- email_hash = 3120ff119c3b5dfdf8a1a71723efccd7f3ae150e90c6dd78d0dfba39c6848457
- created_at = 2023-08-16T00:00:00
- updated_at = 2026-08-15T12:00:00

Action 4.4.S14 — Create and inspect digital.web_users

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create and inspect digital.web_users. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **pgAdmin 4 Query Tool**.
2. LOOK FOR: Confirm the active connection is northstar_digital and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open sql/postgres/source/01_create_digital_schema.sql in VS Code beside the database editor.
4. TYPE THIS: Build the digital.web_users definition one column and constraint at a time from the reference block below.

5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows northstar_digital.
8. LOOK FOR: No error in the Messages pane and the table under northstar_digital > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL psql, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **pgAdmin 4 Query Tool**.
2. LOOK FOR: Confirm the active connection is northstar_digital and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open sql/postgres/source/01_create_digital_schema.sql in VS Code beside the database editor.
4. TYPE THIS: Build the digital.web_users definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows northstar_digital.
8. LOOK FOR: No error in the Messages pane and the table under northstar_digital > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

DDL section — build and explain it before execution

Language: PostgreSQL SQL

Exact file: sql/postgres/source/01_create_digital_schema.sql

Execution location: pgAdmin 4 Query Tool, database northstar_digital.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE IF NOT EXISTS digital.web_users
(
    web_user_id          bigint PRIMARY KEY,
    customer_id          integer NULL,
    anonymous_cookie_id  varchar(64) NOT NULL,
    email_hash           char(64) NULL,
    created_at           timestamptz NOT NULL,
    updated_at           timestamptz NOT NULL,
    CONSTRAINT uq_web_users_cookie UNIQUE (anonymous_cookie_id)
);
```

Line-by-line DDL explanation

- **Line 1:** `CREATE TABLE IF NOT EXISTS digital.web_users` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `IF` begins a conditional decision; `NOT` reverses a Boolean condition; `EXISTS` tests whether a matching object or row is present; `digital.web_users` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `(` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `web_user_id bigint PRIMARY KEY,` — `web_user_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bigint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `customer_id integer NULL,` — `customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `integer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `anonymous_cookie_id varchar(64) NOT NULL,` — `anonymous_cookie_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `varchar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `64` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `email_hash char(64) NULL,` — `email_hash` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `char` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `64` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `created_at timestamptz NOT NULL,` — `created_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestamptz` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `updated_at timestamptz NOT NULL,` — `updated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestamptz` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition;

NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 9:** CONSTRAINT uq_web_users_cookie UNIQUE (anonymous_cookie_id) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; uq_web_users_cookie is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UNIQUE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; anonymous_cookie_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S14.V — Verify metadata and row count for digital.web_users

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

pgAdmin 4 Query Tool query window connected to northstar_digital

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM digital.web_users;
```

Where to enter it:

pgAdmin 4 Query Tool query window connected to northstar_digital. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM digital.web_users;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `digital.web_users` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record `VERIFIED BY USER` and proceed without recreating it.
- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for digital.web_users

Show pgAdmin 4 Query Tool with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: docs/images/phase04_digital_web_users_schema_verified.png. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled digital.web_users at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 15 — digital.campaigns

Real-world object: campaigns in the PostgreSQL Digital domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to PostgreSQL Digital; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per campaign identifier. Verify that every column and example row describes that same unit.

Primary key: campaign_id

Foreign keys: No foreign key; this object is a parent/reference table or independent event table.

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
campaign_id	INTEGER	No	Yes	—	Campaign identifier	801	the storage type selected for the business meaning and source-system behavior of this column. Required because the business record is incomplete without it
campaign_code	VARCHAR(40)	No	No	—	Campaign business key	CMP-0801	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
campaign_name	VARCHAR(150)	No	No	—	Campaign name	Winter Essentials	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
channel	VARCHAR(30)	No	No	—	Marketing channel	PAID_SEARCH	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
start_date	DATE	No	No	—	Campaign start	2026-01-01	a calendar date with no time-of-day component. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
end_date	DATE	No	No	—	Campaign end	2026-01-31	a calendar date with no time-of-day component. Required because the business record is incomplete without it
budget_amount	NUMERIC(14,2)	No	No	—	Planned spend	2500.00	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it
currency_code	CHAR(3)	No	No	—	Budget currency	USD	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
objective	VARCHAR(60)	No	No	—	Campaign objective	CONVERSION	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
created_at	TIMESTAMPZ	No	No	—	Source creation time	2025-12-15T12:00:00Z	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
updated_at	TIMESTAMPZ	No	No	—	Source update time	2025-12-15T12:00:00Z	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- campaign_id = 8001
- campaign_code = CMP-08001
- campaign_name = Seasonal Campaign 1
- channel = Email
- start_date = 2025-10-02
- end_date = 2025-11-16
- budget_amount = 18670.77
- currency_code = USD
- created_at = 2025-10-02T00:00:00

Action 4.4.S15 — Create and inspect digital.campaigns

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create and inspect digital.campaigns. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **pgAdmin 4 Query Tool**.
2. LOOK FOR: Confirm the active connection is northstar_digital and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open sql/postgres/source/01_create_digital_schema.sql in VS Code beside the database editor.
4. TYPE THIS: Build the digital.campaigns definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows northstar_digital.
8. LOOK FOR: No error in the Messages pane and the table under northstar_digital > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL psql, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **pgAdmin 4 Query Tool**.
2. LOOK FOR: Confirm the active connection is northstar_digital and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open sql/postgres/source/01_create_digital_schema.sql in VS Code beside the database editor.
4. TYPE THIS: Build the digital.campaigns definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows northstar_digital.
8. LOOK FOR: No error in the Messages pane and the table under northstar_digital > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: PostgreSQL SQL

Exact file: sql/postgres/source/01_create_digital_schema.sql

Execution location: pgAdmin 4 Query Tool, database northstar_digital.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE IF NOT EXISTS digital.campaigns
(
    campaign_id      integer PRIMARY KEY,
    campaign_code    varchar(30) NOT NULL UNIQUE,
    campaign_name     varchar(200) NOT NULL,
    channel          varchar(40) NOT NULL,
    start_date       date NOT NULL,
    end_date         date NOT NULL,
    budget_amount    numeric(14,2) NOT NULL,
    currency_code     varchar(3) NOT NULL,
    created_at       timestampz NOT NULL,
    updated_at       timestampz NOT NULL,
    CONSTRAINT ck_campaign_dates CHECK (end_date >= start_date)
);
```

Line-by-line DDL explanation

- **Line 1:** CREATE TABLE IF NOT EXISTS digital.campaigns — CREATE defines a new database object; TABLE the database object that stores rows and columns; IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; digital.campaigns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** campaign_id integer PRIMARY KEY, — campaign_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; integer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, ar-

gument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

er, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 12:** updated_at timestamptz NOT NULL, — updated_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timestamptz is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** CONSTRAINT ck_campaign_dates CHECK (end_date >= start_date) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ck_campaign_dates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CHECK is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; end_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; >= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; start_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S15.V — Verify metadata and row count for digital.campaigns

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

pgAdmin 4 Query Tool query window connected to northstar_digital

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM digital.campaigns;
```

Where to enter it:

pgAdmin 4 Query Tool query window connected to northstar_digital. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM digital.campaigns;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `digital.campaigns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record VERIFIED BY USER and proceed without recreating it.
- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for digital.campaigns

Show pgAdmin 4 Query Tool with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: docs/images/phase04_digital_campaigns_schema_verified.png. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled digital.campaigns at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 16 — digital.web_sessions

Real-world object: web_sessions in the PostgreSQL Digital domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to PostgreSQL Digital; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per session identifier. Verify that every column and example row describes that same unit.

Primary key: session_id

Foreign keys: web_user_id → web_users.web_user_id; campaign_id → campaigns.campaign_id

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
session_id	BIGINT	No	Yes	—	Session identifier	7100001	a whole-number type used where identifiers or counts may exceed the normal integer range. Required because the business record is incomplete without it
session_key	VARCHAR(50)	No	No	—	Session business key; controlled duplicates included	SES-07100001	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
web_user_id	BIGINT	Yes	No	web_users.web_user_id	Known digital user	700001	a whole-number type used where identifiers or counts may exceed the normal integer range. Nullable because the source can legitimately omit it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
session_start	TIMESTAMPZ	No	No	—	Session start time	2026-01-04T14:00:00Z	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
session_end	TIMESTAMPZ	Yes	No	—	Session end time	2026-01-04T14:17:00Z	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Nullable because the source can legitimately omit it
device_type	VARCHAR(30)	No	No	—	Desktop, mobile, or tablet	MOBILE	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
traffic_source	VARCHAR(50)	No	No	—	Acquisition source	google	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
campaign_id	INTEGER	Yes	No	campaigns.campaign_id	Attributed campaign	801	the storage type selected for the business meaning and source-system behavior of this column. Nullable because the source can legitimately omit it
landing_page	VARCHAR(300)	No	No	—	First page	/category/electronics	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
converted_order_id	BIGINT	Yes	No	—	Loose ERP order reference	9000001	a whole-number type used where identifiers or counts may exceed the normal integer range. Nullable because the source can legitimately omit it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
created_at	TIMESTAMPZ	No	No	—	Source creation time	2026-01-04T14:00:00Z	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- session_id = 1000001
- web_user_id = 700047
- session_start_utc = 2025-08-23T04:30:18-04:00
- session_end_utc = 2025-08-23T08:30:48+00:00
- source_channel = Direct
- campaign_id = ``
- device_type = mobile
- country_code = US
- converted_order_id = ``
- created_at = 2026-08-15T12:00:00+00:00

Action 4.4.S16 — Create and inspect digital.web_sessions

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create and inspect digital.web_sessions. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **pgAdmin 4 Query Tool**.
2. LOOK FOR: Confirm the active connection is northstar_digital and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open sql/postgres/source/01_create_digital_schema.sql in VS Code beside the database editor.
4. TYPE THIS: Build the digital.web_sessions definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows northstar_digital.
8. LOOK FOR: No error in the Messages pane and the table under northstar_digital > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL `psql`, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **pgAdmin 4 Query Tool**.
2. LOOK FOR: Confirm the active connection is `northstar_digital` and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/postgres/source/01_create_digital_schema.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `digital.web_sessions` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows `northstar_digital`.
8. LOOK FOR: No error in the Messages pane and the table under `northstar_digital > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: PostgreSQL SQL

Exact file: `sql/postgres/source/01_create_digital_schema.sql`

Execution location: pgAdmin 4 Query Tool, database `northstar_digital`.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.


```
CREATE TABLE IF NOT EXISTS digital.web_sessions
(
  session_id          bigint PRIMARY KEY,
  web_user_id         bigint NULL REFERENCES digital.web_users(web_user_id),
  session_start_utc   timestampz NOT NULL,
  session_end_utc     timestampz NULL,
  source_channel       varchar(40) NOT NULL,
  campaign_id         integer NULL REFERENCES digital.campaigns(campaign_id),
  device_type         varchar(30) NOT NULL,
  country_code        varchar(10) NOT NULL,
  converted_order_id  bigint NULL,
  created_at          timestampz NOT NULL,
  CONSTRAINT ck_session_end CHECK (session_end_utc IS NULL OR session_end_utc >= session_start_utc)
);
```

Line-by-line DDL explanation

- **Line 1:** `CREATE TABLE IF NOT EXISTS digital.web_sessions` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `IF` begins a conditional decision; `NOT` reverses a Boolean condition; `EXISTS` tests whether a matching object or row is present; `digital.web_sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `(` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `session_id bigint PRIMARY KEY,` — `session_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bigint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `web_user_id bigint NULL REFERENCES digital.web_users(web_user_id),` — `web_user_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bigint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `REFERENCES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `digital.web_users` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `web_user_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `session_start_utc timestampz NOT NULL,` — `session_start_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestampz` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `session_end_utc timestampz NULL,` — `session_end_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestampz` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `source_channel varchar(40) NOT NULL,` — `source_channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `varchar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `40` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `campaign_id integer NULL REFERENCES digital.campaigns(campaign_id),` — `campaign_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `integer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `REFERENCES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `digital.campaigns` is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; campaign_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 9:** device_type varchar(30) NOT NULL, — device_type is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; varchar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** country_code varchar(10) NOT NULL, — country_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; varchar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 10 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** converted_order_id bigint NULL, — converted_order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bigint is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** created_at timestamptz NOT NULL, — created_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timestamptz is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** CONSTRAINT ck_session_end CHECK (session_end_utc IS NULL OR session_end_utc >= session_start_utc) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ck_session_end is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CHECK is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; session_end_utc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; OR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; session_end_utc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; >= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; session_start_utc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S16.V — Verify metadata and row count for digital.web_sessions

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

pgAdmin 4 Query Tool query window connected to northstar_digital

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM digital.web_sessions;
```

Where to enter it:

pgAdmin 4 Query Tool query window connected to northstar_digital. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM digital.web_sessions;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `digital.web_sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record `VERIFIED BY USER` and proceed without recreating it.
- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for `digital.web_sessions`

Show pgAdmin 4 Query Tool with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: `docs/images/phase04_digital_web_sessions_schema_verified.png`. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled `digital.web_sessions` at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 17 — `digital.web_events`

Real-world object: `web_events` in the PostgreSQL Digital domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to PostgreSQL Digital; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per event row identifier. Verify that every column and example row describes that same unit.

Primary key: `event_id`

Foreign keys: `session_id` → `web_sessions.session_id`

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
event_id	BIGINT	No	Yes	—	Event row identifier	7200001	a whole-number type used where identifiers or counts may exceed the normal integer range. Required because the business record is incomplete without it
event_key	VARCHAR(50)	No	No	—	Event business key; controlled duplicates included	EVT-07200001	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
session_id	BIGINT	No	No	web_sessions.session_id	Parent session	7100001	a whole-number type used where identifiers or counts may exceed the normal integer range. Required because the business record is incomplete without it
event_timestamp	TIMESTAMPZ	No	No	—	Event time	2026-01-04T14:02:00Z	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
event_type	VARCHAR(40)	No	No	—	PAGE_VIEW, PRODUCT_VIEW, ADD_TO_CART, CHECKOUT, PURCHASE	PRODUCT_VIEW	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
page_url	VARCHAR(500)	Yes	No	—	Page URL	/product/NRT-ELE-10001	text with either a stated maximum length or engine-managed variable length. Nullable because the source can legitimately omit it
product_id	INTEGER	Yes	No	—	Loose ERP product reference	10001	the storage type selected for the business meaning and source-system behavior of this column. Nullable because the source can legitimately omit it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
order_id	BIGINT	Yes	No	—	Loose ERP order reference	9000001	a whole-number type used where identifiers or counts may exceed the normal integer range. Nullable because the source can legitimately omit it
event_properties	JSONB	No	No	—	Flexible event payload	{"position":1}	structured JSON content that can preserve nested attributes. Required because the business record is incomplete without it
created_at	TIMESTAMPTZ	No	No	—	Source creation time	2026-01-04T14:02:01Z	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- event_id = 2000001
- session_id = 1000001
- event_timestamp = 2020-01-01T00:00:00+00:00
- event_type = page_view
- page_url = https://northstar.example/page-view
- product_id = ``
- order_id = ``
- event_properties = {"synthetic": true, "sequence": 0}
- created_at = 2026-08-15T12:00:00+00:00

Action 4.4.S17 — Create and inspect digital.web_events

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create and inspect digital.web_events. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **pgAdmin 4 Query Tool**.

2. LOOK FOR: Confirm the active connection is `northstar_digital` and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/postgres/source/01_create_digital_schema.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `digital.web_events` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows `northstar_digital`.
8. LOOK FOR: No error in the Messages pane and the table under `northstar_digital > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL psql, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **pgAdmin 4 Query Tool**.
2. LOOK FOR: Confirm the active connection is `northstar_digital` and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open `sql/postgres/source/01_create_digital_schema.sql` in VS Code beside the database editor.
4. TYPE THIS: Build the `digital.web_events` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows `northstar_digital`.
8. LOOK FOR: No error in the Messages pane and the table under `northstar_digital > Tables` after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: PostgreSQL SQL

Exact file: sql/postgres/source/01_create_digital_schema.sql

Execution location: pgAdmin 4 Query Tool, database northstar_digital.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE IF NOT EXISTS digital.web_events
(
    event_id          bigint PRIMARY KEY,
    session_id        bigint NOT NULL REFERENCES digital.web_sessions(session_id),
    event_timestamp    timestampz NOT NULL,
    event_type         varchar(50) NOT NULL,
    page_url           text NULL,
    product_id         integer NULL,
    order_id           bigint NULL,
    event_properties    jsonb NULL,
    created_at         timestampz NOT NULL
);
```

Line-by-line DDL explanation

- Line 1:** `CREATE TABLE IF NOT EXISTS digital.web_events` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `IF` begins a conditional decision; `NOT` reverses a Boolean condition; `EXISTS` tests whether a matching object or row is present; `digital.web_events` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `(` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `event_id bigint PRIMARY KEY,` — `event_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bigint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `session_id bigint NOT NULL REFERENCES digital.web_sessions(session_id),` — `session_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bigint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `REFERENCES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `digital.web_sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `session_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `),` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 5:** `event_timestamp timestampz NOT NULL,` — `event_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestampz` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 6:** `event_type varchar(50) NOT NULL,` — `event_type` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `varchar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `50` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** `page_url text NULL`, — `page_url` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `product_id integer NULL`, — `product_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `integer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `order_id bigint NULL`, — `order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bigint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `event_properties jsonb NULL`, — `event_properties` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `jsonb` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** `created_at timestamptz NOT NULL` — `created_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestamptz` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `);` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S17.V — Verify metadata and row count for `digital.web_events`

Application:

pgAdmin 4 or PostgreSQL `psql`, as named in the action

Window or panel:

pgAdmin 4 Query Tool query window connected to `northstar_digital`

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM digital.web_events;
```

Where to enter it:

pgAdmin 4 Query Tool query window connected to northstar_digital. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM digital.web_events;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `digital.web_events` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record VERIFIED BY USER and proceed without recreating it.
- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for digital.web_events

Show pgAdmin 4 Query Tool with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: docs/images/phase04_digital_web_events_schema_verified.png. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled digital.web_events at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 18 — digital.campaign_touchpoints

Real-world object: campaign touchpoints in the PostgreSQL Digital domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to PostgreSQL Digital; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per marketing touchpoint identifier. Verify that every column and example row describes that same unit.

Primary key: touchpoint_id

Foreign keys: campaign_id → campaigns.campaign_id; web_user_id → web_users.web_user_id; session_id → web_sessions.session_id

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
touchpoint_id	BIGINT	No	Yes	—	Marketing touchpoint identifier	7300001	a whole-number type used where identifiers or counts may exceed the normal integer range. Required because the business record is incomplete without it
campaign_id	INTEGER	No	No	campaigns.campaign_id	Campaign	801	the storage type selected for the business meaning and source-system behavior of this column. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
web_user_id	BIGINT	Yes	No	web_users.web_user_id	Known user	700001	a whole-number type used where identifiers or counts may exceed the normal integer range. Nullable because the source can legitimately omit it
session_id	BIGINT	Yes	No	web_sessions.session_id	Related session	7100001	a whole-number type used where identifiers or counts may exceed the normal integer range. Nullable because the source can legitimately omit it
touchpoint_timestamp	TIMESTAMPZ	No	No	—	Touchpoint time	2026-01-04T13:58:00Z	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it
touchpoint_type	VARCHAR(30)	No	No	—	IMPRESSION, CLICK, EMAIL_OPEN, or EMAIL_CLICK	CLICK	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
cost_amount	NUMERIC(12,4)	No	No	—	Allocated cost	0.8500	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it
currency_code	CHAR(3)	No	No	—	Cost currency	USD	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
created_at	TIMESTAMPZ	No	No	—	Source creation time	2026-01-04T13:58:01Z	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- touchpoint_id = 3000001

- web_user_id = 700079
- session_id = 1000002
- campaign_id = 8008
- touchpoint_timestamp = 2025-07-20T11:58:52+00:00
- touchpoint_type = click
- attribution_weight = 1.0
- created_at = 2026-08-15T12:00:00+00:00

Action 4.4.S18 — Create and inspect digital.campaign_touchpoints

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create and inspect digital.campaign_touchpoints. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **pgAdmin 4 Query Tool**.
2. LOOK FOR: Confirm the active connection is northstar_digital and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open sql/postgres/source/01_create_digital_schema.sql in VS Code beside the database editor.
4. TYPE THIS: Build the digital.campaign_touchpoints definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows northstar_digital.
8. LOOK FOR: No error in the Messages pane and the table under northstar_digital > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL psql, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **pgAdmin 4 Query Tool**.
2. LOOK FOR: Confirm the active connection is northstar_digital and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open sql/postgres/source/01_create_digital_schema.sql in VS Code beside the database editor.
4. TYPE THIS: Build the digital.campaign_touchpoints definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.

6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows northstar_digital.
8. LOOK FOR: No error in the Messages pane and the table under northstar_digital > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: PostgreSQL SQL

Exact file: sql/postgres/source/01_create_digital_schema.sql

Execution location: pgAdmin 4 Query Tool, database northstar_digital.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE IF NOT EXISTS digital.campaign_touchpoints
(
    touchpoint_id      bigint PRIMARY KEY,
    web_user_id        bigint NULL REFERENCES digital.web_users(web_user_id),
    session_id         bigint NULL REFERENCES digital.web_sessions(session_id),
    campaign_id        integer NOT NULL REFERENCES digital.campaigns(campaign_id),
    touchpoint_timestamp timestampz NOT NULL,
    touchpoint_type     varchar(40) NOT NULL,
    attribution_weight  numeric(8,6) NOT NULL,
    created_at         timestampz NOT NULL,
    CONSTRAINT ck_touchpoint_weight CHECK (attribution_weight >= 0 AND attribution_weight <= 1)
);
```

Line-by-line DDL explanation

- **Line 1:** CREATE TABLE IF NOT EXISTS digital.campaign_touchpoints — CREATE defines a new database object; TABLE the database object that stores rows and columns; IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; digital.campaign_touchpoints is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 10:** created_at timestampz NOT NULL, — created_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timestampz is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** CONSTRAINT ck_touchpoint_weight CHECK (attribution_weight >= 0 AND attribution_weight <= 1) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ck_touchpoint_weight is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CHECK is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; attribution_weight is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; >= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; AND is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; attribution_weight is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; <= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S18.V — Verify metadata and row count for digital.campaign_touchpoints

Application:

pgAdmin 4 or PostgreSQL psqL, as named in the action

Window or panel:

pgAdmin 4 Query Tool query window connected to northstar_digital

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psqL, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.

6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM digital.campaign_touchpoints;
```

Where to enter it:

pgAdmin 4 Query Tool query window connected to northstar_digital. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM digital.campaign_touchpoints;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `digital.campaign_touchpoints` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record VERIFIED BY USER and proceed without recreating it.
- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for digital.campaign_touchpoints

Show pgAdmin 4 Query Tool with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: docs/images/phase04_digital_campaign_touchpoints_schema_verified.png. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled digital.campaign_touchpoints at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

Source object 19 — digital.marketing_spend

Real-world object: marketing_spend in the PostgreSQL Digital domain.

Why it belongs in this source: The object is generated and operated with the transaction or digital behavior assigned to PostgreSQL Digital; the placement creates a realistic heterogeneous integration boundary.

Declared grain: one row per spend row identifier. Verify that every column and example row describes that same unit.

Primary key: marketing_spend_id

Foreign keys: campaign_id → campaigns.campaign_id

Index purpose: Primary, unique, foreign-key, timestamp, and business-key indexes support integrity or extraction access patterns; do not add indexes without a query reason.

Column-by-column design

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
marketing_spend_id	BIGINT	No	Yes	—	Spend row identifier	7400001	a whole-number type used where identifiers or counts may exceed the normal integer range. Required because the business record is incomplete without it
spend_date	DATE	No	No	—	Spend date	2026-01-04	a calendar date with no time-of-day component. Required because the business record is incomplete without it

Column	Data type	Null?	PK	FK	Business definition	Example	Why the type/constraint fits
campaign_id	INTEGER	No	No	campaigns.campaign_id	Campaign	801	the storage type selected for the business meaning and source-system behavior of this column. Required because the business record is incomplete without it
channel	VARCHAR(30)	No	No	—	Marketing channel	PAID_SEARCH	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
spend_amount	NUMERIC(14,2)	No	No	—	Actual spend	125.40	an exact fixed-precision number suitable for currency or ratios where binary floating-point rounding is undesirable. Required because the business record is incomplete without it
currency_code	CHAR(3)	No	No	—	Spend currency	USD	text with either a stated maximum length or engine-managed variable length. Required because the business record is incomplete without it
impressions	INTEGER	No	No	—	Delivered impressions	25000	the storage type selected for the business meaning and source-system behavior of this column. Required because the business record is incomplete without it
clicks	INTEGER	No	No	—	Clicks	680	the storage type selected for the business meaning and source-system behavior of this column. Required because the business record is incomplete without it
created_at	TIMESTAMPZ	No	No	—	Source creation time	2026-01-05T02:00:00Z	a date-and-time value used for event time, creation time, update time, or incremental watermarks. Required because the business record is incomplete without it

Realistic example record

- spend_id = 4000001
- spend_date = 2025-10-02
- campaign_id = 8001
- channel = Email
- spend_amount = 678.34
- impressions = 25257
- clicks = 1259
- currency_code = USD
- created_at = 2026-08-15T12:00:00+00:00

Action 4.4.S19 — Create and inspect digital.marketing_spend

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create and inspect digital.marketing_spend. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open **pgAdmin 4 Query Tool**.
2. LOOK FOR: Confirm the active connection is northstar_digital and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open sql/postgres/source/01_create_digital_schema.sql in VS Code beside the database editor.
4. TYPE THIS: Build the digital.marketing_spend definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows northstar_digital.
8. LOOK FOR: No error in the Messages pane and the table under northstar_digital > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL psql, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open **pgAdmin 4 Query Tool**.
2. LOOK FOR: Confirm the active connection is northstar_digital and the query editor is not connected to master, postgres, or another database.
3. CLICK: Open sql/postgres/source/01_create_digital_schema.sql in VS Code beside the database editor.

4. TYPE THIS: Build the `digital.marketing_spend` definition one column and constraint at a time from the reference block below.
5. PRESS: Save the source file with Ctrl+S before execution.
6. CLICK: Copy only this table's current verified DDL into a new query window or use the phase's full ordered script after every table section is understood.
7. PRESS: Execute/F5 only after the database selector shows `northstar_digital`.
8. LOOK FOR: No error in the Messages pane and the table under `northstar_digital` > Tables after Refresh.
9. CLICK: Expand Columns, Keys, Constraints, and Indexes to compare them with the design table.
10. DO NOT CONTINUE UNTIL: A metadata query confirms the column names and types.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

DDL section — build and explain it before execution

Language: PostgreSQL SQL

Exact file: `sql/postgres/source/01_create_digital_schema.sql`

Execution location: pgAdmin 4 Query Tool, database `northstar_digital`.

Input: The requirements and column design above.

Output: One table plus its constraints and indexes when the statement is run in the correct database.

```
CREATE TABLE IF NOT EXISTS digital.marketing_spend
(
    spend_id          bigint PRIMARY KEY,
    spend_date        date NOT NULL,
    campaign_id       integer NOT NULL REFERENCES digital.campaigns(campaign_id),
    channel           varchar(40) NOT NULL,
    spend_amount      numeric(14,2) NOT NULL,
    impressions       bigint NOT NULL,
    clicks            bigint NOT NULL,
    currency_code     varchar(3) NOT NULL,
    created_at        timestamptz NOT NULL,
    CONSTRAINT uq_marketing_spend_grain UNIQUE (spend_date, campaign_id, channel),
    CONSTRAINT ck_spend_nonnegative CHECK (spend_amount >= 0 AND impressions >= 0 AND clicks >= 0)
);
```

Line-by-line DDL explanation

- **Line 1:** CREATE TABLE IF NOT EXISTS digital.marketing_spend — CREATE defines a new database object; TABLE the database object that stores rows and columns; IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; digital.marketing_spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** spend_id bigint PRIMARY KEY, — spend_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bigint is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** spend_date date NOT NULL, — spend_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** campaign_id integer NOT NULL REFERENCES digital.campaigns(campaign_id), — campaign_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; integer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; digital.campaigns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; campaign_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** channel varchar(40) NOT NULL, — channel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; varchar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 40 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** spend_amount numeric(14,2) NOT NULL, — spend_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; numeric is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 14 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** impressions bigint NOT NULL, — impressions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bigint is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** clicks bigint NOT NULL, — clicks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bigint is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 10:** `currency_code varchar(3) NOT NULL`, — `currency_code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `varchar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** `created_at timestamptz NOT NULL`, — `created_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestamptz` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `CONSTRAINT uq_marketing_spend_grain UNIQUE (spend_date, campaign_id, channel)`, — `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `uq_marketing_spend_grain` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UNIQUE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spend_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `campaign_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `CONSTRAINT ck_spend_nonnegative CHECK (spend_amount >= 0 AND impressions >= 0 AND clicks >= 0)` — `CONSTRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ck_spend_nonnegative` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CHECK` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spend_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `AND` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `impressions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `AND` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 14:** `);` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Verification query

Action 4.4.S19.V — Verify metadata and row count for `digital.marketing_spend`

Application:

pgAdmin 4 or PostgreSQL `psql`, as named in the action

Window or panel:

pgAdmin 4 Query Tool query window connected to `northstar_digital`

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4** or **PostgreSQL psql**, as named in the action to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (SQL):

```
SELECT COUNT(*) AS row_count FROM digital.marketing_spend;
```

Where to enter it:

pgAdmin 4 Query Tool query window connected to northstar_digital. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT(*) AS row_count FROM digital.marketing_spend;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `digital.marketing_spend` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use

`git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Object-already-exists branch

- **If the script reports that the object already exists:** Do not drop it automatically. Inspect its columns, keys, constraints, and origin. Compare with Git history and your journal.
- **If the object matches exactly and was created by your verified earlier run:** Record `VERIFIED BY USER` and proceed without recreating it.
- **If it differs:** Script or document the existing definition, preserve evidence, and decide whether to migrate it or use the documented development reset only after confirming that no needed data will be lost.
- **If the table is empty:** That is expected before Phase 6. Do not treat zero rows as a schema failure.
- **If rows already exist:** Record the count and source before any load; never silently append to an unknown baseline.

Screenshot checkpoint for `digital.marketing_spend`

Show pgAdmin 4 Query Tool with Object Explorer/server tree, the expanded table columns, and the successful metadata query. Hide login names, server addresses beyond localhost, and credentials. Recommended filename: `docs/images/phase04_digital_marketing_spend_schema_verified.png`. This screenshot proves the definition you inspected, not that later data loads succeeded.

Interview-ready explanation

"I modeled `digital.marketing_spend` at the declared grain, selected types and nullability from business meaning, enforced stable identifiers and relationships, and verified the physical schema independently rather than assuming the DDL ran correctly."

[Back to the Phase 4 lesson index](#)

File-feed object 1 — `returns.csv`

Purpose: one row per returned product line, including quantity, reason, status, and dates.

Physical format: `csv`. CSV is flat and inspectable; JSON Lines keeps one independently parseable event per line and supports nested content.

Expected generated location: `datasets/raw/files/returns.csv` inside a batch-controlled extract/output structure.

Grain: one row per returned product line, including quantity, reason, status, and dates.

Dependencies: Generated parent/business keys must correspond to valid source data except for explicitly injected quality-test cases.

Downstream targets: `bronze.file_returns`, an entity-specific Silver table or quarantine, and the mapped Gold dimension/fact when applicable.

Action 4.4.FD1 — Inspect and reconcile `returns.csv`

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Inspect and reconcile `returns.csv`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: In VS Code Explorer, expand `datasets`, then `raw`, then `files` only after Phase 5 creates the batch output.
2. LOOK FOR: Locate `returns.csv` under the current batch, not the small sample fixture unless the lesson explicitly requests it.
3. CLICK: Open the file in VS Code as text.
4. LOOK FOR: Inspect the header or first JSON object, delimiters, quoting, date representation, null representation, and line endings.
5. CLICK: Open the batch manifest beside the feed.
6. LOOK FOR: Match the relative path, row or event count, byte size, and checksum entry.
7. DO NOT EDIT: Raw generated output is evidence; correct the generator and create a new batch instead of hand-fixing a raw file.
8. DO NOT CONTINUE UNTIL: The feed structure and manifest entry agree.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: In VS Code Explorer, expand `datasets`, then `raw`, then `files` only after Phase 5 creates the batch output.
2. LOOK FOR: Locate `returns.csv` under the current batch, not the small sample fixture unless the lesson explicitly requests it.
3. CLICK: Open the file in VS Code as text.
4. LOOK FOR: Inspect the header or first JSON object, delimiters, quoting, date representation, null representation, and line endings.
5. CLICK: Open the batch manifest beside the feed.
6. LOOK FOR: Match the relative path, row or event count, byte size, and checksum entry.
7. DO NOT EDIT: Raw generated output is evidence; correct the generator and create a new batch instead of hand-fixing a raw file.
8. DO NOT CONTINUE UNTIL: The feed structure and manifest entry agree.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Expected example — do not type this as output

```
return_id,order_id,order_item_id,product_id,customer_id,return_date,return_quantity,return_reason,refund_amount,currency_code,source_file_date
RET-00000001,9000071,10000142,99999999,100013,04/22/2025,1,Late delivery,140.14,USD,2025-01-01
RET-00000002,9000175,10000353,100000000,100034,20-Mar-2026,1,Wrong size,29.73,USD,2025-01-01
RET-00000003,9000123,10000237,10026,100009,2024/06/29,1,Changed mind,15.04,USD,2026-08-15
```

What to validate

1. The file exists only after the generator or extractor writes it.
2. The filename extension is not hidden or doubled, such as .csv.txt.
3. CSV headers are unique and match the data dictionary; JSON Lines has exactly one complete object per nonblank line.
4. The manifest count matches an independent count.
5. The SHA-256 checksum matches a fresh calculation.
6. Expected dirty rows remain observable for Silver tests; raw data is not “cleaned” manually.
7. No real customer or employee information is present; the data is fictional.

Git rule

Generated raw batches are normally ignored because they can be recreated and may be large. Small non-sensitive sample fixtures under datasets/sample/ can be tracked for tests. Confirm the rule with `git check-ignore -v` before staging.

Interview-ready explanation

“I treated returns.csv as a source contract with a declared grain, schema, manifest entry, checksum, and downstream mapping. I preserved raw evidence and corrected generation logic rather than editing an output file.”

File-feed object 2 — supplier_cost_updates.csv

Purpose: one row per supplier-product cost change effective on a business date.

Physical format: csv. CSV is flat and inspectable; JSON Lines keeps one independently parseable event per line and supports nested content.

Expected generated location: datasets/raw/files/supplier_cost_updates.csv inside a batch-controlled extract/output structure.

Grain: one row per supplier-product cost change effective on a business date.

Dependencies: Generated parent/business keys must correspond to valid source data except for explicitly injected quality-test cases.

Downstream targets: bronze.file_supplier_cost_updates, an entity-specific Silver table or quarantine, and the mapped Gold dimension/fact when applicable.

Action 4.4.FD2 — Inspect and reconcile supplier_cost_updates.csv**Application:**

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Inspect and reconcile `supplier_cost_updates.csv`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: In VS Code Explorer, expand datasets, then raw, then files only after Phase 5 creates the batch output.
2. LOOK FOR: Locate `supplier_cost_updates.csv` under the current batch, not the small sample fixture unless the lesson explicitly requests it.
3. CLICK: Open the file in VS Code as text.
4. LOOK FOR: Inspect the header or first JSON object, delimiters, quoting, date representation, null representation, and line endings.
5. CLICK: Open the batch manifest beside the feed.
6. LOOK FOR: Match the relative path, row or event count, byte size, and checksum entry.
7. DO NOT EDIT: Raw generated output is evidence; correct the generator and create a new batch instead of hand-fixing a raw file.
8. DO NOT CONTINUE UNTIL: The feed structure and manifest entry agree.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: In VS Code Explorer, expand datasets, then raw, then files only after Phase 5 creates the batch output.
2. LOOK FOR: Locate `supplier_cost_updates.csv` under the current batch, not the small sample fixture unless the lesson explicitly requests it.
3. CLICK: Open the file in VS Code as text.
4. LOOK FOR: Inspect the header or first JSON object, delimiters, quoting, date representation, null representation, and line endings.
5. CLICK: Open the batch manifest beside the feed.
6. LOOK FOR: Match the relative path, row or event count, byte size, and checksum entry.
7. DO NOT EDIT: Raw generated output is evidence; correct the generator and create a new batch instead of hand-fixing a raw file.
8. DO NOT CONTINUE UNTIL: The feed structure and manifest entry agree.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Expected example — do not type this as output

```
supplier_cost_update_id,supplier_code,sku,effective_date,new_unit_cost,currency_code
SCU-00000001,SUP-05001,NRT-ELE-10013,2026-02-18,33.95,USD
SCU-00000002,SUP-05001,NRT-OUT-10017,2026-02-21,64.76,USD
SCU-00000003,SUP-05001,NRT-ELE-10002,2026-03-15,22.45,USD
```

What to validate

1. The file exists only after the generator or extractor writes it.
2. The filename extension is not hidden or doubled, such as .csv.txt.
3. CSV headers are unique and match the data dictionary; JSON Lines has exactly one complete object per nonblank line.
4. The manifest count matches an independent count.
5. The SHA-256 checksum matches a fresh calculation.
6. Expected dirty rows remain observable for Silver tests; raw data is not “cleaned” manually.
7. No real customer or employee information is present; the data is fictional.

Git rule

Generated raw batches are normally ignored because they can be recreated and may be large. Small non-sensitive sample fixtures under datasets/sample/ can be tracked for tests. Confirm the rule with `git check-ignore -v` before staging.

Interview-ready explanation

“I treated supplier_cost_updates.csv as a source contract with a declared grain, schema, manifest entry, checksum, and downstream mapping. I preserved raw evidence and corrected generation logic rather than editing an output file.”

File-feed object 3 — promotion_calendar.csv

Purpose: one row per promotion definition and active date range.

Physical format: csv. CSV is flat and inspectable; JSON Lines keeps one independently parseable event per line and supports nested content.

Expected generated location: datasets/raw/files/promotion_calendar.csv inside a batch-controlled extract/output structure.

Grain: one row per promotion definition and active date range.

Dependencies: Generated parent/business keys must correspond to valid source data except for explicitly injected quality-test cases.

Downstream targets: bronze.file_promotion_calendar, an entity-specific Silver table or quarantine, and the mapped Gold dimension/fact when applicable.

Action 4.4.FD3 — Inspect and reconcile promotion_calendar.csv**Application:**

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Inspect and reconcile `promotion_calendar.csv`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: In VS Code Explorer, expand datasets, then raw, then files only after Phase 5 creates the batch output.
2. LOOK FOR: Locate `promotion_calendar.csv` under the current batch, not the small sample fixture unless the lesson explicitly requests it.
3. CLICK: Open the file in VS Code as text.
4. LOOK FOR: Inspect the header or first JSON object, delimiters, quoting, date representation, null representation, and line endings.
5. CLICK: Open the batch manifest beside the feed.
6. LOOK FOR: Match the relative path, row or event count, byte size, and checksum entry.
7. DO NOT EDIT: Raw generated output is evidence; correct the generator and create a new batch instead of hand-fixing a raw file.
8. DO NOT CONTINUE UNTIL: The feed structure and manifest entry agree.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: In VS Code Explorer, expand datasets, then raw, then files only after Phase 5 creates the batch output.
2. LOOK FOR: Locate `promotion_calendar.csv` under the current batch, not the small sample fixture unless the lesson explicitly requests it.
3. CLICK: Open the file in VS Code as text.
4. LOOK FOR: Inspect the header or first JSON object, delimiters, quoting, date representation, null representation, and line endings.
5. CLICK: Open the batch manifest beside the feed.
6. LOOK FOR: Match the relative path, row or event count, byte size, and checksum entry.
7. DO NOT EDIT: Raw generated output is evidence; correct the generator and create a new batch instead of hand-fixing a raw file.
8. DO NOT CONTINUE UNTIL: The feed structure and manifest entry agree.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Expected example — do not type this as output

```
promotion_code,promotion_name,start_date,end_date,discount_type,discount_value,channel_code
WELCOME10,New Customer Welcome,2025-01-01,2026-12-31,PERCENT,10,ONLINE
WEEKEND15,Weekend Event,2026-01-01,2026-12-31,PERCENT,15,ALL
FREESHIP,Free Shipping,2026-01-01,2026-12-31,SHIPPING,100,ONLINE
```

What to validate

1. The file exists only after the generator or extractor writes it.
2. The filename extension is not hidden or doubled, such as .csv.txt.
3. CSV headers are unique and match the data dictionary; JSON Lines has exactly one complete object per nonblank line.
4. The manifest count matches an independent count.
5. The SHA-256 checksum matches a fresh calculation.
6. Expected dirty rows remain observable for Silver tests; raw data is not “cleaned” manually.
7. No real customer or employee information is present; the data is fictional.

Git rule

Generated raw batches are normally ignored because they can be recreated and may be large. Small non-sensitive sample fixtures under datasets/sample/ can be tracked for tests. Confirm the rule with `git check-ignore -v` before staging.

Interview-ready explanation

“I treated promotion_calendar.csv as a source contract with a declared grain, schema, manifest entry, checksum, and downstream mapping. I preserved raw evidence and corrected generation logic rather than editing an output file.”

File-feed object 4 — shipment_tracking_events.jsonl

Purpose: one JSON object per carrier tracking event, with nested location/status details.

Physical format: jsonl. CSV is flat and inspectable; JSON Lines keeps one independently parseable event per line and supports nested content.

Expected generated location: datasets/raw/files/shipment_tracking_events.jsonl inside a batch-controlled extract/output structure.

Grain: one JSON object per carrier tracking event, with nested location/status details.

Dependencies: Generated parent/business keys must correspond to valid source data except for explicitly injected quality-test cases.

Downstream targets: bronze.file_shipment_tracking_events, an entity-specific Silver table or quarantine, and the mapped Gold dimension/fact when applicable.

Action 4.4.FD4 — Inspect and reconcile shipment_tracking_events.jsonl**Application:**

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Inspect and reconcile `shipment_tracking_events.jsonl`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: In VS Code Explorer, expand datasets, then raw, then files only after Phase 5 creates the batch output.
2. LOOK FOR: Locate `shipment_tracking_events.jsonl` under the current batch, not the small sample fixture unless the lesson explicitly requests it.
3. CLICK: Open the file in VS Code as text.
4. LOOK FOR: Inspect the header or first JSON object, delimiters, quoting, date representation, null representation, and line endings.
5. CLICK: Open the batch manifest beside the feed.
6. LOOK FOR: Match the relative path, row or event count, byte size, and checksum entry.
7. DO NOT EDIT: Raw generated output is evidence; correct the generator and create a new batch instead of hand-fixing a raw file.
8. DO NOT CONTINUE UNTIL: The feed structure and manifest entry agree.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: In VS Code Explorer, expand datasets, then raw, then files only after Phase 5 creates the batch output.
2. LOOK FOR: Locate `shipment_tracking_events.jsonl` under the current batch, not the small sample fixture unless the lesson explicitly requests it.
3. CLICK: Open the file in VS Code as text.
4. LOOK FOR: Inspect the header or first JSON object, delimiters, quoting, date representation, null representation, and line endings.
5. CLICK: Open the batch manifest beside the feed.
6. LOOK FOR: Match the relative path, row or event count, byte size, and checksum entry.
7. DO NOT EDIT: Raw generated output is evidence; correct the generator and create a new batch instead of hand-fixing a raw file.
8. DO NOT CONTINUE UNTIL: The feed structure and manifest entry agree.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 4 lesson index](#)

Expected example — do not type this as output

```
{
  "tracking_number": "NST9000002665",
  "order_id": 9000002,
  "carrier": "National Ground",
  "events": [
    {
      "event_type": "LABEL_CREATED",
      "event_timestamp": "2024-02-12T00:00:00",
      "location": {
        "city": "Origin",
        "country": "US"
      }
    },
    {
      "event_type": "IN_TRANSIT",
      "event_timestamp": "2024-02-13T00:00:00",
      "location": {
        "city": "Hub",
        "country": "US"
      }
    },
    {
      "event_type": "DELIVERED",
      "event_timestamp": "2024-02-13T00:00:00",
      "location": {
        "city": "Destination",
        "country": "US"
      }
    }
  ]
},
{
  "tracking_number": "NST9000006571",
  "order_id": 9000006,
  "carrier": "ParcelOne",
  "events": [
    {
      "event_type": "LABEL_CREATED",
      "event_timestamp": "2024-10-19T00:00:00",
      "location": {
        "city": "Origin",
        "country": "US"
      }
    },
    {
      "event_type": "IN_TRANSIT",
      "event_timestamp": "2024-10-20T00:00:00",
      "location": {
        "city": "Hub",
        "country": "US"
      }
    },
    {
      "event_type": "DELIVERED",
      "event_timestamp": "2024-10-25T00:00:00",
      "location": {
        "city": "Destination",
        "country": "US"
      }
    }
  ]
},
{
  "tracking_number": "NST9000007582",
  "order_id": 9000007,
  "carrier": "National Ground",
  "events": [
    {
      "event_type": "LABEL_CREATED",
      "event_timestamp": "2025-11-09T00:00:00",
      "location": {
        "city": "Origin",
        "country": "US"
      }
    },
    {
      "event_type": "IN_TRANSIT",
      "event_timestamp": "2025-11-10T00:00:00",
      "location": {
        "city": "Hub",
        "country": "US"
      }
    },
    {
      "event_type": "DELIVERED",
      "event_timestamp": "2025-11-15T00:00:00",
      "location": {
        "city": "Destination",
        "country": "US"
      }
    }
  ]
},
{
  "tracking_number": "NST9000011599",
  "order_id": 9000011,
  "carrier": "ParcelOne",
  "events": [
    {
      "event_type": "LABEL_CREATED",
      "event_timestamp": "2026-02-27T00:00:00",
      "location": {
        "city": "Origin",
        "country": "US"
      }
    },
    {
      "event_type": "IN_TRANSIT",
      "event_timestamp": "2026-02-28T00:00:00",
      "location": {
        "city": "Hub",
        "country": "US"
      }
    },
    {
      "event_type": "DELIVERED",
      "event_timestamp": "2026-03-02T00:00:00",
      "location": {
        "city": "Destination",
        "country": "US"
      }
    }
  ]
}
```

What to validate

1. The file exists only after the generator or extractor writes it.
2. The filename extension is not hidden or doubled, such as .csv.txt.
3. CSV headers are unique and match the data dictionary; JSON Lines has exactly one complete object per nonblank line.
4. The manifest count matches an independent count.
5. The SHA-256 checksum matches a fresh calculation.
6. Expected dirty rows remain observable for Silver tests; raw data is not “cleaned” manually.
7. No real customer or employee information is present; the data is fictional.

Git rule

Generated raw batches are normally ignored because they can be recreated and may be large. Small non-sensitive sample fixtures under datasets/sample/ can be tracked for tests. Confirm the rule with `git check-ignore -v` before staging.

Interview-ready explanation

“I treated shipment_tracking_events.jsonl as a source contract with a declared grain, schema, manifest entry, checksum, and downstream mapping. I preserved raw evidence and corrected generation logic rather than editing an output file.”

What your screen should look like now

- The new Phase 4 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 4.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 4.5](#).

Lesson 4.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 4 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Source-system separation

Question:

Where should ERP and digital data live?

Options considered:

- One SQL Server database.
- SQL Server ERP plus PostgreSQL digital.
- Only flat files.

Beginner path selected:

Use SQL Server for ERP and PostgreSQL for digital activity.

Why this path was selected:

Two engines create realistic integration work while remaining manageable locally.

Tradeoff:

More installation, drivers, and dialect handling are required.

What a company might do instead:

A company may use SaaS sources, managed cloud databases, event platforms, and CDC.

How to explain this in an interview:

I separated operational domains across SQL Server and PostgreSQL to demonstrate heterogeneous-source ingestion and dialect-aware engineering.

If you see Result A

The database/schema/table exists and metadata matches the declared columns, keys, and grain. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

The object is absent, already exists with a different definition, or you are connected to the wrong database. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: VS Code, SSMS, pgAdmin, and PowerShell.

Window/panel: Show database tree and column/constraint metadata for representative source tables plus data dictionary.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase04_design-and-create-the-sql-server-and-postgresql-source-_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 4's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 4 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 4.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.

4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

Action 4.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Action 4.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.

4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Interpret `git status --short`

- ?? means untracked: Git sees a new path but it is not staged.
- M means modified in the working tree but not staged.
- M means the staged version differs from the last commit.
- MM means a staged version exists and the file has additional unstaged edits.
- A means staged as a new file.
- D means deletion; stop and investigate unless deletion is deliberate.
- R means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 4.5.G4 — Inspect the unstaged diff for `sql/sqlserver/source/01_create_erp_database.sql`

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "sql/sqlserver/source/01_create_erp_database.sql"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "sql/sqlserver/source/01_create_erp_database.sql"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the

surrounding explanation; -- is a command option or flag; its exact behavior is defined beside the command; "sql/sqlserver/source/01_create_erp_database.sql" is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Action 4.5.G5 — Stage only the reviewed file sql/sqlserver/source/01_create_erp_database.sql

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "sql/sqlserver/source/01_create_erp_database.sql"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "sql/sqlserver/source/01_create_erp_database.sql"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"sql/sqlserver/source/01_create_erp_database.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 4.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Action 4.5.G7 — Create the Phase 4 commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "feat: define northstar source schemas"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "feat: define northstar source schemas"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"feat: define northstar source schemas"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use

git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Action 4.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Action 4.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 4 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message `feat: define northstar source schemas` and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 4.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 4 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

"Phase 4 created the verified boundary described by **Design and create the SQL Server and PostgreSQL source systems**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed."

Mini quiz

1. Multiple choice: Why does Phase 4 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

My Phase 4 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 4.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 5 uses each column, constraint, and relationship as a generation requirement.

Phase 4 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]



Output independently inspected — [STATUS]



Failure/alternate branch understood — [STATUS]



Screenshot captured and redacted — [STATUS]



Targeted Git commit exists — [STATUS]



Branch pushed and verified on GitHub — [STATUS]



Quiz completed before answer key — [STATUS]



Journal completed — [STATUS]



Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 5 — Build and validate the synthetic-data generator

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: Complete source schemas and data-quality scenarios.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 6 loads the generated parent and child data into the two database engines.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 5 lesson index

1. [Lesson 5.1 — Understand the phase and starting screen](#)
2. [Lesson 5.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 5.3 — Create and understand the phase files](#)
4. [Lesson 5.4 — Build and validate required objects](#)
5. [Lesson 5.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 5.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** VS Code, VS Code PowerShell terminal, Python virtual environment, and File Explorer.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 5 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 4 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 5 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

VS Code, VS Code PowerShell terminal, Python virtual environment, and File Explorer. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 5 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.

- No new Phase 5 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 5 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 5.1 completion checklist

☐

I read why Phase 5 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 5.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 5 lesson index](#)

Lesson 5.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 5 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

A reproducible Python generator that creates all ERP, digital, CSV, and nested JSON source data at configurable scales, keeps normal foreign keys coherent, injects controlled quality defects, writes a batch manifest/checksums, and validates its own output.

2. Why it matters

A portfolio project needs data rich enough to exercise engineering decisions without exposing real people or employer information. A fixed seed and quality fixture turn messy data from random surprise into a repeatable test case.

3. Words to know

Term	Plain-language definition
Module	Python file/package imported or run with <code>python -m</code> .
Function	Named reusable block with inputs and output/side effect.
Loop	Repeats logic over records or tables.
Dictionary	Key/value mapping used for configuration and records.

Term	Plain-language definition
DataFrame	Table-like in-memory data structure from pandas/Spark.
Random seed	Starting value that makes pseudo-random business content reproducible.
Configuration	Values separated from logic, here in YAML and CLI arguments.
Logging	Structured messages about progress and failures.
Exception	Error object raised when code cannot safely continue.
Referential integrity	Child references point to existing parent records for normal data.
Distribution	Pattern of values/likelihoods rather than uniform randomness.
Fixture	Known data/expectation used by tests.
Docstring/type hint	Description and expected Python types that improve readability/tools.

4. What must already be completed

Phases 1–4 complete. `.venv` is active and dependencies installed. Source databases need not contain rows yet.

5. Applications to open

Application	How to open it	Why it is open
VS Code	Repository root; open <code>scripts/data_generation/generate_northstar_data.py</code> .	Read functions, imports, CLI, and TODO work.
Windows PowerShell	Repository root with <code>.venv</code> active.	Run generator/tests and inspect output.
File Explorer or VS Code Explorer	Open <code>datasets/sample/</code> .	Inspect generated files and metadata.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run `Get-Location`; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
<code>python -c "import sys, pandas, faker, yaml; print(sys.executable); print(pandas.version)"</code>	Confirms interpreter and core imports.	Avoid debugging generator logic with the wrong Python environment.
<code>python -m scripts.data_generation.generate_northstar_data --scale quick --seed 20260815 --output-dir datasets\generated\practice_quick</code>	Runs generator as a package module with explicit scale, seed, output.	Module execution preserves imports and separates practice output from included sample.
<code>python -m scripts.validation.validate_generated_data --input-dir datasets\generated\practice_quick</code>	Checks required non-empty files and normal source relationships.	Generation success message is not enough.
<code>Get-Content data-sets\generated\practice_quick\metadata\generation_summary.csv</code>	Prints per-file row counts.	You must know what was generated before loading.
<code>python -m pytest -q tests\unit\test_generator.py tests\data_quality\test_sample_quality_fixture.py</code>	Runs deterministic generation and intentional-dirty-data tests.	Proves same seed and expected defects behave as designed.

Python concepts in this generator

- Imports bring in tested libraries instead of rewriting Faker/DataFrame/YAML behavior.
- Functions each generate one subject or perform one responsibility.
- Loops create repeated fictional entities and events.
- Dictionaries represent record fields and table collections.
- DataFrames provide column operations and CSV writing.
- `random.Random(seed)` and `Faker.seed(seed)` make business content repeatable.
- `argparse` gives `--scale`, `--seed`, `--output-dir`, and `--config`.
- Logging says what happened; exceptions stop unsafe output.
- `if __name__ == "__main__":` runs `main()` only when launched, not when imported by tests.

MY TASK

Run quick mode twice with the same seed into two folders, prove selected business files are byte-identical, inspect every controlled issue type, and complete one small generator enhancement with dictionary/test updates.

STARTER SKELETON

```
from __future__ import annotations

import logging
import random
from typing import Any

LOGGER = logging.getLogger(__name__)

def generate_loyalty_points(
    order_total: float,
    loyalty_tier: str,
    rng: random.Random,
) -> int:
    """Return fictional points earned for one valid order."""
    # TODO(USER): define a deterministic base rule.
    base_points = _____
    # TODO(USER): define tier multiplier mapping.
    multiplier_by_tier: dict[str, float] = {_____}
    # TODO(USER): avoid negative points and return int.
    return _____

# TODO(USER): add unit tests for zero, normal, and tier cases.
```

Acceptance criteria: Two same-seed business files match, normal references validate, expected dirty rows are observed, and any enhancement updates code, data dictionary, config if needed, and deterministic tests.

HINT 1 — CONCEPT

Reproducibility applies to business content; batch timestamps are allowed to differ.

HINT 2 — PSEUDOCODE

Generate parents before children and choose IDs from parent DataFrames.

HINT 3 — NEAR-COMPLETE PATTERN

Copy the existing small function/docstring/type-hint style and add tests before scaling.

13. Expected files or output

- 23 generated source files plus four metadata files for quick mode.
- 60 quick customers and 200 quick orders under documented configuration.
- Controlled duplicate, email, quantity, date, reference, currency, order, and timezone issues.
- Passing validation/test output.
- One personally explained small change or documented attempt.

14. How to verify

- Compare selected files with Get-FileHash or pytest same-seed test.
- Open expected fixture and find examples in source CSVs.
- Check order customer IDs all exist.
- Confirm order dates are not before customer signup.
- Confirm no real personal data or real company contact is used.

15. Automated tests

- tests/unit/test_generator.py.
- tests/data_quality/test_sample_quality_fixture.py.
- Standalone generated-data validator.
- Later source row-count reconciliation after load.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
No module named scripts	Script was launched by path from wrong folder/interpreter.	Get-Location; python -c "import sys; print(sys.path[0])".	Run from repository root with python -m scripts.data_generation..	Generator starts.
FileNotFound config	Current folder is not repository root.	Get-Location; Test-Path config\data_generation.yml.	Enter verified root or pass explicit config.	Config loads.
Output differs with same seed	Timestamp fields/batch ID compared or nondeterministic operation added.	Diff selected business CSVs and isolate columns.	Exclude run metadata from reproducibility assertion or seed/sort all business choices.	Business files match.
Generation is slow	Scale too large for debugging.	Read --scale and config.	Use quick; only run portfolio after correctness.	Quick completes and tests pass.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why does generation order matter for foreign keys?
2. Why are realistic distributions better than uniform choices?
3. Why should dirty data be controlled rather than random?
4. What does the main guard do?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. What makes two generator runs reproducible?
2. Which file records expected injected issue counts?
3. Why should portfolio scale wait until logic passes quick scale?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture PowerShell showing generator and validation success, plus Explorer showing the four output subfolders and an open manifest. Include the seed/scale, not a password.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

Generator read and explained — [STATUS]

☐

Quick run completed — [STATUS]

☐

Same-seed behavior verified — [STATUS]

☐

Normal referential integrity verified — [STATUS]

☐

Controlled issues located — [STATUS]

☐

Tests passed — [STATUS]

☐

Small user-owned task completed — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-5. The quiz answer is in [the answer-key section immediately after the quiz in this workbook#phase-5] (the answer-key section immediately after the quiz in this workbook#phase-5).

Next phase: Phase 6 loads the generated ERP and digital files into the two local source databases.

Required action cards

Action 5.2.1 — Open config/data_generation.yml. Compare quick, small, medium, and portfolio counts. Use quick while debuggi

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open config/data_generation.yml. Compare quick, small, medium, and portfolio counts. Use quick while debuggi. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.

3. Perform only this immediate task: Open `config/data_generation.yml`. Compare quick, small, medium, and portfolio counts. Use quick while debugging.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 1****Action**

Open `config/data_generation.yml`. Compare quick, small, medium, and portfolio counts. Use quick while debugging.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 5 lesson index](#)

Action 5.2.2 — Open the generator. Read imports first: standard library, pandas, YAML, Faker, shared logging.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open the generator. Read imports first: standard library, pandas, YAML, Faker, shared logging. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open the generator. Read imports first: standard library, pandas, YAML, Faker, shared logging.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 2

Action

Open the generator. Read imports first: standard library, pandas, YAML, Faker, shared logging.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 5 lesson index](#)

Action 5.2.3 — Find GenerationContext; explain why seed, scale, output, counts, issues, batch ID, and timestamp travel toge

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Find GenerationContext; explain why seed, scale, output, counts, issues, batch ID, and timestamp travel toge. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Find GenerationContext; explain why seed, scale, output, counts, issues, batch ID, and timestamp travel together.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 3****Action**

Find GenerationContext; explain why seed, scale, output, counts, issues, batch ID, and timestamp travel together.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 5 lesson index](#)

Action 5.2.4 — Trace generation order: reference tables → customers/addresses → orders/items/payments/shipments → inventory →

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Trace generation order: reference tables → customers/addresses → orders/items/payments/shipments → inventory →. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Trace generation order: reference tables → customers/addresses → orders/items/payments/shipments → inventory → digital → files. Parents must exist before children can choose their keys.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 4****Action**

Trace generation order: reference tables → customers/addresses → orders/items/payments/shipments → inventory → digital → files. Parents must exist before children can choose their keys.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 5 lesson index](#)

Action 5.2.5 — Find one weighted choice. Explain why store/online or channel values should not be equally likely by default.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Find one weighted choice. Explain why store/online or channel values should not be equally likely by default. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Find one weighted choice. Explain why store/online or channel values should not be equally likely by default.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 5

Action

Find one weighted choice. Explain why store/online or channel values should not be equally likely by default.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 5 lesson index](#)

Action 5.2.6 — Find controlled issue injection. The generator intentionally creates invalid rows; Silver, not the generator,

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Find controlled issue injection. The generator intentionally creates invalid rows; Silver, not the generator, . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Find controlled issue injection. The generator intentionally creates invalid rows; Silver, not the generator, decides trusted behavior.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 6****Action**

Find controlled issue injection. The generator intentionally creates invalid rows; Silver, not the generator, decides trusted behavior.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 5 lesson index](#)

Action 5.2.7 — Run quick mode with the fixed seed to a new practice folder. Do not overwrite evidence from another run without

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run quick mode with the fixed seed to a new practice folder. Do not overwrite evidence from another run without. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run quick mode with the fixed seed to a new practice folder. Do not overwrite evidence from another run without recording it.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 7

Action

Run quick mode with the fixed seed to a new practice folder. Do not overwrite evidence from another run without recording it.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 5 lesson index](#)

Action 5.2.8 — Open metadata/generation_summary.csv, batch_manifest.json, validation_results.json, and `expected_qualit

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open metadata/generation_summary.csv, batch_manifest.json, validation_results.json, and expected_quality. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open metadata/generation_summary.csv, batch_manifest.json, validation_results.json, and expected_quality_issues.json.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 8****Action**

Open metadata/generation_summary.csv, batch_manifest.json, validation_results.json, and expected_quality_issues.json.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 5 lesson index](#)

Action 5.2.9 — Run the standalone validation and pytest generator tests.

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run the standalone validation and pytest generator tests. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run the standalone validation and pytest generator tests.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 9****Action**

Run the standalone validation and pytest generator tests.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.



I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 5 lesson index](#)

Action 5.2.10 — Complete the TODO exercise: add one fictional loyalty_points_balance field to a throwaway branch or add one

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Complete the TODO exercise: add one fictional loyalty_points_balance field to a throwaway branch or add one . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.

3. Perform only this immediate task: Complete the TODO exercise: add one fictional `loyalty_points_balance` field to a throwaway branch or add one generator-only summary metric; update dictionary/tests before merging.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 10****Action**

Complete the TODO exercise: add one fictional `loyalty_points_balance` field to a throwaway branch or add one generator-only summary metric; update dictionary/tests before merging.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 5 lesson index](#)

Action 5.2.C1 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -c "import sys, pandas, faker, yaml; print(sys.executable); print(pandas.__version__)"
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -c "import sys, pandas, faker, yaml; print(sys.executable); print(pandas.__version__)"` — `python` invokes Python when that command name is registered; `-c` creates the named branch before switching; `"import sys, pandas, faker, yaml; print(sys.executable); print(pandas.__version__)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Action 5.2.C2 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m scripts.data_generation.generate_northstar_data --scale quick --seed 20260815 --output-dir datasets\generated\practice_quick
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m scripts.data_generation.generate_northstar_data --scale quick --seed 20260815 --output-dir datasets\generated\practice_quick` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.data_generation.generate_northstar_data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--scale` is a command option or flag; its exact behavior is defined beside the command; `quick` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--seed` is a command option or flag; its exact behavior is defined beside the command; `20260815` is a numeric literal used by the command or expression; `--output` is a command option or flag; its exact behavior is defined beside the command; `-dir` is a command option or flag; its exact behavior is defined beside the command; `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `generated` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `practice_quick` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Action 5.2.C3 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m scripts.validation.validate_generated_data --input-dir datasets\generated\practice_quick
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m scripts.validation.validate_generated_data --input-dir datasets\generated\practice_quick` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.validate_generated_data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--input` is a command option or flag; its exact behavior is

defined beside the command; `-dir` is a command option or flag; its exact behavior is defined beside the command; `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `generated` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `practice_quick` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Action 5.2.C4 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Get-Content datasets\generated\practice_quick\metadata\generation_summary.csv
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** Get-Content datasets\generated\practice_quick\metadata\generation_summary.csv — Get-Content is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datasets is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; generated is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; practice_quick is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; metadata is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; generation_summary.csv is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use

`git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Action 5.2.C5 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m pytest -q tests\unit\test_generator.py tests\data_quality\test_sample_quality_fixture.py
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m pytest -q tests\unit\test_generator.py tests\data_quality\test_sample_quality_fixture.py` — python invokes Python when that command name is registered; `-m` asks Python to run an installed module; `pytest` the Python test runner; `-q` is a command option or flag; its exact behavior is defined beside the command; `tests` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `unit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_generator.py` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tests` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `data_quality` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_sample_quality_fixture.py` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 5.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 5 starting-screen checklist and re-verify the active context.

Lesson 5.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 5 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — config/

Exact folder name: config

Exact parent: repository root

Why it is needed now: Phase 5 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: config.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 5.D1.1 — Create the folder config

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\config" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\config" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\config" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\config" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\config" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\config" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — scripts/

Exact folder name: `scripts`

Exact parent: `repository root`

Why it is needed now: Phase 5 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: scripts.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 5.D2.1 — Create the folder scripts

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\scripts" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\scripts" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\scripts" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\scripts" -PathType Container` and require True.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\scripts" -PathType Container`.

2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — tests/

Exact folder name: tests

Exact parent: repository root

Why it is needed now: Phase 5 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: tests.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 5.D3.1 — Create the folder tests

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.

7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\tests" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\tests" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\tests"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\tests" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\tests" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\tests" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — `scripts/data_generation/`

Exact folder name: `data_generation`

Exact parent: `scripts`

Why it is needed now: Phase 5 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `scripts` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `data_generation`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 5.D4.1 — Create the folder `scripts/data_generation`

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\scripts\data_generation" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\scripts\data_generation" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\scripts\data_generation"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\scripts\data_generation" -PathType Container` and require True.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts\data_generation" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\scripts\data_generation" -PathType Container`.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — scripts/validation/

Exact folder name: validation

Exact parent: scripts

Why it is needed now: Phase 5 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `scripts` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `validation`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 5.D5.1 — Create the folder scripts/validation

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\scripts\validation" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\scripts\validation" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\scripts\validation" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\scripts\validation" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts\validation" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\scripts\validation" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — tests/unit/

Exact folder name: `unit`

Exact parent: `tests`

Why it is needed now: Phase 5 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path tests one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `unit`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 5.D6.1 — Create the folder tests/unit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\tests\unit" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\tests\unit" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\tests\unit` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\tests\unit" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\tests\unit" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run Test-Path ".\tests\unit" -PathType Container.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — config/data_generation.yml

Exact filename: data_generation.yml

Filename extension: .yml. In this project it identifies human-readable configuration and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: config relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: human-readable configuration for Phase 5.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? Yes—review carefully before Git staging.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in config one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: data_generation.yml.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled data_generation.yml and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 5.F1.1 — Create the file config/data_generation.yml from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\config\data_generation.yml" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType File -Path ".\config\data_generation.yml" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; File is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\config\data_generation.yml" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\config\data_generation.yml" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "config/data_generation.yml"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `config`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `config/data_generation.yml`.
3. In PowerShell at the repository root, run `Test-Path ".\config\data_generation.yml"` and require True.
4. Run `git status --short -- "config/data_generation.yml"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for config/data_generation.yml

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: YAML

Exact file: `config/data_generation.yml`

Purpose: human-readable configuration.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 5.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
default_seed: 20260815
default_scale: quick
scales:
  quick:
    customers: 60
```

```

products: 30
suppliers: 8
stores: 5
employees: 20
orders: 200
web_users: 80
web_sessions: 300
campaigns: 8
history_days: 90
small:
  customers: 500
  products: 150
  suppliers: 25
  stores: 10
  employees: 80
  orders: 3000
  web_users: 700
  web_sessions: 5000
  campaigns: 15
  history_days: 365
medium:
  customers: 5000
  products: 500
  suppliers: 80
  stores: 30
  employees: 350
  orders: 50000
  web_users: 7000
  web_sessions: 80000
  campaigns: 40
  history_days: 730
portfolio:
  customers: 25000
  products: 1500
  suppliers: 250
  stores: 75
  employees: 1200
  orders: 500000
  web_users: 35000
  web_sessions: 750000
  campaigns: 100
  history_days: 1095
quality_issues:
  duplicate_business_keys: 3
  null_emails: 3
  malformed_emails: 2
  whitespace_and_case: 4
  invalid_state_codes: 2
  impossible_quantities: 2
  unknown_file_references: 2
  mixed_date_formats: 6
  late_arriving_records: 2
  out_of_order_events: 2
  currency_inconsistencies: 2
  timezone_differences: 4

```

Line-by-line explanation

- **Line 1:** default_seed: 20260815 — default_seed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 20260815 is a numeric literal used by the command or expression
- **Line 2:** default_scale: quick — default_scale is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quick is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** scales: — scales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** quick: — quick is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** customers: 60 — customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 60 is a numeric literal used by the command or expression

- [illegible]

- **Line 43:** orders: 500000 — orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 500000 is a numeric literal used by the command or expression
- **Line 44:** web_users: 35000 — web_users is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 35000 is a numeric literal used by the command or expression
- **Line 45:** web_sessions: 750000 — web_sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 750000 is a numeric literal used by the command or expression
- **Line 46:** campaigns: 100 — campaigns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression
- **Line 47:** history_days: 1095 — history_days is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1095 is a numeric literal used by the command or expression
- **Line 48:** quality_issues: — quality_issues is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 49:** duplicate_business_keys: 3 — duplicate_business_keys is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression
- **Line 50:** null_emails: 3 — null_emails is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression
- **Line 51:** malformed_emails: 2 — malformed_emails is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression
- **Line 52:** whitespace_and_case: 4 — whitespace_and_case is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 4 is a numeric literal used by the command or expression
- **Line 53:** invalid_state_codes: 2 — invalid_state_codes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression
- **Line 54:** impossible_quantities: 2 — impossible_quantities is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression
- **Line 55:** unknown_file_references: 2 — unknown_file_references is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression
- **Line 56:** mixed_date_formats: 6 — mixed_date_formats is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 6 is a numeric literal used by the command or expression
- **Line 57:** late_arriving_records: 2 — late_arriving_records is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression
- **Line 58:** out_of_order_events: 2 — out_of_order_events is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression

- **Line 59:** `currency_inconsistencies: 2` — `currency_inconsistencies` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression
- **Line 60:** `timezone_differences: 4` — `timezone_differences` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 4 is a numeric literal used by the command or expression

[Back to the Phase 5 lesson index](#)

File introduction — `scripts/data_generation/generate_northstar_data.py`

Exact filename: `generate_northstar_data.py`

Filename extension: `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `scripts/data_generation` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 5.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `scripts/data_generation` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `generate_northstar_data.py`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `generate_northstar_data.py` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 5.F2.1 — Create the file `scripts/data_generation/generate_northstar_data.py` from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\scripts\data_generation\generate_northstar_data.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\scripts\data_generation\generate_northstar_data.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\scripts\data_generation\generate_northstar_data.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use

`git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\scripts\data_generation\generate_northstar_data.py" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "scripts/data_generation/generate_northstar_data.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `scripts/data_generation`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `scripts/data_generation/generate_northstar_data.py`.
3. In PowerShell at the repository root, run `Test-Path ".\scripts\data_generation\generate_northstar_data.py" and require True.`
4. Run `git status --short -- "scripts/data_generation/generate_northstar_data.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `scripts/data_generation/generate_northstar_data.py`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `scripts/data_generation/generate_northstar_data.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 5.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```

"""Generate reproducible, entirely fictional Northstar Retail source data.

Run from the repository root in an activated Python virtual environment:
python -m scripts.data_generation.generate_northstar_data --scale quick --seed 20260815
"""
from __future__ import annotations

import argparse
import hashlib
import json
import logging
import random
import sys
import uuid
from dataclasses import dataclass
from datetime import UTC, date, datetime, timedelta
from pathlib import Path
from typing import Any

import pandas as pd
import yaml
from faker import Faker

from scripts.utilities.logging_utils import configure_logging

LOGGER = logging.getLogger("northstar.generator")

REGIONS = {
    "Northeast": [("NY", "New York"), ("MA", "Boston"), ("PA", "Philadelphia")],
    "Mid-Atlantic": [("VA", "Arlington"), ("MD", "Bethesda"), ("DC", "Washington")],
    "Southeast": [("GA", "Atlanta"), ("NC", "Charlotte"), ("FL", "Orlando")],
    "Midwest": [("IL", "Chicago"), ("OH", "Columbus"), ("MI", "Detroit")],
    "West": [("CA", "San Diego"), ("WA", "Seattle"), ("CO", "Denver")],
}

CATEGORIES = [
    (1, "Electronics"), (2, "Home Office"), (3, "Kitchen"), (4, "Fitness"),
    (5, "Outdoor"), (6, "Personal Care"), (7, "Pet Supplies"), (8, "Travel"),
]

CHANNELS = ["Paid Search", "Social", "Email", "Affiliate", "Organic", "Direct"]
EVENT_SEQUENCE = ["page_view", "product_view", "add_to_cart", "checkout_start", "purchase"]
BRANDS = ["Aurora Works", "Blue Peak", "Cedar & Finch", "Northline", "Orbit & Oak"]
PRODUCT_NOUNS = ["Lamp", "Keyboard", "Bottle", "Backpack", "Blender", "Headphones", "Mat", "Organizer"]

@dataclass(frozen=True)
class GenerationContext:
    seed: int
    scale: str
    output_dir: Path
    counts: dict[str, int]
    issues: dict[str, int]
    batch_id: str
    generated_at: datetime

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Generate fictional Northstar Retail data.")
    parser.add_argument("--scale", choices=["quick", "small", "medium", "portfolio"], default="quick")
    parser.add_argument("--seed", type=int, default=20260815)
    parser.add_argument("--output-dir", type=Path, default=Path("datasets/sample"))
    parser.add_argument("--config", type=Path, default=Path("config/data_generation.yml"))
    parser.add_argument(
        "--as-of-utc",
        default="2026-08-15T12:00:00+00:00",
        help="Deterministic dataset as-of timestamp; use ISO 8601.",
    )
    return parser.parse_args()

def load_context(args: argparse.Namespace) -> GenerationContext:
    with args.config.open("r", encoding="utf-8") as handle:
        config = yaml.safe_load(handle)
    counts = dict(config["scales"][args.scale])
    issues = dict(config["quality_issues"])
    stamp = datetime.fromisoformat(args.as_of_utc.replace("Z", "+00:00")).astimezone(UTC)
    stable = hashlib.sha256(f"{args.seed}:{args.scale}".encode()).hexdigest()[:12]
    return GenerationContext(
        seed=args.seed,
        scale=args.scale,
        output_dir=args.output_dir,
        counts=counts,
        issues=issues,
        batch_id=f"GEN-{stamp:%Y%m%dT%H%M%S}-#{stable}",
        generated_at=stamp,
    )

```

```

def weighted_choice(rng: random.Random, values: list[Any], weights: list[float]) -> Any:
    return rng.choices(values, weights=weights, k=1)[0]

def write_frame(frame: pd.DataFrame, path: Path) -> dict[str, Any]:
    path.parent.mkdir(parents=True, exist_ok=True)
    frame.to_csv(path, index=False, date_format="%Y-%m-%dT%H:%M:%S")
    digest = hashlib.sha256(path.read_bytes()).hexdigest()
    return {"file": path.as_posix(), "rows": int(len(frame)), "sha256": digest}

def random_date(rng: random.Random, start: date, end: date) -> date:
    return start + timedelta(days=rng.randint(0, max(0, (end - start).days)))

def generate_reference_data(ctx: GenerationContext, fake: Faker, rng: random.Random) -> dict[str, pd.DataFrame]:
    now = ctx.generated_at.replace(tzinfo=None)
    categories = pd.DataFrame([
        {"category_id": cid, "category_name": name, "parent_category_id": None,
         "created_at": datetime(2024, 1, 1), "updated_at": now}
        for cid, name in CATEGORIES
    ])

    suppliers_rows = []
    for i in range(ctx.counts["suppliers"]):
        sid = 5001 + i
        suppliers_rows.append({
            "supplier_id": sid,
            "supplier_code": f"SUP-{sid:05d}",
            "supplier_name": f"{fake.unique.company()} Supply",
            "country_code": weighted_choice(rng, ["US", "CA", "MX", "GB"], [0.72, 0.12, 0.10, 0.06]),
            "lead_time_days": weighted_choice(rng, [3, 7, 10, 14, 21, 30], [0.08, 0.20, 0.25, 0.28, 0.14, 0.05]),
            "active_flag": 1 if rng.random() > 0.04 else 0,
            "created_at": datetime(2024, 1, 1),
            "updated_at": now,
        })
    suppliers = pd.DataFrame(suppliers_rows)

    products_rows = []
    for i in range(ctx.counts["products"]):
        pid = 10001 + i
        category_id, category_name = rng.choice(CATEGORIES)
        cost = round(rng.lognormvariate(3.0, 0.65), 2)
        margin = rng.uniform(1.35, 2.8)
        products_rows.append({
            "product_id": pid,
            "sku": f"NRT-{category_name[:3].upper()}-{pid}",
            "product_name": f"{rng.choice(BRANDS)} {rng.choice(PRODUCT_NOUNS)} {i + 1}",
            "category_id": category_id,
            "supplier_id": int(rng.choice(suppliers["supplier_id"].tolist())),
            "brand": rng.choice(BRANDS),
            "unit_cost": cost,
            "list_price": round(cost * margin, 2),
            "currency_code": "USD",
            "active_flag": 1 if rng.random() > 0.03 else 0,
            "created_at": datetime(2024, 1, 1),
            "updated_at": now,
        })
    products = pd.DataFrame(products_rows)

    stores_rows = []
    location_pool = [(region, state, city) for region, values in REGIONS.items() for state, city in values]
    for i in range(ctx.counts["stores"]):
        store_id = 201 + i
        region, state_code, city = location_pool[i % len(location_pool)]
        stores_rows.append({
            "store_id": store_id,
            "store_code": f"{state_code}-{city[:3].upper()}-{i + 1:02d}",
            "store_name": f"Northstar {city} {i // len(location_pool) + 1}",
            "region": region,
            "state_code": state_code,
            "city": city,
            "open_date": random_date(rng, date(2017, 1, 1), date(2024, 12, 31)),
            "active_flag": 1,
            "created_at": datetime(2024, 1, 1),
            "updated_at": now,
        })
    stores = pd.DataFrame(stores_rows)

    employees_rows = []
    for i in range(ctx.counts["employees"]):
        employee_id = 301 + i
        store_id = int(stores.iloc[i % len(stores)]["store_id"])
        employees_rows.append({
            "employee_id": employee_id,
            "employee_number": f"EMP-{employee_id:05d}",
            "store_id": store_id,

```

```

        "manager_employee_id": None,
        "first_name": fake.first_name(),
        "last_name": fake.last_name(),
        "job_title": weighted_choice(rng, ["Sales Associate", "Store Manager", "Inventory Specialist"], [0.72, 0.10, 0.18]),
        "hire_date": random_date(rng, date(2019, 1, 1), date(2025, 12, 31)),
        "active_flag": 1 if rng.random() > 0.05 else 0,
        "created_at": datetime(2024, 1, 1),
        "updated_at": now,
    })
employees = pd.DataFrame(employees_rows)
for store_id, group in employees.groupby("store_id"):
    manager_id = int(group.iloc[0]["employee_id"])
    employees.loc[group.index, "manager_employee_id"] = manager_id
    employees.loc[group.index[0], "job_title"] = "Store Manager"
    employees.loc[group.index[0], "manager_employee_id"] = None

return {
    "product_categories": categories,
    "suppliers": suppliers,
    "products": products,
    "stores": stores,
    "employees": employees,
}

def generate_customers(ctx: GenerationContext, fake: Faker, rng: random.Random) -> tuple[pd.DataFrame, pd.DataFrame]:
    today = date(2026, 8, 1)
    rows, addresses = [], []
    for i in range(ctx.counts["customers"]):
        customer_id = 100001 + i
        signup = random_date(rng, date(2023, 1, 1), today - timedelta(days=7))
        first, last = fake.first_name(), fake.last_name()
        customer_number = f"CUS-{customer_id:07d}"
        email = f"{first}.{last}.{customer_id}@example.test".lower().replace("'", "")
        rows.append({
            "customer_id": customer_id,
            "customer_number": customer_number,
            "first_name": first,
            "last_name": last,
            "email": email,
            "phone": fake.numerify("+1-555-####"),
            "signup_date": signup,
            "loyalty_tier": weighted_choice(rng, ["Bronze", "Silver", "Gold", "Platinum"], [0.48, 0.30, 0.17, 0.05]),
            "marketing_opt_in": 1 if rng.random() < 0.68 else 0,
            "created_at": datetime.combine(signup, datetime.min.time()),
            "updated_at": ctx.generated_at.replace(tzinfo=None),
        })
    region, candidates = rng.choice(list(REGIONS.items()))
    state_code, city = rng.choice(candidates)
    addresses.append({
        "address_id": 500001 + i,
        "customer_id": customer_id,
        "address_type": "Shipping",
        "line1": fake.street_address(),
        "line2": None,
        "city": city,
        "state_code": state_code,
        "postal_code": fake.postcode()[1:10],
        "country_code": "US",
        "is_default": 1,
        "created_at": datetime.combine(signup, datetime.min.time()),
        "updated_at": ctx.generated_at.replace(tzinfo=None),
    })
    customers = pd.DataFrame(rows)
    address_frame = pd.DataFrame(addresses)

    # Controlled issues. These counts are deterministic and recorded in the fixture.
    for idx in range(min(ctx.issues["null_emails"], len(customers))):
        customers.loc[idx, "email"] = None
    for idx in range(ctx.issues["null_emails"], min(ctx.issues["null_emails"] + ctx.issues["malformed_emails"], len(customers))):
        customers.loc[idx, "email"] = f"bad-email-{idx}"
    start = ctx.issues["null_emails"] + ctx.issues["malformed_emails"]
    for idx in range(start, min(start + ctx.issues["whitespace_and_case"], len(customers))):
        customers.loc[idx, "first_name"] = f" {str(customers.loc[idx, 'first_name']).upper()} "
        customers.loc[idx, "loyalty_tier"] = str(customers.loc[idx, "loyalty_tier"]).upper()
    for idx in range(min(ctx.issues["invalid_state_codes"], len(address_frame))):
        address_frame.loc[idx, "state_code"] = ["VIRGINIA", "XX"][idx % 2]
    duplicate_count = min(ctx.issues["duplicate_business_keys"], max(0, len(customers) - 1))
    for idx in range(duplicate_count):
        customers.loc[len(customers) - 1 - idx, "customer_number"] = customers.loc[idx, "customer_number"]
    return customers, address_frame

def generate_orders(
    ctx: GenerationContext,
    refs: dict[str, pd.DataFrame],
    customers: pd.DataFrame,

```

```

rng: random.Random,
) -> dict[str, pd.DataFrame]:
    order_rows, item_rows, payment_rows, shipment_rows = [], [], [], []
    customer_lookup = customers.set_index("customer_id")["signup_date"].to_dict()
    customer_ids = customers["customer_id"].tolist()
    product_ids = refs["products"]["product_id"].tolist()
    product_lookup = refs["products"].set_index("product_id").to_dict("index")
    store_ids = refs["stores"]["store_id"].tolist()
    employee_by_store = refs["employees"].groupby("store_id")["employee_id"].apply(list).to_dict()
    max_order_date = date(2026, 7, 31)

    for i in range(ctx.counts["orders"]):
        order_id = 9_000_001 + i
        customer_id = int(rng.choice(customer_ids))
        signup = customer_lookup[customer_id]
        start = max(signup, date(2024, 1, 1))
        order_date = random_date(rng, start, max_order_date)
        hour = weighted_choice(rng, list(range(8, 23)), [1, 2, 3, 5, 7, 9, 10, 10, 9, 8, 7, 6, 5, 4, 2])
        order_ts = datetime.combine(order_date, datetime.min.time()) + timedelta(hours=hour, minutes=rng.randint(0, 59))
        channel = weighted_choice(rng, ["ONLINE", "STORE"], [0.56, 0.44])
        store_id = int(rng.choice(store_ids))
        employee_id = None if channel == "ONLINE" else int(rng.choice(employee_by_store[store_id]))
        status = weighted_choice(rng, ["COMPLETED", "SHIPPED", "CANCELLED"], [0.72, 0.24, 0.04])
        promo = rng.choice([None, None, None, "WELCOME10", "WEEKEND15", "FREESHIP"])
        order_rows.append({
            "order_id": order_id,
            "order_number": f"ORD-{order_id:09d}",
            "customer_id": customer_id,
            "store_id": store_id,
            "employee_id": employee_id,
            "channel_code": channel,
            "order_timestamp": order_ts,
            "order_status": status,
            "promotion_code": promo,
            "currency_code": "USD",
            "created_at": order_ts,
            "updated_at": ctx.generated_at.replace(tzinfo=None),
        })
        line_count = weighted_choice(rng, [1, 2, 3, 4, 5], [0.43, 0.29, 0.16, 0.08, 0.04])
        subtotal = tax = shipping = 0.0
        chosen = rng.sample(product_ids, k=min(line_count, len(product_ids)))
        for line_no, product_id in enumerate(chosen, start=1):
            product = product_lookup[product_id]
            quantity = weighted_choice(rng, [1, 2, 3, 4], [0.72, 0.20, 0.06, 0.02])
            unit_price = float(product["list_price"])
            discount_rate = 0.0 if promo is None else rng.choice([0.05, 0.10, 0.15])
            discount_amount = round(quantity * unit_price * discount_rate, 2)
            line_subtotal = round(quantity * unit_price - discount_amount, 2)
            line_tax = round(line_subtotal * rng.choice([0.0, 0.05, 0.06, 0.08]), 2)
            item_rows.append({
                "order_item_id": 10_000_001 + len(item_rows),
                "order_id": order_id,
                "line_number": line_no,
                "product_id": product_id,
                "quantity": quantity,
                "unit_price": unit_price,
                "unit_cost": float(product["unit_cost"]),
                "discount_amount": discount_amount,
                "tax_amount": line_tax,
                "created_at": order_ts,
                "updated_at": ctx.generated_at.replace(tzinfo=None),
            })
            subtotal += line_subtotal
            tax += line_tax
        shipping = 0 if channel == "STORE" and subtotal <= 75 else rng.choice([4.99, 7.99, 9.99])
        total = round(subtotal + tax + shipping, 2)
        payment_rows.append({
            "payment_id": 20_000_001 + i,
            "order_id": order_id,
            "payment_timestamp": order_ts + timedelta(minutes=rng.randint(1, 10)),
            "payment_method": rng.choice(["CARD", "GIFT_CARD", "DIGITAL_WALLET"], [0.73, 0.10, 0.17]),
            "payment_status": "REFUNDED" if status == "CANCELLED" else "CAPTURED",
            "amount": total,
            "currency_code": "USD",
            "transaction_reference": f"PAY-{uuid.UUID(int=rng.getrandbits(128)).hex[:16].upper()}",
            "created_at": order_ts,
            "updated_at": ctx.generated_at.replace(tzinfo=None),
        })
        if channel == "ONLINE" and status != "CANCELLED":
            promised = order_date + timedelta(days=rng.choice([2, 3, 4, 5]))
            shipped = order_date + timedelta(days=rng.choice([0, 1, 1, 2]))
            delivered = shipped + timedelta(days=rng.choice([1, 2, 3, 4, 6]))
            shipment_rows.append({
                "shipment_id": 30_000_001 + len(shipment_rows),
                "order_id": order_id,
                "carrier": rng.choice(["ParcelOne", "SwiftShip", "National Ground"]),
                "tracking_number": f"NST{order_id}{rng.randint(100, 999)}",
            })

```

```

        "ship_date": shipped,
        "promised_delivery_date": promised,
        "actual_delivery_date": delivered,
        "shipment_status": "DELIVERED",
        "shipping_cost": shipping,
        "created_at": datetime.combine(shipped, datetime.min.time()),
        "updated_at": ctx.generated_at.replace(tzinfo=None),
    })

orders = pd.DataFrame(order_rows)
items = pd.DataFrame(item_rows)
payments = pd.DataFrame(payment_rows)
shipments = pd.DataFrame(shipment_rows)
for idx in range(min(ctx.issues["impossible_quantities"], len(items))):
    items.loc[idx, "quantity"] = [-2, 0][idx % 2]
for idx in range(min(ctx.issues["currency_inconsistencies"], len(payments))):
    payments.loc[idx, "currency_code"] = ["usd", "EUR"][idx % 2]
return {"orders": orders, "order_items": items, "payments": payments, "shipments": shipments}

def generate_inventory(ctx: GenerationContext, refs: dict[str, pd.DataFrame], rng: random.Random) -> dict[str, pd.DataFrame]:
    transactions, snapshots = [], []
    snapshot_date = date(2026, 7, 31)
    for store_id in refs["stores"]["store_id"].tolist():
        sample_size = min(len(refs["products"]), max(10, len(refs["products"]) // 3))
        for product_id in rng.sample(refs["products"]["product_id"].tolist(), sample_size):
            starting = rng.randint(5, 120)
            received = rng.randint(0, 60)
            sold = rng.randint(0, min(50, starting + received))
            adjusted = rng.choice([-2, -1, 0, 0, 0, 1, 2])
            on_hand = starting + received - sold + adjusted
            base_id = 40_000_001 + len(transactions)
            for movement_type, qty, days_back in [
                ("OPENING", starting, 30), ("RECEIPT", received, 18), ("SALE", -sold, 5), ("ADJUSTMENT", adjusted, 1)
            ]:
                if qty == 0:
                    continue
                transactions.append({
                    "inventory_transaction_id": base_id + len(transactions),
                    "store_id": store_id,
                    "product_id": product_id,
                    "transaction_timestamp": datetime.combine(snapshot_date - timedelta(days=days_back), datetime.min.time()),
                    "transaction_type": movement_type,
                    "quantity_change": qty,
                    "reference_type": "SYNTHETIC",
                    "reference_id": None,
                    "created_at": ctx.generated_at.replace(tzinfo=None),
                })
            snapshots.append({
                "inventory_snapshot_id": 50_000_001 + len(snapshots),
                "snapshot_date": snapshot_date,
                "store_id": store_id,
                "product_id": product_id,
                "quantity_on_hand": on_hand,
                "quantity_reserved": min(max(0, on_hand), rng.randint(0, 8)),
                "reorder_point": rng.randint(5, 20),
                "created_at": ctx.generated_at.replace(tzinfo=None),
            })
    return {
        "inventory_transactions": pd.DataFrame(transactions),
        "inventory_snapshots": pd.DataFrame(snapshots),
    }

def generate_digital(
    ctx: GenerationContext,
    customers: pd.DataFrame,
    orders: pd.DataFrame,
    fake: Faker,
    rng: random.Random,
) -> dict[str, pd.DataFrame]:
    web_users = []
    linked_customer_ids = customers["customer_id"].sample(
        n=min(ctx.counts["web_users"], len(customers)), random_state=ctx.seed
    ).tolist()
    for i in range(ctx.counts["web_users"]):
        user_id = 700001 + i
        customer_id = linked_customer_ids[i] if i < len(linked_customer_ids) and rng.random() < 0.75 else None
        created = random_date(rng, date(2023, 1, 1), date(2026, 7, 1))
        web_users.append({
            "web_user_id": user_id,
            "customer_id": customer_id,
            "anonymous_cookie_id": uuid.UUID(int=rng.getrandbits(128)).hex,
            "email_hash": hashlib.sha256(f"web-{user_id}@example.test".encode()).hexdigest(),
            "created_at": datetime.combine(created, datetime.min.time()),
            "updated_at": ctx.generated_at.replace(tzinfo=None),
        })

```

```

web_users_df = pd.DataFrame(web_users)

campaigns = []
for i in range(ctx.counts["campaigns"]):
    campaign_id = 8001 + i
    start = random_date(rng, date(2025, 1, 1), date(2026, 6, 30))
    campaigns.append({
        "campaign_id": campaign_id,
        "campaign_code": f"CMP-{campaign_id:05d}",
        "campaign_name": f"{rng.choice(['Launch', 'Seasonal', 'Retention', 'Awareness'])} Campaign {i + 1}",
        "channel": rng.choice(CHANNELS[:-2]),
        "start_date": start,
        "end_date": start + timedelta(days=rng.choice([14, 30, 45, 60])),
        "budget_amount": round(rng.uniform(500, 25000), 2),
        "currency_code": "USD",
        "created_at": datetime.combine(start, datetime.min.time()),
        "updated_at": ctx.generated_at.replace(tzinfo=None),
    })
campaigns_df = pd.DataFrame(campaigns)

sessions, events, touchpoints = [], [], []
order_choices = orders[orders["channel_code"] == "ONLINE"]["order_id"].tolist()
for i in range(ctx.counts["web_sessions"]):
    session_id = 1_000_001 + i
    web_user_id = int(rng.choice(web_users_df["web_user_id"].tolist()))
    start_date = random_date(rng, date(2025, 1, 1), date(2026, 7, 31))
    started = datetime.combine(start_date, datetime.min.time()) + timedelta(
        hours=rng.randint(0, 23), minutes=rng.randint(0, 59), seconds=rng.randint(0, 59)
    )
    duration = max(5, int(rng.lognormvariate(4.2, 0.8)))
    ended = started + timedelta(seconds=duration)
    channel = weighted_choice(rng, CHANNELS, [0.24, 0.19, 0.11, 0.07, 0.25, 0.14])
    campaign_id = None
    if channel in {"Paid Search", "Social", "Email", "Affiliate"} and rng.random() < 0.75:
        campaign_id = int(rng.choice(campaigns_df["campaign_id"].tolist()))
    converted = rng.random() < (0.08 if channel in {"Email", "Paid Search"} else 0.035)
    order_id = int(rng.choice(order_choices)) if converted and order_choices else None
    sessions.append({
        "session_id": session_id,
        "web_user_id": web_user_id,
        "session_start_utc": started.isoformat() + "+00:00",
        "session_end_utc": ended.isoformat() + "+00:00",
        "source_channel": channel,
        "campaign_id": campaign_id,
        "device_type": weighted_choice(rng, ["mobile", "desktop", "tablet"], [0.61, 0.34, 0.05]),
        "country_code": "US",
        "converted_order_id": order_id,
        "created_at": ctx.generated_at.isoformat(),
    })
    event_types = EVENT_SEQUENCE if converted else EVENT_SEQUENCE[: rng.randint(1, 4)]
    current = started
    for event_type in event_types:
        current += timedelta(seconds=rng.randint(2, 80))
        events.append({
            "event_id": 2_000_001 + len(events),
            "session_id": session_id,
            "event_timestamp": current.isoformat() + "+00:00",
            "event_type": event_type,
            "page_url": f"https://northstar.example/{event_type.replace('_', '-')}",
            "product_id": None,
            "order_id": order_id if event_type == "purchase" else None,
            "event_properties": json.dumps({"synthetic": True, "sequence": len(events)}),
            "created_at": ctx.generated_at.isoformat(),
        })
    if campaign_id is not None:
        touchpoints.append({
            "touchpoint_id": 3_000_001 + len(touchpoints),
            "web_user_id": web_user_id,
            "session_id": session_id,
            "campaign_id": campaign_id,
            "touchpoint_timestamp": started.isoformat() + "+00:00",
            "touchpoint_type": "click",
            "attribution_weight": 1.0,
            "created_at": ctx.generated_at.isoformat(),
        })

sessions_df = pd.DataFrame(sessions)
events_df = pd.DataFrame(events)
for idx in range(min(ctx.issues["out_of_order_events"], len(events_df))):
    events_df.loc[idx, "event_timestamp"] = "2020-01-01T00:00:00+00:00"
for idx in range(min(ctx.issues["timezone_differences"], len(sessions_df))):
    value = pd.Timestamp(sessions_df.loc[idx, "session_start_utc"])
    sessions_df.loc[idx, "session_start_utc"] = value.tz_convert("America/New_York").isoformat()

spend = []
for _, campaign in campaigns_df.iterrows():
    current = campaign["start_date"]

```

```

while current <= campaign["end_date"]:
    spend.append({
        "spend_id": 4_000_001 + len(spend),
        "spend_date": current,
        "campaign_id": int(campaign["campaign_id"]),
        "channel": campaign["channel"],
        "spend_amount": round(rng.uniform(10, max(20, float(campaign["budget_amount"]) / 20)), 2),
        "impressions": rng.randint(100, 50000),
        "clicks": rng.randint(5, 2000),
        "currency_code": "USD",
        "created_at": ctx.generated_at.isoformat(),
    })
    current += timedelta(days=1)
return {
    "web_users": web_users_df,
    "web_sessions": sessions_df,
    "web_events": events_df,
    "campaigns": campaigns_df,
    "campaign_touchpoints": pd.DataFrame(touchpoints),
    "marketing_spend": pd.DataFrame(spend),
}

def generate_file_feeds(
    ctx: GenerationContext,
    refs: dict[str, pd.DataFrame],
    orders: dict[str, pd.DataFrame],
    rng: random.Random,
) -> dict[str, Any]:
    items = orders["order_items"]
    candidates = items[items["quantity"] > 0].sample(
        n=min(max(1, len(items) // 12), 10000), random_state=ctx.seed
    )
    returns = []
    order_dates = orders["orders"].set_index("order_id")["order_timestamp"].to_dict()
    for _, item in candidates.iterrows():
        purchase_dt = pd.Timestamp(order_dates[item["order_id"]]).to_pydatetime()
        return_dt = purchase_dt + timedelta(days=rng.randint(1, 45))
        returns.append({
            "return_id": f"RET-{len(returns)+1:08d}",
            "order_id": int(item["order_id"]),
            "order_item_id": int(item["order_item_id"]),
            "product_id": int(item["product_id"]),
            "customer_id": int(orders["orders"].loc[orders["orders"]["order_id"] == item["order_id"], "customer_id"].iloc[0]),
            "return_date": return_dt.date().isoformat(),
            "return_quantity": 1,
            "return_reason": rng.choice(["Damaged", "Wrong size", "Changed mind", "Late delivery", "Not as described"]),
            "refund_amount": round(float(item["unit_price"]) - float(item["discount_amount"]) / max(1, int(item["quantity"])), 2),
            "currency_code": "USD",
            "source_file_date": ctx.generated_at.date().isoformat(),
        })
    returns_df = pd.DataFrame(returns)
    for idx in range(min(ctx.issues["unknown_file_references"], len(returns_df))):
        returns_df.loc[idx, "product_id"] = 99999999 + idx
    for idx in range(min(ctx.issues["mixed_date_formats"], len(returns_df))):
        parsed = pd.Timestamp(returns_df.loc[idx, "return_date"])
        formats = ["%m/%d/%Y", "%d-%b-%Y", "%Y/%m/%d"]
        returns_df.loc[idx, "return_date"] = parsed.strftime(formats[idx % len(formats)])
    for idx in range(min(ctx.issues["late_arriving_records"], len(returns_df))):
        returns_df.loc[idx, "source_file_date"] = "2025-01-01"

    cost_updates = []
    for _, product in refs["products"].sample(n=min(len(refs["products"]), 1000), random_state=ctx.seed).iterrows():
        effective = random_date(rng, date(2026, 1, 1), date(2026, 7, 31))
        cost_updates.append({
            "supplier_cost_update_id": f"SCU-{len(cost_updates)+1:08d}",
            "supplier_code": refs["suppliers"].loc[
                refs["suppliers"]["supplier_id"] == product["supplier_id"], "supplier_code"
            ].iloc[0],
            "sku": product["sku"],
            "effective_date": effective.isoformat(),
            "new_unit_cost": round(float(product["unit_cost"]) * rng.uniform(0.92, 1.12), 2),
            "currency_code": "USD",
        })
    cost_df = pd.DataFrame(cost_updates)

    promotions = pd.DataFrame([
        {"promotion_code": "WELCOME10", "promotion_name": "New Customer Welcome", "start_date": "2025-01-01", "end_date": "2026-12-31", "dis-
count_type": "PERCENT", "discount_value": 10, "channel_code": "ONLINE"},
        {"promotion_code": "WEEKEND15", "promotion_name": "Weekend Event", "start_date": "2026-01-01", "end_date": "2026-12-31", "dis-
count_type": "PERCENT", "discount_value": 15, "channel_code": "ALL"},
        {"promotion_code": "FREESHIP", "promotion_name": "Free Shipping", "start_date": "2026-01-01", "end_date": "2026-12-31", "dis-
count_type": "SHIPPING", "discount_value": 100, "channel_code": "ONLINE"},
    ])

    tracking = []
    for _, shipment in orders["shipments"].iterrows():

```



```

        shipped = pd.Timestamp(shipment["ship_date"])
        delivered = pd.Timestamp(shipment["actual_delivery_date"])
        tracking.append({
            "tracking_number": shipment["tracking_number"],
            "order_id": int(shipment["order_id"]),
            "carrier": shipment["carrier"],
            "events": [
                {"event_type": "LABEL_CREATED", "event_timestamp": shipped.isoformat(), "location": {"city": "Origin", "country": "US"}},
                {"event_type": "IN_TRANSIT", "event_timestamp": (shipped + timedelta(days=1)).isoformat(), "location": {"city": "Hub", "country": "US"}},
                {"event_type": "DELIVERED", "event_timestamp": delivered.isoformat(), "location": {"city": "Destination", "country": "US"}},
            ],
        })
    }
    return {
        "returns.csv": returns_df,
        "supplier_cost_updates.csv": cost_df,
        "promotion_calendar.csv": promotions,
        "shipment_tracking_events.jsonl": tracking,
    }

def validate(frames: dict[str, pd.DataFrame]) -> list[dict[str, Any]]:
    tests = []
    def check(name: str, passed: bool, details: str) -> None:
        tests.append({"test": name, "status": "PASS" if passed else "FAIL", "details": details})
    check("customers_not_empty", len(frames["customers"]) > 0, f"rows={len(frames['customers'])}")
    check("orders_not_empty", len(frames["orders"]) > 0, f"rows={len(frames['orders'])}")
    normal_orders = set(frames["orders"]["customer_id"]) - set(frames["customers"]["customer_id"])
    check("orders_customer_fk", len(normal_orders) == 0, f"orphans={len(normal_orders)}")
    order_items_orphans = set(frames["order_items"]["order_id"]) - set(frames["orders"]["order_id"])
    check("items_order_fk", len(order_items_orphans) == 0, f"orphans={len(order_items_orphans)}")
    signup = frames["customers"].set_index("customer_id")["signup_date"].to_dict()
    invalid_dates = sum(pd.Timestamp(row.order_timestamp).date() < signup[row.customer_id] for row in frames["orders"].itertuples())
    check("signup_before_order", invalid_dates == 0, f"invalid={invalid_dates}")
    return tests

def write_outputs(
    ctx: GenerationContext,
    sqlserver: dict[str, pd.DataFrame],
    postgres: dict[str, pd.DataFrame],
    files: dict[str, Any],
) -> None:
    output = ctx.output_dir
    if output.exists():
        # Only remove generated subfolders, never the repository or arbitrary parent folder.
        for child in ["sqlserver", "postgres", "files", "metadata"]:
            target = output / child
            if target.exists():
                import shutil
                shutil.rmtree(target)
    manifests = []
    for name, frame in sqlserver.items():
        manifests.append({"source_system": "sqlserver", **write_frame(frame, output / "sqlserver" / f"{name}.csv")})
    for name, frame in postgres.items():
        manifests.append({"source_system": "postgres", **write_frame(frame, output / "postgres" / f"{name}.csv")})
    for filename, value in files.items():
        path = output / "files" / filename
        path.parent.mkdir(parents=True, exist_ok=True)
        if filename.endswith(".jsonl"):
            with path.open("w", encoding="utf-8") as handle:
                for record in value:
                    handle.write(json.dumps(record) + "\n")
            rows = len(value)
            manifests.append({"source_system": "files", "file": path.as_posix(), "rows": rows, "sha256": hash-
lib.sha256(path.read_bytes()).hexdigest()})
        else:
            manifests.append({"source_system": "files", **write_frame(value, path)})

all_frames = {**sqlserver, **postgres}
tests = validate(all_frames)
metadata = output / "metadata"
metadata.mkdir(parents=True, exist_ok=True)
(metadata / "batch_manifest.json").write_text(json.dumps({
    "batch_id": ctx.batch_id,
    "generated_at_utc": ctx.generated_at.isoformat(),
    "seed": ctx.seed,
    "scale": ctx.scale,
    "files": manifests,
}, indent=2, encoding="utf-8"))
(metadata / "validation_results.json").write_text(json.dumps(tests, indent=2, encoding="utf-8"))
expected = {
    "fixture_purpose": "Expected controlled issues for teaching Silver data-quality handling.",
    "seed": ctx.seed,
    "scale": ctx.scale,
    "configured_issue_counts": ctx.issues,
    "notes": [

```

```

        "A configured issue can affect more than one column.",
        "The fixture records injections, while validation code measures observed rows by rule.",
        "All names, contacts, companies, and identifiers are fictional.",
    ],
}
(metadata / "expected_quality_issues.json").write_text(json.dumps(expected, indent=2), encoding="utf-8")
summary = pd.DataFrame([{"source_system": item["source_system"], "file": Path(item["file"]).name, "rows": item["rows"]} for item in manifests])
summary.to_csv(metadata / "generation_summary.csv", index=False)
failed = [test for test in tests if test["status"] != "PASS"]
if failed:
    raise RuntimeError(f"Generation validation failed: {failed}")
LOGGER.info("Generated %s files for batch %s", len(manifests), ctx.batch_id)
LOGGER.info("Output folder: %s", output.resolve())

def main() -> int:
    configure_logging()
    args = parse_args()
    ctx = load_context(args)
    rng = random.Random(ctx.seed)
    Faker.seed(ctx.seed)
    fake = Faker("en_US")
    fake.seed_instance(ctx.seed)
    LOGGER.info("Generating scale=%s seed=%s", ctx.scale, ctx.seed)
    refs = generate_reference_data(ctx, fake, rng)
    customers, addresses = generate_customers(ctx, fake, rng)
    orders = generate_orders(ctx, refs, customers, rng)
    inventory = generate_inventory(ctx, refs, rng)
    sqlserver = {"**refs", "customers": customers, "addresses": addresses, "**orders", "**inventory"}
    postgres = generate_digital(ctx, customers, orders["orders"], fake, rng)
    file_feeds = generate_file_feeds(ctx, refs, orders, rng)
    write_outputs(ctx, sqlserver, postgres, file_feeds)
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

Line-by-line explanation

- Line 1:** `"""Generate reproducible, entirely fictional Northstar Retail source data. — """` is a quoted literal; the quotation marks preserve spaces or mark text; `Generate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `reproducible` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `entirely` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fictional` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Northstar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Retail` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it
- Line 2:** blank line used only to separate logical sections for readability.
- Line 3:** `Run from the repository root in an activated Python virtual environment: —` `Run` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `repository` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `root` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `an` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `activated` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Python` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `virtual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `environment` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `python -m scripts.data_generation.generate_northstar_data --scale quick --seed 20260815 —` `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module;

`scripts.data_generation.generate_northstar_data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--scale` is a command option or flag; its exact behavior is defined beside the command; `quick` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--seed` is a command option or flag; its exact behavior is defined beside the command; `20260815` is a numeric literal used by the command or expression

- **Line 5:** `""" — """` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 6:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `import argparse` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `argparse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `import hashlib` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `hashlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `import json` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `json` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** `import logging` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `logging` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `import random` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `random` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `import sys` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sys` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** `import uuid` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `uuid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** `from dataclasses import dataclass` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dataclasses` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dataclass` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `from datetime import UTC, date, datetime, timedelta` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UTC` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timedelta` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** `from pathlib import Path` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pathlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** `from typing import Any` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `typing` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or liter-

al whose project-specific role is described in the surrounding explanation; Any is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 19:** blank line used only to separate logical sections for readability.
- **Line 20:** `import pandas as pd` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pandas` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** `import yaml` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `yaml` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** `from faker import Faker` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `faker` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Faker` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** blank line used only to separate logical sections for readability.
- **Line 24:** `from scripts.utilities.logging_utils import configure_logging` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scripts.utilities.logging_utils` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `configure_logging` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** blank line used only to separate logical sections for readability.
- **Line 26:** `LOGGER = logging.getLogger("northstar.generator")` — `LOGGER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `logging.getLogger` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"northstar.generator"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:** blank line used only to separate logical sections for readability.
- **Line 28:** `REGIONS = {` — `REGIONS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** `"Northeast": [("NY", "New York"), ("MA", "Boston"), ("PA", "Philadelphia")]` — `"Northeast"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"NY"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"New York"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"MA"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Boston"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 30:** `"Mid-Atlantic": [("VA", "Arlington"), ("MD", "Bethesda"), ("DC", "Washington")]` — `"Mid-Atlantic"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"VA"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Arlington"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"MD"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

"Bethesda" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 31:** "Southeast": [("GA", "Atlanta"), ("NC", "Charlotte"), ("FL", "Orlando")], — "Southeast" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "GA" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Atlanta" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "NC" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Charlotte" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 32:** "Midwest": [("IL", "Chicago"), ("OH", "Columbus"), ("MI", "Detroit")], — "Midwest" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "IL" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Chicago" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "OH" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Columbus" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 33:** "West": [("CA", "San Diego"), ("WA", "Seattle"), ("CO", "Denver")], — "West" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "CA" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "San Diego" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "WA" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Seattle" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 34:** } — } is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** CATEGORIES = [— CATEGORIES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 36:** (1, "Electronics"), (2, "Home Office"), (3, "Kitchen"), (4, "Fitness"), — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Electronics" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Home Office" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.

- **Line 37:** (5, "Outdoor"), (6, "Personal Care"), (7, "Pet Supplies"), (8, "Travel"), — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 5 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Outdoor" is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 6 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Personal Care" is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 7 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 38:**] —] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 39:** CHANNELS = ["Paid Search", "Social", "Email", "Affiliate", "Organic", "Direct"] — CHANNELS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Paid Search" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Social" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Email" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Affiliate" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Organic" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 40:** EVENT_SEQUENCE = ["page_view", "product_view", "add_to_cart", "checkout_start", "purchase"] — EVENT_SEQUENCE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "page_view" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "product_view" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "add_to_cart" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "checkout_start" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "purchase" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 41:** BRANDS = ["Aurora Works", "Blue Peak", "Cedar & Finch", "Northline", "Orbit & Oak"] — BRANDS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Aurora Works" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Blue Peak" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Cedar & Finch" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Northline" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Orbit & Oak" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 42:** PRODUCT_NOUNS = ["Lamp", "Keyboard", "Bottle", "Backpack", "Blender", "Headphones", "Mat", "Organizer"] — PRODUCT_NOUNS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Lamp" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Keyboard" is a quoted literal; the quotation marks preserve spaces or mark text; , is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Bottle" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Backpack" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Blender" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 43:** blank line used only to separate logical sections for readability.
- **Line 44:** blank line used only to separate logical sections for readability.
- **Line 45:** `@dataclass(frozen=True)` — `dataclass` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frozen` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** `class GenerationContext:` — `class` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `GenerationContext` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 47:** `seed: int` — `seed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 48:** `scale: str` — `scale` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 49:** `output_dir: Path` — `output_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 50:** `counts: dict[str, int]` — `counts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dict` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 51:** `issues: dict[str, int]` — `issues` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dict` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 52:** `batch_id: str` — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 53:** `generated_at: datetime` — `generated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 54:** blank line used only to separate logical sections for readability.
- **Line 55:** blank line used only to separate logical sections for readability.

- **Line 56:** `def parse_args() -> argparse.Namespace:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parse_args` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `argparse.Namespace` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 57:** `parser = argparse.ArgumentParser(description="Generate fictional Northstar Retail data.")` — `parser` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `argparse.ArgumentParser` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `description` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Generate fictional Northstar Retail data."` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 58:** `parser.add_argument("--scale", choices=["quick", "small", "medium", "portfolio"], default="quick")` — `parser.add_argument` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--scale` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `choices` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `["quick"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"small"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"medium"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default="quick"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 59:** `parser.add_argument("--seed", type=int, default=20260815)` — `parser.add_argument` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--seed` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `type` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `20260815` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 60:** `parser.add_argument("--output-dir", type=Path, default=Path("datasets/sample"))` — `parser.add_argument` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--output-dir` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `type` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 61:** `parser.add_argument("--config", type=Path, default=Path("config/data_generation.yml"))` — `parser.add_argument` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--config` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `type` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 62:** `parser.add_argument(` — `parser.add_argument` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 63:** `--as-of-utc`, — `--as-of-utc` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 64:** `default="2026-08-15T12:00:00+00:00"`, — `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"2026-08-15T12:00:00+00:00"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 65:** `help="Deterministic dataset as-of timestamp; use ISO 8601."`, — `help` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Deterministic dataset as-of timestamp; use ISO 8601."` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 66:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 67:** `return parser.parse_args()` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parser.parse_args` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 68:** blank line used only to separate logical sections for readability.
- **Line 69:** blank line used only to separate logical sections for readability.
- **Line 70:** `def load_context(args: argparse.Namespace) -> GenerationContext:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `load_context` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `argparse.Namespace` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `GenerationContext` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 71:** `with args.config.open("r", encoding="utf-8") as handle:` — `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.config.open` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"r"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `encoding` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

"utf-8" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; as is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; handle is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

whose project-specific role is described in the surrounding explanation; 12 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.

- **Line 77:** `return GenerationType(` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `GenerationContext` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 78:** `seed=args.seed,` — `seed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.seed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 79:** `scale=args.scale,` — `scale` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.scale` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 80:** `output_dir=args.output_dir,` — `output_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.output_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 81:** `counts=counts,` — `counts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `counts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 82:** `issues=issues,` — `issues` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `issues` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 83:** `batch_id=f"GEN-{stamp:%Y%m%dT%H%M%SZ}-{stable}"`, — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"GEN-{stamp:%Y%m%dT%H%M%SZ}-{stable}"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 84:** `generated_at=stamp,` — `generated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `stamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 85:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 86:** blank line used only to separate logical sections for readability.
- **Line 87:** blank line used only to separate logical sections for readability.
- **Line 88:** `def weighted_choice(rng: random.Random, values: list[Any], weights: list[float]) -> Any:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `weighted_choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `random.Random` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `values` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `list` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

Any is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 89:** `return rng.choices(values, weights=weights, k=1)[0]` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choices` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `values` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `weights` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `weights` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `k` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 90:** blank line used only to separate logical sections for readability.
- **Line 91:** blank line used only to separate logical sections for readability.
- **Line 92:** `def write_frame(frame: pd.DataFrame, path: Path) -> dict[str, Any]:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `write_frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; remaining punctuation and identifiers continue the same statement.
- **Line 93:** `path.parent.mkdir(parents=True, exist_ok=True)` — `path.parent.mkdir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parents` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exist_ok` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 94:** `frame.to_csv(path, index=False, date_format="%Y-%m-%dT%H:%M:%S")` — `frame.to_csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `index` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `False` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date_format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"%Y-%m-%dT%H:%M:%S"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 95:** `digest = hashlib.sha256(path.read_bytes()).hexdigest()` — `digest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `hashlib.sha256` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.read_bytes` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `hexdigest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 96:** `return {"file": path.as_posix(), "rows": int(len(frame)), "sha256": digest}` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"file"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.as_posix` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"rows"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 97:** blank line used only to separate logical sections for readability.
- **Line 98:** blank line used only to separate logical sections for readability.
- **Line 99:** `def random_date(rng: random.Random, start: date, end: date) -> date:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `random_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `random.Random` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `start` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `end` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 100:** `return start + timedelta(days=rng.randint(0, max(0, (end - start).days)))` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `start` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timedelta` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `days` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.randint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `max` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 101:** blank line used only to separate logical sections for readability.
- **Line 102:** blank line used only to separate logical sections for readability.
- **Line 103:** `def generate_reference_data(ctx: GenerationContext, fake: Faker, rng: random.Random) -> dict[str, pd.DataFrame]:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `generate_reference_data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx: GenerationContext` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fake: Faker` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng: random.Random` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `) ->` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dict[str, pd.DataFrame]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

described in the surrounding explanation; `generate_reference_data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `GenerationContext` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fake` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Faker` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 104:** `now = ctx.generated_at.replace(tzinfo=None)` — `now` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.generated_at.replace` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tzinfo` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 105:** `categories = pd.DataFrame([` — `categories` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( [` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 106:** `{"category_id": cid, "category_name": name, "parent_category_id": None,` — `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"category_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"category_name"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"parent_category_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 107:** `"created_at": datetime(2024, 1, 1), "updated_at": now}` — `"created_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2024` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"updated_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 108:** `for cid, name in CATEGORIES` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument,

identifier, or literal whose project-specific role is described in the surrounding explanation; CATEGORIES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 109:** `)) — ))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 110:** blank line used only to separate logical sections for readability.
- **Line 111:** `suppliers_rows = []` — `suppliers_rows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 112:** `for i in range(ctx.counts["suppliers"]):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `range` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.counts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"suppliers"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 113:** `sid = 5001 + i` — `sid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `5001` is a numeric literal used by the command or expression; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 114:** `suppliers_rows.append({` — `suppliers_rows.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `({` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 115:** `"supplier_id": sid,` — `"supplier_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 116:** `"supplier_code": f"SUP-{sid:05d}",` — `"supplier_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SUP-{sid:05d}"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 117:** `"supplier_name": f"{fake.unique.company()} Supply",` — `"supplier_name"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{fake.unique.company()} Supply` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 118:** `"country_code": weighted_choice(rng, ["US", "CA", "MX", "GB"], [0.72, 0.12, 0.10, 0.06]),` — `"country_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `weighted_choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"US"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"CA"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"MX"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 119:** "lead_time_days": weighted_choice(rng, [3, 7, 10, 14, 21, 30], [0.08, 0.20, 0.25, 0.28, 0.14, 0.05]), — "lead_time_days" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; weighted_choice is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 7 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 10 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 120:** "active_flag": 1 if rng.random() > 0.04 else 0, — "active_flag" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.random is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; > is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0.04 is a numeric literal used by the command or expression; else is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 121:** "created_at": datetime(2024, 1, 1), — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datetime is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2024 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 122:** "updated_at": now, — "updated_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; now is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 123:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 124:** suppliers = pd.DataFrame(suppliers_rows) — suppliers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; suppliers_rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 125:** blank line used only to separate logical sections for readability.
- **Line 126:** products_rows = [] — products_rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 127:** for i in range(ctx.counts["products"]): — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; i is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; range is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.counts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"products"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 128:** `pid = 10001 + i` — `pid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `10001` is a numeric literal used by the command or expression; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 129:** `category_id, category_name = rng.choice(CATEGORIES)` — `category_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `category_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CATEGORIES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 130:** `cost = round(rng.lognormvariate(3.0, 0.65), 2)` — `cost` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `round` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.lognormvariate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3.0` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0.65` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 131:** `margin = rng.uniform(1.35, 2.8)` — `margin` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.uniform` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1.35` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2.8` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 132:** `products_rows.append({` — `products_rows.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 133:** `"product_id": pid,` — `"product_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 134:** `"sku": f"NRT-{{category_name[:3].upper()}}-{{pid}}",` — `"sku"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"NRT-{{category_name[:3].upper()}}-{{pid}}"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 135:** `"product_name": f"{{rng.choice(BRANDS)}} {rng.choice(PRODUCT_NOUNS)} {i + 1}",` — `"product_name"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier,

- or literal whose project-specific role is described in the surrounding explanation; "{rng.choice(BRANDS)} {rng.choice(PRODUCT_NOUNS)} {i + 1}" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 136:** "category_id": category_id, — "category_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; category_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 137:** "supplier_id": int(rng.choice(suppliers["supplier_id"].tolist())), — "supplier_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; int is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.choice is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; suppliers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "supplier_id" is a quoted literal; the quotation marks preserve spaces or mark text;] . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tolist is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ())), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 138:** "brand": rng.choice(BRANDS), — "brand" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.choice is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BRANDS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 139:** "unit_cost": cost, — "unit_cost" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cost is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 140:** "list_price": round(cost * margin, 2), — "list_price" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; round is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cost is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; * is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; margin is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 141:** "currency_code": "USD", — "currency_code" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "USD" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 142:** "active_flag": 1 if rng.random() > 0.03 else 0, — "active_flag" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.random is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; > is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0.03 is a numeric literal used by the command or expression; else is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 143:** "created_at": datetime(2024, 1, 1), — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datetime is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2024 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 144:** "updated_at": now, — "updated_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; now is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 145:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 146:** products = pd.DataFrame(products_rows) — products is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; products_rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 147:** blank line used only to separate logical sections for readability.
- **Line 148:** stores_rows = [] — stores_rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 149:** location_pool = [(region, state, city) for region, values in REGIONS.items() for state, city in values] — location_pool is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; region is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; state is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; city is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; region is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 150:** for i in range(ctx.counts["stores"]): — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; i is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; range is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.counts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "stores" is a quoted literal; the quotation marks preserve spaces or mark text;) : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 151:** store_id = 201 + i — store_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 201 is a numeric literal used by the command or expression; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

ation; i is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 152:** `region, state_code, city = location_pool[i % len(location_pool)]` — `region` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `state_code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `city` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `location_pool` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `location_pool` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 153:** `stores_rows.append({` — `stores_rows.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 154:** `"store_id": store_id,` — `"store_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `store_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 155:** `"store_code": f"{state_code}-{city[:3].upper()}-{i + 1:02d}"`, — `"store_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{state_code}-{city[:3].upper()}-{i + 1:02d}"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 156:** `"store_name": f"Northstar {city} {i // len(location_pool) + 1}"`, — `"store_name"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Northstar {city} {i // len(location_pool) + 1}"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 157:** `"region": region,` — `"region"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `region` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 158:** `"state_code": state_code,` — `"state_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `state_code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 159:** `"city": city,` — `"city"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `city` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 160:** `"open_date": random_date(rng, date(2017, 1, 1), date(2024, 12, 31))`, — `"open_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `random_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is

a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2017 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 161:** "active_flag": 1, — "active_flag" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 162:** "created_at": datetime(2024, 1, 1), — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datetime is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2024 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 163:** "updated_at": now, — "updated_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; now is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 164:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 165:** stores = pd.DataFrame(stores_rows) — stores is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stores_rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 166:** blank line used only to separate logical sections for readability.
- **Line 167:** employees_rows = [] — employees_rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 168:** for i in range(ctx.counts["employees"]): — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; i is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; range is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.counts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "employees" is a quoted literal; the quotation marks preserve spaces or mark text;) : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 169:** employee_id = 301 + i — employee_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 301 is a numeric literal used by the command or expression; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; i is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 170:** store_id = int(stores.iloc[i % len(stores)]["store_id"]) — store_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `stores.iloc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `stores` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` `]` `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"store_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 171:** `employees_rows.append({` — `employees_rows.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 172:** `"employee_id": employee_id,` — `"employee_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `employee_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 173:** `"employee_number": f"EMP-{employee_id:05d}",` — `"employee_number"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"EMP-{employee_id:05d}"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 174:** `"store_id": store_id,` — `"store_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `store_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 175:** `"manager_employee_id": None,` — `"manager_employee_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 176:** `"first_name": fake.first_name(),` — `"first_name"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fake.first_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 177:** `"last_name": fake.last_name(),` — `"last_name"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fake.last_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 178:** `"job_title": weighted_choice(rng, ["Sales Associate", "Store Manager", "Inventory Specialist"], [0.72, 0.10, 0.18]),` — `"job_title"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `weighted_choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Sales Associate"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Store Manager"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the

surrounding explanation; "Inventory Specialist" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 179:** "hire_date": random_date(rng, date(2019, 1, 1), date(2025, 12, 31)), — "hire_date" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; random_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2019 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 180:** "active_flag": 1 if rng.random() > 0.05 else 0, — "active_flag" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.random is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; > is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0.05 is a numeric literal used by the command or expression; else is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 181:** "created_at": datetime(2024, 1, 1), — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datetime is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2024 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 182:** "updated_at": now, — "updated_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; now is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 183:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 184:** employees = pd.DataFrame(employees_rows) — employees is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; employees_rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 185:** for store_id, group in employees.groupby("store_id"): — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; store_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; group is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; employees.groupby is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "store_id" is a quoted literal; the quotation marks preserve spaces

or mark text;) : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 186:** `manager_id = int(group.iloc[0]["employee_id"])` — `manager_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `group.iloc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `]` `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"employee_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 187:** `employees.loc[group.index, "manager_employee_id"] = manager_id` — `employees.loc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `group.index` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"manager_employee_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manager_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 188:** `employees.loc[group.index[0], "job_title"] = "Store Manager"` — `employees.loc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `group.index` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `]` `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"job_title"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Store Manager"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 189:** `employees.loc[group.index[0], "manager_employee_id"] = None` — `employees.loc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `group.index` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `]` `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"manager_employee_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 190:** blank line used only to separate logical sections for readability.
- **Line 191:** `return {` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 192:** `"product_categories": categories,` — `"product_categories"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `categories` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 193:** `"suppliers": suppliers,` — `"suppliers"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `suppliers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in

the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 194:** "products": products, — "products" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; products is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 195:** "stores": stores, — "stores" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stores is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 196:** "employees": employees, — "employees" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; employees is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 197:** } — } is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 198:** blank line used only to separate logical sections for readability.
- **Line 199:** blank line used only to separate logical sections for readability.
- **Line 200:** def generate_customers(ctx: GenerationContext, fake: Faker, rng: random.Random) -> tuple[pd.DataFrame, pd.DataFrame]: — def is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; generate_customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; GenerationContext is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fake is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Faker is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 201:** today = date(2026, 8, 1) — today is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2026 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 8 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 202:** rows, addresses = [], [] — rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; addresses is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 203:** for i in range(ctx.counts["customers"]): — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; i is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; range is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.counts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"customers"` is a quoted literal; the quotation marks preserve spaces or mark text; `])` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 204:** `customer_id = 100001 + i` — `customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `100001` is a numeric literal used by the command or expression; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 205:** `signup = random_date(rng, date(2023, 1, 1), today - timedelta(days=7))` — `signup` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `random_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2023` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 206:** `first, last = fake.first_name(), fake.last_name()` — `first` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `last` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fake.first_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fake.last_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 207:** `customer_number = f"CUS-{customer_id:07d}"` — `customer_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"CUS-{customer_id:07d}"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 208:** `email = f"{first}.{last}.{customer_id}@example.test".lower().replace(" ", "")` — `email` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{first}.{last}.{customer_id}@example.test"` is a quoted literal; the quotation marks preserve spaces or mark text; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `lower` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `replace` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `" "` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `" "` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 209:** `rows.append({` — `rows.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 210:** `"customer_id": customer_id,` — `"customer_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

surrounding explanation; `customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 211:** `"customer_number": customer_number, — "customer_number" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation`
- **Line 212:** `"first_name": first, — "first_name" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; first is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation`
- **Line 213:** `"last_name": last, — "last_name" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; last is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation`
- **Line 214:** `"email": email, — "email" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; email is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation`
- **Line 215:** `"phone": fake.numerify("+1-555-####"), — "phone" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fake.numerify is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; " +1-555-####" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation`
- **Line 216:** `"signup_date": signup, — "signup_date" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; signup is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation`
- **Line 217:** `"loyalty_tier": weighted_choice(rng, ["Bronze", "Silver", "Gold", "Platinum"], [0.48, 0.30, 0.17, 0.05]), — "loyalty_tier" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; weighted_choice is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Bronze" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Silver" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Gold" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.`
- **Line 218:** `"marketing_opt_in": 1 if rng.random() < 0.68 else 0, — "marketing_opt_in" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.random is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; < is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0.68 is a numeric literal used by the command or expression; else is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;`

0 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 219:** "created_at": datetime.combine(signup, datetime.min.time()), — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datetime.combine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; signup is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datetime.min.time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 220:** "updated_at": ctx.generated_at.replace(tzinfo=None), — "updated_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.generated_at.replace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tzinfo is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 221:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 222:** region, candidates = rng.choice(list(REGIONS.items())) — region is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; candidates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.choice is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; list is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REGIONS.items is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ()) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 223:** state_code, city = rng.choice(candidates) — state_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; city is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.choice is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; candidates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 224:** addresses.append({ — addresses.append is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ({ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 225:** "address_id": 500001 + i, — "address_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 500001 is a numeric literal used by the command or expression; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; i is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 226:** "customer_id": customer_id, — "customer_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 227:** "address_type": "Shipping", — "address_type" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Shipping" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 228:** "line1": fake.street_address(), — "line1" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fake.street_address is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 229:** "line2": None, — "line2" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 230:** "city": city, — "city" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; city is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 231:** "state_code": state_code, — "state_code" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; state_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 232:** "postal_code": fake.postcode()[:10], — "postal_code" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fake.postcode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ()[: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 10 is a numeric literal used by the command or expression;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 233:** "country_code": "US", — "country_code" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "US" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 234:** "is_default": 1, — "is_default" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 235:** "created_at": datetime.combine(signup, datetime.min.time()), — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datetime.combine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; signup is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datetime.min.time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ()), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 236:** "updated_at": ctx.generated_at.replace(tzinfo=None), — "updated_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.generated_at.replace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tzinfo is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tion;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 237:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 238:** customers = pd.DataFrame(rows) — customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 239:** address_frame = pd.DataFrame(addresses) — address_frame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; addresses is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 240:** blank line used only to separate logical sections for readability.
- **Line 241:** # Controlled issues. These counts are deterministic and recorded in the fixture. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 242:** for idx in range(min(ctx.issues["null_emails"], len(customers))): — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; idx is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; range is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; min is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.issues is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "null_emails" is a quoted literal; the quotation marks preserve spaces or mark text;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; len is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 243:** customers.loc[idx, "email"] = None — customers.loc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; idx is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "email" is a quoted literal; the quotation marks preserve spaces or mark text;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 244:** for idx in range(ctx.issues["null_emails"], min(ctx.issues["null_emails"] + ctx.issues["malformed_emails"], len(customers))): — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; idx is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; range is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.issues is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "null_emails" is a quoted literal; the quotation marks preserve spaces or mark text;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; min is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

ctx.issues is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- [illegible]

described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customers.loc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; idx is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 250:** for idx in range(min(ctx.issues["invalid_state_codes"], len(address_frame))): — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; idx is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; range is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; min is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.issues is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "invalid_state_codes" is a quoted literal; the quotation marks preserve spaces or mark text;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; len is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 251:** address_frame.loc[idx, "state_code"] = ["VIRGINIA", "XX"][idx % 2] — address_frame.loc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; idx is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "state_code" is a quoted literal; the quotation marks preserve spaces or mark text;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "VIRGINIA" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "XX" is a quoted literal; the quotation marks preserve spaces or mark text;] [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 252:** duplicate_count = min(ctx.issues["duplicate_business_keys"], max(0, len(customers) - 1)) — duplicate_count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; min is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.issues is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "duplicate_business_keys" is a quoted literal; the quotation marks preserve spaces or mark text;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; max is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 253:** for idx in range(duplicate_count): — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; idx is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; range is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; duplicate_count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 254:** customers.loc[len(customers) - 1 - idx, "customer_number"] = customers.loc[idx, "customer_number"] — customers.loc is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-` is a command option or flag; its exact behavior is defined beside the command; `1` is a numeric literal used by the command or expression; `-` is a command option or flag; its exact behavior is defined beside the command; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"customer_number"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 255:** `return customers, address_frame` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `address_frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 256:** blank line used only to separate logical sections for readability.
- **Line 257:** blank line used only to separate logical sections for readability.
- **Line 258:** `def generate_orders(` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `generate_orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 259:** `ctx: GenerationContext,` — `ctx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `GenerationContext` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 260:** `refs: dict[str, pd.DataFrame],` — `refs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dict` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 261:** `customers: pd.DataFrame,` — `customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 262:** `rng: random.Random,` — `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `random.Random` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 263:** `) -> dict[str, pd.DataFrame]:` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `dict` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 264:** `order_rows, item_rows, payment_rows, shipment_rows = [], [], [], []` — `order_rows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; item_rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; payment_rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; shipment_rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

- **Line 269:** `store_ids = refs["stores"]["store_id"].tolist()` — `store_ids` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `refs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"stores"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"store_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tolist` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 270:** `employee_by_store = refs["employees"].groupby("store_id")["employee_id"].apply(list).to_dict()` — `employee_by_store` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `refs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"employees"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `groupby` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"store_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"employee_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 271:** `max_order_date = date(2026, 7, 31)` — `max_order_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2026` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `7` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `31` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 272:** blank line used only to separate logical sections for readability.
- **Line 273:** `for i in range(ctx.counts["orders"]):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `range` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.counts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"orders"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 274:** `order_id = 9_000_001 + i` — `order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `9_000_001` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 275:** `customer_id = int(rng.choice(customer_ids))` — `customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer_ids` is a name, keyword, argument, identifier, or literal whose project-

specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 276:** `signup = customer_lookup[customer_id]` — `signup` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer_lookup` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 277:** `start = max(signup, date(2024, 1, 1))` — `start` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `max` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `signup` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2024` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 278:** `order_date = random_date(rng, start, max_order_date)` — `order_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `random_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `start` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `max_order_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 279:** `hour = weighted_choice(rng, list(range(8, 23)), [1, 2, 3, 5, 7, 9, 10, 10, 9, 8, 7, 6, 5, 4, 2])` — `hour` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `weighted_choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `list` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `range` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `8` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 280:** `order_ts = datetime.combine(order_date, datetime.min.time()) + timedelta(hours=hour, minutes=rng.randint(0, 59))` — `order_ts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime.combine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime.min.time` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

described in the surrounding explanation; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timedelta is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; hours is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 281:** `channel = weighted_choice(rng, ["ONLINE", "STORE"], [0.56, 0.44])` — `channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `weighted_choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"ONLINE"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"STORE"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 282:** `store_id = int(rng.choice(store_ids))` — `store_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `store_ids` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.
- **Line 283:** `employee_id = None if channel == "ONLINE" else int(rng.choice(employee_by_store[store_id]))` — `employee_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"ONLINE"` is a quoted literal; the quotation marks preserve spaces or mark text; `else` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 284:** `status = weighted_choice(rng, ["COMPLETED", "SHIPPED", "CANCELLED"], [0.72, 0.24, 0.04])` — `status` shows tracked, staged, modified, and untracked work; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `weighted_choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"COMPLETED"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SHIPPED"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"CANCELLED"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 285:** `promo = rng.choice([None, None, None, "WELCOME10", "WEEKEND15", "FREESHIP"])` — `promo` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `([` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"WELCOME10"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 286:** `order_rows.append({` — `order_rows.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `({` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 287:** `"order_id": order_id,` — `"order_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 288:** `"order_number": f"ORD-{order_id:09d}"`, — `"order_number"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"ORD-{order_id:09d}"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 289:** `"customer_id": customer_id,` — `"customer_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 290:** `"store_id": store_id,` — `"store_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `store_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 291:** `"employee_id": employee_id,` — `"employee_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `employee_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 292:** `"channel_code": channel,` — `"channel_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 293:** `"order_timestamp": order_ts,` — `"order_timestamp"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_ts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 294:** `"order_status": status,` — `"order_status"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `status` shows tracked, staged, modified, and untracked work; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 295:** "promotion_code": promo, — "promotion_code" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; promo is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 296:** "currency_code": "USD", — "currency_code" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "USD" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 297:** "created_at": order_ts, — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_ts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 298:** "updated_at": ctx.generated_at.replace(tzinfo=None), — "updated_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.generated_at.replace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tzinfo is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 299:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 300:** line_count = weighted_choice(rng, [1, 2, 3, 4, 5], [0.43, 0.29, 0.16, 0.08, 0.04]) — line_count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; weighted_choice is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 301:** subtotal = tax = shipping = 0.0 — subtotal is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tax is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; shipping is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0.0 is a numeric literal used by the command or expression
- **Line 302:** chosen = rng.sample(product_ids, k=min(line_count, len(product_ids))) — chosen is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.sample is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product_ids is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; k is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; min is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; line_count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 303:** `for line_no, product_id in enumerate(chosen, start=1):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `line_no` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `product_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `enumerate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `chosen` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `start` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 304:** `product = product_lookup[product_id]` — `product` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `product_lookup` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `product_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 305:** `quantity = weighted_choice(rng, [1, 2, 3, 4], [0.72, 0.20, 0.06, 0.02])` — `quantity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `weighted_choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 306:** `unit_price = float(product["list_price"])` — `unit_price` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `float` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `product` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"list_price"` is a quoted literal; the quotation marks preserve spaces or mark text; `])` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 307:** `discount_rate = 0.0 if promo is None else rng.choice([0.05, 0.10, 0.15])` — `discount_rate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0.0` is a numeric literal used by the command or expression; `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `promo` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `else` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0.05` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 308:** `discount_amount = round(quantity * unit_price * discount_rate, 2)` — `discount_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `round` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quantity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `unit_price` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `discount_rate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 309:** `line_subtotal = round(quantity * unit_price - discount_amount, 2)` — `line_subtotal` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `round` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quantity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `unit_price` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-` is a command option or flag; its exact behavior is defined beside the command; `discount_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 310:** `line_tax = round(line_subtotal * rng.choice([0.0, 0.05, 0.06, 0.08]), 2)` — `line_tax` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `round` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `line_subtotal` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0.0` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0.05` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 311:** `item_rows.append({` — `item_rows.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 312:** `"order_item_id": 10_000_001 + len(item_rows),` — `"order_item_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `10_000_001` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `item_rows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 313:** `"order_id": order_id,` — `"order_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 314:** "line_number": line_no, — "line_number" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; line_no is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 315:** "product_id": product_id, — "product_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 316:** "quantity": quantity, — "quantity" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quantity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 317:** "unit_price": unit_price, — "unit_price" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; unit_price is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 318:** "unit_cost": float(product["unit_cost"]), — "unit_cost" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; float is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "unit_cost" is a quoted literal; the quotation marks preserve spaces or mark text;]) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 319:** "discount_amount": discount_amount, — "discount_amount" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; discount_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 320:** "tax_amount": line_tax, — "tax_amount" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; line_tax is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 321:** "created_at": order_ts, — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_ts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 322:** "updated_at": ctx.generated_at.replace(tzinfo=None), — "updated_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.generated_at.replace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tzinfo is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 323:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 324:** subtotal += line_subtotal — subtotal is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; += is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; `line_subtotal` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 325:** `tax += line_tax` — `tax` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `line_tax` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 326:** `shipping = 0 if channel == "STORE" or subtotal >= 75 else rng.choice([4.99, 7.99, 9.99])` — `shipping` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"STORE"` is a quoted literal; the quotation marks preserve spaces or mark text; `or` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `subtotal` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `75` is a numeric literal used by the command or expression; `else` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 327:** `total = round(subtotal + tax + shipping, 2)` — `total` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `round` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `subtotal` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tax` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `shipping` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 328:** `payment_rows.append({` — `payment_rows.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 329:** `"payment_id": 20_000_001 + i,` — `"payment_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `20_000_001` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 330:** `"order_id": order_id,` — `"order_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 331:** `"payment_timestamp": order_ts + timedelta(minutes=rng.randint(1, 10)),` — `"payment_timestamp"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_ts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timedelta` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `minutes` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.randint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `,` is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 332:** "payment_method": weighted_choice(rng, ["CARD", "GIFT_CARD", "DIGITAL_WALLET"], [0.73, 0.10, 0.17]), — "payment_method" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; weighted_choice is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "CARD" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "GIFT_CARD" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "DIGITAL_WALLET" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 333:** "payment_status": "REFUNDED" if status == "CANCELLED" else "CAPTURED", — "payment_status" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "REFUNDED" is a quoted literal; the quotation marks preserve spaces or mark text; if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; status shows tracked, staged, modified, and untracked work; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "CANCELLED" is a quoted literal; the quotation marks preserve spaces or mark text; else is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "CAPTURED" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 334:** "amount": total, — "amount" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; total is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 335:** "currency_code": "USD", — "currency_code" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "USD" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 336:** "transaction_reference": f"PAY-{uuid.UUID(int=rng.getrandbits(128)).hex[:16].upper()}", — "transaction_reference" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "PAY-{uuid.UUID(int=rng.getrandbits(128)).hex[:16].upper()}" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 337:** "created_at": order_ts, — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_ts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 338:** "updated_at": ctx.generated_at.replace(tzinfo=None), — "updated_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.generated_at.replace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tzinfo is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 339:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 340:** `if channel == "ONLINE" and status != "CANCELLED":` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"ONLINE"` is a quoted literal; the quotation marks preserve spaces or mark text; and `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `status` shows tracked, staged, modified, and untracked work; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"CANCELLED"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 341:** `promised = order_date + timedelta(days=rng.choice([2, 3, 4, 5]))` — `promised` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timedelta` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `days` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 342:** `shipped = order_date + timedelta(days=rng.choice([0, 1, 1, 2]))` — `shipped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timedelta` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `days` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 343:** `delivered = shipped + timedelta(days=rng.choice([1, 2, 3, 4, 6]))` — `delivered` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `shipped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timedelta` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `days` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 344:** `shipment_rows.append({` — `shipment_rows.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 345:** `"shipment_id": 30_000_001 + len(shipment_rows),` — `"shipment_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30_000_001` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument,

- identifier, or literal whose project-specific role is described in the surrounding explanation; shipment_rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 346:** "order_id": order_id, — "order_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 347:** "carrier": rng.choice(["ParcelOne", "SwiftShip", "National Ground"]), — "carrier" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.choice is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ([is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "ParcelOne" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "SwiftShip" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "National Ground" is a quoted literal; the quotation marks preserve spaces or mark text;]) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 348:** "tracking_number": f"NST{order_id}{rng.randint(100, 999)}", — "tracking_number" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "NST{order_id}{rng.randint(100, 999)}" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 349:** "ship_date": shipped, — "ship_date" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; shipped is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 350:** "promised_delivery_date": promised, — "promised_delivery_date" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; promised is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 351:** "actual_delivery_date": delivered, — "actual_delivery_date" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; delivered is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 352:** "shipment_status": "DELIVERED", — "shipment_status" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "DELIVERED" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 353:** "shipping_cost": shipping, — "shipping_cost" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; shipping is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 354:** "created_at": datetime.combine(shipped, datetime.min.time()), — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datetime.combine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; shipped is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datetime.min.time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ()) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 355:** `"updated_at": ctx.generated_at.replace(tzinfo=None),` — `"updated_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.generated_at.replace` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tzinfo` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 356:** `})` — `})` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 357:** blank line used only to separate logical sections for readability.
- **Line 358:** `orders = pd.DataFrame(order_rows)` — `orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_rows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 359:** `items = pd.DataFrame(item_rows)` — `items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `item_rows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 360:** `payments = pd.DataFrame(payment_rows)` — `payments` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `payment_rows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 361:** `shipments = pd.DataFrame(shipment_rows)` — `shipments` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `shipment_rows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 362:** `for idx in range(min(ctx.issues["impossible_quantities"], len(items))):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `range` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `min` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.issues` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"impossible_quantities"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 363:** `items.loc[idx, "quantity"] = [-2, 0][idx % 2]` — `items.loc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quantity"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[-` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `][` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 364:** `for idx in range(min(ctx.issues["currency_inconsistencies"], len(payments))):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `range` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `min` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.issues` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"currency_inconsistencies"` is a quoted literal; the quotation marks preserve spaces or mark text; `]`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 365:** `payments.loc[idx, "currency_code"] = ["usd", "EUR"][idx % 2]` — `payments.loc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"currency_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"usd"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"EUR"` is a quoted literal; the quotation marks preserve spaces or mark text; `][` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 366:** `return {"orders": orders, "order_items": items, "payments": payments, "shipments": shipments}` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"orders"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_items"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"payments"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 367:** blank line used only to separate logical sections for readability.
- **Line 368:** blank line used only to separate logical sections for readability.
- **Line 369:** `def generate_inventory(ctx: GenerationContext, refs: dict[str, pd.DataFrame], rng: random.Random) -> dict[str, pd.DataFrame]:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `generate_inventory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx: GenerationContext` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `refs: dict[str, pd.DataFrame]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng: random.Random` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `) ->` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dict[str, pd.DataFrame]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

ject-specific role is described in the surrounding explanation; `generate_inventory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `GenerationContext` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `refs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dict` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 370:** `transactions, snapshots = [], []` — `transactions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `snapshots` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[],` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 371:** `snapshot_date = date(2026, 7, 31)` — `snapshot_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2026` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `7` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `31` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 372:** `for store_id in refs["stores"]["store_id"].tolist():` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `store_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `refs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"stores"` is a quoted literal; the quotation marks preserve spaces or mark text; `][` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"store_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `].` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tolist` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `():` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 373:** `sample_size = min(len(refs["products"]), max(10, len(refs["products"]) // 3))` — `sample_size` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `min` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `refs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"products"` is a quoted literal; the quotation marks preserve spaces or mark text; `]),` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `max` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- [illegible]

fier, or literal whose project-specific role is described in the surrounding explanation; starting is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; received is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; - is a command option or flag; its exact behavior is defined beside the command; sold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; adjusted is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 380:** `base_id = 40_000_001 + len(transactions) —` `base_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `40_000_001` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `transactions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 381:** `for movement_type, qty, days_back in [` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `movement_type` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `qty` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `days_back` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 382:** `("OPENING", starting, 30), ("RECEIPT", received, 18), ("SALE", -sold, 5), ("ADJUSTMENT", adjusted, 1) —` `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"OPENING"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `starting` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"RECEIPT"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `received` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 383:** `]:` — `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 384:** `if qty == 0:` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `qty` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 385:** `continue` — `continue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 386:** `transactions.append({` — `transactions.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 387:** `"inventory_transaction_id": base_id + len(transactions),` — `"inventory_transaction_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `base_id` is a name, keyword, argument, identifier, or

literal whose project-specific role is described in the surrounding explanation; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; len is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; trans- actions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 388:** "store_id": store_id, — "store_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; store_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 389:** "product_id": product_id, — "product_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 390:** "transaction_timestamp": datetime.combine(snapshot_date - timedelta(days=days_back), date- time.min.time()), — "transaction_timestamp" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datetime.combine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; snapshot_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; - is a command option or flag; its exact behavior is defined beside the command; timedelta is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; days is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; days_back is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 391:** "transaction_type": movement_type, — "transaction_type" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; movement_type is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 392:** "quantity_change": qty, — "quantity_change" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; qty is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 393:** "reference_type": "SYNTHETIC", — "reference_type" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "SYNTHETIC" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 394:** "reference_id": None, — "reference_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 395:** "created_at": ctx.generated_at.replace(tzinfo=None), — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.generated_at.replace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tzinfo is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tion;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 396:** `}}` — `}}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 397:** `snapshots.append({` — `snapshots.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 398:** `"inventory_snapshot_id": 50_000_001 + len(snapshots),` — `"inventory_snapshot_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `50_000_001` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `snapshots` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 399:** `"snapshot_date": snapshot_date,` — `"snapshot_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `snapshot_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 400:** `"store_id": store_id,` — `"store_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `store_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 401:** `"product_id": product_id,` — `"product_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `product_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 402:** `"quantity_on_hand": on_hand,` — `"quantity_on_hand"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `on_hand` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 403:** `"quantity_reserved": min(max(0, on_hand), rng.randint(0, 8)),` — `"quantity_reserved"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `min` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `max` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `on_hand` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.randint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 404:** `"reorder_point": rng.randint(5, 20),` — `"reorder_point"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.randint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `5` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `20` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 405:** "created_at": ctx.generated_at.replace(tzinfo=None), — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.generated_at.replace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tzinfo is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 406:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 407:** return { — return is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 408:** "inventory_transactions": pd.DataFrame(transactions), — "inventory_transactions" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; transactions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 409:** "inventory_snapshots": pd.DataFrame(snapshots), — "inventory_snapshots" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; snapshots is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 410:** } — } is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 411:** blank line used only to separate logical sections for readability.
- **Line 412:** blank line used only to separate logical sections for readability.
- **Line 413:** def generate_digital(— def is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; generate_digital is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 414:** ctx: GenerationContext, — ctx is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; GenerationContext is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 415:** customers: pd.DataFrame, — customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 416:** orders: pd.DataFrame, — orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 417:** fake: Faker, — fake is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Faker is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 418:** rng: random.Random, — rng is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

role is described in the surrounding explanation; `random`. `Random` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 419:** `) -> dict[str, pd.DataFrame]:` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `dict` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 420:** `web_users = []` — `web_users` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 421:** `linked_customer_ids = customers["customer_id"].sample(` — `linked_customer_ids` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"customer_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sample` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 422:** `n=min(ctx.counts["web_users"], len(customers)), random_state=ctx.seed` — `n` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `min` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.counts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web_users"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 423:** `) .tolist()` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tolist` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 424:** `for i in range(ctx.counts["web_users"]):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `range` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.counts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web_users"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 425:** `user_id = 700001 + i` — `user_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `700001` is a numeric literal used by the command or expression; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding

explanation; i is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 426:** `customer_id = linked_customer_ids[i]` if `i < len(linked_customer_ids)` and `rng.random() < 0.75` else `None` — `customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `linked_customer_ids` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `linked_customer_ids` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 427:** `created = random_date(rng, date(2023, 1, 1), date(2026, 7, 1))` — `created` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `random_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2023` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 428:** `web_users.append({` — `web_users.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 429:** `"web_user_id": user_id,` — `"web_user_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `user_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 430:** `"customer_id": customer_id,` — `"customer_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 431:** `"anonymous_cookie_id": uuid.UUID(int=rng.getrandbits(128)).hex,` — `"anonymous_cookie_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `uuid.UUID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.getrandbits` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `128` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `hex` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 432:** `"email_hash": hashlib.sha256(f"web-{user_id}@example.test").encode()).hexdigest(),` — `"email_hash"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or lit-

eral whose project-specific role is described in the surrounding explanation; `hashlib.sha256` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web-{user_id}@example.test"` is a quoted literal; the quotation marks preserve spaces or mark text; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `encode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( ) .` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `hexdigest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( ) ,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 433:** `"created_at": datetime.combine(created, datetime.min.time())`, — `"created_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime.combine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `created` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime.min.time` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( ) ,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 434:** `"updated_at": ctx.generated_at.replace(tzinfo=None)`, — `"updated_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.generated_at.replace` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tzinfo` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `) ,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 435:** `} )` — `} )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 436:** `web_users_df = pd.DataFrame(web_users)` — `web_users_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `web_users` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 437:** blank line used only to separate logical sections for readability.
- **Line 438:** `campaigns = []` — `campaigns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 439:** `for i in range(ctx.counts["campaigns"]):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `range` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.counts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"campaigns"` is a quoted literal; the quotation marks preserve spaces or mark text; `)] :` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 440:** `campaign_id = 8001 + i` — `campaign_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

project-specific role is described in the surrounding explanation; 8001 is a numeric literal used by the command or expression; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; i is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 441:** `start = random_date(rng, date(2025, 1, 1), date(2026, 6, 30))` — `start` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `random_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2025` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 442:** `campaigns.append({` — `campaigns.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( {` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 443:** `"campaign_id": campaign_id,` — `"campaign_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `campaign_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 444:** `"campaign_code": f"CMP-{campaign_id:05d}"`, — `"campaign_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"CMP-{campaign_id:05d}"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 445:** `"campaign_name": f"{rng.choice(['Launch', 'Seasonal', 'Retention', 'Awareness'])} Campaign {i + 1}"`, — `"campaign_name"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{rng.choice(['Launch', 'Seasonal', 'Retention', 'Awareness'])} Campaign {i + 1}"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 446:** `"channel": rng.choice(CHANNELS[:-2])`, — `"channel"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CHANNELS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[ :-` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression; `)]`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 447:** `"start_date": start,` — `"start_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `start` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 448:** `"end_date": start + timedelta(days=rng.choice([14, 30, 45, 60]))`, — `"end_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `start` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timedelta` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; days is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.choice is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 14 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 449:** "budget_amount": round(rng.uniform(500, 25000), 2), — "budget_amount" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; round is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.uniform is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 500 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 25000 is a numeric literal used by the command or expression;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 450:** "currency_code": "USD", — "currency_code" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "USD" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 451:** "created_at": datetime.combine(start, datetime.min.time()), — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datetime.combine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; start is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datetime.min.time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ()), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 452:** "updated_at": ctx.generated_at.replace(tzinfo=None), — "updated_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.generated_at.replace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tzinfo is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 453:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 454:** campaigns_df = pd.DataFrame(campaigns) — campaigns_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; campaigns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 455:** blank line used only to separate logical sections for readability.
- **Line 456:** sessions, events, touchpoints = [], [], [] — sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; events is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; touch-points is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [] , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [] , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 457:** `order_choices = orders[orders["channel_code"] == "ONLINE"]["order_id"].tolist()` — `order_choices` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"channel_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"ONLINE"` is a quoted literal; the quotation marks preserve spaces or mark text; `][` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 458:** `for i in range(ctx.counts["web_sessions"]):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `range` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.counts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web_sessions"` is a quoted literal; the quotation marks preserve spaces or mark text; `]):` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 459:** `session_id = 1_000_001 + i` — `session_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1_000_001` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `i` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 460:** `web_user_id = int(rng.choice(web_users_df["web_user_id"].tolist()))` — `web_user_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `web_users_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web_user_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `].` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tolist` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( ))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 461:** `start_date = random_date(rng, date(2025, 1, 1), date(2026, 7, 31))` — `start_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `random_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(2025, 1, 1), date(2026, 7, 31))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

ject-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2025 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- [illegible]

whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 467:** `channel = weighted_choice(rng, CHANNELS, [0.24, 0.19, 0.11, 0.07, 0.25, 0.14])` — `channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `weighted_choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CHANNELS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0.24` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0.19` is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 468:** `campaign_id = None` — `campaign_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 469:** `if channel in {"Paid Search", "Social", "Email", "Affiliate"} and rng.random() < 0.75:` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Paid Search"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Social"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Email"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Affiliate"` is a quoted literal; the quotation marks preserve spaces or mark text; `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 470:** `campaign_id = int(rng.choice(campaigns_df["campaign_id"].tolist()))` — `campaign_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `campaigns_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"campaign_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tolist` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( ) )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 471:** `converted = rng.random() < (0.08 if channel in {"Email", "Paid Search"} else 0.035)` — `converted` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.random` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0.08` is a numeric literal used by the command or expression; `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; "Email" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 472:** `order_id = int(rng.choice(order_choices))` if converted and `order_choices` else None — `order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_choices` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `converted` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_choices` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 473:** `sessions.append({` — `sessions.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 474:** `"session_id": session_id,` — `"session_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `session_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 475:** `"web_user_id": web_user_id,` — `"web_user_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `web_user_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 476:** `"session_start_utc": started.isoformat() + "+00:00",` — `"session_start_utc"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `started.isoformat` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `" +00:00"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 477:** `"session_end_utc": ended.isoformat() + "+00:00",` — `"session_end_utc"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ended.isoformat` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `" +00:00"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 478:** `"source_channel": channel,` — `"source_channel"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 479:** `"campaign_id": campaign_id,` — `"campaign_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `campaign_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 480:** `"device_type": weighted_choice(rng, ["mobile", "desktop", "tablet"], [0.61, 0.34, 0.05]),` — `"device_type"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `weighted_choice` is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "mobile" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "desktop" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "tablet" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 481:** "country_code": "US", — "country_code" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "US" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 482:** "converted_order_id": order_id, — "converted_order_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 483:** "created_at": ctx.generated_at.isoformat(), — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.generated_at.isoformat is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 484:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 485:** event_types = EVENT_SEQUENCE if converted else EVENT_SEQUENCE[: rng.randint(1, 4)] — event_types is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; EVENT_SEQUENCE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; converted is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; else is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; EVENT_SEQUENCE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.randint is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 486:** current = started — current is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; started is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 487:** for event_type in event_types: — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; event_type is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; event_types is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 488:** current += timedelta(seconds=rng.randint(2, 80)) — current is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; += is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timedelta is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; seconds is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

surrounding explanation; `rng.randint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `80` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 489:** `events.append({` — `events.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 490:** `"event_id": 2_000_001 + len(events),` — `"event_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2_000_001` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `events` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 491:** `"session_id": session_id,` — `"session_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `session_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 492:** `"event_timestamp": current.isoformat() + "+00:00",` — `"event_timestamp"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `current.isoformat` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `" +00:00"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 493:** `"event_type": event_type,` — `"event_type"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `event_type` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 494:** `"page_url": f"https://northstar.example/{event_type.replace('_', '-')}"` — `"page_url"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"https://northstar.example/{event_type.replace('_', '-')}"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 495:** `"product_id": None,` — `"product_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 496:** `"order_id": order_id if event_type == "purchase" else None,` — `"order_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `event_type` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"purchase"` is a quoted literal; the quotation marks preserve spaces or mark text; `else` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 497:** "event_properties": json.dumps({"synthetic": True, "sequence": len(events)}), — "event_properties" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; json.dumps is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ({ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "synthetic" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; True is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "sequence" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; len is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 498:** "created_at": ctx.generated_at.isoformat(), — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.generated_at.isoformat is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 499:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 500:** if campaign_id is not None: — if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; campaign_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 501:** touchpoints.append({ — touchpoints.append is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ({ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 502:** "touchpoint_id": 3_000_001 + len(touchpoints), — "touchpoint_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3_000_001 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; len is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; touchpoints is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 503:** "web_user_id": web_user_id, — "web_user_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; web_user_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 504:** "session_id": session_id, — "session_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; session_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 505:** "campaign_id": campaign_id, — "campaign_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; campaign_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 506:** "touchpoint_timestamp": started.isoformat() + "+00:00", — "touchpoint_timestamp" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; started.isoformat is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "+00:00" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 507:** "touchpoint_type": "click", — "touchpoint_type" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "click" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 508:** "attribution_weight": 1.0, — "attribution_weight" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1.0 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 509:** "created_at": ctx.generated_at.isoformat(), — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.generated_at.isoformat is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 510:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 511:** blank line used only to separate logical sections for readability.
- **Line 512:** sessions_df = pd.DataFrame(sessions) — sessions_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 513:** events_df = pd.DataFrame(events) — events_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; events is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 514:** for idx in range(min(ctx.issues["out_of_order_events"], len(events_df))): — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; idx is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; range is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; min is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.issues is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "out_of_order_events" is a quoted literal; the quotation marks preserve spaces or mark text;] , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; len is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 515:** events_df.loc[idx, "event_timestamp"] = "2020-01-01T00:00:00+00:00" — events_df.loc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; idx is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "event_timestamp" is a quoted literal; the quotation marks preserve spaces or mark text;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "2020-01-01T00:00:00+00:00" is a quoted literal; the quotation marks preserve spaces or mark text

- **Line 516:** `for idx in range(min(ctx.issues["timezone_differences"], len(sessions_df))):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `range` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `min` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.issues` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"timezone_differences"` is a quoted literal; the quotation marks preserve spaces or mark text; `],` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 517:** `value = pd.Timestamp(sessions_df.loc[idx, "session_start_utc"])` — `value` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.Timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sessions_df.loc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"session_start_utc"` is a quoted literal; the quotation marks preserve spaces or mark text; `])` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 518:** `sessions_df.loc[idx, "session_start_utc"] = value.tz_convert("America/New_York").isoformat()` — `sessions_df.loc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"session_start_utc"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `value.tz_convert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"America/New_York"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `isoformat` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 519:** blank line used only to separate logical sections for readability.
- **Line 520:** `spend = []` — `spend` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 521:** `for _, campaign in campaigns_df.iterrows():` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `_` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `campaign` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `campaigns_df.iterrows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 522:** `current = campaign["start_date"]` — `current` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `campaign` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"start_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; "start_date" is a quoted literal; the quotation marks preserve spaces or mark text;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 523:** while current <= campaign["end_date"]: — while is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; current is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; <= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; campaign is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "end_date" is a quoted literal; the quotation marks preserve spaces or mark text;]: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 524:** spend.append({ — spend.append is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ({ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 525:** "spend_id": 4_000_001 + len(spend), — "spend_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 4_000_001 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; len is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 526:** "spend_date": current, — "spend_date" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; current is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 527:** "campaign_id": int(campaign["campaign_id"]), — "campaign_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; int is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; campaign is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "campaign_id" is a quoted literal; the quotation marks preserve spaces or mark text;] , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 528:** "channel": campaign["channel"], — "channel" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; campaign is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "channel" is a quoted literal; the quotation marks preserve spaces or mark text;] , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 529:** "spend_amount": round(rng.uniform(10, max(20, float(campaign["budget_amount"]) / 20)), 2), — "spend_amount" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; round is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.uniform is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 10 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; max is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 20 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 530:** "impressions": rng.randint(100, 50000), — "impressions" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.randint is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 50000 is a numeric literal used by the command or expression;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 531:** "clicks": rng.randint(5, 2000), — "clicks" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rng.randint is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 5 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2000 is a numeric literal used by the command or expression;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 532:** "currency_code": "USD", — "currency_code" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "USD" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 533:** "created_at": ctx.generated_at.isoformat(), — "created_at" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.generated_at.isoformat is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 534:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 535:** current += timedelta(days=1) — current is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; += is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timedelta is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; days is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 536:** return { — return is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 537:** "web_users": web_users_df, — "web_users" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; web_users_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 538:** "web_sessions": sessions_df, — "web_sessions" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sessions_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 539:** "web_events": events_df, — "web_events" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; events_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 540:** "campaigns": campaigns_df, — "campaigns" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; campaigns_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 541:** `"campaign_touchpoints": pd.DataFrame(touchpoints),` — `"campaign_touchpoints"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `touchpoints` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 542:** `"marketing_spend": pd.DataFrame(spend),` — `"marketing_spend"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spend` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 543:** `}` — `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 544:** blank line used only to separate logical sections for readability.
- **Line 545:** blank line used only to separate logical sections for readability.
- **Line 546:** `def generate_file_feeds(` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `generate_file_feeds` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 547:** `ctx: GenerationContext,` — `ctx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `GenerationContext` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 548:** `refs: dict[str, pd.DataFrame],` — `refs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dict` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 549:** `orders: dict[str, pd.DataFrame],` — `orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dict` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 550:** `rng: random.Random,` — `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `random.Random` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 551:** `) -> dict[str, Any]:` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `dict` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Any` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

specific role is described in the surrounding explanation;] : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 552:** `items = orders["order_items"]` — `items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_items"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 553:** `candidates = items[items["quantity"] > 0].sample(` — `candidates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quantity"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sample` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 554:** `n=min(max(1, len(items) // 12), 10000), random_state=ctx.seed` — `n` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `min` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `max` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 555:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 556:** `returns = []` — `returns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 557:** `order_dates = orders["orders"].set_index("order_id")["order_timestamp"].to_dict()` — `order_dates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"orders"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.set_index` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_timestamp"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 558:** `for _, item in candidates.iterrows():` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `_` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or

literal whose project-specific role is described in the surrounding explanation; `item` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `candidates.iterrows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- Line 559:** `purchase_dt = pd.Timestamp(order_dates[item["order_id"]]).to_pydatetime()` — `purchase_dt` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.Timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_dates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `item` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `to_pydatetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 560:** `return_dt = purchase_dt + timedelta(days=rng.randint(1, 45))` — `return_dt` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `purchase_dt` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timedelta` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `days` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.randint` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 561:** `returns.append({` — `returns.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 562:** `"return_id": f"RET-{len(returns)+1:08d}"`, — `"return_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"RET-{len(returns)+1:08d}"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 563:** `"order_id": int(item["order_id"])`, — `"order_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `item` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `[]`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 564:** `"order_item_id": int(item["order_item_id"])`, — `"order_item_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `item` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_item_id"` is a quoted liter-

al; the quotation marks preserve spaces or mark text; `]`), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 565:** `"product_id": int(item["product_id"])`, — `"product_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `item` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"product_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `]`), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 566:** `"customer_id": int(orders["orders"].loc[orders["orders"]["order_id"] == item["order_id"], "customer_id"].iloc[0])`, — `"customer_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"orders"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.loc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 567:** `"return_date": return_dt.date().isoformat()`, — `"return_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `return_dt.date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `isoformat` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 568:** `"return_quantity": 1`, — `"return_quantity"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 569:** `"return_reason": rng.choice(["Damaged", "Wrong size", "Changed mind", "Late delivery", "Not as described"])`, — `"return_reason"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.choice` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( [` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Damaged"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Wrong size"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Changed mind"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Late delivery"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 570:** `"refund_amount": round(float(item["unit_price"]) - float(item["discount_amount"]) / max(1, int(item["quantity"])), 2)`, — `"refund_amount"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `round` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `float` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `item` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"unit_price"` is a quoted literal; the quotation

marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-` is a command option or flag; its exact behavior is defined beside the command; `float` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 571:** `"currency_code": "USD",` — `"currency_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"USD"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 572:** `"source_file_date": ctx.generated_at.date().isoformat(),` — `"source_file_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.generated_at.date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `isoformat` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 573:** `})` — `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 574:** `returns_df = pd.DataFrame(returns)` — `returns_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `returns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 575:** `for idx in range(min(ctx.issues["unknown_file_references"], len(returns_df))):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `range` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `min` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.issues` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"unknown_file_references"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 576:** `returns_df.loc[idx, "product_id"] = 9999999 + idx` — `returns_df.loc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"product_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `9999999` is a numeric literal used by the command or expression; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 577:** `for idx in range(min(ctx.issues["mixed_date_formats"], len(returns_df))):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `range` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `min` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

role is described in the surrounding explanation; `ctx.issues` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"mixed_date_formats"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- Line 578:** `parsed = pd.Timestamp(returns_df.loc[idx, "return_date"])` — `parsed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.Timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `returns_df.loc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"return_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 579:** `formats = ["%m/%d/%Y", "%d-%b-%Y", "%Y/%m/%d"]` — `formats` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"%m/%d/%Y"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"%d-%b-%Y"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"%Y/%m/%d"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 580:** `returns_df.loc[idx, "return_date"] = parsed.strftime(formats[idx % len(formats)])` — `returns_df.loc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"return_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parsed.strftime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `formats` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 581:** `for idx in range(min(ctx.issues["late_arriving_records"], len(returns_df))):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `range` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `min` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.issues` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"late_arriving_records"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 582:** `returns_df.loc[idx, "source_file_date"] = "2025-01-01"` — `returns_df.loc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `idx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"source_file_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"2025-01-01"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 583:** blank line used only to separate logical sections for readability.
- **Line 584:** `cost_updates = []` — `cost_updates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 585:** `for _, product in refs["products"].sample(n=min(len(refs["products"]), 1000), random_state=ctx.seed).iterrows():` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `_` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `product` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `refs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"products"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sample` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `n` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 586:** `effective = random_date(rng, date(2026, 1, 1), date(2026, 7, 31))` — `effective` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `random_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2026` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 587:** `cost_updates.append({` — `cost_updates.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 588:** `"supplier_cost_update_id": f"SCU-{len(cost_updates)+1:08d}"`, — `"supplier_cost_update_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SCU-{len(cost_updates)+1:08d}"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 589:** `"supplier_code": refs["suppliers"].loc[` — `"supplier_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `refs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"suppliers"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `loc` is a name, keyword, argument, identifier, or literal whose project-

specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 590:** `refs["suppliers"]["supplier_id"] == product["supplier_id"], "supplier_code"` — `refs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"suppliers"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"supplier_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `product` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"supplier_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `]`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"supplier_code"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 591:** `].iloc[0], — ].` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `iloc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `]`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 592:** `"sku": product["sku"], — "sku"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `product` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"sku"` is a quoted literal; the quotation marks preserve spaces or mark text; `]`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 593:** `"effective_date": effective.isoformat(), — "effective_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `effective.isoformat` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 594:** `"new_unit_cost": round(float(product["unit_cost"]) * rng.uniform(0.92, 1.12), 2), — "new_unit_cost"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `round` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `float` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `product` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"unit_cost"` is a quoted literal; the quotation marks preserve spaces or mark text; `])` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng.uniform` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 595:** `"currency_code": "USD", — "currency_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"USD"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 596:** `}) — })` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 597:** `cost_df = pd.DataFrame(cost_updates) — cost_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cost_updates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 598:** blank line used only to separate logical sections for readability.
- **Line 599:** `promotions = pd.DataFrame([` — `promotions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `([` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 600:** `{"promotion_code": "WELCOME10", "promotion_name": "New Customer Welcome", "start_date": "2025-01-01", "end_date": "2026-12-31", "discount_type": "PERCENT", "discount_value": 10, "channel_code": "ONLINE"},` — `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"promotion_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"WELCOME10"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"promotion_name"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"New Customer Welcome"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"start_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"2025-01-01"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 601:** `{"promotion_code": "WEEKEND15", "promotion_name": "Weekend Event", "start_date": "2026-01-01", "end_date": "2026-12-31", "discount_type": "PERCENT", "discount_value": 15, "channel_code": "ALL"},` — `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"promotion_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"WEEKEND15"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"promotion_name"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Weekend Event"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"start_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"2026-01-01"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 602:** `{"promotion_code": "FREESHIP", "promotion_name": "Free Shipping", "start_date": "2026-01-01", "end_date": "2026-12-31", "discount_type": "SHIPPING", "discount_value": 100, "channel_code": "ONLINE"},` — `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"promotion_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"FREESHIP"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"promotion_name"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Free Shipping"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"start_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"2026-01-01"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 603:** `)]` — `)]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 604:** blank line used only to separate logical sections for readability.
- **Line 605:** `tracking = []` — `tracking` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 606:** `for _, shipment in orders["shipments"].iterrows():` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `_` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `shipment` is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "shipments" is a quoted literal; the quotation marks preserve spaces or mark text;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; iterrows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 607:** `shipped = pd.Timestamp(shipment["ship_date"])` — `shipped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.Timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `shipment` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"ship_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `])` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 608:** `delivered = pd.Timestamp(shipment["actual_delivery_date"])` — `delivered` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.Timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `shipment` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"actual_delivery_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `])` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 609:** `tracking.append({` — `tracking.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 610:** `"tracking_number": shipment["tracking_number"],` — `"tracking_number"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `shipment` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"tracking_number"` is a quoted literal; the quotation marks preserve spaces or mark text; `],` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 611:** `"order_id": int(shipment["order_id"]),` — `"order_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `shipment` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `]),` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 612:** `"carrier": shipment["carrier"],` — `"carrier"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `shipment` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"carrier"` is a quoted literal; the quotation marks preserve spaces or mark text; `],` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 613:** `"events": [` — `"events"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 614:** `{"event_type": "LABEL_CREATED", "event_timestamp": shipped.isoformat(), "location": {"city": "Origin", "country": "US"}},` — `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

described in the surrounding explanation; "event_type" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "LABEL_CREATED" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "event_timestamp" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; shipped.isoformat is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "location" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 615:** {"event_type": "IN_TRANSIT", "event_timestamp": (shipped + timedelta(days=1)).isoformat(), "location": {"city": "Hub", "country": "US"}}, — { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "event_type" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "IN_TRANSIT" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "event_timestamp" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; shipped is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timedelta is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 616:** {"event_type": "DELIVERED", "event_timestamp": delivered.isoformat(), "location": {"city": "Destination", "country": "US"}}, — { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "event_type" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "DELIVERED" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "event_timestamp" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; delivered.isoformat is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "location" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 617:**] , —] , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 618:** }) — }) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 619:** return { — return is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 620:** "returns.csv": returns_df, — "returns.csv" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; returns_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 621:** "supplier_cost_updates.csv": cost_df, — "supplier_cost_updates.csv" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cost_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 622:** "promotion_calendar.csv": promotions, — "promotion_calendar.csv" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; promotions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 623:** "shipment_tracking_events.jsonl": tracking, — "shipment_tracking_events.jsonl" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tracking is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 624:** } — } is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 625:** blank line used only to separate logical sections for readability.
- **Line 626:** blank line used only to separate logical sections for readability.
- **Line 627:** def validate(frames: dict[str, pd.DataFrame]) -> list[dict[str, Any]]: — def is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; validate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; frames is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dict is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;]) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -> is a command option or flag; its exact behavior is defined beside the command; remaining punctuation and identifiers continue the same statement.
- **Line 628:** tests = [] — tests is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 629:** def check(name: str, passed: bool, details: str) -> None: — def is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; check is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; passed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bool is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; details is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 630:** tests.append({"test": name, "status": "PASS" if passed else "FAIL", "details": details}) — tests.append is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ({ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "test" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "status" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "PASS" is a quoted literal; the quotation marks preserve spaces or mark text; if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; passed is a name, keyword, argument,

identifier, or literal whose project-specific role is described in the surrounding explanation; else is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 631:** `check("customers_not_empty", len(frames["customers"]) > 0, f"rows={len(frames['customers'])}")` — `check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"customers_not_empty"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frames` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"customers"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 632:** `check("orders_not_empty", len(frames["orders"]) > 0, f"rows={len(frames['orders'])}")` — `check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"orders_not_empty"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frames` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"orders"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 633:** `normal_orders = set(frames["orders"]["customer_id"]) - set(frames["customers"]["customer_id"])` — `normal_orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frames` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"orders"` is a quoted literal; the quotation marks preserve spaces or mark text; `][` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"customer_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `])` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-` is a command option or flag; its exact behavior is defined beside the command; `set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 634:** `check("orders_customer_fk", len(normal_orders) == 0, f"orphans={len(normal_orders)}")` — `check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"orders_customer_fk"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `normal_orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 635:** `order_items_orphans = set(frames["order_items"]["order_id"]) - set(frames["orders"]["order_id"])` — `order_items_orphans` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frames` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_items"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-` is a command option or flag; its exact behavior is defined beside the command; `set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 636:** `check("items_order_fk", len(order_items_orphans) == 0, f"orphans={len(order_items_orphans)}")` — `check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"items_order_fk"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_items_orphans` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 637:** `signup = frames["customers"].set_index("customer_id")["signup_date"].to_dict()` — `signup` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frames` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"customers"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.set_index` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"customer_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `["signup_date"]` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 638:** `invalid_dates = sum(pd.Timestamp(row.order_timestamp).date() < signup[row.customer_id] for row in frames["orders"].itertuples())` — `invalid_dates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sum` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.Timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row.order_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `).` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `signup` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 639:** `check("signup_before_order", invalid_dates == 0, f"invalid={invalid_dates}")` — `check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "signup_before_order" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; invalid_dates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "invalid={invalid_dates}" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 640:** return tests — return is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tests is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 641:** blank line used only to separate logical sections for readability.
- **Line 642:** blank line used only to separate logical sections for readability.
- **Line 643:** def write_outputs(— def is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; write_outputs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 644:** ctx: GenerationContext, — ctx is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; GenerationContext is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 645:** sqlserver: dict[str, pd.DataFrame], — sqlserver is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dict is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 646:** postgres: dict[str, pd.DataFrame], — postgres is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dict is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 647:** files: dict[str, Any], — files is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dict is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Any is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 648:**) -> None: —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -> is a command option or flag; its exact behavior is defined beside the command; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 649:** `output = ctx.output_dir` — `output` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.output_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 650:** `if output.exists():` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `output.exists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()`: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 651:** `# Only remove generated subfolders, never the repository or arbitrary parent folder.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 652:** `for child in ["sqlserver", "postgres", "files", "metadata"]:` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `child` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"sqlserver"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"postgres"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"files"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"metadata"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 653:** `target = output / child` — `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `output` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `child` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 654:** `if target.exists():` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.exists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()`: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 655:** `import shutil` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `shutil` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 656:** `shutil.rmtree(target)` — `shutil.rmtree` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 657:** `manifests = []` — `manifests` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 658:** `for name, frame in sqlserver.items():` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlserver.items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()`: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 659:** `manifests.append({"source_system": "sqlserver", **write_frame(frame, output / "sqlserver" / f"{name}.csv")})` — `manifests.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"source_system"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"sqlserver"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `write_frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `output / "sqlserver" / f"{name}.csv"` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

described in the surrounding explanation; "sqlserver" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; write_frame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; frame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; output is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 660:** for name, frame in postgres.items(): — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; frame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; postgres.items is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 661:** manifests.append({"source_system": "postgres", **write_frame(frame, output / "postgres" / f"{name}.csv")}) — manifests.append is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ({ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "source_system" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "postgres" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; write_frame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; frame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; output is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 662:** for filename, value in files.items(): — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; filename is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; files.items is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 663:** path = output / "files" / filename — path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; output is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; "files" is a quoted literal; the quotation marks preserve spaces or mark text; / is a path, relative location, branch-style name, or file argument; verify every character before running it; filename is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 664:** path.parent.mkdir(parents=True, exist_ok=True) — path.parent.mkdir is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; parents is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; True is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; exist_ok is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific

role is described in the surrounding explanation; True is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 665:** `if filename.endswith(".jsonl"):` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filename.endswith` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `".jsonl"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 666:** `with path.open("w", encoding="utf-8") as handle:` — `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.open` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"w"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `encoding` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"utf-8"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `handle` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 667:** `for record in value:` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `record` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `value` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 668:** `handle.write(json.dumps(record) + "\n")` — `handle.write` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `json.dumps` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `record` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\n"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 669:** `rows = len(value)` — `rows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `value` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 670:** `manifests.append({"source_system": "files", "file": path.as_posix(), "rows": rows, "sha256": hashlib.sha256(path.read_bytes()).hexdigest()})` — `manifests.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"source_system"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"files"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"file"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.as_posix` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"rows"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 671:** `else:` — `else` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 672:** `manifests.append({"source_system": "files", **write_frame(value, path)})` — `manifests.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"source_system"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"files"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `write_frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `value` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 673:** blank line used only to separate logical sections for readability.
- **Line 674:** `all_frames = {**sqlserver, **postgres}` — `all_frames` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlserver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `postgres` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 675:** `tests = validate(all_frames)` — `tests` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `validate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `all_frames` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 676:** `metadata = output / "metadata"` — `metadata` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `output` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"metadata"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 677:** `metadata.mkdir(parents=True, exist_ok=True)` — `metadata.mkdir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parents` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exist_ok` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 678:** `(metadata / "batch_manifest.json").write_text(json.dumps({` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `metadata` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"batch_manifest.json"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `write_text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; `json.dumps` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 679:** `"batch_id": ctx.batch_id, — "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; :` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 680:** `"generated_at_utc": ctx.generated_at.isoformat(), — "generated_at_utc" is a quoted literal; the quotation marks preserve spaces or mark text; :` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.generated_at.isoformat` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 681:** `"seed": ctx.seed, — "seed" is a quoted literal; the quotation marks preserve spaces or mark text; :` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.seed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 682:** `"scale": ctx.scale, — "scale" is a quoted literal; the quotation marks preserve spaces or mark text; :` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.scale` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 683:** `"files": manifests, — "files" is a quoted literal; the quotation marks preserve spaces or mark text; :` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manifests` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 684:** `}, indent=2), encoding="utf-8") — },` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `indent` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression; `)` , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `encoding` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"utf-8"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 685:** `(metadata / "validation_results.json").write_text(json.dumps(tests, indent=2), encoding="utf-8") — (` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `metadata` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"validation_results.json"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `write_text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `json.dumps` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tests` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `indent` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 686:** `expected = { — expected` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 687:** `"fixture_purpose": "Expected controlled issues for teaching Silver data-quality handling."`, — `"fixture_purpose"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Expected controlled issues for teaching Silver data-quality handling."` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 688:** `"seed": ctx.seed`, — `"seed"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.seed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 689:** `"scale": ctx.scale`, — `"scale"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.scale` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 690:** `"configured_issue_counts": ctx.issues`, — `"configured_issue_counts"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.issues` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 691:** `"notes": [` — `"notes"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 692:** `"A configured issue can affect more than one column."`, — `"A configured issue can affect more than one column."` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 693:** `"The fixture records injections, while validation code measures observed rows by rule."`, — `"The fixture records injections, while validation code measures observed rows by rule."` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 694:** `"All names, contacts, companies, and identifiers are fictional."`, — `"All names, contacts, companies, and identifiers are fictional."` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 695:** `]`, — `]`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 696:** `}` — `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 697:** `(metadata / "expected_quality_issues.json").write_text(json.dumps(expected, indent=2), encoding="utf-8")` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `metadata` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"expected_quality_issues.json"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `write_text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `json.dumps` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `expected` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `indent` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 698:** `summary = pd.DataFrame([{"source_system": item["source_system"], "file": Path(item["file"]).name, "rows": item["rows"]} for item in manifests])` — `summary` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

surrounding explanation; ({ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "source_system" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; item is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "source_system" is a quoted literal; the quotation marks preserve spaces or mark text;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "file" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 699:** `summary.to_csv(metadata / "generation_summary.csv", index=False)` — `summary.to_csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `metadata` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; "generation_summary.csv" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `index` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `False` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 700:** `failed = [test for test in tests if test["status"] != "PASS"]` — `failed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tests` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "status" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 701:** `if failed:` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 702:** `raise RuntimeError(f"Generation validation failed: {failed}")` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `RuntimeError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Generation validation failed: {failed}" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 703:** `LOGGER.info("Generated %s files for batch %s", len(manifests), ctx.batch_id)` — `LOGGER.info` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Generated %s files for batch %s" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manifests` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.batch_id` is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 704:** `LOGGER.info("Output folder: %s", output.resolve())` — `LOGGER.info` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Output folder: %s"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `output.resolve` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 705:** blank line used only to separate logical sections for readability.
- **Line 706:** blank line used only to separate logical sections for readability.
- **Line 707:** `def main() -> int:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 708:** `configure_logging()` — `configure_logging` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 709:** `args = parse_args()` — `args` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parse_args` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 710:** `ctx = load_context(args)` — `ctx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `load_context` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 711:** `rng = random.Random(ctx.seed)` — `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `random.Random` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.seed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 712:** `Faker.seed(ctx.seed)` — `Faker.seed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.seed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 713:** `fake = Faker("en_US")` — `fake` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Faker` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"en_US"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 714:** `fake.seed_instance(ctx.seed)` — `fake.seed_instance` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx.seed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 715:** `LOGGER.info("Generating scale=%s seed=%s", ctx.scale, ctx.seed)` — `LOGGER.info` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Generating scale=%s seed=%s" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.scale is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ctx.seed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

- **Line 720:** `sqlserver = {**refs, "customers": customers, "addresses": addresses, **orders, **inventory}` — `sqlserver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `refs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"customers"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"addresses"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `addresses` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 721:** `postgres = generate_digital(ctx, customers, orders["orders"], fake, rng)` — `postgres` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `generate_digital` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"orders"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 722:** `file_feeds = generate_file_feeds(ctx, refs, orders, rng)` — `file_feeds` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `generate_file_feeds` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `refs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rng` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.
- **Line 723:** `write_outputs(ctx, sqlserver, postgres, file_feeds)` — `write_outputs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ctx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlserver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `postgres` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `file_feeds` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.
- **Line 724:** `return 0` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression.
- **Line 725:** blank line used only to separate logical sections for readability.
- **Line 726:** blank line used only to separate logical sections for readability.

- **Line 727:** `if __name__ == "__main__":` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__name__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"__main__"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 728:** `sys.exit(main())` — `sys.exit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 5 lesson index](#)

File introduction — `scripts/validation/validate_generated_data.py`

Exact filename: `validate_generated_data.py`

Filename extension: `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `scripts/validation` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 5.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `scripts/validation` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `validate_generated_data.py`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `validate_generated_data.py` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 5.F3.1 — Create the file `scripts/validation/validate_generated_data.py` from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\scripts\validation\validate_generated_data.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\scripts\validation\validate_generated_data.py" -ErrorAction Stop`
— `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\scripts\validation\validate_generated_data.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use

`git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\scripts\validation\validate_generated_data.py" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "scripts/validation/validate_generated_data.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `scripts/validation`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `scripts/validation/validate_generated_data.py`.
3. In PowerShell at the repository root, run `Test-Path ".\scripts\validation\validate_generated_data.py"` and require `True`.
4. Run `git status --short -- "scripts/validation/validate_generated_data.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `scripts/validation/validate_generated_data.py`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `scripts/validation/validate_generated_data.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 5.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```

"""Validate generated files without requiring SQL Server, PostgreSQL, or Databricks."""
from __future__ import annotations

import argparse
import json
from pathlib import Path

import pandas as pd

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--input-dir", type=Path, default=Path("datasets/sample"))
    args = parser.parse_args()
    customer = pd.read_csv(args.input_dir / "sqlserver/customers.csv")
    orders = pd.read_csv(args.input_dir / "sqlserver/orders.csv")
    items = pd.read_csv(args.input_dir / "sqlserver/order_items.csv")
    manifest = json.loads((args.input_dir / "metadata/batch_manifest.json").read_text(encoding="utf-8"))
    failures = []
    if customer.empty: failures.append("customers.csv is empty")
    if orders.empty: failures.append("orders.csv is empty")
    if set(orders.customer_id) - set(customer.customer_id): failures.append("orders contain unexpected customer orphans")
    if set(items.order_id) - set(orders.order_id): failures.append("order_items contain unexpected order orphans")
    if len(manifest["files"]) < 20: failures.append("manifest contains fewer files than expected")
    if failures:
        raise SystemExit("VALIDATION FAILED\n- " + "\n- ".join(failures))
    print(f"VALIDATION PASSED: {len(manifest['files'])} files, {len(customer)} customers, {len(orders)} orders")

if __name__ == "__main__":
    main()

```

Line-by-line explanation

- **Line 1:** `"""Validate generated files without requiring SQL Server, PostgreSQL, or Databricks."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `Validate generated files without requiring SQL Server, PostgreSQL, or Databricks.` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 2:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** blank line used only to separate logical sections for readability.
- **Line 4:** `import argparse` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `argparse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `import json` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `json` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `from pathlib import Path` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pathlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `import pandas as pd` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pandas` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** blank line used only to separate logical sections for readability.
- **Line 10:** blank line used only to separate logical sections for readability.
- **Line 11:** `def main() -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 12:** `parser = argparse.ArgumentParser()` — `parser` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `argparse.ArgumentParser` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `parser.add_argument("--input-dir", type=Path, default=Path("datasets/sample"))` — `parser.add_argument` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--input-dir` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `type` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 14:** `args = parser.parse_args()` — `args` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parser.parse_args` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** `customer = pd.read_csv(args.input_dir / "sqlserver/customers.csv")` — `customer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.read_csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.input_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"sqlserver/customers.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `orders = pd.read_csv(args.input_dir / "sqlserver/orders.csv")` — `orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.read_csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.input_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"sqlserver/orders.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** `items = pd.read_csv(args.input_dir / "sqlserver/order_items.csv")` — `items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.read_csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.input_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"sqlserver/order_items.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** `manifest = json.loads((args.input_dir / "metadata/batch_manifest.json").read_text(encoding="utf-8"))` — `manifest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier,

or literal whose project-specific role is described in the surrounding explanation; `json.loads` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `((` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.input_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"metadata/batch_manifest.json"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `read_text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `encoding` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 19:** `failures = []` — `failures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** `if customer.empty: failures.append("customers.csv is empty")` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer.empty` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failures.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"customers.csv is empty"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** `if orders.empty: failures.append("orders.csv is empty")` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders.empty` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failures.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"orders.csv is empty"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** `if set(orders.customer_id) - set(customer.customer_id): failures.append("orders contain unexpected customer orphans")` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders.customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-` is a command option or flag; its exact behavior is defined beside the command; `set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer.customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failures.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 23:** `if set(items.order_id) - set(orders.order_id): failures.append("order_items contain unexpected order orphans")` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `items.order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-` is a command option or flag; its exact behavior is defined beside the command; `set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders.order_id` is a name, keyword, argument, identifier, or

literal whose project-specific role is described in the surrounding explanation;) : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; failures.append is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 24:** `if len(manifest["files"]) < 20: failures.append("manifest contains fewer files than expected")` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manifest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"files"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `20` is a numeric literal used by the command or expression; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failures.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 25:** `if failures:` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** `raise SystemExit("VALIDATION FAILED\n- " + "\n- ".join(failures))` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SystemExit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"VALIDATION FAILED\n- "` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\n- "` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` `.` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` `join` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:** `print(f"VALIDATION PASSED: {len(manifest['files'])} files, {len(customer)} customers, {len(orders)} orders")` — `print` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"VALIDATION PASSED: {len(manifest['files'])} files, {len(customer)} customers, {len(orders)} orders"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** blank line used only to separate logical sections for readability.
- **Line 29:** blank line used only to separate logical sections for readability.
- **Line 30:** `if __name__ == "__main__":` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__name__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"__main__"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** `main()` — `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 5 lesson index](#)

File introduction — tests/unit/test_generator.py

Exact filename: test_generator.py

Filename extension: .py. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: tests/unit relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 5.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in tests/unit one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: test_generator.py.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled test_generator.py and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 5.F4.1 — Create the file tests/unit/test_generator.py from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\tests\unit\test_generator.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\tests\unit\test_generator.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\tests\unit\test_generator.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\tests\unit\test_generator.py" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "tests/unit/test_generator.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `tests/unit`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `tests/unit/test_generator.py`.
3. In PowerShell at the repository root, run `Test-Path ".\tests\unit\test_generator.py"` and require True.
4. Run `git status --short -- "tests/unit/test_generator.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for tests/unit/test_generator.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `tests/unit/test_generator.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 5.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
from __future__ import annotations

import json
import subprocess
import sys
from pathlib import Path

import pandas as pd

def run_generator(output: Path) -> None:
    subprocess.run([
        sys.executable, "-m", "scripts.data_generation.generate_northstar_data",
        "--scale", "quick", "--seed", "20260815", "--output-dir", str(output),
    ], check=True)

def test_quick_generator_counts_and_integrity(tmp_path: Path) -> None:
    output = tmp_path / "generated"
    run_generator(output)
    customers = pd.read_csv(output / "sqlserver/customers.csv")
```

```

orders = pd.read_csv(output / "sqlserver/orders.csv")
items = pd.read_csv(output / "sqlserver/order_items.csv")
assert len(customers) == 60
assert len(orders) == 200
assert not (set(orders.customer_id) - set(customers.customer_id))
assert not (set(items.order_id) - set(orders.order_id))
results = json.loads((output / "metadata/validation_results.json").read_text())
assert all(row["status"] == "PASS" for row in results)

def test_same_seed_reproduces_business_content(tmp_path: Path) -> None:
    left, right = tmp_path / "left", tmp_path / "right"
    run_generator(left); run_generator(right)
    for relative in ["sqlserver/customers.csv", "sqlserver/orders.csv", "postgres/web_sessions.csv", "files/returns.csv"]:
        assert (left / relative).read_bytes() == (right / relative).read_bytes()

```

Line-by-line explanation

- **Line 1:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** `import json` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `json` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `import subprocess` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `subprocess` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `import sys` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sys` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `from pathlib import Path` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pathlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `import pandas as pd` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pandas` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** blank line used only to separate logical sections for readability.
- **Line 10:** blank line used only to separate logical sections for readability.
- **Line 11:** `def run_generator(output: Path) -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_generator` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `output` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `subprocess.run([` — `subprocess.run` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `sys.executable, "-m", "scripts.data_generation.generate_northstar_data",` — `sys.executable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"-m"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument,

identifier, or literal whose project-specific role is described in the surrounding explanation;

"scripts.data_generation.generate_northstar_data" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 14:** "--scale", "quick", "--seed", "20260815", "--output-dir", str(output), — "--scale" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "quick" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "--seed" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "20260815" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "--output-dir" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 15:**], check=True) —], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; check is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; True is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** blank line used only to separate logical sections for readability.
- **Line 17:** blank line used only to separate logical sections for readability.
- **Line 18:** def test_quick_generator_counts_and_integrity(tmp_path: Path) -> None: — def is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test_quick_generator_counts_and_integrity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tmp_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -> is a command option or flag; its exact behavior is defined beside the command; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** output = tmp_path / "generated" — output is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tmp_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; "generated" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 20:** run_generator(output) — run_generator is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; output is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** customers = pd.read_csv(output / "sqlserver/customers.csv") — customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.read_csv is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; output is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; "sqlserver/customers.csv" is a path, relative location, branch-style name, or file argument; verify every character before running it;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; orders.order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 28:** results = json.loads((output / "metadata/validation_results.json").read_text()) — results is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; json.loads is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ((is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; output is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; "metadata/validation_results.json" is a path, relative location, branch-style name, or file argument; verify every character before running it;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; read_text is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** assert all(row["status"] == "PASS" for row in results) — assert is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; all is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; row is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "status" is a quoted literal; the quotation marks preserve spaces or mark text;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "PASS" is a quoted literal; the quotation marks preserve spaces or mark text; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; row is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 30:** blank line used only to separate logical sections for readability.
- **Line 31:** blank line used only to separate logical sections for readability.
- **Line 32:** def test_same_seed_reproduces_business_content(tmp_path: Path) -> None: — def is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test_same_seed_reproduces_business_content is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tmp_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -> is a command option or flag; its exact behavior is defined beside the command; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** left, right = tmp_path / "left", tmp_path / "right" — left is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; right is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tmp_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; "left" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tmp_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; "right" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 34:** run_generator(left); run_generator(right) — run_generator is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; left is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_generator` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `right` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 35:** `for relative in ["sqlserver/customers.csv", "sqlserver/orders.csv", "postgres/web_sessions.csv", "files/returns.csv"]:` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `relative` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"sqlserver/customers.csv"` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"sqlserver/orders.csv"` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"postgres/web_sessions.csv"` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"files/returns.csv"` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 36:** `assert (left / relative).read_bytes() == (right / relative).read_bytes()` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `left` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` `relative` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `read_bytes` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `right` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` remaining punctuation and identifiers continue the same statement.

[Back to the Phase 5 lesson index](#)

What your screen should look like now

- Every Phase 5 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 5.3 completion checklist

☐

`config/data_generation.yml` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`scripts/data_generation/generate_northstar_data.py` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`scripts/validation/validate_generated_data.py` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`tests/unit/test_generator.py` was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 5.4](#) and recheck the root and branch.

Lesson 5.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Action 5.4.G1 — Inspect the current state without changing it

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Inspect the current state without changing it. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 5 lesson index](#)

Action 5.4.G2 — Create the smallest required object or decision

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Create the smallest required object or decision. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.

4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 5 lesson index](#)

Action 5.4.G3 — Run the phase-specific verification

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run the phase-specific verification. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 5 lesson index](#)

Action 5.4.G4 — Demonstrate one safe failure or alternate-result branch

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Demonstrate one safe failure or alternate-result branch. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 5 lesson index](#)

Action 5.4.G5 — Record evidence and prepare the downstream handoff

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Record evidence and prepare the downstream handoff. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

What your screen should look like now

- The new Phase 5 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 5.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 5.5](#).

Lesson 5.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 5 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Deterministic synthetic data

Question:

How can fake data be realistic yet reproducible?

Options considered:

- Unseeded random generation.
- Deterministic seeded generation with configuration.
- Manually typed fixtures only.

Beginner path selected:

Use seeded Faker/pandas generation plus controlled dirty-data injection.

Why this path was selected:

Reruns can be compared, tests know the expected defects, and volumes can be configured.

Tradeoff:

Synthetic distributions do not prove production behavior.

What a company might do instead:

Companies use masked production-like data, governed test-data management, and privacy review.

How to explain this in an interview:

I used deterministic seeds and an expected-quality-issue manifest so the same defects could test Bronze/Silver behavior repeatedly.

Design decision — Python scripts versus notebooks

Question:

Should synthetic-data generation and local loading be built as reusable Python scripts or interactive notebooks?

Options considered:

- Versioned .py scripts with configuration and command-line arguments.
- Jupyter notebooks.
- Manual spreadsheet generation.

Beginner path selected:

Use Python scripts for local generation, loading, extraction, validation, and repeatable command-line execution; use Databricks notebooks only where the remote platform requires them.

Why this path was selected:

Scripts are easier to test, parameterize, review, run non-interactively, and reuse in CI. The learner still sees each section assembled and explained before the full script.

Tradeoff:

Scripts provide less immediate visual exploration than notebooks and require deliberate logging and small tests.

What a company might do instead:

A company may package Python modules, use workflow frameworks, containerize execution, and reserve notebooks for exploration or platform-specific tasks.

How to explain this in an interview:

I chose scripts for reproducible local engineering workflows and notebooks for Databricks execution, separating exploratory convenience from production-style testability.

If you see Result A

Generation completes with deterministic business content, expected counts, and intentional defect totals. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

Counts or checksums differ because configuration, seed, package version, or generation order changed. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: VS Code, VS Code PowerShell terminal, Python virtual environment, and File Explorer.

Window/panel: Show generator logs, summary counts, validation results, and controlled dirty-data manifest.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase05_build-and-validate-the-synthetic-data-generator_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 5's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 5 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 5.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Action 5.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Action 5.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Interpret git status --short

- ?? means untracked: Git sees a new path but it is not staged.
- M means modified in the working tree but not staged.
- M means the staged version differs from the last commit.
- MM means a staged version exists and the file has additional unstaged edits.
- A means staged as a new file.
- D means deletion; stop and investigate unless deletion is deliberate.
- R means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 5.5.G4 — Inspect the unstaged diff for config/data_generation.yml

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "config/data_generation.yml"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** git diff -- "config/data_generation.yml" — git calls the Git version-control program; diff is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -- is a command option or flag; its exact behavior is defined beside the command; "config/data_generation.yml" is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Action 5.5.G5 — Stage only the reviewed file `config/data_generation.yml`

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "config/data_generation.yml"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "config/data_generation.yml"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"config/data_generation.yml"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 5.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Action 5.5.G7 — Create the Phase 5 commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "feat: build deterministic synthetic data generator"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "feat: build deterministic synthetic data generator"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"feat: build deterministic synthetic data generator"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Action 5.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Action 5.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 5 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message `feat: build deterministic synthetic data generator and the intended changed paths`.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 5.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 5 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

"Phase 5 created the verified boundary described by **Build and validate the synthetic-data generator**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed."

Mini quiz

1. Multiple choice: Why does Phase 5 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

My Phase 5 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 5.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 6 loads the generated parent and child data into the two database engines.

Phase 5 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]



Output independently inspected — [STATUS]



Failure/alternate branch understood — [STATUS]



Screenshot captured and redacted — [STATUS]



Targeted Git commit exists — [STATUS]



Branch pushed and verified on GitHub — [STATUS]



Quiz completed before answer key — [STATUS]



Journal completed — [STATUS]



Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 6 — Load and verify the local source databases

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: Generated files plus empty, verified source schemas.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 7 reads the populated sources into traceable extraction batches.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 6 lesson index

1. [Lesson 6.1 — Understand the phase and starting screen](#)
2. [Lesson 6.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 6.3 — Create and understand the phase files](#)
4. [Lesson 6.4 — Build and validate required objects](#)
5. [Lesson 6.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 6.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** VS Code, PowerShell, SSMS, pgAdmin, SQL Server, and PostgreSQL.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 6 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 5 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 6 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

VS Code, PowerShell, SSMS, pgAdmin, SQL Server, and PostgreSQL. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 6 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.

- No new Phase 6 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 6 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 6.1 completion checklist

☐

I read why Phase 6 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 6.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 6 lesson index](#)

Lesson 6.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 6 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

Repeatable local source-loading processes using secret-free configuration. The beginner path uses pandas/SQLAlchemy in dependency order; the scalable path introduces SQL Server bulk copy and PostgreSQL COPY concepts. Row counts, transactions, failures, and duplicate-run behavior are recorded.

2. Why it matters

Generated files are not database sources until rows are loaded and reconciled. Loading teaches drivers, connection strings, environment variables, transaction scope, dependency order, bulk tradeoffs, and why rerunning append logic without a batch strategy can duplicate data.

3. Words to know

Term	Plain-language definition
Connection string	Structured host/database/auth/driver information.
Environment variable	Configuration value read at runtime; secrets stay outside code.
Transaction	All-or-nothing group of database changes.

Term	Plain-language definition
Commit / rollback	Make transaction permanent or undo it.
Chunk	Subset of rows loaded at a time.
Bulk load	Database-optimized high-throughput ingestion such as bcp or COPY.
Dependency order	Parent tables loaded before child foreign keys.
Duplicate-run protection	Rule preventing repeated input from creating duplicate trusted rows.
Trusted connection	SQL Server Windows authentication from the current identity.
Row-count reconciliation	Generated count compared to database count and inserted delta.
Retry	Repeat a transient operation under limits.
Staging	Temporary landing table used before validation/merge.

4. What must already be completed

Source tables exist; quick data validates; .env, ODBC, psycopg, SQLAlchemy are installed. You know your local SQL Server instance and PostgreSQL credentials privately.

5. Applications to open

Application	How to open it	Why it is open
VS Code	Repository root; open .env.example and loader scripts.	Create private .env and read load order.
Windows PowerShell	Repository root with .env active.	Run loaders and config checks.
SSMS	Connected to NorthstarRetail_ERP.	Validate ERP counts and dirty fixtures.
pgAdmin or psql	Connected to northstar_digital.	Validate digital counts.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run Get-Location; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
Copy-Item .env.example .env	Creates a private local configuration copy.	Code needs environment-specific values without changing a public template.
git check-ignore .env	Asks Git whether .env is ignored.	Secrets must not be tracked.
python -c "from scripts.utilities.config import load_environment; load_environment(); import os; print(os.getenv('SQLSERVER_HOST')); print(os.getenv('POSTGRES_HOST'))"	Loads config and prints only non-secret hosts.	Confirms .env parsing without displaying passwords.
python -m scripts.loading.load_sqlserver_source --input-dir datasets/sample/sqlserver --mode replace-dev	Loads ERP CSVs in dependency order into SQL Server.	Source tables must contain data before extraction.

Line or action	Meaning	Why it belongs here
python -m scripts.loading.load_postgres_source --input-dir datasets\sample\postgres --mode replace-dev	Loads digital CSVs in dependency order into PostgreSQL.	Digital source needs separate driver/dialect path.
-- Execute sql/sqlserver/validation/01_validate_erp_source.sql	Shows counts, orphans, malformed emails, impossible quantities, duplicate business keys.	A successful loader process does not prove correct population.
-- Execute sql/postgres/validation/01_validate_digital_source.sql	Shows counts, orphan events, out-of-order events, and conversion overview.	Verifies digital load and intentional event issue.

Beginner versus bulk loading

Method	Best now	Benefit	Cost/risk
pandas + SQLAlchemy	learning/small fixtures	readable, same language, logs, transactions	slower, type inference needs care
SQL Server bcp/BULK INSERT	larger local files	high throughput	path permissions, mapping, error files, transaction design
PostgreSQL COPY\copy	larger CSV	optimized server/client loading	quoting/types/error handling and client/server path distinction

Do not label the simple method 'bad'. It is appropriate for a small learning batch; the decision changes with volume and operating requirements.

MY TASK

Load both sources, reconcile every generated file count to its database table, then design (do not blindly implement) a checksum-based duplicate-run guard and a bulk-load migration plan.

STARTER SKELETON

```
def already_loaded(connection, source_checksum: str) -> bool:
    """Return True when this exact generated input was committed before."""
    # TODO(USER): query audit.source_load_batch or a child file table.
    result = connection.execute(_____, {"checksum": source_checksum})
    return _____

def record_success(connection, batch_id: str, checksum: str, rows: int) -> None:
    # TODO(USER): insert audit record inside the same transaction.
    _____
```

Acceptance criteria: Both databases match generated counts, normal FK checks pass, controlled dirty rows remain available for Silver, .env is untracked, and duplicate-run/bulk strategy is documented.

HINT 1 — CONCEPT

Load parents before children and keep explicit table order.

HINT 2 — PSEUDOCODE

A primary-key failure is safer than silent duplication, but not a complete operational rerun strategy.

HINT 3 — NEAR-COMPLETE PATTERN

Record checksum and success in the same transaction as data changes so they cannot disagree.

13. Expected files or output

- Populated ERP and digital source tables.
- Loader logs with row counts.
- Source validation result evidence.
- Private .env, ignored.
- Duplicate-run and bulk-load decision notes.

14. How to verify

- Generated summary count equals table count for each source.
- No FK orphans.
- Known invalid quantities/emails/events remain observable.
- A failed test transaction does not leave unexplained partial rows.
- `git status` never lists `.env`.

15. Automated tests

- Source validation SQL.
- Manual generated-to-source count matrix.
- Integration test can be added when local DBs are available; CI remains infrastructure-free.
- Secret scanner and ignored-file check.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
pyodbc driver error	Exact driver/server/auth mismatch.	pyodbc driver list and SQL Server context in SSMS.	Copy exact driver name/verified host to <code>.env</code> ; use intended auth.	Python <code>SELECT 1</code> succeeds.
FK constraint failure	Load order or source file mismatch.	Read table/key in exception; query parent ID.	Do not disable constraint; correct generation/order/input.	Transaction rerun passes.
PostgreSQL password failed	Wrong private credential/user/database.	Test <code>psql</code> with same host/port/user/database.	Correct <code>.env</code> after authorized reset; never print password.	Loader connects.
Primary key violation on re-run	Append repeated same IDs.	Check before counts and input checksum.	Use <code>replace-dev</code> in disposable setup or implement guarded batch merge.	Counts equal one batch.
Partial source load	Transaction boundary or DDL/autocommit issue.	Inspect counts/audit/status after error.	Rollback/reset only disposable data, fix cause, reload in one controlled transaction.	All tables reconcile.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why do environment variables improve security but not automatically guarantee it?
2. Why are table load order and transaction scope different concerns?
3. When is pandas loading appropriate?
4. What should a duplicate-run guard record?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. Which file should contain your real local password?
2. Why should a child table not load before its parent?
3. What is one advantage and one risk of bulk loading?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture loader log counts and SSMS/pgAdmin validation result grids. Crop `.env` and connection dialogs containing secrets. Save the image under `docs/images/` with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
feat: load and reconcile synthetic source databases
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

Private config created/ignored — [STATUS]

☐

SQL Server load reconciled — [STATUS]

☐

PostgreSQL load reconciled — [STATUS]

☐

Controlled issues preserved — [STATUS]

☐

Transaction behavior understood — [STATUS]

☐

Duplicate-run plan documented — [STATUS]

☐

Bulk alternatives explained — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-6. The quiz answer is in [the answer-key section immediately after the quiz in this workbook#phase-6] (the answer-key section immediately after the quiz in this workbook#phase-6).

Next phase: Phase 7 extracts a timestamped, manifest-driven batch for the local-first Databricks handoff.

Required action cards

Action 6.2.1 — Copy `.env.example` to `.env`; never edit the example with real secrets. Confirm `.env` is ignored before typi

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Copy `.env.example` to `.env`; never edit the example with real secrets. Confirm `.env` is ignored before typi. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Copy `.env.example` to `.env`; never edit the example with real secrets. Confirm `.env` is ignored before typing credentials.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 1****Action**

Copy `.env.example` to `.env`; never edit the example with real secrets. Confirm `.env` is ignored before typing credentials.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 6 lesson index](#)

Action 6.2.2 — Set SQLSERVER_HOST to the verified server name; use trusted connection for local Windows authentication when

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Set SQLSERVER_HOST to the verified server name; use trusted connection for local Windows authentication when. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Set SQLSERVER_HOST to the verified server name; use trusted connection for local Windows authentication when configured. Set exact ODBC driver name.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 2

Action

Set SQLSERVER_HOST to the verified server name; use trusted connection for local Windows authentication when configured. Set exact ODBC driver name.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 6 lesson index](#)

Action 6.2.3 — Set PostgreSQL host, port, database, username, and private password in .env.

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Set PostgreSQL host, port, database, username, and private password in .env. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Set PostgreSQL host, port, database, username, and private password in .env.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.

6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL `psql`, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 3

Action

Set PostgreSQL host, port, database, username, and private password in `.env`.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 6 lesson index](#)

Action 6.2.4 — Open each loader and inspect TABLE_ORDER. Parent tables must precede products/orders/items/events.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Open each loader and inspect TABLE_ORDER. Parent tables must precede products/orders/items/events. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open each loader and inspect TABLE_ORDER. Parent tables must precede products/orders/items/events.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 4

Action

Open each loader and inspect TABLE_ORDER. Parent tables must precede products/orders/items/events.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 6 lesson index](#)

Action 6.2.5 — For a first disposable load, use `--mode replace-dev only` after confirming these source databases contain no

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

For a first disposable load, use `--mode replace-dev only` after confirming these source databases contain no . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: For a first disposable load, use `--mode replace-dev only` after confirming these source databases contain no valuable data. The scripts delete/truncate in dependency order and then append generated rows.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 5

Action

For a first disposable load, use `--mode replace-dev` only after confirming these source databases contain no valuable data. The scripts delete/truncate in dependency order and then append generated rows.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 6 lesson index](#)

Action 6.2.6 — Run SQL Server loader. It uses one transaction and logs before/inserted/after counts. If a child fails, transa

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run SQL Server loader. It uses one transaction and logs before/inserted/after counts. If a child fails, transa. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run SQL Server loader. It uses one transaction and logs before/inserted/after counts. If a child fails, transaction behavior should prevent a partial commit within the connection scope.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 6****Action**

Run SQL Server loader. It uses one transaction and logs before/inserted/after counts. If a child fails, transaction behavior should prevent a partial commit within the connection scope.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 6 lesson index](#)

Action 6.2.7 — Run PostgreSQL loader and review logged counts.

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run PostgreSQL loader and review logged counts. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run PostgreSQL loader and review logged counts.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL psql, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 7

Action

Run PostgreSQL loader and review logged counts.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 6 lesson index](#)

Action 6.2.8 — Run source validation SQL in SSMS and pgAdmin. Controlled invalid business values should be visible; database

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run source validation SQL in SSMS and pgAdmin. Controlled invalid business values should be visible; database . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run source validation SQL in SSMS and pgAdmin. Controlled invalid business values should be visible; database foreign-key violations should be zero.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 8****Action**

Run source validation SQL in SSMS and pgAdmin. Controlled invalid business values should be visible; database foreign-key violations should be zero.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 6 lesson index](#)

Action 6.2.9 — Record generated versus loaded counts. If you rerun append mode, primary keys should fail rather than silently

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Record generated versus loaded counts. If you rerun append mode, primary keys should fail rather than silently. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Record generated versus loaded counts. If you rerun append mode, primary keys should fail rather than silently duplicate; this is a signal that source-load batch protection needs design.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 9

Action

Record generated versus loaded counts. If you rerun append mode, primary keys should fail rather than silently duplicate; this is a signal that source-load batch protection needs design.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 6 lesson index](#)

Action 6.2.10 — Study scalable options: SQL Server bcp/BULK INSERT and PostgreSQL \copy/COPY. Do not switch until the simp

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Study scalable options: SQL Server bcp/BULK INSERT and PostgreSQL \copy/COPY. Do not switch until the simp. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Study scalable options: SQL Server bcp/BULK INSERT and PostgreSQL \copy/COPY. Do not switch until the simple path works and you can preserve transactions, mappings, and logs.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 10****Action**

Study scalable options: SQL Server bcp/BULK INSERT and PostgreSQL \copy/COPY. Do not switch until the simple path works and you can preserve transactions, mappings, and logs.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 6 lesson index](#)

Action 6.2.11 — Complete TODO: add an explicit load batch record or file checksum guard before supporting routine append rerun

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Complete TODO: add an explicit load batch record or file checksum guard before supporting routine append rerun. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Complete TODO: add an explicit load batch record or file checksum guard before supporting routine append reruns.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 11

Action

Complete TODO: add an explicit load batch record or file checksum guard before supporting routine append reruns.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.



I can describe what changed on the screen or filesystem.



I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 6 lesson index](#)

Action 6.2.C1 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Copy-Item .env.example .env
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `Copy-Item .env.example .env` — `Copy-Item` copies a selected file or folder; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `env.example` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Action 6.2.C2 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
git check-ignore .env
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git check-ignore .env` — `git` calls the Git version-control program; `check-ignore` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Action 6.2.C3 — Windows PowerShell

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -c "from scripts.utilities.config import load_environment; load_environment(); import os; print(os.getenv('SQLSERVER_HOST')); print(os.getenv('POSTGRES_HOST'))"
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -c "from scripts.utilities.config import load_environment; load_environment(); import os; print(os.getenv('SQLSERVER_HOST')); print(os.getenv('POSTGRES_HOST'))"` — python invokes Python when that command name is registered; `-c` creates the named branch before switching; `"from scripts.utilities.config import load_environment; load_environment(); import os; print(os.getenv('SQLSERVER_HOST')); print(os.getenv('POSTGRES_HOST'))"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Action 6.2.C4 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m scripts.loading.load_sqlserver_source --input-dir datasets\sample\sqlserver --mode replace-dev
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m scripts.loading.load_sqlserver_source --input-dir datasets\sample\sqlserver --mode replace-dev` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.loading.load_sqlserver_source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--input` is a command option or flag; its exact behavior is

defined beside the command; `-dir` is a command option or flag; its exact behavior is defined beside the command; `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sample` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlserver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--mode` is a command option or flag; its exact behavior is defined beside the command; `replace-dev` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Action 6.2.C5 — Windows PowerShell

Application:

pgAdmin 4 or PostgreSQL `psql`, as named in the action

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m scripts.loading.load_postgres_source --input-dir datasets\sample\postgres --mode replace-dev
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m scripts.loading.load_postgres_source --input-dir datasets\sample\postgres --mode replace-dev` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.loading.load_postgres_source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--input` is a command option or flag; its exact behavior is defined beside the command; `-dir` is a command option or flag; its exact behavior is defined beside the command; `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sample` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `postgres` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--mode` is a command option or flag; its exact behavior is defined beside the command; `replace-dev` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Action 6.2.C6 — SSMS query window

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SSMS query window

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
-- Execute sql/sqlserver/validation/01_validate_erp_source.sql
```

Where to enter it:

SSMS query window. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** -- Execute sql/sqlserver/validation/01_validate_erp_source.sql is a comment for a human reader; it normally does not change data or execute business logic.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Action 6.2.C7 — pgAdmin Query Tool

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:
pgAdmin Query Tool

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
-- Execute sql/postgres/validation/01_validate_digital_source.sql
```

Where to enter it:

pgAdmin Query Tool. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** -- Execute sql/postgres/validation/01_validate_digital_source.sql is a comment for a human reader; it normally does not change data or execute business logic.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 6.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 6 starting-screen checklist and re-verify the active context.

Lesson 6.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 6 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — scripts/

Exact folder name: `scripts`

Exact parent: `repository root`

Why it is needed now: Phase 6 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `scripts`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 6.D1.1 — Create the folder scripts

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\scripts" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\scripts" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\scripts"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\scripts" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run Test-Path ".\scripts" -PathType Container.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/

Exact folder name: sql

Exact parent: repository root

Why it is needed now: Phase 6 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: sql.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 6.D2.1 — Create the folder sql

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\sql" -PathType Container` and require True.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql" -PathType Container`.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — scripts/loading/

Exact folder name: loading

Exact parent: scripts

Why it is needed now: Phase 6 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `scripts` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: loading.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 6.D3.1 — Create the folder scripts/loading

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\scripts\loading" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\scripts\loading" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\scripts\loading` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\scripts\loading" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts\loading" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\scripts\loading" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/postgres/

Exact folder name: postgres

Exact parent: sql

Why it is needed now: Phase 6 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `sql` one level at a time.

4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: postgres.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 6.D4.1 — Create the folder sql/postgres

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\postgres" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\sql\postgres" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\sql\postgres" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\postgres" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\postgres" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\postgres" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/sqlserver/

Exact folder name: sqlserver

Exact parent: sql

Why it is needed now: Phase 6 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path sql one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: sqlserver.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 6.D5.1 — Create the folder sql/sqlserver

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\sqlserver" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\sqlserver" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\sqlserver"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\sqlserver" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\sqlserver" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\sqlserver" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/postgres/validation/

Exact folder name: validation

Exact parent: sql/postgres

Why it is needed now: Phase 6 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path sql/postgres one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: validation.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 6.D6.1 — Create the folder sql/postgres/validation

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.

2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\postgres\validation" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\postgres\validation" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\postgres\validation"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\postgres\validation" -PathType Container` and require True.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\postgres\validation" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\postgres\validation" -PathType Container`.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/sqlserver/validation/

Exact folder name: validation

Exact parent: sql/sqlserver

Why it is needed now: Phase 6 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path sql/sqlserver one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: validation.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method**Action 6.D7.1 — Create the folder sql/sqlserver/validation**

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\sqlserver\validation" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\sqlserver\validation" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\sql\sqlserver\validation` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\sqlserver\validation" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\sqlserver\validation" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\sqlserver\validation" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — .env.example

Exact filename: `.env.example`

Filename extension: `.example`. In this project it identifies project text or configuration and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `.` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project text or configuration for Phase 6.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? Yes—review carefully before Git staging.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in . one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: .env.example.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled .env.example and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 6.F1.1 — Create the file .env.example from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\env.example" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\env.example" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\env.example"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\env.example" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- ".env.example"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `..`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `.env.example`.
3. In PowerShell at the repository root, run `Test-Path ".\env.example"` and require `True`.
4. Run `git status --short -- ".env.example"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for .env.example

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: configuration text

Exact file: `.env.example`

Purpose: project text or configuration.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 6.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Copy this file to .env. Never commit .env.
# Track A uses Windows Authentication for SQL Server by default.

APP_ENV=dev
LOG_LEVEL=INFO

SQLSERVER_SERVER=localhost
SQLSERVER_INSTANCE=MSSQLSERVER
SQLSERVER_SOURCE_DATABASE=NorthstarRetail_ERP
SQLSERVER_WAREHOUSE_DATABASE=NorthstarRetail_DW
SQLSERVER_DRIVER=ODBC Driver 18 for SQL Server
SQLSERVER_TRUSTED_CONNECTION=yes
SQLSERVER_ENCRYPT=yes
SQLSERVER_TRUST_SERVER_CERTIFICATE=yes
# Use these only if you intentionally choose SQL Server Authentication:
SQLSERVER_USERNAME=
SQLSERVER_PASSWORD=

POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DATABASE=northstar_digital
POSTGRES_USERNAME=postgres
POSTGRES_PASSWORD=CHANGE_ME_LOCALLY

# Track B only. Do not put live cloud secrets here if the file might be shared.
DATABRICKS_HOST=
DATABRICKS_HTTP_PATH=
DATABRICKS_TOKEN=
DATABRICKS_CATALOG=northstar
DATABRICKS_BRONZE_SCHEMA=bronze
DATABRICKS_SILVER_SCHEMA=silver
DATABRICKS_GOLD_SCHEMA=gold
DATABRICKS_VOLUME_PATH=/Volumes/northstar/landing/raw
```

Line-by-line explanation

- **Line 1:** `# Copy this file to .env. Never commit .env.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `# Track A uses Windows Authentication for SQL Server by default.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** blank line used only to separate logical sections for readability.

- **Line 4:** APP_ENV=dev — APP_ENV is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dev is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** LOG_LEVEL=INFO — LOG_LEVEL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INFO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** SQLSERVER_SERVER=localhost — SQLSERVER_SERVER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; localhost is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** SQLSERVER_INSTANCE=MSSQLSERVER — SQLSERVER_INSTANCE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; MSSQLSERVER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** SQLSERVER_SOURCE_DATABASE=NorthstarRetail_ERP — SQLSERVER_SOURCE_DATABASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NorthstarRetail_ERP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** SQLSERVER_WAREHOUSE_DATABASE=NorthstarRetail_DW — SQLSERVER_WAREHOUSE_DATABASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NorthstarRetail_DW is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** SQLSERVER_DRIVER=ODBC Driver 18 for SQL Server — SQLSERVER_DRIVER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ODBC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Driver is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 18 is a numeric literal used by the command or expression; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** SQLSERVER_TRUSTED_CONNECTION=yes — SQLSERVER_TRUSTED_CONNECTION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; yes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** SQLSERVER_ENCRYPT=yes — SQLSERVER_ENCRYPT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; yes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** SQLSERVER_TRUST_SERVER_CERTIFICATE=yes — SQLSERVER_TRUST_SERVER_CERTIFICATE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; yes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** # Use these only if you intentionally choose SQL Server Authentication: is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 16:** SQLSERVER_USERNAME= — SQLSERVER_USERNAME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** SQLSERVER_PASSWORD= — SQLSERVER_PASSWORD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** blank line used only to separate logical sections for readability.

- **Line 19:** `POSTGRES_HOST=localhost` — `POSTGRES_HOST` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `localhost` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** `POSTGRES_PORT=5432` — `POSTGRES_PORT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `5432` is a numeric literal used by the command or expression
- **Line 21:** `POSTGRES_DATABASE=northstar_digital` — `POSTGRES_DATABASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `northstar_digital` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** `POSTGRES_USERNAME=postgres` — `POSTGRES_USERNAME` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `postgres` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** `POSTGRES_PASSWORD=CHANGE_ME_LOCALLY` — `POSTGRES_PASSWORD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CHANGE_ME_LOCALLY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** blank line used only to separate logical sections for readability.
- **Line 25:** `# Track B only. Do not put live cloud secrets here if the file might be shared.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 26:** `DATABRICKS_HOST=` — `DATABRICKS_HOST` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:** `DATABRICKS_HTTP_PATH=` — `DATABRICKS_HTTP_PATH` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** `DATABRICKS_TOKEN=` — `DATABRICKS_TOKEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** `DATABRICKS_CATALOG=northstar` — `DATABRICKS_CATALOG` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `northstar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 30:** `DATABRICKS_BRONZE_SCHEMA=bronze` — `DATABRICKS_BRONZE_SCHEMA` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** `DATABRICKS_SILVER_SCHEMA=silver` — `DATABRICKS_SILVER_SCHEMA` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `silver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 32:** `DATABRICKS_GOLD_SCHEMA=gold` — `DATABRICKS_GOLD_SCHEMA` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** `DATABRICKS_VOLUME_PATH=/Volumes/northstar/landing/raw` — `DATABRICKS_VOLUME_PATH` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Volumes` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `northstar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `landing` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

File introduction — scripts/loading/load_sqlserver_source.py

Exact filename: load_sqlserver_source.py

Filename extension: .py. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: scripts/loading relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 6.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in scripts/loading one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: load_sqlserver_source.py.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled load_sqlserver_source.py and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 6.F2.1 — Create the file scripts/loading/load_sqlserver_source.py from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.

3. **CLICK:** Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. **TYPE THIS:** Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. **PRESS:** Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. **LOOK FOR:** Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. **DO NOT CONTINUE UNTIL:** Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\scripts\loading\load_sqlserver_source.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\scripts\loading\load_sqlserver_source.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\scripts\loading\load_sqlserver_source.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\scripts\loading\load_sqlserver_source.py" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "scripts/loading/load_sqlserver_source.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `scripts/loading`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `scripts/loading/load_sqlserver_source.py`.
3. In PowerShell at the repository root, run `Test-Path ".\scripts\loading\load_sqlserver_source.py"` and require True.
4. Run `git status --short -- "scripts/loading/load_sqlserver_source.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `scripts/loading/load_sqlserver_source.py`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `scripts/loading/load_sqlserver_source.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 6.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
"""Load generated ERP CSVs into local SQL Server staging/source tables."""
from __future__ import annotations

import argparse
import logging
from pathlib import Path

import pandas as pd
from sqlalchemy import text

from scripts.utilities.config import load_environment
from scripts.utilities.database import sqlserver_engine
from scripts.utilities.logging_utils import configure_logging

LOGGER = logging.getLogger("northstar.load_sqlserver")
```

```

TABLE_ORDER = [
    "product_categories", "suppliers", "products", "stores", "employees", "customers",
    "addresses", "orders", "order_items", "payments", "shipments",
    "inventory_transactions", "inventory_snapshots",
]

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--input-dir", type=Path, default=Path("datasets/sample/sqlserver"))
    parser.add_argument("--mode", choices=["append", "replace-dev"], default="append")
    args = parser.parse_args()
    configure_logging(); load_environment()
    engine = sqlserver_engine()
    with engine.begin() as connection:
        for table in TABLE_ORDER:
            path = args.input_dir / f"{table}.csv"
            if not path.exists():
                raise FileNotFoundError(f"Missing expected generated file: {path}")
            frame = pd.read_csv(path)
            if args.mode == "replace-dev":
                connection.execute(text(f"DELETE FROM erp.{table};"))
            before = connection.execute(text(f"SELECT COUNT_BIG(*) FROM erp.{table};")).scalar_one()
            frame.to_sql(table, connection, schema="erp", if_exists="append", index=False, chunksize=1000)
            after = connection.execute(text(f"SELECT COUNT_BIG(*) FROM erp.{table};")).scalar_one()
            LOGGER.info("erp.%s before=%s inserted=%s after=%s", table, before, len(frame), after)

if __name__ == "__main__":
    main()

```

Line-by-line explanation

- **Line 1:** `""Load generated ERP CSVs into local SQL Server staging/source tables.""` — `""` is a quoted literal; the quotation marks preserve spaces or mark text; `Load generated ERP CSVs into local SQL Server staging/source tables.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 2:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** blank line used only to separate logical sections for readability.
- **Line 4:** `import argparse` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `argparse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `import logging` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `logging` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `from pathlib import Path` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pathlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `import pandas as pd` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pandas` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `from sqlalchemy import text` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlalchemy` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** blank line used only to separate logical sections for readability.
- **Line 11:** `from scripts.utilities.config import load_environment` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scripts.utilities.config` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

import is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; load_environment is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 12:** from scripts.utilities.database import sqlserver_engine — from is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; scripts.utilities.database is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; import is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sqlserver_engine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** from scripts.utilities.logging_utils import configure_logging — from is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; scripts.utilities.logging_utils is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; import is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; configure_logging is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** blank line used only to separate logical sections for readability.
- **Line 15:** LOGGER = logging.getLogger("northstar.load_sqlserver") — LOGGER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; logging.getLogger is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "northstar.load_sqlserver" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** TABLE_ORDER = [— TABLE_ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** "product_categories", "suppliers", "products", "stores", "employees", "customers", — "product_categories" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "suppliers" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "products" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "stores" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "employees" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "customers" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** "addresses", "orders", "order_items", "payments", "shipments", — "addresses" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "orders" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "order_items" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "payments" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "shipments" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** "inventory_transactions", "inventory_snapshots", — "inventory_transactions" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "inventory_snapshots" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:**] —] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** blank line used only to separate logical sections for readability.
- **Line 22:** blank line used only to separate logical sections for readability.

- **Line 23:** `def main() -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** `parser = argparse.ArgumentParser()` — `parser` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `argparse.ArgumentParser` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** `parser.add_argument("--input-dir", type=Path, default=Path("datasets/sample/sqlserver"))` — `parser.add_argument` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--input-dir` is a quoted literal; the quotation marks preserve spaces or mark text; `"` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `type` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 26:** `parser.add_argument("--mode", choices=["append", "replace-dev"], default="append")` — `parser.add_argument` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--mode` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `choices` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `["append", "replace-dev"]` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `replace-dev` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 27:** `args = parser.parse_args()` — `args` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parser.parse_args` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** `configure_logging(); load_environment()` — `configure_logging` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `load_environment` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** `engine = sqlserver_engine()` — `engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlserver_engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 30:** `with engine.begin() as connection:` — `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `engine.begin` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

identifier, or literal whose project-specific role is described in the surrounding explanation; as is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; connection is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 31:** for table in TABLE_ORDER: — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; TABLE_ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 32:** path = args.input_dir / f"{table}.csv" — path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; args.input_dir is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{table}.csv" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 33:** if not path.exists(): — if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; path.exists is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (): is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** raise FileNotFoundError(f"Missing expected generated file: {path}") — raise is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FileNotFoundError is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Missing expected generated file: {path}" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** frame = pd.read_csv(path) — frame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pd.read_csv is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 36:** if args.mode == "replace-dev": — if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; args.mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replace-dev" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 37:** connection.execute(text(f"DELETE FROM erp.{table};")) — connection.execute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; text is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "DELETE FROM erp.{table};" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 38:** before = connection.execute(text(f"SELECT COUNT_BIG(*) FROM erp.{table};")).scalar_one() — before is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; connection.execute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; text is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

ject-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "SELECT COUNT_BIG(*) FROM erp.{table};" is a quoted literal; the quotation marks preserve spaces or mark text;)). is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; scalar_one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 39:** `frame.to_sql(table, connection, schema="erp", if_exists="append", index=False, chunksize=1000)` — `frame.to_sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "erp" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `if_exists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 40:** `after = connection.execute(text(f"SELECT COUNT_BIG(*) FROM erp.{table}")).scalar_one()` — `after` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection.execute` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "SELECT COUNT_BIG(*) FROM erp.{table};" is a quoted literal; the quotation marks preserve spaces or mark text;)). is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scalar_one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** `LOGGER.info("erp.%s before=%s inserted=%s after=%s", table, before, len(frame), after)` — `LOGGER.info` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "erp.%s before=%s inserted=%s after=%s" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `before` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 42:** blank line used only to separate logical sections for readability.
- **Line 43:** blank line used only to separate logical sections for readability.
- **Line 44:** `if __name__ == "__main__":` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__name__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "__main__" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 45:** `main()` — `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 6 lesson index](#)

File introduction — `scripts/loading/load_postgres_source.py`

Exact filename: `load_postgres_source.py`

Filename extension: `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `scripts/loading` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 6.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `scripts/loading` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `load_postgres_source.py`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `load_postgres_source.py` and an empty file under the correct parent folder.
10. PRESS: `Ctrl+S` after adding each explained section.

PowerShell path

Action 6.F3.1 — Create the file `scripts/loading/load_postgres_source.py` from PowerShell

Application:

pgAdmin 4 or PostgreSQL `psql`, as named in the action

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\scripts\loading\load_postgres_source.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\scripts\loading\load_postgres_source.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\scripts\loading\load_postgres_source.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\scripts\loading\load_postgres_source.py" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "scripts/loading/load_postgres_source.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `scripts/loading`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `scripts/loading/load_postgres_source.py`.
3. In PowerShell at the repository root, run `Test-Path ".\scripts\loading\load_postgres_source.py"` and require True.
4. Run `git status --short -- "scripts/loading/load_postgres_source.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for scripts/loading/load_postgres_source.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `scripts/loading/load_postgres_source.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 6.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
"""Load generated digital CSVs into local PostgreSQL."""
from __future__ import annotations

import argparse
import logging
from pathlib import Path
```

```

import pandas as pd
from sqlalchemy import text

from scripts.utilities.config import load_environment
from scripts.utilities.database import postgres_engine
from scripts.utilities.logging_utils import configure_logging

LOGGER = logging.getLogger("northstar.load_postgres")
TABLE_ORDER = ["web_users", "campaigns", "web_sessions", "web_events", "campaign_touchpoints", "marketing_spend"]

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--input-dir", type=Path, default=Path("datasets/sample/postgres"))
    parser.add_argument("--mode", choices=["append", "replace-dev"], default="append")
    args = parser.parse_args()
    configure_logging(); load_environment()
    engine = postgres_engine()
    with engine.begin() as connection:
        for table in TABLE_ORDER:
            path = args.input_dir / f"{table}.csv"
            if not path.exists():
                raise FileNotFoundError(f"Missing expected generated file: {path}")
            frame = pd.read_csv(path)
            if args.mode == "replace-dev":
                connection.execute(text(f"TRUNCATE TABLE digital.{table} RESTART IDENTITY CASCADE;"))
            before = connection.execute(text(f"SELECT COUNT(*) FROM digital.{table};")).scalar_one()
            frame.to_sql(table, connection, schema="digital", if_exists="append", index=False, chunksize=1000, method="multi")
            after = connection.execute(text(f"SELECT COUNT(*) FROM digital.{table};")).scalar_one()
            LOGGER.info("digital.%s before=%s inserted=%s after=%s", table, before, len(frame), after)

if __name__ == "__main__":
    main()

```

Line-by-line explanation

- **Line 1:** `"""Load generated digital CSVs into local PostgreSQL."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `Load generated digital CSVs into local PostgreSQL.` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 2:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** blank line used only to separate logical sections for readability.
- **Line 4:** `import argparse` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `argparse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `import logging` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `logging` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `from pathlib import Path` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pathlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `import pandas as pd` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pandas` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `from sqlalchemy import text` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlalchemy` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** blank line used only to separate logical sections for readability.

- **Line 11:** `from scripts.utilities.config import load_environment` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scripts.utilities.config` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `load_environment` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `from scripts.utilities.database import postgres_engine` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scripts.utilities.database` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `postgres_engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `from scripts.utilities.logging_utils import configure_logging` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scripts.utilities.logging_utils` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `configure_logging` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** blank line used only to separate logical sections for readability.
- **Line 15:** `LOGGER = logging.getLogger("northstar.load_postgres")` — `LOGGER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `logging.getLogger` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"northstar.load_postgres"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `TABLE_ORDER = ["web_users", "campaigns", "web_sessions", "web_events", "campaign_touchpoints", "marketing_spend"]` — `TABLE_ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web_users"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"campaigns"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web_sessions"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web_events"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"campaign_touchpoints"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 17:** blank line used only to separate logical sections for readability.
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** `def main() -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** `parser = argparse.ArgumentParser()` — `parser` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `argparse.ArgumentParser` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** `parser.add_argument("--input-dir", type=Path, default=Path("datasets/sample/postgres"))` — `parser.add_argument` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"--input-dir"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `type=Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default=Path("datasets/sample/postgres")` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tion; type is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; default is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 22:** `parser.add_argument("--mode", choices=["append", "replace-dev"], default="append")` — `parser.add_argument` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "--mode" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `choices` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "append" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replace-dev" is a quoted literal; the quotation marks preserve spaces or mark text;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 23:** `args = parser.parse_args()` — `args` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parser.parse_args` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** `configure_logging(); load_environment()` — `configure_logging` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `load_environment` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** `engine = postgres_engine()` — `engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `postgres_engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** `with engine.begin() as connection:` — `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `engine.begin` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:** `for table in TABLE_ORDER:` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `TABLE_ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** `path = args.input_dir / f"{table}.csv"` — `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.input_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; `verify every character before running it`; `f` is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; "{table}.csv" is a quoted literal; the quotation marks preserve spaces or mark text

- **Line 29:** `if not path.exists():` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.exists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 30:** `raise FileNotFoundError(f"Missing expected generated file: {path}")` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FileNotFoundError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Missing expected generated file: {path}"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** `frame = pd.read_csv(path)` — `frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.read_csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 32:** `if args.mode == "replace-dev":` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"replace-dev"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** `connection.execute(text(f"TRUNCATE TABLE digital.{table} RESTART IDENTITY CASCADE;"))` — `connection.execute` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"TRUNCATE TABLE digital.{table} RESTART IDENTITY CASCADE;"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** `before = connection.execute(text(f"SELECT COUNT(*) FROM digital.{table};")).scalar_one()` — `before` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection.execute` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SELECT COUNT(*) FROM digital.{table};"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scalar_one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** `frame.to_sql(table, connection, schema="digital", if_exists="append", index=False, chunksize=1000, method="multi")` — `frame.to_sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding

explanation; "digital" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; if_exists is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 36:** after = connection.execute(text(f"SELECT COUNT(*) FROM digital.{table};")).scalar_one() — after is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; connection.execute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; text is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "SELECT COUNT(*) FROM digital.{table};" is a quoted literal; the quotation marks preserve spaces or mark text;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; scalar_one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 37:** LOGGER.info("digital.%s before=%s inserted=%s after=%s", table, before, len(frame), after) — LOGGER.info is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "digital.%s before=%s inserted=%s after=%s" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; before is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; len is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; frame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 38:** blank line used only to separate logical sections for readability.
- **Line 39:** blank line used only to separate logical sections for readability.
- **Line 40:** if __name__ == "__main__": — if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; __name__ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "__main__" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** main() — main is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 6 lesson index](#)

File introduction — sql/sqlserver/validation/01_validate_erp_source.sql

Exact filename: 01_validate_erp_source.sql

Filename extension: .sql. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: sql/sqlserver/validation relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 6.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in sql/sqlserver/validation one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 01_validate_erp_source.sql.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 01_validate_erp_source.sql and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 6.F4.1 — Create the file sql/sqlserver/validation/01_validate_erp_source.sql from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\sqlserver\validation\01_validate_erp_source.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\sqlserver\validation\01_validate_erp_source.sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\sqlserver\validation\01_validate_erp_source.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\sql\sqlserver\validation\01_validate_erp_source.sql" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add -Force, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/sqlserver/validation/01_validate_erp_source.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `sql/sqlserver/validation`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `sql/sqlserver/validation/01_validate_erp_source.sql`.
3. In PowerShell at the repository root, run `Test-Path ".\sql\sqlserver\validation\01_validate_erp_source.sql"` and require True.
4. Run `git status --short -- "sql/sqlserver/validation/01_validate_erp_source.sql"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `sql/sqlserver/validation/01_validate_erp_source.sql`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: `sql/sqlserver/validation/01_validate_erp_source.sql`

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 6.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: SQL Server / T-SQL
-- Run in: SSMS, NorthstarRetail_ERP.
USE NorthstarRetail_ERP;
GO

-- Row counts. Replace expected values with the generation summary for your chosen scale.
SELECT 'customers' AS table_name, COUNT_BIG(*) AS row_count FROM erp.customers
UNION ALL SELECT 'orders', COUNT_BIG(*) FROM erp.orders
UNION ALL SELECT 'order_items', COUNT_BIG(*) FROM erp.order_items
UNION ALL SELECT 'products', COUNT_BIG(*) FROM erp.products
UNION ALL SELECT 'shipments', COUNT_BIG(*) FROM erp.shipments;

-- Referential integrity checks should return zero because source foreign keys are enabled.
SELECT COUNT_BIG(*) AS orphan_order_customers
FROM erp.orders AS o
LEFT JOIN erp.customers AS c ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL;

SELECT COUNT_BIG(*) AS orphan_order_items
FROM erp.order_items AS oi
LEFT JOIN erp.orders AS o ON o.order_id = oi.order_id
WHERE o.order_id IS NULL;

-- Controlled dirty-data observations; nonzero values are expected in the learning dataset.
SELECT COUNT_BIG(*) AS null_or_malformed_emails
FROM erp.customers
WHERE email IS NULL OR email NOT LIKE '%@%_%';

SELECT COUNT_BIG(*) AS impossible_quantities
FROM erp.order_items
WHERE quantity <= 0;

SELECT customer_number, COUNT_BIG(*) AS duplicate_count
FROM erp.customers
```

```
GROUP BY customer_number
HAVING COUNT_BIG(*) > 1;
```

Line-by-line explanation

- **Line 1:** -- Dialect: SQL Server / T-SQL is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** -- Run in: SSMS, NorthstarRetail_ERP. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** USE NorthstarRetail_ERP; — USE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NorthstarRetail_ERP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** -- Row counts. Replace expected values with the generation summary for your chosen scale. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 7:** SELECT 'customers' AS table_name, COUNT_BIG(*) AS row_count FROM erp.customers — SELECT requests data without changing it unless combined with a modifying statement; 'customers' is a quoted literal; the quotation marks preserve spaces or mark text; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; row_count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; erp.customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** UNION ALL SELECT 'orders', COUNT_BIG(*) FROM erp.orders — UNION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ALL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 'orders' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; erp.orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** UNION ALL SELECT 'order_items', COUNT_BIG(*) FROM erp.order_items — UNION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ALL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 'order_items' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; erp.order_items is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** UNION ALL SELECT 'products', COUNT_BIG(*) FROM erp.products — UNION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ALL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 'products' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; erp.products is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** UNION ALL SELECT 'shipments', COUNT_BIG(*) FROM erp.shipments; — UNION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ALL is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 'shipments' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; erp.shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** -- Referential integrity checks should return zero because source foreign keys are enabled. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 14:** SELECT COUNT_BIG(*) AS orphan_order_customers — SELECT requests data without changing it unless combined with a modifying statement; COUNT_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; orphan_order_customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** FROM erp.orders AS o — FROM identifies the source table or query; erp.orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; o is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** LEFT JOIN erp.customers AS c ON c.customer_id = o.customer_id — LEFT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; c is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; c.customer_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; o.customer_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** WHERE c.customer_id IS NULL; — WHERE filters rows to those meeting a condition; c.customer_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** SELECT COUNT_BIG(*) AS orphan_order_items — SELECT requests data without changing it unless combined with a modifying statement; COUNT_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; orphan_order_items is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** FROM erp.order_items AS oi — FROM identifies the source table or query; erp.order_items is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; oi is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** LEFT JOIN erp.orders AS o ON o.order_id = oi.order_id — LEFT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; erp.orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; o is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; o.order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; `oi.order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 22:** `WHERE o.order_id IS NULL;` — `WHERE` filters rows to those meeting a condition; `o.order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** blank line used only to separate logical sections for readability.
- **Line 24:** `-- Controlled dirty-data observations; nonzero values are expected in the learning dataset.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 25:** `SELECT COUNT_BIG(*) AS null_or_malformed_emails` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT_BIG` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `null_or_malformed_emails` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** `FROM erp.customers` — `FROM` identifies the source table or query; `erp.customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:** `WHERE email IS NULL OR email NOT LIKE '%@%._%';` — `WHERE` filters rows to those meeting a condition; `email` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `OR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `email` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `LIKE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'%@%._%'` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 28:** blank line used only to separate logical sections for readability.
- **Line 29:** `SELECT COUNT_BIG(*) AS impossible_quantities` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT_BIG` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `impossible_quantities` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 30:** `FROM erp.order_items` — `FROM` identifies the source table or query; `erp.order_items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** `WHERE quantity <= 0;` — `WHERE` filters rows to those meeting a condition; `quantity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression
- **Line 32:** blank line used only to separate logical sections for readability.
- **Line 33:** `SELECT customer_number, COUNT_BIG(*) AS duplicate_count` — `SELECT` requests data without changing it unless combined with a modifying statement; `customer_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `COUNT_BIG` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `duplicate_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** `FROM erp.customers` — `FROM` identifies the source table or query; `erp.customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** `GROUP BY customer_number` — `GROUP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 36:** `HAVING COUNT_BIG(*) > 1;` — `HAVING` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `COUNT_BIG` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier,

or literal whose project-specific role is described in the surrounding explanation; > is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression

[Back to the Phase 6 lesson index](#)

File introduction — sql/postgres/validation/01_validate_digital_source.sql

Exact filename: 01_validate_digital_source.sql

Filename extension: .sql. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: sql/postgres/validation relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 6.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in sql/postgres/validation one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 01_validate_digital_source.sql.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 01_validate_digital_source.sql and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 6.F5.1 — Create the file sql/postgres/validation/01_validate_digital_source.sql from PowerShell

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\postgres\validation\01_validate_digital_source.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\postgres\validation\01_validate_digital_source.sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\postgres\validation\01_validate_digital_source.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\sql\postgres\validation\01_validate_digital_source.sql" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/postgres/validation/01_validate_digital_source.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `sql/postgres/validation`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `sql/postgres/validation/01_validate_digital_source.sql`.
3. In PowerShell at the repository root, run `Test-Path ".\sql\postgres\validation\01_validate_digital_source.sql"` and require True.
4. Run `git status --short -- "sql/postgres/validation/01_validate_digital_source.sql"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `sql/postgres/validation/01_validate_digital_source.sql`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: `sql/postgres/validation/01_validate_digital_source.sql`

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 6.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: PostgreSQL SQL
-- Run in: pgAdmin Query Tool connected to northstar_digital.
SELECT 'web_users' AS table_name, COUNT(*) AS row_count FROM digital.web_users
UNION ALL SELECT 'web_sessions', COUNT(*) FROM digital.web_sessions
UNION ALL SELECT 'web_events', COUNT(*) FROM digital.web_events
```



```

UNION ALL SELECT 'campaigns', COUNT(*) FROM digital.campaigns
UNION ALL SELECT 'marketing_spend', COUNT(*) FROM digital.marketing_spend;

SELECT COUNT(*) AS orphan_events
FROM digital.web_events AS e
LEFT JOIN digital.web_sessions AS s ON s.session_id = e.session_id
WHERE s.session_id IS NULL;

SELECT COUNT(*) AS out_of_order_events
FROM digital.web_events AS e
JOIN digital.web_sessions AS s ON s.session_id = e.session_id
WHERE e.event_timestamp < s.session_start_utc;

SELECT source_channel, COUNT(*) AS sessions, COUNT(converted_order_id) AS converted_sessions
FROM digital.web_sessions
GROUP BY source_channel
ORDER BY sessions DESC;

```

Line-by-line explanation

- **Line 1:** -- Dialect: PostgreSQL SQL is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** -- Run in: pgAdmin Query Tool connected to northstar_digital. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** SELECT 'web_users' AS table_name, COUNT(*) AS row_count FROM digital.web_users — SELECT requests data without changing it unless combined with a modifying statement; 'web_users' is a quoted literal; the quotation marks preserve spaces or mark text; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; row_count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; digital.web_users is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** UNION ALL SELECT 'web_sessions', COUNT(*) FROM digital.web_sessions — UNION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ALL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 'web_sessions' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; digital.web_sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** UNION ALL SELECT 'web_events', COUNT(*) FROM digital.web_events — UNION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ALL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 'web_events' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; digital.web_events is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** UNION ALL SELECT 'campaigns', COUNT(*) FROM digital.campaigns — UNION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ALL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 'campaigns' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; digital.campaigns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** UNION ALL SELECT 'marketing_spend', COUNT(*) FROM digital.marketing_spend; — UNION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ALL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 'marketing_spend' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; digital.marketing_spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** SELECT COUNT(*) AS orphan_events — SELECT requests data without changing it unless combined with a modifying statement; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; orphan_events is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** FROM digital.web_events AS e — FROM identifies the source table or query; digital.web_events is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; e is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** LEFT JOIN digital.web_sessions AS s ON s.session_id = e.session_id — LEFT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; digital.web_sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s.session_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; e.session_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** WHERE s.session_id IS NULL; — WHERE filters rows to those meeting a condition; s.session_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** blank line used only to separate logical sections for readability.
- **Line 14:** SELECT COUNT(*) AS out_of_order_events — SELECT requests data without changing it unless combined with a modifying statement; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; out_of_order_events is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** FROM digital.web_events AS e — FROM identifies the source table or query; digital.web_events is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; e is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** JOIN digital.web_sessions AS s ON s.session_id = e.session_id — JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; digital.web_sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s.session_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; `e.session_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 17:** `WHERE e.event_timestamp < s.session_start_utc;` — `WHERE` filters rows to those meeting a condition; `e.event_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `s.session_start_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** `SELECT source_channel, COUNT(*) AS sessions, COUNT(converted_order_id) AS converted_sessions` — `SELECT` requests data without changing it unless combined with a modifying statement; `source_channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `converted_order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 20:** `FROM digital.web_sessions` — `FROM` identifies the source table or query; `digital.web_sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** `GROUP BY source_channel` — `GROUP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** `ORDER BY sessions DESC;` — `ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DESC` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 6 lesson index](#)

What your screen should look like now

- Every Phase 6 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 6.3 completion checklist

☐

`.env.example` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`scripts/loading/load_sqlserver_source.py` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`scripts/loading/load_postgres_source.py` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`sql/sqlserver/validation/01_validate_erp_source.sql` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`sql/postgres/validation/01_validate_digital_source.sql` was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 6.4](#) and recheck the root and branch.

Lesson 6.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Action 6.4.G1 — Inspect the current state without changing it

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Inspect the current state without changing it. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 6 lesson index](#)

Action 6.4.G2 — Create the smallest required object or decision

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create the smallest required object or decision. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.

4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 6 lesson index](#)

Action 6.4.G3 — Run the phase-specific verification

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run the phase-specific verification. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 6 lesson index](#)

Action 6.4.G4 — Demonstrate one safe failure or alternate-result branch

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Demonstrate one safe failure or alternate-result branch. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 6 lesson index](#)

Action 6.4.G5 — Record evidence and prepare the downstream handoff

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Record evidence and prepare the downstream handoff. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

What your screen should look like now

- The new Phase 6 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 6.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 6.5](#).

Lesson 6.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 6 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Verification before automation

Question:

Should the phase prioritize a visible manual verification or jump directly to automation?

Options considered:

- Manual verification first, then automation.
- Automation without a known-good baseline.
- Manual operation forever.

Beginner path selected:

Establish one verified manual baseline and then automate repeatable checks.

Why this path was selected:

A known-good example makes failures interpretable and prevents automation from hiding a wrong assumption.

Tradeoff:

The first run takes longer.

What a company might do instead:

Teams often pair infrastructure as code and automated tests with controlled bootstrap and acceptance procedures.

How to explain this in an interview:

I established a verified baseline, then encoded the same contract in tests and orchestration so automation preserved meaning rather than just speed.

If you see Result A

Loaded table counts match generated counts and foreign-key checks pass. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

A table is empty, partially loaded, duplicated, or has rejected/constraint-failing rows. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: VS Code, PowerShell, SSMS, pgAdmin, SQL Server, and PostgreSQL.

Window/panel: Show SSMS/pgAdmin row counts and loader audit results.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase06_load-and-verify-the-local-source-databases_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 6's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 6 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 6.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Action 6.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Action 6.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Interpret git status --short

- ?? means untracked: Git sees a new path but it is not staged.
- M means modified in the working tree but not staged.
- M means the staged version differs from the last commit.
- MM means a staged version exists and the file has additional unstaged edits.
- A means staged as a new file.
- D means deletion; stop and investigate unless deletion is deliberate.
- R means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 6.5.G4 — Inspect the unstaged diff for .env.example

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- ".env.example"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- ".env.example"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `".env.example"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Action 6.5.G5 — Stage only the reviewed file `.env.example`

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- ".env.example"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- ".env.example"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `".env.example"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 6.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Action 6.5.G7 — Create the Phase 6 commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "feat: load and validate local source databases"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "feat: load and validate local source databases"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"feat: load and validate local source databases"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Action 6.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Action 6.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 6 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message feat: load and validate local source databases and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 6.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 6 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

"Phase 6 created the verified boundary described by **Load and verify the local source databases**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed."

Mini quiz

1. Multiple choice: Why does Phase 6 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

My Phase 6 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 6.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 7 reads the populated sources into traceable extraction batches.

Phase 6 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 7 — Build full and incremental local extraction

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: Populated sources and reproducible full/incremental extract logic.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 8 creates the cloud landing location that receives those local batches.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 7 lesson index

1. [Lesson 7.1 — Understand the phase and starting screen](#)
2. [Lesson 7.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 7.3 — Create and understand the phase files](#)
4. [Lesson 7.4 — Build and validate required objects](#)
5. [Lesson 7.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 7.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** VS Code, PowerShell, SSMS, pgAdmin, Python, and File Explorer.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 7 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 6 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 7 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

VS Code, PowerShell, SSMS, pgAdmin, Python, and File Explorer. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 7 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 7 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 7 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 7.1 completion checklist

☐

I read why Phase 7 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 7.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 7 lesson index](#)

Lesson 7.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 7 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

A local extraction process that reads both databases and file feeds, writes timestamped Parquet/file outputs, and creates a manifest with batch ID, mode, watermark, row counts, and checksums. You will design failure and watermark behavior before relying on incremental mode.

2. Why it matters

A batch is the unit of movement, retry, audit, and reconciliation. Without a stable batch ID and manifest, a cloud upload is just a collection of files whose origin and completeness cannot be proved.

3. Words to know

Term	Plain-language definition
Extraction	Reading data out of a source without yet applying business transformation.
Full extraction	Read the complete selected source scope.
Incremental extraction	Read rows new/changed after a stored source position.
High-water mark	Latest source timestamp/position fully committed by a successful batch.
Tie-breaker	Second key used when many rows share a timestamp.
Manifest	Batch inventory with counts, checksums, paths, mode, and status.
Parquet	Columnar file format preserving types efficiently.
Checksum	SHA-256 file fingerprint.
Atomic handoff	Consumer sees a complete batch, not partially written files.
Idempotency	Safe repeat behavior for the same logical extraction window.
Batch status	RUNNING/SUCCESS/FAILED evidence.
Partial failure	Some files written before the process stops.

4. What must already be completed

Both source databases are populated/reconciled; file feeds exist; `.env` works; `.venv` active; enough disk space is available.

5. Applications to open

Application	How to open it	Why it is open
VS Code	Open extractor, manifest examples, and runbook.	Understand logic before execution.
Windows PowerShell	Repository root with <code>.venv</code> active.	Run full/incremental extract and checks.
SSMS and pgAdmin	Source contexts.	Verify watermark/update columns and independent counts.
File Explorer	Open generated batch folder.	Confirm immutable folder shape before upload.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run `Get-Location`; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
<code>python -m scripts.extraction.extract_local_sources --mode full --file-feed-dir datasets\sample\files --output-root datasets\raw</code>	Reads local sources and writes one timestamped batch folder.	A full baseline is the safest first handoff.
<code>Get-ChildItem datasets\raw -Directory Sort-Object LastWriteTime -Descending Select-Object -First 1 FullName, Name, LastWriteTime</code>	Finds the newest batch folder without guessing its timestamp.	The exact folder name becomes workflow batch ID.
<code>Get-Content datasets\raw\EXT-YYYYMMDDTH-HMMSSZ\batch_manifest.json</code>	Prints manifest.	You should inspect evidence before upload.
<code>Get-FileHash datasets\raw\EXT-YYYYMMDDTH-HMMSSZ\sqlserver\orders.parquet -Algorithm SHA256</code>	Computes SHA-256 independently.	Proves local bytes match manifest.
<code>SELECT COUNT_BIG(*) AS orders, MAX(updated_at) AS max_updated_at FROM erp.orders;</code>	Gets independent source count and candidate watermark.	Manifest must reconcile to source and watermarks need evidence.
<code>SELECT COUNT(*) AS sessions, MAX(created_at) AS max_created_at FROM digital.web_sessions;</code>	Gets digital count and event/source position evidence.	PostgreSQL tables may not share the same update column strategy.

Incremental decision rule

A generic `WHERE updated_at > watermark` is not valid for every table: some tables have only `created_at`, snapshots use a business date, events may be append-only, and file feeds use filenames/dates/checksums. A production-ready extractor stores a watermark per source object and may use `(updated_at, primary_key)` to avoid equal-timestamp gaps.

Failure rule

Write to a unique batch folder, keep status `FAILED` on exception, do not publish/upload it as trusted, and do not advance the committed watermark. A later run can retry the same logical bounds with a new attempt ID or safely replace the failed folder under an explicit policy.

MY TASK

Run/reconcile a full batch, independently verify one checksum, simulate one safe failure, and refactor/design per-table incremental metadata with a tie-breaker for at least orders and web events.

STARTER SKELETON

```
# config/incremental_sources.example.yml
sources:
- source_system: SQLSERVER_ERP
  table: erp.orders
  watermark_column: updated_at
  tie_breaker_column: order_id
  last_success_value: TODO(USER)
  last_success_tie_breaker: TODO(USER)
- source_system: POSTGRES_DIGITAL
```

```

table: digital.web_events
strategy: append_only
watermark_column: event_timestamp
tie_breaker_column: event_id
last_success_value: TODO(USER)

```

TODO(USER): state when values are committed and how deleted rows are handled.

Acceptance criteria: One full batch has exact source/file reconciliation and checksum proof; failed batch behavior is recorded; incremental design is per table with committed watermark/tie-breaker policy.

HINT 1 — CONCEPT

Watermark is a success checkpoint, not simply MAX(source) at process start.

HINT 2 — PSEUDOCODE

Use the primary key as a tie-breaker when timestamps can repeat.

HINT 3 — NEAR-COMPLETE PATTERN

Adapt the manifest structure already written; add attempt/status/watermark metadata before changing extraction filters.

13. Expected files or output

- datasets/raw/EXT-.../ with sqlserver, postgres, files, manifest.
- Source-to-extract count matrix.
- Independent checksum match.
- FAILED practice manifest/evidence.
- Per-table incremental strategy document or config skeleton.

14. How to verify

- Every manifest file exists and hash matches.
- Full counts equal source counts.
- Failed attempt is not labeled SUCCESS.
- No watermark was advanced from failed attempt.
- Exact batch ID is ready for volume upload.

15. Automated tests

- Manifest schema/required fields test (add as user task).
- Checksum recomputation.
- Source count SQL reconciliation.
- Incremental boundary tests with equal timestamps and no-change rerun.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
updated_at column not found	Generic incremental filter applied to a table without that column.	Inspect data dictionary/ source schema.	Use per-table strategy and column mapping; full mode until implemented.	Each query uses valid source column.
Manifest FAILED	Source/file/permission/write error occurred.	Read error field and terminal exception.	Fix root cause; do not upload partial batch; rerun.	New complete SUCCESS batch reconciles.
Checksum differs	File changed, wrong path, or text transfer altered bytes.	Compare size/time/path; re-compute local and uploaded copy later.	Use immutable binary upload and exact batch folder.	Hashes match.
Parquet package error	pyarrow missing/wrong environment.	python -m pip show pyarrow; interpreter path.	Install requirements in .env.	Small Parquet write/read succeeds.
Disk space failure	Output/portfolio scale too large.	Get-PSDrive -Name C and batch size.	Use quick/small, clear only known failed/generated folders after backup.	Full batch completes.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why is a manifest more than a file list?
2. When should a watermark advance?
3. Why might (timestamp, key) be safer than timestamp alone?
4. What happens to a partial failed batch?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. Which extraction mode should establish the first baseline?
2. What does a checksum prove and not prove?
3. Why does one generic updated_at filter fail across these sources?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture the SUCCESS manifest, source count queries, and matching checksum. Also capture the safe FAILED attempt status without exposing credentials.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
feat: extract manifest-driven local source batches
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

- ☐ Full batch extracted — [STATUS]
- ☐ All files/counts reconciled — [STATUS]
- ☐ Checksum independently verified — [STATUS]
- ☐ Failure behavior tested — [STATUS]
- ☐ Watermark commit rule explained — [STATUS]
- ☐ Per-table incremental design completed — [STATUS]
- ☐ Batch ready for Databricks upload — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-7. The quiz answer is in [the answer-key section immediately after the quiz in this workbook#phase-7] (the answer-key section immediately after the quiz in this workbook#phase-7).

Next phase: Phase 8 creates and verifies the Databricks workspace resources that will receive this exact batch.

Required action cards

Action 7.2.1 — Open scripts/extraction/extract_local_sources.py. Identify source table lists, batch ID creation, output fol

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open scripts/extraction/extract_local_sources.py. Identify source table lists, batch ID creation, output fol. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open scripts/extraction/extract_local_sources.py. Identify source table lists, batch ID creation, output folder, checksum, and finally manifest write.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 1

Action

Open scripts/extraction/extract_local_sources.py. Identify source table lists, batch ID creation, output folder, checksum, and finally manifest write.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 7 lesson index](#)

Action 7.2.2 — Open docs/runbook.md. Note that watermark advances only after successful downstream acceptance—not merely af

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open docs/runbook.md. Note that watermark advances only after successful downstream acceptance—not merely af. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open docs/runbook.md. Note that watermark advances only after successful downstream acceptance—not merely after one file writes.
4. Pause while the application finishes the operation; do not click the same control twice.

5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 2****Action**

Open `docs/runbook.md`. Note that watermark advances only after successful downstream acceptance—not merely after one file writes.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 7 lesson index](#)

Action 7.2.3 — For the first batch, use full mode. This establishes a known baseline and avoids pretending incremental logic

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

For the first batch, use full mode. This establishes a known baseline and avoids pretending incremental logic. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: For the first batch, use full mode. This establishes a known baseline and avoids pretending incremental logic is verified.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 3

Action

For the first batch, use full mode. This establishes a known baseline and avoids pretending incremental logic is verified.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 7 lesson index](#)

Action 7.2.4 — Run extraction. Copy the exact EXT- . . . batch folder name into the journal.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run extraction. Copy the exact EXT- . . . batch folder name into the journal. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run extraction. Copy the exact EXT- . . . batch folder name into the journal.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 4****Action**

Run extraction. Copy the exact EXT- . . . batch folder name into the journal.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 7 lesson index](#)

Action 7.2.5 — Open batch_manifest.json. Confirm status SUCCESS, mode full, source entries, row counts, checksums, and path

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open `batch_manifest.json`. Confirm status SUCCESS, mode full, source entries, row counts, checksums, and path. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open `batch_manifest.json`. Confirm status SUCCESS, mode full, source entries, row counts, checksums, and paths.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 5****Action**

Open `batch_manifest.json`. Confirm status SUCCESS, mode full, source entries, row counts, checksums, and paths.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 7 lesson index](#)

Action 7.2.6 — Compare extracted row counts to source SQL counts. For file feeds, compare line/record counts carefully: CSV h

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Compare extracted row counts to source SQL counts. For file feeds, compare line/record counts carefully: CSV h. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Compare extracted row counts to source SQL counts. For file feeds, compare line/record counts carefully: CSV header is not a data row; JSONL has one object per line.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 6

Action

Compare extracted row counts to source SQL counts. For file feeds, compare line/record counts carefully: CSV header is not a data row; JSONL has one object per line.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 7 lesson index](#)

Action 7.2.7 — Recompute a file checksum in PowerShell and compare to manifest.

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Recompute a file checksum in PowerShell and compare to manifest. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Recompute a file checksum in PowerShell and compare to manifest.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 7****Action**

Recompute a file checksum in PowerShell and compare to manifest.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 7 lesson index](#)

Action 7.2.8 — Design incremental behavior: choose updated_at only for tables that have it, a tie-breaker key for equal tim

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Design incremental behavior: choose updated_at only for tables that have it, a tie-breaker key for equal tim. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Design incremental behavior: choose updated_at only for tables that have it, a tie-breaker key for equal timestamps, and a separate strategy for append-only/event tables or snapshots.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 8

Action

Design incremental behavior: choose updated_at only for tables that have it, a tie-breaker key for equal timestamps, and a separate strategy for append-only/event tables or snapshots.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 7 lesson index](#)

Action 7.2.9 — Simulate a failure safely by pointing a practice run to a missing file-feed folder or invalid output—not by co

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Simulate a failure safely by pointing a practice run to a missing file-feed folder or invalid output—not by co. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Simulate a failure safely by pointing a practice run to a missing file-feed folder or invalid output—not by corrupting source data. Inspect FAILED manifest and confirm no watermark is advanced.
4. Pause while the application finishes the operation; do not click the same control twice.

5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 9****Action**

Simulate a failure safely by pointing a practice run to a missing file-feed folder or invalid output—not by corrupting source data. Inspect FAILED manifest and confirm no watermark is advanced.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 7 lesson index](#)

Action 7.2.10 — Complete TODO: store per-table watermarks rather than applying one generic updated_at filter to tables that

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Complete TODO: store per-table watermarks rather than applying one generic updated_at filter to tables that . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Complete TODO: store per-table watermarks rather than applying one generic updated_at filter to tables that do not have that column. The workbook reference extractor is intentionally a teaching skeleton here.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 10

Action

Complete TODO: store per-table watermarks rather than applying one generic updated_at filter to tables that do not have that column. The workbook reference extractor is intentionally a teaching skeleton here.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 7 lesson index](#)

Action 7.2.C1 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m scripts.extraction.extract_local_sources --mode full --file-feed-dir datasets\sample\files --output-root datasets\raw
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m scripts.extraction.extract_local_sources --mode full --file-feed-dir datasets\sample\files --output-root datasets\raw` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.extraction.extract_local_sources` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--mode` is a command option or flag; its exact behavior is defined beside the command; `full` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--file` is a command option or flag; its exact behavior is defined beside the command; `-feed` is a command option or flag; its exact behavior is defined beside the command; `-dir` is a command option or flag; its exact behavior is defined beside the command; `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sample` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `files` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--output` is a command option or flag; its exact behavior is defined beside the command; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Action 7.2.C2 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Get-ChildItem datasets\raw -Directory | Sort-Object LastWriteTime -Descending | Select-Object -First 1 FullName, Name, LastWriteTime
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** Get-ChildItem datasets\raw -Directory | Sort-Object LastWriteTime -Descending | Select-Object -First 1 FullName, Name, LastWriteTime — Get-ChildItem lists files and folders; datasets is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; raw is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Directory is a command option or flag; its exact behavior is defined beside the command; Sort-Object is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; LastWriteTime is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Descending is a command option or flag; its exact behavior is defined beside the com-

mand; `Select-Object` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-First` is a command option or flag; its exact behavior is defined beside the command; `1` is a numeric literal used by the command or expression; `FullName` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Action 7.2.C3 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Get-Content datasets\raw\EXT-YYYYMMDDTHMMSSZ\batch_manifest.json
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** Get-Content datasets\raw\EXT-YYYYMMDDTHMMSSZ\batch_manifest.json — Get-Content is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datasets is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; raw is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; EXT-YYYYMMDDTHMMSSZ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_manifest.json is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Action 7.2.C4 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Get-FileHash datasets\raw\EXT-YYYYMMDDTHMMSSZ\sqlserver\orders.parquet -Algorithm SHA256
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `Get-FileHash datasets\raw\EXT-YYYYMMDDTHHMMSSZ\sqlserver\orders.parquet -Algorithm SHA256` — `Get-FileHash` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `EXT-YYYYMMDDTHHMMSSZ` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlserver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders.parquet` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Algorithm` is a command option or flag; its exact behavior is defined beside the command; `SHA256` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Action 7.2.C5 — SSMS query window

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SSMS query window

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT COUNT_BIG(*) AS orders, MAX(updated_at) AS max_updated_at FROM erp.orders;
```

Where to enter it:

SSMS query window. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT COUNT_BIG(*) AS orders, MAX(updated_at) AS max_updated_at FROM erp.orders;` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT_BIG` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `MAX` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `updated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `max_updated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Action 7.2.C6 — pgAdmin Query Tool

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

pgAdmin Query Tool

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT COUNT(*) AS sessions, MAX(created_at) AS max_created_at FROM digital.web_sessions;
```

Where to enter it:

pgAdmin Query Tool. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** SELECT COUNT(*) AS sessions, MAX(created_at) AS max_created_at FROM digital.web_sessions; — SELECT requests data without changing it unless combined with a modifying statement; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; MAX is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; created_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; max_created_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 7.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 7 starting-screen checklist and re-verify the active context.

Lesson 7.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 7 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — scripts/

Exact folder name: `scripts`

Exact parent: `repository root`

Why it is needed now: Phase 7 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `scripts`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 7.D1.1 — Create the folder scripts

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\scripts" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\scripts" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\scripts"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\scripts" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run Test-Path ".\scripts" -PathType Container.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — scripts/extraction/

Exact folder name: extraction

Exact parent: scripts

Why it is needed now: Phase 7 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path scripts one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: extraction.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 7.D2.1 — Create the folder scripts/extraction

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\scripts\extraction" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\scripts\extraction" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\scripts\extraction` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\scripts\extraction" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts\extraction" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\scripts\extraction" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — scripts/utilities/

Exact folder name: `utilities`

Exact parent: `scripts`

Why it is needed now: Phase 7 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `scripts` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `utilities`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 7.D3.1 — Create the folder scripts/utilities

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\scripts\utilities" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\scripts\utilities" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\scripts\utilities" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\scripts\utilities" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts\utilities" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\scripts\utilities" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — `scripts/extraction/extract_local_sources.py`

Exact filename: `extract_local_sources.py`

Filename extension: `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `scripts/extraction` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 7.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in scripts/extraction one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `extract_local_sources.py`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `extract_local_sources.py` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 7.F1.1 — Create the file `scripts/extraction/extract_local_sources.py` from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\scripts\extraction\extract_local_sources.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\scripts\extraction\extract_local_sources.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\scripts\extraction\extract_local_sources.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\scripts\extraction\extract_local_sources.py" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "scripts/extraction/extract_local_sources.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `scripts/extraction`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `scripts/extraction/extract_local_sources.py`.
3. In PowerShell at the repository root, run `Test-Path ".\scripts\extraction\extract_local_sources.py"` and require `True`.
4. Run `git status --short -- "scripts/extraction/extract_local_sources.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `scripts/extraction/extract_local_sources.py`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `scripts/extraction/extract_local_sources.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 7.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
"""Extract local SQL Server, PostgreSQL, and file feeds for Track A."""
from __future__ import annotations

import argparse
import hashlib
import json
import logging
import shutil
from datetime import UTC, datetime
from pathlib import Path

import pandas as pd
from sqlalchemy import text

from scripts.utilities.config import load_environment
from scripts.utilities.database import postgres_engine, sqlserver_engine
from scripts.utilities.logging_utils import configure_logging

LOGGER = logging.getLogger("northstar.extract")
SQLSERVER_TABLES = ["customers", "addresses", "products", "product_categories", "suppliers", "stores", "employees", "orders", "order_items",
"payments", "shipments", "inventory_transactions", "inventory_snapshots"]
POSTGRES_TABLES = ["web_users", "web_sessions", "web_events", "campaigns", "campaign_touchpoints", "marketing_spend"]

def checksum(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()

def extract_table(connection, query: str, output: Path) -> dict[str, object]:
    frame = pd.read_sql_query(text(query), connection)
    output.parent.mkdir(parents=True, exist_ok=True)
    frame.to_parquet(output, index=False)
    return {"file": output.as_posix(), "rows": len(frame), "sha256": checksum(output)}

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--mode", choices=["full", "incremental"], default="full")
    parser.add_argument("--watermark", default="1900-01-01T00:00:00")
    parser.add_argument("--file-feed-dir", type=Path, default=Path("datasets/sample/files"))
    parser.add_argument("--output-root", type=Path, default=Path("datasets/raw"))
    args = parser.parse_args()
```

```

configure_logging(); load_environment()
now = datetime.now(UTC)
batch_id = f"EXT-{now:%Y%m%dT%H%M%S}"
batch_root = args.output_root / batch_id
manifest: dict[str, object] = {"batch_id": batch_id, "started_at_utc": now.isoformat(), "mode": args.mode, "watermark": args.watermark,
"files": []}
sqlserver_filter = "" if args.mode == "full" else " WHERE updated_at > :watermark"
postgres_filter = "" if args.mode == "full" else " WHERE updated_at > :watermark"
params = {"watermark": args.watermark}
try:
    with sqlserver_engine().connect() as connection:
        for table in SQLSERVER_TABLES:
            query = f"SELECT * FROM erp.{table}{sqlserver_filter}"
            frame = pd.read_sql_query(text(query), connection, params=params)
            path = batch_root / "sqlserver" / f"{table}.parquet"
            path.parent.mkdir(parents=True, exist_ok=True); frame.to_parquet(path, index=False)
            manifest["files"].append({"source_system": "sqlserver", "table": table, "file": path.as_posix(), "rows": len(frame), "sha256":
checksum(path)})
        with postgres_engine().connect() as connection:
            for table in POSTGRES_TABLES:
                query = f"SELECT * FROM digital.{table}{postgres_filter}"
                frame = pd.read_sql_query(text(query), connection, params=params)
                path = batch_root / "postgres" / f"{table}.parquet"
                path.parent.mkdir(parents=True, exist_ok=True); frame.to_parquet(path, index=False)
                manifest["files"].append({"source_system": "postgres", "table": table, "file": path.as_posix(), "rows": len(frame), "sha256":
checksum(path)})
            for source in args.file_feed_dir.glob("*"):
                if source.is_file():
                    target = batch_root / "files" / source.name
                    target.parent.mkdir(parents=True, exist_ok=True); shutil.copy2(source, target)
                    row_count = sum(1 for _ in target.open("rb")) - (1 if target.suffix == ".csv" else 0)
                    manifest["files"].append({"source_system": "files", "file": target.as_posix(), "rows": max(0, row_count), "sha256": check-
sum(target)})
            manifest["status"] = "SUCCESS"
except Exception as exc:
    manifest["status"] = "FAILED"; manifest["error"] = str(exc)
    raise
finally:
    batch_root.mkdir(parents=True, exist_ok=True)
    (batch_root / "batch_manifest.json").write_text(json.dumps(manifest, indent=2), encoding="utf-8")
    LOGGER.info("Wrote manifest %s", (batch_root / "batch_manifest.json").resolve())

if __name__ == "__main__":
    main()

```

Line-by-line explanation

- **Line 1:** `"""Extract local SQL Server, PostgreSQL, and file feeds for Track A."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `Extract local SQL Server, PostgreSQL, and file feeds for Track A.` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 2:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** blank line used only to separate logical sections for readability.
- **Line 4:** `import argparse` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `argparse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `import hashlib` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `hashlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `import json` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `json` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `import logging` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `logging` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `import shutil` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `shutil` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 9:** `from datetime import UTC, datetime` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UTC` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `from pathlib import Path` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pathlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** blank line used only to separate logical sections for readability.
- **Line 12:** `import pandas as pd` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pandas` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `from sqlalchemy import text` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlalchemy` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** blank line used only to separate logical sections for readability.
- **Line 15:** `from scripts.utilities.config import load_environment` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scripts.utilities.config` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `load_environment` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `from scripts.utilities.database import postgres_engine, sqlserver_engine` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scripts.utilities.database` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `postgres_engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlserver_engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** `from scripts.utilities.logging_utils import configure_logging` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scripts.utilities.logging_utils` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `configure_logging` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** `LOGGER = logging.getLogger("northstar.extract")` — `LOGGER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `logging.getLogger` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"northstar.extract"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** `SQLSERVER_TABLES = ["customers", "addresses", "products", "product_categories", "suppliers", "stores", "employees", "orders", "order_items", "payments", "shipments", "inventory_transactions", "inventory_snapshots"]` — `SQLSERVER_TABLES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"customers"` is a quoted literal; the quotation marks

preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "addresses" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "products" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "product_categories" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "suppliers" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 21:** `POSTGRES_TABLES = ["web_users", "web_sessions", "web_events", "campaigns", "campaign_touchpoints", "marketing_spend"]` — `POSTGRES_TABLES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web_users"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web_sessions"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web_events"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"campaigns"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"campaign_touchpoints"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 22:** blank line used only to separate logical sections for readability.
- **Line 23:** blank line used only to separate logical sections for readability.
- **Line 24:** `def checksum(path: Path) -> str:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `checksum` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** `return hashlib.sha256(path.read_bytes()).hexdigest()` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `hashlib.sha256` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.read_bytes` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `hexdigest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** blank line used only to separate logical sections for readability.
- **Line 27:** blank line used only to separate logical sections for readability.
- **Line 28:** `def extract_table(connection, query: str, output: Path) -> dict[str, object]:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `extract_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `output` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 29:** `frame = pd.read_sql_query(text(query), connection)` — `frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.read_sql_query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 30:** `output.parent.mkdir(parents=True, exist_ok=True)` — `output.parent.mkdir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parents` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exist_ok` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** `frame.to_parquet(output, index=False)` — `frame.to_parquet` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `output` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `index` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `False` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 32:** `return {"file": output.as_posix(), "rows": len(frame), "sha256": checksum(output)}` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"file"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `output.as_posix` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(`), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"rows"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 33:** blank line used only to separate logical sections for readability.
- **Line 34:** blank line used only to separate logical sections for readability.
- **Line 35:** `def main() -> None` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

project-specific role is described in the surrounding explanation; `parser.parse_args` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 42:** `configure_logging(); load_environment()` — `configure_logging` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `load_environment` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 43:** `now = datetime.now(UTC)` — `now` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime.now` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UTC` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 44:** `batch_id = f"EXT-{now:%Y%m%dT%H%M%SZ}"` — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"EXT-{now:%Y%m%dT%H%M%SZ}"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 45:** `batch_root = args.output_root / batch_id` — `batch_root` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.output_root` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** `manifest: dict[str, object] = {"batch_id": batch_id, "started_at_utc": now.isoformat(), "mode": args.mode, "watermark": args.watermark, "files": []}` — `manifest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dict` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `object` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 47:** `sqlserver_filter = "" if args.mode == "full" else " WHERE updated_at > :watermark"` — `sqlserver_filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `""` is a quoted literal; the quotation marks preserve spaces or mark text; `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"full"` is a quoted literal; the quotation marks preserve spaces or mark text; `else` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `" WHERE updated_at > :watermark"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 48:** `postgres_filter = "" if args.mode == "full" else " WHERE updated_at > :watermark"` — `postgres_filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `""` is a quoted literal; the quotation marks preserve spaces or mark text; `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"full"` is a quoted literal; the quotation marks preserve spaces or mark text; `else` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; " WHERE updated_at > :watermark" is a quoted literal; the quotation marks preserve spaces or mark text

- **Line 49:** `params = {"watermark": args.watermark}` — `params` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"watermark"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.watermark` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 50:** `try:` — `try` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 51:** `with sqlserver_engine().connect() as connection:` — `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlserver_engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connect` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 52:** `for table in SQLSERVER_TABLES:` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQLSERVER_TABLES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 53:** `query = f"SELECT * FROM erp.{table}{sqlserver_filter}"` — `query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SELECT * FROM erp.{table}{sqlserver_filter}"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 54:** `frame = pd.read_sql_query(text(query), connection, params=params)` — `frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.read_sql_query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `params` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 55:** `path = batch_root / "sqlserver" / f"{table}.parquet"` — `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_root` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"sqlserver"` is a quoted literal; the quotation marks preserve spaces or mark text; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{table}.parquet"` is a quoted literal; the quotation marks preserve spaces or mark text

- **Line 56:** `path.parent.mkdir(parents=True, exist_ok=True); frame.to_parquet(path, index=False)` — `path.parent.mkdir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parents` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exist_ok` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frame.to_parquet` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 57:** `manifest["files"].append({"source_system": "sqlserver", "table": table, "file": path.as_posix(), "rows": len(frame), "sha256": checksum(path)})` — `manifest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"files"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"source_system"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"sqlserver"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"table"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 58:** `with postgres_engine().connect() as connection:` — `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `postgres_engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connect` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 59:** `for table in POSTGRES_TABLES:` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `POSTGRES_TABLES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 60:** `query = f"SELECT * FROM digital.{table}{postgres_filter}"` — `query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SELECT * FROM digital.{table}{postgres_filter}"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 61:** `frame = pd.read_sql_query(text(query), connection, params=params)` — `frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.read_sql_query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; connection is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; params is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 62:** `path = batch_root / "postgres" / f"{table}.parquet"` — `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_root` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"postgres"` is a quoted literal; the quotation marks preserve spaces or mark text; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{table}.parquet"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 63:** `path.parent.mkdir(parents=True, exist_ok=True); frame.to_parquet(path, index=False)` — `path.parent.mkdir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parents` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exist_ok` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frame.to_parquet` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 64:** `manifest["files"].append({"source_system": "postgres", "table": table, "file": path.as_posix(), "rows": len(frame), "sha256": checksum(path)})` — `manifest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"files"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"source_system"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"postgres"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"table"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 65:** `for source in args.file_feed_dir.glob("*"):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.file_feed_dir.glob` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 66:** `if source.is_file():` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source.is_file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 67:** `target = batch_root / "files" / source.name` — `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_root` is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; "files" is a quoted literal; the quotation marks preserve spaces or mark text; / is a path, relative location, branch-style name, or file argument; verify every character before running it; source.name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- Line 68:** `target.parent.mkdir(parents=True, exist_ok=True); shutil.copy2(source, target)` — `target.parent.mkdir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parents` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exist_ok` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `shutil.copy2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 69:** `row_count = sum(1 for _ in target.open("rb")) - (1 if target.suffix == ".csv" else 0)` — `row_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sum` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `_` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.open` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"rb"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 70:** `manifest["files"].append({"source_system": "files", "file": target.as_posix(), "rows": max(0, row_count), "sha256": checksum(target)})` — `manifest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"files"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"source_system"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"files"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"file"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 71:** `manifest["status"] = "SUCCESS"` — `manifest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SUCCESS"` is a quoted literal; the quotation marks preserve spaces or mark text
- Line 72:** `except Exception as exc:` — `except` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Exception` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exc` is a name, keyword, argument,

identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 73:** `manifest["status"] = "FAILED"; manifest["error"] = str(exc)` — `manifest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"FAILED"` is a quoted literal; the quotation marks preserve spaces or mark text; `manifest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"error"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 74:** `raise` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 75:** `finally:` — `finally` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 76:** `batch_root.mkdir(parents=True, exist_ok=True)` — `batch_root.mkdir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parents` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exist_ok` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 77:** `(batch_root / "batch_manifest.json").write_text(json.dumps(manifest, indent=2), encoding="utf-8")` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_root` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"batch_manifest.json"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `write_text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `json.dumps` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manifest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `indent` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 78:** `LOGGER.info("Wrote manifest %s", (batch_root / "batch_manifest.json").resolve())` — `LOGGER.info` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Wrote manifest %s"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_root` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"batch_manifest.json"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `resolve` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding

explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 79:** blank line used only to separate logical sections for readability.
- **Line 80:** blank line used only to separate logical sections for readability.
- **Line 81:** `if __name__ == "__main__":` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__name__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"__main__"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 82:** `main()` — `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 7 lesson index](#)

File introduction — `scripts/utilities/checksums.py`

Exact filename: `checksums.py`

Filename extension: `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `scripts/utilities` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 7.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `scripts/utilities` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `checksums.py`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `checksums.py` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 7.F2.1 — Create the file `scripts/utilities/checksums.py` from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\scripts\utilities\checksums.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType File -Path ".\scripts\utilities\checksums.py" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; File is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\scripts\utilities\checksums.py" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\scripts\utilities\checksums.py" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "scripts/utilities/checksums.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `scripts/utilities`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `scripts/utilities/checksums.py`.
3. In PowerShell at the repository root, run `Test-Path ".\scripts\utilities\checksums.py"` and require `True`.
4. Run `git status --short -- "scripts/utilities/checksums.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for scripts/utilities/checksums.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `scripts/utilities/checksums.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 7.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
"""Small checksum functions used by extraction and reconciliation scripts."""
from __future__ import annotations

import hashlib
from pathlib import Path

def sha256_file(path: str | Path) -> str:
    """Return the SHA-256 checksum of a file without loading it all into memory."""
    file_path = Path(path)
    digest = hashlib.sha256()
    with file_path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()
```

Line-by-line explanation

- **Line 1:** `"""Small checksum functions used by extraction and reconciliation scripts."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `Small checksum functions used by extraction and reconciliation scripts.` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 2:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** blank line used only to separate logical sections for readability.
- **Line 4:** `import hashlib` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `hashlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `from pathlib import Path` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pathlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `def sha256_file(path: str | Path) -> str:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sha256_file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `"""Return the SHA-256 checksum of a file without loading it all into memory."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `Return the SHA-256 checksum of a file without loading it all into memory.` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 10:** `file_path = Path(path)` — `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `file_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 11:** `digest = hashlib.sha256()` — `digest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `hashlib.sha256` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `with file_path.open("rb") as handle:` — `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `file_path.open` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"rb"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `handle` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `for chunk in iter(lambda: handle.read(1024 * 1024), b""):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `chunk` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `iter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `lambda` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `handle.read` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1024` is a numeric literal used by the command or expression; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1024` is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 14:** `digest.update(chunk)` — `digest.update` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `chunk` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** `return digest.hexdigest()` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `digest.hexdigest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 7 lesson index](#)

File introduction — `scripts/utilities/database.py`

Exact filename: `database.py`

Filename extension: `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `scripts/utilities` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 7.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.

2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in scripts/utilities one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `database.py`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `database.py` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 7.F3.1 — Create the file `scripts/utilities/database.py` from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\scripts\utilities\database.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\scripts\utilities\database.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is

a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\scripts\utilities\database.py" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\scripts\utilities\database.py" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add -Force, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "scripts/utilities/database.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use TODO(USER) where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under scripts/utilities.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with scripts/utilities/database.py.
3. In PowerShell at the repository root, run Test-Path ".\scripts\utilities\database.py" and require True.
4. Run git status --short -- "scripts/utilities/database.py" to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for scripts/utilities/database.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: scripts/utilities/database.py

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 7.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
"""Safe SQLAlchemy connection factories for local SQL Server and PostgreSQL."""
from __future__ import annotations

from urllib.parse import quote_plus

from sqlalchemy import Engine, create_engine

from scripts.utilities.config import required_env


def sqlserver_engine() -> Engine:
    """Create a SQL Server engine using values from .env, never hard-coded secrets."""
    server = required_env("SQLSERVER_HOST")
    database = required_env("SQLSERVER_SOURCE_DATABASE")
    driver = required_env("SQLSERVER_ODBC_DRIVER")
    trusted = required_env("SQLSERVER_TRUSTED_CONNECTION").lower() == "yes"
    trust_certificate = required_env("SQLSERVER_TRUST_SERVER_CERTIFICATE")

    if trusted:
        odbc = (
            f"DRIVER={{driver}};SERVER={server};DATABASE={database};"
            f"Trusted_Connection=yes;TrustServerCertificate={trust_certificate};"
        )
    else:
        username = required_env("SQLSERVER_USERNAME")
        password = required_env("SQLSERVER_PASSWORD")
        odbc = (
            f"DRIVER={{driver}};SERVER={server};DATABASE={database};"
            f"UID={username};PWD={password};TrustServerCertificate={trust_certificate};"
        )
    return create_engine(f"mssql+pyodbc:///?odbc_connect={quote_plus(odbc)}", future=True)


def sqlserver_warehouse_engine() -> Engine:
    """Create a SQL Server warehouse engine using the warehouse database name."""
    import os

    original = os.environ.get("SQLSERVER_SOURCE_DATABASE")
    os.environ["SQLSERVER_SOURCE_DATABASE"] = required_env("SQLSERVER_WAREHOUSE_DATABASE")
    try:
        return sqlserver_engine()
    finally:
        if original is None:
            os.environ.pop("SQLSERVER_SOURCE_DATABASE", None)
        else:
            os.environ["SQLSERVER_SOURCE_DATABASE"] = original


def postgres_engine() -> Engine:
    """Create a PostgreSQL engine using values from .env."""
```

```

user = required_env("POSTGRES_USER")
password = quote_plus(required_env("POSTGRES_PASSWORD"))
host = required_env("POSTGRES_HOST")
port = required_env("POSTGRES_PORT")
database = required_env("POSTGRES_DATABASE")
return create_engine(
    f"postgresql+psycopg://:{user}:{password}@{host}:{port}/{database}", future=True
)

```

Line-by-line explanation

- **Line 1:** `"""Safe SQLAlchemy connection factories for local SQL Server and PostgreSQL."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `"Safe SQLAlchemy connection factories for local SQL Server and PostgreSQL."` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 2:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** blank line used only to separate logical sections for readability.
- **Line 4:** `from urllib.parse import quote_plus` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `urllib.parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quote_plus` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** `from sqlalchemy import Engine, create_engine` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlalchemy` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `create_engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `from scripts.utilities.config import required_env` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scripts.utilities.config` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** blank line used only to separate logical sections for readability.
- **Line 10:** blank line used only to separate logical sections for readability.
- **Line 11:** `def sqlserver_engine() -> Engine:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlserver_engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `Engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `"""Create a SQL Server engine using values from .env, never hard-coded secrets."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `"Create a SQL Server engine using values from .env, never hard-coded secrets."` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 13:** `server = required_env("SQLSERVER_HOST")` — `server` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SQLSERVER_HOST"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 14:** `database = required_env("SQLSERVER_SOURCE_DATABASE")` — `database` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SQLSERVER_SOURCE_DATABASE"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** `driver = required_env("SQLSERVER_ODBC_DRIVER")` — `driver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SQLSERVER_ODBC_DRIVER"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `trusted = required_env("SQLSERVER_TRUSTED_CONNECTION").lower() == "yes"` — `trusted` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SQLSERVER_TRUSTED_CONNECTION"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `lower` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"yes"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 17:** `trust_certificate = required_env("SQLSERVER_TRUST_SERVER_CERTIFICATE")` — `trust_certificate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SQLSERVER_TRUST_SERVER_CERTIFICATE"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** `if trusted:` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `trusted` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** `odbc = (` — `odbc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** `f"DRIVER={{driver}};SERVER={server};DATABASE={database};"` — `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"DRIVER={{driver}};SERVER={server};DATABASE={database};"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 22:** `f"Trusted_Connection=yes;TrustServerCertificate={trust_certificate};"` — `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Trusted_Connection=yes;TrustServerCertificate={trust_certificate};"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 23:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** `else:` — `else` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** `username = required_env("SQLSERVER_USERNAME")` — `username` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_env` is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "SQLSERVER_USERNAME" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 26:** password = required_env("SQLSERVER_PASSWORD") — password is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; required_env is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "SQLSERVER_PASSWORD" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:** odbc = (— odbc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** f"DRIVER={{driver}};SERVER={server};DATABASE={database};" — f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "DRIVER={{driver}};SERVER={server};DATABASE={database};" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 29:** f"UID={username};PWD={password};TrustServerCertificate={trust_certificate};" — f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "UID={username};PWD={password};TrustServerCertificate={trust_certificate};" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 30:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** return create_engine(f"mssql+pyodbc:///odbc_connect={quote_plus(odbc)}", future=True) — return is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; create_engine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "mssql+pyodbc:///odbc_connect={quote_plus(odbc)}" is a path, relative location, branch-style name, or file argument; verify every character before running it; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; future is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; True is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 32:** blank line used only to separate logical sections for readability.
- **Line 33:** blank line used only to separate logical sections for readability.
- **Line 34:** def sqlserver_warehouse_engine() -> Engine: — def is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sqlserver_warehouse_engine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -> is a command option or flag; its exact behavior is defined beside the command; Engine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** """Create a SQL Server warehouse engine using the warehouse database name.""" — """ is a quoted literal; the quotation marks preserve spaces or mark text; "Create a SQL Server warehouse engine using the warehouse database name." is a quoted literal; the quotation marks preserve spaces or mark text; """ is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 36:** import os — import is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; os is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 37:** blank line used only to separate logical sections for readability.
- **Line 38:** original = os.environ.get("SQLSERVER_SOURCE_DATABASE") — original is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; os.environ.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

- (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "SQLSERVER_SOURCE_DATABASE" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 39:** `os.environ["SQLSERVER_SOURCE_DATABASE"] = required_env("SQLSERVER_WAREHOUSE_DATABASE")` — `os.environ` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SQLSERVER_SOURCE_DATABASE"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SQLSERVER_WAREHOUSE_DATABASE"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 40:** `try:` — `try` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 41:** `return sqlserver_engine()` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlserver_engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 42:** `finally:` — `finally` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 43:** `if original is None:` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `original` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 44:** `os.environ.pop("SQLSERVER_SOURCE_DATABASE", None)` — `os.environ.pop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SQLSERVER_SOURCE_DATABASE"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 45:** `else:` — `else` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 46:** `os.environ["SQLSERVER_SOURCE_DATABASE"] = original` — `os.environ` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SQLSERVER_SOURCE_DATABASE"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `original` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
 - **Line 47:** blank line used only to separate logical sections for readability.
 - **Line 48:** blank line used only to separate logical sections for readability.
 - **Line 49:** `def postgres_engine() -> Engine:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `postgres_engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `Engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 50:** `""Create a PostgreSQL engine using values from .env.""` — `""` is a quoted literal; the quotation marks preserve spaces or mark text; `"Create a PostgreSQL engine using values from .env."` is a quoted literal; the quotation marks preserve spaces or mark text; `""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 51:** `user = required_env("POSTGRES_USER")` — `user` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"POSTGRES_USER"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 52:** `password = quote_plus(required_env("POSTGRES_PASSWORD"))` — `password` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quote_plus` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"POSTGRES_PASSWORD"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 53:** `host = required_env("POSTGRES_HOST")` — `host` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"POSTGRES_HOST"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 54:** `port = required_env("POSTGRES_PORT")` — `port` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"POSTGRES_PORT"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 55:** `database = required_env("POSTGRES_DATABASE")` — `database` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"POSTGRES_DATABASE"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 56:** `return create_engine(` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `create_engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 57:** `f"postgresql+psycopg://{user}:{password}@{host}:{port}/{database}", future=True` — `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `postgresql+psycopg://{user}:{password}@{host}:{port}/{database}"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `future` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 58:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 7 lesson index](#)

What your screen should look like now

- Every Phase 7 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 7.3 completion checklist

☐

scripts/extraction/extract_local_sources.py was created by me, saved, located correctly, and inspected — [STATUS]

☐

scripts/utilities/checksums.py was created by me, saved, located correctly, and inspected — [STATUS]

☐

scripts/utilities/database.py was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 7.4](#) and recheck the root and branch.

Lesson 7.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Action 7.4.G1 — Inspect the current state without changing it

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Inspect the current state without changing it. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 7 lesson index](#)

Action 7.4.G2 — Create the smallest required object or decision

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Create the smallest required object or decision. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 7 lesson index](#)

Action 7.4.G3 — Run the phase-specific verification

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run the phase-specific verification. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

Action 7.4.G4 — Demonstrate one safe failure or alternate-result branch

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Demonstrate one safe failure or alternate-result branch. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 7 lesson index](#)

Action 7.4.G5 — Record evidence and prepare the downstream handoff

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Record evidence and prepare the downstream handoff. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 7 lesson index](#)

What your screen should look like now

- The new Phase 7 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 7.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 7.5](#).

Lesson 7.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 7 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — CSV versus Parquet extracts

Question:

Which local interchange format should the beginner path use?

Options considered:

- CSV/JSON plus manifest.
- Parquet.
- Direct JDBC from Databricks to localhost.

Beginner path selected:

Use CSV and JSON with manifests and checksums for Track A.

Why this path was selected:

The files are inspectable with basic tools and make schema/quoting issues visible.

Tradeoff:

They are larger and less type-safe than Parquet.

What a company might do instead:

Production pipelines often use Parquet/Delta in cloud object storage and CDC or direct networked ingestion.

How to explain this in an interview:

I chose inspectable files for the local handoff, added checksums and manifests, and documented Parquet as a scale-oriented alternative.

Design decision — Full versus incremental loading

Question:

When should the extractor read every source row and when should it read only new or changed rows?

Options considered:

- Full extraction for every run.
- Incremental extraction from the first run.
- A full baseline followed by audited incremental batches.

Beginner path selected:

Create and reconcile one full baseline, then use incremental extraction with a timestamp-plus-key boundary and advance the watermark only after success.

Why this path was selected:

The baseline proves completeness and supplies a recovery point; incremental batches then teach efficiency, replay safety, and late/change handling.

Tradeoff:

Two modes require more tests, metadata, and correction logic than a full-only learning pipeline.

What a company might do instead:

A company may use CDC, transaction-log positions, event streams, snapshot isolation, backfill tooling, and automated restart/reconciliation controls.

How to explain this in an interview:

I established a full baseline before introducing incremental watermarks so efficiency did not replace completeness or recoverability.

If you see Result A

The full baseline reconciles and the incremental batch advances its watermark only after success. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

No rows qualify, duplicate timestamps create an ambiguous boundary, or a failed batch has partial output. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: VS Code, PowerShell, SSMS, pgAdmin, Python, and File Explorer.

Window/panel: Show successful manifest, checksum, watermark, source counts, and a safe failed-attempt record.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase07_build-full-and-incremental-local-extraction_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 7's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 7 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 7.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.

7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Action 7.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Action 7.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Interpret `git status --short`

- ?? means untracked: Git sees a new path but it is not staged.
- M means modified in the working tree but not staged.
- M means the staged version differs from the last commit.
- MM means a staged version exists and the file has additional unstaged edits.
- A means staged as a new file.
- D means deletion; stop and investigate unless deletion is deliberate.
- R means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 7.5.G4 — Inspect the unstaged diff for scripts/extraction/extract_local_sources.py

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "scripts/extraction/extract_local_sources.py"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "scripts/extraction/extract_local_sources.py"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding ex-

planation; -- is a command option or flag; its exact behavior is defined beside the command; "scripts/extraction/extract_local_sources.py" is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Action 7.5.G5 — Stage only the reviewed file scripts/extraction/extract_local_sources.py

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "scripts/extraction/extract_local_sources.py"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "scripts/extraction/extract_local_sources.py"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"scripts/extraction/extract_local_sources.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 7.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches `The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Action 7.5.G7 — Create the Phase 7 commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "feat: add manifest-driven incremental extracts"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "feat: add manifest-driven incremental extracts"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"feat: add manifest-driven incremental extracts"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use

`git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Action 7.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Action 7.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 7 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message feat: add manifest-driven incremental extracts and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 7.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 7 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

"Phase 7 created the verified boundary described by **Build full and incremental local extraction**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed."

Mini quiz

1. Multiple choice: Why does Phase 7 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

My Phase 7 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 7.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 8 creates the cloud landing location that receives those local batches.

Phase 7 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 8 — Configure Databricks, governed storage, notebooks, and compute

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: A valid local batch, manifest, and verified Databricks environment.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 9 ingests that exact batch into append-only raw Delta tables.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 8 lesson index

1. [Lesson 8.1 — Understand the phase and starting screen](#)
2. [Lesson 8.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 8.3 — Create and understand the phase files](#)
4. [Lesson 8.4 — Build and validate required objects](#)
5. [Lesson 8.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 8.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** Web browser, Databricks workspace, VS Code, and File Explorer.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 8 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 7 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 8 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

Web browser, Databricks workspace, VS Code, and File Explorer. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 8 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 8 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 8 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 8.1 completion checklist

☐

I read why Phase 8 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 8.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 8 lesson index](#)

Lesson 8.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 8 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

An available Databricks workspace configured with a usable catalog, landing, bronze, silver, gold, and quality schemas, a governed volume for Track A files, Git-friendly source notebooks, and a successful `00_environment_check` run using the exact uploaded batch ID.

2. Why it matters

Databricks is the remote lakehouse and orchestration environment. Before transforming data, you must prove identity, object permissions, storage paths, Spark, parameters, and Delta read/write. This isolates environment failures from business-code failures.

3. Words to know

Term	Plain-language definition
Workspace	Databricks environment containing notebooks, data, compute, SQL, and Jobs.
Compute	Resources that execute Spark/SQL. Free Edition commonly provides serverless managed compute subject to quotas.
Catalog	Top governed namespace in Unity Catalog.
Schema	Namespace below a catalog for tables/volumes.
Volume	Governed file storage for non-tabular files and batch handoff.
Notebook	Document with executable cells and Markdown.
Git folder	Databricks workspace folder linked to a Git repository; older documentation may call it Repos.
Cluster	User-configured Spark compute; availability varies by tier.
SQL warehouse	Compute endpoint for Databricks SQL and BI queries.
Secret	Protected credential value referenced without printing it.
Widget	Notebook input parameter.
Task value	Small output passed/recorded between Job tasks.
Serverless	Provider-managed compute; feature/quotas vary by workspace.
Catalog Explorer	UI for catalogs, schemas, tables, volumes, and permissions.

4. What must already be completed

A verified SUCCESS extraction batch exists. Databricks account/workspace availability and current limitations were reviewed in Phase 2. The feature branch is pushed if using Git folder integration.

5. Applications to open

Application	How to open it	Why it is open
Web browser	Open the Databricks workspace URL.	Create/inspect workspace objects and run notebook.
VS Code	Open databricks/notebooks/00_environment_check.py and databricks/README.md.	Understand every cell before importing/running.
File Explorer	Open exact local datasets/raw/EXT-... batch.	Upload the correct immutable folder.
Windows PowerShell	Repository root.	Read manifest and verify batch ID/checksums before upload.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run `Get-Location`; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
<code>\$BatchId = 'EXT-YYYYMMDDTHHMMSSZ'; Get-Content "data-sets\raw\$BatchId\batch_manifest.json"</code>	Prints the local manifest before upload.	The remote parameter must match a complete local batch.
<code>print(spark.version)</code>	Prints Spark and resolved parameters/path.	Hidden defaults cause wrong-catalog and wrong-batch failures.
<code>SHOW SCHEMAS IN northstar_retail;</code>	Lists schemas in the chosen catalog.	Verifies namespaces independently of notebook variables.
<code>dbutils.fs.ls(f"/Volumes/{catalog}/{landing_schema}/{volume}/{batch_id}")</code>	Lists uploaded batch contents.	Proves compute can access the exact handoff.
<code>SELECT * FROM northstar_retail.quality.environment_check ORDER BY checked_at_utc DESC LIMIT 10;</code>	Reads the Delta test table written by Spark.	Proves table creation and SQL access.

Compute choice summary

- Use available serverless notebook/job compute for Track A when the workspace provides it.
- A SQL warehouse runs Databricks SQL and BI queries; it is not the same execution surface as a PySpark notebook task.
- Free Edition is quota-limited and, as verified on 2026-08-16, Jobs concurrency is limited; the documented graph stays below five parallel tasks. Verify current limits again when you run.

Catalog fallback

If `CREATE CATALOG` is denied, do not request administrator rights merely for a name. Use a writable catalog, record it, and pass it consistently. The architecture is about governed namespaces, not the literal string `northstar_retail`.

MY TASK

Create/identify the governed namespaces and volume, upload one exact SUCCESS batch, import/sync the notebooks, run `00_environment_check`, and document any tier/UI variance with a screenshot and official source.

STARTER SKELETON

```
TODO(USER) Databricks environment record
Workspace/tier/cloud: _____
Catalog actually used: _____
Schemas: _____
Volume path: _____
Git folder path or import fallback: _____
Compute type: _____
SQL warehouse availability: _____
Batch ID: _____
Environment check run ID/time: _____
Tier/UI variance and official source: _____
```

Acceptance criteria: The exact batch path is readable, the test Delta table contains a PASS row, all parameters are recorded, and no notebook contains a token/password.

HINT 1 — CONCEPT

Use actual writable resources; names are parameters, not hard-coded truth.

HINT 2 — PSEUDOCODE

Resolve volume path from catalog/schema/volume and append the exact manifest batch_id.

HINT 3 — NEAR-COMPLETE PATTERN

Run the environment notebook reference in this workbook cell by cell, comparing output to the environment record.

13. Expected files or output

- Five schemas in a writable catalog.
- Managed volume with preserved batch folder.
- Workspace Git folder or documented import fallback.
- PASS row in quality.environment_check.
- Spark/version/path parameter evidence.
- No Track B credential requirement for Track A.

14. How to verify

- Volume browser and `dbutils.fs.ls` show the same files.
- Batch ID matches local manifest.
- Spark can write/read Delta table.
- Catalog Explorer can preview the table.
- Git folder points to intended repository/branch or fallback is documented.

15. Automated tests

- `00_environment_check` assertions.
- Manual uploaded checksum comparison for at least one file if tooling permits download/read.
- Environment status task value equals PASS when run as a Job later.
- No secret scanner finding in Git.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
PERMISSION_DENIED creating catalog/schema/volume	Workspace identity lacks privilege.	Inspect object named in error; <code>SHOW CATALOGS/grants</code> if allowed.	Use permitted catalog/schema or request minimum grant; do not use admin token workaround.	Environment check writes table.
Batch path not found	Upload flattened/renamed/wrong parameter.	Volume browser; print batch_root; compare manifest ID.	Recreate exact folder structure and parameter.	ls returns contents.

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
Git folder authentication fails	Credential/provider/branch configuration issue.	Workspace user settings and official Git folder docs.	Use supported OAuth/PAT securely or manual import fallback.	Notebooks visible and current.
Compute unavailable/quota exceeded	Tier quota or temporary capacity.	Workspace banner/run error and limits page.	Use quick data, stop repeated retries, resume when quota permits; do not claim run.	Environment run completes.
Notebook creates wrong catalog	Default widgets not replaced.	Print widgets/current_catalog.	Set job/notebook parameters consistently; clean accidental disposable object only after review.	Objects live in recorded catalog.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why does Track A use a volume instead of JDBC to localhost?
2. What is the difference between Spark compute and a SQL warehouse?
3. Why is environment validation a separate first task?
4. What should you do when the UI differs from the workbook?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. What path pattern identifies a Unity Catalog volume?
2. Which parameter must exactly match the extraction folder?
3. Why are Databricks secrets unnecessary for local-source ingestion in Track A?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture Catalog Explorer with schemas/volume, the preserved batch folder, and the environment-check PASS output/table. Hide personal email where unnecessary.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
docs: verify databricks workspace and track a volume handoff
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

Workspace/tier documented — [STATUS]

☐

Writable catalog chosen — [STATUS]

☐

Schemas created — [STATUS]

☐

Volume and batch uploaded — [STATUS]

☐

Git/import path established — [STATUS]

☐

Environment notebook passed — [STATUS]

☐

Delta table verified — [STATUS]

☐

No secret exposed — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-8. The quiz answer is in [the answer-key section immediately after the quiz in this workbook#phase-8] (the answer-key section immediately after the quiz in this workbook#phase-8).

Next phase: Phase 9 ingests the uploaded SQL Server, PostgreSQL, CSV, and JSON extracts into Bronze Delta tables.

Required action cards

Action 8.2.1 — Sign in to the available Databricks workspace. Record cloud/region/workspace tier and today's date. Free Editi

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Sign in to the available Databricks workspace. Record cloud/region/workspace tier and today's date. Free Editi. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Sign in to the available Databricks workspace. Record cloud/region/workspace tier and today's date. Free Edition and paid workspaces can show different menus and privileges.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 1

Action

Sign in to the available Databricks workspace. Record cloud/region/workspace tier and today's date. Free Edition and paid workspaces can show different menus and privileges.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.2.2 — In the left navigation, open or . Look for an existing writable catalog. If you can create one, choose and nam

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

In the left navigation, open or . Look for an existing writable catalog. If you can create one, choose and nam. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: In the left navigation, open **Catalog** or **Catalog Explorer**. Look for an existing writable catalog. If you can create one, choose **Create catalog** and name it `northstar_retail`. If permission is denied, use a catalog the workspace grants, commonly `workspace`, and pass that actual name to every widget/job parameter.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 2****Action**

In the left navigation, open **Catalog** or **Catalog Explorer**. Look for an existing writable catalog. If you can create one, choose **Create catalog** and name it `northstar_retail`. If permission is denied, use a catalog the workspace grants, commonly `workspace`, and pass that actual name to every widget/job parameter.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.2.3 — Inside the chosen catalog, create schemas landing, bronze, silver, gold, and quality using or the SQ

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Inside the chosen catalog, create schemas landing, bronze, silver, gold, and quality using or the SQ. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Inside the chosen catalog, create schemas landing, bronze, silver, gold, and quality using **Create > Schema** or the SQL editor. Add descriptive comments when the UI allows.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 3

Action

Inside the chosen catalog, create schemas landing, bronze, silver, gold, and quality using **Create > Schema** or the SQL editor. Add descriptive comments when the UI allows.

Before continuing

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.2.4 — Open landing; choose , name it track_a_files, and choose a managed volume unless your workspace administra

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open landing; choose , name it track_a_files, and choose a managed volume unless your workspace administra. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open landing; choose **Create > Volume**, name it `track_a_files`, and choose a managed volume unless your workspace administrator has defined an external location. Record the volume path / Volumes/<catalog>/landing/track_a_files.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 4****Action**

Open landing; choose **Create > Volume**, name it `track_a_files`, and choose a managed volume unless your workspace administrator has defined an external location. Record the volume path /Volumes/<catalog>/landing/track_a_files.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.2.5 — Open the volume. Create or upload the exact extraction folder EXT- . . . , preserving subfolders sqlserver, `p

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open the volume. Create or upload the exact extraction folder EXT- . . . , preserving subfolders sqlserver, `p. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open the volume. Create or upload the exact extraction folder EXT- . . . , preserving subfolders sqlserver, postgres, and files, plus batch_manifest.json. If the UI cannot upload a folder recursively, create the subfolders in the volume browser and upload files in groups. Do not flatten filenames.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL psql, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 5

Action

Open the volume. Create or upload the exact extraction folder EXT- . . . , preserving subfolders sqlserver, postgres, and files, plus batch_manifest.json. If the UI cannot upload a folder recursively, create the subfolders in the volume browser and upload files in groups. Do not flatten filenames.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.2.6 — Open . Choose (or current equivalent). Enter the public repository URL, GitHub provider, and feature branch. F

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open . Choose (or current equivalent). Enter the public repository URL, GitHub provider, and feature branch. F. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open **Workspace**. Choose **Create > Git folder** (or current equivalent). Enter the public repository URL, GitHub provider, and feature branch. For write/pull operations, configure Git credentials through user settings according to current official docs; do not paste a token into a notebook.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 6****Action**

Open **Workspace**. Choose **Create > Git folder** (or current equivalent). Enter the public repository URL, GitHub provider, and feature branch. For write/pull operations, configure Git credentials through user settings according to current official docs; do not paste a token into a notebook.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.2.7 — If Git folder is unavailable, create a normal workspace folder and use to upload the .py source notebooks. R

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

If Git folder is unavailable, create a normal workspace folder and use to upload the .py source notebooks. R. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: If Git folder is unavailable, create a normal workspace folder and use **Import** to upload the .py source notebooks. Record that this is a manual fallback.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 7

Action

If Git folder is unavailable, create a normal workspace folder and use **Import** to upload the .py source notebooks. Record that this is a manual fallback.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.2.8 — Open `00_environment_check.py` as a notebook. Attach/choose available compute if prompted. Free Edition often

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open `00_environment_check.py` as a notebook. Attach/choose available compute if prompted. Free Edition often . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open `00_environment_check.py` as a notebook. Attach/choose available compute if prompted. Free Edition often uses serverless compute automatically; do not invent a cluster that the tier does not offer.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 8****Action**

Open `00_environment_check.py` as a notebook. Attach/choose available compute if prompted. Free Edition often uses serverless compute automatically; do not invent a cluster that the tier does not offer.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.2.9 — Read every cell. In the widget area, set the actual catalog, schemas, volume, and exact batch_id. The notebo

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Read every cell. In the widget area, set the actual catalog, schemas, volume, and exact batch_id. The notebook. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Read every cell. In the widget area, set the actual catalog, schemas, volume, and exact batch_id. The notebook creates missing permitted objects, writes a tiny Delta test row, lists the uploaded batch path, and sets task values.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 9****Action**

Read every cell. In the widget area, set the actual catalog, schemas, volume, and exact batch_id. The notebook creates missing permitted objects, writes a tiny Delta test row, lists the uploaded batch path, and sets task values.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.2.10 — Run . Read output from each cell. A failure to list the batch is not fixed by changing code blindly; verify pa

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run . Read output from each cell. A failure to list the batch is not fixed by changing code blindly; verify pa. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run **Run all**. Read output from each cell. A failure to list the batch is not fixed by changing code blindly; verify path/upload first.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 10

Action

Run **Run all**. Read output from each cell. A failure to list the batch is not fixed by changing code blindly; verify path/upload first.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.2.11 — Open Catalog Explorer and locate <catalog>.quality.environment_check. Preview rows and find your batch ID.

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open Catalog Explorer and locate <catalog>.quality.environment_check. Preview rows and find your batch ID. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open Catalog Explorer and locate <catalog>.quality.environment_check. Preview rows and find your batch ID.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 11****Action**

Open Catalog Explorer and locate <catalog>.quality.environment_check. Preview rows and find your batch ID.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.2.12 — Open and inspect available SQL warehouse/serverless options. Do not create paid capacity without understanding

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open and inspect available SQL warehouse/serverless options. Do not create paid capacity without understanding. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open **SQL** and inspect available SQL warehouse/serverless options. Do not create paid capacity without understanding cost. The SQL warehouse is for SQL/BI consumption; Spark notebook compute executes PySpark tasks.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 12

Action

Open **SQL** and inspect available SQL warehouse/serverless options. Do not create paid capacity without understanding cost. The SQL warehouse is for SQL/BI consumption; Spark notebook compute executes PySpark tasks.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.2.13 — Read the secret-management official page. Track A needs no database password in Databricks because it reads up

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Read the secret-management official page. Track A needs no database password in Databricks because it reads up. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Read the secret-management official page. Track A needs no database password in Databricks because it reads uploaded files. Create secrets only for a real Track B connection and never display them.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 13****Action**

Read the secret-management official page. Track A needs no database password in Databricks because it reads uploaded files. Create secrets only for a real Track B connection and never display them.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.2.C1 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
$BatchId = 'EXT-YYYYMMDDTHHMMSSZ'; Get-Content "datasets\raw\$BatchId\batch_manifest.json"
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `$BatchId = 'EXT-YYYYMMDDTHHMMSSZ'; Get-Content "datasets\raw\$BatchId\batch_manifest.json"` — `$BatchId` is a variable or environment-value reference rather than literal output; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'EXT-YYYYMMDDTHHMMSSZ'` is a quoted literal; the quotation marks preserve spaces or mark text; `Get-Content` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"datasets\raw\$BatchId\batch_manifest.json"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

Action 8.2.C2 — Databricks notebook cell

Application:

Databricks workspace in a web browser

Window or panel:

Databricks notebook cell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Python instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Python):

```
print(spark.version)
print({"catalog": catalog, "batch_root": batch_root, "batch_id": batch_id})
```

Where to enter it:

Databricks notebook cell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `print(spark.version)` — `print` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.version` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `print({"catalog": catalog, "batch_root": batch_root, "batch_id": batch_id})` — `print` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"catalog"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `catalog` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_root"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_root` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use

`git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

Action 8.2.C3 — Databricks SQL editor or %sql notebook cell

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor or %sql notebook cell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SHOW SCHEMAS IN northstar_retail;
```

Where to enter it:

Databricks SQL editor or %sql notebook cell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SHOW SCHEMAS IN northstar_retail;` — `SHOW` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCHEMAS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `northstar_retail` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

Action 8.2.C4 — Databricks notebook cell

Application:

Databricks workspace in a web browser

Window or panel:

Databricks notebook cell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Python instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Python):

```
dbutils.fs.ls(f"/Volumes/{catalog}/{landing_schema}/{volume}/{batch_id}")
```

Where to enter it:

Databricks notebook cell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `dbutils.fs.ls(f"/Volumes/{catalog}/{landing_schema}/{volume}/{batch_id}")` — `dbutils.fs.ls` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/{catalog}/{landing_schema}/{volume}/{batch_id}"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

Action 8.2.C5 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT * FROM northstar_retail.quality.environment_check ORDER BY checked_at_utc DESC LIMIT 10;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT * FROM northstar_retail.quality.environment_check ORDER BY checked_at_utc DESC LIMIT 10; —` `SELECT` requests data without changing it unless combined with a modifying statement; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `northstar_retail.quality.environment_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `checked_at_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DESC` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `LIMIT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `10` is a numeric literal used by the command or expression

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 8.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 8 starting-screen checklist and re-verify the active context.

Lesson 8.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 8 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — databricks/

Exact folder name: databricks

Exact parent: repository root

Why it is needed now: Phase 8 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: databricks.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 8.D1.1 — Create the folder databricks

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\databricks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\databricks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\databricks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\databricks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\databricks" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — databricks/notebooks/

Exact folder name: notebooks

Exact parent: databricks

Why it is needed now: Phase 8 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `databricks` one level at a time.

4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: notebooks.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 8.D2.1 — Create the folder databricks/notebooks

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\databricks\notebooks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\databricks\notebooks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\databricks\notebooks` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\databricks\notebooks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\databricks\notebooks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\databricks\notebooks" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — databricks/README.md

Exact filename: `README.md`

Filename extension: `.md`. In this project it identifies project documentation and helps the correct application choose syntax

highlighting or execution behavior.

Exact parent folder: databricks relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 8.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in databricks one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: README.md.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled README.md and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 8.F1.1 — Create the file databricks/README.md from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\databricks\README.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\databricks\README.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\README.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\databricks\README.md" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "databricks/README.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under databricks.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `databricks/README.md`.
3. In PowerShell at the repository root, run `Test-Path ".\databricks\README.md"` and require True.
4. Run `git status --short -- "databricks/README.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for databricks/README.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: `databricks/README.md`

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 8.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Databricks learning assets

- `notebooks/00...12`: Git-friendly Databricks source notebooks in execution order.
- `jobs/job_definition.example.yml`: version-controlled dependency map.
- `jobs/job_task_map.md`: human-readable dependency diagram.

## Track A boundary

Your local SQL Server and PostgreSQL instances are not reachable from a cloud workspace simply because they use the name `localhost`. Track A therefore extracts locally, uploads a batch folder to a Unity Catalog volume, processes it in Databricks, and exports Gold files for local download. This manual boundary is intentional, honest, and inexpensive for learning.

## Workspace-specific placeholders

Replace catalog, schema, volume, user email, and batch ID values with resources that your workspace permits. Do not commit tokens or passwords. Read Phase 8 and Phase 14 before creating a Job.
```

Line-by-line explanation

- **Line 1:** `# Databricks learning assets` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** `- `notebooks/00...12`: Git-friendly Databricks source notebooks in execution order.`—is a command option or flag; its exact behavior is defined beside the command; `notebooks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `00` is a nu-

meric literal used by the command or expression;...is a path, relative location, branch-style name, or file argument; verify every character before running it;12is a numeric literal used by the command or expression;:is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Git-friendlyis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Databricksis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;sourceis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;notebooksis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 4:** - \jobs/job_definition.example.yml`: version-controlled dependency map.—is a command option or flag; its exact behavior is defined beside the command;jobsis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;/is a path, relative location, branch-style name, or file argument; verify every character before running it;job_definition.example.ymlis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;:is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;version-controlledis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;dependencyis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;mapis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;.` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 5:** - \jobs/job_task_map.md`: human-readable dependency diagram.—is a command option or flag; its exact behavior is defined beside the command;jobsis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;/is a path, relative location, branch-style name, or file argument; verify every character before running it;job_task_map.md is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;:is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;human-readableis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;dependencyis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;diagramis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;.` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** ## Track A boundary is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** Your local SQL Server and PostgreSQL instances are not reachable from a cloud workspace simply because they use the name \localhost`. Track A therefore extracts locally, uploads a batch folder to a Unity Catalog volume, processes it in Databricks, and exports Gold files for local download. This manual boundary is intentional, honest, and inexpensive for learning.—Youris a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;localis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;SQLis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Serveris a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;andis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;PostgreSQLis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;instancesis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;areis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;notis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;reachableis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;fromis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;a` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 10:** blank line used only to separate logical sections for readability.

- **Line 11:** `## Workspace-specific placeholders` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** Replace `catalog`, `schema`, `volume`, `user email`, and `batch ID` values with resources that your workspace permits. Do not commit tokens or passwords. Read Phase 8 and Phase 14 before creating a Job. — Replace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `catalog` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `volume` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `user` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `email` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

[Back to the Phase 8 lesson index](#)

File introduction — `databricks/notebooks/00_environment_check.py`

Exact filename: `00_environment_check.py`

Filename extension: `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `databricks/notebooks` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 8.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? Yes—review carefully before Git staging.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `databricks/notebooks` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `00_environment_check.py`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `00_environment_check.py` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 8.F2.1 — Create the file `databricks/notebooks/00_environment_check.py` from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\databricks\notebooks\00_environment_check.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\databricks\notebooks\00_environment_check.py" -ErrorAction Stop`
— `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\notebooks\00_environment_check.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\databricks\notebooks\00_environment_check.py" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "databricks/notebooks/00_environment_check.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `databricks/notebooks`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `databricks/notebooks/00_environment_check.py`.
3. In PowerShell at the repository root, run `Test-Path ".\databricks\notebooks\00_environment_check.py"` and require `True`.

4. Run `git status --short -- "databricks/notebooks/00_environment_check.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for databricks/notebooks/00_environment_check.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: databricks/notebooks/00_environment_check.py

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 8.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Databricks notebook source
# Northstar Retail End-to-End Data Platform
# This file is a Git-friendly Databricks source notebook.
# Track A reads files that you manually uploaded to a Unity Catalog volume.
# It never attempts to reach SQL Server or PostgreSQL on localhost.

# COMMAND -----
# MAGIC %md
# MAGIC # 00 – Environment check
# MAGIC **Purpose:** verify Spark, parameters, catalog, schemas, volume access, and Delta read/write.
# MAGIC **Input:** widget values. **Output:** one disposable Delta test table and a task value.

# COMMAND -----
from datetime import datetime, timezone
from pyspark.sql import functions as F

for name, default in {
    "catalog": "northstar_retail",
    "landing_schema": "landing",
    "bronze_schema": "bronze",
    "silver_schema": "silver",
    "gold_schema": "gold",
    "quality_schema": "quality",
    "volume": "track_a_files",
    "batch_id": "MANUAL-REPLACE-ME",
}.items():
    dbutils.widgets.text(name, default)

catalog = dbutils.widgets.get("catalog")
landing_schema = dbutils.widgets.get("landing_schema")
bronze_schema = dbutils.widgets.get("bronze_schema")
silver_schema = dbutils.widgets.get("silver_schema")
gold_schema = dbutils.widgets.get("gold_schema")
quality_schema = dbutils.widgets.get("quality_schema")
volume = dbutils.widgets.get("volume")
batch_id = dbutils.widgets.get("batch_id")

if batch_id == "MANUAL-REPLACE-ME":
    raise ValueError("Set batch_id to the exact extraction folder name, such as EXT-20260815T180000Z.")

# COMMAND -----
# Creating a catalog may require workspace permissions. If this fails, use a catalog you can access.
spark.sql(f"CREATE CATALOG IF NOT EXISTS `{catalog}`")
for schema_name in [landing_schema, bronze_schema, silver_schema, gold_schema, quality_schema]:
    spark.sql(f"CREATE SCHEMA IF NOT EXISTS `{catalog}`.`{schema_name}`")
spark.sql(f"CREATE VOLUME IF NOT EXISTS `{catalog}`.`{landing_schema}`.`{volume}`")

volume_root = f"/Volumes/{catalog}/{landing_schema}/{volume}"
batch_root = f"{volume_root}/{batch_id}"
print({"catalog": catalog, "batch_root": batch_root, "spark_version": spark.version})

# COMMAND -----
# Write and read a tiny Delta table. This proves Spark and the Gold schema work.
test_table = f"`{catalog}`.`{quality_schema}`.environment_check"
test_df = spark.createDataFrame([(batch_id, datetime.now(timezone.utc), "PASS")], [{"batch_id", "checked_at_utc", "status"}])
(test_df.write.format("delta").mode("append").saveAsTable(test_table))
assert spark.table(test_table).filter(F.col("batch_id") == batch_id).count() >= 1

# Verify the uploaded batch folder. It is normal for this to fail before you upload extracts.
try:
    uploaded = dbutils.fs.ls(batch_root)
    print(f"Found {len(uploaded)} objects in {batch_root}")
except Exception as exc:
    raise FileNotFoundError(
        f"Databricks cannot find {batch_root}. Upload the entire local extraction batch folder "
        "to the volume and keep the batch folder name unchanged."
    ) from exc

# COMMAND -----
```



```
dbutils.jobs.taskValues.set(key="environment_status", value="PASS")
dbutils.jobs.taskValues.set(key="batch_root", value=batch_root)
print("ENVIRONMENT CHECK PASSED")
```

Line-by-line explanation

- **Line 1:** `# Databricks notebook source` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `# Northstar Retail End-to-End Data Platform` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** `# This file is a Git-friendly Databricks source notebook.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** `# Track A reads files that you manually uploaded to a Unity Catalog volume.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** `# It never attempts to reach SQL Server or PostgreSQL on localhost.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 9:** `# MAGIC %md` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 10:** `# MAGIC # 00 – Environment check` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 11:** `# MAGIC **Purpose:**` verify Spark, parameters, catalog, schemas, volume access, and Delta read/write. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 12:** `# MAGIC **Input:**` widget values. `**Output:**` one disposable Delta test table and a task value. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 13:** blank line used only to separate logical sections for readability.
- **Line 14:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 15:** `from datetime import datetime, timezone —` `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timezone` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `from pyspark.sql import functions as F —` `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `functions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** blank line used only to separate logical sections for readability.
- **Line 18:** `for name, default in { —` `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** `"catalog": "northstar_retail", —` `"catalog"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"northstar_retail"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 20:** "landing_schema": "landing", — "landing_schema" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "landing" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** "bronze_schema": "bronze", — "bronze_schema" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "bronze" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** "silver_schema": "silver", — "silver_schema" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "silver" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** "gold_schema": "gold", — "gold_schema" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "gold" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** "quality_schema": "quality", — "quality_schema" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "quality" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** "volume": "track_a_files", — "volume" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "track_a_files" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** "batch_id": "MANUAL-REPLACE-ME", — "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "MANUAL-REPLACE-ME" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:** }.items(): — }. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; items is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** dbutils.widgets.text(name, default) — dbutils.widgets.text is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; default is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** blank line used only to separate logical sections for readability.
- **Line 30:** catalog = dbutils.widgets.get("catalog") — catalog is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.widgets.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "catalog" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** landing_schema = dbutils.widgets.get("landing_schema") — landing_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.widgets.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "landing_schema" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 32:** bronze_schema = dbutils.widgets.get("bronze_schema") — bronze_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 33:** `silver_schema = dbutils.widgets.get("silver_schema")` — `silver_schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"silver_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** `gold_schema = dbutils.widgets.get("gold_schema")` — `gold_schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"gold_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** `quality_schema = dbutils.widgets.get("quality_schema")` — `quality_schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quality_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 36:** `volume = dbutils.widgets.get("volume")` — `volume` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"volume"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 37:** `batch_id = dbutils.widgets.get("batch_id")` — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 38:** blank line used only to separate logical sections for readability.
- **Line 39:** `if batch_id == "MANUAL-REPLACE-ME":` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"MANUAL-REPLACE-ME"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 40:** `raise ValueError("Set batch_id to the exact extraction folder name, such as EXT-20260815T180000Z.")` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ValueError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Set batch_id to the exact extraction folder name, such as EXT-20260815T180000Z."` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** blank line used only to separate logical sections for readability.

- **Line 42:** # COMMAND ----- is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 43:** # Creating a catalog may require workspace permissions. If this fails, use a catalog you can access. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 44:** spark.sql(f"CREATE CATALOG IF NOT EXISTS \{catalog}")—spark.sql is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "CREATE CATALOG IF NOT EXISTS \{catalog}" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 45:** for schema_name in [landing_schema, bronze_schema, silver_schema, gold_schema, quality_schema]: — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; schema_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; landing_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; silver_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gold_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 46:** spark.sql(f"CREATE SCHEMA IF NOT EXISTS \{catalog}\.{schema_name}")—spark.sql is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "CREATE SCHEMA IF NOT EXISTS \{catalog}\.{schema_name}" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 47:** spark.sql(f"CREATE VOLUME IF NOT EXISTS \{catalog}\.{landing_schema}\.{volume}")—spark.sql is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "CREATE VOLUME IF NOT EXISTS \{catalog}\.{landing_schema}\.{volume}" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 48:** blank line used only to separate logical sections for readability.
- **Line 49:** volume_root = f"/Volumes/{catalog}/{landing_schema}/{volume}" — volume_root is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "/Volumes/{catalog}/{landing_schema}/{volume}" is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 50:** batch_root = f"{volume_root}/{batch_id}" — batch_root is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{volume_root}/{batch_id}" is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 51:** print({"catalog": catalog, "batch_root": batch_root, "spark_version": spark.version}) — print is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ({ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "catalog" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; catalog is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_root" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_root is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "spark_version" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 52:** blank line used only to separate logical sections for readability.
- **Line 53:** # COMMAND ----- is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 54:** # Write and read a tiny Delta table. This proves Spark and the Gold schema work. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 55:** test_table = f"\{catalog}\.{quality_schema}\.environment_check"-test_table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "\{catalog}\.{quality_schema}\.environment_check" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 56:** test_df = spark.createDataFrame([(batch_id, datetime.now(timezone.utc), "PASS")], [{"batch_id", "checked_at_utc", "status"}]) — test_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spark.createDataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ([(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; datetime.now is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timezone.utc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "PASS" is a quoted literal; the quotation marks preserve spaces or mark text;)], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 57:** (test_df.write.format("delta").mode("append").saveAsTable(test_table)) — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test_df.write.format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "append" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test_table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 58:** assert spark.table(test_table).filter(F.col("batch_id") == batch_id).count() >= 1 — assert is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spark.table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test_table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 59:** blank line used only to separate logical sections for readability.
- **Line 60:** # Verify the uploaded batch folder. It is normal for this to fail before you upload extracts. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 61:** try: — try is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 62:** uploaded = dbutils.fs.ls(batch_root) — uploaded is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.fs.ls is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_root is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 63:** print(f"Found {len(uploaded)} objects in {batch_root}") — print is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Found {len(uploaded)} objects in {batch_root}" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 64:** except Exception as exc: — except is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Exception is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; as is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; exc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 65:** raise FileNotFoundError(— raise is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FileNotFoundError is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 66:** f"Databricks cannot find {batch_root}. Upload the entire local extraction batch folder " — f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Databricks cannot find {batch_root}. Upload the entire local extraction batch folder " is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 67:** "to the volume and keep the batch folder name unchanged." — "to the volume and keep the batch folder name unchanged." is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 68:**) from exc —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; from is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; exc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 69:** blank line used only to separate logical sections for readability.
- **Line 70:** # COMMAND ----- is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 71:** dbutils.jobs.taskValues.set(key="environment_status", value="PASS") — dbutils.jobs.taskValues.set is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "environment_status" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "PASS" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 72:** `dbutils.jobs.taskValues.set(key="batch_root", value=batch_root)` — `dbutils.jobs.taskValues.set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_root"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `value` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_root` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 73:** `print("ENVIRONMENT CHECK PASSED")` — `print` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"ENVIRONMENT CHECK PASSED"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 8 lesson index](#)

What your screen should look like now

- Every Phase 8 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 8.3 completion checklist

☐

`databricks/README.md` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`databricks/notebooks/00_environment_check.py` was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with `Ctrl+S`. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 8.4](#) and recheck the root and branch.

Lesson 8.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Action 8.4.G1 — Inspect the current state without changing it

Application:

Web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Inspect the current state without changing it. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.4.G2 — Create the smallest required object or decision

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Create the smallest required object or decision. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location

with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.4.G3 — Run the phase-specific verification

Application:

Web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run the phase-specific verification. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.4.G4 — Demonstrate one safe failure or alternate-result branch

Application:

Web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Demonstrate one safe failure or alternate-result branch. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

Action 8.4.G5 — Record evidence and prepare the downstream handoff

Application:

Web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Record evidence and prepare the downstream handoff. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 8 lesson index](#)

What your screen should look like now

- The new Phase 8 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 8.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 8.5](#).

Lesson 8.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 8 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Databricks compute and storage

Question:

Which UI and compute path should the learner use when tiers differ?

Options considered:

- Serverless when available.
- Classic compute when allowed.
- Skip execution.

Beginner path selected:

Use the compute option the workspace actually offers and a governed volume or My Files fallback.

Why this path was selected:

It is tier-aware and avoids claiming unavailable features.

Tradeoff:

Free or trial quotas and labels vary.

What a company might do instead:

Enterprises define Unity Catalog objects, external locations, policies, service principals, and job clusters.

How to explain this in an interview:

I verified the workspace tier first and selected an available compute/storage path while keeping object names and validation contracts consistent.

Design decision — Databricks storage options

Question:

Where should Track A batch files be placed when workspace features and permissions vary?

Options considered:

- A governed Unity Catalog volume.
- Workspace or user file storage available to the account.
- An external cloud storage location.

Beginner path selected:

Use a governed volume when permitted; otherwise use the workspace-supported user-file fallback and record the exact path, limitation, and cleanup procedure.

Why this path was selected:

The path remains observable and tier-aware without inventing permissions. A governed volume teaches catalog/schema ownership while the fallback keeps the learning path usable.

Tradeoff:

Fallback storage can have tighter quotas, weaker sharing, or different governance and should not be presented as the enterprise target.

What a company might do instead:

Use external locations and cloud object storage with storage credentials, managed identities, access controls, lifecycle policies, and infrastructure as code.

How to explain this in an interview:

I selected the most governed storage object my workspace actually allowed and parameterized the path so the pipeline contract survived tier differences.

If you see Result A

A writable catalog/schema/storage path and available compute run the environment check. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

The workspace exposes a different tier/UI, lacks permission, or offers a different compute/storage object. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: Web browser, Databricks workspace, VS Code, and File Explorer.

Window/panel: Show workspace/catalog/schema/volume/compute plus successful environment-check output.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase08_configure-databricks-governed-storage-notebooks-and-com_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 8's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 8 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 8.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

Action 8.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a

command option or flag; its exact behavior is defined beside the command; -current is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

Action 8.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

Interpret `git status --short`

- ?? means untracked: Git sees a new path but it is not staged.
- M means modified in the working tree but not staged.
- M means the staged version differs from the last commit.
- MM means a staged version exists and the file has additional unstaged edits.
- A means staged as a new file.
- D means deletion; stop and investigate unless deletion is deliberate.
- R means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 8.5.G4 — Inspect the unstaged diff for databricks/README.md

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "databricks/README.md"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "databricks/README.md"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"databricks/README.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

Action 8.5.G5 — Stage only the reviewed file databricks/README.md

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "databricks/README.md"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "databricks/README.md"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"databricks/README.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 8.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

Action 8.5.G7 — Create the Phase 8 commit

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "feat: configure databricks environment and storage"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "feat: configure databricks environment and storage"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"feat: configure databricks environment and storage"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

Action 8.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

Action 8.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 8 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message `feat: configure databricks environment and storage` and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 8.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 8 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

"Phase 8 created the verified boundary described by **Configure Databricks, governed storage, notebooks, and compute**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed."

Mini quiz

1. Multiple choice: Why does Phase 8 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** PLANNED, CREATED BY USER, and RUN BY USER describe earlier states; only inspected evidence supports verification.

My Phase 8 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 8.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 9 ingests that exact batch into append-only raw Delta tables.

Phase 8 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 9 — Build the Bronze ingestion layer

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: Bronze tables that preserve values and ingestion metadata.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 10 cleans, types, deduplicates, and quarantines Bronze records.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 9 lesson index

1. [Lesson 9.1 — Understand the phase and starting screen](#)
2. [Lesson 9.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 9.3 — Create and understand the phase files](#)
4. [Lesson 9.4 — Build and validate required objects](#)
5. [Lesson 9.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 9.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** Databricks workspace, Catalog Explorer, notebooks, and Jobs-compatible compute.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 9 files remain absent until their creation lesson.

- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 8 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 9 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

Databricks workspace, Catalog Explorer, notebooks, and Jobs-compatible compute. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 9 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 9 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 9 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 9.1 completion checklist

☐

I read why Phase 9 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]



I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 9.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 9 lesson index](#)

Lesson 9.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 9 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

Separate Bronze Delta tables for SQL Server extracts, PostgreSQL extracts, CSV feeds, and nested JSON tracking events. Raw values are preserved while source system, file path, ingestion timestamp, batch ID, record hash, unreadable-record quarantine, row counts, and manifest reconciliation are added.

2. Why it matters

Bronze is the replayable evidence layer. Heavy cleaning here would erase the source form and make later rule disputes difficult. The layer should answer which batch/file produced a row and support safe same-batch reruns.

3. Words to know

Term	Plain-language definition
Raw preservation	Keep source values as received wherever readable.
Append-only	New batches are added rather than editing historical raw rows; same-batch replacement is a controlled exception for idempotency.
Record hash	Hash of the raw record used for trace/change evidence.
Input file name	Physical source path recorded per row.
PERMISSIVE mode	Reader captures malformed data where possible rather than silently dropping.
Quarantine	Unreadable records and error context retained outside trusted table.
Delta table	Transactional lakehouse table.
Schema inference	Reader guesses types; Bronze CSV intentionally keeps strings to defer business casting.
Manifest reconciliation	Expected file rows compared to read/written rows.
Rerun	Repeat a batch/task after failure.
Batch replacement	Delete only target rows with same batch ID before append.
Source system	Origin label such as SQLSERVER_ERP, POSTGRES_DIGITAL, FILE_FEED.

4. What must already be completed

Phase 8 environment check passes and the exact batch folder is readable. Source manifest is present. Notebooks 01–03 are synced/imported.

5. Applications to open

Application	How to open it	Why it is open
Databricks workspace	Open notebooks 01, 02, 03.	Run and inspect Bronze ingestion.
Catalog Explorer	Open Bronze and quality schemas.	Preview tables/quarantine and properties.
VS Code	Open the source notebook and Bronze SQL pattern.	Read audit/idempotency functions.
Local manifest	Open in VS Code/browser.	Provide expected row counts/checksums.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run Get-Location; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
<code>display(spark.table(f"{catalog}. {bronze_schema}.erp_customers").filter(F.col("_batch_id") == batch_id).limit(10))</code>	Shows raw customer rows and audit columns for one batch.	You must visually verify preservation and metadata before Silver.
<code>SELECT _batch_id, COUNT(*) AS rows, COUNT(DISTINCT _record_hash) AS distinct_hashes</code>	Counts Bronze orders by batch and hash.	Detects duplicate batch or duplicate raw rows.
<code>manifest_text = "".join(r.value for r in spark.read.text(f"{batch_root}/ batch_manifest.json").collect())</code>	Reads and parses local extraction manifest from volume.	Bronze must compare against source evidence, not only itself.
<code>SELECT return_date, COUNT(*) AS rows</code>	Shows raw mixed date strings.	Proves Bronze did not force one format.
<code>SELECT * FROM north- star_retail.bronze.file_read_quarantine ORDER BY quarantined_at_utc DESC LIMIT 20;</code>	Inspects unreadable file error evidence.	A failed record/file must remain diagnosable.

Bronze contract

Do: preserve raw values, retain nested structures, add technical metadata, identify batch/file/source, count rows, and fail visibly on unreadable inputs.

Do not: title-case names, validate emails as trusted, parse every date into one business meaning, choose duplicate winners, join sources, calculate profit, or resolve dimension keys. Those are Silver/Gold responsibilities.

Manifest formula

For each readable file: $\text{manifest expected rows} = \text{Bronze rows for that file and batch}$. For malformed records under permissive parsing: $\text{expected rows} = \text{accepted raw rows} + \text{row-level unreadable quarantine}$, with exact reader behavior documented.

MY TASK

Add manifest expected-row validation to each Bronze notebook, demonstrate same-batch idempotency, and document one unreadable-file quarantine test without changing the valid source batch.

STARTER SKELETON

```
def expected_rows(manifest: dict, source_system: str, filename: str) -> int:
    # TODO(USER): find exactly one matching manifest entry.
    matches = [
        item for item in manifest["files"]
        if item["source_system"] == _____ and item["file"].endswith(_____)
    ]
    if len(matches) != 1:
        raise ValueError(_____)
    return int(matches[0]["rows"])

# TODO(USER): compare expected, input DataFrame, and target batch counts.
assert _____ == _____ == _____
```

Acceptance criteria: Every Bronze subject has audit columns, current-batch count matches manifest, same-batch rerun is stable, controlled dirty values remain raw, and unreadable input creates evidence and failure.

HINT 1 — CONCEPT

Match manifest entries by source system plus terminal filename, not an unqualified substring.

HINT 2 — PSEUDOCODE

Compute expected/input/target once and include all values in an assertion message.

HINT 3 — NEAR-COMPLETE PATTERN

Follow `append_once` and add manifest lookup before the write; do not clean values.

13. Expected files or output

- 13 ERP Bronze tables, 6 digital Bronze tables, 4 file Bronze tables.
- Audit fields on all.
- Manifest reconciliation output.
- Stable same-batch count.
- Optional practice quarantine record for unreadable copied input.

14. How to verify

- Batch and file metadata trace every row.
- All source manifest entries have a Bronze outcome.
- Raw issue examples remain unchanged.
- Nested JSON events are visible as nested data.
- No duplicate batch count after rerun.

15. Automated tests

- Manifest-to-Bronze count checks.
- Same-batch before/after count equality.
- Required audit columns/not-null tests.
- Unreadable-path task failure and quarantine test.
- Record-hash stability for unchanged same-batch input.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
Path/file not found	Upload structure or widget mismatch.	Print batch_root; ls parent/subfolder; compare manifest.	Correct path/upload; do not hard-code random path.	Reader counts expected file.
Schema inference changed types	Reader options or source file evolved.	printSchema and compare previous batch.	Keep CSV Bronze strings; record additive drift; explicit Silver casts.	Schema drift test documented.
Target doubled on rerun	Batch deletion filter not applied or batch ID changed.	Group target by batch; print parameter.	Use exact stable batch ID and replace only that batch.	Count stable.
Manifest count differs	Wrong file, header count, parsing loss, or manifest method.	Compare source entry/path/input/quarantine/target.	Correct counting/parsing and document malformed behavior; do not alter expected silently.	Equation balances.
JSON corrupt record	Malformed nested feed.	Read with permissive/corrupt-record column and inspect source line.	Quarantine error/record and fail required gate per policy.	No silent loss.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why preserve malformed business values in Bronze?
2. What makes a Bronze rerun safe?
3. How do source manifest and target count differ as evidence?
4. What belongs in unreadable-record quarantine?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. Which audit field connects all rows to one run?
2. Should Bronze title-case customer names? Why?
3. What equation reconciles a file with quarantined unreadable rows?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture one Bronze table with raw defect plus audit columns, manifest reconciliation output, and same-batch count before/after rerun.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
feat: ingest manifest-reconciled bronze delta tables
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

All source types ingested — [STATUS]

☐

Audit columns verified — [STATUS]

☐

Manifest reconciliation added — [STATUS]

☐

Raw preservation checked — [STATUS]

☐

Nested JSON checked — [STATUS]

☐

Same-batch rerun stable — [STATUS]

☐

Unreadable failure/quarantine tested — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-9. The quiz answer is in [the answer-key section immediately after the quiz in this workbook#phase-9] (the answer-key section immediately after the quiz in this workbook#phase-9).

Next phase: Phase 10 converts Bronze evidence into typed, standardized, conformed Silver tables and quarantines rule failures.

Required action cards

Action 9.2.1 — Open 01_bronze_sqlserver. Read widget configuration, with_audit_columns, append_once, explicit table list

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open 01_bronze_sqlserver. Read widget configuration, with_audit_columns, append_once, explicit table list. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open 01_bronze_sqlserver. Read widget configuration, with_audit_columns, append_once, explicit table list, and reconciliation assertion.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 1

Action

Open `01_bronze_sqlserver`. Read widget configuration, `with_audit_columns`, `append_once`, explicit table list, and reconciliation assertion.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Action 9.2.2 — Confirm every Job/notebook parameter matches Phase 8. Run SQL Server Bronze. Inspect results table and one row

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Confirm every Job/notebook parameter matches Phase 8. Run SQL Server Bronze. Inspect results table and one row. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Confirm every Job/notebook parameter matches Phase 8. Run SQL Server Bronze. Inspect results table and one row table schema.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 2****Action**

Confirm every Job/notebook parameter matches Phase 8. Run SQL Server Bronze. Inspect results table and one row table schema.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Action 9.2.3 — Open 02_bronze_postgres, run, and inspect timestampz/event source values without standardizing them yet.

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open 02_bronze_postgres, run, and inspect timestampz/event source values without standardizing them yet. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open 02_bronze_postgres, run, and inspect timestampz/event source values without standardizing them yet.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL psql, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 3

Action

Open 02_bronze_postgres, run, and inspect timestampz/event source values without standardizing them yet.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Action 9.2.4 — Open 03_bronze_files. CSV is read with header and permissive mode; JSONL is parsed as nested arrays/structs.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open 03_bronze_files. CSV is read with header and permissive mode; JSONL is parsed as nested arrays/structs. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open 03_bronze_files. CSV is read with header and permissive mode; JSONL is parsed as nested arrays/structs. Run it.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 4****Action**

Open 03_bronze_files. CSV is read with header and permissive mode; JSONL is parsed as nested arrays/structs. Run it.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Action 9.2.5 — In Catalog Explorer, preview `bronze.erp_customers`, `bronze.digital_web_sessions`, `bronze.file_returns`, and

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

In Catalog Explorer, preview `bronze.erp_customers`, `bronze.digital_web_sessions`, `bronze.file_returns`, and. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: In Catalog Explorer, preview `bronze.erp_customers`, `bronze.digital_web_sessions`, `bronze.file_returns`, and `bronze.file_shipment_tracking_events`.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 5

Action

In Catalog Explorer, preview `bronze.erp_customers`, `bronze.digital_web_sessions`, `bronze.file_returns`, and `bronze.file_shipment_tracking_events`.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Action 9.2.6 — Verify audit columns exist on every Bronze table and `_batch_id` equals the exact batch.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Verify audit columns exist on every Bronze table and `_batch_id` equals the exact batch. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Verify audit columns exist on every Bronze table and `_batch_id` equals the exact batch.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.

6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 6

Action

Verify audit columns exist on every Bronze table and `_batch_id` equals the exact batch.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Action 9.2.7 — Compare target counts to manifest entries. The Bronze notebook reference pattern reconciles DataFrame to target

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Compare target counts to manifest entries. The Bronze notebook reference pattern reconciles DataFrame to target. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Compare target counts to manifest entries. The Bronze notebook reference pattern reconciles DataFrame to target; complete the user task to parse the manifest and assert expected file rows too.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 7

Action

Compare target counts to manifest entries. The Bronze notebook reference pattern reconciles DataFrame to target; complete the user task to parse the manifest and assert expected file rows too.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Action 9.2.8 — Inspect controlled dirty values in Bronze: malformed email, whitespace/case, impossible quantity, mixed return

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Inspect controlled dirty values in Bronze: malformed email, whitespace/case, impossible quantity, mixed return. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Inspect controlled dirty values in Bronze: malformed email, whitespace/case, impossible quantity, mixed return dates, out-of-order event, currency case. They should not be silently corrected.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 8****Action**

Inspect controlled dirty values in Bronze: malformed email, whitespace/case, impossible quantity, mixed return dates, out-of-order event, currency case. They should not be silently corrected.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Action 9.2.9 — Run one same-batch rerun. The notebook deletes only that batch's rows and appends once. Count before/after sho

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one same-batch rerun. The notebook deletes only that batch's rows and appends once. Count before/after sho. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run one same-batch rerun. The notebook deletes only that batch's rows and appends once. Count before/after should remain unchanged.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 9****Action**

Run one same-batch rerun. The notebook deletes only that batch's rows and appends once. Count before/after should remain unchanged.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Action 9.2.10 — Simulate an unreadable file only in a disposable copied batch. Verify file_read_quarantine receives context

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Simulate an unreadable file only in a disposable copied batch. Verify file_read_quarantine receives context . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Simulate an unreadable file only in a disposable copied batch. Verify file_read_quarantine receives context and task fails; restore the valid batch rather than bypassing failure.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 10

Action

Simulate an unreadable file only in a disposable copied batch. Verify `file_read_quarantine` receives context and task fails; restore the valid batch rather than bypassing failure.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Action 9.2.11 — Record table count, batch ID, source path, and one raw example in the journal.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Record table count, batch ID, source path, and one raw example in the journal. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Record table count, batch ID, source path, and one raw example in the journal.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 11****Action**

Record table count, batch ID, source path, and one raw example in the journal.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location

with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Action 9.2.C1 — Databricks notebook cell

Application:

Databricks workspace in a web browser

Window or panel:

Databricks notebook cell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Python instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Python):

```
display(spark.table(f"{catalog}.{bronze_schema}.erp_customers").filter(F.col("_batch_id") == batch_id).limit(10))
```

Where to enter it:

Databricks notebook cell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `display(spark.table(f"{catalog}.{bronze_schema}.erp_customers").filter(F.col("_batch_id") == batch_id).limit(10))` — `display` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{bronze_schema}.erp_customers"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `limit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `10` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Action 9.2.C2 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained SQL instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT _batch_id, COUNT(*) AS rows, COUNT(DISTINCT _record_hash) AS distinct_hashes
FROM northstar_retail.bronze.erp_orders
GROUP BY _batch_id
ORDER BY _batch_id;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT _batch_id, COUNT(*) AS rows, COUNT(DISTINCT _record_hash) AS distinct_hashes` — `SELECT` requests data without changing it unless combined with a modifying statement; `_batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DISTINCT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `_record_hash` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 2:** `FROM northstar_retail.bronze.erp_orders` — `FROM` identifies the source table or query; `northstar_retail.bronze.erp_orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `GROUP BY _batch_id` — `GROUP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `_batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `ORDER BY _batch_id;` — `ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `_batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Action 9.2.C3 — Databricks notebook cell

Application:

Databricks workspace in a web browser

Window or panel:

Databricks notebook cell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Python instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Python):

```
manifest_text = "".join(r.value for r in spark.read.text(f"{batch_root}/batch_manifest.json").collect())
import json
manifest = json.loads(manifest_text)
print(manifest["batch_id"], len(manifest["files"]))
```

Where to enter it:

Databricks notebook cell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `manifest_text = "".join(r.value for r in spark.read.text(f"{batch_root}/batch_manifest.json").collect())` — `manifest_text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `" "` is a quoted literal; the quotation marks preserve spaces or mark text; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `join` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `r.value` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `r` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 2:** `import json` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `json` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `manifest = json.loads(manifest_text)` — `manifest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `json.loads` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manifest_text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `print(manifest["batch_id"], len(manifest["files"]))` — `print` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manifest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, ar-

gument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mani - fest is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "files" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Action 9.2.C4 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT return_date, COUNT(*) AS rows
FROM northstar_retail.bronze.file_returns
WHERE _batch_id = 'EXT-YYYYMMDDTHMMSSZ'
GROUP BY return_date
ORDER BY return_date
LIMIT 20;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT return_date, COUNT(*) AS rows` — `SELECT` requests data without changing it unless combined with a modifying statement; `return_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `FROM northstar_retail.bronze.file_returns` — `FROM` identifies the source table or query; `northstar_retail.bronze.file_returns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `WHERE _batch_id = 'EXT-YYYYMMDDTHMMSSZ'` — `WHERE` filters rows to those meeting a condition; `_batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'EXT-YYYYMMDDTHMMSSZ'` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 4:** `GROUP BY return_date` — `GROUP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `return_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `ORDER BY return_date` — `ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `return_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `LIMIT 20;` — `LIMIT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `20` is a numeric literal used by the command or expression

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Action 9.2.C5 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT * FROM northstar_retail.bronze.file_read_quarantine ORDER BY quarantined_at_utc DESC LIMIT 20;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT * FROM northstar_retail.bronze.file_read_quarantine ORDER BY quarantined_at_utc DESC LIMIT 20;` — `SELECT` requests data without changing it unless combined with a modifying statement; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `northstar_retail.bronze.file_read_quarantine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quarantined_at_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DESC` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `LIMIT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `20` is a numeric literal used by the command or expression

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 9.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 9 starting-screen checklist and re-verify the active context.

Lesson 9.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 9 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — databricks/

Exact folder name: databricks

Exact parent: repository root

Why it is needed now: Phase 9 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: databricks.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 9.D1.1 — Create the folder databricks

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\databricks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\databricks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks"` is a path, relat-

ive location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\databricks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\databricks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\databricks" -PathType Container`.
2. Require the output `True` before creating a child file.

3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/

Exact folder name: sql

Exact parent: repository root

Why it is needed now: Phase 9 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: sql.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 9.D2.1 — Create the folder sql

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\sql" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — databricks/notebooks/

Exact folder name: notebooks

Exact parent: databricks

Why it is needed now: Phase 9 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `databricks` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `notebooks`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 9.D3.1 — Create the folder `databricks/notebooks`

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\databricks\notebooks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\databricks\notebooks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\notebooks"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\databricks\notebooks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\databricks\notebooks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\databricks\notebooks" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/databricks/

Exact folder name: databricks

Exact parent: sql

Why it is needed now: Phase 9 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `sql` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `databricks`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method**Action 9.D4.1 — Create the folder sql/databricks**

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\databricks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\databricks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\sql\databricks` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\databricks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\databricks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\databricks" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/databricks/bronze/

Exact folder name: bronze

Exact parent: sql/databricks

Why it is needed now: Phase 9 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path sql/databricks one level at a time.

4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: bronze.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 9.D5.1 — Create the folder sql/databricks/bronze

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\databricks\bronze" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\sql\databricks\bronze" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\sql\databricks\bronze" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\databricks\bronze" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\databricks\bronze" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\databricks\bronze" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — databricks/notebooks/01_bronze_sqlserver.py

Exact filename: `01_bronze_sqlserver.py`

Filename extension: `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax

highlighting or execution behavior.

Exact parent folder: databricks/notebooks relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 9.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in databricks/notebooks one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 01_bronze_sqlserver.py.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 01_bronze_sqlserver.py and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 9.F1.1 — Create the file databricks/notebooks/01_bronze_sqlserver.py from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\databricks\notebooks\01_bronze_sqlserver.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\databricks\notebooks\01_bronze_sqlserver.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\notebooks\01_bronze_sqlserver.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\databricks\notebooks\01_bronze_sqlserver.py" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "databricks/notebooks/01_bronze_sqlserver.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `databricks/notebooks`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `databricks/notebooks/01_bronze_sqlserver.py`.
3. In PowerShell at the repository root, run `Test-Path ".\databricks\notebooks\01_bronze_sqlserver.py"` and require True.
4. Run `git status --short -- "databricks/notebooks/01_bronze_sqlserver.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `databricks/notebooks/01_bronze_sqlserver.py`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `databricks/notebooks/01_bronze_sqlserver.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 9.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Databricks notebook source
# Northstar Retail End-to-End Data Platform
# This file is a Git-friendly Databricks source notebook.
# Track A reads files that you manually uploaded to a Unity Catalog volume.
# It never attempts to reach SQL Server or PostgreSQL on localhost.

# COMMAND -----
# MAGIC %md
# MAGIC # 01 – Bronze SQL Server extracts
# MAGIC Reads Track A Parquet extracts from the uploaded volume; SQL Server itself remains local.

# COMMAND -----

from __future__ import annotations
from datetime import datetime, timezone
from functools import reduce
from pyspark.sql import DataFrame
from pyspark.sql import functions as F

for name, default in {
    "catalog": "northstar_retail", "landing_schema": "landing", "bronze_schema": "bronze",
    "volume": "track_a_files", "batch_id": "MANUAL-REPLACE-ME"
}.items():
    dbutils.widgets.text(name, default)
```

```

catalog = dbutils.widgets.get("catalog")
landing_schema = dbutils.widgets.get("landing_schema")
bronze_schema = dbutils.widgets.get("bronze_schema")
volume = dbutils.widgets.get("volume")
batch_id = dbutils.widgets.get("batch_id")
if batch_id == "MANUAL-REPLACE-ME":
    raise ValueError("Pass the extraction batch_id from the workflow.")
batch_root = f"/Volumes/{catalog}/{landing_schema}/{volume}/{batch_id}"

def with_audit_columns(df: DataFrame, source_system: str) -> DataFrame:
    """Preserve raw columns and add repeatable ingestion metadata."""
    payload = F.to_json(F.struct(*[F.col(c).cast("string").alias(c) for c in df.columns]))
    return (
        df.withColumn("_source_system", F.lit(source_system))
        .withColumn("_source_file", F.input_file_name())
        .withColumn("_ingested_at_utc", F.current_timestamp())
        .withColumn("_batch_id", F.lit(batch_id))
        .withColumn("_record_hash", F.sha2(payload, 256))
    )

def append_once(df: DataFrame, table_name: str) -> None:
    """Append this batch once; an exact rerun replaces the same batch safely."""
    full_name = f`${catalog}`.`{bronze_schema}`.`{table_name}`
    if spark.catalog.tableExists(full_name.replace("`", "")):
        spark.sql(f"DELETE FROM {full_name} WHERE _batch_id = '{batch_id}'")
    (df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full_name))
    loaded = spark.table(full_name).filter(F.col("_batch_id") == batch_id).count()
    if loaded != df.count():
        raise AssertionError(f"Bronze reconciliation failed for {table_name}: input={df.count()} target={loaded}")

# COMMAND -----
tables = [
    "customers", "addresses", "products", "product_categories", "suppliers", "stores",
    "employees", "orders", "order_items", "payments", "shipments",
    "inventory_transactions", "inventory_snapshots",
]
results = []
for table in tables:
    path = f"{batch_root}/sqlserver/{table}.parquet"
    raw = spark.read.format("parquet").load(path)
    audited = with_audit_columns(raw, "SQLSERVER_ERP")
    append_once(audited, f"erp_{table}")
    results.append((table, raw.count()))
display(spark.createDataFrame(results, ["table_name", "rows_loaded"]))
dbutils.jobs.taskValues.set(key="bronze_sqlserver_status", value="PASS")

```

Line-by-line explanation

- **Line 1:** # Databricks notebook source is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** # Northstar Retail End-to-End Data Platform is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** # This file is a Git-friendly Databricks source notebook. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** # Track A reads files that you manually uploaded to a Unity Catalog volume. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** # It never attempts to reach SQL Server or PostgreSQL on localhost. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** # COMMAND ----- is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 9:** # MAGIC %md is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 10:** # MAGIC # 01 – Bronze SQL Server extracts is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 11:** # MAGIC Reads Track A Parquet extracts from the uploaded volume; SQL Server itself remains local. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** # COMMAND ----- is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 14:** blank line used only to separate logical sections for readability.

- **Line 15:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `from datetime import datetime, timezone` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timezone` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** `from functools import reduce` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `functools` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `reduce` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** `from pyspark.sql import DataFrame` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** `from pyspark.sql import functions as F` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `functions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** blank line used only to separate logical sections for readability.
- **Line 21:** `for name, default in {` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** `"catalog": "northstar_retail", "landing_schema": "landing", "bronze_schema": "bronze",` — `"catalog"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"northstar_retail"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"landing_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"landing"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** `"volume": "track_a_files", "batch_id": "MANUAL-REPLACE-ME"` — `"volume"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"track_a_files"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"MANUAL-REPLACE-ME"` is a quoted literal; the quotation marks preserve spaces or mark text

- **Line 24:** `}.items():` — `}.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** `dbutils.widgets.text(name, default)` — `dbutils.widgets.text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** blank line used only to separate logical sections for readability.
- **Line 27:** `catalog = dbutils.widgets.get("catalog")` — `catalog` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"catalog"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** `landing_schema = dbutils.widgets.get("landing_schema")` — `landing_schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"landing_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** `bronze_schema = dbutils.widgets.get("bronze_schema")` — `bronze_schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 30:** `volume = dbutils.widgets.get("volume")` — `volume` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"volume"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** `batch_id = dbutils.widgets.get("batch_id")` — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 32:** `if batch_id == "MANUAL-REPLACE-ME":` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"MANUAL-REPLACE-ME"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** `raise ValueError("Pass the extraction batch_id from the workflow.")` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ValueError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Pass the extraction batch_id from the workflow." is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 34:** batch_root = f"/Volumes/{catalog}/{landing_schema}/{volume}/{batch_id}" — batch_root is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "/Volumes/{catalog}/{landing_schema}/{volume}/{batch_id}" is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 35:** blank line used only to separate logical sections for readability.
- **Line 36:** blank line used only to separate logical sections for readability.
- **Line 37:** def with_audit_columns(df: DataFrame, source_system: str) -> DataFrame: — def is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; with_audit_columns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source_system is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -> is a command option or flag; its exact behavior is defined beside the command; remaining punctuation and identifiers continue the same statement.
- **Line 38:** """Preserve raw columns and add repeatable ingestion metadata.""" — "" is a quoted literal; the quotation marks preserve spaces or mark text; "Preserve raw columns and add repeatable ingestion metadata." is a quoted literal; the quotation marks preserve spaces or mark text; "" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 39:** payload = F.to_json(F.struct(*[F.col(c).cast("string").alias(c) for c in df.columns])) — payload is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.to_json is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.struct is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [* is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; c is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 40:** return (— return is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** df.withColumn("_source_system", F.lit(source_system)) — df.withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_source_system" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source_system is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 42:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 43:** `.withColumn("_ingested_at_utc", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at_utc"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 44:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 45:** `.withColumn("_record_hash", F.sha2(payload, 256))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_record_hash"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.sha2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `payload` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `256` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 47:** blank line used only to separate logical sections for readability.
- **Line 48:** blank line used only to separate logical sections for readability.
- **Line 49:** `def append_once(df: DataFrame, table_name: str) -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `append_once` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; remaining punctuation and identifiers continue the same statement.
- **Line 50:** `"""Append this batch once; an exact rerun replaces the same batch safely."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `Append this batch once; an exact rerun replaces the`

same batch safely." is a quoted literal; the quotation marks preserve spaces or mark text; "" is a quoted literal; the quotation marks preserve spaces or mark text

- Line 51:** `full_name = f"\{catalog}\.{bronze_schema}\.{table_name}"`—`full_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\{catalog}\.{bronze_schema}\.{table_name}"` is a quoted literal; the quotation marks preserve spaces or mark text
- Line 52:** `if spark.catalog.tableExists(full_name.replace("\", "")):`—`if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.catalog.tableExists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full_name.replace` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `""` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `""` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 53:** `spark.sql(f"DELETE FROM {full_name} WHERE _batch_id = '{batch_id}')` — `spark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"DELETE FROM {full_name} WHERE _batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 54:** `(df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full_name))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"mergeSchema"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` remaining punctuation and identifiers continue the same statement.
- Line 55:** `loaded = spark.table(full_name).filter(F.col("_batch_id") == batch_id).count()` — `loaded` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 56:** `if loaded != df.count():` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `loaded` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df.count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

tifier, or literal whose project-specific role is described in the surrounding explanation; () : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 57:** `raise AssertionError(f"Bronze reconciliation failed for {table_name}: input={df.count()} target={loaded}")` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AssertionError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Bronze reconciliation failed for {table_name}: input={df.count()} target={loaded}"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 58:** blank line used only to separate logical sections for readability.
- **Line 59:** blank line used only to separate logical sections for readability.
- **Line 60:** `# COMMAND` ----- is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 61:** `tables = [` — `tables` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 62:** `"customers", "addresses", "products", "product_categories", "suppliers", "stores",` — `"customers"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"addresses"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"products"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"product_categories"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"suppliers"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"stores"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 63:** `"employees", "orders", "order_items", "payments", "shipments",` — `"employees"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"orders"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_items"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"payments"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"shipments"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 64:** `"inventory_transactions", "inventory_snapshots",` — `"inventory_transactions"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"inventory_snapshots"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 65:** `] — ]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 66:** `results = []` — `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 67:** `for table in tables:` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tables` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 68:** `path = f"{batch_root}/sqlserver/{table}.parquet"` — `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{batch_root}/sqlserver/{table}.parquet"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 69:** `raw = spark.read.format("parquet").load(path)` — `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"parquet"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `load` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 70:** `audited = with_audit_columns(raw, "SQLSERVER_ERP")` — `audited` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `with_audit_columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SQLSERVER_ERP"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 71:** `append_once(audited, f"erp_{table}")` — `append_once` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `audited` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"erp_{table}"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 72:** `results.append((table, raw.count()))` — `results.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `((` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw.count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 73:** `display(spark.createDataFrame(results, ["table_name", "rows_loaded"]))` — `display` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.createDataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"table_name"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"rows_loaded"` is a quoted literal; the quotation marks preserve spaces or mark text; `])` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 74:** `dbutils.jobs.taskValues.set(key="bronze_sqlserver_status", value="PASS")` — `dbutils.jobs.taskValues.set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

role is described in the surrounding explanation; key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "bronze_sqlserver_status" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "PASS" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 9 lesson index](#)

File introduction — databricks/notebooks/02_bronze_postgres.py

Exact filename: 02_bronze_postgres.py

Filename extension: .py. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: databricks/notebooks relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 9.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in databricks/notebooks one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 02_bronze_postgres.py.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 02_bronze_postgres.py and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 9.F2.1 — Create the file databricks/notebooks/02_bronze_postgres.py from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\databricks\notebooks\02_bronze_postgres.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\databricks\notebooks\02_bronze_postgres.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\notebooks\02_bronze_postgres.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\databricks\notebooks\02_bronze_postgres.py" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "databricks/notebooks/02_bronze_postgres.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `databricks/notebooks`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `databricks/notebooks/02_bronze_postgres.py`.
3. In PowerShell at the repository root, run `Test-Path ".\databricks\notebooks\02_bronze_postgres.py"` and require True.
4. Run `git status --short -- "databricks/notebooks/02_bronze_postgres.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for databricks/notebooks/02_bronze_postgres.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `databricks/notebooks/02_bronze_postgres.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 9.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Databricks notebook source
# Northstar Retail End-to-End Data Platform
```

```

# This file is a Git-friendly Databricks source notebook.
# Track A reads files that you manually uploaded to a Unity Catalog volume.
# It never attempts to reach SQL Server or PostgreSQL on localhost.

# COMMAND -----
# MAGIC %md
# MAGIC # 02 – Bronze PostgreSQL extracts
# MAGIC Reads Track A Parquet extracts from the uploaded volume; PostgreSQL itself remains local.

# COMMAND -----

from __future__ import annotations
from datetime import datetime, timezone
from functools import reduce
from pyspark.sql import DataFrame
from pyspark.sql import functions as F

for name, default in {
    "catalog": "northstar_retail", "landing_schema": "landing", "bronze_schema": "bronze",
    "volume": "track_a_files", "batch_id": "MANUAL-REPLACE-ME"
}.items():
    dbutils.widgets.text(name, default)

catalog = dbutils.widgets.get("catalog")
landing_schema = dbutils.widgets.get("landing_schema")
bronze_schema = dbutils.widgets.get("bronze_schema")
volume = dbutils.widgets.get("volume")
batch_id = dbutils.widgets.get("batch_id")
if batch_id == "MANUAL-REPLACE-ME":
    raise ValueError("Pass the extraction batch_id from the workflow.")
batch_root = f"/Volumes/{catalog}/{landing_schema}/{volume}/{batch_id}"

def with_audit_columns(df: DataFrame, source_system: str) -> DataFrame:
    """Preserve raw columns and add repeatable ingestion metadata."""
    payload = F.to_json(F.struct(*[F.col(c).cast("string").alias(c) for c in df.columns]))
    return (
        df.withColumn("_source_system", F.lit(source_system))
        .withColumn("_source_file", F.input_file_name())
        .withColumn("_ingested_at_utc", F.current_timestamp())
        .withColumn("_batch_id", F.lit(batch_id))
        .withColumn("_record_hash", F.sha2(payload, 256))
    )

def append_once(df: DataFrame, table_name: str) -> None:
    """Append this batch once; an exact rerun replaces the same batch safely."""
    full_name = f"`{catalog}`.`{bronze_schema}`.`{table_name}`"
    if spark.catalog.tableExists(full_name.replace("`", "")):
        spark.sql(f"DELETE FROM {full_name} WHERE _batch_id = '{batch_id}'")
    (df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full_name))
    loaded = spark.table(full_name).filter(F.col("_batch_id") == batch_id).count()
    if loaded != df.count():
        raise AssertionError(f"Bronze reconciliation failed for {table_name}: input={df.count()} target={loaded}")

# COMMAND -----
tables = ["web_users", "web_sessions", "web_events", "campaigns", "campaign_touchpoints", "marketing_spend"]
results = []
for table in tables:
    path = f"{batch_root}/postgres/{table}.parquet"
    raw = spark.read.format("parquet").load(path)
    audited = with_audit_columns(raw, "POSTGRES_DIGITAL")
    append_once(audited, f"digital_{table}")
    results.append((table, raw.count()))
display(spark.createDataFrame(results, ["table_name", "rows_loaded"]))
dbutils.jobs.taskValues.set(key="bronze_postgres_status", value="PASS")

```

Line-by-line explanation

- **Line 1:** # Databricks notebook source is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** # Northstar Retail End-to-End Data Platform is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** # This file is a Git-friendly Databricks source notebook. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** # Track A reads files that you manually uploaded to a Unity Catalog volume. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** # It never attempts to reach SQL Server or PostgreSQL on localhost. is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 9:** `# MAGIC %md` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 10:** `# MAGIC # 02 – Bronze PostgreSQL extracts` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 11:** `# MAGIC Reads Track A Parquet extracts from the uploaded volume; PostgreSQL itself remains local.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 14:** blank line used only to separate logical sections for readability.
- **Line 15:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `from datetime import datetime, timezone` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timezone` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** `from functools import reduce` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `functools` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `reduce` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** `from pyspark.sql import DataFrame` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** `from pyspark.sql import functions as F` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `functions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** blank line used only to separate logical sections for readability.
- **Line 21:** `for name, default in {` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** `"catalog": "northstar_retail", "landing_schema": "landing", "bronze_schema": "bronze",` — `"catalog"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"northstar_retail"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"landing_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"landing"` is a quoted literal; the quotation marks preserve spaces or mark

text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "bronze_schema" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "bronze" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 23:** "volume": "track_a_files", "batch_id": "MANUAL-REPLACE-ME" — "volume" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "track_a_files" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "MANUAL-REPLACE-ME" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 24:** }.items(): — }. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; items is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (): is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** dbutils.widgets.text(name, default) — dbutils.widgets.text is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; default is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** blank line used only to separate logical sections for readability.
- **Line 27:** catalog = dbutils.widgets.get("catalog") — catalog is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.widgets.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "catalog" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** landing_schema = dbutils.widgets.get("landing_schema") — landing_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.widgets.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "landing_schema" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** bronze_schema = dbutils.widgets.get("bronze_schema") — bronze_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.widgets.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "bronze_schema" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 30:** volume = dbutils.widgets.get("volume") — volume is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.widgets.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "volume" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** batch_id = dbutils.widgets.get("batch_id") — batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.widgets.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 32:** `if batch_id == "MANUAL-REPLACE-ME":` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"MANUAL-REPLACE-ME"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** `raise ValueError("Pass the extraction batch_id from the workflow.")` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ValueError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Pass the extraction batch_id from the workflow."` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** `batch_root = f"/Volumes/{catalog}/{landing_schema}/{volume}/{batch_id}"` — `batch_root` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/{catalog}/{landing_schema}/{volume}/{batch_id}"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 35:** blank line used only to separate logical sections for readability.
- **Line 36:** blank line used only to separate logical sections for readability.
- **Line 37:** `def with_audit_columns(df: DataFrame, source_system: str) -> DataFrame:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `with_audit_columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; remaining punctuation and identifiers continue the same statement.
- **Line 38:** `"""Preserve raw columns and add repeatable ingestion metadata."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `"Preserve raw columns and add repeatable ingestion metadata."` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 39:** `payload = F.to_json(F.struct(*[F.col(c).cast("string").alias(c) for c in df.columns]))` — `payload` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.to_json` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.struct` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `c` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 40:** `return (` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** `df.withColumn("_source_system", F.lit(source_system))` — `df.withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_system"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 43:** `.withColumn("_ingested_at_utc", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at_utc"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 44:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 45:** `.withColumn("_record_hash", F.sha2(payload, 256))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_record_hash"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.sha2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `payload` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `256` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 47:** blank line used only to separate logical sections for readability.
- **Line 48:** blank line used only to separate logical sections for readability.
- **Line 49:** `def append_once(df: DataFrame, table_name: str) -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `append_once` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -> is a command option or flag; its exact behavior is defined beside the command; remaining punctuation and identifiers continue the same statement.

- **Line 50:** `""Append this batch once; an exact rerun replaces the same batch safely.""` — "" is a quoted literal; the quotation marks preserve spaces or mark text; "Append this batch once; an exact rerun replaces the same batch safely." is a quoted literal; the quotation marks preserve spaces or mark text; "" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 51:** `full_name = f"\{catalog}\.{bronze_schema}\.{table_name}"`—full_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;""{catalog}\.{bronze_schema}\.{table_name}" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 52:** `if spark.catalog.tableExists(full_name.replace("\", "")):`—if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;spark.catalog.tableExists is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;full_name.replace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;"" is a quoted literal; the quotation marks preserve spaces or mark text;; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;"" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 53:** `spark.sql(f"DELETE FROM {full_name} WHERE _batch_id = '{batch_id}'")` — spark.sql is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "DELETE FROM {full_name} WHERE _batch_id = '{batch_id}'" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 54:** `(df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full_name))` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; df.write.format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "append" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "mergeSchema" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 55:** `loaded = spark.table(full_name).filter(F.col("_batch_id") == batch_id).count()` — loaded is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spark.table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; full_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; filter is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 56:** `if loaded != df.count():` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `loaded` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df.count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `():` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 57:** `raise AssertionError(f"Bronze reconciliation failed for {table_name}: input={df.count()} target={loaded}")` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AssertionError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Bronze reconciliation failed for {table_name}: input={df.count()} target={loaded}"` is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 58:** blank line used only to separate logical sections for readability.
- **Line 59:** blank line used only to separate logical sections for readability.
- **Line 60:** `# COMMAND` ----- is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 61:** `tables = ["web_users", "web_sessions", "web_events", "campaigns", "campaign_touchpoints", "marketing_spend"]` — `tables` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web_users"` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web_sessions"` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web_events"` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"campaigns"` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"campaign_touchpoints"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 62:** `results = []` — `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 63:** `for table in tables:` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tables` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 64:** `path = f"{batch_root}/postgres/{table}.parquet"` — `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{batch_root}/postgres/{table}.parquet"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 65:** `raw = spark.read.format("parquet").load(path)` — `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "parquet" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; load is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 66:** `audited = with_audit_columns(raw, "POSTGRES_DIGITAL")` — `audited` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `with_audit_columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"POSTGRES_DIGITAL"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 67:** `append_once(audited, f"digital_{table}")` — `append_once` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `audited` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"digital_{table}"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 68:** `results.append((table, raw.count()))` — `results.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `((` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw.count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 69:** `display(spark.createDataFrame(results, ["table_name", "rows_loaded"]))` — `display` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.createDataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"table_name"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"rows_loaded"` is a quoted literal; the quotation marks preserve spaces or mark text; `]))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 70:** `dbutils.jobs.taskValues.set(key="bronze_postgres_status", value="PASS")` — `dbutils.jobs.taskValues.set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze_postgres_status"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `value` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

File introduction — databricks/notebooks/03_bronze_files.py

Exact filename: 03_bronze_files.py

Filename extension: .py. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: databricks/notebooks relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 9.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in databricks/notebooks one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 03_bronze_files.py.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 03_bronze_files.py and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 9.F3.1 — Create the file databricks/notebooks/03_bronze_files.py from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.

4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\databricks\notebooks\03_bronze_files.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\databricks\notebooks\03_bronze_files.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\notebooks\03_bronze_files.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\databricks\notebooks\03_bronze_files.py" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "databricks/notebooks/03_bronze_files.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `databricks/notebooks`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `databricks/notebooks/03_bronze_files.py`.
3. In PowerShell at the repository root, run `Test-Path ".\databricks\notebooks\03_bronze_files.py"` and require True.
4. Run `git status --short -- "databricks/notebooks/03_bronze_files.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for databricks/notebooks/03_bronze_files.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `databricks/notebooks/03_bronze_files.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 9.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Databricks notebook source
# Northstar Retail End-to-End Data Platform
# This file is a Git-friendly Databricks source notebook.
# Track A reads files that you manually uploaded to a Unity Catalog volume.
# It never attempts to reach SQL Server or PostgreSQL on localhost.

# COMMAND -----
# MAGIC %md
# MAGIC # 03 – Bronze file feeds
# MAGIC CSV values remain strings in Bronze. Nested tracking JSON is preserved and parsed as a struct/array.

# COMMAND -----

from __future__ import annotations
from datetime import datetime, timezone
from functools import reduce
from pyspark.sql import DataFrame
```

```

from pyspark.sql import functions as F

for name, default in {
    "catalog": "northstar_retail", "landing_schema": "landing", "bronze_schema": "bronze",
    "volume": "track_a_files", "batch_id": "MANUAL-REPLACE-ME"
}.items():
    dbutils.widgets.text(name, default)

catalog = dbutils.widgets.get("catalog")
landing_schema = dbutils.widgets.get("landing_schema")
bronze_schema = dbutils.widgets.get("bronze_schema")
volume = dbutils.widgets.get("volume")
batch_id = dbutils.widgets.get("batch_id")
if batch_id == "MANUAL-REPLACE-ME":
    raise ValueError("Pass the extraction batch_id from the workflow.")
batch_root = f"/Volumes/{catalog}/{landing_schema}/{volume}/{batch_id}"

def with_audit_columns(df: DataFrame, source_system: str) -> DataFrame:
    """Preserve raw columns and add repeatable ingestion metadata."""
    payload = F.to_json(F.struct(*[F.col(c).cast("string").alias(c) for c in df.columns]))
    return (
        df.withColumn("_source_system", F.lit(source_system))
        .withColumn("_source_file", F.input_file_name())
        .withColumn("_ingested_at_utc", F.current_timestamp())
        .withColumn("_batch_id", F.lit(batch_id))
        .withColumn("_record_hash", F.sha2(payload, 256))
    )

def append_once(df: DataFrame, table_name: str) -> None:
    """Append this batch once; an exact rerun replaces the same batch safely."""
    full_name = f`${catalog}`.`{bronze_schema}`.`{table_name}`
    if spark.catalog.tableExists(full_name.replace("`", "")):
        spark.sql(f"DELETE FROM {full_name} WHERE _batch_id = '{batch_id}'")
    (df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full_name))
    loaded = spark.table(full_name).filter(F.col("_batch_id") == batch_id).count()
    if loaded != df.count():
        raise AssertionError(f"Bronze reconciliation failed for {table_name}: input={df.count()} target={loaded}")

# COMMAND -----
file_specs = {
    "file_returns": (f"{batch_root}/files/returns.csv", "csv"),
    "file_supplier_cost_updates": (f"{batch_root}/files/supplier_cost_updates.csv", "csv"),
    "file_promotion_calendar": (f"{batch_root}/files/promotion_calendar.csv", "csv"),
    "file_shipment_tracking_events": (f"{batch_root}/files/shipment_tracking_events.jsonl", "json"),
}
results = []
for table_name, (path, kind) in file_specs.items():
    try:
        if kind == "csv":
            raw = spark.read.option("header", "true").option("mode", "PERMISSIVE").csv(path)
        else:
            raw = spark.read.option("mode", "PERMISSIVE").json(path)
        audited = with_audit_columns(raw, "FILE_FEED")
        append_once(audited, table_name)
        results.append((table_name, raw.count(), "PASS"))
    except Exception as exc:
        # Create a visible quarantine record and fail the task so the quality gate cannot be bypassed.
        quarantine = spark.createDataFrame([(batch_id, path, str(exc), datetime.now(timezone.utc))], ["batch_id", "source_path", "error_message", "quarantined_at_utc"])
        quarantine.write.format("delta").mode("append").saveAsTable(f`${catalog}`.`{bronze_schema}`.`file_read_quarantine`)
        raise

display(spark.createDataFrame(results, ["table_name", "rows_loaded", "status"]))
dbutils.jobs.taskValues.set(key="bronze_files_status", value="PASS")

```

Line-by-line explanation

- **Line 1:** # Databricks notebook source is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** # Northstar Retail End-to-End Data Platform is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** # This file is a Git-friendly Databricks source notebook. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** # Track A reads files that you manually uploaded to a Unity Catalog volume. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** # It never attempts to reach SQL Server or PostgreSQL on localhost. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** blank line used only to separate logical sections for readability.

- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 9:** `# MAGIC %md` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 10:** `# MAGIC # 03 – Bronze file feeds` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 11:** `# MAGIC CSV values remain strings in Bronze. Nested tracking JSON is preserved and parsed as a struct/array.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 14:** blank line used only to separate logical sections for readability.
- **Line 15:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `from datetime import datetime, timezone` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timezone` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** `from functools import reduce` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `functools` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `reduce` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** `from pyspark.sql import DataFrame` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** `from pyspark.sql import functions as F` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `functions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** blank line used only to separate logical sections for readability.
- **Line 21:** `for name, default in {` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** `"catalog": "northstar_retail", "landing_schema": "landing", "bronze_schema": "bronze",` — `"catalog"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"northstar_retail"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"landing_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"landing"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding

explanation; "bronze_schema" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "bronze" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 23:** "volume": "track_a_files", "batch_id": "MANUAL-REPLACE-ME" — "volume" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "track_a_files" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "MANUAL-REPLACE-ME" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 24:** }.items(): — }. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; items is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** dbutils.widgets.text(name, default) — dbutils.widgets.text is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; default is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** blank line used only to separate logical sections for readability.
- **Line 27:** catalog = dbutils.widgets.get("catalog") — catalog is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.widgets.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "catalog" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** landing_schema = dbutils.widgets.get("landing_schema") — landing_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.widgets.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "landing_schema" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** bronze_schema = dbutils.widgets.get("bronze_schema") — bronze_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.widgets.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "bronze_schema" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 30:** volume = dbutils.widgets.get("volume") — volume is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.widgets.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "volume" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** batch_id = dbutils.widgets.get("batch_id") — batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.widgets.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

"batch_id" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 32:** `if batch_id == "MANUAL-REPLACE-ME":` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"MANUAL-REPLACE-ME"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** `raise ValueError("Pass the extraction batch_id from the workflow.")` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ValueError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Pass the extraction batch_id from the workflow."` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** `batch_root = f"/Volumes/{catalog}/{landing_schema}/{volume}/{batch_id}"` — `batch_root` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/{catalog}/{landing_schema}/{volume}/{batch_id}"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 35:** blank line used only to separate logical sections for readability.
- **Line 36:** blank line used only to separate logical sections for readability.
- **Line 37:** `def with_audit_columns(df: DataFrame, source_system: str) -> DataFrame:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `with_audit_columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; remaining punctuation and identifiers continue the same statement.
- **Line 38:** `"""Preserve raw columns and add repeatable ingestion metadata."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `"Preserve raw columns and add repeatable ingestion metadata."` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 39:** `payload = F.to_json(F.struct(*[F.col(c).cast("string").alias(c) for c in df.columns]))` — `payload` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.to_json` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.struct` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `c` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 40:** `return (` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** `df.withColumn("_source_system", F.lit(source_system))` — `df.withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_system"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 43:** `.withColumn("_ingested_at_utc", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at_utc"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 44:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 45:** `.withColumn("_record_hash", F.sha2(payload, 256))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_record_hash"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.sha2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `payload` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `256` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 47:** blank line used only to separate logical sections for readability.
- **Line 48:** blank line used only to separate logical sections for readability.
- **Line 49:** `def append_once(df: DataFrame, table_name: str) -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `append_once` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -> is a command option or flag; its exact behavior is defined beside the command; remaining punctuation and identifiers continue the same statement.

- **Line 50:** `""`Append this batch once; an exact rerun replaces the same batch safely.`""` — `""` is a quoted literal; the quotation marks preserve spaces or mark text; `"Append this batch once; an exact rerun replaces the same batch safely."` is a quoted literal; the quotation marks preserve spaces or mark text; `""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 51:** `full_name = f"\{catalog}\.{bronze_schema}\.{table_name}"`—`full_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\{catalog}\.{bronze_schema}\.{table_name}"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 52:** `if spark.catalog.tableExists(full_name.replace("\", "")):`—`if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.catalog.tableExists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full_name.replace` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `""` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `""` is a quoted literal; the quotation marks preserve spaces or mark text; `);` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 53:** `spark.sql(f"DELETE FROM {full_name} WHERE _batch_id = '{batch_id}')` — `spark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"DELETE FROM {full_name} WHERE _batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 54:** `(df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full_name))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"mergeSchema"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 55:** `loaded = spark.table(full_name).filter(F.col("_batch_id") == batch_id).count()` — `loaded` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 56:** `if loaded != df.count():` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `loaded` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df.count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `():` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 57:** `raise AssertionError(f"Bronze reconciliation failed for {table_name}: input={df.count()} target={loaded}")` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AssertionError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Bronze reconciliation failed for {table_name}: input={df.count()} target={loaded}"` is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 58:** blank line used only to separate logical sections for readability.
- **Line 59:** blank line used only to separate logical sections for readability.
- **Line 60:** `# COMMAND` ----- is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 61:** `file_specs = {` — `file_specs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 62:** `"file_returns": (f"{batch_root}/files/returns.csv", "csv"),` — `"file_returns"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{batch_root}/files/returns.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"csv"` is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 63:** `"file_supplier_cost_updates": (f"{batch_root}/files/supplier_cost_updates.csv", "csv"),` — `"file_supplier_cost_updates"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{batch_root}/files/supplier_cost_updates.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"csv"` is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 64:** `"file_promotion_calendar": (f"{batch_root}/files/promotion_calendar.csv", "csv"),` — `"file_promotion_calendar"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{batch_root}/files/promotion_calendar.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"csv"` is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 65:** `"file_shipment_tracking_events": (f"{batch_root}/files/shipment_tracking_events.jsonl", "json"),` — `"file_shipment_tracking_events"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{batch_root}/files/shipment_tracking_events.jsonl"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"json"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 66:** `}` — `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 67:** `results = []` — `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 68:** `for table_name, (path, kind) in file_specs.items():` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `kind` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `file_specs.items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 69:** `try:` — `try` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 70:** `if kind == "csv":` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `kind` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"csv"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 71:** `raw = spark.read.option("header", "true").option("mode", "PERMISSIVE").csv(path)` — `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"mode"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 72:** `else:` — `else` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 73:** `raw = spark.read.option("mode", "PERMISSIVE").json(path)` — `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "mode" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "PERMISSIVE" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; json is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 74:** `audited = with_audit_columns(raw, "FILE_FEED")` — `audited` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `with_audit_columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"FILE_FEED"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 75:** `append_once(audited, table_name)` — `append_once` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `audited` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 76:** `results.append((table_name, raw.count(), "PASS"))` — `results.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `((` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw.count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 77:** `except Exception as exc:` — `except` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Exception` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 78:** `# Create a visible quarantine record and fail the task so the quality gate cannot be bypassed.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 79:** `quarantine = spark.createDataFrame([(batch_id, path, str(exc), datetime.now(timezone.utc))], ["batch_id", "source_path", "error_message", "quarantined_at_utc"])` — `quarantine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.createDataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `([(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 80:** `quarantine.write.format("delta").mode("append").saveAsTable(f"\{catalog}\.{bronze_schema}\.file_read_quarantine")`—`quarantine.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\{catalog}\.{bronze_schema}\.file_read_quarantine"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 81:** `raise` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 82:** blank line used only to separate logical sections for readability.
- **Line 83:** `display(spark.createDataFrame(results, ["table_name", "rows_loaded", "status"]))` — `display` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.createDataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"table_name"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"rows_loaded"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 84:** `dbutils.jobs.taskValues.set(key="bronze_files_status", value="PASS")` — `dbutils.jobs.taskValues.set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze_files_status"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `value` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 9 lesson index](#)

File introduction — `sql/databricks/bronze/01_bronze_audit_pattern.sql`

Exact filename: `01_bronze_audit_pattern.sql`

Filename extension: `.sql`. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `sql/databricks/bronze` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 9.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in sql/databricks/bronze one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 01_bronze_audit_pattern.sql.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 01_bronze_audit_pattern.sql and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 9.F4.1 — Create the file sql/databricks/bronze/01_bronze_audit_pattern.sql from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\databricks\bronze\01_bronze_audit_pattern.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\databricks\bronze\01_bronze_audit_pattern.sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\databricks\bronze\01_bronze_audit_pattern.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\sql\databricks\bronze\01_bronze_audit_pattern.sql" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add -Force, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/databricks/bronze/01_bronze_audit_pattern.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `sql/databricks/bronze`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `sql/databricks/bronze/01_bronze_audit_pattern.sql`.
3. In PowerShell at the repository root, run `Test-Path ".\sql\databricks\bronze\01_bronze_audit_pattern.sql"` and require True.
4. Run `git status --short -- "sql/databricks/bronze/01_bronze_audit_pattern.sql"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `sql/databricks/bronze/01_bronze_audit_pattern.sql`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: `sql/databricks/bronze/01_bronze_audit_pattern.sql`

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 9.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: Databricks SQL / Spark SQL
-- Teaching pattern. The runnable Bronze implementation is in databricks/notebooks/01-03.
SELECT
  source.*,
  input_file_name() AS _source_file,
  current_timestamp() AS _ingested_at_utc,
  '${batch_id}' AS _batch_id,
  sha2(to_json(struct(source.*)), 256) AS _record_hash
FROM read_files('${input_path}', format => 'parquet') AS source;
```

Line-by-line explanation

- **Line 1:** `-- Dialect: Databricks SQL / Spark SQL` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `-- Teaching pattern. The runnable Bronze implementation is in databricks/notebooks/01-03.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** `SELECT` — `SELECT` requests data without changing it unless combined with a modifying statement
- **Line 4:** `source.*`, — `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.*`, is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 5:** `input_file_name() AS _source_file`, — `input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `_source_file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 6:** `current_timestamp()` AS `_ingested_at_utc`, — `current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `_ingested_at_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `'${batch_id}'` AS `_batch_id`, — `'${batch_id}'` is a quoted literal; the quotation marks preserve spaces or mark text; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `_batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `sha2(to_json(struct(source.*)), 256)` AS `_record_hash` — `sha2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `to_json` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `struct` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.*`) is a path, relative location, branch-style name, or file argument; verify every character before running it; `256` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `_record_hash` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `FROM read_files('${input_path}', format => 'parquet')` AS `source`; — `FROM` identifies the source table or query; `read_files` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'${input_path}'` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=>` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'parquet'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 9 lesson index](#)

What your screen should look like now

- Every Phase 9 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 9.3 completion checklist

☐

`databricks/notebooks/01_bronze_sqlserver.py` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`databricks/notebooks/02_bronze_postgres.py` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`databricks/notebooks/03_bronze_files.py` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`sql/databricks/bronze/01_bronze_audit_pattern.sql` was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 9.4](#) and recheck the root and branch.

Lesson 9.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Bronze object 1 — `bronze.erp_product_categories`

Input: `product_categories.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.erp_product_categories`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for `product_categories` in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B1 — Ingest `product_categories` into `bronze.erp_product_categories`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest `product_categories` into `bronze.erp_product_categories`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `product_categories`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `product_categories`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/product_categories.csv"
target_table = f"{catalog}.bronze.erp_product_categories"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/product_categories.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/product_categories.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.erp_product_categories"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_product_categories"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `bronze_df = (raw_df — bronze_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; raw_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation`
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta")` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 9:** `.mode("append")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table)` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for `product_categories` from the uploaded manifest.
2. Count rows in `bronze.erp_product_categories` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: `[ENTER VERIFIED RESULT]`.

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.erp_product_categories` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 2 — `bronze.erp_suppliers`

Input: `suppliers.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.erp_suppliers`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for `suppliers` in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B2 — Ingest suppliers into `bronze.erp_suppliers`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest suppliers into `bronze.erp_suppliers`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `suppliers`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `suppliers`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/suppliers.csv"
target_table = f"{catalog}.bronze.erp_suppliers"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/suppliers.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/suppliers.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.erp_suppliers"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_suppliers"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `bronze_df = (raw_df` — `bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 6:** `.withColumn("_source_file", F.input_file_name())` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_source_file" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.input_file_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ()) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_ingested_at" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.current_timestamp is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ()) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta")` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze_df.write.format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "append" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table)` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; target_table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for suppliers from the uploaded manifest.
2. Count rows in `bronze.erp_suppliers` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: [ENTER VERIFIED RESULT].

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.erp_suppliers` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 3 — `bronze.erp_products`

Input: `products.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.erp_products`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for products in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B3 — Ingest products into `bronze.erp_products`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest products into `bronze.erp_products`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for products.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/01_bronze_sqlserver.py in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter batch_id equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for products.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/products.csv"
target_table = f"{catalog}.bronze.erp_products"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/products.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/products.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 2:** `target_table = f"{catalog}.bronze.erp_products"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_products"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `bronze_df = (raw_df — bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta")` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark

text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 10:** `.saveAsTable(target_table)` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for products from the uploaded manifest.
2. Count rows in `bronze.erp_products` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: `[ENTER VERIFIED RESULT]`.

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.erp_products` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 4 — `bronze.erp_stores`

Input: `stores.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.erp_stores`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for stores in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B4 — Ingest stores into `bronze.erp_stores`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest stores into `bronze.erp_stores`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for stores.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for stores.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/stores.csv"
target_table = f"{catalog}.bronze.erp_stores"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/stores.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/stores.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.erp_stores"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_stores"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `bronze_df = (raw_df` — `bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta")` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table)` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for stores from the uploaded manifest.
2. Count rows in `bronze.erp_stores` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: [ENTER VERIFIED RESULT].

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.erp_stores` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 5 — `bronze.erp_employees`

Input: `employees.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.erp_employees`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for `employees` in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B5 — Ingest employees into `bronze.erp_employees`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest employees into `bronze.erp_employees`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `employees`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `employees`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.

8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/employees.csv"
target_table = f"{catalog}.bronze.erp_employees"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/employees.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/employees.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.erp_employees"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_employees"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

tion; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "header" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "true" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; csv is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 4:** `bronze_df = (raw_df — bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id)) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name()) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp()) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta") —` `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append") —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table)) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for employees from the uploaded manifest.
2. Count rows in `bronze.erp_employees` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: `[ENTER VERIFIED RESULT]`.

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.erp_employees` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 6 — `bronze.erp_customers`

Input: `customers.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.erp_customers`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for customers in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B6 — Ingest customers into `bronze.erp_customers`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest customers into `bronze.erp_customers`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.

2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for customers.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for customers.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/customers.csv"
target_table = f"{catalog}.bronze.erp_customers"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/customers.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/customers.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.erp_customers"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_customers"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `bronze_df = (raw_df` — `bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier,

or literal whose project-specific role is described in the surrounding explanation; "_ingested_at" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.current_timestamp is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ()) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 8:** `(bronze_df.write.format("delta")` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "append" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for customers from the uploaded manifest.
2. Count rows in `bronze.erp_customers` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: [ENTER VERIFIED RESULT].

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.erp_customers` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 7 — `bronze.erp_addresses`

Input: `addresses.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.erp_addresses`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for addresses in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B7 — Ingest addresses into bronze.erp_addresses

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest addresses into bronze.erp_addresses. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open databricks/notebooks/01_bronze_sqlserver.py in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter batch_id equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for addresses.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/01_bronze_sqlserver.py in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter batch_id equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for addresses.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/addresses.csv"
target_table = f"{catalog}.bronze.erp_addresses"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/addresses.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/addresses.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.erp_addresses"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f"{catalog}.bronze.erp_addresses"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 4:** `bronze_df = (raw_df — bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta")` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table)` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for addresses from the uploaded manifest.
2. Count rows in `bronze.erp_addresses` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: [ENTER VERIFIED RESULT].

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: PATH_NOT_FOUND, TABLE_OR_VIEW_NOT_FOUND, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created bronze.erp_addresses as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 8 — bronze.erp_orders

Input: orders.csv or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table bronze.erp_orders.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for orders in one batch.

Required metadata: _batch_id, _source_file, _ingested_at, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B8 — Ingest orders into bronze.erp_orders

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest orders into bronze.erp_orders. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open databricks/notebooks/01_bronze_sqlserver.py in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter batch_id equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for orders.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.

9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for orders.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/orders.csv"
target_table = f"{catalog}.bronze.erp_orders"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
```

```
.mode("append")
.saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/orders.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/orders.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.erp_orders"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_orders"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `bronze_df = (raw_df — bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta")` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument,

identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 9:** `.mode("append")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "append" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table)` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; target_table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for orders from the uploaded manifest.
2. Count rows in `bronze.erp_orders` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: [ENTER VERIFIED RESULT].

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.erp_orders` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 9 — `bronze.erp_order_items`

Input: `order_items.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.erp_order_items`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for `order_items` in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B9 — Ingest `order_items` into `bronze.erp_order_items`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest `order_items` into `bronze.erp_order_items`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `order_items`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `order_items`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/order_items.csv"
target_table = f"{catalog}.bronze.erp_order_items"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/order_items.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/order_items.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.erp_order_items"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_order_items"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `bronze_df = (raw_df` — `bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 6:** `.withColumn("_source_file", F.input_file_name())` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_source_file" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.input_file_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ()) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_ingested_at" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.current_timestamp is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ()) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta")` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze_df.write.format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "append" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table)` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; target_table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for order_items from the uploaded manifest.
2. Count rows in bronze.erp_order_items where _batch_id equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: [ENTER VERIFIED RESULT].

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: PATH_NOT_FOUND, TABLE_OR_VIEW_NOT_FOUND, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.erp_order_items` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 10 — `bronze.erp_payments`

Input: `payments.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.erp_payments`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for payments in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B10 — Ingest payments into `bronze.erp_payments`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest payments into `bronze.erp_payments`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for payments.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/01_bronze_sqlserver.py in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter batch_id equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for payments.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/payments.csv"
target_table = f"{catalog}.bronze.erp_payments"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/payments.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/payments.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 2:** `target_table = f"{catalog}.bronze.erp_payments"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_payments"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `bronze_df = (raw_df — bronze_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; raw_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation`
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta")` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark

text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 10:** `.saveAsTable(target_table)` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for payments from the uploaded manifest.
2. Count rows in `bronze.erp_payments` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: `[ENTER VERIFIED RESULT]`.

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.erp_payments` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 11 — `bronze.erp_shipments`

Input: `shipments.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.erp_shipments`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for `shipments` in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B11 — Ingest shipments into `bronze.erp_shipments`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest shipments into bronze.erp_shipments. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open databricks/notebooks/01_bronze_sqlserver.py in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter batch_id equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for shipments.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/01_bronze_sqlserver.py in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter batch_id equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for shipments.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/shipments.csv"
target_table = f"{catalog}.bronze.erp_shipments"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/shipments.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/shipments.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.erp_shipments"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_shipments"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `bronze_df = (raw_df` — `bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta")` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for shipments from the uploaded manifest.
2. Count rows in `bronze.erp_shipments` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: [ENTER VERIFIED RESULT].

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.erp_shipments` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 12 — `bronze.erp_inventory_transactions`

Input: `inventory_transactions.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.erp_inventory_transactions`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for `inventory_transactions` in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B12 — Ingest `inventory_transactions` into `bronze.erp_inventory_transactions`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest `inventory_transactions` into `bronze.erp_inventory_transactions`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `inventory_transactions`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `inventory_transactions`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.

6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/inventory_transactions.csv"
target_table = f"{catalog}.bronze.erp_inventory_transactions"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("batch_id", F.lit(batch_id))
    .withColumn("source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/inventory_transactions.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/inventory_transactions.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.erp_inventory_transactions"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_inventory_transactions"` is a quoted literal; the quotation marks preserve spaces or mark text

- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `bronze_df = (raw_df — bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta")` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table)` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

er, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for `inventory_transactions` from the uploaded manifest.
2. Count rows in `bronze.erp_inventory_transactions` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: `[ENTER VERIFIED RESULT]`.

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.erp_inventory_transactions` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 13 — `bronze.erp_inventory_snapshots`

Input: `inventory_snapshots.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.erp_inventory_snapshots`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for `inventory_snapshots` in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B13 — Ingest `inventory_snapshots` into `bronze.erp_inventory_snapshots`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest `inventory_snapshots` into `bronze.erp_inventory_snapshots`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `inventory_snapshots`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/01_bronze_sqlserver.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `inventory_snapshots`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/inventory_snapshots.csv"
target_table = f"{catalog}.bronze.erp_inventory_snapshots"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/inventory_snapshots.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/sqlserver/inventory_snapshots.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- Line 2:** `target_table = f"{catalog}.bronze.erp_inventory_snapshots"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_inventory_snapshots"` is a quoted literal; the quotation marks preserve spaces or mark text
- Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `bronze_df = (raw_df` — `bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or

literal whose project-specific role is described in the surrounding explanation; "_source_file" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.input_file_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_ingested_at" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.current_timestamp is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta")` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze_df.write.format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "append" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table)` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; target_table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for inventory_snapshots from the uploaded manifest.
2. Count rows in bronze.erp_inventory_snapshots where _batch_id equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: [ENTER VERIFIED RESULT].

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: PATH_NOT_FOUND, TABLE_OR_VIEW_NOT_FOUND, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created bronze.erp_inventory_snapshots as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 14 — bronze.digital_web_users

Input: web_users.csv or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table bronze.digital_web_users.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for web_users in one batch.

Required metadata: _batch_id, _source_file, _ingested_at, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B14 — Ingest web_users into bronze.digital_web_users

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest web_users into bronze.digital_web_users. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open databricks/notebooks/02_bronze_postgres.py in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter batch_id equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for web_users.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/02_bronze_postgres.py in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter batch_id equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for web_users.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.

8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/web_users.csv"
target_table = f"{catalog}.bronze.digital_web_users"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/web_users.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/web_users.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.digital_web_users"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.digital_web_users"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

tion; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "header" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "true" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; csv is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 4:** `bronze_df = (raw_df — bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id)) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name()) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta") —` `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append") —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table)) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for `web_users` from the uploaded manifest.
2. Count rows in `bronze.digital_web_users` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: `[ENTER VERIFIED RESULT]`.

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.digital_web_users` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 15 — `bronze.digital_campaigns`

Input: `campaigns.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.digital_campaigns`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for `campaigns` in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B15 — Ingest campaigns into `bronze.digital_campaigns`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest campaigns into `bronze.digital_campaigns`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/02_bronze_postgres.py` in the Databricks workspace or Git folder.

2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for campaigns.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/02_bronze_postgres.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for campaigns.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/campaigns.csv"
target_table = f"{catalog}.bronze.digital_campaigns"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/campaigns.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/campaigns.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- Line 2:** `target_table = f"{catalog}.bronze.digital_campaigns"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.digital_campaigns"` is a quoted literal; the quotation marks preserve spaces or mark text
- Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `bronze_df = (raw_df` — `bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier,

or literal whose project-specific role is described in the surrounding explanation; "_ingested_at" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.current_timestamp is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ()) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 8:** `(bronze_df.write.format("delta")` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "append" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for campaigns from the uploaded manifest.
2. Count rows in `bronze.digital_campaigns` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: [ENTER VERIFIED RESULT].

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.digital_campaigns` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 16 — `bronze.digital_web_sessions`

Input: `web_sessions.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.digital_web_sessions`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for `web_sessions` in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B16 — Ingest web_sessions into bronze.digital_web_sessions

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest web_sessions into bronze.digital_web_sessions. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open databricks/notebooks/02_bronze_postgres.py in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter batch_id equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for web_sessions.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/02_bronze_postgres.py in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter batch_id equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for web_sessions.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/web_sessions.csv"
target_table = f"{catalog}.bronze.digital_web_sessions"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/web_sessions.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/web_sessions.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.digital_web_sessions"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.digital_web_sessions"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 4:** `bronze_df = (raw_df — bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id)) — .` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name()) — .` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp()) — .` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta") — (` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append") — .` is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table)) — .` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for `web_sessions` from the uploaded manifest.
2. Count rows in `bronze.digital_web_sessions` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: [ENTER VERIFIED RESULT].

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: PATH_NOT_FOUND, TABLE_OR_VIEW_NOT_FOUND, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created bronze.digital_web_sessions as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 17 — `bronze.digital_web_events`

Input: `web_events.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.digital_web_events`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for `web_events` in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B17 — Ingest `web_events` into `bronze.digital_web_events`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest `web_events` into `bronze.digital_web_events`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/02_bronze_postgres.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `web_events`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/02_bronze_postgres.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `web_events`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.

7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/web_events.csv"
target_table = f"{catalog}.bronze.digital_web_events"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/web_events.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/web_events.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.digital_web_events"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.digital_web_events"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option`

is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "header" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "true" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; csv is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 4:** `bronze_df = (raw_df — bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id)) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name()) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta") —` `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append") —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table)) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for `web_events` from the uploaded manifest.
2. Count rows in `bronze.digital_web_events` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: `[ENTER VERIFIED RESULT]`.

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.digital_web_events` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 18 — `bronze.digital_campaign_touchpoints`

Input: `campaign_touchpoints.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.digital_campaign_touchpoints`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for `campaign_touchpoints` in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B18 — Ingest `campaign_touchpoints` into `bronze.digital_campaign_touchpoints`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest `campaign_touchpoints` into `bronze.digital_campaign_touchpoints`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open databricks/notebooks/02_bronze_postgres.py in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter batch_id equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for campaign_touchpoints.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/02_bronze_postgres.py in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter batch_id equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for campaign_touchpoints.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/campaign_touchpoints.csv"
target_table = f"{catalog}.bronze.digital_campaign_touchpoints"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/campaign_touchpoints.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/campaign_touchpoints.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- Line 2:** `target_table = f"{catalog}.bronze.digital_campaign_touchpoints"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.digital_campaign_touchpoints"` is a quoted literal; the quotation marks preserve spaces or mark text
- Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `bronze_df = (raw_df` — `bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-

specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta")` — `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table)` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for `campaign_touchpoints` from the uploaded manifest.
2. Count rows in `bronze.digital_campaign_touchpoints` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: `[ENTER VERIFIED RESULT]`.

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.digital_campaign_touchpoints` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 19 — `bronze.digital_marketing_spend`

Input: `marketing_spend.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.digital_marketing_spend`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business clean-

ing.

Grain: One Bronze row per input record for `marketing_spend` in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B19 — Ingest `marketing_spend` into `bronze.digital_marketing_spend`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest `marketing_spend` into `bronze.digital_marketing_spend`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/02_bronze_postgres.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `marketing_spend`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/02_bronze_postgres.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `marketing_spend`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/marketing_spend.csv"
target_table = f"{catalog}.bronze.digital_marketing_spend"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/marketing_spend.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/postgres/marketing_spend.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.digital_marketing_spend"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.digital_marketing_spend"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted

literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; csv is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 4:** `bronze_df = (raw_df — bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id)) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name()) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta") —` `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append") —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table)) —` . is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for `marketing_spend` from the uploaded manifest.
2. Count rows in `bronze.digital_marketing_spend` where `_batch_id` equals the current batch.

3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: [ENTER VERIFIED RESULT].

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: PATH_NOT_FOUND, TABLE_OR_VIEW_NOT_FOUND, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.digital_marketing_spend` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 20 — `bronze.file_returns`

Input: `returns.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.file_returns`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for returns in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B20 — Ingest returns into `bronze.file_returns`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest returns into `bronze.file_returns`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/03_bronze_files.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for returns.

5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/03_bronze_files.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for returns.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/file/returns.csv"
target_table = f"{catalog}.bronze.file_returns"
```

```
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/file/returns.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/file/returns.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.file_returns"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.file_returns"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `bronze_df = (raw_df` — `bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier,

er, or literal whose project-specific role is described in the surrounding explanation; ()) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 8:** `(bronze_df.write.format("delta")` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for returns from the uploaded manifest.
2. Count rows in `bronze.file_returns` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: [ENTER VERIFIED RESULT].

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.file_returns` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 21 — `bronze.file_supplier_cost_updates`

Input: `supplier_cost_updates.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.file_supplier_cost_updates`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for `supplier_cost_updates` in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B21 — Ingest `supplier_cost_updates` into `bronze.file_supplier_cost_updates`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest `supplier_cost_updates` into `bronze.file_supplier_cost_updates`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/03_bronze_files.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `supplier_cost_updates`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/03_bronze_files.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `supplier_cost_updates`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/file/supplier_cost_updates.csv"
target_table = f"{catalog}.bronze.file_supplier_cost_updates"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/file/supplier_cost_updates.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/file/supplier_cost_updates.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.file_supplier_cost_updates"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.file_supplier_cost_updates"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, ar-

gument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 4:** `bronze_df = (raw_df — bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta")` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for `supplier_cost_updates` from the uploaded manifest.
2. Count rows in `bronze.file_supplier_cost_updates` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: [ENTER VERIFIED RESULT].

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: PATH_NOT_FOUND, TABLE_OR_VIEW_NOT_FOUND, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.file_supplier_cost_updates` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 22 — `bronze.file_promotion_calendar`

Input: `promotion_calendar.csv` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.file_promotion_calendar`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for `promotion_calendar` in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B22 — Ingest `promotion_calendar` into `bronze.file_promotion_calendar`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest `promotion_calendar` into `bronze.file_promotion_calendar`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/03_bronze_files.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `promotion_calendar`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.

9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/03_bronze_files.py` in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter `batch_id` equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for `promotion_calendar`.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/file/promotion_calendar.csv"
target_table = f"{catalog}.bronze.file_promotion_calendar"
raw_df = spark.read.option("header", "true").csv(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
```

```
.mode("append")
.saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/file/promotion_calendar.csv"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/file/promotion_calendar.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.file_promotion_calendar"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.file_promotion_calendar"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.option("header", "true").csv(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"header"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `bronze_df = (raw_df — bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 8:** `(bronze_df.write.format("delta")` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for `promotion_calendar` from the uploaded manifest.
2. Count rows in `bronze.file_promotion_calendar` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: `[ENTER VERIFIED RESULT]`.

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: `PATH_NOT_FOUND`, `TABLE_OR_VIEW_NOT_FOUND`, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created `bronze.file_promotion_calendar` as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

Bronze object 23 — `bronze.file_shipment_tracking_events`

Input: `shipment_tracking_events.jsonl` or the equivalent entity extract in the uploaded Track A batch.

Target: Delta table `bronze.file_shipment_tracking_events`.

Responsibility: Preserve raw source columns while adding ingestion metadata; Bronze is not the place for business cleaning.

Grain: One Bronze row per input record for `shipment_tracking_events` in one batch.

Required metadata: `_batch_id`, `_source_file`, `_ingested_at`, and source/system identifiers used by the project audit pattern.

Schema rule: Capture or declare the source shape deliberately. Unexpected columns/types must be visible, not silently discarded.

Rerun rule: Test the same batch twice. The implementation must either reject duplicate batch ingestion or prove a controlled idempotent strategy; unbounded append is not acceptable evidence.

Action 9.4.B23 — Ingest shipment_tracking_events into bronze.file_shipment_tracking_events

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Ingest shipment_tracking_events into bronze.file_shipment_tracking_events. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open databricks/notebooks/03_bronze_files.py in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter batch_id equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for shipment_tracking_events.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/03_bronze_files.py in the Databricks workspace or Git folder.
2. LOOK FOR: Confirm the notebook language is Python and the expected compute is attached or serverless is available.
3. LOOK FOR: Confirm the widget/parameter batch_id equals the uploaded batch folder exactly.
4. CLICK: Add or locate the entity-specific ingestion cell for shipment_tracking_events.
5. TYPE THIS: Construct the entity path and target table from parameters; do not hard-code another user's catalog.
6. PRESS: Run the inspection cell first and display the raw schema and sample rows.
7. PRESS: Run the write cell only after the manifest entry and schema are understood.
8. LOOK FOR: A completed cell, a Delta target in Catalog Explorer, and the current batch metadata columns.
9. PRESS: Run the independent source-versus-Bronze count query.
10. DO NOT CONTINUE UNTIL: Row counts and batch ID match and a rerun test has a documented result.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 9 lesson index](#)

Teaching code section

```
source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/file/shipment_tracking_events.jsonl"
target_table = f"{catalog}.bronze.file_shipment_tracking_events"
raw_df = spark.read.json(source_path)
bronze_df = (raw_df
    .withColumn("_batch_id", F.lit(batch_id))
    .withColumn("_source_file", F.input_file_name())
    .withColumn("_ingested_at", F.current_timestamp()))
(bronze_df.write.format("delta")
    .mode("append")
    .saveAsTable(target_table))
```

Line-by-line explanation

- **Line 1:** `source_path = "/Volumes/<catalog>/landing/northstar/<batch_id>/file/shipment_tracking_events.jsonl"` — `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"/Volumes/<catalog>/landing/northstar/<batch_id>/file/shipment_tracking_events.jsonl"` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 2:** `target_table = f"{catalog}.bronze.file_shipment_tracking_events"` — `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.file_shipment_tracking_events"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 3:** `raw_df = spark.read.json(source_path)` — `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.read.json` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `bronze_df = (raw_df` — `bronze_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; `raw_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.withColumn("_source_file", F.input_file_name())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_source_file"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.input_file_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.withColumn("_ingested_at", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_ingested_at"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(bronze_df.write.format("delta")` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.mode("append")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(target_table)` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Manifest reconciliation

1. Read the expected count and checksum for `shipment_tracking_events` from the uploaded manifest.
2. Count rows in `bronze.file_shipment_tracking_events` where `_batch_id` equals the current batch.
3. Confirm every row has a source file and ingestion timestamp.
4. Compare the distinct source-file list with the manifest path.
5. Record the actual result: [ENTER VERIFIED RESULT].

Quarantine behavior

Bronze preserves parseable raw rows. A completely unreadable file, malformed JSON line, missing required header, or manifest mismatch should fail or enter a technical quarantine according to the notebook contract. Business-invalid values remain available for Silver rules.

Common error and exact recovery

Representative error: PATH_NOT_FOUND, TABLE_OR_VIEW_NOT_FOUND, permission denied, or a schema mismatch.

Meaning: The input path, catalog/schema, permission, or assumed format does not match the uploaded batch.

Do not do: Do not create a new empty file at the missing path and do not change the batch ID until the mismatch disappears.

First diagnostic: List the volume directory and print the computed path.

Safest fix: Correct the upload location or parameter, then rerun only this entity cell.

Verification: The exact batch path is visible and counts reconcile.

Interview-ready explanation

"I created bronze.file_shipment_tracking_events as an auditable raw Delta table. I attached batch and file lineage, reconciled against a manifest, and tested same-batch behavior before allowing Silver to read it."

What your screen should look like now

- The new Phase 9 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 9.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 9.5](#).

Lesson 9.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 9 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Verification before automation

Question:

Should the phase prioritize a visible manual verification or jump directly to automation?

Options considered:

- Manual verification first, then automation.
- Automation without a known-good baseline.
- Manual operation forever.

Beginner path selected:

Establish one verified manual baseline and then automate repeatable checks.

Why this path was selected:

A known-good example makes failures interpretable and prevents automation from hiding a wrong assumption.

Tradeoff:

The first run takes longer.

What a company might do instead:

Teams often pair infrastructure as code and automated tests with controlled bootstrap and acceptance procedures.

How to explain this in an interview:

I established a verified baseline, then encoded the same contract in tests and orchestration so automation preserved meaning rather than just speed.

If you see Result A

Bronze row counts and metadata reconcile to the manifest, including rerun behavior. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

A file is missing, malformed, quarantined, duplicated, or matched to the wrong manifest entry. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: Databricks workspace, Catalog Explorer, notebooks, and Jobs-compatible compute.

Window/panel: Show Bronze tables, metadata columns, manifest reconciliation, and same-batch behavior.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase09_build-the-bronze-ingestion-layer_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 9's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 9 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 9.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Action 9.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Action 9.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Interpret git status --short

- ?? means untracked: Git sees a new path but it is not staged.
- M means modified in the working tree but not staged.
- M means the staged version differs from the last commit.
- MM means a staged version exists and the file has additional unstaged edits.
- A means staged as a new file.
- D means deletion; stop and investigate unless deletion is deliberate.
- R means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 9.5.G4 — Inspect the unstaged diff for databricks/notebooks/ 01_bronze_sqlserver.py

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "databricks/notebooks/01_bronze_sqlserver.py"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "databricks/notebooks/01_bronze_sqlserver.py"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"databricks/notebooks/01_bronze_sqlserver.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Action 9.5.G5 — Stage only the reviewed file databricks/notebooks/01_bronze_sqlserver.py

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "databricks/notebooks/01_bronze_sqlserver.py"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "databricks/notebooks/01_bronze_sqlserver.py"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"databricks/notebooks/01_bronze_sqlserver.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 9.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Action 9.5.G7 — Create the Phase 9 commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "feat: ingest source batches into bronze"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "feat: ingest source batches into bronze"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"feat: ingest source batches into bronze"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Action 9.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):


```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Action 9.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 9 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message `feat: ingest source batches into bronze` and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 9.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 9 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

"Phase 9 created the verified boundary described by **Build the Bronze ingestion layer**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed."

Mini quiz

1. Multiple choice: Why does Phase 9 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

My Phase 9 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 9.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 10 cleans, types, deduplicates, and quarantines Bronze records.

Phase 9 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]



Output independently inspected — [STATUS]



Failure/alternate branch understood — [STATUS]



Screenshot captured and redacted — [STATUS]



Targeted Git commit exists — [STATUS]



Branch pushed and verified on GitHub — [STATUS]



Quiz completed before answer key — [STATUS]



Journal completed — [STATUS]



Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 10 — Build the Silver cleaning and conformance layer

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: Conformed Silver entities and explicit rejection rules.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 11 converts descriptive entities into durable dimensions.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 10 lesson index

1. [Lesson 10.1 — Understand the phase and starting screen](#)
2. [Lesson 10.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 10.3 — Create and understand the phase files](#)
4. [Lesson 10.4 — Build and validate required objects](#)
5. [Lesson 10.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 10.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** Databricks notebooks, Catalog Explorer, and SQL/Python cells.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 10 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 9 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 10 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

Databricks notebooks, Catalog Explorer, and SQL/Python cells. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 10 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.

- No new Phase 10 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 10 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 10.1 completion checklist

☐

I read why Phase 10 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 10.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 10 lesson index](#)

Lesson 10.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 10 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

Trusted-detail Silver tables for customers, products, sales, shipments, inventory, returns, promotions, web sessions/events, campaigns, and marketing spend, plus rule-specific quarantine tables and before/after quality evidence.

2. Why it matters

Silver is where source representations become governed data. It must distinguish correctable formatting differences from invalid business records, choose duplicate winners reproducibly, standardize time and currency assumptions, and preserve reject evidence so trusted counts can be explained.

3. Words to know

Term	Plain-language definition
Casting	Convert a value to a data type such as date, integer, decimal, timestamp.
Standardization	Represent equivalent values consistently, such as uppercase codes.
Validation	Test whether a value meets a rule.
Business key	Stable meaningful identifier used to deduplicate/conform.

Term	Plain-language definition
Window function	Calculation across related rows, used to rank duplicate candidates.
Conformed data	Shared representation across sources/domains.
UTC normalization	Convert timestamp instants to a common time zone.
Late-arriving record	Record received after expected time or related entity.
Change detection	Identify changed attributes, often with a hash.
MERGE	Transactional insert/update matching pattern.
Reject rule	Condition that keeps a row out of trusted output.
Quality flag	Non-blocking indicator retained for analysis.
Before/after example	Concrete raw value and resulting trusted/quarantine outcome.

4. What must already be completed

All Bronze tasks pass with manifest reconciliation and stable batch rerun. You know expected issue fixture counts. Silver notebooks 04–07 are available.

5. Applications to open

Application	How to open it	Why it is open
Databricks notebooks 04–07	Workspace Git folder/import.	Run domain transformations and inspect output.
Catalog Explorer / SQL editor	Silver and quality schemas.	Query trusted/quarantine tables.
VS Code	Open notebook source and SQL teaching patterns.	Read rules and complete TODO work.
Expected fixture	Uploaded/local JSON.	Compare configured defects with observed rules.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run `Get-Location`; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
<code>display(spark.table(f"{catalog}.{silver_schema}.customers").filter(F.col("batch_id") == batch_id).select("customer_number", "first_name", "email", "email_valid_flag", "state_code").limit(20))</code>	Shows standardized trusted customer fields/flags.	Validates correction and non-blocking flag behavior.
<code>SELECT quality_rule, COUNT(*) AS rejected_rows</code>	Counts customer rejects by rule.	Trusted output must explain exclusions.
<code>SELECT</code>	Summarizes typed valid sales.	Provides the independent pre-Gold revenue population.
<code>SELECT event_type, event_timestamp_utc, session_start_utc, out_of_order_flag</code>	Shows why events were rejected.	Time ordering is a business-quality rule, not a reader error.

Line or action	Meaning	Why it belongs here
bronze_count = spark.table(f"{catalog}.{bronze_schema}.erp_order_items").filter(F.col("_batch_id") == batch_id).count()	Reconciles sales line input to valid/reject outcomes.	No silent loss is allowed.

Tiny toy example

Input library rows: card A-1, name KIROLOS, email bad; another newer card A-1, name KIROLOS, valid email. Silver trims/title-cases, validates email, ranks by updated time, keeps the newer business-key winner, and quarantines the duplicate row. This teaches the pattern without revealing the final Northstar answer first.

Correct versus reject

Whitespace/case/date-format variations are usually correctable if unambiguous. Missing business key, impossible quantity, unreadable date, invalid relationship, or unsupported currency may require quarantine. A malformed email can be a quality flag rather than full-row rejection when sales analysis does not require sending email; document the policy.

MY TASK

Complete natural-key enrichment in `silver.sales`, add at least one Spark SQL equivalent to a PySpark rule, reconcile every Silver table to input/reject/dedup outcomes, and explain one correction-versus-quarantine decision.

STARTER SKELETON

```
# TODO(USER): enrich valid sales with natural business keys before Gold.
customer_keys = spark.table(f"{catalog}.{silver_schema}.customers").select(
    "customer_id", "customer_number"
)
product_keys = spark.table(f"{catalog}.{silver_schema}.products").select(
    "product_id", "sku"
)
store_keys = (spark.table(f"{catalog}.{bronze_schema}.erp_stores")
    .filter(F.col("_batch_id") == batch_id)
    .select("store_id", F.upper(F.trim("store_code")).alias("store_code")))

enriched = (sales
    .join(customer_keys, "customer_id", "left")
    .join(product_keys, "product_id", "left")
    .join(store_keys, "store_id", "left")
    # TODO(USER): employee_number and null/unknown policy.
)
# TODO(USER): test that required natural keys are non-null or intentionally UNKNOWN.
```

Acceptance criteria: Every trusted Silver table has declared grain and rules; input reconciles to valid/quarantine/dedup; timestamps/currency policy is consistent; sales carries natural keys for Gold lookup; rerun is stable.

HINT 1 — CONCEPT

Gold facts should join dimensions using natural/business keys and event date, not regex-extracted IDs.

HINT 2 — PSEUDOCODE

Build small lookup DataFrames from conformed/bronze parent tables, then left join and test null rates.

HINT 3 — NEAR-COMPLETE PATTERN

Use the near-complete skeleton and add employee/promotion/channel plus an explicit unknown policy.

13. Expected files or output

- Silver customers/products/sales/shipments/inventory/returns/promotions/web sessions/events/campaigns/spend.
- Quality quarantine tables with rule/batch/time.
- Natural-key enriched sales.
- Before/after examples.
- Layer reconciliation table.

14. How to verify

- Types are correct and codes standardized.
- No impossible quantities in trusted sales.
- UTC timestamps and event order are valid.
- Business-key duplicate winner is deterministic.
- Non-USD spend does not enter trusted Track A amounts.
- Input/valid/reject relationships are documented.

15. Automated tests

- Not-null/unique business keys.
- Accepted codes/currencies.
- Email validity flag counts.
- Date/numeric ranges.
- Referential integrity/natural-key availability.
- Same-batch replacement stability.
- Observed fixture count report.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
Cast produces null	Unexpected format or invalid value.	Filter source non-null where casted value null; display raw values.	Add explicit format/coalesce or quarantine; do not silently default.	No unexplained cast nulls.
Duplicate winner changes on rerun	Ranking order lacks deterministic tie-breaker.	Inspect equal updated_at rows/window order.	Add stable secondary key descending.	Same winner/hash each run.
Join multiplies rows	Lookup not unique at join key/grain.	Group lookup key HAVING count>1; compare counts before/after.	Deduplicate/conform lookup first; assert uniqueness.	Output count relationship expected.
Time zone shifted incorrectly	Timestamp already includes offset but conversion assumes local zone.	Inspect raw timestamp/parsed timezone.	Parse instant correctly; convert once to UTC; unit-test offsets.	Known instant aligns.
Quarantine duplicates on re-run	Append without batch replacement/unique grain.	Group by batch/rule/business key.	Delete/merge same batch before append.	Stable count.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. What makes a formatting issue correctable but a quantity issue rejectable?
2. How does a deterministic window resolve duplicate business keys?
3. Why should Silver sales carry natural keys?
4. Why can injected issue counts differ from observed rule counts?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. Which layer should normalize timestamps to UTC?
2. What test detects a row-multiplying join?
3. Why might malformed email be a flag rather than a full-row reject?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture a before/after customer example, sales quarantine count, digital event quarantine, and a reconciliation output proving no silent loss.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
feat: build conformed silver tables and quality quarantines
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

Customer rules verified — [STATUS]

☐

Product/cost rules verified — [STATUS]

☐

Sales/return/shipping/inventory rules verified — [STATUS]

☐

Digital/marketing rules verified — [STATUS]

☐

Natural keys enriched — [STATUS]

☐

Quarantines reconciled — [STATUS]

☐

Same-batch rerun stable — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-10. The quiz answer is in the answer-key section immediately after the quiz in this workbook#phase-10.

Next phase: Phase 11 creates Gold dimensions, surrogate keys, unknown members, Type 1 corrections, and SCD Type 2 history.

Required action cards

Action 10.2.1 — Start with a toy example unrelated to Northstar: library patrons with ' KIR0LLO5 ', duplicate card number

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Start with a toy example unrelated to Northstar: library patrons with ' KIR0LLO5 ', duplicate card number. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Start with a toy example unrelated to Northstar: library patrons with ' KIROLL0S ', duplicate card numbers, and invalid email. Decide what is corrected, flagged, or quarantined before reading final customer code.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 1****Action**

Start with a toy example unrelated to Northstar: library patrons with ' KIROLL0S ', duplicate card numbers, and invalid email. Decide what is corrected, flagged, or quarantined before reading final customer code.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location

with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Action 10.2.2 — Open 04_silver_customers. Trace trim/case/email validation/address join/window ranking/quarantine/batch repl

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open 04_silver_customers. Trace trim/case/email validation/address join/window ranking/quarantine/batch repl. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open 04_silver_customers. Trace trim/case/email validation/address join/window ranking/quarantine/batch replacement. Note malformed email may be retained with email_valid_flag=false if business key is valid; duplicate/missing business key is quarantined.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 2

Action

Open 04_silver_customers. Trace trim/case/email validation/address join/window ranking/quarantine/batch replacement. Note malformed email may be retained with email_valid_flag=false if business key is valid; duplicate/missing business key is quarantined.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Action 10.2.3 — Run customer notebook and compare input = trusted + duplicate/missing-key quarantine under documented rule. Co

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run customer notebook and compare input = trusted + duplicate/missing-key quarantine under documented rule. Co. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run customer notebook and compare input = trusted + duplicate/missing-key quarantine under documented rule. Count invalid-email flags separately because they may remain trusted for non-email analysis.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 3****Action**

Run customer notebook and compare input = trusted + duplicate/missing-key quarantine under documented rule. Count invalid-email flags separately because they may remain trusted for non-email analysis.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Action 10.2.4 — Open/run 05_silver_products. Check SKU/category/supplier conformance and latest valid supplier cost by effective date

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open/run 05_silver_products. Check SKU/category/supplier conformance and latest valid supplier cost by effective date. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open/run 05_silver_products. Check SKU/category/supplier conformance and latest valid supplier cost by effective date.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 4****Action**

Open/run 05_silver_products. Check SKU/category/supplier conformance and latest valid supplier cost by effective date.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Action 10.2.5 — Open/run 06_silver_sales. Impossible quantities are quarantined; valid line amounts are derived. Returns par

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open/run 06_silver_sales. Impossible quantities are quarantined; valid line amounts are derived. Returns par. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open/run 06_silver_sales. Impossible quantities are quarantined; valid line amounts are derived. Returns parse multiple formats; unsupported dates/quantities quarantine. Shipments and inventory are typed and flagged.
4. Pause while the application finishes the operation; do not click the same control twice.

5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 5****Action**

Open/run 06_silver_sales. Impossible quantities are quarantined; valid line amounts are derived. Returns parse multiple formats; unsupported dates/quantities quarantine. Shipments and inventory are typed and flagged.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Action 10.2.6 — Open/run 07_silver_digital. Timestamps normalize to UTC, sessions derive duration/conversion, out-of-order e

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open/run 07_silver_digital. Timestamps normalize to UTC, sessions derive duration/conversion, out-of-order e. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open/run 07_silver_digital. Timestamps normalize to UTC, sessions derive duration/conversion, out-of-order events quarantine, and non-USD/negative spend is excluded under Track A currency policy.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 6

Action

Open/run 07_silver_digital. Timestamps normalize to UTC, sessions derive duration/conversion, out-of-order events quarantine, and non-USD/negative spend is excluded under Track A currency policy.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Action 10.2.7 — For each Silver table, write input tables, output grain, transformation rules, reject rules, PySpark pattern,

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

For each Silver table, write input tables, output grain, transformation rules, reject rules, PySpark pattern, . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: For each Silver table, write input tables, output grain, transformation rules, reject rules, PySpark pattern, Spark SQL equivalent, tests, and before/after examples in the journal or source mapping.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 7****Action**

For each Silver table, write input tables, output grain, transformation rules, reject rules, PySpark pattern, Spark SQL equivalent, tests, and before/after examples in the journal or source mapping.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Action 10.2.8 — Complete natural-key enrichment: ensure Silver sales carries customer_number, sku, store_code, `employee`

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Complete natural-key enrichment: ensure Silver sales carries customer_number, sku, store_code, employee. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Complete natural-key enrichment: ensure Silver sales carries customer_number, sku, store_code, employee_number, promotion_code, and channel_code rather than forcing Gold facts to reverse-engineer IDs from strings.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 8****Action**

Complete natural-key enrichment: ensure Silver sales carries customer_number, sku, store_code, employee_number, promotion_code, and channel_code rather than forcing Gold facts to reverse-engineer IDs from strings.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Action 10.2.9 — Add observed issue-count results to a quality summary; explain why injection counts and observed rule counts c

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Add observed issue-count results to a quality summary; explain why injection counts and observed rule counts c. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Add observed issue-count results to a quality summary; explain why injection counts and observed rule counts can differ when one row triggers multiple fields/rules.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 9

Action

Add observed issue-count results to a quality summary; explain why injection counts and observed rule counts can differ when one row triggers multiple fields/rules.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Action 10.2.10 — Run same-batch rerun and prove trusted/quarantine counts do not grow unexpectedly.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run same-batch rerun and prove trusted/quarantine counts do not grow unexpectedly. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run same-batch rerun and prove trusted/quarantine counts do not grow unexpectedly.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 10****Action**

Run same-batch rerun and prove trusted/quarantine counts do not grow unexpectedly.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Action 10.2.C1 — Databricks notebook cell

Application:

Databricks workspace in a web browser

Window or panel:

Databricks notebook cell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Python instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Python):

```
display(spark.table(f"{catalog}.{silver_schema}.customers").filter(F.col("batch_id") == batch_id).select("customer_number","first_name","email","email_valid_flag","state_code").limit(20))
```

Where to enter it:

Databricks notebook cell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `display(spark.table(f"{catalog}.{silver_schema}.customers").filter(F.col("batch_id") == batch_id).select("customer_number","first_name","email","email_valid_flag","state_code").limit(20))` — `display` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.{silver_schema}.customers" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Action 10.2.C2 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT quality_rule, COUNT(*) AS rejected_rows
FROM northstar_retail.quality.customer_quarantine
WHERE batch_id = 'EXT-YYYYMMDDTHHMMSSZ'
GROUP BY quality_rule;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT quality_rule, COUNT(*) AS rejected_rows` — `SELECT` requests data without changing it unless combined with a modifying statement; `quality_rule` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rejected_rows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `FROM northstar_retail.quality.customer_quarantine` — `FROM` identifies the source table or query; `northstar_retail.quality.customer_quarantine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `WHERE batch_id = 'EXT-YYYYMMDDTHHMMSSZ'` — `WHERE` filters rows to those meeting a condition; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'EXT-YYYYMMDDTHHMMSSZ'` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 4:** `GROUP BY quality_rule;` — `GROUP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quality_rule` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Action 10.2.C3 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.

2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT
  COUNT(*) AS valid_lines,
  SUM(gross_sales_amount) AS gross_sales,
  SUM(discount_dec) AS discounts,
  SUM(net_sales_amount) AS net_sales,
  SUM(gross_profit_amount) AS gross_profit
FROM northstar_retail.silver.sales
WHERE batch_id = 'EXT-YYYYMMDDTHHMMSSZ';
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** SELECT — SELECT requests data without changing it unless combined with a modifying statement
- **Line 2:** COUNT(*) AS valid_lines, — COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid_lines is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** SUM(gross_sales_amount) AS gross_sales, — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gross_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gross_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** SUM(discount_dec) AS discounts, — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; discount_dec is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; discounts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** SUM(net_sales_amount) AS net_sales, — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; net_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; net_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** SUM(gross_profit_amount) AS gross_profit — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

gross_profit_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gross_profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** FROM northstar_retail.silver.sales — FROM identifies the source table or query; northstar_retail.silver.sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** WHERE batch_id = 'EXT-YYYYMMDDTHMMSSZ' ; — WHERE filters rows to those meeting a condition; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'EXT-YYYYMMDDTHMMSSZ' is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Action 10.2.C4 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT event_type, event_timestamp_utc, session_start_utc, out_of_order_flag
FROM northstar_retail.quality.digital_event_quarantine
WHERE batch_id = 'EXT-YYYYMMDDTHHMMSSZ'
ORDER BY event_timestamp_utc
LIMIT 20;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, and values as inputs.

What each part means:

- **Line 1:** `SELECT event_type, event_timestamp_utc, session_start_utc, out_of_order_flag` — `SELECT` requests data without changing it unless combined with a modifying statement; `event_type` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `event_timestamp_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `session_start_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `out_of_order_flag` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `FROM northstar_retail.quality.digital_event_quarantine` — `FROM` identifies the source table or query; `northstar_retail.quality.digital_event_quarantine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `WHERE batch_id = 'EXT-YYYYMMDDTHHMMSSZ'` — `WHERE` filters rows to those meeting a condition; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'EXT-YYYYMMDDTHHMMSSZ'` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 4:** `ORDER BY event_timestamp_utc` — `ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `event_timestamp_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 5:** `LIMIT 20`; — `LIMIT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `20` is a numeric literal used by the command or expression

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Action 10.2.C5 — Databricks notebook cell

Application:

Databricks workspace in a web browser

Window or panel:

Databricks notebook cell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Python instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Python):

```
bronze_count = spark.table(f"{catalog}.{bronze_schema}.erp_order_items").filter(F.col("_batch_id") == batch_id).count()
valid_count = spark.table(f"{catalog}.{silver_schema}.sales").filter(F.col("batch_id") == batch_id).count()
reject_count = spark.table(f"{catalog}.{quality_schema}.sales_quarantine").filter(F.col("batch_id") == batch_id).count()
print({"bronze": bronze_count, "valid": valid_count, "reject": reject_count, "balanced": bronze_count == valid_count + reject_count})
```

Where to enter it:

Databricks notebook cell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `bronze_count = spark.table(f"{catalog}.{bronze_schema}.erp_order_items").filter(F.col("_batch_id") == batch_id).count()` — `bronze_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 2:** `valid_count = spark.table(f"{catalog}.{silver_schema}.sales").filter(F.col("batch_id") == batch_id).count()` — `valid_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 3:** `reject_count = spark.table(f"{catalog}.{quality_schema}.sales_quarantine").filter(F.col("batch_id") == batch_id).count()` — `reject_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.{quality_schema}.sales_quarantine"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 4:** `print({"bronze": bronze_count, "valid": valid_count, "reject": reject_count, "balanced": bronze_count == valid_count + reject_count})` — `print` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"valid"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"reject"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 10.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 10 starting-screen checklist and re-verify the active context.

Lesson 10.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 10 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — databricks/

Exact folder name: databricks

Exact parent: repository root

Why it is needed now: Phase 10 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.

6. TYPE THIS: databricks.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 10.D1.1 — Create the folder databricks

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\databricks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\databricks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\databricks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\databricks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\databricks" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/

Exact folder name: `sql`

Exact parent: repository root

Why it is needed now: Phase 10 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `sql`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 10.D2.1 — Create the folder `sql`

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run Test-Path ".\sql" -PathType Container.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — databricks/notebooks/

Exact folder name: notebooks

Exact parent: databricks

Why it is needed now: Phase 10 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path databricks one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: notebooks.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 10.D3.1 — Create the folder databricks/notebooks

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\databricks\notebooks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\databricks\notebooks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\databricks\notebooks` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\databricks\notebooks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\databricks\notebooks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\databricks\notebooks" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — `sql/databricks/`

Exact folder name: `databricks`

Exact parent: `sql`

Why it is needed now: Phase 10 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `sql` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `databricks`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method**Action 10.D4.1 — Create the folder `sql/databricks`**

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\databricks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\databricks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\sql\databricks` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\databricks" -PathType Container` and require True.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\databricks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\databricks" -PathType Container`.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/databricks/silver/

Exact folder name: silver

Exact parent: sql/databricks

Why it is needed now: Phase 10 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path sql/databricks one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: silver.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 10.D5.1 — Create the folder sql/databricks/silver

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\databricks\silver" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\databricks\silver" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\sql\databricks\silver` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\databricks\silver" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\databricks\silver" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\databricks\silver" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — databricks/notebooks/04_silver_customers.py

Exact filename: `04_silver_customers.py`

Filename extension: `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `databricks/notebooks` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 10.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-

specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in databricks/notebooks one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 04_silver_customers.py.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 04_silver_customers.py and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 10.F1.1 — Create the file databricks/notebooks/04_silver_customers.py from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\databricks\notebooks\04_silver_customers.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\databricks\notebooks\04_silver_customers.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\notebooks\04_silver_customers.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\databricks\notebooks\04_silver_customers.py" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add -Force, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "databricks/notebooks/04_silver_customers.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `databricks/notebooks`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `databricks/notebooks/04_silver_customers.py`.
3. In PowerShell at the repository root, run `Test-Path ".\databricks\notebooks\04_silver_customers.py"` and require `True`.
4. Run `git status --short -- "databricks/notebooks/04_silver_customers.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for databricks/notebooks/04_silver_customers.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `databricks/notebooks/04_silver_customers.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 10.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Databricks notebook source
# Northstar Retail End-to-End Data Platform
# This file is a Git-friendly Databricks source notebook.
# Track A reads files that you manually uploaded to a Unity Catalog volume.
# It never attempts to reach SQL Server or PostgreSQL on localhost.

# COMMAND -----
# MAGIC %md
# MAGIC # 04 – Silver customers
# MAGIC Standardizes names and codes, validates email, chooses one business-key winner, and quarantines rejects.

# COMMAND -----

from __future__ import annotations
from pyspark.sql import functions as F
from pyspark.sql.window import Window

for name, default in {
    "catalog": "northstar_retail", "bronze_schema": "bronze", "silver_schema": "silver",
    "quality_schema": "quality", "batch_id": "MANUAL-REPLACE-ME"
}.items(): dbutils.widgets.text(name, default)
catalog = dbutils.widgets.get("catalog")
bronze_schema = dbutils.widgets.get("bronze_schema")
silver_schema = dbutils.widgets.get("silver_schema")
quality_schema = dbutils.widgets.get("quality_schema")
batch_id = dbutils.widgets.get("batch_id")
if batch_id == "MANUAL-REPLACE-ME": raise ValueError("Pass batch_id from the workflow.")

def replace_batch(df, table_name: str):
    full = f"{catalog}`.`{silver_schema}`.`{table_name}`"
    if spark.catalog.tableExists(full.replace("`", "")):
        spark.sql(f"DELETE FROM {full} WHERE batch_id = '{batch_id}'")
    df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full)
```

```

def quarantine(df, rule_name: str, table_name: str):
    if df.limit(1).count():
        (df.withColumn("quality_rule", F.lit(rule_name))
         .withColumn("batch_id", F.lit(batch_id))
         .withColumn("quarantined_at_utc", F.current_timestamp())
         .write.format("delta").mode("append").option("mergeSchema", "true")
         .saveAsTable(f"{catalog}`.`{quality_schema}`.`{table_name}`"))

# COMMAND -----
bronze = spark.table(f"{catalog}`.`{bronze_schema}`.`erp_customers`").filter(F.col("batch_id") == batch_id)
addresses = spark.table(f"{catalog}`.`{bronze_schema}`.`erp_addresses`").filter(F.col("batch_id") == batch_id)
address_one = (
    addresses.withColumn("rank", F.row_number().over(Window.partitionBy("customer_id").orderBy(F.col("is_default").desc(), F.col("updated_at").desc()))
               .filter(F.col("rank") == 1)
               .select("customer_id", F.upper(F.trim("state_code")).alias("state_code"), F.upper(F.trim("country_code")).alias("country_code")))
)
cleaned = (
    bronze.withColumn("customer_number", F.upper(F.trim("customer_number")))
           .withColumn("first_name", F.initcap(F.trim("first_name")))
           .withColumn("last_name", F.initcap(F.trim("last_name")))
           .withColumn("email", F.lower(F.trim("email")))
           .withColumn("loyalty_tier", F.initcap(F.lower(F.trim("loyalty_tier"))))
           .withColumn("signup_date", F.to_date("signup_date"))
           .join(address_one, "customer_id", "left")
           .withColumn("email_valid_flag", F.col("email").rlike(r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"))
           .withColumn("batch_id", F.lit(batch_id))
           .withColumn("load_timestamp_utc", F.current_timestamp())
           .withColumn("source_system", F.lit("SQLSERVER_ERP"))
)
window = Window.partitionBy("customer_number").orderBy(F.col("updated_at").desc(), F.col("customer_id").desc())
ranked = cleaned.withColumn("business_key_rank", F.row_number().over(window))
rejects = ranked.filter((F.col("customer_number").isNull()) | (F.col("business_key_rank") > 1))
quarantine(rejects, "MISSING_OR_DUPLICATE_CUSTOMER_BUSINESS_KEY", "customer_quarantine")
valid = ranked.filter((F.col("customer_number").isNotNull()) & (F.col("business_key_rank") == 1)).drop("business_key_rank")
replace_batch(valid, "customers")
print({"input": bronze.count(), "valid": valid.count(), "quarantined": rejects.count()})
dbutils.jobs.taskValues.set(key="silver_customers_status", value="PASS")

```

Line-by-line explanation

- **Line 1:** # Databricks notebook source is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** # Northstar Retail End-to-End Data Platform is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** # This file is a Git-friendly Databricks source notebook. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** # Track A reads files that you manually uploaded to a Unity Catalog volume. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** # It never attempts to reach SQL Server or PostgreSQL on localhost. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** # COMMAND ----- is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 9:** # MAGIC %md is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 10:** # MAGIC # 04 – Silver customers is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 11:** # MAGIC Standardizes names and codes, validates email, chooses one business-key winner, and quarantines rejects. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** # COMMAND ----- is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 14:** blank line used only to separate logical sections for readability.
- **Line 15:** from __future__ import annotations — from is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; __future__ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; import is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; annotations is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 16:** `from pyspark.sql import functions as F` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `functions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** `from pyspark.sql.window import Window` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql.window` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Window` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** `for name, default in {` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** `"catalog": "northstar_retail", "bronze_schema": "bronze", "silver_schema": "silver",` — `"catalog"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"northstar_retail"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"silver_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"silver"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** `"quality_schema": "quality", "batch_id": "MANUAL-REPLACE-ME"` — `"quality_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quality"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"MANUAL-REPLACE-ME"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 22:** `} .items(): dbutils.widgets.text(name, default)` — `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** `catalog = dbutils.widgets.get("catalog")` — `catalog` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"catalog"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 24:** `bronze_schema = dbutils.widgets.get("bronze_schema")` — `bronze_schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** `silver_schema = dbutils.widgets.get("silver_schema")` — `silver_schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"silver_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** `quality_schema = dbutils.widgets.get("quality_schema")` — `quality_schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quality_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:** `batch_id = dbutils.widgets.get("batch_id")` — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** `if batch_id == "MANUAL-REPLACE-ME": raise ValueError("Pass batch_id from the workflow.")` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"MANUAL-REPLACE-ME"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ValueError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Pass batch_id from the workflow."` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** blank line used only to separate logical sections for readability.
- **Line 30:** blank line used only to separate logical sections for readability.
- **Line 31:** `def replace_batch(df, table_name: str):` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `replace_batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 32:** `full = f"\{catalog}\{silver_schema}\{table_name}"` — `full` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\{catalog}\{silver_schema}\{table_name}"` is a quoted literal; the quotation marks preserve spaces or mark text

tion;fis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;""{catalog}`.`{silver_schema}`.`{table_name}`"" is a quoted literal; the quotation marks preserve spaces or mark text

- **Line 33:** `if spark.catalog.tableExists(full.replace("\", "")):`—if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.catalog.tableExists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full.replace` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `""` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `""` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** `spark.sql(f"DELETE FROM {full} WHERE batch_id = '{batch_id}')` — `spark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"DELETE FROM {full} WHERE batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** `df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full)` — `df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"mergeSchema"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 36:** blank line used only to separate logical sections for readability.
- **Line 37:** blank line used only to separate logical sections for readability.
- **Line 38:** `def quarantine(df, rule_name: str, table_name: str):` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quarantine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rule_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 39:** `if df.limit(1).count():` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df.limit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 40:** `(df.withColumn("quality_rule", F.lit(rule_name)))` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df.withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "quality_rule" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rule_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** `.withColumn("batch_id", F.lit(batch_id))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** `.withColumn("quarantined_at_utc", F.current_timestamp())` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "quarantined_at_utc" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 43:** `.write.format("delta").mode("append").option("mergeSchema", "true")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "append" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "mergeSchema" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 44:** `.saveAsTable(f"{catalog}`.`{quality_schema}`.`{table_name}`")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}`.`{quality_schema}`.`{table_name}`"` is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 45:** blank line used only to separate logical sections for readability.
- **Line 46:** blank line used only to separate logical sections for readability.
- **Line 47:** `# COMMAND` ----- is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 48:** `bronze = spark.table(f"{catalog}`.`{bronze_schema}`.`erp_customers`").filter(F.col("_batch_id") == batch_id)` — `bronze` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, ar-

argument, identifier, or literal whose project-specific role is described in the surrounding explanation;fis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;"{catalog}`.`{bronze_schema}`.erp_customers"is a quoted literal; the quotation marks preserve spaces or mark text;).is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;filteris a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;F.colis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;"_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 49:** addresses = spark.table(f"\{catalog}`.`{bronze_schema}`.erp_addresses").filter(F.col("_batch_id") == batch_id)—addresses is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;spark.table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;fis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;"{catalog}`.`{bronze_schema}`.erp_addresses" is a quoted literal; the quotation marks preserve spaces or mark text;).is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;"_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 50:** address_one = (— address_one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 51:** addresses.withColumn("rank", F.row_number().over(Window.partitionBy("customer_id").orderBy(F.col("is_default").desc(), F.col("updated_at").desc())) — addresses.withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "rank" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.row_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; over is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Window.partitionBy is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "customer_id" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 52:** .filter(F.col("rank") == 1) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "rank" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 53:** .select("customer_id", F.upper(F.trim("state_code")).alias("state_code"), F.upper(F.trim("country_code")).alias("country_code")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; select is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.trim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "email" is a quoted literal; the quotation marks preserve spaces or mark text;))) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 60:** `.withColumn("loyalty_tier", F.initcap(F.lower(F.trim("loyalty_tier"))))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "loyalty_tier" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.initcap is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lower is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.trim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "loyalty_tier" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 61:** `.withColumn("signup_date", F.to_date("signup_date"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "signup_date" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.to_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "signup_date" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 62:** `.join(address_one, "customer_id", "left")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; join is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; address_one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "customer_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "left" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 63:** `.withColumn("email_valid_flag", F.col("email").rlike(r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "email_valid_flag" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "email" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rlike is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; r is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 64:** `.withColumn("batch_id", F.lit(batch_id))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-

specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 65:** `.withColumn("load_timestamp_utc", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "load_timestamp_utc" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 66:** `.withColumn("source_system", F.lit("SQLSERVER_ERP"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "source_system" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "SQLSERVER_ERP" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 67:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 68:** `window = Window.partitionBy("customer_number").orderBy(F.col("updated_at").desc(), F.col("customer_id").desc())` — `window` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Window.partitionBy` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "customer_number" is a quoted literal; the quotation marks preserve spaces or mark text;) `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orderBy` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "updated_at" is a quoted literal; the quotation marks preserve spaces or mark text;) `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 69:** `ranked = cleaned.withColumn("business_key_rank", F.row_number().over(window))` — `ranked` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cleaned.withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "business_key_rank" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.row_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `over` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `window` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 70:** `rejects = ranked.filter((F.col("customer_number").isNull()) | (F.col("business_key_rank") > 1))` — `rejects` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ranked.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ((is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier,

er, or literal whose project-specific role is described in the surrounding explanation; "customer_number" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; isNull is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 71:** quarantine(rejects, "MISSING_OR_DUPLICATE_CUSTOMER_BUSINESS_KEY", "customer_quarantine") — quarantine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rejects is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "MISSING_OR_DUPLICATE_CUSTOMER_BUSINESS_KEY" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "customer_quarantine" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 72:** valid = ranked.filter((F.col("customer_number").isNull()) & (F.col("business_key_rank") == 1)).drop("business_key_rank") — valid is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ranked.filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ((is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "customer_number" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; isNull is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 73:** replace_batch(valid, "customers") — replace_batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "customers" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 74:** print({"input": bronze.count(), "valid": valid.count(), "quarantined": rejects.count()}) — print is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ({ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "input" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze.count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "valid" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid.count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "quarantined" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 75:** dbutils.jobs.taskValues.set(key="silver_customers_status", value="PASS") — dbutils.jobs.taskValues.set is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "silver_customers_status" is a quoted literal; the

quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "PASS" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 10 lesson index](#)

File introduction — databricks/notebooks/05_silver_products.py

Exact filename: 05_silver_products.py

Filename extension: .py. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: databricks/notebooks relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 10.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in databricks/notebooks one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 05_silver_products.py.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 05_silver_products.py and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 10.F2.1 — Create the file databricks/notebooks/05_silver_products.py from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\databricks\notebooks\05_silver_products.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\databricks\notebooks\05_silver_products.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\notebooks\05_silver_products.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\databricks\notebooks\05_silver_products.py" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "databricks/notebooks/05_silver_products.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `databricks/notebooks`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `databricks/notebooks/05_silver_products.py`.
3. In PowerShell at the repository root, run `Test-Path ".\databricks\notebooks\05_silver_products.py"` and require True.
4. Run `git status --short -- "databricks/notebooks/05_silver_products.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for databricks/notebooks/05_silver_products.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `databricks/notebooks/05_silver_products.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 10.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Databricks notebook source
# Northstar Retail End-to-End Data Platform
# This file is a Git-friendly Databricks source notebook.
# Track A reads files that you manually uploaded to a Unity Catalog volume.
# It never attempts to reach SQL Server or PostgreSQL on localhost.
```



```

# COMMAND -----
# MAGIC %md
# MAGIC # 05 – Silver products
# MAGIC Conforms product, category, supplier, and latest cost information.

# COMMAND -----

from __future__ import annotations
from pyspark.sql import functions as F
from pyspark.sql.window import Window

for name, default in {
    "catalog": "northstar_retail", "bronze_schema": "bronze", "silver_schema": "silver",
    "quality_schema": "quality", "batch_id": "MANUAL-REPLACE-ME"
}.items(): dbutils.widgets.text(name, default)
catalog = dbutils.widgets.get("catalog")
bronze_schema = dbutils.widgets.get("bronze_schema")
silver_schema = dbutils.widgets.get("silver_schema")
quality_schema = dbutils.widgets.get("quality_schema")
batch_id = dbutils.widgets.get("batch_id")
if batch_id == "MANUAL-REPLACE-ME": raise ValueError("Pass batch_id from the workflow.")

def replace_batch(df, table_name: str):
    full = f"{catalog}`.`{silver_schema}`.`{table_name}`"
    if spark.catalog.tableExists(full.replace("`", "")):
        spark.sql(f"DELETE FROM {full} WHERE batch_id = '{batch_id}'")
    df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full)

def quarantine(df, rule_name: str, table_name: str):
    if df.limit(1).count():
        (df.withColumn("quality_rule", F.lit(rule_name))
         .withColumn("batch_id", F.lit(batch_id))
         .withColumn("quarantined_at_utc", F.current_timestamp())
         .write.format("delta").mode("append").option("mergeSchema", "true")
         .saveAsTable(f"{catalog}`.`{quality_schema}`.`{table_name}`"))

# COMMAND -----
products = spark.table(f"{catalog}`.`{bronze_schema}`.`erp_products`").filter(F.col("_batch_id") == batch_id)
categories = spark.table(f"{catalog}`.`{bronze_schema}`.`erp_product_categories`").filter(F.col("_batch_id") == batch_id)
suppliers = spark.table(f"{catalog}`.`{bronze_schema}`.`erp_suppliers`").filter(F.col("_batch_id") == batch_id)
costs = spark.table(f"{catalog}`.`{bronze_schema}`.`file_supplier_cost_updates`").filter(F.col("_batch_id") == batch_id)
latest_cost = (
    costs.withColumn("effective_date_parsed", F.to_date("effective_date"))
         .withColumn("rank", F.row_number().over(Window.partitionBy(F.upper(F.trim("sku"))).orderBy(F.col("effective_date_parsed").desc())))
         .filter(F.col("rank") == 1)
         .select(F.upper(F.trim("sku")).alias("cost_sku"), F.col("new_unit_cost").cast("decimal(12,2)").alias("latest_unit_cost"), "effective_date_parsed")
)
cleaned = (
    products.alias("p")
    .join(categories.alias("c"), F.col("p.category_id") == F.col("c.category_id"), "left")
    .join(suppliers.alias("s"), F.col("p.supplier_id") == F.col("s.supplier_id"), "left")
    .join(latest_cost.alias("lc"), F.upper(F.trim(F.col("p.sku"))) == F.col("lc.cost_sku"), "left")
    .select(
        F.col("p.product_id"), F.upper(F.trim(F.col("p.sku"))).alias("sku"),
        F.trim(F.col("p.product_name")).alias("product_name"), F.initcap(F.trim(F.col("c.category_name"))).alias("category_name"),
        F.trim(F.col("p.brand")).alias("brand"), F.upper(F.trim(F.col("s.supplier_code"))).alias("supplier_code"),
        F.coalesce(F.col("lc.latest_unit_cost"), F.col("p.unit_cost").cast("decimal(12,2)")).alias("unit_cost"),
        F.col("p.list_price").cast("decimal(12,2)").alias("list_price"), F.upper(F.trim(F.col("p.currency_code"))).alias("currency_code"),
        F.col("p.active_flag").cast("boolean").alias("active_flag"), F.col("p.updated_at"),
        F.lit(batch_id).alias("batch_id"), F.current_timestamp().alias("load_timestamp_utc"), F.lit("SQLSERVER_ERP").alias("source_system")
    )
)
invalid = cleaned.filter(F.col("sku").isNull() | (F.col("unit_cost") < 0) | (F.col("list_price") < 0))
quarantine(invalid, "INVALID_PRODUCT", "product_quarantine")
valid = cleaned.join(invalid.select("product_id"), "product_id", "left_anti")
replace_batch(valid, "products")
print(f"input: {products.count()}, 'valid': {valid.count()}, 'quarantined': {invalid.count()}")
dbutils.jobs.taskValues.set(key="silver_products_status", value="PASS")

```

Line-by-line explanation

- **Line 1:** # Databricks notebook source is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** # Northstar Retail End-to-End Data Platform is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** # This file is a Git-friendly Databricks source notebook. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** # Track A reads files that you manually uploaded to a Unity Catalog volume. is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 5:** `# It never attempts to reach SQL Server or PostgreSQL on localhost.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 9:** `# MAGIC %md` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 10:** `# MAGIC # 05 – Silver products` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 11:** `# MAGIC Conforms product, category, supplier, and latest cost information.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 14:** blank line used only to separate logical sections for readability.
- **Line 15:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `from pyspark.sql import functions as F` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `functions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** `from pyspark.sql.window import Window` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql.window` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Window` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** `for name, default in {` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** `"catalog": "northstar_retail", "bronze_schema": "bronze", "silver_schema": "silver",` — `"catalog"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"northstar_retail"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"silver_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"silver"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** `"quality_schema": "quality", "batch_id": "MANUAL-REPLACE-ME"` — `"quality_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quality"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "MANUAL-REPLACE-ME" is a quoted literal; the quotation marks preserve spaces or mark text

role is described in the surrounding explanation; "Pass batch_id from the workflow." is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 29:** blank line used only to separate logical sections for readability.
- **Line 30:** blank line used only to separate logical sections for readability.
- **Line 31:** `def replace_batch(df, table_name: str):` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `replace_batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 32:** `full = f"\{catalog}\.silver_schema\.\{table_name}"` — `full` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\{catalog}\.silver_schema\.\{table_name}"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 33:** `if spark.catalog.tableExists(full.replace("\", "")):` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.catalog.tableExists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full.replace` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `""` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `""` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** `spark.sql(f"DELETE FROM {full} WHERE batch_id = '{batch_id}'")` — `spark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"DELETE FROM {full} WHERE batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** `df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full)` — `df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"mergeSchema"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 36:** blank line used only to separate logical sections for readability.
- **Line 37:** blank line used only to separate logical sections for readability.
- **Line 38:** `def quarantine(df, rule_name: str, table_name: str):` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quarantine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rule_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 39:** `if df.limit(1).count():` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df.limit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 40:** `(df.withColumn("quality_rule", F.lit(rule_name)))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df.withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quality_rule"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rule_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** `.withColumn("batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** `.withColumn("quarantined_at_utc", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quarantined_at_utc"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 43:** `.write.format("delta").mode("append").option("mergeSchema", "true")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

"mergeSchema" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 44:** `.saveAsTable(f"\{catalog}\{quality_schema}\{table_name}")` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\{catalog}\{quality_schema}\{table_name}"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 45:** blank line used only to separate logical sections for readability.
- **Line 46:** blank line used only to separate logical sections for readability.
- **Line 47:** `# COMMAND` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 48:** `products = spark.table(f"\{catalog}\{bronze_schema}\erp_products").filter(F.col("_batch_id") == batch_id)` `products` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f"\{catalog}\{bronze_schema}\erp_products"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 49:** `categories = spark.table(f"\{catalog}\{bronze_schema}\erp_product_categories").filter(F.col("_batch_id") == batch_id)` `categories` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f"\{catalog}\{bronze_schema}\erp_product_categories"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 50:** `suppliers = spark.table(f"\{catalog}\{bronze_schema}\erp_suppliers").filter(F.col("_batch_id") == batch_id)` `suppliers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f"\{catalog}\{bronze_schema}\erp_suppliers"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 51:** `costs = spark.table(f"\{catalog}\.{bronze_schema}.file_supplier_cost_updates").filter(F.col("_batch_id") == batch_id)`—`costs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\{catalog}\.{bronze_schema}.file_supplier_cost_updates"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.`
- **Line 52:** `latest_cost = ( — latest_cost` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 53:** `costs.withColumn("effective_date_parsed", F.to_date("effective_date"))` — `costs.withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"effective_date_parsed"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.to_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"effective_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 54:** `.withColumn("rank", F.row_number().over(Window.partitionBy(F.upper(F.trim("sku"))).orderBy(F.col("effective_date_parsed").desc())))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"rank"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.row_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `over` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Window.partitionBy` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.upper` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 55:** `.filter(F.col("rank") == 1)` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"rank"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- Line 56:** `.select(F.upper(F.trim("sku")).alias("cost_sku"), F.col("new_unit_cost").cast("decimal(12,2)").alias("latest_unit_cost"), "effective_date_parsed")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; select is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.upper is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.trim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "sku" is a quoted literal; the quotation marks preserve spaces or mark text;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "cost_sku" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- Line 57:** `)` —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 58:** `cleaned =` (— cleaned is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 59:** `products.alias("p")` — products.alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "p" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 60:** `.join(categories.alias("c"), F.col("p.category_id") == F.col("c.category_id"), "left")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; join is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; categories.alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "c" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "p.category_id" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 61:** `.join(suppliers.alias("s"), F.col("p.supplier_id") == F.col("s.supplier_id"), "left")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; join is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; suppliers.alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "s" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "p.supplier_id" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 62:** `.join(latest_cost.alias("lc"), F.upper(F.trim(F.col("p.sku")))) == F.col("lc.cost_sku"), "left")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; join is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; latest_cost.alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "p.sku" is a quoted literal; the quotation marks preserve spaces or mark text;)) F.upper is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "lc.cost_sku" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

described in the surrounding explanation; "lc" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.upper is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.trim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 63:** .select(— . is a path, relative location, branch-style name, or file argument; verify every character before running it; select is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 64:** F.col("p.product_id"), F.upper(F.trim(F.col("p.sku"))).alias("sku"), — F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "p.product_id" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.upper is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.trim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "p.sku" is a quoted literal; the quotation marks preserve spaces or mark text;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 65:** F.trim(F.col("p.product_name")).alias("product_name"), F.initcap(F.trim(F.col("c.category_name"))).alias("category_name"), — F.trim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "p.product_name" is a quoted literal; the quotation marks preserve spaces or mark text;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "product_name" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.initcap is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 66:** F.trim(F.col("p.brand")).alias("brand"), F.upper(F.trim(F.col("s.supplier_code"))).alias("supplier_code"), — F.trim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "p.brand" is a quoted literal; the quotation marks preserve spaces or mark text;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "brand" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.upper is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 67:** F.coalesce(F.col("lc.latest_unit_cost"), F.col("p.unit_cost").cast("decimal(12,2)")).alias("unit_cost"), — F.coalesce is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "lc.latest_unit_cost" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "p.unit_cost" is a quoted literal; the quotation marks preserve spaces or mark text;). is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 68:** `F.col("p.list_price").cast("decimal(12,2)").alias("list_price"), F.up-`
`per(F.trim(F.col("p.currency_code"))).alias("currency_code"),` — `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"p.list_price"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"decimal(12,2)"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"list_price"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 69:** `F.col("p.active_flag").cast("boolean").alias("active_flag"), F.col("p.updated_at"),` — `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"p.active_flag"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"boolean"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"active_flag"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 70:** `F.lit(batch_id).alias("batch_id"), F.current_timestamp().alias("load_timestamp_utc"),`
`F.lit("SQLSERVER_ERP").alias("source_system")` — `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 71:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 72:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 73:** `invalid = cleaned.filter(F.col("sku").isNull() | (F.col("unit_cost") < 0) | (F.col("list_price") < 0))` — `invalid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cleaned.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"sku"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `isNull` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 74:** `quarantine(invalid, "INVALID_PRODUCT", "product_quarantine")` — `quarantine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `invalid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"INVALID_PRODUCT"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"product_quarantine"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 75:** `valid = cleaned.join(invalid.select("product_id"), "product_id", "left_anti")` — `valid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cleaned.join` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `invalid.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"product_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"product_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"left_anti"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 76:** `replace_batch(valid, "products")` — `replace_batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"products"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 77:** `print({"input": products.count(), "valid": valid.count(), "quarantined": invalid.count()})` — `print` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"input"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `products.count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"valid"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid.count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quarantined"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 78:** `dbutils.jobs.taskValues.set(key="silver_products_status", value="PASS")` — `dbutils.jobs.taskValues.set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "silver_products_status" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "PASS" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 10 lesson index](#)

File introduction — databricks/notebooks/06_silver_sales.py

Exact filename: 06_silver_sales.py

Filename extension: .py. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: databricks/notebooks relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 10.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in databricks/notebooks one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 06_silver_sales.py.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 06_silver_sales.py and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 10.F3.1 — Create the file databricks/notebooks/06_silver_sales.py from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\databricks\notebooks\06_silver_sales.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\databricks\notebooks\06_silver_sales.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\notebooks\06_silver_sales.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\databricks\notebooks\06_silver_sales.py" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "databricks/notebooks/06_silver_sales.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `databricks/notebooks`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `databricks/notebooks/06_silver_sales.py`.
3. In PowerShell at the repository root, run `Test-Path ".\databricks\notebooks\06_silver_sales.py"` and require True.
4. Run `git status --short -- "databricks/notebooks/06_silver_sales.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for databricks/notebooks/06_silver_sales.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `databricks/notebooks/06_silver_sales.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 10.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Databricks notebook source
# Northstar Retail End-to-End Data Platform
```

```

# This file is a Git-friendly Databricks source notebook.
# Track A reads files that you manually uploaded to a Unity Catalog volume.
# It never attempts to reach SQL Server or PostgreSQL on localhost.

# COMMAND -----
# MAGIC %md
# MAGIC # 06 – Silver sales, shipments, inventory, returns, and promotions

# COMMAND -----

from __future__ import annotations
from pyspark.sql import functions as F
from pyspark.sql.window import Window

for name, default in {
    "catalog": "northstar_retail", "bronze_schema": "bronze", "silver_schema": "silver",
    "quality_schema": "quality", "batch_id": "MANUAL-REPLACE-ME"
}.items(): dbutils.widgets.text(name, default)
catalog = dbutils.widgets.get("catalog")
bronze_schema = dbutils.widgets.get("bronze_schema")
silver_schema = dbutils.widgets.get("silver_schema")
quality_schema = dbutils.widgets.get("quality_schema")
batch_id = dbutils.widgets.get("batch_id")
if batch_id == "MANUAL-REPLACE-ME": raise ValueError("Pass batch_id from the workflow.")

def replace_batch(df, table_name: str):
    full = f"`{catalog}`.`{silver_schema}`.`{table_name}`"
    if spark.catalog.tableExists(full.replace("`", "")):
        spark.sql(f"DELETE FROM {full} WHERE batch_id = '{batch_id}'")
    df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full)

def quarantine(df, rule_name: str, table_name: str):
    if df.limit(1).count():
        (df.withColumn("quality_rule", F.lit(rule_name))
         .withColumn("batch_id", F.lit(batch_id))
         .withColumn("quarantined_at_utc", F.current_timestamp())
         .write.format("delta").mode("append").option("mergeSchema", "true")
         .saveAsTable(f"`{catalog}`.`{quality_schema}`.`{table_name}`"))

# COMMAND -----
orders = spark.table(f"`{catalog}`.`{bronze_schema}`.`erp_orders`").filter(F.col("_batch_id") == batch_id)
items = spark.table(f"`{catalog}`.`{bronze_schema}`.`erp_order_items`").filter(F.col("_batch_id") == batch_id)
products = spark.table(f"`{catalog}`.`{silver_schema}`.`products`").filter(F.col("batch_id") == batch_id)
valid_sales = (
    items.alias("i").join(orders.alias("o"), "order_id", "inner")
    .withColumn("quantity_int", F.col("quantity").cast("int"))
    .withColumn("unit_price_dec", F.col("unit_price").cast("decimal(12,2)"))
    .withColumn("unit_cost_dec", F.col("unit_cost").cast("decimal(12,2)"))
    .withColumn("discount_dec", F.col("discount_amount").cast("decimal(12,2)"))
    .withColumn("tax_dec", F.col("tax_amount").cast("decimal(12,2)"))
)
invalid_sales = valid_sales.filter((F.col("quantity_int") <= 0) | F.col("product_id").isNull())
quarantine(invalid_sales, "IMPOSSIBLE_QUANTITY_OR_PRODUCT", "sales_quarantine")
sales = (
    valid_sales.filter(F.col("quantity_int") > 0)
    .select(
        "order_item_id", "order_id", "line_number", "product_id", F.col("o.customer_id").alias("customer_id"),
        F.col("o.store_id").alias("store_id"), F.col("o.employee_id").alias("employee_id"), F.col("o.order_number").alias("order_number"),
        F.upper(F.trim(F.col("o.channel_code"))).alias("channel_code"), F.col("o.promotion_code").alias("promotion_code"),
        F.to_timestamp(F.col("o.order_timestamp")).alias("order_timestamp_utc"), F.to_date(F.col("o.order_timestamp")).alias("order_date"),
        "quantity_int", "unit_price_dec", "unit_cost_dec", "discount_dec", "tax_dec",
        (F.col("quantity_int") * F.col("unit_price_dec")).alias("gross_sales_amount"),
        (F.col("quantity_int") * F.col("unit_price_dec") - F.col("discount_dec")).alias("net_sales_amount"),
        (F.col("quantity_int") * F.col("unit_cost_dec")).alias("cost_of_goods_sold"),
        ((F.col("quantity_int") * F.col("unit_price_dec") - F.col("discount_dec")) - (F.col("quantity_int") *
        F.col("unit_cost_dec"))).alias("gross_profit_amount"),
        F.upper(F.trim(F.col("o.currency_code"))).alias("currency_code"),
        F.lit(batch_id).alias("batch_id"), F.current_timestamp().alias("load_timestamp_utc"), F.lit("SQLSERVER_ERP").alias("source_system")
    )
)
replace_batch(sales, "sales")

# Separate subject tables are cleaned with their own grains.
shipments = spark.table(f"`{catalog}`.`{bronze_schema}`.`erp_shipments`").filter(F.col("_batch_id") == batch_id)
shipments_clean = (shipments
    .withColumn("ship_date", F.to_date("ship_date"))
    .withColumn("promised_delivery_date", F.to_date("promised_delivery_date"))
    .withColumn("actual_delivery_date", F.to_date("actual_delivery_date"))
    .withColumn("on_time_flag", F.col("actual_delivery_date") <= F.col("promised_delivery_date"))
    .withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system",
    F.lit("SQLSERVER_ERP")))
replace_batch(shipments_clean, "shipments")

```

```

inventory = spark.table(f"`{catalog}`.`{bronze_schema}`.`erp_inventory_snapshots`).filter(F.col("_batch_id") == batch_id)
inventory_clean = (inventory
    .withColumn("snapshot_date", F.to_date("snapshot_date"))
    .withColumn("quantity_on_hand", F.col("quantity_on_hand").cast("int"))
    .withColumn("quantity_reserved", F.col("quantity_reserved").cast("int"))
    .withColumn("reorder_point", F.col("reorder_point").cast("int"))
    .withColumn("stockout_risk_flag", (F.col("quantity_on_hand") - F.col("quantity_reserved")) <= F.col("reorder_point"))
    .withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system",
F.lit("SQLSERVER_ERP")))
replace_batch(inventory_clean, "inventory_snapshots")

returns_raw = spark.table(f"`{catalog}`.`{bronze_schema}`.`file_returns`).filter(F.col("_batch_id") == batch_id)
parsed_return_date = F.coalesce(F.to_date("return_date", "yyyy-MM-dd"), F.to_date("return_date", "MM/dd/yyyy"), F.to_date("return_date", "dd-
MMM-yyyy"), F.to_date("return_date", "yyyy/MM/dd"))
returns_clean = (returns_raw
    .withColumn("return_date_parsed", parsed_return_date)
    .withColumn("return_quantity_int", F.col("return_quantity").cast("int"))
    .withColumn("refund_amount_dec", F.col("refund_amount").cast("decimal(12,2)"))
    .withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system",
F.lit("FILE_FEED")))
return_rejects = returns_clean.filter(F.col("return_date_parsed").isNull() | (F.col("return_quantity_int") <= 0))
quarantine(return_rejects, "INVALID_RETURN", "return_quarantine")
replace_batch(returns_clean.join(return_rejects.select("return_id"), "return_id", "left_anti"), "returns")

promotions_raw = spark.table(f"`{catalog}`.`{bronze_schema}`.`file_promotion_calendar`).filter(F.col("_batch_id") == batch_id)
promotions = (promotions_raw
    .withColumn("promotion_code", F.upper(F.trim("promotion_code")))
    .withColumn("start_date", F.to_date("start_date"))
    .withColumn("end_date", F.to_date("end_date"))
    .withColumn("discount_value", F.col("discount_value").cast("decimal(12,4)"))
    .withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system",
F.lit("FILE_FEED")))
replace_batch(promotions, "promotions")
print({"sales_input": items.count(), "sales_valid": sales.count(), "sales_quarantined": invalid_sales.count()})
dbutils.jobs.taskValues.set(key="silver_sales_status", value="PASS")

```

Line-by-line explanation

- **Line 1:** # Databricks notebook source is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** # Northstar Retail End-to-End Data Platform is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** # This file is a Git-friendly Databricks source notebook. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** # Track A reads files that you manually uploaded to a Unity Catalog volume. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** # It never attempts to reach SQL Server or PostgreSQL on localhost. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** # COMMAND ----- is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 9:** # MAGIC %md is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 10:** # MAGIC # 06 – Silver sales, shipments, inventory, returns, and promotions is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 11:** blank line used only to separate logical sections for readability.
- **Line 12:** # COMMAND ----- is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 13:** blank line used only to separate logical sections for readability.
- **Line 14:** from __future__ import annotations — from is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; __future__ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; import is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; annotations is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** from pyspark.sql import functions as F — from is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pyspark.sql is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; import is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; functions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; as is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

F is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 16:** `from pyspark.sql.window import Window` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql.window` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Window` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** blank line used only to separate logical sections for readability.
- **Line 18:** `for name, default in {` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** `"catalog": "northstar_retail", "bronze_schema": "bronze", "silver_schema": "silver",` — `"catalog"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"northstar_retail"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"silver_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"silver"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** `"quality_schema": "quality", "batch_id": "MANUAL-REPLACE-ME"` — `"quality_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quality"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"MANUAL-REPLACE-ME"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 21:** `}.items(): dbutils.widgets.text(name, default)` — `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** `catalog = dbutils.widgets.get("catalog")` — `catalog` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"catalog"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** `bronze_schema = dbutils.widgets.get("bronze_schema")` — `bronze_schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze_schema"` is a quoted literal; the quotation marks preserve spaces or mark

text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 24:** `silver_schema = dbutils.widgets.get("silver_schema")` — `silver_schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"silver_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** `quality_schema = dbutils.widgets.get("quality_schema")` — `quality_schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quality_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** `batch_id = dbutils.widgets.get("batch_id")` — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:** `if batch_id == "MANUAL-REPLACE-ME": raise ValueError("Pass batch_id from the workflow.")` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"MANUAL-REPLACE-ME"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ValueError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Pass batch_id from the workflow."` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** blank line used only to separate logical sections for readability.
- **Line 29:** blank line used only to separate logical sections for readability.
- **Line 30:** `def replace_batch(df, table_name: str):` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `replace_batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** `full = f"\{catalog}\.{silver_schema}\.{table_name}"` — `full` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\{catalog}\.{silver_schema}\.{table_name}"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 32:** `if spark.catalog.tableExists(full.replace("\", "")):` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.catalog.tableExists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; full.replace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; " " is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; " " is a quoted literal; the quotation marks preserve spaces or mark text;)) ` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 33:** `spark.sql(f"DELETE FROM {full} WHERE batch_id = '{batch_id}'")` — `spark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"DELETE FROM {full} WHERE batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** `df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full)` — `df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"mergeSchema"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 35:** blank line used only to separate logical sections for readability.
- **Line 36:** blank line used only to separate logical sections for readability.
- **Line 37:** `def quarantine(df, rule_name: str, table_name: str):` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quarantine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rule_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 38:** `if df.limit(1).count():` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df.limit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `)` . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( ) :` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 39:** `(df.withColumn("quality_rule", F.lit(rule_name)))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df.withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quality_rule"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier,

er, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rule_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 40:** `.withColumn("batch_id", F.lit(batch_id))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** `.withColumn("quarantined_at_utc", F.current_timestamp())` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "quarantined_at_utc" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.current_timestamp is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ()) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** `.write.format("delta").mode("append").option("mergeSchema", "true")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; write.format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "append" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "mergeSchema" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 43:** `.saveAsTable(f"\{catalog}\.{quality_schema}\.{table_name}")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "\{catalog}\.{quality_schema}\.{table_name}" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 44:** blank line used only to separate logical sections for readability.
- **Line 45:** blank line used only to separate logical sections for readability.
- **Line 46:** `# COMMAND` ----- is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 47:** `orders = spark.table(f"\{catalog}\.{bronze_schema}\.erp_orders").filter(F.col("_batch_id") == batch_id)` — orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spark.table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "\{catalog}\.{bronze_schema}\.erp_orders" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword,

argument, identifier, or literal whose project-specific role is described in the surrounding explanation;F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;"_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 48:** `items = spark.table(f"\{catalog}\.{bronze_schema}.erp_order_items").filter(F.col("_batch_id") == batch_id)`—`items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\{catalog}\.{bronze_schema}.erp_order_items"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 49:** `products = spark.table(f"\{catalog}\.{silver_schema}.products").filter(F.col("batch_id") == batch_id)`—`products` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\{catalog}\.{silver_schema}.products"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 50:** `valid_sales = ( — valid_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 51:** `items.alias("i").join(orders.alias("o"), "order_id", "inner")` — `items.alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"i"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `join` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders.alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"o"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 52:** `.withColumn("quantity_int", F.col("quantity").cast("int"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quantity_int"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose

argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "decimal(12,2)" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 57:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 58:** invalid_sales = valid_sales.filter((F.col("quantity_int") <= 0) | F.col("product_id").isNull()) — invalid_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid_sales.filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "quantity_int" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; <= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 59:** quarantine(invalid_sales, "IMPOSSIBLE_QUANTITY_OR_PRODUCT", "sales_quarantine") — quarantine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; invalid_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "IMPOSSIBLE_QUANTITY_OR_PRODUCT" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "sales_quarantine" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 60:** sales = (— sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 61:** valid_sales.filter(F.col("quantity_int") > 0) — valid_sales.filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "quantity_int" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; > is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 62:** .select(— . is a path, relative location, branch-style name, or file argument; verify every character before running it; select is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 63:** "order_item_id", "order_id", "line_number", "product_id", F.col("o.customer_id").alias("customer_id"), — "order_item_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "order_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "line_number" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "product_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "o.customer_id" is a quoted literal; the quota-

tion marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 64:** `F.col("o.store_id").alias("store_id"), F.col("o.employee_id").alias("employee_id"), F.col("o.order_number").alias("order_number"), — F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"o.store_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"store_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"o.employee_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 65:** `F.upper(F.trim(F.col("o.channel_code"))).alias("channel_code"), F.col("o.promotion_code").alias("promotion_code"), — F.upper` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"o.channel_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `)))` . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"channel_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 66:** `F.to_timestamp(F.col("o.order_timestamp")).alias("order_timestamp_utc"), F.to_date(F.col("o.order_timestamp")).alias("order_date"), — F.to_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"o.order_timestamp"` is a quoted literal; the quotation marks preserve spaces or mark text; `))` . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_timestamp_utc"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.to_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 67:** `"quantity_int", "unit_price_dec", "unit_cost_dec", "discount_dec", "tax_dec", — "quantity_int"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"unit_price_dec"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"unit_cost_dec"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"discount_dec"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"tax_dec"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 68:** `(F.col("quantity_int") * F.col("unit_price_dec")).alias("gross_sales_amount"), — (` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding

explanation; "quantity_int" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; * is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "unit_price_dec" is a quoted literal; the quotation marks preserve spaces or mark text;)). is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

in the surrounding explanation; "o.currency_code" is a quoted literal; the quotation marks preserve spaces or mark text;))) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "currency_code" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 73:** `F.lit(batch_id).alias("batch_id"), F.current_timestamp().alias("load_timestamp_utc"), F.lit("SQLSERVER_ERP").alias("source_system")` — `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 74:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 75:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 76:** `replace_batch(sales, "sales")` — `replace_batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"sales"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 77:** blank line used only to separate logical sections for readability.
- **Line 78:** `# Separate subject tables are cleaned with their own grains.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 79:** `shipments = spark.table(f"{catalog}`.`{bronze_schema}`.erp_shipments").filter(F.col("_batch_id") == batch_id)` — `shipments` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}`.`{bronze_schema}`.erp_shipments"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 80:** `shipments_clean = (shipments` — `shipments_clean` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `shipments` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 81:** `.withColumn("ship_date", F.to_date("ship_date"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "ship_date" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.to_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "ship_date" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 82:** .withColumn("promised_delivery_date", F.to_date("promised_delivery_date")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "promised_delivery_date" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.to_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "promised_delivery_date" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 83:** .withColumn("actual_delivery_date", F.to_date("actual_delivery_date")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "actual_delivery_date" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.to_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "actual_delivery_date" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 84:** .withColumn("on_time_flag", F.col("actual_delivery_date") <= F.col("promised_delivery_date")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "on_time_flag" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "actual_delivery_date" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; <= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 85:** .withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system", F.lit("SQLSERVER_ERP")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "load_timestamp_utc" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 86:** replace_batch(shipments_clean, "shipments") — replace_batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; shipments_clean is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;
 , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "shipments" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 87:** blank line used only to separate logical sections for readability.
- **Line 88:** `inventory = spark.table(f"\{catalog}\.{bronze_schema}\.erp_inventory_snapshots").filter(F.col("_batch_id") == batch_id)`—`inventory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}\.{bronze_schema}\.erp_inventory_snapshots"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 89:** `inventory_clean = (inventory — inventory_clean` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `inventory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 90:** `.withColumn("snapshot_date", F.to_date("snapshot_date"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"snapshot_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.to_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"snapshot_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 91:** `.withColumn("quantity_on_hand", F.col("quantity_on_hand").cast("int"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quantity_on_hand"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quantity_on_hand"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"int"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 92:** `.withColumn("quantity_reserved", F.col("quantity_reserved").cast("int"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quantity_reserved"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quantity_reserved"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, ar-

gument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "int" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 93:** `.withColumn("reorder_point", F.col("reorder_point").cast("int"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "reorder_point" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "reorder_point" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "int" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 94:** `.withColumn("stockout_risk_flag", (F.col("quantity_on_hand") - F.col("quantity_reserved"))) <= F.col("reorder_point"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "stockout_risk_flag" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "quantity_on_hand" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; - is a command option or flag; its exact behavior is defined beside the command; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 95:** `.withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system", F.lit("SQLSERVER_ERP"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "load_timestamp_utc" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 96:** `replace_batch(inventory_clean, "inventory_snapshots")` — replace_batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; inventory_clean is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "inventory_snapshots" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 97:** blank line used only to separate logical sections for readability.
- **Line 98:** `returns_raw = spark.table(f"{catalog}`.{bronze_schema}`.file_returns").filter(F.col("_batch_id") == batch_id)-returns_raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spark.table is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;fis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;""{catalog}`.`bronze_schema`.`file_returns`"is a quoted literal; the quotation marks preserve spaces or mark text;),is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;filteris a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;F.colis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 99:** `parsed_return_date = F.coalesce(F.to_date("return_date", "yyyy-MM-dd"), F.to_date("return_date", "MM/dd/yyyy"), F.to_date("return_date", "dd-MMM-yyyy"), F.to_date("return_date", "yyyy/MM/dd"))` — `parsed_return_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.coalesce` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.to_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"return_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"yyyy-MM-dd"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.to_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 100:** `returns_clean = (returns_raw — returns_clean` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `returns_raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 101:** `.withColumn("return_date_parsed", parsed_return_date)` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"return_date_parsed"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parsed_return_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 102:** `.withColumn("return_quantity_int", F.col("return_quantity").cast("int"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"return_quantity_int"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"return_quantity"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"int"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 103:** `.withColumn("refund_amount_dec", F.col("refund_amount").cast("decimal(12,2)"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"refund_amount_dec"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, ar-

gument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "refund_amount" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "decimal(12,2)" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 104:** `.withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system", F.lit("FILE_FEED"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "load_timestamp_utc" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 105:** `return_rejects = returns_clean.filter(F.col("return_date_parsed").isNull() | (F.col("return_quantity_int") <= 0))` — return_rejects is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; returns_clean.filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "return_date_parsed" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; isNull is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 106:** `quarantine(return_rejects, "INVALID_RETURN", "return_quarantine")` — quarantine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; return_rejects is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "INVALID_RETURN" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "return_quarantine" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.
- **Line 107:** `replace_batch(returns_clean.join(return_rejects.select("return_id"), "return_id", "left_anti"), "returns")` — replace_batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; returns_clean.join is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; return_rejects.select is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "return_id" is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "return_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "left_anti" is a quoted literal; the quotation marks

preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 108:** blank line used only to separate logical sections for readability.
- **Line 109:** `promotions_raw = spark.table(f"\{catalog}\.`{bronze_schema}`.file_promotion_calendar").filter(F.col("_batch_id") == batch_id)`—`promotions_raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\{catalog}\.`{bronze_schema}`.file_promotion_calendar"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 110:** `promotions = (promotions_raw — promotions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `promotions_raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 111:** `.withColumn("promotion_code", F.upper(F.trim("promotion_code")))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"promotion_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.upper` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"promotion_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `)))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 112:** `.withColumn("start_date", F.to_date("start_date"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"start_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.to_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"start_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 113:** `.withColumn("end_date", F.to_date("end_date"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"end_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.to_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"end_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 114:** `.withColumn("discount_value", F.col("discount_value").cast("decimal(12,4)"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword,

argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "discount_value" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "discount_value" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "decimal(12,4)" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 115:** `.withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system", F.lit("FILE_FEED"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "load_timestamp_utc" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 116:** `replace_batch(promotions, "promotions")` — replace_batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; promotions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "promotions" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 117:** `print({"sales_input": items.count(), "sales_valid": sales.count(), "sales_quarantined": invalid_sales.count()})` — print is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ({ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "sales_input" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; items.count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "sales_valid" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sales.count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "sales_quarantined" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 118:** `dbutils.jobs.taskValues.set(key="silver_sales_status", value="PASS")` — dbutils.jobs.taskValues.set is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "silver_sales_status" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "PASS" is a quoted literal; the quotation marks

preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 10 lesson index](#)

File introduction — databricks/notebooks/07_silver_digital.py

Exact filename: 07_silver_digital.py

Filename extension: .py. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: databricks/notebooks relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 10.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in databricks/notebooks one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 07_silver_digital.py.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 07_silver_digital.py and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 10.F4.1 — Create the file databricks/notebooks/07_silver_digital.py from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.

2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\databricks\notebooks\07_silver_digital.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\databricks\notebooks\07_silver_digital.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\notebooks\07_silver_digital.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\databricks\notebooks\07_silver_digital.py" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "databricks/notebooks/07_silver_digital.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `databricks/notebooks`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `databricks/notebooks/07_silver_digital.py`.
3. In PowerShell at the repository root, run `Test-Path ".\databricks\notebooks\07_silver_digital.py"` and require True.
4. Run `git status --short -- "databricks/notebooks/07_silver_digital.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for databricks/notebooks/07_silver_digital.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `databricks/notebooks/07_silver_digital.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 10.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Databricks notebook source
# Northstar Retail End-to-End Data Platform
# This file is a Git-friendly Databricks source notebook.
# Track A reads files that you manually uploaded to a Unity Catalog volume.
# It never attempts to reach SQL Server or PostgreSQL on localhost.

# COMMAND -----
# MAGIC %md
# MAGIC # 07 – Silver digital and marketing data

# COMMAND -----
```

```

from __future__ import annotations
from pyspark.sql import functions as F
from pyspark.sql.window import Window

for name, default in {
    "catalog": "northstar_retail", "bronze_schema": "bronze", "silver_schema": "silver",
    "quality_schema": "quality", "batch_id": "MANUAL-REPLACE-ME"
}.items(): dbutils.widgets.text(name, default)
catalog = dbutils.widgets.get("catalog")
bronze_schema = dbutils.widgets.get("bronze_schema")
silver_schema = dbutils.widgets.get("silver_schema")
quality_schema = dbutils.widgets.get("quality_schema")
batch_id = dbutils.widgets.get("batch_id")
if batch_id == "MANUAL-REPLACE-ME": raise ValueError("Pass batch_id from the workflow.")

def replace_batch(df, table_name: str):
    full = f"{catalog}`.`{silver_schema}`.`{table_name}`"
    if spark.catalog.tableExists(full.replace("`", "")):
        spark.sql(f"DELETE FROM {full} WHERE batch_id = '{batch_id}'")
    df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full)

def quarantine(df, rule_name: str, table_name: str):
    if df.limit(1).count():
        (df.withColumn("quality_rule", F.lit(rule_name))
         .withColumn("batch_id", F.lit(batch_id))
         .withColumn("quarantined_at_utc", F.current_timestamp())
         .write.format("delta").mode("append").option("mergeSchema", "true")
         .saveAsTable(f"{catalog}`.`{quality_schema}`.`{table_name}`"))

# COMMAND -----
users = spark.table(f"{catalog}`.`{bronze_schema}`.`digital_web_users`").filter(F.col("_batch_id") == batch_id)
sessions = spark.table(f"{catalog}`.`{bronze_schema}`.`digital_web_sessions`").filter(F.col("_batch_id") == batch_id)
events = spark.table(f"{catalog}`.`{bronze_schema}`.`digital_web_events`").filter(F.col("_batch_id") == batch_id)
campaigns = spark.table(f"{catalog}`.`{bronze_schema}`.`digital_campaigns`").filter(F.col("_batch_id") == batch_id)
spend = spark.table(f"{catalog}`.`{bronze_schema}`.`digital_marketing_spend`").filter(F.col("_batch_id") == batch_id)

clean_sessions = (sessions
    .withColumn("session_start_utc", F.to_utc_timestamp(F.to_timestamp("session_start_utc"), "UTC"))
    .withColumn("session_end_utc", F.to_utc_timestamp(F.to_timestamp("session_end_utc"), "UTC"))
    .withColumn("source_channel", F.initcap(F.trim("source_channel")))
    .withColumn("device_type", F.lower(F.trim("device_type")))
    .withColumn("duration_seconds", F.col("session_end_utc").cast("long") - F.col("session_start_utc").cast("long"))
    .withColumn("converted_flag", F.col("converted_order_id").isNull())
    .withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system", F.lit("POST-
GRES_DIGITAL")))

clean_events = (events
    .withColumn("event_timestamp_utc", F.to_utc_timestamp(F.to_timestamp("event_timestamp"), "UTC"))
    .withColumn("event_type", F.lower(F.trim("event_type")))
    .join(clean_sessions.select("session_id", "session_start_utc", "session_id", "left")
    .withColumn("out_of_order_flag", F.col("event_timestamp_utc") < F.col("session_start_utc"))
    .withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system", F.lit("POST-
GRES_DIGITAL")))
    quarantine(clean_events.filter(F.col("out_of_order_flag")), "OUT_OF_ORDER_EVENT", "digital_event_quarantine")
    replace_batch(clean_sessions, "web_sessions")
    replace_batch(clean_events.filter(~F.col("out_of_order_flag")), "web_events")

clean_campaigns = (campaigns
    .withColumn("campaign_code", F.upper(F.trim("campaign_code")))
    .withColumn("start_date", F.to_date("start_date"))
    .withColumn("end_date", F.to_date("end_date"))
    .withColumn("budget_amount", F.col("budget_amount").cast("decimal(14,2)"))
    .withColumn("currency_code", F.upper(F.trim("currency_code")))
    .withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system", F.lit("POST-
GRES_DIGITAL")))
    replace_batch(clean_campaigns, "campaigns")

clean_spend = (spend
    .withColumn("spend_date", F.to_date("spend_date"))
    .withColumn("channel", F.initcap(F.trim("channel")))
    .withColumn("spend_amount", F.col("spend_amount").cast("decimal(14,2)"))
    .withColumn("impressions", F.col("impressions").cast("long"))
    .withColumn("clicks", F.col("clicks").cast("long"))
    .withColumn("currency_code", F.upper(F.trim("currency_code")))
    .withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system", F.lit("POST-
GRES_DIGITAL")))
    spend_rejects = clean_spend.filter((F.col("spend_amount") < 0) | (~F.col("currency_code").isin("USD")))
    quarantine(spend_rejects, "NEGATIVE_OR_NON_USD_SPEND", "marketing_spend_quarantine")
    replace_batch(clean_spend.join(spend_rejects.select("spend_id", "spend_id", "left_anti"), "marketing_spend")
    print(f"Sessions: {clean_sessions.count()}, Events: {clean_events.count()}")
    clean_events.filter(F.col("out_of_order_flag")).count())
    dbutils.jobs.taskValues.set(key="silver_digital_status", value="PASS")

```

Line-by-line explanation

- **Line 1:** `# Databricks notebook source` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `# Northstar Retail End-to-End Data Platform` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** `# This file is a Git-friendly Databricks source notebook.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** `# Track A reads files that you manually uploaded to a Unity Catalog volume.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** `# It never attempts to reach SQL Server or PostgreSQL on localhost.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 9:** `# MAGIC %md` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 10:** `# MAGIC # 07 – Silver digital and marketing data` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 11:** blank line used only to separate logical sections for readability.
- **Line 12:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 13:** blank line used only to separate logical sections for readability.
- **Line 14:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** `from pyspark.sql import functions as F` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `functions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `from pyspark.sql.window import Window` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql.window` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Window` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** blank line used only to separate logical sections for readability.
- **Line 18:** `for name, default in {` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** `"catalog": "northstar_retail", "bronze_schema": "bronze", "silver_schema": "silver",` — `"catalog"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"northstar_retail"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"silver_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"sil-`

ver" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

- **Line 27:** `if batch_id == "MANUAL-REPLACE-ME": raise ValueError("Pass batch_id from the workflow.")` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"MANUAL-REPLACE-ME"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ValueError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Pass batch_id from the workflow."` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** blank line used only to separate logical sections for readability.
- **Line 29:** blank line used only to separate logical sections for readability.
- **Line 30:** `def replace_batch(df, table_name: str):` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `replace_batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** `full = f"{catalog}.{silver_schema}.{table_name}"` — `full` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.{silver_schema}.{table_name}"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 32:** `if spark.catalog.tableExists(full.replace("\", "")):` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.catalog.tableExists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full.replace` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `""` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** `spark.sql(f"DELETE FROM {full} WHERE batch_id = '{batch_id}'")` — `spark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"DELETE FROM {full} WHERE batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** `df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full)` — `df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"mergeSchema"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

explanation; "mergeSchema" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 35:** blank line used only to separate logical sections for readability.
- **Line 36:** blank line used only to separate logical sections for readability.
- **Line 37:** `def quarantine(df, rule_name: str, table_name: str):` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quarantine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rule_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 38:** `if df.limit(1).count():` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df.limit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.
- **Line 39:** `(df.withColumn("quality_rule", F.lit(rule_name)))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df.withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quality_rule"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rule_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.
- **Line 40:** `.withColumn("batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.
- **Line 41:** `.withColumn("quarantined_at_utc", F.current_timestamp())` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quarantined_at_utc"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.
- **Line 42:** `.write.format("delta").mode("append").option("mergeSchema", "true")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "append" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "mergeSchema" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 43:** `.saveAsTable(f"\{catalog}\.{quality_schema}\.{table_name}")`—is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\{catalog}\.{quality_schema}\.{table_name}"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 44:** blank line used only to separate logical sections for readability.
- **Line 45:** blank line used only to separate logical sections for readability.
- **Line 46:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 47:** `users = spark.table(f"\{catalog}\.{bronze_schema}\.digital_web_users").filter(F.col("_batch_id") == batch_id)`—`users` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\{catalog}\.{bronze_schema}\.digital_web_users"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 48:** `sessions = spark.table(f"\{catalog}\.{bronze_schema}\.digital_web_sessions").filter(F.col("_batch_id") == batch_id)`—`sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\{catalog}\.{bronze_schema}\.digital_web_sessions"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 49:** `events = spark.table(f"\{catalog}\.{bronze_schema}\.digital_web_events").filter(F.col("_batch_id") == batch_id)`—`events` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\{catalog}\.{bronze_schema}\.digital_web_events"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

project-specific role is described in the surrounding explanation;spark.tableis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;fis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;"{catalog}`.`bronze_schema`.`digital_web_events"is a quoted literal; the quotation marks preserve spaces or mark text;);is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;filteris a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;F.colis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;"_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 50:** campaigns = spark.table(f"\{catalog}`.`bronze_schema`.`digital_campaigns").filter(F.col("_batch_id") == batch_id)–campaignsis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;=is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;spark.tableis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;fis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;"{catalog}`.`bronze_schema`.`digital_campaigns"is a quoted literal; the quotation marks preserve spaces or mark text;);is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;filteris a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;F.colis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;"_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 51:** spend = spark.table(f"\{catalog}`.`bronze_schema`.`digital_marketing_spend").filter(F.col("_batch_id") == batch_id)–spendis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;=is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;spark.tableis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;fis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;"{catalog}`.`bronze_schema`.`digital_marketing_spend"is a quoted literal; the quotation marks preserve spaces or mark text;);is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;filteris a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;F.colis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;"_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 52:** blank line used only to separate logical sections for readability.
- **Line 53:** clean_sessions = (sessions — clean_sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 54:** .withColumn("session_start_utc", F.to_utc_timestamp(F.to_timestamp("session_start_utc"), "UTC")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "session_start_utc" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

ation; F.to_utc_timestamp is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.to_timestamp is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "session_start_utc" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "UTC" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 55:** .withColumn("session_end_utc", F.to_utc_timestamp(F.to_timestamp("session_end_utc"), "UTC")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "session_end_utc" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.to_utc_timestamp is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.to_timestamp is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "session_end_utc" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "UTC" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 56:** .withColumn("source_channel", F.initcap(F.trim("source_channel"))) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "source_channel" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.initcap is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.trim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "source_channel" is a quoted literal; the quotation marks preserve spaces or mark text;))) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 57:** .withColumn("device_type", F.lower(F.trim("device_type"))) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "device_type" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lower is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.trim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "device_type" is a quoted literal; the quotation marks preserve spaces or mark text;))) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 58:** .withColumn("duration_seconds", F.col("session_end_utc").cast("long") - F.col("session_start_utc").cast("long")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "duration_seconds" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "session_end_utc" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; "long" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 59:** `.withColumn("converted_flag", F.col("converted_order_id").isNotNull())` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "converted_flag" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "converted_order_id" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `isNotNull` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 60:** `.withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system", F.lit("POSTGRES_DIGITAL"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "load_timestamp_utc" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 61:** blank line used only to separate logical sections for readability.
- **Line 62:** `clean_events = (events —` `clean_events` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `events` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 63:** `.withColumn("event_timestamp_utc", F.to_utc_timestamp(F.to_timestamp("event_timestamp"), "UTC"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "event_timestamp_utc" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.to_utc_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.to_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "event_timestamp" is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "UTC" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 64:** `.withColumn("event_type", F.lower(F.trim("event_type")))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "event_type" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lower` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

"event_type" is a quoted literal; the quotation marks preserve spaces or mark text;))) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 65:** `.join(clean_sessions.select("session_id", "session_start_utc"), "session_id", "left")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; join is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; clean_sessions.select is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "session_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "session_start_utc" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "session_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "left" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 66:** `.withColumn("out_of_order_flag", F.col("event_timestamp_utc") < F.col("session_start_utc"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "out_of_order_flag" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "event_timestamp_utc" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; < is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 67:** `.withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system", F.lit("POSTGRES_DIGITAL"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "load_timestamp_utc" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 68:** `quarantine(clean_events.filter(F.col("out_of_order_flag")), "OUT_OF_ORDER_EVENT", "digital_event_quarantine")` — quarantine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; clean_events.filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "out_of_order_flag" is a quoted literal; the quotation marks preserve spaces or mark text;)) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "OUT_OF_ORDER_EVENT" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "digital_event_quarantine" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 69:** `replace_batch(clean_sessions, "web_sessions")` — `replace_batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `clean_sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web_sessions"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 70:** `replace_batch(clean_events.filter(~F.col("out_of_order_flag")), "web_events")` — `replace_batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `clean_events.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"out_of_order_flag"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"web_events"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 71:** blank line used only to separate logical sections for readability.
- **Line 72:** `clean_campaigns = (campaigns — clean_campaigns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `campaigns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 73:** `.withColumn("campaign_code", F.upper(F.trim("campaign_code")))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"campaign_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.upper` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"campaign_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 74:** `.withColumn("start_date", F.to_date("start_date"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"start_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.to_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"start_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 75:** `.withColumn("end_date", F.to_date("end_date"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"end_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.to_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"end_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 76:** `.withColumn("budget_amount", F.col("budget_amount").cast("decimal(14,2)"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "budget_amount" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "budget_amount" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "decimal(14,2)" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 77:** `.withColumn("currency_code", F.upper(F.trim("currency_code")))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "currency_code" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.upper is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.trim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "currency_code" is a quoted literal; the quotation marks preserve spaces or mark text;))) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 78:** `.withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system", F.lit("POSTGRES_DIGITAL"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "load_timestamp_utc" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 79:** `replace_batch(clean_campaigns, "campaigns")` — replace_batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; clean_campaigns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "campaigns" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 80:** blank line used only to separate logical sections for readability.
- **Line 81:** `clean_spend = (spend — clean_spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation`
- **Line 82:** `.withColumn("spend_date", F.to_date("spend_date"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "spend_date" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.to_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "spend_date" is a quoted literal; the

quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

fier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.trim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "currency_code" is a quoted literal; the quotation marks preserve spaces or mark text;))) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- Line 88:** `.withColumn("batch_id", F.lit(batch_id)).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("source_system", F.lit("POSTGRES_DIGITAL"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "load_timestamp_utc" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- Line 89:** `spend_rejects = clean_spend.filter((F.col("spend_amount") < 0) | (~F.col("currency_code").isin("USD")))` — spend_rejects is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; clean_spend.filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ((is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "spend_amount" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; < is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 90:** `quarantine(spend_rejects, "NEGATIVE_OR_NON_USD_SPEND", "marketing_spend_quarantine")` — quarantine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spend_rejects is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "NEGATIVE_OR_NON_USD_SPEND" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "marketing_spend_quarantine" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 91:** `replace_batch(clean_spend.join(spend_rejects.select("spend_id"), "spend_id", "left_anti"), "marketing_spend")` — replace_batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; clean_spend.join is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spend_rejects.select is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "spend_id" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "spend_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "left_anti" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 92:** `print({"sessions": clean_sessions.count(), "events": clean_events.count(), "event_quarantine": clean_events.filter(F.col("out_of_order_flag")).count()})` — `print` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{ }` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"sessions"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `clean_sessions.count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"events"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `clean_events.count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"event_quarantine"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 93:** `dbutils.jobs.taskValues.set(key="silver_digital_status", value="PASS")` — `dbutils.jobs.taskValues.set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"silver_digital_status"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `value` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 10 lesson index](#)

File introduction — `sql/databricks/silver/01_customer_quality_patterns.sql`

Exact filename: `01_customer_quality_patterns.sql`

Filename extension: `.sql`. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `sql/databricks/silver` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 10.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `sql/databricks/silver` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `01_customer_quality_patterns.sql`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `01_customer_quality_patterns.sql` and an empty file under the correct parent folder.
10. PRESS: `Ctrl+S` after adding each explained section.

Action 10.F5.1 — Create the file `sql/databricks/silver/01_customer_quality_patterns.sql` from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\databricks\silver\01_customer_quality_patterns.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\databricks\silver\01_customer_quality_patterns.sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\sql\databricks\silver\01_customer_quality_patterns.sql` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\sql\databricks\silver\01_customer_quality_patterns.sql" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/databricks/silver/01_customer_quality_patterns.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `sql/databricks/silver`.

2. Right-click the editor tab and choose **Copy Path**; compare the ending with `sql/databricks/silver/01_customer_quality_patterns.sql`.
3. In PowerShell at the repository root, run `Test-Path ".\sql\databricks\silver\01_customer_quality_patterns.sql"` and require True.
4. Run `git status --short -- "sql/databricks/silver/01_customer_quality_patterns.sql"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `sql/databricks/silver/01_customer_quality_patterns.sql`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: `sql/databricks/silver/01_customer_quality_patterns.sql`

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 10.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: Databricks SQL / Spark SQL
-- Tiny teaching example: clean and rank duplicate business keys.
WITH cleaned AS (
  SELECT
    upper(trim(customer_number)) AS customer_number,
    initcap(trim(first_name)) AS first_name,
    lower(trim(email)) AS email,
    updated_at,
    row_number() OVER (
      PARTITION BY upper(trim(customer_number))
      ORDER BY updated_at DESC, customer_id DESC
    ) AS business_key_rank
  FROM IDENTIFIER(:bronze_customer_table)
)
SELECT * FROM cleaned WHERE business_key_rank = 1;
```

Line-by-line explanation

- **Line 1:** `-- Dialect: Databricks SQL / Spark SQL` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `-- Tiny teaching example: clean and rank duplicate business keys.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** `WITH cleaned AS (` — `WITH` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cleaned` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `SELECT` — `SELECT` requests data without changing it unless combined with a modifying statement
- **Line 5:** `upper(trim(customer_number)) AS customer_number,` — `upper` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `initcap(trim(first_name)) AS first_name,` — `initcap` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `first_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `first_name` is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** lower(trim(email)) AS email, — lower is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; trim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; email is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; email is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** updated_at, — updated_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** row_number() OVER (— row_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; OVER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** PARTITION BY upper(trim(customer_number)) — PARTITION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; upper is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; trim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** ORDER BY updated_at DESC, customer_id DESC — ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; updated_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DESC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DESC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:**) AS business_key_rank —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; business_key_rank is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** FROM IDENTIFIER(:bronze_customer_table) — FROM identifies the source table or query; IDENTIFIER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze_customer_table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** SELECT * FROM cleaned WHERE business_key_rank = 1; — SELECT requests data without changing it unless combined with a modifying statement; * is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; cleaned is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; business_key_rank is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression

[Back to the Phase 10 lesson index](#)

File introduction — `sql/databricks/silver/02_sales_quality_patterns.sql`

Exact filename: `02_sales_quality_patterns.sql`

Filename extension: `.sql`. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `sql/databricks/silver` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 10.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `sql/databricks/silver` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `02_sales_quality_patterns.sql`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `02_sales_quality_patterns.sql` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 10.F6.1 — Create the file `sql/databricks/silver/02_sales_quality_patterns.sql` from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.

2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\databricks\silver\02_sales_quality_patterns.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\databricks\silver\02_sales_quality_patterns.sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\databricks\silver\02_sales_quality_patterns.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\sql\databricks\silver\02_sales_quality_patterns.sql" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/databricks/silver/02_sales_quality_patterns.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `sql/databricks/silver`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `sql/databricks/silver/02_sales_quality_patterns.sql`.
3. In PowerShell at the repository root, run `Test-Path ".\sql\databricks\silver\02_sales_quality_patterns.sql"` and require True.
4. Run `git status --short -- "sql/databricks/silver/02_sales_quality_patterns.sql"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `sql/databricks/silver/02_sales_quality_patterns.sql`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: `sql/databricks/silver/02_sales_quality_patterns.sql`

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 10.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: Databricks SQL / Spark SQL
SELECT
  order_item_id,
  CAST(quantity AS INT) AS quantity,
  CAST(unit_price AS DECIMAL(12,2)) AS unit_price,
  CAST(discount_amount AS DECIMAL(12,2)) AS discount_amount,
  CAST(quantity AS INT) * CAST(unit_price AS DECIMAL(12,2)) AS gross_sales_amount,
  CAST(quantity AS INT) * CAST(unit_price AS DECIMAL(12,2)) - CAST(discount_amount AS DECIMAL(12,2)) AS net_sales_amount
FROM IDENTIFIER(:bronze_order_item_table)
WHERE CAST(quantity AS INT) > 0;
```

Line-by-line explanation

- **Line 1:** `-- Dialect: Databricks SQL / Spark SQL` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `SELECT` — `SELECT` requests data without changing it unless combined with a modifying statement
- **Line 3:** `order_item_id,` — `order_item_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `CAST(quantity AS INT) AS quantity,` — `CAST` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quantity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quantity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `CAST(unit_price AS DECIMAL(12,2)) AS unit_price,` — `CAST` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `unit_price` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DECIMAL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `12` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `unit_price` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 6:** `CAST(discount_amount AS DECIMAL(12,2)) AS discount_amount,` — `CAST` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `discount_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DECIMAL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `12` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `discount_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 7:** `CAST(quantity AS INT) * CAST(unit_price AS DECIMAL(12,2)) AS gross_sales_amount,` — `CAST` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quantity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CAST` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `unit_price` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DECIM-`

AL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 8:** `CAST(quantity AS INT) * CAST(unit_price AS DECIMAL(12,2)) - CAST(discount_amount AS DECIMAL(12,2)) AS net_sales_amount` — CAST is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quantity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; * is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CAST is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; unit_price is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DECIMAL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 9:** `FROM IDENTIFIER(:bronze_order_item_table)` — FROM identifies the source table or query; IDENTIFIER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze_order_item_table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `WHERE CAST(quantity AS INT) > 0;` — WHERE filters rows to those meeting a condition; CAST is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quantity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; > is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression

[Back to the Phase 10 lesson index](#)

What your screen should look like now

- Every Phase 10 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 10.3 completion checklist

☐

`databricks/notebooks/04_silver_customers.py` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`databricks/notebooks/05_silver_products.py` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`databricks/notebooks/06_silver_sales.py` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`databricks/notebooks/07_silver_digital.py` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`sql/databricks/silver/01_customer_quality_patterns.sql` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`sql/databricks/silver/02_sales_quality_patterns.sql` was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 10.4](#) and recheck the root and branch.

Lesson 10.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Silver entity 1 — `silver.product_categories`

Bronze input: `bronze.erp_product_categories`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV1 — Build and test `silver.product_categories`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.product_categories`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `product_categories`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for product_categories.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.erp_product_categories")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
.option("replaceWhere", f"_batch_id = '{batch_id}'")
.saveAsTable(f"{catalog}.silver.product_categories"))
```

Line-by-line explanation

- **Line 1:** `source_df = spark.table(f"{catalog}.bronze.erp_product_categories")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_product_categories"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `"*",` — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}')` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"replaceWhere"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.product_categories"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.silver.product_categories"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.product_categories` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 2 — `silver.suppliers`

Bronze input: `bronze.erp_suppliers`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offsetting value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV2 — Build and test `silver.suppliers`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.suppliers`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `suppliers`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for `suppliers`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.erp_suppliers")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable(f"{catalog}.silver.suppliers"))
```

Line-by-line explanation

- Line 1:** `source_df = spark.table(f"{catalog}.bronze.erp_suppliers")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{catalog}.bronze.erp_suppliers` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `"*"`, — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}')` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"replaceWhere"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; `"_batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 10:** `.saveAsTable(f"{catalog}.silver.suppliers"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.silver.suppliers"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.suppliers` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 3 — `silver.products`

Bronze input: `bronze.erp_products`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV3 — Build and test `silver.products`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.products`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for products.

2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for products.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.erp_products")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable(f"{catalog}.silver.products"))
```

Line-by-line explanation

- **Line 1:** `source_df = spark.table(f"{catalog}.bronze.erp_products")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_products"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `"*",` — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}'")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"replaceWhere"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.products")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.silver.products"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.products` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 4 — `silver.stores`

Bronze input: `bronze.erp_stores`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV4 — Build and test `silver.stores`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.stores`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `stores`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for `stores`.

2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.erp_stores")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
```

```

)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
.option("replaceWhere", f"_batch_id = '{batch_id}'")
.saveAsTable(f"{catalog}.silver.stores"))

```

Line-by-line explanation

- **Line 1:** `source_df = spark.table(f"{catalog}.bronze.erp_stores")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_stores"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `"*"`, — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}')` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.stores"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.silver.stores"` is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.stores` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 5 — `silver.employees`

Bronze input: `bronze.erp_employees`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, of-fending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV5 — Build and test `silver.employees`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.employees`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `employees`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for `employees`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.erp_employees")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
.option("replaceWhere", f"_batch_id = '{batch_id}'")
.saveAsTable(f"{catalog}.silver.employees"))
```

Line-by-line explanation

- Line 1:** `source_df = spark.table(f"{catalog}.bronze.erp_employees")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_employees"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `"*"`, — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}'")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"replaceWhere"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.employees")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

fier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.silver.employees" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.employees` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 6 — `silver.customers`

Bronze input: `bronze.erp_customers`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV6 — Build and test `silver.customers`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.customers`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for customers.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for customers.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.erp_customers")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable(f"{catalog}.silver.customers"))
```

Line-by-line explanation

- Line 1:** `source_df = spark.table(f"{catalog}.bronze.erp_customers")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_customers"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `"*"`, — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}')` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"replaceWhere"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; `_batch_id = '{batch_id}'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 10:** `.saveAsTable(f"{catalog}.silver.customers"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.silver.customers"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.customers` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 7 — `silver.addresses`

Bronze input: `bronze.erp_addresses`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV7 — Build and test `silver.addresses`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.addresses`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for addresses.

2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for addresses.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.erp_addresses")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable(f"{catalog}.silver.addresses"))
```

Line-by-line explanation

- **Line 1:** `source_df = spark.table(f"{catalog}.bronze.erp_addresses")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_addresses"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `"*",` — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}'")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"replaceWhere"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.addresses")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.silver.addresses"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.addresses` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 8 — `silver.orders`

Bronze input: `bronze.erp_orders`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV8 — Build and test `silver.orders`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.orders`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for orders.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for orders.

2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.erp_orders")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
```



```

)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
.option("replaceWhere", f"_batch_id = '{batch_id}'")
.saveAsTable(f"{catalog}.silver.orders"))

```

Line-by-line explanation

- **Line 1:** `source_df = spark.table(f"{catalog}.bronze.erp_orders")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_orders"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `"*"`, — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}')` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.orders")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.silver.orders"` is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.orders` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 9 — `silver.order_items`

Bronze input: `bronze.erp_order_items`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, of-fending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV9 — Build and test `silver.order_items`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.order_items`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `order_items`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for `order_items`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.erp_order_items")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
.option("replaceWhere", f"_batch_id = '{batch_id}'")
.saveAsTable(f"{catalog}.silver.order_items"))
```

Line-by-line explanation

- Line 1:** `source_df = spark.table(f"{catalog}.bronze.erp_order_items")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_order_items"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `"*"` — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}'")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"replaceWhere"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.order_items")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

fier, or literal whose project-specific role is described in the surrounding explanation; "{cata-log}.silver.order_items" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.order_items` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 10 — `silver.payments`

Bronze input: `bronze.erp_payments`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV10 — Build and test `silver.payments`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.payments`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for payments.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.

7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for payments.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.erp_payments")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable(f"{catalog}.silver.payments"))
```

Line-by-line explanation

- Line 1:** `source_df = spark.table(f"{catalog}.bronze.erp_payments")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_payments"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `"*"`, — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `stand-`

ardized_df.filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** rejected_df = standardized_df.subtract(valid_df) — rejected_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; standardized_df.subtract is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** (valid_df.write.format("delta").mode("overwrite")) — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid_df.write.format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** .option("replaceWhere", f"_batch_id = '{batch_id}'") — . is a path, relative location, branch-style name, or file argument; verify every character before running it; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id = '{batch_id}'" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** .saveAsTable(f"{catalog}.silver.payments") — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.silver.payments" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.payments` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 11 — `silver.shipments`

Bronze input: `bronze.erp_shipments`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, of-fending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV11 — Build and test `silver.shipments`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.shipments`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for shipments.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for shipments.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.

7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.erp_shipments")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable(f"{catalog}.silver.shipments"))
```

Line-by-line explanation

- **Line 1:** `source_df = spark.table(f"{catalog}.bronze.erp_shipments")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_shipments"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `standardized_df = source_df.select( — standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `"*"`, — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}')` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id = '{batch_id}'" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.shipments")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.silver.shipments" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.shipments` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 12 — `silver.inventory_transactions`

Bronze input: `bronze.erp_inventory_transactions`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, of-fending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV12 — Build and test `silver.inventory_transactions`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.inventory_transactions`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `inventory_transactions`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for `inventory_transactions`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.erp_inventory_transactions")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
.option("replaceWhere", f"_batch_id = '{batch_id}'")
.saveAsTable(f"{catalog}.silver.inventory_transactions"))
```

Line-by-line explanation

- Line 1:** `source_df = spark.table(f"{catalog}.bronze.erp_inventory_transactions")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_inventory_transactions"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `"*"`, — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the

surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source_df.columns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "string" is a quoted literal; the quotation marks preserve spaces or mark text;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 5:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** valid_df = standardized_df.filter(F.col("_batch_id") == batch_id) — valid_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; standardized_df.filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** rejected_df = standardized_df.subtract(valid_df) — rejected_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; standardized_df.subtract is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** (valid_df.write.format("delta").mode("overwrite")) — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid_df.write.format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** .option("replaceWhere", f"_batch_id = '{batch_id}'") — . is a path, relative location, branch-style name, or file argument; verify every character before running it; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id = '{batch_id}'" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** .saveAsTable(f"{catalog}.silver.inventory_transactions") — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.silver.inventory_transactions" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.inventory_transactions` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 13 — `silver.inventory_snapshots`

Bronze input: `bronze.erp_inventory_snapshots`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV13 — Build and test `silver.inventory_snapshots`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.inventory_snapshots`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `inventory_snapshots`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for `inventory_snapshots`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.erp_inventory_snapshots")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
.option("replaceWhere", f"_batch_id = '{batch_id}'")
.saveAsTable(f"{catalog}.silver.inventory_snapshots"))
```

Line-by-line explanation

- Line 1:** `source_df = spark.table(f"{catalog}.bronze.erp_inventory_snapshots")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_inventory_snapshots"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `"*",` — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described

in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}')` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"replaceWhere"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.inventory_snapshots")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.silver.inventory_snapshots"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.inventory_snapshots` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 14 — `silver.web_users`

Bronze input: `bronze.digital_web_users`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, of-fending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV14 — Build and test `silver.web_users`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.web_users`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `web_users`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for `web_users`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.digital_web_users")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable(f"{catalog}.silver.web_users"))
```

Line-by-line explanation

- **Line 1:** `source_df = spark.table(f"{catalog}.bronze.digital_web_users")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the

surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.bronze.digital_web_users" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 2:** `standardized_df = source_df.select( — standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `"*"`, `— "*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key") — F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `) — )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id) — valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `rejected_df = standardized_df.subtract(valid_df) — rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite")) — (` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}')` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"replaceWhere"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.web_users")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.silver.web_users"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.web_users` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 15 — `silver.campaigns`

Bronze input: `bronze.digital_campaigns`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV15 — Build and test `silver.campaigns`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.campaigns`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for campaigns.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for campaigns.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.digital_campaigns")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
 .option("replaceWhere", f"_batch_id = '{batch_id}'")
 .saveAsTable(f"{catalog}.silver.campaigns"))
```

Line-by-line explanation

- Line 1:** `source_df = spark.table(f"{catalog}.bronze.digital_campaigns")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.digital_campaigns"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `"*"` , — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or ex-

pression;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "string" is a quoted literal; the quotation marks preserve spaces or mark text;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 5:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** valid_df = standardized_df.filter(F.col("_batch_id") == batch_id) — valid_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; standardized_df.filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** rejected_df = standardized_df.subtract(valid_df) — rejected_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; standardized_df.subtract is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** (valid_df.write.format("delta").mode("overwrite")) — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid_df.write.format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** .option("replaceWhere", f"_batch_id = '{batch_id}'") — . is a path, relative location, branch-style name, or file argument; verify every character before running it; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id = '{batch_id}'" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** .saveAsTable(f"{catalog}.silver.campaigns") — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.silver.campaigns" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]

- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.campaigns` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 16 — `silver.web_sessions`

Bronze input: `bronze.digital_web_sessions`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV16 — Build and test `silver.web_sessions`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.web_sessions`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `web_sessions`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for `web_sessions`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.digital_web_sessions")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
.option("replaceWhere", f"_batch_id = '{batch_id}'")
.saveAsTable(f"{catalog}.silver.web_sessions"))
```

Line-by-line explanation

- **Line 1:** `source_df = spark.table(f"{catalog}.bronze.digital_web_sessions")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.digital_web_sessions"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `"*",` — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}')` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"replaceWhere"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.web_sessions"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.silver.web_sessions"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.web_sessions` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 17 — `silver.web_events`

Bronze input: `bronze.digital_web_events`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offsetting value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV17 — Build and test `silver.web_events`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.web_events`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `web_events`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for `web_events`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.digital_web_events")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable(f"{catalog}.silver.web_events"))
```

Line-by-line explanation

- Line 1:** `source_df = spark.table(f"{catalog}.bronze.digital_web_events")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.digital_web_events"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `"*"`, — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}')` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"replaceWhere"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; `_batch_id = '{batch_id}'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 10:** `.saveAsTable(f"{catalog}.silver.web_events"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.silver.web_events"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.web_events` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 18 — `silver.campaign_touchpoints`

Bronze input: `bronze.digital_campaign_touchpoints`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV18 — Build and test `silver.campaign_touchpoints`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.campaign_touchpoints`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `campaign_touchpoints`.

2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for campaign_touchpoints.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.digital_campaign_touchpoints")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable(f"{catalog}.silver.campaign_touchpoints"))
```

Line-by-line explanation

- **Line 1:** `source_df = spark.table(f"{catalog}.bronze.digital_campaign_touchpoints")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.digital_campaign_touchpoints"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `"*"`, — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}'")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"replaceWhere"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.campaign_touchpoints"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.silver.campaign_touchpoints"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.campaign_touchpoints` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 19 — `silver.marketing_spend`

Bronze input: `bronze.digital_marketing_spend`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV19 — Build and test `silver.marketing_spend`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.marketing_spend`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `marketing_spend`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for `marketing_spend`.

2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.digital_marketing_spend")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
```



```

)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
.option("replaceWhere", f"_batch_id = '{batch_id}'")
.saveAsTable(f"{catalog}.silver.marketing_spend"))

```

Line-by-line explanation

- **Line 1:** `source_df = spark.table(f"{catalog}.bronze.digital_marketing_spend")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.digital_marketing_spend"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `"*"` , — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)` . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}')` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.marketing_spend"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.silver.marketing_spend"` is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.marketing_spend` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 20 — `silver.returns`

Bronze input: `bronze.file_returns`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, of-fending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV20 — Build and test `silver.returns`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver_returns`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for returns.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for returns.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.file_returns")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
.option("replaceWhere", f"_batch_id = '{batch_id}'")
.saveAsTable(f"{catalog}.silver.returns"))
```

Line-by-line explanation

- Line 1:** `source_df = spark.table(f"{catalog}.bronze.file_returns")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.file_returns"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `"*"`, — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}'")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"replaceWhere"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.returns")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

fier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.silver.returns" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.returns` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 21 — `silver.supplier_cost_updates`

Bronze input: `bronze.file_supplier_cost_updates`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV21 — Build and test `silver.supplier_cost_updates`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.supplier_cost_updates`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `supplier_cost_updates`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.

7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for `supplier_cost_updates`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.file_supplier_cost_updates")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable(f"{catalog}.silver.supplier_cost_updates"))
```

Line-by-line explanation

- Line 1:** `source_df = spark.table(f"{catalog}.bronze.file_supplier_cost_updates")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.file_supplier_cost_updates"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `"*"`, — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `stand-`

ardized_df.filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** rejected_df = standardized_df.subtract(valid_df) — rejected_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; standardized_df.subtract is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** (valid_df.write.format("delta").mode("overwrite")) — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid_df.write.format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** .option("replaceWhere", f"_batch_id = '{batch_id}'") — . is a path, relative location, branch-style name, or file argument; verify every character before running it; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id = '{batch_id}'" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** .saveAsTable(f"{catalog}.silver.supplier_cost_updates")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.silver.supplier_cost_updates" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.supplier_cost_updates` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 22 — `silver.promotion_calendar`

Bronze input: `bronze.file_promotion_calendar`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, of-fending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV22 — Build and test `silver.promotion_calendar`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.promotion_calendar`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `promotion_calendar`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for `promotion_calendar`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.

7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.file_promotion_calendar")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable(f"{catalog}.silver.promotion_calendar"))
```

Line-by-line explanation

- **Line 1:** `source_df = spark.table(f"{catalog}.bronze.file_promotion_calendar")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.file_promotion_calendar"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `standardized_df = source_df.select( — standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `"*"`, — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)]` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 9:** `option("replaceWhere", f"_batch_id = '{batch_id}')` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id = '{batch_id}'" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.promotion_calendar"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.silver.promotion_calendar" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.promotion_calendar` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 23 — `silver.shipment_tracking_events`

Bronze input: `bronze.file_shipment_tracking_events`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, of-fending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV23 — Build and test `silver.shipment_tracking_events`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.shipment_tracking_events`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `shipment_tracking_events`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for `shipment_tracking_events`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.file_shipment_tracking_events")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
.option("replaceWhere", f"_batch_id = '{batch_id}'")
.saveAsTable(f"{catalog}.silver.shipment_tracking_events"))
```

Line-by-line explanation

- Line 1:** `source_df = spark.table(f"{catalog}.bronze.file_shipment_tracking_events")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.file_shipment_tracking_events"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `"*"`, — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the

surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source_df.columns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "string" is a quoted literal; the quotation marks preserve spaces or mark text;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 5:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** valid_df = standardized_df.filter(F.col("_batch_id") == batch_id) — valid_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; standardized_df.filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** rejected_df = standardized_df.subtract(valid_df) — rejected_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; standardized_df.subtract is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** (valid_df.write.format("delta").mode("overwrite")) — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid_df.write.format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** .option("replaceWhere", f"_batch_id = '{batch_id}'") — . is a path, relative location, branch-style name, or file argument; verify every character before running it; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id = '{batch_id}'" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** .saveAsTable(f"{catalog}.silver.shipment_tracking_events")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.silver.shipment_tracking_events" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.shipment_tracking_events` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 24 — `silver.sales`

Bronze input: `bronze.erp_orders`, `bronze.erp_order_items`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV24 — Build and test `silver.sales`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.sales`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for `sales`.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for sales.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
orders_df = spark.table(f"{catalog}.bronze.erp_orders")
items_df = spark.table(f"{catalog}.bronze.erp_order_items")
source_df = items_df.join(orders_df, on=["order_id", "_batch_id"], how="inner")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
.option("replaceWhere", f"_batch_id = '{batch_id}'")
.saveAsTable(f"{catalog}.silver.sales"))
```

Line-by-line explanation

- **Line 1:** `orders_df = spark.table(f"{catalog}.bronze.erp_orders")` — `orders_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_orders"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `items_df = spark.table(f"{catalog}.bronze.erp_order_items")` — `items_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.erp_order_items"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `source_df = items_df.join(orders_df, on=["order_id", "_batch_id"], how="inner")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `items_df.join` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `on` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 4:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `"*"` , — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source_df.columns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression;]) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "string" is a quoted literal; the quotation marks preserve spaces or mark text;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 7:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** valid_df = standardized_df.filter(F.col("_batch_id") == batch_id) — valid_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; standardized_df.filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** rejected_df = standardized_df.subtract(valid_df) — rejected_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; standardized_df.subtract is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid_df is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** (valid_df.write.format("delta").mode("overwrite")) — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid_df.write.format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** .option("replaceWhere", f"_batch_id = '{batch_id}'") — . is a path, relative location, branch-style name, or file argument; verify every character before running it; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id = '{batch_id}'" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** .saveAsTable(f"{catalog}.silver.sales") — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.silver.sales" is a quoted

literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.sales` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

Silver entity 25 — `silver.promotions`

Bronze input: `bronze.file_promotion_calendar`.

Silver grain: One accepted, conformed record at the entity grain defined by the source-to-target mapping.

Cleaning responsibility: trim, cast, standardize, deduplicate, validate, and quarantine.

Rejected output: Entity-specific rejection/quarantine rows with `_batch_id`, business key when available, rule ID, reason, offending value, and rejection timestamp.

Business impact: Incorrect cleaning can duplicate revenue, misidentify customers, lose history, overstate inventory, misattribute marketing, or hide late shipments.

Action 10.4.SV25 — Build and test `silver.promotions`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and test `silver.promotions`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open the Silver notebook responsible for promotions.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require `input = accepted + rejected` unless the contract documents a separate deduplication accounting category.

8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open the Silver notebook responsible for promotions.
2. LOOK FOR: Display a dirty Bronze example before writing a cleaning rule.
3. TYPE THIS: Add one explicit rule for the problem, including a rejection reason when the row is not accepted.
4. PRESS: Run the rule against the current batch only.
5. LOOK FOR: Display accepted and rejected examples side by side.
6. PRESS: Count Bronze input, Silver accepted, and rejected rows.
7. LOOK FOR: Require input = accepted + rejected unless the contract documents a separate deduplication accounting category.
8. PRESS: Run edge-case tests for null, whitespace, malformed date, duplicate business key, negative amount/quantity, and unknown reference as applicable.
9. DO NOT CONTINUE UNTIL: Counts reconcile and the expected dirty fixture produces the expected outcome.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 10 lesson index](#)

Dirty input → rule → expected outputs

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Whitespace/case	padded or mixed-case code	trim and standardize governed code	normalized value	none when otherwise valid	one input becomes one accepted row

Scenario	Dirty input example	Rule	Accepted output	Rejected output	Count expectation
Invalid required value	null, malformed, or impossible value	validate business and type rule	none	original value plus reason	one input becomes one rejected row
Duplicate business key	two candidate rows	deterministic ordering and survivor rule	one survivor	duplicate/reconciliation record	input count remains explainable
Unknown reference	key absent from conformed parent	preserve for unknown-key handling or reject per contract	explicit unknown handling	monitored exception if policy requires	no silent row loss
Date boundary	end before start or impossible timestamp	compare parsed dates	corrected only when rule permits	otherwise reject	no coerced nonsense dates

Teaching code section

```
source_df = spark.table(f"{catalog}.bronze.file_promotion_calendar")
standardized_df = source_df.select(
    "*",
    F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")
)
valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)
rejected_df = standardized_df.subtract(valid_df)
(valid_df.write.format("delta").mode("overwrite")
.option("replaceWhere", f"_batch_id = '{batch_id}'")
.saveAsTable(f"{catalog}.silver.promotions"))
```

Line-by-line explanation

- Line 1:** `source_df = spark.table(f"{catalog}.bronze.file_promotion_calendar")` — `source_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.bronze.file_promotion_calendar"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `standardized_df = source_df.select(` — `standardized_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `"*"`, — `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `F.trim(F.col(source_df.columns[0]).cast("string")).alias("_example_standardized_key")` — `F.trim` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source_df.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"string"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 6:** `valid_df = standardized_df.filter(F.col("_batch_id") == batch_id)` — `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** `rejected_df = standardized_df.subtract(valid_df)` — `rejected_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized_df.subtract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `(valid_df.write.format("delta").mode("overwrite"))` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid_df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.option("replaceWhere", f"_batch_id = '{batch_id}')` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id = '{batch_id}'" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.saveAsTable(f"{catalog}.silver.promotions"))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.silver.promotions" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Edge-case test worksheet

- Actual Bronze rows: [ENTER COUNT]
- Accepted Silver rows: [ENTER COUNT]
- Rejected rows: [ENTER COUNT]
- Deduplicated-but-accounted rows: [ENTER COUNT OR 0]
- Reconciliation equation: [WRITE THE EQUATION AND RESULT]
- Controlled dirty fixture observed: [YES/NO + RULE ID]
- Rerun result for same batch: [ENTER RESULT]

Common error and recovery

If a cast turns invalid text into null, do not filter nulls without recording them. Add a pre-cast raw column or parse-valid flag, write a rejection reason, reconcile counts, and rerun the batch-scoped partition. If a duplicate rule chooses different survivors across runs, add deterministic tie-break columns and test the ordering.

Interview-ready explanation

"I built `silver.promotions` as a conformed entity with explicit acceptance and rejection rules. I demonstrated dirty input, validated edge cases, and reconciled every input row so cleaning could not silently remove data."

What your screen should look like now

- The new Phase 10 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 10.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 10.5](#).

Lesson 10.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 10 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Verification before automation

Question:

Should the phase prioritize a visible manual verification or jump directly to automation?

Options considered:

- Manual verification first, then automation.
- Automation without a known-good baseline.
- Manual operation forever.

Beginner path selected:

Establish one verified manual baseline and then automate repeatable checks.

Why this path was selected:

A known-good example makes failures interpretable and prevents automation from hiding a wrong assumption.

Tradeoff:

The first run takes longer.

What a company might do instead:

Teams often pair infrastructure as code and automated tests with controlled bootstrap and acceptance procedures.

How to explain this in an interview:

I established a verified baseline, then encoded the same contract in tests and orchestration so automation preserved meaning rather than just speed.

If you see Result A

Accepted plus rejected plus duplicate outcomes reconcile to Bronze input. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

A cleaning rule silently drops rows, changes grain, or produces an unexplained null/type result. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: Databricks notebooks, Catalog Explorer, and SQL/Python cells.

Window/panel: Show dirty input, accepted output, rejected output, and count reconciliation.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase10_build-the-silver-cleaning-and-conformance-layer_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 10's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 10 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 10.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.

3. **CLICK:** Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. **TYPE THIS:** Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. **PRESS:** Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. **LOOK FOR:** Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. **DO NOT CONTINUE UNTIL:** Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Action 10.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Action 10.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Interpret git status --short

- ?? means untracked: Git sees a new path but it is not staged.
- M means modified in the working tree but not staged.
- M means the staged version differs from the last commit.
- MM means a staged version exists and the file has additional unstaged edits.
- A means staged as a new file.
- D means deletion; stop and investigate unless deletion is deliberate.
- R means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 10.5.G4 — Inspect the unstaged diff for databricks/notebooks/04_silver_customers.py

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "databricks/notebooks/04_silver_customers.py"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** git diff -- "databricks/notebooks/04_silver_customers.py" — git calls the Git version-control program; diff is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -- is a command option or flag; its exact behavior is defined beside the command; "databricks/notebooks/

04_silver_customers.py" is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Action 10.5.G5 — Stage only the reviewed file databricks/notebooks/04_silver_customers.py

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "databricks/notebooks/04_silver_customers.py"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "databricks/notebooks/04_silver_customers.py"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"databricks/notebooks/04_silver_customers.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 10.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Action 10.5.G7 — Create the Phase 10 commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "feat: conform and quarantine silver entities"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "feat: conform and quarantine silver entities"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"feat: conform and quarantine silver entities"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Action 10.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command op-

tion or flag; its exact behavior is defined beside the command; --oneLine is a command option or flag; its exact behavior is defined beside the command; --decorate is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Action 10.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 10 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message `feat: conform and quarantine silver entities` and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 10.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 10 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

“Phase 10 created the verified boundary described by **Build the Silver cleaning and conformance layer**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed.”

Mini quiz

1. Multiple choice: Why does Phase 10 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** PLANNED, CREATED BY USER, and RUN BY USER describe earlier states; only inspected evidence supports verification.

My Phase 10 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 10.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 11 converts descriptive entities into durable dimensions.

Phase 10 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 11 — Build Gold dimensions, surrogate keys, and SCD behavior

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: Verified dimensions with unknown rows and effective dates.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 12 resolves dimension keys while building facts at declared grains.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 11 lesson index

1. [Lesson 11.1 — Understand the phase and starting screen](#)
2. [Lesson 11.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 11.3 — Create and understand the phase files](#)
4. [Lesson 11.4 — Build and validate required objects](#)
5. [Lesson 11.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 11.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** Databricks notebooks, Catalog Explorer, and SQL/Python cells.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.

- **Files that should not exist yet:** New Phase 11 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 10 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 11 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

Databricks notebooks, Catalog Explorer, and SQL/Python cells. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 11 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 11 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 11 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 11.1 completion checklist

☐

I read why Phase 11 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]



I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 11.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 11 lesson index](#)

Lesson 11.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 11 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required
Ownership status at start: Workbook instructions exist; your execution is not yet verified.
Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

Nine business dimensions: date, customer, product, store, employee, supplier, promotion, channel, and campaign. Customer/product preserve selected history with SCD Type 2; other dimensions use Type 1 where appropriate. Every dimension has a surrogate key, natural key, audit fields, and an unknown row.

2. Why it matters

Dimensions provide consistent descriptive context across facts. Surrogate keys separate warehouse identity from source identifiers; SCD2 preserves historical truth; unknown rows let facts load while unresolved context remains measurable.

3. Words to know

Term	Plain-language definition
Dimension grain	One row per current entity or per historical entity version.
Natural key	Business identifier from source, such as SKU.
Surrogate key	Warehouse key used by facts.
SCD Type 1	Overwrite correction; no prior version retained.
SCD Type 2	Expire old row and insert new effective-dated version.
Effective start/end	Date range when a dimension version is valid.
Current flag	Boolean marking open current version.
Tracked attribute	Column whose change creates a new Type 2 version.
Record hash	Hash of tracked attributes used to detect change.
Unknown member	Key 0 row for missing/invalid/late context.
Inferred member	Minimal placeholder created for a known natural key before full attributes arrive.
Overlap	Two versions valid for same key/date; invalid SCD state.
Event-time lookup	Fact joins the dimension version valid when event happened.

4. What must already be completed

Relevant Silver tables pass and natural keys are stable/unique. Date range requirements and Type 1/2 attribute decisions are documented.

5. Applications to open

Application	How to open it	Why it is open
Databricks notebook 08	Workspace.	Build dimensions and complete user-owned patterns.
Catalog Explorer / SQL editor	Gold schema.	Inspect keys/history/unknown rows.
VS Code	Notebook source, SCD SQL pattern, data dictionary.	Review implementation and tracked columns.
SCD diagram	docs/images/scd2_timeline.svg.	Explain version behavior visually.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run `Get-Location`; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
SELECT customer_number, COUNT(*) AS versions,	Finds keys with history and invalid current counts.	SCD2 must have exactly one current row; history itself is expected.
SELECT a.customer_number, a.customer_key AS a_key, b.customer_key AS b_key,	Detects overlapping SCD2 date ranges.	Overlap makes event-time lookup ambiguous.
SELECT 'dim_customer' AS table_name, COUNT(*) AS unknown_rows FROM northstar_retail.gold.dim_customer WHERE customer_key=0	Checks required unknown rows.	Fact foreign keys need a governed fallback.
version_key = (F.abs(F.xxhash64(F.concat_ws(" ", F.col(business_key), F.col("record_hash")))) + F.lit(1)).cast("long")	Creates deterministic surrogate key from natural key plus version hash.	Different historical versions must not reuse one key.

SCD2 worked example

Customer CUS-100001 is Silver tier from 2026-01-01 through 2026-05-31. On 2026-06-01 the source reports Gold. The old row remains with current false/end 2026-05-31; a new surrogate row starts 2026-06-01 and ends 9999-12-31/current true. A May sale joins Silver version; a July sale joins Gold version.

Full refresh versus incremental

`dim_date` can be regenerated deterministically. Small static channel mappings can be full-replaced in development. SCD2 dimensions require incremental change processing or carefully reconstructed full history; blindly overwriting loses version lineage.

MY TASK

Fix/complete version-specific surrogate key generation, add unknown members for all dimensions, run one controlled SCD2 change, and produce a dimension audit matrix with grain, SCD type, natural key, tracked fields, unknown policy, and tests.

STARTER SKELETON

```
def add_unknown_member(df, schema):
    # TODO(USER): create one row matching schema with key 0 and safe labels.
    unknown = spark.createDataFrame([_____], schema=schema)
    return unknown.unionByName(df)

def scd2_version_key(business_key: str):
    # TODO(USER): combine natural key with record_hash so versions differ.
    return (F.abs(F.xxhash64(F.concat_ws("||", F.col(business_key), _____))) + 1).cast("long")

# TODO(USER): MERGE unknown row once and test exactly one row with key 0.
```

Acceptance criteria: All nine dimensions exist; natural/surrogate keys and grains are valid; every dimension has one unknown row; controlled SCD2 change creates non-overlapping versions and unique keys; rerun does not duplicate current rows.

HINT 1 — CONCEPT

A Type 2 surrogate key identifies a version, not only an entity.

HINT 2 — PSEUDOCODE

Hash tracked fields first, then combine business key and record hash for deterministic version key.

HINT 3 — NEAR-COMPLETE PATTERN

Copy the SQL Server warehouse unknown-row values or data dictionary to build one schema-compatible row per dimension.

13. Expected files or output

- 9 Gold dimensions.
- Unknown key 0 in each.
- Customer/product history fields.
- One controlled multi-version example.
- Dimension audit/test matrix.
- Stable rerun results.

14. How to verify

- Unique surrogate keys.
- One current version/natural key.
- No overlaps/gaps policy documented.
- Unknown exactly once.
- Tracked-change creates new key/version; untracked correction follows stated Type 1/no-change behavior.
- Date dimension covers source event range.

15. Automated tests

- Surrogate and natural/current uniqueness.
- SCD overlap/current-count tests.
- Record hash change/no-change tests.
- Unknown member tests.
- Type 1 rerun tests.
- Date completeness/uniqueness.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
Duplicate surrogate key after SCD change	Key derived only from business key.	Group by surrogate key and inspect hashes/versions.	Derive version key from business key + record hash/version start.	Unique test passes.
Multiple current rows	Merge inserted changed row without expiring old or rerun duplicate.	Query current count per business key.	Expire matching current once; insert changed once; idempotent staging.	Exactly one current.
Date ranges overlap	End/start logic wrong or multiple changes same date.	Overlap self-join query.	Define date/timestamp granularity; end old before new; handle same-day changes.	Zero overlaps.
Unknown row duplicated	Unknown append on every run.	Count key 0.	MERGE/insert-if-not-exists or deterministic full replacement.	Exactly one.
Facts later map all to unknown	Natural keys absent/mismatched case or event-time condition.	Compare Silver natural keys to dimensions.	Standardize same codes and test lookup match rate before facts.	Unknown rate within threshold.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why can one customer have multiple surrogate keys?
2. What determines tracked attributes?
3. What is wrong with overlapping SCD date ranges?
4. When would you use unknown versus inferred member?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. What two inputs should a deterministic SCD2 version key include?
2. How many current rows should one business key have?
3. Why must fact lookup include event date?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture one controlled customer/product history with old/new keys/dates/current flags, plus unknown-member and overlap test results.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
feat: build gold dimensions with scd2 and unknown members
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

All dimension grains stated — [STATUS]

☐

Type 1 dimensions verified — [STATUS]

☐

Customer/product SCD2 verified — [STATUS]

☐

Version keys unique — [STATUS]

☐

Unknown rows complete — [STATUS]

☐

Overlap/current tests pass — [STATUS]

☐

Late-arriving policy documented — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-11. The quiz answer is in the answer-key section immediately after the quiz in this workbook#phase-11.

Next phase: Phase 12 resolves those dimensions into six facts and validates additive/non-additive measures.

Required action cards

Action 11.2.1 — Use a toy employee department example: Type 1 fixes a misspelling; Type 2 preserves a department transfer. Dec

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Use a toy employee department example: Type 1 fixes a misspelling; Type 2 preserves a department transfer. Dec. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Use a toy employee department example: Type 1 fixes a misspelling; Type 2 preserves a department transfer. Decide which question requires history before reading final code.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 1

Action

Use a toy employee department example: Type 1 fixes a misspelling; Type 2 preserves a department transfer. Decide which question requires history before reading final code.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Action 11.2.2 — State each dimension grain. `dim_customer` and `dim_product`: one row per natural key per effective historical

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

State each dimension grain. `dim_customer` and `dim_product`: one row per natural key per effective historical. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: State each dimension grain. `dim_customer` and `dim_product`: one row per natural key per effective historical version. Type 1 dimensions: one row per natural key.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 2****Action**

State each dimension grain. `dim_customer` and `dim_product`: one row per natural key per effective historical version. Type 1 dimensions: one row per natural key.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Action 11.2.3 — Open 08_gold_dimensions. Inspect deterministic key helper and Type 1 merge.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open 08_gold_dimensions. Inspect deterministic key helper and Type 1 merge. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open 08_gold_dimensions. Inspect deterministic key helper and Type 1 merge.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 3

Action

Open 08_gold_dimensions. Inspect deterministic key helper and Type 1 merge.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Action 11.2.4 — Correct/complete SCD2 surrogate generation so a new record hash/version receives a new surrogate key; a busine

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Correct/complete SCD2 surrogate generation so a new record hash/version receives a new surrogate key; a busine. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Correct/complete SCD2 surrogate generation so a new record hash/version receives a new surrogate key; a business-key-only hash is not sufficient because versions would collide.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 4****Action**

Correct/complete SCD2 surrogate generation so a new record hash/version receives a new surrogate key; a business-key-only hash is not sufficient because versions would collide.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Action 11.2.5 — Review tracked attributes: customer name/email/state/tier and product name/category/brand/supplier/cost/price.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Review tracked attributes: customer name/email/state/tier and product name/category/brand/supplier/cost/price. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Review tracked attributes: customer name/email/state/tier and product name/category/brand/supplier/cost/price. Decide whether every listed change truly needs history and document tradeoff.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 5

Action

Review tracked attributes: customer name/email/state/tier and product name/category/brand/supplier/cost/price. Decide whether every listed change truly needs history and document tradeoff.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Action 11.2.6 — Build `dim_date` with one row per date and integer YYYYMMDD key; verify date range covers all facts plus fut

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build `dim_date` with one row per date and integer YYYYMMDD key; verify date range covers all facts plus fut. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Build `dim_date` with one row per date and integer YYYYMMDD key; verify date range covers all facts plus future reporting need.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.

6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 6

Action

Build `dim_date` with one row per date and integer `YYYYMMDD` key; verify date range covers all facts plus future reporting need.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Action 11.2.7 — Run Type 1 dimensions and check natural-key uniqueness.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run Type 1 dimensions and check natural-key uniqueness. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run Type 1 dimensions and check natural-key uniqueness.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 7****Action**

Run Type 1 dimensions and check natural-key uniqueness.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Action 11.2.8 — Run SCD2 initial load. Insert unknown row key 0 into every dimension using a rerun-safe MERGE/union policy.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run SCD2 initial load. Insert unknown row key 0 into every dimension using a rerun-safe MERGE/union policy. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run SCD2 initial load. Insert unknown row key 0 into every dimension using a rerun-safe MERGE/union policy.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 8

Action

Run SCD2 initial load. Insert unknown row key 0 into every dimension using a rerun-safe MERGE/union policy.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Action 11.2.9 — Create a controlled SCD2 test: in a copied Silver test row, change one tracked customer tier or product cost,

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Create a controlled SCD2 test: in a copied Silver test row, change one tracked customer tier or product cost, . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Create a controlled SCD2 test: in a copied Silver test row, change one tracked customer tier or product cost, run a new batch/date, and verify old row expires and new row starts without overlap.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 9****Action**

Create a controlled SCD2 test: in a copied Silver test row, change one tracked customer tier or product cost, run a new batch/date, and verify old row expires and new row starts without overlap.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Action 11.2.10 — Test exactly one current row per natural key, non-overlapping dates, unique surrogate key, and unknown row pre

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Test exactly one current row per natural key, non-overlapping dates, unique surrogate key, and unknown row pre. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Test exactly one current row per natural key, non-overlapping dates, unique surrogate key, and unknown row presence.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 10

Action

Test exactly one current row per natural key, non-overlapping dates, unique surrogate key, and unknown row presence.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Action 11.2.11 — Document late-arriving policy: facts use 0 initially or an inferred member; choose one for Track A and how rep

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Document late-arriving policy: facts use 0 initially or an inferred member; choose one for Track A and how rep. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Document late-arriving policy: facts use 0 initially or an inferred member; choose one for Track A and how repair will update facts.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.

6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 11

Action

Document late-arriving policy: facts use 0 initially or an inferred member; choose one for Track A and how repair will update facts.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Action 11.2.12 — Do not use current dimension row for historical fact lookup; Phase 12 will join by natural key plus event date

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Do not use current dimension row for historical fact lookup; Phase 12 will join by natural key plus event date. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Do not use current dimension row for historical fact lookup; Phase 12 will join by natural key plus event date.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 12

Action

Do not use current dimension row for historical fact lookup; Phase 12 will join by natural key plus event date.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Action 11.2.C1 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT customer_number, COUNT(*) AS versions,
       SUM(CASE WHEN is_current THEN 1 ELSE 0 END) AS current_versions
FROM northstar_retail.gold.dim_customer
GROUP BY customer_number
HAVING current_versions <> 1 OR versions > 1
ORDER BY versions DESC;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT customer_number, COUNT(*) AS versions,` — `SELECT` requests data without changing it unless combined with a modifying statement; `customer_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `versions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `SUM(CASE WHEN is_current THEN 1 ELSE 0 END) AS current_versions` — `SUM` adds numeric values in the current filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is_current` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `ELSE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `END` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 3:** `FROM northstar_retail.gold.dim_customer` — `FROM` identifies the source table or query; `northstar_retail.gold.dim_customer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `GROUP BY customer_number` — `GROUP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `HAVING current_versions <> 1 OR versions > 1` — `HAVING` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `current_versions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<>` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `OR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `versions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression
- **Line 6:** `ORDER BY versions DESC;` — `ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `versions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DESC` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Action 11.2.C2 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.

2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT a.customer_number, a.customer_key AS a_key, b.customer_key AS b_key,
       a.effective_start_date, a.effective_end_date,
       b.effective_start_date, b.effective_end_date
FROM northstar_retail.gold.dim_customer a
JOIN northstar_retail.gold.dim_customer b
  ON a.customer_number=b.customer_number
 AND a.customer_key < b.customer_key
 AND a.effective_start_date <= b.effective_end_date
 AND b.effective_start_date <= a.effective_end_date;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT a.customer_number, a.customer_key AS a_key, b.customer_key AS b_key,` — `SELECT` requests data without changing it unless combined with a modifying statement; `a.customer_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a.customer_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `b.customer_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `b_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `a.effective_start_date, a.effective_end_date,` — `a.effective_start_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a.effective_end_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `b.effective_start_date, b.effective_end_date` — `b.effective_start_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `b.effective_end_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `FROM northstar_retail.gold.dim_customer a` — `FROM` identifies the source table or query; `northstar_retail.gold.dim_customer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `JOIN northstar_retail.gold.dim_customer b` — `JOIN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `northstar_retail.gold.dim_customer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

b is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 6:** `ON a.customer_number=b.customer_number` — `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a.customer_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `b.customer_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `AND a.customer_key < b.customer_key` — `AND` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a.customer_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `b.customer_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `AND a.effective_start_date <= b.effective_end_date` — `AND` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a.effective_start_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `b.effective_end_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `AND b.effective_start_date <= a.effective_end_date;` — `AND` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `b.effective_start_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a.effective_end_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Action 11.2.C3 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT 'dim_customer' AS table_name, COUNT(*) AS unknown_rows FROM northstar_retail.gold.dim_customer WHERE customer_key=0
UNION ALL SELECT 'dim_product', COUNT(*) FROM northstar_retail.gold.dim_product WHERE product_key=0
UNION ALL SELECT 'dim_store', COUNT(*) FROM northstar_retail.gold.dim_store WHERE store_key=0;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** SELECT 'dim_customer' AS table_name, COUNT(*) AS unknown_rows FROM northstar_retail.gold.dim_customer WHERE customer_key=0 — SELECT requests data without changing it unless combined with a modifying statement; 'dim_customer' is a quoted literal; the quotation marks preserve spaces or mark text; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; unknown_rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; northstar_retail.gold.dim_customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; remaining punctuation and identifiers continue the same statement.

- **Line 2:** UNION ALL SELECT 'dim_product', COUNT(*) FROM northstar_retail.gold.dim_product WHERE product_key=0 — UNION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ALL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 'dim_product' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; northstar_retail.gold.dim_product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; product_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 3:** UNION ALL SELECT 'dim_store', COUNT(*) FROM northstar_retail.gold.dim_store WHERE store_key=0; — UNION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ALL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 'dim_store' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; northstar_retail.gold.dim_store is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; store_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Action 11.2.C4 — Databricks notebook cell

Application:

Databricks workspace in a web browser

Window or panel:

Databricks notebook cell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Python instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Python):

```
version_key = (F.abs(F.xxhash64(F.concat_ws("||", F.col("business_key"), F.col("record_hash")))) + F.lit(1)).cast("long")
```

Where to enter it:

Databricks notebook cell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `version_key = (F.abs(F.xxhash64(F.concat_ws("||", F.col("business_key"), F.col("record_hash")))) + F.lit(1)).cast("long")` — `version_key` is a name, keyword, argument, identifier, or literal whose project-specific role

is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.abs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.xxhash64 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.concat_ws is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; " | | " is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 11.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 11 starting-screen checklist and re-verify the active context.

Lesson 11.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 11 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — databricks/

Exact folder name: databricks

Exact parent: repository root

Why it is needed now: Phase 11 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: databricks.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 11.D1.1 — Create the folder databricks

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\databricks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\databricks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\databricks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\databricks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\databricks" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/

Exact folder name: `sql`

Exact parent: `repository root`

Why it is needed now: Phase 11 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `repository root` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `sql`.
7. PRESS: Enter.

8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 11.D2.1 — Create the folder sql

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — databricks/notebooks/

Exact folder name: `notebooks`

Exact parent: `databricks`

Why it is needed now: Phase 11 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path databricks one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: notebooks.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 11.D3.1 — Create the folder databricks/notebooks

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\databricks\notebooks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\databricks\notebooks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\notebooks"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\databricks\notebooks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\databricks\notebooks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run Test-Path ".\databricks\notebooks" -PathType Container.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/databricks/

Exact folder name: databricks

Exact parent: sql

Why it is needed now: Phase 11 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path sql one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: databricks.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 11.D4.1 — Create the folder sql/databricks

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\databricks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\databricks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\sql\databricks` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\databricks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\databricks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\databricks" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/databricks/gold/

Exact folder name: gold

Exact parent: sql/databricks

Why it is needed now: Phase 11 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path sql/databricks one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: gold.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method**Action 11.D5.1 — Create the folder sql/databricks/gold**

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\databricks\gold" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\databricks\gold" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\sql\databricks\gold` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\databricks\gold" -PathType Container` and require True.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\databricks\gold" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\databricks\gold" -PathType Container`.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — databricks/notebooks/08_gold_dimensions.py

Exact filename: `08_gold_dimensions.py`

Filename extension: `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `databricks/notebooks` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 11.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `databricks/notebooks` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.

6. CLICK: Select **New File**.
7. TYPE THIS: `08_gold_dimensions.py`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `08_gold_dimensions.py` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 11.F1.1 — Create the file `databricks/notebooks/08_gold_dimensions.py` from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\databricks\notebooks\08_gold_dimensions.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\databricks\notebooks\08_gold_dimensions.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\notebooks\08_gold_dimensions.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\databricks\notebooks\08_gold_dimensions.py" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "databricks/notebooks/08_gold_dimensions.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.

5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use TODO(USER) where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under databricks/notebooks.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with databricks/notebooks/08_gold_dimensions.py.
3. In PowerShell at the repository root, run Test-Path ".\databricks\notebooks\08_gold_dimensions.py" and require True.
4. Run git status --short -- "databricks/notebooks/08_gold_dimensions.py" to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for databricks/notebooks/08_gold_dimensions.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: databricks/notebooks/08_gold_dimensions.py

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 11.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Databricks notebook source
# Northstar Retail End-to-End Data Platform
# This file is a Git-friendly Databricks source notebook.
# Track A reads files that you manually uploaded to a Unity Catalog volume.
# It never attempts to reach SQL Server or PostgreSQL on localhost.

# COMMAND -----
# MAGIC %md
# MAGIC # 08 – Gold dimensions
# MAGIC Builds unknown rows, Type 1 dimensions, and SCD Type 2 customer/product dimensions.

# COMMAND -----
from __future__ import annotations
from datetime import date
from delta.tables import DeltaTable
from pyspark.sql import functions as F
from pyspark.sql.window import Window

for name, default in {"catalog": "northstar_retail", "bronze_schema": "bronze", "silver_schema": "silver", "gold_schema": "gold", "batch_id": "MANUAL-REPLACE-ME"}.items(): dbutils.widgets.text(name, default)
catalog=dbutils.widgets.get("catalog"); bronze_schema=dbutils.widgets.get("bronze_schema"); silver_schema=dbutils.widgets.get("silver_schema"); gold_schema=dbutils.widgets.get("gold_schema"); batch_id=dbutils.widgets.get("batch_id")
if batch_id=="MANUAL-REPLACE-ME": raise ValueError("Pass batch_id from the workflow.")

def stable_positive_key(column_name: str):
    # xxhash64 is deterministic across reruns. Absolute value keeps keys positive; 0 is reserved for Unknown.
    return (F.abs(F.xxhash64(F.col(column_name))) + F.lit(1)).cast("long")

def type1_upsert(source, table_name: str, business_key: str):
    target_name=f"{catalog}.{gold_schema}.{table_name}"
    if not spark.catalog.tableExists(target_name):
        source.write.format("delta").mode("overwrite").saveAsTable(target_name)
        return
    target=DeltaTable.forName(spark,target_name)
    (target.alias("t").merge(source.alias("s"), f"t.{business_key}=s.{business_key}")
     .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute())

def scd2_merge(source, table_name: str, business_key: str, hash_columns: list[str], surrogate_key: str):
    """SCD2 teaching pattern: expire changed current rows, then insert new versions."""
    target_name=f"{catalog}.{gold_schema}.{table_name}"
    hash_expr=F.sha2(F.concat_ws("||", *[F.coalesce(F.col(c).cast("string"),F.lit("<NULL>")) for c in hash_columns]),256)
    staged=(source.withColumn("record_hash",hash_expr)
              .withColumn("effective_start_date",F.current_date())
              .withColumn("effective_end_date",F.lit("9999-12-31").cast("date"))
              .withColumn("is_current",F.lit(True))
              .withColumn(surrogate_key, (F.abs(F.xxhash64(F.col(business_key), F.col("record_hash"), F.col("batch_id")))) +
F.lit(1)).cast("long")))
    if not spark.catalog.tableExists(target_name):
        staged.write.format("delta").mode("overwrite").saveAsTable(target_name); return
    target=DeltaTable.forName(spark,target_name)
    current=spark.table(target_name).filter("is_current=true").select(business_key,"record_hash")
```

```

changed=(staged.alias("s").join(current.alias("t"),business_key,"left")
    .filter(F.col("t.record_hash").isNull() | (F.col("s.record_hash") != F.col("t.record_hash"))
    .select("s.*"))
(target.alias("t")).merge(changed.alias("s"),f"t.{business_key}=s.{business_key} AND t.is_current=true")
    .whenMatchedUpdate(set={"is_current":"false","effective_end_date":"date_sub(current_date(),1)"}).execute())
changed.write.format("delta").mode("append").saveAsTable(target_name)

# COMMAND -----
# Date dimension: one row per calendar date.
dates=(spark.range(0,3653).select(F.date_add(F.lit("2020-01-01").cast("date"),F.col("id").cast("int")).alias("full_date"))
    .withColumn("date_key",F.date_format("full_date","yyyyMMdd").cast("int"))
    .withColumn("day_number",F.dayofmonth("full_date"))
    .withColumn("day_name",F.date_format("full_date","EEEE"))
    .withColumn("week_number",F.weekofyear("full_date"))
    .withColumn("month_number",F.month("full_date"))
    .withColumn("month_name",F.date_format("full_date","MMMM"))
    .withColumn("quarter_number",F.quarter("full_date"))
    .withColumn("calendar_year",F.year("full_date"))
    .withColumn("is_weekend",F.dayofweek("full_date").isin(1,7))
    .with-
Column("load_timestamp_utc",F.current_timestamp()).withColumn("batch_id",F.lit(batch_id)).withColumn("source_system",F.lit("GENERATED")))
dates.write.format("delta").mode("overwrite").option("overwriteSchema","true").saveAsTable(f"{catalog}.{gold_schema}.dim_date")

customers=spark.table(f"{catalog}.{silver_schema}.customers").filter(F.col("batch_id")==batch_id).select("customer_number","first_name","last_name","email","state_code","loyalty_tier","
signup_date","load_timestamp_utc","batch_id","source_system")
scd2_merge(customers,"dim_customer","customer_number",[first_name,"last_name","email","state_code","loyalty_tier"],"customer_key")
products=spark.table(f"{catalog}.{silver_schema}.products").filter(F.col("batch_id")==batch_id).select("sku","product_name","category_name","brand","supplier_code","unit_cost","list_pr
ice","load_timestamp_utc","batch_id","source_system")
scd2_merge(products,"dim_product","sku",[product_name,"category_name","brand","supplier_code","unit_cost","list_price"],"product_key")

# Type 1 dimensions. Complete TODO(USER) employee and supplier enrichment after practicing the store example.
stores=(spark.table(f"{catalog}.{bronze_schema}.erp_stores").filter(F.col("_batch_id")==batch_id)
    .se-
lect(stable_positive_key("store_code").cast("int").alias("store_key"),F.upper(F.trim("store_code")).alias("store_code"),"store_name","region",
"state_code","city",F.to_date("open_date").alias("open_date"),F.col("active_flag").cast("boolean").alias("active_flag"),F.current_timestamp()).
alias("load_timestamp_utc"),F.lit(batch_id).alias("batch_id"),F.lit("SQLSERVER_ERP").alias("source_system"))
type1_upsert(stores,"dim_store","store_code")

employees=(spark.table(f"{catalog}.{bronze_schema}.erp_employees").filter(F.col("_batch_id")==batch_id)
    .se-
lect(stable_positive_key("employee_number").cast("int").alias("employee_key"),F.upper(F.trim("employee_number")).alias("employee_number"),F.co
ncat_ws(" ",F.init-
cap(F.trim("first_name")),F.initcap(F.trim("last_name"))).alias("employee_name"),"job_title",F.col("store_id").cast("string").alias("store_cod
e"),F.col("act-
ive_flag").cast("boolean").alias("active_flag"),F.current_timestamp().alias("load_timestamp_utc"),F.lit(batch_id).alias("batch_id"),F.lit("SQL
SERVER_ERP").alias("source_system"))
type1_upsert(employees,"dim_employee","employee_number")

suppliers=(spark.table(f"{catalog}.{bronze_schema}.erp_suppliers").filter(F.col("_batch_id")==batch_id)
    .se-
lect(stable_positive_key("supplier_code").cast("int").alias("supplier_key"),F.upper(F.trim("supplier_code")).alias("supplier_code"),"supplier_
name","coun-
try_code",F.col("lead_time_days").cast("int").alias("lead_time_days"),F.col("active_flag").cast("boolean").alias("active_flag"),F.current_time
stamp().alias("load_timestamp_utc"),F.lit(batch_id).alias("batch_id"),F.lit("SQLSERVER_ERP").alias("source_system"))
type1_upsert(suppliers,"dim_supplier","supplier_code")

promotions=(spark.table(f"{catalog}.{silver_schema}.promotions").filter(F.col("batch_id")==batch_id)
    .se-
lect(stable_positive_key("promotion_code").cast("int").alias("promotion_key"),"promotion_code","promotion_name","discount_type","discount_valu
e","start_date","end_date","channel_code","load_timestamp_utc","batch_id","source_system"))
type1_upsert(promotions,"dim_promotion","promotion_code")

channels=spark.createDataFrame([(1,"STORE","Store","Sales"),(2,"ONLINE","Online","Sales"),(3,"PAID SEARCH","Paid Search","Marketing"),(4,"SO-
CIAL","Social","Marketing"),(5,"EMAIL","Email","Marketing"),(6,"AFFILIATE","Affiliate","Marketing"),(7,"ORGANIC","Organic","Marketing"),
(8,"DIRECT","Direct","Marketing")],[chan-
nel_key","chan-
nel_code","channel_name","channel_group"])).withColumn("load_timestamp_utc",F.current_timestamp()).withColumn("batch_id",F.lit(batch_id)).withC
olumn("source_system",F.lit("CONFORMED"))
type1_upsert(channels,"dim_channel","channel_code")

campaigns=(spark.table(f"{catalog}.{silver_schema}.campaigns").filter(F.col("batch_id")==batch_id)
    .se-
lect(stable_positive_key("campaign_code").cast("int").alias("campaign_key"),"campaign_code","campaign_name","channel","start_date","end_date",
"budget_amount","currency_code","load_timestamp_utc","batch_id","source_system"))
type1_upsert(campaigns,"dim_campaign","campaign_code")

# Add required Unknown rows with key 0 after normal upserts; MERGE makes reruns safe.
# TODO(USER): write and validate the unknown-member MERGE using Phase 11 and the optional reference-answer section later in this workbook.
# This deliberate stop prevents a Job from reporting success before the learner-owned requirement is complete.
raise NotImplementedError("TODO(USER): add and test key-0 Unknown rows for all dimensions, then remove this deliberate stop and set gold_di-
mensions_status=PASS.")

```

Line-by-line explanation

- **Line 1:** `# Databricks notebook source` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `# Northstar Retail End-to-End Data Platform` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** `# This file is a Git-friendly Databricks source notebook.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** `# Track A reads files that you manually uploaded to a Unity Catalog volume.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** `# It never attempts to reach SQL Server or PostgreSQL on localhost.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 9:** `# MAGIC %md` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 10:** `# MAGIC # 08 – Gold dimensions` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 11:** `# MAGIC Builds unknown rows, Type 1 dimensions, and SCD Type 2 customer/product dimensions.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 14:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** `from datetime import date` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `from delta.tables import DeltaTable` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `delta.tables` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DeltaTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** `from pyspark.sql import functions as F` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `functions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** `from pyspark.sql.window import Window` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql.window` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Window` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** blank line used only to separate logical sections for readability.
- **Line 20:** `for name, default in {"catalog": "northstar_retail", "bronze_schema": "bronze", "silver_schema": "silver", "gold_schema": "gold", "batch_id": "MANUAL-REPLACE-ME"}.items(): dbutils.widgets.text(name, default)` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; default is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "catalog" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "northstar_retail" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "bronze_schema" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 21:** `catalog=dbutils.widgets.get("catalog"); bronze_schema=dbutils.widgets.get("bronze_schema"); silver_schema=dbutils.widgets.get("silver_schema"); gold_schema=dbutils.widgets.get("gold_schema"); batch_id=dbutils.widgets.get("batch_id")` — `catalog` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"catalog"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"bronze_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 22:** `if batch_id=="MANUAL-REPLACE-ME": raise ValueError("Pass batch_id from the workflow.")` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"MANUAL-REPLACE-ME"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ValueError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Pass batch_id from the workflow."` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** blank line used only to separate logical sections for readability.
- **Line 24:** blank line used only to separate logical sections for readability.
- **Line 25:** `def stable_positive_key(column_name: str):` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `stable_positive_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `column_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** `# xxhash64 is deterministic across reruns. Absolute value keeps keys positive; 0 is reserved for Unknown.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 27:** `return (F.abs(F.xxhash64(F.col(column_name))) + F.lit(1)).cast("long")` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.abs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.xxhash64` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-

specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; column_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;))) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 28:** blank line used only to separate logical sections for readability.
- **Line 29:** blank line used only to separate logical sections for readability.
- **Line 30:** `def type1_upsert(source, table_name: str, business_key: str):` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `type1_upsert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 31:** `target_name=f"{catalog}.{gold_schema}.{table_name}"` — `target_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.{gold_schema}.{table_name}"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 32:** `if not spark.catalog.tableExists(target_name):` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.catalog.tableExists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** `source.write.format("delta").mode("overwrite").saveAsTable(target_name)` — `source.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** `return` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** `target=DeltaTable.forName(spark,target_name)` — `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DeltaTable.forName` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding

explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; target_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 36:** (target.alias("t").merge(source.alias("s"), f"t.{business_key}=s.{business_key}") — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; target.alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "t" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; merge is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source.alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "s" is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 37:** .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute() — . is a path, relative location, branch-style name, or file argument; verify every character before running it; whenMatchedUpdateAll is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; whenNotMatchedInsertAll is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; execute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 38:** blank line used only to separate logical sections for readability.
- **Line 39:** blank line used only to separate logical sections for readability.
- **Line 40:** def scd2_merge(source, table_name: str, business_key: str, hash_columns: list[str], surrogate_key: str): — def is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; scd2_merge is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; business_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 41:** """"SCD2 teaching pattern: expire changed current rows, then insert new versions."" — "" is a quoted literal; the quotation marks preserve spaces or mark text; "SCD2 teaching pattern: expire changed current rows, then insert new versions." is a quoted literal; the quotation marks preserve spaces or mark text; "" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 42:** target_name=f"{catalog}.{gold_schema}.{table_name}" — target_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.{gold_schema}.{table_name}" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 43:** hash_expr=F.sha2(F.concat_ws("||", *[F.coalesce(F.col(c).cast("string"),F.lit("<NULL>")) for c in hash_columns]),256) — hash_expr is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.sha2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; `F.concat_ws` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"||"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.coalesce` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 44:** `staged=(source.withColumn("record_hash",hash_expr) — staged` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source.withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"record_hash"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `hash_expr` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 45:** `.withColumn("effective_start_date",F.current_date()) — .` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"effective_start_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** `.withColumn("effective_end_date",F.lit("9999-12-31").cast("date")) — .` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"effective_end_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"9999-12-31"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"date"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 47:** `.withColumn("is_current",F.lit(True)) — .` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"is_current"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `True` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 48:** `.withColumn(surrogate_key, (F.abs(F.xxhash64(F.col(business_key), F.col("record_hash"), F.col("batch_id")))+ F.lit(1)).cast("long")) — .` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `surrogate_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.abs`

is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.xxhash64 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 49:** `if not spark.catalog.tableExists(target_name):` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.catalog.tableExists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 50:** `staged.write.format("delta").mode("overwrite").saveAsTable(target_name);` `return` — `staged.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 51:** `target=DeltaTable.forName(spark,target_name)` — `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DeltaTable.forName` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 52:** `current=spark.table(target_name).filter("is_current=true").select(business_key,"record_hash")` — `current` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"is_current=true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 53:** `changed=(staged.alias("s").join(current.alias("t"),business_key,"left")` — `changed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `staged.alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the

surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "s" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; join is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; current.alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "t" is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 54:** `.filter(F.col("t.record_hash").isNull() | (F.col("s.record_hash") != F.col("t.record_hash")))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "t.record_hash" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; isNull is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 55:** `.select("s.*")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; select is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "s.*" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 56:** `(target.alias("t").merge(changed.alias("s"),f"t.{business_key}=s.{business_key} AND t.is_current=true")` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; target.alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "t" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; merge is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; changed.alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "s" is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 57:** `.whenMatchedUp-date(set={"is_current":"false","effective_end_date":"date_sub(current_date(),1)"}).execute())` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; whenMatchedUp-date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; set is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ={ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "is_current" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "false" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "effective_end_date" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "date_sub(cur-

rent_date(),1)" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 58:** `changed.write.format("delta").mode("append").saveAsTable(target_name)` — `changed.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 59:** blank line used only to separate logical sections for readability.
- **Line 60:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 61:** `# Date dimension: one row per calendar date.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 62:**
`dates=(spark.range(0,3653).select(F.date_add(F.lit("2020-01-01").cast("date"),F.col("id").cast("int")).alias("full_date"))` — `dates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.range` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3653` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.date_add` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 63:** `.withColumn("date_key",F.date_format("full_date","yyyyMMdd").cast("int"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"date_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.date_format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"full_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"yyyyMMdd"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 64:** `.withColumn("day_number",F.dayofmonth("full_date"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"day_number"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.dayofmonth` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"full_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "full_date" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 71:** `.withColumn("is_weekend",F.dayofweek("full_date").isin(1,7))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "is_weekend" is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.dayofweek` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "full_date" is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `isin` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.

- **Line 72:** `.withColumn("load_timestamp_utc",F.current_timestamp()).withColumn("batch_id",F.lit(batch_id)).withColumn("source_system",F.lit("GENERATED"))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "load_timestamp_utc" is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 73:** `dates.write.format("delta").mode("overwrite").option("overwriteSchema","true").saveAsTable(f"{catalog}.{gold_schema}.dim_date")` — `dates.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwriteSchema" is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 74:** blank line used only to separate logical sections for readability.

- **Line 75:** `customers=spark.table(f"{catalog}.{silver_schema}.customers").filter(F.col("batch_id")==batch_id).select("customer_number","first_name","last_name","email","state_code","loyalty_tier","signup_date","load_timestamp_utc","batch_id","source_system")` — `customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.{silver_schema}.customers" is a quoted literal; the quotation marks preserve spaces or mark text; `.` is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 76:** `scd2_merge(customers,"dim_customer","customer_number",["first_name","last_name","email","state_code","loyalty_tier"],"customer_key")` — `scd2_merge` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"dim_customer"` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"customer_number"` is a quoted literal; the quotation marks preserve spaces or mark text; , [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"first_name"` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"last_name"` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 77:** `products=spark.table(f"{catalog}.{silver_schema}.products").filter(F.col("batch_id")==batch_id).select("sku","product_name","category_name","brand","supplier_code","unit_cost","list_price","load_timestamp_utc","batch_id","source_system")` — `products` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f"{catalog}.{silver_schema}.products"` is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 78:** `scd2_merge(products,"dim_product","sku",["product_name","category_name","brand","supplier_code","unit_cost","list_price"],"product_key")` — `scd2_merge` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `products` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"dim_product"` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"sku"` is a quoted literal; the quotation marks preserve spaces or mark text; , [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"product_name"` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"category_name"` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 79:** blank line used only to separate logical sections for readability.
- **Line 80:** `# Type 1 dimensions. Complete TODO(USER) employee and supplier enrichment after practicing the store example.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 81:** `stores=(spark.table(f"{catalog}.{bronze_schema}.erp_stores").filter(F.col("_batch_id")==batch_id)` — `stores` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; =(is a name, keyword, argument, identifier, or literal whose project-specific role is described

in the surrounding explanation; spark.table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.{bronze_schema}.erp_stores" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

• **Line 82:** .se-

```
lect(stable_positive_key("store_code").cast("int").alias("store_key"),F.upper(F.trim("store_code")).alias("store_code"),"store_name","region","state_code","city",F.to_date("open_date").alias("open_date"),F.col("active_flag").cast("boolean").alias("active_flag"),F.current_timestamp().alias("load_timestamp_utc"),F.lit(batch_id).alias("batch_id"),F.lit("SQLSERVER_ERP").alias("source_system")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; select is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stable_positive_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "store_code" is a quoted literal; the quotation marks preserve spaces or mark text; ) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "int" is a quoted literal; the quotation marks preserve spaces or mark text; ) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
```

• **Line 83:** type1_upsert(stores,"dim_store","store_code") — type1_upsert is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stores is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "dim_store" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "store_code" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

• **Line 84:** blank line used only to separate logical sections for readability.

• **Line 85:** employees=(spark.table(f"{catalog}.

```
{bronze_schema}.erp_employees").filter(F.col("_batch_id")==batch_id) — employees is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; =( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spark.table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.{bronze_schema}.erp_employees" is a quoted literal; the quotation marks preserve spaces or mark text; ) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
```

• **Line 86:** .se-

```
lect(stable_positive_key("employee_number").cast("int").alias("employee_key"),F.upper(F.trim("employee_number")).alias("employee_number"),F.concat_ws(" ",F.init-
```

cap(F.trim("first_name")),F.initcap(F.trim("last_name"))).alias("employee_name"),"job_title",F.col("store_id").cast("string").alias("store_code"),F.col("active_flag").cast("boolean").alias("active_flag"),F.current_timestamp().alias("load_timestamp_utc"),F.lit(batch_id).alias("batch_id"),F.lit("SQLSERVER_ERP").alias("source_system")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; select is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stable_positive_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "employee_number" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "int" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 87:** type1_upsert(employees,"dim_employee","employee_number") — type1_upsert is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; employees is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "dim_employee" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "employee_number" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 88:** blank line used only to separate logical sections for readability.
- **Line 89:** suppliers=(spark.table(f"{catalog}."{bronze_schema}.erp_suppliers").filter(F.col("_batch_id")==batch_id) — suppliers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; =(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spark.table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}."{bronze_schema}.erp_suppliers" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 90:** .select(stable_positive_key("supplier_code").cast("int").alias("supplier_key"),F.upper(F.trim("supplier_code"))).alias("supplier_code"),"supplier_name","country_code",F.col("lead_time_days").cast("int").alias("lead_time_days"),F.col("active_flag").cast("boolean").alias("active_flag"),F.current_timestamp().alias("load_timestamp_utc"),F.lit(batch_id).alias("batch_id"),F.lit("SQLSERVER_ERP").alias("source_system")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; select is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stable_positive_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "supplier_code" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

in the surrounding explanation; "int" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 91:** type1_upsert(suppliers,"dim_supplier","supplier_code") — type1_upsert is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; suppliers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "dim_supplier" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "supplier_code" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 92:** blank line used only to separate logical sections for readability.
- **Line 93:** promotions=(spark.table(f"{catalog}.{silver_schema}.promotions").filter(F.col("batch_id")==batch_id) — promotions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; =(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spark.table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.{silver_schema}.promotions" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 94:** .select(stable_positive_key("promotion_code").cast("int").alias("promotion_key"),"promotion_code","promotion_name","discount_type","discount_value","start_date","end_date","channel_code","load_timestamp_utc","batch_id","source_system")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; select is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stable_positive_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "promotion_code" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "int" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 95:** type1_upsert(promotions,"dim_promotion","promotion_code") — type1_upsert is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; promotions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "dim_promotion" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "promotion_code" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 96:** blank line used only to separate logical sections for readability.
- **Line 97:** channels=spark.createDataFrame([(1,"STORE","Store","Sales"),(2,"ONLINE","Online","Sales"),(3,"PAID SEARCH","Paid Search","Marketing"),(4,"SOCIAL","Social","Marketing"),(5,"EMAIL","Email","Marketing"),(6,"AFFILIATE","Affiliate","Marketing"),(7,"ORGANIC","Organic","Marketing"),(8,"DIRECT","Direct","Marketing")],["chan-

nel_key", "chan-
nel_code", "channel_name", "channel_group"])).withColumn("load_timestamp_utc", F.current_timestamp()).withColumn("batch_id", F.lit(batch_id)).withColumn("source_system", F.lit("CONFORMED")) — channels is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spark.createDataFrame is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "STORE" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Store" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Sales" is a quoted literal; the quotation marks preserve spaces or mark text;), (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 98:** type1_upsert(channels, "dim_channel", "channel_code") — type1_upsert is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; channels is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "dim_channel" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "channel_code" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 99:** blank line used only to separate logical sections for readability.
- **Line 100:** campaigns=(spark.table(f"{catalog}.{silver_schema}.campaigns").filter(F.col("batch_id")==batch_id) — campaigns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; =(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spark.table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.{silver_schema}.campaigns" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 101:** .select(stable_positive_key("campaign_code").cast("int").alias("campaign_key"), "campaign_code", "campaign_name", "channel", "start_date", "end_date", "budget_amount", "currency_code", "load_timestamp_utc", "batch_id", "source_system")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; select is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stable_positive_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "campaign_code" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "int" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 102:** type1_upsert(campaigns, "dim_campaign", "campaign_code") — type1_upsert is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; campaigns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "dim_campaign" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "campaign_code" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 103:** blank line used only to separate logical sections for readability.
- **Line 104:** # Add required Unknown rows with key 0 after normal upserts; MERGE makes reruns safe. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 105:** # TODO(USER): write and validate the unknown-member MERGE using Phase 11 and the optional reference-answer section later in this workbook. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 106:** # This deliberate stop prevents a Job from reporting success before the learner-owned requirement is complete. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 107:** raise NotImplementedError("TODO(USER): add and test key-0 Unknown rows for all dimensions, then remove this deliberate stop and set gold_dimensions_status=PASS.") — raise is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NotImplementedError is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "TODO(USER): add and test key-0 Unknown rows for all dimensions, then remove this deliberate stop and set gold_dimensions_status=PASS." is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 11 lesson index](#)

File introduction — sql/databricks/gold/01_scd2_merge_pattern.sql

Exact filename: 01_scd2_merge_pattern.sql

Filename extension: .sql. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: sql/databricks/gold relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 11.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in sql/databricks/gold one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 01_scd2_merge_pattern.sql.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 01_scd2_merge_pattern.sql and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 11.F2.1 — Create the file sql/databricks/gold/01_scd2_merge_pattern.sql from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\databricks\gold\01_scd2_merge_pattern.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\databricks\gold\01_scd2_merge_pattern.sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\databricks\gold\01_scd2_merge_pattern.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\sql\databricks\gold\01_scd2_merge_pattern.sql" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/databricks/gold/01_scd2_merge_pattern.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `sql/databricks/gold`.

2. Right-click the editor tab and choose **Copy Path**; compare the ending with `sql/databricks/gold/01_scd2_merge_pattern.sql`.
3. In PowerShell at the repository root, run `Test-Path ".\sql\databricks\gold\01_scd2_merge_pattern.sql"` and require `True`.
4. Run `git status --short -- "sql/databricks/gold/01_scd2_merge_pattern.sql"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `sql/databricks/gold/01_scd2_merge_pattern.sql`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: `sql/databricks/gold/01_scd2_merge_pattern.sql`

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 11.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: Databricks SQL / Spark SQL
-- Near-complete SCD2 pattern. Test on a copy before adapting it.
MERGE INTO IDENTIFIER(:target_dimension) AS target
USING IDENTIFIER(:changed_source_rows) AS source
ON target.business_key = source.business_key AND target.is_current = true
WHEN MATCHED AND target.record_hash <> source.record_hash THEN UPDATE SET
    target.is_current = false,
    target.effective_end_date = date_sub(current_date(), 1);

-- A second INSERT step adds new and changed current versions. The two-step reason is explained in Phase 11.
```

Line-by-line explanation

- **Line 1:** `-- Dialect: Databricks SQL / Spark SQL` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `-- Near-complete SCD2 pattern. Test on a copy before adapting it.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** `MERGE INTO IDENTIFIER(:target_dimension) AS target` — `MERGE` compares source and target rows to update or insert deliberately; `INTO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IDENTIFIER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(:)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_dimension` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `USING IDENTIFIER(:changed_source_rows) AS source` — `USING` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IDENTIFIER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(:)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `changed_source_rows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `ON target.business_key = source.business_key AND target.is_current = true` — `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source.business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AND` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.is_current` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `true` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `WHEN MATCHED AND target.record_hash <> source.record_hash THEN UPDATE SET` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `MATCHED` is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AND is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; target.record_hash is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; <> is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source.record_hash is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; THEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UPDATE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SET is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** target.is_current = false, — target.is_current is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; false is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** target.effective_end_date = date_sub(current_date(), 1); — target.effective_end_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; date_sub is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; current_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** blank line used only to separate logical sections for readability.
- **Line 10:** -- A second INSERT step adds new and changed current versions. The two-step reason is explained in Phase 11. is a comment for a human reader; it normally does not change data or execute business logic.

[Back to the Phase 11 lesson index](#)

What your screen should look like now

- Every Phase 11 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 11.3 completion checklist



databricks/notebooks/08_gold_dimensions.py was created by me, saved, located correctly, and inspected — [STATUS]



sql/databricks/gold/01_scd2_merge_pattern.sql was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 11.4](#) and recheck the root and branch.

Lesson 11.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Gold dimension 1 — `gold.dim_date`

Grain: One row per version of the date business entity when history is preserved; otherwise one current row per business key.

Natural/business key: `date_business_key` or the exact mapped source key documented for the dimension.

Surrogate key: `date_key`, generated independently of source identifiers.

Unknown-member strategy: Key 0 with readable values so facts with late, missing, or invalid references remain countable and monitorable.

Change behavior: Type 1 or deterministic rebuild behavior as documented for this dimension.

Source columns: The conformed Silver fields listed in the source-to-target mapping; do not pull raw Bronze values into Gold.

Dimension lookup use: Facts resolve the natural key plus event/effective date to the correct surrogate key.

Action 11.4.D1 — Build and validate `gold.dim_date`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and validate `gold.dim_date`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/08_gold_dimensions.py`.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for `dim_date` in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply Type 1 or deterministic rebuild behavior as documented for this dimension.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/08_gold_dimensions.py`.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for `dim_date` in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.

7. TYPE THIS: Apply Type 1 or deterministic rebuild behavior as documented for this dimension.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Teaching merge pattern

```
-- Teaching pattern; adapt the mapped columns for dim_date.  
MERGE INTO gold.dim_date AS target  
USING staged_dim_date AS source  
ON target.date_business_key = source.date_business_key  
  
WHEN MATCHED THEN UPDATE SET  
    target.updated_at = current_timestamp()  
WHEN NOT MATCHED THEN INSERT *;
```

Line-by-line explanation

- **Line 1:** `-- Teaching pattern; adapt the mapped columns for dim_date.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `MERGE INTO gold.dim_date AS target` — `MERGE` compares source and target rows to update or insert deliberately; `INTO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gold.dim_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `USING staged_dim_date AS source` — `USING` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `staged_dim_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 4:** `ON target.date_business_key = source.date_business_key` — `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.date_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source.date_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** `WHEN MATCHED THEN UPDATE SET` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `MATCHED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UPDATE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SET` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `target.updated_at = current_timestamp()` — `target.updated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `WHEN NOT MATCHED THEN INSERT *;` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `MATCHED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INSERT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation queries and gates

- Surrogate key uniqueness.
- Business-key/current-row uniqueness.
- Unknown key 0 exists exactly once.
- Required descriptive attributes are not null for known rows.
- SCD2 effective windows do not overlap and exactly one current row exists per active business key.
- Initial-load count reconciles to the Silver survivor count plus the unknown row.
- Same-batch rerun is idempotent.
- A sample lookup before/after a Type 2 change resolves to the expected key.

Interview-ready explanation

"I built `dim_date` with a source-independent surrogate key, an explicit unknown member, deterministic change handling, and validation for key uniqueness and effective-date integrity. Facts use the dimension key that was valid at their event date."

Gold dimension 2 — `gold.dim_customer`

Grain: One row per version of the customer business entity when history is preserved; otherwise one current row per business key.

Natural/business key: `customer_business_key` or the exact mapped source key documented for the dimension.

Surrogate key: `customer_key`, generated independently of source identifiers.

Unknown-member strategy: Key 0 with readable values so facts with late, missing, or invalid references remain countable and monitorable.

Change behavior: SCD Type 2 for history-bearing attributes.

Source columns: The conformed Silver fields listed in the source-to-target mapping; do not pull raw Bronze values into Gold.

Dimension lookup use: Facts resolve the natural key plus event/effective date to the correct surrogate key.

Action 11.4.D2 — Build and validate `gold.dim_customer`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and validate `gold.dim_customer`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/08_gold_dimensions.py`.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for `dim_customer` in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply SCD Type 2 for history-bearing attributes.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/08_gold_dimensions.py`.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for `dim_customer` in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply SCD Type 2 for history-bearing attributes.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Teaching merge pattern

```
-- Teaching pattern; adapt the mapped columns for dim_customer.  
MERGE INTO gold.dim_customer AS target  
USING staged_dim_customer AS source  
ON target.customer_business_key = source.customer_business_key  
AND target.is_current = TRUE  
WHEN MATCHED THEN UPDATE SET  
    target.updated_at = current_timestamp()  
WHEN NOT MATCHED THEN INSERT *;
```

Line-by-line explanation

- **Line 1:** `-- Teaching pattern; adapt the mapped columns for dim_customer.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `MERGE INTO gold.dim_customer AS target` — `MERGE` compares source and target rows to update or insert deliberately; `INTO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gold.dim_customer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `USING staged_dim_customer AS source` — `USING` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `staged_dim_customer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `ON target.customer_business_key = source.customer_business_key` — `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.customer_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source.customer_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `AND target.is_current = TRUE` — `AND` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.is_current` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `TRUE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `WHEN MATCHED THEN UPDATE SET` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `MATCHED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UPDATE` is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; SET is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** `target.updated_at = current_timestamp()` — `target.updated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `WHEN NOT MATCHED THEN INSERT *;` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `MATCHED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INSERT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Detailed SCD Type 2 timeline test

1. Select one fictional business key and record its original descriptive attributes.
2. Run the initial dimension load and record surrogate key, `effective_from`, `effective_to`, and `is_current`.
3. Change one history-tracked source attribute in a controlled later batch.
4. Run the merge.
5. Confirm the old row is closed immediately before the new row begins according to the documented boundary convention.
6. Confirm exactly one row is current.
7. Confirm a fact dated before the change resolves to the old surrogate key and a fact dated after the change resolves to the new key.
8. Rerun the changed batch and confirm no third row appears.

Validation queries and gates

- Surrogate key uniqueness.
- Business-key/current-row uniqueness.
- Unknown key 0 exists exactly once.
- Required descriptive attributes are not null for known rows.
- SCD2 effective windows do not overlap and exactly one current row exists per active business key.
- Initial-load count reconciles to the Silver survivor count plus the unknown row.
- Same-batch rerun is idempotent.
- A sample lookup before/after a Type 2 change resolves to the expected key.

Interview-ready explanation

"I built `dim_customer` with a source-independent surrogate key, an explicit unknown member, deterministic change handling, and validation for key uniqueness and effective-date integrity. Facts use the dimension key that was valid at their event date."

Gold dimension 3 — `gold.dim_product`

Grain: One row per version of the product business entity when history is preserved; otherwise one current row per business key.

Natural/business key: `product_business_key` or the exact mapped source key documented for the dimension.

Surrogate key: `product_key`, generated independently of source identifiers.

Unknown-member strategy: Key 0 with readable values so facts with late, missing, or invalid references remain countable and monitorable.

Change behavior: SCD Type 2 for history-bearing attributes.

Source columns: The conformed Silver fields listed in the source-to-target mapping; do not pull raw Bronze values into Gold.

Dimension lookup use: Facts resolve the natural key plus event/effective date to the correct surrogate key.

Action 11.4.D3 — Build and validate `gold.dim_product`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and validate `gold.dim_product`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/08_gold_dimensions.py`.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for `dim_product` in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply SCD Type 2 for history-bearing attributes.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/08_gold_dimensions.py`.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for `dim_product` in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply SCD Type 2 for history-bearing attributes.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Teaching merge pattern

```
-- Teaching pattern; adapt the mapped columns for dim_product.  
MERGE INTO gold.dim_product AS target  
USING staged_dim_product AS source  
ON target.product_business_key = source.product_business_key  
AND target.is_current = TRUE  
WHEN MATCHED THEN UPDATE SET  
    target.updated_at = current_timestamp()  
WHEN NOT MATCHED THEN INSERT *;
```

Line-by-line explanation

- **Line 1:** `-- Teaching pattern; adapt the mapped columns for dim_product.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `MERGE INTO gold.dim_product AS target` — `MERGE` compares source and target rows to update or insert deliberately; `INTO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gold.dim_product` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `USING staged_dim_product AS source` — `USING` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `staged_dim_product` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `ON target.product_business_key = source.product_business_key` — `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.product_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source.product_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `AND target.is_current = TRUE` — `AND` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.is_current` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `TRUE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `WHEN MATCHED THEN UPDATE SET` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `MATCHED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UPDATE` is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; SET is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** `target.updated_at = current_timestamp()` — `target.updated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `WHEN NOT MATCHED THEN INSERT *;` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `MATCHED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INSERT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Detailed SCD Type 2 timeline test

1. Select one fictional business key and record its original descriptive attributes.
2. Run the initial dimension load and record surrogate key, `effective_from`, `effective_to`, and `is_current`.
3. Change one history-tracked source attribute in a controlled later batch.
4. Run the merge.
5. Confirm the old row is closed immediately before the new row begins according to the documented boundary convention.
6. Confirm exactly one row is current.
7. Confirm a fact dated before the change resolves to the old surrogate key and a fact dated after the change resolves to the new key.
8. Rerun the changed batch and confirm no third row appears.

Validation queries and gates

- Surrogate key uniqueness.
- Business-key/current-row uniqueness.
- Unknown key 0 exists exactly once.
- Required descriptive attributes are not null for known rows.
- SCD2 effective windows do not overlap and exactly one current row exists per active business key.
- Initial-load count reconciles to the Silver survivor count plus the unknown row.
- Same-batch rerun is idempotent.
- A sample lookup before/after a Type 2 change resolves to the expected key.

Interview-ready explanation

"I built `dim_product` with a source-independent surrogate key, an explicit unknown member, deterministic change handling, and validation for key uniqueness and effective-date integrity. Facts use the dimension key that was valid at their event date."

Gold dimension 4 — `gold.dim_store`

Grain: One row per version of the store business entity when history is preserved; otherwise one current row per business key.

Natural/business key: `store_business_key` or the exact mapped source key documented for the dimension.

Surrogate key: `store_key`, generated independently of source identifiers.

Unknown-member strategy: Key 0 with readable values so facts with late, missing, or invalid references remain countable and monitorable.

Change behavior: Type 1 or deterministic rebuild behavior as documented for this dimension.

Source columns: The conformed Silver fields listed in the source-to-target mapping; do not pull raw Bronze values into Gold.

Dimension lookup use: Facts resolve the natural key plus event/effective date to the correct surrogate key.

Action 11.4.D4 — Build and validate `gold.dim_store`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and validate `gold.dim_store`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/08_gold_dimensions.py`.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for `dim_store` in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply Type 1 or deterministic rebuild behavior as documented for this dimension.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/08_gold_dimensions.py`.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for `dim_store` in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply Type 1 or deterministic rebuild behavior as documented for this dimension.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Teaching merge pattern

```
-- Teaching pattern; adapt the mapped columns for dim_store.  
MERGE INTO gold.dim_store AS target  
USING staged_dim_store AS source  
ON target.store_business_key = source.store_business_key  
  
WHEN MATCHED THEN UPDATE SET  
    target.updated_at = current_timestamp()  
WHEN NOT MATCHED THEN INSERT *;
```

Line-by-line explanation

- **Line 1:** `-- Teaching pattern; adapt the mapped columns for dim_store.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `MERGE INTO gold.dim_store AS target` — `MERGE` compares source and target rows to update or insert deliberately; `INTO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gold.dim_store` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `USING staged_dim_store AS source` — `USING` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `staged_dim_store` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `ON target.store_business_key = source.store_business_key` — `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.store_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source.store_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** `WHEN MATCHED THEN UPDATE SET` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `MATCHED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UPDATE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SET` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** `target.updated_at = current_timestamp()` — `target.updated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `WHEN NOT MATCHED THEN INSERT *;` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `MATCHED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INSERT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation queries and gates

- Surrogate key uniqueness.
- Business-key/current-row uniqueness.
- Unknown key 0 exists exactly once.
- Required descriptive attributes are not null for known rows.
- SCD2 effective windows do not overlap and exactly one current row exists per active business key.
- Initial-load count reconciles to the Silver survivor count plus the unknown row.
- Same-batch rerun is idempotent.
- A sample lookup before/after a Type 2 change resolves to the expected key.

Interview-ready explanation

"I built `dim_store` with a source-independent surrogate key, an explicit unknown member, deterministic change handling, and validation for key uniqueness and effective-date integrity. Facts use the dimension key that was valid at their event date."

Gold dimension 5 — `gold.dim_employee`

Grain: One row per version of the `employee` business entity when history is preserved; otherwise one current row per business key.

Natural/business key: `employee_business_key` or the exact mapped source key documented for the dimension.

Surrogate key: `employee_key`, generated independently of source identifiers.

Unknown-member strategy: Key 0 with readable values so facts with late, missing, or invalid references remain countable and monitorable.

Change behavior: Type 1 or deterministic rebuild behavior as documented for this dimension.

Source columns: The conformed Silver fields listed in the source-to-target mapping; do not pull raw Bronze values into Gold.

Dimension lookup use: Facts resolve the natural key plus event/effective date to the correct surrogate key.

Action 11.4.D5 — Build and validate `gold.dim_employee`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and validate `gold.dim_employee`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open databricks/notebooks/08_gold_dimensions.py.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for dim_employee in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply Type 1 or deterministic rebuild behavior as documented for this dimension.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/08_gold_dimensions.py.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for dim_employee in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply Type 1 or deterministic rebuild behavior as documented for this dimension.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location

with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Teaching merge pattern

```
-- Teaching pattern; adapt the mapped columns for dim_employee.  
MERGE INTO gold.dim_employee AS target  
USING staged_dim_employee AS source  
ON target.employee_business_key = source.employee_business_key  
  
WHEN MATCHED THEN UPDATE SET  
    target.updated_at = current_timestamp()  
WHEN NOT MATCHED THEN INSERT *;
```

Line-by-line explanation

- **Line 1:** `-- Teaching pattern; adapt the mapped columns for dim_employee.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `MERGE INTO gold.dim_employee AS target` — `MERGE` compares source and target rows to update or insert deliberately; `INTO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gold.dim_employee` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `USING staged_dim_employee AS source` — `USING` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `staged_dim_employee` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `ON target.employee_business_key = source.employee_business_key` — `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.employee_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source.employee_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** `WHEN MATCHED THEN UPDATE SET` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `MATCHED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UPDATE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SET` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `target.updated_at = current_timestamp()` — `target.updated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `WHEN NOT MATCHED THEN INSERT *` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `MATCHED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INSERT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation queries and gates

- Surrogate key uniqueness.

- Business-key/current-row uniqueness.
- Unknown key 0 exists exactly once.
- Required descriptive attributes are not null for known rows.
- SCD2 effective windows do not overlap and exactly one current row exists per active business key.
- Initial-load count reconciles to the Silver survivor count plus the unknown row.
- Same-batch rerun is idempotent.
- A sample lookup before/after a Type 2 change resolves to the expected key.

Interview-ready explanation

"I built `dim_employee` with a source-independent surrogate key, an explicit unknown member, deterministic change handling, and validation for key uniqueness and effective-date integrity. Facts use the dimension key that was valid at their event date."

Gold dimension 6 — `gold.dim_supplier`

Grain: One row per version of the supplier business entity when history is preserved; otherwise one current row per business key.

Natural/business key: `supplier_business_key` or the exact mapped source key documented for the dimension.

Surrogate key: `supplier_key`, generated independently of source identifiers.

Unknown-member strategy: Key 0 with readable values so facts with late, missing, or invalid references remain countable and monitorable.

Change behavior: Type 1 or deterministic rebuild behavior as documented for this dimension.

Source columns: The conformed Silver fields listed in the source-to-target mapping; do not pull raw Bronze values into Gold.

Dimension lookup use: Facts resolve the natural key plus event/effective date to the correct surrogate key.

Action 11.4.D6 — Build and validate `gold.dim_supplier`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and validate `gold.dim_supplier`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/08_gold_dimensions.py`.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for `dim_supplier` in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply Type 1 or deterministic rebuild behavior as documented for this dimension.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/08_gold_dimensions.py`.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for `dim_supplier` in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply Type 1 or deterministic rebuild behavior as documented for this dimension.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Teaching merge pattern

```
-- Teaching pattern; adapt the mapped columns for dim_supplier.  
MERGE INTO gold.dim_supplier AS target  
USING staged_dim_supplier AS source  
ON target.supplier_business_key = source.supplier_business_key  
  
WHEN MATCHED THEN UPDATE SET
```

```
target.updated_at = current_timestamp()  
WHEN NOT MATCHED THEN INSERT *;
```

Line-by-line explanation

- **Line 1:** `--` Teaching pattern; adapt the mapped columns for `dim_supplier`. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `MERGE INTO gold.dim_supplier AS target` — `MERGE` compares source and target rows to update or insert deliberately; `INTO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gold.dim_supplier` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `USING staged_dim_supplier AS source` — `USING` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `staged_dim_supplier` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `ON target.supplier_business_key = source.supplier_business_key` — `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.supplier_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source.supplier_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** `WHEN MATCHED THEN UPDATE SET` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `MATCHED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UPDATE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SET` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `target.updated_at = current_timestamp()` — `target.updated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `WHEN NOT MATCHED THEN INSERT *;` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `MATCHED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INSERT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation queries and gates

- Surrogate key uniqueness.
- Business-key/current-row uniqueness.
- Unknown key 0 exists exactly once.
- Required descriptive attributes are not null for known rows.
- SCD2 effective windows do not overlap and exactly one current row exists per active business key.
- Initial-load count reconciles to the Silver survivor count plus the unknown row.
- Same-batch rerun is idempotent.
- A sample lookup before/after a Type 2 change resolves to the expected key.

Interview-ready explanation

"I built `dim_supplier` with a source-independent surrogate key, an explicit unknown member, deterministic change handling, and validation for key uniqueness and effective-date integrity. Facts use the dimension key that was valid at their event date."

Gold dimension 7 — `gold.dim_promotion`

Grain: One row per version of the promotion business entity when history is preserved; otherwise one current row per business key.

Natural/business key: `promotion_business_key` or the exact mapped source key documented for the dimension.

Surrogate key: `promotion_key`, generated independently of source identifiers.

Unknown-member strategy: Key 0 with readable values so facts with late, missing, or invalid references remain countable and monitorable.

Change behavior: Type 1 or deterministic rebuild behavior as documented for this dimension.

Source columns: The conformed Silver fields listed in the source-to-target mapping; do not pull raw Bronze values into Gold.

Dimension lookup use: Facts resolve the natural key plus event/effective date to the correct surrogate key.

Action 11.4.D7 — Build and validate `gold.dim_promotion`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and validate `gold.dim_promotion`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/08_gold_dimensions.py`.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for `dim_promotion` in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply Type 1 or deterministic rebuild behavior as documented for this dimension.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/08_gold_dimensions.py`.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for `dim_promotion` in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.

5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply Type 1 or deterministic rebuild behavior as documented for this dimension.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Teaching merge pattern

```
-- Teaching pattern; adapt the mapped columns for dim_promotion.
MERGE INTO gold.dim_promotion AS target
USING staged_dim_promotion AS source
ON target.promotion_business_key = source.promotion_business_key

WHEN MATCHED THEN UPDATE SET
    target.updated_at = current_timestamp()
WHEN NOT MATCHED THEN INSERT *;
```

Line-by-line explanation

- **Line 1:** -- Teaching pattern; adapt the mapped columns for dim_promotion. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** MERGE INTO gold.dim_promotion AS target — MERGE compares source and target rows to update or insert deliberately; INTO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gold.dim_promotion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; target is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** USING staged_dim_promotion AS source — USING is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; staged_dim_promotion is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 4:** `ON target.promotion_business_key = source.promotion_business_key` — ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.promotion_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source.promotion_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** `WHEN MATCHED THEN UPDATE SET` — WHEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; MATCHED is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; THEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UPDATE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SET is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `target.updated_at = current_timestamp()` — `target.updated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `WHEN NOT MATCHED THEN INSERT *;` — WHEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; MATCHED is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; THEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INSERT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation queries and gates

- Surrogate key uniqueness.
- Business-key/current-row uniqueness.
- Unknown key 0 exists exactly once.
- Required descriptive attributes are not null for known rows.
- SCD2 effective windows do not overlap and exactly one current row exists per active business key.
- Initial-load count reconciles to the Silver survivor count plus the unknown row.
- Same-batch rerun is idempotent.
- A sample lookup before/after a Type 2 change resolves to the expected key.

Interview-ready explanation

"I built `dim_promotion` with a source-independent surrogate key, an explicit unknown member, deterministic change handling, and validation for key uniqueness and effective-date integrity. Facts use the dimension key that was valid at their event date."

Gold dimension 8 — `gold.dim_channel`

Grain: One row per version of the `channel` business entity when history is preserved; otherwise one current row per business key.

Natural/business key: `channel_business_key` or the exact mapped source key documented for the dimension.

Surrogate key: `channel_key`, generated independently of source identifiers.

Unknown-member strategy: Key 0 with readable values so facts with late, missing, or invalid references remain countable and monitorable.

Change behavior: Type 1 or deterministic rebuild behavior as documented for this dimension.

Source columns: The conformed Silver fields listed in the source-to-target mapping; do not pull raw Bronze values into Gold.

Dimension lookup use: Facts resolve the natural key plus event/effective date to the correct surrogate key.

Action 11.4.D8 — Build and validate gold.dim_channel

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and validate gold.dim_channel. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open databricks/notebooks/08_gold_dimensions.py.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for dim_channel in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply Type 1 or deterministic rebuild behavior as documented for this dimension.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/08_gold_dimensions.py.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for dim_channel in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply Type 1 or deterministic rebuild behavior as documented for this dimension.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Teaching merge pattern

```
-- Teaching pattern; adapt the mapped columns for dim_channel.
MERGE INTO gold.dim_channel AS target
  USING staged_dim_channel AS source
  ON target.channel_business_key = source.channel_business_key

WHEN MATCHED THEN UPDATE SET
  target.updated_at = current_timestamp()
WHEN NOT MATCHED THEN INSERT *;
```

Line-by-line explanation

- **Line 1:** `-- Teaching pattern; adapt the mapped columns for dim_channel.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `MERGE INTO gold.dim_channel AS target` — `MERGE` compares source and target rows to update or insert deliberately; `INTO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gold.dim_channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `USING staged_dim_channel AS source` — `USING` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `staged_dim_channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `ON target.channel_business_key = source.channel_business_key` — `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.channel_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source.channel_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** `WHEN MATCHED THEN UPDATE SET` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `MATCHED` is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; THEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UPDATE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SET is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** `target.updated_at = current_timestamp()` — `target.updated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `WHEN NOT MATCHED THEN INSERT *;` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `MATCHED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INSERT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation queries and gates

- Surrogate key uniqueness.
- Business-key/current-row uniqueness.
- Unknown key 0 exists exactly once.
- Required descriptive attributes are not null for known rows.
- SCD2 effective windows do not overlap and exactly one current row exists per active business key.
- Initial-load count reconciles to the Silver survivor count plus the unknown row.
- Same-batch rerun is idempotent.
- A sample lookup before/after a Type 2 change resolves to the expected key.

Interview-ready explanation

"I built `dim_channel` with a source-independent surrogate key, an explicit unknown member, deterministic change handling, and validation for key uniqueness and effective-date integrity. Facts use the dimension key that was valid at their event date."

Gold dimension 9 — `gold.dim_campaign`

Grain: One row per version of the campaign business entity when history is preserved; otherwise one current row per business key.

Natural/business key: `campaign_business_key` or the exact mapped source key documented for the dimension.

Surrogate key: `campaign_key`, generated independently of source identifiers.

Unknown-member strategy: Key 0 with readable values so facts with late, missing, or invalid references remain countable and monitorable.

Change behavior: Type 1 or deterministic rebuild behavior as documented for this dimension.

Source columns: The conformed Silver fields listed in the source-to-target mapping; do not pull raw Bronze values into Gold.

Dimension lookup use: Facts resolve the natural key plus event/effective date to the correct surrogate key.

Action 11.4.D9 — Build and validate `gold.dim_campaign`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and validate `gold.dim_campaign`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/08_gold_dimensions.py`.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for `dim_campaign` in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply Type 1 or deterministic rebuild behavior as documented for this dimension.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open `databricks/notebooks/08_gold_dimensions.py`.
2. LOOK FOR: Confirm the current batch's required Silver inputs exist and passed quality gates.
3. TYPE THIS: Declare the dimension grain and natural/business key for `dim_campaign` in a Markdown comment before code.
4. TYPE THIS: Create or verify the unknown member with surrogate key 0 and readable Unknown attributes.
5. TYPE THIS: Build the staged source with one deterministic row per business key.
6. PRESS: Run the initial-load section and inspect keys and counts.
7. TYPE THIS: Apply Type 1 or deterministic rebuild behavior as documented for this dimension.
8. PRESS: Run a controlled changed-record example.
9. LOOK FOR: Verify current-row uniqueness, effective dates when used, and no overlapping SCD2 windows.
10. PRESS: Rerun the same batch.
11. DO NOT CONTINUE UNTIL: The rerun creates no unintended duplicate and a sample fact lookup returns the expected surrogate key.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 11 lesson index](#)

Teaching merge pattern

```
-- Teaching pattern; adapt the mapped columns for dim_campaign.  
MERGE INTO gold.dim_campaign AS target  
USING staged_dim_campaign AS source  
ON target.campaign_business_key = source.campaign_business_key  
  
WHEN MATCHED THEN UPDATE SET  
    target.updated_at = current_timestamp()  
WHEN NOT MATCHED THEN INSERT *;
```

Line-by-line explanation

- **Line 1:** `-- Teaching pattern; adapt the mapped columns for dim_campaign.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `MERGE INTO gold.dim_campaign AS target` — `MERGE` compares source and target rows to update or insert deliberately; `INTO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gold.dim_campaign` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `USING staged_dim_campaign AS source` — `USING` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `staged_dim_campaign` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `ON target.campaign_business_key = source.campaign_business_key` — `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.campaign_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source.campaign_business_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** `WHEN MATCHED THEN UPDATE SET` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `MATCHED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UPDATE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SET` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `target.updated_at = current_timestamp()` — `target.updated_at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `current_timestamp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `WHEN NOT MATCHED THEN INSERT *;` — `WHEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `MATCHED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `THEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INSERT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

surrounding explanation; * is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation queries and gates

- Surrogate key uniqueness.
- Business-key/current-row uniqueness.
- Unknown key 0 exists exactly once.
- Required descriptive attributes are not null for known rows.
- SCD2 effective windows do not overlap and exactly one current row exists per active business key.
- Initial-load count reconciles to the Silver survivor count plus the unknown row.
- Same-batch rerun is idempotent.
- A sample lookup before/after a Type 2 change resolves to the expected key.

Interview-ready explanation

"I built `dim_campaign` with a source-independent surrogate key, an explicit unknown member, deterministic change handling, and validation for key uniqueness and effective-date integrity. Facts use the dimension key that was valid at their event date."

What your screen should look like now

- The new Phase 11 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 11.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 11.5](#).

Lesson 11.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 11 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Type 1 versus Type 2 dimensions

Question:

Which changes need history?

Options considered:

- Type 1 overwrite all attributes.
- Type 2 all attributes.
- Selective Type 1/Type 2.

Beginner path selected:

Use Type 2 for customer and product history; simpler dimensions use Type 1 where appropriate.

Why this path was selected:

It demonstrates effective dating where historical context changes metric interpretation without adding unnecessary history everywhere.

Tradeoff:

SCD2 logic is more complex and creates multiple rows per business key.

What a company might do instead:

Companies define attribute-level change policies, late-arriving logic, audit metadata, and performance strategies.

How to explain this in an interview:

I selected SCD behavior by business meaning: customer/product changes preserve history, while corrective reference attributes can overwrite.

If you see Result A

Unknown members exist, current-row uniqueness holds, and an SCD2 change creates one valid new version. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

Overlapping dates, duplicate current rows, unstable keys, or an unexpired old version appear. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: Databricks notebooks, Catalog Explorer, and SQL/Python cells.

Window/panel: Show unknown member, SCD2 timeline, current-row uniqueness, and sample surrogate-key lookup.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase11_build-gold-dimensions-surrogate-keys-and-scd-behavior_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 11's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 11 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 11.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Action 11.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Action 11.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Interpret `git status --short`

- `??` means untracked: Git sees a new path but it is not staged.
- `M` means modified in the working tree but not staged.
- `M` means the staged version differs from the last commit.
- `MM` means a staged version exists and the file has additional unstaged edits.
- `A` means staged as a new file.
- `D` means deletion; stop and investigate unless deletion is deliberate.
- `R` means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 11.5.G4 — Inspect the unstaged diff for databricks/notebooks/08_gold_dimensions.py

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.

4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "databricks/notebooks/08_gold_dimensions.py"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "databricks/notebooks/08_gold_dimensions.py"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"databricks/notebooks/08_gold_dimensions.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Action 11.5.G5 — Stage only the reviewed file databricks/notebooks/08_gold_dimensions.py

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "databricks/notebooks/08_gold_dimensions.py"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "databricks/notebooks/08_gold_dimensions.py"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"databricks/notebooks/08_gold_dimensions.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 11.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Action 11.5.G7 — Create the Phase 11 commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "feat: build conformed gold dimensions"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "feat: build conformed gold dimensions"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"feat: build conformed gold dimensions"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Action 11.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Action 11.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 11 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message `feat: build conformed gold dimensions and the intended changed paths`.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 11.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.

7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 11 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

“Phase 11 created the verified boundary described by **Build Gold dimensions, surrogate keys, and SCD behavior**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed.”

Mini quiz

1. Multiple choice: Why does Phase 11 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

My Phase 11 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 11.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 12 resolves dimension keys while building facts at declared grains.

Phase 11 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 12 — Build Gold fact tables and reconciled business measures

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: Gold dimensions and facts with reconciled totals.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 13 converts correctness expectations into executable blocking tests.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 12 lesson index

1. Lesson 12.1 — Understand the phase and starting screen
2. Lesson 12.2 — Perform the notebook-guided actions one at a time
3. Lesson 12.3 — Create and understand the phase files
4. Lesson 12.4 — Build and validate required objects
5. Lesson 12.5 — Verify, troubleshoot, commit, teach back, and stop safely

Lesson 12.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** Databricks notebooks, Catalog Explorer, and SQL/Python cells.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 12 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 11 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 12 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

Databricks notebooks, Catalog Explorer, and SQL/Python cells. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 12 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 12 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 12 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 12.1 completion checklist

☐

I read why Phase 12 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 12.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 12 lesson index](#)

Lesson 12.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 12 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

Six facts at explicit grain: sales, returns, inventory snapshot, shipments, web sessions, and marketing spend. Each resolves dimension keys—including event-time SCD2—derives measures, prevents duplicate grain, handles unknown keys, reconciles to Silver, and optionally produces justified daily/monthly aggregates.

2. Why it matters

Facts are where double-counting becomes expensive. Grain determines keys, joins, and measure additivity. Resolving dimensions by natural key and event date preserves history; unknown handling and reconciliation keep incomplete context visible without dropping business events.

3. Words to know

Term	Plain-language definition
Transaction fact	One row per event, such as order item or return.
Periodic snapshot	One row per entity combination at a time point, such as daily stock.
Factless fact	Relationship/event fact with counts but few numeric measures.
Dimension lookup	Map a natural key/event date to a surrogate key.
Unknown-key rate	Share of facts using key 0.
Additive	Safe to sum across relevant dimensions.
Semi-additive	Safe across some dimensions but not time, such as inventory snapshot.
Non-additive	Ratio/percentage recalculated from components.
Degenerate dimension	Business identifier stored in fact, such as order_number.
Currency grain	Currency context that must be consistent/conversion-aware.
Event date / load date	When business event happened versus when platform processed it.
Aggregate table	Pre-summarized table used only when workload justifies it.
Duplicate prevention	Unique business grain and same-batch merge/replace.

4. What must already be completed

Gold dimensions pass all tests; Silver facts carry stable natural keys and valid measures; unknown rows exist. You can state each fact grain before coding.

5. Applications to open

Application	How to open it	Why it is open
Databricks notebook 09	Workspace.	Build fact_sales pattern and user-owned remaining facts.
Databricks SQL editor	Gold/Silver schemas.	Run grain/reconciliation/unknown checks.
VS Code	Open data dictionary, mapping, reference appendix after attempt.	Complete fact TODOs.
Architecture/star diagram	Open Gold star schema.	Trace relationships.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run Get-Location; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
SELECT order_item_id, COUNT(*) AS duplicate_count	Checks declared sales grain uniqueness.	Duplicate lines inflate every sales KPI.
WITH s AS (	Reconciles sales revenue Silver to Gold.	Dimension enrichment must not add/drop/multiply measures.
SELECT	Measures unresolved dimension context.	Key 0 is allowed but must be visible and thresholded.
SELECT snapshot_date_key, SUM(quantity_on_hand) AS on_hand	Shows stock per snapshot date.	Demonstrates correct time grain rather than summing all dates.
SELECT COUNT(*) AS delivered, SUM(CASE WHEN on_time_flag THEN 1 ELSE 0 END) AS on_time,	Validates shipment-level on-time numerator/denominator.	Prevents using order or line grain.

Fact relationship table

Fact	Grain	Core dimensions	Additive measures	Caution
fact_sales	valid order item	date, customer, product, store, employee, promotion, channel	quantity, sales, discount, cost, profit	orders are distinct, margin is ratio
fact_returns	return event/line	return date, customer, product, store	return quantity, refund	date-role compatibility
fact_inventory_snapshot	date-store-product	date, store, product	quantity/value across store/product	not across time
fact_shipments	shipment	ship/promised/delivery dates, customer, store	shipment count/cost	on-time is ratio
fact_web_sessions	session	date, customer, campaign, channel	session/event counts/duration	anonymous customer 0 legitimate
fact_marketing_spend	date-campaign-channel	date, campaign, channel	spend, impressions, clicks	currency/attribution

Ownership boundary

The workbook contains a complete teaching pattern for fact_sales and explicit TODOs for the remaining facts. You should implement them after the toy exercise, use three hint levels, and only then inspect the optional reference-answer section later in this workbook. This makes the project yours while keeping acceptance tests unambiguous.

MY TASK

Complete all five remaining fact transformations, replace unsafe ID shortcuts with natural-key/event-date lookups, add unique-grain and reconciliation tests, and justify whether any aggregate table is necessary.

STARTER SKELETON

```
# Example skeleton: fact_returns
returns = spark.table(f"{catalog}.{silver_schema}.returns").filter(F.col("batch_id") == batch_id)
# TODO(USER): join Silver sales/order context for store_code and purchase details.
# TODO(USER): lookup dim_customer/dim_product versions using natural key + return/purchase date policy.
# TODO(USER): map unresolved references to key 0 and retain return_id/order_item_id.
fact_returns = returns.select(
    (F.abs(F.xxhash64("return_id")) + 1).alias("return_key"),
    "return_id",
    # TODO(USER): remaining keys/measures/audit columns
)
# TODO(USER): assert unique return_id and reconcile quantity/refund to Silver.
```

Acceptance criteria: All six facts exist at documented unique grain, use correct dimension/date lookups, reconcile measures/rows under documented exclusions, track unknown rates, and remain stable on same-batch rerun.

HINT 1 — CONCEPT

Write one sentence grain, choose immutable business identifier, then list dimensions/measures before code.

HINT 2 — PSEUDOCODE

Pseudocode: Silver fact → natural key lookup → effective-date lookup → coalesce key 0 → derive measures → unique/reconcile tests.

HINT 3 — NEAR-COMPLETE PATTERN

Follow fact_sales structure, but adapt date roles and semi-additive behavior; open reference only after a failed attempt.

13. Expected files or output

- 6 Gold fact Delta tables.
- Grain/relationship matrix.
- Unknown-key monitoring.
- Silver-to-Gold count/amount reconciliations.
- Same-batch stable counts.
- Aggregate decision record.

14. How to verify

- Unique business grain per fact.
- No orphan surrogate keys.
- Sales arithmetic identity.
- Inventory selected-date behavior.
- Shipment sample on-time logic.
- Web conversion links do not duplicate order attribution unexpectedly.
- Spend USD/ranges valid.

15. Automated tests

- Unique-key and not-null tests.
- Dimension RI and unknown-rate thresholds.
- Date/numeric ranges.
- Silver-to-Gold row/amount reconciliation.
- Event-time SCD lookup test using controlled history.
- Same-batch rerun.
- Additivity tests/semantic cautions.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
All facts use unknown key	Natural key missing/case mismatch or date join wrong.	Anti-join Silver keys to dimension; inspect date ranges.	Carry standardized natural keys; join on key + event date; use 0 only unresolved.	Unknown rate expected.
Fact count exceeds Silver	Many-to-many dimension join/overlap.	Compare count after each join; dimension uniqueness/overlap tests.	Fix dimension SCD state or lookup condition.	Count reconciles.
Revenue differs	Rows lost/duplicated or formula/type/rounding changed.	Reconcile row/quantity/gross/discount/net stepwise.	Fix first divergent boundary; preserve decimal types.	Difference tolerance met.
Inventory total enormous	Summed multiple snapshots.	Group by snapshot date.	Filter latest/selected date; label semi-additive.	One-date validation matches source.
Fact rerun duplicates	Append without stable batch/grain delete/merge.	Group grain by batch/count.	Replace same batch or MERGE unique identifier.	Stable.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why is inventory semi-additive?
2. Why does a fact keep order_number even though it is not a dimension table?
3. How should an anonymous web session map customer?
4. Why is event-time SCD lookup necessary?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. What is the grain of fact_marketing_spend?
2. Which measure should not be summed across inventory dates?
3. What does a high unknown-product rate tell you?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture the star schema, six fact counts/grains, one SCD event-time lookup, revenue reconciliation, and same-batch duplicate test.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
feat: build and reconcile gold fact tables
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

- ☐ All six grains declared — [STATUS]
- ☐ fact_sales verified — [STATUS]
- ☐ returns/inventory/shipments completed — [STATUS]
- ☐ web/spend completed — [STATUS]
- ☐ SCD lookups verified — [STATUS]
- ☐ Unknown rates acceptable — [STATUS]
- ☐ Reconciliations pass — [STATUS]
- ☐ Rerun stable — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-12. The quiz answer is in the answer-key section immediately after the quiz in this workbook#phase-12.

Next phase: Phase 13 expands the tests into a stored, blocking publication quality gate.

Required action cards

Action 12.2.1 — Write the six grain statements before opening final code: sales one valid order item; returns one return event

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Write the six grain statements before opening final code: sales one valid order item; returns one return event. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Write the six grain statements before opening final code: sales one valid order item; returns one return event/line; inventory one snapshot date-store-product; shipments one shipment; web sessions one session; spend one date-campaign-channel.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 1

Action

Write the six grain statements before opening final code: sales one valid order item; returns one return event/line; inventory one snapshot date-store-product; shipments one shipment; web sessions one session; spend one date-campaign-channel.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Action 12.2.2 — Use a toy movie-ticket example to practice dimension lookup: a theater name change produces two versions; a ti

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Use a toy movie-ticket example to practice dimension lookup: a theater name change produces two versions; a ti. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Use a toy movie-ticket example to practice dimension lookup: a theater name change produces two versions; a ticket joins the version valid on show date.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 2****Action**

Use a toy movie-ticket example to practice dimension lookup: a theater name change produces two versions; a ticket joins the version valid on show date.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Action 12.2.3 — Open 09_gold_facts. Replace any ID regex shortcut with joins from Silver natural keys. For customer/product

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open 09_gold_facts. Replace any ID regex shortcut with joins from Silver natural keys. For customer/product . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open 09_gold_facts. Replace any ID regex shortcut with joins from Silver natural keys. For customer/product SCD2, include event_date BETWEEN effective_start_date AND effective_end_date.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 3****Action**

Open 09_gold_facts. Replace any ID regex shortcut with joins from Silver natural keys. For customer/product SCD2, include event_date BETWEEN effective_start_date AND effective_end_date.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Action 12.2.4 — Complete fact_sales, validating one row per order_item_id and amount identities.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Complete fact_sales, validating one row per order_item_id and amount identities. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Complete fact_sales, validating one row per order_item_id and amount identities.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 4

Action

Complete fact_sales, validating one row per order_item_id and amount identities.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Action 12.2.5 — Complete fact_returns: parse return date already in Silver, lookup customer/product/store/date, retain retur

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Complete fact_returns: parse return date already in Silver, lookup customer/product/store/date, retain retur. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Complete fact_returns: parse return date already in Silver, lookup customer/product/store/date, retain return ID/order item, quantity/refund/reason/currency. Unknown source references map to 0 and are counted.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 5****Action**

Complete fact_returns: parse return date already in Silver, lookup customer/product/store/date, retain return ID/order item, quantity/refund/reason/currency. Unknown source references map to 0 and are counted.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Action 12.2.6 — Complete fact_inventory_snapshot: one date/store/product; derive available quantity, risk flag, inventory va

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Complete fact_inventory_snapshot: one date/store/product; derive available quantity, risk flag, inventory va. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Complete fact_inventory_snapshot: one date/store/product; derive available quantity, risk flag, inventory value using event-valid product cost. Never aggregate stock across dates.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 6

Action

Complete fact_inventory_snapshot: one date/store/product; derive available quantity, risk flag, inventory value using event-valid product cost. Never aggregate stock across dates.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.



I can describe what changed on the screen or filesystem.



I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Action 12.2.7 — Complete fact_shipments: one shipment; date roles ship/promised/delivery, customer/store, carrier, delivery

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Complete fact_shipments: one shipment; date roles ship/promised/delivery, customer/store, carrier, delivery . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.

2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Complete fact_shipments: one shipment; date roles ship/promised/delivery, customer/store, carrier, delivery days, on-time flag.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 7****Action**

Complete fact_shipments: one shipment; date roles ship/promised/delivery, customer/store, carrier, delivery days, on-time flag.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

Action 12.2.8 — Complete fact_web_sessions: one session; customer/campaign/channel/date, duration, event counts, converted f

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Complete fact_web_sessions: one session; customer/campaign/channel/date, duration, event counts, converted f. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Complete fact_web_sessions: one session; customer/campaign/channel/date, duration, event counts, converted flag/order ID. Unknown anonymous customer can legitimately be 0.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 8

Action

Complete fact_web_sessions: one session; customer/campaign/channel/date, duration, event counts, converted flag/order ID. Unknown anonymous customer can legitimately be 0.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Action 12.2.9 — Complete fact_marketing_spend: one date/campaign/channel, USD spend, impressions, clicks.

Application:

Windows File Explorer

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Complete fact_marketing_spend: one date/campaign/channel, USD spend, impressions, clicks. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Complete fact_marketing_spend: one date/campaign/channel, USD spend, impressions, clicks.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows File Explorer. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 9

Action

Complete `fact_marketing_spend`: one date/campaign/channel, USD spend, impressions, clicks.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Action 12.2.10 — For every fact, enforce unique grain, required dimension keys/not-null, unknown threshold, range, and Silver r

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

For every fact, enforce unique grain, required dimension keys/not-null, unknown threshold, range, and Silver r. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: For every fact, enforce unique grain, required dimension keys/not-null, unknown threshold, range, and Silver reconciliation.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 10****Action**

For every fact, enforce unique grain, required dimension keys/not-null, unknown threshold, range, and Silver reconciliation.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Action 12.2.11 — Create daily/monthly sales aggregate only after querying demonstrates need; document exact grain and component

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Create daily/monthly sales aggregate only after querying demonstrates need; document exact grain and component. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Create daily/monthly sales aggregate only after querying demonstrates need; document exact grain and components. Ratios should not be stored as sums unless semantics are safe.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 11

Action

Create daily/monthly sales aggregate only after querying demonstrates need; document exact grain and components. Ratios should not be stored as sums unless semantics are safe.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Action 12.2.12 — Rerun same batch and prove no duplicate fact grain.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Rerun same batch and prove no duplicate fact grain. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Rerun same batch and prove no duplicate fact grain.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 12****Action**

Rerun same batch and prove no duplicate fact grain.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Action 12.2.C1 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT order_item_id, COUNT(*) AS duplicate_count
FROM northstar_retail.gold.fact_sales
GROUP BY order_item_id
HAVING COUNT(*) > 1;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT order_item_id, COUNT(*) AS duplicate_count` — `SELECT` requests data without changing it unless combined with a modifying statement; `order_item_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `duplicate_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 2:** FROM northstar_retail.gold.fact_sales — FROM identifies the source table or query; northstar_retail.gold.fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** GROUP BY order_item_id — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_item_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** HAVING COUNT(*) > 1; — HAVING is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; > is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Action 12.2.C2 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
WITH s AS (
  SELECT SUM(net_sales_amount) AS amount FROM northstar_retail.silver.sales WHERE batch_id='EXT-YYYYMMDDTHHMMSSZ'
), g AS (
  SELECT SUM(net_sales_amount) AS amount FROM northstar_retail.gold.fact_sales WHERE batch_id='EXT-YYYYMMDDTHHMMSSZ'
)
SELECT s.amount AS silver_amount, g.amount AS gold_amount, g.amount-s.amount AS difference FROM s CROSS JOIN g;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** WITH s AS (— WITH is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** SELECT SUM(net_sales_amount) AS amount FROM northstar_retail.silver.sales WHERE batch_id='EXT-YYYYMMDDTHHMMSSZ' — SELECT requests data without changing it unless combined with a modifying statement; SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; net_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; northstar_retail.silver.sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 3:** `) , g AS ( — )`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `g` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `SELECT SUM(net_sales_amount) AS amount FROM northstar_retail.gold.fact_sales WHERE batch_id='EXT-YYYYMMDDTHHMMSSZ' — SELECT` requests data without changing it unless combined with a modifying statement; `SUM` adds numeric values in the current filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `net_sales_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `northstar_retail.gold.fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `WHERE` filters rows to those meeting a condition; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `) — )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `SELECT s.amount AS silver_amount, g.amount AS gold_amount, g.amount-s.amount AS difference FROM s CROSS JOIN g; — SELECT` requests data without changing it unless combined with a modifying statement; `s.amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `silver_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `g.amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gold_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `g.amount-s.amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `difference` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Action 12.2.C3 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT
  COUNT(*) AS rows,
  SUM(CASE WHEN customer_key=0 THEN 1 ELSE 0 END) AS unknown_customer,
  SUM(CASE WHEN product_key=0 THEN 1 ELSE 0 END) AS unknown_product,
  SUM(CASE WHEN store_key=0 THEN 1 ELSE 0 END) AS unknown_store
FROM northstar_retail.gold.fact_sales
WHERE batch_id='EXT-YYYYMMDDTHHMMSSZ';
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** SELECT — SELECT requests data without changing it unless combined with a modifying statement
- **Line 2:** COUNT(*) AS rows, — COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** SUM(CASE WHEN customer_key=0 THEN 1 ELSE 0 END) AS unknown_customer, — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; THEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; ELSE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 4:** SUM(CASE WHEN product_key=0 THEN 1 ELSE 0 END) AS unknown_product, — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; THEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; ELSE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** SUM(CASE WHEN store_key=0 THEN 1 ELSE 0 END) AS unknown_store — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; store_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; THEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; ELSE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 6:** FROM northstar_retail.gold.fact_sales — FROM identifies the source table or query; northstar_retail.gold.fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** WHERE batch_id='EXT-YYYYMMDDTHHMMSSZ'; — WHERE filters rows to those meeting a condition; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'EXT-YYYYMMDDTHHMMSSZ' is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Action 12.2.C4 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT snapshot_date_key, SUM(quantity_on_hand) AS on_hand
FROM northstar_retail.gold.fact_inventory_snapshot
GROUP BY snapshot_date_key
ORDER BY snapshot_date_key;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** SELECT snapshot_date_key, SUM(quantity_on_hand) AS on_hand — SELECT requests data without changing it unless combined with a modifying statement; snapshot_date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quantity_on_hand is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on_hand is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** FROM northstar_retail.gold.fact_inventory_snapshot — FROM identifies the source table or query; northstar_retail.gold.fact_inventory_snapshot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** GROUP BY snapshot_date_key — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; snapshot_date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** ORDER BY snapshot_date_key; — ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; snapshot_date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Action 12.2.C5 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.

7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT COUNT(*) AS delivered, SUM(CASE WHEN on_time_flag THEN 1 ELSE 0 END) AS on_time,  
      SUM(CASE WHEN on_time_flag THEN 1 ELSE 0 END) / NULLIF(COUNT(*),0) AS on_time_rate  
FROM northstar_retail.gold.fact_shipments  
WHERE delivery_date_key <> 0;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** SELECT COUNT(*) AS delivered, SUM(CASE WHEN on_time_flag THEN 1 ELSE 0 END) AS on_time, — SELECT requests data without changing it unless combined with a modifying statement; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; delivered is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on_time_flag is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; THEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 2:** SUM(CASE WHEN on_time_flag THEN 1 ELSE 0 END) / NULLIF(COUNT(*),0) AS on_time_rate — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on_time_flag is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; THEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; ELSE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.
- **Line 3:** FROM northstar_retail.gold.fact_shipments — FROM identifies the source table or query; northstar_retail.gold.fact_shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** WHERE delivery_date_key <> 0; — WHERE filters rows to those meeting a condition; delivery_date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; <> is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 12.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 12 starting-screen checklist and re-verify the active context.

Lesson 12.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 12 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — databricks/

Exact folder name: databricks

Exact parent: repository root

Why it is needed now: Phase 12 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: databricks.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 12.D1.1 — Create the folder databricks

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.

6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\databricks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\databricks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\databricks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\databricks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\databricks" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — `sql/`

Exact folder name: `sql`

Exact parent: `repository root`

Why it is needed now: Phase 12 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `repository root` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `sql`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 12.D2.1 — Create the folder `sql`

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql" -PathType Container` and require True.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql" -PathType Container`.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — databricks/notebooks/

Exact folder name: notebooks

Exact parent: databricks

Why it is needed now: Phase 12 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path databricks one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: notebooks.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 12.D3.1 — Create the folder databricks/notebooks

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\databricks\notebooks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\databricks\notebooks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\notebooks"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\databricks\notebooks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\databricks\notebooks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\databricks\notebooks" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/databricks/

Exact folder name: databricks

Exact parent: sql

Why it is needed now: Phase 12 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path sql one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: databricks.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 12.D4.1 — Create the folder sql/databricks

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\databricks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\sql\databricks" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\sql\databricks" is a path,

relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\sql\databricks" -PathType Container` and require True.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\databricks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\databricks" -PathType Container`.
2. Require the output True before creating a child file.

3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/databricks/gold/

Exact folder name: gold

Exact parent: sql/databricks

Why it is needed now: Phase 12 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path sql/databricks one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: gold.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 12.D5.1 — Create the folder sql/databricks/gold

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):


```
New-Item -ItemType Directory -Path ".\sql\databricks\gold" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\databricks\gold" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\databricks\gold"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\sql\databricks\gold" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\databricks\gold" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\databricks\gold" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — databricks/notebooks/09_gold_facts.py

Exact filename: `09_gold_facts.py`

Filename extension: `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `databricks/notebooks` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 12.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `databricks/notebooks` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `09_gold_facts.py`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `09_gold_facts.py` and an empty file under the correct parent folder.
10. PRESS: `Ctrl+S` after adding each explained section.

PowerShell path

Action 12.F1.1 — Create the file `databricks/notebooks/09_gold_facts.py` from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\databricks\notebooks\09_gold_facts.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\databricks\notebooks\09_gold_facts.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\notebooks\09_gold_facts.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\databricks\notebooks\09_gold_facts.py" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "databricks/notebooks/09_gold_facts.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `databricks/notebooks`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `databricks/notebooks/09_gold_facts.py`.
3. In PowerShell at the repository root, run `Test-Path ".\databricks\notebooks\09_gold_facts.py"` and require `True`.
4. Run `git status --short -- "databricks/notebooks/09_gold_facts.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for databricks/notebooks/09_gold_facts.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: databricks/notebooks/09_gold_facts.py

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 12.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Databricks notebook source
# Northstar Retail End-to-End Data Platform
# This file is a Git-friendly Databricks source notebook.
# Track A reads files that you manually uploaded to a Unity Catalog volume.
# It never attempts to reach SQL Server or PostgreSQL on localhost.

# COMMAND -----
# MAGIC %md
# MAGIC # 09 – Gold facts
# MAGIC Looks up effective dimension keys at the event date and builds facts at declared grain.

# COMMAND -----
from pyspark.sql import functions as F
from pyspark.sql.window import Window
for name, default in {"catalog": "northstar_retail", "silver_schema": "silver", "gold_schema": "gold", "batch_id": "MANUAL-REPLACE-ME"}.items():
    dbutils.widgets.text(name, default)
catalog=dbutils.widgets.get("catalog"); silver_schema=dbutils.widgets.get("silver_schema"); gold_schema=dbutils.widgets.get("gold_schema");
batch_id=dbutils.widgets.get("batch_id")
if batch_id=="MANUAL-REPLACE-ME": raise ValueError("Pass batch_id from workflow.")

def overwrite_batch(df, table_name: str):
    full=f"{catalog}.{gold_schema}.{table_name}"
    if spark.catalog.tableExists(full): spark.sql(f"DELETE FROM {full} WHERE batch_id='{batch_id}'")
    df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full)

sales=spark.table(f"{catalog}.{silver_schema}.sales").filter(F.col("batch_id")==batch_id)
customers=spark.table(f"{catalog}.{gold_schema}.dim_customer")
products=spark.table(f"{catalog}.{gold_schema}.dim_product")
stores=spark.table(f"{catalog}.{gold_schema}.dim_store")
employees=spark.table(f"{catalog}.{gold_schema}.dim_employee")
promotions=spark.table(f"{catalog}.{gold_schema}.dim_promotion")
channels=spark.table(f"{catalog}.{gold_schema}.dim_channel")

fact_sales=(sales.alias("s")
    .join(customers.alias("c"), (F.col("s.customer_id").cast("long")==F.regexp_extract(F.col("c.customer_number"), r"(\d+)$", 1).cast("long"))) &
    (F.col("s.order_date").between(F.col("c.effective_start_date"), F.col("c.effective_end_date"))), "left")
    .join(products.alias("p"), F.col("s.product_id").cast("long")==F.regexp_extract(F.col("p.sku"), r"(\d+)$", 1).cast("long"), "left")
    .join(stores.alias("st"), F.col("s.store_id").cast("long")==F.regexp_extract(F.col("st.store_code"), r"(\d+)$", 1).cast("long"), "left")
    .join(employees.alias("e"), F.col("s.employee_id").cast("long")==F.regexp_extract(F.col("e.employee_number"), r"(\d+)$", 1).cast("long"), "left")
    .join(promotions.alias("pr"), F.coalesce(F.col("s.promotion_code"), F.lit("NONE"))==F.col("pr.promotion_code"), "left")
    .join(channels.alias("ch"), F.col("s.channel_code")==F.col("ch.channel_code"), "left")
    .select(
        (F.abs(F.xxhash64("s.order_item_id"))+1).alias("sales_key"), "s.order_item_id", "s.order_id", "s.order_number",
        F.date_format("s.order_date", "yyyyMMdd").cast("int").alias("order_date_key"),
        F.coalesce("c.customer_key", F.lit(0)).alias("customer_key"), F.coalesce("p.product_key", F.lit(0)).alias("product_key"),
        F.coalesce("st.store_key", F.lit(0)).alias("store_key"), F.coalesce("e.employee_key", F.lit(0)).alias("employee_key"),
        F.coalesce("pr.promotion_key", F.lit(0)).alias("promotion_key"), F.coalesce("ch.channel_key", F.lit(0)).alias("channel_key"),
        F.col("s.quant-
ity_int").ali-
as("quant-
ity"), F.col("s.gross_sales_amount"), F.col("s.discount_dec").alias("discount_amount"), F.col("s.net_sales_amount"), F.col("s.tax_dec").alias("tax-
amount"),

F.col("s.unit_cost_dec").alias("unit_cost_amount"), F.col("s.cost_of_goods_sold"), F.col("s.gross_profit_amount"), "s.currency_code", F.current_timest-
amp().alias("load_timestamp_utc"), F.lit(batch_id).alias("batch_id"), F.lit("CONFORMED").alias("source_system"))
overwrite_batch(fact_sales, "fact_sales")

# TODO(USER): Complete the remaining fact lookups after validating fact sales.
# Acceptance criteria and near-complete patterns are in Phase 12. The skeleton below creates
# the expected table names only when your source table is available, preventing false results.
for target, source in [
    ("fact_returns", "returns"), ("fact_inventory_snapshot", "inventory_snapshots"),
    ("fact_shipments", "shipments"), ("fact_web_sessions", "web_sessions"), ("fact_marketing_spend", "marketing_spend")
]:
    source_name=f"{catalog}.{silver_schema}.{source}"
    if not spark.catalog.tableExists(source_name):
        raise ValueError(f"Required Silver table missing: {source_name}")
    # Reference implementation is deliberately separated in the optional reference-answer section later in this workbook.

if fact_sales.groupBy("order_item_id").count().filter("count>1").count(): raise AssertionError("Duplicate fact_sales grain")
raise NotImplementedError("TODO(USER): build and validate fact_returns, fact_inventory_snapshot, fact_shipments, fact_web_sessions, and fact_marketing_spend, then remove this deliberate stop and set gold_facts_status=PASS.")
```

Line-by-line explanation

- **Line 1:** `# Databricks notebook source` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `# Northstar Retail End-to-End Data Platform` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** `# This file is a Git-friendly Databricks source notebook.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** `# Track A reads files that you manually uploaded to a Unity Catalog volume.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** `# It never attempts to reach SQL Server or PostgreSQL on localhost.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 9:** `# MAGIC %md` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 10:** `# MAGIC # 09 – Gold facts` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 11:** `# MAGIC Looks up effective dimension keys at the event date and builds facts at declared grain.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 14:** `from pyspark.sql import functions as F — from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `functions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.
- **Line 15:** `from pyspark.sql.window import Window — from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pyspark.sql.window` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Window` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.
- **Line 16:** `for name, default in {"catalog": "northstar_retail", "silver_schema": "silver", "gold_schema": "gold", "batch_id": "MANUAL-REPLACE-ME"}.items(): dbutils.widgets.text(name, default) — for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `default` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"catalog"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"northstar_retail"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"silver_schema"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 17:** `catalog=dbutils.widgets.get("catalog"); silver_schema=dbutils.widgets.get("silver_schema"); gold_schema=dbutils.widgets.get("gold_schema"); batch_id=dbutils.widgets.get("batch_id") — catalog` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; "catalog" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; silver_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.widgets.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "silver_schema" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 18:** `if batch_id=="MANUAL-REPLACE-ME": raise ValueError("Pass batch_id from workflow.")` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"MANUAL-REPLACE-ME"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ValueError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Pass batch_id from workflow."` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** blank line used only to separate logical sections for readability.
- **Line 20:** blank line used only to separate logical sections for readability.
- **Line 21:** `def overwrite_batch(df, table_name: str):` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `overwrite_batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `df` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** `full=f"{catalog}.{gold_schema}.{table_name}"` — `full` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.{gold_schema}.{table_name}"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 23:** `if spark.catalog.tableExists(full): spark.sql(f"DELETE FROM {full} WHERE batch_id='{batch_id}')` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.catalog.tableExists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"DELETE FROM {full} WHERE batch_id='{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** `df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable(full)` — `df.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"mergeSchema"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"true"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

surrounding explanation; "append" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "mergeSchema" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 25:** blank line used only to separate logical sections for readability.
- **Line 26:** `sales=spark.table(f"{catalog}.{silver_schema}.sales").filter(F.col("batch_id")==batch_id)` — `sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.{silver_schema}.sales"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 27:** `customers=spark.table(f"{catalog}.{gold_schema}.dim_customer")` — `customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.{gold_schema}.dim_customer"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** `products=spark.table(f"{catalog}.{gold_schema}.dim_product")` — `products` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.{gold_schema}.dim_product"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** `stores=spark.table(f"{catalog}.{gold_schema}.dim_store")` — `stores` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.{gold_schema}.dim_store"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 30:** `employees=spark.table(f"{catalog}.{gold_schema}.dim_employee")` — `employees` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.{gold_schema}.dim_employee"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 31:** `promotions=spark.table(f"{catalog}.{gold_schema}.dim_promotion")` — `promotions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.{gold_schema}.dim_promotion"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 32:** `channels=spark.table(f"{catalog}.{gold_schema}.dim_channel")` — `channels` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.{gold_schema}.dim_channel"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** blank line used only to separate logical sections for readability.
- **Line 34:** `fact_sales=(sales.alias("s")` — `fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sales.alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"s"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** `.join(customers.alias("c"),(F.col("s.customer_id").cast("long")==F.regexp_extract(F.col("c.customer_number"),r"(\d+)$",1).cast("long")) & (F.col("s.order_date").between(F.col("c.effective_start_date"),F.col("c.effective_end_date"))),"left")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `join` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customers.alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"c"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`, `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"s.customer_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`. `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 36:** `.join(products.alias("p"),F.col("s.product_id").cast("long")==F.regexp_extract(F.col("p.sku"),r"(\d+)$",1).cast("long"),"left")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `join` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `products.alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"p"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`, `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"s.product_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`. `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 37:** `.join(stores.alias("st"),F.col("s.store_id").cast("long")==F.regexp_extract(F.col("st.store_code"),r"(\d+)`

`$",1).cast("long"),"left")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `join` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `stores.alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"st"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"s.store_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 38:** `.join(employ-`

`ees.alias("e"),F.col("s.employee_id").cast("long")==F.regexp_extract(F.col("e.employee_number"),r"(\d+)$",1).cast("long"),"left")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `join` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `employees.alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"e"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"s.employee_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`. is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 39:** `.join(promo-`

`tions.alias("pr"),F.coalesce(F.col("s.promotion_code"),F.lit("NONE"))==F.col("pr.promotion_code"),"left")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `join` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `promotions.alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"pr"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.coalesce` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"s.promotion_code"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 40:** `.join(channels.alias("ch"),F.col("s.channel_code")==F.col("ch.channel_code"),"left")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; `join` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `channels.alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"ch"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"s.channel_code"` is a quoted literal; the quotation marks preserve spaces or mark text; `)==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 41:** `.select( — .` is a path, relative location, branch-style name, or file argument; verify every character before running it; `select` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** `(F.abs(F.xxhash64("s.order_item_id"))+1).alias("sales_key"), "s.order_item_id", "s.order_id", "s.order_number", — (` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.abs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.xxhash64` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"s.order_item_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"sales_key"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 43:** `F.date_format("s.order_date", "yyyyMMdd").cast("int").alias("order_date_key"), — F.date_format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"s.order_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"yyyyMMdd"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"int"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 44:** `F.coalesce("c.customer_key", F.lit(0)).alias("customer_key"), F.coalesce("p.product_key", F.lit(0)).alias("product_key"), — F.coalesce` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"c.customer_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"customer_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 45:** `F.coalesce("st.store_key", F.lit(0)).alias("store_key"), F.coalesce("e.employee_key", F.lit(0)).alias("employee_key"), — F.coalesce` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"st.store_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)` `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `alias` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"store_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` `,` is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

• **Line 46:** F.co-

alesce("pr.promotion_key",F.lit(0)).alias("promotion_key"),F.coalesce("ch.channel_key",F.lit(0)).alias("channel_key"), — F.coalesce is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "pr.promotion_key" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "promotion_key" is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

• **Line 47:** F.col("s.quant-

ity_int").ali-

as("quant-

ity"),F.col("s.gross_sales_amount"),F.col("s.discount_dec").alias("discount_amount"),F.col("s.net_sales_amount"),F.col("s.tax_dec").alias("tax_amount"), — F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "s.quantity_int" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "quantity" is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "s.gross_sales_amount" is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

• **Line 48:**

F.col("s.unit_cost_dec").alias("unit_cost_amount"),F.col("s.cost_of_goods_sold"),F.col("s.gross_profit_amount"),"s.currency_code",F.current_timestamp().alias("load_timestamp_utc"),F.lit(batch_id).alias("batch_id"),F.lit("CONFORMED").alias("source_system")) — F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "s.unit_cost_dec" is a quoted literal; the quotation marks preserve spaces or mark text;) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; alias is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "unit_cost_amount" is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "s.cost_of_goods_sold" is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

• **Line 49:** overwrite_batch(fact_sales,"fact_sales") — overwrite_batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "fact_sales" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

• **Line 50:** blank line used only to separate logical sections for readability.

- **Line 51:** # TODO(USER): Complete the remaining fact lookups after validating fact_sales. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 52:** # Acceptance criteria and near-complete patterns are in Phase 12. The skeleton below creates is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 53:** # the expected table names only when your source table is available, preventing false results. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 54:** for target, source in [— for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; target is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 55:** ("fact_returns", "returns"), ("fact_inventory_snapshot", "inventory_snapshots"), — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "fact_returns" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "returns" is a quoted literal; the quotation marks preserve spaces or mark text;), (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "fact_inventory_snapshot" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "inventory_snapshots" is a quoted literal; the quotation marks preserve spaces or mark text;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 56:** ("fact_shipments", "shipments"), ("fact_web_sessions", "web_sessions"), ("fact_marketing_spend", "marketing_spend") — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "fact_shipments" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "shipments" is a quoted literal; the quotation marks preserve spaces or mark text;), (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "fact_web_sessions" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "web_sessions" is a quoted literal; the quotation marks preserve spaces or mark text;), (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "fact_marketing_spend" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "marketing_spend" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 57:**]: —]: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 58:** source_name=f"{catalog}.{silver_schema}.{source}" — source_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{catalog}.{silver_schema}.{source}" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 59:** if not spark.catalog.tableExists(source_name): — if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spark.catalog.tableExists is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;): is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 60:** raise ValueError(f"Required Silver table missing: {source_name}") — raise is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ValueError is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Required Silver table missing: {source_name}" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 61:** # Reference implementation is deliberately separated in the optional reference-answer section later in this workbook. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 62:** blank line used only to separate logical sections for readability.
- **Line 63:** `if fact_sales.groupBy("order_item_id").count().filter("count>1").count(): raise AssertionError("Duplicate fact_sales grain")` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact_sales.groupBy` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"order_item_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"count>1"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 64:** `raise NotImplementedError("TODO(USER): build and validate fact_returns, fact_inventory_snapshot, fact_shipments, fact_web_sessions, and fact_marketing_spend, then remove this deliberate stop and set gold_facts_status=PASS.")` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NotImplementedError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"TODO(USER): build and validate fact_returns, fact_inventory_snapshot, fact_shipments, fact_web_sessions, and fact_marketing_spend, then remove this deliberate stop and set gold_facts_status=PASS."` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 12 lesson index](#)

File introduction — `sql/databricks/gold/02_kpi_views.sql`

Exact filename: `02_kpi_views.sql`

Filename extension: `.sql`. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `sql/databricks/gold` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 12.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `sql/databricks/gold` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `02_kpi_views.sql`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `02_kpi_views.sql` and an empty file under the correct parent folder.

10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 12.F2.1 — Create the file `sql/databricks/gold/02_kpi_views.sql` from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\databricks\gold\02_kpi_views.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item` -ItemType File -Path ".\sql\databricks\gold\02_kpi_views.sql" -ErrorAction Stop — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\sql\databricks\gold\02_kpi_views.sql` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\sql\databricks\gold\02_kpi_views.sql" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/databricks/gold/02_kpi_views.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.

7. Use TODO(USER) where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under sql/databricks/gold.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with sql/databricks/gold/02_kpi_views.sql.
3. In PowerShell at the repository root, run Test-Path ".\sql\databricks\gold\02_kpi_views.sql" and require True.
4. Run git status --short -- "sql/databricks/gold/02_kpi_views.sql" to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for sql/databricks/gold/02_kpi_views.sql

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: sql/databricks/gold/02_kpi_views.sql

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 12.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: Databricks SQL / Spark SQL
CREATE OR REPLACE VIEW IDENTIFIER(:catalog || '.gold_semantic.executive_daily_kpi') AS
SELECT
  d.full_date,
  SUM(f.net_sales_amount) AS total_revenue,
  SUM(f.gross_profit_amount) AS gross_profit,
  COUNT(DISTINCT f.order_id) AS orders,
  SUM(f.quantity) AS units_sold,
  SUM(f.net_sales_amount) / NULLIF(COUNT(DISTINCT f.order_id), 0) AS average_order_value
FROM IDENTIFIER(:catalog || '.gold_fact_sales') AS f
JOIN IDENTIFIER(:catalog || '.gold.dim_date') AS d ON d.date_key = f.order_date_key
GROUP BY d.full_date;
```

Line-by-line explanation

- **Line 1:** -- Dialect: Databricks SQL / Spark SQL is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** CREATE OR REPLACE VIEW IDENTIFIER(:catalog || '.gold_semantic.executive_daily_kpi') AS — CREATE defines a new database object; OR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REPLACE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VIEW is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IDENTIFIER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; catalog is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '.gold_semantic.executive_daily_kpi' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** SELECT — SELECT requests data without changing it unless combined with a modifying statement
- **Line 4:** d.full_date, — d.full_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** SUM(f.net_sales_amount) AS total_revenue, — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.net_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; total_revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** SUM(f.gross_profit_amount) AS gross_profit, — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.gross_profit_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

role is described in the surrounding explanation; gross_profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- Line 7:** COUNT(DISTINCT f.order_id) AS orders, — COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DISTINCT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 8:** SUM(f.quantity) AS units_sold, — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.quantity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; units_sold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 9:** SUM(f.net_sales_amount) / NULLIF(COUNT(DISTINCT f.order_id), 0) AS average_order_value — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.net_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; NULLIF is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DISTINCT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 10:** FROM IDENTIFIER(:catalog || '.gold.fact_sales') AS f — FROM identifies the source table or query; IDENTIFIER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; catalog is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '.gold.fact_sales' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 11:** JOIN IDENTIFIER(:catalog || '.gold.dim_date') AS d ON d.date_key = f.order_date_key — JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IDENTIFIER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; catalog is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '.gold.dim_date' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; d is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; d.date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.order_date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 12:** GROUP BY d.full_date; — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

specific role is described in the surrounding explanation; `d.full_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 12 lesson index](#)

What your screen should look like now

- Every Phase 12 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 12.3 completion checklist

☐

`databricks/notebooks/09_gold_facts.py` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`sql/databricks/gold/02_kpi_views.sql` was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with `Ctrl+S`. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 12.4](#) and recheck the root and branch.

Lesson 12.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Gold fact 1 — `gold.fact_sales`

Declared grain: one row per valid order item.

Dimension keys: date, customer, product, store, employee, promotion, channel. Each lookup must name its business key and effective-date rule.

Measures: `quantity`, `gross_sales_amount`, `discount_amount`, `net_sales_amount`, `cost_of_goods_sold`, `gross_profit_amount`.

Additivity: Monetary and quantity measures are generally additive across compatible dimensions; snapshot balances are not safely additive across snapshot dates; ratios and flags must be aggregated with documented denominators.

Unknown keys: Use key 0 rather than dropping the fact, then monitor the unknown-key rate.

Duplicate prevention: Define a stable business-event key at the declared grain and enforce same-batch idempotency.

Incremental rule: Process the named batch or watermark range and do not reprocess unrelated history unless the repair strategy says so.

Action 12.4.F1 — Build and reconcile `gold.fact_sales`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and reconcile gold.fact_sales. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open databricks/notebooks/09_gold_facts.py.
2. TYPE THIS: Write the grain one row per valid order item above the fact-building section.
3. LOOK FOR: Confirm every required dimension is already verified.
4. TYPE THIS: Select one row per business event at the declared grain.
5. TYPE THIS: Resolve each dimension key by business key and event/effective date.
6. TYPE THIS: Map missing dimension references to unknown key 0 and increment a monitored exception count.
7. TYPE THIS: Derive measures only from documented source columns and formulas.
8. TYPE THIS: Add a deterministic duplicate-prevention key and current batch metadata.
9. PRESS: Run the batch-scoped write.
10. PRESS: Run source-to-fact count and amount reconciliation.
11. PRESS: Run the same batch again.
12. DO NOT CONTINUE UNTIL: Counts/totals reconcile, no unintended duplicate appears, and sample business questions return explainable results.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/09_gold_facts.py.
2. TYPE THIS: Write the grain one row per valid order item above the fact-building section.
3. LOOK FOR: Confirm every required dimension is already verified.
4. TYPE THIS: Select one row per business event at the declared grain.
5. TYPE THIS: Resolve each dimension key by business key and event/effective date.
6. TYPE THIS: Map missing dimension references to unknown key 0 and increment a monitored exception count.
7. TYPE THIS: Derive measures only from documented source columns and formulas.
8. TYPE THIS: Add a deterministic duplicate-prevention key and current batch metadata.
9. PRESS: Run the batch-scoped write.
10. PRESS: Run source-to-fact count and amount reconciliation.
11. PRESS: Run the same batch again.
12. DO NOT CONTINUE UNTIL: Counts/totals reconcile, no unintended duplicate appears, and sample business questions return explainable results.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Teaching fact-build pattern

```
fact_source = spark.table("gold_stage.fact_sales")
deduped = fact_source.dropDuplicates(["_business_event_key"])
resolved = (deduped
    .withColumn("date_key", F.coalesce(F.col("date_key"), F.lit(0)))
    .withColumn("_batch_id", F.lit(batch_id)))
(resolved.write.format("delta")
    .mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable("gold.fact_sales"))
```

Line-by-line explanation

- **Line 1:** `fact_source = spark.table("gold_stage.fact_sales")` — `fact_source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"gold_stage.fact_sales"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `deduped = fact_source.dropDuplicates(["_business_event_key"])` — `deduped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact_source.dropDuplicates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `["_business_event_key"]` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `resolved = (deduped` — `resolved` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `deduped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `.withColumn("date_key", F.coalesce(F.col("date_key"), F.lit(0)))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"date_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.coalesce` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"date_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `(resolved.write.format("delta")` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `resolved.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.mode("overwrite")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `.option("replaceWhere", f"_batch_id = '{batch_id}'")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"replaceWhere"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id = '{batch_id}'"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.saveAsTable("gold.fact_sales")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"gold.fact_sales"` is a quoted literal; the quotation marks preserve spaces or mark text; `))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Reconciliation worksheet

Check	Upstream value	Gold value	Difference	Pass condition	My verified result
Accepted event rows	[ENTER]	[ENTER]	[CALCULATE]	zero unless explicitly aggregated	[ENTER]
Primary quantity	[ENTER]	[ENTER]	[CALCULATE]	zero within declared logic	[ENTER]
Primary amount	[ENTER]	[ENTER]	[CALCULATE]	within rounding tolerance	[ENTER]
Unknown dimension keys	[ENTER]	[ENTER]	n/a	monitored and explained	[ENTER]
Duplicate business-event keys	n/a	[ENTER]	n/a	zero	[ENTER]
Same-batch rerun row growth	n/a	[ENTER]	n/a	zero unintended growth	[ENTER]

Sample business questions

1. What does one row mean, and which dimensions can filter it?
2. Which measures can be summed across dates, and which cannot?
3. Which source rows reconcile to one chosen fact row?
4. What happens when a dimension record arrives late?
5. What query proves that a rerun did not duplicate the business event?

Interview-ready explanation

"I built `fact_sales` at one row per valid order item, resolved conformed dimension keys with explicit unknown handling, derived documented measures, and reconciled both counts and amounts. A deterministic event key made incremental reruns testable."

Gold fact 2 — `gold.fact_returns`

Declared grain: one row per accepted returned product line.

Dimension keys: return date, customer, product, store, channel. Each lookup must name its business key and effective-date rule.

Measures: `return_quantity`, `refund_amount`.

Additivity: Monetary and quantity measures are generally additive across compatible dimensions; snapshot balances are not safely additive across snapshot dates; ratios and flags must be aggregated with documented denominators.

Unknown keys: Use key 0 rather than dropping the fact, then monitor the unknown-key rate.

Duplicate prevention: Define a stable business-event key at the declared grain and enforce same-batch idempotency.

Incremental rule: Process the named batch or watermark range and do not reprocess unrelated history unless the repair strategy says so.

Action 12.4.F2 — Build and reconcile `gold.fact_returns`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and reconcile `gold.fact_returns`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/09_gold_facts.py`.
2. TYPE THIS: Write the grain `one row per accepted returned product line` above the fact-building section.
3. LOOK FOR: Confirm every required dimension is already verified.
4. TYPE THIS: Select `one row per business event` at the declared grain.
5. TYPE THIS: Resolve each dimension key by business key and event/effective date.
6. TYPE THIS: Map missing dimension references to unknown key 0 and increment a monitored exception count.
7. TYPE THIS: Derive measures only from documented source columns and formulas.
8. TYPE THIS: Add a deterministic duplicate-prevention key and current batch metadata.
9. PRESS: Run the batch-scoped write.
10. PRESS: Run source-to-fact count and amount reconciliation.
11. PRESS: Run the same batch again.
12. DO NOT CONTINUE UNTIL: Counts/totals reconcile, no unintended duplicate appears, and sample business questions return explainable results.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/09_gold_facts.py.
2. TYPE THIS: Write the grain one row per accepted returned product line above the fact-building section.
3. LOOK FOR: Confirm every required dimension is already verified.
4. TYPE THIS: Select one row per business event at the declared grain.
5. TYPE THIS: Resolve each dimension key by business key and event/effective date.
6. TYPE THIS: Map missing dimension references to unknown key 0 and increment a monitored exception count.
7. TYPE THIS: Derive measures only from documented source columns and formulas.
8. TYPE THIS: Add a deterministic duplicate-prevention key and current batch metadata.
9. PRESS: Run the batch-scoped write.
10. PRESS: Run source-to-fact count and amount reconciliation.
11. PRESS: Run the same batch again.
12. DO NOT CONTINUE UNTIL: Counts/totals reconcile, no unintended duplicate appears, and sample business questions return explainable results.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Teaching fact-build pattern

```
fact_source = spark.table("gold_stage.fact_returns")
deduped = fact_source.dropDuplicates(["_business_event_key"])
resolved = (deduped
    .withColumn("date_key", F.coalesce(F.col("date_key"), F.lit(0)))
    .withColumn("_batch_id", F.lit(batch_id)))
(resolved.write.format("delta")
    .mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable("gold.fact_returns"))
```

Line-by-line explanation

- **Line 1:** `fact_source = spark.table("gold_stage.fact_returns")` — `fact_source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; spark.table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "gold_stage.fact_returns" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 2:** deduped = fact_source.dropDuplicates(["_business_event_key"]) — deduped is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_source.dropDuplicates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ([is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_business_event_key" is a quoted literal; the quotation marks preserve spaces or mark text;]) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** resolved = (deduped — resolved is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; deduped is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** .withColumn("date_key", F.coalesce(F.col("date_key"), F.lit(0))) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "date_key" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.coalesce is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "date_key" is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** .withColumn("_batch_id", F.lit(batch_id))) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;))) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** (resolved.write.format("delta") — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; resolved.write.format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** .mode("overwrite") — . is a path, relative location, branch-style name, or file argument; verify every character before running it; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** .option("replaceWhere", f"_batch_id = '{batch_id}'") — . is a path, relative location, branch-style name, or file argument; verify every character before running it; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; "_batch_id = '{batch_id}'" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 9:** .saveAsTable("gold.fact_returns")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "gold.fact_returns" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Reconciliation worksheet

Check	Upstream value	Gold value	Difference	Pass condition	My verified result
Accepted event rows	[ENTER]	[ENTER]	[CALCULATE]	zero unless explicitly aggregated	[ENTER]
Primary quantity	[ENTER]	[ENTER]	[CALCULATE]	zero within declared logic	[ENTER]
Primary amount	[ENTER]	[ENTER]	[CALCULATE]	within rounding tolerance	[ENTER]
Unknown dimension keys	[ENTER]	[ENTER]	n/a	monitored and explained	[ENTER]
Duplicate business-event keys	n/a	[ENTER]	n/a	zero	[ENTER]
Same-batch rerun row growth	n/a	[ENTER]	n/a	zero unintended growth	[ENTER]

Sample business questions

1. What does one row mean, and which dimensions can filter it?
2. Which measures can be summed across dates, and which cannot?
3. Which source rows reconcile to one chosen fact row?
4. What happens when a dimension record arrives late?
5. What query proves that a rerun did not duplicate the business event?

Interview-ready explanation

"I built fact_returns at one row per accepted returned product line, resolved conformed dimension keys with explicit unknown handling, derived documented measures, and reconciled both counts and amounts. A deterministic event key made incremental reruns testable."

Gold fact 3 — gold.fact_inventory_snapshot

Declared grain: one row per product-store snapshot date.

Dimension keys: snapshot date, product, store. Each lookup must name its business key and effective-date rule.

Measures: quantity_on_hand, quantity_reserved, reorder_point, stockout_risk_flag.

Additivity: Monetary and quantity measures are generally additive across compatible dimensions; snapshot balances are not safely additive across snapshot dates; ratios and flags must be aggregated with documented denominators.

Unknown keys: Use key 0 rather than dropping the fact, then monitor the unknown-key rate.

Duplicate prevention: Define a stable business-event key at the declared grain and enforce same-batch idempotency.

Incremental rule: Process the named batch or watermark range and do not reprocess unrelated history unless the repair strategy says so.

Action 12.4.F3 — Build and reconcile gold.fact_inventory_snapshot

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and reconcile gold.fact_inventory_snapshot. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open databricks/notebooks/09_gold_facts.py.
2. TYPE THIS: Write the grain one row per product-store snapshot date above the fact-building section.
3. LOOK FOR: Confirm every required dimension is already verified.
4. TYPE THIS: Select one row per business event at the declared grain.
5. TYPE THIS: Resolve each dimension key by business key and event/effective date.
6. TYPE THIS: Map missing dimension references to unknown key 0 and increment a monitored exception count.
7. TYPE THIS: Derive measures only from documented source columns and formulas.
8. TYPE THIS: Add a deterministic duplicate-prevention key and current batch metadata.
9. PRESS: Run the batch-scoped write.
10. PRESS: Run source-to-fact count and amount reconciliation.
11. PRESS: Run the same batch again.
12. DO NOT CONTINUE UNTIL: Counts/totals reconcile, no unintended duplicate appears, and sample business questions return explainable results.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/09_gold_facts.py.
2. TYPE THIS: Write the grain one row per product-store snapshot date above the fact-building section.
3. LOOK FOR: Confirm every required dimension is already verified.
4. TYPE THIS: Select one row per business event at the declared grain.
5. TYPE THIS: Resolve each dimension key by business key and event/effective date.
6. TYPE THIS: Map missing dimension references to unknown key 0 and increment a monitored exception count.
7. TYPE THIS: Derive measures only from documented source columns and formulas.
8. TYPE THIS: Add a deterministic duplicate-prevention key and current batch metadata.
9. PRESS: Run the batch-scoped write.
10. PRESS: Run source-to-fact count and amount reconciliation.
11. PRESS: Run the same batch again.
12. DO NOT CONTINUE UNTIL: Counts/totals reconcile, no unintended duplicate appears, and sample business questions return explainable results.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Teaching fact-build pattern

```
fact_source = spark.table("gold_stage.fact_inventory_snapshot")
deduped = fact_source.dropDuplicates(["_business_event_key"])
resolved = (deduped
    .withColumn("date_key", F.coalesce(F.col("date_key"), F.lit(0)))
    .withColumn("_batch_id", F.lit(batch_id)))
(resolved.write.format("delta")
    .mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable("gold.fact_inventory_snapshot"))
```

Line-by-line explanation

- **Line 1:** `fact_source = spark.table("gold_stage.fact_inventory_snapshot")` — `fact_source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"gold_stage.fact_inventory_snapshot"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `deduped = fact_source.dropDuplicates(["_business_event_key"])` — `deduped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact_source.dropDuplicates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `( [` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_business_event_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `resolved = (deduped` — `resolved` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `deduped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `.withColumn("date_key", F.coalesce(F.col("date_key"), F.lit(0)))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"date_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.coalesce` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the

surrounding explanation; "date_key" is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;))) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `(resolved.write.format("delta")` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; resolved.write.format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.mode("overwrite")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `.option("replaceWhere", f"_batch_id = '{batch_id}'")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id = '{batch_id}'" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.saveAsTable("gold.fact_inventory_snapshot")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "gold.fact_inventory_snapshot" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Reconciliation worksheet

Check	Upstream value	Gold value	Difference	Pass condition	My verified result
Accepted event rows	[ENTER]	[ENTER]	[CALCULATE]	zero unless explicitly aggregated	[ENTER]
Primary quantity	[ENTER]	[ENTER]	[CALCULATE]	zero within declared logic	[ENTER]
Primary amount	[ENTER]	[ENTER]	[CALCULATE]	within rounding tolerance	[ENTER]
Unknown dimension keys	[ENTER]	[ENTER]	n/a	monitored and explained	[ENTER]
Duplicate business-event keys	n/a	[ENTER]	n/a	zero	[ENTER]
Same-batch rerun row growth	n/a	[ENTER]	n/a	zero unintended growth	[ENTER]

Sample business questions

1. What does one row mean, and which dimensions can filter it?
2. Which measures can be summed across dates, and which cannot?

3. Which source rows reconcile to one chosen fact row?
4. What happens when a dimension record arrives late?
5. What query proves that a rerun did not duplicate the business event?

Interview-ready explanation

"I built fact_inventory_snapshot at one row per product-store snapshot date, resolved conformed dimension keys with explicit unknown handling, derived documented measures, and reconciled both counts and amounts. A deterministic event key made incremental reruns testable."

Gold fact 4 — gold.fact_shipments

Declared grain: one row per shipment.

Dimension keys: ship date, delivery date, customer, store, channel. Each lookup must name its business key and effective-date rule.

Measures: shipping_cost, transit_days, on_time_flag.

Additivity: Monetary and quantity measures are generally additive across compatible dimensions; snapshot balances are not safely additive across snapshot dates; ratios and flags must be aggregated with documented denominators.

Unknown keys: Use key 0 rather than dropping the fact, then monitor the unknown-key rate.

Duplicate prevention: Define a stable business-event key at the declared grain and enforce same-batch idempotency.

Incremental rule: Process the named batch or watermark range and do not reprocess unrelated history unless the repair strategy says so.

Action 12.4.F4 — Build and reconcile gold.fact_shipments

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and reconcile gold.fact_shipments. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open databricks/notebooks/09_gold_facts.py.
2. TYPE THIS: Write the grain one row per shipment above the fact-building section.
3. LOOK FOR: Confirm every required dimension is already verified.
4. TYPE THIS: Select one row per business event at the declared grain.
5. TYPE THIS: Resolve each dimension key by business key and event/effective date.
6. TYPE THIS: Map missing dimension references to unknown key 0 and increment a monitored exception count.
7. TYPE THIS: Derive measures only from documented source columns and formulas.
8. TYPE THIS: Add a deterministic duplicate-prevention key and current batch metadata.
9. PRESS: Run the batch-scoped write.
10. PRESS: Run source-to-fact count and amount reconciliation.
11. PRESS: Run the same batch again.
12. DO NOT CONTINUE UNTIL: Counts/totals reconcile, no unintended duplicate appears, and sample business questions return explainable results.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/09_gold_facts.py.
2. TYPE THIS: Write the grain one row per shipment above the fact-building section.
3. LOOK FOR: Confirm every required dimension is already verified.
4. TYPE THIS: Select one row per business event at the declared grain.
5. TYPE THIS: Resolve each dimension key by business key and event/effective date.
6. TYPE THIS: Map missing dimension references to unknown key 0 and increment a monitored exception count.
7. TYPE THIS: Derive measures only from documented source columns and formulas.
8. TYPE THIS: Add a deterministic duplicate-prevention key and current batch metadata.
9. PRESS: Run the batch-scoped write.
10. PRESS: Run source-to-fact count and amount reconciliation.
11. PRESS: Run the same batch again.
12. DO NOT CONTINUE UNTIL: Counts/totals reconcile, no unintended duplicate appears, and sample business questions return explainable results.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Teaching fact-build pattern

```
fact_source = spark.table("gold_stage.fact_shipments")
deduped = fact_source.dropDuplicates(["_business_event_key"])
resolved = (deduped
    .withColumn("date_key", F.coalesce(F.col("date_key"), F.lit(0)))
    .withColumn("_batch_id", F.lit(batch_id)))
(resolved.write.format("delta")
    .mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable("gold.fact_shipments"))
```

Line-by-line explanation

- **Line 1:** `fact_source = spark.table("gold_stage.fact_shipments")` — `fact_source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"gold_stage.fact_shipments"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `deduped = fact_source.dropDuplicates(["_business_event_key"])` — `deduped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact_source.dropDuplicates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_business_event_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `resolved = (deduped — resolved is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; deduped is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation`
- **Line 4:** `.withColumn("date_key", F.coalesce(F.col("date_key"), F.lit(0)))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"date_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.coalesce` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"date_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id)))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `(resolved.write.format("delta")` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `resolved.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.mode("overwrite")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"overwrite"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `.option("replaceWhere", f"_batch_id = '{batch_id}'")` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replacWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id = '{batch_id}'" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 9:** .saveAsTable("gold.fact_shipments")) — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "gold.fact_shipments" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Reconciliation worksheet

Check	Upstream value	Gold value	Difference	Pass condition	My verified result
Accepted event rows	[ENTER]	[ENTER]	[CALCULATE]	zero unless explicitly aggregated	[ENTER]
Primary quantity	[ENTER]	[ENTER]	[CALCULATE]	zero within declared logic	[ENTER]
Primary amount	[ENTER]	[ENTER]	[CALCULATE]	within rounding tolerance	[ENTER]
Unknown dimension keys	[ENTER]	[ENTER]	n/a	monitored and explained	[ENTER]
Duplicate business-event keys	n/a	[ENTER]	n/a	zero	[ENTER]
Same-batch rerun row growth	n/a	[ENTER]	n/a	zero unintended growth	[ENTER]

Sample business questions

1. What does one row mean, and which dimensions can filter it?
2. Which measures can be summed across dates, and which cannot?
3. Which source rows reconcile to one chosen fact row?
4. What happens when a dimension record arrives late?
5. What query proves that a rerun did not duplicate the business event?

Interview-ready explanation

"I built fact_shipments at one row per shipment, resolved conformed dimension keys with explicit unknown handling, derived documented measures, and reconciled both counts and amounts. A deterministic event key made incremental reruns testable."

Gold fact 5 — gold.fact_web_sessions

Declared grain: one row per conformed web session or declared aggregate session grain.

Dimension keys: session date, customer/web user, campaign, channel. Each lookup must name its business key and effective-date rule.

Measures: session_count, converted_flag, converted_order_id.

Additivity: Monetary and quantity measures are generally additive across compatible dimensions; snapshot balances are not safely additive across snapshot dates; ratios and flags must be aggregated with documented denominators.

Unknown keys: Use key 0 rather than dropping the fact, then monitor the unknown-key rate.

Duplicate prevention: Define a stable business-event key at the declared grain and enforce same-batch idempotency.

Incremental rule: Process the named batch or watermark range and do not reprocess unrelated history unless the repair strategy says so.

Action 12.4.F5 — Build and reconcile gold.fact_web_sessions

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and reconcile gold.fact_web_sessions. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open databricks/notebooks/09_gold_facts.py.
2. TYPE THIS: Write the grain one row per conformed web session or declared aggregate session grain above the fact-building section.
3. LOOK FOR: Confirm every required dimension is already verified.
4. TYPE THIS: Select one row per business event at the declared grain.
5. TYPE THIS: Resolve each dimension key by business key and event/effective date.
6. TYPE THIS: Map missing dimension references to unknown key 0 and increment a monitored exception count.
7. TYPE THIS: Derive measures only from documented source columns and formulas.
8. TYPE THIS: Add a deterministic duplicate-prevention key and current batch metadata.
9. PRESS: Run the batch-scoped write.
10. PRESS: Run source-to-fact count and amount reconciliation.
11. PRESS: Run the same batch again.
12. DO NOT CONTINUE UNTIL: Counts/totals reconcile, no unintended duplicate appears, and sample business questions return explainable results.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/09_gold_facts.py.
2. TYPE THIS: Write the grain one row per conformed web session or declared aggregate session grain above the fact-building section.
3. LOOK FOR: Confirm every required dimension is already verified.
4. TYPE THIS: Select one row per business event at the declared grain.
5. TYPE THIS: Resolve each dimension key by business key and event/effective date.
6. TYPE THIS: Map missing dimension references to unknown key 0 and increment a monitored exception count.
7. TYPE THIS: Derive measures only from documented source columns and formulas.
8. TYPE THIS: Add a deterministic duplicate-prevention key and current batch metadata.
9. PRESS: Run the batch-scoped write.
10. PRESS: Run source-to-fact count and amount reconciliation.
11. PRESS: Run the same batch again.
12. DO NOT CONTINUE UNTIL: Counts/totals reconcile, no unintended duplicate appears, and sample business questions return explainable results.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Teaching fact-build pattern

```
fact_source = spark.table("gold_stage.fact_web_sessions")
deduped = fact_source.dropDuplicates(["_business_event_key"])
resolved = (deduped
    .withColumn("date_key", F.coalesce(F.col("date_key"), F.lit(0)))
    .withColumn("_batch_id", F.lit(batch_id)))
(resolved.write.format("delta")
    .mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable("gold.fact_web_sessions"))
```

Line-by-line explanation

- Line 1:** `fact_source = spark.table("gold_stage.fact_web_sessions")` — `fact_source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"gold_stage.fact_web_sessions"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `deduped = fact_source.dropDuplicates(["_business_event_key"])` — `deduped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact_source.dropDuplicates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_business_event_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `resolved = (deduped` — `resolved` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `deduped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `.withColumn("date_key", F.coalesce(F.col("date_key"), F.lit(0)))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "date_key" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.coalesce is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "date_key" is a quoted literal; the quotation marks preserve spaces or mark text;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 5:** `.withColumn("_batch_id", F.lit(batch_id))` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; withColumn is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.lit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;))) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `(resolved.write.format("delta")` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; resolved.write.format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "delta" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `.mode("overwrite")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `.option("replaceWhere", f"_batch_id = '{batch_id}'")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id = '{batch_id}'" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.saveAsTable("gold.fact_web_sessions")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "gold.fact_web_sessions" is a quoted literal; the quotation marks preserve spaces or mark text;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Reconciliation worksheet

Check	Upstream value	Gold value	Difference	Pass condition	My verified result
Accepted event rows	[ENTER]	[ENTER]	[CALCULATE]	zero unless explicitly aggregated	[ENTER]
Primary quantity	[ENTER]	[ENTER]	[CALCULATE]	zero within declared logic	[ENTER]
Primary amount	[ENTER]	[ENTER]	[CALCULATE]	within rounding tolerance	[ENTER]
Unknown dimension keys	[ENTER]	[ENTER]	n/a	monitored and explained	[ENTER]

Check	Upstream value	Gold value	Difference	Pass condition	My verified result
Duplicate business-event keys	n/a	[ENTER]	n/a	zero	[ENTER]
Same-batch rerun row growth	n/a	[ENTER]	n/a	zero unintended growth	[ENTER]

Sample business questions

1. What does one row mean, and which dimensions can filter it?
2. Which measures can be summed across dates, and which cannot?
3. Which source rows reconcile to one chosen fact row?
4. What happens when a dimension record arrives late?
5. What query proves that a rerun did not duplicate the business event?

Interview-ready explanation

"I built `fact_web_sessions` at one row per conformed web session or declared aggregate session grain, resolved conformed dimension keys with explicit unknown handling, derived documented measures, and reconciled both counts and amounts. A deterministic event key made incremental reruns testable."

Gold fact 6 — `gold.fact_marketing_spend`

Declared grain: one row per date-campaign-channel spend record.

Dimension keys: spend date, campaign, channel. Each lookup must name its business key and effective-date rule.

Measures: `spend_amount`, `impressions`, `clicks`.

Additivity: Monetary and quantity measures are generally additive across compatible dimensions; snapshot balances are not safely additive across snapshot dates; ratios and flags must be aggregated with documented denominators.

Unknown keys: Use key 0 rather than dropping the fact, then monitor the unknown-key rate.

Duplicate prevention: Define a stable business-event key at the declared grain and enforce same-batch idempotency.

Incremental rule: Process the named batch or watermark range and do not reprocess unrelated history unless the repair strategy says so.

Action 12.4.F6 — Build and reconcile `gold.fact_marketing_spend`

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Build and reconcile `gold.fact_marketing_spend`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open `databricks/notebooks/09_gold_facts.py`.
2. TYPE THIS: Write the grain one row per date-campaign-channel spend record above the fact-building section.
3. LOOK FOR: Confirm every required dimension is already verified.
4. TYPE THIS: Select one row per business event at the declared grain.
5. TYPE THIS: Resolve each dimension key by business key and event/effective date.
6. TYPE THIS: Map missing dimension references to unknown key 0 and increment a monitored exception count.
7. TYPE THIS: Derive measures only from documented source columns and formulas.
8. TYPE THIS: Add a deterministic duplicate-prevention key and current batch metadata.
9. PRESS: Run the batch-scoped write.

10. PRESS: Run source-to-fact count and amount reconciliation.
11. PRESS: Run the same batch again.
12. DO NOT CONTINUE UNTIL: Counts/totals reconcile, no unintended duplicate appears, and sample business questions return explainable results.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open databricks/notebooks/09_gold_facts.py.
2. TYPE THIS: Write the grain one row per date-campaign-channel spend record above the fact-building section.
3. LOOK FOR: Confirm every required dimension is already verified.
4. TYPE THIS: Select one row per business event at the declared grain.
5. TYPE THIS: Resolve each dimension key by business key and event/effective date.
6. TYPE THIS: Map missing dimension references to unknown key 0 and increment a monitored exception count.
7. TYPE THIS: Derive measures only from documented source columns and formulas.
8. TYPE THIS: Add a deterministic duplicate-prevention key and current batch metadata.
9. PRESS: Run the batch-scoped write.
10. PRESS: Run source-to-fact count and amount reconciliation.
11. PRESS: Run the same batch again.
12. DO NOT CONTINUE UNTIL: Counts/totals reconcile, no unintended duplicate appears, and sample business questions return explainable results.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 12 lesson index](#)

Teaching fact-build pattern

```
fact_source = spark.table("gold_stage.fact_marketing_spend")
deduped = fact_source.dropDuplicates(["_business_event_key"])
resolved = (deduped
    .withColumn("date_key", F.coalesce(F.col("date_key"), F.lit(0)))
    .withColumn("_batch_id", F.lit(batch_id)))
(resolved.write.format("delta")
    .mode("overwrite")
    .option("replaceWhere", f"_batch_id = '{batch_id}'")
    .saveAsTable("gold.fact_marketing_spend"))
```

Line-by-line explanation

- Line 1:** `fact_source = spark.table("gold_stage.fact_marketing_spend")` — `fact_source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"gold_stage.fact_marketing_spend"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `deduped = fact_source.dropDuplicates(["_business_event_key"])` — `deduped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact_source.dropDuplicates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_business_event_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `resolved = (deduped` — `resolved` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `deduped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `.withColumn("date_key", F.coalesce(F.col("date_key"), F.lit(0)))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"date_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.coalesce` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"date_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 5:** `.withColumn("_batch_id", F.lit(batch_id)))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `withColumn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.lit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 6:** `(resolved.write.format("delta"))` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `resolved.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted

literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** `.mode("overwrite")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "overwrite" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `.option("replaceWhere", f"_batch_id = '{batch_id}')` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; option is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "replaceWhere" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "_batch_id = '{batch_id}'" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `.saveAsTable("gold.fact_marketing_spend")` — . is a path, relative location, branch-style name, or file argument; verify every character before running it; saveAsTable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "gold.fact_marketing_spend" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Reconciliation worksheet

Check	Upstream value	Gold value	Difference	Pass condition	My verified result
Accepted event rows	[ENTER]	[ENTER]	[CALCULATE]	zero unless explicitly aggregated	[ENTER]
Primary quantity	[ENTER]	[ENTER]	[CALCULATE]	zero within declared logic	[ENTER]
Primary amount	[ENTER]	[ENTER]	[CALCULATE]	within rounding tolerance	[ENTER]
Unknown dimension keys	[ENTER]	[ENTER]	n/a	monitored and explained	[ENTER]
Duplicate business-event keys	n/a	[ENTER]	n/a	zero	[ENTER]
Same-batch rerun row growth	n/a	[ENTER]	n/a	zero unintended growth	[ENTER]

Sample business questions

1. What does one row mean, and which dimensions can filter it?
2. Which measures can be summed across dates, and which cannot?
3. Which source rows reconcile to one chosen fact row?
4. What happens when a dimension record arrives late?
5. What query proves that a rerun did not duplicate the business event?

Interview-ready explanation

"I built fact_marketing_spend at one row per date-campaign-channel spend record, resolved conformed dimension keys with explicit unknown handling, derived documented measures, and reconciled both counts and amounts. A deterministic event key made incremental reruns testable."

What your screen should look like now

- The new Phase 12 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 12.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 12.5](#).

Lesson 12.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 12 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Verification before automation

Question:

Should the phase prioritize a visible manual verification or jump directly to automation?

Options considered:

- Manual verification first, then automation.
- Automation without a known-good baseline.
- Manual operation forever.

Beginner path selected:

Establish one verified manual baseline and then automate repeatable checks.

Why this path was selected:

A known-good example makes failures interpretable and prevents automation from hiding a wrong assumption.

Tradeoff:

The first run takes longer.

What a company might do instead:

Teams often pair infrastructure as code and automated tests with controlled bootstrap and acceptance procedures.

How to explain this in an interview:

I established a verified baseline, then encoded the same contract in tests and orchestration so automation preserved meaning rather than just speed.

If you see Result A

Each fact passes its declared grain, dimension lookup, unknown-key, and amount/count reconciliation. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

Duplicate grain, unresolved keys beyond policy, or mismatched measures appear. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: Databricks notebooks, Catalog Explorer, and SQL/Python cells.

Window/panel: Show fact grain sample, dimension keys, measures, and count/amount reconciliation.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase12_build-gold-fact-tables-and-reconciled-business-measures_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 12's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 12 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 12.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Action 12.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Action 12.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.

7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Interpret `git status --short`

- `??` means untracked: Git sees a new path but it is not staged.
- `M` means modified in the working tree but not staged.
- `M` means the staged version differs from the last commit.

- MM means a staged version exists and the file has additional unstaged edits.
- A means staged as a new file.
- D means deletion; stop and investigate unless deletion is deliberate.
- R means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 12.5.G4 — Inspect the unstaged diff for databricks/notebooks/09_gold_facts.py

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "databricks/notebooks/09_gold_facts.py"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "databricks/notebooks/09_gold_facts.py"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"databricks/notebooks/09_gold_facts.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Action 12.5.G5 — Stage only the reviewed file databricks/notebooks/09_gold_facts.py

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.

2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "databricks/notebooks/09_gold_facts.py"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "databricks/notebooks/09_gold_facts.py"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"databricks/notebooks/09_gold_facts.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 12.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Action 12.5.G7 — Create the Phase 12 commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "feat: build reconciled gold facts"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "feat: build reconciled gold facts"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"feat: build reconciled gold facts"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Action 12.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Action 12.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.

3. **CLICK:** Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. **TYPE THIS:** Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. **PRESS:** Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. **LOOK FOR:** Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. **DO NOT CONTINUE UNTIL:** Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 12 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message feat: build reconciled gold facts and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 12.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 12 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

"Phase 12 created the verified boundary described by **Build Gold fact tables and reconciled business measures**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed."

Mini quiz

1. Multiple choice: Why does Phase 12 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
4. What command would you use first to confirm the PowerShell current folder before a repository command?
5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?

6. Design choice: Explain one tradeoff of Track A and one production alternative.
7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
8. Ownership: Which status may you select only after personally performing and inspecting the verification?

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** PLANNED, CREATED BY USER, and RUN BY USER describe earlier states; only inspected evidence supports verification.

My Phase 12 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.

- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 12.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 13 converts correctness expectations into executable blocking tests.

Phase 12 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 13 — Build data-quality, reconciliation, and failure gates

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: Runnable transformations and quality gates.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 14 connects them in dependency order and proves repair behavior.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 13 lesson index

1. [Lesson 13.1 — Understand the phase and starting screen](#)
2. [Lesson 13.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 13.3 — Create and understand the phase files](#)
4. [Lesson 13.4 — Build and validate required objects](#)
5. [Lesson 13.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 13.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** VS Code tests, Databricks notebooks, Catalog Explorer, and Job task output.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 13 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 12 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 13 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

VS Code tests, Databricks notebooks, Catalog Explorer, and Job task output. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 13 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 13 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 13 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 13.1 completion checklist

☐

I read why Phase 13 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 13.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 13 lesson index](#)

Lesson 13.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 13 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

A risk-based test suite across local Python, source SQL, Databricks Bronze/Silver/Gold, and SQL Server publication, with stored `quality.test_results`, clear required versus warning checks, schema/freshness/quarantine monitoring, and a demonstrated failed gate that blocks publication.

2. Why it matters

Data quality is not a single 'not null' query. Trust depends on boundary reconciliations, grain, arithmetic, SCD state, freshness, schema stability, quarantine visibility, and failure behavior. A gate has value only when downstream publication truly depends on it.

3. Words to know

Term	Plain-language definition
Assertion	Condition that fails execution when false.
Required check	Failure blocks publication.
Warning check	Visible concern that does not block under current policy.
Threshold	Allowed limit such as unknown-key share $\leq$ 5%.
Freshness	How recent the latest accepted event/snapshot is.
Schema drift	New/missing/changed source column/type.
Reconciliation	Independent comparison across layers.
Test result table	Stored batch/check/actual/expected/status evidence.
False positive/negative	Test fails good data or passes bad data.
Quality gate	Dependency task that stops downstream publication on required failure.
Observability	Logs/metrics/status making operation understandable.
Repair run	Rerun failed and dependent tasks after correction.
Tolerance	Documented acceptable numeric difference, commonly rounding-related.

4. What must already be completed

All required Gold dimensions/facts exist and local tests can run. You have documented expected defects, exclusions, and KPI formulas.

5. Applications to open

Application	How to open it	Why it is open
Databricks notebook 10	Workspace.	Run/store blocking quality checks.
Databricks SQL editor	Quality/Gold/Silver/Bronze.	Inspect results and reconciliation.
VS Code/PowerShell	Repository root.	Run pytest and structural checks.

Application	How to open it	Why it is open
Testing strategy	Open docs/testing_strategy.md.	Map risk to tests and gates.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run Get-Location; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
python -m pytest -q	Runs unit/integration/sample quality/reconciliation tests that require no paid infrastructure.	Local code/fixtures must pass before remote publication logic.
python -m scripts.validation.validate_sql_dialects; python -m scripts.validation.validate_notebooks; python -m scripts.validation.scan_secrets	Checks SQL headers, notebook syntax/JSON, and common secret patterns.	Structural/security defects can break or expose the project before data runs.
SELECT check_name, actual_value, expected_value, operator, status, required_flag, checked_at_utc	Displays stored batch quality results.	A notebook success banner is not durable evidence.
SELECT status, required_flag, COUNT(*) AS checks	Summarizes check outcomes.	Supports a monitoring KPI without hiding failures.
SELECT MAX(d.full_date) AS latest_sales_date, current_date() AS run_date, datediff(current_date(), MAX(d.full_date)) AS age_days	Calculates sales freshness age.	Row validity does not prove data is current.

Layer reconciliation is not always equality

- Source = Bronze for readable raw rows.
- Bronze = Silver trusted + quarantine + deterministic duplicate losers (with categories documented).
- Silver = Gold for event rows after dimension enrichment; dimensions can have history/unknown rows so their counts differ by design.
- Gold = serving warehouse after publication.
- Gold KPI = Power BI measure under identical filters.

Quality gate failure test

A gate is unproven until a required failure blocks publication. Use a controlled disposable threshold or copied test table, capture the failure/dependency skip, then restore and repair. Never leave the repository configured to fail or lower the test just to get green.

MY TASK

Expand the quality notebook/test results to cover all required verification gates, demonstrate a blocked publication and successful repair, and document owner/action for every rule.

STARTER SKELETON

```
# TODO(USER): reusable reconciliation record
def reconcile(name: str, source_value: float, target_value: float, tolerance: float = 0.0):
```

```

difference = target_value - source_value
status = "PASS" if abs(difference) <= tolerance else "FAIL"
return (batch_id, name, source_value, target_value, difference, tolerance, status, True)

# TODO(USER): add rows for counts, quantity, revenue, profit, and quarantine categories.
# TODO(USER): write to quality.reconciliation_results and fail on required FAIL.

```

Acceptance criteria: The valid batch has stored passing required tests; a controlled failure blocks publication; repair succeeds without duplication; every required project gate has a test/evidence owner.

HINT 1 — CONCEPT

Start from risk register: write the test that catches each high-impact risk.

HINT 2 — PSEUDOCODE

Represent checks as data rows, then filter required failures and raise once after storing results.

HINT 3 — NEAR-COMPLETE PATTERN

Use in-workbook record pattern and extend table checks/reconciliations systematically.

13. Expected files or output

- Local validation pass evidence.
- Comprehensive quality.test_results.
- Reconciliation table/report.
- Failed-gate run with publication skipped.
- Repaired successful run.
- Rule ownership/action matrix.

14. How to verify

- No required fail in publishable batch.
- Failure run actually prevents dependent task.
- Quality results include batch/timestamp/actual/expected/status.
- Quarantine is visible and reconciled.
- Freshness is tested against expected source date semantics.
- Schema drift test has approved schema contract.

15. Automated tests

- All matrix tests from docs/testing_strategy.md.
- Controlled failure and repair.
- Duplicate rerun.
- Source-to-Gold revenue.
- SCD2 behavior.
- Publication dependency.
- Semantic validation later.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
All tests pass suspiciously	Tests may not cover data/risk or filter wrong batch.	Print batch, counts, intentionally fail one rule.	Add negative test and verify gate path.	Controlled failure blocks.
Freshness always fails	Synthetic max date differs from today's real date.	Inspect expected source data date/config.	Test against batch/source expected date or label historical fixture; do not fabricate freshness.	Rule meaning correct.
Reconciliation count mismatch	Dedup/quarantine/history rules not included.	Break difference into categorized outcomes.	Use rule-specific equation rather than forcing equality.	Difference explained.

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
Test results duplicate on re-run	Append same batch/check without replacement/merge.	Group by batch/check.	Delete/merge same batch check rows before insert or add attempt ID/current view.	Monitoring shows intended latest attempt.
Publication ran despite FAIL	Dependency not configured or task result ignored.	Inspect Job graph/run dependencies.	Make publish depend on quality task success; raise exception on required fail.	Publish skipped.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why is a green test suite not automatically trustworthy?
2. How does reconciliation differ across layer boundaries?
3. Why can freshness be tricky with a historical synthetic fixture?
4. What proves a quality gate works?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. Which type of failed check must block publication?
2. Why store actual and expected values, not only PASS?
3. What should happen to a same-batch quality rerun?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture valid result table, controlled failed Job task with publish skipped, and repaired successful run. Show exact batch IDs and redact account details.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
test: add blocking data quality and reconciliation gates
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

Local tests pass — [STATUS]

☐

All dimension/fact rules stored — [STATUS]

☐

Layer reconciliations complete — [STATUS]

☐

Freshness/schema/quarantine monitored — [STATUS]

☐

Controlled gate failure demonstrated — [STATUS]

☐

Publication blocked — [STATUS]

☐

Repair pass and rerun stability verified — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-13. The quiz answer is in the answer-key section immediately after the quiz in this workbook#phase-13.

Next phase: Phase 14 creates the full Lakeflow Job in the UI and version control with parameters, dependencies, retries, and run operations.

Required action cards

Action 13.2.1 — Open the test matrix and identify which damaging failure each test catches. Remove/replace tests that have no

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open the test matrix and identify which damaging failure each test catches. Remove/replace tests that have no . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open the test matrix and identify which damaging failure each test catches. Remove/replace tests that have no decision consequence.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 1

Action

Open the test matrix and identify which damaging failure each test catches. Remove/replace tests that have no decision consequence.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Action 13.2.2 — Run local Python unit, data-quality, reconciliation, notebook, SQL dialect, secret, and link validations.

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run local Python unit, data-quality, reconciliation, notebook, SQL dialect, secret, and link validations. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run local Python unit, data-quality, reconciliation, notebook, SQL dialect, secret, and link validations.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 2****Action**

Run local Python unit, data-quality, reconciliation, notebook, SQL dialect, secret, and link validations.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Action 13.2.3 — Open 10_data_quality. Read record function, required flag, result write, task value, and exception.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open 10_data_quality. Read record function, required flag, result write, task value, and exception. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open 10_data_quality. Read record function, required flag, result write, task value, and exception.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 3

Action

Open 10_data_quality. Read record function, required flag, result write, task value, and exception.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Action 13.2.4 — Add missing checks for every required fact/dimension: not-null, unique grain, accepted codes, RI, date/numeric

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Add missing checks for every required fact/dimension: not-null, unique grain, accepted codes, RI, date/numeric. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Add missing checks for every required fact/dimension: not-null, unique grain, accepted codes, RI, date/numeric ranges, SCD current/overlap, source-layer counts, revenue, freshness, schema, quarantine, unknown rates.
4. Pause while the application finishes the operation; do not click the same control twice.

5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 4****Action**

Add missing checks for every required fact/dimension: not-null, unique grain, accepted codes, RI, date/numeric ranges, SCD current/overlap, source-layer counts, revenue, freshness, schema, quarantine, unknown rates.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Action 13.2.5 — Set thresholds from business requirements, not convenience. A 5% unknown threshold is a initial assumption to

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Set thresholds from business requirements, not convenience. A 5% unknown threshold is a initial assumption to . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Set thresholds from business requirements, not convenience. A 5% unknown threshold is a initial assumption to review.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 5

Action

Set thresholds from business requirements, not convenience. A 5% unknown threshold is a initial assumption to review.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Action 13.2.6 — Run the gate on a valid batch and confirm PASS rows are stored.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run the gate on a valid batch and confirm PASS rows are stored. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run the gate on a valid batch and confirm PASS rows are stored.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 6

Action

Run the gate on a valid batch and confirm PASS rows are stored.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Action 13.2.7 — Create a disposable controlled failure: temporarily set a required threshold below the observed valid value or

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Create a disposable controlled failure: temporarily set a required threshold below the observed valid value or. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Create a disposable controlled failure: temporarily set a required threshold below the observed valid value or add a copied duplicate test row. Run the gate and verify task fails. Do not corrupt the canonical Gold table without a rollback plan.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 7****Action**

Create a disposable controlled failure: temporarily set a required threshold below the observed valid value or add a copied duplicate test row. Run the gate and verify task fails. Do not corrupt the canonical Gold table without a rollback plan.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Action 13.2.8 — Verify publication does not run in the Job dependency graph when gate fails. Capture run evidence.

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Verify publication does not run in the Job dependency graph when gate fails. Capture run evidence. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Verify publication does not run in the Job dependency graph when gate fails. Capture run evidence.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 8

Action

Verify publication does not run in the Job dependency graph when gate fails. Capture run evidence.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Action 13.2.9 — Restore condition/data, repair run from quality task, and prove pass. Confirm no duplicate outputs.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Restore condition/data, repair run from quality task, and prove pass. Confirm no duplicate outputs. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Restore condition/data, repair run from quality task, and prove pass. Confirm no duplicate outputs.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 9****Action**

Restore condition/data, repair run from quality task, and prove pass. Confirm no duplicate outputs.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Action 13.2.10 — Create a reconciliation result that compares source/Bronze/Silver/Gold row counts and revenue with rule-specif

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Create a reconciliation result that compares source/Bronze/Silver/Gold row counts and revenue with rule-specif. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Create a reconciliation result that compares source/Bronze/Silver/Gold row counts and revenue with rule-specific explanations rather than expecting every raw count to equal trusted count.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 10

Action

Create a reconciliation result that compares source/Bronze/Silver/Gold row counts and revenue with rule-specific explanations rather than expecting every raw count to equal trusted count.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Action 13.2.11 — Define schema-drift expected columns/types and freshness targets by domain.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Define schema-drift expected columns/types and freshness targets by domain. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Define schema-drift expected columns/types and freshness targets by domain.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 11****Action**

Define schema-drift expected columns/types and freshness targets by domain.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Action 13.2.12 — Document who owns each failed rule and whether quarantine can be released/reprocessed.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Document who owns each failed rule and whether quarantine can be released/reprocessed. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Document who owns each failed rule and whether quarantine can be released/reprocessed.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 12****Action**

Document who owns each failed rule and whether quarantine can be released/reprocessed.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Action 13.2.C1 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m pytest -q
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m pytest -q` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `pytest` the Python test runner; `-q` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Action 13.2.C2 — Windows PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m scripts.validation.validate_sql_dialects; python -m scripts.validation.validate_notebooks; python -m scripts.validation.scan_secrets
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** python -m scripts.validation.validate_sql_dialects; python -m scripts.validation.validate_notebooks; python -m scripts.validation.scan_secrets — python invokes Python when that command name is registered; -m asks Python to run an installed module; scripts.validation.validate_sql_dialects is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; python invokes Python when that command name is registered; -m asks Python to run an installed module; scripts.validation.validate_notebooks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; python invokes Python when that command name is registered; -m asks Python to run an installed module; scripts.validation.scan_secrets is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Action 13.2.C3 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT check_name, actual_value, expected_value, operator, status, required_flag, checked_at_utc
FROM northstar_retail.quality.test_results
WHERE batch_id='EXT-YYYYMMDDTHMMSSZ'
ORDER BY status DESC, check_name;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT check_name, actual_value, expected_value, operator, status, required_flag, checked_at_utc` — `SELECT` requests data without changing it unless combined with a modifying statement; `check_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual_value` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `expected_value` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `operator` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `status` shows tracked, staged, modified, and untracked work; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_flag` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 2:** `FROM northstar_retail.quality.test_results` — `FROM` identifies the source table or query; `northstar_retail.quality.test_results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `WHERE batch_id='EXT-YYYYMMDDTHHMMSSZ'` — `WHERE` filters rows to those meeting a condition; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'EXT-YYYYMMDDTHHMMSSZ'` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 4:** `ORDER BY status DESC, check_name;` — `ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `status` shows tracked, staged, modified, and untracked work; `DESC` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `check_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Action 13.2.C4 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT status, required_flag, COUNT(*) AS checks
FROM northstar_retail.quality.test_results
WHERE batch_id='EXT-YYYYMMDDTHMMSSZ'
GROUP BY status, required_flag;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT status, required_flag, COUNT(*) AS checks` — `SELECT` requests data without changing it unless combined with a modifying statement; `status` shows tracked, staged, modified, and untracked work; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_flag` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `checks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `FROM northstar_retail.quality.test_results` — `FROM` identifies the source table or query; `northstar_retail.quality.test_results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `WHERE batch_id='EXT-YYYYMMDDTHHMMSSZ'` — `WHERE` filters rows to those meeting a condition; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'EXT-YYYYMMDDTHHMMSSZ'` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 4:** `GROUP BY status, required_flag;` — `GROUP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `status` shows tracked, staged, modified, and untracked work; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `required_flag` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Action 13.2.C5 — Databricks SQL editor

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT MAX(d.full_date) AS latest_sales_date, current_date() AS run_date, datediff(current_date(), MAX(d.full_date)) AS age_days
FROM northstar_retail.gold.fact_sales f
JOIN northstar_retail.gold.dim_date d ON d.date_key=f.order_date_key;
```

Where to enter it:

Databricks SQL editor. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** SELECT MAX(d.full_date) AS latest_sales_date, current_date() AS run_date, datediff(current_date(), MAX(d.full_date)) AS age_days — SELECT requests data without changing it unless combined with a modifying statement; MAX is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; d.full_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-

specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; latest_sales_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; current_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; run_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 2:** FROM northstar_retail.gold.fact_sales f — FROM identifies the source table or query; northstar_retail.gold.fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** JOIN northstar_retail.gold.dim_date d ON d.date_key=f.order_date_key; — JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; northstar_retail.gold.dim_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; d is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; d.date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.order_date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 13.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 13 starting-screen checklist and re-verify the active context.

Lesson 13.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 13 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — databricks/

Exact folder name: databricks

Exact parent: repository root

Why it is needed now: Phase 13 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: databricks.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 13.D1.1 — Create the folder databricks

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\databricks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\databricks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\databricks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\databricks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\databricks" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/

Exact folder name: `sql`

Exact parent: `repository root`

Why it is needed now: Phase 13 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `sql`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 13.D2.1 — Create the folder `sql`

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql"` is a path, relative location,

branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql" -PathType Container` and require True.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql" -PathType Container`.
2. Require the output True before creating a child file.

3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — tests/

Exact folder name: tests

Exact parent: repository root

Why it is needed now: Phase 13 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: tests.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 13.D3.1 — Create the folder tests

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\tests" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\tests" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\tests"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\tests" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\tests" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\tests" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — databricks/notebooks/

Exact folder name: notebooks

Exact parent: databricks

Why it is needed now: Phase 13 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `databricks` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `notebooks`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 13.D4.1 — Create the folder `databricks/notebooks`

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\databricks\notebooks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\databricks\notebooks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\notebooks"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\databricks\notebooks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\databricks\notebooks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\databricks\notebooks" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/databricks/

Exact folder name: databricks

Exact parent: sql

Why it is needed now: Phase 13 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `sql` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `databricks`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method**Action 13.D5.1 — Create the folder sql/databricks**

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\databricks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\databricks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\sql\databricks` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\databricks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\databricks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\databricks" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — tests/data_quality/

Exact folder name: `data_quality`

Exact parent: `tests`

Why it is needed now: Phase 13 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `tests` one level at a time.

4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: data_quality.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 13.D6.1 — Create the folder tests/data_quality

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\tests\data_quality" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\tests\data_quality" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\tests\data_quality" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\tests\data_quality" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\tests\data_quality" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\tests\data_quality" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — tests/reconciliation/

Exact folder name: reconciliation

Exact parent: tests

Why it is needed now: Phase 13 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path tests one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: reconciliation.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 13.D7.1 — Create the folder tests/reconciliation

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\tests\reconciliation" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\tests\reconciliation" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\tests\reconciliation` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\tests\reconciliation" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\tests\reconciliation" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run Test-Path ".\tests\reconciliation" -PathType Container.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/databricks/quality/

Exact folder name: quality

Exact parent: sql/databricks

Why it is needed now: Phase 13 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path sql/databricks one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: quality.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 13.D8.1 — Create the folder sql/databricks/quality

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\databricks\quality" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\databricks\quality" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\sql\databricks\quality` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\sql\databricks\quality" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\databricks\quality" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\databricks\quality" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — databricks/notebooks/10_data_quality.py

Exact filename: `10_data_quality.py`

Filename extension: `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `databricks/notebooks` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 13.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `databricks/notebooks` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `10_data_quality.py`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `10_data_quality.py` and an empty file under the correct parent folder.
10. PRESS: `Ctrl+S` after adding each explained section.

PowerShell path

Action 13.F1.1 — Create the file `databricks/notebooks/10_data_quality.py` from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\databricks\notebooks\10_data_quality.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\databricks\notebooks\10_data_quality.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\notebooks\10_data_quality.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\databricks\notebooks\10_data_quality.py" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "databricks/notebooks/10_data_quality.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `databricks/notebooks`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `databricks/notebooks/10_data_quality.py`.
3. In PowerShell at the repository root, run `Test-Path ".\databricks\notebooks\10_data_quality.py"` and require `True`.

4. Run `git status --short -- "databricks/notebooks/10_data_quality.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for databricks/notebooks/10_data_quality.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: databricks/notebooks/10_data_quality.py

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 13.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Databricks notebook source
# Northstar Retail End-to-End Data Platform
# This file is a Git-friendly Databricks source notebook.
# Track A reads files that you manually uploaded to a Unity Catalog volume.
# It never attempts to reach SQL Server or PostgreSQL on localhost.

# COMMAND -----
# MAGIC %md
# MAGIC # 10 – Data-quality gate
# MAGIC A failed required check raises an exception so publication cannot run.

# COMMAND -----
from pyspark.sql import functions as F
for name, default in {"catalog": "northstar_retail", "gold_schema": "gold", "quality_schema": "quality", "batch_id": "MANUAL-REPLACE-ME"}.items():
    dbutils.widgets.text(name, default)
catalog=dbutils.widgets.get("catalog"); gold_schema=dbutils.widgets.get("gold_schema"); quality_schema=dbutils.widgets.get("quality_schema");
batch_id=dbutils.widgets.get("batch_id")
if batch_id=="MANUAL-REPLACE-ME": raise ValueError("Pass batch_id from workflow.")

checks=[]
def record(name: str, actual: float, expected: float, operator: str, required: bool=True):
    passed={"==":actual==expected,">":actual>expected,"<=":actual<=expected}[operator]
    checks.append((batch_id,name,float(actual),float(expected),operator,"PASS" if passed else "FAIL",required))

sales=spark.table(f"{catalog}.{gold_schema}.fact_sales").filter(F.col("batch_id")==batch_id)
record("fact_sales_not_empty",sales.count(),1,">=")
record("fact_sales_unique_order_item",sales.count()-sales.select("order_item_id").distinct().count(),0,"==")
record("fact_sales_positive_quantity",sales.filter(F.col("quantity")<=0).count(),0,"==")
record("fact_sales_revenue_identity",sales.filter(F.abs(F.col("net_sales_amount")-(F.col("gross_sales_amount")-F.col("dis-
count_amount")))>F.lit(0.01)).count(),0,"==")
record("fact_sales_unknown_product_share",sales.filter(F.col("product_key")==0).count()/max(1,sales.count()),0.05,"<=")
record("fact_sales_unknown_customer_share",sales.filter(F.col("customer_key")==0).count()/max(1,sales.count()),0.05,"<=")

results=spark.createDataFrame(checks,
["batch_id","check_name","actual_value","expected_value","operator","status","required_flag"]).withColumn("checked_at_utc",F.current_timestamp
())
results.write.format("delta").mode("append").saveAsTable(f"{catalog}.{quality_schema}.test_results")
display(results)
failed=results.filter((F.col("required_flag")) & (F.col("status")=="FAIL")).count()
dbutils.jobs.taskValues.set(key="quality_status",value="PASS" if failed==0 else "FAIL")
if failed: raise AssertionError(f"QUALITY GATE FAILED: {failed} required checks failed. Read the test_results table before repairing the
run.")
print("QUALITY GATE PASSED")
```

Line-by-line explanation

- **Line 1:** `# Databricks notebook source` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `# Northstar Retail End-to-End Data Platform` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** `# This file is a Git-friendly Databricks source notebook.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** `# Track A reads files that you manually uploaded to a Unity Catalog volume.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** `# It never attempts to reach SQL Server or PostgreSQL on localhost.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `# COMMAND -----` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 9:** `# MAGIC %md` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 10:** # MAGIC # 10 — Data-quality gate is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 11:** # MAGIC A failed required check raises an exception so publication cannot run. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** # COMMAND ----- is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 14:** from pyspark.sql import functions as F — from is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pyspark.sql is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; import is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; functions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; as is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** for name, default in {"catalog": "northstar_retail", "gold_schema": "gold", "quality_schema": "quality", "batch_id": "MANUAL-REPLACE-ME"}.items(): dbutils.widgets.text(name, default) — for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; default is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "catalog" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "northstar_retail" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "gold_schema" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 16:** catalog=dbutils.widgets.get("catalog"); gold_schema=dbutils.widgets.get("gold_schema"); quality_schema=dbutils.widgets.get("quality_schema"); batch_id=dbutils.widgets.get("batch_id") — catalog is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.widgets.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "catalog" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gold_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dbutils.widgets.get is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "gold_schema" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 17:** if batch_id=="MANUAL-REPLACE-ME": raise ValueError("Pass batch_id from workflow.") — if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "MANUAL-REPLACE-ME" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; raise is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ValueError is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Pass batch_id from workflow." is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** `checks=[]` — `checks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** `def record(name: str, actual: float, expected: float, operator: str, required: bool=True):` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `record` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `float` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `expected` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 21:** `passed={ "==" : actual==expected, ">=" : actual>=expected, "<=" : actual<=expected } [operator]` — `passed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `= {` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"=="` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `expected` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>="` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 22:** `checks.append((batch_id,name,float(actual),float(expected),operator,"PASS" if passed else "FAIL",required))` — `checks.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `((` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `float` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `float` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 23:** blank line used only to separate logical sections for readability.
- **Line 24:** `sales=spark.table(f"{catalog}.{gold_schema}.fact_sales").filter(F.col("batch_id")==batch_id)` — `sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.{gold_schema}.fact_sales"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 25:** `record("fact_sales_not_empty", sales.count(), 1, ">=")` — record is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "fact_sales_not_empty" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sales.count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ">=" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** `record("fact_sales_unique_order_item", sales.count() - sales.select("order_item_id").distinct().count(), 0, "=")` — record is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "fact_sales_unique_order_item" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sales.count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () - is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sales.select is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "order_item_id" is a quoted literal; the quotation marks preserve spaces or mark text;). is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; distinct is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (). is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 27:** `record("fact_sales_positive_quantity", sales.filter(F.col("quantity") <= 0).count(), 0, "=")` — record is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "fact_sales_positive_quantity" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sales.filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "quantity" is a quoted literal; the quotation marks preserve spaces or mark text;) <= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression;). is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 28:** `record("fact_sales_revenue_identity", sales.filter(F.abs(F.col("net_sales_amount") - (F.col("gross_sales_amount") - F.col("discount_amount")) > F.lit(0.01)).count(), 0, "=")` — record is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "fact_sales_revenue_identity" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sales.filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.abs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; F.col is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "net_sales_amount" is a quoted literal; the quotation marks preserve spaces or mark text;) - (is a name, keyword, argument, identifier, or literal whose project-

specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 29:** `record("fact_sales_unknown_product_share", sales.filter(F.col("product_key")==0).count()/max(1, sales.count()), 0.05, "<=")` — `record` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"fact_sales_unknown_product_share"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sales.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"product_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `)==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 30:** `record("fact_sales_unknown_customer_share", sales.filter(F.col("customer_key")==0).count()/max(1, sales.count()), 0.05, "<=")` — `record` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"fact_sales_unknown_customer_share"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sales.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"customer_key"` is a quoted literal; the quotation marks preserve spaces or mark text; `)==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 31:** blank line used only to separate logical sections for readability.
- **Line 32:** `results=spark.createDataFrame(checks, [ "batch_id", "check_name", "actual_value", "expected_value", "operator", "status", "required_flag" ]).withColumn("checked_at_utc", F.current_timestamp())` — `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spark.createDataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `checks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"check_name"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"actual_value"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 33:** `results.write.format("delta").mode("append").saveAsTable(f"{catalog}.{quality_schema}.test_results")` — `results.write.format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"delta"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"append"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `saveAsTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{catalog}.{qual-`

ity_schema}.test_results" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 34:** `display(results)` — `display` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** `failed=results.filter((F.col("required_flag")) & (F.col("status")=="FAIL")).count()` — `failed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `results.filter` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `((` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"required_flag"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `F.col` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 36:** `dbutils.jobs.taskValues.set(key="quality_status",value="PASS" if failed==0 else "FAIL")` — `dbutils.jobs.taskValues.set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"quality_status"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `value` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 37:** `if failed: raise AssertionError(f"QUALITY GATE FAILED: {failed} required checks failed. Read the test_results table before repairing the run.")` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AssertionError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"QUALITY GATE FAILED: {failed} required checks failed. Read the test_results table before repairing the run."` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 38:** `print("QUALITY GATE PASSED")` — `print` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"QUALITY GATE PASSED"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 13 lesson index](#)

File introduction — `sql/databricks/quality/01_quality_gate.sql`

Exact filename: `01_quality_gate.sql`

Filename extension: `.sql`. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: sql/databricks/quality relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 13.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in sql/databricks/quality one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 01_quality_gate.sql.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 01_quality_gate.sql and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 13.F2.1 — Create the file sql/databricks/quality/01_quality_gate.sql from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\databricks\quality\01_quality_gate.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\databricks\quality\01_quality_gate.sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\databricks\quality\01_quality_gate.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\sql\databricks\quality\01_quality_gate.sql" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/databricks/quality/01_quality_gate.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `sql/databricks/quality`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `sql/databricks/quality/01_quality_gate.sql`.
3. In PowerShell at the repository root, run `Test-Path ".\sql\databricks\quality\01_quality_gate.sql"` and require True.
4. Run `git status --short -- "sql/databricks/quality/01_quality_gate.sql"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `sql/databricks/quality/01_quality_gate.sql`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: `sql/databricks/quality/01_quality_gate.sql`

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 13.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: Databricks SQL / Spark SQL
-- These assertion queries should each return zero failed rows.
SELECT COUNT(*) AS duplicate_order_item_rows
FROM (
  SELECT order_item_id FROM IDENTIFIER(:fact_sales_table)
  GROUP BY order_item_id HAVING COUNT(*) > 1
);

SELECT COUNT(*) AS invalid_revenue_identity_rows
FROM IDENTIFIER(:fact_sales_table)
WHERE abs(net_sales_amount - (gross_sales_amount - discount_amount)) > 0.01;
```

Line-by-line explanation

- **Line 1:** `-- Dialect: Databricks SQL / Spark SQL` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `-- These assertion queries should each return zero failed rows.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** `SELECT COUNT(*) AS duplicate_order_item_rows` — `SELECT` requests data without changing it unless combined with a modifying statement; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-

specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; duplicate_order_item_rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 4:** FROM (— FROM identifies the source table or query; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** SELECT order_item_id FROM IDENTIFIER(:fact_sales_table) — SELECT requests data without changing it unless combined with a modifying statement; order_item_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; IDENTIFIER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales_table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** GROUP BY order_item_id HAVING COUNT(*) > 1 — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_item_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; HAVING is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; > is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression
- **Line 7:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** SELECT COUNT(*) AS invalid_revenue_identity_rows — SELECT requests data without changing it unless combined with a modifying statement; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; invalid_revenue_identity_rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** FROM IDENTIFIER(:fact_sales_table) — FROM identifies the source table or query; IDENTIFIER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (: is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales_table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** WHERE abs(net_sales_amount - (gross_sales_amount - discount_amount)) > 0.01; — WHERE filters rows to those meeting a condition; abs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; net_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; - is a command option or flag; its exact behavior is defined beside the command; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gross_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; - is a command option or flag; its exact behavior is defined beside the command; discount_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; > is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0.01 is a numeric literal used by the command or expression

[Back to the Phase 13 lesson index](#)

File introduction — tests/data_quality/test_sample_quality_fixture.py

Exact filename: test_sample_quality_fixture.py

Filename extension: .py. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: tests/data_quality relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source

SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 13.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in tests/data_quality one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: test_sample_quality_fixture.py.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled test_sample_quality_fixture.py and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 13.F3.1 — Create the file tests/data_quality/test_sample_quality_fixture.py from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\tests\data_quality\test_sample_quality_fixture.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\tests\data_quality\test_sample_quality_fixture.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\tests\data_quality\test_sample_quality_fixture.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\tests\data_quality\test_sample_quality_fixture.py" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "tests/data_quality/test_sample_quality_fixture.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `tests/data_quality`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `tests/data_quality/test_sample_quality_fixture.py`.
3. In PowerShell at the repository root, run `Test-Path ".\tests\data_quality\test_sample_quality_fixture.py"` and require True.
4. Run `git status --short -- "tests/data_quality/test_sample_quality_fixture.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `tests/data_quality/test_sample_quality_fixture.py`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `tests/data_quality/test_sample_quality_fixture.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 13.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
from __future__ import annotations

import json
from pathlib import Path

import pandas as pd

ROOT = Path("datasets/sample")

def test_controlled_dirty_data_is_present() -> None:
    customers = pd.read_csv(ROOT / "sqlserver/customers.csv")
    items = pd.read_csv(ROOT / "sqlserver/order_items.csv")
    expected = json.loads((ROOT / "metadata/expected_quality_issues.json").read_text())["configured_issue_counts"]
    assert customers.email.isna().sum() >= expected["null_emails"]
    assert (items.quantity <= 0).sum() >= expected["impossible_quantities"]
    assert customers.customer_number.duplicated(keep=False).sum() >= expected["duplicate_business_keys"]

def test_no_unexpected_order_orphans() -> None:
    customers = pd.read_csv(ROOT / "sqlserver/customers.csv")
    orders = pd.read_csv(ROOT / "sqlserver/orders.csv")
    assert not (set(orders.customer_id) - set(customers.customer_id))
```

Line-by-line explanation

- **Line 1:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** `import json` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `json` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `from pathlib import Path` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pathlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** `import pandas as pd` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pandas` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `R00T = Path("datasets/sample")` — `R00T` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"datasets/sample"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** blank line used only to separate logical sections for readability.
- **Line 10:** blank line used only to separate logical sections for readability.
- **Line 11:** `def test_controlled_dirty_data_is_present() -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_controlled_dirty_data_is_present` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `customers = pd.read_csv(R00T / "sqlserver/customers.csv")` — `customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.read_csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `R00T` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"sqlserver/customers.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `items = pd.read_csv(R00T / "sqlserver/order_items.csv")` — `items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.read_csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `R00T` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"sqlserver/order_items.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 14:** `expected = json.loads((ROOT / "metadata/expected_quality_issues.json").read_text())["configured_issue_counts"]` — `expected` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `json.loads` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `((` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ROOT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"metadata/expected_quality_issues.json"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `read_text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"configured_issue_counts"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** `assert customers.email.isna().sum() >= expected["null_emails"]` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customers.email.isna` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sum` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `expected` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"null_emails"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `assert (items.quantity <= 0).sum() >= expected["impossible_quantities"]` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `items.quantity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sum` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `expected` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"impossible_quantities"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 17:** `assert customers.customer_number.duplicated(keep=False).sum() >= expected["duplicate_business_keys"]` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customers.customer_number.duplicated` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `keep` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `False` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sum` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `expected` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** blank line used only to separate logical sections for readability.

- **Line 20:** `def test_no_unexpected_order_orphans() -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_no_unexpected_order_orphans` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** `customers = pd.read_csv(ROOT / "sqlserver/customers.csv")` — `customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.read_csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ROOT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"sqlserver/customers.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** `orders = pd.read_csv(ROOT / "sqlserver/orders.csv")` — `orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.read_csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ROOT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"sqlserver/orders.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** `assert not (set(orders.customer_id) - set(customers.customer_id))` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders.customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-` is a command option or flag; its exact behavior is defined beside the command; `set` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customers.customer_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 13 lesson index](#)

File introduction — tests/reconciliation/test_sample_reconciliation.py

Exact filename: `test_sample_reconciliation.py`

Filename extension: `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `tests/reconciliation` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 13.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in tests/reconciliation one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: test_sample_reconciliation.py.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled test_sample_reconciliation.py and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 13.F4.1 — Create the file tests/reconciliation/test_sample_reconciliation.py from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\tests\reconciliation\test_sample_reconciliation.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\tests\reconciliation\test_sample_reconciliation.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\tests\reconciliation\test_sample_reconciliation.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\tests\reconciliation\test_sample_reconciliation.py" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "tests/reconciliation/test_sample_reconciliation.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `tests/reconciliation`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `tests/reconciliation/test_sample_reconciliation.py`.
3. In PowerShell at the repository root, run `Test-Path ".\tests\reconciliation\test_sample_reconciliation.py"` and require `True`.
4. Run `git status --short -- "tests/reconciliation/test_sample_reconciliation.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for tests/reconciliation/test_sample_reconciliation.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `tests/reconciliation/test_sample_reconciliation.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 13.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
from __future__ import annotations

from pathlib import Path

import pandas as pd

ROOT = Path("datasets/sample")


def test_valid_line_revenue_formula_is_reproducible() -> None:
    items = pd.read_csv(ROOT / "sqlserver/order_items.csv")
    valid = items[items.quantity > 0].copy()
    valid["net"] = valid.quantity * valid.unit_price - valid.discount_amount
    valid["profit"] = valid["net"] - valid.quantity * valid.unit_cost
    assert valid["net"].notna().all()
    assert abs(valid["profit"].sum() - (valid["net"].sum() - (valid.quantity * valid.unit_cost).sum())) < 0.01


def test_inventory_snapshot_grain_is_unique() -> None:
    snapshot = pd.read_csv(ROOT / "sqlserver/inventory_snapshots.csv")
    assert not snapshot.duplicated(["snapshot_date", "store_id", "product_id"]).any()
```

Line-by-line explanation

- **Line 1:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** `from pathlib import Path` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pathlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** blank line used only to separate logical sections for readability.

- **Line 5:** `import pandas as pd` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pandas` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** `R00T = Path("datasets/sample")` — `R00T` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"datasets/sample"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** blank line used only to separate logical sections for readability.
- **Line 10:** `def test_valid_line_revenue_formula_is_reproducible() -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_valid_line_revenue_formula_is_reproducible` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** `items = pd.read_csv(R00T / "sqlserver/order_items.csv")` — `items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.read_csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `R00T` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"sqlserver/order_items.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `valid = items[items.quantity > 0].copy()` — `valid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `items.quantity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `].` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `copy` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `valid["net"] = valid.quantity * valid.unit_price - valid.discount_amount` — `valid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"net"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid.quantity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid.unit_price` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-` is a command option or flag; its exact behavior is defined beside the command; `valid.discount_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** `valid["profit"] = valid["net"] - valid.quantity * valid.unit_cost` — `valid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

"profit" is a quoted literal; the quotation marks preserve spaces or mark text;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "net" is a quoted literal; the quotation marks preserve spaces or mark text;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; - is a command option or flag; its exact behavior is defined beside the command; valid.quantity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; * is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 15:** `assert valid["net"].notna().all()` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"net"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `notna` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `all` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `assert abs(valid["profit"].sum() - (valid["net"].sum() - (valid.quantity * valid.unit_cost).sum())) < 0.01` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `abs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"profit"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sum` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-` is a command option or flag; its exact behavior is defined beside the command; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `valid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 17:** blank line used only to separate logical sections for readability.
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** `def test_inventory_snapshot_grain_is_unique() -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_inventory_snapshot_grain_is_unique` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** `snapshot = pd.read_csv(ROOT / "sqlserver/inventory_snapshots.csv")` — `snapshot` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.read_csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ROOT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"sqlserver/inventory_snapshots.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** `assert not snapshot.duplicated(["snapshot_date", "store_id", "product_id"]).any()` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `snapshot.duplicated` is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; ([is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "snapshot_date" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "store_id" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "product_id" is a quoted literal; the quotation marks preserve spaces or mark text;]) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; any is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 13 lesson index](#)

What your screen should look like now

- Every Phase 13 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 13.3 completion checklist

☐

databricks/notebooks/10_data_quality.py was created by me, saved, located correctly, and inspected — [STATUS]

☐

sql/databricks/quality/01_quality_gate.sql was created by me, saved, located correctly, and inspected — [STATUS]

☐

tests/data_quality/test_sample_quality_fixture.py was created by me, saved, located correctly, and inspected — [STATUS]

☐

tests/reconciliation/test_sample_reconciliation.py was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 13.4](#) and recheck the root and branch.

Lesson 13.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Quality test 1 — required files and manifest completeness

Risk: A batch can be incomplete while later code still finds some files.

Passing means: The actual result meets the stated threshold or invariant for the exact batch and layer.

Failing means: Data or metadata cannot be trusted for its next use; the result must identify the entity, batch, actual value, expected value, severity, and rule ID.

Pipeline behavior: A critical integrity, reconciliation, duplicate, or required-field failure blocks publication. A warning-level issue is recorded and reviewed without being mislabeled as success.

Action 13.4.T1 — Implement and prove the required files and manifest completeness quality test

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Implement and prove the required files and manifest completeness quality test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for required files and manifest completeness.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for required files and manifest completeness.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Teaching assertion

```
def test_required_files_and_manifest_completeness():  
    actual = run_quality_check("required files and manifest completeness")  
    assert actual["status"] == "PASS", actual
```

Line-by-line explanation

- Line 1:** `def test_required_files_and_manifest_completeness():` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_required_files_and_manifest_completeness` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `actual = run_quality_check("required files and manifest completeness")` — `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_quality_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"required files and manifest completeness"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `assert actual["status"] == "PASS", actual` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Controlled-failure record

- Valid run ID/batch: [ENTER]
- Valid result: [ENTER ACTUAL + EXPECTED + PASS]
- Toy defect introduced: [DESCRIBE]
- Failed result: [ENTER ACTUAL + EXPECTED + FAIL]
- Downstream task behavior: [SKIPPED/BLOCKED/OTHER + EVIDENCE]
- Repair made: [DESCRIBE]
- Repaired result: [ENTER]

Troubleshooting

If a test passes when the toy defect exists, suspect that the query filters the defect out, reads the wrong batch/table, compares the wrong grain, or treats null incorrectly. Print the test population and a failing example. Do not weaken the threshold merely to make the status green.

Interview-ready explanation

"I designed required files and manifest completeness from a business/data risk, stored actual and expected values, demonstrated a controlled failure, and verified that the blocking behavior protected downstream publication."

Quality test 2 — manifest checksum match

Risk: A file can be changed, truncated, or replaced after generation.

Passing means: The actual result meets the stated threshold or invariant for the exact batch and layer.

Failing means: Data or metadata cannot be trusted for its next use; the result must identify the entity, batch, actual value, expected value, severity, and rule ID.

Pipeline behavior: A critical integrity, reconciliation, duplicate, or required-field failure blocks publication. A warning-level issue is recorded and reviewed without being mislabeled as success.

Action 13.4.T2 — Implement and prove the manifest checksum match quality test

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Implement and prove the `manifest checksum match` quality test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or `databricks/notebooks/10_data_quality.py`.
3. TYPE THIS: Build the smallest test query or assertion for `manifest checksum match`.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for manifest checksum match.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Teaching assertion

```
def test_manifest_checksum_match():
    actual = run_quality_check("manifest checksum match")
    assert actual["status"] == "PASS", actual
```

Line-by-line explanation

- **Line 1:** `def test_manifest_checksum_match():` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_manifest_checksum_match` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `actual = run_quality_check("manifest checksum match")` — `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_quality_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"manifest checksum match"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `assert actual["status"] == "PASS", actual` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Controlled-failure record

- Valid run ID/batch: [ENTER]
- Valid result: [ENTER ACTUAL + EXPECTED + PASS]
- Toy defect introduced: [DESCRIBE]
- Failed result: [ENTER ACTUAL + EXPECTED + FAIL]
- Downstream task behavior: [SKIPPED/BLOCKED/OTHER + EVIDENCE]
- Repair made: [DESCRIBE]
- Repaired result: [ENTER]

Troubleshooting

If a test passes when the toy defect exists, suspect that the query filters the defect out, reads the wrong batch/table, compares the wrong grain, or treats null incorrectly. Print the test population and a failing example. Do not weaken the threshold merely to make the status green.

Interview-ready explanation

"I designed `manifest_checksum_match` from a business/data risk, stored actual and expected values, demonstrated a controlled failure, and verified that the blocking behavior protected downstream publication."

Quality test 3 — Bronze source-to-target row reconciliation

Risk: Ingestion can omit or duplicate raw records.

Passing means: The actual result meets the stated threshold or invariant for the exact batch and layer.

Failing means: Data or metadata cannot be trusted for its next use; the result must identify the entity, batch, actual value, expected value, severity, and rule ID.

Pipeline behavior: A critical integrity, reconciliation, duplicate, or required-field failure blocks publication. A warning-level issue is recorded and reviewed without being mislabeled as success.

Action 13.4.T3 — Implement and prove the Bronze source-to-target row reconciliation quality test

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Implement and prove the Bronze source-to-target row reconciliation quality test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for Bronze source-to-target row reconciliation.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for Bronze source-to-target row reconciliation.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Teaching assertion

```
def test_bronze_source_to_target_row_reconciliation():
    actual = run_quality_check("Bronze source-to-target row reconciliation")
    assert actual["status"] == "PASS", actual
```

Line-by-line explanation

- **Line 1:** `def test_bronze_source_to_target_row_reconciliation():` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_bronze_source_to_target_row_reconciliation` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `actual = run_quality_check("Bronze source-to-target row reconciliation")` — `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_quality_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Bronze source-to-target row reconciliation"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `assert actual["status"] == "PASS", actual` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Controlled-failure record

- Valid run ID/batch: [ENTER]
- Valid result: [ENTER ACTUAL + EXPECTED + PASS]
- Toy defect introduced: [DESCRIBE]
- Failed result: [ENTER ACTUAL + EXPECTED + FAIL]
- Downstream task behavior: [SKIPPED/BLOCKED/OTHER + EVIDENCE]
- Repair made: [DESCRIBE]
- Repaired result: [ENTER]

Troubleshooting

If a test passes when the toy defect exists, suspect that the query filters the defect out, reads the wrong batch/table, compares the wrong grain, or treats null incorrectly. Print the test population and a failing example. Do not weaken the threshold merely to make the status green.

Interview-ready explanation

"I designed Bronze source-to-target row reconciliation from a business/data risk, stored actual and expected values, demonstrated a controlled failure, and verified that the blocking behavior protected downstream publication."

Quality test 4 — required-key null check

Risk: Rows without required identifiers cannot be joined or deduplicated reliably.

Passing means: The actual result meets the stated threshold or invariant for the exact batch and layer.

Failing means: Data or metadata cannot be trusted for its next use; the result must identify the entity, batch, actual value, expected value, severity, and rule ID.

Pipeline behavior: A critical integrity, reconciliation, duplicate, or required-field failure blocks publication. A warning-level issue is recorded and reviewed without being mislabeled as success.

Action 13.4.T4 — Implement and prove the required-key null check quality test

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Implement and prove the required-key null check quality test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for required-key null check.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.

2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for required-key null check.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Teaching assertion

```
def test_required_key_null_check():
    actual = run_quality_check("required-key null check")
    assert actual["status"] == "PASS", actual
```

Line-by-line explanation

- **Line 1:** `def test_required_key_null_check():` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_required_key_null_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `actual = run_quality_check("required-key null check")` — `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_quality_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

ation; "required-key null check" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 3:** `assert actual["status"] == "PASS", actual` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Controlled-failure record

- Valid run ID/batch: [ENTER]
- Valid result: [ENTER ACTUAL + EXPECTED + PASS]
- Toy defect introduced: [DESCRIBE]
- Failed result: [ENTER ACTUAL + EXPECTED + FAIL]
- Downstream task behavior: [SKIPPED/BLOCKED/OTHER + EVIDENCE]
- Repair made: [DESCRIBE]
- Repaired result: [ENTER]

Troubleshooting

If a test passes when the toy defect exists, suspect that the query filters the defect out, reads the wrong batch/table, compares the wrong grain, or treats null incorrectly. Print the test population and a failing example. Do not weaken the threshold merely to make the status green.

Interview-ready explanation

"I designed required-key null check from a business/data risk, stored actual and expected values, demonstrated a controlled failure, and verified that the blocking behavior protected downstream publication."

Quality test 5 — business-key duplicate check

Risk: Duplicates can multiply facts or create ambiguous dimension matches.

Passing means: The actual result meets the stated threshold or invariant for the exact batch and layer.

Failing means: Data or metadata cannot be trusted for its next use; the result must identify the entity, batch, actual value, expected value, severity, and rule ID.

Pipeline behavior: A critical integrity, reconciliation, duplicate, or required-field failure blocks publication. A warning-level issue is recorded and reviewed without being mislabeled as success.

Action 13.4.T5 — Implement and prove the business-key duplicate check quality test

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Implement and prove the business-key duplicate check quality test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for business-key duplicate check.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for business-key duplicate check.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Teaching assertion

```
def test_business_key_duplicate_check():
    actual = run_quality_check("business-key duplicate check")
    assert actual["status"] == "PASS", actual
```

Line-by-line explanation

- **Line 1:** `def test_business_key_duplicate_check():` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_business_key_duplicate_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `actual = run_quality_check("business-key duplicate check")` — `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_quality_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"business-key duplicate check"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `assert actual["status"] == "PASS", actual` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Controlled-failure record

- Valid run ID/batch: [ENTER]
- Valid result: [ENTER ACTUAL + EXPECTED + PASS]
- Toy defect introduced: [DESCRIBE]
- Failed result: [ENTER ACTUAL + EXPECTED + FAIL]
- Downstream task behavior: [SKIPPED/BLOCKED/OTHER + EVIDENCE]
- Repair made: [DESCRIBE]
- Repaired result: [ENTER]

Troubleshooting

If a test passes when the toy defect exists, suspect that the query filters the defect out, reads the wrong batch/table, compares the wrong grain, or treats null incorrectly. Print the test population and a failing example. Do not weaken the threshold merely to make the status green.

Interview-ready explanation

"I designed `business-key duplicate check` from a business/data risk, stored actual and expected values, demonstrated a controlled failure, and verified that the blocking behavior protected downstream publication."

Quality test 6 — foreign-key orphan check

Risk: Child rows can reference parents that do not exist.

Passing means: The actual result meets the stated threshold or invariant for the exact batch and layer.

Failing means: Data or metadata cannot be trusted for its next use; the result must identify the entity, batch, actual value,

expected value, severity, and rule ID.

Pipeline behavior: A critical integrity, reconciliation, duplicate, or required-field failure blocks publication. A warning-level issue is recorded and reviewed without being mislabeled as success.

Action 13.4.T6 — Implement and prove the foreign-key orphan check quality test

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Implement and prove the foreign-key orphan check quality test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for foreign-key orphan check.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for foreign-key orphan check.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Teaching assertion

```
def test_foreign_key_orphan_check():
    actual = run_quality_check("foreign-key orphan check")
    assert actual["status"] == "PASS", actual
```

Line-by-line explanation

- **Line 1:** `def test_foreign_key_orphan_check():` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_foreign_key_orphan_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `actual = run_quality_check("foreign-key orphan check")` — `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_quality_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"foreign-key orphan check"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `assert actual["status"] == "PASS", actual` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Controlled-failure record

- Valid run ID/batch: [ENTER]
- Valid result: [ENTER ACTUAL + EXPECTED + PASS]
- Toy defect introduced: [DESCRIBE]
- Failed result: [ENTER ACTUAL + EXPECTED + FAIL]
- Downstream task behavior: [SKIPPED/BLOCKED/OTHER + EVIDENCE]
- Repair made: [DESCRIBE]
- Repaired result: [ENTER]

Troubleshooting

If a test passes when the toy defect exists, suspect that the query filters the defect out, reads the wrong batch/table, compares the wrong grain, or treats null incorrectly. Print the test population and a failing example. Do not weaken the threshold merely to make the status green.

Interview-ready explanation

"I designed foreign-key orphan check from a business/data risk, stored actual and expected values, demonstrated a controlled failure, and verified that the blocking behavior protected downstream publication."

Quality test 7 — accepted-plus-rejected reconciliation

Risk: Cleaning can silently discard rows.

Passing means: The actual result meets the stated threshold or invariant for the exact batch and layer.

Failing means: Data or metadata cannot be trusted for its next use; the result must identify the entity, batch, actual value, expected value, severity, and rule ID.

Pipeline behavior: A critical integrity, reconciliation, duplicate, or required-field failure blocks publication. A warning-level issue is recorded and reviewed without being mislabeled as success.

Action 13.4.T7 — Implement and prove the accepted-plus-rejected reconciliation quality test

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Implement and prove the accepted-plus-rejected reconciliation quality test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for accepted-plus-rejected reconciliation.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.

10. PRESS: Rerun and require PASS.

11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for accepted-plus-rejected reconciliation.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Teaching assertion

```
def test_accepted_plus_rejected_reconciliation():
    actual = run_quality_check("accepted-plus-rejected reconciliation")
    assert actual["status"] == "PASS", actual
```

Line-by-line explanation

- **Line 1:** `def test_accepted_plus_rejected_reconciliation():` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_accepted_plus_rejected_reconciliation` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `actual = run_quality_check("accepted-plus-rejected reconciliation")` — `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_quality_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"accepted-plus-rejected reconciliation"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `assert actual["status"] == "PASS", actual` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Controlled-failure record

- Valid run ID/batch: [ENTER]
- Valid result: [ENTER ACTUAL + EXPECTED + PASS]
- Toy defect introduced: [DESCRIBE]
- Failed result: [ENTER ACTUAL + EXPECTED + FAIL]
- Downstream task behavior: [SKIPPED/BLOCKED/OTHER + EVIDENCE]
- Repair made: [DESCRIBE]
- Repaired result: [ENTER]

Troubleshooting

If a test passes when the toy defect exists, suspect that the query filters the defect out, reads the wrong batch/table, compares the wrong grain, or treats null incorrectly. Print the test population and a failing example. Do not weaken the threshold merely to make the status green.

Interview-ready explanation

"I designed `accepted-plus-rejected_reconciliation` from a business/data risk, stored actual and expected values, demonstrated a controlled failure, and verified that the blocking behavior protected downstream publication."

Quality test 8 — SCD2 current-row uniqueness

Risk: Multiple current versions make fact lookups ambiguous.

Passing means: The actual result meets the stated threshold or invariant for the exact batch and layer.

Failing means: Data or metadata cannot be trusted for its next use; the result must identify the entity, batch, actual value, expected value, severity, and rule ID.

Pipeline behavior: A critical integrity, reconciliation, duplicate, or required-field failure blocks publication. A warning-level issue is recorded and reviewed without being mislabeled as success.

Action 13.4.T8 — Implement and prove the SCD2 current-row uniqueness quality test

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Implement and prove the SCD2 current-row uniqueness quality test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for SCD2 current-row uniqueness.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for SCD2 current-row uniqueness.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Teaching assertion

```
def test_scd2_current_row_uniqueness():
    actual = run_quality_check("SCD2 current-row uniqueness")
    assert actual["status"] == "PASS", actual
```

Line-by-line explanation

- **Line 1:** `def test_scd2_current_row_uniqueness():` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_scd2_current_row_uniqueness` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `actual = run_quality_check("SCD2 current-row uniqueness")` — `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_quality_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SCD2 current-row uniqueness"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `assert actual["status"] == "PASS", actual` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Controlled-failure record

- Valid run ID/batch: [ENTER]
- Valid result: [ENTER ACTUAL + EXPECTED + PASS]
- Toy defect introduced: [DESCRIBE]
- Failed result: [ENTER ACTUAL + EXPECTED + FAIL]
- Downstream task behavior: [SKIPPED/BLOCKED/OTHER + EVIDENCE]

- Repair made: [DESCRIBE]
- Repaired result: [ENTER]

Troubleshooting

If a test passes when the toy defect exists, suspect that the query filters the defect out, reads the wrong batch/table, compares the wrong grain, or treats null incorrectly. Print the test population and a failing example. Do not weaken the threshold merely to make the status green.

Interview-ready explanation

"I designed SCD2 current-row uniqueness from a business/data risk, stored actual and expected values, demonstrated a controlled failure, and verified that the blocking behavior protected downstream publication."

Quality test 9 — SCD2 effective-window overlap

Risk: Overlapping history windows can resolve one event to multiple versions.

Passing means: The actual result meets the stated threshold or invariant for the exact batch and layer.

Failing means: Data or metadata cannot be trusted for its next use; the result must identify the entity, batch, actual value, expected value, severity, and rule ID.

Pipeline behavior: A critical integrity, reconciliation, duplicate, or required-field failure blocks publication. A warning-level issue is recorded and reviewed without being mislabeled as success.

Action 13.4.T9 — Implement and prove the SCD2 effective-window overlap quality test

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Implement and prove the SCD2 effective-window overlap quality test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for SCD2 effective-window overlap.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for SCD2 effective-window overlap.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Teaching assertion

```
def test_scd2_effective_window_overlap():
    actual = run_quality_check("SCD2 effective-window overlap")
    assert actual["status"] == "PASS", actual
```

Line-by-line explanation

- **Line 1:** `def test_scd2_effective_window_overlap():` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_scd2_effective_window_overlap` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is

a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 2:** `actual = run_quality_check("SCD2 effective-window overlap")` — `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_quality_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SCD2 effective-window overlap"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `assert actual["status"] == "PASS", actual` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Controlled-failure record

- Valid run ID/batch: [ENTER]
- Valid result: [ENTER ACTUAL + EXPECTED + PASS]
- Toy defect introduced: [DESCRIBE]
- Failed result: [ENTER ACTUAL + EXPECTED + FAIL]
- Downstream task behavior: [SKIPPED/BLOCKED/OTHER + EVIDENCE]
- Repair made: [DESCRIBE]
- Repaired result: [ENTER]

Troubleshooting

If a test passes when the toy defect exists, suspect that the query filters the defect out, reads the wrong batch/table, compares the wrong grain, or treats null incorrectly. Print the test population and a failing example. Do not weaken the threshold merely to make the status green.

Interview-ready explanation

"I designed SCD2 effective-window overlap from a business/data risk, stored actual and expected values, demonstrated a controlled failure, and verified that the blocking behavior protected downstream publication."

Quality test 10 — fact grain duplicate check

Risk: A repeated business event overstates counts and amounts.

Passing means: The actual result meets the stated threshold or invariant for the exact batch and layer.

Failing means: Data or metadata cannot be trusted for its next use; the result must identify the entity, batch, actual value, expected value, severity, and rule ID.

Pipeline behavior: A critical integrity, reconciliation, duplicate, or required-field failure blocks publication. A warning-level issue is recorded and reviewed without being mislabeled as success.

Action 13.4.T10 — Implement and prove the fact grain duplicate check quality test

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Implement and prove the fact `grain duplicate check` quality test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or `databricks/notebooks/10_data_quality.py`.
3. TYPE THIS: Build the smallest test query or assertion for `fact grain duplicate check`.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or `databricks/notebooks/10_data_quality.py`.
3. TYPE THIS: Build the smallest test query or assertion for `fact grain duplicate check`.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Teaching assertion

```
def test_fact_grain_duplicate_check():
    actual = run_quality_check("fact grain duplicate check")
    assert actual["status"] == "PASS", actual
```

Line-by-line explanation

- **Line 1:** `def test_fact_grain_duplicate_check():` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_fact_grain_duplicate_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `actual = run_quality_check("fact grain duplicate check")` — `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_quality_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"fact grain duplicate check"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `assert actual["status"] == "PASS", actual` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Controlled-failure record

- Valid run ID/batch: [ENTER]
- Valid result: [ENTER ACTUAL + EXPECTED + PASS]
- Toy defect introduced: [DESCRIBE]
- Failed result: [ENTER ACTUAL + EXPECTED + FAIL]
- Downstream task behavior: [SKIPPED/BLOCKED/OTHER + EVIDENCE]
- Repair made: [DESCRIBE]
- Repaired result: [ENTER]

Troubleshooting

If a test passes when the toy defect exists, suspect that the query filters the defect out, reads the wrong batch/table, compares the wrong grain, or treats null incorrectly. Print the test population and a failing example. Do not weaken the threshold merely to make the status green.

Interview-ready explanation

"I designed fact grain duplicate check from a business/data risk, stored actual and expected values, demonstrated a controlled failure, and verified that the blocking behavior protected downstream publication."

Quality test 11 — sales count and amount reconciliation

Risk: Gold revenue or quantities can diverge from accepted Silver sales.

Passing means: The actual result meets the stated threshold or invariant for the exact batch and layer.

Failing means: Data or metadata cannot be trusted for its next use; the result must identify the entity, batch, actual value, expected value, severity, and rule ID.

Pipeline behavior: A critical integrity, reconciliation, duplicate, or required-field failure blocks publication. A warning-level issue is recorded and reviewed without being mislabeled as success.

Action 13.4.T11 — Implement and prove the sales count and amount reconciliation quality test

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Implement and prove the sales count and amount reconciliation quality test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for sales count and amount reconciliation.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for sales count and amount reconciliation.

4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Teaching assertion

```
def test_sales_count_and_amount_reconciliation():
    actual = run_quality_check("sales count and amount reconciliation")
    assert actual["status"] == "PASS", actual
```

Line-by-line explanation

- **Line 1:** `def test_sales_count_and_amount_reconciliation():` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_sales_count_and_amount_reconciliation` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `actual = run_quality_check("sales count and amount reconciliation")` — `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_quality_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"sales count and amount reconciliation"` is a quoted literal; the quotation marks

preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 3:** `assert actual["status"] == "PASS", actual` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Controlled-failure record

- Valid run ID/batch: [ENTER]
- Valid result: [ENTER ACTUAL + EXPECTED + PASS]
- Toy defect introduced: [DESCRIBE]
- Failed result: [ENTER ACTUAL + EXPECTED + FAIL]
- Downstream task behavior: [SKIPPED/BLOCKED/OTHER + EVIDENCE]
- Repair made: [DESCRIBE]
- Repaired result: [ENTER]

Troubleshooting

If a test passes when the toy defect exists, suspect that the query filters the defect out, reads the wrong batch/table, compares the wrong grain, or treats null incorrectly. Print the test population and a failing example. Do not weaken the threshold merely to make the status green.

Interview-ready explanation

"I designed sales count and amount reconciliation from a business/data risk, stored actual and expected values, demonstrated a controlled failure, and verified that the blocking behavior protected downstream publication."

Quality test 12 — unknown-key rate threshold

Risk: Excessive key 0 usage hides conformance or late-arriving problems.

Passing means: The actual result meets the stated threshold or invariant for the exact batch and layer.

Failing means: Data or metadata cannot be trusted for its next use; the result must identify the entity, batch, actual value, expected value, severity, and rule ID.

Pipeline behavior: A critical integrity, reconciliation, duplicate, or required-field failure blocks publication. A warning-level issue is recorded and reviewed without being mislabeled as success.

Action 13.4.T12 — Implement and prove the unknown-key rate threshold quality test

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Implement and prove the unknown-key rate threshold quality test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for unknown-key rate threshold.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for unknown-key rate threshold.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Teaching assertion

```
def test_unknown_key_rate_threshold():
    actual = run_quality_check("unknown-key rate threshold")
    assert actual["status"] == "PASS", actual
```

Line-by-line explanation

- **Line 1:** `def test_unknown_key_rate_threshold():` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_unknown_key_rate_threshold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `actual = run_quality_check("unknown-key rate threshold")` — `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_quality_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"unknown-key rate threshold"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `assert actual["status"] == "PASS", actual` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Controlled-failure record

- Valid run ID/batch: [ENTER]
- Valid result: [ENTER ACTUAL + EXPECTED + PASS]
- Toy defect introduced: [DESCRIBE]
- Failed result: [ENTER ACTUAL + EXPECTED + FAIL]
- Downstream task behavior: [SKIPPED/BLOCKED/OTHER + EVIDENCE]
- Repair made: [DESCRIBE]
- Repaired result: [ENTER]

Troubleshooting

If a test passes when the toy defect exists, suspect that the query filters the defect out, reads the wrong batch/table, compares the wrong grain, or treats null incorrectly. Print the test population and a failing example. Do not weaken the threshold merely to make the status green.

Interview-ready explanation

"I designed unknown-key rate threshold from a business/data risk, stored actual and expected values, demonstrated a controlled failure, and verified that the blocking behavior protected downstream publication."

Quality test 13 — freshness and batch-completion check

Risk: A successful old batch can be mistaken for current data.

Passing means: The actual result meets the stated threshold or invariant for the exact batch and layer.

Failing means: Data or metadata cannot be trusted for its next use; the result must identify the entity, batch, actual value,

expected value, severity, and rule ID.

Pipeline behavior: A critical integrity, reconciliation, duplicate, or required-field failure blocks publication. A warning-level issue is recorded and reviewed without being mislabeled as success.

Action 13.4.T13 — Implement and prove the freshness and batch-completion check quality test

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Implement and prove the freshness and batch-completion check quality test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for freshness and batch-completion check.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for freshness and batch-completion check.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.

11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Teaching assertion

```
def test_freshness_and_batch_completion_check():
    actual = run_quality_check("freshness and batch-completion check")
    assert actual["status"] == "PASS", actual
```

Line-by-line explanation

- **Line 1:** `def test_freshness_and_batch_completion_check():` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_freshness_and_batch_completion_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `actual = run_quality_check("freshness and batch-completion check")` — `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_quality_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"freshness and batch-completion check"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `assert actual["status"] == "PASS", actual` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks

preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; actual is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Controlled-failure record

- Valid run ID/batch: [ENTER]
- Valid result: [ENTER ACTUAL + EXPECTED + PASS]
- Toy defect introduced: [DESCRIBE]
- Failed result: [ENTER ACTUAL + EXPECTED + FAIL]
- Downstream task behavior: [SKIPPED/BLOCKED/OTHER + EVIDENCE]
- Repair made: [DESCRIBE]
- Repaired result: [ENTER]

Troubleshooting

If a test passes when the toy defect exists, suspect that the query filters the defect out, reads the wrong batch/table, compares the wrong grain, or treats null incorrectly. Print the test population and a failing example. Do not weaken the threshold merely to make the status green.

Interview-ready explanation

"I designed freshness and batch-completion check from a business/data risk, stored actual and expected values, demonstrated a controlled failure, and verified that the blocking behavior protected downstream publication."

Quality test 14 — warehouse publication gate

Risk: Unverified Gold data can reach reporting.

Passing means: The actual result meets the stated threshold or invariant for the exact batch and layer.

Failing means: Data or metadata cannot be trusted for its next use; the result must identify the entity, batch, actual value, expected value, severity, and rule ID.

Pipeline behavior: A critical integrity, reconciliation, duplicate, or required-field failure blocks publication. A warning-level issue is recorded and reviewed without being mislabeled as success.

Action 13.4.T14 — Implement and prove the warehouse publication gate quality test

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Implement and prove the warehouse publication gate quality test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for warehouse publication gate.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.

7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. LOOK FOR: Read the risk statement and write what a passing and failing result mean before code.
2. CLICK: Open the named local test file or databricks/notebooks/10_data_quality.py.
3. TYPE THIS: Build the smallest test query or assertion for warehouse publication gate.
4. PRESS: Run it against the valid sample or current verified batch.
5. LOOK FOR: Require a PASS result with actual and expected values stored.
6. TYPE THIS: Introduce a tiny toy failure in a disposable fixture or temporary DataFrame; do not corrupt verified project data.
7. PRESS: Run the test again.
8. LOOK FOR: Require a controlled failure and verify downstream publication is skipped when the test is blocking.
9. TYPE THIS: Remove or repair the toy failure.
10. PRESS: Rerun and require PASS.
11. DO NOT CONTINUE UNTIL: You have evidence for valid, failed, and repaired behavior.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 13 lesson index](#)

Teaching assertion

```
def test_warehouse_publication_gate():
    actual = run_quality_check("warehouse publication gate")
    assert actual["status"] == "PASS", actual
```

Line-by-line explanation

- **Line 1:** `def test_warehouse_publication_gate():` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_warehouse_publication_gate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `actual = run_quality_check("warehouse publication gate")` — `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run_quality_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"warehouse publication gate"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `assert actual["status"] == "PASS", actual` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"status"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PASS"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Controlled-failure record

- Valid run ID/batch: [ENTER]
- Valid result: [ENTER ACTUAL + EXPECTED + PASS]
- Toy defect introduced: [DESCRIBE]
- Failed result: [ENTER ACTUAL + EXPECTED + FAIL]
- Downstream task behavior: [SKIPPED/BLOCKED/OTHER + EVIDENCE]
- Repair made: [DESCRIBE]
- Repaired result: [ENTER]

Troubleshooting

If a test passes when the toy defect exists, suspect that the query filters the defect out, reads the wrong batch/table, compares the wrong grain, or treats null incorrectly. Print the test population and a failing example. Do not weaken the threshold merely to make the status green.

Interview-ready explanation

"I designed `warehouse_publication_gate` from a business/data risk, stored actual and expected values, demonstrated a controlled failure, and verified that the blocking behavior protected downstream publication."

What your screen should look like now

- The new Phase 13 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 13.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 13.5](#).

Lesson 13.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 13 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Verification before automation

Question:

Should the phase prioritize a visible manual verification or jump directly to automation?

Options considered:

- Manual verification first, then automation.
- Automation without a known-good baseline.
- Manual operation forever.

Beginner path selected:

Establish one verified manual baseline and then automate repeatable checks.

Why this path was selected:

A known-good example makes failures interpretable and prevents automation from hiding a wrong assumption.

Tradeoff:

The first run takes longer.

What a company might do instead:

Teams often pair infrastructure as code and automated tests with controlled bootstrap and acceptance procedures.

How to explain this in an interview:

I established a verified baseline, then encoded the same contract in tests and orchestration so automation preserved meaning rather than just speed.

Design decision — Test framework choices

Question:

Which testing tools should validate Python logic, data quality, SQL contracts, and repository structure?

Options considered:

- pytest plus explicit SQL/Databricks quality queries and reconciliation tables.
- Only manual inspection.

- A single vendor-specific data-testing framework for every layer.

Beginner path selected:

Use pytest for local Python/fixture/repository tests and explicit SQL or PySpark checks for database and lakehouse contracts.

Why this path was selected:

The combination is transparent, free, CI-friendly, and teaches the difference between unit, contract, data-quality, and reconciliation tests without hiding logic behind one tool.

Tradeoff:

The learner maintains more than one testing style and must standardize result metadata manually.

What a company might do instead:

A company might add Great Expectations, Soda, dbt tests, Deequ, contract registries, observability platforms, and environment-specific integration suites.

How to explain this in an interview:

I matched the test tool to the execution layer: pytest for deterministic local logic and executable SQL/PySpark gates for live data contracts, all tied to batch evidence.

If you see Result A

Required tests record PASS with expected and actual values and permit publication. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

A blocking rule records FAIL and downstream publication remains skipped. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: VS Code tests, Databricks notebooks, Catalog Explorer, and Job task output.

Window/panel: Show valid test, controlled blocking failure, and repaired result.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase13_build-data-quality-reconciliation-and-failure-gates_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 13's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 13 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 13.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Action 13.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Action 13.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Interpret `git status --short`

- `??` means untracked: Git sees a new path but it is not staged.
- `M` means modified in the working tree but not staged.
- `M` means the staged version differs from the last commit.
- `MM` means a staged version exists and the file has additional unstaged edits.
- `A` means staged as a new file.
- `D` means deletion; stop and investigate unless deletion is deliberate.
- `R` means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 13.5.G4 — Inspect the unstaged diff for databricks/notebooks/10_data_quality.py

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.

4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "databricks/notebooks/10_data_quality.py"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "databricks/notebooks/10_data_quality.py"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"databricks/notebooks/10_data_quality.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Action 13.5.G5 — Stage only the reviewed file databricks/notebooks/10_data_quality.py

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "databricks/notebooks/10_data_quality.py"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "databricks/notebooks/10_data_quality.py"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"databricks/notebooks/10_data_quality.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 13.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Action 13.5.G7 — Create the Phase 13 commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.

7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "test: add blocking quality and reconciliation gates"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "test: add blocking quality and reconciliation gates"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"test: add blocking quality and reconciliation gates"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Action 13.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Action 13.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.

7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 13 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message test: add blocking quality and reconciliation gates and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 13.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 13 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

"Phase 13 created the verified boundary described by **Build data-quality, reconciliation, and failure gates**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed."

Mini quiz

1. Multiple choice: Why does Phase 13 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** PLANNED, CREATED BY USER, and RUN BY USER describe earlier states; only inspected evidence supports verification.

My Phase 13 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 13.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 14 connects them in dependency order and proves repair behavior.

Phase 13 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 14 — Create and operate a multi-task Databricks Job

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: A successful, idempotent Databricks Job run.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 15 exports only verified Gold results and loads them into the local warehouse.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 14 lesson index

1. [Lesson 14.1 — Understand the phase and starting screen](#)
2. [Lesson 14.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 14.3 — Create and understand the phase files](#)
4. [Lesson 14.4 — Build and validate required objects](#)
5. [Lesson 14.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 14.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** Databricks Jobs & Pipelines, notebook task pages, run history, and notifications settings.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 14 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 13 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 14 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

Databricks Jobs & Pipelines, notebook task pages, run history, and notifications settings. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 14 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 14 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 14 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 14.1 completion checklist

☐

I read why Phase 14 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 14.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 14 lesson index](#)

Lesson 14.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 14 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

A parameterized Lakeflow Job containing environment validation, three Bronze tasks, four Silver domain tasks, Gold dimensions, Gold facts, quality gate, publication, and final reconciliation, with correct dependencies, compute, retries, timeouts, notifications, schedule decision, repair-run procedure, and a version-controlled definition.

2. Why it matters

A set of notebooks becomes a pipeline only when execution order, shared parameters, failure behavior, retries, and operational evidence are controlled. Orchestration prevents a user from accidentally publishing before upstream tasks or tests finish.

3. Words to know

Term	Plain-language definition
Job	Named orchestrated workflow.
Task	One executable unit inside a Job.
Dependency	Task that must succeed before another starts.
Parameter	Runtime value passed to tasks, such as batch ID.
Retry	Automatic rerun for likely transient failure.
Timeout	Maximum task/job runtime before failure.
Notification	Message on selected run events.
Schedule	Time/trigger for starting a Job.
Repair run	Rerun failed/skipped downstream tasks within an existing run.
Run history	Stored status, timing, parameters, logs, task graph.
Concurrent run	Multiple Job instances executing at once.
Job cluster/serverless	Compute selected for Job tasks; availability varies.
Declarative Automation Bundle	Current Databricks version-controlled deployment packaging; formerly Asset Bundles in older naming.
Task dependency diagram	Visual execution graph.

4. What must already be completed

Notebooks 00–12 exist in workspace; manual runs through quality are understood; exact parameter names are consistent; current workspace jobs/compute limitations are reviewed.

5. Applications to open

Application	How to open it	Why it is open
Databricks Jobs & Pipelines / Workflows UI	Left navigation current equivalent.	Create/configure/run Job.
Workspace notebooks	Open task files.	Confirm paths/parameters.

Application	How to open it	Why it is open
VS Code	Open job_definition.example.yml and task map.	Version-control graph.
Email/notification destination	Only if appropriate and permitted.	Test notifications without exposing data.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run Get-Location; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
batch_id = EXT-YYYYMMDDTHHMMSSZ	Selects one uploaded extraction batch for every task.	A Job without a concrete batch can mix/default wrong input.
print({name: dbutils.widgets.get(name) for name in ["catalog", "batch_id"]})	Shows key task parameters.	Detects parameter propagation failure.
python -c "import yaml, pathlib; p=yaml.safe_load(pathlib.Path('databricks/jobs/job_definition.example.yml').read_text()); print([t['task_key'] for t in p['resources']['jobs']['northstar_retail_daily']['tasks']])"	Parses job YAML and lists tasks.	Version-controlled definition must be valid YAML and complete.
python -m pytest -q tests\integration\test_repository_contract.py	Checks every YAML notebook path exists and Gold dictionary contract.	Prevents stale orchestration references.

Retry policy

Retry transient storage/compute/network availability failures a small number of times with delay. Do not repeatedly retry deterministic schema, permission, missing-path, or quality assertion failures; retries waste quota and hide diagnosis.

Scheduling Track A

Because the local extraction/upload is manual, an automatic daily Job can find no new batch or rerun an old one. The beginner default is manual run after upload. An advanced readiness trigger reads a control object/manifest and selects an unprocessed batch safely.

Version control

The example YAML captures logical resources and real repository-relative notebook paths. A deployable Databricks bundle additionally needs workspace target configuration and supported CLI authentication; keep that optional until the UI Job is verified.

MY TASK

Build the entire Job in the UI, align it to the YAML and dependency diagram, run a valid batch, demonstrate blocked publication and repair, and document manual-versus-scheduled trigger decision.

STARTER SKELETON

```
resources:
  jobs:
```



```
northstar_retail_daily:
  name: northstar-retail-daily
  max_concurrent_runs: 1
  parameters:
    - name: batch_id
      default: TODO(USER)-REQUIRED
  tasks:
    - task_key: TODO(USER)
      depends_on:
        - task_key: TODO(USER)
      notebook_task:
        notebook_path: ../notebooks/TODOTODO(USER).py
        base_parameters:
          batch_id: "{{job.parameters.batch_id}}"
        max_retries: TODO(USER)
        timeout_seconds: TODO(USER)
# TODO(USER): fill only after building/validating UI graph.
```

Acceptance criteria: UI graph and YAML contain all real notebook tasks/dependencies; all tasks receive same parameters; valid run succeeds; failed gate skips publish; repair succeeds; concurrent/schedule/retry decisions are documented.

HINT 1 — CONCEPT

Start with the final gate chain and work backward to prerequisites.

HINT 2 — PSEUDOCODE

Dependency pseudocode: environment → Bronze parallel → Silver domains → dimensions → facts → quality → publish → reconcile.

HINT 3 — NEAR-COMPLETE PATTERN

Compare every task to in-workbook `job_definition.example.yml` reference, but enter it yourself in UI and verify paths.

13. Expected files or output

- Lakeflow Job graph with 13 tasks.
- Shared parameter set.
- Valid run ID/history.
- Failed/skipped/repair evidence.
- Updated version-controlled YAML/task map.
- Trigger/retry/timeout/notification decision log.

14. How to verify

- Every task path opens a real notebook.
- No task bypasses quality before publication.
- Parallel tasks stay within workspace limits.
- Max concurrent runs 1.
- Run parameters consistent.
- Repair does not duplicate data.
- YAML parses and contract test passes.

15. Automated tests

- Job dependency graph review.
- Valid end-to-end run.
- Controlled failure/skip/repair.
- Same-batch idempotency.
- YAML parsing and path tests.
- Notification test if configured.
- Manual/schedule readiness behavior.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
Notebook path invalid	Workspace Git/import path differs from YAML.	Open task source selector and repository paths.	Point to actual synced notebook; update workspace-neutral mapping.	Task opens/runs.
Parameter default used accidentally	Run dialog not set or task mapping wrong.	Inspect run/task parameters/log output.	Require/validate batch placeholder and pass Job parameters.	All print exact batch.
Retries repeat quality failure	Retry configured for deterministic assertion.	Run attempts/error same each time.	Reduce retry or classify; fix data/code, use repair.	Quota not wasted.
Publish runs too early	Missing/wrong dependency.	Inspect graph edges and run timeline.	Make publish directly depend on successful quality gate.	Failure skips publish.
Schedule reruns old batch	Manual handoff has no readiness selection.	Inspect scheduled parameter/history.	Keep manual or implement processed-batch control/readiness gate.	Each run selects new intended batch.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why set max concurrent runs to one?
2. Which failures deserve automatic retry?
3. Why is a clock schedule risky in Track A?
4. What does a repair run preserve compared with starting blindly?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. Which task must `publish_gold` depend on?
2. How many parallel Bronze tasks are in the documented graph?
3. What current Databricks term replaces older Asset Bundles naming?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture full task graph, run parameters, successful run history, and controlled failure with publish skipped/repair. Show run ID; redact email if needed.

Save the image under `docs/images/` with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
feat: orchestrate northstar lakeflow job with quality gate
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

All tasks created — [STATUS]

☐

Dependencies match diagram — [STATUS]

☐

Parameters propagate — [STATUS]

☐

Compute/retries/timeouts documented — [STATUS]

☐

Concurrency/schedule decision documented — [STATUS]

☐

Valid run passes — [STATUS]

☐

Failure/skip/repair proven — [STATUS]

☐

YAML/path tests pass — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-14. The quiz answer is in the answer-key section immediately after the quiz in this workbook#phase-14.

Next phase: Phase 15 exports validated Gold and transactionally publishes it into local SQL Server NorthstarRetail_DW.

Required action cards

Action 14.2.1 — Open databricks/jobs/job_task_map.md and explain every arrow. The graph allows three Bronze tasks in parallel

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Open databricks/jobs/job_task_map.md and explain every arrow. The graph allows three Bronze tasks in parallel. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open databricks/jobs/job_task_map.md and explain every arrow. The graph allows three Bronze tasks in parallel and requires all relevant Silver domains before dimensions.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 1

Action

Open `databricks/jobs/job_task_map.md` and explain every arrow. The graph allows three Bronze tasks in parallel and requires all relevant Silver domains before dimensions.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#)

Action 14.2.2 — In Databricks left navigation open or , choose , name it northstar-retail-daily.

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

In Databricks left navigation open or , choose , name it northstar-retail-daily. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: In Databricks left navigation open **Jobs & Pipelines** or **Workflows**, choose **Create > Job**, name it northstar-retail-daily.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 2****Action**

In Databricks left navigation open **Jobs & Pipelines** or **Workflows**, choose **Create > Job**, name it northstar-retail-daily.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#)

Action 14.2.3 — Add Job parameters for catalog, schemas, volume, and batch_id. Set batch_id default to an unmistakable placeho

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Add Job parameters for catalog, schemas, volume, and batch_id. Set batch_id default to an unmistakable placeho. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Add Job parameters for catalog, schemas, volume, and batch_id. Set batch_id default to an unmistakable placeholder; require a real value at run time.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 3

Action

Add Job parameters for catalog, schemas, volume, and batch_id. Set batch_id default to an unmistakable placeholder; require a real value at run time.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.



I can describe what changed on the screen or filesystem.



I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#)

Action 14.2.4 — Add task `environment_check`, type Notebook, select `00_environment_check`, pass parameters with Job parameter

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Add task `environment_check`, type Notebook, select `00_environment_check`, pass parameters with Job parameter. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.

2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Add task `environment_check`, type Notebook, select `00_environment_check`, pass parameters with Job parameter references, and choose available serverless/job compute.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 4

Action

Add task `environment_check`, type Notebook, select `00_environment_check`, pass parameters with Job parameter references, and choose available serverless/job compute.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

Action 14.2.5 — Add three Bronze tasks depending on environment_check. Configure at most two retries with a sensible interval

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Add three Bronze tasks depending on environment_check. Configure at most two retries with a sensible interval . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Add three Bronze tasks depending on environment_check. Configure at most two retries with a sensible interval for transient reads; deterministic data/assertion failures should still remain visible.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 5

Action

Add three Bronze tasks depending on environment_check. Configure at most two retries with a sensible interval for transient reads; deterministic data/assertion failures should still remain visible.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#)

Action 14.2.6 — Add Silver tasks with exact dependencies: customers/products depend on relevant Bronze files, sales also depen

Application:

pgAdmin 4 or PostgreSQL psql, as named in the action

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Add Silver tasks with exact dependencies: customers/products depend on relevant Bronze files, sales also depen. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Add Silver tasks with exact dependencies: customers/products depend on relevant Bronze files, sales also depends on products, digital depends on PostgreSQL Bronze.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

pgAdmin 4 or PostgreSQL `psql`, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 6****Action**

Add Silver tasks with exact dependencies: customers/products depend on relevant Bronze files, sales also depends on products, digital depends on PostgreSQL Bronze.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#)

Action 14.2.7 — Add Gold dimensions after all four Silver domains, Gold facts after dimensions, quality after facts, publish a

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Add Gold dimensions after all four Silver domains, Gold facts after dimensions, quality after facts, publish a. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Add Gold dimensions after all four Silver domains, Gold facts after dimensions, quality after facts, publish after quality, final reconciliation after publish.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 7****Action**

Add Gold dimensions after all four Silver domains, Gold facts after dimensions, quality after facts, publish after quality, final reconciliation after publish.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#)

Action 14.2.8 — Set max_concurrent_runs=1 so the same target cannot be modified by overlapping runs.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Set max_concurrent_runs=1 so the same target cannot be modified by overlapping runs. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Set max_concurrent_runs=1 so the same target cannot be modified by overlapping runs.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 8

Action

Set `max_concurrent_runs=1` so the same target cannot be modified by overlapping runs.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#)

Action 14.2.9 — Set task/job timeouts based on quick/small observations, not arbitrary production claims. Record default and r

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Set task/job timeouts based on quick/small observations, not arbitrary production claims. Record default and reason. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Set task/job timeouts based on quick/small observations, not arbitrary production claims. Record default and reason.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 9****Action**

Set task/job timeouts based on quick/small observations, not arbitrary production claims. Record default and reason.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location

with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#)

Action 14.2.10 — Configure notifications if the workspace supports them: failure and success for your own address, no data in m

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Configure notifications if the workspace supports them: failure and success for your own address, no data in m. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Configure notifications if the workspace supports them: failure and success for your own address, no data in message body. Test one controlled failure notification.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 10

Action

Configure notifications if the workspace supports them: failure and success for your own address, no data in message body. Test one controlled failure notification.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.



I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#)

Action 14.2.11 — Do not schedule until manual parameterized runs work. Track A has a manual upload, so a daily clock schedule c

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Do not schedule until manual parameterized runs work. Track A has a manual upload, so a daily clock schedule c. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.

3. Perform only this immediate task: Do not schedule until manual parameterized runs work. Track A has a manual upload, so a daily clock schedule can start without a new batch. Choose manual trigger or design a batch-readiness check/notification.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 11****Action**

Do not schedule until manual parameterized runs work. Track A has a manual upload, so a daily clock schedule can start without a new batch. Choose manual trigger or design a batch-readiness check/notification.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

Action 14.2.12 — Run with exact batch ID. Watch graph; open task logs and outputs. Record run ID.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run with exact batch ID. Watch graph; open task logs and outputs. Record run ID. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run with exact batch ID. Watch graph; open task logs and outputs. Record run ID.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 12

Action

Run with exact batch ID. Watch graph; open task logs and outputs. Record run ID.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#)

Action 14.2.13 — Use the controlled quality failure from Phase 13 to verify skipped publish. Restore and use from the earliest

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Use the controlled quality failure from Phase 13 to verify skipped publish. Restore and use from the earliest . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Use the controlled quality failure from Phase 13 to verify skipped publish. Restore and use **Repair run** from the earliest affected task.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 13****Action**

Use the controlled quality failure from Phase 13 to verify skipped publish. Restore and use **Repair run** from the earliest affected task.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#)

Action 14.2.14 — Compare the UI Job to `job_definition.example.yml`. Update the YAML with actual workspace-neutral logic but ke

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Compare the UI Job to `job_definition.example.yml`. Update the YAML with actual workspace-neutral logic but ke. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Compare the UI Job to `job_definition.example.yml`. Update the YAML with actual workspace-neutral logic but keep secrets/individual paths out. For deployable automation, create a root bundle later using current official Declarative Automation Bundles docs.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 14****Action**

Compare the UI Job to `job_definition.example.yml`. Update the YAML with actual workspace-neutral logic but keep secrets/individual paths out. For deployable automation, create a root bundle later using current official Declarative Automation Bundles docs.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#)

Action 14.2.C1 — Databricks Job run parameter dialog

Application:

Databricks workspace in a web browser

Window or panel:

Databricks Job run parameter dialog

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Text instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Text):

```
batch_id = EXT-YYYYMMDDTHHMMSSZ
```

Where to enter it:

Databricks Job run parameter dialog. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `batch_id = EXT-YYYYMMDDTHHMMSSZ` — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `EXT-YYYYMMDDTHHMMSSZ` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

Action 14.2.C2 — Databricks notebook task cell/log

Application:

Databricks workspace in a web browser

Window or panel:

Databricks notebook task cell/log

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Python instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Python):

```
print({name: dbutils.widgets.get(name) for name in ["catalog", "batch_id"]})
```

Where to enter it:

Databricks notebook task cell/log. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `print({name: dbutils.widgets.get(name) for name in ["catalog", "batch_id"]})` — `print` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `;` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dbutils.widgets.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

Action 14.2.C3 — Windows PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -c "import yaml, pathlib; p=yaml.safe_load(pathlib.Path('databricks/jobs/job_definition.example.yaml').read_text()); print([t['task_key'] for t in p['resources']['jobs']['northstar_retail_daily']['tasks'])"
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -c "import yaml, pathlib; p=yaml.safe_load(pathlib.Path('databricks/jobs/job_definition.example.yaml').read_text()); print([t['task_key'] for t in p['resources']['jobs']['northstar_retail_daily']['tasks']])"` — `python` invokes Python when that command name is registered; `-c` creates the named branch before switching; `"import yaml, pathlib; p=yaml.safe_load(pathlib.Path('databricks/jobs/job_definition.example.yaml').read_text()); print([t['task_key'] for t in p['resources']['jobs']['northstar_retail_daily']['tasks']])"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

Action 14.2.C4 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m pytest -q tests\integration\test_repository_contract.py
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m pytest -q tests\integration\test_repository_contract.py` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `pytest` the Python test runner; `-q` is a command option or flag; its exact behavior is defined beside the command; `tests` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `integration` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_repository_contract.py` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 14.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 14 starting-screen checklist and re-verify the active context.

Lesson 14.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 14 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — databricks/

Exact folder name: databricks

Exact parent: repository root

Why it is needed now: Phase 14 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: databricks.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 14.D1.1 — Create the folder databricks

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.

6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\databricks" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\databricks" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\databricks" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\databricks" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\databricks" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — databricks/jobs/

Exact folder name: jobs

Exact parent: databricks

Why it is needed now: Phase 14 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path databricks one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: jobs.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 14.D2.1 — Create the folder databricks/jobs

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\databricks\jobs" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\databricks\jobs" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\databricks\jobs` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\databricks\jobs" -PathType Container` and require True.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\databricks\jobs" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\databricks\jobs" -PathType Container`.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — databricks/jobs/job_definition.example.yml

Exact filename: `job_definition.example.yml`

Filename extension: `.yml`. In this project it identifies human-readable configuration and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `databricks/jobs` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: human-readable configuration for Phase 14.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? Yes—review carefully before Git staging.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `databricks/jobs` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.

6. CLICK: Select **New File**.
7. TYPE THIS: `job_definition.example.yml`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `job_definition.example.yml` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 14.F1.1 — Create the file `databricks/jobs/job_definition.example.yml` from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\databricks\jobs\job_definition.example.yml" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\databricks\jobs\job_definition.example.yml" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\jobs\job_definition.example.yml"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\databricks\jobs\job_definition.example.yml" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "databricks/jobs/job_definition.example.yml"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.

5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use TODO(USER) where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under databricks/jobs.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with databricks/jobs/job_definition.example.yml.
3. In PowerShell at the repository root, run Test-Path ".\databricks\jobs\job_definition.example.yml" and require True.
4. Run git status --short -- "databricks/jobs/job_definition.example.yml" to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for databricks/jobs/job_definition.example.yml

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: YAML

Exact file: databricks/jobs/job_definition.example.yml

Purpose: human-readable configuration.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 14.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Example version-controlled Lakeflow Job definition.
# Paths are repository-relative and point to files the learner manually creates through this workbook.
# Adapt this file to your workspace or convert it into a root databricks.yml bundle.
resources:
  jobs:
    northstar_retail_daily:
      name: northstar-retail-daily
      max_concurrent_runs: 1
      timeout_seconds: 7200
      parameters:
        - name: catalog
          default: northstar_retail
        - name: landing_schema
          default: landing
        - name: bronze_schema
          default: bronze
        - name: silver_schema
          default: silver
        - name: gold_schema
          default: gold
        - name: quality_schema
          default: quality
        - name: volume
          default: track_a_files
        - name: batch_id
          default: MANUAL-REPLACE-ME
      tasks:
        - task_key: environment_check
          notebook_task:
            notebook_path: ../notebooks/00_environment_check.py
            base_parameters: {catalog: "{{job.parameters.catalog}}", landing_schema: "{{job.parameters.landing_schema}}", bronze_schema:
"{{job.parameters.bronze_schema}}", silver_schema: "{{job.parameters.silver_schema}}", gold_schema: "{{job.parameters.gold_schema}}", qual-
ity_schema: "{{job.parameters.quality_schema}}", volume: "{{job.parameters.volume}}", batch_id: "{{job.parameters.batch_id}}"}
          - task_key: bronze_sqlserver
            depends_on: [{task_key: environment_check}]
            notebook_task: {notebook_path: ../notebooks/01_bronze_sqlserver.py, base_parameters: {catalog: "{{job.parameters.catalog}}", land-
ing_schema: "{{job.parameters.landing_schema}}", bronze_schema: "{{job.parameters.bronze_schema}}", volume: "{{job.parameters.volume}}",
batch_id: "{{job.parameters.batch_id}}"}
            max_retries: 2
            min_retry_interval_millis: 60000
          - task_key: bronze_postgres
            depends_on: [{task_key: environment_check}]
            notebook_task: {notebook_path: ../notebooks/02_bronze_postgres.py, base_parameters: {catalog: "{{job.parameters.catalog}}", land-
ing_schema: "{{job.parameters.landing_schema}}", bronze_schema: "{{job.parameters.bronze_schema}}", volume: "{{job.parameters.volume}}",
batch_id: "{{job.parameters.batch_id}}"}
            max_retries: 2
          - task_key: bronze_files
            depends_on: [{task_key: environment_check}]
            notebook_task: {notebook_path: ../notebooks/03_bronze_files.py, base_parameters: {catalog: "{{job.parameters.catalog}}", land-
ing_schema: "{{job.parameters.landing_schema}}", bronze_schema: "{{job.parameters.bronze_schema}}", volume: "{{job.parameters.volume}}",
batch_id: "{{job.parameters.batch_id}}"}
            max_retries: 2
          - task_key: silver_customers
            depends_on: [{task_key: bronze_sqlserver}, {task_key: bronze_files}]
            notebook_task: {notebook_path: ../notebooks/04_silver_customers.py, base_parameters: {catalog: "{{job.parameters.catalog}}",
```

```

bronze_schema: "{{job.parameters.bronze_schema}}", silver_schema: "{{job.parameters.silver_schema}}", quality_schema: "{{job.parameters.quality_schema}}", batch_id: "{{job.parameters.batch_id}}"}
- task_key: silver_products
  depends_on: [{task_key: bronze_sqlserver}, {task_key: bronze_files}]
  notebook_task: {notebook_path: ../notebooks/05_silver_products.py, base_parameters: {catalog: "{{job.parameters.catalog}}", bronze_schema: "{{job.parameters.bronze_schema}}", silver_schema: "{{job.parameters.silver_schema}}", quality_schema: "{{job.parameters.quality_schema}}", batch_id: "{{job.parameters.batch_id}}"}
- task_key: silver_sales
  depends_on: [{task_key: bronze_sqlserver}, {task_key: bronze_files}, {task_key: silver_products}]
  notebook_task: {notebook_path: ../notebooks/06_silver_sales.py, base_parameters: {catalog: "{{job.parameters.catalog}}", bronze_schema: "{{job.parameters.bronze_schema}}", silver_schema: "{{job.parameters.silver_schema}}", quality_schema: "{{job.parameters.quality_schema}}", batch_id: "{{job.parameters.batch_id}}"}
- task_key: silver_digital
  depends_on: [{task_key: bronze_postgres}]
  notebook_task: {notebook_path: ../notebooks/07_silver_digital.py, base_parameters: {catalog: "{{job.parameters.catalog}}", bronze_schema: "{{job.parameters.bronze_schema}}", silver_schema: "{{job.parameters.silver_schema}}", quality_schema: "{{job.parameters.quality_schema}}", batch_id: "{{job.parameters.batch_id}}"}
- task_key: gold_dimensions
  depends_on: [{task_key: silver_customers}, {task_key: silver_products}, {task_key: silver_sales}, {task_key: silver_digital}]
  notebook_task: {notebook_path: ../notebooks/08_gold_dimensions.py, base_parameters: {catalog: "{{job.parameters.catalog}}", bronze_schema: "{{job.parameters.bronze_schema}}", silver_schema: "{{job.parameters.silver_schema}}", gold_schema: "{{job.parameters.gold_schema}}", batch_id: "{{job.parameters.batch_id}}"}
- task_key: gold_facts
  depends_on: [{task_key: gold_dimensions}]
  notebook_task: {notebook_path: ../notebooks/09_gold_facts.py, base_parameters: {catalog: "{{job.parameters.catalog}}", silver_schema: "{{job.parameters.silver_schema}}", gold_schema: "{{job.parameters.gold_schema}}", batch_id: "{{job.parameters.batch_id}}"}
- task_key: data_quality_gate
  depends_on: [{task_key: gold_facts}]
  notebook_task: {notebook_path: ../notebooks/10_data_quality.py, base_parameters: {catalog: "{{job.parameters.catalog}}", gold_schema: "{{job.parameters.gold_schema}}", quality_schema: "{{job.parameters.quality_schema}}", batch_id: "{{job.parameters.batch_id}}"}
- task_key: publish_gold
  depends_on: [{task_key: data_quality_gate}]
  notebook_task: {notebook_path: ../notebooks/11_publish_gold.py, base_parameters: {catalog: "{{job.parameters.catalog}}", landing_schema: "{{job.parameters.landing_schema}}", gold_schema: "{{job.parameters.gold_schema}}", quality_schema: "{{job.parameters.quality_schema}}", volume: "{{job.parameters.volume}}", batch_id: "{{job.parameters.batch_id}}"}
- task_key: final_reconciliation
  depends_on: [{task_key: publish_gold}]
  notebook_task: {notebook_path: ../notebooks/12_demo_queries.py, base_parameters: {catalog: "{{job.parameters.catalog}}", gold_schema: "{{job.parameters.gold_schema}}", quality_schema: "{{job.parameters.quality_schema}}", batch_id: "{{job.parameters.batch_id}}"}

```

Line-by-line explanation

- Line 1:** # Example version-controlled Lakeflow Job definition. is a comment for a human reader; it normally does not change data or execute business logic.
- Line 2:** # Paths are repository-relative and point to files the learner manually creates through this workbook. is a comment for a human reader; it normally does not change data or execute business logic.
- Line 3:** # Adapt this file to your workspace or convert it into a root databricks.yml bundle. is a comment for a human reader; it normally does not change data or execute business logic.
- Line 4:** resources: — resources is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 5:** jobs: — jobs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 6:** northstar_retail_daily: — northstar_retail_daily is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 7:** name: northstar-retail-daily — name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; northstar-retail-daily is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 8:** max_concurrent_runs: 1 — max_concurrent_runs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression
- Line 9:** timeout_seconds: 7200 — timeout_seconds is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 7200 is a numeric literal used by the command or expression
- Line 10:** parameters: — parameters is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 24:** default: track_a_files — default is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; track_a_files is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** - name: batch_id — - is a command option or flag; its exact behavior is defined beside the command; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** default: MANUAL-REPLACE-ME — default is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; MANUAL-REPLACE-ME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:** tasks: — tasks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** - task_key: environment_check — - is a command option or flag; its exact behavior is defined beside the command; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; environment_check is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** notebook_task: — notebook_task is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 30:** notebook_path: ../notebooks/00_environment_check.py — notebook_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ../ is a path, relative location, branch-style name, or file argument; verify every character before running it; notebooks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; 00_environment_check.py is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** base_parameters: {catalog: "{{job.parameters.catalog}}", landing_schema: "{{job.parameters.landing_schema}}", bronze_schema: "{{job.parameters.bronze_schema}}", silver_schema: "{{job.parameters.silver_schema}}", gold_schema: "{{job.parameters.gold_schema}}", quality_schema: "{{job.parameters.quality_schema}}", volume: "{{job.parameters.volume}}", batch_id: "{{job.parameters.batch_id}}"} — base_parameters is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; catalog is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{{job.parameters.catalog}}" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; landing_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{{job.parameters.landing_schema}}" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze_schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 32:** - task_key: bronze_sqlserver — - is a command option or flag; its exact behavior is defined beside the command; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze_sqlserver is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** depends_on: [{task_key: environment_check}] — depends_on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [{ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; task_key is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; environment_check is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; }} is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 34:** notebook_task: {notebook_path: ../notebooks/01_bronze_sqlserver.py, base_parameters: {catalog: "{{job.parameters.catalog}}", landing_schema: "{{job.parameters.landing_schema}}", bronze_schema: "{{job.parameters.bronze_schema}}", volume: "{{job.parameters.volume}}", batch_id: "{{job.parameters.batch_id}}"}} — notebook_task is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; notebook_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ../ is a path, relative location, branch-style name, or file argument; verify every character before running it; notebooks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; 01_bronze_sqlserver.py is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; base_parameters is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 35:** max_retries: 2 — max_retries is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression
- **Line 36:** min_retry_interval_millis: 60000 — min_retry_interval_millis is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 60000 is a numeric literal used by the command or expression
- **Line 37:** - task_key: bronze_postgres — - is a command option or flag; its exact behavior is defined beside the command; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze_postgres is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 38:** depends_on: [{task_key: environment_check}] — depends_on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [{ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; environment_check is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; }} is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 39:** notebook_task: {notebook_path: ../notebooks/02_bronze_postgres.py, base_parameters: {catalog: "{{job.parameters.catalog}}", landing_schema: "{{job.parameters.landing_schema}}", bronze_schema: "{{job.parameters.bronze_schema}}", volume: "{{job.parameters.volume}}", batch_id: "{{job.parameters.batch_id}}"}} — notebook_task is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; notebook_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ../ is a path, relative location, branch-style name, or file argument; verify every character before running it; notebooks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; 02_bronze_postgres.py is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; base_parameters is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 40:** `max_retries: 2` — `max_retries` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression
- **Line 41:** `- task_key: bronze_files` — `-` is a command option or flag; its exact behavior is defined beside the command; `task_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_files` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** `depends_on: [{task_key: environment_check}]` — `depends_on` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `task_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `environment_check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `}]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 43:** `notebook_task: {notebook_path: ../notebooks/03_bronze_files.py, base_parameters: {catalog: "{{job.parameters.catalog}}", landing_schema: "{{job.parameters.landing_schema}}", bronze_schema: "{{job.parameters.bronze_schema}}", volume: "{{job.parameters.volume}}", batch_id: "{{job.parameters.batch_id}}"} }` — `notebook_task` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `notebook_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `../` is a path, relative location, branch-style name, or file argument; verify every character before running it; `notebooks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `03_bronze_files.py` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `base_parameters` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 44:** `max_retries: 2` — `max_retries` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression
- **Line 45:** `- task_key: silver_customers` — `-` is a command option or flag; its exact behavior is defined beside the command; `task_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `silver_customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** `depends_on: [{task_key: bronze_sqlserver}, {task_key: bronze_files}]` — `depends_on` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `task_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_sqlserver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `task_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bronze_files` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `}]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 47:** `notebook_task: {notebook_path: ../notebooks/04_silver_customers.py, base_parameters: {catalog: "{{job.parameters.catalog}}", bronze_schema: "{{job.parameters.bronze_schema}}", silver_schema: "{{job.parameters.silver_schema}}", quality_schema: "{{job.parameters.quality_schema}}", batch_id:`

"{{job.parameters.batch_id}}}" — notebook_task is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; notebook_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ../ is a path, relative location, branch-style name, or file argument; verify every character before running it; notebooks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; 04_silver_customers.py is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; base_parameters is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 48:** - task_key: silver_products — - is a command option or flag; its exact behavior is defined beside the command; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; silver_products is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 49:** depends_on: [{task_key: bronze_sqlserver}, {task_key: bronze_files}] — depends_on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [{ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze_sqlserver is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; }, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze_files is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; }] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 50:** notebook_task: {notebook_path: ../notebooks/05_silver_products.py, base_parameters: {catalog: "{{job.parameters.catalog}}", bronze_schema: "{{job.parameters.bronze_schema}}", silver_schema: "{{job.parameters.silver_schema}}", quality_schema: "{{job.parameters.quality_schema}}", batch_id: "{{job.parameters.batch_id}}"} — notebook_task is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; notebook_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ../ is a path, relative location, branch-style name, or file argument; verify every character before running it; notebooks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; 05_silver_products.py is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; base_parameters is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 51:** - task_key: silver_sales — - is a command option or flag; its exact behavior is defined beside the command; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; silver_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 52:** depends_on: [{task_key: bronze_sqlserver}, {task_key: bronze_files}, {task_key: silver_products}] — depends_on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the

surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [{ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze_sqlserver is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; }, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze_files is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; }, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 53:** notebook_task: {notebook_path: ../notebooks/06_silver_sales.py, base_parameters: {catalog: "{{job.parameters.catalog}}", bronze_schema: "{{job.parameters.bronze_schema}}", silver_schema: "{{job.parameters.silver_schema}}", quality_schema: "{{job.parameters.quality_schema}}", batch_id: "{{job.parameters.batch_id}}"}}} — notebook_task is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; notebook_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ../ is a path, relative location, branch-style name, or file argument; verify every character before running it; notebooks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; 06_silver_sales.py is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; base_parameters is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 54:** - task_key: silver_digital — - is a command option or flag; its exact behavior is defined beside the command; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; silver_digital is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 55:** depends_on: [{task_key: bronze_postgres}] — depends_on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [{ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze_postgres is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; }] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 56:** notebook_task: {notebook_path: ../notebooks/07_silver_digital.py, base_parameters: {catalog: "{{job.parameters.catalog}}", bronze_schema: "{{job.parameters.bronze_schema}}", silver_schema: "{{job.parameters.silver_schema}}", quality_schema: "{{job.parameters.quality_schema}}", batch_id: "{{job.parameters.batch_id}}"}}} — notebook_task is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; notebook_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ../ is a path, relative location, branch-style name, or file argument; verify every character before running it; notebooks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; 07_silver_digital.py is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described

in the surrounding explanation; `base_parameters` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 57:** `- task_key: gold_dimensions —` `-` is a command option or flag; its exact behavior is defined beside the command; `task_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gold_dimensions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 58:** `depends_on: [{task_key: silver_customers}, {task_key: silver_products}, {task_key: silver_sales}, {task_key: silver_digital}]` — `depends_on` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `task_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `silver_customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `task_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `silver_products` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 59:** `notebook_task: {notebook_path: ../notebooks/08_gold_dimensions.py, base_parameters: {catalog: "{{job.parameters.catalog}}", bronze_schema: "{{job.parameters.bronze_schema}}", silver_schema: "{{job.parameters.silver_schema}}", gold_schema: "{{job.parameters.gold_schema}}", batch_id: "{{job.parameters.batch_id}}"} }` — `notebook_task` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `notebook_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `../` is a path, relative location, branch-style name, or file argument; verify every character before running it; `notebooks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `08_gold_dimensions.py` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `base_parameters` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 60:** `- task_key: gold_facts —` `-` is a command option or flag; its exact behavior is defined beside the command; `task_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gold_facts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 61:** `depends_on: [{task_key: gold_dimensions}]` — `depends_on` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `task_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gold_dimensions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `}]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 62:** `notebook_task: {notebook_path: ../notebooks/09_gold_facts.py, base_parameters: {catalog: "{{job.parameters.catalog}}", silver_schema: "{{job.parameters.silver_schema}}", gold_schema: "{{job.parameters.gold_schema}}", batch_id: "{{job.parameters.batch_id}}"} }` — `notebook_task` is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; notebook_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . . / is a path, relative location, branch-style name, or file argument; verify every character before running it; notebooks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; 09_gold_facts.py is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; base_parameters is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 63:** - task_key: data_quality_gate — - is a command option or flag; its exact behavior is defined beside the command; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data_quality_gate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 64:** depends_on: [{task_key: gold_facts}] — depends_on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [{ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gold_facts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; }] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 65:** notebook_task: {notebook_path: ../notebooks/10_data_quality.py, base_parameters: {catalog: "{{job.parameters.catalog}}", gold_schema: "{{job.parameters.gold_schema}}", quality_schema: "{{job.parameters.quality_schema}}", batch_id: "{{job.parameters.batch_id}}"}} — notebook_task is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; notebook_path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . . / is a path, relative location, branch-style name, or file argument; verify every character before running it; notebooks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; 10_data_quality.py is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; base_parameters is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 66:** - task_key: publish_gold — - is a command option or flag; its exact behavior is defined beside the command; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; publish_gold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 67:** depends_on: [{task_key: data_quality_gate}] — depends_on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [{ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; task_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data_quality_gate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; }] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 68:** `notebook_task: {notebook_path: ../notebooks/11_publish_gold.py, base_parameters: {catalog: "{{job.parameters.catalog}}", landing_schema: "{{job.parameters.landing_schema}}", gold_schema: "{{job.parameters.gold_schema}}", quality_schema: "{{job.parameters.quality_schema}}", volume: "{{job.parameters.volume}}", batch_id: "{{job.parameters.batch_id}}"} }` — `notebook_task` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `notebook_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `../` is a path, relative location, branch-style name, or file argument; verify every character before running it; `notebooks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `11_publish_gold.py` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `base_parameters` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 69:** `- task_key: final_reconciliation` — `-` is a command option or flag; its exact behavior is defined beside the command; `task_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `final_reconciliation` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 70:** `depends_on: [{task_key: publish_gold}]` — `depends_on` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `task_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `publish_gold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `}]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 71:** `notebook_task: {notebook_path: ../notebooks/12_demo_queries.py, base_parameters: {catalog: "{{job.parameters.catalog}}", gold_schema: "{{job.parameters.gold_schema}}", quality_schema: "{{job.parameters.quality_schema}}", batch_id: "{{job.parameters.batch_id}}"} }` — `notebook_task` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `notebook_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `../` is a path, relative location, branch-style name, or file argument; verify every character before running it; `notebooks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `12_demo_queries.py` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `base_parameters` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

[Back to the Phase 14 lesson index](#)

File introduction — databricks/jobs/job_task_map.md

Exact filename: `job_task_map.md`

Filename extension: `.md`. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `databricks/jobs` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 14.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in databricks/jobs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `job_task_map.md`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `job_task_map.md` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 14.F2.1 — Create the file `databricks/jobs/job_task_map.md` from PowerShell

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\databricks\jobs\job_task_map.md" -ErrorAction Stop
```


Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\databricks\jobs\job_task_map.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\databricks\jobs\job_task_map.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\databricks\jobs\job_task_map.md" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "databricks/jobs/job_task_map.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use TODO(USER) where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `databricks/jobs`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `databricks/jobs/job_task_map.md`.
3. In PowerShell at the repository root, run `Test-Path ".\databricks\jobs\job_task_map.md"` and require True.
4. Run `git status --short -- "databricks/jobs/job_task_map.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for databricks/jobs/job_task_map.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: `databricks/jobs/job_task_map.md`

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 14.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Lakeflow Jobs task map

```mermaid
flowchart TD
 E[environment_check] --> B1[bronze_sqlserver]
 E --> B2[bronze_postgres]
 E --> B3[bronze_files]
 B1 --> S1[silver_customers]
 B3 --> S1
 B1 --> S2[silver_products]
 B3 --> S2
 B1 --> S3[silver_sales]
 B3 --> S3
 S2 --> S3
 B2 --> S4[silver_digital]
 S1 --> D[gold_dimensions]
 S2 --> D
 S3 --> D
 S4 --> D
 D --> F[gold_facts]
 F --> Q[data_quality_gate]
 Q --> P[publish_gold]
 P --> R[final_reconciliation]
```

The quality gate is intentionally between Gold construction and publication. A failed required check must prevent publication. In Databricks Free Edition, compute and concurrency are quota-limited; this graph uses at most three parallel Bronze tasks and four parallel prerequisites for Gold dimensions, below the documented five-task concurrent Jobs limit as accessed on 2026-08-16.

#### #### Line-by-line explanation

[illegible]

```

or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** `F --> Q[data_quality_gate]` – `F` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-->` is a command option or flag; its exact behavior is defined beside the command; `Q` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `data_quality_gate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** `Q --> P[publish_gold]` – `Q` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-->` is a command option or flag; its exact behavior is defined beside the command; `P` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `publish_gold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `P` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** `P --> R[final_reconciliation]` – `P` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-->` is a command option or flag; its exact behavior is defined beside the command; `R` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `final_reconciliation` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** `'''` –
- **Line 25:** blank line used only to separate logical sections for readability.
- **Line 26:** `The quality gate is intentionally between Gold construction and publication. A failed required check must prevent publication. In Databricks Free Edition, compute and concurrency are quota-limited; this graph uses at most three parallel Bronze tasks and four parallel prerequisites for Gold dimensions, below the documented five-task concurrent Jobs limit as accessed on 2026-08-16.` – `The` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quality` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `intentionally` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `between` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Gold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `construction` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `publication` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `A` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

```

[Back to the Phase 14 lesson index](#phase-14-lesson-index)

## What your screen should look like now

- Every Phase 14 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

## Lesson 14.3 completion checklist

- [ ] `databricks/jobs/job\_definition.example.yml` was created by me, saved, located correctly, and inspected – [STATUS]
- [ ] `databricks/jobs/job\_task\_map.md` was created by me, saved, located correctly, and inspected – [STATUS]

## Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 14.4] (#phase-14-lesson-4) and recheck the root and branch.

<a id="phase-14-lesson-4"></a>

## Lesson 14.4 – Build and validate required objects

## What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

<a id="job-environmentcheck"></a>

### Databricks Job task 1 – `environment\_check`

```

Notebook: `databricks/notebooks/00_environment_check.py`
Depends on: No upstream task; this is the environment/start gate.
Inputs: Job parameters, upstream verified tables/files, and task outputs named in the task map.
Output: A validated table set, quality status, export, or demonstration result for the current Job run.
Retry policy: Retry only transient failures; deterministic data-quality failures require diagnosis rather than repeated compute cost.
Timeout: Set a finite threshold large enough for the fixture but short enough to surface a stuck task.
Idempotency expectation: Repairing or rerunning the task for the same batch must not create unintended duplicates.

```

<a id="action-14-4-j1"></a>

## Action 14.4.J1 – Create and validate the `environment\_check` Lakeflow Job task

**\*\*Application:\*\***  
Databricks workspace in a web browser

**\*\*Window or panel:\*\***  
Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**\*\*Starting condition:\*\***  
The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**\*\*Current folder or location:\*\***  
The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

**\*\*What you are about to do:\*\***  
Create and validate the `environment\_check` Lakeflow Job task. This is one bounded construction or inspection task.

**\*\*Why this action comes now:\*\***  
It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**\*\*Exact physical actions:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name `environment\_check` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to `databricks/notebooks/00\_environment\_check.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select No upstream task; this is the environment/start gate..
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*Command or code:\*\***  
No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**\*\*Where to enter it:\*\***  
Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**\*\*What each part means:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name `environment\_check` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to `databricks/notebooks/00\_environment\_check.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select No upstream task; this is the environment/start gate..
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*What should happen:\*\***  
A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**\*\*Expected example:\*\***  
> Your exact values may differ. The expected result is the same \*type\* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**\*\*What may look different:\*\***  
Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**\*\*How to verify success:\*\***  
Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**\*\*Do not continue unless:\*\***  
You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

**\*\*Common mistake:\*\***  
Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***  
Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#phase-14-lesson-index)

#### #### Run-history verification

1. Open the run from the **\*\*Start time\*\*** or matrix view.
2. Click ``environment_check``.
3. Record the run ID, task attempt, notebook path, parameters, start/end time, compute, and final state.
4. Open output and locate the entity/batch-specific validation summary.
5. If the task fails, copy the first meaningful exception and the deepest caused-by message, not only the red status.
6. Confirm dependent tasks are skipped when the failure is blocking.

#### #### Repair-run branch

- Open the failed run.
- Click **\*\*Repair run\*\***.
- Select the failed task and automatically dependent tasks shown by Databricks.
- Override parameters only when the correction requires it; record every override.
- Run the repair.
- Verify the repaired column/status in the matrix and inspect outputs again.
- Confirm tasks that already succeeded were not unnecessarily duplicated.

#### #### Common mistake

Selecting a similarly named notebook from a Drafts folder, omitting a parameter because a previous interactive run had a widget value, or using retries to hide a deterministic failure.

#### #### Interview-ready explanation

"I configured ``environment_check`` as an explicit Job node with declared dependencies, parameters, finite retry/timeout behavior, and output validation. I used repair runs for failed/dependent tasks and checked same-batch idempotency."

<a id="job-bronzeqlserver"></a>

### Databricks Job task 2 – ``bronze_sqlserver``

**\*\*Notebook:\*\*** ``databricks/notebooks/01_bronze_sqlserver.py``

**\*\*Depends on:\*\*** ``environment_check``

**\*\*Inputs:\*\*** Job parameters, upstream verified tables/files, and task outputs named in the task map.

**\*\*Output:\*\*** A validated table set, quality status, export, or demonstration result for the current Job run.

**\*\*Retry policy:\*\*** Retry only transient failures; deterministic data-quality failures require diagnosis rather than repeated compute cost.

**\*\*Timeout:\*\*** Set a finite threshold large enough for the fixture but short enough to surface a stuck task.

**\*\*Idempotency expectation:\*\*** Repairing or rerunning the task for the same batch must not create unintended duplicates.

<a id="action-14-4-j2"></a>

## Action 14.4.J2 – Create and validate the ``bronze_sqlserver`` Lakeflow Job task

**\*\*Application:\*\***

Databricks workspace in a web browser

**\*\*Window or panel:\*\***

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**\*\*Starting condition:\*\***

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**\*\*Current folder or location:\*\***

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

**\*\*What you are about to do:\*\***

Create and validate the ``bronze_sqlserver`` Lakeflow Job task. This is one bounded construction or inspection task.

**\*\*Why this action comes now:\*\***

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**\*\*Exact physical actions:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name ``bronze_sqlserver`` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to ``databricks/notebooks/01_bronze_sqlserver.py`` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add ``batch_id``, ``catalog``, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select ``environment_check``.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.

12. PRESS: Run now only when all required dependencies and parameters are verified.  
 13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.  
 14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*Command or code:\*\***

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**\*\*Where to enter it:\*\***  
 Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**\*\*What each part means:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name ``bronze_sqlserver`` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to ``databricks/notebooks/01_bronze_sqlserver.py`` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add ``batch_id``, ``catalog``, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select ``environment_check``.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*What should happen:\*\***  
 A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**\*\*Expected example:\*\***

> Your exact values may differ. The expected result is the same *\*type\** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**\*\*What may look different:\*\***  
 Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**\*\*How to verify success:\*\***  
 Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**\*\*Do not continue unless:\*\***  
 You can point to the visible result and explain what it proves. Mark the action ``VERIFIED BY USER`` only after that check.

**\*\*Common mistake:\*\***  
 Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***  
 Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***  
 You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***  
 The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#phase-14-lesson-index)

#### Run-history verification

1. Open the run from the **\*\*Start time\*\*** or matrix view.
2. Click ``bronze_sqlserver``.
3. Record the run ID, task attempt, notebook path, parameters, start/end time, compute, and final state.
4. Open output and locate the entity/batch-specific validation summary.
5. If the task fails, copy the first meaningful exception and the deepest caused-by message, not only the red status.
6. Confirm dependent tasks are skipped when the failure is blocking.

#### Repair-run branch

- Open the failed run.
- Click **\*\*Repair run\*\***.
- Select the failed task and automatically dependent tasks shown by Databricks.
- Override parameters only when the correction requires it; record every override.
- Run the repair.
- Verify the repaired column/status in the matrix and inspect outputs again.
- Confirm tasks that already succeeded were not unnecessarily duplicated.

#### Common mistake

Selecting a similarly named notebook from a Drafts folder, omitting a parameter because a previous interactive run had a widget value, or using retries to hide a deterministic failure.

#### #### Interview-ready explanation

"I configured `bronze\_sqlserver` as an explicit Job node with declared dependencies, parameters, finite retry/timeout behavior, and output validation. I used repair runs for failed/dependent tasks and checked same-batch idempotency."

<a id="job-bronzepostgres"></a>

### Databricks Job task 3 – `bronze\_postgres`

**\*\*Notebook:\*\*** `databricks/notebooks/02\_bronze\_postgres.py`

**\*\*Depends on:\*\*** `environment\_check`

**\*\*Inputs:\*\*** Job parameters, upstream verified tables/files, and task outputs named in the task map.

**\*\*Output:\*\*** A validated table set, quality status, export, or demonstration result for the current Job run.

**\*\*Retry policy:\*\*** Retry only transient failures; deterministic data-quality failures require diagnosis rather than repeated compute cost.

**\*\*Timeout:\*\*** Set a finite threshold large enough for the fixture but short enough to surface a stuck task.

**\*\*Idempotency expectation:\*\*** Repairing or rerunning the task for the same batch must not create unintended duplicates.

<a id="action-14-4-j3"></a>

## Action 14.4.J3 – Create and validate the `bronze\_postgres` Lakeflow Job task

**\*\*Application:\*\***

Databricks workspace in a web browser

**\*\*Window or panel:\*\***

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**\*\*Starting condition:\*\***

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**\*\*Current folder or location:\*\***

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

**\*\*What you are about to do:\*\***

Create and validate the `bronze\_postgres` Lakeflow Job task. This is one bounded construction or inspection task.

**\*\*Why this action comes now:\*\***

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**\*\*Exact physical actions:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name `bronze\_postgres` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to `databricks/notebooks/02\_bronze\_postgres.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `environment\_check`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*Command or code:\*\***

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**\*\*Where to enter it:\*\***

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**\*\*What each part means:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name `bronze\_postgres` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to `databricks/notebooks/02\_bronze\_postgres.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `environment\_check`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*What should happen:\*\***

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**\*\*Expected example:\*\***



> Your exact values may differ. The expected result is the same *\*type\** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**\*\*What may look different:\*\***  
Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**\*\*How to verify success:\*\***  
Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**\*\*Do not continue unless:\*\***  
You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

**\*\*Common mistake:\*\***  
Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***  
Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***  
You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***  
The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#phase-14-lesson-index)

#### #### Run-history verification

1. Open the run from the **\*\*Start time\*\*** or matrix view.
2. Click `bronze\_postgres`.
3. Record the run ID, task attempt, notebook path, parameters, start/end time, compute, and final state.
4. Open output and locate the entity/batch-specific validation summary.
5. If the task fails, copy the first meaningful exception and the deepest caused-by message, not only the red status.
6. Confirm dependent tasks are skipped when the failure is blocking.

#### #### Repair-run branch

- Open the failed run.
- Click **\*\*Repair run\*\***.
- Select the failed task and automatically dependent tasks shown by Databricks.
- Override parameters only when the correction requires it; record every override.
- Run the repair.
- Verify the repaired column/status in the matrix and inspect outputs again.
- Confirm tasks that already succeeded were not unnecessarily duplicated.

#### #### Common mistake

Selecting a similarly named notebook from a Drafts folder, omitting a parameter because a previous interactive run had a widget value, or using retries to hide a deterministic failure.

#### #### Interview-ready explanation

"I configured `bronze\_postgres` as an explicit Job node with declared dependencies, parameters, finite retry/timeout behavior, and output validation. I used repair runs for failed/dependent tasks and checked same-batch idempotency."

```

Databricks Job task 4 – `bronze_files`

Notebook: `databricks/notebooks/03_bronze_files.py`
Depends on: `environment_check`
Inputs: Job parameters, upstream verified tables/files, and task outputs named in the task map.
Output: A validated table set, quality status, export, or demonstration result for the current Job run.
Retry policy: Retry only transient failures; deterministic data-quality failures require diagnosis rather than repeated compute cost.
Timeout: Set a finite threshold large enough for the fixture but short enough to surface a stuck task.
Idempotency expectation: Repairing or rerunning the task for the same batch must not create unintended duplicates.

Action 14.4.J4 – Create and validate the `bronze_files` Lakeflow Job task

Application:
Databricks workspace in a web browser

Window or panel:
Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:
The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:
```

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

**\*\*What you are about to do:\*\***

Create and validate the `bronze\_files` Lakeflow Job task. This is one bounded construction or inspection task.

**\*\*Why this action comes now:\*\***

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**\*\*Exact physical actions:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name `bronze\_files` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to `databricks/notebooks/03\_bronze\_files.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `environment\_check`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*Command or code:\*\***

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**\*\*Where to enter it:\*\***

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**\*\*What each part means:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name `bronze\_files` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to `databricks/notebooks/03\_bronze\_files.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `environment\_check`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*What should happen:\*\***

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**\*\*Expected example:\*\***

> Your exact values may differ. The expected result is the same *\*type\** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**\*\*What may look different:\*\***

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**\*\*How to verify success:\*\***

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**\*\*Do not continue unless:\*\***

You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

**\*\*Common mistake:\*\***

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#phase-14-lesson-index)

#### Run-history verification

1. Open the run from the **Start time** or matrix view.
2. Click `bronze_files``.
3. Record the run ID, task attempt, notebook path, parameters, start/end time, compute, and final state.
4. Open output and locate the entity/batch-specific validation summary.
5. If the task fails, copy the first meaningful exception and the deepest caused-by message, not only the red status.
6. Confirm dependent tasks are skipped when the failure is blocking.

#### #### Repair-run branch

- Open the failed run.
- Click **Repair run**.
- Select the failed task and automatically dependent tasks shown by Databricks.
- Override parameters only when the correction requires it; record every override.
- Run the repair.
- Verify the repaired column/status in the matrix and inspect outputs again.
- Confirm tasks that already succeeded were not unnecessarily duplicated.

#### #### Common mistake

Selecting a similarly named notebook from a Drafts folder, omitting a parameter because a previous interactive run had a widget value, or using retries to hide a deterministic failure.

#### #### Interview-ready explanation

"I configured `bronze_files`` as an explicit Job node with declared dependencies, parameters, finite retry/timeout behavior, and output validation. I used repair runs for failed/dependent tasks and checked same-batch idempotency."

<a id="job-silvercustomers"></a>

### Databricks Job task 5 – `silver_customers``

**Notebook:** `databricks/notebooks/04_silver_customers.py``

**Depends on:** `bronze_sqlserver``

**Inputs:** Job parameters, upstream verified tables/files, and task outputs named in the task map.

**Output:** A validated table set, quality status, export, or demonstration result for the current Job run.

**Retry policy:** Retry only transient failures; deterministic data-quality failures require diagnosis rather than repeated compute cost.

**Timeout:** Set a finite threshold large enough for the fixture but short enough to surface a stuck task.

**Idempotency expectation:** Repairing or rerunning the task for the same batch must not create unintended duplicates.

<a id="action-14-4-j5"></a>

## Action 14.4.J5 – Create and validate the `silver_customers`` Lakeflow Job task

**Application:**

Databricks workspace in a web browser

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

**What you are about to do:**

Create and validate the `silver_customers`` Lakeflow Job task. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. CLICK: In the Databricks left sidebar, select **Jobs & Pipelines**. A workspace may show **Workflows** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **Create Job** for the first task.
3. TYPE THIS: Enter task name `silver_customers`` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **Notebook** as the task type.
5. CLICK: Browse to `databricks/notebooks/04_silver_customers.py`` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch_id``, `catalog``, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `bronze_sqlserver``.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**\*\*What each part means:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name `silver\_customers` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to `databricks/notebooks/04\_silver\_customers.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `bronze\_sqlserver`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*What should happen:\*\***

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**\*\*Expected example:\*\***

> Your exact values may differ. The expected result is the same *\*type\** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**\*\*What may look different:\*\***

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**\*\*How to verify success:\*\***

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**\*\*Do not continue unless:\*\***

You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

**\*\*Common mistake:\*\***

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#phase-14-lesson-index)

#### #### Run-history verification

1. Open the run from the **\*\*Start time\*\*** or matrix view.
2. Click `silver\_customers`.
3. Record the run ID, task attempt, notebook path, parameters, start/end time, compute, and final state.
4. Open output and locate the entity/batch-specific validation summary.
5. If the task fails, copy the first meaningful exception and the deepest caused-by message, not only the red status.
6. Confirm dependent tasks are skipped when the failure is blocking.

#### #### Repair-run branch

- Open the failed run.
- Click **\*\*Repair run\*\***.
- Select the failed task and automatically dependent tasks shown by Databricks.
- Override parameters only when the correction requires it; record every override.
- Run the repair.
- Verify the repaired column/status in the matrix and inspect outputs again.
- Confirm tasks that already succeeded were not unnecessarily duplicated.

#### #### Common mistake

Selecting a similarly named notebook from a Drafts folder, omitting a parameter because a previous interactive run had a widget value, or using retries to hide a deterministic failure.

#### #### Interview-ready explanation

"I configured `silver\_customers` as an explicit Job node with declared dependencies, parameters, finite retry/timeout behavior, and output validation. I used repair runs for failed/dependent tasks and checked same-batch idempotency."

<a id="job-silverproducts"></a>

### Databricks Job task 6 – `silver\_products`

**\*\*Notebook:\*\*** `databricks/notebooks/05\_silver\_products.py`

**\*\*Depends on:\*\*** `bronze\_sqlserver`, `bronze\_files`

```

Inputs: Job parameters, upstream verified tables/files, and task outputs named in the task map.
Output: A validated table set, quality status, export, or demonstration result for the current Job run.
Retry policy: Retry only transient failures; deterministic data-quality failures require diagnosis rather than repeated compute cost.
Timeout: Set a finite threshold large enough for the fixture but short enough to surface a stuck task.
Idempotency expectation: Repairing or rerunning the task for the same batch must not create unintended duplicates.

Action 14.4.J6 – Create and validate the `silver_products` Lakeflow Job task

Application:
Databricks workspace in a web browser

Window or panel:
Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:
The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:
The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:
Create and validate the `silver_products` Lakeflow Job task. This is one bounded construction or inspection task.

Why this action comes now:
It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:
1. CLICK: In the Databricks left sidebar, select **Jobs & Pipelines**. A workspace may show **Workflows** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **Create Job** for the first task.
3. TYPE THIS: Enter task name `silver_products` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **Notebook** as the task type.
5. CLICK: Browse to `databricks/notebooks/05_silver_products.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `bronze_sqlserver`, `bronze_files`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

Command or code:
No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:
Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:
1. CLICK: In the Databricks left sidebar, select **Jobs & Pipelines**. A workspace may show **Workflows** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **Create Job** for the first task.
3. TYPE THIS: Enter task name `silver_products` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **Notebook** as the task type.
5. CLICK: Browse to `databricks/notebooks/05_silver_products.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `bronze_sqlserver`, `bronze_files`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

What should happen:
A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:
> Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:
Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:
Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

```

**\*\*Do not continue unless:\*\***  
You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

**\*\*Common mistake:\*\***  
Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***  
Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***  
You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***  
The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#phase-14-lesson-index)

#### #### Run-history verification

1. Open the run from the **\*\*Start time\*\*** or matrix view.
2. Click `silver\_products`.
3. Record the run ID, task attempt, notebook path, parameters, start/end time, compute, and final state.
4. Open output and locate the entity/batch-specific validation summary.
5. If the task fails, copy the first meaningful exception and the deepest caused-by message, not only the red status.
6. Confirm dependent tasks are skipped when the failure is blocking.

#### #### Repair-run branch

- Open the failed run.
- Click **\*\*Repair run\*\***.
- Select the failed task and automatically dependent tasks shown by Databricks.
- Override parameters only when the correction requires it; record every override.
- Run the repair.
- Verify the repaired column/status in the matrix and inspect outputs again.
- Confirm tasks that already succeeded were not unnecessarily duplicated.

#### #### Common mistake

Selecting a similarly named notebook from a Drafts folder, omitting a parameter because a previous interactive run had a widget value, or using retries to hide a deterministic failure.

#### #### Interview-ready explanation

"I configured `silver\_products` as an explicit Job node with declared dependencies, parameters, finite retry/timeout behavior, and output validation. I used repair runs for failed/dependent tasks and checked same-batch idempotency."

<a id="job-silversales"></a>  
**### Databricks Job task 7 – `silver\_sales`**

**\*\*Notebook:\*\*** `databricks/notebooks/06\_silver\_sales.py`  
**\*\*Depends on:\*\*** `bronze\_sqlserver`, `silver\_customers`, `silver\_products`  
**\*\*Inputs:\*\*** Job parameters, upstream verified tables/files, and task outputs named in the task map.  
**\*\*Output:\*\*** A validated table set, quality status, export, or demonstration result for the current Job run.  
**\*\*Retry policy:\*\*** Retry only transient failures; deterministic data-quality failures require diagnosis rather than repeated compute cost.  
**\*\*Timeout:\*\*** Set a finite threshold large enough for the fixture but short enough to surface a stuck task.  
**\*\*Idempotency expectation:\*\*** Repairing or rerunning the task for the same batch must not create unintended duplicates.

<a id="action-14-4-j7"></a>  
**## Action 14.4.J7 – Create and validate the `silver\_sales` Lakeflow Job task**

**\*\*Application:\*\***  
Databricks workspace in a web browser

**\*\*Window or panel:\*\***  
Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**\*\*Starting condition:\*\***  
The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**\*\*Current folder or location:\*\***  
The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

**\*\*What you are about to do:\*\***  
Create and validate the `silver\_sales` Lakeflow Job task. This is one bounded construction or inspection task.

**\*\*Why this action comes now:\*\***  
It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**\*\*Exact physical actions:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.

3. TYPE THIS: Enter task name `silver\_sales` exactly; avoid spaces when the project map uses underscores.

4. CLICK: Choose **Notebook** as the task type.

5. CLICK: Browse to `databricks/notebooks/06\_silver\_sales.py` in the verified workspace/Git folder.

6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.

7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.

8. CLICK: Under dependencies, select `bronze\_sqlserver`, `silver\_customers`, `silver\_products`.

9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.

10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.

11. CLICK: Save task.

12. PRESS: Run now only when all required dependencies and parameters are verified.

13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.

14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

- CLICK: In the Databricks left sidebar, select **Jobs & Pipelines**. A workspace may show **Workflows** in an alternate interface.
- CLICK: Open the Northstar Retail Job or select **Create Job** for the first task.
- TYPE THIS: Enter task name `silver\_sales` exactly; avoid spaces when the project map uses underscores.
- CLICK: Choose **Notebook** as the task type.
- CLICK: Browse to `databricks/notebooks/06\_silver\_sales.py` in the verified workspace/Git folder.
- CLICK: Select the compute option available to your workspace and appropriate to the notebook.
- TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
- CLICK: Under dependencies, select `bronze\_sqlserver`, `silver\_customers`, `silver\_products`.
- CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
- CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
- CLICK: Save task.
- PRESS: Run now only when all required dependencies and parameters are verified.
- LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
- DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

> Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#phase-14-lesson-index)

#### Run-history verification

- Open the run from the **Start time** or matrix view.
- Click `silver\_sales`.
- Record the run ID, task attempt, notebook path, parameters, start/end time, compute, and final state.
- Open output and locate the entity/batch-specific validation summary.
- If the task fails, copy the first meaningful exception and the deepest caused-by message, not only the red status.
- Confirm dependent tasks are skipped when the failure is blocking.

#### Repair-run branch

- Open the failed run.
- Click **Repair run**.
- Select the failed task and automatically dependent tasks shown by Databricks.

- Override parameters only when the correction requires it; record every override.
- Run the repair.
- Verify the repaired column/status in the matrix and inspect outputs again.
- Confirm tasks that already succeeded were not unnecessarily duplicated.

#### Common mistake

Selecting a similarly named notebook from a Drafts folder, omitting a parameter because a previous interactive run had a widget value, or using retries to hide a deterministic failure.

#### Interview-ready explanation

"I configured `silver\_sales` as an explicit Job node with declared dependencies, parameters, finite retry/timeout behavior, and output validation. I used repair runs for failed/dependent tasks and checked same-batch idempotency."

<a id="job-silverdigital"></a>

### Databricks Job task 8 – `silver\_digital`

**\*\*Notebook:\*\*** `databricks/notebooks/07\_silver\_digital.py`

**\*\*Depends on:\*\*** `bronze\_postgres`

**\*\*Inputs:\*\*** Job parameters, upstream verified tables/files, and task outputs named in the task map.

**\*\*Output:\*\*** A validated table set, quality status, export, or demonstration result for the current Job run.

**\*\*Retry policy:\*\*** Retry only transient failures; deterministic data-quality failures require diagnosis rather than repeated compute cost.

**\*\*Timeout:\*\*** Set a finite threshold large enough for the fixture but short enough to surface a stuck task.

**\*\*Idempotency expectation:\*\*** Repairing or rerunning the task for the same batch must not create unintended duplicates.

<a id="action-14-4-j8"></a>

## Action 14.4.J8 – Create and validate the `silver\_digital` Lakeflow Job task

**\*\*Application:\*\***

Databricks workspace in a web browser

**\*\*Window or panel:\*\***

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**\*\*Starting condition:\*\***

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**\*\*Current folder or location:\*\***

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

**\*\*What you are about to do:\*\***

Create and validate the `silver\_digital` Lakeflow Job task. This is one bounded construction or inspection task.

**\*\*Why this action comes now:\*\***

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**\*\*Exact physical actions:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name `silver\_digital` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to `databricks/notebooks/07\_silver\_digital.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `bronze\_postgres`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*Command or code:\*\***

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**\*\*Where to enter it:\*\***

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**\*\*What each part means:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name `silver\_digital` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to `databricks/notebooks/07\_silver\_digital.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `bronze\_postgres`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.



12. PRESS: Run now only when all required dependencies and parameters are verified.

13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.

14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*What should happen:\*\***  
A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**\*\*Expected example:\*\***

> Your exact values may differ. The expected result is the same *\*type\** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**\*\*What may look different:\*\***  
Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**\*\*How to verify success:\*\***  
Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**\*\*Do not continue unless:\*\***  
You can point to the visible result and explain what it proves. Mark the action ``VERIFIED BY USER`` only after that check.

**\*\*Common mistake:\*\***  
Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***  
Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***  
You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***  
The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#phase-14-lesson-index)

#### Run-history verification

1. Open the run from the **\*\*Start time\*\*** or matrix view.
2. Click ``silver_digital``.
3. Record the run ID, task attempt, notebook path, parameters, start/end time, compute, and final state.
4. Open output and locate the entity/batch-specific validation summary.
5. If the task fails, copy the first meaningful exception and the deepest caused-by message, not only the red status.
6. Confirm dependent tasks are skipped when the failure is blocking.

#### Repair-run branch

- Open the failed run.
- Click **\*\*Repair run\*\***.
- Select the failed task and automatically dependent tasks shown by Databricks.
- Override parameters only when the correction requires it; record every override.
- Run the repair.
- Verify the repaired column/status in the matrix and inspect outputs again.
- Confirm tasks that already succeeded were not unnecessarily duplicated.

#### Common mistake

Selecting a similarly named notebook from a Drafts folder, omitting a parameter because a previous interactive run had a widget value, or using retries to hide a deterministic failure.

#### Interview-ready explanation

"I configured ``silver_digital`` as an explicit Job node with declared dependencies, parameters, finite retry/timeout behavior, and output validation. I used repair runs for failed/dependent tasks and checked same-batch idempotency."

<a id="job-golddimensions"></a>  
 #### Databricks Job task 9 – ``gold_dimensions``

**\*\*Notebook:\*\*** ``databricks/notebooks/08_gold_dimensions.py``  
**\*\*Depends on:\*\*** ``silver_customers``, ``silver_products``, ``silver_digital``, ``bronze_files``  
**\*\*Inputs:\*\*** Job parameters, upstream verified tables/files, and task outputs named in the task map.  
**\*\*Output:\*\*** A validated table set, quality status, export, or demonstration result for the current Job run.  
**\*\*Retry policy:\*\*** Retry only transient failures; deterministic data-quality failures require diagnosis rather than repeated compute cost.  
**\*\*Timeout:\*\*** Set a finite threshold large enough for the fixture but short enough to surface a stuck task.  
**\*\*Idempotency expectation:\*\*** Repairing or rerunning the task for the same batch must not create unintended duplicates.

<a id="action-14-4-j9"></a>  
 ## Action 14.4.J9 – Create and validate the ``gold_dimensions`` Lakeflow Job task

**\*\*Application:\*\***  
 Databricks workspace in a web browser

**\*\*Window or panel:\*\***

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**\*\*Starting condition:\*\***

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**\*\*Current folder or location:\*\***

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

**\*\*What you are about to do:\*\***

Create and validate the `gold\_dimensions` Lakeflow Job task. This is one bounded construction or inspection task.

**\*\*Why this action comes now:\*\***

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**\*\*Exact physical actions:\*\***

1. CLICK: In the Databricks left sidebar, select **Jobs & Pipelines**. A workspace may show **Workflows** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **Create Job** for the first task.
3. TYPE THIS: Enter task name `gold\_dimensions` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **Notebook** as the task type.
5. CLICK: Browse to `databricks/notebooks/08\_gold\_dimensions.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `silver\_customers`, `silver\_products`, `silver\_digital`, `bronze\_files`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*Command or code:\*\***

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**\*\*Where to enter it:\*\***

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**\*\*What each part means:\*\***

1. CLICK: In the Databricks left sidebar, select **Jobs & Pipelines**. A workspace may show **Workflows** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **Create Job** for the first task.
3. TYPE THIS: Enter task name `gold\_dimensions` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **Notebook** as the task type.
5. CLICK: Browse to `databricks/notebooks/08\_gold\_dimensions.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `silver\_customers`, `silver\_products`, `silver\_digital`, `bronze\_files`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*What should happen:\*\***

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**\*\*Expected example:\*\***

> Your exact values may differ. The expected result is the same *\*type\** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**\*\*What may look different:\*\***

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**\*\*How to verify success:\*\***

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**\*\*Do not continue unless:\*\***

You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

**\*\*Common mistake:\*\***

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#phase-14-lesson-index)

#### #### Run-history verification

1. Open the run from the **\*\*Start time\*\*** or matrix view.
2. Click ``gold_dimensions``.
3. Record the run ID, task attempt, notebook path, parameters, start/end time, compute, and final state.
4. Open output and locate the entity/batch-specific validation summary.
5. If the task fails, copy the first meaningful exception and the deepest caused-by message, not only the red status.
6. Confirm dependent tasks are skipped when the failure is blocking.

#### #### Repair-run branch

- Open the failed run.
- Click **\*\*Repair run\*\***.
- Select the failed task and automatically dependent tasks shown by Databricks.
- Override parameters only when the correction requires it; record every override.
- Run the repair.
- Verify the repaired column/status in the matrix and inspect outputs again.
- Confirm tasks that already succeeded were not unnecessarily duplicated.

#### #### Common mistake

Selecting a similarly named notebook from a Drafts folder, omitting a parameter because a previous interactive run had a widget value, or using retries to hide a deterministic failure.

#### #### Interview-ready explanation

"I configured ``gold_dimensions`` as an explicit Job node with declared dependencies, parameters, finite retry/timeout behavior, and output validation. I used repair runs for failed/dependent tasks and checked same-batch idempotency."

<a id="job-goldfacts"></a>

## Databricks Job task 10 – ``gold_facts``

**\*\*Notebook:\*\*** ``databricks/notebooks/09_gold_facts.py``  
**\*\*Depends on:\*\*** ``gold_dimensions``, ``silver_sales``, ``silver_digital``  
**\*\*Inputs:\*\*** Job parameters, upstream verified tables/files, and task outputs named in the task map.  
**\*\*Output:\*\*** A validated table set, quality status, export, or demonstration result for the current Job run.  
**\*\*Retry policy:\*\*** Retry only transient failures; deterministic data-quality failures require diagnosis rather than repeated compute cost.  
**\*\*Timeout:\*\*** Set a finite threshold large enough for the fixture but short enough to surface a stuck task.  
**\*\*Idempotency expectation:\*\*** Repairing or rerunning the task for the same batch must not create unintended duplicates.

<a id="action-14-4-j10"></a>

## Action 14.4.J10 – Create and validate the ``gold_facts`` Lakeflow Job task

**\*\*Application:\*\***

Databricks workspace in a web browser

**\*\*Window or panel:\*\***

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**\*\*Starting condition:\*\***

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**\*\*Current folder or location:\*\***

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

**\*\*What you are about to do:\*\***

Create and validate the ``gold_facts`` Lakeflow Job task. This is one bounded construction or inspection task.

**\*\*Why this action comes now:\*\***

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**\*\*Exact physical actions:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name ``gold_facts`` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to ``databricks/notebooks/09_gold_facts.py`` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add ``batch_id``, ``catalog``, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select ``gold_dimensions``, ``silver_sales``, ``silver_digital``.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*Command or code:\*\***

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**\*\*Where to enter it:\*\***

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**\*\*What each part means:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name ``gold_facts`` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to ``databricks/notebooks/09_gold_facts.py`` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add ``batch_id``, ``catalog``, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select ``gold_dimensions``, ``silver_sales``, ``silver_digital``.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*What should happen:\*\***

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**\*\*Expected example:\*\***

> Your exact values may differ. The expected result is the same *\*type\** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**\*\*What may look different:\*\***

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**\*\*How to verify success:\*\***

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**\*\*Do not continue unless:\*\***

You can point to the visible result and explain what it proves. Mark the action ``VERIFIED BY USER`` only after that check.

**\*\*Common mistake:\*\***

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#phase-14-lesson-index)

**#### Run-history verification**

1. Open the run from the **\*\*Start time\*\*** or matrix view.
2. Click ``gold_facts``.
3. Record the run ID, task attempt, notebook path, parameters, start/end time, compute, and final state.
4. Open output and locate the entity/batch-specific validation summary.
5. If the task fails, copy the first meaningful exception and the deepest caused-by message, not only the red status.
6. Confirm dependent tasks are skipped when the failure is blocking.

**#### Repair-run branch**

- Open the failed run.
- Click **\*\*Repair run\*\***.
- Select the failed task and automatically dependent tasks shown by Databricks.
- Override parameters only when the correction requires it; record every override.
- Run the repair.
- Verify the repaired column/status in the matrix and inspect outputs again.
- Confirm tasks that already succeeded were not unnecessarily duplicated.

**#### Common mistake**

Selecting a similarly named notebook from a Drafts folder, omitting a parameter because a previous interactive run had a widget value, or using retries to hide a deterministic failure.

**#### Interview-ready explanation**

"I configured `gold\_facts` as an explicit Job node with declared dependencies, parameters, finite retry/timeout behavior, and output validation. I used repair runs for failed/dependent tasks and checked same-batch idempotency."

<a id="job-dataquality"></a>

### Databricks Job task 11 – `data\_quality`

**\*\*Notebook:\*\*** `databricks/notebooks/10\_data\_quality.py`

**\*\*Depends on:\*\*** `gold\_facts`

**\*\*Inputs:\*\*** Job parameters, upstream verified tables/files, and task outputs named in the task map.

**\*\*Output:\*\*** A validated table set, quality status, export, or demonstration result for the current Job run.

**\*\*Retry policy:\*\*** Retry only transient failures; deterministic data-quality failures require diagnosis rather than repeated compute cost.

**\*\*Timeout:\*\*** Set a finite threshold large enough for the fixture but short enough to surface a stuck task.

**\*\*Idempotency expectation:\*\*** Repairing or rerunning the task for the same batch must not create unintended duplicates.

<a id="action-14-4-j11"></a>

## Action 14.4.J11 – Create and validate the `data\_quality` Lakeflow Job task

**\*\*Application:\*\***

Databricks workspace in a web browser

**\*\*Window or panel:\*\***

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**\*\*Starting condition:\*\***

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**\*\*Current folder or location:\*\***

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

**\*\*What you are about to do:\*\***

Create and validate the `data\_quality` Lakeflow Job task. This is one bounded construction or inspection task.

**\*\*Why this action comes now:\*\***

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**\*\*Exact physical actions:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name `data\_quality` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to `databricks/notebooks/10\_data\_quality.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `gold\_facts`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*Command or code:\*\***

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**\*\*Where to enter it:\*\***

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**\*\*What each part means:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name `data\_quality` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to `databricks/notebooks/10\_data\_quality.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `gold\_facts`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*What should happen:\*\***

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**\*\*Expected example:\*\***

> Your exact values may differ. The expected result is the same *\*type\** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**\*\*What may look different:\*\***

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**\*\*How to verify success:\*\***

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**\*\*Do not continue unless:\*\***

You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

**\*\*Common mistake:\*\***

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#phase-14-lesson-index)

#### #### Run-history verification

1. Open the run from the **\*\*Start time\*\*** or matrix view.
2. Click `data\_quality`.
3. Record the run ID, task attempt, notebook path, parameters, start/end time, compute, and final state.
4. Open output and locate the entity/batch-specific validation summary.
5. If the task fails, copy the first meaningful exception and the deepest caused-by message, not only the red status.
6. Confirm dependent tasks are skipped when the failure is blocking.

#### #### Repair-run branch

- Open the failed run.
- Click **\*\*Repair run\*\***.
- Select the failed task and automatically dependent tasks shown by Databricks.
- Override parameters only when the correction requires it; record every override.
- Run the repair.
- Verify the repaired column/status in the matrix and inspect outputs again.
- Confirm tasks that already succeeded were not unnecessarily duplicated.

#### #### Common mistake

Selecting a similarly named notebook from a Drafts folder, omitting a parameter because a previous interactive run had a widget value, or using retries to hide a deterministic failure.

#### #### Interview-ready explanation

"I configured `data\_quality` as an explicit Job node with declared dependencies, parameters, finite retry/timeout behavior, and output validation. I used repair runs for failed/dependent tasks and checked same-batch idempotency."

<a id="job-publishgold"></a>

### Databricks Job task 12 – `publish\_gold`

**\*\*Notebook:\*\*** `databricks/notebooks/11\_publish\_gold.py`

**\*\*Depends on:\*\*** `data\_quality`

**\*\*Inputs:\*\*** Job parameters, upstream verified tables/files, and task outputs named in the task map.

**\*\*Output:\*\*** A validated table set, quality status, export, or demonstration result for the current Job run.

**\*\*Retry policy:\*\*** Retry only transient failures; deterministic data-quality failures require diagnosis rather than repeated compute cost.

**\*\*Timeout:\*\*** Set a finite threshold large enough for the fixture but short enough to surface a stuck task.

**\*\*Idempotency expectation:\*\*** Repairing or rerunning the task for the same batch must not create unintended duplicates.

<a id="action-14-4-j12"></a>

## Action 14.4.J12 – Create and validate the `publish\_gold` Lakeflow Job task

**\*\*Application:\*\***

Databricks workspace in a web browser

**\*\*Window or panel:\*\***

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**\*\*Starting condition:\*\***

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**\*\*Current folder or location:\*\***

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

**\*\*What you are about to do:\*\***

Create and validate the `publish\_gold` Lakeflow Job task. This is one bounded construction or inspection task.

**\*\*Why this action comes now:\*\***

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**\*\*Exact physical actions:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name `publish\_gold` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to `databricks/notebooks/11\_publish\_gold.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `data\_quality`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*Command or code:\*\***

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**\*\*Where to enter it:\*\***

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**\*\*What each part means:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name `publish\_gold` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to `databricks/notebooks/11\_publish\_gold.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `data\_quality`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*What should happen:\*\***

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**\*\*Expected example:\*\***

> Your exact values may differ. The expected result is the same *\*type\** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**\*\*What may look different:\*\***

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**\*\*How to verify success:\*\***

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**\*\*Do not continue unless:\*\***

You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

**\*\*Common mistake:\*\***

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#phase-14-lesson-index)

#### Run-history verification

1. Open the run from the **\*\*Start time\*\*** or matrix view.
2. Click `publish\_gold`.
3. Record the run ID, task attempt, notebook path, parameters, start/end time, compute, and final state.

4. Open output and locate the entity/batch-specific validation summary.
5. If the task fails, copy the first meaningful exception and the deepest caused-by message, not only the red status.
6. Confirm dependent tasks are skipped when the failure is blocking.

#### #### Repair-run branch

- Open the failed run.
- Click **\*\*Repair run\*\***.
- Select the failed task and automatically dependent tasks shown by Databricks.
- Override parameters only when the correction requires it; record every override.
- Run the repair.
- Verify the repaired column/status in the matrix and inspect outputs again.
- Confirm tasks that already succeeded were not unnecessarily duplicated.

#### #### Common mistake

Selecting a similarly named notebook from a Drafts folder, omitting a parameter because a previous interactive run had a widget value, or using retries to hide a deterministic failure.

#### #### Interview-ready explanation

"I configured `publish\_gold` as an explicit Job node with declared dependencies, parameters, finite retry/timeout behavior, and output validation. I used repair runs for failed/dependent tasks and checked same-batch idempotency."

<a id="job-demoqueries"></a>

### Databricks Job task 13 – `demo\_queries`

**\*\*Notebook:\*\*** `databricks/notebooks/12\_demo\_queries.py`

**\*\*Depends on:\*\*** `publish\_gold`

**\*\*Inputs:\*\*** Job parameters, upstream verified tables/files, and task outputs named in the task map.

**\*\*Output:\*\*** A validated table set, quality status, export, or demonstration result for the current Job run.

**\*\*Retry policy:\*\*** Retry only transient failures; deterministic data-quality failures require diagnosis rather than repeated compute cost.

**\*\*Timeout:\*\*** Set a finite threshold large enough for the fixture but short enough to surface a stuck task.

**\*\*Idempotency expectation:\*\*** Repairing or rerunning the task for the same batch must not create unintended duplicates.

<a id="action-14-4-j13"></a>

## Action 14.4.J13 – Create and validate the `demo\_queries` Lakeflow Job task

**\*\*Application:\*\***

Databricks workspace in a web browser

**\*\*Window or panel:\*\***

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**\*\*Starting condition:\*\***

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**\*\*Current folder or location:\*\***

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

**\*\*What you are about to do:\*\***

Create and validate the `demo\_queries` Lakeflow Job task. This is one bounded construction or inspection task.

**\*\*Why this action comes now:\*\***

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**\*\*Exact physical actions:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.
3. TYPE THIS: Enter task name `demo\_queries` exactly; avoid spaces when the project map uses underscores.
4. CLICK: Choose **\*\*Notebook\*\*** as the task type.
5. CLICK: Browse to `databricks/notebooks/12\_demo\_queries.py` in the verified workspace/Git folder.
6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.
7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.
8. CLICK: Under dependencies, select `publish\_gold`.
9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.
10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.
11. CLICK: Save task.
12. PRESS: Run now only when all required dependencies and parameters are verified.
13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.
14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**\*\*Command or code:\*\***

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**\*\*Where to enter it:\*\***

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**\*\*What each part means:\*\***

1. CLICK: In the Databricks left sidebar, select **\*\*Jobs & Pipelines\*\***. A workspace may show **\*\*Workflows\*\*** in an alternate interface.
2. CLICK: Open the Northstar Retail Job or select **\*\*Create Job\*\*** for the first task.



3. TYPE THIS: Enter task name `demo\_queries` exactly; avoid spaces when the project map uses underscores.

4. CLICK: Choose **Notebook** as the task type.

5. CLICK: Browse to `databricks/notebooks/12\_demo\_queries.py` in the verified workspace/Git folder.

6. CLICK: Select the compute option available to your workspace and appropriate to the notebook.

7. TYPE THIS: Add `batch\_id`, `catalog`, and other parameters exactly as the notebook reads them.

8. CLICK: Under dependencies, select `publish\_gold`.

9. CLICK: Open advanced settings and set retries/timeouts from the documented policy; do not choose unlimited retries.

10. CLICK: Add failure notification only to a channel/address you are authorized to use, or record that notifications are not configured.

11. CLICK: Save task.

12. PRESS: Run now only when all required dependencies and parameters are verified.

13. LOOK FOR: Open task output and confirm the exact notebook, parameter values, compute, and result.

14. DO NOT CONTINUE UNTIL: The task's output and downstream dependency behavior match the Job map.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

> Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 14 lesson index](#phase-14-lesson-index)

#### Run-history verification

1. Open the run from the **Start time** or matrix view.
2. Click `demo\_queries`.
3. Record the run ID, task attempt, notebook path, parameters, start/end time, compute, and final state.
4. Open output and locate the entity/batch-specific validation summary.
5. If the task fails, copy the first meaningful exception and the deepest caused-by message, not only the red status.
6. Confirm dependent tasks are skipped when the failure is blocking.

#### Repair-run branch

- Open the failed run.
- Click **Repair run**.
- Select the failed task and automatically dependent tasks shown by Databricks.
- Override parameters only when the correction requires it; record every override.
- Run the repair.
- Verify the repaired column/status in the matrix and inspect outputs again.
- Confirm tasks that already succeeded were not unnecessarily duplicated.

#### Common mistake

Selecting a similarly named notebook from a Drafts folder, omitting a parameter because a previous interactive run had a widget value, or using retries to hide a deterministic failure.

#### Interview-ready explanation

"I configured `demo\_queries` as an explicit Job node with declared dependencies, parameters, finite retry/timeout behavior, and output validation. I used repair runs for failed/dependent tasks and checked same-batch idempotency."

## What your screen should look like now

- The new Phase 14 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

## ## Lesson 14.4 completion checklist

- [ ] Every required object was built separately – [STATUS]
- [ ] Every object has a declared responsibility/grain – [STATUS]
- [ ] Inputs and outputs were inspected – [STATUS]
- [ ] A rerun or alternate-result branch was tested where applicable – [STATUS]
- [ ] Counts, keys, measures, relationships, or settings were independently validated – [STATUS]
- [ ] I can explain one failure and its safest recovery – [STATUS]

## ## Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 14.5](#phase-14-lesson-5).

<a id="phase-14-lesson-5"></a>

## ## Lesson 14.5 – Verify, troubleshoot, commit, teach back, and stop safely

### ## What your screen should look like before starting

The Phase 14 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

### ## Design decision – Verification before automation

**\*\*Question:\*\***

Should the phase prioritize a visible manual verification or jump directly to automation?

**\*\*Options considered:\*\***

- Manual verification first, then automation.
- Automation without a known-good baseline.
- Manual operation forever.

**\*\*Beginner path selected:\*\***

Establish one verified manual baseline and then automate repeatable checks.

**\*\*Why this path was selected:\*\***

A known-good example makes failures interpretable and prevents automation from hiding a wrong assumption.

**\*\*Tradeoff:\*\***

The first run takes longer.

**\*\*What a company might do instead:\*\***

Teams often pair infrastructure as code and automated tests with controlled bootstrap and acceptance procedures.

**\*\*How to explain this in an interview:\*\***

I established a verified baseline, then encoded the same contract in tests and orchestration so automation preserved meaning rather than just speed.

### ## If you see Result A

The Job DAG completes and the run history shows successful dependent tasks. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

### ## If you see Result B

A task fails or is skipped; repair targets only the failed/downstream tasks after the cause is fixed. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

### ## If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

### ## Phase-specific verification gate

- [ ] Required files/objects exist in the exact location.
- [ ] I personally created or configured them.
- [ ] I ran the phase commands in the documented context.
- [ ] I inspected the output rather than relying on a green icon alone.
- [ ] Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.
- [ ] Same-batch/rerun behavior is known where applicable.
- [ ] A controlled failure or realistic alternate-result branch was tested safely where applicable.
- [ ] No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.
- [ ] Actual verified result: `[ENTER VERIFIED RESULT]`.

### ## Point-of-failure troubleshooting record

**\*\*Exact or representative error message:\*\***

```
`[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]`

What it means in plain language:
`[WRITE YOUR INTERPRETATION]`

Most likely cause:
`[WRITE ONE TESTABLE HYPOTHESIS]`

What not to do:
Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:
`[ENTER THE SMALLEST OBSERVATIONAL CHECK]`

How to read the diagnostic result:
`[RESULT A MEANS... / RESULT B MEANS...]`

Safest fix:
`[ENTER THE NARROW FIX]`

Alternative fix when applicable:
`[ENTER OR N/A]`

How to verify the fix:
`[ENTER THE INDEPENDENT RECHECK]`

How to return to the main tutorial:
Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: Databricks Jobs & Pipelines, notebook task pages, run history, and notifications settings.
Window/panel: Show Job DAG, run matrix, failed task, skipped dependency, and repair-run success.
Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.
Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.
Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.
Recommended filename: `docs/images/phase14_create-and-operate-a-multi-task-databricks-job_verified.png`.
Recommended repository folder: `docs/images/`.
README placement: The section describing Phase 14's verified result; do not use the image before the result is actually verified.
What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

`[ADD YOUR VERIFIED SCREENSHOT HERE]`

Git checkpoint after the verified Phase 14 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 14.5.G1 – Verify the repository root before Git operations

Application:
Windows PowerShell or the VS Code PowerShell terminal

Window or panel:
VS Code integrated PowerShell terminal

Starting condition:
The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:
The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:
Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:
The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches `The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.` or the more specific location stated below.
```

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.  
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.  
5. PRESS: Press **\*\*Enter\*\*** for a one-line terminal command, **\*\*F5/Execute\*\*** for the named database editor, or the notebook cell Run control as stated.  
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.  
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**\*\*Command or code (PowerShell):\*\***

```
```powershell  
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

Action 14.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

Action 14.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.

7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

Interpret `git status --short`

- `??` means untracked: Git sees a new path but it is not staged.
- `M` means modified in the working tree but not staged.
- `M` means the staged version differs from the last commit.

- MM means a staged version exists and the file has additional unstaged edits.
- A means staged as a new file.
- D means deletion; stop and investigate unless deletion is deliberate.
- R means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 14.5.G4 — Inspect the unstaged diff for databricks/jobs/job_definition.example.yml

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "databricks/jobs/job_definition.example.yml"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "databricks/jobs/job_definition.example.yml"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"databricks/jobs/job_definition.example.yml"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

Action 14.5.G5 — Stage only the reviewed file `databricks/jobs/job_definition.example.yml`

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.

2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "databricks/jobs/job_definition.example.yml"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "databricks/jobs/job_definition.example.yml"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"databricks/jobs/job_definition.example.yml"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 14.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

Action 14.5.G7 — Create the Phase 14 commit

Application:

Databricks workspace in a web browser

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "feat: orchestrate northstar lakehouse job"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "feat: orchestrate northstar lakehouse job"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"feat: orchestrate northstar lakehouse job"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

Action 14.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

Action 14.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The selected Databricks workspace, catalog, schema, volume, notebook, or Job named in the action. or the more specific location stated below.

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 14 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message `feat: orchestrate northstar lakehouse job` and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 14.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 14 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

“Phase 14 created the verified boundary described by **Create and operate a multi-task Databricks Job**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed.”

Mini quiz

1. Multiple choice: Why does Phase 14 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
4. What command would you use first to confirm the PowerShell current folder before a repository command?

5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** PLANNED, CREATED BY USER, and RUN BY USER describe earlier states; only inspected evidence supports verification.

My Phase 14 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.

- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 14.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 15 exports only verified Gold results and loads them into the local warehouse.

Phase 14 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 15 — Publish Gold data into the SQL Server dimensional warehouse

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: A validated SQL Server dimensional warehouse.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 16 imports trusted tables, builds relationships, and defines DAX measures.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 15 lesson index

1. [Lesson 15.1 — Understand the phase and starting screen](#)
2. [Lesson 15.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 15.3 — Create and understand the phase files](#)
4. [Lesson 15.4 — Build and validate required objects](#)
5. [Lesson 15.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 15.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** Databricks, File Explorer, PowerShell, SSMS, and Python.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 15 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 14 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 15 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

Databricks, File Explorer, PowerShell, SSMS, and Python. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 15 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 15 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 15 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 15.1 completion checklist

☐

I read why Phase 15 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 15.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 15 lesson index](#)

Lesson 15.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 15 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

A local SQL Server warehouse named NorthstarRetail_DW with dw, stg, audit, and semantic schemas, dimension/fact DDL, unknown rows, constraints/indexes, staging and transactional publication, load audit/reconciliation, semantic views, and exact SSMS inspection steps. Track A uses Gold export/download; Track B direct publication remains optional.

2. Why it matters

Gold Delta is business-ready, but a SQL Server serving mirror teaches warehouse DDL, transactional stage-and-publish, relational constraints/indexes, local Power BI connectivity, and cross-platform reconciliation. The manual download preserves the honest localhost boundary.

3. Words to know

Term	Plain-language definition
Serving warehouse	Curated database optimized for analyst/BI consumption.
Staging schema	Temporary load tables before target publish.
Transactional publication	Stage/validate/delete-merge/insert/audit commit as one unit.
Full development reload	Replace target scope safely in disposable development.
Incremental publication	Apply new/changed rows while preserving history.
Audit table	Records batch process status/timing/errors.
Reconciliation table	Stores source/target values/differences/status.
Clustered/nonclustered index	SQL Server physical/access structures; design depends on workload.
Primary/foreign key	Relational integrity constraints in serving layer.
Semantic view	Stable SQL view exposing business-friendly columns.
Parquet export	Columnar Gold files downloaded from volume.
Direct publication	Track B write over reachable network/JDBC/connector, requiring security/network support.
Rollback	Transaction reverts target changes after failure.

4. What must already be completed

Job quality gate passes; all Gold tables exist; publish notebook exports manifest/path; local SQL Server/SSMS/ODBC works; downloaded files are in one known folder; disk space available.

5. Applications to open

Application	How to open it	Why it is open
Databricks Job/notebook 11	Workspace.	Export Gold after quality gate.
Databricks volume browser	Gold export folder.	Download exact batch export/manifest.
Windows PowerShell	Repository root with .venv active.	Create DB through SSMS then run local loader.

Application	How to open it	Why it is open
SSMS	Local server.	Run warehouse DDL, inspect/stage/validate.
VS Code	Warehouse DDL, loader, validation, semantic views.	Review transaction logic before execution.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run Get-Location; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
SELECT @@SERVERNAME AS server_name, SUSER_SNAME() AS login_name, DB_NAME() AS database_name;	Verifies server/login/context.	ERP and DW have similar names; context mistakes are dangerous.
-- Execute the entire 01_create_warehouse.sql file.	Creates DW schemas/tables/constraints/indexes/ unknown rows.	Loader needs target contract.
-- Execute 02_create_staging_tables.sql, then 03_semantic_views.sql.	Creates staging mirrors and business-friendly views.	Publication isolates incoming files and Power BI can use stable semantic access.
python -m scripts.loading.load_gold_to_sqlserver --input-dir "C: \\Users\\User\\Downloads\\gold_exports\\EXT- YYYYMMDDTHMMSSZ" --mode full-dev	Stages and transactionally publishes all Gold tables into the local DW.	This is Track A's controlled local serving handoff.
-- Execute sql/sqlserver/validation/02_valid- ate_warehouse.sql	Checks duplicate fact grain, dimension RI, SCD current rows, revenue identity, audit/reconcili- ation.	Loader success does not prove model correct- ness.
SELECT TOP (20) * FROM semant- ic.vw_executive_daily_kpi ORDER BY full_date DESC;	Queries serving semantic view.	Confirms views compile and aggregate loaded warehouse.

Safe full-dev order

Stage every table first. Validate stage counts/schema. Delete fact tables before dimensions to satisfy foreign keys. Insert dimensions in dependency order, then facts. Recreate/retain unknown rows consistently. Record audit/reconciliation and commit once. On exception, roll back.

Incremental tradeoff

An incremental warehouse publish must MERGE Type 1 dimensions, append/expire SCD2 versions correctly, upsert facts by immutable grain, handle deletes/corrections, and reconcile the batch. Full-dev is simpler and transparent for small learning data; it is not a production claim.

MY TASK

Review/fix the local loader for fact-before-dimension deletion and explicit target columns, publish one validated batch, prove rollback with a safe staged failure, reconcile Gold to DW, and verify a same-batch reload.

STARTER SKELETON

```
# TODO(USER): safe full-dev publication outline
with engine.begin() as connection:
    stage_all_exports(connection, input_dir)
    validate_stage_counts(connection, manifest)
    for fact in reversed(FACT_TABLES):
        connection.execute(text(f"DELETE FROM dw.{fact}"))
    for dimension in reversed(DIMENSION_TABLES):
        connection.execute(text(f"DELETE FROM dw.{dimension} WHERE {dimension_key(dimension)} <> 0"))
    for dimension in DIMENSION_TABLES:
        insert_explicit_columns(connection, dimension)
    for fact in FACT_TABLES:
        insert_explicit_columns(connection, fact)
    record_reconciliation(connection, batch_id)
# TODO(USER): test a forced error rolls back target counts.
```

Acceptance criteria: Warehouse objects exist; publication is transactional and FK-safe; all Gold/SQL Server counts/amounts reconcile; invalid staged run rolls back; same-batch full-dev reload reproduces results; semantic views query successfully.

HINT 1 — CONCEPT

Delete child facts before parent dimensions; insert parents before children.

HINT 2 — PSEUDOCODE

Ask SQL Server `sys.columns` for insertable non-computed columns and use explicit lists.

HINT 3 — NEAR-COMPLETE PATTERN

Stage all tables and validate first; adapt skeleton inside one `engine.begin()` transaction.

13. Expected files or output

- NorthstarRetail_DW and four schemas.
- 9 dimensions, 6 facts, audit/reconciliation, staging tables, semantic views.
- SUCCESS audit/run artifact.
- Gold-to-DW reconciliation.
- Rollback and rerun evidence.

14. How to verify

- All target counts equal export manifest.
- No fact duplicates/orphan keys.
- SCD2 current/unknown rows valid.
- Revenue/profit match Gold.
- Forced failure leaves pre-run target state.
- Semantic view values match base query.
- ERP source unaffected.

15. Automated tests

- Warehouse validation SQL.
- Gold-to-DW count/amount checks.
- Transaction rollback test.
- Same-batch reproducibility.
- Constraint/index inspection.
- Semantic view query and KPI validation.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
FK delete conflict	Dimensions deleted before facts.	Read constraint/table error; inspect deletion order.	Delete child facts first inside transaction.	Reload succeeds.
Computed column insert error	SELECT * includes computed/non-insertable target column.	Inspect <code>sys.columns.is_computed</code> and stage schema.	Use explicit insertable column list.	fact_inventory loads.
Missing Gold export	Fact TODO incomplete or download structure lost.	List input directory and Databricks manifest.	Complete Gold, rerun gated publish, preserve folders.	All 15 exports found.
Partial warehouse changes	Operations not in one transaction or DDL auto-commit boundary.	Compare audit/status/counts after forced failure.	Stage/validate then data DML in one transaction; avoid DDL in load transaction.	Rollback restores state.
Power BI later sees no tables	Wrong database/schema/permission or import selection.	SSMS context and view/table queries.	Use NorthstarRetail_DW and read-only rights on dw/semantic.	SQL query succeeds.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why stage all exports before deleting targets?
2. Why delete facts before dimensions?
3. What does a transaction protect here?
4. What additional controls does Track B require?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. What is the SQL Server warehouse database name?
2. Which columns should be excluded from explicit insert lists?
3. Why is full-dev acceptable here but not automatically production-ready?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture Object Explorer warehouse schemas, SUCCESS audit row, reconciliation results, and semantic view output with batch ID. Do not show passwords/server secrets.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
feat: publish validated gold to sql server warehouse
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

Warehouse DDL/staging/views created — [STATUS]

☐

Loader reviewed/fixed — [STATUS]

☐

Gold export complete — [STATUS]

☐

Transactional load succeeded — [STATUS]

☐

Gold-to-DW reconciliation passed — [STATUS]

☐

Rollback tested — [STATUS]

☐

Same-batch reload stable — [STATUS]

☐

SSMS inspection captured — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-15. The quiz answer is in the answer-key section immediately after the quiz in this workbook#phase-15.

Next phase: Phase 16 creates the Power BI semantic model and validates DAX measures against the serving warehouse.

Required action cards

Action 15.2.1 — Confirm 11_publish_gold ran only after quality PASS and printed /gold_exports/<batch_id> plus row manifest

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Confirm 11_publish_gold ran only after quality PASS and printed /gold_exports/<batch_id> plus row manifest. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Confirm 11_publish_gold ran only after quality PASS and printed /gold_exports/<batch_id> plus row manifest.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 1

Action

Confirm `11_publish_gold` ran only after quality PASS and printed `/gold_exports/<batch_id>` plus row manifest.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Action 15.2.2 — Download the entire Gold export folder, preserving each `table.parquet` directory/file structure and ``_manifests`

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Download the entire Gold export folder, preserving each table.parquet directory/file structure and _manifest. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Download the entire Gold export folder, preserving each table.parquet directory/file structure and _manifest. Record local download path/checksum where practical.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 2****Action**

Download the entire Gold export folder, preserving each table.parquet directory/file structure and _manifest. Record local download path/checksum where practical.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Action 15.2.3 — Open SSMS and reconnect to verified local server. Run context query.

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Open SSMS and reconnect to verified local server. Run context query. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open SSMS and reconnect to verified local server. Run context query.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 3

Action

Open SSMS and reconnect to verified local server. Run context query.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Action 15.2.4 — Open/execute sql/sqlserver/warehouse/01_create_warehouse.sql, then 02_create_staging_tables.sql, then `03_

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Open/execute sql/sqlserver/warehouse/01_create_warehouse.sql, then 02_create_staging_tables.sql, then `03_. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open/execute `sql/sqlserver/warehouse/01_create_warehouse.sql`, then `02_create_staging_tables.sql`, then `03_semantic_views.sql`. Read messages and object inventory.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 4****Action**

Open/execute `sql/sqlserver/warehouse/01_create_warehouse.sql`, then `02_create_staging_tables.sql`, then `03_semantic_views.sql`. Read messages and object inventory.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Action 15.2.5 — Refresh Object Explorer: NorthstarRetail_DW > Tables and Views. Inspect dw, stg, audit, semantic obj

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Refresh Object Explorer: NorthstarRetail_DW > Tables and Views. Inspect dw, stg, audit, semantic obj. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Refresh Object Explorer: NorthstarRetail_DW > Tables and Views. Inspect dw, stg, audit, semantic objects.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 5

Action

Refresh Object Explorer: NorthstarRetail_DW > Tables and Views. Inspect dw, stg, audit, semantic objects.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Action 15.2.6 — Before loading, open scripts/loading/load_gold_to_sqlserver.py. Confirm it stages all exports, deletes facts

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Before loading, open scripts/loading/load_gold_to_sqlserver.py. Confirm it stages all exports, deletes facts. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Before loading, open scripts/loading/load_gold_to_sqlserver.py. Confirm it stages all exports, deletes facts before dimensions for a full development reload, preserves transaction, uses explicit insertable columns, and writes audit status. If your current version differs, complete/fix the TODO before running.

4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 6****Action**

Before loading, open `scripts/loading/load_gold_to_sqlserver.py`. Confirm it stages all exports, deletes facts before dimensions for a full development reload, preserves transaction, uses explicit insertable columns, and writes audit status. If your current version differs, complete/fix the TODO before running.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Action 15.2.7 — Run local loader with exact download folder and full-dev only in this disposable learning warehouse. Do not

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run local loader with exact download folder and full-dev only in this disposable learning warehouse. Do not . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run local loader with exact download folder and full-dev only in this disposable learning warehouse. Do not point it at ERP source.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 7

Action

Run local loader with exact download folder and full-dev only in this disposable learning warehouse. Do not point it at ERP source.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Action 15.2.8 — If any table fails, transaction should roll back target changes and audit/log should show failure. Fix first c

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

If any table fails, transaction should roll back target changes and audit/log should show failure. Fix first c. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: If any table fails, transaction should roll back target changes and audit/log should show failure. Fix first cause; do not manually load remaining tables.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 8****Action**

If any table fails, transaction should roll back target changes and audit/log should show failure. Fix first cause; do not manually load remaining tables.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Action 15.2.9 — Run `sql/sqlserver/validation/02_validate_warehouse.sql` and KPI validation queries. Insert/record cross-syste

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run `sql/sqlserver/validation/02_validate_warehouse.sql` and KPI validation queries. Insert/record cross-system. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run `sql/sqlserver/validation/02_validate_warehouse.sql` and KPI validation queries. Insert/record cross-system reconciliation in audit table.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 9****Action**

Run `sql/sqlserver/validation/02_validate_warehouse.sql` and KPI validation queries. Insert/record cross-system reconciliation in audit table.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Action 15.2.10 — Inspect row counts, unknown rows, SCD2 versions, fact grain, indexes, relationships, and semantic views in SSM

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Inspect row counts, unknown rows, SCD2 versions, fact grain, indexes, relationships, and semantic views in SSM. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Inspect row counts, unknown rows, SCD2 versions, fact grain, indexes, relationships, and semantic views in SSMS.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 10

Action

Inspect row counts, unknown rows, SCD2 versions, fact grain, indexes, relationships, and semantic views in SSMS.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Action 15.2.11 — Run a same-batch publication again. Full-dev should reproduce counts; incremental mode remains a user-owned ex

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run a same-batch publication again. Full-dev should reproduce counts; incremental mode remains a user-owned ex. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run a same-batch publication again. Full-dev should reproduce counts; incremental mode remains a user-owned extension with tested MERGE semantics.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 11****Action**

Run a same-batch publication again. Full-dev should reproduce counts; incremental mode remains a user-owned extension with tested MERGE semantics.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Action 15.2.12 — Track B: only with network-accessible managed SQL Server/Azure SQL, firewall/private endpoint, supported drive

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Track B: only with network-accessible managed SQL Server/Azure SQL, firewall/private endpoint, supported drive. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Track B: only with network-accessible managed SQL Server/Azure SQL, firewall/private endpoint, supported driver, secret-managed identity, and current platform support. Never expose local SQL Server to the internet for a portfolio demo.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 12

Action

Track B: only with network-accessible managed SQL Server/Azure SQL, firewall/private endpoint, supported driver, secret-managed identity, and current platform support. Never expose local SQL Server to the internet for a portfolio demo.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Action 15.2.C1 — SSMS query window

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SSMS query window

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT @@SERVERNAME AS server_name, SUSER_SNAME() AS login_name, DB_NAME() AS database_name;
```

Where to enter it:

SSMS query window. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** SELECT @@SERVERNAME AS server_name, SUSER_SNAME() AS login_name, DB_NAME() AS database_name; — SELECT requests data without changing it unless combined with a modifying statement; SERVERNAME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; server_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUSER_SNAME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; login_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DB_NAME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Action 15.2.C2 — SSMS query window

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SSMS query window

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
-- Execute the entire 01_create_warehouse.sql file.
```

Where to enter it:

SSMS query window. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `-- Execute the entire 01_create_warehouse.sql file.` is a comment for a human reader; it normally does not change data or execute business logic.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Action 15.2.C3 — SSMS query window

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SSMS query window

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
-- Execute 02_create_staging_tables.sql, then 03_semantic_views.sql.
```

Where to enter it:

SSMS query window. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** -- Execute 02_create_staging_tables.sql, then 03_semantic_views.sql. is a comment for a human reader; it normally does not change data or execute business logic.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use

git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Action 15.2.C4 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m scripts.loading.load_gold_to_sqlserver --input-dir "C:\Users\User\Downloads\gold_exports\EXT-YYYYMMDDTHHMMSSZ" --mode full-dev
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m scripts.loading.load_gold_to_sqlserver --input-dir "C:\Users\User\Downloads\gold_exports\EXT-YYYYMMDDTHHMMSSZ" --mode full-dev` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.loading.load_gold_to_sqlserver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--input` is a command option or flag; its exact behavior is defined beside the command; `-dir` is a command option or flag; its exact behavior is defined beside the command; `"C:\Users\User\Downloads\gold_exports\EXT-YYYYMMDDTHHMMSSZ"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `--mode` is a command option or flag; its exact behavior is defined beside the command; `full-dev` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Action 15.2.C5 — SSMS query window

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SSMS query window

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
-- Execute sql/sqlserver/validation/02_validate_warehouse.sql
```

Where to enter it:

SSMS query window. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** -- Execute sql/sqlserver/validation/02_validate_warehouse.sql is a comment for a human reader; it normally does not change data or execute business logic.

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Action 15.2.C6 — SSMS query window

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SSMS query window

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT TOP (20) * FROM semantic.vw_executive_daily_kpi ORDER BY full_date DESC;
```

Where to enter it:

SSMS query window. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT TOP (20) * FROM semantic.vw_executive_daily_kpi ORDER BY full_date DESC;` — `SELECT` requests data without changing it unless combined with a modifying statement; `TOP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `20` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `semantic.vw_executive_daily_kpi` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DESC` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 15.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 15 starting-screen checklist and re-verify the active context.

Lesson 15.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 15 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — scripts/

Exact folder name: scripts

Exact parent: repository root

Why it is needed now: Phase 15 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: scripts.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 15.D1.1 — Create the folder scripts

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\scripts" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\scripts" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\scripts"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\scripts" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\scripts" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/

Exact folder name: `sql`

Exact parent: `repository root`

Why it is needed now: Phase 15 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: sql.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 15.D2.1 — Create the folder sql

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\sql" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run Test-Path ".\sql" -PathType Container.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — scripts/loading/

Exact folder name: loading

Exact parent: scripts

Why it is needed now: Phase 15 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path scripts one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: loading.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 15.D3.1 — Create the folder scripts/loading

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\scripts\loading" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\scripts\loading" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\scripts\loading` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\scripts\loading" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts\loading" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\scripts\loading" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/sqlserver/

Exact folder name: `sqlserver`

Exact parent: `sql`

Why it is needed now: Phase 15 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `sql` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `sqlserver`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method**Action 15.D4.1 — Create the folder sql/sqlserver**

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\sqlserver" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\sql\sqlserver" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\sql\sqlserver" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\sqlserver" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\sqlserver" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\sqlserver" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/sqlserver/validation/

Exact folder name: `validation`

Exact parent: `sql/sqlserver`

Why it is needed now: Phase 15 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `sql/sqlserver` one level at a time.

4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: validation.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 15.D5.1 — Create the folder sql/sqlserver/validation

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\sqlserver\validation" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\sql\sqlserver\validation" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\sql\sqlserver\validation" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\sqlserver\validation" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\sqlserver\validation" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\sqlserver\validation" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/sqlserver/warehouse/

Exact folder name: warehouse

Exact parent: sql/sqlserver

Why it is needed now: Phase 15 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path sql/sqlserver one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: warehouse.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 15.D6.1 — Create the folder sql/sqlserver/warehouse

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\sqlserver\warehouse" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\sqlserver\warehouse" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\sqlserver\warehouse"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql\sqlserver\warehouse" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\sqlserver\warehouse" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\sqlserver\warehouse" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — `sql/sqlserver/warehouse/01_create_warehouse.sql`

Exact filename: `01_create_warehouse.sql`

Filename extension: `.sql`. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `sql/sqlserver/warehouse` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 15.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `sql/sqlserver/warehouse` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `01_create_warehouse.sql`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `01_create_warehouse.sql` and an empty file under the correct parent folder.
10. PRESS: `Ctrl+S` after adding each explained section.

PowerShell path

Action 15.F1.1 — Create the file `sql/sqlserver/warehouse/01_create_warehouse.sql` from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\sqlserver\warehouse\01_create_warehouse.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\sqlserver\warehouse\01_create_warehouse.sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\sqlserver\warehouse\01_create_warehouse.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use

`git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\sql\sqlserver\warehouse\01_create_warehouse.sql" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/sqlserver/warehouse/01_create_warehouse.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `sql/sqlserver/warehouse`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `sql/sqlserver/warehouse/01_create_warehouse.sql`.
3. In PowerShell at the repository root, run `Test-Path ".\sql\sqlserver\warehouse\01_create_warehouse.sql"` and require `True`.
4. Run `git status --short -- "sql/sqlserver/warehouse/01_create_warehouse.sql"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `sql/sqlserver/warehouse/01_create_warehouse.sql`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: `sql/sqlserver/warehouse/01_create_warehouse.sql`

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 15.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```

-- Dialect: SQL Server / T-SQL
-- Purpose: create the local serving warehouse NorthstarRetail_DW.
-- Run in: SSMS connected to your local SQL Server instance.
-- Safety: creates missing objects; it does not drop populated tables.

USE master;
GO
IF DB_ID(N'NorthstarRetail_DW') IS NULL CREATE DATABASE NorthstarRetail_DW;
GO
USE NorthstarRetail_DW;
GO
IF SCHEMA_ID(N'dw') IS NULL EXEC(N'CREATE SCHEMA dw AUTHORIZATION dbo;');
IF SCHEMA_ID(N'stg') IS NULL EXEC(N'CREATE SCHEMA stg AUTHORIZATION dbo;');
IF SCHEMA_ID(N'audit') IS NULL EXEC(N'CREATE SCHEMA audit AUTHORIZATION dbo;');
IF SCHEMA_ID(N'semantic') IS NULL EXEC(N'CREATE SCHEMA semantic AUTHORIZATION dbo;');
GO

IF OBJECT_ID(N'audit.etl_batch', N'U') IS NULL
CREATE TABLE audit.etl_batch
(
    batch_id VARCHAR(80) PRIMARY KEY,
    process_name VARCHAR(100) NOT NULL,
    status VARCHAR(20) NOT NULL,
    started_at_utc DATETIME2(3) NOT NULL,
    finished_at_utc DATETIME2(3) NULL,
    error_message NVARCHAR(2000) NULL
);
GO
IF OBJECT_ID(N'audit.reconciliation_result', N'U') IS NULL
CREATE TABLE audit.reconciliation_result
(
    reconciliation_id BIGINT IDENTITY(1,1) PRIMARY KEY,
    batch_id VARCHAR(80) NOT NULL,
    check_name VARCHAR(150) NOT NULL,
    source_value DECIMAL(38,6) NULL,
    target_value DECIMAL(38,6) NULL,
    difference_value DECIMAL(38,6) NULL,
    status VARCHAR(20) NOT NULL,
    checked_at_utc DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
);
GO

IF OBJECT_ID(N'dw.dim_date', N'U') IS NULL
CREATE TABLE dw.dim_date
(
    date_key INT NOT NULL PRIMARY KEY,
    full_date DATE NOT NULL UNIQUE,
    day_number TINYINT NOT NULL,
    day_name VARCHAR(10) NOT NULL,
    week_number TINYINT NOT NULL,
    month_number TINYINT NOT NULL,
    month_name VARCHAR(10) NOT NULL,
    quarter_number TINYINT NOT NULL,
    calendar_year SMALLINT NOT NULL,
    is_weekend BIT NOT NULL,
    load_timestamp_utc DATETIME2(3) NOT NULL,
    batch_id VARCHAR(80) NOT NULL,
    source_system VARCHAR(30) NOT NULL
);
GO

IF OBJECT_ID(N'dw.dim_customer', N'U') IS NULL
CREATE TABLE dw.dim_customer
(
    customer_key BIGINT NOT NULL PRIMARY KEY,
    customer_number VARCHAR(25) NOT NULL,
    first_name NVARCHAR(80) NULL,
    last_name NVARCHAR(80) NULL,
    email NVARCHAR(255) NULL,
    state_code VARCHAR(10) NULL,
    loyalty_tier VARCHAR(20) NULL,
    signup_date DATE NULL,
    effective_start_date DATE NOT NULL,
    effective_end_date DATE NOT NULL,
    is_current BIT NOT NULL,
    record_hash CHAR(64) NULL,
    load_timestamp_utc DATETIME2(3) NOT NULL,
    batch_id VARCHAR(80) NOT NULL,
    source_system VARCHAR(30) NOT NULL
);
GO
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE object_id=OBJECT_ID(N'dw.dim_customer') AND name=N'IX_dim_customer_business_current')
CREATE INDEX IX_dim_customer_business_current ON dw.dim_customer(customer_number, is_current) INCLUDE (customer_key, effective_start_date, effective_end_date);
GO

IF OBJECT_ID(N'dw.dim_product', N'U') IS NULL

```

```

CREATE TABLE dw.dim_product
(
    product_key BIGINT NOT NULL PRIMARY KEY,
    sku VARCHAR(30) NOT NULL,
    product_name NVARCHAR(200) NULL,
    category_name NVARCHAR(100) NULL,
    brand NVARCHAR(100) NULL,
    supplier_code VARCHAR(20) NULL,
    unit_cost DECIMAL(12,2) NULL,
    list_price DECIMAL(12,2) NULL,
    effective_start_date DATE NOT NULL,
    effective_end_date DATE NOT NULL,
    is_current BIT NOT NULL,
    record_hash CHAR(64) NULL,
    load_timestamp_utc DATETIME2(3) NOT NULL,
    batch_id VARCHAR(80) NOT NULL,
    source_system VARCHAR(30) NOT NULL
);
GO
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE object_id=OBJECT_ID(N'dw.dim_product') AND name=N'IX_dim_product_business_current')
CREATE INDEX IX_dim_product_business_current ON dw.dim_product(sku, is_current) INCLUDE (product_key, effective_start_date, effective_end_date);
GO

IF OBJECT_ID(N'dw.dim_store', N'U') IS NULL
CREATE TABLE dw.dim_store
(
    store_key INT NOT NULL PRIMARY KEY,
    store_code VARCHAR(20) NOT NULL,
    store_name NVARCHAR(150) NULL,
    region VARCHAR(30) NULL,
    state_code VARCHAR(10) NULL,
    city NVARCHAR(100) NULL,
    open_date DATE NULL,
    active_flag BIT NULL,
    load_timestamp_utc DATETIME2(3) NOT NULL,
    batch_id VARCHAR(80) NOT NULL,
    source_system VARCHAR(30) NOT NULL
);
GO

IF OBJECT_ID(N'dw.dim_employee', N'U') IS NULL
CREATE TABLE dw.dim_employee
(
    employee_key INT NOT NULL PRIMARY KEY,
    employee_number VARCHAR(20) NOT NULL,
    employee_name NVARCHAR(170) NULL,
    job_title NVARCHAR(100) NULL,
    store_code VARCHAR(20) NULL,
    active_flag BIT NULL,
    load_timestamp_utc DATETIME2(3) NOT NULL,
    batch_id VARCHAR(80) NOT NULL,
    source_system VARCHAR(30) NOT NULL
);
GO

IF OBJECT_ID(N'dw.dim_supplier', N'U') IS NULL
CREATE TABLE dw.dim_supplier
(
    supplier_key INT NOT NULL PRIMARY KEY,
    supplier_code VARCHAR(20) NOT NULL,
    supplier_name NVARCHAR(150) NULL,
    country_code VARCHAR(10) NULL,
    lead_time_days INT NULL,
    active_flag BIT NULL,
    load_timestamp_utc DATETIME2(3) NOT NULL,
    batch_id VARCHAR(80) NOT NULL,
    source_system VARCHAR(30) NOT NULL
);
GO

IF OBJECT_ID(N'dw.dim_promotion', N'U') IS NULL
CREATE TABLE dw.dim_promotion
(
    promotion_key INT NOT NULL PRIMARY KEY,
    promotion_code VARCHAR(30) NOT NULL,
    promotion_name NVARCHAR(150) NULL,
    discount_type VARCHAR(30) NULL,
    discount_value DECIMAL(12,4) NULL,
    start_date DATE NULL,
    end_date DATE NULL,
    channel_code VARCHAR(20) NULL,
    load_timestamp_utc DATETIME2(3) NOT NULL,
    batch_id VARCHAR(80) NOT NULL,
    source_system VARCHAR(30) NOT NULL
);
GO

```



```

IF OBJECT_ID(N'dw.dim_channel', N'U') IS NULL
CREATE TABLE dw.dim_channel
(
    channel_key INT NOT NULL PRIMARY KEY,
    channel_code VARCHAR(30) NOT NULL,
    channel_name VARCHAR(80) NOT NULL,
    channel_group VARCHAR(80) NULL,
    load_timestamp_utc DATETIME2(3) NOT NULL,
    batch_id VARCHAR(80) NOT NULL,
    source_system VARCHAR(30) NOT NULL
);
GO

IF OBJECT_ID(N'dw.dim_campaign', N'U') IS NULL
CREATE TABLE dw.dim_campaign
(
    campaign_key INT NOT NULL PRIMARY KEY,
    campaign_code VARCHAR(30) NOT NULL,
    campaign_name NVARCHAR(200) NULL,
    channel VARCHAR(40) NULL,
    start_date DATE NULL,
    end_date DATE NULL,
    budget_amount DECIMAL(14,2) NULL,
    currency_code VARCHAR(3) NULL,
    load_timestamp_utc DATETIME2(3) NOT NULL,
    batch_id VARCHAR(80) NOT NULL,
    source_system VARCHAR(30) NOT NULL
);
GO

IF OBJECT_ID(N'dw.fact_sales', N'U') IS NULL
CREATE TABLE dw.fact_sales
(
    sales_key BIGINT NOT NULL PRIMARY KEY,
    order_item_id BIGINT NOT NULL UNIQUE,
    order_id BIGINT NOT NULL,
    order_number VARCHAR(30) NOT NULL,
    order_date_key INT NOT NULL,
    customer_key BIGINT NOT NULL,
    product_key BIGINT NOT NULL,
    store_key INT NOT NULL,
    employee_key INT NOT NULL,
    promotion_key INT NOT NULL,
    channel_key INT NOT NULL,
    quantity INT NOT NULL,
    gross_sales_amount DECIMAL(16,2) NOT NULL,
    discount_amount DECIMAL(16,2) NOT NULL,
    net_sales_amount DECIMAL(16,2) NOT NULL,
    tax_amount DECIMAL(16,2) NOT NULL,
    unit_cost_amount DECIMAL(16,2) NOT NULL,
    cost_of_goods_sold DECIMAL(16,2) NOT NULL,
    gross_profit_amount DECIMAL(16,2) NOT NULL,
    currency_code VARCHAR(3) NOT NULL,
    load_timestamp_utc DATETIME2(3) NOT NULL,
    batch_id VARCHAR(80) NOT NULL,
    source_system VARCHAR(30) NOT NULL,
    CONSTRAINT FK_fact_sales_date FOREIGN KEY (order_date_key) REFERENCES dw.dim_date(date_key),
    CONSTRAINT FK_fact_sales_customer FOREIGN KEY (customer_key) REFERENCES dw.dim_customer(customer_key),
    CONSTRAINT FK_fact_sales_product FOREIGN KEY (product_key) REFERENCES dw.dim_product(product_key),
    CONSTRAINT FK_fact_sales_store FOREIGN KEY (store_key) REFERENCES dw.dim_store(store_key),
    CONSTRAINT FK_fact_sales_employee FOREIGN KEY (employee_key) REFERENCES dw.dim_employee(employee_key),
    CONSTRAINT FK_fact_sales_promotion FOREIGN KEY (promotion_key) REFERENCES dw.dim_promotion(promotion_key),
    CONSTRAINT FK_fact_sales_channel FOREIGN KEY (channel_key) REFERENCES dw.dim_channel(channel_key)
);
GO
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE object_id=OBJECT_ID(N'dw.fact_sales') AND name=N'IX_fact_sales_date_product')
CREATE INDEX IX_fact_sales_date_product ON dw.fact_sales(order_date_key, product_key) INCLUDE (net_sales_amount, gross_profit_amount, quantity);
GO

IF OBJECT_ID(N'dw.fact_returns', N'U') IS NULL
CREATE TABLE dw.fact_returns
(
    return_key BIGINT NOT NULL PRIMARY KEY,
    return_id VARCHAR(40) NOT NULL UNIQUE,
    order_item_id BIGINT NULL,
    return_date_key INT NOT NULL,
    customer_key BIGINT NOT NULL,
    product_key BIGINT NOT NULL,
    store_key INT NOT NULL,
    return_quantity INT NOT NULL,
    refund_amount DECIMAL(16,2) NOT NULL,
    return_reason NVARCHAR(100) NULL,
    currency_code VARCHAR(3) NOT NULL,
    load_timestamp_utc DATETIME2(3) NOT NULL,
    batch_id VARCHAR(80) NOT NULL,

```

```

    source_system VARCHAR(30) NOT NULL
);
GO

IF OBJECT_ID(N'dw.fact_inventory_snapshot', N'U') IS NULL
CREATE TABLE dw.fact_inventory_snapshot
(
    inventory_snapshot_key BIGINT NOT NULL PRIMARY KEY,
    snapshot_date_key INT NOT NULL,
    store_key INT NOT NULL,
    product_key BIGINT NOT NULL,
    quantity_on_hand INT NOT NULL,
    quantity_reserved INT NOT NULL,
    quantity_available AS (quantity_on_hand - quantity_reserved) PERSISTED,
    reorder_point INT NOT NULL,
    stockout_risk_flag BIT NOT NULL,
    inventory_value_amount DECIMAL(18,2) NULL,
    load_timestamp_utc DATETIME2(3) NOT NULL,
    batch_id VARCHAR(80) NOT NULL,
    source_system VARCHAR(30) NOT NULL,
    CONSTRAINT UQ_fact_inventory_grain UNIQUE (snapshot_date_key, store_key, product_key)
);
GO

IF OBJECT_ID(N'dw.fact_shipments', N'U') IS NULL
CREATE TABLE dw.fact_shipments
(
    shipment_key BIGINT NOT NULL PRIMARY KEY,
    shipment_id BIGINT NOT NULL UNIQUE,
    order_id BIGINT NOT NULL,
    ship_date_key INT NOT NULL,
    promised_date_key INT NOT NULL,
    delivery_date_key INT NOT NULL,
    customer_key BIGINT NOT NULL,
    store_key INT NOT NULL,
    carrier NVARCHAR(100) NULL,
    delivery_days INT NULL,
    on_time_flag BIT NOT NULL,
    shipping_cost DECIMAL(16,2) NOT NULL,
    load_timestamp_utc DATETIME2(3) NOT NULL,
    batch_id VARCHAR(80) NOT NULL,
    source_system VARCHAR(30) NOT NULL
);
GO

IF OBJECT_ID(N'dw.fact_web_sessions', N'U') IS NULL
CREATE TABLE dw.fact_web_sessions
(
    web_session_key BIGINT NOT NULL PRIMARY KEY,
    session_id BIGINT NOT NULL UNIQUE,
    session_date_key INT NOT NULL,
    customer_key BIGINT NOT NULL,
    campaign_key INT NOT NULL,
    channel_key INT NOT NULL,
    session_count INT NOT NULL,
    duration_seconds INT NULL,
    page_view_count INT NULL,
    add_to_cart_count INT NULL,
    converted_flag BIT NOT NULL,
    converted_order_id BIGINT NULL,
    load_timestamp_utc DATETIME2(3) NOT NULL,
    batch_id VARCHAR(80) NOT NULL,
    source_system VARCHAR(30) NOT NULL
);
GO

IF OBJECT_ID(N'dw.fact_marketing_spend', N'U') IS NULL
CREATE TABLE dw.fact_marketing_spend
(
    marketing_spend_key BIGINT NOT NULL PRIMARY KEY,
    spend_date_key INT NOT NULL,
    campaign_key INT NOT NULL,
    channel_key INT NOT NULL,
    spend_amount DECIMAL(16,2) NOT NULL,
    impressions BIGINT NOT NULL,
    clicks BIGINT NOT NULL,
    currency_code VARCHAR(3) NOT NULL,
    load_timestamp_utc DATETIME2(3) NOT NULL,
    batch_id VARCHAR(80) NOT NULL,
    source_system VARCHAR(30) NOT NULL,
    CONSTRAINT UQ_fact_marketing_grain UNIQUE (spend_date_key, campaign_key, channel_key)
);
GO

-- Unknown members. Key 0 means the source value is missing, invalid, or late arriving.
IF NOT EXISTS (SELECT 1 FROM dw.dim_customer WHERE customer_key = 0)
INSERT dw.dim_customer(customer_key, customer_number, first_name, last_name, email, state_code, loyalty_tier, signup_date, effect-
```

```

ive_start_date, effective_end_date, is_current, record_hash, load_timestamp_utc, batch_id, source_system)
VALUES (0, 'UNKNOWN', 'Unknown', 'Unknown', NULL, NULL, 'Unknown', NULL, '19000101', '99991231', 1, NULL, SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM');
IF NOT EXISTS (SELECT 1 FROM dw.dim_product WHERE product_key = 0)
INSERT dw.dim_product VALUES (0, 'UNKNOWN', 'Unknown Product', 'Unknown', NULL, NULL, NULL, 0, SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM');
VALUES (0, 'UNKNOWN', 'Unknown Product', 'Unknown', 'Unknown', 'UNKNOWN', NULL, NULL, '19000101', '99991231', 1, NULL, SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM');
IF NOT EXISTS (SELECT 1 FROM dw.dim_store WHERE store_key = 0)
INSERT dw.dim_store VALUES (0, 'UNKNOWN', 'Unknown Store', 'Unknown', NULL, NULL, NULL, 0, SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM');
IF NOT EXISTS (SELECT 1 FROM dw.dim_employee WHERE employee_key = 0)
INSERT dw.dim_employee VALUES (0, 'UNKNOWN', 'Unknown Employee', NULL, NULL, 0, SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM');
IF NOT EXISTS (SELECT 1 FROM dw.dim_supplier WHERE supplier_key = 0)
INSERT dw.dim_supplier VALUES (0, 'UNKNOWN', 'Unknown Supplier', NULL, NULL, 0, SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM');
IF NOT EXISTS (SELECT 1 FROM dw.dim_promotion WHERE promotion_key = 0)
INSERT dw.dim_promotion VALUES (0, 'NONE', 'No Promotion', NULL, NULL, NULL, NULL, NULL, SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM');
IF NOT EXISTS (SELECT 1 FROM dw.dim_channel WHERE channel_key = 0)
INSERT dw.dim_channel VALUES (0, 'UNKNOWN', 'Unknown Channel', 'Unknown', SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM');
IF NOT EXISTS (SELECT 1 FROM dw.dim_campaign WHERE campaign_key = 0)
INSERT dw.dim_campaign VALUES (0, 'NONE', 'No Campaign', NULL, NULL, NULL, NULL, NULL, SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM');
GO

SELECT s.name AS schema_name, t.name AS table_name
FROM sys.tables t JOIN sys.schemas s ON s.schema_id=t.schema_id
WHERE s.name IN ('dw','audit','stg')
ORDER BY s.name,t.name;

```

Line-by-line explanation

- **Line 1:** -- Dialect: SQL Server / T-SQL is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** -- Purpose: create the local serving warehouse NorthstarRetail_DW. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** -- Run in: SSMS connected to your local SQL Server instance. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** -- Safety: creates missing objects; it does not drop populated tables. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** USE master; — USE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; master is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** IF DB_ID(N'NorthstarRetail_DW') IS NULL CREATE DATABASE NorthstarRetail_DW; — IF begins a conditional decision; DB_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'NorthstarRetail_DW' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CREATE defines a new database object; DATABASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NorthstarRetail_DW is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** USE NorthstarRetail_DW; — USE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NorthstarRetail_DW is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** IF SCHEMA_ID(N'dw') IS NULL EXEC(N'CREATE SCHEMA dw AUTHORIZATION dbo;'); — IF begins a conditional decision; SCHEMA_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 19:** CREATE TABLE audit.etl_batch — CREATE defines a new database object; TABLE the database object that stores rows and columns; audit.etl_batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** batch_id VARCHAR(80) PRIMARY KEY, — batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 80 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** process_name VARCHAR(100) NOT NULL, — process_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** status VARCHAR(20) NOT NULL, — status shows tracked, staged, modified, and untracked work; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 20 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** started_at_utc DATETIME2(3) NOT NULL, — started_at_utc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** finished_at_utc DATETIME2(3) NULL, — finished_at_utc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** error_message NVARCHAR(2000) NULL — error_message is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NVARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2000 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 28:** G0 — G0 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** IF OBJECT_ID(N'audit.reconciliation_result', N'U') IS NULL — IF begins a conditional decision; OB-JECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'audit.reconciliation_result' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 30:** CREATE TABLE audit.reconciliation_result — CREATE defines a new database object; TABLE the database object that stores rows and columns; audit.reconciliation_result is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 32:** reconciliation_id BIGINT IDENTITY(1,1) PRIMARY KEY, — reconciliation_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BIGINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IDENTITY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** batch_id VARCHAR(80) NOT NULL, — batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 80 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** check_name VARCHAR(150) NOT NULL, — check_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 150 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** source_value DECIMAL(38,6) NULL, — source_value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DECIMAL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 38 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 6 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 36:** target_value DECIMAL(38,6) NULL, — target_value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DECIMAL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

fier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 38 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 6 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 37:** difference_value DECIMAL(38,6) NULL, — difference_value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DECIMAL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 38 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 6 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 38:** status VARCHAR(20) NOT NULL, — status shows tracked, staged, modified, and untracked work; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 20 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 39:** checked_at_utc DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — checked_at_utc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 40:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** blank line used only to separate logical sections for readability.
- **Line 43:** IF OBJECT_ID(N'dw.dim_date', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.dim_date' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 44:** CREATE TABLE dw.dim_date — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.dim_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 45:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 55:** `is_weekend BIT NOT NULL`, — `is_weekend` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 56:** `load_timestamp_utc DATETIME2(3) NOT NULL`, — `load_timestamp_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 57:** `batch_id VARCHAR(80) NOT NULL`, — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 80 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 58:** `source_system VARCHAR(30) NOT NULL` — `source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 59:** `) ;` —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 60:** `G0` — `G0` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 61:** blank line used only to separate logical sections for readability.
- **Line 62:** `IF OBJECT_ID(N'dw.dim_customer', N'U') IS NULL` — `IF` begins a conditional decision; `OBJECT_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'dw.dim_customer'` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'U'` is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 63:** `CREATE TABLE dw.dim_customer` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `dw.dim_customer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 64:** `(` — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 65:** `customer_key BIGINT NOT NULL PRIMARY KEY`, — `customer_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

- **Line 74:** `effective_end_date DATE NOT NULL`, — `effective_end_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 75:** `is_current BIT NOT NULL`, — `is_current` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 76:** `record_hash CHAR(64) NULL`, — `record_hash` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `64` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 77:** `load_timestamp_utc DATETIME2(3) NOT NULL`, — `load_timestamp_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 78:** `batch_id VARCHAR(80) NOT NULL`, — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `80` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 79:** `source_system VARCHAR(30) NOT NULL` — `source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 80:** `) ;` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 81:** `G0` — `G0` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 82:** `IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE object_id=OBJECT_ID(N'dw.dim_customer') AND name=N'IX_dim_customer_business_current')` — `IF` begins a conditional decision; `NOT` reverses a Boolean condition; `EXISTS` tests whether a matching object or row is present; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SELECT` requests data without changing it unless combined with a modifying statement; `1` is a numeric literal used by the command or expression; `FROM` identifies the source table or query; `sys.indexes` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `WHERE` filters rows to those meeting a condition; `object_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `OBJECT_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 83:** `CREATE INDEX IX_dim_customer_business_current ON dw.dim_customer(customer_number, is_current) INCLUDE (customer_key, effective_start_date, effective_end_date);` — `CREATE` defines a new database object;

INDEX is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IX_dim_customer_business_current is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim_customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is_current is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INCLUDE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 84:** G0 — G0 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 85:** blank line used only to separate logical sections for readability.
- **Line 86:** IF OBJECT_ID('dw.dim_product', 'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.dim_product' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 87:** CREATE TABLE dw.dim_product — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.dim_product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 88:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 89:** product_key BIGINT NOT NULL PRIMARY KEY, — product_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BIGINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 90:** sku VARCHAR(30) NOT NULL, — sku is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 91:** product_name NVARCHAR(200) NULL, — product_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NVARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 200 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 92:** category_name NVARCHAR(100) NULL, — category_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NVARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 100:** record_hash CHAR(64) NULL, — record_hash is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 64 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 101:** load_timestamp_utc DATETIME2(3) NOT NULL, — load_timestamp_utc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 102:** batch_id VARCHAR(80) NOT NULL, — batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 80 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 103:** source_system VARCHAR(30) NOT NULL — source_system is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 104:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 105:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 106:** IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE object_id=OBJECT_ID(N'dw.dim_product') AND name=N'IX_dim_product_business_current') — IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 1 is a numeric literal used by the command or expression; FROM identifies the source table or query; sys.indexes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; object_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 107:** CREATE INDEX IX_dim_product_business_current ON dw.dim_product(sku, is_current) INCLUDE (product_key, effective_start_date, effective_end_date); — CREATE defines a new database object; INDEX is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IX_dim_product_business_current is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim_product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sku is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is_current is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INCLUDE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 108:** G0 — G0 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 109:** blank line used only to separate logical sections for readability.
- **Line 110:** IF OBJECT_ID(N'dw.dim_store', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.dim_store' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 111:** CREATE TABLE dw.dim_store — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.dim_store is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 112:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 113:** store_key INT NOT NULL PRIMARY KEY, — store_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 114:** store_code VARCHAR(20) NOT NULL, — store_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 20 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 115:** store_name NVARCHAR(150) NULL, — store_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NVARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 150 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 116:** region VARCHAR(30) NULL, — region is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 117:** state_code VARCHAR(10) NULL, — state_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 10 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 118:** `city NVARCHAR(100) NULL, — city` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NVARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `100` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 119:** `open_date DATE NULL, — open_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 120:** `active_flag BIT NULL, — active_flag` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 121:** `load_timestamp_utc DATETIME2(3) NOT NULL, — load_timestamp_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 122:** `batch_id VARCHAR(80) NOT NULL, — batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `80` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 123:** `source_system VARCHAR(30) NOT NULL — source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 124:** `); — )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 125:** `G0 — G0` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 126:** blank line used only to separate logical sections for readability.
- **Line 127:** `IF OBJECT_ID(N'dw.dim_employee', N'U') IS NULL — IF` begins a conditional decision; `OBJECT_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'dw.dim_employee'` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'U'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 128:** `CREATE TABLE dw.dim_employee` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `dw.dim_employee` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 129:** `(` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 130:** `employee_key INT NOT NULL PRIMARY KEY,` — `employee_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 131:** `employee_number VARCHAR(20) NOT NULL,` — `employee_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `20` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 132:** `employee_name NVARCHAR(170) NULL,` — `employee_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NVARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `170` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 133:** `job_title NVARCHAR(100) NULL,` — `job_title` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NVARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `100` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 134:** `store_code VARCHAR(20) NULL,` — `store_code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `20` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 135:** `active_flag BIT NULL,` — `active_flag` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 136:** `load_timestamp_utc DATETIME2(3) NOT NULL,` — `load_timestamp_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 137:** `batch_id VARCHAR(80) NOT NULL`, — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `80` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 138:** `source_system VARCHAR(30) NOT NULL` — `source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 139:** `);` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 140:** `G0` — `G0` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 141:** blank line used only to separate logical sections for readability.
- **Line 142:** `IF OBJECT_ID(N'dw.dim_supplier', N'U') IS NULL` — `IF` begins a conditional decision; `OBJECT_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'dw.dim_supplier'` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'U'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 143:** `CREATE TABLE dw.dim_supplier` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `dw.dim_supplier` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 144:** `(` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 145:** `supplier_key INT NOT NULL PRIMARY KEY`, — `supplier_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 146:** `supplier_code VARCHAR(20) NOT NULL`, — `supplier_code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `20` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 147:** `supplier_name NVARCHAR(150) NULL`, — `supplier_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NVARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `150` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

ject-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 148:** country_code VARCHAR(10) NULL, — country_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 10 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 149:** lead_time_days INT NULL, — lead_time_days is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 150:** active_flag BIT NULL, — active_flag is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BIT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 151:** load_timestamp_utc DATETIME2(3) NOT NULL, — load_timestamp_utc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 152:** batch_id VARCHAR(80) NOT NULL, — batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 80 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 153:** source_system VARCHAR(30) NOT NULL — source_system is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 154:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 155:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 156:** blank line used only to separate logical sections for readability.
- **Line 157:** IF OBJECT_ID(N'dw.dim_promotion', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.dim_promotion' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 158:** CREATE TABLE dw.dim_promotion — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.dim_promotion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 159:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 160:** promotion_key INT NOT NULL PRIMARY KEY, — promotion_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 161:** promotion_code VARCHAR(30) NOT NULL, — promotion_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 162:** promotion_name NVARCHAR(150) NULL, — promotion_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NVARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 150 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 163:** discount_type VARCHAR(30) NULL, — discount_type is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 164:** discount_value DECIMAL(12,4) NULL, — discount_value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DECIMAL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 12 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 4 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 165:** start_date DATE NULL, — start_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 166:** end_date DATE NULL, — end_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 167:** channel_code VARCHAR(20) NULL, — channel_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or

literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 20 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 168:** `load_timestamp_utc DATETIME2(3) NOT NULL`, — `load_timestamp_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 169:** `batch_id VARCHAR(80) NOT NULL`, — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 80 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 170:** `source_system VARCHAR(30) NOT NULL` — `source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 171:** `)` ; — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 172:** `GO` — `GO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 173:** blank line used only to separate logical sections for readability.
- **Line 174:** `IF OBJECT_ID(N'dw.dim_channel' , N'U') IS NULL` — `IF` begins a conditional decision; `OBJECT_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'dw.dim_channel'` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'U'` is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 175:** `CREATE TABLE dw.dim_channel` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `dw.dim_channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 176:** `(` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 177:** `channel_key INT NOT NULL PRIMARY KEY`, — `channel_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 178:** `channel_code VARCHAR(30) NOT NULL`, — `channel_code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 179:** `channel_name VARCHAR(80) NOT NULL`, — `channel_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `80` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 180:** `channel_group VARCHAR(80) NULL`, — `channel_group` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `80` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 181:** `load_timestamp_utc DATETIME2(3) NOT NULL`, — `load_timestamp_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 182:** `batch_id VARCHAR(80) NOT NULL`, — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `80` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 183:** `source_system VARCHAR(30) NOT NULL` — `source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 184:** `)`; — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 185:** `G0` — `G0` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 186:** blank line used only to separate logical sections for readability.
- **Line 187:** `IF OBJECT_ID(N'dw.dim_campaign', N'U') IS NULL` — `IF` begins a conditional decision; `OBJECT_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'dw.dim_campaign'` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 188:** CREATE TABLE dw.dim_campaign — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.dim_campaign is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 189:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 190:** campaign_key INT NOT NULL PRIMARY KEY, — campaign_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 191:** campaign_code VARCHAR(30) NOT NULL, — campaign_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 30 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 192:** campaign_name NVARCHAR(200) NULL, — campaign_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NVARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 200 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 193:** channel VARCHAR(40) NULL, — channel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 40 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 194:** start_date DATE NULL, — start_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 195:** end_date DATE NULL, — end_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 196:** budget_amount DECIMAL(14,2) NULL, — budget_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DECIMAL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 14 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tion; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 197:** `currency_code VARCHAR(3) NULL`, — `currency_code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 198:** `load_timestamp_utc DATETIME2(3) NOT NULL`, — `load_timestamp_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 199:** `batch_id VARCHAR(80) NOT NULL`, — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `80` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 200:** `source_system VARCHAR(30) NOT NULL` — `source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 201:** `) ; — )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 202:** `G0` — `G0` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 203:** blank line used only to separate logical sections for readability.
- **Line 204:** `IF OBJECT_ID(N'dw.fact_sales', N'U') IS NULL` — `IF` begins a conditional decision; `OBJECT_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'dw.fact_sales'` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'U'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 205:** `CREATE TABLE dw.fact_sales` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `dw.fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 206:** `( — (` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 207:** `sales_key BIGINT NOT NULL PRIMARY KEY`, — `sales_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

identifier, or literal whose project-specific role is described in the surrounding explanation; order_date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; employee_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim_employee is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; employee_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 235:** CONSTRAINT FK_fact_sales_promotion FOREIGN KEY (promotion_key) REFERENCES dw.dim_promotion(promotion_key), — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FK_fact_sales_promotion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FOREIGN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; promotion_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim_promotion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; promotion_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 236:** CONSTRAINT FK_fact_sales_channel FOREIGN KEY (channel_key) REFERENCES dw.dim_channel(channel_key) — CONSTRAINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FK_fact_sales_channel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FOREIGN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; channel_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REFERENCES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim_channel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; channel_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 237:**) ; —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 238:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 239:** IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE object_id=OBJECT_ID(N'dw.fact_sales') AND name=N'IX_fact_sales_date_product') — IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 1 is a numeric literal used by the command or expression; FROM identifies the source table or query; sys.indexes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; object_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 240:** CREATE INDEX IX_fact_sales_date_product ON dw.fact_sales(order_date_key, product_key) INCLUDE (net_sales_amount, gross_profit_amount, quantity); — CREATE defines a new database object; INDEX is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IX_fact_sales_date_product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.fact_sales is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INCLUDE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 241:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 242:** blank line used only to separate logical sections for readability.
- **Line 243:** IF OBJECT_ID(N'dw.fact_returns', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.fact_returns' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 244:** CREATE TABLE dw.fact_returns — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.fact_returns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 245:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 246:** return_key BIGINT NOT NULL PRIMARY KEY, — return_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BIGINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PRIMARY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KEY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 247:** return_id VARCHAR(40) NOT NULL UNIQUE, — return_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 40 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UNIQUE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 248:** order_item_id BIGINT NULL, — order_item_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BIGINT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 249:** return_date_key INT NOT NULL, — return_date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 250:** customer_key BIGINT NOT NULL, — customer_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BIGINT is a name, keyword, argument, identifier, or

literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 259:** `source_system VARCHAR(30) NOT NULL` — `source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30` is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 260:** `) ; — )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 261:** `GO` — `GO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 262:** blank line used only to separate logical sections for readability.
- **Line 263:** `IF OBJECT_ID(N'dw.fact_inventory_snapshot', N'U') IS NULL` — IF begins a conditional decision; `OBJECT_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'dw.fact_inventory_snapshot'` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'U'` is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 264:** `CREATE TABLE dw.fact_inventory_snapshot` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `dw.fact_inventory_snapshot` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 265:** `( — (` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 266:** `inventory_snapshot_key BIGINT NOT NULL PRIMARY KEY,` — `inventory_snapshot_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 267:** `snapshot_date_key INT NOT NULL,` — `snapshot_date_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 268:** `store_key INT NOT NULL,` — `store_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 269:** `product_key BIGINT NOT NULL,` — `product_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 270:** `quantity_on_hand INT NOT NULL`, — `quantity_on_hand` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 271:** `quantity_reserved INT NOT NULL`, — `quantity_reserved` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 272:** `quantity_available AS (quantity_on_hand - quantity_reserved) PERSISTED`, — `quantity_available` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quantity_on_hand` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-` is a command option or flag; its exact behavior is defined beside the command; `quantity_reserved` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PERSISTED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 273:** `reorder_point INT NOT NULL`, — `reorder_point` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 274:** `stockout_risk_flag BIT NOT NULL`, — `stockout_risk_flag` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 275:** `inventory_value_amount DECIMAL(18,2) NULL`, — `inventory_value_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DECIMAL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `18` is a numeric literal used by the command or expression; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 276:** `load_timestamp_utc DATETIME2(3) NOT NULL`, — `load_timestamp_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 277:** `batch_id VARCHAR(80) NOT NULL`, — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `80` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or lit-

eral whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 278:** `source_system VARCHAR(30) NOT NULL, — source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 279:** `CONSTRAINT UQ_fact_inventory_grain UNIQUE (snapshot_date_key, store_key, product_key) — CON-`
`STRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UQ_fact_inventory_grain` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UNIQUE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `snapshot_date_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `store_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `product_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 280:** `); — )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 281:** `GO — GO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 282:** blank line used only to separate logical sections for readability.
- **Line 283:** `IF OBJECT_ID(N'dw.fact_shipments', 'U') IS NULL — IF` begins a conditional decision; `OBJECT_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'dw.fact_shipments'` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'U'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 284:** `CREATE TABLE dw.fact_shipments — CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `dw.fact_shipments` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 285:** `( — (` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 286:** `shipment_key BIGINT NOT NULL PRIMARY KEY, — shipment_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 287:** `shipment_id BIGINT NOT NULL UNIQUE, — shipment_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UNIQUE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

described in the surrounding explanation; 2 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 298:** `load_timestamp_utc DATETIME2(3) NOT NULL, — load_timestamp_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 299:** `batch_id VARCHAR(80) NOT NULL, — batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `80` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 300:** `source_system VARCHAR(30) NOT NULL — source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 301:** `) ; — )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 302:** `G0 — G0` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 303:** blank line used only to separate logical sections for readability.
- **Line 304:** `IF OBJECT_ID(N'dw.fact_web_sessions', N'U') IS NULL — IF` begins a conditional decision; `OBJECT_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'dw.fact_web_sessions'` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'U'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 305:** `CREATE TABLE dw.fact_web_sessions — CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `dw.fact_web_sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 306:** `( — (` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 307:** `web_session_key BIGINT NOT NULL PRIMARY KEY, — web_session_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 308:** `session_id BIGINT NOT NULL UNIQUE, — session_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

fier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; UNIQUE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 319:** `load_timestamp_utc DATETIME2(3) NOT NULL`, — `load_timestamp_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATETIME2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `3` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 320:** `batch_id VARCHAR(80) NOT NULL`, — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `80` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 321:** `source_system VARCHAR(30) NOT NULL` — `source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 322:** `);` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 323:** `G0` — `G0` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 324:** blank line used only to separate logical sections for readability.
- **Line 325:** `IF OBJECT_ID(N'dw.fact_marketing_spend', N'U') IS NULL` — `IF` begins a conditional decision; `OBJECT_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'dw.fact_marketing_spend'` is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'U'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 326:** `CREATE TABLE dw.fact_marketing_spend` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `dw.fact_marketing_spend` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 327:** `(` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 328:** `marketing_spend_key BIGINT NOT NULL PRIMARY KEY`, — `marketing_spend_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BIGINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIMARY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `KEY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 329:** `spend_date_key INT NOT NULL`, — `spend_date_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `INT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

eral whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 338:** `source_system VARCHAR(30) NOT NULL, — source_system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `30` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 339:** `CONSTRAINT UQ_fact_marketing_grain UNIQUE (spend_date_key, campaign_key, channel_key) — CON-`
`STRAINT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UQ_fact_marketing_grain` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UNIQUE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `spend_date_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `campaign_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `channel_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 340:** `); — )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 341:** `GO — GO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 342:** blank line used only to separate logical sections for readability.
- **Line 343:** `-- Unknown members. Key 0 means the source value is missing, invalid, or late arriving.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 344:** `IF NOT EXISTS (SELECT 1 FROM dw.dim_customer WHERE customer_key = 0) — IF` begins a conditional decision; `NOT` reverses a Boolean condition; `EXISTS` tests whether a matching object or row is present; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SELECT` requests data without changing it unless combined with a modifying statement; `1` is a numeric literal used by the command or expression; `FROM` identifies the source table or query; `dw.dim_customer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `WHERE` filters rows to those meeting a condition; `customer_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 345:** `INSERT dw.dim_customer(customer_key, customer_number, first_name, last_name, email, state_code,`
`loyalty_tier, signup_date, effective_start_date, effective_end_date, is_current, record_hash,`
`load_timestamp_utc, batch_id, source_system) — INSERT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dw.dim_customer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer_number` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `first_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `last_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `email` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 346:** `VALUES (0, 'UNKNOWN', 'Unknown', 'Unknown', NULL, NULL, 'Unknown', NULL, '19000101',`
`'99991231', 1, NULL, SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM'); — VALUES` is a name, keyword, argument, identifier,

surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.

- **Line 351:** INSERT dw.dim_store VALUES (0, 'UNKNOWN', 'Unknown Store', 'Unknown', NULL, NULL, NULL, 0, SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM'); — INSERT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim_store is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VALUES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'UNKNOWN' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Unknown Store' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Unknown' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 352:** IF NOT EXISTS (SELECT 1 FROM dw.dim_employee WHERE employee_key = 0) — IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 1 is a numeric literal used by the command or expression; FROM identifies the source table or query; dw.dim_employee is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; employee_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 353:** INSERT dw.dim_employee VALUES (0, 'UNKNOWN', 'Unknown Employee', NULL, NULL, 0, SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM'); — INSERT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim_employee is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VALUES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'UNKNOWN' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Unknown Employee' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 354:** IF NOT EXISTS (SELECT 1 FROM dw.dim_supplier WHERE supplier_key = 0) — IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 1 is a numeric literal used by the command or expression; FROM identifies the source table or query; dw.dim_supplier is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; supplier_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 355:** INSERT dw.dim_supplier VALUES (0, 'UNKNOWN', 'Unknown Supplier', NULL, NULL, 0, SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM'); — INSERT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim_supplier is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VALUES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'UNKNOWN' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; 'Unknown Supplier' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 356:** IF NOT EXISTS (SELECT 1 FROM dw.dim_promotion WHERE promotion_key = 0) — IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 1 is a numeric literal used by the command or expression; FROM identifies the source table or query; dw.dim_promotion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; promotion_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 357:** INSERT dw.dim_promotion VALUES (0, 'NONE', 'No Promotion', NULL, NULL, NULL, NULL, NULL, SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM'); — INSERT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim_promotion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VALUES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'NONE' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'No Promotion' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 358:** IF NOT EXISTS (SELECT 1 FROM dw.dim_channel WHERE channel_key = 0) — IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 1 is a numeric literal used by the command or expression; FROM identifies the source table or query; dw.dim_channel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; channel_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 359:** INSERT dw.dim_channel VALUES (0, 'UNKNOWN', 'Unknown Channel', 'Unknown', SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM'); — INSERT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim_channel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VALUES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'UNKNOWN' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Unknown Channel' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Unknown' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 360:** IF NOT EXISTS (SELECT 1 FROM dw.dim_campaign WHERE campaign_key = 0) — IF begins a conditional decision; NOT reverses a Boolean condition; EXISTS tests whether a matching object or row is present; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; 1 is a numeric literal used by the command or expression; FROM identifies the source table or query; dw.dim_campaign is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; campaign_key is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.

- **Line 361:** INSERT dw.dim_campaign VALUES (0, 'NONE', 'No Campaign', NULL, NULL, NULL, NULL, NULL, SYSUTCDATETIME(), 'SYSTEM', 'SYSTEM'); — INSERT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim_campaign is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VALUES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'NONE' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'No Campaign' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 362:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 363:** blank line used only to separate logical sections for readability.
- **Line 364:** SELECT s.name AS schema_name, t.name AS table_name — SELECT requests data without changing it unless combined with a modifying statement; s.name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; schema_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; t.name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 365:** FROM sys.tables t JOIN sys.schemas s ON s.schema_id=t.schema_id — FROM identifies the source table or query; sys.tables is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; t is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sys.schemas is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s.schema_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; t.schema_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 366:** WHERE s.name IN ('dw', 'audit', 'stg') — WHERE filters rows to those meeting a condition; s.name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'audit' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stg' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 367:** ORDER BY s.name, t.name; — ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s.name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; t.name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

File introduction — sql/sqlserver/warehouse/02_create_staging_tables.sql

Exact filename: 02_create_staging_tables.sql

Filename extension: .sql. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: sql/sqlserver/warehouse relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 15.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in sql/sqlserver/warehouse one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 02_create_staging_tables.sql.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 02_create_staging_tables.sql and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 15.F2.1 — Create the file sql/sqlserver/warehouse/02_create_staging_tables.sql from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.

3. **CLICK:** Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. **TYPE THIS:** Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. **PRESS:** Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. **LOOK FOR:** Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. **DO NOT CONTINUE UNTIL:** Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\sqlserver\warehouse\02_create_staging_tables.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\sqlserver\warehouse\02_create_staging_tables.sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\sql\sqlserver\warehouse\02_create_staging_tables.sql` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\sql\sqlserver\warehouse\02_create_staging_tables.sql" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/sqlserver/warehouse/02_create_staging_tables.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `sql/sqlserver/warehouse`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `sql/sqlserver/warehouse/02_create_staging_tables.sql`.
3. In PowerShell at the repository root, run `Test-Path ".\sql\sqlserver\warehouse\02_create_staging_tables.sql"` and require True.
4. Run `git status --short -- "sql/sqlserver/warehouse/02_create_staging_tables.sql"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `sql/sqlserver/warehouse/02_create_staging_tables.sql`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: `sql/sqlserver/warehouse/02_create_staging_tables.sql`

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 15.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: SQL Server / T-SQL
-- Purpose: create empty staging tables that mirror warehouse targets.
-- Run after 01_create_warehouse.sql in NorthstarRetail_DW.
USE NorthstarRetail_DW;
GO

DECLARE @table sysname;
DECLARE table_cursor CURSOR LOCAL FAST_FORWARD FOR
SELECT name FROM sys.tables WHERE schema_id = SCHEMA_ID('dw');
OPEN table_cursor;
FETCH NEXT FROM table_cursor INTO @table;
WHILE @@FETCH_STATUS = 0
BEGIN
```

```

DECLARE @stage sysname = N'stg_' + @table;
IF OBJECT_ID(QUOTENAME('stg') + N'.' + QUOTENAME(@stage), N'U') IS NULL
BEGIN
    DECLARE @sql nvarchar(max) = N'SELECT TOP (0) * INTO stg.' + QUOTENAME(@stage) + N' FROM dw.' + QUOTENAME(@table) + N';';
    EXEC sys.sp_executesql @sql;
END;
FETCH NEXT FROM table_cursor INTO @table;
END;
CLOSE table_cursor;
DEALLOCATE table_cursor;
GO
SELECT TABLE_SCHEMA, TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_SCHEMA='stg' ORDER BY TABLE_NAME;

```

Line-by-line explanation

- **Line 1:** -- Dialect: SQL Server / T-SQL is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** -- Purpose: create empty staging tables that mirror warehouse targets. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** -- Run after 01_create_warehouse.sql in NorthstarRetail_DW. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** USE NorthstarRetail_DW; — USE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NorthstarRetail_DW is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** DECLARE @table sysname; — DECLARE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sysname is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** DECLARE table_cursor CURSOR LOCAL FAST_FORWARD FOR — DECLARE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table_cursor is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CURSOR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; LOCAL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FAST_FORWARD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FOR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** SELECT name FROM sys.tables WHERE schema_id = SCHEMA_ID('dw'); — SELECT requests data without changing it unless combined with a modifying statement; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; sys.tables is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; schema_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SCHEMA_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** OPEN table_cursor; — OPEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table_cursor is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** FETCH NEXT FROM table_cursor INTO @table; — FETCH is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NEXT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; table_cursor is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INTO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** WHILE @@FETCH_STATUS = 0 — WHILE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FETCH_STATUS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or

literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression

- **Line 13:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** DECLARE @stage sysname = N'stg_' + @table; — DECLARE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stage is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sysname is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stg_' is a quoted literal; the quotation marks preserve spaces or mark text; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** IF OBJECT_ID(QUOTENAME('stg') + N'.' + QUOTENAME(@stage), N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; QUOTENAME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stg' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '.' is a quoted literal; the quotation marks preserve spaces or mark text; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; QUOTENAME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 16:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** DECLARE @sql nvarchar(max) = N'SELECT TOP (0) * INTO stg.' + QUOTENAME(@stage) + N' FROM dw.' + QUOTENAME(@table) + N';'; — DECLARE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sql is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; nvarchar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; max is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'SELECT TOP (0) * INTO stg.' is a quoted literal; the quotation marks preserve spaces or mark text; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; QUOTENAME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 18:** EXEC sys.sp_executesql @sql; — EXEC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sys.sp_executesql is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sql is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** FETCH NEXT FROM table_cursor INTO @table; — FETCH is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NEXT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; table_cursor is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INTO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 22:** `CLOSE table_cursor;` — `CLOSE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_cursor` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** `DEALLOCATE table_cursor;` — `DEALLOCATE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table_cursor` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** `G0` — `G0` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** `SELECT TABLE_SCHEMA, TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_SCHEMA='stg' ORDER BY TABLE_NAME;` — `SELECT` requests data without changing it unless combined with a modifying statement; `TABLE_SCHEMA` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `TABLE_NAME` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FROM` identifies the source table or query; `INFORMATION_SCHEMA.TABLES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `WHERE` filters rows to those meeting a condition; `TABLE_SCHEMA` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'stg'` is a quoted literal; the quotation marks preserve spaces or mark text; `ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

[Back to the Phase 15 lesson index](#)

File introduction — `scripts/loading/load_gold_to_sqlserver.py`

Exact filename: `load_gold_to_sqlserver.py`

Filename extension: `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `scripts/loading` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: executable Python logic for Phase 15.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `scripts/loading` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `load_gold_to_sqlserver.py`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `load_gold_to_sqlserver.py` and an empty file under the correct parent folder.
10. PRESS: `Ctrl+S` after adding each explained section.

PowerShell path

Action 15.F3.1 — Create the file scripts/loading/load_gold_to_sqlserver.py from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\scripts\loading\load_gold_to_sqlserver.py" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType File -Path ".\scripts\loading\load_gold_to_sqlserver.py" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; File is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\scripts\loading\load_gold_to_sqlserver.py" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\scripts\loading\load_gold_to_sqlserver.py" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "scripts/loading/load_gold_to_sqlserver.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `scripts/loading`.

2. Right-click the editor tab and choose **Copy Path**; compare the ending with `scripts/loading/load_gold_to_sqlserver.py`.
3. In PowerShell at the repository root, run `Test-Path ".\scripts\loading\load_gold_to_sqlserver.py"` and require `True`.
4. Run `git status --short -- "scripts/loading/load_gold_to_sqlserver.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `scripts/loading/load_gold_to_sqlserver.py`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Python

Exact file: `scripts/loading/load_gold_to_sqlserver.py`

Purpose: executable Python logic.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 15.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
"""Track A local loader: publish downloaded Gold exports to NorthstarRetail_DW safely.

The required beginner mode is ``full-dev`` for a disposable learning warehouse. It stages and
validates every table before changing targets, deletes child facts before parent dimensions,
inserts parents before children with explicit noncomputed columns, and commits once.
"""
from __future__ import annotations

import argparse
import json
import logging
from datetime import UTC, datetime
from pathlib import Path
from typing import Any

import pandas as pd
from sqlalchemy import Connection, text

from scripts.utilities.config import load_environment
from scripts.utilities.database import sqlserver_warehouse_engine
from scripts.utilities.logging_utils import configure_logging

LOGGER = logging.getLogger("northstar.publish_gold")
DIMENSION_TABLES = [
    "dim_date", "dim_customer", "dim_product", "dim_store", "dim_employee",
    "dim_supplier", "dim_promotion", "dim_channel", "dim_campaign",
]
FACT_TABLES = [
    "fact_sales", "fact_returns", "fact_inventory_snapshot", "fact_shipments",
    "fact_web_sessions", "fact_marketing_spend",
]
TABLE_ORDER = DIMENSION_TABLES + FACT_TABLES

def read_export(path: Path) -> pd.DataFrame:
    """Read one downloaded Spark Parquet directory/file or CSV file."""
    if path.suffix.lower() == ".parquet":
        return pd.read_parquet(path)
    if path.suffix.lower() == ".csv":
        return pd.read_csv(path)
    raise ValueError(f"Unsupported Gold export path: {path}")

def find_export(input_dir: Path, table: str) -> Path:
    """Find exactly one ``table.parquet`` or ``table.csv`` export."""
    candidates = [input_dir / f"{table}.parquet", input_dir / f"{table}.csv"]
    existing = [path for path in candidates if path.exists()]
    if len(existing) != 1:
        raise FileNotFoundError(
            f"Expected exactly one Gold export for {table}; found {existing} in {input_dir}"
        )
    return existing[0]

def quote_identifier(name: str) -> str:
    """Quote a trusted SQL Server identifier after a strict local allow-list check."""
    if not name.replace("_", "").isalnum():
        raise ValueError(f"Unsafe identifier: {name!r}")
    return f"{{name}}"
```

```
def insertable_columns(connection: Connection, table: str) -> list[str]:
    """Return target columns excluding identity and computed columns, in physical order."""
    rows = connection.execute(
```

```

    text(
        """
        SELECT c.name
        FROM sys.columns AS c
        WHERE c.object_id = OBJECT_ID(:object_name)
              AND c.is_identity = 0
              AND c.is_computed = 0
        ORDER BY c.column_id;
        """
    ),
    {"object_name": f"dw.{table}"},
).fetchall()
columns = [str(row[0]) for row in rows]
if not columns:
    raise RuntimeError(f"No insertable columns found for dw.{table}")
return columns

def stage_all(connection: Connection, input_dir: Path) -> tuple[dict[str, pd.DataFrame], list[dict[str, Any]]]:
    """Read and stage every table before any target row is deleted."""
    frames: dict[str, pd.DataFrame] = {}
    results: list[dict[str, Any]] = []
    for table in TABLE_ORDER:
        export_path = find_export(input_dir, table)
        frame = read_export(export_path)
        if frame.empty and table in {"dim_date", "dim_channel":
            raise ValueError(f"Required reference table {table} is empty")
        stage = f"stg_{table}"
        connection.execute(text(f"TRUNCATE TABLE stg.{quote_identifier(stage)};"))
        frame.to_sql(
            stage,
            connection,
            schema="stg",
            if_exists="append",
            index=False,
            chunksize=1000,
            method="multi",
        )
        staged_count = int(
            connection.execute(text(f"SELECT COUNT_BIG(*) FROM stg.{quote_identifier(stage)};")).scalar_one()
        )
        if staged_count != len(frame):
            raise AssertionError(
                f"Staging mismatch for {table}: file={len(frame)} stage={staged_count}"
            )
        frames[table] = frame
        results.append(
            {
                "table": table,
                "export_path": str(export_path),
                "file_rows": len(frame),
                "staged_rows": staged_count,
            }
        )
    return frames, results

def full_dev_publish(connection: Connection, results: list[dict[str, Any]]) -> None:
    """Replace a disposable development warehouse in foreign-key-safe order."""
    for table in FACT_TABLES:
        connection.execute(text(f"DELETE FROM dw.{quote_identifier(table)};"))
    for table in reversed(DIMENSION_TABLES):
        connection.execute(text(f"DELETE FROM dw.{quote_identifier(table)};"))

    for table in TABLE_ORDER:
        columns = insertable_columns(connection, table)
        quoted_columns = ", ".join(quote_identifier(column) for column in columns)
        stage = f"stg_{table}"
        connection.execute(
            text(
                f"INSERT INTO dw.{quote_identifier(table)} ({quoted_columns}) "
                f"SELECT {quoted_columns} FROM stg.{quote_identifier(stage)};"
            )
        )
        target_count = int(
            connection.execute(text(f"SELECT COUNT_BIG(*) FROM dw.{quote_identifier(table)};")).scalar_one()
        )
        result = next(item for item in results if item["table"] == table)
        result["target_rows"] = target_count
        if target_count != result["staged_rows"]:
            raise AssertionError(
                f"Target mismatch for {table}: stage={result['staged_rows']} target={target_count}"
            )

def write_artifact(batch_id: str, status: str, results: list[dict[str, Any]], error: str | None = None) -> Path:
    """Write a local nonsecret publication summary."""

```

```

output_dir = Path("run_artifacts")
output_dir.mkdir(exist_ok=True)
path = output_dir / f"{batch_id}.json"
path.write_text(
    json.dumps({"batch_id": batch_id, "status": status, "error": error, "tables": results}, indent=2),
    encoding="utf-8",
)
return path

def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input-dir", type=Path, required=True)
    parser.add_argument("--mode", choices=["full-dev", "incremental"], default="full-dev")
    args = parser.parse_args()
    if args.mode == "incremental":
        raise NotImplementedError(
            "TODO(USER): implement and test the Phase 15 incremental strategy only after full-dev is verified."
        )
    if not args.input_dir.exists():
        raise FileNotFoundError(args.input_dir)

    configure_logging()
    load_environment()
    batch_id = f"PUB-{datetime.now(UTC):%Y%m%d%H%M%S}"
    engine = sqlserver_warehouse_engine()
    results: list[dict[str, Any]] = []
    try:
        with engine.begin() as connection:
            connection.execute(
                text(
                    "INSERT INTO audit.etl_batch"
                    "(batch_id, process_name, status, started_at_utc) "
                    "VALUES (:batch_id, 'publish_gold', 'RUNNING', SYSUTCDATETIME())"
                ),
                {"batch_id": batch_id},
            )
            _, results = stage_all(connection, args.input_dir)
            full_dev_publish(connection, results)
            connection.execute(
                text(
                    "UPDATE audit.etl_batch SET status='SUCCESS', "
                    "finished_at_utc=SYSUTCDATETIME() WHERE batch_id=:batch_id"
                ),
                {"batch_id": batch_id},
            )
            artifact = write_artifact(batch_id, "SUCCESS", results)
            LOGGER.info("Published batch %s; artifact=%s", batch_id, artifact)
    except Exception as exc:
        # The data transaction above rolls back. Record failure in a separate transaction.
        try:
            with engine.begin() as connection:
                connection.execute(
                    text(
                        "INSERT INTO audit.etl_batch"
                        "(batch_id, process_name, status, started_at_utc, finished_at_utc, error_message) "
                        "VALUES (:batch_id, 'publish_gold', 'FAILED', SYSUTCDATETIME(), "
                        "SYSUTCDATETIME(), :error_message)"
                    ),
                    {"batch_id": batch_id, "error_message": str(exc)[:2000]},
                )
            except Exception:
                LOGGER.exception("Could not record failed publication audit row")
            write_artifact(batch_id, "FAILED", results, str(exc))
            raise

if __name__ == "__main__":
    main()

```

Line-by-line explanation

- Line 1:** `"""Track A local loader: publish downloaded Gold exports to NorthstarRetail_DW safely. — """` is a quoted literal; the quotation marks preserve spaces or mark text; `Track` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `A` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `local` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `loader` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `publish` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `downloaded` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Gold` is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; exports is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; to is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NorthstarRetail_DW is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** The required beginner mode is \full-dev` for a disposable learning warehouse. It stages and—Theis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;requiredis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;beginneris a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;modeis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;isis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;full-devis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;foris a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;ais a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;disposableis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;learningis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;warehouseis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;.` is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.
- **Line 4:** validates every table before changing targets, deletes child facts before parent dimensions, — validates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; every is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; before is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; changing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; targets is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; deletes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; child is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; facts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; before is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; parent is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 5:** inserts parents before children with explicit noncomputed columns, and commits once. — inserts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; parents is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; before is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; children is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; with is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; explicit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; noncomputed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; columns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; commits is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; once is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 6:** "" — "" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 7:** from __future__ import annotations — from is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; __future__ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; import is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; annotations is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** `import argparse` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `argparse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `import json` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `json` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** `import logging` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `logging` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `from datetime import UTC, datetime` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `UTC` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `datetime` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `from pathlib import Path` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pathlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** `from typing import Any` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `typing` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Any` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** blank line used only to separate logical sections for readability.
- **Line 16:** `import pandas as pd` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pandas` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** `from sqlalchemy import Connection, text` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlalchemy` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** `from scripts.utilities.config import load_environment` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scripts.utilities.config` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `load_environment` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** `from scripts.utilities.database import sqlserver_warehouse_engine` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scripts.utilities.database` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlserver_warehouse_engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** `from scripts.utilities.logging_utils import configure_logging` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scripts.utilities.logging_utils` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

specific role is described in the surrounding explanation; `configure_logging` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 22:** blank line used only to separate logical sections for readability.
- **Line 23:** `LOGGER = logging.getLogger("northstar.publish_gold")` — `LOGGER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `logging.getLogger` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"northstar.publish_gold"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** `DIMENSION_TABLES = [` — `DIMENSION_TABLES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** `"dim_date", "dim_customer", "dim_product", "dim_store", "dim_employee",` — `"dim_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"dim_customer"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"dim_product"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"dim_store"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"dim_employee"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** `"dim_supplier", "dim_promotion", "dim_channel", "dim_campaign",` — `"dim_supplier"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"dim_promotion"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"dim_channel"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"dim_campaign"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:** `]` — `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** `FACT_TABLES = [` — `FACT_TABLES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** `"fact_sales", "fact_returns", "fact_inventory_snapshot", "fact_shipments",` — `"fact_sales"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"fact_returns"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"fact_inventory_snapshot"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"fact_shipments"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 30:** `"fact_web_sessions", "fact_marketing_spend",` — `"fact_web_sessions"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"fact_marketing_spend"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** `]` — `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 32:** `TABLE_ORDER = DIMENSION_TABLES + FACT_TABLES` — `TABLE_ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DIMENSION_TABLES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

FACT_TABLES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 33:** blank line used only to separate logical sections for readability.
- **Line 34:** blank line used only to separate logical sections for readability.
- **Line 35:** `def read_export(path: Path) -> pd.DataFrame:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `read_export` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `pd.DataFrame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 36:** `"""Read one downloaded Spark Parquet directory/file or CSV file."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `"Read one downloaded Spark Parquet directory/file or CSV file."` is a path, relative location, branch-style name, or file argument; verify every character before running it; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 37:** `if path.suffix.lower() == ".parquet":` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.suffix.lower` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `".parquet"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 38:** `return pd.read_parquet(path)` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.read_parquet` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 39:** `if path.suffix.lower() == ".csv":` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.suffix.lower` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `".csv"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 40:** `return pd.read_csv(path)` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pd.read_csv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** `raise ValueError(f"Unsupported Gold export path: {path}")` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ValueError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Unsupported Gold export path: {path}"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** blank line used only to separate logical sections for readability.
- **Line 43:** blank line used only to separate logical sections for readability.
- **Line 44:** `def find_export(input_dir: Path, table: str) -> Path:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `find_export` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `input_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; remaining punctuation and identifiers continue the same statement.

- **Line 45:** `"""Find exactly one \table.parquet` or `table.csv` export."""`—`"""` is a quoted literal; the quotation marks preserve spaces or mark text; `Find exactly one \table.parquet` or `table.csv` export.` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 46:** `candidates = [input_dir / f"{table}.parquet", input_dir / f"{table}.csv"]` — `candidates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `input_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{table}.parquet"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `input_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{table}.csv"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 47:** `existing = [path for path in candidates if path.exists()]` — `existing` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `candidates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.exists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 48:** `if len(existing) != 1:` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `existing` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 49:** `raise FileNotFoundError` (— `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FileNotFoundError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 50:** `f"Expected exactly one Gold export for {table}; found {existing} in {input_dir}"` — `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Expected exactly one Gold export for {table}; found {existing} in {input_dir}"` is a quoted literal; the quotation marks preserve spaces or mark text

- **Line 51:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 52:** `return existing[0]` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `existing` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 53:** blank line used only to separate logical sections for readability.
- **Line 54:** blank line used only to separate logical sections for readability.
- **Line 55:** `def quote_identifier(name: str) -> str` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quote_identifier` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 56:** `"""Quote a trusted SQL Server identifier after a strict local allow-list check."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `Quote a trusted SQL Server identifier after a strict local allow-list check.` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 57:** `if not name.replace("_", "").isalnum():` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `name.replace` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"_"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `""` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `isalnum` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 58:** `raise ValueError(f"Unsafe identifier: {name!r}")` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ValueError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Unsafe identifier: {name!r}"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 59:** `return f"[{name}]"` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"[{name}]"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 60:** blank line used only to separate logical sections for readability.
- **Line 61:** blank line used only to separate logical sections for readability.
- **Line 62:** `def insertable_columns(connection: Connection, table: str) -> list[str]` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `insertable_columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `list[str]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -> is a command option or flag; its exact behavior is defined beside the command; remaining punctuation and identifiers continue the same statement.

- **Line 63:** `""Return target columns excluding identity and computed columns, in physical order.""` — `""` is a quoted literal; the quotation marks preserve spaces or mark text; `Return target columns excluding identity and computed columns, in physical order.` is a quoted literal; the quotation marks preserve spaces or mark text; `""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 64:** `rows = connection.execute(` — `rows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection.execute` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 65:** `text(` — `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 66:** `""` — `""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 67:** `SELECT c.name` — `SELECT` requests data without changing it unless combined with a modifying statement; `c.name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 68:** `FROM sys.columns AS c` — `FROM` identifies the source table or query; `sys.columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `c` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 69:** `WHERE c.object_id = OBJECT_ID(:object_name)` — `WHERE` filters rows to those meeting a condition; `c.object_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `OBJECT_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `object_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 70:** `AND c.is_identity = 0` — `AND` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `c.is_identity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression
- **Line 71:** `AND c.is_computed = 0` — `AND` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `c.is_computed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression
- **Line 72:** `ORDER BY c.column_id;` — `ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `c.column_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 73:** `""` — `""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 74:** `),` — `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 75:** `{"object_name": f"dw.{table}"}`, — `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"object_name"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"dw.{table}"` is a quoted literal; the quotation marks preserve spaces or mark text; `}`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 76:** `).fetchall()` — `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fetchall` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 77:** `columns = [str(row[0]) for row in rows]` — `columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 78:** `if not columns:` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 79:** `raise RuntimeError(f"No insertable columns found for dw.{table}")` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `RuntimeError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"No insertable columns found for dw.{table}"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 80:** `return columns` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 81:** blank line used only to separate logical sections for readability.
- **Line 82:** blank line used only to separate logical sections for readability.
- **Line 83:** `def stage_all(connection: Connection, input_dir: Path) -> tuple[dict[str, pd.DataFrame], list[dict[str, Any]]]:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `stage_all` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `input_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; remaining punctuation and identifiers continue the same statement.
- **Line 84:** `"""Read and stage every table before any target row is deleted."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `Read and stage every table before any target row is deleted.` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 85:** `frames: dict[str, pd.DataFrame] = {}` — `frames` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dict` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument,

(is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Required reference table {table} is empty" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 92:** stage = f"stg_{table}" — stage is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "stg_{table}" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 93:** connection.execute(text(f"TRUNCATE TABLE stg.{quote_identifier(stage)};")) — connection.execute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; text is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "TRUNCATE TABLE stg.{quote_identifier(stage)};" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 94:** frame.to_sql(— frame.to_sql is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 95:** stage, — stage is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 96:** connection, — connection is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 97:** schema="stg", — schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "stg" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 98:** if_exists="append", — if_exists is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "append" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 99:** index=False, — index is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; False is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 100:** chunksize=1000, — chunksize is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1000 is a numeric literal used by the command or expression; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 101:** method="multi", — method is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "multi" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 102:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 103:** staged_count = int(— staged_count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; int is a name, keyword, argument, identifier, or liter-

al whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 104:** `connection.execute(text(f"SELECT COUNT_BIG(*) FROM stg.{quote_identifier(stage)};"))` — `connection.execute` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SELECT COUNT_BIG(*) FROM stg.{quote_identifier(stage)};"` is a quoted literal; the quotation marks preserve spaces or mark text;)) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scalar_one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 105:** `) — )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 106:** `if staged_count != len(frame):` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `staged_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 107:** `raise AssertionError(` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AssertionError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 108:** `f"Staging mismatch for {table}: file={len(frame)} stage={staged_count}"` — `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Staging mismatch for {table}: file={len(frame)} stage={staged_count}"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 109:** `) — )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 110:** `frames[table] = frame` — `frames` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 111:** `results.append(` — `results.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 112:** `{` — `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 113:** `"table": table,` — `"table"` is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 114:** `"export_path": str(export_path),` — `"export_path"` is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `export_path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 115:** `"file_rows": len(frame),` — `"file_rows"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `len` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frame` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 116:** `"staged_rows": staged_count,` — `"staged_rows"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `staged_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 117:** `}` — `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 118:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 119:** `return frames, results` — `return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `frames` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 120:** blank line used only to separate logical sections for readability.
- **Line 121:** blank line used only to separate logical sections for readability.
- **Line 122:** `def full_dev_publish(connection: Connection, results: list[dict[str, Any]]) -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full_dev_publish` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `list` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dict` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 123:** `"""Replace a disposable development warehouse in foreign-key-safe order."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `Replace a disposable development warehouse in foreign-key-safe order.` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 124:** `for table in FACT_TABLES:` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FACT_TABLES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 125:** `connection.execute(text(f"DELETE FROM dw.{quote_identifier(table)}"))` — `connection.execute` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"DELETE FROM dw.{quote_identifier(table)}"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 126:** `for table in reversed(DIMENSION_TABLES):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier,

er, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reversed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DIMENSION_TABLES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 127:** `connection.execute(text(f"DELETE FROM dw.{quote_identifier(table)}"))` — `connection.execute` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"DELETE FROM dw.{quote_identifier(table)}"` is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 128:** blank line used only to separate logical sections for readability.
- **Line 129:** `for table in TABLE_ORDER:` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `TABLE_ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 130:** `columns = insertable_columns(connection, table)` — `columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `insertable_columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 131:** `quoted_columns = ", ".join(quote_identifier(column) for column in columns)` — `quoted_columns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `", "` is a quoted literal; the quotation marks preserve spaces or mark text; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `join` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quote_identifier` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `column` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `column` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 132:** `stage = f"stg_{table}"` — `stage` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"stg_{table}"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 133:** `connection.execute(` — `connection.execute` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 134:** `text(` — `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 135:** `f"INSERT INTO dw.{quote_identifier(table)} ({quoted_columns}) "` — `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"INSERT INTO dw.{quote_identifier(table)} ({quoted_columns}) "` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 136:** `f"SELECT {quoted_columns} FROM stg.{quote_identifier(stage)};"` — `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SELECT {quoted_columns} FROM stg.{quote_identifier(stage)};"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 137:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 138:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 139:** `target_count = int(` — `target_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `int` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 140:** `connection.execute(text(f"SELECT COUNT_BIG(*) FROM dw.{quote_identifier(table)};"))` — `connection.execute` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SELECT COUNT_BIG(*) FROM dw.{quote_identifier(table)};"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scalar_one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 141:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 142:** `result = next(item for item in results if item["table"] == table)` — `result` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `next` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `item` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `item` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `item` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 143:** `result["target_rows"] = target_count` — `result` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"target_rows"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 144:** `if target_count != result["staged_rows"]:` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `result` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

"staged_rows" is a quoted literal; the quotation marks preserve spaces or mark text;] : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 145:** raise AssertionError(— raise is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AssertionError is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 146:** f"Target mismatch for {table}: stage={result['staged_rows']} target={target_count}" — f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Target mismatch for {table}: stage={result['staged_rows']} target={target_count}" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 147:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 148:** blank line used only to separate logical sections for readability.
- **Line 149:** blank line used only to separate logical sections for readability.
- **Line 150:** def write_artifact(batch_id: str, status: str, results: list[dict[str, Any]], error: str | None = None) -> Path: — def is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; write_artifact is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; status shows tracked, staged, modified, and untracked work; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; results is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 151:** """Write a local nonsecret publication summary.""" — " is a quoted literal; the quotation marks preserve spaces or mark text; "Write a local nonsecret publication summary." is a quoted literal; the quotation marks preserve spaces or mark text; " is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 152:** output_dir = Path("run_artifacts") — output_dir is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "run_artifacts" is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 153:** output_dir.mkdir(exist_ok=True) — output_dir.mkdir is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; exist_ok is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; True is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 154:** path = output_dir / f"{batch_id}.json" — path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; output_dir is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "{batch_id}.json" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 155:** path.write_text(— path.write_text is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 156:** json.dumps({"batch_id": batch_id, "status": status, "error": error, "tables": results}, indent=2), — json.dumps is a name, keyword, argument, identifier, or literal whose project-specific role is described in

the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "batch_id" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "status" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; status shows tracked, staged, modified, and untracked work; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "error" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 157:** encoding="utf-8" , — encoding is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "utf-8" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 158:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 159:** return path — return is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 160:** blank line used only to separate logical sections for readability.
- **Line 161:** blank line used only to separate logical sections for readability.
- **Line 162:** def main() -> None: — def is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; main is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -> is a command option or flag; its exact behavior is defined beside the command; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 163:** parser = argparse.ArgumentParser(description=__doc__) — parser is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; argparse.ArgumentParser is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; description is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; __doc__ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 164:** parser.add_argument("--input-dir", type=Path, required=True) — parser.add_argument is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "--input-dir" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; type is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; required is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; True is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 165:** parser.add_argument("--mode", choices=["full-dev", "incremental"], default="full-dev") — parser.add_argument is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "--mode" is a quoted literal; the quotation marks preserve spaces or mark text; , is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; choices is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [=] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "full-dev" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "incremental" is a quoted literal; the quotation marks preserve spaces or mark text;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; default is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 166:** `args = parser.parse_args()` — `args` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parser.parse_args` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 167:** `if args.mode == "incremental":` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.mode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"incremental"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 168:** `raise NotImplementedError()` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NotImplementedError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 169:** `"TODO(USER): implement and test the Phase 15 incremental strategy only after full-dev is verified."` — `"TODO(USER): implement and test the Phase 15 incremental strategy only after full-dev is verified."` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 170:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 171:** `if not args.input_dir.exists():` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.input_dir.exists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 172:** `raise FileNotFoundError(args.input_dir)` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `FileNotFoundError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.input_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 173:** blank line used only to separate logical sections for readability.
- **Line 174:** `configure_logging()` — `configure_logging` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 175:** `load_environment()` — `load_environment` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 176:** `batch_id = f"PUB-{datetime.now(UTC):%Y%m%dT%H%M%S}"` — `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"PUB-{datetime.now(UTC):%Y%m%dT%H%M%S}"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 177:** `engine = sqlserver_warehouse_engine()` — `engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlserver_warehouse_engine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 178:** `results: list[dict[str, Any]] = []` — `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `list` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dict` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Any` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 179:** `try:` — `try` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 180:** `with engine.begin() as connection:` — `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `engine.begin` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 181:** `connection.execute(` — `connection.execute` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 182:** `text(` — `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 183:** `"INSERT INTO audit.etl_batch"` — `"INSERT INTO audit.etl_batch"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 184:** `"(batch_id, process_name, status, started_at_utc) "` — `"(batch_id, process_name, status, started_at_utc) "` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 185:** `"VALUES (:batch_id, 'publish_gold', 'RUNNING', SYSUTCDATETIME())"` — `"VALUES (:batch_id, 'publish_gold', 'RUNNING', SYSUTCDATETIME())"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 186:** `), — ),` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 187:** `{"batch_id": batch_id},` — `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 188:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 189:** `_, results = stage_all(connection, args.input_dir)` — `_` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `stage_all` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `args.input_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 190:** `full_dev_publish(connection, results)` — `full_dev_publish` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 191:** `connection.execute(` — `connection.execute` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 192:** `text(` — `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 193:** `"UPDATE audit.etl_batch SET status='SUCCESS', "` — `"UPDATE audit.etl_batch SET status='SUCCESS', "` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 194:** `"finished_at_utc=SYSUTCDATETIME() WHERE batch_id=:batch_id"` — `"finished_at_utc=SYSUTCDATETIME() WHERE batch_id=:batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 195:** `),` — `),` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 196:** `{"batch_id": batch_id},` — `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 197:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 198:** `artifact = write_artifact(batch_id, "SUCCESS", results)` — `artifact` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `write_artifact` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"SUCCESS"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 199:** `LOGGER.info("Published batch %s; artifact=%s", batch_id, artifact)` — `LOGGER.info` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Published batch %s; artifact=%s"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `artifact` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 200:** `except Exception as exc:` — `except` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Exception` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 201:** `# The data transaction above rolls back. Record failure in a separate transaction.` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 202:** `try:` — `try` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 203:** `with engine.begin() as connection:` — `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `engine.begin` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 204:** `connection.execute(` — `connection.execute` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 205:** `text(` — `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 206:** `"INSERT INTO audit.etl_batch"` — `"INSERT INTO audit.etl_batch"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 207:** `"(batch_id, process_name, status, started_at_utc, finished_at_utc, error_message) "` — `"(batch_id, process_name, status, started_at_utc, finished_at_utc, error_message) "` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 208:** `"VALUES (:batch_id, 'publish_gold', 'FAILED', SYSUTCDATETIME(), "` — `"VALUES (:batch_id, 'publish_gold', 'FAILED', SYSUTCDATETIME(), "` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 209:** `"SYSUTCDATETIME(), :error_message)"` — `"SYSUTCDATETIME(), :error_message)"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 210:** `), — ),` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 211:** `{"batch_id": batch_id, "error_message": str(exc)[:2000]}`, — `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"batch_id"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"error_message"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)[:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2000` is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 212:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 213:** `except Exception:` — `except` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Exception` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 214:** `LOGGER.exception("Could not record failed publication audit row")` — `LOGGER.exception` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Could not record failed publication audit row"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 215:** `write_artifact(batch_id, "FAILED", results, str(exc))` — `write_artifact` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"FAILED"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; results is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; exc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 216:** raise — raise is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 217:** blank line used only to separate logical sections for readability.
- **Line 218:** blank line used only to separate logical sections for readability.
- **Line 219:** if __name__ == "__main__": — if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; __name__ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "__main__" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 220:** main() — main is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 15 lesson index](#)

File introduction — sql/sqlserver/validation/02_validate_warehouse.sql

Exact filename: 02_validate_warehouse.sql

Filename extension: .sql. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: sql/sqlserver/validation relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 15.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in sql/sqlserver/validation one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 02_validate_warehouse.sql.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 02_validate_warehouse.sql and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 15.F4.1 — Create the file `sql/sqlserver/validation/02_validate_warehouse.sql` from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\sqlserver\validation\02_validate_warehouse.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\sqlserver\validation\02_validate_warehouse.sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\sqlserver\validation\02_validate_warehouse.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\sql\sqlserver\validation\02_validate_warehouse.sql" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/sqlserver/validation/02_validate_warehouse.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `sql/sqlserver/validation`.

2. Right-click the editor tab and choose **Copy Path**; compare the ending with `sql/sqlserver/validation/02_validate_warehouse.sql`.
3. In PowerShell at the repository root, run `Test-Path ".\sql\sqlserver\validation\02_validate_warehouse.sql"` and require `True`.
4. Run `git status --short -- "sql/sqlserver/validation/02_validate_warehouse.sql"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `sql/sqlserver/validation/02_validate_warehouse.sql`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: `sql/sqlserver/validation/02_validate_warehouse.sql`

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 15.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: SQL Server / T-SQL
USE NorthstarRetail_DW;
GO

-- Duplicate fact grain should be zero.
SELECT order_item_id, COUNT_BIG(*) AS duplicate_count
FROM dw.fact_sales GROUP BY order_item_id HAVING COUNT_BIG(*) > 1;

-- Orphan dimension keys should be zero; key 0 is valid and means Unknown.
SELECT COUNT_BIG(*) AS orphan_customer_keys
FROM dw.fact_sales f LEFT JOIN dw.dim_customer d ON d.customer_key=f.customer_key
WHERE d.customer_key IS NULL;

-- SCD2 dimensions: no more than one current row per business key.
SELECT customer_number, COUNT_BIG(*) AS current_count
FROM dw.dim_customer WHERE is_current=1
GROUP BY customer_number HAVING COUNT_BIG(*) > 1;

SELECT sku, COUNT_BIG(*) AS current_count
FROM dw.dim_product WHERE is_current=1
GROUP BY sku HAVING COUNT_BIG(*) > 1;

-- Revenue identity: net sales equals gross minus discount.
SELECT COUNT_BIG(*) AS invalid_revenue_rows
FROM dw.fact_sales
WHERE ABS(net_sales_amount - (gross_sales_amount - discount_amount)) > 0.01;

-- Inspect the most recent ETL and reconciliation results.
SELECT TOP (20) * FROM audit.etl_batch ORDER BY started_at_utc DESC;
SELECT TOP (50) * FROM audit.reconciliation_result ORDER BY checked_at_utc DESC;
```

Line-by-line explanation

- **Line 1:** `-- Dialect: SQL Server / T-SQL` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `USE NorthstarRetail_DW;` — `USE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NorthstarRetail_DW` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `GO` — `GO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** `-- Duplicate fact grain should be zero.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** `SELECT order_item_id, COUNT_BIG(*) AS duplicate_count` — `SELECT` requests data without changing it unless combined with a modifying statement; `order_item_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `COUNT_BIG` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `duplicate_count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `FROM dw.fact_sales GROUP BY order_item_id HAVING COUNT_BIG(*) > 1;` — `FROM` identifies the source table or query; `dw.fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in

the surrounding explanation; GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_item_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; HAVING is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; > is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression

- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** -- Orphan dimension keys should be zero; key 0 is valid and means Unknown. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 10:** SELECT COUNT_BIG(*) AS orphan_customer_keys — SELECT requests data without changing it unless combined with a modifying statement; COUNT_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; orphan_customer_keys is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** FROM dw.fact_sales f LEFT JOIN dw.dim_customer d ON d.customer_key=f.customer_key — FROM identifies the source table or query; dw.fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; LEFT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim_customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; d is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; d.customer_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.customer_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** WHERE d.customer_key IS NULL; — WHERE filters rows to those meeting a condition; d.customer_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** blank line used only to separate logical sections for readability.
- **Line 14:** -- SCD2 dimensions: no more than one current row per business key. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 15:** SELECT customer_number, COUNT_BIG(*) AS current_count — SELECT requests data without changing it unless combined with a modifying statement; customer_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; current_count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** FROM dw.dim_customer WHERE is_current=1 — FROM identifies the source table or query; dw.dim_customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; is_current is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression
- **Line 17:** GROUP BY customer_number HAVING COUNT_BIG(*) > 1; — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; HAVING is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tion; COUNT_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; > is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression

- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** SELECT sku, COUNT_BIG(*) AS current_count — SELECT requests data without changing it unless combined with a modifying statement; sku is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; current_count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** FROM dw.dim_product WHERE is_current=1 — FROM identifies the source table or query; dw.dim_product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHERE filters rows to those meeting a condition; is_current is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression
- **Line 21:** GROUP BY sku HAVING COUNT_BIG(*) > 1; — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sku is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; HAVING is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; > is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression
- **Line 22:** blank line used only to separate logical sections for readability.
- **Line 23:** -- Revenue identity: net sales equals gross minus discount. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 24:** SELECT COUNT_BIG(*) AS invalid_revenue_rows — SELECT requests data without changing it unless combined with a modifying statement; COUNT_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; invalid_revenue_rows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** FROM dw.fact_sales — FROM identifies the source table or query; dw.fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** WHERE ABS(net_sales_amount - (gross_sales_amount - discount_amount)) > 0.01; — WHERE filters rows to those meeting a condition; ABS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; net_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; - is a command option or flag; its exact behavior is defined beside the command; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gross_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; - is a command option or flag; its exact behavior is defined beside the command; discount_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;)) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; > is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0.01 is a numeric literal used by the command or expression
- **Line 27:** blank line used only to separate logical sections for readability.
- **Line 28:** -- Inspect the most recent ETL and reconciliation results. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 29:** SELECT TOP (20) * FROM audit.etl_batch ORDER BY started_at_utc DESC; — SELECT requests data without changing it unless combined with a modifying statement; TOP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 20 is a numeric literal used by the

command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; * is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; audit.etl_batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; started_at_utc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DESC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 30:** SELECT TOP (50) * FROM audit.reconciliation_result ORDER BY checked_at_utc DESC; — SELECT requests data without changing it unless combined with a modifying statement; TOP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 50 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; * is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; audit.reconciliation_result is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; checked_at_utc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DESC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 15 lesson index](#)

File introduction — sql/sqlserver/warehouse/03_semantic_views.sql

Exact filename: 03_semantic_views.sql

Filename extension: .sql. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: sql/sqlserver/warehouse relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: database or lakehouse definitions and queries for Phase 15.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in sql/sqlserver/warehouse one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: 03_semantic_views.sql.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled 03_semantic_views.sql and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 15.F5.1 — Create the file sql/sqlserver/warehouse/03_semantic_views.sql from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\sqlserver\warehouse\03_semantic_views.sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType File -Path ".\sql\sqlserver\warehouse\03_semantic_views.sql" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; File is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\sql\sqlserver\warehouse\03_semantic_views.sql" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\sql\sqlserver\warehouse\03_semantic_views.sql" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/sqlserver/warehouse/03_semantic_views.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `sql/sqlserver/warehouse`.

2. Right-click the editor tab and choose **Copy Path**; compare the ending with `sql/sqlserver/warehouse/03_semantic_views.sql`.
3. In PowerShell at the repository root, run `Test-Path ".\sql\sqlserver\warehouse\03_semantic_views.sql"` and require True.
4. Run `git status --short -- "sql/sqlserver/warehouse/03_semantic_views.sql"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `sql/sqlserver/warehouse/03_semantic_views.sql`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: SQL

Exact file: `sql/sqlserver/warehouse/03_semantic_views.sql`

Purpose: database or lakehouse definitions and queries.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 15.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: SQL Server / T-SQL
-- Purpose: thin semantic access views for analysts and Power BI.
USE NorthstarRetail_DW;
GO

CREATE OR ALTER VIEW semantic.vw_sales_detail AS
SELECT
    fs.order_number,
    dd.full_date AS order_date,
    dc.customer_number,
    CONCAT(dc.first_name, ' ', dc.last_name) AS customer_name,
    dp.sku,
    dp.product_name,
    dp.category_name,
    ds.store_name,
    ds.region,
    dch.channel_name,
    fs.quantity,
    fs.net_sales_amount,
    fs.cost_of_goods_sold,
    fs.gross_profit_amount,
    fs.discount_amount,
    fs.tax_amount
FROM dw.fact_sales AS fs
JOIN dw.dim_date AS dd ON dd.date_key = fs.order_date_key
JOIN dw.dim_customer AS dc ON dc.customer_key = fs.customer_key
JOIN dw.dim_product AS dp ON dp.product_key = fs.product_key
JOIN dw.dim_store AS ds ON ds.store_key = fs.store_key
JOIN dw.dim_channel AS dch ON dch.channel_key = fs.channel_key;
GO

CREATE OR ALTER VIEW semantic.vw_executive_daily_kpi AS
SELECT
    dd.full_date,
    SUM(fs.net_sales_amount) AS total_revenue,
    SUM(fs.gross_profit_amount) AS gross_profit,
    COUNT(DISTINCT fs.order_id) AS orders,
    SUM(fs.quantity) AS units_sold,
    CAST(SUM(fs.net_sales_amount) / NULLIF(COUNT(DISTINCT fs.order_id), 0) AS DECIMAL(18,2)) AS average_order_value
FROM dw.fact_sales AS fs
JOIN dw.dim_date AS dd ON dd.date_key = fs.order_date_key
GROUP BY dd.full_date;
GO

CREATE OR ALTER VIEW semantic.vw_data_quality_monitor AS
SELECT batch_id, check_name, source_value, target_value, difference_value, status, checked_at_utc
FROM audit.reconciliation_result;
GO
```

Line-by-line explanation

- **Line 1:** `-- Dialect: SQL Server / T-SQL` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `-- Purpose: thin semantic access views for analysts and Power BI.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** `USE NorthstarRetail_DW;` — `USE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NorthstarRetail_DW` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `GO` — `GO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** CREATE OR ALTER VIEW semantic.vw_sales_detail AS — CREATE defines a new database object; OR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ALTER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VIEW is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; semantic.vw_sales_detail is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** SELECT — SELECT requests data without changing it unless combined with a modifying statement
- **Line 8:** fs.order_number, — fs.order_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** dd.full_date AS order_date, — dd.full_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** dc.customer_number, — dc.customer_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** CONCAT(dc.first_name, ' ', dc.last_name) AS customer_name, — CONCAT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dc.first_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ' ' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dc.last_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** dp.sku, — dp.sku is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** dp.product_name, — dp.product_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** dp.category_name, — dp.category_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** ds.store_name, — ds.store_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** ds.region, — ds.region is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** dch.channel_name, — dch.channel_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** fs.quantity, — fs.quantity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** fs.net_sales_amount, — fs.net_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

planation; dch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dch.channel_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fs.channel_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 30:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** blank line used only to separate logical sections for readability.
- **Line 32:** CREATE OR ALTER VIEW semantic.vw_executive_daily_kpi AS — CREATE defines a new database object; OR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ALTER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VIEW is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; semantic.vw_executive_daily_kpi is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** SELECT — SELECT requests data without changing it unless combined with a modifying statement
- **Line 34:** dd.full_date, — dd.full_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** SUM(fs.net_sales_amount) AS total_revenue, — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fs.net_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; total_revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 36:** SUM(fs.gross_profit_amount) AS gross_profit, — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fs.gross_profit_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gross_profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 37:** COUNT(DISTINCT fs.order_id) AS orders, — COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DISTINCT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fs.order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 38:** SUM(fs.quantity) AS units_sold, — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fs.quantity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; units_sold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 39:** CAST(SUM(fs.net_sales_amount) / NULLIF(COUNT(DISTINCT fs.order_id), 0) AS DECIMAL(18,2)) AS average_order_value — CAST is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

fs.net_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; NULLIF is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DISTINCT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 40:** FROM dw.fact_sales AS fs — FROM identifies the source table or query; dw.fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** JOIN dw.dim_date AS dd ON dd.date_key = fs.order_date_key — JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dd is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dd.date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fs.order_date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** GROUP BY dd.full_date; — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dd.full_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 43:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 44:** blank line used only to separate logical sections for readability.
- **Line 45:** CREATE OR ALTER VIEW semantic.vw_data_quality_monitor AS — CREATE defines a new database object; OR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ALTER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VIEW is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; semantic.vw_data_quality_monitor is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** SELECT batch_id, check_name, source_value, target_value, difference_value, status, checked_at_utc — SELECT requests data without changing it unless combined with a modifying statement; batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; check_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source_value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; target_value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; difference_value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; status shows tracked, staged, modified, and untracked work; remaining punctuation and identifiers continue the same statement.
- **Line 47:** FROM audit.reconciliation_result; — FROM identifies the source table or query; audit.reconciliation_result is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 48:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

What your screen should look like now

- Every Phase 15 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 15.3 completion checklist

☐

sql/sqlserver/warehouse/01_create_warehouse.sql was created by me, saved, located correctly, and inspected — [STATUS]

☐

sql/sqlserver/warehouse/02_create_staging_tables.sql was created by me, saved, located correctly, and inspected — [STATUS]

☐

scripts/loading/load_gold_to_sqlserver.py was created by me, saved, located correctly, and inspected — [STATUS]

☐

sql/sqlserver/validation/02_validate_warehouse.sql was created by me, saved, located correctly, and inspected — [STATUS]

☐

sql/sqlserver/warehouse/03_semantic_views.sql was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 15.4](#) and recheck the root and branch.

Lesson 15.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

SQL Server warehouse object 1 — `dw.dim_date`

Object type: dimension.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: `load_batch_id`, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W1 — Create, load, and validate `dw.dim_date`

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.dim_date. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_date.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_date.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.dim_date', N'U') IS NULL
BEGIN
    CREATE TABLE dw.dim_date
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.dim_date', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.dim_date' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.dim_date — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.dim_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.dim_date through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 2 — dw.dim_customer

Object type: dimension.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W2 — Create, load, and validate dw.dim_customer

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.dim_customer. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_customer.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_customer.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.dim_customer', N'U') IS NULL
BEGIN
    CREATE TABLE dw.dim_customer
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.dim_customer', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.dim_customer' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.dim_customer — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.dim_customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.dim_customer through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 3 — dw.dim_product

Object type: dimension.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W3 — Create, load, and validate dw.dim_product

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.dim_product. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_product.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_product.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.dim_product', N'U') IS NULL
BEGIN
    CREATE TABLE dw.dim_product
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.dim_product', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.dim_product' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.dim_product — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.dim_product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.dim_product through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 4 — dw.dim_store

Object type: dimension.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W4 — Create, load, and validate dw.dim_store

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.dim_store. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_store.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_store.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.dim_store', N'U') IS NULL
BEGIN
    CREATE TABLE dw.dim_store
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.dim_store', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.dim_store' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.dim_store — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.dim_store is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.dim_store through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 5 — dw.dim_employee

Object type: dimension.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W5 — Create, load, and validate dw.dim_employee

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.dim_employee. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_employee.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_employee.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.dim_employee', N'U') IS NULL
BEGIN
    CREATE TABLE dw.dim_employee
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.dim_employee', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.dim_employee' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.dim_employee — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.dim_employee is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.dim_employee through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 6 — dw.dim_supplier

Object type: dimension.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W6 — Create, load, and validate dw.dim_supplier

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.dim_supplier. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_supplier.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_supplier.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.dim_supplier', N'U') IS NULL
BEGIN
    CREATE TABLE dw.dim_supplier
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.dim_supplier', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.dim_supplier' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.dim_supplier — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.dim_supplier is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.dim_supplier through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 7 — dw.dim_promotion

Object type: dimension.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W7 — Create, load, and validate dw.dim_promotion

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.dim_promotion. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_promotion.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_promotion.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.dim_promotion', N'U') IS NULL
BEGIN
    CREATE TABLE dw.dim_promotion
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.dim_promotion', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.dim_promotion' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.dim_promotion — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.dim_promotion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.dim_promotion through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 8 — dw.dim_channel

Object type: dimension.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W8 — Create, load, and validate dw.dim_channel

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.dim_channel. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_channel.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_channel.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.dim_channel', N'U') IS NULL
BEGIN
    CREATE TABLE dw.dim_channel
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.dim_channel', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.dim_channel' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.dim_channel — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.dim_channel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.dim_channel through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 9 — dw.dim_campaign

Object type: dimension.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W9 — Create, load, and validate dw.dim_campaign

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.dim_campaign. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_campaign.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.dim_campaign.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.dim_campaign', N'U') IS NULL
BEGIN
    CREATE TABLE dw.dim_campaign
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.dim_campaign', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.dim_campaign' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.dim_campaign — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.dim_campaign is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.dim_campaign through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 10 — dw.fact_sales

Object type: fact.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W10 — Create, load, and validate dw.fact_sales

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.fact_sales. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.fact_sales.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.fact_sales.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.fact_sales', N'U') IS NULL
BEGIN
    CREATE TABLE dw.fact_sales
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.fact_sales', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.fact_sales' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.fact_sales — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.fact_sales through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 11 — dw.fact_returns

Object type: fact.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W11 — Create, load, and validate dw.fact_returns

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.fact_returns. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.fact_returns.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.fact_returns.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.fact_returns', N'U') IS NULL
BEGIN
    CREATE TABLE dw.fact_returns
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.fact_returns', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.fact_returns' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.fact_returns — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.fact_returns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.fact_returns through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 12 — dw.fact_inventory_snapshot

Object type: fact.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W12 — Create, load, and validate dw.fact_inventory_snapshot

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.fact_inventory_snapshot. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.fact_inventory_snapshot.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.fact_inventory_snapshot.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.fact_inventory_snapshot', N'U') IS NULL
BEGIN
    CREATE TABLE dw.fact_inventory_snapshot
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.fact_inventory_snapshot', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.fact_inventory_snapshot' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.fact_inventory_snapshot — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.fact_inventory_snapshot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.fact_inventory_snapshot through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 13 — dw.fact_shipments

Object type: fact.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W13 — Create, load, and validate dw.fact_shipments

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.fact_shipments. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.fact_shipments.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.fact_shipments.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.fact_shipments', N'U') IS NULL
BEGIN
    CREATE TABLE dw.fact_shipments
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.fact_shipments', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.fact_shipments' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.fact_shipments — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.fact_shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.fact_shipments through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 14 — dw.fact_web_sessions

Object type: fact.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W14 — Create, load, and validate dw.fact_web_sessions

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.fact_web_sessions. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.fact_web_sessions.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.fact_web_sessions.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.fact_web_sessions', N'U') IS NULL
BEGIN
    CREATE TABLE dw.fact_web_sessions
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- Line 1:** `IF OBJECT_ID(N'dw.fact_web_sessions', N'U') IS NULL` — `IF` begins a conditional decision; `OBJECT_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'dw.fact_web_sessions'` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'U'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `BEGIN` — `BEGIN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `CREATE TABLE dw.fact_web_sessions` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `dw.fact_web_sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `(` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 5:** `-- TODO(USER): add the mapped columns and SQL Server data types.` is a comment for a human reader; it normally does not change data or execute business logic.
- Line 6:** `load_batch_id VARCHAR(100) NOT NULL,` — `load_batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `100` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.fact_web_sessions through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 15 — dw.fact_marketing_spend

Object type: fact.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W15 — Create, load, and validate dw.fact_marketing_spend

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.fact_marketing_spend. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.fact_marketing_spend.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.fact_marketing_spend.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.fact_marketing_spend', N'U') IS NULL
BEGIN
    CREATE TABLE dw.fact_marketing_spend
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.fact_marketing_spend', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.fact_marketing_spend' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.fact_marketing_spend — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.fact_marketing_spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.fact_marketing_spend through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 16 — dw.load_batch

Object type: audit.

Responsibility: Preserve or present warehouse metadata and loaded tables in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W16 — Create, load, and validate dw.load_batch

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.load_batch. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.load_batch.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.load_batch.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.load_batch', N'U') IS NULL
BEGIN
    CREATE TABLE dw.load_batch
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.load_batch', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.load_batch' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.load_batch — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.load_batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw_load_batch through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 17 — dw.quality_result

Object type: audit.

Responsibility: Preserve or present warehouse metadata and loaded tables in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W17 — Create, load, and validate dw.quality_result

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.quality_result. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.quality_result.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.quality_result.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.quality_result', N'U') IS NULL
BEGIN
    CREATE TABLE dw.quality_result
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.quality_result', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.quality_result' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.quality_result — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.quality_result is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.quality_result through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 18 — dw.reconciliation_result

Object type: audit.

Responsibility: Preserve or present warehouse metadata and loaded tables in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W18 — Create, load, and validate dw.reconciliation_result

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate dw.reconciliation_result. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.reconciliation_result.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns dw.reconciliation_result.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'dw.reconciliation_result', N'U') IS NULL
BEGIN
    CREATE TABLE dw.reconciliation_result
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'dw.reconciliation_result', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'dw.reconciliation_result' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE dw.reconciliation_result — CREATE defines a new database object; TABLE the database object that stores rows and columns; dw.reconciliation_result is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published dw.reconciliation_result through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 19 — stg.dim_date

Object type: staging.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W19 — Create, load, and validate stg.dim_date

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate `stg.dim_date`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database `NorthstarRetail_DW` in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.dim_date`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database `NorthstarRetail_DW` in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.dim_date`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'stg.dim_date', N'U') IS NULL
BEGIN
    CREATE TABLE stg.dim_date
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'stg.dim_date', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stg.dim_date' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE stg.dim_date — CREATE defines a new database object; TABLE the database object that stores rows and columns; stg.dim_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published stg.dim_date through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 20 — stg.dim_customer

Object type: staging.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W20 — Create, load, and validate stg.dim_customer

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate stg.dim_customer. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns stg.dim_customer.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns stg.dim_customer.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'stg.dim_customer', N'U') IS NULL
BEGIN
    CREATE TABLE stg.dim_customer
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'stg.dim_customer', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stg.dim_customer' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE stg.dim_customer — CREATE defines a new database object; TABLE the database object that stores rows and columns; stg.dim_customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published stg.dim_customer through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 21 — stg.dim_product

Object type: staging.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W21 — Create, load, and validate stg.dim_product

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate stg.dim_product. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns stg.dim_product.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns stg.dim_product.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'stg.dim_product', N'U') IS NULL
BEGIN
    CREATE TABLE stg.dim_product
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'stg.dim_product', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stg.dim_product' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE stg.dim_product — CREATE defines a new database object; TABLE the database object that stores rows and columns; stg.dim_product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published stg.dim_product through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 22 — stg.dim_store

Object type: staging.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W22 — Create, load, and validate stg.dim_store

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate `stg.dim_store`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database `NorthstarRetail_DW` in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.dim_store`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database `NorthstarRetail_DW` in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.dim_store`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'stg.dim_store', N'U') IS NULL
BEGIN
    CREATE TABLE stg.dim_store
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'stg.dim_store', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stg.dim_store' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE stg.dim_store — CREATE defines a new database object; TABLE the database object that stores rows and columns; stg.dim_store is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DE-FAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published stg.dim_store through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 23 — stg.dim_employee

Object type: staging.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W23 — Create, load, and validate stg.dim_employee

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate `stg.dim_employee`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database `NorthstarRetail_DW` in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.dim_employee`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database `NorthstarRetail_DW` in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.dim_employee`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'stg.dim_employee', N'U') IS NULL
BEGIN
    CREATE TABLE stg.dim_employee
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'stg.dim_employee', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stg.dim_employee' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE stg.dim_employee — CREATE defines a new database object; TABLE the database object that stores rows and columns; stg.dim_employee is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published stg.dim_employee through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 24 — stg.dim_supplier

Object type: staging.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W24 — Create, load, and validate stg.dim_supplier

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate `stg.dim_supplier`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database `NorthstarRetail_DW` in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.dim_supplier`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database `NorthstarRetail_DW` in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.dim_supplier`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'stg.dim_supplier', N'U') IS NULL
BEGIN
    CREATE TABLE stg.dim_supplier
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'stg.dim_supplier', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stg.dim_supplier' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE stg.dim_supplier — CREATE defines a new database object; TABLE the database object that stores rows and columns; stg.dim_supplier is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published stg.dim_supplier through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 25 — stg.dim_promotion

Object type: staging.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W25 — Create, load, and validate stg.dim_promotion

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Create, load, and validate `stg.dim_promotion`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database `NorthstarRetail_DW` in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.dim_promotion`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database `NorthstarRetail_DW` in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.dim_promotion`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'stg.dim_promotion', N'U') IS NULL
BEGIN
    CREATE TABLE stg.dim_promotion
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'stg.dim_promotion', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ' stg.dim_promotion' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE stg.dim_promotion — CREATE defines a new database object; TABLE the database object that stores rows and columns; stg.dim_promotion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published stg.dim_promotion through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 26 — stg.dim_channel

Object type: staging.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W26 — Create, load, and validate stg.dim_channel

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate `stg.dim_channel`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database `NorthstarRetail_DW` in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.dim_channel`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database `NorthstarRetail_DW` in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.dim_channel`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'stg.dim_channel', N'U') IS NULL
BEGIN
    CREATE TABLE stg.dim_channel
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'stg.dim_channel', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stg.dim_channel' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE stg.dim_channel — CREATE defines a new database object; TABLE the database object that stores rows and columns; stg.dim_channel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published stg.dim_channel through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 27 — stg.dim_campaign

Object type: staging.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W27 — Create, load, and validate stg.dim_campaign

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate stg.dim_campaign. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns stg.dim_campaign.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns stg.dim_campaign.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'stg.dim_campaign', N'U') IS NULL
BEGIN
    CREATE TABLE stg.dim_campaign
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'stg.dim_campaign', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stg.dim_campaign' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE stg.dim_campaign — CREATE defines a new database object; TABLE the database object that stores rows and columns; stg.dim_campaign is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published stg.dim_campaign through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 28 — stg.fact_sales

Object type: staging.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W28 — Create, load, and validate stg.fact_sales

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate stg.fact_sales. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns stg.fact_sales.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns stg.fact_sales.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'stg.fact_sales', N'U') IS NULL
BEGIN
    CREATE TABLE stg.fact_sales
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'stg.fact_sales', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stg.fact_sales' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE stg.fact_sales — CREATE defines a new database object; TABLE the database object that stores rows and columns; stg.fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published stg.fact_sales through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 29 — stg.fact_returns

Object type: staging.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W29 — Create, load, and validate stg.fact_returns

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate stg.fact_returns. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns stg.fact_returns.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns stg.fact_returns.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'stg.fact_returns', N'U') IS NULL
BEGIN
    CREATE TABLE stg.fact_returns
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'stg.fact_returns', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stg.fact_returns' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE stg.fact_returns — CREATE defines a new database object; TABLE the database object that stores rows and columns; stg.fact_returns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published stg.fact_ returns through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 30 — stg.fact_inventory_snapshot

Object type: staging.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W30 — Create, load, and validate stg.fact_inventory_snapshot

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate stg.fact_inventory_snapshot. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns stg.fact_inventory_snapshot.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns stg.fact_inventory_snapshot.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'stg.fact_inventory_snapshot', N'U') IS NULL
BEGIN
    CREATE TABLE stg.fact_inventory_snapshot
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'stg.fact_inventory_snapshot', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stg.fact_inventory_snapshot' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE stg.fact_inventory_snapshot — CREATE defines a new database object; TABLE the database object that stores rows and columns; stg.fact_inventory_snapshot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published stg.fact_inventory_snapshot through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 31 — stg.fact_shipments

Object type: staging.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W31 — Create, load, and validate stg.fact_shipments

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate `stg.fact_shipments`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database `NorthstarRetail_DW` in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.fact_shipments`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database `NorthstarRetail_DW` in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.fact_shipments`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'stg.fact_shipments', N'U') IS NULL
BEGIN
    CREATE TABLE stg.fact_shipments
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'stg.fact_shipments', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ' stg.fact_shipments' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE stg.fact_shipments — CREATE defines a new database object; TABLE the database object that stores rows and columns; stg.fact_shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published stg.fact_shipments through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 32 — stg.fact_web_sessions

Object type: staging.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W32 — Create, load, and validate stg.fact_web_sessions

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate `stg.fact_web_sessions`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.fact_web_sessions`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns `stg.fact_web_sessions`.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'stg.fact_web_sessions', N'U') IS NULL
BEGIN
    CREATE TABLE stg.fact_web_sessions
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'stg.fact_web_sessions', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ' stg.fact_web_sessions' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE stg.fact_web_sessions — CREATE defines a new database object; TABLE the database object that stores rows and columns; stg.fact_web_sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published stg.fact_web_sessions through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 33 — stg.fact_marketing_spend

Object type: staging.

Responsibility: Preserve or present exported Gold file for the verified batch in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W33 — Create, load, and validate stg.fact_marketing_spend

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate stg.fact_marketing_spend. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns stg.fact_marketing_spend.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns stg.fact_marketing_spend.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'stg.fact_marketing_spend', N'U') IS NULL
BEGIN
    CREATE TABLE stg.fact_marketing_spend
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'stg.fact_marketing_spend', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'stg.fact_marketing_spend' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE stg.fact_marketing_spend — CREATE defines a new database object; TABLE the database object that stores rows and columns; stg.fact_marketing_spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published stg.fact_marketing_spend through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 34 — reporting.vw_sales

Object type: semantic view.

Responsibility: Preserve or present warehouse metadata and loaded tables in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W34 — Create, load, and validate reporting.vw_sales

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate reporting.vw_sales. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns reporting.vw_sales.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns reporting.vw_sales.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'reporting.vw_sales', N'U') IS NULL
BEGIN
    CREATE TABLE reporting.vw_sales
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'reporting.vw_sales', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ' reporting.vw_sales ' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE reporting.vw_sales — CREATE defines a new database object; TABLE the database object that stores rows and columns; reporting.vw_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published reporting.vw_sales through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 35 — reporting.vw_returns

Object type: semantic view.

Responsibility: Preserve or present warehouse metadata and loaded tables in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W35 — Create, load, and validate reporting.vw_returns

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate reporting.vw_returns. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns reporting.vw_returns.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns reporting.vw_returns.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'reporting.vw_returns', N'U') IS NULL
BEGIN
    CREATE TABLE reporting.vw_returns
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- Line 1:** `IF OBJECT_ID(N'reporting.vw_returns', N'U') IS NULL` — `IF` begins a conditional decision; `OBJECT_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'reporting.vw_returns'` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'U'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 2:** `BEGIN` — `BEGIN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** `CREATE TABLE reporting.vw_returns` — `CREATE` defines a new database object; `TABLE` the database object that stores rows and columns; `reporting.vw_returns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 4:** `(` — `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 5:** `-- TODO(USER): add the mapped columns and SQL Server data types.` is a comment for a human reader; it normally does not change data or execute business logic.
- Line 6:** `load_batch_id VARCHAR(100) NOT NULL,` — `load_batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VARCHAR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `100` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOT` reverses a Boolean condition; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published reporting.vw_returns through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 36 — reporting.vw_inventory

Object type: semantic view.

Responsibility: Preserve or present warehouse metadata and loaded tables in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W36 — Create, load, and validate reporting.vw_inventory

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate reporting.vw_inventory. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns reporting.vw_inventory.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns reporting.vw_inventory.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'reporting.vw_inventory', N'U') IS NULL
BEGIN
    CREATE TABLE reporting.vw_inventory
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'reporting.vw_inventory', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ' reporting.vw_inventory' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE reporting.vw_inventory — CREATE defines a new database object; TABLE the database object that stores rows and columns; reporting.vw_inventory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published reporting.vw_inventory through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 37 — reporting.vw_shipments

Object type: semantic view.

Responsibility: Preserve or present warehouse metadata and loaded tables in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W37 — Create, load, and validate reporting.vw_shipments

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate reporting.vw_shipments. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns reporting.vw_shipments.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns reporting.vw_shipments.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'reporting.vw_shipments', N'U') IS NULL
BEGIN
    CREATE TABLE reporting.vw_shipments
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'reporting.vw_shipments', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ' reporting.vw_shipments' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE reporting.vw_shipments — CREATE defines a new database object; TABLE the database object that stores rows and columns; reporting.vw_shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published reporting.vw_shipments through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

SQL Server warehouse object 38 — reporting.vw_marketing

Object type: semantic view.

Responsibility: Preserve or present warehouse metadata and loaded tables in the local dimensional warehouse without inventing direct cloud-to-laptop connectivity.

Grain: Match the corresponding Gold object exactly for dimensions/facts; staging retains one row per exported record; audit tables retain one row per load/check event.

Keys and constraints: Dimensions use surrogate primary keys and business-key/history rules; facts use foreign keys/indexes appropriate to the declared grain; staging favors load/reconciliation metadata before final constraints.

Load metadata: load_batch_id, source filename/checksum, loaded timestamp, row count, and load status.

Rerun rule: The loader identifies the batch and target state before inserting, replacing, or rejecting a duplicate load.

Action 15.4.W38 — Create, load, and validate reporting.vw_marketing

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Create, load, and validate reporting.vw_marketing. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns reporting.vw_marketing.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open SSMS and connect to the verified local SQL Server instance.
2. LOOK FOR: Select database NorthstarRetail_DW in the query-window database list.
3. CLICK: Open the warehouse DDL file that owns reporting.vw_marketing.
4. TYPE THIS: Build the object from the mapped Gold columns, keys, constraints, audit metadata, and declared grain.
5. PRESS: Save the SQL file.
6. PRESS: Execute the object section or the verified ordered script.
7. CLICK: Refresh the target schema in Object Explorer.
8. LOOK FOR: Inspect Columns, Keys, Constraints, and Indexes.
9. CLICK: In File Explorer, locate the current batch's downloaded/exported Gold file; do not use an older batch with the same table name.
10. PRESS: Run the local loader for the staging table or object after checksum/count verification.
11. PRESS: Run row-count, duplicate-key, foreign-key, unknown-member, and amount reconciliation queries.
12. DO NOT CONTINUE UNTIL: The warehouse result matches the exported Gold result within documented tolerance.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 15 lesson index](#)

Teaching DDL skeleton

```
IF OBJECT_ID(N'reporting.vw_marketing', N'U') IS NULL
BEGIN
    CREATE TABLE reporting.vw_marketing
    (
        -- TODO(USER): add the mapped columns and SQL Server data types.
        load_batch_id VARCHAR(100) NOT NULL,
        loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO
```

Line-by-line explanation

- **Line 1:** IF OBJECT_ID(N'reporting.vw_marketing', N'U') IS NULL — IF begins a conditional decision; OBJECT_ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ' reporting.vw_marketing' is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; N is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'U' is a quoted literal; the quotation marks preserve spaces or mark text;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** BEGIN — BEGIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** CREATE TABLE reporting.vw_marketing — CREATE defines a new database object; TABLE the database object that stores rows and columns; reporting.vw_marketing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** (— (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** -- TODO(USER): add the mapped columns and SQL Server data types. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** load_batch_id VARCHAR(100) NOT NULL, — load_batch_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VARCHAR is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 7:** loaded_at DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME() — loaded_at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DATETIME2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NOT reverses a Boolean condition; NULL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DEFAULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SYSUTCDATETIME is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:**); —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** END; — END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Validation matrix

Validation	Exported Gold	SQL Server warehouse	Pass condition	My result
Row count	[ENTER]	[ENTER]	exact match	[ENTER]
Distinct grain key	[ENTER]	[ENTER]	exact match	[ENTER]
Primary amount/quantity	[ENTER]	[ENTER]	documented tolerance	[ENTER]
Unknown key count	[ENTER]	[ENTER]	exact match	[ENTER]
Duplicate load of same batch	n/a	[ENTER]	prevented or idempotent	[ENTER]

Common error and recovery

If a bulk/load operation truncates text, changes decimals, parses dates incorrectly, or fails on constraints, stop the transaction. Compare exported schema/values with staging types, preserve the failed batch, correct the mapping, and rerun the staging load. Do not disable constraints globally or edit the exported evidence file.

Interview-ready explanation

"I published reporting.vw_marketing through an honest Track A file handoff, loaded staging with batch metadata, applied dimensional constraints, and reconciled row counts and measures back to the verified Gold export."

What your screen should look like now

- The new Phase 15 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 15.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]



I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 15.5](#).

Lesson 15.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 15 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Local export-and-load versus direct JDBC publication

Question:

How should cloud Gold data reach laptop SQL Server?

Options considered:

- Manual verified export and local load.
- Direct Databricks JDBC to localhost.
- Cloud warehouse target.

Beginner path selected:

Use manual export-and-load for Track A.

Why this path was selected:

A cloud workspace cannot normally reach a laptop loopback address; the selected method is honest and reproducible.

Tradeoff:

The handoff is not fully automated.

What a company might do instead:

A company uses routable private connectivity or publishes to a managed cloud warehouse.

How to explain this in an interview:

I avoided false localhost connectivity and used a controlled export with row-count and checksum reconciliation before local warehouse loading.

If you see Result A

Export manifest, staging counts, warehouse counts, and control totals reconcile. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

A file/table is missing, the local loader partially fails, or totals differ across the handoff. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate



Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: Databricks, File Explorer, PowerShell, SSMS, and Python.

Window/panel: Show export manifest, staging load, warehouse tables, and reconciliation query.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase15_publish-gold-data-into-the-sql-server-dimensional-wareh_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 15's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 15 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. **PRESS:** In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 15.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. **CLICK:** Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. **LOOK FOR:** Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. **CLICK:** Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. **TYPE THIS:** Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. **PRESS:** Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. **LOOK FOR:** Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. **DO NOT CONTINUE UNTIL:** Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Action 15.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use

git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Action 15.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Interpret `git status --short`

- `??` means untracked: Git sees a new path but it is not staged.
- `M` means modified in the working tree but not staged.
- `M` means the staged version differs from the last commit.
- `MM` means a staged version exists and the file has additional unstaged edits.
- `A` means staged as a new file.
- `D` means deletion; stop and investigate unless deletion is deliberate.
- `R` means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 15.5.G4 — Inspect the unstaged diff for sql/sqlserver/warehouse/01_create_warehouse.sql

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "sql/sqlserver/warehouse/01_create_warehouse.sql"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "sql/sqlserver/warehouse/01_create_warehouse.sql"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"sql/sqlserver/warehouse/01_create_warehouse.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Action 15.5.G5 — Stage only the reviewed file `sql/sqlserver/warehouse/01_create_warehouse.sql`

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "sql/sqlserver/warehouse/01_create_warehouse.sql"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "sql/sqlserver/warehouse/01_create_warehouse.sql"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"sql/sqlserver/warehouse/01_create_warehouse.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 15.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Action 15.5.G7 — Create the Phase 15 commit

Application:

SQL Server Management Studio (SSMS)

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.

2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "feat: publish gold into sql server warehouse"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "feat: publish gold into sql server warehouse"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"feat: publish gold into sql server warehouse"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Action 15.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Action 15.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

Usually the repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` plus the database named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches Usually the repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project plus the database named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 15 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message feat: publish gold into sql server warehouse and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 15.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 15 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

“Phase 15 created the verified boundary described by **Publish Gold data into the SQL Server dimensional warehouse**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed.”

Mini quiz

1. Multiple choice: Why does Phase 15 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.

4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

My Phase 15 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.

- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 15.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 16 imports trusted tables, builds relationships, and defines DAX measures.

Phase 15 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Phase 16 — Build and validate the Power BI semantic model

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: A validated star-schema semantic model and measures.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 17 uses those governed fields to construct report pages and visual interactions.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 16 lesson index

1. [Lesson 16.1 — Understand the phase and starting screen](#)
2. [Lesson 16.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 16.3 — Create and understand the phase files](#)
4. [Lesson 16.4 — Build and validate required objects](#)
5. [Lesson 16.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 16.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** Power BI Desktop, Power Query Editor, Model view, DAX formula bar, and SSMS.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 16 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 15 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 16 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

Power BI Desktop, Power Query Editor, Model view, DAX formula bar, and SSMS. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 16 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 16 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 16 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 16.1 completion checklist

☐

I read why Phase 16 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 16.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 16 lesson index](#)

Lesson 16.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 16 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

A Power BI semantic model over NorthstarRetail_DW with a proper date table, one-to-many single-direction relationships, hidden technical columns, documented business measures, useful hierarchies, display folders, and a Databricks SQL view alternative. This workbook supplies exact model documentation and optional reference DAX; you create and verify the .pbix yourself so the portfolio evidence is genuinely yours.

2. Why it matters

The semantic layer gives every report the same business definitions. Without it, two analysts can calculate revenue, returns, or conversion differently even when they read the same warehouse tables.

3. Words to know

Term	Plain-language definition
Semantic model	Reusable tables, relationships, measures, formats, and business definitions used by reports.
DAX	Data Analysis Expressions, the formula language used for Power BI measures and calculated objects.
Measure	A calculation evaluated under the current report filters, such as Total Revenue.
Filter context	The active filters from slicers, rows, columns, relationships, and visual selections.
Import	Power BI copies data into its in-memory model during refresh.
DirectQuery	Power BI sends queries to the source when a visual needs data.
Cardinality	How many matching rows can exist on each side of a relationship.
Cross-filter direction	The direction in which a filter travels through a relationship.
Date table	A complete one-row-per-date dimension used for time intelligence.
Explicit measure	A named DAX calculation rather than an automatic column aggregation.
Display folder	A model organization label for related measures.
Hierarchy	An ordered drill path, such as Year > Quarter > Month > Date.
Default summarization	Whether a numeric column is summed automatically; keys should normally be Do not summarize.
Databricks SQL view	A saved SQL query that presents business-friendly columns as an alternate semantic access path.

4. What must already be completed

Phase 15 is verified: NorthstarRetail_DW exists, semantic views query successfully, dimensions and facts reconcile to the selected Gold batch, and the account you will use can read the dw or semantic schema.

5. Applications to open

Application	How to open it	Why it is open
Power BI Desktop	Windows Start menu > search Power BI Desktop > Open.	Create the model and report file.
SSMS	Open from Start and connect to the verified local SQL Server.	Validate every measure against SQL.
VS Code	Open the repository and powerbi/ folder.	Read the model contract and copy reviewed DAX one measure at a time.
Databricks SQL editor	Workspace > SQL > SQL Editor, if your tier exposes it.	Create/query the optional semantic views.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run Get-Location; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
SELECT SUM(net_sales_amount) AS total_revenue,	Creates the unfiltered SQL baseline for three core measures.	The semantic model must reproduce warehouse truth before visual design begins.
Total Revenue =	Defines a reusable revenue measure that responds to report filters.	An explicit measure is documented and behaves consistently across visuals.
Gross Margin Percentage =	Calculates margin safely and returns blank when the denominator is zero.	DIVIDE handles zero/blank denominators more safely than /.
SELECT SUM(f.net_sales_amount) AS filtered_revenue	Builds a filter-matched validation query.	An all-data match does not prove relationships or filter context work.
SELECT order_date_key,	Provides a simple alternate semantic query path over Gold.	It demonstrates that the business definition is portable across serving tools.

Import versus DirectQuery decision

For the small local Track A portfolio, Import is the default: it is simpler, fast for interactive visuals, and does not require the local SQL Server to answer every click. DirectQuery keeps data at the source but adds source performance, concurrency, transformation, and feature considerations. Record the decision rather than calling one mode universally better.

Relationship rule

A dimension has one row per key/version and filters a fact with many rows. Start with single-direction dimension-to-fact relationships. Bidirectional filtering can solve specific designs but can also create multiple filter paths and ambiguous totals; do not enable it as a quick fix.

Required measures

powerbi/measures.dax contains Total Revenue, Gross Profit, Gross Margin Percentage, Promotion Gross Profit, Orders, Units Sold, Average Order Value, Return Rate, On-Time Delivery Percentage, Marketing Spend, Return on Ad Spend, Conversion Rate, New Customers, Returning Customers, Inventory On Hand, and Stockout Risk Count. Each measure includes a business explanation and SQL validation method.

MY TASK

Build the Power BI model yourself, create all required relationships and measures, validate every measure under at least one nontrivial filter, and capture model/validation evidence without publishing fabricated KPI values.

STARTER SKELETON

```
-- TODO(USER): type this only after defining the business rule.
Average Order Value =
DIVIDE (
    [Total Revenue],
    [Orders]
)

-- TODO(USER): validate the numerator and denominator separately in SQL.
-- TODO(USER): set format and description in Model view.
-- TODO(USER): record the filtered test in your learning journal.
```

Acceptance criteria: All 15 required tables load; relationships match the contract with no unexplained ambiguity; dim_date is marked; technical columns are hidden; every required measure has format, description, display folder, and SQL validation evidence; the saved PBIX is labeled as your own verified work.

HINT 1 — CONCEPT

A measure is only trustworthy when its table grain, relationship path, and denominator are understood.

HINT 2 — PSEUDOCODE

Create base measures first, validate them, then create ratios with DIVIDE and validate filtered cases.

HINT 3 — NEAR-COMPLETE PATTERN

Use powerbi/measures.dax and sql/sqlserver/analytics/kpi_validation_queries.sql; copy one measure, place it in a simple visual, match filters in SQL, and record the difference.

13. Expected files or output

- A local Northstar_Retail_Portfolio.pbix created by you.
- Model view screenshot showing the star-schema relationships.
- 15 documented DAX measures with correct formats.
- Measure validation table with SQL result, DAX result, difference, status, and filter definition.
- Optional Databricks SQL semantic-view/query evidence.

14. How to verify

- Every dimension key is unique on the one side.
- No relationship is many-to-many unless explicitly justified.
- Cross-filter direction is Single by default.
- Dates filter all applicable facts through the intended date key.
- Every unfiltered and filtered DAX result matches SQL within documented rounding tolerance.
- A zero-denominator case returns blank or the documented safe value.
- No technical ID is accidentally summed in a visual.

15. Automated tests

- SQL-to-DAX baseline tests for all measures.

- One-date, date-range, channel, store, product, and no-data filter tests.
- Relationship uniqueness query for each dimension key.
- Ambiguous-path inspection in Manage relationships.
- Refresh test after a controlled warehouse reload.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
The relationship cannot be created because a column contains duplicate values	The supposed dimension key is not unique or an SCD2 business key was used instead of surrogate key.	Run <code>GROUP BY key HAVING COUNT(*) > 1</code> in SSMS.	Use the dimension surrogate key on the one side and fix duplicate dimension grain upstream.	The duplicate query returns zero and relationship is 1:*
Power BI detected ambiguous relationships	Multiple active paths or bidirectional filters connect the same tables.	Open Manage relationships and trace every active path.	Return to single-direction star relationships; use inactive role-playing dates plus explicit DAX only when required.	No ambiguity warning and filtered totals match SQL.
Measure total looks wrong	Ratio/average was calculated row-by-row or an implicit aggregation was used.	Show numerator and denominator measures beside the total; run matching SQL.	Create explicit base measures and use DIVIDE at the intended grain.	Detail rows and grand total both match their SQL definitions.
Time intelligence is blank or inconsistent	Date table is not marked, dates are missing, or relationship uses the wrong date key.	Inspect min/max dates and missing dates in <code>dim_date</code> ; check relationship.	Use a complete date dimension, mark it, and activate the intended date relationship.	Date slicer and monthly totals match SQL.
Refresh cannot connect	Wrong server/database, SQL service stopped, or credentials changed.	Confirm SSMS connection with the same server and database.	Start the service, correct data source settings, and re-enter credentials through Power BI UI.	Refresh completes and row counts remain reconciled.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why is a semantic model more than a set of charts?
2. Why does filter context change a measure result?
3. Why is a surrogate key used in fact-to-dimension relationships?
4. When might DirectQuery be chosen instead of Import?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. Which cross-filter direction is the default for this star schema?
2. Why should Gross Margin Percentage use DIVIDE?
3. What must be identical between a DAX validation and its SQL query?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture Model view with all relationship lines, the marked date-table property, the Measures display folders, and one DAX-versus-SQL validation. Hide machine/user names and credentials.

Save the image under `docs/images/` with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
feat: document and validate power bi semantic model
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

Connected to NorthstarRetail_DW — [STATUS]

☐

15 model tables loaded — [STATUS]

☐

Relationships created and checked — [STATUS]

☐

Date table marked and hierarchy created — [STATUS]

☐

Technical columns hidden — [STATUS]

☐

All required DAX measures created — [STATUS]

☐

Filtered SQL validations passed — [STATUS]

☐

Model screenshot captured — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-16. The quiz answer is in the answer-key section immediately after the quiz in this workbook#phase-16.

Next phase: Phase 17 turns the trusted semantic model into audience-specific report pages and analyst consumption examples.

Required action cards

Action 16.2.1 — Open powerbi/README.md, powerbi/semantic_model.md, and powerbi/measures.dax in VS Code. Read the table g

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Open powerbi/README.md, powerbi/semantic_model.md, and powerbi/measures.dax in VS Code. Read the table g. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open powerbi/README.md, powerbi/semantic_model.md, and powerbi/measures.dax in VS Code. Read the table grain and relationship table before opening Power BI.

4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 1****Action**

Open `powerbi/README.md`, `powerbi/semantic_model.md`, and `powerbi/measures.dax` in VS Code. Read the table grain and relationship table before opening Power BI.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

Action 16.2.2 — Open Power BI Desktop. On the welcome page choose ; if the welcome page is closed, use .

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Open Power BI Desktop. On the welcome page choose ; if the welcome page is closed, use . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open Power BI Desktop. On the welcome page choose **Get data**; if the welcome page is closed, use **Home > Get data > SQL Server**.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 2

Action

Open Power BI Desktop. On the welcome page choose **Get data**; if the welcome page is closed, use **Home > Get data > SQL Server**.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

Action 16.2.3 — Enter the exact local server name already verified in SSMS and database NorthstarRetail_DW. Choose for Track

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Enter the exact local server name already verified in SSMS and database NorthstarRetail_DW. Choose for Track. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Enter the exact local server name already verified in SSMS and database Northstar-Retail_DW. Choose **Import** for Track A unless you have a measured reason to practice DirectQuery. Do not type a password into any repository file.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 3

Action

Enter the exact local server name already verified in SSMS and database NorthstarRetail_DW. Choose **Import** for Track A unless you have a measured reason to practice DirectQuery. Do not type a password into any repository file.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

Action 16.2.4 — Choose the appropriate Windows or database authentication in the credential dialog. In Navigator select the ni

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Choose the appropriate Windows or database authentication in the credential dialog. In Navigator select the ni. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Choose the appropriate Windows or database authentication in the credential dialog. In Navigator select the nine dw.dim_* tables and six dw.fact_* tables. Select **Transform Data**, not Load, so you can inspect types first.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 4****Action**

Choose the appropriate Windows or database authentication in the credential dialog. In Navigator select the nine dw.dim_* tables and six dw.fact_* tables. Select **Transform Data**, not Load, so you can inspect types first.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

Action 16.2.5 — In Power Query, inspect each query name, column type, row count preview, and error indicator. Do not add busin

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

In Power Query, inspect each query name, column type, row count preview, and error indicator. Do not add busin. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: In Power Query, inspect each query name, column type, row count preview, and error indicator. Do not add business calculations that belong in the warehouse or DAX. Rename only if the model contract documents the rename, then choose **Close & Apply**.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 5****Action**

In Power Query, inspect each query name, column type, row count preview, and error indicator. Do not add business calculations that belong in the warehouse or DAX. Rename only if the model contract documents the rename, then choose **Close & Apply**.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

Action 16.2.6 — Open . Create relationships from each fact foreign key to its dimension primary key using the table in `powerb`

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Open . Create relationships from each fact foreign key to its dimension primary key using the table in `powerbi`. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open **Model view**. Create relationships from each fact foreign key to its dimension primary key using the table in powerbi/semantic_model.md. Set cardinality to Many-to-one (*:1), cross-filter direction Single, and active unless the contract specifically says otherwise.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 6****Action**

Open **Model view**. Create relationships from each fact foreign key to its dimension primary key using the table in powerbi/semantic_model.md. Set cardinality to Many-to-one (*:1), cross-filter direction Single, and active unless the contract specifically says otherwise.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

Action 16.2.7 — Select `dim_date`, open , choose `full_date`, and confirm. Create the Date hierarchy Year > Quarter > Month >

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Select `dim_date`, open , choose `full_date`, and confirm. Create the Date hierarchy Year > Quarter > Month > . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Select `dim_date`, open **Table tools > Mark as date table**, choose `full_date`, and confirm. Create the Date hierarchy Year > Quarter > Month > Full Date.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 7

Action

Select `dim_date`, open **Table tools > Mark as date table**, choose `full_date`, and confirm. Create the Date hierarchy Year > Quarter > Month > Full Date.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

Action 16.2.8 — Hide surrogate keys, batch IDs, hashes, load timestamps, and other technical columns from Report view. Set key

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Hide surrogate keys, batch IDs, hashes, load timestamps, and other technical columns from Report view. Set key. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Hide surrogate keys, batch IDs, hashes, load timestamps, and other technical columns from Report view. Set keys to **Do not summarize**. Add descriptions and display folders to measures.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 8****Action**

Hide surrogate keys, batch IDs, hashes, load timestamps, and other technical columns from Report view. Set keys to **Do not summarize**. Add descriptions and display folders to measures.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

Action 16.2.9 — Use to create a one-row table named `_Measures`, or use a documented existing table. Create each measure from

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Use to create a one-row table named `_Measures`, or use a documented existing table. Create each measure from . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Use **Home > Enter data** to create a one-row table named `_Measures`, or use a documented existing table. Create each measure from `measures.dax` one at a time, then move it to the matching display folder.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 9

Action

Use **Home > Enter data** to create a one-row table named `_Measures`, or use a documented existing table. Create each measure from `measures.dax` one at a time, then move it to the matching display folder.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use `Back` or `Cancel` rather than `Delete` or `Overwrite`. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

Action 16.2.10 — Format currencies, whole numbers, percentages, and decimal precision exactly as documented. Do not use FORMAT()

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Format currencies, whole numbers, percentages, and decimal precision exactly as documented. Do not use FORMAT(). This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Format currencies, whole numbers, percentages, and decimal precision exactly as documented. Do not use FORMAT() merely for visual formatting because it changes a numeric result into text.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 10****Action**

Format currencies, whole numbers, percentages, and decimal precision exactly as documented. Do not use FORMAT() merely for visual formatting because it changes a numeric result into text.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

Action 16.2.11 — For every measure, run the corresponding SQL validation query in SSMS using the same date/channel/store filter

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

For every measure, run the corresponding SQL validation query in SSMS using the same date/channel/store filter. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: For every measure, run the corresponding SQL validation query in SSMS using the same date/channel/store filters. Record the DAX result, SQL result, difference, and status.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 11

Action

For every measure, run the corresponding SQL validation query in SSMS using the same date/channel/store filters. Record the DAX result, SQL result, difference, and status.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

Action 16.2.12 — Create a temporary matrix with date, channel, and product category to test totals under multiple filters. Test

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create a temporary matrix with date, channel, and product category to test totals under multiple filters. Test. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Create a temporary matrix with date, channel, and product category to test totals under multiple filters. Test an empty-selection case, one date, multiple dates, and all dates.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 12****Action**

Create a temporary matrix with date, channel, and product category to test totals under multiple filters. Test an empty-selection case, one date, multiple dates, and all dates.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

Action 16.2.13 — Save the model as Northstar_Retail_Portfolio.pbix in a local ignored folder. Add only screenshots and docume

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Save the model as Northstar_Retail_Portfolio.pbix in a local ignored folder. Add only screenshots and docume. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Save the model as Northstar_Retail_Portfolio.pbix in a local ignored folder. Add only screenshots and documentation to Git unless you intentionally decide to version the PBIX and understand its binary-file tradeoff.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 13

Action

Save the model as Northstar_Retail_Portfolio.pbix in a local ignored folder. Add only screenshots and documentation to Git unless you intentionally decide to version the PBIX and understand its binary-file tradeoff.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

Action 16.2.14 — Optional alternate path: in Databricks SQL, run the views under sql/databricks/gold/ and compare a KPI to th

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Optional alternate path: in Databricks SQL, run the views under sql/databricks/gold/ and compare a KPI to th. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Optional alternate path: in Databricks SQL, run the views under sql/databricks/gold/ and compare a KPI to the same Gold table. Record the warehouse/catalog and access mode used.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 14****Action**

Optional alternate path: in Databricks SQL, run the views under `sql/databricks/gold/` and compare a KPI to the same Gold table. Record the warehouse/catalog and access mode used.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

Action 16.2.C1 — SSMS query window (SQL Server / T-SQL)

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SSMS query window (SQL Server / T-SQL)

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT SUM(net_sales_amount) AS total_revenue,
       SUM(gross_profit_amount) AS gross_profit,
       COUNT(DISTINCT order_id) AS orders
FROM dw.fact_sales;
```

Where to enter it:

SSMS query window (SQL Server / T-SQL). Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT SUM(net_sales_amount) AS total_revenue,` — `SELECT` requests data without changing it unless combined with a modifying statement; `SUM` adds numeric values in the current filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `net_sales_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `total_revenue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `SUM(gross_profit_amount) AS gross_profit,` — `SUM` adds numeric values in the current filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gross_profit_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gross_profit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 3:** `COUNT(DISTINCT order_id) AS orders` — `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DISTINCT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `orders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `FROM dw.fact_sales;` — `FROM` identifies the source table or query; `dw.fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

Action 16.2.C2 — Power BI Desktop DAX formula bar

Application:

Power BI Desktop

Window or panel:

Power BI Desktop DAX formula bar

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained Dax instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Power BI Desktop** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Dax):

```
Total Revenue =  
SUM ( fact_sales[net_sales_amount] )
```

Where to enter it:

Power BI Desktop DAX formula bar. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** Total Revenue = — Total is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** SUM (fact_sales[net_sales_amount]) — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; net_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

Action 16.2.C3 — Power BI Desktop DAX formula bar

Application:

Power BI Desktop

Window or panel:

Power BI Desktop DAX formula bar

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained Dax instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Power BI Desktop** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Dax):

```
Gross Margin Percentage =  
DIVIDE ( [Gross Profit], [Total Revenue] )
```

Where to enter it:

Power BI Desktop DAX formula bar. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** Gross Margin Percentage = — Gross is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Margin is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Percentage is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** DIVIDE ([Gross Profit], [Total Revenue]) — DIVIDE performs protected division in DAX; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gross is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Total is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

Action 16.2.C4 — SSMS query window (SQL Server / T-SQL)

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SSMS query window (SQL Server / T-SQL)

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT SUM(f.net_sales_amount) AS filtered_revenue
FROM dw.fact_sales AS f
JOIN dw.dim_date AS d ON d.date_key = f.order_date_key
WHERE d.full_date >= '2025-01-01'
AND d.full_date < '2025-02-01';
```

Where to enter it:

SSMS query window (SQL Server / T-SQL). Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT SUM(f.net_sales_amount) AS filtered_revenue` — `SELECT` requests data without changing it unless combined with a modifying statement; `SUM` adds numeric values in the current filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f.net_sales_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filtered_revenue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `FROM dw.fact_sales AS f` — `FROM` identifies the source table or query; `dw.fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `JOIN dw.dim_date AS d ON d.date_key = f.order_date_key` — `JOIN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dw.dim_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `d` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `d.date_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f.order_date_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `WHERE d.full_date >= '2025-01-01'` — `WHERE` filters rows to those meeting a condition; `d.full_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'2025-01-01'` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 5:** `AND d.full_date < '2025-02-01'`; — `AND` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `d.full_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'2025-02-01'` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

Action 16.2.C5 — Databricks SQL editor (Databricks SQL / Spark SQL)

Application:

Databricks workspace in a web browser

Window or panel:

Databricks SQL editor (Databricks SQL / Spark SQL)

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT order_date_key,
       SUM(net_sales_amount) AS total_revenue,
       SUM(gross_profit_amount) AS gross_profit
FROM northstar_retail.gold.fact_sales
GROUP BY order_date_key
ORDER BY order_date_key;
```

Where to enter it:

Databricks SQL editor (Databricks SQL / Spark SQL). Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT order_date_key, —` `SELECT` requests data without changing it unless combined with a modifying statement; `order_date_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `SUM(net_sales_amount) AS total_revenue, —` `SUM` adds numeric values in the current filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `net_sales_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `total_revenue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `SUM(gross_profit_amount) AS gross_profit —` `SUM` adds numeric values in the current filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gross_profit_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gross_profit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `FROM northstar_retail.gold.fact_sales —` `FROM` identifies the source table or query; `northstar_retail.gold.fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `GROUP BY order_date_key —` `GROUP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_date_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `ORDER BY order_date_key; —` `ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_date_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use

`git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 16.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 16 starting-screen checklist and re-verify the active context.

Lesson 16.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 16 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — docs/

Exact folder name: docs

Exact parent: repository root

Why it is needed now: Phase 16 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: docs.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 16.D1.1 — Create the folder docs

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\docs" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\docs" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run Test-Path ".\docs" -PathType Container.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — powerbi/

Exact folder name: powerbi

Exact parent: repository root

Why it is needed now: Phase 16 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: powerbi.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 16.D2.1 — Create the folder powerbi

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\powerbi" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\powerbi" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\powerbi"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\powerbi" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\powerbi" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\powerbi" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — powerbi/semantic_model.md

Exact filename: `semantic_model.md`

Filename extension: `.md`. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `powerbi` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 16.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `powerbi` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `semantic_model.md`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `semantic_model.md` and an empty file under the correct parent folder.
10. PRESS: `Ctrl+S` after adding each explained section.

PowerShell path

Action 16.F1.1 — Create the file `powerbi/semantic_model.md` from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\powerbi\semantic_model.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\powerbi\semantic_model.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\powerbi\semantic_model.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\powerbi\semantic_model.md" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "powerbi/semantic_model.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `powerbi`.

2. Right-click the editor tab and choose **Copy Path**; compare the ending with powerbi/semantic_model.md.
3. In PowerShell at the repository root, run Test-Path ".\powerbi\semantic_model.md" and require True.
4. Run git status --short -- "powerbi/semantic_model.md" to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for powerbi/semantic_model.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: powerbi/semantic_model.md

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 16.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Power BI Semantic Model

## Model rule

Use a star schema. Every relationship normally filters from one dimension to one fact in a single direction. Avoid fact-to-fact relationships and bidirectional filtering unless you can explain and test the ambiguity it introduces.

## Relationships

| From dimension column | To fact column | Cardinality | Cross-filter | Active? |
|---|---|---|---|---|
| dim_date[date_key] | fact_sales[order_date_key] | One-to-many | Single | Yes |
| dim_date[date_key] | fact_returns[return_date_key] | One-to-many | Single | Yes |
| dim_date[date_key] | fact_inventory_snapshot[snapshot_date_key] | One-to-many | Single | Yes |
| dim_date[date_key] | fact_shipments[ship_date_key] | One-to-many | Single | Yes |
| dim_date[date_key] | fact_shipments[promised_date_key] | One-to-many | Single | No |
| dim_date[date_key] | fact_shipments[delivery_date_key] | One-to-many | Single | No |
| dim_date[date_key] | fact_web_sessions[session_date_key] | One-to-many | Single | Yes |
| dim_date[date_key] | fact_marketing_spend[spend_date_key] | One-to-many | Single | Yes |
| dim_customer[customer_key] | fact_sales[customer_key] | One-to-many | Single | Yes |
| dim_customer[customer_key] | fact_shipments[customer_key] | One-to-many | Single | Yes |
| dim_customer[customer_key] | fact_web_sessions[customer_key] | One-to-many | Single | Yes |
| dim_customer[customer_key] | fact_returns[customer_key] | One-to-many | Single | Yes |
| dim_product[product_key] | fact_sales[product_key] | One-to-many | Single | Yes |
| dim_product[product_key] | fact_returns[product_key] | One-to-many | Single | Yes |
| dim_product[product_key] | fact_inventory_snapshot[product_key] | One-to-many | Single | Yes |
| dim_employee[employee_key] | fact_sales[employee_key] | One-to-many | Single | Yes |
| dim_promotion[promotion_key] | fact_sales[promotion_key] | One-to-many | Single | Yes |
| dim_store[store_key] | sales/returns/inventory/shipments store_key | One-to-many | Single | Yes |
| dim_channel[channel_key] | sales/web_sessions/marketing_spend channel_key | One-to-many | Single | Yes |
| dim_campaign[campaign_key] | web_sessions/marketing_spend campaign_key | One-to-many | Single | Yes |
| dim_supplier[supplier_code] | dim_product[supplier_code] | One-to-many | Single | Yes – documented snowflake exception |

### Supplier relationship note

'dim_supplier' is the one documented snowflake exception in the beginner model because the required facts do not carry a supplier key. Supplier filters travel through 'dim_product'. Verify 'dim_supplier[supplier_code]' is unique, keep the relationship single-direction, and do not add a second supplier-to-fact path. A larger production model may denormalize supplier attributes into 'dim_product' or add supplier keys to appropriate procurement/inventory facts.

## Date table

In Model view, select 'dim_date', then **Table tools > Mark as date table > Mark as date table**, choose 'full_date', and confirm. Sort 'month_name' by 'month_number'. Create a hierarchy: 'calendar_year > quarter_number > month_name > full_date'.

## Hidden technical columns

Hide surrogate keys from report view after relationships are correct: all '*_key' columns, batch ID, record hash, load timestamp, source system, SCD effective dates unless an audit page needs them. Do not delete them from the model.

## Display folders and formats

- '_Measures\Sales': Total Revenue, Gross Profit, Gross Margin %, Promotion Gross Profit, Orders, Units Sold, Average Order Value.
- '_Measures>Returns and Shipping': Return Rate, On-Time Delivery %.
- '_Measures\Marketing': Marketing Spend, Attributed Revenue, Return on Ad Spend, Conversion Rate.
- '_Measures\Customer': Purchasing Customers, New Customers, Returning Customers.
- '_Measures\Inventory': Inventory On Hand, Stockout Risk Count.

Format money as currency with two decimals, counts as whole numbers with thousands separators, and rates as percentages with one or two decimals. Add the business definition from 'docs/kpi_definitions.md' to each measure's Description property.

## Import versus DirectQuery decision

Use **Import** for Track A: the local warehouse is modest, development is faster, and you can control refresh manually. DirectQuery is an advanced choice when near-real-time access, data volume, governance, and capacity justify the latency and modeling tradeoffs. Record the choice in 'docs/assumption_log.md'.
```

Attribution warning

The included `Attributed Revenue` measure uses converting session order IDs, equivalent to a simplified last-converting-session link. It is not a multi-touch causal model. Test that an order ID appears once before presenting ROAS.

Line-by-line explanation

- **Line 1:** `# Power BI Semantic Model` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** `## Model rule` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** Use a star schema. Every relationship normally filters from one dimension to one fact in a single direction. Avoid fact-to-fact relationships and bidirectional filtering unless you can explain and test the ambiguity it introduces. — Use is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; star is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Every is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; relationship is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; normally is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; filters is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; from is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dimension is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** `## Relationships` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** `| From dimension column | To fact column | Cardinality | Cross-filter | Active? | —` From is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dimension is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; column is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; To is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; column is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Cardinality is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Cross-filter is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Active is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.
- **Line 10:** `| --- | --- | --- | --- | --- | —` --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command.
- **Line 11:** `| dim_date[date_key] | fact_sales[order_date_key] | One-to-many | Single | Yes | —` dim_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; One-to-many is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Single is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Yes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.

Yes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

tion; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; delivery_date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; One-to-many is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Single is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; No is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

tifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; One-to-many is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Single is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Yes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

to-many is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Single is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Yes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 29:** | dim_channel[channel_key] | sales/web_sessions/marketing_spend channel_key | One-to-many | Single | Yes | — dim_channel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; channel_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; web_sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; marketing_spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; channel_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; One-to-many is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Single is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 30:** | dim_campaign[campaign_key] | web_sessions/marketing_spend campaign_key | One-to-many | Single | Yes | — dim_campaign is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; campaign_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; web_sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; marketing_spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; campaign_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; One-to-many is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Single is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Yes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** | dim_supplier[supplier_code] | dim_product[supplier_code] | One-to-many | Single | Yes — documented snowflake exception | — dim_supplier is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; supplier_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dim_product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; supplier_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; One-to-many is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Single is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Yes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; documented is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 32:** blank line used only to separate logical sections for readability.
- **Line 33:** ### Supplier relationship note is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 34:** blank line used only to separate logical sections for readability.
- **Line 35:** \dim_supplier` is the one documented snowflake exception in the beginner model because the required facts do not carry a supplier key. Supplier filters travel through `dim_product`. Verify `dim_supplier[supplier_code]` is unique, keep the relationship single-direction, and do not add a second supplier-to-fact path. A larger production model may denormalize supplier attributes into `dim_product` or add supplier keys to appropriate procurement/inventory facts.—dim_supplier is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the

surrounding explanation;documentedis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;snowflakeis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;exceptionis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;inis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;theis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;beginneris a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;modelis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;because`is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 36:** blank line used only to separate logical sections for readability.
- **Line 37:** `## Date table` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 38:** blank line used only to separate logical sections for readability.
- **Line 39:** In Model view, select `\dim_date``, then **Table tools > Mark as date table > Mark as date table**, choose ``full_date``, and confirm. Sort ``month_name`` by ``month_number``. Create a hierarchy: ``calendar_year > quarter_number > month_name > full_date``.–Inis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Modelis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;viewis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;selectis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;dim_dateis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;thenis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;**is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Tableis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;toolsis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;>`is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 40:** blank line used only to separate logical sections for readability.
- **Line 41:** `## Hidden technical columns` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 42:** blank line used only to separate logical sections for readability.
- **Line 43:** Hide surrogate keys from report view after relationships are correct: all `*_key`` columns, batch ID, record hash, load timestamp, source system, SCD effective dates unless an audit page needs them. Do not delete them from the model.–Hideis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;surrogateis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;keysis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;fromis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;reportis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;viewis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;afteris a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;relationships is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;areis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;correctis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;all`is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 44:** blank line used only to separate logical sections for readability.
- **Line 45:** `## Display folders and formats` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 46:** blank line used only to separate logical sections for readability.

- Line 47:** - `_Measures\Sales``: Total Revenue, Gross Profit, Gross Margin %, Promotion Gross Profit, Orders, Units Sold, Average Order Value.—is a command option or flag; its exact behavior is defined beside the command; `_Measures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Total` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Revenue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Gross` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Profit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Gross` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Margin`` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 48:** - `_Measures>Returns and Shipping``: Return Rate, On-Time Delivery %.—is a command option or flag; its exact behavior is defined beside the command; `_Measures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Returns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Shipping` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Rate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `On-Time` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Delivery` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ``` is a path, relative location, branch-style name, or file argument; verify every character before running it
- Line 49:** - `_Measures\Marketing``: Marketing Spend, Attributed Revenue, Return on Ad Spend, Conversion Rate.—is a command option or flag; its exact behavior is defined beside the command; `_Measures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Marketing` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Marketing` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Marketing` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Spend` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Attributed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Revenue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Return` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `on`` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 50:** - `_Measures\Customer``: Purchasing Customers, New Customers, Returning Customers.—is a command option or flag; its exact behavior is defined beside the command; `_Measures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Customer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Purchasing` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `New` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

fier, or literal whose project-specific role is described in the surrounding explanation;Customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Returning is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Customers' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 51:** - _Measures\Inventory`: Inventory On Hand, Stockout Risk Count.-- is a command option or flag; its exact behavior is defined beside the command; Measures is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Inventory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Inventory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; On is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Hand is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Stockout is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Risk is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 52:** blank line used only to separate logical sections for readability.
- **Line 53:** Format money as currency with two decimals, counts as whole numbers with thousands separators, and rates as percentages with one or two decimals. Add the business definition from \docs\kpi_definitions.md` to each measure's Description property.-Format is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; money is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;; as is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; currency is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; with is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; two is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; decimals is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; counts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; as is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; whole is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; numbers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 54:** blank line used only to separate logical sections for readability.
- **Line 55:** ## Import versus DirectQuery decision is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 56:** blank line used only to separate logical sections for readability.
- **Line 57:** Use ****Import**** for Track A: the local warehouse is modest, development is faster, and you can control refresh manually. DirectQuery is an advanced choice when near-real-time access, data volume, governance, and capacity justify the latency and modeling tradeoffs. Record the choice in \docs\assumption_log.md`. -Use is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Import is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Track is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; A is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is**

described in the surrounding explanation;localis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;warehouseis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 58:** blank line used only to separate logical sections for readability.
- **Line 59:** `## Attribution` warning is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 60:** blank line used only to separate logical sections for readability.
- **Line 61:** The `included \Attributed Revenue`` measure uses converting session order IDs, equivalent to a simplified last-converting-session link. It is not a multi-touch causal model. Test that an order ID appears once before presenting ROAS.–Theis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;includedis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Attributedis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Revenueis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;measureis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;usesis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;convertingis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;sessionis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;orderid is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;IDsis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;;is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;equivalent` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

[Back to the Phase 16 lesson index](#)

File introduction — `powerbi/measures.dax`

Exact filename: `measures.dax`

Filename extension: `.dax`. In this project it identifies Power BI measures and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `powerbi` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: Power BI measures for Phase 16.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `powerbi` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `measures.dax`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `measures.dax` and an empty file under the correct parent folder.
10. PRESS: `Ctrl+S` after adding each explained section.

Action 16.F2.1 — Create the file `powerbi/measures.dax` from PowerShell

Application:

Power BI Desktop

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Power BI Desktop** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\powerbi\measures.dax" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\powerbi\measures.dax" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\powerbi\measures.dax"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\powerbi\measures.dax" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "powerbi/measures.dax"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `powerbi`.

2. Right-click the editor tab and choose **Copy Path**; compare the ending with `powerbi/measures.dax`.
3. In PowerShell at the repository root, run `Test-Path ".\powerbi\measures.dax"` and require `True`.
4. Run `git status --short -- "powerbi/measures.dax"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `powerbi/measures.dax`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: DAX

Exact file: `powerbi/measures.dax`

Purpose: Power BI measures.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 16.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
// Northstar Retail Power BI measures
// Put these in a dedicated empty table named _Measures.
// Replace column names only if your loaded warehouse schema differs.

Total Revenue =
SUM ( fact_sales[net_sales_amount] )

Gross Profit =
SUM ( fact_sales[gross_profit_amount] )

Gross Margin % =
DIVIDE ( [Gross Profit], [Total Revenue] )

Promotion Gross Profit =
CALCULATE (
    [Gross Profit],
    KEEPFILTERS ( dim_promotion[promotion_key] <> 0 )
)

Orders =
DISTINCTCOUNT ( fact_sales[order_id] )

Units Sold =
SUM ( fact_sales[quantity] )

Average Order Value =
DIVIDE ( [Total Revenue], [Orders] )

Returned Units =
SUM ( fact_returns[return_quantity] )

Return Rate =
DIVIDE ( [Returned Units], [Units Sold] )

Delivered Shipments =
CALCULATE (
    COUNTROWS ( fact_shipments ),
    NOT ISBLANK ( fact_shipments[delivery_date_key] )
)

On-Time Shipments =
CALCULATE (
    [Delivered Shipments],
    fact_shipments[on_time_flag] = TRUE ()
)

On-Time Delivery % =
DIVIDE ( [On-Time Shipments], [Delivered Shipments] )

Marketing Spend =
SUM ( fact_marketing_spend[spend_amount] )

Web Sessions =
SUM ( fact_web_sessions[session_count] )

Converted Sessions =
CALCULATE (
    [Web Sessions],
    fact_web_sessions[converted_flag] = TRUE ()
)

Conversion Rate =
DIVIDE ( [Converted Sessions], [Web Sessions] )

Attributed Revenue =
VAR ConvertedOrders =
    FILTER (
```

```

VALUES ( fact_web_sessions[converted_order_id] ),
NOT ISBLANK ( fact_web_sessions[converted_order_id] )
)
RETURN
CALCULATE (
[Total Revenue],
TREATAS ( ConvertedOrders, fact_sales[order_id] )
)

Return on Ad Spend =
DIVIDE ( [Attributed Revenue], [Marketing Spend] )

Purchasing Customers =
CALCULATE (
DISTINCTCOUNT ( fact_sales[customer_key] ),
fact_sales[customer_key] <> 0
)

New Customers =
VAR PeriodStart = MIN ( dim_date[full_date] )
VAR PeriodEnd = MAX ( dim_date[full_date] )
RETURN
COUNTROWS (
FILTER (
VALUES ( fact_sales[customer_key] ),
fact_sales[customer_key] <> 0
&& VAR FirstOrderDate =
CALCULATE (
MIN ( dim_date[full_date] ),
REMOVEFILTERS ( dim_date )
)
RETURN
FirstOrderDate >= PeriodStart
&& FirstOrderDate <= PeriodEnd
)
)

Returning Customers =
[Purchasing Customers] - [New Customers]

Inventory On Hand =
SUM ( fact_inventory_snapshot[quantity_on_hand] )

Stockout Risk Count =
CALCULATE (
COUNTROWS ( fact_inventory_snapshot ),
fact_inventory_snapshot[stockout_risk_flag] = TRUE ()
)

```

Line-by-line explanation

- **Line 1:** // Northstar Retail Power BI measures is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** // Put these in a dedicated empty table named _Measures. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** // Replace column names only if your loaded warehouse schema differs. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** Total Revenue = — Total is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** SUM (fact_sales[net_sales_amount]) — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; net_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** Gross Profit = — Gross is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 9:** SUM (fact_sales[gross_profit_amount]) — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gross_profit_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** blank line used only to separate logical sections for readability.
- **Line 11:** Gross Margin % = — Gross is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Margin is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** DIVIDE ([Gross Profit], [Total Revenue]) — DIVIDE performs protected division in DAX; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gross is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Total is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** blank line used only to separate logical sections for readability.
- **Line 14:** Promotion Gross Profit = — Promotion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gross is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** CALCULATE (— CALCULATE evaluates an expression under modified DAX filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** [Gross Profit], — [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gross is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** KEEPFILTERS (dim_promotion[promotion_key] <> 0) — KEEPFILTERS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dim_promotion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; promotion_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; <> is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** blank line used only to separate logical sections for readability.
- **Line 20:** Orders = — Orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** DISTINCTCOUNT (fact_sales[order_id]) — DISTINCTCOUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or-

der_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; J is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- [illegible]

described in the surrounding explanation; Units is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Sold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 34:** blank line used only to separate logical sections for readability.
- **Line 35:** Delivered Shipments = — Delivered is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 36:** CALCULATE (— CALCULATE evaluates an expression under modified DAX filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 37:** COUNTRROWS (fact_shipments), — COUNTRROWS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 38:** NOT ISBLANK (fact_shipments[delivery_date_key]) — NOT reverses a Boolean condition; ISBLANK is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; delivery_date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 39:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 40:** blank line used only to separate logical sections for readability.
- **Line 41:** On-Time Shipments = — On-Time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** CALCULATE (— CALCULATE evaluates an expression under modified DAX filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 43:** [Delivered Shipments], — [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Delivered is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 44:** fact_shipments[on_time_flag] = TRUE () — fact_shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on_time_flag is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; TRUE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 45:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** blank line used only to separate logical sections for readability.
- **Line 47:** On-Time Delivery % = — On-Time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Delivery is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 48:** DIVIDE ([On-Time Shipments], [Delivered Shipments]) — DIVIDE performs protected division in DAX; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding

explanation; On-Time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Delivered is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 49:** blank line used only to separate logical sections for readability.
- **Line 50:** Marketing Spend = — Marketing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 51:** SUM (fact_marketing_spend[spend_amount]) — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_marketing_spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spend_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 52:** blank line used only to separate logical sections for readability.
- **Line 53:** Web Sessions = — Web is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 54:** SUM (fact_web_sessions[session_count]) — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_web_sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; session_count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 55:** blank line used only to separate logical sections for readability.
- **Line 56:** Converted Sessions = — Converted is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 57:** CALCULATE (— CALCULATE evaluates an expression under modified DAX filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 58:** [Web Sessions], — [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Web is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 59:** fact_web_sessions[converted_flag] = TRUE () — fact_web_sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; converted_flag is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; TRUE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 60:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 61:** blank line used only to separate logical sections for readability.

- **Line 62:** Conversion Rate = — Conversion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Rate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 63:** DIVIDE ([Converted Sessions], [Web Sessions]) — DIVIDE performs protected division in DAX; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Converted is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Web is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 64:** blank line used only to separate logical sections for readability.
- **Line 65:** Attributed Revenue = — Attributed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 66:** VAR ConvertedOrders = — VAR declares a reusable DAX variable; ConvertedOrders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 67:** FILTER (— FILTER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 68:** VALUES (fact_web_sessions[converted_order_id]) , — VALUES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_web_sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; converted_order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 69:** NOT ISBLANK (fact_web_sessions[converted_order_id]) — NOT reverses a Boolean condition; ISBLANK is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_web_sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; converted_order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 70:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 71:** RETURN — RETURN returns the final expression after DAX variables are defined
- **Line 72:** CALCULATE (— CALCULATE evaluates an expression under modified DAX filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 73:** [Total Revenue], — [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Total is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 74:** TREATAS (ConvertedOrders, fact_sales[order_id]) — TREATAS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ConvertedOrders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 75:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 76:** blank line used only to separate logical sections for readability.
- **Line 77:** Return on Ad Spend = — Return is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Ad is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 78:** DIVIDE ([Attributed Revenue], [Marketing Spend]) — DIVIDE performs protected division in DAX; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Attributed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Marketing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 79:** blank line used only to separate logical sections for readability.
- **Line 80:** Purchasing Customers = — Purchasing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 81:** CALCULATE (— CALCULATE evaluates an expression under modified DAX filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 82:** DISTINCTCOUNT (fact_sales[customer_key]), — DISTINCTCOUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 83:** fact_sales[customer_key] <> 0 — fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; <> is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression
- **Line 84:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 85:** blank line used only to separate logical sections for readability.
- **Line 86:** New Customers = — New is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 87:** `VAR PeriodStart = MIN ( dim_date[full_date] )` — VAR declares a reusable DAX variable; PeriodStart is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; MIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dim_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 88:** `VAR PeriodEnd = MAX ( dim_date[full_date] )` — VAR declares a reusable DAX variable; PeriodEnd is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; MAX is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dim_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 89:** `RETURN` — RETURN returns the final expression after DAX variables are defined
- **Line 90:** `COUNTROWS` (— COUNTROWS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 91:** `FILTER` (— FILTER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 92:** `VALUES ( fact_sales[customer_key] )`, — VALUES is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 93:** `fact_sales[customer_key] <> 0` — `fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<>` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression
- **Line 94:** `&& VAR FirstOrderDate =` — VAR declares a reusable DAX variable; FirstOrderDate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 95:** `CALCULATE` (— CALCULATE evaluates an expression under modified DAX filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 96:** `MIN ( dim_date[full_date] )`, — MIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dim_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 97:** REMOVEFILTERS (dim_date) — REMOVEFILTERS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dim_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 98:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 99:** RETURN — RETURN returns the final expression after DAX variables are defined
- **Line 100:** FirstOrderDate >= PeriodStart — FirstOrderDate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; >= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PeriodStart is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 101:** && FirstOrderDate <= PeriodEnd — FirstOrderDate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; <= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PeriodEnd is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 102:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 103:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 104:** blank line used only to separate logical sections for readability.
- **Line 105:** Returning Customers = — Returning is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 106:** [Purchasing Customers] - [New Customers] — [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Purchasing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; - is a command option or flag; its exact behavior is defined beside the command; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; New is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 107:** blank line used only to separate logical sections for readability.
- **Line 108:** Inventory On Hand = — Inventory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; On is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Hand is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 109:** SUM (fact_inventory_snapshot[quantity_on_hand]) — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_inventory_snapshot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quantity_on_hand is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 110:** blank line used only to separate logical sections for readability.
- **Line 111:** Stockout Risk Count = — Stockout is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Risk is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 112:** CALCULATE (— CALCULATE evaluates an expression under modified DAX filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 113:** COUNTROWS (fact_inventory_snapshot), — COUNTROWS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or

literal whose project-specific role is described in the surrounding explanation; `fact_inventory_snapshot` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 114:** `fact_inventory_snapshot[stockout_risk_flag] = TRUE ()` — `fact_inventory_snapshot` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `stockout_risk_flag` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `TRUE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 115:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 16 lesson index](#)

File introduction — docs/kpi_definitions.csv

Exact filename: `kpi_definitions.csv`

Filename extension: `.csv`. In this project it identifies structured rows used as a definition, mapping, or fixture and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `docs` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: structured rows used as a definition, mapping, or fixture for Phase 16.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `docs` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `kpi_definitions.csv`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `kpi_definitions.csv` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 16.F3.1 — Create the file docs/kpi_definitions.csv from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\kpi_definitions.csv" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType File -Path ".\docs\kpi_definitions.csv" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; File is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\docs\kpi_definitions.csv" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\docs\kpi_definitions.csv" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/kpi_definitions.csv"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under docs.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `docs/kpi_definitions.csv`.
3. In PowerShell at the repository root, run `Test-Path ".\docs\kpi_definitions.csv"` and require `True`.
4. Run `git status --short -- "docs/kpi_definitions.csv"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

[Back to the Phase 16 lesson index](#)

What your screen should look like now

- Every Phase 16 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.

- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 16.3 completion checklist



powerbi/semantic_model.md was created by me, saved, located correctly, and inspected — [STATUS]



powerbi/measures.dax was created by me, saved, located correctly, and inspected — [STATUS]



docs/kpi_definitions.csv was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 16.4](#) and recheck the root and branch.

Lesson 16.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Power BI measure 1 — Total Revenue

Business meaning: Net merchandise sales after line discounts, before subtracting cost; tax is excluded.

Business formula: `SUM(net_sales_amount)`

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: `fact_sales`

Important filters: Exclude cancelled/invalid lines upstream; selected date/channel/product/store filters apply.

Recommended format: Currency.

Action 16.4.M1 — Create, format, and validate the Total Revenue DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Total Revenue DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.

4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Total Revenue shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Total Revenue shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Total Revenue =  
SUM ( fact_sales[net_sales_amount] )
```

Line-by-line DAX explanation

- **Line 1:** Total Revenue = — Total is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** SUM (fact_sales[net_sales_amount]) — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; net_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Compare DAX card to SQL under the same date and slicer filters.

Common interpretation mistake: Adding tax or shipping without stating it, or summing order totals at line grain.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Total Revenue from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 2 — Gross Profit

Business meaning: Revenue remaining after cost of goods sold.

Business formula: SUM(net_sales_amount - cost_of_goods_sold)

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: fact_sales

Important filters: Same population as revenue; USD-only in beginner track.

Recommended format: Currency.

Action 16.4.M2 — Create, format, and validate the Gross Profit DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Gross Profit DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Gross Profit shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Gross Profit shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Gross Profit =
SUM ( fact_sales[gross_profit_amount] )
```

Line-by-line DAX explanation

- **Line 1:** Gross Profit = — Gross is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** SUM (fact_sales[gross_profit_amount]) — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gross_profit_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Net revenue minus COGS must equal stored gross profit within rounding tolerance.

Common interpretation mistake: Treating gross profit as operating profit; marketing, payroll, and overhead are not included.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Gross Profit from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 3 — Gross Margin %

Business meaning: Gross profit as a share of revenue.

Business formula: Gross Profit / Total Revenue

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: fact_sales

Important filters: Same filter context for numerator and denominator.

Recommended format: Percentage.

Action 16.4.M3 — Create, format, and validate the Gross Margin % DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Gross Margin % DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Gross Margin % shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Gross Margin % shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.

8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Gross Margin % =
DIVIDE ( [Gross Profit], [Total Revenue] )
```

Line-by-line DAX explanation

- **Line 1:** Gross Margin % = — Gross is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Margin is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** DIVIDE ([Gross Profit], [Total Revenue]) — DIVIDE performs protected division in DAX; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gross is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Total is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Calculate from independently queried profit and revenue; do not average row margins.

Common interpretation mistake: Averaging product-level percentages instead of dividing aggregate amounts.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Gross Margin % from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 4 — Promotion Gross Profit

Business meaning: Gross profit contributed by accepted sales that used a known promotion. Use it to compare promotion contribution, not to claim causal incremental lift.

Business formula: SUM(gross_profit_amount) for promotion_key <> 0

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: fact_sales[gross_profit_amount,promotion_key]; dim_promotion[promotion_key,promotion_name,promotion_code,start_date,end_date]

Important filters: Exclude unknown/no-promotion key 0; use the same accepted-sales, date, channel, and USD filters as Gross Profit.

Recommended format: Currency.

Action 16.4.M4 — Create, format, and validate the Promotion Gross Profit DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Promotion Gross Profit DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.

5. TYPE THIS: Enter the exact measure definition for Promotion Gross Profit shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Promotion Gross Profit shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

DAX definition

```
Promotion Gross Profit =  
CALCULATE (  
    [Gross Profit],  
    KEEPFILTERS ( dim_promotion[promotion_key] <> 0 )  
)
```

Line-by-line DAX explanation

- **Line 1:** Promotion Gross Profit = — Promotion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gross is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** CALCULATE (— CALCULATE evaluates an expression under modified DAX filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** [Gross Profit], — [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gross is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** KEEPFILTERS (dim_promotion[promotion_key] <> 0) — KEEPFILTERS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dim_promotion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; promotion_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; <> is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Compare the DAX measure by promotion to the grouped SQL result under identical dates, channels, and promotion filters.

Common interpretation mistake: Treating observed promoted-sales profit as incremental causal lift while ignoring baseline demand, cannibalization, and discount cost.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Promotion Gross Profit from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 5 — Orders

Business meaning: Distinct completed or shipped customer orders represented in sales.

Business formula: COUNT(DISTINCT order_id)

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: fact_sales

Important filters: Invalid/cancelled orders excluded in Silver policy.

Recommended format: Whole number or appropriate numeric format.

Action 16.4.M5 — Create, format, and validate the Orders DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Orders DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Orders shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Orders shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.

11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Orders =  
DISTINCTCOUNT ( fact_sales[order_id] )
```

Line-by-line DAX explanation

- **Line 1:** Orders = — Orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** DISTINCTCOUNT (fact_sales[order_id]) — DISTINCTCOUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or-der_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Compare to distinct valid order IDs in Silver.

Common interpretation mistake: Counting fact rows, which counts lines rather than orders.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]

- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Orders from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 6 — Units Sold

Business meaning: Total valid product units on sales lines.

Business formula: SUM(quantity)

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: fact_sales

Important filters: Quantity must be positive after Silver quarantine.

Recommended format: Whole number or appropriate numeric format.

Action 16.4.M6 — Create, format, and validate the Units Sold DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Units Sold DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Units Sold shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for `Units Sold` shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Units Sold =  
SUM ( fact_sales[quantity] )
```

Line-by-line DAX explanation

- **Line 1:** `Units Sold =` — `Units` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Sold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 2:** `SUM ( fact_sales[quantity] )` — SUM adds numeric values in the current filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quantity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Reconcile to valid Silver quantities.

Common interpretation mistake: Including returned units as negative sales without defining net units separately.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined `Units Sold` from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 7 — Average Order Value

Business meaning: Average net merchandise revenue per distinct order.

Business formula: Total Revenue / Orders

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: `fact_sales`

Important filters: Same context as revenue and orders.

Recommended format: Currency.

Action 16.4.M7 — Create, format, and validate the Average Order Value DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Average Order Value DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Average Order Value shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Average Order Value shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location

with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Average Order Value =  
DIVIDE ( [Total Revenue], [Orders] )
```

Line-by-line DAX explanation

- **Line 1:** Average Order Value = — Average is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Order is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** DIVIDE ([Total Revenue], [Orders]) — DIVIDE performs protected division in DAX; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Total is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Aggregate each order first and average, then compare.

Common interpretation mistake: Using average line revenue.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Average Order Value from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 8 — Returned Units

Business meaning: A governed semantic calculation used by one or more report pages.

Business formula: See the DAX expression and supporting measures below.

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: See expression dependencies and semantic model mapping.

Important filters: Use the documented date role and exclude invalid source rows upstream.

Recommended format: Whole number or appropriate numeric format.

Action 16.4.M8 — Create, format, and validate the Returned Units DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Returned Units DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Returned Units shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Returned Units shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Returned Units =  
SUM ( fact_returns[return_quantity] )
```

Line-by-line DAX explanation

- **Line 1:** Returned Units = — Returned is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Units is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** SUM (fact_returns[return_quantity]) — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_returns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; return_quantity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Compare the measure with an independently filtered SQL query at the same grain and date context.

Common interpretation mistake: Using a different denominator, date role, or filter population than the documented business definition.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Returned Units from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 9 — Return Rate

Business meaning: Returned units divided by sold units for a comparable filtered population.

Business formula: $\text{SUM}(\text{return_quantity}) / \text{SUM}(\text{sold quantity})$

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: fact_returns, fact_sales, shared dimensions

Important filters: Use compatible date definition; report purchase-date or return-date view explicitly.

Recommended format: Percentage.

Action 16.4.M9 — Create, format, and validate the Return Rate DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Return Rate DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Return Rate shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.

4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Return Rate shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Return Rate =
DIVIDE ( [Returned Units], [Units Sold] )
```

Line-by-line DAX explanation

- **Line 1:** Return Rate = — Return is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Rate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** DIVIDE ([Returned Units], [Units Sold]) — DIVIDE performs protected division in DAX; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Returned is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Units is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Units is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Sold is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Trace a small product/date sample from sales to returns.

Common interpretation mistake: Dividing return rows by order rows, or mixing return-date and purchase-date contexts silently.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Return Rate from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 10 — Delivered Shipments

Business meaning: A governed semantic calculation used by one or more report pages.

Business formula: See the DAX expression and supporting measures below.

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: See expression dependencies and semantic model mapping.

Important filters: Use the documented date role and exclude invalid source rows upstream.

Recommended format: Whole number or appropriate numeric format.

Action 16.4.M10 — Create, format, and validate the Delivered Shipments DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Delivered Shipments DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.

4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Delivered Shipments shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Delivered Shipments shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Delivered Shipments =  
CALCULATE (  
    COUNTROWS ( fact_shipments ),  
    NOT ISBLANK ( fact_shipments[delivery_date_key] )  
)
```

Line-by-line DAX explanation

- **Line 1:** Delivered Shipments = — Delivered is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** CALCULATE (— CALCULATE evaluates an expression under modified DAX filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** COUNTROWS (fact_shipments), — COUNTROWS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** NOT ISBLANK (fact_shipments[delivery_date_key]) — NOT reverses a Boolean condition; ISBLANK is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; delivery_date_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Compare the measure with an independently filtered SQL query at the same grain and date context.

Common interpretation mistake: Using a different denominator, date role, or filter population than the documented business definition.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Delivered Shipments from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 11 — On-Time Shipments

Business meaning: A governed semantic calculation used by one or more report pages.

Business formula: See the DAX expression and supporting measures below.

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use

compatible populations.

Required tables/columns: See expression dependencies and semantic model mapping.

Important filters: Use the documented date role and exclude invalid source rows upstream.

Recommended format: Whole number or appropriate numeric format.

Action 16.4.M11 — Create, format, and validate the On-Time Shipments DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the On-Time Shipments DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for On-Time Shipments shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for On-Time Shipments shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.

11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
On-Time Shipments =  
CALCULATE (  
    [Delivered Shipments],  
    fact_shipments[on_time_flag] = TRUE ()  
)
```

Line-by-line DAX explanation

- **Line 1:** On-Time Shipments = — On-Time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** CALCULATE (— CALCULATE evaluates an expression under modified DAX filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** [Delivered Shipments], — [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Delivered is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** fact_shipments[on_time_flag] = TRUE () — fact_shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on_time_flag is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; TRUE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

planation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 5:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Compare the measure with an independently filtered SQL query at the same grain and date context.

Common interpretation mistake: Using a different denominator, date role, or filter population than the documented business definition.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined On-Time Shipments from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 12 — On-Time Delivery %

Business meaning: Share of delivered shipments whose actual delivery date is on or before promised date.

Business formula: On-time shipments / delivered shipments

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: fact_shipments

Important filters: Delivered shipments with a non-null actual date only.

Recommended format: Percentage.

Action 16.4.M12 — Create, format, and validate the On-Time Delivery % DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the On-Time Delivery % DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.

3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for On-Time Delivery % shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for On-Time Delivery % shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
On-Time Delivery % =  
DIVIDE ( [On-Time Shipments], [Delivered Shipments] )
```

Line-by-line DAX explanation

- **Line 1:** On-Time Delivery % = — On-Time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Delivery is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** DIVIDE ([On-Time Shipments], [Delivered Shipments]) — DIVIDE performs protected division in DAX; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; On-Time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Delivered is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Manually inspect promised and actual dates for a sample.

Common interpretation mistake: Using orders as denominator when one order can have multiple shipments.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined On-Time Delivery % from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 13 — Marketing Spend

Business meaning: Recorded media spend in the governed reporting currency.

Business formula: SUM(spend_amount)

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: fact_marketing_spend

Important filters: USD only in Track A; date and campaign filters apply.

Recommended format: Currency.

Action 16.4.M13 — Create, format, and validate the Marketing Spend DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Marketing Spend DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Marketing Spend shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Marketing Spend shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Marketing Spend =  
SUM ( fact_marketing_spend[spend_amount] )
```

Line-by-line DAX explanation

- **Line 1:** Marketing Spend = — Marketing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** SUM (fact_marketing_spend[spend_amount]) — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_marketing_spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spend_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Reconcile to accepted Silver spend rows.

Common interpretation mistake: Mixing currencies or campaign budgets with actual spend.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Marketing Spend from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 14 — Web Sessions

Business meaning: A governed semantic calculation used by one or more report pages.

Business formula: See the DAX expression and supporting measures below.

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: See expression dependencies and semantic model mapping.

Important filters: Use the documented date role and exclude invalid source rows upstream.

Recommended format: Whole number or appropriate numeric format.

Action 16.4.M14 — Create, format, and validate the Web Sessions DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Web Sessions DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Web Sessions shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for `Web Sessions` shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use `Back` or `Cancel` rather than `Delete` or `Overwrite`. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Web Sessions =  
SUM ( fact_web_sessions[session_count] )
```

Line-by-line DAX explanation

- **Line 1:** `Web Sessions =` — `Web` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `SUM ( fact_web_sessions[session_count] )` — `SUM` adds numeric values in the current filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact_web_sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described

in the surrounding explanation; session_count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Compare the measure with an independently filtered SQL query at the same grain and date context.

Common interpretation mistake: Using a different denominator, date role, or filter population than the documented business definition.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Web Sessions from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 15 — Converted Sessions

Business meaning: A governed semantic calculation used by one or more report pages.

Business formula: See the DAX expression and supporting measures below.

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: See expression dependencies and semantic model mapping.

Important filters: Use the documented date role and exclude invalid source rows upstream.

Recommended format: Whole number or appropriate numeric format.

Action 16.4.M15 — Create, format, and validate the Converted Sessions DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Converted Sessions DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.

3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Converted Sessions shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Converted Sessions shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Converted Sessions =  
CALCULATE (  
    [Web Sessions],  
    fact_web_sessions[converted_flag] = TRUE ()  
)
```

Line-by-line DAX explanation

- **Line 1:** `Converted Sessions =` — `Converted` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `CALCULATE (` — `CALCULATE` evaluates an expression under modified DAX filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `[Web Sessions],` — `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Web` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `fact_web_sessions[converted_flag] = TRUE ()` — `fact_web_sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `converted_flag` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `TRUE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Compare the measure with an independently filtered SQL query at the same grain and date context.

Common interpretation mistake: Using a different denominator, date role, or filter population than the documented business definition.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined `Converted Sessions` from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 16 — Conversion Rate

Business meaning: Share of web sessions that are linked to a completed order.

Business formula: `Converted sessions / sessions`

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use

compatible populations.

Required tables/columns: fact_web_sessions

Important filters: Bot traffic is out of scope; filter by session date/channel/campaign.

Recommended format: Percentage.

Action 16.4.M16 — Create, format, and validate the Conversion Rate DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Conversion Rate DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Conversion Rate shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Conversion Rate shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.

11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Conversion Rate =  
DIVIDE ( [Converted Sessions], [Web Sessions] )
```

Line-by-line DAX explanation

- **Line 1:** Conversion Rate = — Conversion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Rate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** DIVIDE ([Converted Sessions], [Web Sessions]) — DIVIDE performs protected division in DAX; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Converted is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;], is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Web is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Compare converted session count to non-null converted_order_id values.

Common interpretation mistake: Dividing orders by page views or users without labeling the denominator.

- Power BI filter/slicer state: [ENTER]

- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Conversion Rate from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 17 — Attributed Revenue

Business meaning: A governed semantic calculation used by one or more report pages.

Business formula: See the DAX expression and supporting measures below.

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: See expression dependencies and semantic model mapping.

Important filters: Use the documented date role and exclude invalid source rows upstream.

Recommended format: Currency.

Action 16.4.M17 — Create, format, and validate the Attributed Revenue DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Attributed Revenue DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Attributed Revenue shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for `Attributed Revenue` shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use `Back` or `Cancel` rather than `Delete` or `Overwrite`. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Attributed Revenue =  
VAR ConvertedOrders =  
    FILTER (  
        VALUES ( fact_web_sessions[converted_order_id] ),  
        NOT ISBLANK ( fact_web_sessions[converted_order_id] )  
    )  
RETURN  
    CALCULATE (  
        [Total Revenue],  
        ConvertedOrders
```

```
TREATAS ( ConvertedOrders, fact_sales[order_id] )
)
```

Line-by-line DAX explanation

- **Line 1:** `Attributed Revenue =` — `Attributed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Revenue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `VAR ConvertedOrders =` — `VAR` declares a reusable DAX variable; `ConvertedOrders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `FILTER (` — `FILTER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `VALUES ( fact_web_sessions[converted_order_id] ),` — `VALUES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact_web_sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `converted_order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `NOT ISBLANK ( fact_web_sessions[converted_order_id] )` — `NOT` reverses a Boolean condition; `ISBLANK` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact_web_sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `converted_order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `RETURN` — `RETURN` returns the final expression after DAX variables are defined
- **Line 8:** `CALCULATE (` — `CALCULATE` evaluates an expression under modified DAX filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `[Total Revenue],` — `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Total` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Revenue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `TREATAS ( ConvertedOrders, fact_sales[order_id] )` — `TREATAS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ConvertedOrders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `order_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** `)` — `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Compare the measure with an independently filtered SQL query at the same grain and date context.

Common interpretation mistake: Using a different denominator, date role, or filter population than the documented business definition.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Attributed Revenue from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 18 — Return on Ad Spend

Business meaning: Attributed sales revenue divided by marketing spend under a stated attribution model.

Business formula: Attributed Revenue / Marketing Spend

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: fact_web_sessions, fact_sales, fact_marketing_spend, dim_campaign

Important filters: Beginner model uses last converting session; exclude unattributed revenue from numerator.

Recommended format: Percentage.

Action 16.4.M18 — Create, format, and validate the Return on Ad Spend DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Return on Ad Spend DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Return on Ad Spend shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.

8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Return on Ad Spend shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Return on Ad Spend =  
DIVIDE ( [Attributed Revenue], [Marketing Spend] )
```

Line-by-line DAX explanation

- **Line 1:** Return on Ad Spend = — Return is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Ad is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** DIVIDE ([Attributed Revenue], [Marketing Spend]) — DIVIDE performs protected division in DAX; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Attributed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Marketing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: List converted order IDs and verify each is counted once before comparing spend.

Common interpretation mistake: Calling total company revenue campaign-attributed revenue.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Return on Ad Spend from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 19 — Purchasing Customers

Business meaning: A governed semantic calculation used by one or more report pages.

Business formula: See the DAX expression and supporting measures below.

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: See expression dependencies and semantic model mapping.

Important filters: Use the documented date role and exclude invalid source rows upstream.

Recommended format: Whole number or appropriate numeric format.

Action 16.4.M19 — Create, format, and validate the Purchasing Customers DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Purchasing Customers DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Purchasing Customers shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Purchasing Customers shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Purchasing Customers =  
CALCULATE (  
    DISTINCTCOUNT ( fact_sales[customer_key] ),  
    fact_sales[customer_key] <> 0  
)
```

Line-by-line DAX explanation

- **Line 1:** Purchasing Customers = — Purchasing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** CALCULATE (— CALCULATE evaluates an expression under modified DAX filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** DISTINCTCOUNT (fact_sales[customer_key]), — DISTINCTCOUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** fact_sales[customer_key] <> 0 — fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; <> is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression
- **Line 5:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Compare the measure with an independently filtered SQL query at the same grain and date context.

Common interpretation mistake: Using a different denominator, date role, or filter population than the documented business definition.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Purchasing Customers from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 20 — New Customers

Business meaning: Customers whose first completed purchase occurs in the selected period.

Business formula: Distinct customers with first order date in period

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: fact_sales, dim_date

Important filters: Selected reporting period; unknown customer excluded.

Recommended format: Whole number or appropriate numeric format.

Action 16.4.M20 — Create, format, and validate the New Customers DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the New Customers DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for New Customers shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.

9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for New Customers shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
New Customers =  
VAR PeriodStart = MIN ( dim_date[full_date] )
```

```

VAR PeriodEnd = MAX ( dim_date[full_date] )
RETURN
COUNTROWS (
    FILTER (
        VALUES ( fact_sales[customer_key] ),
        fact_sales[customer_key] <> 0
        && VAR FirstOrderDate =
            CALCULATE (
                MIN ( dim_date[full_date] ),
                REMOVEFILTERS ( dim_date )
            )
        RETURN
            FirstOrderDate >= PeriodStart
            && FirstOrderDate <= PeriodEnd
    )
)

```

Line-by-line DAX explanation

- **Line 1:** `New Customers =` — `New` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `VAR PeriodStart = MIN ( dim_date[full_date] )` — `VAR` declares a reusable DAX variable; `PeriodStart` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `MIN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dim_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `VAR PeriodEnd = MAX ( dim_date[full_date] )` — `VAR` declares a reusable DAX variable; `PeriodEnd` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `MAX` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dim_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `full_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `RETURN` — `RETURN` returns the final expression after DAX variables are defined
- **Line 5:** `COUNTROWS (` — `COUNTROWS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `FILTER (` — `FILTER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `VALUES ( fact_sales[customer_key] ),` — `VALUES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `fact_sales[customer_key] <> 0` — `fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customer_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; <> is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression

- **Line 9:** && VAR FirstOrderDate = — VAR declares a reusable DAX variable; FirstOrderDate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** CALCULATE (— CALCULATE evaluates an expression under modified DAX filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** MIN (dim_date[full_date]), — MIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dim_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; full_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** REMOVEFILTERS (dim_date) — REMOVEFILTERS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dim_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** RETURN — RETURN returns the final expression after DAX variables are defined
- **Line 15:** FirstOrderDate >= PeriodStart — FirstOrderDate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; >= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PeriodStart is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** && FirstOrderDate <= PeriodEnd — FirstOrderDate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; <= is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PeriodEnd is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Compare a list of customer first-order dates between SQL and Power BI.

Common interpretation mistake: Using signup date when the business question is first purchase.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined New Customers from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 21 — Returning Customers

Business meaning: Customers purchasing in the period whose first purchase occurred before the period.

Business formula: Purchasing customers - New Customers

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: fact_sales, dim_date

Important filters: Selected reporting period; unknown customer excluded.

Recommended format: Whole number or appropriate numeric format.

Action 16.4.M21 — Create, format, and validate the Returning Customers DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Returning Customers DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Returning Customers shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Returning Customers shown below.
6. PRESS: Enter.

7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Returning Customers =
[Purchasing Customers] - [New Customers]
```

Line-by-line DAX explanation

- **Line 1:** Returning Customers = — Returning is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** [Purchasing Customers] - [New Customers] — [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Purchasing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; - is a command option or flag; its exact behavior is defined beside the command; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; New is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Customers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Pick customers around the period boundary and inspect order history.

Common interpretation mistake: Counting repeated orders rather than distinct customers.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Returning Customers from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 22 — Inventory On Hand

Business meaning: Physical quantity recorded at the selected snapshot date.

Business formula: SUM(quantity_on_hand)

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: fact_inventory_snapshot

Important filters: A single snapshot date is usually required; do not sum across dates.

Recommended format: Whole number or appropriate numeric format.

Action 16.4.M22 — Create, format, and validate the Inventory On Hand DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Inventory On Hand DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated _Measures table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Inventory On Hand shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.

8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Inventory On Hand shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Inventory On Hand =  
SUM ( fact_inventory_snapshot[quantity_on_hand] )
```

Line-by-line DAX explanation

- **Line 1:** Inventory On Hand = — Inventory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; On is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Hand is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** SUM (fact_inventory_snapshot[quantity_on_hand]) — SUM adds numeric values in the current filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_inventory_snapshot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quantity_on_hand is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Check one date/store/product against the source snapshot.

Common interpretation mistake: Summing inventory over multiple dates, which double-counts stock.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]
- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Inventory On Hand from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

Power BI measure 23 — Stockout Risk Count

Business meaning: Count of store-product snapshot rows at or below their reorder threshold.

Business formula: COUNTROWS where stockout_risk_flag = 1

Filter context: Slicers and relationships change the rows visible to the expression; numerator and denominator must use compatible populations.

Required tables/columns: fact_inventory_snapshot

Important filters: Use latest or selected snapshot date.

Recommended format: Whole number or appropriate numeric format.

Action 16.4.M23 — Create, format, and validate the Stockout Risk Count DAX measure

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create, format, and validate the Stockout Risk Count DAX measure. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Stockout Risk Count shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open Power BI Desktop and select **Model view** or **Report view**.
2. CLICK: In the Fields/Data pane, select the dedicated `_Measures` table.
3. CLICK: On the ribbon, select **New measure**.
4. LOOK FOR: Confirm the DAX formula bar appears and begins with a default measure name.
5. TYPE THIS: Enter the exact measure definition for Stockout Risk Count shown below.
6. PRESS: Enter.
7. CLICK: Select the measure and set its format: currency, whole number, decimal, or percentage as defined by the KPI.
8. CLICK: Assign the measure to the appropriate display folder when the model uses display folders.
9. CLICK: Add a temporary card or table visual with the measure and the same filters used by the SQL validation query.
10. PRESS: Run the independent SQL query in SSMS.
11. LOOK FOR: Compare Power BI and SQL values at full precision before display rounding.
12. DO NOT CONTINUE UNTIL: The values agree or every difference is explained and corrected.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 16 lesson index](#)

DAX definition

```
Stockout Risk Count =
CALCULATE (
    COUNTROWS ( fact_inventory_snapshot ),
    fact_inventory_snapshot[stockout_risk_flag] = TRUE ()
)
```

Line-by-line DAX explanation

- **Line 1:** Stockout Risk Count = — Stockout is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Risk is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** CALCULATE (— CALCULATE evaluates an expression under modified DAX filter context; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** COUNTROWS (fact_inventory_snapshot), — COUNTROWS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_inventory_snapshot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** fact_inventory_snapshot[stockout_risk_flag] = TRUE () — fact_inventory_snapshot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stockout_risk_flag is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; TRUE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:**) —) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

SQL/DAX validation

Validation method: Recompute available quantity <= reorder point for a sample.

Common interpretation mistake: Calling a low-stock warning an actual stockout without distinguishing zero inventory.

- Power BI filter/slicer state: [ENTER]
- SQL WHERE conditions: [ENTER]
- SQL result before formatting: [ENTER]
- DAX result before display rounding: [ENTER]
- Difference: [ENTER]

- Status: [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

Common error and exact recovery

If Power BI and SQL differ, first compare date role, relationship direction, hidden filters, null/unknown keys, distinct-count grain, and numerator/denominator populations. Do not rewrite the DAX until the comparison query matches the semantic definition.

Interview-ready explanation

"I defined Stockout Risk Count from a documented business formula, used reusable base measures where appropriate, formatted it explicitly, and validated it against SQL under identical filters rather than trusting the visual alone."

What your screen should look like now

- The new Phase 16 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 16.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 16.5](#).

Lesson 16.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 16 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Power BI Import versus DirectQuery

Question:

How should Power BI access the local dimensional warehouse?

Options considered:

- Import.
- DirectQuery.
- Composite model.

Beginner path selected:

Use Import for the required beginner model.

Why this path was selected:

It is responsive, supports the full modeling experience, and suits the synthetic batch volume.

Tradeoff:

Refresh is required to see new warehouse data.

What a company might do instead:

Companies choose by latency, scale, governance, gateway, and semantic-model requirements.

How to explain this in an interview:

I selected Import because the project is batch-oriented and small; I validated refresh totals against SQL and documented DirectQuery tradeoffs.

If you see Result A

Relationships have the intended cardinality/direction and DAX matches filtered SQL. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

Power BI proposes an ambiguous or many-to-many relationship, or a measure differs under the same filter. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: Power BI Desktop, Power Query Editor, Model view, DAX formula bar, and SSMS.

Window/panel: Show Power BI model view, relationship settings, DAX measure, and matching SQL result.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase16_build-and-validate-the-power-bi-semantic-model_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 16's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 16 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 16.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use

git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

Action 16.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

Action 16.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

Interpret git status --short

- ?? means untracked: Git sees a new path but it is not staged.
- M means modified in the working tree but not staged.
- M means the staged version differs from the last commit.
- MM means a staged version exists and the file has additional unstaged edits.
- A means staged as a new file.
- D means deletion; stop and investigate unless deletion is deliberate.
- R means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 16.5.G4 — Inspect the unstaged diff for powerbi/semantic_model.md

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.

6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "powerbi/semantic_model.md"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "powerbi/semantic_model.md"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"powerbi/semantic_model.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

Action 16.5.G5 — Stage only the reviewed file powerbi/semantic_model.md

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "powerbi/semantic_model.md"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "powerbi/semantic_model.md"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"powerbi/semantic_model.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 16.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.

4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

Action 16.5.G7 — Create the Phase 16 commit

Application:

Power BI Desktop

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Power BI Desktop** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "feat: build validated power bi semantic model"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "feat: build validated power bi semantic model"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"feat: build validated power bi semantic model"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

Action 16.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

Action 16.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 16 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message `feat: build validated power bi semantic model and the intended changed paths`.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 16.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.

9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 16 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

"Phase 16 created the verified boundary described by **Build and validate the Power BI semantic model**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed."

Mini quiz

1. Multiple choice: Why does Phase 16 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

My Phase 16 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 16.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 17 uses those governed fields to construct report pages and visual interactions.

Phase 16 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 17 — Build and validate all Power BI report pages

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: Accurate report pages and SQL-to-DAX validation evidence.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 18 reviews every location where secrets, compute cost, or sensitive data can appear.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 17 lesson index

1. [Lesson 17.1 — Understand the phase and starting screen](#)
2. [Lesson 17.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 17.3 — Create and understand the phase files](#)
4. [Lesson 17.4 — Build and validate required objects](#)
5. [Lesson 17.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 17.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** Power BI Desktop Report view, Format pane, Selection pane, and SSMS validation windows.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 17 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 16 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 17 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

Power BI Desktop Report view, Format pane, Selection pane, and SSMS validation windows. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 17 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.

- No new Phase 17 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 17 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 17.1 completion checklist

☐

I read why Phase 17 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 17.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 17 lesson index](#)

Lesson 17.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 17 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

A nine-page Power BI report design, an executive KPI view, a data-quality monitoring page, a dashboard wireframe, drill-through plan, validation checklist, and at least 20 analyst SQL questions with query skeletons. You populate visuals only from your verified model and label incomplete pages honestly.

2. Why it matters

A technically correct platform has value only when people can answer business questions safely. Consumption design connects each audience, question, metric, filter, visual, and validation rule.

3. Words to know

Term	Plain-language definition
Consumption layer	The reports, SQL queries, exports, or applications through which people use trusted data.
KPI card	A visual displaying one important measure, often with period context or target.
Slicer	A report control that changes filter context.

Term	Plain-language definition
Drill-through	Navigation from a summary item to a detail page filtered to that item.
Tooltip page	A report page shown when a user hovers over a visual data point.
Wireframe	A layout plan showing where visuals and controls belong without claiming real results.
Executive view	A compact view of outcomes, changes, risks, and decisions rather than operational detail.
Operational view	A detailed view used to act on current process conditions.
Misleading chart	A visual whose scale, aggregation, category count, or encoding can cause a false interpretation.
Accessibility	Design that remains usable with keyboard navigation, readable text, adequate contrast, and meaningful labels.

4. What must already be completed

The Phase 16 semantic model is verified and saved. Every measure used by a report page has a business definition and at least one SQL validation.

5. Applications to open

Application	How to open it	Why it is open
Power BI Desktop	Open your verified PBIX.	Build and test report pages.
SSMS	Connect to NorthstarRetail_DW.	Run analyst and visual-validation queries.
VS Code	Open powerbi/dashboard_requirements.md and the SQL analytics folder.	Follow the report contract and record evidence.
Web browser	Open the local HTML notebook/GitHub preview after screenshots are added.	Review portfolio presentation without publishing private details.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run `Get-Location`; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
SELECT TOP (20)	Returns the daily series used to validate a trend visual.	A chart should be checked at its displayed grain, not only as a grand total.
SELECT p.category_name,	Creates the category comparison baseline.	It verifies relationship filtering and category grouping.
Get-Content .\sql\sqlserver\analytics\20_analyst_questions.sql Select-String '^-- [0-9][0-9].'	Lists the analyst question headings included in the reference file.	You should know the scope before attempting queries.
python -m scripts.validation.validate_markdown_links	Checks report documentation links and screenshot placeholders resolve.	Portfolio navigation is part of the consumption experience.

Page contract

Each page in `powerbi/dashboard_requirements.md` states audience, business question, measures, dimensions, slicers, recommended visuals, drill-through, SQL validation, and a common misleading alternative. A visual is not selected because it looks impressive; it is selected because its encoding fits the question.

Do not fabricate results

Wireframes may contain labels such as `[TOTAL REVENUE]` but not invented numbers. After your pipeline runs, replace placeholders only with values visible in a validated model and capture the data-through date and batch evidence.

MY TASK

Build and validate the nine report pages, complete at least 20 analyst questions, add one useful drill-through path, and create screenshots that prove your own execution without exposing credentials or inventing performance claims.

STARTER SKELETON

```
PAGE: Inventory
AUDIENCE: TODO(USER)
DECISION: TODO(USER)
GRAIN SHOWN: TODO(USER)
MEASURES: Inventory On Hand; Stockout Risk Count
DIMENSIONS: TODO(USER)
SLICERS: TODO(USER)
VISUALS: TODO(USER)
DRILL-THROUGH: TODO(USER)
SQL VALIDATION: TODO(USER)
MISLEADING CHOICE TO AVOID: TODO(USER)
SCREENSHOT FILE: docs/images/TODO_USER_inventory_page.png
```

Acceptance criteria: All nine pages have a documented audience and question; every visible KPI uses an explicit validated measure; interactions and drill-through work; each critical visual matches a same-grain SQL query; accessibility checks are recorded; screenshots show only real verified results or clearly marked placeholders.

HINT 1 — CONCEPT

Start with the decision and grain, then choose the visual.

HINT 2 — PSEUDOCODE

For each visual: identify measure, grouping columns, filters, and SQL GROUP BY; compare the resulting rows.

HINT 3 — NEAR-COMPLETE PATTERN

Use the page contracts, add a table visual temporarily, match it to the provided SQL patterns, then replace it with the final visual after validation.

13. Expected files or output

- Nine named Power BI pages.
- One drill-through flow and useful tooltip behavior.
- Executive KPI and data-quality monitoring views.
- At least 20 attempted analyst queries with notes.
- Completed screenshot checklist and evidence files.
- A wireframe that contains no fabricated numbers.

14. How to verify

- Every page answers its stated business question.
- All visual grains and filters are documented.
- No implicit measures are used for published KPIs.
- Critical chart table views match SQL.
- Slicer interactions and synced slicers are intentional.

- Drill-through retains the correct context.
- Data-quality page exposes failed/quarantined checks rather than hiding them.
- Screenshots include a verified data-through context.

15. Automated tests

- Visual-to-SQL same-grain comparisons.
- Slicer interaction matrix.
- Drill-through context test.
- Empty/no-data state test.
- Accessibility/tab-order/alt-text review.
- Screenshot privacy and truthfulness review.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
A chart changes when an unrelated slicer is used	Synced slicer or visual interaction is broader than intended.	Use Edit interactions and inspect Sync slicers.	Disable the invalid interaction or redesign the page contract.	Only intended visuals change.
Grand total differs from the sum of visible percentages	The measure is recalculated in total filter context, which is often correct for ratios.	Display numerator/denominator at row and total levels.	Explain the ratio behavior; do not sum percentages unless the business rule truly requires it.	Total matches the SQL ratio definition.
Too many categories are unreadable	A categorical visual is overloaded.	Count distinct categories and inspect audience need.	Use Top N with an explicit Other/selection path, a table, or drill-through; disclose the filter.	All omitted categories remain accessible and no hidden bias is introduced.
Map or region visual is blank/wrong	Geographic category is ambiguous or unsupported.	Inspect data category, country/state fields, and null/invalid codes.	Set correct data category and use a bar/matrix if geography cannot be resolved safely.	Locations or alternative visual match source counts.
Screenshot shows a result that later changes	Refresh batch or filters were not recorded.	Check batch/load timestamp and active filters.	Capture a data-through date and evidence note; replace stale screenshots after a material change.	Screenshot and evidence point to the same run.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. How does the audience change the amount of detail on a page?
2. Why should a visual be validated at its displayed grain?
3. Why can a grand-total percentage correctly differ from the sum of row percentages?
4. What makes a dashboard screenshot trustworthy?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. What should be defined before choosing a visual?
2. Which page must expose quality failures and quarantine counts?
3. What is the safest response when a page is not yet verified?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture each completed report page, one drill-through result, one Edit interactions view, and one SQL-to-visual validation. Use the Phase 17 in-workbook checklist; do not show fabricated numbers.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
feat: add validated consumption layer design and evidence
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

Executive Overview complete — [STATUS]

☐

Sales/Product/Store/Customer pages complete — [STATUS]

☐

Marketing/Inventory/Shipping pages complete — [STATUS]

☐

Data Quality page complete — [STATUS]

☐

Drill-through and interactions tested — [STATUS]

☐

20 analyst queries attempted — [STATUS]

☐

Visual-to-SQL checks passed — [STATUS]

☐

Screenshots reviewed and saved — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-17. The quiz answer is in the answer-key section immediately after the quiz in this workbook#phase-17.

Next phase: Phase 18 protects configuration, credentials, access, logs, and cost while keeping the synthetic portfolio safe to publish.

Required action cards

Action 17.2.1 — Read `powerbi/dashboard_requirements.md`. For each page, write the audience and one decision the page should s

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Read `powerbi/dashboard_requirements.md`. For each page, write the audience and one decision the page should s. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Read `powerbi/dashboard_requirements.md`. For each page, write the audience and one decision the page should support before adding a visual.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 1****Action**

Read `powerbi/dashboard_requirements.md`. For each page, write the audience and one decision the page should support before adding a visual.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

Action 17.2.2 — In Power BI Report view, rename the first page . Open and choose Fit to page while designing.

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

In Power BI Report view, rename the first page . Open and choose Fit to page while designing. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: In Power BI Report view, rename the first page **Executive Overview**. Open **View > Page view** and choose Fit to page while designing.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 2

Action

In Power BI Report view, rename the first page **Executive Overview**. Open **View > Page view** and choose Fit to page while designing.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.



I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

Action 17.2.3 — Add a horizontal date slicer and channel/store slicers that apply only where useful. Use intentionally; do not

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Add a horizontal date slicer and channel/store slicers that apply only where useful. Use intentionally; do not. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.

3. Perform only this immediate task: Add a horizontal date slicer and channel/store slicers that apply only where useful. Use **View > Sync slicers** intentionally; do not make every slicer global without testing.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 3****Action**

Add a horizontal date slicer and channel/store slicers that apply only where useful. Use **View > Sync slicers** intentionally; do not make every slicer global without testing.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

Action 17.2.4 — Add KPI cards for revenue, gross profit, gross margin, orders, average order value, return rate, on-time deliv

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Add KPI cards for revenue, gross profit, gross margin, orders, average order value, return rate, on-time deliv. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Add KPI cards for revenue, gross profit, gross margin, orders, average order value, return rate, on-time delivery, and stockout risk. Add a visible "Data through" date sourced from the model.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 4

Action

Add KPI cards for revenue, gross profit, gross margin, orders, average order value, return rate, on-time delivery, and stockout risk. Add a visible "Data through" date sourced from the model.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

Action 17.2.5 — Add a monthly revenue/profit trend and category/store comparison. Use consistent units and zero baselines wher

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Add a monthly revenue/profit trend and category/store comparison. Use consistent units and zero baselines wher. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Add a monthly revenue/profit trend and category/store comparison. Use consistent units and zero baselines where a bar length represents magnitude.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 5****Action**

Add a monthly revenue/profit trend and category/store comparison. Use consistent units and zero baselines where a bar length represents magnitude.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

Action 17.2.6 — Create the remaining pages: Sales Performance; Product and Category; Store and Region; Customer; Marketing Fun

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create the remaining pages: Sales Performance; Product and Category; Store and Region; Customer; Marketing Fun. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Create the remaining pages: Sales Performance; Product and Category; Store and Region; Customer; Marketing Funnel; Inventory; Shipping and Returns; Data Quality.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 6****Action**

Create the remaining pages: Sales Performance; Product and Category; Store and Region; Customer; Marketing Funnel; Inventory; Shipping and Returns; Data Quality.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

Action 17.2.7 — For each page, use the audience/question/visual/field/filter/drill-through/validation specification in the das

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

For each page, use the audience/question/visual/field/filter/drill-through/validation specification in the das. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: For each page, use the audience/question/visual/field/filter/drill-through/validation specification in the dashboard requirements. Limit visuals to those needed for the decision.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 7

Action

For each page, use the audience/question/visual/field/filter/drill-through/validation specification in the dashboard requirements. Limit visuals to those needed for the decision.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

Action 17.2.8 — Create a Product Detail drill-through page or drill-through region that includes product, trend, returns, inve

Application:

Windows File Explorer

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create a Product Detail drill-through page or drill-through region that includes product, trend, returns, inve. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Create a Product Detail drill-through page or drill-through region that includes product, trend, returns, inventory, and margin. Test right-click drill-through from at least two source pages.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Windows File Explorer. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 8****Action**

Create a Product Detail drill-through page or drill-through region that includes product, trend, returns, inventory, and margin. Test right-click drill-through from at least two source pages.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

Action 17.2.9 — Create tooltips for definitions or context, not hidden essential facts. Add alt text to visuals and set tab or

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Create tooltips for definitions or context, not hidden essential facts. Add alt text to visuals and set tab or. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Create tooltips for definitions or context, not hidden essential facts. Add alt text to visuals and set tab order through **View > Selection** and **View > Tab order** where available.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 9

Action

Create tooltips for definitions or context, not hidden essential facts. Add alt text to visuals and set tab order through **View > Selection** and **View > Tab order** where available.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

Action 17.2.10 — Open and test how every slicer/visual affects other visuals. Disable interactions that imply an invalid compar

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Open and test how every slicer/visual affects other visuals. Disable interactions that imply an invalid compar. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open **Format > Edit interactions** and test how every slicer/visual affects other visuals. Disable interactions that imply an invalid comparison.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 10****Action**

Open **Format > Edit interactions** and test how every slicer/visual affects other visuals. Disable interactions that imply an invalid comparison.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

Action 17.2.11 — For each visual, use or export a small summarized set and compare it to an SSMS query with identical filters a

Application:

SQL Server Management Studio (SSMS)

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

For each visual, use or export a small summarized set and compare it to an SSMS query with identical filters a. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: For each visual, use **Show as a table** or export a small summarized set and compare it to an SSMS query with identical filters and grouping.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 11

Action

For each visual, use **Show as a table** or export a small summarized set and compare it to an SSMS query with identical filters and grouping.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

Action 17.2.12 — Run the analyst question file under sql/sqlserver/analytics/. Attempt each TODO(USER) before opening the r

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run the analyst question file under `sql/sqlserver/analytics/`. Attempt each `TODO(USER)` before opening the r. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run the analyst question file under `sql/sqlserver/analytics/`. Attempt each `TODO(USER)` before opening the reference appendix. Add your own business interpretation below the query result.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 12****Action**

Run the analyst question file under `sql/sqlserver/analytics/`. Attempt each `TODO(USER)` before opening the reference appendix. Add your own business interpretation below the query result.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

Action 17.2.13 — Use the Phase 17 screenshot checkpoints in this workbook and the screenshot checkpoints in this workbook.

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Use the Phase 17 screenshot checkpoints in this workbook and the screenshot checkpoints in this workbook. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Use the Phase 17 screenshot checkpoints in this workbook and the screenshot checkpoints in this workbook. Capture only verified pages; use placeholder boxes in README for pages not yet built.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 13

Action

Use the Phase 17 screenshot checkpoints in this workbook and the screenshot checkpoints in this workbook. Capture only verified pages; use placeholder boxes in README for pages not yet built.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

Action 17.2.14 — Do not publish a public report unless you understand tenant settings, sharing permissions, synthetic-data labe

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Do not publish a public report unless you understand tenant settings, sharing permissions, synthetic-data labe. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Do not publish a public report unless you understand tenant settings, sharing permissions, synthetic-data labeling, and account visibility. A PBIX plus screenshots is sufficient for the local-first portfolio.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 14****Action**

Do not publish a public report unless you understand tenant settings, sharing permissions, synthetic-data labeling, and account visibility. A PBIX plus screenshots is sufficient for the local-first portfolio.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

Action 17.2.C1 — SSMS query window (SQL Server / T-SQL)

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SSMS query window (SQL Server / T-SQL)

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):

```
SELECT TOP (20)
    d.full_date,
    SUM(f.net_sales_amount) AS total_revenue,
    SUM(f.gross_profit_amount) AS gross_profit
FROM dw.fact_sales AS f
JOIN dw.dim_date AS d ON d.date_key = f.order_date_key
GROUP BY d.full_date
ORDER BY d.full_date DESC;
```

Where to enter it:

SSMS query window (SQL Server / T-SQL). Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** SELECT TOP (20) — SELECT requests data without changing it unless combined with a modifying statement; TOP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the

surrounding explanation; 20 is a numeric literal used by the command or expression;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 2:** `d.full_date, — d.full_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `SUM(f.net_sales_amount) AS total_revenue, — SUM` adds numeric values in the current filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f.net_sales_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `total_revenue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `SUM(f.gross_profit_amount) AS gross_profit — SUM` adds numeric values in the current filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f.gross_profit_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gross_profit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `FROM dw.fact_sales AS f — FROM` identifies the source table or query; `dw.fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `JOIN dw.dim_date AS d ON d.date_key = f.order_date_key — JOIN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dw.dim_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `d` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `d.date_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f.order_date_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `GROUP BY d.full_date — GROUP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `d.full_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `ORDER BY d.full_date DESC; — ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `d.full_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DESC` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

Action 17.2.C2 — SSMS query window (SQL Server / T-SQL)

Application:

SQL Server Management Studio (SSMS)

Window or panel:

SSMS query window (SQL Server / T-SQL)

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained Sql instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Sql):


```

SELECT p.category_name,
       SUM(f.net_sales_amount) AS revenue,
       SUM(f.gross_profit_amount) AS gross_profit
FROM dw.fact_sales AS f
JOIN dw.dim_product AS p ON p.product_key = f.product_key
GROUP BY p.category_name
ORDER BY revenue DESC;

```

Where to enter it:

SSMS query window (SQL Server / T-SQL). Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `SELECT p.category_name,` — `SELECT` requests data without changing it unless combined with a modifying statement; `p.category_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `SUM(f.net_sales_amount) AS revenue,` — `SUM` adds numeric values in the current filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f.net_sales_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `revenue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `SUM(f.gross_profit_amount) AS gross_profit` — `SUM` adds numeric values in the current filter context; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f.gross_profit_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gross_profit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `FROM dw.fact_sales AS f` — `FROM` identifies the source table or query; `dw.fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `JOIN dw.dim_product AS p ON p.product_key = f.product_key` — `JOIN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dw.dim_product` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `p` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `p.product_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f.product_key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `GROUP BY p.category_name` — `GROUP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `p.category_name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `ORDER BY revenue DESC;` — `ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `revenue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DESC` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

Action 17.2.C3 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Get-Content .\sql\sqlserver\analytics\20_analyst_questions.sql | Select-String '^-- [0-9][0-9]\.'
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `Get-Content .\sql\sqlserver\analytics\20_analyst_questions.sql | Select-String '^-- [0-9][0-9]\.'`
 — `Get-Content` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sqlserver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `analytics` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `20_analyst_questions.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Select-String` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'^-- [0-9][0-9]\.'` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

Action 17.2.C4 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m scripts.validation.validate_markdown_links
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m scripts.validation.validate_markdown_links` — python invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.validate_markdown_links` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 17.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]



I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 17 starting-screen checklist and re-verify the active context.

Lesson 17.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 17 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — powerbi/

Exact folder name: powerbi

Exact parent: repository root

Why it is needed now: Phase 17 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: powerbi.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 17.D1.1 — Create the folder powerbi

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\powerbi" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\powerbi" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\powerbi"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\powerbi" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\powerbi" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\powerbi" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/

Exact folder name: `sql`

Exact parent: `repository root`

Why it is needed now: Phase 17 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `repository root` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `sql`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method**Action 17.D2.1 — Create the folder `sql`**

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\sql" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\sql" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\sql" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/sqlserver/

Exact folder name: `sqlserver`

Exact parent: `sql`

Why it is needed now: Phase 17 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path sql one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: sqlserver.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 17.D3.1 — Create the folder sql/sqlserver

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\sqlserver" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\sql\sqlserver" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\sql\sqlserver" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\sql\sqlserver" -PathType Container` and require True.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\sqlserver" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\sqlserver" -PathType Container`.

2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

Folder introduction — sql/sqlserver/analytics/

Exact folder name: analytics

Exact parent: sql/sqlserver

Why it is needed now: Phase 17 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path sql/sqlserver one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: analytics.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 17.D4.1 — Create the folder sql/sqlserver/analytics

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.

7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\sql\sqlserver\analytics" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\sql\sqlserver\analytics" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\sqlserver\analytics"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\sql\sqlserver\analytics" -PathType Container` and require `True`.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\sql\sqlserver\analytics" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\sql\sqlserver\analytics" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — `powerbi/dashboard_requirements.md`

Exact filename: `dashboard_requirements.md`

Filename extension: `.md`. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `powerbi` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 17.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `powerbi` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `dashboard_requirements.md`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `dashboard_requirements.md` and an empty file under the correct parent folder.
10. PRESS: `Ctrl+S` after adding each explained section.

PowerShell path

Action 17.F1.1 — Create the file `powerbi/dashboard_requirements.md` from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\powerbi\dashboard_requirements.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType File -Path ".\powerbi\dashboard_requirements.md" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; File is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\powerbi\dashboard_requirements.md" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\powerbi\dashboard_requirements.md" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "powerbi/dashboard_requirements.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `powerbi`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `powerbi/dashboard_requirements.md`.
3. In PowerShell at the repository root, run `Test-Path ".\powerbi\dashboard_requirements.md"` and require `True`.
4. Run `git status --short -- "powerbi/dashboard_requirements.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for `powerbi/dashboard_requirements.md`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: powerbi/dashboard_requirements.md

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 17.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Power BI Report Requirements
```

No actual performance values are included. Replace every placeholder only after running and validating your pipeline.

```
| Page | Audience | Business question | Fields/KPIs | Slicers | Visuals | Drill-through | Validation | Misleading choice to avoid |
| --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Executive Overview | Executives | Are revenue, profit, orders, delivery, and returns on track? | Revenue, gross profit, margin, orders, AOV, return rate, on-time delivery; monthly trend; channel mix | Date, region, channel | KPI cards, line chart, decomposition or bar chart | Sales Performance | Validate every KPI card to SQL; verify date slicer affects all visuals | Do not use a pie chart with many categories or a truncated trend axis. |
| Sales Performance | Sales leadership | Where are sales and profit growing or declining? | Revenue/profit by month, channel, store, region; order and AOV trends | Date, channel, region, store | Lines, clustered bars, matrix, waterfall for change | Order detail or Store and Region | Check totals and drill rows against semantic.vw_sales_detail | Do not mix revenue and margin percentages on one unlabeled axis. |
| Product and Category | Merchandising | Which products and categories drive profitable demand? | Revenue, units, gross profit, margin, discount by SKU/category | Date, category, brand, supplier | Treemap only for high-level mix, bars, scatter price vs margin, matrix | Product detail | Validate top N against analyst query 07 | Top revenue is not automatically top profit. |
| Store and Region | Retail operations | Which stores and regions outperform after accounting for volume? | Revenue, profit, orders, AOV, return rate by store/region | Date, region, store | Map only if location quality is verified; bars and matrix preferred | Store detail | Compare a selected store to SQL query 03 | Avoid maps that imply area size equals performance. |
| Customer | CRM and Leadership | How many new and returning customers buy, and how do segments behave? | New/returning, purchasing customers, revenue by tier/state | Date, loyalty tier, state | Cards, cohort-style matrix, bars | Customer history | Inspect first purchase dates for sample customers | Do not label signup as acquisition unless business definition says so. |
| Marketing Funnel | Marketing | Which channels and campaigns create efficient, attributable conversion? | Spend, sessions, conversion, attributed revenue, ROAS | Date, channel, campaign, device | Funnel, scatter spend vs revenue, campaign table | Campaign detail | Verify converted order IDs are unique before ROAS | Correlation and last-touch attribution are not causation. |
| Inventory | Inventory operations | Which store-product combinations are at stockout risk? | On hand, available, inventory value, risk count | Snapshot date, region, store, category | Cards, heatmap/matrix, ranked bars | Product-store detail | Use one snapshot date; recompute risk formula | Never sum snapshot stock across dates. |
| Shipping and Returns | Operations and CX | Where are delivery delays and returns concentrated? | On-time %, delivery days, returned units, return rate, refund amount | Ship/return date, carrier, region, reason | Cards, bars, distribution, matrix | Shipment/order detail | Sample promised vs actual dates and returned lines | Do not compare shipment count directly with order count without split-shipment context. |
| Data Quality | Data engineering and owners | Can decision-makers trust the latest batch? | Pass/fail, quarantines, unknown keys, freshness, reconciliation differences | Batch, date, source, rule | Status cards, rule table, trend | Batch detail | Compare to quality.test_results and audit.reconciliation_result | A green dashboard is meaningless if failed tests were filtered out. |
```

```
## Suggested executive wireframe
```

```
``text
+-----+
| Northstar Retail – Executive Overview | [Date] [Region] [Channel] |
+-----+-----+-----+-----+-----+-----+
| Revenue | Gross | Margin % | Orders | AOV | On-time / Returns |
| [VERIFY] | [VERIFY] | [VERIFY] | [VERIFY] | [VERIFY] | [VERIFY] |
+-----+-----+-----+-----+-----+-----+
| Monthly Revenue and Gross Profit Trend | Revenue / Profit by Channel |
| [YOUR VERIFIED VISUAL] | [YOUR VERIFIED VISUAL] |
+-----+-----+-----+-----+-----+-----+
| Top/Bottom Category or Store | Alerts: Stockout / DQ / Shipping |
| [YOUR VERIFIED VISUAL] | [YOUR VERIFIED VISUAL] |
+-----+-----+-----+-----+-----+-----+
```

```
#### Line-by-line explanation
```

```
- **Line 1:** `# Power BI Report Requirements` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** `No actual performance values are included. Replace every placeholder only after running and validating your pipeline.` – `No` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `actual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `performance` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `values` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `are` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `included` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Replace` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `every` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `placeholder` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `only` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `after` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** `| Page | Audience | Business question | Fields/KPIs | Slicers | Visuals | Drill-through | Validation | Misleading choice to avoid |` – `Page` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Audience` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Business` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `question` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Fields` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `KPIs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Slicers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Visuals` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Drill-through` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Validation` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Misleading choice to avoid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.
```


cific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 12:** `| Marketing Funnel | Marketing | Which channels and campaigns create efficient, attributable conversion? | Spend, sessions, conversion, attributed revenue, ROAS | Date, channel, campaign, device | Funnel, scatter spend vs revenue, campaign table | Campaign detail | Verify converted order IDs are unique before ROAS | Correlation and last-touch attribution are not causation. |` - `Marketing` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Funnel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Marketing` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Which` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `channels` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `campaigns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `create` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `efficient` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `attributable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `conversion` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 13:** `| Inventory | Inventory operations | Which store-product combinations are at stockout risk? | On hand, available, inventory value, risk count | Snapshot date, region, store, category | Cards, heatmap/matrix, ranked bars | Product-store detail | Use one snapshot date; recompute risk formula | Never sum snapshot stock across dates. |` - `Inventory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Inventory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `operations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Which` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `store-product` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `combinations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `are` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `at` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `stockout` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `risk` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `On` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `hand` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 14:** `| Shipping and Returns | Operations and CX | Where are delivery delays and returns concentrated? | On-time %, delivery days, returned units, return rate, refund amount | Ship/return date, carrier, region, reason | Cards, bars, distribution, matrix | Shipment/order detail | Sample promised vs actual dates and returned lines | Do not compare shipment count directly with order count without split-shipment context. |` - `Shipping` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Returns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Operations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CX` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Where` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `are` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `delivery` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `delays` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `returns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 15:** `| Data Quality | Data engineering and owners | Can decision-makers trust the latest batch? | Pass/fail, quarantines, unknown keys, freshness, reconciliation differences | Batch, date, source, rule | Status cards, rule table, trend | Batch detail | Compare to quality.test_results and audit.reconciliation_result | A green dashboard is meaningless if failed tests were filtered out. |` - `Data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Quality` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `engineering` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `owners` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Can` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `decision-makers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `trust` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `latest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 16:** blank line used only to separate logical sections for readability.

- **Line 17:** `## Suggested executive wireframe` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 18:** blank line used only to separate logical sections for readability.

- **Line 19:** `\\`text` - `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 20:** `+-----+` -
`+-----+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 21:** `| Northstar Retail - Executive Overview [Date] [Region] [Channel] |` - `Northstar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Retail` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Executive` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Overview` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Region` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Channel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 22:** `+-----+` - `+-----+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 23:** `| Revenue | Gross | Margin % | Orders | AOV | On-time / Returns |` - `Revenue` is a name, keyword, argument,

```
**Exact filename:** `20 analyst questions.sql`
```

```

**Filename extension:** `.sql`. In this project it identifies database or lakehouse definitions and queries and helps the correct application
choose syntax highlighting or execution behavior.
**Exact parent folder:** `sql/sqlserver/analytics` relative to the repository root.
**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transforma-
tions, tests, documentation, and report assets without guessing.
**What the file will contain:** database or lakehouse definitions and queries for Phase 17.
**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence
may depend on its exact path.
**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific arti-
fact.
**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.
**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

#### VS Code click path

1. CLICK: In VS Code, open Explorer from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `sql/sqlserver/analytics` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select New File.
7. TYPE THIS: `20_analyst_questions.sql`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `20_analyst_questions.sql` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

#### PowerShell path

<a id="action-17-f2-1"></a>
Action 17.F2.1 – Create the file `sql/sqlserver/analytics/20_analyst_questions.sql` from PowerShell

Application:
Windows PowerShell or the VS Code PowerShell terminal

Window or panel:
VS Code integrated PowerShell terminal opened at the repository root

Starting condition:
The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, cata-
log, schema, table, or file is the correct one.

Current folder or location:
The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named
in the action.

What you are about to do:
Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:
The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by
later output.

Exact physical actions:

1. CLICK: Bring Windows PowerShell or the VS Code PowerShell terminal to the front.
2. LOOK FOR: Confirm the current location matches `The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and
the open Power BI Desktop project/model view named in the action.` or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press Enter for a one-line terminal command, F5/Execute for the named database editor, or the notebook cell Run control as
stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```powershell
New-Item -ItemType File -Path ".\sql\sqlserver\analytics\20_analyst_questions.sql" -ErrorAction Stop

```

### Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\sqlserver\analytics\20_analyst_questions.sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.`

`\sql\sqlserver\analytics\20_analyst_questions.sql` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### **Does it change anything?**

This can change files, Git state, packages, database objects, or data.

### **Is it safe to rerun?**

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### **What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### **Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### **What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### **How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### **Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

### **Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

### **Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

### **What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

### **How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

## **File-creation decision branches**

### **If the command creates the file**

Run `Test-Path -LiteralPath ".\sql\sqlserver\analytics\20_analyst_questions.sql" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

### **If PowerShell says the file already exists**

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/sqlserver/analytics/20_analyst_questions.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

### **If PowerShell says part of the path does not exist**

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

## File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use TODO(USER) where the workbook requires your own decision or verified value.

## File verification

1. In the VS Code Explorer, confirm the file appears under sql/sqlserver/analytics.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with sql/sqlserver/analytics/20\_analyst\_questions.sql.
3. In PowerShell at the repository root, run Test-Path ".\sql\sqlserver\analytics\20\_analyst\_questions.sql" and require True.
4. Run git status --short -- "sql/sqlserver/analytics/20\_analyst\_questions.sql" to see only this file's Git state.
5. Run the phase-specific validator before committing.

## Optional reference answer for sql/sqlserver/analytics/20\_analyst\_questions.sql

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** SQL

**Exact file:** sql/sqlserver/analytics/20\_analyst\_questions.sql

**Purpose:** database or lakehouse definitions and queries.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 17.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: SQL Server / T-SQL
-- Teaching analyst questions. Complete each TODO(USER), compare to the reference appendix,
-- and record your validation in the learning journal.
USE NorthstarRetail_DW;
GO

-- 01. Revenue by month.
SELECT d.calendar_year, d.month_number, SUM(f.net_sales_amount) AS revenue
FROM dw.fact_sales f JOIN dw.dim_date d ON d.date_key=f.order_date_key
GROUP BY d.calendar_year,d.month_number ORDER BY d.calendar_year,d.month_number;

-- 02. Gross profit by category.
SELECT p.category_name, SUM(f.gross_profit_amount) AS gross_profit
FROM dw.fact_sales f JOIN dw.dim_product p ON p.product_key=f.product_key
GROUP BY p.category_name ORDER BY gross_profit DESC;

-- 03. Best-performing stores by revenue. TODO(USER): add order count and gross margin.
SELECT s.store_name, SUM(f.net_sales_amount) AS revenue
FROM dw.fact_sales f JOIN dw.dim_store s ON s.store_key=f.store_key
GROUP BY s.store_name ORDER BY revenue DESC;

-- 04. Online versus store average order value.
SELECT c.channel_name, SUM(f.net_sales_amount)/NULLIF(COUNT(DISTINCT f.order_id),0) AS average_order_value
FROM dw.fact_sales f JOIN dw.dim_channel c ON c.channel_key=f.channel_key
GROUP BY c.channel_name;

-- 05. Return rate by category. TODO(USER): define numerator and denominator at compatible grain.
-- Hint: aggregate return quantity and sold quantity separately before joining.

-- 06. On-time delivery percentage by carrier.
SELECT carrier, 100.0*SUM(CASE WHEN on_time_flag=1 THEN 1 ELSE 0 END)/NULLIF(COUNT_BIG(*),0) AS on_time_pct
FROM dw.fact_shipments GROUP BY carrier;

-- 07. Top ten products by units sold.
SELECT TOP (10) p.sku,p.product_name,SUM(f.quantity) AS units
FROM dw.fact_sales f JOIN dw.dim_product p ON p.product_key=f.product_key
GROUP BY p.sku,p.product_name ORDER BY units DESC;

-- 08. Products with negative gross profit. TODO(USER): add a date filter.
SELECT p.sku,p.product_name,SUM(f.gross_profit_amount) AS gross_profit
FROM dw.fact_sales f JOIN dw.dim_product p ON p.product_key=f.product_key
GROUP BY p.sku,p.product_name HAVING SUM(f.gross_profit_amount)<0;

-- 09. Discount amount by promotion.
SELECT pr.promotion_name,SUM(f.discount_amount) AS discount_amount,SUM(f.net_sales_amount) AS revenue
FROM dw.fact_sales f JOIN dw.dim_promotion pr ON pr.promotion_key=f.promotion_key
GROUP BY pr.promotion_name ORDER BY discount_amount DESC;
```



```
-- 10. New customers by month. TODO(USER): use dim_customer.signup_date and avoid double-counting SCD2 versions.

-- 11. Returning customers by month. TODO(USER): define returning as a customer with an earlier order.

-- 12. Conversion rate by source channel.
SELECT c.channel_name,100.0*SUM(CASE WHEN f.converted_flag=1 THEN 1 ELSE 0 END)/NULLIF(SUM(f.session_count),0) AS conversion_pct
FROM dw.fact_web_sessions f JOIN dw.dim_channel c ON c.channel_key=f.channel_key
GROUP BY c.channel_name;

-- 13. Marketing spend and clicks by campaign.
SELECT c.campaign_name,SUM(f.spend_amount) AS spend,SUM(f.clicks) AS clicks
FROM dw.fact_marketing_spend f JOIN dw.dim_campaign c ON c.campaign_key=f.campaign_key
GROUP BY c.campaign_name ORDER BY spend DESC;

-- 14. Return on ad spend. TODO(USER): decide and document an attribution rule before joining sales and spend.

-- 15. Stockout-risk products by store.
SELECT s.store_name,p.product_name,COUNT_BIG(*) AS risk_snapshot_count
FROM dw.fact_inventory_snapshot f
JOIN dw.dim_store s ON s.store_key=f.store_key
JOIN dw.dim_product p ON p.product_key=f.product_key
WHERE f.stockout_risk_flag=1 GROUP BY s.store_name,p.product_name;

-- 16. Inventory value by category.
SELECT p.category_name,SUM(f.inventory_value_amount) AS inventory_value
FROM dw.fact_inventory_snapshot f JOIN dw.dim_product p ON p.product_key=f.product_key
GROUP BY p.category_name;

-- 17. Average delivery days by region.
SELECT s.region,AVG(1.0*f.delivery_days) AS average_delivery_days
FROM dw.fact_shipments f JOIN dw.dim_store s ON s.store_key=f.store_key
GROUP BY s.region;

-- 18. Customers whose attributes changed under SCD2.
SELECT customer_number,COUNT_BIG(*) AS version_count
FROM dw.dim_customer GROUP BY customer_number HAVING COUNT_BIG(*)>1;

-- 19. Sales using unknown dimension members.
SELECT
SUM(CASE WHEN customer_key=0 THEN 1 ELSE 0 END) AS unknown_customer_rows,
SUM(CASE WHEN product_key=0 THEN 1 ELSE 0 END) AS unknown_product_rows,
SUM(CASE WHEN store_key=0 THEN 1 ELSE 0 END) AS unknown_store_rows
FROM dw.fact_sales;

-- 20. Daily data-quality failures.
SELECT CAST(changed_at_utc AS date) AS check_date,COUNT_BIG(*) AS failed_checks
FROM audit.reconciliation_result WHERE status='FAIL'
GROUP BY CAST(changed_at_utc AS date) ORDER BY check_date;
```

## Line-by-line explanation

- **Line 1:** -- Dialect: SQL Server / T-SQL is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** -- Teaching analyst questions. Complete each TODO(USER), compare to the reference appendix, is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** -- and record your validation in the learning journal. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** USE NorthstarRetail\_DW; — USE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NorthstarRetail\_DW is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** -- 01. Revenue by month. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 8:** SELECT d.calendar\_year, d.month\_number, SUM(f.net\_sales\_amount) AS revenue — SELECT requests data without changing it unless combined with a modifying statement; d.calendar\_year is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; d.month\_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.net\_sales\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 9:** FROM dw.fact\_sales f JOIN dw.dim\_date d ON d.date\_key=f.order\_date\_key — FROM identifies the source table or query; dw.fact\_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim\_date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; d is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.order\_date\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** GROUP BY d.calendar\_year,d.month\_number ORDER BY d.calendar\_year,d.month\_number; — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; d.calendar\_year is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; d.month\_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; d.calendar\_year is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; d.month\_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** blank line used only to separate logical sections for readability.
- **Line 12:** -- 02. Gross profit by category. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 13:** SELECT p.category\_name, SUM(f.gross\_profit\_amount) AS gross\_profit — SELECT requests data without changing it unless combined with a modifying statement; p.category\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.gross\_profit\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gross\_profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** FROM dw.fact\_sales f JOIN dw.dim\_product p ON p.product\_key=f.product\_key — FROM identifies the source table or query; dw.fact\_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim\_product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.product\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.product\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** GROUP BY p.category\_name ORDER BY gross\_profit DESC; — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.category\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

surrounding explanation; gross\_profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DESC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 16:** blank line used only to separate logical sections for readability.
- **Line 17:** -- 03. Best-performing stores by revenue. TODO(USER): add order count and gross margin. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 18:** SELECT s.store\_name, SUM(f.net\_sales\_amount) AS revenue — SELECT requests data without changing it unless combined with a modifying statement; s.store\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.net\_sales\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** FROM dw.fact\_sales f JOIN dw.dim\_store s ON s.store\_key=f.store\_key — FROM identifies the source table or query; dw.fact\_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim\_store is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s.store\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.store\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** GROUP BY s.store\_name ORDER BY revenue DESC; — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s.store\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DESC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** blank line used only to separate logical sections for readability.
- **Line 22:** -- 04. Online versus store average order value. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 23:** SELECT c.channel\_name, SUM(f.net\_sales\_amount)/NULLIF(COUNT(DISTINCT f.order\_id),0) AS average\_order\_value — SELECT requests data without changing it unless combined with a modifying statement; c.channel\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.net\_sales\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; )/ is a path, relative location, branch-style name, or file argument; verify every character before running it; NULLIF is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DISTINCT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 24:** FROM dw.fact\_sales f JOIN dw.dim\_channel c ON c.channel\_key=f.channel\_key — FROM identifies the source table or query; dw.fact\_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim\_channel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

er, or literal whose project-specific role is described in the surrounding explanation; c is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; c.channel\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.channel\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 25:** GROUP BY c.channel\_name; — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; c.channel\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** blank line used only to separate logical sections for readability.
- **Line 27:** -- 05. Return rate by category. TODO(USER): define numerator and denominator at compatible grain. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 28:** -- Hint: aggregate return quantity and sold quantity separately before joining. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 29:** blank line used only to separate logical sections for readability.
- **Line 30:** -- 06. On-time delivery percentage by carrier. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 31:** SELECT carrier, 100.0\*SUM(CASE WHEN on\_time\_flag=1 THEN 1 ELSE 0 END)/NULLIF(COUNT\_BIG(\*),0) AS on\_time\_pct — SELECT requests data without changing it unless combined with a modifying statement; carrier is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100.0 is a numeric literal used by the command or expression; \* is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on\_time\_flag is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 32:** FROM dw.fact\_shipments GROUP BY carrier; — FROM identifies the source table or query; dw.fact\_shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; carrier is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** blank line used only to separate logical sections for readability.
- **Line 34:** -- 07. Top ten products by units sold. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 35:** SELECT TOP (10) p.sku,p.product\_name,SUM(f.quantity) AS units — SELECT requests data without changing it unless combined with a modifying statement; TOP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 10 is a numeric literal used by the command or expression; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.sku is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.product\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.quantity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 36:** FROM dw.fact\_sales f JOIN dw.dim\_product p ON p.product\_key=f.product\_key — FROM identifies the source table or query; dw.fact\_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim\_product is a name, keyword, argument, identifier,

er, or literal whose project-specific role is described in the surrounding explanation; p is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.product\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.product\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 37:** GROUP BY p.sku,p.product\_name ORDER BY units DESC; — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.sku is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.product\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; units is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DESC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 38:** blank line used only to separate logical sections for readability.
- **Line 39:** -- 08. Products with negative gross profit. TODO(USER): add a date filter. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 40:** SELECT p.sku,p.product\_name,SUM(f.gross\_profit\_amount) AS gross\_profit — SELECT requests data without changing it unless combined with a modifying statement; p.sku is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.product\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.gross\_profit\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gross\_profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 41:** FROM dw.fact\_sales f JOIN dw.dim\_product p ON p.product\_key=f.product\_key — FROM identifies the source table or query; dw.fact\_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim\_product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.product\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.product\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** GROUP BY p.sku,p.product\_name HAVING SUM(f.gross\_profit\_amount)<0; — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.sku is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.product\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; HAVING is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.gross\_profit\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; )< is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression
- **Line 43:** blank line used only to separate logical sections for readability.

- **Line 44:** -- 09. Discount amount by promotion. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 45:** SELECT pr.promotion\_name,SUM(f.discount\_amount) AS discount\_amount,SUM(f.net\_sales\_amount) AS revenue — SELECT requests data without changing it unless combined with a modifying statement; pr.promotion\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.discount\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; discount\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 46:** FROM dw.fact\_sales f JOIN dw.dim\_promotion pr ON pr.promotion\_key=f.promotion\_key — FROM identifies the source table or query; dw.fact\_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim\_promotion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pr is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pr.promotion\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.promotion\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 47:** GROUP BY pr.promotion\_name ORDER BY discount\_amount DESC; — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pr.promotion\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; discount\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DESC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 48:** blank line used only to separate logical sections for readability.
- **Line 49:** -- 10. New customers by month. TODO(USER): use dim\_customer.signup\_date and avoid double-counting SCD2 versions. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 50:** blank line used only to separate logical sections for readability.
- **Line 51:** -- 11. Returning customers by month. TODO(USER): define returning as a customer with an earlier order. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 52:** blank line used only to separate logical sections for readability.
- **Line 53:** -- 12. Conversion rate by source channel. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 54:** SELECT c.channel\_name,100.0\*SUM(CASE WHEN f.converted\_flag=1 THEN 1 ELSE 0 END)/NULLIF(SUM(f.session\_count),0) AS conversion\_pct — SELECT requests data without changing it unless combined with a modifying statement; c.channel\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100.0 is a numeric literal used by the command or expression; \* is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.converted\_flag is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

ation; 1 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.

- **Line 55:** FROM dw.fact\_web\_sessions f JOIN dw.dim\_channel c ON c.channel\_key=f.channel\_key — FROM identifies the source table or query; dw.fact\_web\_sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim\_channel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; c is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; c.channel\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.channel\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 56:** GROUP BY c.channel\_name; — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; c.channel\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 57:** blank line used only to separate logical sections for readability.
- **Line 58:** -- 13. Marketing spend and clicks by campaign. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 59:** SELECT c.campaign\_name,SUM(f.spend\_amount) AS spend,SUM(f.clicks) AS clicks — SELECT requests data without changing it unless combined with a modifying statement; c.campaign\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.spend\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 60:** FROM dw.fact\_marketing\_spend f JOIN dw.dim\_campaign c ON c.campaign\_key=f.campaign\_key — FROM identifies the source table or query; dw.fact\_marketing\_spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim\_campaign is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; c is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; c.campaign\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.campaign\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 61:** GROUP BY c.campaign\_name ORDER BY spend DESC; — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; c.campaign\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DESC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 62:** blank line used only to separate logical sections for readability.
- **Line 63:** -- 14. Return on ad spend. TODO(USER): decide and document an attribution rule before joining sales and spend. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 64:** blank line used only to separate logical sections for readability.

- **Line 65:** -- 15. Stockout-risk products by store. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 66:** SELECT s.store\_name,p.product\_name,COUNT\_BIG(\*) AS risk\_snapshot\_count — SELECT requests data without changing it unless combined with a modifying statement; s.store\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.product\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT\_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (\*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; risk\_snapshot\_count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 67:** FROM dw.fact\_inventory\_snapshot f — FROM identifies the source table or query; dw.fact\_inventory\_snapshot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 68:** JOIN dw.dim\_store s ON s.store\_key=f.store\_key — JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim\_store is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s.store\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.store\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 69:** JOIN dw.dim\_product p ON p.product\_key=f.product\_key — JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim\_product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.product\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.product\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 70:** WHERE f.stockout\_risk\_flag=1 GROUP BY s.store\_name,p.product\_name; — WHERE filters rows to those meeting a condition; f.stockout\_risk\_flag is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s.store\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.product\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 71:** blank line used only to separate logical sections for readability.
- **Line 72:** -- 16. Inventory value by category. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 73:** SELECT p.category\_name,SUM(f.inventory\_value\_amount) AS inventory\_value — SELECT requests data without changing it unless combined with a modifying statement; p.category\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.inventory\_value\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; inventory\_value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation



- **Line 74:** FROM dw.fact\_inventory\_snapshot f JOIN dw.dim\_product p ON p.product\_key=f.product\_key — FROM identifies the source table or query; dw.fact\_inventory\_snapshot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim\_product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.product\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.product\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 75:** GROUP BY p.category\_name; — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.category\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 76:** blank line used only to separate logical sections for readability.
- **Line 77:** -- 17. Average delivery days by region. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 78:** SELECT s.region,AVG(1.0\*f.delivery\_days) AS average\_delivery\_days — SELECT requests data without changing it unless combined with a modifying statement; s.region is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AVG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1.0 is a numeric literal used by the command or expression; \* is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.delivery\_days is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; average\_delivery\_days is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 79:** FROM dw.fact\_shipments f JOIN dw.dim\_store s ON s.store\_key=f.store\_key — FROM identifies the source table or query; dw.fact\_shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim\_store is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s.store\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f.store\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 80:** GROUP BY s.region; — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s.region is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 81:** blank line used only to separate logical sections for readability.
- **Line 82:** -- 18. Customers whose attributes changed under SCD2. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 83:** SELECT customer\_number,COUNT\_BIG(\*) AS version\_count — SELECT requests data without changing it unless combined with a modifying statement; customer\_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT\_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (\*) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; version\_count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 84:** FROM dw.dim\_customer GROUP BY customer\_number HAVING COUNT\_BIG(\*)>1; — FROM identifies the source table or query; dw.dim\_customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer\_number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; HAVING is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT\_BIG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (\*)> is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression
- **Line 85:** blank line used only to separate logical sections for readability.
- **Line 86:** -- 19. Sales using unknown dimension members. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 87:** SELECT — SELECT requests data without changing it unless combined with a modifying statement
- **Line 88:** SUM(CASE WHEN customer\_key=0 THEN 1 ELSE 0 END) AS unknown\_customer\_rows, — SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; THEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; ELSE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 89:** SUM(CASE WHEN product\_key=0 THEN 1 ELSE 0 END) AS unknown\_product\_rows, — SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; THEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; ELSE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 90:** SUM(CASE WHEN store\_key=0 THEN 1 ELSE 0 END) AS unknown\_store\_rows — SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; store\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; THEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; ELSE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; END is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 91:** FROM dw.fact\_sales; — FROM identifies the source table or query; dw.fact\_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 92:** blank line used only to separate logical sections for readability.
- **Line 93:** -- 20. Daily data-quality failures. is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 94:** `SELECT CAST(check_at_utc AS date) AS check_date, COUNT_BIG(*) AS failed_checks` — `SELECT` requests data without changing it unless combined with a modifying statement; `CAST` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `checked_at_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `check_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `COUNT_BIG` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(*)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 95:** `FROM audit.reconciliation_result WHERE status='FAIL'` — `FROM` identifies the source table or query; `audit.reconciliation_result` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `WHERE` filters rows to those meeting a condition; `status` shows tracked, staged, modified, and untracked work; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'FAIL'` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 96:** `GROUP BY CAST(check_at_utc AS date) ORDER BY check_date;` — `GROUP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CAST` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `checked_at_utc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `AS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ORDER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `check_date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 17 lesson index](#)

## File introduction — `sql/sqlserver/analytics/kpi_validation_queries.sql`

**Exact filename:** `kpi_validation_queries.sql`

**Filename extension:** `.sql`. In this project it identifies database or lakehouse definitions and queries and helps the correct application choose syntax highlighting or execution behavior.

**Exact parent folder:** `sql/sqlserver/analytics` relative to the repository root.

**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

**What the file will contain:** database or lakehouse definitions and queries for Phase 17.

**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.

**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

### VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `sql/sqlserver/analytics` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.

6. CLICK: Select **New File**.
7. TYPE THIS: `kpi_validation_queries.sql`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `kpi_validation_queries.sql` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

## PowerShell path

### Action 17.F3.1 — Create the file `sql/sqlserver/analytics/kpi_validation_queries.sql` from PowerShell

#### Application:

Windows PowerShell or the VS Code PowerShell terminal

#### Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

#### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

#### Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

#### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

#### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

#### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\sql\sqlserver\analytics\kpi_validation_queries.sql" -ErrorAction Stop
```

#### Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\sql\sqlserver\analytics\kpi_validation_queries.sql" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\sql\sqlserver\analytics\kpi_validation_queries.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined

beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### **Does it change anything?**

This can change files, Git state, packages, database objects, or data.

### **Is it safe to rerun?**

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### **What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### **Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### **What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### **How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### **Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

### **Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

### **Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

### **What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

### **How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

## **File-creation decision branches**

### **If the command creates the file**

Run `Test-Path -LiteralPath ".\sql\sqlserver\analytics\kpi_validation_queries.sql" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

### **If PowerShell says the file already exists**

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "sql/sqlserver/analytics/kpi_validation_queries.sql"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

### **If PowerShell says part of the path does not exist**

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

## **File-content construction**

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.

3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use TODO(USER) where the workbook requires your own decision or verified value.

## File verification

1. In the VS Code Explorer, confirm the file appears under sql/sqlserver/analytics.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with sql/sqlserver/analytics/kpi\_validation\_queries.sql.
3. In PowerShell at the repository root, run Test-Path ".\sql\sqlserver\analytics\kpi\_validation\_queries.sql" and require True.
4. Run git status --short -- "sql/sqlserver/analytics/kpi\_validation\_queries.sql" to see only this file's Git state.
5. Run the phase-specific validator before committing.

## Optional reference answer for sql/sqlserver/analytics/kpi\_validation\_queries.sql

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** SQL

**Exact file:** sql/sqlserver/analytics/kpi\_validation\_queries.sql

**Purpose:** database or lakehouse definitions and queries.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 17.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
-- Dialect: SQL Server / T-SQL
-- Independent SQL checks used to validate Power BI measures.
USE NorthstarRetail_DW;
GO
SELECT SUM(net_sales_amount) AS Total_Revenue FROM dw.fact_sales;
SELECT SUM(gross_profit_amount) AS Gross_Profit FROM dw.fact_sales;
SELECT SUM(gross_profit_amount)/NULLIF(SUM(net_sales_amount),0) AS Gross_Margin_Ratio FROM dw.fact_sales;
SELECT COUNT(DISTINCT order_id) AS Orders FROM dw.fact_sales;
SELECT SUM(quantity) AS Units_Sold FROM dw.fact_sales;
SELECT SUM(net_sales_amount)/NULLIF(COUNT(DISTINCT order_id),0) AS Average_Order_Value FROM dw.fact_sales;
SELECT 1.0*SUM(return_quantity)/NULLIF((SELECT SUM(quantity) FROM dw.fact_sales),0) AS Return_Rate FROM dw.fact_returns;
SELECT 1.0*SUM(CASE WHEN on_time_flag=1 THEN 1 ELSE 0 END)/NULLIF(COUNT_BIG(*),0) AS On_Time_Delivery_Rate FROM dw.fact_shipments;
SELECT SUM(spend_amount) AS Marketing_Spend FROM dw.fact_marketing_spend;
SELECT 1.0*SUM(CASE WHEN converted_flag=1 THEN 1 ELSE 0 END)/NULLIF(SUM(session_count),0) AS Conversion_Rate FROM dw.fact_web_sessions;
SELECT SUM(quantity_on_hand) AS Inventory_On_Hand, SUM(CASE WHEN stockout_risk_flag=1 THEN 1 ELSE 0 END) AS Stockout_Risk_Count FROM
dw.fact_inventory_snapshot;
-- Promotion Gross Profit by promotion. This is contribution, not causal incremental lift.
SELECT
 p.promotion_code,
 p.promotion_name,
 SUM(s.gross_profit_amount) AS Promotion_Gross_Profit
FROM dw.fact_sales AS s
INNER JOIN dw.dim_promotion AS p
 ON p.promotion_key = s.promotion_key
WHERE s.promotion_key <> 0
GROUP BY p.promotion_code, p.promotion_name
ORDER BY Promotion_Gross_Profit DESC;
```

## Line-by-line explanation

- **Line 1:** -- Dialect: SQL Server / T-SQL is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** -- Independent SQL checks used to validate Power BI measures. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** USE NorthstarRetail\_DW; — USE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NorthstarRetail\_DW is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** GO — GO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** SELECT SUM(net\_sales\_amount) AS Total\_Revenue FROM dw.fact\_sales; — SELECT requests data without changing it unless combined with a modifying statement; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; net\_sales\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; Total\_Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; dw.fact\_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 6:** SELECT SUM(gross\_profit\_amount) AS Gross\_Profit FROM dw.fact\_sales; — SELECT requests data without changing it unless combined with a modifying statement; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gross\_profit\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gross\_Profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; dw.fact\_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** SELECT SUM(gross\_profit\_amount)/NULLIF(SUM(net\_sales\_amount),0) AS Gross\_Margin\_Ratio FROM dw.fact\_sales; — SELECT requests data without changing it unless combined with a modifying statement; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gross\_profit\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; )/ is a path, relative location, branch-style name, or file argument; verify every character before running it; NULLIF is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; net\_sales\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- **Line 8:** SELECT COUNT(DISTINCT order\_id) AS Orders FROM dw.fact\_sales; — SELECT requests data without changing it unless combined with a modifying statement; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DISTINCT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order\_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Orders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; dw.fact\_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** SELECT SUM(quantity) AS Units\_Sold FROM dw.fact\_sales; — SELECT requests data without changing it unless combined with a modifying statement; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quantity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Units\_Sold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; dw.fact\_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** SELECT SUM(net\_sales\_amount)/NULLIF(COUNT(DISTINCT order\_id),0) AS Average\_Order\_Value FROM dw.fact\_sales; — SELECT requests data without changing it unless combined with a modifying statement; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; net\_sales\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; )/ is a path, relative location, branch-style name, or file argument; verify every character before running it; NULLIF is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DISTINCT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order\_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

surrounding explanation; ), is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- Line 11:** `SELECT 1.0*SUM(return_quantity)/NULLIF((SELECT SUM(quantity) FROM dw.fact_sales),0) AS Return_Rate FROM dw.fact_returns;` — SELECT requests data without changing it unless combined with a modifying statement; 1.0 is a numeric literal used by the command or expression; \* is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; return\_quantity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; )/ is a path, relative location, branch-style name, or file argument; verify every character before running it; NULLIF is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SELECT requests data without changing it unless combined with a modifying statement; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- Line 12:** `SELECT 1.0*SUM(CASE WHEN on_time_flag=1 THEN 1 ELSE 0 END)/NULLIF(COUNT_BIG(*),0) AS On_Time_Delivery_Rate FROM dw.fact_shipments;` — SELECT requests data without changing it unless combined with a modifying statement; 1.0 is a numeric literal used by the command or expression; \* is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on\_time\_flag is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; THEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- Line 13:** `SELECT SUM(spend_amount) AS Marketing_Spend FROM dw.fact_marketing_spend;` — SELECT requests data without changing it unless combined with a modifying statement; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spend\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Marketing\_Spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; FROM identifies the source table or query; dw.fact\_marketing\_spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 14:** `SELECT 1.0*SUM(CASE WHEN converted_flag=1 THEN 1 ELSE 0 END)/NULLIF(SUM(session_count),0) AS Conversion_Rate FROM dw.fact_web_sessions;` — SELECT requests data without changing it unless combined with a modifying statement; 1.0 is a numeric literal used by the command or expression; \* is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; converted\_flag is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; THEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; remaining punctuation and identifiers continue the same statement.
- Line 15:** `SELECT SUM(quantity_on_hand) AS Inventory_On_Hand, SUM(CASE WHEN stockout_risk_flag=1 THEN 1 ELSE 0 END) AS Stockout_Risk_Count FROM dw.fact_inventory_snapshot;` — SELECT requests data without changing it unless combined with a modifying statement; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quantity\_on\_hand is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Inventory\_On\_Hand is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or



literal whose project-specific role is described in the surrounding explanation; SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; WHEN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 16:** -- Promotion Gross Profit by promotion. This is contribution, not causal incremental lift. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 17:** SELECT — SELECT requests data without changing it unless combined with a modifying statement
- **Line 18:** p.promotion\_code, — p.promotion\_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** p.promotion\_name, — p.promotion\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** SUM(s.gross\_profit\_amount) AS Promotion\_Gross\_Profit — SUM adds numeric values in the current filter context; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s.gross\_profit\_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Promotion\_Gross\_Profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** FROM dw.fact\_sales AS s — FROM identifies the source table or query; dw.fact\_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** INNER JOIN dw.dim\_promotion AS p — INNER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JOIN is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dw.dim\_promotion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; AS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** ON p.promotion\_key = s.promotion\_key — ON is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.promotion\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s.promotion\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** WHERE s.promotion\_key <> 0 — WHERE filters rows to those meeting a condition; s.promotion\_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; <> is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression
- **Line 25:** GROUP BY p.promotion\_code, p.promotion\_name — GROUP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.promotion\_code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; p.promotion\_name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** ORDER BY Promotion\_Gross\_Profit DESC; — ORDER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BY is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Promotion\_Gross\_Profit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DESC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:** blank line used only to separate logical sections for readability.

[Back to the Phase 17 lesson index](#)

## What your screen should look like now

- Every Phase 17 file you actually created appears under the correct parent folder.

- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

## Lesson 17.3 completion checklist

---

☐

powerbi/dashboard\_requirements.md was created by me, saved, located correctly, and inspected — [STATUS]

☐

sql/sqlserver/analytics/20\_analyst\_questions.sql was created by me, saved, located correctly, and inspected — [STATUS]

☐

sql/sqlserver/analytics/kpi\_validation\_queries.sql was created by me, saved, located correctly, and inspected — [STATUS]

## Safe stopping point

---

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 17.4](#) and recheck the root and branch.

## Lesson 17.4 — Build and validate required objects

---

### What your screen should look like before starting

---

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

### Report page 1 — Executive Overview

**Audience and decision:** This page groups the measures and dimensions needed for a reader to answer the page's named business question without mixing incompatible grains.

**Required fields/measures:**

- Total Revenue
- Gross Profit
- Gross Margin %
- Orders
- Average Order Value
- On-Time Delivery %

**Page-size rule:** Use the documented canvas and keep consistent margins, alignment, reading order, and title placement.

**Slicer rule:** Date plus only the relevant dimensions. A slicer must not imply that it filters a measure when no valid relationship exists.

**Validation rule:** Every KPI and chart total is compared to a table visual and an independently filtered SQL query.

**Misleading alternatives to avoid:** Truncated axes without a reason, pie charts with too many categories, averaging row-level percentages, dual axes that suggest false correlation, totals across snapshot dates, and colors used as the only status signal.

## Action 17.4.P1 — Build and validate the Executive Overview report page

---

**Application:**

Power BI Desktop

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

**What you are about to do:**

Build and validate the Executive Overview report page. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Executive Overview.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Executive Overview.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.

14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

#### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

#### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

#### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

#### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

#### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

#### Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

#### Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

#### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

#### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

### Literal visual build worksheet

Visual order	Visual type	Field placement	Aggregation/measure	Formatting	Interaction check	My verified result
1	Page title/text box	Executive Overview	none	consistent title size	n/a	[ENTER]
2	KPI cards	headline measures above	explicit measures only	units/decimals/percent	all page slicers	[ENTER]
3	Primary chart	page-specific date/category axis	explicit measure	title, labels, sort	test each slicer	[ENTER]
4	Secondary comparison	appropriate dimension	explicit measure	avoid misleading axis	test cross-highlight	[ENTER]
5	Detail table/matrix	keys hidden; readable attributes	measures	totals/subtotals deliberate	drill/tooltip check	[ENTER]
6	Slicers	date and relevant dimensions	no implicit aggregation	dropdown/list/range	inspect every visual	[ENTER]

#### Screenshot checkpoint

- **Application:** Power BI Desktop.
- **Window:** Report view with Executive Overview selected and the Filters pane showing no accidental hidden filter.
- **Include:** Page title, all visuals, current slicer state, and one visible validation element if readable.
- **Hide:** Server names beyond local examples, Windows username when unnecessary, account email, tokens, passwords, and unrelated desktop content.
- **Crop out:** Other applications and empty desktop area.
- **Recommended filename:** docs/images/phase17\_01\_executive-overview\_verified.png.

- **README placement:** Dashboard/report-pages section after the architecture and validation sections.
- **What it proves:** The verified page existed with the shown filter state; it does not prove every other filter combination.

## Interview-ready explanation

"I built the Executive Overview page from governed measures, set interactions deliberately, avoided grain/denominator traps, and reconciled displayed values to SQL under matching filters before using the screenshot as portfolio evidence."

## Report page 2 — Sales Performance

**Audience and decision:** This page groups the measures and dimensions needed for a reader to answer the page's named business question without mixing incompatible grains.

### Required fields/measures:

- Total Revenue
- Orders
- Units Sold
- Average Order Value
- dim\_date
- dim\_channel

**Page-size rule:** Use the documented canvas and keep consistent margins, alignment, reading order, and title placement.

**Slicer rule:** Date plus only the relevant dimensions. A slicer must not imply that it filters a measure when no valid relationship exists.

**Validation rule:** Every KPI and chart total is compared to a table visual and an independently filtered SQL query.

**Misleading alternatives to avoid:** Truncated axes without a reason, pie charts with too many categories, averaging row-level percentages, dual axes that suggest false correlation, totals across snapshot dates, and colors used as the only status signal.

## Action 17.4.P2 — Build and validate the Sales Performance report page

### Application:

Power BI Desktop

### Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

### Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

### What you are about to do:

Build and validate the Sales Performance report page. This is one bounded construction or inspection task.

### Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

### Exact physical actions:

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Sales Performance.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.

11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

#### Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

#### Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

#### What each part means:

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Sales Performance.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

#### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

#### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

#### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

#### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

#### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

#### Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

### Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

### Literal visual build worksheet

Visual order	Visual type	Field placement	Aggregation/measure	Formatting	Interaction check	My verified result
1	Page title/text box	Sales Performance	none	consistent title size	n/a	[ENTER]
2	KPI cards	headline measures above	explicit measures only	units/decimals/percent	all page slicers	[ENTER]
3	Primary chart	page-specific date/category axis	explicit measure	title, labels, sort	test each slicer	[ENTER]
4	Secondary comparison	appropriate dimension	explicit measure	avoid misleading axis	test cross-highlight	[ENTER]
5	Detail table/matrix	keys hidden; readable attributes	measures	totals/subtotals deliberate	drill/tooltip check	[ENTER]
6	Slicers	date and relevant dimensions	no implicit aggregation	dropdown/list/range	inspect every visual	[ENTER]

### Screenshot checkpoint

- **Application:** Power BI Desktop.
- **Window:** Report view with Sales Performance selected and the Filters pane showing no accidental hidden filter.
- **Include:** Page title, all visuals, current slicer state, and one visible validation element if readable.
- **Hide:** Server names beyond local examples, Windows username when unnecessary, account email, tokens, passwords, and unrelated desktop content.
- **Crop out:** Other applications and empty desktop area.
- **Recommended filename:** docs/images/phase17\_02\_sales-performance\_verified.png.
- **README placement:** Dashboard/report-pages section after the architecture and validation sections.
- **What it proves:** The verified page existed with the shown filter state; it does not prove every other filter combination.

### Interview-ready explanation

"I built the Sales Performance page from governed measures, set interactions deliberately, avoided grain/denominator traps, and reconciled displayed values to SQL under matching filters before using the screenshot as portfolio evidence."

### Report page 3 — Product and Category

**Audience and decision:** This page groups the measures and dimensions needed for a reader to answer the page's named business question without mixing incompatible grains.

#### Required fields/measures:

- Total Revenue
- Gross Profit
- Gross Margin %
- Units Sold
- dim\_product

**Page-size rule:** Use the documented canvas and keep consistent margins, alignment, reading order, and title placement.

**Slicer rule:** Date plus only the relevant dimensions. A slicer must not imply that it filters a measure when no valid relationship exists.

**Validation rule:** Every KPI and chart total is compared to a table visual and an independently filtered SQL query.

**Misleading alternatives to avoid:** Truncated axes without a reason, pie charts with too many categories, averaging row-level percentages, dual axes that suggest false correlation, totals across snapshot dates, and colors used as the only status signal.

## Action 17.4.P3 — Build and validate the Product and Category report page

---

### Application:

Power BI Desktop

### Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

### Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

### What you are about to do:

Build and validate the Product and Category report page. This is one bounded construction or inspection task.

### Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

### Exact physical actions:

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Product and Category.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

### Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

### Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

### What each part means:

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Product and Category.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.



9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

#### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

#### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

#### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

#### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

#### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

#### Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

#### Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

#### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

#### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

#### Literal visual build worksheet

Visual order	Visual type	Field placement	Aggregation/measure	Formatting	Interaction check	My verified result
1	Page title/text box	Product and Category	none	consistent title size	n/a	[ENTER]
2	KPI cards	headline measures above	explicit measures only	units/decimals/percent	all page slicers	[ENTER]
3	Primary chart	page-specific date/category axis	explicit measure	title, labels, sort	test each slicer	[ENTER]
4	Secondary comparison	appropriate dimension	explicit measure	avoid misleading axis	test cross-highlight	[ENTER]
5	Detail table/matrix	keys hidden; readable attributes	measures	totals/subtotals deliberate	drill/tooltip check	[ENTER]
6	Slicers	date and relevant dimensions	no implicit aggregation	dropdown/list/range	inspect every visual	[ENTER]

## Screenshot checkpoint

- **Application:** Power BI Desktop.
- **Window:** Report view with Product and Category selected and the Filters pane showing no accidental hidden filter.
- **Include:** Page title, all visuals, current slicer state, and one visible validation element if readable.
- **Hide:** Server names beyond local examples, Windows username when unnecessary, account email, tokens, passwords, and unrelated desktop content.
- **Crop out:** Other applications and empty desktop area.
- **Recommended filename:** docs/images/phase17\_03\_product-and-category\_verified.png.
- **README placement:** Dashboard/report-pages section after the architecture and validation sections.
- **What it proves:** The verified page existed with the shown filter state; it does not prove every other filter combination.

## Interview-ready explanation

"I built the Product and Category page from governed measures, set interactions deliberately, avoided grain/denominator traps, and reconciled displayed values to SQL under matching filters before using the screenshot as portfolio evidence."

## Report page 4 — Store and Region

**Audience and decision:** This page groups the measures and dimensions needed for a reader to answer the page's named business question without mixing incompatible grains.

### Required fields/measures:

- Total Revenue
- Gross Profit
- Orders
- dim\_store

**Page-size rule:** Use the documented canvas and keep consistent margins, alignment, reading order, and title placement.

**Slicer rule:** Date plus only the relevant dimensions. A slicer must not imply that it filters a measure when no valid relationship exists.

**Validation rule:** Every KPI and chart total is compared to a table visual and an independently filtered SQL query.

**Misleading alternatives to avoid:** Truncated axes without a reason, pie charts with too many categories, averaging row-level percentages, dual axes that suggest false correlation, totals across snapshot dates, and colors used as the only status signal.

## Action 17.4.P4 — Build and validate the Store and Region report page

---

### Application:

Power BI Desktop

### Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

### Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

### What you are about to do:

Build and validate the Store and Region report page. This is one bounded construction or inspection task.

### Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

### Exact physical actions:

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Store and Region.
4. PRESS: Enter.

5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

#### Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

#### Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

#### What each part means:

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Store and Region.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

#### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

#### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

#### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

### Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

### Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

## Literal visual build worksheet

Visual order	Visual type	Field placement	Aggregation/measure	Formatting	Interaction check	My verified result
1	Page title/text box	Store and Region	none	consistent title size	n/a	[ENTER]
2	KPI cards	headline measures above	explicit measures only	units/decimals/percent	all page slicers	[ENTER]
3	Primary chart	page-specific date/category axis	explicit measure	title, labels, sort	test each slicer	[ENTER]
4	Secondary comparison	appropriate dimension	explicit measure	avoid misleading axis	test cross-highlight	[ENTER]
5	Detail table/matrix	keys hidden; readable attributes	measures	totals/subtotals deliberate	drill/tooltip check	[ENTER]
6	Slicers	date and relevant dimensions	no implicit aggregation	dropdown/list/range	inspect every visual	[ENTER]

### Screenshot checkpoint

- **Application:** Power BI Desktop.
- **Window:** Report view with Store and Region selected and the Filters pane showing no accidental hidden filter.
- **Include:** Page title, all visuals, current slicer state, and one visible validation element if readable.
- **Hide:** Server names beyond local examples, Windows username when unnecessary, account email, tokens, passwords, and unrelated desktop content.
- **Crop out:** Other applications and empty desktop area.
- **Recommended filename:** docs/images/phase17\_04\_store-and-region\_verified.png.
- **README placement:** Dashboard/report-pages section after the architecture and validation sections.
- **What it proves:** The verified page existed with the shown filter state; it does not prove every other filter combination.

### Interview-ready explanation

"I built the Store and Region page from governed measures, set interactions deliberately, avoided grain/denominator traps, and reconciled displayed values to SQL under matching filters before using the screenshot as portfolio evidence."

## Report page 5 — Customer

**Audience and decision:** This page groups the measures and dimensions needed for a reader to answer the page's named business question without mixing incompatible grains.

### Required fields/measures:

- New Customers
- Returning Customers
- Purchasing Customers
- dim\_customer

**Page-size rule:** Use the documented canvas and keep consistent margins, alignment, reading order, and title placement.  
**Slicer rule:** Date plus only the relevant dimensions. A slicer must not imply that it filters a measure when no valid relationship exists.  
**Validation rule:** Every KPI and chart total is compared to a table visual and an independently filtered SQL query.  
**Misleading alternatives to avoid:** Truncated axes without a reason, pie charts with too many categories, averaging row-level percentages, dual axes that suggest false correlation, totals across snapshot dates, and colors used as the only status signal.

## Action 17.4.P5 — Build and validate the Customer report page

---

### Application:

Power BI Desktop

### Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

### Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

### What you are about to do:

Build and validate the Customer report page. This is one bounded construction or inspection task.

### Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

### Exact physical actions:

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Customer.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

### Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

### Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

### What each part means:

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.

2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Customer.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

### Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

### Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

## Literal visual build worksheet

Visual order	Visual type	Field placement	Aggregation/measure	Formatting	Interaction check	My verified result
1	Page title/text box	Customer	none	consistent title size	n/a	[ENTER]
2	KPI cards	headline measures above	explicit measures only	units/decimals/percent	all page slicers	[ENTER]
3	Primary chart	page-specific date/category axis	explicit measure	title, labels, sort	test each slicer	[ENTER]

Visual order	Visual type	Field placement	Aggregation/ measure	Formatting	Interaction check	My verified result
4	Secondary comparison	appropriate dimension	explicit measure	avoid misleading axis	test cross-highlight	[ENTER]
5	Detail table/matrix	keys hidden; readable attributes	measures	totals/subtotals deliberate	drill/tooltip check	[ENTER]
6	Slicers	date and relevant dimensions	no implicit aggregation	dropdown/list/range	inspect every visual	[ENTER]

### Screenshot checkpoint

- **Application:** Power BI Desktop.
- **Window:** Report view with Customer selected and the Filters pane showing no accidental hidden filter.
- **Include:** Page title, all visuals, current slicer state, and one visible validation element if readable.
- **Hide:** Server names beyond local examples, Windows username when unnecessary, account email, tokens, passwords, and unrelated desktop content.
- **Crop out:** Other applications and empty desktop area.
- **Recommended filename:** docs/images/phase17\_05\_customer\_verified.png.
- **README placement:** Dashboard/report-pages section after the architecture and validation sections.
- **What it proves:** The verified page existed with the shown filter state; it does not prove every other filter combination.

### Interview-ready explanation

"I built the Customer page from governed measures, set interactions deliberately, avoided grain/denominator traps, and reconciled displayed values to SQL under matching filters before using the screenshot as portfolio evidence."

## Report page 6 — Marketing Funnel

**Audience and decision:** This page groups the measures and dimensions needed for a reader to answer the page's named business question without mixing incompatible grains.

**Required fields/measures:**

- Marketing Spend
- Web Sessions
- Converted Sessions
- Conversion Rate
- Return on Ad Spend
- dim\_campaign

**Page-size rule:** Use the documented canvas and keep consistent margins, alignment, reading order, and title placement.

**Slicer rule:** Date plus only the relevant dimensions. A slicer must not imply that it filters a measure when no valid relationship exists.

**Validation rule:** Every KPI and chart total is compared to a table visual and an independently filtered SQL query.

**Misleading alternatives to avoid:** Truncated axes without a reason, pie charts with too many categories, averaging row-level percentages, dual axes that suggest false correlation, totals across snapshot dates, and colors used as the only status signal.

## Action 17.4.P6 — Build and validate the Marketing Funnel report page

---

**Application:**

Power BI Desktop

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

**What you are about to do:**

Build and validate the Marketing Funnel report page. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Marketing Funnel.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.



14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Marketing Funnel.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

### Literal visual build worksheet

Visual order	Visual type	Field placement	Aggregation/measure	Formatting	Interaction check	My verified result
1	Page title/text box	Marketing Funnel	none	consistent title size	n/a	[ENTER]
2	KPI cards	headline measures above	explicit measures only	units/decimals/percent	all page slicers	[ENTER]
3	Primary chart	page-specific date/category axis	explicit measure	title, labels, sort	test each slicer	[ENTER]
4	Secondary comparison	appropriate dimension	explicit measure	avoid misleading axis	test cross-highlight	[ENTER]
5	Detail table/matrix	keys hidden; readable attributes	measures	totals/subtotals deliberate	drill/tooltip check	[ENTER]
6	Slicers	date and relevant dimensions	no implicit aggregation	dropdown/list/range	inspect every visual	[ENTER]

### Screenshot checkpoint

- **Application:** Power BI Desktop.
- **Window:** Report view with Marketing Funnel selected and the Filters pane showing no accidental hidden filter.
- **Include:** Page title, all visuals, current slicer state, and one visible validation element if readable.
- **Hide:** Server names beyond local examples, Windows username when unnecessary, account email, tokens, passwords, and unrelated desktop content.
- **Crop out:** Other applications and empty desktop area.
- **Recommended filename:** docs/images/phase17\_06\_marketing-funnel\_verified.png.
- **README placement:** Dashboard/report-pages section after the architecture and validation sections.
- **What it proves:** The verified page existed with the shown filter state; it does not prove every other filter combination.

### Interview-ready explanation

"I built the Marketing Funnel page from governed measures, set interactions deliberately, avoided grain/denominator traps, and reconciled displayed values to SQL under matching filters before using the screenshot as portfolio evidence."

### Report page 7 — Inventory

**Audience and decision:** This page groups the measures and dimensions needed for a reader to answer the page's named business question without mixing incompatible grains.

#### Required fields/measures:

- Inventory On Hand
- Stockout Risk Count
- dim\_product
- dim\_store
- dim\_date

**Page-size rule:** Use the documented canvas and keep consistent margins, alignment, reading order, and title placement.

**Slicer rule:** Date plus only the relevant dimensions. A slicer must not imply that it filters a measure when no valid relationship exists.

**Validation rule:** Every KPI and chart total is compared to a table visual and an independently filtered SQL query.

**Misleading alternatives to avoid:** Truncated axes without a reason, pie charts with too many categories, averaging row-level percentages, dual axes that suggest false correlation, totals across snapshot dates, and colors used as the only status signal.

## Action 17.4.P7 — Build and validate the Inventory report page

### Application:

Power BI Desktop

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

**What you are about to do:**

Build and validate the Inventory report page. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Inventory.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Inventory.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.

12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

### Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

### Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

### Literal visual build worksheet

Visual order	Visual type	Field placement	Aggregation/ measure	Formatting	Interaction check	My verified result
1	Page title/text box	Inventory	none	consistent title size	n/a	[ENTER]
2	KPI cards	headline measures above	explicit measures only	units/decimals/percent	all page slicers	[ENTER]
3	Primary chart	page-specific date/category axis	explicit measure	title, labels, sort	test each slicer	[ENTER]
4	Secondary comparison	appropriate dimension	explicit measure	avoid misleading axis	test cross-highlight	[ENTER]
5	Detail table/matrix	keys hidden; readable attributes	measures	totals/subtotals deliberate	drill/tooltip check	[ENTER]
6	Slicers	date and relevant dimensions	no implicit aggregation	dropdown/list/range	inspect every visual	[ENTER]

### Screenshot checkpoint

- **Application:** Power BI Desktop.
- **Window:** Report view with Inventory selected and the Filters pane showing no accidental hidden filter.
- **Include:** Page title, all visuals, current slicer state, and one visible validation element if readable.
- **Hide:** Server names beyond local examples, Windows username when unnecessary, account email, tokens, passwords, and unrelated desktop content.

- **Crop out:** Other applications and empty desktop area.
- **Recommended filename:** docs/images/phase17\_07\_inventory\_verified.png.
- **README placement:** Dashboard/report-pages section after the architecture and validation sections.
- **What it proves:** The verified page existed with the shown filter state; it does not prove every other filter combination.

### Interview-ready explanation

"I built the Inventory page from governed measures, set interactions deliberately, avoided grain/denominator traps, and reconciled displayed values to SQL under matching filters before using the screenshot as portfolio evidence."

## Report page 8 — Shipping and Returns

**Audience and decision:** This page groups the measures and dimensions needed for a reader to answer the page's named business question without mixing incompatible grains.

### Required fields/measures:

- On-Time Delivery %
- Delivered Shipments
- Return Rate
- Returned Units
- dim\_date

**Page-size rule:** Use the documented canvas and keep consistent margins, alignment, reading order, and title placement.

**Slicer rule:** Date plus only the relevant dimensions. A slicer must not imply that it filters a measure when no valid relationship exists.

**Validation rule:** Every KPI and chart total is compared to a table visual and an independently filtered SQL query.

**Misleading alternatives to avoid:** Truncated axes without a reason, pie charts with too many categories, averaging row-level percentages, dual axes that suggest false correlation, totals across snapshot dates, and colors used as the only status signal.

## Action 17.4.P8 — Build and validate the Shipping and Returns report page

### Application:

Power BI Desktop

### Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

### Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

### What you are about to do:

Build and validate the Shipping and Returns report page. This is one bounded construction or inspection task.

### Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

### Exact physical actions:

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Shipping and Returns.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.

10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

#### Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

#### Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

#### What each part means:

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Shipping and Returns.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

#### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

#### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

#### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

#### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

#### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

**Literal visual build worksheet**

Visual order	Visual type	Field placement	Aggregation/measure	Formatting	Interaction check	My verified result
1	Page title/text box	Shipping and Returns	none	consistent title size	n/a	[ENTER]
2	KPI cards	headline measures above	explicit measures only	units/decimals/percent	all page slicers	[ENTER]
3	Primary chart	page-specific date/category axis	explicit measure	title, labels, sort	test each slicer	[ENTER]
4	Secondary comparison	appropriate dimension	explicit measure	avoid misleading axis	test cross-highlight	[ENTER]
5	Detail table/matrix	keys hidden; readable attributes	measures	totals/subtotals deliberate	drill/tooltip check	[ENTER]
6	Slicers	date and relevant dimensions	no implicit aggregation	dropdown/list/range	inspect every visual	[ENTER]

**Screenshot checkpoint**

- **Application:** Power BI Desktop.
- **Window:** Report view with Shipping and Returns selected and the Filters pane showing no accidental hidden filter.
- **Include:** Page title, all visuals, current slicer state, and one visible validation element if readable.
- **Hide:** Server names beyond local examples, Windows username when unnecessary, account email, tokens, passwords, and unrelated desktop content.
- **Crop out:** Other applications and empty desktop area.
- **Recommended filename:** docs/images/phase17\_08\_shipping-and-returns\_verified.png.
- **README placement:** Dashboard/report-pages section after the architecture and validation sections.
- **What it proves:** The verified page existed with the shown filter state; it does not prove every other filter combination.

**Interview-ready explanation**

"I built the Shipping and Returns page from governed measures, set interactions deliberately, avoided grain/denominator traps, and reconciled displayed values to SQL under matching filters before using the screenshot as portfolio evidence."

**Report page 9 — Data Quality**

**Audience and decision:** This page groups the measures and dimensions needed for a reader to answer the page's named business question without mixing incompatible grains.

**Required fields/measures:**

- quality result status
- rule ID
- actual value
- expected value
- batch ID
- layer/entity

**Page-size rule:** Use the documented canvas and keep consistent margins, alignment, reading order, and title placement.

**Slicer rule:** Date plus only the relevant dimensions. A slicer must not imply that it filters a measure when no valid relationship exists.

**Validation rule:** Every KPI and chart total is compared to a table visual and an independently filtered SQL query.

**Misleading alternatives to avoid:** Truncated axes without a reason, pie charts with too many categories, averaging row-level percentages, dual axes that suggest false correlation, totals across snapshot dates, and colors used as the only status signal.

## Action 17.4.P9 — Build and validate the Data Quality report page

---

**Application:**

Power BI Desktop

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

**What you are about to do:**

Build and validate the Data Quality report page. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Data Quality.
4. PRESS: Enter.
5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

1. CLICK: In Power BI Desktop Report view, select the + icon to create a blank page.
2. RIGHT-CLICK: Right-click the new page tab and select **Rename page**.
3. TYPE THIS: Data Quality.
4. PRESS: Enter.



5. CLICK: With the page selected, open the Format pane and set the documented page/canvas size; use a standard 16:9 canvas unless the requirement says otherwise.
6. CLICK: Add each visual one at a time; after adding a visual, select its fields before adding another.
7. CLICK: Add KPI/card visuals for the page's headline measures.
8. CLICK: Add the primary trend or comparison chart and assign the documented axis, values, legend, and tooltips.
9. CLICK: Add slicers for date and the page-specific dimensions; choose dropdown/list/between behavior deliberately.
10. CLICK: Format titles, display units, decimal places, percentage symbols, labels, and accessibility alt text.
11. CLICK: Open **Edit interactions** and test how every slicer affects every visual; disable only interactions that are intentionally misleading.
12. CLICK: Add a report-page tooltip or drill-through target only where the requirement calls for it, then test Back navigation.
13. PRESS: Compare every headline value with a temporary validation table and SQL query under the same filters.
14. PRESS: Ctrl+S and save the Power BI project/file to its documented local location.
15. LOOK FOR: Capture a screenshot only after values and interactions are verified; redact sensitive connection details.
16. DO NOT CONTINUE UNTIL: Page layout, values, filters, interactions, tooltips/drill-through, and accessibility checks pass.

### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

### Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

### Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 17 lesson index](#)

### Literal visual build worksheet

Visual order	Visual type	Field placement	Aggregation/measure	Formatting	Interaction check	My verified result
1	Page title/text box	Data Quality	none	consistent title size	n/a	[ENTER]
2	KPI cards	headline measures above	explicit measures only	units/decimals/percent	all page slicers	[ENTER]
3	Primary chart	page-specific date/category axis	explicit measure	title, labels, sort	test each slicer	[ENTER]
4	Secondary comparison	appropriate dimension	explicit measure	avoid misleading axis	test cross-highlight	[ENTER]

Visual order	Visual type	Field placement	Aggregation/ measure	Formatting	Interaction check	My verified result
5	Detail table/matrix	keys hidden; readable attributes	measures	totals/subtotals deliberate	drill/tooltip check	[ENTER]
6	Slicers	date and relevant dimensions	no implicit aggregation	dropdown/list/range	inspect every visual	[ENTER]

### Screenshot checkpoint

- **Application:** Power BI Desktop.
- **Window:** Report view with Data Quality selected and the Filters pane showing no accidental hidden filter.
- **Include:** Page title, all visuals, current slicer state, and one visible validation element if readable.
- **Hide:** Server names beyond local examples, Windows username when unnecessary, account email, tokens, passwords, and unrelated desktop content.
- **Crop out:** Other applications and empty desktop area.
- **Recommended filename:** docs/images/phase17\_09\_data-quality-verified.png.
- **README placement:** Dashboard/report-pages section after the architecture and validation sections.
- **What it proves:** The verified page existed with the shown filter state; it does not prove every other filter combination.

### Interview-ready explanation

"I built the Data Quality page from governed measures, set interactions deliberately, avoided grain/denominator traps, and reconciled displayed values to SQL under matching filters before using the screenshot as portfolio evidence."

## What your screen should look like now

- The new Phase 17 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

## Lesson 17.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

## Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 17.5](#).

## Lesson 17.5 — Verify, troubleshoot, commit, teach back, and stop safely

### What your screen should look like before starting

The Phase 17 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

## Design decision — Verification before automation

---

### Question:

Should the phase prioritize a visible manual verification or jump directly to automation?

### Options considered:

- Manual verification first, then automation.
- Automation without a known-good baseline.
- Manual operation forever.

### Beginner path selected:

Establish one verified manual baseline and then automate repeatable checks.

### Why this path was selected:

A known-good example makes failures interpretable and prevents automation from hiding a wrong assumption.

### Tradeoff:

The first run takes longer.

### What a company might do instead:

Teams often pair infrastructure as code and automated tests with controlled bootstrap and acceptance procedures.

### How to explain this in an interview:

I established a verified baseline, then encoded the same contract in tests and orchestration so automation preserved meaning rather than just speed.

## If you see Result A

---

Each page answers its audience question and visible values match the validated semantic model. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

## If you see Result B

---

A visual is blank, misleading, differently filtered, over-aggregated, or inconsistent with SQL/DAX validation. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

## If you see an error

---

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

## Phase-specific verification gate

---

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.



Actual verified result: [ENTER VERIFIED RESULT].

## Point-of-failure troubleshooting record

---

### Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

### What it means in plain language:

[WRITE YOUR INTERPRETATION]

### Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

### What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

### First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

### How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

### Safest fix:

[ENTER THE NARROW FIX]

### Alternative fix when applicable:

[ENTER OR N/A]

### How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

### How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

## Screenshot checkpoint

---

**Application(s) that should be visible:** Power BI Desktop Report view, Format pane, Selection pane, and SSMS validation windows.

**Window/panel:** Show each verified report page under a documented slicer state.

**Sensitive information to hide:** Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

**Include:** The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

**Crop out:** Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

**Recommended filename:** docs/images/phase17\_build-and-validate-all-power-bi-report-pages\_verified.png.

**Recommended repository folder:** docs/images/.

**README placement:** The section describing Phase 17's verified result; do not use the image before the result is actually verified.

**What it proves:** The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

## Git checkpoint after the verified Phase 17 changes

---

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

### Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

## Action 17.5.G1 — Verify the repository root before Git operations

---

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

### Does it change anything?

This is intended to inspect or validate state rather than change it.

### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

## Action 17.5.G2 — Verify the active Git branch

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
git branch --show-current
```

#### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

#### Does it change anything?

This is intended to inspect or validate state rather than change it.

#### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

#### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

#### How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

## Action 17.5.G3 — Inspect all current Git changes

---

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
git status --short
```

### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

### Does it change anything?

This is intended to inspect or validate state rather than change it.

### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:



Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

**Interpret `git status --short`**

- `??` means untracked: Git sees a new path but it is not staged.
- `M` means modified in the working tree but not staged.
- `M` means the staged version differs from the last commit.
- `MM` means a staged version exists and the file has additional unstaged edits.
- `A` means staged as a new file.
- `D` means deletion; stop and investigate unless deletion is deliberate.
- `R` means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

## Action 17.5.G4 — Inspect the unstaged diff for `powerbi/dashboard_requirements.md`

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
git diff -- "powerbi/dashboard_requirements.md"
```

### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git diff -- "powerbi/dashboard_requirements.md"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"powerbi/dashboard_requirements.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

### Does it change anything?

This is intended to inspect or validate state rather than change it.

### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

## Action 17.5.G5 — Stage only the reviewed file `powerbi/dashboard_requirements.md`

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project` and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git add -- "powerbi/dashboard_requirements.md"
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git add -- "powerbi/dashboard_requirements.md"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"powerbi/dashboard_requirements.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

### How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

## Action 17.5.G6 — Inspect the staged diff

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git diff --cached
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

## Action 17.5.G7 — Create the Phase 17 commit

---

**Application:**

Power BI Desktop

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Power BI Desktop** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git commit -m "feat: build verified power bi report pages"
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git commit -m "feat: build verified power bi report pages"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"feat: build verified power bi report pages"` is a quoted literal; the quotation marks preserve spaces or mark text

**Does it change anything?**

This can change files, Git state, packages, database objects, or data.

**Is it safe to rerun?**

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

## Action 17.5.G8 — Verify the new commit

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git log -1 --oneline --decorate
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use



git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

## Action 17.5.G9 — Push the feature branch and set upstream

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project and the open Power BI Desktop project/model view named in the action. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git push -u origin "$(git branch --show-current)"
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

**Does it change anything?**

This can change files, Git state, packages, database objects, or data.

**Is it safe to rerun?**

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 17 lesson index](#)

**Verify on GitHub**

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message `feat: build verified power bi report pages` and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

## Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

## Explain it in your own words

---

Without copying the model answer, explain:

1. What you created or configured in Phase 17.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

## My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

## OPTIONAL REFERENCE ANSWER — PHASE 17 TEACH-BACK MODEL

---

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

“Phase 17 created the verified boundary described by **Build and validate all Power BI report pages**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed.”

## Mini quiz

---

1. Multiple choice: Why does Phase 17 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
  2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
  3. Fill in the blank: The statement that defines what one row represents is the table or entity \_\_\_\_\_.
  4. What command would you use first to confirm the PowerShell current folder before a repository command?
  5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
  6. Design choice: Explain one tradeoff of Track A and one production alternative.
  7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
  8. Ownership: Which status may you select only after personally performing and inspecting the verification?
- 

## Mini quiz answer key and explanations

---

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.

5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** PLANNED, CREATED BY USER, and RUN BY USER describe earlier states; only inspected evidence supports verification.

## My Phase 17 Learning Journal

---

**Date completed:**

[ENTER DATE]

**What I created myself:**

[WRITE YOUR ANSWER]

**Commands I ran:**

[WRITE YOUR ANSWER]

**What confused me:**

[WRITE YOUR ANSWER]

**What error occurred:**

[WRITE YOUR ANSWER]

**What the error meant:**

[WRITE YOUR ANSWER]

**How I fixed it:**

[WRITE YOUR ANSWER]

**What I learned:**

[WRITE YOUR ANSWER]

**What I can now explain without help:**

[WRITE YOUR ANSWER]

**What I still need to practice:**

[WRITE YOUR ANSWER]

## What your screen should look like now

---

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

## Lesson 17.5 completion checklist

---

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

## Safe stopping point

---

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

## How this phase connects to the next phase

---

Phase 18 reviews every location where secrets, compute cost, or sensitive data can appear.

## Phase 17 completion checklist

---

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [ ] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

# Phase 18 — Secure secrets, control cost, and review operational settings

[Back to the master table of contents](#)

**Track:** Track A required

**Status when first opened:** PLANNED

**Completion rule:** The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

## Why this phase happens now

---

**Earlier work this phase depends on:** A repository with secrets excluded and cost-sensitive settings recorded.

**What would go wrong if it happened earlier:** You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

**Later work that depends on it:** Phase 19 automates local, infrastructure-free quality checks on every change.

**Decision being made:** Choose and verify the simplest local-first method that teaches the professional concept without

pretending that local services are publicly reachable.

**Alternatives:** A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

**Beginner path selected:** Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

**Reason for the order:** Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

## Phase 18 lesson index

---

1. [Lesson 18.1 — Understand the phase and starting screen](#)
2. [Lesson 18.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 18.3 — Create and understand the phase files](#)
4. [Lesson 18.4 — Build and validate required objects](#)
5. [Lesson 18.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

## Lesson 18.1 — Understand the phase and starting screen

---

### What your screen should look like before starting

---

- **Application open:** VS Code, PowerShell, Git, GitHub, Databricks settings, and Power BI options.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 18 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 17 must be verified when this phase has a prerequisite.

## Three-level explanation of the phase

---

### Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

### Level 2 — Northstar Retail project explanation

Phase 18 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

### Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

## Applications to open

---

VS Code, PowerShell, Git, GitHub, Databricks settings, and Power BI options. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

## Phase status gate

---



I changed this phase from [ ] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

## What your screen should look like now

---

- The Phase 18 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 18 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 18 master-tracker row is marked [-] In progress only after you personally updated it.

## Lesson 18.1 completion checklist

---

☐

I read why Phase 18 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

## Safe stopping point

---

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 18.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 18 lesson index](#)

## Lesson 18.2 — Perform the notebook-guided actions one at a time

---

### What your screen should look like before starting

---

The Phase 18 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

### Retained phase lesson, exercises, rules, and expected results

---

**Track:** Track A required

**Ownership status at start:** Workbook instructions exist; your execution is not yet verified.

**Evidence rule:** Replace placeholders only after the verification steps pass.

## 1. What you are building

A no-secrets configuration pattern, .gitignore, .env.example, Databricks secret guidance, least-privilege SQL roles, read-only analyst access, development/production separation, log-redaction rules, synthetic-data classification, secret-scanning checklist, and cost shutdown checklist.

## 2. Why it matters

A portfolio project can still leak real passwords, tokens, machine details, or cloud spend. Security and cost controls are design requirements, not cleanup after the pipeline works.

## 3. Words to know

Term	Plain-language definition
Secret	A password, token, private key, connection string, or credential that grants access.
Environment variable	A process-level name/value used to supply configuration without writing a secret into code.
.env file	A local text file commonly used for environment variables; it must be ignored by Git.
Least privilege	Grant only the minimum access needed for a task.
Read-only role	A database permission group that can query approved objects but cannot change them.
Secret scope	A Databricks-managed namespace for secret values, subject to workspace capabilities and permissions.
PII	Personally identifiable information; synthetic names/emails should still be handled as if their shape were sensitive.
Redaction	Removing or masking sensitive content from logs, screenshots, and error messages.
Configuration environment	A named set of settings such as development or production.
Compute auto-termination	Stopping idle cloud compute automatically to reduce cost.
Secret scanning	Searching repository content for credential-like patterns before commit/push.

## 4. What must already be completed

Local source/warehouse connections work through environment variables, the repository has no real credentials, and you can identify every service/database account used by the project.

## 5. Applications to open

Application	How to open it	Why it is open
VS Code	Open .gitignore, .env.example, config files, and security guide.	Review what is safe to publish.
Windows PowerShell	Open in repository root.	Create local .env, inspect tracked files, and run scans.
SSMS and pgAdmin	Connect with an administrative learning account only while creating read roles.	Grant and test minimum database permissions.
Databricks workspace settings/secret interface	Open only if available on your workspace tier.	Store cloud-connected credentials without notebooks or Git.
GitHub repository Settings/Security	Open the repository in a browser.	Review visibility, Actions, and secret alerts if available.

## 6. Exact click-by-click steps

**Beginner rhythm:** Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.



## 7. Exact commands, one code block at a time

## 8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run `Get-Location`; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

## TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

### Line-by-line code explanation

Line or action	Meaning	Why it belongs here
Copy-Item .env.example .env	Creates a local ignored configuration file and opens it for editing.	Scripts need connection settings, but real values must not be placed in tracked templates.
git check-ignore -v .env	Shows why .env is ignored and whether any sensitive file is staged/untracked.	Assuming an ignore rule works is not sufficient.
python -m scripts.validation.scan_secrets	Scans text files for several common credential patterns.	It catches obvious mistakes before GitHub receives them.
IF DATA-BASE_PRINCIPAL_ID(N'northstar_analyst_reader') IS NULL	Creates a reusable read-only semantic-schema role without embedding credentials.	Power BI/analysts should not need table-write or database-owner permission.
-- TODO(USER): create/login names and passwords outside Git.	Creates a group role with read access to current and future digital tables.	The extraction account does not require object ownership or write privileges.

### Local-first security boundary

Databricks does not receive local SQL Server/PostgreSQL passwords in Track A. Python runs on your Windows machine, extracts to files, and you manually upload a batch. This is less automated but avoids pretending a cloud workspace can safely reach localhost.

### Cost truthfulness

Free or trial labels, quotas, and availability can change. Use the official source register near the beginning of this workbook, read the current account page before starting compute, record what you actually used, enable auto-stop where available, and never claim “zero cost” unless your own billing confirms it.

## MY TASK

Prove that secrets are excluded from Git, create/test read-only database roles in a safe local environment, document the Track A credential boundary, review logs/screenshots, and complete the security and cost checklists.

## STARTER SKELETON

```
SECRET/RESOURCE: TODO(USER)
OWNER: TODO(USER)
USED BY: TODO(USER)
STORAGE LOCATION: environment variable / secret manager / identity
MINIMUM PERMISSIONS: TODO(USER)
ROTATION/REVOCACTION METHOD: TODO(USER)
LOG REDACTION TEST: TODO(USER)
COST CONTROL: TODO(USER)
EVIDENCE: TODO(USER)
```

**Acceptance criteria:** No real credential or private key is tracked; .env ignore behavior is proven; source extraction and analyst access use least privilege; log/screenshots are reviewed; cloud/local cost shutdown steps are recorded; an accidental-secret response procedure is understood.

## HINT 1 — CONCEPT

Treat every value that grants access as a secret even when the data is synthetic.

## HINT 2 — PSEUDOCODE

For each component list identity, minimum actions, storage method, test for denied write, and shutdown/rotation path.

## HINT 3 — NEAR-COMPLETE PATTERN

Use `.env.example` for variable names only, local `.env` for values, database group roles for permissions, and a pre-push scan plus manual staged-diff review.

### 13. Expected files or output

- Verified ignored `.env` file.
- Passing secret-pattern scan and reviewed staged diff.
- Read-only SQL Server and PostgreSQL role test evidence.
- Completed secret inventory and classification notes.
- Cost/shutdown checklist with actual services used.
- Sanitized logs and screenshot review.

### 14. How to verify

- `git ls-files` contains no `.env` or credential file.
- Read-only users can SELECT required objects and cannot modify them.
- Track A Databricks notebooks contain no local database connection string.
- Logs do not print complete connection URLs/passwords/tokens.
- Generated sample data is explicitly labeled synthetic.
- Cloud compute/warehouse auto-stop or manual shutdown is recorded.
- Secret rotation steps are known before any public push.

### 15. Automated tests

- Ignore-rule test with `git check-ignore`.
- Secret scanner plus manual staged-diff review.
- Positive SELECT and negative INSERT permission tests.
- Log redaction inspection.
- Repository history spot check before publication.
- Cloud billing/resource inventory check.

### 16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
<code>.env</code> appears in git status	Ignore rule is missing, malformed, or the file was already tracked.	<code>git check-ignore -v .env</code> and <code>git ls-files .env</code> .	Add rule; if already tracked, use <code>git rm --cached .env</code> , rotate any exposed credential, then commit the removal.	<code>.env</code> is ignored/untracked and credential is rotated if needed.
Permission denied for SELECT	Role/user mapping, schema USAGE, or object SELECT grant is missing.	Inspect current user and database role membership.	Grant only the missing read permission; do not make the user owner/db_owner.	SELECT succeeds; write still fails.
Read-only account can modify data	It inherited a broader role or direct grant.	Inspect role memberships and effective grants.	Revoke broad membership/grants, reconnect, retest.	INSERT/UPDATE/DELETE are denied.
Secret scanner reports a template	Example resembles a real token/password pattern.	Open the exact file/line and determine whether it is a real secret.	Rotate if real; replace with unmistakable placeholder if fake; document false positive only when justified.	Scan passes and no usable credential exists.
Unexpected cloud charge/quota failure	Compute/storage remained active or free-tier quota was exceeded.	Open provider billing/quota and resource/run history.	Stop/delete unused resources, reduce scale/schedule, and document cost guardrails before retry.	No unintended active resource and next run fits budget/quota.

For deeper tool-specific diagnosis, use [the troubleshooting index](#) near the end of [this workbook](#).

## 17. Explain it in your own words

---

1. Why is `.env.example` safe while `.env` is not?
2. Why does a Power BI reader not need warehouse write access?
3. Why are synthetic names still classified as PII-like?
4. What must happen after a real secret is committed?

Write your answers in [the learning-journal fields in this phase](the learning-journal fields in this phase). Do not copy the wording above.

## 18. Mini quiz

---

1. Which command proves the ignore rule that applies to `.env`?
2. Does deleting a committed secret from the latest file make it safe again?
3. Why does Track A not place local database credentials in Databricks?

Record your answers before opening the answer key.

## 19. Screenshot checkpoint

---

Capture passing scans, ignore proof, role membership and denied-write tests, plus compute auto-stop/resource shutdown settings. Redact usernames, server endpoints, tokens, and billing identifiers.

Save the image under `docs/images/` with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

## 20. Git checkpoint and suggested commit message

---

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
security: add least privilege secret and cost controls
```

Push the feature branch only after tests pass.

## 21. Completion checklist

---

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

`.env` ignore behavior proven — [STATUS]

☐

No secrets tracked or staged — [STATUS]

☐

SQL Server read role tested — [STATUS]

☐

PostgreSQL read role tested — [STATUS]

☐

Databricks credential boundary documented — [STATUS]

☐

Logs/screenshots sanitized — [STATUS]

☐

Synthetic classification documented — [STATUS]

☐

Cost shutdown controls verified — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: [the optional reference-answer section later in this workbook#phase-18](the optional reference-answer section later in this workbook#phase-18). The quiz answer is in [the answer-key section immediately after the quiz in this workbook#phase-18](the answer-key section immediately after the quiz in this workbook#phase-18).

---

**Next phase:** Phase 19 automates the repository checks through GitHub Actions and a safe branch/pull-request workflow.

## Required action cards

---

### Action 18.2.1 — Open .gitignore and confirm .env, .venv, logs, run artifacts, generated raw data, PBIX working files, an

---

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Open .gitignore and confirm .env, .venv, logs, run artifacts, generated raw data, PBIX working files, an. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open .gitignore and confirm .env, .venv, logs, run artifacts, generated raw data, PBIX working files, and local connection files are excluded as intended.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

#### Step 1

**Action**

Open .gitignore and confirm .env, .venv, logs, run artifacts, generated raw data, PBIX working files, and local connection files are excluded as intended.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.2.2 — Copy .env.example to .env in the repository root. Fill only local values in .env; never edit .env.examp

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Copy .env.example to .env in the repository root. Fill only local values in .env; never edit .env.examp. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Copy .env.example to .env in the repository root. Fill only local values in .env; never edit .env.example to contain a real password.
4. Pause while the application finishes the operation; do not click the same control twice.

5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 2****Action**

Copy `.env.example` to `.env` in the repository root. Fill only local values in `.env`; never edit `.env.example` to contain a real password.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.2.3 — Run `git check-ignore -v .env` and `git status --short`. Stop if `.env` appears as an untracked/staged file

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

### Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

### Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run `git check-ignore -v .env` and `git status --short`. Stop if `.env` appears as an untracked/staged file. This is one bounded construction or inspection task.

### Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

### Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run `git check-ignore -v .env` and `git status --short`. Stop if `.env` appears as an untracked/staged file rather than ignored.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

### Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

### Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

### What each part means:

## Step 3

### Action

Run `git check-ignore -v .env` and `git status --short`. Stop if `.env` appears as an untracked/staged file rather than ignored.

### Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.2.4 — Review every script for `os.getenv/configuration` helpers and confirm errors identify missing variable names w

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Review every script for `os.getenv/configuration` helpers and confirm errors identify missing variable names w. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Review every script for `os.getenv/configuration` helpers and confirm errors identify missing variable names without printing their values.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.



**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 4****Action**

Review every script for `os.getenv/configuration` helpers and confirm errors identify missing variable names without printing their values.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.2.5 — Run the included secret-pattern scanner. Also inspect Git staged changes manually because a simple pattern sca

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run the included secret-pattern scanner. Also inspect Git staged changes manually because a simple pattern sca. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run the included secret-pattern scanner. Also inspect Git staged changes manually because a simple pattern scanner cannot guarantee safety.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 5****Action**

Run the included secret-pattern scanner. Also inspect Git staged changes manually because a simple pattern scanner cannot guarantee safety.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.2.6 — In SSMS, create a warehouse read-only role and a separate login/user only when you understand local authentication

---

**Application:**

SQL Server Management Studio (SSMS)

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

In SSMS, create a warehouse read-only role and a separate login/user only when you understand local authentication. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: In SSMS, create a warehouse read-only role and a separate login/user only when you understand local authentication. Grant SELECT on semantic and/or dw, not write access to source/warehouse tables.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

SQL Server Management Studio (SSMS). When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 6****Action**

In SSMS, create a warehouse read-only role and a separate login/user only when you understand local authentication. Grant SELECT on semantic and/or dw, not write access to source/warehouse tables.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.2.7 — In PostgreSQL, create a read-only role for extraction practice with CONNECT, USAGE on the digital schema, and

**Application:**

pgAdmin 4 or PostgreSQL psql, as named in the action

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

In PostgreSQL, create a read-only role for extraction practice with CONNECT, USAGE on the digital schema, and . This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: In PostgreSQL, create a read-only role for extraction practice with CONNECT, USAGE on the digital schema, and SELECT on intended tables. Do not grant CREATE or ownership.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

pgAdmin 4 or PostgreSQL `psql`, as named in the action. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 7****Action**

In PostgreSQL, create a read-only role for extraction practice with CONNECT, USAGE on the digital schema, and SELECT on intended tables. Do not grant CREATE or ownership.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.2.8 — For Databricks Track A, no local database credential is needed in Databricks because extracts are manually upl

---

**Application:**

Databricks workspace in a web browser

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

For Databricks Track A, no local database credential is needed in Databricks because extracts are manually upl. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: For Databricks Track A, no local database credential is needed in Databricks because extracts are manually uploaded. For Track B, use secret management or supported identity, never plaintext notebook widgets.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 8****Action**

For Databricks Track A, no local database credential is needed in Databricks because extracts are manually uploaded. For Track B, use secret management or supported identity, never plaintext notebook widgets.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.2.9 — Inspect logs and run artifacts. Ensure connection URLs are masked, exceptions do not print passwords, and scre

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Inspect logs and run artifacts. Ensure connection URLs are masked, exceptions do not print passwords, and scre. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Inspect logs and run artifacts. Ensure connection URLs are masked, exceptions do not print passwords, and screenshots hide account identifiers, tokens, email addresses, server public names, and workspace URLs when unnecessary.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 9****Action**

Inspect logs and run artifacts. Ensure connection URLs are masked, exceptions do not print passwords, and screenshots hide account identifiers, tokens, email addresses, server public names, and workspace URLs when unnecessary.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.



**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.2.10 — Classify synthetic customer fields in the data dictionary. Use labels such as Synthetic PII-like so future rea

---

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Classify synthetic customer fields in the data dictionary. Use labels such as Synthetic PII-like so future rea. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Classify synthetic customer fields in the data dictionary. Use labels such as Synthetic PII-like so future real-data handling expectations are visible.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

## Step 10

### Action

Classify synthetic customer fields in the data dictionary. Use labels such as Synthetic PII-like so future real-data handling expectations are visible.

### Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

### Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

### Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.2.11 — Set development configuration names, catalogs, schemas, and sample volume separately from any optional product

### Application:

Visual Studio Code

### Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Set development configuration names, catalogs, schemas, and sample volume separately from any optional product. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Set development configuration names, catalogs, schemas, and sample volume separately from any optional production environment. Never reuse a production credential in a portfolio repository.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 11****Action**

Set development configuration names, catalogs, schemas, and sample volume separately from any optional production environment. Never reuse a production credential in a portfolio repository.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.2.12 — Record compute choices and shutdown controls: local services, Databricks serverless/job quotas, SQL warehouse

---

**Application:**

Power BI Desktop

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Record compute choices and shutdown controls: local services, Databricks serverless/job quotas, SQL warehouse . This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Record compute choices and shutdown controls: local services, Databricks serverless/job quotas, SQL warehouse auto-stop, cloud database tiers, storage, egress, and Power BI licensing. Stop or delete unused cloud resources.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

## Step 12

### Action

Record compute choices and shutdown controls: local services, Databricks serverless/job quotas, SQL warehouse auto-stop, cloud database tiers, storage, egress, and Power BI licensing. Stop or delete unused cloud resources.

### Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

### Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

### Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.2.13 — Before every push, run the secret scan, inspect staged diff, and use the secret-scanning checklist in `docs/se

### Application:

Visual Studio Code

### Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

### Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Before every push, run the secret scan, inspect staged diff, and use the secret-scanning checklist in `docs/se`. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Before every push, run the secret scan, inspect staged diff, and use the secret-scanning checklist in docs/security\_and\_cost.md.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 13****Action**

Before every push, run the secret scan, inspect staged diff, and use the secret-scanning checklist in docs/security\_and\_cost.md.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.2.14 — If a secret was ever committed, remove/rotate it immediately; deleting one file or commit is not enough because

---

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

If a secret was ever committed, remove/rotate it immediately; deleting one file or commit is not enough because. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: If a secret was ever committed, remove/rotate it immediately; deleting one file or commit is not enough because the credential may already be copied or remain in history.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

### Step 14

**Action**

If a secret was ever committed, remove/rotate it immediately; deleting one file or commit is not enough because the credential may already be copied or remain in history.

### Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

### Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

### Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.2.C1 — Windows PowerShell

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

Windows PowerShell

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.



### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (Powershell):

```
Copy-Item .env.example .env
```

### Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** Copy-Item .env.example .env — Copy-Item copies a selected file or folder; . is a path, relative location, branch-style name, or file argument; verify every character before running it; env.example is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; env is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

## Action 18.2.C2 — Windows PowerShell

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Powershell):**

```
notepad .env
```

**Where to enter it:**

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `notepad .env` — `notepad` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### Does it change anything?

This is intended to inspect or validate state rather than change it.

### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

### How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

## Action 18.2.C3 — Windows PowerShell

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

Windows PowerShell

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (Powershell):

```
git check-ignore -v .env
```

### Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git check-ignore -v .env` — `git` calls the Git version-control program; `check-ignore` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-v` is a command option or flag; its exact behavior is defined beside the command; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### Does it change anything?

This is intended to inspect or validate state rather than change it.

### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

## Action 18.2.C4 — Windows PowerShell

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Powershell):**

```
git status --short
```

**Where to enter it:**

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

## Action 18.2.C5 — Windows PowerShell

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Powershell):**

```
python -m scripts.validation.scan_secrets
```

**Where to enter it:**

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `python -m scripts.validation.scan_secrets` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.scan_secrets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

## Action 18.2.C6 — SSMS query window (SQL Server / T-SQL)

**Application:**

SQL Server Management Studio (SSMS)

**Window or panel:**

SSMS query window (SQL Server / T-SQL)

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Sql instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **SQL Server Management Studio (SSMS)** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Sql):**

```
IF DATABASE_PRINCIPAL_ID(N'northstar_analyst_reader') IS NULL
CREATE ROLE northstar_analyst_reader;
GRANT SELECT ON SCHEMA::semantic TO northstar_analyst_reader;
-- TODO(USER): CREATE USER [your_local_read_login] FOR LOGIN [your_local_read_login];
-- TODO(USER): ALTER ROLE northstar_analyst_reader ADD MEMBER [your_local_read_login];
```



**Where to enter it:**

SSMS query window (SQL Server / T-SQL). Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `IF DATABASE_PRINCIPAL_ID(N'northstar_analyst_reader') IS NULL` — `IF` begins a conditional decision; `DATABASE_PRINCIPAL_ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `N` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'northstar_analyst_reader'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NULL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `CREATE ROLE northstar_analyst_reader;` — `CREATE` defines a new database object; `ROLE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `northstar_analyst_reader` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `GRANT SELECT ON SCHEMA::semantic TO northstar_analyst_reader;` — `GRANT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SELECT` requests data without changing it unless combined with a modifying statement; `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCHEMA` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `::` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `semantic` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `TO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `northstar_analyst_reader` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `-- TODO(USER): CREATE USER [your_local_read_login] FOR LOGIN [your_local_read_login];` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 5:** `-- TODO(USER): ALTER ROLE northstar_analyst_reader ADD MEMBER [your_local_read_login];` is a comment for a human reader; it normally does not change data or execute business logic.

**Does it change anything?**

This can change files, Git state, packages, database objects, or data.

**Is it safe to rerun?**

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

## Action 18.2.C7 — pgAdmin Query Tool or psql (PostgreSQL SQL)

---

**Application:**

pgAdmin 4 or PostgreSQL psql, as named in the action

**Window or panel:**

pgAdmin Query Tool or psql (PostgreSQL SQL)

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Sql instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **pgAdmin 4 or PostgreSQL psql, as named in the action** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Sql):**

```
-- TODO(USER): create/login names and passwords outside Git.
CREATE ROLE northstar_digital_reader NOLOGIN;
GRANT CONNECT ON DATABASE northstar_digital TO northstar_digital_reader;
GRANT USAGE ON SCHEMA digital TO northstar_digital_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA digital TO northstar_digital_reader;
ALTER DEFAULT PRIVILEGES IN SCHEMA digital
GRANT SELECT ON TABLES TO northstar_digital_reader;
```

**Where to enter it:**

pgAdmin Query Tool or psql (PostgreSQL SQL). Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `-- TODO(USER): create/login names and passwords outside Git.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `CREATE ROLE northstar_digital_reader NOLOGIN;` — `CREATE` defines a new database object; `ROLE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `northstar_digital_reader` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NOLOGIN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** `GRANT CONNECT ON DATABASE northstar_digital TO northstar_digital_reader;` — `GRANT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CONNECT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATABASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `northstar_digital` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `TO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `northstar_digital_reader` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `GRANT USAGE ON SCHEMA digital TO northstar_digital_reader;` — `GRANT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `USAGE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCHEMA` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `digital` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `TO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `northstar_digital_reader` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `GRANT SELECT ON ALL TABLES IN SCHEMA digital TO northstar_digital_reader;` — `GRANT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SELECT` requests data without changing it unless combined with a modifying statement; `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ALL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `TABLES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCHEMA` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `digital` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `TO` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `northstar_digital_reader` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `ALTER DEFAULT PRIVILEGES IN SCHEMA digital` — `ALTER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DEFAULT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PRIVILEGES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `IN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCHEMA` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `digital` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `GRANT SELECT ON TABLES TO northstar_digital_reader;` — `GRANT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SELECT` requests data without changing it unless combined with a modifying statement; `ON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `TABLES` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `TO` is a name, keyword, argument, identifier, or

literal whose project-specific role is described in the surrounding explanation; northstar\_digital\_reader is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

### How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

## What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

## Lesson 18.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

## Safe stopping point

---

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 18 starting-screen checklist and re-verify the active context.

## Lesson 18.3 — Create and understand the phase files

---

### What your screen should look like before starting

---

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 18 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

#### Folder introduction — docs/

**Exact folder name:** docs

**Exact parent:** repository root

**Why it is needed now:** Phase 18 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

**What will eventually be inside:** Only files whose purpose matches this folder's name.

**Why another folder is not appropriate:** Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

**Naming rules:** Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

#### VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: docs.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

#### PowerShell method

### Action 18.D1.1 — Create the folder docs

---

#### Application:

Windows PowerShell or the VS Code PowerShell terminal

#### Window or panel:

VS Code integrated PowerShell terminal at the repository root

#### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

#### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

#### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

#### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

#### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop
```

### Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

**Folder-creation decision branches****If the folder is created**

Run `Test-Path -LiteralPath ".\docs" -PathType Container` and require `True`.

**If PowerShell says the folder already exists**

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\docs" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

**If the parent path does not exist**

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

**Verify the folder**

1. Run `Test-Path ".\docs" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

**Folder introduction — scripts/**

**Exact folder name:** `scripts`

**Exact parent:** `repository root`

**Why it is needed now:** Phase 18 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

**What will eventually be inside:** Only files whose purpose matches this folder's name.

**Why another folder is not appropriate:** Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

**Naming rules:** Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

**VS Code method**

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `repository root` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `scripts`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

**PowerShell method****Action 18.D2.1 — Create the folder scripts**

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal at the repository root

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
New-Item -ItemType Directory -Path ".\scripts" -ErrorAction Stop
```

**Where to enter it:**

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** New-Item -ItemType Directory -Path ".\scripts" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\scripts" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This can change files, Git state, packages, database objects, or data.

**Is it safe to rerun?**

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.



**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

**Folder-creation decision branches****If the folder is created**

Run `Test-Path -LiteralPath ".\scripts" -PathType Container` and require `True`.

**If PowerShell says the folder already exists**

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

**If the parent path does not exist**

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

**Verify the folder**

1. Run `Test-Path ".\scripts" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

**Folder introduction — scripts/validation/**

**Exact folder name:** `validation`

**Exact parent:** `scripts`

**Why it is needed now:** Phase 18 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

**What will eventually be inside:** Only files whose purpose matches this folder's name.

**Why another folder is not appropriate:** Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

**Naming rules:** Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

**VS Code method**

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `scripts` one level at a time.

4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: validation.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

## PowerShell method

### Action 18.D3.1 — Create the folder scripts/validation

#### Application:

Windows PowerShell or the VS Code PowerShell terminal

#### Window or panel:

VS Code integrated PowerShell terminal at the repository root

#### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

#### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

#### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

#### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

#### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\scripts\validation" -ErrorAction Stop
```

#### Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\scripts\validation" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\scripts\validation" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This can change files, Git state, packages, database objects, or data.

**Is it safe to rerun?**

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

**Folder-creation decision branches****If the folder is created**

Run `Test-Path -LiteralPath ".\scripts\validation" -PathType Container` and require `True`.

**If PowerShell says the folder already exists**

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts\validation" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

**If the parent path does not exist**

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

**Verify the folder**

1. Run `Test-Path ".\scripts\validation" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

## File introduction — docs/security\_and\_cost.md

**Exact filename:** security\_and\_cost.md

**Filename extension:** .md. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

**Exact parent folder:** docs relative to the repository root.

**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

**What the file will contain:** project documentation for Phase 18.

**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.

**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

### VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in docs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: security\_and\_cost.md.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled security\_and\_cost.md and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

### PowerShell path

## Action 18.F1.1 — Create the file docs/security\_and\_cost.md from PowerShell

---

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\security_and_cost.md" -ErrorAction Stop
```

#### Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\docs\security_and_cost.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs\security_and_cost.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

#### Does it change anything?

This can change files, Git state, packages, database objects, or data.

#### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

#### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

## How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

## File-creation decision branches

### If the command creates the file

Run `Test-Path -LiteralPath ".\docs\security_and_cost.md" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

### If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/security_and_cost.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

### If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

## File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

## File verification

1. In the VS Code Explorer, confirm the file appears under docs.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `docs/security_and_cost.md`.
3. In PowerShell at the repository root, run `Test-Path ".\docs\security_and_cost.md"` and require True.
4. Run `git status --short -- "docs/security_and_cost.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

## Optional reference answer for docs/security\_and\_cost.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** Markdown

**Exact file:** `docs/security_and_cost.md`

**Purpose:** project documentation.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 18.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
Security, Configuration, and Cost Controls

Secret handling

- `.env` is local and ignored. `.env.example` contains names and safe placeholders only.
- Never commit database passwords, Databricks tokens, Power BI credentials, connection strings with secrets, or private keys.
- Use `git check-ignore .env` before the first commit.
- If a secret reaches Git, assume it is compromised: revoke/rotate it first, then remove history using an approved method. Deleting the visible line is not enough.
- Use Databricks secret management or supported cloud identities for Track B. Do not display secret values in notebook output.
- Redact logs and screenshots.

Least privilege

| Identity | Minimum learning access |
|---|---|
| Local source loader | Insert/select in source schemas; no server-admin need after setup |
| Local extractor | Read-only on source tables and watermark metadata |
| Databricks workflow | Read/write on project volume, Bronze/Silver/Gold/quality schemas only |
| Local Gold publisher | Stage/load in warehouse plus audit writes |
| Power BI/analyst | Read-only on `dw` tables or `semantic` views |

Use separate identities in a real environment. Track A may use your local Windows identity for simplicity, but document that this is not a production access model.
```

```

Development versus production configuration

- `config/dev.example.yml` is a template; create `config/dev.yml` locally and ignore it if it contains environment-specific values.
- Use environment variables for secrets.
- Keep catalog/schema/database names parameterized.
- Development can use safe full reloads; production usually needs tested incremental strategies and deployment controls.
- The destructive reset scripts contain guards. Never remove the guard in shared environments.

Synthetic PII practice

Although all identities are fictional, mask or hide names/emails in executive pages, limit detail access, and classify fields. This builds habits without using real people.

Cost-aware choices

- Generate `quick` while debugging, `small` after logic works, and `portfolio` only for a final performance demonstration.
- Free/community offerings have quota and feature limits that can change; verify the current official page before relying on them.
- Use serverless/job compute only for active tasks; avoid repeated blind retries.
- Stop local database services when you no longer need them if your computer resources are constrained.
- Import mode in Power BI is the default Track A choice; refresh intentionally.
- Track B can create charges for databases, compute, storage, egress, networking, and Power BI capacity/licensing. Set budgets/alerts before creating resources.

Secret-scanning checklist

- [] `.env` is ignored and absent from `git ls-files`.
- [] No password/token/private-key pattern appears in tracked text.
- [] Notebook outputs contain no credentials.
- [] Screenshots hide server details and account email where unnecessary.
- [] GitHub Actions uses encrypted repository/environment secrets only when needed.
- [] Public sample data is quick-scale and fictional.
- [] Connection test logs redact passwords.
- [] Any exposed credential was revoked, not merely deleted from a file.

```

## Line-by-line explanation

- **Line 1:** # Security, Configuration, and Cost Controls is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** ## Secret handling is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** - \.env` is local and ignored. \.env.example` contains names and safe placeholders only.—is a command option or flag; its exact behavior is defined beside the command;.is a path, relative location, branch-style name, or file argument; verify every character before running it;env is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;isis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;localis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;andis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;ignoredis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;is is a path, relative location, branch-style name, or file argument; verify every character before running it;is a path, relative location, branch-style name, or file argument; verify every character before running it;env.example is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;containsis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;names` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 6:** - Never commit database passwords, Databricks tokens, Power BI credentials, connection strings with secrets, or private keys. — - is a command option or flag; its exact behavior is defined beside the command; Never is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; commit records the staged snapshot; database is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; passwords is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Databricks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tokens is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Power is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BI is a name, keyword, argument, identifier, or literal whose project-specific role is described

in the surrounding explanation; credentials is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 7:** - Use `\git check-ignore .env` before the first commit.— is a command option or flag; its exact behavior is defined beside the command; Use is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; git calls the Git version-control program; check-ignore is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; env is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; before is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; first is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; commit records the staged snapshot; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 8:** - If a secret reaches Git, assume it is compromised: revoke/rotate it first, then remove history using an approved method. Deleting the visible line is not enough. — is a command option or flag; its exact behavior is defined beside the command; If is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; secret is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reaches is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Git is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; assume is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; it is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; compromised is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 9:** - Use Databricks secret management or supported cloud identities for Track B. Do not display secret values in notebook output. — is a command option or flag; its exact behavior is defined beside the command; Use is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Databricks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; secret is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; management is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; supported is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cloud is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; identities is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Track is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; B is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 10:** - Redact logs and screenshots. — is a command option or flag; its exact behavior is defined beside the command; Redact is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; logs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; screenshots is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 11:** blank line used only to separate logical sections for readability.
- **Line 12:** ## Least privilege is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 13:** blank line used only to separate logical sections for readability.
- **Line 14:** | Identity | Minimum learning access | — Identity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Minimum is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; learning is a name, keyword, ar-



gument, identifier, or literal whose project-specific role is described in the surrounding explanation; access is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 15:** | --- | --- | — --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command
- **Line 16:** | Local source loader | Insert/select in source schemas; no server-admin need after setup | — Local is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; loader is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Insert is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; select is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; schemas is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; no is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; server-admin is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; need is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 17:** | Local extractor | Read-only on source tables and watermark metadata | — Local is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; extractor is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Read-only is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tables is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; watermark is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; metadata is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** | Databricks workflow | Read/write on project volume, Bronze/Silver/Gold/quality schemas only | — Databricks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; workflow is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Read is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; write is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; project is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; volume is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Bronze is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; Silver is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 19:** | Local Gold publisher | Stage/load in warehouse plus audit writes | — Local is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; publisher is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Stage is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; load is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; warehouse is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; plus is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; audit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; writes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** | Power BI/analyst | Read-only on \dw\ tables or \semantic\ views | —Power is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tion;Blis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;/is a path, relative location, branch-style name, or file argument; verify every character before running it;analystis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Read-onlyis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;onis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;dwis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;tablesis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Oris a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;semanticis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;views` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 21:** blank line used only to separate logical sections for readability.
- **Line 22:** Use separate identities in a real environment. Track A may use your local Windows identity for simplicity, but document that this is not a production access model. — Use is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; separate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; identities is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; real is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; environment is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Track is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; A is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; may is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; use is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 23:** blank line used only to separate logical sections for readability.
- **Line 24:** ## Development versus production configuration is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 25:** blank line used only to separate logical sections for readability.
- **Line 26:** - `config/dev.example.yml` is a template; create `config/dev.yml` locally and ignore it if it contains environment-specific values.—is a command option or flag; its exact behavior is defined beside the command;configis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;/is a path, relative location, branch-style name, or file argument; verify every character before running it;dev.example.ymlis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;isis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;ais a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;templateis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;createis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;configis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;/is a path, relative location, branch-style name, or file argument; verify every character before running it;dev.ymlis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;locally` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 27:** - Use environment variables for secrets. — - is a command option or flag; its exact behavior is defined beside the command; Use is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; environment is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; variables is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; secrets is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 28:** - Keep catalog/schema/database names parameterized. — - is a command option or flag; its exact behavior is defined beside the command; Keep is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; catalog is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; database is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; names is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; parameterized is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 29:** - Development can use safe full reloads; production usually needs tested incremental strategies and deployment controls. — - is a command option or flag; its exact behavior is defined beside the command; Development is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; can is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; use is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; safe is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; full is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reloads is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; production is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; usually is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; needs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tested is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; incremental is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 30:** - The destructive reset scripts contain guards. Never remove the guard in shared environments. — - is a command option or flag; its exact behavior is defined beside the command; The is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; destructive is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reset is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; scripts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; contain is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; guards is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Never is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remove is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; guard is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 31:** blank line used only to separate logical sections for readability.
- **Line 32:** ## Synthetic PII practice is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 33:** blank line used only to separate logical sections for readability.
- **Line 34:** Although all identities are fictional, mask or hide names/emails in executive pages, limit detail access, and classify fields. This builds habits without using real people. — Although is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; all is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; identities is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fictional is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mask is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; hide is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; names is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; emails is a name, keyword, argu-

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 35:** blank line used only to separate logical sections for readability.
- **Line 36:** `## Cost-aware choices` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 37:** blank line used only to separate logical sections for readability.
- **Line 38:** `- Generate \quick\ while debugging, \small\ after logic works, and \portfolio\ only for a final performance demonstration.`—is a command option or flag; its exact behavior is defined beside the command; `Generate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quick` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `while` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `debugging` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `small` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `after` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `logic` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `works` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and\` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 39:** `- Free/community offerings have quota and feature limits` that can change; verify the current official page before relying on them. — is a command option or flag; its exact behavior is defined beside the command; `Free` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `community` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `offerings` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `have` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quota` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `feature` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `limits` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `that` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `can` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 40:** `- Use serverless/job compute only for active tasks; avoid repeated blind retries.` — is a command option or flag; its exact behavior is defined beside the command; `Use` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `serverless` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `job` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `compute` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `only` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `active` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tasks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `avoid` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `repeated` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 41:** `- Stop local database services when you no longer need them if your computer resources are constrained.` — is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `local` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `database` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `services` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `when` is a name, keyword, argument, identifier, or literal whose project-

specific role is described in the surrounding explanation; you is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; no is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; longer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; need is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; them is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 42:** - Import mode in Power BI is the default Track A choice; refresh intentionally. — - is a command option or flag; its exact behavior is defined beside the command; Import is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Power is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BI is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; default is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Track is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; A is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; choice is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 43:** - Track B can create charges for databases, compute, storage, egress, networking, and Power BI capacity/licensing. Set budgets/alerts before creating resources. — - is a command option or flag; its exact behavior is defined beside the command; Track is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; B is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; can is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; create is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; charges is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; databases is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; compute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; storage is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 44:** blank line used only to separate logical sections for readability.
- **Line 45:** ## Secret-scanning checklist is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 46:** blank line used only to separate logical sections for readability.
- **Line 47:** - [ ] \.env` is ignored and absent from `git ls-files`.—is a command option or flag; its exact behavior is defined beside the command;[is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;]is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;is a path, relative location, branch-style name, or file argument; verify every character before running it;env is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;isis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;ignored is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;absent is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;from is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;git calls the Git version-control program;ls-files` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 48:** - [ ] No password/token/private-key pattern appears in tracked text. — - is a command option or flag; its exact behavior is defined beside the command; [ is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; No is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; password is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; token is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; private-key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pattern is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; appears is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 49:** - [ ] Notebook outputs contain no credentials. — - is a command option or flag; its exact behavior is defined beside the command; [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Notebook is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; outputs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; contain is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; no is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; credentials is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 50:** - [ ] Screenshots hide server details and account email where unnecessary. — - is a command option or flag; its exact behavior is defined beside the command; [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Screenshots is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; hide is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; details is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; account is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; email is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; where is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; unnecessary is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 51:** - [ ] GitHub Actions uses encrypted repository/environment secrets only when needed. — - is a command option or flag; its exact behavior is defined beside the command; [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; GitHub is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Actions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; uses is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; encrypted is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; repository is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; environment is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; secrets is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; only is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 52:** - [ ] Public sample data is quick-scale and fictional. — - is a command option or flag; its exact behavior is defined beside the command; [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Public is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sample is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quick-

scale is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fictional is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 53:** - [ ] Connection test logs redact passwords. — - is a command option or flag; its exact behavior is defined beside the command; [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Connection is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; logs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; redact is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; passwords is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 54:** - [ ] Any exposed credential was revoked, not merely deleted from a file. — - is a command option or flag; its exact behavior is defined beside the command; [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Any is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; exposed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; credential is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; was is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; revoked is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; merely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; deleted is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

[Back to the Phase 18 lesson index](#)

## File introduction — scripts/validation/scan\_secrets.py

**Exact filename:** scan\_secrets.py

**Filename extension:** .py. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

**Exact parent folder:** scripts/validation relative to the repository root.

**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

**What the file will contain:** executable Python logic for Phase 18.

**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.

**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

## VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in scripts/validation one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: scan\_secrets.py.
8. PRESS: Enter.

9. LOOK FOR: A new editor tab labeled `scan_secrets.py` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

## PowerShell path

# Action 18.F2.1 — Create the file `scripts/validation/scan_secrets.py` from PowerShell

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\scripts\validation\scan_secrets.py" -ErrorAction Stop
```

### Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `New-Item` -Item Type `File` -Path `".\scripts\validation\scan_secrets.py"` -Error Action `Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\scripts\validation\scan_secrets.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation



**Does it change anything?**

This can change files, Git state, packages, database objects, or data.

**Is it safe to rerun?**

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

**File-creation decision branches****If the command creates the file**

Run `Test-Path -LiteralPath ".\scripts\validation\scan_secrets.py" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

**If PowerShell says the file already exists**

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "scripts/validation/scan_secrets.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

**If PowerShell says part of the path does not exist**

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

**File-content construction**

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.

5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use TODO(USER) where the workbook requires your own decision or verified value.

## File verification

1. In the VS Code Explorer, confirm the file appears under scripts/validation.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with scripts/validation/scan\_secrets.py.
3. In PowerShell at the repository root, run Test-Path ".\scripts\validation\scan\_secrets.py" and require True.
4. Run git status --short -- "scripts/validation/scan\_secrets.py" to see only this file's Git state.
5. Run the phase-specific validator before committing.

## Optional reference answer for scripts/validation/scan\_secrets.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** Python

**Exact file:** scripts/validation/scan\_secrets.py

**Purpose:** executable Python logic.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 18.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
"""Simple educational secret-pattern scanner; not a replacement for enterprise tooling."""
from __future__ import annotations

import re
from pathlib import Path

PATTERNS = {
 "private key": re.compile(r"-----BEGIN (?:(RSA |EC |OPENSSH)?PRIVATE KEY-----"),
 "Databricks token-like": re.compile(r"\bdapi[a-zA-Z0-9]{20,}\b"),
 "GitHub token-like": re.compile(r"\bgh[pousr]_[A-Za-z0-9]{20,}\b"),
 "password assignment": re.compile(r"(?i)\b(password|pwd)\s*[=:]\s*['\"]{8,}['\"]"),
}

SKIP = {".git", ".venv", "datasets", "logs", "run_artifacts"}
EXTENSIONS = {".py", ".ps1", ".sql", ".md", ".yml", ".yaml", ".json", ".dax", ".txt"}

def main() -> None:
 findings = []
 for path in Path(".").rglob("**"):
 if not path.is_file() or path.suffix.lower() not in EXTENSIONS or any(part in SKIP for part in path.parts):
 continue
 text = path.read_text(encoding="utf-8", errors="ignore")
 for label, pattern in PATTERNS.items():
 if pattern.search(text): findings.append(f"{path}: {label}")
 if findings:
 raise SystemExit("Possible secrets found; review and rotate real credentials:\n- " + "\n- ".join(findings))
 print("Secret-pattern scan passed.")

if __name__ == "__main__": main()
```

## Line-by-line explanation

- **Line 1:** """Simple educational secret-pattern scanner; not a replacement for enterprise tooling.""" — "" is a quoted literal; the quotation marks preserve spaces or mark text; "Simple educational secret-pattern scanner; not a replacement for enterprise tooling." is a quoted literal; the quotation marks preserve spaces or mark text; "" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 2:** from \_\_future\_\_ import annotations — from is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; \_\_future\_\_ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; import is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; annotations is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** blank line used only to separate logical sections for readability.
- **Line 4:** import re — import is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; re is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** from pathlib import Path — from is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pathlib is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; import is a name, keyword, argument, identifier, or

literal whose project-specific role is described in the surrounding explanation; Path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** PATTERNS = { — PATTERNS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** "private key": re.compile(r"-----BEGIN (? :RSA |EC |OPENSSH )?PRIVATE KEY-----"), — "private key" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; re.compile is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; r is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "-----BEGIN (? :RSA |EC |OPENSSH )?PRIVATE KEY-----" is a quoted literal; the quotation marks preserve spaces or mark text; ) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** "Databricks token-like": re.compile(r"\bdapi[a-zA-Z0-9]{20,}\b"), — "Databricks token-like" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; re.compile is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; r is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "\bdapi[a-zA-Z0-9]{20,}\b" is a path, relative location, branch-style name, or file argument; verify every character before running it; ) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** "GitHub token-like": re.compile(r"\bgh[pousr]\_[A-Za-z0-9\_]{20,}\b"), — "GitHub token-like" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; re.compile is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; r is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "\bgh[pousr]\_[A-Za-z0-9\_]{20,}\b" is a path, relative location, branch-style name, or file argument; verify every character before running it; ) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** "password assignment": re.compile(r"(?i)\b(password|pwd)\s\*[:]\s\*['\"][^<'\"]{8,}['\""]), — "password assignment" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; re.compile is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; r is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "(?i)\b(password|pwd)\s\*[:]\s\*['\"][^<'\"]{8,}['\"]" is a path, relative location, branch-style name, or file argument; verify every character before running it; ] [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; < is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '\" ] { 8 , } [ ' is a path, relative location, branch-style name, or file argument; verify every character before running it; "]" is a quoted literal; the quotation marks preserve spaces or mark text; ) , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** } — } is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** SKIP = { ".git", ".venv", "datasets", "logs", "run\_artifacts" } — SKIP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ".git" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ".venv" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "datasets" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "logs" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "run\_artifacts" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 14:** `EXTENSIONS = {".py", ".ps1", ".sql", ".md", ".yaml", ".yaml", ".json", ".dax", ".txt"}` — `EXTENSIONS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `".py"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `".ps1"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `".sql"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `".md"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `".yaml"` is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.
- **Line 15:** blank line used only to separate logical sections for readability.
- **Line 16:** blank line used only to separate logical sections for readability.
- **Line 17:** `def main() -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** `findings = []` — `findings` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** `for path in Path(".").rglob("*"):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"."` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rglob` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"*"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** `if not path.is_file() or path.suffix.lower() not in EXTENSIONS or any(part in SKIP for part in path.parts):` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.is_file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `or` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.suffix.lower` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `EXTENSIONS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `or` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `any` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 21:** `continue` — `continue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 22:** `text = path.read_text(encoding="utf-8", errors="ignore")` — `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.read_text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `encoding` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"utf-8"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `errors` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"ignore"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** `for label, pattern in PATTERNS.items():` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `label` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pattern` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PATTERNS.items` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** `if pattern.search(text): findings.append(f"{path}: {label}")` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pattern.search` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `findings.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{path}: {label}"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** `if findings:` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `findings` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** `raise SystemExit("Possible secrets found; review and rotate real credentials:\n- " + "\n- ".join(findings))` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SystemExit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Possible secrets found; review and rotate real credentials:\n- "` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\n- "` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` `.` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` `join` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `findings` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:** `print("Secret-pattern scan passed.")` — `print` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Secret-pattern scan passed."` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** blank line used only to separate logical sections for readability.
- **Line 29:** blank line used only to separate logical sections for readability.

- **Line 30:** `if __name__ == "__main__": main()` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__name__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"__main__"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 18 lesson index](#)

## File introduction — .env.example

**Exact filename:** `.env.example`

**Filename extension:** `.example`. In this project it identifies project text or configuration and helps the correct application choose syntax highlighting or execution behavior.

**Exact parent folder:** `.` relative to the repository root.

**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

**What the file will contain:** project text or configuration for Phase 18.

**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

**Could it contain secrets?** Yes—review carefully before Git staging.

**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

### VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `.` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `.env.example`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `.env.example` and an empty file under the correct parent folder.
10. PRESS: `Ctrl+S` after adding each explained section.

### PowerShell path

## Action 18.F3.1 — Create the file .env.example from PowerShell

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\env.example" -ErrorAction Stop
```

### Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\env.example" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\env.example"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

**File-creation decision branches****If the command creates the file**

Run `Test-Path -LiteralPath ".\env.example" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

**If PowerShell says the file already exists**

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- ".env.example"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

**If PowerShell says part of the path does not exist**

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

**File-content construction**

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

**File verification**

1. In the VS Code Explorer, confirm the file appears under ..
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `.env.example`.
3. In PowerShell at the repository root, run `Test-Path ".\env.example"` and require True.
4. Run `git status --short -- ".env.example"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

**Optional reference answer for .env.example**

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** configuration text

**Exact file:** `.env.example`

**Purpose:** project text or configuration.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 18.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
Copy this file to .env. Never commit .env.
Track A uses Windows Authentication for SQL Server by default.

APP_ENV=dev
LOG_LEVEL=INFO
```



```

SQLSERVER_SERVER=localhost
SQLSERVER_INSTANCE=MSSQLSERVER
SQLSERVER_SOURCE_DATABASE=NorthstarRetail_ERP
SQLSERVER_WAREHOUSE_DATABASE=NorthstarRetail_DW
SQLSERVER_DRIVER=ODBC Driver 18 for SQL Server
SQLSERVER_TRUSTED_CONNECTION=yes
SQLSERVER_ENCRYPT=yes
SQLSERVER_TRUST_SERVER_CERTIFICATE=yes
Use these only if you intentionally choose SQL Server Authentication:
SQLSERVER_USERNAME=
SQLSERVER_PASSWORD=

POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DATABASE=northstar_digital
POSTGRES_USERNAME=postgres
POSTGRES_PASSWORD=CHANGE_ME_LOCALLY

Track B only. Do not put live cloud secrets here if the file might be shared.
DATABRICKS_HOST=
DATABRICKS_HTTP_PATH=
DATABRICKS_TOKEN=
DATABRICKS_CATALOG=northstar
DATABRICKS_BRONZE_SCHEMA=bronze
DATABRICKS_SILVER_SCHEMA=silver
DATABRICKS_GOLD_SCHEMA=gold
DATABRICKS_VOLUME_PATH=/Volumes/northstar/landing/raw

```

## Line-by-line explanation

- **Line 1:** # Copy this file to .env. Never commit .env. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** # Track A uses Windows Authentication for SQL Server by default. is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 3:** blank line used only to separate logical sections for readability.
- **Line 4:** APP\_ENV=dev — APP\_ENV is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dev is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** LOG\_LEVEL=INFO — LOG\_LEVEL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; INFO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** SQLSERVER\_SERVER=localhost — SQLSERVER\_SERVER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; localhost is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** SQLSERVER\_INSTANCE=MSSQLSERVER — SQLSERVER\_INSTANCE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; MSSQLSERVER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** SQLSERVER\_SOURCE\_DATABASE=NorthstarRetail\_ERP — SQLSERVER\_SOURCE\_DATABASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NorthstarRetail\_ERP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** SQLSERVER\_WAREHOUSE\_DATABASE=NorthstarRetail\_DW — SQLSERVER\_WAREHOUSE\_DATABASE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; NorthstarRetail\_DW is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** SQLSERVER\_DRIVER=ODBC Driver 18 for SQL Server — SQLSERVER\_DRIVER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ODBC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Driver is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tion; 18 is a numeric literal used by the command or expression; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 12:** `SQLSERVER_TRUSTED_CONNECTION=yes` — `SQLSERVER_TRUSTED_CONNECTION` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `yes` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `SQLSERVER_ENCRYPT=yes` — `SQLSERVER_ENCRYPT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `yes` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** `SQLSERVER_TRUST_SERVER_CERTIFICATE=yes` — `SQLSERVER_TRUST_SERVER_CERTIFICATE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `yes` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** `# Use these only if you intentionally choose SQL Server Authentication:` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 16:** `SQLSERVER_USERNAME=` — `SQLSERVER_USERNAME` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** `SQLSERVER_PASSWORD=` — `SQLSERVER_PASSWORD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** `POSTGRES_HOST=localhost` — `POSTGRES_HOST` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `localhost` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** `POSTGRES_PORT=5432` — `POSTGRES_PORT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `5432` is a numeric literal used by the command or expression
- **Line 21:** `POSTGRES_DATABASE=northstar_digital` — `POSTGRES_DATABASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `northstar_digital` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 22:** `POSTGRES_USERNAME=postgres` — `POSTGRES_USERNAME` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `postgres` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** `POSTGRES_PASSWORD=CHANGE_ME_LOCALLY` — `POSTGRES_PASSWORD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CHANGE_ME_LOCALLY` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** blank line used only to separate logical sections for readability.
- **Line 25:** `# Track B only. Do not put live cloud secrets here if the file might be shared.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 26:** `DATABRICKS_HOST=` — `DATABRICKS_HOST` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 27:** `DATABRICKS_HTTP_PATH=` — `DATABRICKS_HTTP_PATH` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** `DATABRICKS_TOKEN=` — `DATABRICKS_TOKEN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 29:** DATABRICKS\_CATALOG=northstar — DATABRICKS\_CATALOG is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; northstar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 30:** DATABRICKS\_BRONZE\_SCHEMA=bronze — DATABRICKS\_BRONZE\_SCHEMA is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bronze is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** DATABRICKS\_SILVER\_SCHEMA=silver — DATABRICKS\_SILVER\_SCHEMA is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; silver is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 32:** DATABRICKS\_GOLD\_SCHEMA=gold — DATABRICKS\_GOLD\_SCHEMA is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** DATABRICKS\_VOLUME\_PATH=/Volumes/northstar/landing/raw — DATABRICKS\_VOLUME\_PATH is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; /= is a path, relative location, branch-style name, or file argument; verify every character before running it; Volumes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; northstar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; landing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; raw is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 18 lesson index](#)

## File introduction — .gitignore

**Exact filename:** .gitignore

**Filename extension:** (no extension). In this project it identifies project text or configuration and helps the correct application choose syntax highlighting or execution behavior.

**Exact parent folder:** . relative to the repository root.

**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

**What the file will contain:** project text or configuration for Phase 18.

**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.

**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

## VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in . one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: .gitignore.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled .gitignore and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

## PowerShell path

## Action 18.F4.1 — Create the file .gitignore from PowerShell

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\gitignore" -ErrorAction Stop
```

### Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** New-Item -ItemType File -Path ".\gitignore" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; File is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\gitignore" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

**File-creation decision branches****If the command creates the file**

Run `Test-Path -LiteralPath ".\.\.gitignore" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

**If PowerShell says the file already exists**

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- ".gitignore"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

**If PowerShell says part of the path does not exist**

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

**File-content construction**

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

**File verification**

1. In the VS Code Explorer, confirm the file appears under `..`

2. Right-click the editor tab and choose **Copy Path**; compare the ending with `.gitignore`.
3. In PowerShell at the repository root, run `Test-Path ".\.\.gitignore"` and require `True`.
4. Run `git status --short -- ".gitignore"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

## Optional reference answer for `.gitignore`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** text

**Exact file:** `.gitignore`

**Purpose:** project text or configuration.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 18.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
Python
.venv/
__pycache__/
*.py[cod]
.pytest_cache/
.ruff_cache/
.coverage
htmlcov/

Secrets and local configuration
.env
.env.*
!.env.example
config/dev.yml
*.pem
*.key
*.pfx
*.p12

Generated data: keep only the intentionally small sample folder
/datasets/raw/**
!/datasets/raw/**/.gitkeep
/datasets/generated/**
/datasets/tmp/**

Logs and manifests created during local runs
/logs/**
!/logs/.gitkeep
/run_artifacts/**

Power BI and database artifacts
*.pbix
*.pbix
*.bak
*.bacpac
*.mdf
*.ldf

Databricks and local tooling
.databricks/
.vscode/settings.json
.idea/

OS
Thumbs.db
.DS_Store
```

## Line-by-line explanation

- **Line 1:** `# Python` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `.venv/` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `venv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 3:** `__pycache__/` — `__pycache__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 4:** `*.py[cod]` — `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `py` invokes the Windows Python launcher or Python Install Manager command shim; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cod` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding

explanation; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 5:** `.pytest_cache/` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `pytest_cache` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 6:** `.ruff_cache/` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `ruff_cache` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 7:** `.coverage` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `coverage` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `htmlcov/` — `htmlcov` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 9:** blank line used only to separate logical sections for readability.
- **Line 10:** `# Secrets and local configuration` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 11:** `.env` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `.env.*` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.*` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 13:** `!.env.example` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `env.example` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** `config/dev.yml` — `config` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `dev.yml` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** `*.pem` — `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pem` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `*.key` — `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `key` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** `*.pfx` — `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pfx` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** `*.p12` — `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `p12` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** blank line used only to separate logical sections for readability.
- **Line 20:** `# Generated data: keep only the intentionally small sample folder` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 21:** `/datasets/raw/**` — `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/**` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 22:** `!/datasets/raw/**/.gitkeep` — `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; `/**/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `gitkeep` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 23:** `/datasets/generated/**` — `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `generated` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/**` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 24:** `/datasets/tmp/**` — `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `tmp` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/**` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 25:** blank line used only to separate logical sections for readability.
- **Line 26:** `# Logs and manifests created during local runs` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 27:** `/logs/**` — `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `logs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/**` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 28:** `!/logs/.gitkeep` — `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `logs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `gitkeep` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 29:** `/run_artifacts/**` — `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `run_artifacts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/**` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 30:** blank line used only to separate logical sections for readability.
- **Line 31:** `# Power BI and database artifacts` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 32:** `*.pbix` — `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pbix` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** `*.pbit` — `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pbit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** `*.bak` — `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bak` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 35:** `*.bacpac` — `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `bacpac` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 36:** `*.mdf` — `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `mdf` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 37:** `*.ldf` — `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ldf` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 38:** blank line used only to separate logical sections for readability.
- **Line 39:** `# Databricks and local tooling` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 40:** `.databricks/` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `databricks` is a name, keyword, argument, identifier, or literal whose project-specific role is



described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 41:** `.vscode/settings.json` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `vscode` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; `settings.json` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** `.idea/` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `idea` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 43:** blank line used only to separate logical sections for readability.
- **Line 44:** `# OS` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 45:** `Thumbs.db` — `Thumbs.db` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** `.DS_Store` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `DS_Store` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 18 lesson index](#)

## What your screen should look like now

---

- Every Phase 18 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

## Lesson 18.3 completion checklist

---

☐

`docs/security_and_cost.md` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`scripts/validation/scan_secrets.py` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`.env.example` was created by me, saved, located correctly, and inspected — [STATUS]

☐

`.gitignore` was created by me, saved, located correctly, and inspected — [STATUS]

## Safe stopping point

---

Save every open file with `Ctrl+S`. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 18.4](#) and recheck the root and branch.

## Lesson 18.4 — Build and validate required objects

---

### What your screen should look like before starting

---

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

## Action 18.4.G1 — Inspect the current state without changing it

---

**Application:**

VS Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Inspect the current state without changing it. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.4.G2 — Create the smallest required object or decision

---

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Create the smallest required object or decision. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.4.G3 — Run the phase-specific verification

---

**Application:**

VS Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run the phase-specific verification. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.4.G4 — Demonstrate one safe failure or alternate-result branch

---

**Application:**

VS Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Demonstrate one safe failure or alternate-result branch. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.

4. Continue only after verification.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## Action 18.4.G5 — Record evidence and prepare the downstream handoff

**Application:**

VS Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Record evidence and prepare the downstream handoff. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 18 lesson index](#)

## What your screen should look like now

- The new Phase 18 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

## Lesson 18.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]



A rerun or alternate-result branch was tested where applicable — [STATUS]



Counts, keys, measures, relationships, or settings were independently validated — [STATUS]



I can explain one failure and its safest recovery — [STATUS]

## Safe stopping point

---

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 18.5](#).

## Lesson 18.5 — Verify, troubleshoot, commit, teach back, and stop safely

---

### What your screen should look like before starting

---

The Phase 18 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

### Design decision — Verification before automation

---

#### Question:

Should the phase prioritize a visible manual verification or jump directly to automation?

#### Options considered:

- Manual verification first, then automation.
- Automation without a known-good baseline.
- Manual operation forever.

#### Beginner path selected:

Establish one verified manual baseline and then automate repeatable checks.

#### Why this path was selected:

A known-good example makes failures interpretable and prevents automation from hiding a wrong assumption.

#### Tradeoff:

The first run takes longer.

#### What a company might do instead:

Teams often pair infrastructure as code and automated tests with controlled bootstrap and acceptance procedures.

#### How to explain this in an interview:

I established a verified baseline, then encoded the same contract in tests and orchestration so automation preserved meaning rather than just speed.

### If you see Result A

---

.env stays ignored, the secret scan is clean, and idle compute is stopped. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

### If you see Result B

---

A credential-like value is staged, printed, or stored in an unsafe location, or compute remains active. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

### If you see an error

---

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.



## Phase-specific verification gate

---

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

## Point-of-failure troubleshooting record

---

### Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

### What it means in plain language:

[WRITE YOUR INTERPRETATION]

### Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

### What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

### First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

### How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

### Safest fix:

[ENTER THE NARROW FIX]

### Alternative fix when applicable:

[ENTER OR N/A]

### How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

### How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

## Screenshot checkpoint

---

**Application(s) that should be visible:** VS Code, PowerShell, Git, GitHub, Databricks settings, and Power BI options.

**Window/panel:** Show ignored .env, secret-scan output, Databricks compute state, and cost checklist.

**Sensitive information to hide:** Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

**Include:** The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

**Crop out:** Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

**Recommended filename:** docs/images/phase18\_secure-secrets-control-cost-and-review-operational-sett\_verified.png.

**Recommended repository folder:** docs/images/.

**README placement:** The section describing Phase 18's verified result; do not use the image before the result is actually verified.

**What it proves:** The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

## Git checkpoint after the verified Phase 18 changes

---

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

### Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

## Action 18.5.G1 — Verify the repository root before Git operations

---

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

## Action 18.5.G2 — Verify the active Git branch

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git branch --show-current
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

## Action 18.5.G3 — Inspect all current Git changes

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git status --short
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

**Interpret `git status --short`**

- `??` means untracked: Git sees a new path but it is not staged.
- `M` means modified in the working tree but not staged.
- `M` means the staged version differs from the last commit.
- `MM` means a staged version exists and the file has additional unstaged edits.
- `A` means staged as a new file.
- `D` means deletion; stop and investigate unless deletion is deliberate.
- `R` means rename; inspect both old and new paths.

- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

## Action 18.5.G4 — Inspect the unstaged diff for docs/security\_and\_cost.md

---

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
git diff -- "docs/security_and_cost.md"
```

### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git diff -- "docs/security_and_cost.md"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"docs/security_and_cost.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

### Does it change anything?

This is intended to inspect or validate state rather than change it.

### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

## Action 18.5.G5 — Stage only the reviewed file docs/security\_and\_cost.md

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.



4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
git add -- "docs/security_and_cost.md"
```

#### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `git add -- "docs/security_and_cost.md"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"docs/security_and_cost.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

#### Does it change anything?

This can change files, Git state, packages, database objects, or data.

#### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

#### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

## Action 18.5.G6 — Inspect the staged diff

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git diff --cached
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

## Action 18.5.G7 — Create the Phase 18 commit

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
git commit -m "security: protect secrets and document cost controls"
```

#### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `git commit -m "security: protect secrets and document cost controls"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"security: protect secrets and document cost controls"` is a quoted literal; the quotation marks preserve spaces or mark text

#### Does it change anything?

This can change files, Git state, packages, database objects, or data.

#### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

## Action 18.5.G8 — Verify the new commit

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git log -1 --oneline --decorate
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

## Action 18.5.G9 — Push the feature branch and set upstream

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

#### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

#### Does it change anything?

This can change files, Git state, packages, database objects, or data.

#### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

### How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 18 lesson index](#)

## Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message security: protect secrets and document cost controls and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

## Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

## Explain it in your own words

---

Without copying the model answer, explain:

1. What you created or configured in Phase 18.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

## My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

## OPTIONAL REFERENCE ANSWER — PHASE 18 TEACH-BACK MODEL

---

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

“Phase 18 created the verified boundary described by **Secure secrets, control cost, and review operational settings**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed.”

## Mini quiz

---

1. Multiple choice: Why does Phase 18 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
3. Fill in the blank: The statement that defines what one row represents is the table or entity \_\_\_\_\_.



4. What command would you use first to confirm the PowerShell current folder before a repository command?
  5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
  6. Design choice: Explain one tradeoff of Track A and one production alternative.
  7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
  8. Ownership: Which status may you select only after personally performing and inspecting the verification?
- 

## Mini quiz answer key and explanations

---

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

## My Phase 18 Learning Journal

---

### Date completed:

[ENTER DATE]

### What I created myself:

[WRITE YOUR ANSWER]

### Commands I ran:

[WRITE YOUR ANSWER]

### What confused me:

[WRITE YOUR ANSWER]

### What error occurred:

[WRITE YOUR ANSWER]

### What the error meant:

[WRITE YOUR ANSWER]

### How I fixed it:

[WRITE YOUR ANSWER]

### What I learned:

[WRITE YOUR ANSWER]

### What I can now explain without help:

[WRITE YOUR ANSWER]

### What I still need to practice:

[WRITE YOUR ANSWER]

## What your screen should look like now

---

- The phase output and final validation are saved or visible.

- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

## Lesson 18.5 completion checklist

---

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

## Safe stopping point

---

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

## How this phase connects to the next phase

---

Phase 19 automates local, infrastructure-free quality checks on every change.

## Phase 18 completion checklist

---

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [ ] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

# Phase 19 — Build GitHub Actions continuous integration

[Back to the master table of contents](#)

**Track:** Track A required

**Status when first opened:** PLANNED

**Completion rule:** The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

## Why this phase happens now

---

**Earlier work this phase depends on:** A passing CI workflow and reviewable branch history.

**What would go wrong if it happened earlier:** You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

**Later work that depends on it:** Phase 20 documents only verified work and links each claim to evidence.

**Decision being made:** Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

**Alternatives:** A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

**Beginner path selected:** Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

**Reason for the order:** Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

## Phase 19 lesson index

---

1. [Lesson 19.1 — Understand the phase and starting screen](#)
2. [Lesson 19.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 19.3 — Create and understand the phase files](#)
4. [Lesson 19.4 — Build and validate required objects](#)
5. [Lesson 19.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

## Lesson 19.1 — Understand the phase and starting screen

---

### What your screen should look like before starting

---

- **Application open:** VS Code, PowerShell, GitHub repository Actions page, and workflow logs.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 19 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 18 must be verified when this phase has a prerequisite.

## Three-level explanation of the phase

---

### Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

## Level 2 — Northstar Retail project explanation

Phase 19 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

## Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

## Applications to open

---

VS Code, PowerShell, GitHub repository Actions page, and workflow logs. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

## Phase status gate

---

☐

I changed this phase from [ ] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

## What your screen should look like now

---

- The Phase 19 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 19 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 19 master-tracker row is marked [-] In progress only after you personally updated it.

## Lesson 19.1 completion checklist

---

☐

I read why Phase 19 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

## Safe stopping point

---

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 19.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 19 lesson index](#)

## Lesson 19.2 — Perform the notebook-guided actions one at a time

### What your screen should look like before starting

The Phase 19 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

### Retained phase lesson, exercises, rules, and expected results

**Track:** Track A required

**Ownership status at start:** Workbook instructions exist; your execution is not yet verified.

**Evidence rule:** Replace placeholders only after the verification steps pass.

### 1. What you are building

A GitHub Actions workflow that runs on pushes and pull requests without paid database or Databricks infrastructure. It validates required files, SQL dialect labels, notebook syntax/JSON, secret patterns, relative Markdown links, Python linting, and deterministic local tests. You also practice branches, pull requests, failed-run diagnosis, and safe merges.

### 2. Why it matters

Continuous integration catches repeatable problems before they reach the main branch. It does not replace human review or external-platform testing, but it makes the repository contract visible and enforceable.

### 3. Words to know

Term	Plain-language definition
CI	Continuous Integration: automatically checking changes when code is pushed or proposed.
CD	Continuous Delivery/Deployment: preparing or releasing approved changes; advanced deployment remains optional here.
Workflow	A YAML file in <code>.github/workflows/</code> that defines automated jobs and steps.
Trigger	An event such as <code>push</code> or <code>pull_request</code> that starts a workflow.
Runner	The temporary machine that executes workflow steps.
Job	A group of steps that runs on one runner.
Step	One action or shell command inside a job.
Action	A reusable GitHub Actions component such as <code>checkout</code> or <code>setup-python</code> .
Artifact	A file retained from a workflow run; not required for this lightweight validation job.
Pull request	A proposed branch merge with review, discussion, and automated checks.
Required check	A status that repository rules can require before merge.
Lint	Static code-style and likely-error checks.
Deterministic test	A test that produces the same result for the same inputs.

### 4. What must already be completed

The project runs locally in a virtual environment, sample generation is deterministic, validation scripts exist, and no test requires a live SQL Server, PostgreSQL, Databricks, or Power BI service in GitHub Actions.

### 5. Applications to open

Application	How to open it	Why it is open
VS Code	Open <code>.github/workflows/validate_project.yml</code> , tests, and validation scripts.	Read and edit the automation.
Windows PowerShell	Open in repository root with <code>.venv</code> active.	Run the exact CI checks locally.

Application	How to open it	Why it is open
GitHub browser	Open repository > Actions and Pull requests.	Inspect runs, logs, checks, and merge state.
Git	Use in PowerShell/VS Code Source Control.	Create branch, commit, push, and update PR.

## 6. Exact click-by-click steps

**Beginner rhythm:** Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

## 7. Exact commands, one code block at a time

## 8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run `Get-Location`; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

## TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

### Line-by-line code explanation

Line or action	Meaning	Why it belongs here
<code>python -m scripts.validation.check_required_files</code>	Runs the repository-contract validations used by CI.	Local reproduction is faster and clearer than repeatedly pushing blind fixes.
<code>python -m ruff check scripts tests</code>	Runs static Python checks and all deterministic tests.	The workflow should not be the first place these checks run.
<code>git switch main</code>	Creates a focused feature branch from current main.	GitHub flow isolates work and makes review safer.
<code>git add .github tests scripts/validation docs</code>	Stages selected paths, commits, and publishes the branch.	A pull request requires the branch on GitHub.

### What this workflow proves

It proves the repository can be installed on a clean Python runner; deterministic sample/unit tests pass; required paths exist; SQL files declare their dialect; notebooks parse; simple secret patterns are absent; and relative documentation links resolve. It does **not** prove that SQL Server, PostgreSQL, Databricks Jobs, or Power BI succeeded in your environment. Those require the manual evidence gates in the workbook.

### Failure-reading order

Read the first failed step, the exact command, and the first actionable error. Later errors are often consequences. Reproduce locally, change one cause, rerun the failed command, then rerun the complete suite.

## MY TASK

Run the full workflow locally, open a small pull request, diagnose one intentionally controlled failure, fix it, obtain green checks, review the diff, and merge through GitHub flow without bypassing evidence or adding secrets.

## STARTER SKELETON

```
TODO(USER): explain every line before changing this workflow.
name: Validate Northstar project
on:
 pull_request:
 permissions:
 contents: read
jobs:
 validate:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
```

```

- uses: actions/setup-python@v5
 with:
 python-version: '3.13'
TODO(USER): install dependencies
TODO(USER): run required-file, dialect, notebook, secret, link, lint, and test checks

```

**Acceptance criteria:** All local checks pass; the pull request displays the expected workflow; a controlled failure is reproduced and fixed; final checks are green; the reviewed diff contains no secrets/generated raw data/false claims; merge and post-merge main check succeed.

## HINT 1 — CONCEPT

A CI failure is a command failure on a clean machine; reproduce the exact command locally first.

## HINT 2 — PSEUDOCODE

Read first red step -> copy command -> run locally -> fix one root cause -> rerun that command -> rerun all.

## HINT 3 — NEAR-COMPLETE PATTERN

Use the existing workflow and validation modules; introduce a reversible broken link or failing assertion on a temporary branch, observe the check, then restore/fix it before merge.

## 13. Expected files or output

- Passing local validation output.
- One focused pull request using the in-workbook template.
- Controlled failed-run screenshot/log note and corrected green run.
- Reviewed Files changed evidence.
- Successful main-branch workflow after merge.

## 14. How to verify

- Workflow YAML parses and appears under GitHub Actions.
- No workflow secret is required for local-only tests.
- Python version and dependency files match project configuration.
- Every CI command can be run locally.
- Controlled failure is real and then removed/fixed.
- PR description distinguishes automated checks from manual platform verification.
- Main remains green after merge.

## 15. Automated tests

- All commands in workflow locally.
- YAML parser/Actions recognition.
- Controlled negative test.
- PR diff secret/generated-file review.
- Post-merge main workflow.

## 16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
Workflow does not appear	File is not under .github/workflows, YAML is invalid, or branch was not pushed.	Check path, git status, remote branch, and YAML indentation.	Move/fix/commit/push the workflow; use spaces, not tabs.	Actions page lists the workflow.
ModuleNotFoundError in Actions	Dependency missing or project root/import path differs.	Read install step and failing import; inspect requirements/pyproject.	Add the legitimate dependency or fix package/import structure; do not install unrelated packages blindly.	Clean local environment and Actions both import it.
Link check fails	A relative path is wrong or referenced file was renamed.	Open the reported Markdown and resolve path relative to that file.	Correct the link or restore the intentional file.	Local link validator and Actions pass.



Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
Secret scan fails	Real credential or token-like example is present.	Open exact file/line and check commit history/staged diff.	Rotate real secret and remove it; use unmistakable placeholders for examples.	Scanner passes and no usable secret remains.
PR checks green but project did not run in Databricks	CI scope is local contract only.	Read workflow steps and manual gate checklist.	Do not overclaim; attach separate Databricks/Power BI evidence after execution.	README status accurately distinguishes automated and manual verification.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

## 17. Explain it in your own words

1. What does CI prove in this project and what does it not prove?
2. Why should a failing Actions command be reproduced locally?
3. Why are database credentials not added to this beginner workflow?
4. How does a pull request reduce risk compared with pushing directly to main?

Write your answers in [the learning-journal fields in this phase](the learning-journal fields in this phase). Do not copy the wording above.

## 18. Mini quiz

1. Where must a GitHub Actions workflow file be stored?
2. Which permission does the validation workflow request?
3. What is the first place to look when a workflow fails?

Record your answers before opening the answer key.

## 19. Screenshot checkpoint

Capture local green checks, the controlled red Actions step, the corrected green PR checks, and reviewed Files changed tab. Redact account notifications or private metadata.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

## 20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
ci: validate northstar project contract
```

Push the feature branch only after tests pass.

## 21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

Workflow understood line by line — [STATUS]

☐

All CI commands pass locally — [STATUS]

☐

Feature branch and PR created — [STATUS]

☐

Controlled failure diagnosed — [STATUS]

☐

Fix produced green checks — [STATUS]

☐

Diff reviewed for safety/truthfulness — [STATUS]

☐

PR merged safely — [STATUS]



Main workflow green — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: [the optional reference-answer section later in this workbook#phase-19](the optional reference-answer section later in this workbook#phase-19). The quiz answer is in [the answer-key section immediately after the quiz in this workbook#phase-19](the answer-key section immediately after the quiz in this workbook#phase-19).

---

**Next phase:** Phase 20 assembles the technical evidence into a professional, truthful portfolio repository.

## Required action cards

---

### Action 19.2.1 — Open `.github/workflows/validate_project.yml`. Read the trigger, permissions, runner, Python version, dependen

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Open `.github/workflows/validate_project.yml`. Read the trigger, permissions, runner, Python version, dependency install, and each named validation step.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open `.github/workflows/validate_project.yml`. Read the trigger, permissions, runner, Python version, dependency install, and each named validation step.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

#### Step 1

**Action**

Open `.github/workflows/validate_project.yml`. Read the trigger, permissions, runner, Python version, dependency install, and each named validation step.

**Before continuing**



I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## Action 19.2.2 — Confirm the workflow uses contents: read and does not require repository secrets. Do not add database passwo

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Confirm the workflow uses contents: read and does not require repository secrets. Do not add database passwo. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Confirm the workflow uses contents: read and does not require repository secrets. Do not add database passwords merely to make cloud CI imitate local integration tests.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 2****Action**

Confirm the workflow uses contents: read and does not require repository secrets. Do not add database passwords merely to make cloud CI imitate local integration tests.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## Action 19.2.3 — In PowerShell, activate .venv and run each workflow command locally in the same order. Fix the first failure

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

In PowerShell, activate .venv and run each workflow command locally in the same order. Fix the first failure. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: In PowerShell, activate .venv and run each workflow command locally in the same order. Fix the first failure rather than continuing blindly.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

### Step 3

**Action**

In PowerShell, activate .venv and run each workflow command locally in the same order. Fix the first failure rather than continuing blindly.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.



I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## Action 19.2.4 — Open the tests and identify what is unit, integration-contract, data-quality-fixture, and reconciliation. Note

**Application:**

Databricks workspace in a web browser

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Open the tests and identify what is unit, integration-contract, data-quality-fixture, and reconciliation. Note. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.

3. Perform only this immediate task: Open the tests and identify what is unit, integration-contract, data-quality-fixture, and reconciliation. Note that these tests validate repository fixtures and reference patterns, not a live Databricks run.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 4****Action**

Open the tests and identify what is unit, integration-contract, data-quality-fixture, and reconciliation. Note that these tests validate repository fixtures and reference patterns, not a live Databricks run.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## Action 19.2.5 — Create a branch named for one small change. Make the change, run tests, review git diff, commit, and push th

---

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

### Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Create a branch named for one small change. Make the change, run tests, review git diff, commit, and push th. This is one bounded construction or inspection task.

### Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

### Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Create a branch named for one small change. Make the change, run tests, review git diff, commit, and push the branch.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

### Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

### Where to enter it:

Windows PowerShell or the VS Code PowerShell terminal. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

### What each part means:

## Step 5

### Action

Create a branch named for one small change. Make the change, run tests, review git diff, commit, and push the branch.

### Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

### Expected example:



Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## Action 19.2.6 — In GitHub open , select your branch into main, use the template, and state what was tested and what remains

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

In GitHub open , select your branch into main, use the template, and state what was tested and what remains . This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: In GitHub open **Pull requests > New pull request**, select your branch into main, use the template, and state what was tested and what remains unverified.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 6****Action**

In GitHub open **Pull requests > New pull request**, select your branch into main, use the template, and state what was tested and what remains unverified.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## Action 19.2.7 — Open the PR checks. If a check fails, click , open the failed job and first red step, read the exact command/e

**Application:**

Databricks workspace in a web browser

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Open the PR checks. If a check fails, click , open the failed job and first red step, read the exact command/e. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open the PR checks. If a check fails, click **Details**, open the failed job and first red step, read the exact command/error, reproduce it locally, and make one focused fix.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 7****Action**

Open the PR checks. If a check fails, click **Details**, open the failed job and first red step, read the exact command/error, reproduce it locally, and make one focused fix.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## Action 19.2.8 — Push the fix to the same branch. Confirm the PR updates automatically and all checks turn green.

---

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Push the fix to the same branch. Confirm the PR updates automatically and all checks turn green. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Push the fix to the same branch. Confirm the PR updates automatically and all checks turn green.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

## What each part means:

### Step 8

#### Action

Push the fix to the same branch. Confirm the PR updates automatically and all checks turn green.

#### Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

#### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

#### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

#### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

#### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

#### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

#### Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

#### Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

#### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

#### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## Action 19.2.9 — Review the Files changed tab for secrets, generated data, unrelated edits, false claims, and broken paths. Mer

#### Application:

Visual Studio Code

#### Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

#### Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Review the Files changed tab for secrets, generated data, unrelated edits, false claims, and broken paths. Merge. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Review the Files changed tab for secrets, generated data, unrelated edits, false claims, and broken paths. Merge only after review and required checks pass.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 9****Action**

Review the Files changed tab for secrets, generated data, unrelated edits, false claims, and broken paths. Merge only after review and required checks pass.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## Action 19.2.10 — After merge, switch to main, pull, delete the local feature branch, and confirm the main-branch workflow suc

---

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

After merge, switch to main, pull, delete the local feature branch, and confirm the main-branch workflow suc. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: After merge, switch to main, pull, delete the local feature branch, and confirm the main-branch workflow succeeds.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

### Step 10

**Action**

After merge, switch to main, pull, delete the local feature branch, and confirm the main-branch workflow succeeds.

**Before continuing**

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## Action 19.2.11 — Optional: configure branch protection/rulesets requiring the validation workflow and a PR. Account/repository

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Optional: configure branch protection/rulesets requiring the validation workflow and a PR. Account/repository . This is one bounded construction or inspection task.



**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Optional: configure branch protection/rulesets requiring the validation workflow and a PR. Account/repository capability can differ, so record what is actually available.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 11****Action**

Optional: configure branch protection/rulesets requiring the validation workflow and a PR. Account/repository capability can differ, so record what is actually available.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## Action 19.2.12 — Keep deployment to Databricks/SQL Server optional. A later Declarative Automation Bundle or cloud deployment m

---

**Application:**

Databricks workspace in a web browser

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Keep deployment to Databricks/SQL Server optional. A later Declarative Automation Bundle or cloud deployment m. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Keep deployment to Databricks/SQL Server optional. A later Declarative Automation Bundle or cloud deployment must use environment identities/secrets and separate validation/deployment stages.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

### Step 12

**Action**

Keep deployment to Databricks/SQL Server optional. A later Declarative Automation Bundle or cloud deployment must use environment identities/secrets and separate validation/deployment stages.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.



I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## Action 19.2.C1 — Windows PowerShell

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (Powershell):

```
python -m scripts.validation.check_required_files
```

#### Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `python -m scripts.validation.check_required_files` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.check_required_files` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

#### Does it change anything?

This is intended to inspect or validate state rather than change it.

#### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

#### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.2.C2 — Windows PowerShell

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Powershell):**

```
python -m scripts.validation.validate_sql_dialects
```

**Where to enter it:**

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `python -m scripts.validation.validate_sql_dialects` — python invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.validate_sql_dialects` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.2.C3 — Windows PowerShell

**Application:**

Databricks workspace in a web browser

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Databricks workspace in a web browser** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (Powershell):

```
python -m scripts.validation.validate_notebooks
```

#### Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `python -m scripts.validation.validate_notebooks` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.validate_notebooks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

#### Does it change anything?

This is intended to inspect or validate state rather than change it.

#### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.2.C4 — Windows PowerShell

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Powershell):**

```
python -m scripts.validation.scan_secrets
```

**Where to enter it:**

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `python -m scripts.validation.scan_secrets` — python invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.scan_secrets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.



**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.2.C5 — Windows PowerShell

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (Powershell):

```
python -m scripts.validation.validate_markdown_links
```

#### Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `python -m scripts.validation.validate_markdown_links` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.validate_markdown_links` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

#### Does it change anything?

This is intended to inspect or validate state rather than change it.

#### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.2.C6 — Windows PowerShell

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Powershell):**

```
python -m ruff check scripts tests
```

**Where to enter it:**

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `python -m ruff check scripts tests` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `ruff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `check` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scripts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tests` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.2.C7 — Windows PowerShell

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (Powershell):

```
python -m pytest -q
```

#### Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `python -m pytest -q` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `pytest` the Python test runner; `-q` is a command option or flag; its exact behavior is defined beside the command

#### Does it change anything?

This is intended to inspect or validate state rather than change it.

#### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.2.C8 — Windows PowerShell

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Powershell):**

```
git switch main
```

**Where to enter it:**

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git switch main` — `git` calls the Git version-control program; `switch` changes the active branch; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.2.C9 — Windows PowerShell

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (Powershell):

```
git pull
```

#### Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `git pull` — `git` calls the Git version-control program; `pull` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

#### Does it change anything?

This is intended to inspect or validate state rather than change it.

#### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.



**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.2.C10 — Windows PowerShell

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Powershell):**

```
git switch -c docs/phase-19-ci-evidence
```

**Where to enter it:**

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git switch -c docs/phase-19-ci-evidence` — `git` calls the Git version-control program; `switch` changes the active branch; `-c` creates the named branch before switching; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `phase-19-ci-evidence` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.2.C11 — Windows PowerShell

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (Powershell):

```
git add .github tests scripts/validation docs
```

#### Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `git add .github tests scripts/validation docs` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `github` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tests` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `scripts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `validation` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

#### Does it change anything?

This can change files, Git state, packages, database objects, or data.

#### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.2.C12 — Windows PowerShell

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Powershell):**

```
git status
```

**Where to enter it:**

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git status` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work

### Does it change anything?

This is intended to inspect or validate state rather than change it.

### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

### How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.2.C13 — Windows PowerShell

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

Windows PowerShell

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (Powershell):

```
git commit -m "ci: validate northstar project contract"
```

### Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git commit -m "ci: validate northstar project contract"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"ci: validate northstar project contract"` is a quoted literal; the quotation marks preserve spaces or mark text

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.2.C14 — Windows PowerShell

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Powershell):**

```
git push -u origin docs/phase-19-ci-evidence
```

**Where to enter it:**

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git push -u origin docs/phase-19-ci-evidence` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `phase-19-ci-evidence` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

### How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

## Lesson 19.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]





I distinguished typed text from observed output — [STATUS]



I recorded differences caused by versions/tier — [STATUS]



I stopped at the first unexplained error — [STATUS]

## Safe stopping point

---

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 19 starting-screen checklist and re-verify the active context.

## Lesson 19.3 — Create and understand the phase files

---

### What your screen should look like before starting

---

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 19 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

#### Folder introduction — .github/

**Exact folder name:** .github

**Exact parent:** repository root

**Why it is needed now:** Phase 19 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

**What will eventually be inside:** Only files whose purpose matches this folder's name.

**Why another folder is not appropriate:** Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

**Naming rules:** Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

#### VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: .github.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

#### PowerShell method

### Action 19.D1.1 — Create the folder .github

---

#### Application:

Windows PowerShell or the VS Code PowerShell terminal

#### Window or panel:

VS Code integrated PowerShell terminal at the repository root

#### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

#### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

#### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\github" -ErrorAction Stop
```

### Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\github" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\github" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

**Folder-creation decision branches****If the folder is created**

Run `Test-Path -LiteralPath ".\github" -PathType Container` and require True.

**If PowerShell says the folder already exists**

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\github" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

**If the parent path does not exist**

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

**Verify the folder**

1. Run `Test-Path ".\github" -PathType Container`.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

**Folder introduction — scripts/**

**Exact folder name:** scripts

**Exact parent:** repository root

**Why it is needed now:** Phase 19 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

**What will eventually be inside:** Only files whose purpose matches this folder's name.

**Why another folder is not appropriate:** Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

**Naming rules:** Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

**VS Code method**

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: scripts.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

**PowerShell method**

## Action 19.D2.1 — Create the folder scripts

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal at the repository root

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\scripts" -ErrorAction Stop
```

### Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\scripts" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\scripts"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

**Folder-creation decision branches****If the folder is created**

Run `Test-Path -LiteralPath ".\scripts" -PathType Container` and require `True`.

**If PowerShell says the folder already exists**

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

**If the parent path does not exist**

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

**Verify the folder**

1. Run `Test-Path ".\scripts" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

**Folder introduction — tests/**

**Exact folder name:** `tests`

**Exact parent:** repository root

**Why it is needed now:** Phase 19 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

**What will eventually be inside:** Only files whose purpose matches this folder's name.

**Why another folder is not appropriate:** Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

**Naming rules:** Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

### VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: tests.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

### PowerShell method

## Action 19.D3.1 — Create the folder tests

---

#### Application:

Windows PowerShell or the VS Code PowerShell terminal

#### Window or panel:

VS Code integrated PowerShell terminal at the repository root

#### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

#### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

#### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

#### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

#### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\tests" -ErrorAction Stop
```

#### Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

## What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\tests" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\tests"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

## Does it change anything?

This can change files, Git state, packages, database objects, or data.

## Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

## What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

## Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

## What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

## How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

## Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

## Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

## Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

## What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

## How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Folder-creation decision branches

### If the folder is created

Run `Test-Path -LiteralPath ".\tests" -PathType Container` and require `True`.

### If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\tests" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

### If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

### Verify the folder

1. Run Test-Path ".\tests" -PathType Container.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

## Folder introduction — .github/workflows/

**Exact folder name:** workflows

**Exact parent:** .github

**Why it is needed now:** Phase 19 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

**What will eventually be inside:** Only files whose purpose matches this folder's name.

**Why another folder is not appropriate:** Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

**Naming rules:** Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

### VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path .github one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: workflows.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

### PowerShell method

## Action 19.D4.1 — Create the folder .github/workflows

---

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal at the repository root

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.



3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\github\workflows" -ErrorAction Stop
```

#### Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\github\workflows" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\github\workflows` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

#### Does it change anything?

This can change files, Git state, packages, database objects, or data.

#### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

**Folder-creation decision branches****If the folder is created**

Run `Test-Path -LiteralPath ".\github\workflows" -PathType Container` and require `True`.

**If PowerShell says the folder already exists**

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\github\workflows" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

**If the parent path does not exist**

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

**Verify the folder**

1. Run `Test-Path ".\github\workflows" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

**Folder introduction — scripts/validation/**

**Exact folder name:** `validation`

**Exact parent:** `scripts`

**Why it is needed now:** Phase 19 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

**What will eventually be inside:** Only files whose purpose matches this folder's name.

**Why another folder is not appropriate:** Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

**Naming rules:** Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

**VS Code method**

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `scripts` one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: `validation`.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

**PowerShell method****Action 19.D5.1 — Create the folder scripts/validation**

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal at the repository root

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
New-Item -ItemType Directory -Path ".\scripts\validation" -ErrorAction Stop
```

**Where to enter it:**

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** New-Item -ItemType Directory -Path ".\scripts\validation" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\scripts\validation" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This can change files, Git state, packages, database objects, or data.

**Is it safe to rerun?**

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

**Folder-creation decision branches****If the folder is created**

Run `Test-Path -LiteralPath ".\scripts\validation" -PathType Container` and require `True`.

**If PowerShell says the folder already exists**

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\scripts\validation" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

**If the parent path does not exist**

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

**Verify the folder**

1. Run `Test-Path ".\scripts\validation" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

**Folder introduction — tests/integration/**

**Exact folder name:** `integration`

**Exact parent:** `tests`

**Why it is needed now:** Phase 19 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

**What will eventually be inside:** Only files whose purpose matches this folder's name.

**Why another folder is not appropriate:** Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

**Naming rules:** Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

**VS Code method**

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path `tests` one level at a time.

4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: integration.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

## PowerShell method

### Action 19.D6.1 — Create the folder tests/integration

---

#### Application:

Windows PowerShell or the VS Code PowerShell terminal

#### Window or panel:

VS Code integrated PowerShell terminal at the repository root

#### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

#### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

#### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

#### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

#### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\tests\integration" -ErrorAction Stop
```

#### Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\tests\integration" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\tests\integration" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This can change files, Git state, packages, database objects, or data.

**Is it safe to rerun?**

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

**Folder-creation decision branches****If the folder is created**

Run `Test-Path -LiteralPath ".\tests\integration" -PathType Container` and require `True`.

**If PowerShell says the folder already exists**

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\tests\integration" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

**If the parent path does not exist**

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

**Verify the folder**

1. Run `Test-Path ".\tests\integration" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

## File introduction — .github/workflows/validate\_project.yml

**Exact filename:** validate\_project.yml

**Filename extension:** .yml. In this project it identifies human-readable configuration and helps the correct application choose syntax highlighting or execution behavior.

**Exact parent folder:** .github/workflows relative to the repository root.

**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

**What the file will contain:** human-readable configuration for Phase 19.

**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.

**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

### VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in .github/workflows one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: validate\_project.yml.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled validate\_project.yml and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

### PowerShell path

## Action 19.F1.1 — Create the file .github/workflows/validate\_project.yml from PowerShell

---

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.

4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\github\workflows\validate_project.yml" -ErrorAction Stop
```

### Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\github\workflows\validate_project.yml" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\github\workflows\validate_project.yml` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.



## What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

## How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## File-creation decision branches

### If the command creates the file

Run `Test-Path -LiteralPath ".\github\workflows\validate_project.yml" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

### If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- ".github/workflows/validate_project.yml"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

### If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

## File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

## File verification

1. In the VS Code Explorer, confirm the file appears under `.github/workflows`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `.github/workflows/validate_project.yml`.
3. In PowerShell at the repository root, run `Test-Path ".github\workflows\validate_project.yml"` and require True.
4. Run `git status --short -- ".github/workflows/validate_project.yml"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

## Optional reference answer for `.github/workflows/validate_project.yml`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** YAML

**Exact file:** `.github/workflows/validate_project.yml`

**Purpose:** human-readable configuration.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 19.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
name: Validate Northstar project

on:
 push:
 branches: [main, "feature/**", "fix/**", "docs/**"]
 pull_request:

permissions:
 contents: read

jobs:
 validate:
 runs-on: ubuntu-latest
 steps:
 - name: Check out repository
 uses: actions/checkout@v4

 - name: Set up Python 3.13
 uses: actions/setup-python@v5
 with:
 python-version: "3.13"
```

```

cache: pip

- name: Install dependencies
 run: |
 python -m pip install --upgrade pip
 python -m pip install -r requirements-dev.txt

- name: Required files
 run: python -m scripts.validation.check_required_files

- name: SQL dialect declarations
 run: python -m scripts.validation.validate_sql_dialects

- name: Notebook validation
 run: python -m scripts.validation.validate_notebooks

- name: Secret-pattern scan
 run: python -m scripts.validation.scan_secrets

- name: Relative Markdown links
 run: python -m scripts.validation.validate_markdown_links

- name: Lint Python
 run: python -m ruff check scripts tests

- name: Unit, integration, quality, and reconciliation tests
 run: python -m pytest -q

```

## Line-by-line explanation

- **Line 1:** name: Validate Northstar project — name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Validate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Northstar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; project is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** on: — on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** push: — push sends local commits to a remote repository; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** branches: [main, "feature/\*\*", "fix/\*\*", "docs/\*\*"] — branches is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; main is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "feature/\*\*" is a path, relative location, branch-style name, or file argument; verify every character before running it; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "fix/\*\*" is a path, relative location, branch-style name, or file argument; verify every character before running it; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "docs/\*\*" is a path, relative location, branch-style name, or file argument; verify every character before running it; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** pull\_request: — pull\_request is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** permissions: — permissions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** contents: read — contents is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; read is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** blank line used only to separate logical sections for readability.

- **Line 11:** jobs: — jobs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** validate: — validate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** runs-on: ubuntu-latest — runs-on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ubuntu-latest is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** steps: — steps is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** - name: Check out repository — - is a command option or flag; its exact behavior is defined beside the command; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Check is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; out is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; repository is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** uses: actions/checkout@v4 — uses is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; actions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; checkout is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; v4 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** blank line used only to separate logical sections for readability.
- **Line 18:** - name: Set up Python 3.13 — - is a command option or flag; its exact behavior is defined beside the command; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Set is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Python is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3.13 is a numeric literal used by the command or expression
- **Line 19:** uses: actions/setup-python@v5 — uses is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; actions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; setup-python is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; v5 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** with: — with is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** python-version: "3.13" — python-version is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "3.13" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 22:** cache: pip — cache is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pip the Python package installer module
- **Line 23:** blank line used only to separate logical sections for readability.
- **Line 24:** - name: Install dependencies — - is a command option or flag; its exact behavior is defined beside the command; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Install is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

is described in the surrounding explanation; dependencies is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 25:** run: | — run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** python -m pip install --upgrade pip — python invokes Python when that command name is registered; -m asks Python to run an installed module; pip the Python package installer module; install is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; --upgrade is a command option or flag; its exact behavior is defined beside the command; pip the Python package installer module
- **Line 27:** python -m pip install -r requirements-dev.txt — python invokes Python when that command name is registered; -m asks Python to run an installed module; pip the Python package installer module; install is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -r is a command option or flag; its exact behavior is defined beside the command; requirements-dev.txt is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** blank line used only to separate logical sections for readability.
- **Line 29:** - name: Required files — - is a command option or flag; its exact behavior is defined beside the command; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Required is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; files is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 30:** run: python -m scripts.validation.check\_required\_files — run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; python invokes Python when that command name is registered; -m asks Python to run an installed module; scripts.validation.check\_required\_files is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 31:** blank line used only to separate logical sections for readability.
- **Line 32:** - name: SQL dialect declarations — - is a command option or flag; its exact behavior is defined beside the command; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dialect is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; declarations is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 33:** run: python -m scripts.validation.validate\_sql\_dialects — run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; python invokes Python when that command name is registered; -m asks Python to run an installed module; scripts.validation.validate\_sql\_dialects is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 34:** blank line used only to separate logical sections for readability.
- **Line 35:** - name: Notebook validation — - is a command option or flag; its exact behavior is defined beside the command; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Notebook is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; validation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 36:** run: python -m scripts.validation.validate\_notebooks — run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; python invokes Python when that command name is registered; -m asks Python to run an installed module; scripts.validation.validate\_notebooks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 37:** blank line used only to separate logical sections for readability.
- **Line 38:** - name: Secret-pattern scan — - is a command option or flag; its exact behavior is defined beside the command; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Secret-pattern is a name, keyword, argument, identifier, or literal whose project-

specific role is described in the surrounding explanation; scan is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 39:** run: python -m scripts.validation.scan\_secrets — run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; python invokes Python when that command name is registered; -m asks Python to run an installed module; scripts.validation.scan\_secrets is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 40:** blank line used only to separate logical sections for readability.
- **Line 41:** - name: Relative Markdown links — - is a command option or flag; its exact behavior is defined beside the command; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Relative is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Markdown is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; links is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 42:** run: python -m scripts.validation.validate\_markdown\_links — run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; python invokes Python when that command name is registered; -m asks Python to run an installed module; scripts.validation.validate\_markdown\_links is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 43:** blank line used only to separate logical sections for readability.
- **Line 44:** - name: Lint Python — - is a command option or flag; its exact behavior is defined beside the command; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Lint is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Python is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 45:** run: python -m ruff check scripts tests — run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; python invokes Python when that command name is registered; -m asks Python to run an installed module; ruff is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; check is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; scripts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tests is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 46:** blank line used only to separate logical sections for readability.
- **Line 47:** - name: Unit, integration, quality, and reconciliation tests — - is a command option or flag; its exact behavior is defined beside the command; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Unit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; integration is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quality is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reconciliation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tests is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 48:** run: python -m pytest -q — run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; python invokes Python when that command name is registered; -m asks Python to run an installed module; pytest the Python test runner; -q is a command option or flag; its exact behavior is defined beside the command

[Back to the Phase 19 lesson index](#)

## File introduction — pyproject.toml

**Exact filename:** pyproject.toml

**Filename extension:** .toml. In this project it identifies project text or configuration and helps the correct application choose syntax highlighting or execution behavior.

**Exact parent folder:** . relative to the repository root.

**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

**What the file will contain:** project text or configuration for Phase 19.

**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.

**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

### VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in . one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: pyproject.toml.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled pyproject.toml and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

### PowerShell path

## Action 19.F2.1 — Create the file pyproject.toml from PowerShell

---

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. **PRESS:** Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. **LOOK FOR:** Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. **DO NOT CONTINUE UNTIL:** Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\pyproject.toml" -ErrorAction Stop
```

#### Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\pyproject.toml" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\pyproject.toml"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

#### Does it change anything?

This can change files, Git state, packages, database objects, or data.

#### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

#### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

## How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## File-creation decision branches

### If the command creates the file

Run `Test-Path -LiteralPath ".\pyproject.toml" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

### If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "pyproject.toml"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

### If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

## File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

## File verification

1. In the VS Code Explorer, confirm the file appears under `..`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `pyproject.toml`.
3. In PowerShell at the repository root, run `Test-Path ".\pyproject.toml"` and require True.
4. Run `git status --short -- "pyproject.toml"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

## Optional reference answer for `pyproject.toml`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** TOML

**Exact file:** `pyproject.toml`

**Purpose:** project text or configuration.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 19.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
[build-system]
requires = ["setuptools>=75", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "northstar-retail-data-platform"
version = "0.1.0"
description = "Beginner-owned end-to-end retail data engineering portfolio project"
requires-python = ">=3.13,<3.15"
readme = "README.md"
license = {text = "MIT"}
authors = [{name = "Kirolos Boules"}]

[tool.pytest.ini_options]
addopts = "-q"
testpaths = ["tests"]
pythonpath = ["."]

[tool.ruff]
line-length = 100
target-version = "py313"
exclude = ["datasets", "notebooks"]

[tool.ruff.lint]
```



```
select = ["E", "F", "I", "B", "UP"]
ignore = ["E501"]
```

## Line-by-line explanation

- **Line 1:** [build-system] — [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; build-system is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** requires = ["setuptools>=75", "wheel"] — requires is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "setuptools>=75" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "wheel" is a quoted literal; the quotation marks preserve spaces or mark text; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** build-backend = "setuptools.build\_meta" — build-backend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "setuptools.build\_meta" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** [project] — [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; project is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** name = "northstar-retail-data-platform" — name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "northstar-retail-data-platform" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 7:** version = "0.1.0" — version is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "0.1.0" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 8:** description = "Beginner-owned end-to-end retail data engineering portfolio project" — description is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Beginner-owned end-to-end retail data engineering portfolio project" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 9:** requires-python = ">=3.13,<3.15" — requires-python is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ">=3.13,<3.15" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 10:** readme = "README.md" — readme is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "README.md" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 11:** license = {text = "MIT"} — license is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; { is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; text is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "MIT" is a quoted literal; the quotation marks preserve spaces or mark text; } is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** authors = [{name = "Kirosos Boules"}] — authors is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [{ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; name is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Kiro-los Boules" is a quoted literal; the quotation marks preserve spaces or mark text; } } is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 13:** blank line used only to separate logical sections for readability.
- **Line 14:** [tool.pytest.ini\_options] — [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tool.pytest.ini\_options is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** addopts = "-q" — addopts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "-q" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 16:** testpaths = ["tests"] — testpaths is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "tests" is a quoted literal; the quotation marks preserve spaces or mark text; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** pythonpath = ["."] — pythonpath is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "." is a quoted literal; the quotation marks preserve spaces or mark text; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** [tool.ruff] — [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tool.ruff is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** line-length = 100 — line-length is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 100 is a numeric literal used by the command or expression
- **Line 21:** target-version = "py313" — target-version is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "py313" is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 22:** exclude = ["datasets", "notebooks"] — exclude is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "datasets" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "notebooks" is a quoted literal; the quotation marks preserve spaces or mark text; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** blank line used only to separate logical sections for readability.
- **Line 24:** [tool.ruff.lint] — [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tool.ruff.lint is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** select = ["E", "F", "I", "B", "UP"] — select is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "E" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "F" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "I" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "B" is a quoted literal; the quotation marks preserve spaces or mark text; , is a name, keyword, argument, identifier, or literal whose project-specific

role is described in the surrounding explanation; "UP" is a quoted literal; the quotation marks preserve spaces or mark text; remaining punctuation and identifiers continue the same statement.

- **Line 26:** ignore = ["E501"] — ignore is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "E501" is a quoted literal; the quotation marks preserve spaces or mark text; ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 19 lesson index](#)

## File introduction — scripts/validation/check\_required\_files.py

**Exact filename:** check\_required\_files.py

**Filename extension:** .py. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

**Exact parent folder:** scripts/validation relative to the repository root.

**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

**What the file will contain:** executable Python logic for Phase 19.

**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.

**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

### VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in scripts/validation one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: check\_required\_files.py.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled check\_required\_files.py and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

### PowerShell path

## Action 19.F3.1 — Create the file scripts/validation/check\_required\_files.py from PowerShell

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\scripts\validation\check_required_files.py" -ErrorAction Stop
```

### Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\scripts\validation\check_required_files.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\scripts\validation\check_required_files.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

**File-creation decision branches****If the command creates the file**

Run `Test-Path -LiteralPath ".\scripts\validation\check_required_files.py" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

**If PowerShell says the file already exists**

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "scripts/validation/check_required_files.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

**If PowerShell says part of the path does not exist**

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

**File-content construction**

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

**File verification**

1. In the VS Code Explorer, confirm the file appears under `scripts/validation`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `scripts/validation/check_required_files.py`.
3. In PowerShell at the repository root, run `Test-Path ".\scripts\validation\check_required_files.py"` and require True.
4. Run `git status --short -- "scripts/validation/check_required_files.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

**Optional reference answer for `scripts/validation/check_required_files.py`**

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** Python

**Exact file:** `scripts/validation/check_required_files.py`

**Purpose:** executable Python logic.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 19.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
"""CI-friendly structural check for required project artifacts."""
from __future__ import annotations
```

```

from pathlib import Path

REQUIRED = [
 "README.md",
 "docs/architecture.md",
 "docs/business_requirements.md",
 "sql/sqlserver/source/01_create_erp_database.sql",
 "sql/postgres/source/01_create_digital_schema.sql",
 "scripts/data_generation/generate_northstar_data.py",
 "databricks/notebooks/00_environment_check.py",
 "databricks/jobs/job_definition.example.yml",
 "powerbi/measures.dax",
 ".github/workflows/validate_project.yml",
]

def main() -> None:
 missing = [item for item in REQUIRED if not Path(item).exists()]
 if missing:
 raise SystemExit("Missing required files:\n- " + "\n- ".join(missing))
 print(f"Required-file check passed: {len(REQUIRED)} artifacts found.")

if __name__ == "__main__":
 main()

```

## Line-by-line explanation

- **Line 1:** `"""CI-friendly structural check for required project artifacts."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `CI-friendly structural check for required project artifacts.` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 2:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** blank line used only to separate logical sections for readability.
- **Line 4:** `from pathlib import Path` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pathlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** blank line used only to separate logical sections for readability.
- **Line 6:** `REQUIRED = [` — `REQUIRED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `"README.md",` — `"README.md"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `"docs/architecture.md",` — `"docs/architecture.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `"docs/business_requirements.md",` — `"docs/business_requirements.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `"sql/sqlserver/source/01_create_erp_database.sql",` — `"sql/sqlserver/source/01_create_erp_database.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** `"sql/postgres/source/01_create_digital_schema.sql",` — `"sql/postgres/source/01_create_digital_schema.sql"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `"scripts/data_generation/generate_northstar_data.py",` — `"scripts/data_generation/generate_northstar_data.py"` is a path, relative location, branch-style name, or file argument; verify every character before

running it; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 13:** "databricks/notebooks/00\_environment\_check.py", — "databricks/notebooks/00\_environment\_check.py" is a path, relative location, branch-style name, or file argument; verify every character before running it; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** "databricks/jobs/job\_definition.example.yml", — "databricks/jobs/job\_definition.example.yml" is a path, relative location, branch-style name, or file argument; verify every character before running it; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** "powerbi/measures.dax", — "powerbi/measures.dax" is a path, relative location, branch-style name, or file argument; verify every character before running it; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** ".github/workflows/validate\_project.yml", — ".github/workflows/validate\_project.yml" is a path, relative location, branch-style name, or file argument; verify every character before running it; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** ] — ] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** blank line used only to separate logical sections for readability.
- **Line 20:** def main() -> None: — def is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; main is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -> is a command option or flag; its exact behavior is defined beside the command; None is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** missing = [item for item in REQUIRED if not Path(item).exists()] — missing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; item is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; item is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; REQUIRED is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 22:** if missing: — if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; missing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** raise SystemExit("Missing required files:\n- " + "\n- ".join(missing)) — raise is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SystemExit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Missing required files:\n- " is a path, relative location, branch-style name, or file argument; verify every character before running it; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "\n- " is a path, relative location, branch-style name, or file argument; verify every character before running it; . is a path, relative location, branch-style name, or file argument; verify every character before running it; join is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; missing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; )) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** print(f"Required-file check passed: {len(REQUIRED)} artifacts found.") — print is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; f is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "Required-file check passed: {len(REQUIRED)} artifacts found." is a quoted literal; the quotation marks preserve spaces or mark text; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 25:** blank line used only to separate logical sections for readability.
- **Line 26:** blank line used only to separate logical sections for readability.
- **Line 27:** if \_\_name\_\_ == "\_\_main\_\_": — if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; \_\_name\_\_ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "\_\_main\_\_" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 28:** main() — main is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 19 lesson index](#)

## File introduction — scripts/validation/validate\_markdown\_links.py

**Exact filename:** validate\_markdown\_links.py

**Filename extension:** .py. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

**Exact parent folder:** scripts/validation relative to the repository root.

**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

**What the file will contain:** executable Python logic for Phase 19.

**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.

**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

### VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in scripts/validation one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: validate\_markdown\_links.py.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled validate\_markdown\_links.py and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

### PowerShell path

## Action 19.F4.1 — Create the file scripts/validation/validate\_markdown\_links.py from PowerShell

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal opened at the repository root



**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
New-Item -ItemType File -Path ".\scripts\validation\validate_markdown_links.py" -ErrorAction Stop
```

**Where to enter it:**

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** New-Item -ItemType File -Path ".\scripts\validation\validate\_markdown\_links.py" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; File is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\scripts\validation\validate\_markdown\_links.py" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This can change files, Git state, packages, database objects, or data.

**Is it safe to rerun?**

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

**File-creation decision branches****If the command creates the file**

Run `Test-Path -LiteralPath ".\scripts\validation\validate_markdown_links.py" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

**If PowerShell says the file already exists**

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "scripts/validation/validate_markdown_links.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

**If PowerShell says part of the path does not exist**

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

**File-content construction**

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

**File verification**

1. In the VS Code Explorer, confirm the file appears under `scripts/validation`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `scripts/validation/validate_markdown_links.py`.
3. In PowerShell at the repository root, run `Test-Path ".\scripts\validation\validate_markdown_links.py"` and require `True`.
4. Run `git status --short -- "scripts/validation/validate_markdown_links.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

## Optional reference answer for scripts/validation/validate\_markdown\_links.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** Python

**Exact file:** scripts/validation/validate\_markdown\_links.py

**Purpose:** executable Python logic.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 19.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
"""Check repository-relative Markdown links; skip web URLs and anchors."""
from __future__ import annotations

import re
from pathlib import Path
from urllib.parse import unquote

LINK = re.compile(r"(?!)\[[^\]]*\]\\((([^\)]+)\))")

def main() -> None:
 failures = []
 for path in Path(".").rglob("*.md"):
 if any(part in {".git", ".venv"} for part in path.parts): continue
 text = path.read_text(encoding="utf-8")
 for raw in LINK.findall(text):
 target = raw.strip().split()[0].strip("<>")
 if target.startswith(("http://", "https://", "mailto:", "#")): continue
 target = unquote(target.split("#", 1)[0])
 if not target: continue
 resolved = (path.parent / target).resolve()
 if not resolved.exists(): failures.append(f"{path}: {raw}")
 if failures:
 raise SystemExit("Broken relative Markdown links:\n- " + "\n- ".join(failures))
 print("Relative Markdown link check passed.")

if __name__ == "__main__": main()
```

### Line-by-line explanation

- Line 1:** `"""Check repository-relative Markdown links; skip web URLs and anchors."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `Check repository-relative Markdown links; skip web URLs and anchors.` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- Line 2:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `annotations` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 3:** blank line used only to separate logical sections for readability.
- Line 4:** `import re` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `re` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 5:** `from pathlib import Path` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pathlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 6:** `from urllib.parse import unquote` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `urllib.parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `unquote` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 7:** blank line used only to separate logical sections for readability.
- Line 8:** `LINK = re.compile(r"(?!)\[[^\]]*\]\\((([^\)]+)\))")` — `LINK` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `re.compile` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `r` is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "(?<!!)\[[^\]]\*\]\((([^\)]+))\)" is a path, relative location, branch-style name, or file argument; verify every character before running it; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 9:** blank line used only to separate logical sections for readability.
- **Line 10:** blank line used only to separate logical sections for readability.
- **Line 11:** `def main() -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `failures = []` — `failures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `for path in Path(".").rglob("*.md"):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"."` is a quoted literal; the quotation marks preserve spaces or mark text; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rglob` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"*.md"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** `if any(part in {".git", ".venv"} for part in path.parts): continue` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `any` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `part` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `".git"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `".venv"` is a quoted literal; the quotation marks preserve spaces or mark text; `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `part` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 15:** `text = path.read_text(encoding="utf-8")` — `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.read_text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `encoding` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"utf-8"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `for raw in LINK.findall(text):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `LINK.findall` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `text` is a name,

- keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ) : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- Line 17:** `target = raw.strip().split()[0].strip("<>")` — `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw.strip` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `split` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `strip` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<>` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
  - Line 18:** `if target.startswith(("http://", "https://", "mailto:", "#")): continue` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.startswith` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `((` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"http://"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"https://"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"mailto:"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"#"` is a quoted literal; the quotation marks preserve spaces or mark text; `))` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `continue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
  - Line 19:** `target = unquote(target.split("#", 1)[0])` — `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `unquote` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target.split` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"#"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `1` is a numeric literal used by the command or expression; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; `])` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
  - Line 20:** `if not target: continue` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `continue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
  - Line 21:** `resolved = (path.parent / target).resolve()` — `resolved` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.parent` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `target` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `resolve` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
  - Line 22:** `if not resolved.exists(): failures.append(f"{path}: {raw}")` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `resolved.exists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failures.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(f"{path}: {raw}")` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; `resolved.exists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failures.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{path}: {raw}"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 23:** `if failures:` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** `raise SystemExit("Broken relative Markdown links:\n- " + "\n- ".join(failures))` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SystemExit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Broken relative Markdown links:\n- "` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\n- "` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` `.` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` `join` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** `print("Relative Markdown link check passed.")` — `print` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Relative Markdown link check passed."` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** blank line used only to separate logical sections for readability.
- **Line 27:** blank line used only to separate logical sections for readability.
- **Line 28:** `if __name__ == "__main__": main()` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__name__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"__main__"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 19 lesson index](#)

## File introduction — `scripts/validation/validate_notebooks.py`

**Exact filename:** `validate_notebooks.py`

**Filename extension:** `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

**Exact parent folder:** `scripts/validation` relative to the repository root.

**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

**What the file will contain:** executable Python logic for Phase 19.

**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.

**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

## VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in scripts/validation one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `validate_notebooks.py`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `validate_notebooks.py` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

## PowerShell path

### Action 19.F5.1 — Create the file `scripts/validation/validate_notebooks.py` from PowerShell

---

#### Application:

Databricks workspace in a web browser

#### Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

#### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

#### Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

#### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

#### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

#### Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\scripts\validation\validate_notebooks.py" -ErrorAction Stop
```

#### Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

## What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\scripts\validation\validate_notebooks.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\scripts\validation\validate_notebooks.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

## Does it change anything?

This can change files, Git state, packages, database objects, or data.

## Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

## What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

## Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

## What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

## How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

## Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

## Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

## Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

## What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

## How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## File-creation decision branches

### If the command creates the file

Run `Test-Path -LiteralPath ".\scripts\validation\validate_notebooks.py" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

### If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "scripts/validation/validate_notebooks.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.



## If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

## File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

## File verification

1. In the VS Code Explorer, confirm the file appears under `scripts/validation`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `scripts/validation/validate_notebooks.py`.
3. In PowerShell at the repository root, run `Test-Path ".\scripts\validation\validate_notebooks.py"` and require `True`.
4. Run `git status --short -- "scripts/validation/validate_notebooks.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

## Optional reference answer for `scripts/validation/validate_notebooks.py`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** Python

**Exact file:** `scripts/validation/validate_notebooks.py`

**Purpose:** executable Python logic.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 19.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
"""Validate .ipynb JSON and compile Git-friendly Databricks source notebooks."""
from __future__ import annotations

import ast
import json
from pathlib import Path

def main() -> None:
 failures = []
 for path in Path("notebooks").glob("*.ipynb"):
 try:
 payload = json.loads(path.read_text(encoding="utf-8"))
 if payload.get("nbformat") != 4 or "cells" not in payload:
 failures.append(f"{path}: missing notebook v4 fields")
 except Exception as exc:
 failures.append(f"{path}: {exc}")
 for path in Path("databricks/notebooks").glob("*.py"):
 try:
 ast.parse(path.read_text(encoding="utf-8"), filename=str(path))
 except SyntaxError as exc:
 failures.append(f"{path}:{exc.lineno}: {exc.msg}")
 if failures:
 raise SystemExit("Notebook validation failed:\n- " + "\n- ".join(failures))
 print("Notebook validation passed.")

if __name__ == "__main__": main()
```

## Line-by-line explanation

- **Line 1:** `"""Validate .ipynb JSON and compile Git-friendly Databricks source notebooks."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `"Validate .ipynb JSON and compile Git-friendly Databricks source notebooks."` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 2:** `from __future__ import annotations` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__future__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument,

identifier, or literal whose project-specific role is described in the surrounding explanation; annotations is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 3:** blank line used only to separate logical sections for readability.
- **Line 4:** `import ast` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ast` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `import json` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `json` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `from pathlib import Path` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pathlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** `def main() -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `failures = []` — `failures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 11:** `for path in Path("notebooks").glob("*.ipynb"):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"notebooks"` is a quoted literal; the quotation marks preserve spaces or mark text; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `glob` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"*.ipynb"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 12:** `try:` — `try` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `payload = json.loads(path.read_text(encoding="utf-8"))` — `payload` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `json.loads` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.read_text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `encoding` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"utf-8"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** `if payload.get("nbformat") != 4 or "cells" not in payload:` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `payload.get` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"nbformat"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

fier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 4 is a numeric literal used by the command or expression; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "cells" is a quoted literal; the quotation marks preserve spaces or mark text; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; payload is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 15:** `failures.append(f"{path}: missing notebook v4 fields")` — `failures.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{path}: missing notebook v4 fields"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `except Exception as exc:` — `except` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Exception` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exc` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** `failures.append(f"{path}: {exc}")` — `failures.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{path}: {exc}"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** `for path in Path("databricks/notebooks").glob("*.py"):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"databricks/notebooks"` is a path, relative location, branch-style name, or file argument; `verify every character before running it;` `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `glob` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"*.py"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** `try:` — `try` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** `ast.parse(path.read_text(encoding="utf-8"), filename=str(path))` — `ast.parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.read_text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `encoding` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"utf-8"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `filename` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `str` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 21:** `except SyntaxError as exc:` — `except` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SyntaxError` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `as` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;

er, or literal whose project-specific role is described in the surrounding explanation; exc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 22:** `failures.append(f"{path}:{exc.lineno}: {exc.msg}")` — `failures.append` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"{path}:{exc.lineno}: {exc.msg}"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 23:** `if failures:` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 24:** `raise SystemExit("Notebook validation failed:\n- " + "\n- ".join(failures))` — `raise` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SystemExit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Notebook validation failed:\n- "` is a path, relative location, branch-style name, or file argument; verify every character before running it; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"\n- "` is a path, relative location, branch-style name, or file argument; verify every character before running it; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `join` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 25:** `print("Notebook validation passed.")` — `print` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Notebook validation passed."` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 26:** blank line used only to separate logical sections for readability.
- **Line 27:** blank line used only to separate logical sections for readability.
- **Line 28:** `if __name__ == "__main__": main()` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `__name__` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `==` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"__main__"` is a quoted literal; the quotation marks preserve spaces or mark text; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 19 lesson index](#)

## File introduction — `scripts/validation/validate_sql_dialects.py`

**Exact filename:** `validate_sql_dialects.py`

**Filename extension:** `.py`. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

**Exact parent folder:** `scripts/validation` relative to the repository root.

**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

**What the file will contain:** executable Python logic for Phase 19.

**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.

**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

## VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in scripts/validation one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: validate\_sql\_dialects.py.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled validate\_sql\_dialects.py and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

## PowerShell path

### Action 19.F6.1 — Create the file scripts/validation/validate\_sql\_dialects.py from PowerShell

---

#### Application:

Windows PowerShell or the VS Code PowerShell terminal

#### Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

#### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

#### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

#### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

#### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

#### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\scripts\validation\validate_sql_dialects.py" -ErrorAction Stop
```

#### Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

## What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\scripts\validation\validate_sql_dialects.py" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\scripts\validation\validate_sql_dialects.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

## Does it change anything?

This can change files, Git state, packages, database objects, or data.

## Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

## What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

## Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

## What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

## How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

## Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

## Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

## Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

## What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

## How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## File-creation decision branches

### If the command creates the file

Run `Test-Path -LiteralPath ".\scripts\validation\validate_sql_dialects.py" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

### If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "scripts/validation/validate_sql_dialects.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

## If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

## File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

## File verification

1. In the VS Code Explorer, confirm the file appears under `scripts/validation`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `scripts/validation/validate_sql_dialects.py`.
3. In PowerShell at the repository root, run `Test-Path ".\scripts\validation\validate_sql_dialects.py"` and require `True`.
4. Run `git status --short -- "scripts/validation/validate_sql_dialects.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

## Optional reference answer for `scripts/validation/validate_sql_dialects.py`

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** Python

**Exact file:** `scripts/validation/validate_sql_dialects.py`

**Purpose:** executable Python logic.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 19.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
"""Require a dialect declaration near the start of every SQL file."""
from pathlib import Path

ALLOWED = {
 "-- Dialect: SQL Server / T-SQL",
 "-- Dialect: PostgreSQL SQL",
 "-- Dialect: Databricks SQL / Spark SQL",
 "-- Dialect: Portable ANSI-style SQL",
}

def main() -> None:
 failures = []
 for path in Path("sql").rglob("*.sql"):
 first_lines = path.read_text(encoding="utf-8").splitlines()[:5]
 if not any(line.strip() in ALLOWED for line in first_lines):
 failures.append(str(path))
 if failures:
 raise SystemExit("SQL files missing an approved dialect header:\n- " + "\n- ".join(failures))
 print("SQL dialect check passed.")

if __name__ == "__main__": main()
```

## Line-by-line explanation

- **Line 1:** `"""Require a dialect declaration near the start of every SQL file."""` — `"""` is a quoted literal; the quotation marks preserve spaces or mark text; `Require a dialect declaration near the start of every SQL file.` is a quoted literal; the quotation marks preserve spaces or mark text; `"""` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 2:** `from pathlib import Path` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pathlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** blank line used only to separate logical sections for readability.

- **Line 4:** `ALLOWED = {` — `ALLOWED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `{` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `-- Dialect: SQL Server / T-SQL`, — `-- Dialect: SQL Server / T-SQL` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `-- Dialect: PostgreSQL SQL`, — `-- Dialect: PostgreSQL SQL` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `-- Dialect: Databricks SQL / Spark SQL`, — `-- Dialect: Databricks SQL / Spark SQL` is a path, relative location, branch-style name, or file argument; verify every character before running it; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `-- Dialect: Portable ANSI-style SQL`, — `-- Dialect: Portable ANSI-style SQL` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `}` — `}` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** blank line used only to separate logical sections for readability.
- **Line 11:** blank line used only to separate logical sections for readability.
- **Line 12:** `def main() -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 13:** `failures = []` — `failures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 14:** `for path in Path("sql").rglob("*.sql"):` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"sql"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rglob` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;  `"*.sql"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** `first_lines = path.read_text(encoding="utf-8").splitlines()[5]` — `first_lines` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.read_text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `encoding` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"utf-8"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `splitlines` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `5` is a numeric literal used by the command or expression; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `if not any(line.strip() in ALLOWED for line in first_lines):` — `if` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, ar-



gument, identifier, or literal whose project-specific role is described in the surrounding explanation; any is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; line.strip is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ALLOWED is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; line is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; first\_lines is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 17:** failures.append(str(path)) — failures.append is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; str is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; )) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 18:** if failures: — if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; failures is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** raise SystemExit("SQL files missing an approved dialect header:\n- " + "\n- ".join(failures)) — raise is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SystemExit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "SQL files missing an approved dialect header:\n- " is a path, relative location, branch-style name, or file argument; verify every character before running it; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "\n- " is a path, relative location, branch-style name, or file argument; verify every character before running it; . is a path, relative location, branch-style name, or file argument; verify every character before running it; join is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; failures is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; )) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** print("SQL dialect check passed.") — print is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "SQL dialect check passed." is a quoted literal; the quotation marks preserve spaces or mark text; ) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 21:** blank line used only to separate logical sections for readability.
- **Line 22:** blank line used only to separate logical sections for readability.
- **Line 23:** if \_\_name\_\_ == "\_\_main\_\_": main() — if is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; \_\_name\_\_ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; == is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "\_\_main\_\_" is a quoted literal; the quotation marks preserve spaces or mark text; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; main is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; () is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 19 lesson index](#)

## File introduction — tests/integration/test\_repository\_contract.py

**Exact filename:** test\_repository\_contract.py

**Filename extension:** .py. In this project it identifies executable Python logic and helps the correct application choose syntax highlighting or execution behavior.

**Exact parent folder:** tests/integration relative to the repository root.

**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

**What the file will contain:** executable Python logic for Phase 19.

**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.

**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

## VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in tests/integration one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: test\_repository\_contract.py.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled test\_repository\_contract.py and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

## PowerShell path

# Action 19.F7.1 — Create the file tests/integration/test\_repository\_contract.py from PowerShell

---

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.

7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\tests\integration\test_repository_contract.py" -ErrorAction Stop
```

### Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\tests\integration\test_repository_contract.py" -ErrorAction Stop`  
— `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\tests\integration\test_repository_contract.py"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

## How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## File-creation decision branches

### If the command creates the file

Run `Test-Path -LiteralPath ".\tests\integration\test_repository_contract.py" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

### If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "tests/integration/test_repository_contract.py"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

### If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

## File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

## File verification

1. In the VS Code Explorer, confirm the file appears under `tests/integration`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `tests/integration/test_repository_contract.py`.
3. In PowerShell at the repository root, run `Test-Path ".\tests\integration\test_repository_contract.py"` and require True.
4. Run `git status --short -- "tests/integration/test_repository_contract.py"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

## Optional reference answer for tests/integration/test\_repository\_contract.py

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** Python

**Exact file:** `tests/integration/test_repository_contract.py`

**Purpose:** executable Python logic.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 19.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
from pathlib import Path
import yaml

def test_job_notebook_paths_exist() -> None:
 payload = yaml.safe_load(Path("databricks/jobs/job_definition.example.yml").read_text())
 tasks = payload["resources"]["jobs"]["northstar_retail_daily"]["tasks"]
 jobs_dir = Path("databricks/jobs")
 for task in tasks:
 path = (jobs_dir / task["notebook_task"]["notebook_path"]).resolve()
 assert path.exists(), f"Missing job notebook: {path}"

def test_all_gold_tables_are_in_dictionary() -> None:
 text = Path("docs/data_dictionary.csv").read_text(encoding="utf-8")
 for table in ["dim_date", "dim_customer", "dim_product", "dim_store", "dim_employee", "dim_supplier", "dim_promotion", "dim_channel", "dim_campaign", "fact_sales", "fact_returns", "fact_inventory_snapshot", "fact_shipments", "fact_web_sessions", "fact_marketing_spend"]:
 assert table in text
```

## Line-by-line explanation

- **Line 1:** `from pathlib import Path` — `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pathlib` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** `import yaml` — `import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `yaml` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 3:** blank line used only to separate logical sections for readability.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** `def test_job_notebook_paths_exist() -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_job_notebook_paths_exist` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `payload = yaml.safe_load(Path("databricks/jobs/job_definition.example.yml").read_text())` — `payload` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `yaml.safe_load` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"databricks/jobs/job_definition.example.yml"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `read_text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** `tasks = payload["resources"]["jobs"]["northstar_retail_daily"]["tasks"]` — `tasks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `payload` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"resources"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"jobs"` is a quoted literal; the quotation marks preserve spaces or mark text; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"northstar_retail_daily"` is a quoted literal; the quotation marks preserve spaces or mark text; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"tasks"` is a quoted literal; the quotation marks preserve spaces or mark text; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** `jobs_dir = Path("databricks/jobs")` — `jobs_dir` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"databricks/jobs"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 9:** `for task in tasks:` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `task` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tasks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `path = (jobs_dir / task["notebook_task"]["notebook_path"]).resolve()` — `path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; jobs\_dir is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; task is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "notebook\_task" is a quoted literal; the quotation marks preserve spaces or mark text; ][ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; "notebook\_path" is a quoted literal; the quotation marks preserve spaces or mark text; ) . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; resolve is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 11:** `assert path.exists(), f"Missing job notebook: {path}"` — `assert` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `path.exists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()`, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `f` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"Missing job notebook: {path}"` is a quoted literal; the quotation marks preserve spaces or mark text
- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** blank line used only to separate logical sections for readability.
- **Line 14:** `def test_all_gold_tables_are_in_dictionary() -> None:` — `def` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test_all_gold_tables_are_in_dictionary` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `()` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `->` is a command option or flag; its exact behavior is defined beside the command; `None` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** `text = Path("docs/data_dictionary.csv").read_text(encoding="utf-8")` — `text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Path` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"docs/data_dictionary.csv"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `)` . is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `read_text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `encoding` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"utf-8"` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** `for table in ["dim_date", "dim_customer", "dim_product", "dim_store", "dim_employee", "dim_supplier", "dim_promotion", "dim_channel", "dim_campaign", "fact_sales", "fact_returns", "fact_inventory_snapshot", "fact_shipments", "fact_web_sessions", "fact_marketing_spend"]:` — `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"dim_date"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"dim_customer"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"dim_product"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `"dim_store"` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 17:** assert table in text — assert is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; text is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

[Back to the Phase 19 lesson index](#)

## What your screen should look like now

---

- Every Phase 19 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

## Lesson 19.3 completion checklist

---

☐

.github/workflows/validate\_project.yml was created by me, saved, located correctly, and inspected — [STATUS]

☐

pyproject.toml was created by me, saved, located correctly, and inspected — [STATUS]

☐

scripts/validation/check\_required\_files.py was created by me, saved, located correctly, and inspected — [STATUS]

☐

scripts/validation/validate\_markdown\_links.py was created by me, saved, located correctly, and inspected — [STATUS]

☐

scripts/validation/validate\_notebooks.py was created by me, saved, located correctly, and inspected — [STATUS]

☐

scripts/validation/validate\_sql\_dialects.py was created by me, saved, located correctly, and inspected — [STATUS]

☐

tests/integration/test\_repository\_contract.py was created by me, saved, located correctly, and inspected — [STATUS]

## Safe stopping point

---

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 19.4](#) and recheck the root and branch.

## Lesson 19.4 — Build and validate required objects

---

### What your screen should look like before starting

---

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

### Action 19.4.G1 — Inspect the current state without changing it

---

#### Application:

VS Code

#### Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

#### Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

#### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

#### What you are about to do:

Inspect the current state without changing it. This is one bounded construction or inspection task.



**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## Action 19.4.G2 — Create the smallest required object or decision

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Create the smallest required object or decision. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

## Action 19.4.G3 — Run the phase-specific verification

---

**Application:**

VS Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run the phase-specific verification. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## Action 19.4.G4 — Demonstrate one safe failure or alternate-result branch

---

**Application:**

VS Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Demonstrate one safe failure or alternate-result branch. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## Action 19.4.G5 — Record evidence and prepare the downstream handoff

---

**Application:**

VS Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Record evidence and prepare the downstream handoff. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.

4. Continue only after verification.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 19 lesson index](#)

## What your screen should look like now

- The new Phase 19 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

## Lesson 19.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

## Safe stopping point

---

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 19.5](#).

## Lesson 19.5 — Verify, troubleshoot, commit, teach back, and stop safely

---

### What your screen should look like before starting

---

The Phase 19 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

### Design decision — Infrastructure-free CI

---

#### Question:

What should GitHub Actions validate without database or Databricks credentials?

#### Options considered:

- Run only local repository-contract tests.
- Connect CI to all external services.
- No CI.

#### Beginner path selected:

Run syntax, unit, fixture, secret, notebook, link, and dialect checks that require no external infrastructure.

#### Why this path was selected:

It provides repeatable feedback without storing database or workspace credentials.

#### Tradeoff:

It does not prove live external integrations.

#### What a company might do instead:

A company adds ephemeral environments, integration-test credentials, deployment approvals, and environment protections.

#### How to explain this in an interview:

My CI validates the repository contract safely; live platform execution remains separately evidenced rather than falsely simulated.

### If you see Result A

---

GitHub Actions runs every intended check and finishes green on the pushed commit. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

### If you see Result B

---

A workflow does not trigger, YAML is invalid, a test fails, or permissions/actions settings block the run. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

### If you see an error

---

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

### Phase-specific verification gate

---

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

## Point-of-failure troubleshooting record

---

### Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

### What it means in plain language:

[WRITE YOUR INTERPRETATION]

### Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

### What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

### First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

### How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

### Safest fix:

[ENTER THE NARROW FIX]

### Alternative fix when applicable:

[ENTER OR N/A]

### How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

### How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

## Screenshot checkpoint

---

**Application(s) that should be visible:** VS Code, PowerShell, GitHub repository Actions page, and workflow logs.

**Window/panel:** Show workflow YAML, successful Actions run, and one diagnosed/fixed toy failure.

**Sensitive information to hide:** Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

**Include:** The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

**Crop out:** Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

**Recommended filename:** docs/images/phase19\_build-github-actions-continuous-integration\_verified.png.

**Recommended repository folder:** docs/images/.

**README placement:** The section describing Phase 19's verified result; do not use the image before the result is actually verified.



**What it proves:** The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

## Git checkpoint after the verified Phase 19 changes

---

Do not use `git add` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

### Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

## Action 19.5.G1 — Verify the repository root before Git operations

---

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

### Does it change anything?

This is intended to inspect or validate state rather than change it.

### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

### How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.5.G2 — Verify the active Git branch

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git branch --show-current
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.5.G3 — Inspect all current Git changes

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git status --short
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

**Interpret `git status --short`**

- `??` means untracked: Git sees a new path but it is not staged.
- `M` means modified in the working tree but not staged.
- `M` means the staged version differs from the last commit.
- `MM` means a staged version exists and the file has additional unstaged edits.
- `A` means staged as a new file.
- `D` means deletion; stop and investigate unless deletion is deliberate.
- `R` means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

## Action 19.5.G4 — Inspect the unstaged diff for .github/workflows/validate\_project.yml

---

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
git diff -- ".github/workflows/validate_project.yml"
```

### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git diff -- ".github/workflows/validate_project.yml"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `".github/workflows/validate_project.yml"` is a path, relative location, branch-style name, or file argument; verify every character before running it

### Does it change anything?

This is intended to inspect or validate state rather than change it.

### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

### How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.5.G5 — Stage only the reviewed file `.github/workflows/validate_project.yml`

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.

3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
git add -- ".github/workflows/validate_project.yml"
```

#### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `git add -- ".github/workflows/validate_project.yml"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `".github/workflows/validate_project.yml"` is a path, relative location, branch-style name, or file argument; verify every character before running it

#### Does it change anything?

This can change files, Git state, packages, database objects, or data.

#### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.



**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

## Action 19.5.G6 — Inspect the staged diff

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git diff --cached
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.5.G7 — Create the Phase 19 commit

---

**Application:**

Web browser on GitHub

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Web browser on GitHub** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
git commit -m "ci: validate repository contracts in github actions"
```

#### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `git commit -m "ci: validate repository contracts in github actions"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"ci: validate repository contracts in github actions"` is a quoted literal; the quotation marks preserve spaces or mark text

#### Does it change anything?

This can change files, Git state, packages, database objects, or data.

#### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.5.G8 — Verify the new commit

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git log -1 --oneline --decorate
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Action 19.5.G9 — Push the feature branch and set upstream

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

#### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

#### Does it change anything?

This can change files, Git state, packages, database objects, or data.

#### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

### How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 19 lesson index](#)

## Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message `ci: validate repository contracts in github actions` and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

## Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

## Explain it in your own words

---

Without copying the model answer, explain:

1. What you created or configured in Phase 19.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

## My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

## OPTIONAL REFERENCE ANSWER — PHASE 19 TEACH-BACK MODEL

---

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

“Phase 19 created the verified boundary described by **Build GitHub Actions continuous integration**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed.”

## Mini quiz

---

1. Multiple choice: Why does Phase 19 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
3. Fill in the blank: The statement that defines what one row represents is the table or entity \_\_\_\_\_.

4. What command would you use first to confirm the PowerShell current folder before a repository command?
  5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
  6. Design choice: Explain one tradeoff of Track A and one production alternative.
  7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
  8. Ownership: Which status may you select only after personally performing and inspecting the verification?
- 

## Mini quiz answer key and explanations

---

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

## My Phase 19 Learning Journal

---

### Date completed:

[ENTER DATE]

### What I created myself:

[WRITE YOUR ANSWER]

### Commands I ran:

[WRITE YOUR ANSWER]

### What confused me:

[WRITE YOUR ANSWER]

### What error occurred:

[WRITE YOUR ANSWER]

### What the error meant:

[WRITE YOUR ANSWER]

### How I fixed it:

[WRITE YOUR ANSWER]

### What I learned:

[WRITE YOUR ANSWER]

### What I can now explain without help:

[WRITE YOUR ANSWER]

### What I still need to practice:

[WRITE YOUR ANSWER]

## What your screen should look like now

---

- The phase output and final validation are saved or visible.



- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

## Lesson 19.5 completion checklist

---

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

## Safe stopping point

---

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

## How this phase connects to the next phase

---

Phase 20 documents only verified work and links each claim to evidence.

## Phase 19 completion checklist

---

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [ ] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

# Phase 20 — Build the README, evidence package, and portfolio presentation

[Back to the master table of contents](#)

**Track:** Track A required

**Status when first opened:** PLANNED

**Completion rule:** The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

## Why this phase happens now

---

**Earlier work this phase depends on:** Verified evidence, screenshots, metrics, and limitations.

**What would go wrong if it happened earlier:** You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

**Later work that depends on it:** Phase 21 turns the real project into truthful interview explanations.

**Decision being made:** Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

**Alternatives:** A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

**Beginner path selected:** Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

**Reason for the order:** Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

## Phase 20 lesson index

---

1. [Lesson 20.1 — Understand the phase and starting screen](#)
2. [Lesson 20.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 20.3 — Create and understand the phase files](#)
4. [Lesson 20.4 — Build and validate required objects](#)
5. [Lesson 20.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

## Lesson 20.1 — Understand the phase and starting screen

---

### What your screen should look like before starting

---

- **Application open:** VS Code Markdown Preview, GitHub repository page, File Explorer, and image editor only for redaction/cropping.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 20 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 19 must be verified when this phase has a prerequisite.

## Three-level explanation of the phase

---

### Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

### Level 2 — Northstar Retail project explanation

Phase 20 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

### Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

## Applications to open

---

VS Code Markdown Preview, GitHub repository page, File Explorer, and image editor only for redaction/cropping. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

## Phase status gate

---

☐

I changed this phase from [ ] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

## What your screen should look like now

---

- The Phase 20 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 20 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 20 master-tracker row is marked [-] In progress only after you personally updated it.

## Lesson 20.1 completion checklist

---

☐

I read why Phase 20 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

## Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 20.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 20 lesson index](#)

## Lesson 20.2 — Perform the notebook-guided actions one at a time

### What your screen should look like before starting

The Phase 20 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

### Retained phase lesson, exercises, rules, and expected results

**Track:** Track A required

**Ownership status at start:** Workbook instructions exist; your execution is not yet verified.

**Evidence rule:** Replace placeholders only after the verification steps pass.

### 1. What you are building

A professional README, architecture/data-flow/star-schema/lineage diagrams, data dictionary, source-to-target mapping, test plan, troubleshooting guide, runbook, manifest, screenshot checklist, demo script, resume bullets, LinkedIn draft, limitations, future improvements, and verified evidence map.

### 2. Why it matters

Documentation is how another engineer reviews, runs, trusts, and discusses the system. A strong portfolio explains decisions and evidence without pretending reference code or unexecuted services are completed achievements.

### 3. Words to know

Term	Plain-language definition
README	The repository landing page and primary project overview.
Manifest	An inventory of files, purposes, and learning phases.
Evidence map	A table connecting a claim to a test, screenshot, log, query, or run ID.
Architecture diagram	A system-level picture of components and data movement.
Lineage diagram	A picture of how data fields/tables flow from source to consumption.
Limitation	A known boundary or weakness described honestly.
Future improvement	A justified next step, not a claim that it already exists.
Portfolio narrative	Business problem, design, implementation, verification, learning, and tradeoffs told as one story.
GitHub topic	A repository label that helps discovery, such as data-engineering or databricks.
Screenshot checkpoint	A required image that proves an action performed by the learner.

### 4. What must already be completed

All required implementation phases have a current status, automated tests are green, and any unexecuted external platform step is marked Not started/In progress rather than described as complete.

## 5. Applications to open

Application	How to open it	Why it is open
VS Code	Open README, docs, diagrams, manifest, and checklist.	Edit and preview documentation.
Windows PowerShell	Open in repository root.	Validate links, required files, placeholders, and clean Git state.
GitHub browser	Open the branch/PR and repository preview.	Review rendering and configure description/topics.
Image viewer	Open SVG and your PNG screenshots.	Check legibility and privacy.

## 6. Exact click-by-click steps

**Beginner rhythm:** Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

## 7. Exact commands, one code block at a time

## 8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run `Get-Location`; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

## TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

### Line-by-line code explanation

Line or action	Meaning	Why it belongs here
<code>python -m scripts.validation.check_required_files</code>	Checks portfolio-required paths, documentation links, and notebook structure.	A polished narrative with missing files or broken navigation is not publishable.
<code>Get-ChildItem .\docs\images -File   Sort-Object Name   Select-Object Name, Length</code>	Lists diagram and screenshot files with sizes.	Zero-byte/missing evidence files can otherwise be overlooked.
<code>Select-String -Path README.md,docs*.md,powerbi*.md -Pattern '[ADD YOUR TODO(USER) MANUAL-REPLACE-ME]'   Select-Object Path,LineNumber,Line</code>	Finds ownership and verification placeholders.	Every placeholder must remain intentionally unverified or be replaced with evidence.
<code>git status</code>	Shows remaining changes, whitespace errors, and scope.	The final documentation commit should be deliberate and reviewable.

### Claim-to-evidence rule

Write “reference material includes” for reference examples. Write “I verified” only after your checklist points to a run ID, batch ID, query result, test output, or screenshot. Write future tense for unbuilt extensions. The README reference answer deliberately uses placeholders where your own execution must supply proof.

### Diagram formats

Mermaid files are editable text; DOT files provide a second text representation; SVG files are rendered for GitHub/browser viewing. Text alternatives in the documentation explain each diagram when rendering is unavailable.

## MY TASK

Audit and customize the entire portfolio narrative, replace only evidence-backed placeholders, add your screenshots and lessons, verify every link/diagram/file, configure GitHub description/topics, and publish through a reviewed PR.

# STARTER SKELETON

---

```
Verified result: TODO(USER)

- **Claim:** TODO(USER)
- **Business meaning:** TODO(USER)
- **Evidence file/run/query:** TODO(USER)
- **Batch ID and data-through date:** TODO(USER)
- **Validation method:** TODO(USER)
- **Limitation:** TODO(USER)
- **What I personally did:** TODO(USER)
- **What the workbook taught or provided as reference:** TODO(USER)
```

**Acceptance criteria:** The repository tells a coherent business-to-technical story; every referenced file exists; all diagrams and links render; screenshots are private and evidentiary; claims map to proof; limitations and TODOs are explicit; automated checks and documentation PR pass.

## HINT 1 — CONCEPT

---

Review the repository in the order a stranger sees it: title, problem, architecture, evidence, run guide, model, dashboard, tests, limitations.

## HINT 2 — PSEUDOCODE

---

For every achievement sentence ask: what exact file/run/query/screenshot proves it and did I personally execute it?

## HINT 3 — NEAR-COMPLETE PATTERN

---

Use the manifest/checklist/evidence template, search all placeholders, update only those with evidence, run validators, then review the GitHub rendering from a clean browser view.

## 13. Expected files or output

---

- Professional root README with truthful status.
- Complete technical documentation and rendered diagrams.
- the file-responsibility index and final definition-of-done sections in this workbook.
- Verified screenshots and evidence notes.
- Customized resume, LinkedIn, demo, and interview materials.
- Repository description and topics.
- Passing documentation/CI checks.

## 14. How to verify

---

- Every link and referenced path resolves.
- Every diagram matches actual object/task names.
- Every completed claim has evidence.
- No fabricated row count, runtime, KPI, screenshot, or performance statement remains.
- README commands match actual paths and Windows syntax.
- Current status distinguishes planned, created, run, verified, and optional.
- Public screenshots/data contain no real secret or unnecessary identity detail.
- GitHub preview is readable on desktop without local-only links.

## 15. Automated tests

---

- Required-file, link, notebook, SQL dialect, secret, lint, and pytest suite.
- Manifest-to-files comparison.
- Placeholder/claim review.
- Diagram open/render inspection.
- GitHub preview and clean-clone navigation review.
- Interviewer two-minute findability test.

## 16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
README image is broken	Relative path/case differs or SVG was not committed.	Open the raw link relative to README; check exact case with <code>Get-ChildItem</code> .	Correct path/case and commit the image.	GitHub preview renders it.
Manifest references missing file	File was renamed/deleted after manifest generation.	Run <code>required-file/manifest</code> comparison and inspect Git history.	Restore/rename/update the reference and rerun all links.	Manifest and filesystem match.
README claims completed Databricks run without evidence	Template language was not replaced carefully.	Search achievement verbs and check evidence map.	Change to planned/future/status wording or add genuine run evidence.	Every claim is defensible.
GitHub cannot render a huge notebook	Notebook output or embedded data is too large.	Check file size and cell outputs.	Clear unnecessary outputs, keep selected verified samples, and link HTML.	Notebook preview or HTML opens quickly.
Screenshot exposes personal details	Capture included workspace/account/server/browser data.	Zoom and inspect every corner/metadata.	Crop/redact/retake; rotate any exposed secret.	Privacy checklist passes.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

## 17. Explain it in your own words

1. What is the difference between a reference asset and an achievement claim?
2. How does a limitation strengthen rather than weaken a portfolio?
3. What should an interviewer find in the first two minutes?
4. Why are both editable and rendered diagram formats included?

Write your answers in [the learning-journal fields in this phase](the learning-journal fields in this phase). Do not copy the wording above.

## 18. Mini quiz

1. Which document inventories every created file?
2. When may a verified placeholder be replaced with a number?
3. Name two required limitations of Track A.

Record your answers before opening the answer key.

## 19. Screenshot checkpoint

Capture repository landing page, architecture/model diagrams, tests/Actions, source databases, Databricks Job graph, Power BI model/pages, and evidence map according to the [screenshot checkpoints in this workbook](#).

Save the image under `docs/images/` with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

## 20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
docs: complete truthful northstar portfolio package
```

Push the feature branch only after tests pass.

## 21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

README narrative reviewed — [STATUS]

☐

All diagrams/links render — [STATUS]

☐

Manifest and checklist match files — [STATUS]

☐

Claims have evidence — [STATUS]

☐

Limitations/future work honest — [STATUS]

☐

Screenshots complete and private — [STATUS]

☐

Career/demo drafts customized — [STATUS]

☐

Portfolio PR and checks pass — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: [the optional reference-answer section later in this workbook#phase-20](the optional reference-answer section later in this workbook#phase-20). The quiz answer is in [the answer-key section immediately after the quiz in this workbook#phase-20](the answer-key section immediately after the quiz in this workbook#phase-20).

---

**Next phase:** Phase 21 prepares you to explain, defend, troubleshoot, and teach every design choice in interviews.

## Required action cards

---

### Action 20.2.1 — Open root README.md in VS Code Preview. Confirm the first screen states the business problem, local-first ar

---

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Open root README.md in VS Code Preview. Confirm the first screen states the business problem, local-first ar. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open root README.md in VS Code Preview. Confirm the first screen states the business problem, local-first architecture, current verification status, and how to start.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.



## What each part means:

### Step 1

#### Action

Open root README.md in VS Code Preview. Confirm the first screen states the business problem, local-first architecture, current verification status, and how to start.

#### Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

#### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

#### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

#### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

#### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

#### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

#### Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

#### Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

#### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

#### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

## Action 20.2.2 — Review all Mermaid sources and rendered SVGs under docs/images/. Verify arrows, table names, task names, and

#### Application:

Visual Studio Code

#### Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Review all Mermaid sources and rendered SVGs under docs/images/. Verify arrows, table names, task names, and. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Review all Mermaid sources and rendered SVGs under docs/images/. Verify arrows, table names, task names, and Track A/Track B boundaries match the actual files.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 2****Action**

Review all Mermaid sources and rendered SVGs under docs/images/. Verify arrows, table names, task names, and Track A/Track B boundaries match the actual files.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

## Action 20.2.3 — Open the file-responsibility index in this workbook and confirm every listed file exists. Use it to locate eac

---

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Open the file-responsibility index in this workbook and confirm every listed file exists. Use it to locate eac. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open the file-responsibility index in this workbook and confirm every listed file exists. Use it to locate each phase artifact rather than duplicating explanations randomly.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

### Step 3

#### Action

Open the file-responsibility index in this workbook and confirm every listed file exists. Use it to locate each phase artifact rather than duplicating explanations randomly.

#### Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

#### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

#### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

#### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

#### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

#### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

#### Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

#### Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

#### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

#### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

## Action 20.2.4 — Review docs/data\_dictionary.md, source-to-target mapping, KPI definitions, lineage, testing strategy, runboo

#### Application:

Visual Studio Code

#### Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

#### Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Review docs/data\_dictionary.md, source-to-target mapping, KPI definitions, lineage, testing strategy, runboo. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Review docs/data\_dictionary.md, source-to-target mapping, KPI definitions, lineage, testing strategy, runbook, risk register, assumption log, security/cost, and troubleshooting. Resolve contradictions with the code.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 4****Action**

Review docs/data\_dictionary.md, source-to-target mapping, KPI definitions, lineage, testing strategy, runbook, risk register, assumption log, security/cost, and troubleshooting. Resolve contradictions with the code.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

## Action 20.2.5 — Complete the global screenshot checklist. For every image, state what it proves, run/batch/date context, and w

---

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Complete the global screenshot checklist. For every image, state what it proves, run/batch/date context, and w. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Complete the global screenshot checklist. For every image, state what it proves, run/batch/date context, and what sensitive content was checked.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

### Step 5

**Action**

Complete the global screenshot checklist. For every image, state what it proves, run/batch/date context, and what sensitive content was checked.

### Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

### Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

### Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

## Action 20.2.6 — Replace only verified placeholders such as [ADD YOUR VERIFIED ROW COUNT]. Leave unverified placeholders visi

### Application:

Visual Studio Code

### Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

### Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Replace only verified placeholders such as [ADD YOUR VERIFIED ROW COUNT]. Leave unverified placeholders visible. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Replace only verified placeholders such as [ADD YOUR VERIFIED ROW COUNT]. Leave unverified placeholders visible or remove the unsupported claim; never guess.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 6****Action**

Replace only verified placeholders such as [ADD YOUR VERIFIED ROW COUNT]. Leave unverified placeholders visible or remove the unsupported claim; never guess.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.



**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

## Action 20.2.7 — Add a section explaining manual Track A upload/download, local services, synthetic data, free-tier quotas, bat

---

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Add a section explaining manual Track A upload/download, local services, synthetic data, free-tier quotas, bat. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Add a **Limitations** section explaining manual Track A upload/download, local services, synthetic data, free-tier quotas, batch-only scope, and learner-owned TODO transformations.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

### Step 7

**Action**

Add a **Limitations** section explaining manual Track A upload/download, local services, synthetic data, free-tier quotas, batch-only scope, and learner-owned TODO transformations.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

## Action 20.2.8 — Add as conditional extensions, with reason and verification needed. Do not describe an extension as present be

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Add as conditional extensions, with reason and verification needed. Do not describe an extension as present be. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Add **Future improvements** as conditional extensions, with reason and verification needed. Do not describe an extension as present because a documentation link exists.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 8****Action**

Add **Future improvements** as conditional extensions, with reason and verification needed. Do not describe an extension as present because a documentation link exists.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

## Action 20.2.9 — Use docs/demo\_video\_script.md to record a 5–10 minute demo after execution. Open with business problem, then

---

**Application:**

Databricks workspace in a web browser

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Use docs/demo\_video\_script.md to record a 5–10 minute demo after execution. Open with business problem, then. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Use docs/demo\_video\_script.md to record a 5–10 minute demo after execution. Open with business problem, then architecture, one source, one quality issue, one Job run, model, dashboard, tests, and lessons.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

### Step 9

**Action**

Use docs/demo\_video\_script.md to record a 5–10 minute demo after execution. Open with business problem, then architecture, one source, one quality issue, one Job run, model, dashboard, tests, and lessons.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

## Action 20.2.10 — Customize resume/LinkedIn drafts only with evidence you can defend. Replace bracketed placeholders after check

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Customize resume/LinkedIn drafts only with evidence you can defend. Replace bracketed placeholders after check. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Customize resume/LinkedIn drafts only with evidence you can defend. Replace bracketed placeholders after checking your own row counts, dates, test count, and screenshots.
4. Pause while the application finishes the operation; do not click the same control twice.

5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 10****Action**

Customize resume/LinkedIn drafts only with evidence you can defend. Replace bracketed placeholders after checking your own row counts, dates, test count, and screenshots.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

## Action 20.2.11 — In GitHub repository settings, add a clear description, website only if appropriate, and topics such as `data-

---

### Application:

Databricks workspace in a web browser

### Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

### Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

In GitHub repository settings, add a clear description, website only if appropriate, and topics such as `data-`. This is one bounded construction or inspection task.

### Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

### Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: In GitHub repository settings, add a clear description, website only if appropriate, and topics such as data-engineering, data-warehouse, databricks, pyspark, sql-server, postgresql, delta-lake, power-bi, star-schema, portfolio-project.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

### Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

### Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

### What each part means:

## Step 11

### Action

In GitHub repository settings, add a clear description, website only if appropriate, and topics such as data-engineering, data-warehouse, databricks, pyspark, sql-server, postgresql, delta-lake, power-bi, star-schema, portfolio-project.

### Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

## Action 20.2.12 — Run all repository validations. Open the HTML showcase notebook and GitHub notebook preview to check rendering

**Application:**

Databricks workspace in a web browser

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run all repository validations. Open the HTML showcase notebook and GitHub notebook preview to check rendering. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run all repository validations. Open the HTML showcase notebook and GitHub notebook preview to check rendering and relative links.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.



**Where to enter it:**

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 12****Action**

Run all repository validations. Open the HTML showcase notebook and GitHub notebook preview to check rendering and relative links.

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

## Action 20.2.13 — Review the repository as an interviewer: can you find the problem, architecture, data model, run order, eviden

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Review the repository as an interviewer: can you find the problem, architecture, data model, run order, eviden. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Review the repository as an interviewer: can you find the problem, architecture, data model, run order, evidence, limitations, and what Kirolos personally completed within two minutes?
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:****Step 13****Action**

Review the repository as an interviewer: can you find the problem, architecture, data model, run order, evidence, limitations, and what Kirolos personally completed within two minutes?

**Before continuing**☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

**What should happen:**

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**Expected example:**

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**What may look different:**

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**How to verify success:**

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**Do not continue unless:**

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

**Common mistake:**

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**Exact recovery:**

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**What you just accomplished:**

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**How this connects to the project:**

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

## Action 20.2.14 — Commit documentation on a branch, open a PR, review rendered diffs and screenshots, then merge after checks pass

---

**Application:**

Visual Studio Code

**Window or panel:**

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Commit documentation on a branch, open a PR, review rendered diffs and screenshots, then merge after checks pass. This is one bounded construction or inspection task.

**Why this action comes now:**

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Commit documentation on a branch, open a PR, review rendered diffs and screenshots, then merge after checks pass.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

**Command or code:**

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**Where to enter it:**

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**What each part means:**

## Step 14

### Action

Commit documentation on a branch, open a PR, review rendered diffs and screenshots, then merge after checks pass.

### Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

### What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

### Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

### What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

### How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

### Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

### Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

### Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

### What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

### How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

## Action 20.2.C1 — Windows PowerShell

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

Windows PowerShell

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Powershell):**

```
python -m scripts.validation.check_required_files
```

**Where to enter it:**

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `python -m scripts.validation.check_required_files` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.check_required_files` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

## Action 20.2.C2 — Windows PowerShell

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Powershell):**

```
python -m scripts.validation.validate_markdown_links
```

**Where to enter it:**

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `python -m scripts.validation.validate_markdown_links` — python invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.validate_markdown_links` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

## Action 20.2.C3 — Windows PowerShell

**Application:**

Databricks workspace in a web browser

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Powershell):**

```
python -m scripts.validation.validate_notebooks
```

**Where to enter it:**

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `python -m scripts.validation.validate_notebooks` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.validate_notebooks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.



**Do not continue unless:**

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

## Action 20.2.C4 — Windows PowerShell

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Powershell):**

```
Get-ChildItem .\docs\images -File | Sort-Object Name | Select-Object Name, Length
```

**Where to enter it:**

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `Get-ChildItem .\docs\images -File | Sort-Object Name | Select-Object Name, Length` — `Get-ChildItem` lists files and folders; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `images` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-File` is a command option or flag; its exact behavior is defined beside the command; `Sort-Object` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Select-Object` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Length` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

## Action 20.2.C5 — Windows PowerShell

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

Windows PowerShell

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (Powershell):

```
Select-String -Path README.md,docs*.md,powerbi*.md -Pattern '[ADD YOUR|TODO\((USER\)|MANUAL-REPLACE-ME' | Select-Object Path,LineNumber,Line
```

### Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `Select-String -Path README.md,docs\*.md,powerbi\*.md -Pattern '[ADD YOUR|TODO\((USER\)|MANUAL-REPLACE-ME' | Select-Object Path,LineNumber,Line` — `Select-String` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `README.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `powerbi` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Pattern` is a command option or flag; its exact behavior is defined beside the command; remaining punctuation and identifiers continue the same statement.

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

## Action 20.2.C6 — Windows PowerShell

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (Powershell):

```
git status
```

### Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git status` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work

### Does it change anything?

This is intended to inspect or validate state rather than change it.

### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm

the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

## Action 20.2.C7 — Windows PowerShell

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (Powershell):**

```
git diff --check
```

**Where to enter it:**

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git diff --check` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--check` is a command option or flag; its exact behavior is defined beside the command

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

## Action 20.2.C8 — Windows PowerShell

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

Windows PowerShell

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained Powershell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (Powershell):

```
git diff --stat
```

#### Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `git diff --stat` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--stat` is a command option or flag; its exact behavior is defined beside the command

#### Does it change anything?

This is intended to inspect or validate state rather than change it.

#### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.



### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

### How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

## What your screen should look like now

---

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

## Lesson 20.2 completion checklist

---

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

## Safe stopping point

---

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 20 starting-screen checklist and re-verify the active context.

## Lesson 20.3 — Create and understand the phase files

---

## What your screen should look like before starting

---

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 20 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

### Folder introduction — docs/

**Exact folder name:** docs

**Exact parent:** repository root

**Why it is needed now:** Phase 20 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

**What will eventually be inside:** Only files whose purpose matches this folder's name.

**Why another folder is not appropriate:** Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

**Naming rules:** Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

### VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: docs.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

### Action 20.D1.1 — Create the folder docs

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal at the repository root

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop
```

**Where to enter it:**

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\docs" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This can change files, Git state, packages, database objects, or data.

**Is it safe to rerun?**

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

**Folder-creation decision branches****If the folder is created**

Run `Test-Path -LiteralPath ".\docs" -PathType Container` and require `True`.

**If PowerShell says the folder already exists**

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\docs" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

**If the parent path does not exist**

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

**Verify the folder**

1. Run `Test-Path ".\docs" -PathType Container`.
2. Require the output `True` before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

**File introduction — README.md**

**Exact filename:** `README.md`

**Filename extension:** `.md`. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

**Exact parent folder:** `.` relative to the repository root.

**Why this file belongs there:** Files with the same responsibility are grouped so a reviewer can find configuration, source

SQL, transformations, tests, documentation, and report assets without guessing.

**What the file will contain:** project documentation for Phase 20.

**Later dependencies:** Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

**Should Git track it?** Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

**Could it contain secrets?** It should not contain live passwords, tokens, connection strings, or account identifiers.

**What happens if it is missing:** The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

### VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in . one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: README.md.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled README.md and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

### PowerShell path

## Action 20.F1.1 — Create the file README.md from PowerShell

---

#### Application:

Windows PowerShell or the VS Code PowerShell terminal

#### Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

#### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

#### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

#### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

#### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

#### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\README.md" -ErrorAction Stop
```

**Where to enter it:**

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `New-Item -ItemType File -Path ".\README.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\README.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

**Does it change anything?**

This can change files, Git state, packages, database objects, or data.

**Is it safe to rerun?**

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

## File-creation decision branches

### If the command creates the file

Run `Test-Path -LiteralPath ".\README.md" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

### If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "README.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

### If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

## File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

## File verification

1. In the VS Code Explorer, confirm the file appears under `..`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `README.md`.
3. In PowerShell at the repository root, run `Test-Path ".\README.md"` and require `True`.
4. Run `git status --short -- "README.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

## Optional reference answer for README.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

**Language:** Markdown

**Exact file:** `README.md`

**Purpose:** project documentation.

**Inputs:** The requirements, configuration, source data, or upstream objects described in Phase 20.

**Expected outputs:** The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
Northstar Retail End-to-End Data Platform

> A beginner-owned, evidence-driven portfolio project for learning SQL Server, PostgreSQL, Python, Databricks, Delta Lake, dimensional modeling, Lakeflow Jobs, SQL Server warehousing, and Power BI.

Original repository: https://github.com/Losboules/SQL-Data-Warehouse-Project
Reference design assembled: 2026-08-16
Status: This is an instructional reference. Kirolos must manually create, run, test, and verify each component before claiming it as completed.

Business problem

Northstar Retail is a fictional omnichannel retailer. Leadership receives inconsistent answers because ERP sales/inventory, website behavior, marketing, shipments, returns, and supplier updates live in separate systems. This project creates a governed daily path from those sources to trusted dimensions, facts, KPIs, and a Power BI report.

Architecture

(docs/images/overall_architecture.svg)

Track A – default local-first path

1. Generate fictional source files with Python.
2. Load ERP data into local SQL Server and digital data into local PostgreSQL.
3. Extract timestamped Parquet/file batches with manifests and checksums.
4. Manually upload one batch folder to a Databricks Unity Catalog volume.
5. Run parameterized Bronze, Silver, Gold, quality, and publication tasks.
6. Download the validated Gold export.
7. Load it transactionally into local `NorthstarRetail_DW`.
8. Import the star schema into Power BI and validate every measure against SQL.
```

Databricks does **not** connect to the local machine's `localhost` in Track A. Manual upload/download steps are explicit learning boundaries.

### ### Track B – optional cloud-connected path

Use network-accessible managed SQL Server/Azure SQL and PostgreSQL, supported JDBC connections, controlled firewall/private networking, secret-managed identities, and cloud storage. This can incur database, compute, storage, egress, networking, and BI costs.

### ## Technology stack

```
| Area | Technology |
|---|---|
| Version control | Git, GitHub, GitHub Actions |
| Local development | Windows PowerShell, VS Code, Python 3.13.x |
| ERP source and serving warehouse | SQL Server + SSMS |
| Digital source | PostgreSQL + pgAdmin/psql |
| Lakehouse | Databricks, PySpark, Spark SQL, Delta Lake, Unity Catalog volumes |
| Orchestration | Databricks Lakeflow Jobs |
| Modeling | Bronze/Silver/Gold, star schema, SCD Type 1/2, unknown members |
| Semantic/consumption | Power BI and Databricks SQL views |
| Trust | pytest, SQL assertions, quality tables, manifests, checksums, reconciliation |
```

### ## Data sources

- SQL Server ERP: customers, addresses, products, categories, suppliers, stores, employees, orders, items, payments, shipments, inventory movements and snapshots.
- PostgreSQL digital: users, sessions, events, campaigns, touchpoints, and marketing spend.
- Files: returns CSV, supplier cost updates CSV, promotion calendar CSV, nested shipment tracking JSONL.

See [``docs/data_dictionary.md``](docs/data\_dictionary.md) and [``docs/source_to_target_mapping.md``](docs/source\_to\_target\_mapping.md).

### ## Pipeline flow

`SQL Server ERP + PostgreSQL digital + CSV/JSON feeds → timestamped local extraction batch → Databricks Bronze → Silver → Gold → blocking quality gate → Gold export → NorthstarRetail\_DW → Power BI / analyst SQL.`

Track A deliberately uses manual upload and download boundaries; Track B replaces those boundaries with network-accessible sources and secured cloud storage.

### ## Medallion layers

- **Bronze**: raw values, source file/system, ingestion timestamp, batch ID, and record hash.
- **Silver**: typed, trimmed, standardized, deduplicated, conformed, quality-flagged, and quarantined records.
- **Gold**: nine dimensions, six facts, SCD2 history for customer/product, unknown members, and governed aggregates/views.

### ## Data model

![Gold star schema](docs/images/gold\_star\_schema.svg)

Every fact's grain is documented in the workbook and data dictionary. `fact\_sales` is one valid order item. Inventory is a periodic snapshot and must not be summed across dates without a deliberate calculation.

### ## Data-quality strategy

The generator intentionally creates controlled defects. Silver handles them with explicit rules and quarantine. Required tests cover not-null, uniqueness, accepted values, referential integrity, date/numeric ranges, freshness, schema drift, duplicate batches, source/layer reconciliation, SCD behavior, and revenue arithmetic. Publication depends on a passing quality gate.

### ## Orchestration

![Lakeflow Jobs dependency](docs/images/jobs\_dependency.svg)

The version-controlled example is [``databricks/jobs/job_definition.example.yml``](databricks/jobs/job\_definition.example.yml). Every task path exists in `databricks/notebooks`.

### ## Semantic model and dashboard

Power BI uses single-direction dimension-to-fact relationships, a marked date table, explicit measures, display folders, descriptions, and validation SQL. Report pages cover executive, sales, products, stores, customers, marketing, inventory, shipping/returns, and data quality. No `.pbix` or result screenshot is fabricated.

### ## How to run the project safely

#### ### 1. Clone the existing repository

Run in **Windows PowerShell**, from the parent folder where you want the clone:

```
```powershell
git clone https://github.com/Losboules/SQL-Data-Warehouse-Project.git
```

```
Set-Location .\SQL-Data-Warehouse-Project
```

```
git switch -c feature/northstar-platform
```

2. Create Python environment

Run in **Windows PowerShell**, from the repository root:

```
py -3.13 -m venv .venv
```

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

```
.\.venv\Scripts\Activate.ps1
```

```
python -m pip install --upgrade pip
```

```
python -m pip install -r requirements-dev.txt
```

3. Generate and validate quick data

```
python -m scripts.data_generation.generate_northstar_data --scale quick --seed 20260815 --output-dir datasets\sample
```

```
python -m scripts.validation.validate_generated_data --input-dir datasets\sample
```

4. Run local checks

```
python -m scripts.validation.check_required_files
python -m scripts.validation.validate_sql_dialects
python -m scripts.validation.validate_notebooks
python -m scripts.validation.scan_secrets
python -m scripts.validation.validate_markdown_links
python -m pytest -q
```

Follow the single manual-build workbook during construction. Keep this README limited to repository facts and evidence you have personally verified.

Repository structure

```
SQL-Data-Warehouse-Project/
├─ README.md, LICENSE, configuration and dependency files
├─ datasets/           # small sample, ignored raw batches, issue fixtures
├─ docs/              # workbook, phase chapters, diagrams, contracts, portfolio docs
├─ notebooks/         # GitHub showcase notebook and HTML
├─ scripts/           # setup, generation, extraction, loading, validation, utilities
├─ sql/              # dialect-separated SQL Server, PostgreSQL, Databricks SQL
├─ databricks/        # source notebooks and job definition
├─ powerbi/          # model, DAX, page requirements, screenshot checklist
├─ tests/            # unit, integration, data-quality, reconciliation
└─ .github/          # Actions, issue, and pull-request templates
```

During construction, use the file-responsibility index inside the manual-build workbook; do not create a separate file-manifest document.

Screenshots

Replace these placeholders with your own verified evidence:

- [ADD YOUR VERIFIED ROW COUNT]
- [ADD PHASE 4 SOURCE DATABASE SCREENSHOT]
- [ADD PHASE 14 JOB RUN SCREENSHOT]
- [ADD PHASE 15 WAREHOUSE RECONCILIATION SCREENSHOT]
- [ADD PHASE 16 POWER BI MODEL SCREENSHOT]
- [ADD PHASE 17 DASHBOARD SCREENSHOT]

Limitations

- Track A has manual cloud handoffs and daily batch latency.
- The sample and quick generator are reference material; record your own local test result before claiming verification. External-platform execution is not claimed.

- Beginner reporting assumes USD and simplified converting-session attribution.
- The Gold fact notebook reference includes a fact_sales teaching pattern and TODO(USER) ownership work for the remaining facts; optional answers appear only inside the manual-build workbook.
- The project is educational, not production certification.

Future improvements

See [docs/optional_extensions.md](#). Choose one only after the required checklist is personally verified.

What I learned

Replace this section after completion. Suggested prompts: How did grain prevent a mistake? Which failure did the quality gate catch? Why was SCD2 needed? What made a rerun safe? How did SQL and DAX validation differ?

[WRITE YOUR OWN VERIFIED REFLECTION]

Contact

Kirolos Boules

LinkedIn: [ADD LINK]

Email or portfolio: [ADD SAFE PUBLIC CONTACT]

License

MIT. All sample identities and business entities are fictional.

Line-by-line explanation

```
- **Line 1:** `# Northstar Retail End-to-End Data Platform` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** `> A beginner-owned, evidence-driven portfolio project for learning SQL Server, PostgreSQL, Python, Databricks, Delta Lake, dimensional modeling, Lakeflow Jobs, SQL Server warehousing, and Power BI.` - `>` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `A` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `beginner-owned` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `evidence-driven` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `portfolio` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `project` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `for` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `learning` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Server` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** `**Original repository:** [https://github.com/Losboules/SQL-Data-Warehouse-Project](https://github.com/Losboules/SQL-Data-Warehouse-Project)` - `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Original` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `repository` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `https` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `://` is a path, relative location, branch-style name, or file argument; verify every character before running it; `github.com` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Losboules` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `SQL-Data-Warehouse-Project` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 6:** `**Reference design assembled:** 2026-08-16` - `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Reference` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `design` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `assembled` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `2026-08-16` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.
- **Line 7:** `**Status:** This is an instructional reference. Kirolos must manually create, run, test, and verify each component before claiming it as completed.` - `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Status` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `This` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `an` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `instructional` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `reference` is a
```

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Kirolos` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `must` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manually` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** `## Business problem` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 10:** blank line used only to separate logical sections for readability.
- **Line 11:** `Northstar Retail is a fictional omnichannel retailer. Leadership receives inconsistent answers because ERP sales/inventory, website behavior, marketing, shipments, returns, and supplier updates live in separate systems. This project creates a governed daily path from those sources to trusted dimensions, facts, KPIs, and a Power BI report.` – `Northstar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Retail` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fictional` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `omnichannel` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `retailer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Leadership` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `receives` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `inconsistent` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `answers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** `## Architecture` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 14:** blank line used only to separate logical sections for readability.
- **Line 15:** `![[Northstar Retail architecture](docs/images/overall\_architecture.svg)]` – `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Northstar` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Retail` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `architecture` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `!` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `images` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `overall\_architecture.svg` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.
- **Line 16:** blank line used only to separate logical sections for readability.
- **Line 17:** `### Track A – default local-first path` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** `1. Generate fictional source files with Python.` – `1` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Generate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fictional` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `files` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Python` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it.
- **Line 20:** `2. Load ERP data into local SQL Server and digital data into local PostgreSQL.` – `2` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Load` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ERP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `into` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `local` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Server` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `digital` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 21:** `3. Extract timestamped Parquet/file batches with manifests and checksums.` – `3` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Extract` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestamped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Parquet` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batches` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manifests` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `checksums` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 22:** `4. Manually upload one batch folder to a Databricks Unity Catalog volume.` – `4` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Manually` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `upload` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `folder` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `to` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Databricks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Unity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Catalog` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 23:** `5. Run parameterized Bronze, Silver, Gold, quality, and publication tasks.` - `5` is a numeric literal used by the command or expression; `` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Run` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parameterized` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Bronze` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Silver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Gold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quality` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 24:** `6. Download the validated Gold export.` - `6` is a numeric literal used by the command or expression; `` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Download` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `validated` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Gold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `export` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 25:** `7. Load it transactionally into local \NorthstarRetail\_DW\.` - `7` is a numeric literal used by the command or expression; `` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Load` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `it` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `transactionally` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `into` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `local` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `NorthstarRetail\_DW` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 26:** `8. Import the star schema into Power BI and validate every measure against SQL.` - `8` is a numeric literal used by the command or expression; `` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Import` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `star` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `into` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Power` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BI` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `validate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `every` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 27:** blank line used only to separate logical sections for readability.

- **Line 28:** `Databricks does \*\*not\*\* connect to the local machine's \localhost\` in Track A. Manual upload/download steps are explicit learning boundaries.` - `Databricks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `does` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `connect` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `to` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `localhost` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `s` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `localhost` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 29:** blank line used only to separate logical sections for readability.

- **Line 30:** `### Track B – optional cloud-connected path` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 31:** blank line used only to separate logical sections for readability.

- **Line 32:** `Use network-accessible managed SQL Server/Azure SQL and PostgreSQL, supported JDBC connections, controlled firewall/private networking, secret-managed identities, and cloud storage. This can incur database, compute, storage, egress, networking, and BI costs.` - `Use` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `network-accessible` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `managed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Server` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Azure` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PostgreSQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `supported` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 33:** blank line used only to separate logical sections for readability.

- **Line 34:** `## Technology stack` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 35:** blank line used only to separate logical sections for readability.

- **Line 36:** `| Area | Technology |` - `Area` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Technology` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 37:** `|---|---|` - `---` is a command option or flag; its exact behavior is defined beside the command; `---` is a command option or flag; its exact behavior is defined beside the command

- **Line 38:** `| Version control | Git, GitHub, GitHub Actions |` - `Version` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `control` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Git` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `GitHub` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `GitHub` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 48:** `## Data sources` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 49:** blank line used only to separate logical sections for readability.

- **Line 50:** `SQL Server ERP: customers, addresses, products, categories, suppliers, stores, employees, orders, items, payments, shipments, inventory movements and snapshots.` - `SQL` is a command option or flag; its exact behavior is defined beside the command; `Server` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ERP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `customers` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `addresses` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `products` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `categories` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 51:** `PostgreSQL digital: users, sessions, events, campaigns, touchpoints, and marketing spend.` - `PostgreSQL` is a command option or flag; its exact behavior is defined beside the command; `digital` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `users` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sessions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `events` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `campaigns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 52:** `Files: returns CSV, supplier cost updates CSV, promotion calendar CSV, nested shipment tracking JSONL.` - `Files` is a command option or flag; its exact behavior is defined beside the command; `returns` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CSV` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `supplier` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cost` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `updates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CSV` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `promotion` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 53:** blank line used only to separate logical sections for readability.

- **Line 54:** `See [docs/data\_dictionary.md] (docs/data\_dictionary.md) and [docs/source\_to\_target\_mapping.md] (docs/source\_to\_target\_mapping.md).` - `See` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `data\_dictionary.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `data\_dictionary.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 55:** blank line used only to separate logical sections for readability.

- **Line 56:** `## Pipeline flow` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 57:** blank line used only to separate logical sections for readability.

- **Line 58:** `SQL Server ERP + PostgreSQL digital + CSV/JSON feeds → timestamped local extraction batch → Databricks Bronze → Silver → Gold → blocking quality gate → Gold export → NorthstarRetail\_DW → Power BI / analyst SQL.` - `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Server` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ERP` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PostgreSQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `digital` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CSV` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `JSON` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `feeds` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `timestamped` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 59:** blank line used only to separate logical sections for readability.

- **Line 60:** `Track A deliberately uses manual upload and download boundaries; Track B replaces those boundaries with network-accessible sources and secured cloud storage.` - `Track` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `A` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `deliberately` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `uses` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `upload` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `download` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `boundaries` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Track` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `B` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `replaces` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 61:** blank line used only to separate logical sections for readability.

- **Line 62:** `## Medallion layers` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 63:** blank line used only to separate logical sections for readability.

- **Line 64:** `--Bronze:\*\* raw values, source file/system, ingestion timestamp, batch ID, and record hash.` - `` is a command option or flag; its exact behavior is defined beside the command; ``* is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Bronze` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ``* is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `values` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `system` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 65:** `--Silver:\*\* typed, trimmed, standardized, deduplicated, conformed, quality-flagged, and quarantined records.` - `` is a command option or flag; its exact behavior is defined beside the command; ``* is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Silver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ``\* is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `typed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `trimmed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `standardized` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `deduplicated` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 66:** `--Gold:\*\* nine dimensions, six facts, SCD2 history for customer/product, unknown members, and governed aggregates/views.` - `` is a command option or flag; its exact behavior is defined beside the command; ``* is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Gold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ``* is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `nine` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dimensions` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `six` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `facts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCD2` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `history` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 67:** blank line used only to separate logical sections for readability.

- **Line 68:** `## Data model` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 69:** blank line used only to separate logical sections for readability.

- **Line 70:** `[Gold star schema](docs/images/gold\_star\_schema.svg)` - `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Gold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `star` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `schema` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `images` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `gold\_star\_schema.svg` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.

- **Line 71:** blank line used only to separate logical sections for readability.

- **Line 72:** `Every fact's grain is documented in the workbook and data dictionary. \fact\_sales\` is one valid order item. Inventory is a periodic snapshot and must not be summed across dates without a deliberate calculation.` - `Every` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `s` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `grain` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `documented` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `workbook` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dictionary` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 73:** blank line used only to separate logical sections for readability.

- **Line 74:** `## Data-quality strategy` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 75:** blank line used only to separate logical sections for readability.

- **Line 76:** `The generator intentionally creates controlled defects. Silver handles them with explicit rules and quarantine. Required tests cover not-null, uniqueness, accepted values, referential integrity, date/numeric ranges, freshness, schema drift, duplicate batches, source/layer reconciliation, SCD behavior, and revenue arithmetic. Publication depends on a passing quality gate.` - `The` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `generator` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `intentionally` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `creates` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `controlled` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `handles` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Silver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `handles` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `them` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

described in the surrounding explanation; `explicit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 77:** blank line used only to separate logical sections for readability.
- **Line 78:** `## Orchestration` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 79:** blank line used only to separate logical sections for readability.
- **Line 80:** `[Lakeflow Jobs dependency](docs/images/jobs\_dependency.svg)` - `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Lakeflow` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Jobs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dependency` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `images` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `jobs\_dependency.svg` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 81:** blank line used only to separate logical sections for readability.
- **Line 82:** `The version-controlled example is [\\databricks\\jobs\\job\_definition.example.yml\\](databricks\\jobs\\job\_definition.example.yml). Every task path exists in \\databricks\\notebooks\\` - `The` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `version-controlled` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `example` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `databricks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `jobs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `job\_definition.example.yml` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `databricks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 83:** blank line used only to separate logical sections for readability.
- **Line 84:** `## Semantic model and dashboard` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 85:** blank line used only to separate logical sections for readability.
- **Line 86:** `Power BI uses single-direction dimension-to-fact relationships, a marked date table, explicit measures, display folders, descriptions, and validation SQL. Report pages cover executive, sales, products, stores, customers, marketing, inventory, shipping/returns, and data quality. No \\pbix\\ or result screenshot is fabricated.` - `Power` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BI` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `uses` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `single-direction` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dimension-to-fact` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `relationships` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `marked` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `date` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `table` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 87:** blank line used only to separate logical sections for readability.
- **Line 88:** `## How to run the project safely` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 89:** blank line used only to separate logical sections for readability.
- **Line 90:** `### 1. Clone the existing repository` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 91:** blank line used only to separate logical sections for readability.
- **Line 92:** `Run in \*\*Windows PowerShell\*\*, from the parent folder where you want the clone:` - `Run` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\*\*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Windows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PowerShell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\*\*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `parent` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `folder` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `where` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `you` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 93:** blank line used only to separate logical sections for readability.
- **Line 94:** `\\` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 95:** `git clone https://github.com/Losboules/SQL-Data-Warehouse-Project.git` - `git` calls the Git version-control program; `clone` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `https` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `://` is a path, relative location, branch-style name, or file argument; verify every character before running it; `github.com` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Losboules` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `SQL-Data-Warehouse-Project.git` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 96:** `\\` -
- **Line 97:** blank line used only to separate logical sections for readability.
- **Line 98:** `\\` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 99:** `Set-Location .\\SQL-Data-Warehouse-Project` - `Set-Location` changes the current PowerShell folder; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `SQL-Data-Warehouse-Project` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

```

- **Line 100:** ```` -
- **Line 101:** blank line used only to separate logical sections for readability.
- **Line 102:** ````powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 103:** `git switch -c feature/northstar-platform` - `git` calls the Git version-control program; `switch` changes the active branch; `-c` creates the named branch before switching; `feature` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `northstar-platform` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 104:** ```` -
- **Line 105:** blank line used only to separate logical sections for readability.
- **Line 106:** `### 2. Create Python environment` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 107:** blank line used only to separate logical sections for readability.
- **Line 108:** `Run in **Windows PowerShell**` from the repository root: - `Run` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Windows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PowerShell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `repository` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `root` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 109:** blank line used only to separate logical sections for readability.
- **Line 110:** ````powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 111:** `py -3.13 -m venv .venv` - `py` invokes the Windows Python launcher or Python Install Manager command shim; `-3` is a command option or flag; its exact behavior is defined beside the command; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `13` is a numeric literal used by the command or expression; `-m` asks Python to run an installed module; `venv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `venv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 112:** ```` -
- **Line 113:** blank line used only to separate logical sections for readability.
- **Line 114:** ````powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 115:** `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` - `Set-ExecutionPolicy` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Scope` is a command option or flag; its exact behavior is defined beside the command; `Process` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-ExecutionPolicy` is a command option or flag; its exact behavior is defined beside the command; `Bypass` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 116:** ```` -
- **Line 117:** blank line used only to separate logical sections for readability.
- **Line 118:** ````powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 119:** `.\.venv\Scripts\Activate.ps1` - `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `venv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Scripts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Activate.ps1` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 120:** ```` -
- **Line 121:** blank line used only to separate logical sections for readability.
- **Line 122:** ````powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 123:** `python -m pip install --upgrade pip` - `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `pip` the Python package installer module; `install` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--upgrade` is a command option or flag; its exact behavior is defined beside the command; `pip` the Python package installer module
- **Line 124:** ```` -
- **Line 125:** blank line used only to separate logical sections for readability.
- **Line 126:** ````powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 127:** `python -m pip install -r requirements-dev.txt` - `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `pip` the Python package installer module; `install` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-r` is a command option or flag; its exact behavior is defined beside the command; `requirements-dev.txt` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 128:** ```` -
- **Line 129:** blank line used only to separate logical sections for readability.
- **Line 130:** `### 3. Generate and validate quick data` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 131:** blank line used only to separate logical sections for readability.
- **Line 132:** ````powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 133:** `python -m scripts.data_generation.generate_northstar_data --scale quick --seed 20260815 --output-dir datasets\sample` - `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.data_generation.generate_northstar_data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--scale` is a command option or flag; its exact behavior is defined beside the command; `quick` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--seed` is a command option or flag; its exact behavior is defined beside the command; `20260815` is a numeric literal used by the command or expression; `--output` is a command option or flag; its exact behavior is defined beside the command; `-dir` is a command option or flag; its exact behavior is defined beside the command; `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sample` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 134:** ```` -

```



```

- **Line 135:** blank line used only to separate logical sections for readability.
- **Line 136:** `\\`powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 137:** `python -m scripts.validation.validate_generated_data --input-dir datasets/sample` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.validate_generated_data` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--input` is a command option or flag; its exact behavior is defined beside the command; `-dir` is a command option or flag; its exact behavior is defined beside the command; `datasets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sample` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 138:** `\\` -
- **Line 139:** blank line used only to separate logical sections for readability.
- **Line 140:** `### 4. Run local checks` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 141:** blank line used only to separate logical sections for readability.
- **Line 142:** `\\`powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 143:** `python -m scripts.validation.check_required_files` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 144:** `python -m scripts.validation.validate_sql_dialects` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 145:** `python -m scripts.validation.validate_notebooks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 146:** `python -m scripts.validation.scan_secrets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 147:** `python -m scripts.validation.validate_markdown_links` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 148:** `python -m pytest -q` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `pytest` the Python test runner; `-q` is a command option or flag; its exact behavior is defined beside the command
- **Line 149:** `\\` -
- **Line 150:** blank line used only to separate logical sections for readability.
- **Line 151:** `Follow the single manual-build workbook during construction. Keep this README limited to repository facts and evidence you have personally verified.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `single` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manual-build` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `workbook` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `during` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `construction` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Keep` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `this` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `README` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `limited` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 152:** blank line used only to separate logical sections for readability.
- **Line 153:** `## Repository structure` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 154:** blank line used only to separate logical sections for readability.
- **Line 155:** `\\`text` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 156:** `SQL-Data-Warehouse-Project/` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 157:** `| README.md, LICENSE, configuration and dependency files` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `README.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `LICENSE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `configuration` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dependency` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `files` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 158:** `| datasets/` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `small` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sample` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ignored` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `raw` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batches` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `issue` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fixtures` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 159:** `| docs/` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `workbook` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `phase` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `chapters` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `diagrams` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `contracts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

```



```

ment, identifier, or literal whose project-specific role is described in the surrounding explanation; `do` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 170:** blank line used only to separate logical sections for readability.
- **Line 171:** `## Screenshots` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 172:** blank line used only to separate logical sections for readability.
- **Line 173:** `Replace these placeholders with your own verified evidence:` - `Replace` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `these` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `placeholders` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `your` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `own` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `verified` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `evidence` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 174:** blank line used only to separate logical sections for readability.
- **Line 175:** `-[ADD YOUR VERIFIED ROW COUNT]\` - `-[` is a command option or flag; its exact behavior is defined beside the command; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ADD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `YOUR` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `VERIFIED` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ROW` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `COUNT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 176:** blank line used only to separate logical sections for readability.
- **Line 177:** `-[ADD PHASE 4 SOURCE DATABASE SCREENSHOT]\` - `-[` is a command option or flag; its exact behavior is defined beside the command; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ADD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PHASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `4` is a numeric literal used by the command or expression; `SOURCE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `DATABASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCREENSHOT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 178:** `-[ADD PHASE 14 JOB RUN SCREENSHOT]\` - `-[` is a command option or flag; its exact behavior is defined beside the command; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ADD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PHASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `14` is a numeric literal used by the command or expression; `JOB` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `RUN` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCREENSHOT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 179:** `-[ADD PHASE 15 WAREHOUSE RECONCILIATION SCREENSHOT]\` - `-[` is a command option or flag; its exact behavior is defined beside the command; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ADD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PHASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `15` is a numeric literal used by the command or expression; `WAREHOUSE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `RECONCILIATION` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCREENSHOT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 180:** `-[ADD PHASE 16 POWER BI MODEL SCREENSHOT]\` - `-[` is a command option or flag; its exact behavior is defined beside the command; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ADD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PHASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `16` is a numeric literal used by the command or expression; `POWER` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BI` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `MODEL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCREENSHOT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 181:** `-[ADD PHASE 17 DASHBOARD SCREENSHOT]\` - `-[` is a command option or flag; its exact behavior is defined beside the command; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ADD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PHASE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `17` is a numeric literal used by the command or expression; `DASHBOARD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SCREENSHOT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 182:** blank line used only to separate logical sections for readability.
- **Line 183:** `## Limitations` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 184:** blank line used only to separate logical sections for readability.
- **Line 185:** `-[Track A manual cloud handoffs and daily batch latency]\` - `-[` is a command option or flag; its exact behavior is defined beside the command; `Track` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `A` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `has` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manual` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cloud` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `handoffs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `daily` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `latency` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 186:** `-[The sample and quick generator are reference material; record your own local test result before claiming verification. External-platform execution is not claimed.]\` - `-[` is a command option or flag; its exact behavior is defined beside the command; `The` is a

```

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'sample' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'and' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'quick' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'generator' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'are' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'reference' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'material' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'record' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'your' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'own' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 187:** - Beginner reporting assumes USD and simplified converting-session attribution. - '-' is a command option or flag; its exact behavior is defined beside the command; 'Beginner' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'reporting' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'assumes' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'USD' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'and' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'simplified' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'converting-session' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'attribution' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '.' is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 188:** - The Gold fact notebook reference includes a 'fact_sales' teaching pattern and 'TODO(USER)' ownership work for the remaining facts; optional answers appear only inside the manual-build workbook. - '-' is a command option or flag; its exact behavior is defined beside the command; 'The' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Gold' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'fact' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'notebook' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'reference' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'includes' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'a' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'fact_sales' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'teaching' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'pattern' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'and' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 189:** - The project is educational, not production certification. - '-' is a command option or flag; its exact behavior is defined beside the command; 'The' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'project' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'is' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'educational' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ',' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'not' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'production' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'certification' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '.' is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 190:** blank line used only to separate logical sections for readability.

- **Line 191:** '## Future improvements' is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 192:** blank line used only to separate logical sections for readability.

- **Line 193:** 'See [docs/optional_extensions.md]' (docs/optional_extensions.md). Choose one only after the required checklist is personally verified. - 'See' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '[' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'docs' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'optional_extensions.md' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '[' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'docs' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '/' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'optional_extensions.md' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ']' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Choose' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'one' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 194:** blank line used only to separate logical sections for readability.

- **Line 195:** '## What I learned' is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 196:** blank line used only to separate logical sections for readability.

- **Line 197:** '> Replace this section after completion. Suggested prompts: How did grain prevent a mistake? Which failure did the quality gate catch? Why was SCD2 needed? What made a rerun safe? How did SQL and DAX validation differ?' - '>' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'Replace' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'this' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'section' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'after' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'completion' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; '.' is a path, relative location, branch-style name, or file argument; verify every character before running it; 'Suggested' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'prompts' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ':' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'How' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'did' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 198:** blank line used only to separate logical sections for readability.

- **Line 199:** '[WRITE YOUR OWN VERIFIED REFLECTION]' - '[' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'WRITE' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'YOUR' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'OWN' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'VERIFIED' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 'REFLECTION' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ']' is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 200:** blank line used only to separate logical sections for readability.

- **Line 201:** `## Contact` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 202:** blank line used only to separate logical sections for readability.

- **Line 203:** `\*\*Kirolos Boules\*\*` – `\*\*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Kirolos` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Boules` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\*\*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 204:** `LinkedIn: \[ADD LINK]\` – `LinkedIn` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ADD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `LINK` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 205:** `Email or portfolio: \[ADD SAFE PUBLIC CONTACT]\` – `Email` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `or` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `portfolio` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ADD` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SAFE` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PUBLIC` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `CONTACT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 206:** blank line used only to separate logical sections for readability.

- **Line 207:** `## License` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 208:** blank line used only to separate logical sections for readability.

- **Line 209:** `MIT. All sample identities and business entities are fictional.` – `MIT` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `All` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sample` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `identities` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `business` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `entities` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `are` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fictional` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it

[Back to the Phase 20 lesson index](#phase-20-lesson-index)

File introduction – `docs/claim\_to\_evidence\_map.md`

Exact filename: `claim\_to\_evidence\_map.md`

Filename extension: `.md`. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `docs` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 20.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `docs` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `claim\_to\_evidence\_map.md`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `claim\_to\_evidence\_map.md` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 20.F2.1 – Create the file `docs/claim\_to\_evidence\_map.md` from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

```

**Current folder or location:**
The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.

**What you are about to do:**
Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**
The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches `The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.` or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

``powershell
New-Item -ItemType File -Path ".\docs\claim_to_evidence_map.md" -ErrorAction Stop

```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\docs\claim_to_evidence_map.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs\claim_to_evidence_map.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\docs\claim_to_evidence_map.md" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/claim_to_evidence_map.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under docs.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `docs/claim_to_evidence_map.md`.
3. In PowerShell at the repository root, run `Test-Path ".\docs\claim_to_evidence_map.md" and require True`.
4. Run `git status --short -- "docs/claim_to_evidence_map.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for docs/claim_to_evidence_map.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: `docs/claim_to_evidence_map.md`

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 20.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Claim-to-Evidence Map
```

```
Do not convert a reference capability into a personal achievement until the evidence column is complete.
```

```
| Claim | Status | Evidence type | Exact file/run/query | Batch/date/commit | Reviewer note |
```

```

|---|---|---|---|---|
| Quick synthetic fixture is deterministic | Reference only; user verification required | pytest + manifest | `tests/unit/test_generator.py`;
`datasets/sample/generation_manifest.json` | Seed 20260815 | Re-run locally |
| SQL Server ERP is loaded | Not started | SSMS row counts + loader artifact | `[ADD]` | `[ADD]` | Do not claim yet |
| PostgreSQL digital source is loaded | Not started | pgAdmin/psql counts + artifact | `[ADD]` | `[ADD]` | Do not claim yet |
| Databricks Job completed and quality passed | Not started | Job run ID + quality table | `[ADD]` | `[ADD]` | Do not claim yet |
| Gold reconciles to SQL Server warehouse | Not started | export manifest + audit SQL | `[ADD]` | `[ADD]` | Do not claim yet |
| Power BI measures match SQL | Not started | validation table/screenshot | `[ADD]` | `[ADD]` | Do not claim yet |
| GitHub Actions validates repository | Not started | Actions run URL/checks | `[ADD]` | `[ADD]` | Do not claim yet |

```

Line-by-line explanation

- **Line 1:** # Claim-to-Evidence Map is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** Do not convert a reference capability into a personal achievement until the evidence column is complete. — Do is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; convert is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reference is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; capability is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; into is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; personal is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; achievement is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; until is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** | Claim | Status | Evidence type | Exact file/run/query | Batch/date/commit | Reviewer note | — Claim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Status is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Evidence is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; type is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Exact is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.
- **Line 6:** |---|---|---|---|---| — --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command
- **Line 7:** | Quick synthetic fixture is deterministic | Reference only; user verification required | pytest + manifest | `tests/unit/test\_generator.py`; `datasets/sample/generation\_manifest.json` | Seed 20260815 | Re-run locally | — Quick is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; synthetic is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fixture is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; deterministic is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Reference is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; only is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding

explanation;user is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;verification is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;required is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;py-test the Python test runner;+ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 8:** | SQL Server ERP is loaded | Not started | SSMS row counts + loader artifact | \[ADD] | \[ADD] | Do not claim yet | -SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;ERP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;loaded is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;started is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;SSMS is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;row is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;+ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;loader is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 9:** | PostgreSQL digital source is loaded | Not started | pgAdmin/psql counts + artifact | \[ADD] | \[ADD] | Do not claim yet | -PostgreSQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;digital is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;loaded is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;started is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;pgAdmin is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;/ is a path, relative location, branch-style name, or file argument; verify every character before running it;psql is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;+ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 10:** | Databricks Job completed and quality passed | Not started | Job run ID + quality table | \[ADD] | \[ADD] | Do not claim yet | -Databricks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Job is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;completed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;quality is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;passed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;started is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Job is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;+ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 11:** | Gold reconciles to SQL Server warehouse | Not started | export manifest + audit SQL | \[ADD] | \[ADD] | Do not claim yet | -Gold is a name, keyword, argument, identifier, or literal whose project-specific

role is described in the surrounding explanation;reconciles is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;to is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;warehouse is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;started is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;export is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;manifest is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;+ is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;audit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 12:** | Power BI measures match SQL | Not started | validation table/screenshot | \[ADD] | \[ADD] | Do not claim yet | -Power is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;BI is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;measures is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;match is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;started is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;validation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;/ is a path, relative location, branch-style name, or file argument; verify every character before running it; screenshot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;[] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 13:** | GitHub Actions validates repository | Not started | Actions run URL/checks | \[ADD] | \[ADD] | Do not claim yet | -GitHub is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Actions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;validates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;repository is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;started is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Actions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;URL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;/ is a path, relative location, branch-style name, or file argument; verify every character before running it; checks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;[] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

[Back to the Phase 20 lesson index](#)

File introduction — docs/resume_bullets.md

Exact filename: resume_bullets.md

Filename extension: .md. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: docs relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 20.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in docs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `resume_bullets.md`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `resume_bullets.md` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 20.F3.1 — Create the file `docs/resume_bullets.md` from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\resume_bullets.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\docs\resume_bullets.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs\resume_bullets.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\docs\resume_bullets.md" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/resume_bullets.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under docs.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `docs/resume_bullets.md`.
3. In PowerShell at the repository root, run `Test-Path ".\docs\resume_bullets.md"` and require True.
4. Run `git status --short -- "docs/resume_bullets.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for docs/resume_bullets.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: `docs/resume_bullets.md`

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 20.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Resume Bullet Drafts

Use only after replacing bracketed placeholders with results you personally verified. Do not list the project as production experience.

- Built a portfolio-scale omnichannel data platform integrating synthetic SQL Server ERP, PostgreSQL digital, CSV, and nested JSON sources through Python batch extraction and Databricks Delta Bronze/Silver/Gold layers; verified `[ADD SOURCE COUNT]` sources and `[ADD VERIFIED ROW COUNT]` Gold rows.
- Designed a dimensional warehouse with nine conformed dimensions and six fact tables, including SCD Type 2 customer/product history, unknown-member handling, declared fact grains, and source-to-target mappings.
- Orchestrated a parameterized Lakeflow Job with dependency-aware Bronze, Silver, Gold, quality-gate, publication, and reconciliation tasks; demonstrated idempotent repair behavior using batch ID `[ADD BATCH ID]`.
- Implemented data-quality and reconciliation checks for uniqueness, referential integrity, ranges, freshness, schema drift, quarantine, and revenue, achieving `[ADD VERIFIED PASS COUNT]` required checks after resolving `[ADD REAL ISSUE]`.
- Published validated Gold data to a local SQL Server serving warehouse and built a Power BI star model with documented DAX for revenue, gross margin, return rate, delivery, conversion, ROAS, customers, and inventory; reconciled dashboard measures to SQL within `[ADD VERIFIED TOLERANCE]`.
- Added GitHub Actions checks for Python lint/tests, required artifacts, SQL dialect labels, notebook syntax, secret patterns, and relative documentation links.

## One-line project description

`Northstar Retail Data Platform – a beginner-built, evidence-driven batch lakehouse and dimensional warehouse portfolio using SQL Server, PostgreSQL, Python, Databricks, Delta Lake, Jobs, and Power BI.`
```

Line-by-line explanation

- **Line 1:** `# Resume Bullet Drafts` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** Use only after replacing bracketed placeholders with results you personally verified. Do not list the project as production experience. — Use is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; only is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; after is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; replacing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bracketed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; placeholders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; with is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; results is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; you is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; personally is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; verified is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** - Built a portfolio-scale omnichannel data platform integrating synthetic SQL Server ERP, PostgreSQL digital, CSV, and nested JSON sources through Python batch extraction and Databricks Delta Bronze/Silver/Gold layers; verified \[ADD SOURCE COUNT] sources and \[ADD VERIFIED ROW COUNT] Gold rows.—is a command option or flag; its exact behavior is defined beside the command; Built is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; portfolio-scale is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; omnichannel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; platform is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; integrating is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; synthetic is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ERP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 6:** - Designed a dimensional warehouse with nine conformed dimensions and six fact tables, including SCD Type 2 customer/product history, unknown-member handling, declared fact grains, and source-to-target mappings. — - is a command option or flag; its exact behavior is defined beside the command; Designed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dimensional is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; warehouse is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; with is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; nine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; conformed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dimensions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; six is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 7:** - Orchestrated a parameterized Lakeflow Job with dependency-aware Bronze, Silver, Gold, quality-gate, publication, and reconciliation tasks; demonstrated idempotent repair behavior using batch ID \[ADD BATCH ID].—is a command option or flag; its exact behavior is defined beside the command; Orchestrated is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; parameterized is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Lakeflow is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Job is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;With is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;dependency-aware is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Bronze is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Silver is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 8:** - Implemented data-quality and reconciliation checks for uniqueness, referential integrity, ranges, freshness, schema drift, quarantine, and revenue, achieving \[ADD VERIFIED PASS COUNT] required checks after resolving \[ADD REAL ISSUE]. — is a command option or flag; its exact behavior is defined beside the command; Implemented is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data-quality is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reconciliation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; checks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; uniqueness is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; referential is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; integrity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 9:** - Published validated Gold data to a local SQL Server serving warehouse and built a Power BI star model with documented DAX for revenue, gross margin, return rate, delivery, conversion, ROAS, customers, and inventory; reconciled dashboard measures to SQL within \[ADD VERIFIED TOLERANCE]. — is a command option or flag; its exact behavior is defined beside the command; Published is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; validated is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; to is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; local is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; serving is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; warehouse is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 10:** - Added GitHub Actions checks for Python lint/tests, required artifacts, SQL dialect labels, notebook syntax, secret patterns, and relative documentation links. — is a command option or flag; its exact behavior is defined beside the command; Added is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; GitHub is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Actions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; checks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Python is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; lint is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; tests is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; required is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 11:** blank line used only to separate logical sections for readability.
- **Line 12:** `##` One-line project description is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 13:** blank line used only to separate logical sections for readability.
- **Line 14:** `\Northstar Retail Data Platform — a beginner-built, evidence-driven batch lakehouse and dimensional warehouse portfolio using SQL Server, PostgreSQL, Python, Databricks, Delta Lake, Jobs, and Power BI.``—Northstar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Retail is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Platform is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ais a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; beginner-built is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; evidence-driven is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; lakehouse is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dimensional is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

[Back to the Phase 20 lesson index](#)

File introduction — docs/linkedin_post.md

Exact filename: `linkedin_post.md`

Filename extension: `.md`. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: `docs` relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 20.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is `SQL-Data-Warehouse-Project` and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in `docs` one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `linkedin_post.md`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `linkedin_post.md` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 20.F4.1 — Create the file docs/linkedin_post.md from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\linkedin_post.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType File -Path ".\docs\linkedin_post.md" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; File is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\docs\linkedin_post.md" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\docs\linkedin_post.md" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/linkedin_post.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under docs.

2. Right-click the editor tab and choose **Copy Path**; compare the ending with docs/linkedin_post.md.
3. In PowerShell at the repository root, run Test-Path ".\docs\linkedin_post.md" and require True.
4. Run git status --short -- "docs/linkedin_post.md" to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for docs/linkedin_post.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: docs/linkedin_post.md

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 20.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# LinkedIn Post Draft

I have been building Northstar Retail, a fictional end-to-end data engineering portfolio project designed to teach me the decisions behind the tools—not just give me code to copy.

The project connects synthetic SQL Server ERP data, PostgreSQL website/marketing data, and CSV/JSON feeds. I use Python for reproducible data generation and batch manifests, Databricks and Delta Lake for Bronze/Silver/Gold processing, Lakeflow Jobs for dependencies and quality gates, SQL Server as a serving warehouse, and Power BI for the semantic and reporting layer.

The most important lessons so far:

- A fact table must have one explicit grain before measures are written.
- A cloud workspace cannot reach my computer's `localhost`, so the beginner track uses an honest manual file handoff.
- Dirty data is useful when it is controlled: it lets me prove deduplication, quarantine, date parsing, SCD Type 2, and rerun behavior.
- A dashboard number is not trustworthy until it reconciles to source rules and independent SQL.

Verified evidence from my run:

- Batch: `[ADD VERIFIED BATCH ID]`
- Required quality checks passed: `[ADD VERIFIED COUNT]`
- Revenue reconciliation difference: `[ADD VERIFIED DIFFERENCE]`
- Same-batch rerun duplicate difference: `[ADD VERIFIED RESULT]`

Next I am improving `[ADD REAL NEXT STEP]`.

#DataEngineering #SQL #Python #Databricks #DeltaLake #PowerBI #DataWarehouse #PortfolioProject
```

Line-by-line explanation

- **Line 1:** # LinkedIn Post Draft is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** I have been building **Northstar Retail**, a fictional end-to-end data engineering portfolio project designed to teach me the decisions behind the tools—not just give me code to copy. — I is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; have is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; been is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; building is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Northstar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Retail is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **, is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fictional is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; end-to-end is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** The project connects synthetic SQL Server ERP data, PostgreSQL website/marketing data, and CSV/JSON feeds. I use Python for reproducible data generation and batch manifests, Databricks and Delta Lake for Bronze/Silver/Gold processing, Lakeflow Jobs for dependencies and quality gates, SQL Server as a serving warehouse, and Power BI for the semantic and reporting layer. — The is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; project is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; con-

nects is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; synthetic is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ERP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PostgreSQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; website is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.

- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** The most important lessons so far: — The is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; most is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; important is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; lessons is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; so is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; far is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** - A fact table must have one explicit grain before measures are written. — - is a command option or flag; its exact behavior is defined beside the command; A is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; must is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; have is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; explicit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; grain is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; before is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; measures is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 10:** - A cloud workspace cannot reach my computer's \localhost`, so the beginner track uses an honest manual file handoff.—is a command option or flag; its exact behavior is defined beside the command; A is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cloud is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; workspace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cannot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reach is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; my is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; computer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; s is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; localhost is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; so` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 11:** - Dirty data is useful when it is controlled: it lets me prove deduplication, quarantine, date parsing, SCD Type 2, and rerun behavior. — - is a command option or flag; its exact behavior is defined beside the command; Dirty is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific

role is described in the surrounding explanation; useful is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; when is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; it is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; controlled is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; it is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; lets is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 12:** - A dashboard number is not trustworthy until it reconciles to source rules and independent SQL. — - is a command option or flag; its exact behavior is defined beside the command; A is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dashboard is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; trustworthy is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; until is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; it is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reconciles is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; to is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 13:** blank line used only to separate logical sections for readability.
- **Line 14:** Verified evidence from my run: — Verified is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; evidence is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; from is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; my is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 15:** blank line used only to separate logical sections for readability.
- **Line 16:** - Batch: \[ADD VERIFIED BATCH ID]—is a command option or flag; its exact behavior is defined beside the command; Batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ADD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VERIFIED is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BATCH is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 17:** - Required quality checks passed: \[ADD VERIFIED COUNT]—is a command option or flag; its exact behavior is defined beside the command; Required is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quality is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; checks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; passed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ADD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; VERIFIED is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; COUNT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 18:** - Revenue reconciliation difference: `\[ADD VERIFIED DIFFERENCE]`—is a command option or flag; its exact behavior is defined beside the command;Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;reconciliation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;difference is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;[is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;ADD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;VERIFIED is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;DIFFERENCE is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 19:** - Same-batch rerun duplicate difference: `\[ADD VERIFIED RESULT]`—is a command option or flag; its exact behavior is defined beside the command;Same-batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;rerun is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;duplicate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;difference is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;[is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;ADD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;VERIFIED is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;RESULT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 20:** blank line used only to separate logical sections for readability.
- **Line 21:** Next I am improving `\[ADD REAL NEXT STEP]`—Next is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;I is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;am is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;improving is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;[is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;ADD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;REAL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;NEXT is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;STEP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 22:** blank line used only to separate logical sections for readability.
- **Line 23:** `#DataEngineering #SQL #Python #Databricks #DeltaLake #PowerBI #DataWarehouse #PortfolioProject` is a comment for a human reader; it normally does not change data or execute business logic.

[Back to the Phase 20 lesson index](#)

File introduction — docs/demo_video_script.md

Exact filename: demo_video_script.md

Filename extension: .md. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: docs relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 20.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-

specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in docs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: demo_video_script.md.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled demo_video_script.md and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 20.F5.1 — Create the file docs/demo_video_script.md from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\demo_video_script.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\docs\demo_video_script.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs\demo_video_script.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\docs\demo_video_script.md" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add -Force, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/demo_video_script.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under docs.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `docs/demo_video_script.md`.
3. In PowerShell at the repository root, run `Test-Path ".\docs\demo_video_script.md"` and require True.
4. Run `git status --short -- "docs/demo_video_script.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for docs/demo_video_script.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: `docs/demo_video_script.md`

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 20.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Demo Video Script

## 0:00-0:30 – Business problem

“Northstar Retail is a fictional omnichannel retailer whose ERP, website, marketing, shipment, return, and inventory data disagree across systems. My goal was to produce trusted daily metrics with evidence from source through dashboard.”

Show: README business problem and overall architecture. Do not show credentials.

## 0:30-1:30 – Sources and synthetic data

Show SQL Server and PostgreSQL ERDs, then the generator command and sample manifest. Explain fixed seed, scale settings, realistic dependencies, and controlled defects. Show one expected issue fixture and one relevant source row.

## 1:30-2:30 – Lakehouse and workflow

Show Bronze/Silver/Gold contracts and the Lakeflow Job graph. Explain the manual Track A boundary and why localhost is not reachable. Open one Silver notebook and one quarantine rule.

## 2:30-3:30 – Dimensional model and SCD2

Show the star schema and SCD2 timeline. State fact_sales grain in one sentence. Show the current/expired customer versions from your verified run.

## 3:30-4:30 – Quality and publication

Show the test-results table and one failed-then-fixed story. Show Gold export manifest, SQL Server audit row, and revenue reconciliation. Use real batch values only.

## 4:30-5:30 – Power BI and conclusion

Show Model view, date table, measure folder, Executive Overview, and Data Quality page. Put SQL validation next to one KPI. Conclude with limitations and the next extension.

## Recording checklist

- Crop or blur account email, paths, server identifiers, tokens, and passwords.
- Zoom so text is readable.
```

- Keep batch ID visible where it proves lineage.
- Do not claim portfolio-scale timing unless measured.
- Keep the GitHub README open at the final project link.

Line-by-line explanation

- **Line 1:** `# Demo Video` Script is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** `## 0:00–0:30` – Business problem is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** “Northstar Retail is a fictional omnichannel retailer whose ERP, website, marketing, shipment, return, and inventory data disagree across systems. My goal was to produce trusted daily metrics with evidence from source through dashboard.” — Northstar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Retail is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fictional is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; omnichannel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; retailer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; whose is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ERP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; website is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** `Show: README business problem and overall architecture. Do not show credentials.` — Show is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; README is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; business is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; problem is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; overall is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; architecture is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Do is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; show is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** `## 0:30–1:30` – Sources and synthetic data is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 10:** blank line used only to separate logical sections for readability.
- **Line 11:** `Show SQL Server and PostgreSQL ERDs, then the generator command and sample manifest. Explain fixed seed, scale settings, realistic dependencies, and controlled defects. Show one expected issue fixture and one relevant source row.` — Show is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; PostgreSQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ERDs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; then is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; generator is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; command is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** ## 1:30–2:30 – Lakehouse and workflow is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 14:** blank line used only to separate logical sections for readability.
- **Line 15:** Show Bronze/Silver/Gold contracts and the Lakeflow Job graph. Explain the manual Track A boundary and why localhost is not reachable. Open one Silver notebook and one quarantine rule. — Show is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Bronze is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; Silver is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; Gold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; contracts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Lakeflow is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Job is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; graph is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 16:** blank line used only to separate logical sections for readability.
- **Line 17:** ## 2:30–3:30 – Dimensional model and SCD2 is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** Show the star schema and SCD2 timeline. State fact_sales grain in one sentence. Show the current/expired customer versions from your verified run. — Show is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; star is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SCD2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timeline is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; State is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; grain is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 20:** blank line used only to separate logical sections for readability.
- **Line 21:** ## 3:30–4:30 – Quality and publication is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 22:** blank line used only to separate logical sections for readability.
- **Line 23:** Show the test-results table and one failed-then-fixed story. Show Gold export manifest, SQL Server audit row, and revenue reconciliation. Use real batch values only. — Show is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test-results is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; failed-then-fixed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; story is a name, keyword, argument,

identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Show is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; export is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 24:** blank line used only to separate logical sections for readability.
- **Line 25: ## 4:30–5:30 – Power BI and conclusion** is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 26:** blank line used only to separate logical sections for readability.
- **Line 27:** Show Model view, date table, measure folder, Executive Overview, and Data Quality page. Put SQL validation next to one KPI. Conclude with limitations and the next extension. — Show is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Model is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; view is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; measure is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; folder is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Executive is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Overview is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 28:** blank line used only to separate logical sections for readability.
- **Line 29: ## Recording checklist** is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 30:** blank line used only to separate logical sections for readability.
- **Line 31:** - Crop or blur account email, paths, server identifiers, tokens, and passwords. — - is a command option or flag; its exact behavior is defined beside the command; Crop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; blur is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; account is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; email is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; paths is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; identifiers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 32:** - Zoom so text is readable. — - is a command option or flag; its exact behavior is defined beside the command; Zoom is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; so is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; text is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; readable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 33:** - Keep batch ID visible where it proves lineage. — - is a command option or flag; its exact behavior is defined beside the command; Keep is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; visible is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; where is a name, keyword, argument,

identifier, or literal whose project-specific role is described in the surrounding explanation; it is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; proves is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; lineage is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 34:** - Do not claim portfolio-scale timing unless measured. — - is a command option or flag; its exact behavior is defined beside the command; Do is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; claim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; portfolio-scale is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; unless is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; measured is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 35:** - Keep the GitHub README open at the final project link. — - is a command option or flag; its exact behavior is defined beside the command; Keep is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; GitHub is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; README is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; open is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; at is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; final is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; project is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; link is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it

[Back to the Phase 20 lesson index](#)

What your screen should look like now

- Every Phase 20 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 20.3 completion checklist

☐

README.md was created by me, saved, located correctly, and inspected — [STATUS]

☐

docs/claim_to_evidence_map.md was created by me, saved, located correctly, and inspected — [STATUS]

☐

docs/resume_bullets.md was created by me, saved, located correctly, and inspected — [STATUS]

☐

docs/linkedin_post.md was created by me, saved, located correctly, and inspected — [STATUS]

☐

docs/demo_video_script.md was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 20.4](#) and recheck the root and branch.

Lesson 20.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Action 20.4.G1 — Inspect the current state without changing it

Application:

VS Code Markdown Preview

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Inspect the current state without changing it. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code Markdown Preview. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

Action 20.4.G2 — Create the smallest required object or decision

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Create the smallest required object or decision. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

Action 20.4.G3 — Run the phase-specific verification

Application:

VS Code Markdown Preview

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run the phase-specific verification. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code Markdown Preview. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

Action 20.4.G4 — Demonstrate one safe failure or alternate-result branch

Application:

VS Code Markdown Preview

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Demonstrate one safe failure or alternate-result branch. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code Markdown Preview. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 20 lesson index](#)

Action 20.4.G5 — Record evidence and prepare the downstream handoff

Application:

VS Code Markdown Preview

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Record evidence and prepare the downstream handoff. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code Markdown Preview. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

What your screen should look like now

- The new Phase 20 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 20.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 20.5](#).

Lesson 20.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 20 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Verification before automation

Question:

Should the phase prioritize a visible manual verification or jump directly to automation?

Options considered:

- Manual verification first, then automation.
- Automation without a known-good baseline.
- Manual operation forever.

Beginner path selected:

Establish one verified manual baseline and then automate repeatable checks.

Why this path was selected:

A known-good example makes failures interpretable and prevents automation from hiding a wrong assumption.

Tradeoff:

The first run takes longer.

What a company might do instead:

Teams often pair infrastructure as code and automated tests with controlled bootstrap and acceptance procedures.

How to explain this in an interview:

I established a verified baseline, then encoded the same contract in tests and orchestration so automation preserved meaning rather than just speed.

If you see Result A

GitHub renders the README, links, evidence, limitations, and verified claims correctly. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

A link/image is broken or a claim lacks execution evidence and must be softened or removed. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: VS Code Markdown Preview, GitHub repository page, File Explorer, and image editor only for redaction/cropping.

Window/panel: Show GitHub README preview with verified screenshots, limitations, and evidence links.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase20_build-the-readme-evidence-package-and-portfolio-present_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 20's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 20 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 20.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

Action 20.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

Action 20.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

Interpret git status --short

- ?? means untracked: Git sees a new path but it is not staged.
- M means modified in the working tree but not staged.
- M means the staged version differs from the last commit.
- MM means a staged version exists and the file has additional unstaged edits.
- A means staged as a new file.
- D means deletion; stop and investigate unless deletion is deliberate.
- R means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 20.5.G4 — Inspect the unstaged diff for README.md

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "README.md"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "README.md"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"README.md"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

Action 20.5.G5 — Stage only the reviewed file README.md

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "README.md"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "README.md"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"README.md"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 20.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

Action 20.5.G7 — Create the Phase 20 commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "docs: publish verified portfolio evidence"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "docs: publish verified portfolio evidence"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"docs: publish verified portfolio evidence"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

Action 20.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

Action 20.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 20 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message docs: publish verified portfolio evidence and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 20.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 20 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

"Phase 20 created the verified boundary described by **Build the README, evidence package, and portfolio presentation**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed."

Mini quiz

1. Multiple choice: Why does Phase 20 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

My Phase 20 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 20.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 21 turns the real project into truthful interview explanations.

Phase 20 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 21 — Prepare interview explanations and defend every design choice

[Back to the master table of contents](#)

Track: Track A required

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: A completed claim-to-evidence map and defensible decisions.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: Phase 22 evaluates advanced alternatives only after the required path is stable.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 21 lesson index

1. [Lesson 21.1 — Understand the phase and starting screen](#)
2. [Lesson 21.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 21.3 — Create and understand the phase files](#)
4. [Lesson 21.4 — Build and validate required objects](#)
5. [Lesson 21.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 21.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** VS Code, the completed workbook journal, repository evidence, and a timer.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 21 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 20 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 21 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

VS Code, the completed workbook journal, repository evidence, and a timer. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 21 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.
- No new Phase 21 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 21 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 21.1 completion checklist

☐

I read why Phase 21 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 21.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 21 lesson index](#)

Lesson 21.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 21 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Track A required

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

A 30-second explanation, 2-minute story, 5-minute architecture walkthrough, 10-minute technical walkthrough, at least 40 interview questions with beginner-friendly answers and follow-ups, a design-defense worksheet, STAR troubleshooting template, glossary, and final self-assessment rubric.

2. Why it matters

A portfolio helps only when you can explain what happened, why each choice was made, what failed, what evidence proves correctness, and what you would change at greater scale.

3. Words to know

Term	Plain-language definition
Elevator pitch	A short problem-and-solution explanation for someone with limited time.
Tradeoff	A benefit gained while accepting a cost or limitation.
Design defense	Explaining the requirement, alternatives, decision, consequences, and evidence for a choice.
STAR	Situation, Task, Action, Result: a structured story format.
Teach-back	Explaining a concept in your own words to prove understanding.
Follow-up question	A deeper interviewer question that tests whether an answer was memorized or understood.
Scale thought experiment	Explaining what changes if volume, velocity, users, or reliability requirements increase.
Failure mode	A way the system can break and how it is detected/recovered.
Self-assessment rubric	A scored checklist of understanding, execution, evidence, and communication.

4. What must already be completed

You have completed or accurately status-labeled every project phase, recorded at least one real troubleshooting story, and can open the repository evidence during a walkthrough.

5. Applications to open

Application	How to open it	Why it is open
VS Code Markdown Preview	Open docs/interview_walkthrough.md and glossary.	Practice from prompts without changing evidence.
Timer/voice recorder	Use a local phone or desktop recorder.	Measure pacing and hear unclear explanations.
Repository/browser	Open README, diagrams, code, tests, screenshots.	Support answers with evidence rather than memory alone.
Learning journal	Open the learning-journal fields in this phase.	Select real failures and lessons for STAR stories.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run `Get-Location`; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
code .\docs\interview_walkthrough.md	Opens the interview guide and glossary in VS Code.	These are the practice source, not a script to read during an interview.
(Select-String -Path .\docs\interview_walkthrough.md -Pattern '^##[0-9]+.').Count	Counts the labeled interview questions.	The workbook requirement is at least 40.
Select-String -Path the learning-journal fields in this workbook -Pattern 'What broke How I diagnosed What I learned'	Locates the troubleshooting-story prompts.	STAR answers should come from your actual work.
Get-ChildItem .\docs\images*.svg Select-Object Name	Lists diagrams available for a live walkthrough.	You should know which visual supports each explanation.

Answer structure

Use: **requirement -> decision -> implementation -> validation -> tradeoff -> next-scale change**. Example: leadership needed consistent daily product metrics; a star schema was chosen for understandable BI; fact_sales uses one row per order item and surrogate dimension keys; grain/RI/revenue tests validate it; SCD2/history adds complexity; at 100x volume, incremental/partition/CDC and performance testing would become more important.

Honest uncertainty

When you did not execute an optional feature, say so, explain the designed extension, and distinguish what you know conceptually from evidence in this repository.

MY TASK

Record all four timed explanations, answer every in-workbook question aloud, complete the design-defense and STAR worksheets with your evidence, run a follow-up practice session, and score the self-assessment honestly.

STARTER SKELETON

```
DESIGN CHOICE: TODO(USER)
REQUIREMENT: TODO(USER)
ALTERNATIVES CONSIDERED: TODO(USER)
DECISION AND WHY: TODO(USER)
IMPLEMENTATION FILES: TODO(USER)
VALIDATION EVIDENCE: TODO(USER)
TRADEOFF/LIMITATION: TODO(USER)
WHAT CHANGES AT 100x: TODO(USER)
FOLLOW-UP I EXPECT: TODO(USER)
```

Acceptance criteria: Each timed version fits its limit and sounds natural; every major component/design choice can be explained without jargon-only answers; 40+ questions are attempted with evidence; one real STAR story is complete; limitations and 100x changes are defensible; rubric gaps become tasks.

HINT 1 — CONCEPT

Do not memorize paragraphs; memorize the architecture, grains, validations, and tradeoffs.

HINT 2 — PSEUDOCODE

For every answer, connect one requirement to one file/query/run and one limitation.

HINT 3 — NEAR-COMPLETE PATTERN

Use the in-workbook question bank, hide the answer, speak for 60–90 seconds, then compare and add your own verified example/follow-up.

13. Expected files or output

- Four timed recordings or practice notes.
- Attempt log for 40+ questions and follow-ups.
- Completed design-defense worksheets.
- At least one genuine STAR troubleshooting story.
- Glossary teach-back notes.
- Completed self-assessment rubric and learning tasks.

14. How to verify

- 30-second pitch states problem, solution, and honest status.
- 2-minute story covers full flow without drowning in tools.
- 5-minute architecture follows actual arrows/boundaries.
- 10-minute walkthrough includes dirty data, SCD2, grain, gate, and reconciliation evidence.
- Every metric/achievement statement can be traced to evidence.
- You can explain why SQL Server and PostgreSQL are separate and why dialects differ.
- You can state Track A manual limitations and Track B requirements.
- Rubric is not self-scored as complete without proof.

15. Automated tests

- Timed practice against limits.
- Random-question practice without reading answers.
- Follow-up interruption practice.
- Diagram teach-back.
- Evidence lookup under two minutes.
- Self-assessment scored with examples.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
Answer becomes a list of tools	The business requirement and data flow are missing.	Listen for a problem, decision, validation, and tradeoff.	Start with the stakeholder question; mention tools only where they solve a requirement.	A nontechnical listener can summarize the value.
Cannot state fact_sales grain	Model was memorized by table name rather than row meaning.	Open mapping/DDI and identify unique business event.	Repeat: one row per valid order item; show duplicate-grain test.	You can explain keys and measures at that grain.
Claims direct Databricks-to-local SQL connectivity	Track A boundary was forgotten.	Trace local-first diagram and notebook headers.	Explain manual upload/download and optional networked Track B.	Answer matches architecture.
STAR result is vague	No measurable/evidentiary outcome was recorded.	Open journal/test/run evidence.	State pass/fail, reconciled value, prevented publication, or corrected behavior; avoid invented metrics.	Result links to proof.

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
Cannot answer scale follow-up	Tradeoffs were not separated from current implementation.	List current bottlenecks/ boundaries by source, storage, compute, model, serving.	Discuss incremental/CDC, partitioning, concurrent jobs, observability, networking, security, and semantic performance as conditional changes.	Answer is requirement-driven rather than buzzwords.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Explain the entire platform to a nontechnical manager in two minutes.
2. Defend the grain of fact_sales and the use of SCD2.
3. Explain one failure from symptom to verification.
4. State what you would change first at 100 times the data volume and why.

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. What six-part answer structure is recommended?
2. Which manual boundary must always be disclosed for Track A?
3. What kind of story should use the STAR format?

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture your completed design-defense worksheet, self-assessment rubric, and a practice setup showing the architecture/evidence—not recordings or personal details unless you choose to keep them private.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
docs: add interview teach-back and design defense evidence
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

30-second explanation practiced — [STATUS]

☐

2-minute explanation practiced — [STATUS]

☐

5-minute architecture walkthrough practiced — [STATUS]

☐

10-minute technical walkthrough practiced — [STATUS]

☐

40+ questions attempted — [STATUS]

☐

Design defenses completed — [STATUS]

☐

STAR story evidence complete — [STATUS]



Self-assessment gaps converted to tasks — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-21. The quiz answer is in the answer-key section immediately after the quiz in this workbook#phase-21.

Next phase: Phase 22 presents optional extensions as separate choices after the required batch platform is verified.

Required action cards

Action 21.2.0 — Review your completed in-workbook journals and glossary before drafting interview answers

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Review your completed in-workbook journals and glossary before drafting interview answers. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. CLICK: Open this manual-build workbook in VS Code.
2. PRESS: Ctrl+F.
3. TYPE THIS: ## My Phase.
4. PRESS: Enter repeatedly to visit each phase journal you actually completed.
5. LOOK FOR: Record real errors, diagnostic actions, fixes, and lessons that can support truthful STAR stories.
6. PRESS: Ctrl+F.
7. TYPE THIS: # Complete glossary.
8. PRESS: Enter.
9. LOOK FOR: Rehearse the interview-ready definitions only after explaining each term in your own words.
10. DO NOT: Open or create a separate journal, glossary, or troubleshooting file.
11. DO NOT CONTINUE UNTIL: Every interview claim points to a real phase, commit, run result, or screenshot.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. CLICK: Open this manual-build workbook in VS Code.
2. PRESS: Ctrl+F.
3. TYPE THIS: ## My Phase.

4. PRESS: Enter repeatedly to visit each phase journal you actually completed.
5. LOOK FOR: Record real errors, diagnostic actions, fixes, and lessons that can support truthful STAR stories.
6. PRESS: Ctrl+F.
7. TYPE THIS: # Complete glossary.
8. PRESS: Enter.
9. LOOK FOR: Rehearse the interview-ready definitions only after explaining each term in your own words.
10. DO NOT: Open or create a separate journal, glossary, or troubleshooting file.
11. DO NOT CONTINUE UNTIL: Every interview claim points to a real phase, commit, run result, or screenshot.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.2.1 — Read the 30-second version once, then close it and explain the business problem, architecture, and outcome in

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Read the 30-second version once, then close it and explain the business problem, architecture, and outcome in . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Read the 30-second version once, then close it and explain the business problem, architecture, and outcome in your own words. Record and compare.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 1****Action**

Read the 30-second version once, then close it and explain the business problem, architecture, and outcome in your own words. Record and compare.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.2.2 — Practice the 2-minute story: problem; sources; local-first extraction; Databricks medallion layers; dimensiona

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Practice the 2-minute story: problem; sources; local-first extraction; Databricks medallion layers; dimensiona. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Practice the 2-minute story: problem; sources; local-first extraction; Databricks medallion layers; dimensional model; Jobs/quality gate; SQL Server serving; Power BI; verification; limitation.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 2

Action

Practice the 2-minute story: problem; sources; local-first extraction; Databricks medallion layers; dimensional model; Jobs/quality gate; SQL Server serving; Power BI; verification; limitation.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.



I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.2.3 — Use the overall architecture diagram for a 5-minute walkthrough. Point to each arrow and state data format, bo

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Use the overall architecture diagram for a 5-minute walkthrough. Point to each arrow and state data format, bo. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.

3. Perform only this immediate task: Use the overall architecture diagram for a 5-minute walkthrough. Point to each arrow and state data format, boundary, orchestration owner, batch identifier, and failure control.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 3****Action**

Use the overall architecture diagram for a 5-minute walkthrough. Point to each arrow and state data format, boundary, orchestration owner, batch identifier, and failure control.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.2.4 — Use the 10-minute technical script to show one source table, generated dirty record, Bronze metadata, Silver f

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Use the 10-minute technical script to show one source table, generated dirty record, Bronze metadata, Silver f. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Use the 10-minute technical script to show one source table, generated dirty record, Bronze metadata, Silver fix/quarantine, SCD2 dimension version, fact grain, quality result, Job dependency, warehouse reconciliation, and DAX validation.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 4

Action

Use the 10-minute technical script to show one source table, generated dirty record, Bronze metadata, Silver fix/quarantine, SCD2 dimension version, fact grain, quality result, Job dependency, warehouse reconciliation, and DAX validation.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.2.5 — Answer all 40+ questions aloud before reading the optional answer later in this workbook. For each, add one ex

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Answer all 40+ questions aloud before reading the optional answer later in this workbook. For each, add one ex. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Answer all 40+ questions aloud before reading the optional answer later in this workbook. For each, add one example from your own run and one follow-up you still find difficult.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 5****Action**

Answer all 40+ questions aloud before reading the optional answer later in this workbook. For each, add one example from your own run and one follow-up you still find difficult.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.2.6 — Complete the design-defense worksheet for medallion architecture, two source databases, local-first manual bou

Application:

Power BI Desktop

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Complete the design-defense worksheet for medallion architecture, two source databases, local-first manual bou. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Complete the design-defense worksheet for medallion architecture, two source databases, local-first manual boundary, Delta, SCD2, star schema, Gold publication, Power BI Import, and CI scope.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Power BI Desktop. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 6****Action**

Complete the design-defense worksheet for medallion architecture, two source databases, local-first manual boundary, Delta, SCD2, star schema, Gold publication, Power BI Import, and CI scope.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.2.7 — Select a genuine failure from your journal and write a STAR story. The Result must be evidence-backed; “I lear

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Select a genuine failure from your journal and write a STAR story. The Result must be evidence-backed; “I lear. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Select a genuine failure from your journal and write a STAR story. The Result must be evidence-backed; “I learned” alone is not a system result.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 7

Action

Select a genuine failure from your journal and write a STAR story. The Result must be evidence-backed; “I learned” alone is not a system result.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.2.8 — Practice questions that expose limitations: why Databricks cannot reach localhost, what remains manual, what f

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Practice questions that expose limitations: why Databricks cannot reach localhost, what remains manual, what f. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Practice questions that expose limitations: why Databricks cannot reach localhost, what remains manual, what free-tier quotas affected, and which transformations began as TODOs.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 8****Action**

Practice questions that expose limitations: why Databricks cannot reach localhost, what remains manual, what free-tier quotas affected, and which transformations began as TODOs.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.2.9 — Practice scale changes: 100x data, CDC, streaming, partitioning, orchestration concurrency, data contracts, ob

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Practice scale changes: 100x data, CDC, streaming, partitioning, orchestration concurrency, data contracts, ob. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Practice scale changes: 100x data, CDC, streaming, partitioning, orchestration concurrency, data contracts, observability, cloud networking, security, and semantic performance.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 9

Action

Practice scale changes: 100x data, CDC, streaming, partitioning, orchestration concurrency, data contracts, observability, cloud networking, security, and semantic performance.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.2.10 — Ask a friend or use a second pass to interrupt with follow-ups. Do not continue a memorized script; answer the

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Ask a friend or use a second pass to interrupt with follow-ups. Do not continue a memorized script; answer the. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Ask a friend or use a second pass to interrupt with follow-ups. Do not continue a memorized script; answer the exact question, then reconnect to evidence.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 10****Action**

Ask a friend or use a second pass to interrupt with follow-ups. Do not continue a memorized script; answer the exact question, then reconnect to evidence.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.2.11 — Score the final rubric. Any concept below “can explain and prove” becomes a learning task with a file, test, a

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Score the final rubric. Any concept below “can explain and prove” becomes a learning task with a file, test, a. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Score the final rubric. Any concept below “can explain and prove” becomes a learning task with a file, test, and teach-back due before claiming readiness.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 11

Action

Score the final rubric. Any concept below “can explain and prove” becomes a learning task with a file, test, and teach-back due before claiming readiness.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.2.12 — Update resume/LinkedIn statements after this exercise so every verb and metric is something you can defend und

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Update resume/LinkedIn statements after this exercise so every verb and metric is something you can defend under. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Update resume/LinkedIn statements after this exercise so every verb and metric is something you can defend under follow-up questioning.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 12****Action**

Update resume/LinkedIn statements after this exercise so every verb and metric is something you can defend under follow-up questioning.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.2.C1 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
code .\docs\interview_walkthrough.md
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** code .\docs\interview_walkthrough.md — code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; docs is a name, keyword, argument, identifier, or literal whose pro-

ject-specific role is described in the surrounding explanation; `interview_walkthrough.md` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

Action 21.2.C2 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

Open the complete glossary section in this workbook

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** Open the complete glossary section in this workbook — Open is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; complete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; glossary is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; section is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; this is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; workbook is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use

git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

Action 21.2.C3 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
(Select-String -Path .\docs\interview_walkthrough.md -Pattern '^## [0-9]+\.').Count
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** (Select-String -Path .\docs\interview_walkthrough.md -Pattern '^## [0-9]+\.').Count — (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Select-String is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; . is a path, relative location, branch-style name, or file argument; verify every character before running it; docs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; interview_walkthrough.md is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Pattern is a command option or flag; its exact behavior is defined beside the command; '^## [0-9]+\.' is a path, relative location, branch-style name, or file argument; verify every character before running it;). is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use git status and git branch --show-current; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

Action 21.2.C4 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Select-String -Path the learning-journal fields in this workbook -Pattern 'What broke|How I diagnosed|What I learned'
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `Select-String -Path the learning-journal fields in this workbook -Pattern 'What broke|How I diagnosed|What I learned'` — `Select-String` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `learning-journal` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fields` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `this` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `workbook` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Pattern` is a command option or flag; its exact behavior is defined beside the command; `'What broke|How I diagnosed|What I learned'` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

Action 21.2.C5 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.

2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
Get-ChildItem .\docs\images\*.svg | Select-Object Name
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `Get-ChildItem .\docs\images\*.svg | Select-Object Name` — `Get-ChildItem` lists files and folders; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `docs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `images` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `svg` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Select-Object` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Name` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 21.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]

☐

I distinguished typed text from observed output — [STATUS]

☐

I recorded differences caused by versions/tier — [STATUS]

☐

I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 21 starting-screen checklist and re-verify the active context.

Lesson 21.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 21 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — docs/

Exact folder name: docs

Exact parent: repository root

Why it is needed now: Phase 21 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.

3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: docs.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 21.D1.1 — Create the folder docs

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop — New-Item creates a file or folder; -ItemType is a command option or flag; its exact behavior is defined beside the command; Directory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\docs" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or

flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

Folder-creation decision branches

If the folder is created

Run `Test-Path -LiteralPath ".\docs" -PathType Container` and require True.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\docs" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\docs" -PathType Container`.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — docs/interview_walkthrough.md

Exact filename: interview_walkthrough.md

Filename extension: .md. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: docs relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 21.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in docs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: interview_walkthrough.md.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled interview_walkthrough.md and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 21.F1.1 — Create the file docs/interview_walkthrough.md from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.

4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\interview_walkthrough.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\docs\interview_walkthrough.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs\interview_walkthrough.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\docs\interview_walkthrough.md" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/interview_walkthrough.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use TODO(USER) where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under docs.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `docs/interview_walkthrough.md`.
3. In PowerShell at the repository root, run `Test-Path ".\docs\interview_walkthrough.md"` and require True.
4. Run `git status --short -- "docs/interview_walkthrough.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for docs/interview_walkthrough.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: `docs/interview_walkthrough.md`

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 21.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Interview and Teach-Back Preparation
```

```
## 30-second explanation
```

```
Northstar Retail is a fictional omnichannel data platform I am building from local SQL Server, PostgreSQL, and file feeds. I generate reproducible synthetic data, extract it in traceable batches, process Bronze, Silver, and Gold Delta layers in Databricks Jobs, publish a dimensional warehouse back to SQL Server, and define Power BI measures. I only present results after reconciliation and data-quality gates pass.
```

```
## 2-minute explanation
```

```
Leadership's reports disagree because operational sales and inventory, digital behavior, marketing, shipping, and returns are separated. The local-first architecture keeps SQL Server and PostgreSQL on my Windows computer, uses Python for generation, load, manifest-driven extraction, and publication, and uses a manual Unity Catalog volume boundary because cloud Databricks cannot reach localhost. Bronze preserves source evidence, Silver standardizes and quarantines controlled defects, and Gold uses conformed dimensions, SCD Type 2 for customers/products, and facts with explicit grain. A Lakeflow Job enforces dependencies and blocks publication on failed tests. The validated Gold export is loaded transactionally into 'NorthstarRetail_DW', and Power BI uses a single-direction star schema and documented DAX measures. My proof is the batch manifest, job run, quality results, SQL reconciliation, and matching Power BI values—not file existence alone.
```

```
## 5-minute architecture walkthrough
```

```
1. Requirements: define stakeholders, grains, KPI formulas, date roles, quality expectations, and exclusions first.
```

2. ****Sources:**** normalized ERP in SQL Server, digital model in PostgreSQL, and file feeds. All records are fictional and reproducible.

3. ****Local orchestration boundary:**** Python extracts full or incremental batches with batch ID, row count, and checksum; I manually upload one immutable folder.

4. ****Lakehouse:**** Bronze appends/replaces the batch with audit metadata; Silver casts, cleans, deduplicates, conforms, and quarantines; Gold resolves SCD-aware dimensions and fact grains.

5. ****Workflow:**** environment, parallel Bronze tasks, dependent Silver domains, dimensions, facts, data-quality gate, publish, and final reconciliation.

6. ****Serving:**** download validated Gold, stage and load SQL Server warehouse in a transaction, then refresh Power BI.

7. ****Trust:**** reconcile source-to-Bronze, Bronze-to-Silver, Silver-to-Gold, revenue, duplicate reruns, and semantic measures.

8. ****Limitations:**** manual handoffs and simplified attribution are intentional Track A constraints; Track B covers managed networking and direct JDBC as an extension.

10-minute technical walkthrough outline

- Minute 0–1: business problem and definitions.
- Minute 1–2: source ERDs and generated quality fixtures.
- Minute 2–3: repository, configuration, credentials, and batch manifest.
- Minute 3–5: medallion contracts, example customer cleanup, and quarantine.
- Minute 5–6: SCD2 timeline and event-time dimension lookup.
- Minute 6–7: fact_sales grain, revenue arithmetic, unknown keys.
- Minute 7–8: Jobs dependency and failure/repair behavior.
- Minute 8–9: SQL Server publication and Power BI relationships/measures.
- Minute 9–10: verification evidence, limitations, and next scaling step.

Interview questions and beginner-friendly answers

1. What business problem does the project solve?

****Answer:**** It brings operational sales, digital behavior, batch files, inventory, shipping, returns, and marketing into a governed daily platform so different teams use the same definitions. The technical success criterion is not merely moving data; it is producing traceable, reconciled facts and reusable semantic measures.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.

2. Why use Bronze, Silver, and Gold?

****Answer:**** Each layer has a different contract. Bronze preserves the source and batch evidence, Silver applies explicit quality and conformance rules, and Gold presents stable dimensional grains. Separating them makes reruns, debugging, lineage, and ownership clearer than one giant transformation.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.

3. Why use both SQL Server and PostgreSQL?

****Answer:**** The fictional company has separate operational domains and the project demonstrates heterogeneous-source integration. It also teaches that SQL dialects, drivers, tools, and operational design differ; it is not an argument that every real company needs both.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.

4. Why is the source model normalized but the warehouse dimensional?

****Answer:**** Operational systems normalize data to support correct transactional updates and reduce repeated facts. Analytics favors dimensions and facts because users repeatedly filter, group, and aggregate; a star schema reduces join ambiguity and makes business grain visible.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.

5. What is the grain of fact_sales?

****Answer:**** One valid order item, identified by `order_item_id`. Quantity, revenue, discounts, cost, and profit are additive at that grain, and order-level metrics use distinct `order_id` rather than counting fact rows.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.

6. What is grain and why declare it first?

****Answer:**** Grain is exactly what one row represents. Without it, keys and measures can silently double-count. Declaring grain determines uniqueness tests, dimension lookups, additive behavior, and valid joins.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.

7. What is a surrogate key?

****Answer:**** A warehouse-controlled key such as `customer_key`. It is separate from the source `customer_number` and allows multiple historical versions, unknown members, and source-system integration without changing the natural business identifier.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.

8. How did you prevent duplicate loads?

****Answer:**** Each task receives one batch ID. Bronze replaces the same batch before appending; Silver uses batch replacement or merge; Gold enforces deterministic grain and uniqueness checks; the Job permits only one concurrent run. I verify a repair rerun leaves counts stable.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.

9. What makes a pipeline idempotent?

****Answer:**** Running the same input and batch again produces the same trusted state rather than an extra copy. Idempotency requires stable identities, controlled overwrite/merge scopes, transactions where appropriate, and tests—not merely a statement in documentation.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.

10. How did you handle SCD Type 2?

****Answer:**** I compare a hash of tracked attributes on the current dimension row. A change expires the old version with an end date and current flag false, then inserts a new current version. Facts look up the version whose effective dates contain the event date.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
11. Why Type 2 for customer and product?

****Answer:**** Historical customer tier/address attributes and product category/cost/descriptions can change, and past reports may need the attributes that were true at the event time. Type 1 would overwrite history. I choose tracked columns deliberately because Type 2 on every field creates noise.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
12. What is the unknown member?

****Answer:**** Dimension key 0 represents missing, invalid, or late-arriving dimension context. It keeps fact foreign keys valid and makes the problem measurable. It is not permission to ignore source quality; unknown-key rates are quality checks.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
13. How do you handle late-arriving dimensions?

****Answer:**** The fact can temporarily use key 0 while retaining the natural key and batch evidence. A later repair resolves the dimension and updates the fact under a controlled rule. Alternatively, a minimal inferred member can be created, but that policy must be explicit.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
14. How did you validate revenue?

****Answer:**** I use the same valid-line population and formula—quantity times unit price minus line discount—from Silver through Gold, SQL Server, and Power BI. I compare row counts and sums, define exclusions, and accept only documented rounding tolerance.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
15. What does a data-quality gate do?

****Answer:**** It stores test results and raises a task failure when a required rule fails. Publication depends on that task, so a bad batch cannot be exported just because a user clicks the next notebook.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
16. What kinds of dirty data are included?

****Answer:**** Controlled duplicate business keys, null/malformed emails, whitespace/case issues, invalid codes, impossible quantities, unknown references, mixed date formats, late records, out-of-order events, currency differences, and time-zone variations. A fixture records configured injections.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
17. Why intentionally generate dirty data?

****Answer:**** Real engineering is mostly about behavior when inputs are imperfect. Controlled defects make quality rules testable and repeatable without using real personal data. Clean-only fixtures would let transformations appear correct without proving rejection and quarantine paths.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
18. What is quarantine?

****Answer:**** A governed table or path for records that cannot enter a trusted layer under current rules. It retains the source values, rule, batch, and timestamp so owners can investigate. Quarantine counts must reconcile with layer input.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
19. What is the difference between ETL and ELT here?

****Answer:**** Local source loading and extraction include ETL-like movement, while Databricks primarily follows ELT: land raw data first, then transform in the lakehouse. The important point is where transformation occurs and which copy is preserved.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
20. What happens when a Databricks task fails?

****Answer:**** Dependent tasks do not run. I inspect the exact task log and quality/quarantine evidence, fix the cause in version control, rerun local checks, then use a repair run from the earliest affected task. Same-batch behavior must not duplicate data.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
21. Why can't Databricks connect to localhost?

****Answer:**** The Databricks compute runs in a remote environment; its localhost is not my Windows computer. Track A therefore uses an uploaded file batch. Track B needs a network-accessible managed source, firewall/private networking, supported driver, and secret-managed identity.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
22. Why is Track A partly manual?

****Answer:**** It is a low-cost, honest learning architecture. The manual boundary isolates local systems from cloud networking while still teaching source extraction, manifests, lakehouse layers, orchestration, publication, and consumption. I label the operational limitation rather than disguising it.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
23. What is a high-water mark?

****Answer:**** The latest source update value that a successful extraction has fully processed. The next incremental run requests rows greater than that value. It advances only after success, and tie-breaker keys may be needed when timestamps are not unique.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
24. How do you handle a failure halfway through extraction?

****Answer:**** The extractor writes to a unique batch folder and records status in a manifest. I do not advance the watermark until the batch is complete. A failed partial batch is not treated as trusted input; I rerun with the same logical bounds or a new controlled batch.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
25. Why use checksums?

****Answer:**** A checksum helps prove that the file uploaded or processed is the same bytes the extractor wrote. It detects accidental change or corruption; it does not prove the data is semantically correct.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
26. What is Delta Lake adding?

****Answer:**** Delta tables provide transactional table behavior over lake files, schema metadata, versioned changes, and merge/update patterns. In this project they support batch replacement, merge, reproducible table access, and governed medallion layers.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
27. Why use a star schema?

****Answer:**** It gives each business event a clear fact grain and shared descriptive dimensions. BI relationships are predictable, SQL is understandable, and measures aggregate correctly when keys and relationships are designed well.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
28. What are additive and non-additive measures?

****Answer:**** Revenue and quantity are generally additive across all relevant dimensions. Ratios such as margin percentage are recalculated from aggregated numerator and denominator. Inventory snapshots are not additive across time because summing daily stock double-counts the same inventory.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
29. How do you define return rate?

****Answer:**** Returned units divided by sold units for a compatible population. I must state whether date filtering is by purchase date or return date, and I do not divide return rows by orders unless the business explicitly wants an order-return incidence metric.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
30. How do you define on-time delivery?

****Answer:**** Delivered shipments with actual delivery date on or before promised date divided by delivered shipments with an actual date. The grain is shipment, not order, because one order may split into multiple shipments.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
31. How is ROAS limited?

****Answer:**** The beginner model links converting sessions to order IDs and is essentially simplified last-session attribution. It does not establish causality or allocate multi-touch credit. I check duplicate links and present the attribution assumption beside the metric.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
32. Why use a semantic layer?

****Answer:**** It centralizes relationships, date behavior, measure formulas, formatting, descriptions, and reusable business definitions. Otherwise each report author can compute a different version of revenue or return rate.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
33. Why prefer explicit DAX measures?

****Answer:**** They are named, documented, format-controlled, reusable, and testable. Implicit sums hide business logic and can encourage users to aggregate technical columns incorrectly.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
34. Import or DirectQuery for Track A?

****Answer:**** Import is the default because the local warehouse is modest, interactive performance is predictable, and refresh can be controlled after publication. DirectQuery has freshness advantages but adds source load, latency, feature, and modeling constraints.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
35. How do you avoid ambiguous Power BI relationships?

****Answer:**** I use one-to-many, single-direction dimension-to-fact relationships. Secondary shipment date roles are inactive and activated in specific measures if needed. I avoid direct fact-to-fact joins and bidirectional filtering without a tested reason.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
36. What is data lineage?

****Answer:**** Evidence connecting a report value back through a measure, serving table, Gold fact/dimension, Silver transformation, Bronze record, source file/table, and batch. This project uses mappings, batch IDs, source/file metadata, task order, and reconciliation results.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
37. How do you protect secrets?

****Answer:**** Passwords stay in ignored ``.env`` files; Databricks uses secret management or identities; logs/screenshots are redacted; CI scans common patterns. If a secret is committed, I revoke or rotate it before cleaning history.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
38. What does least privilege mean here?

****Answer:**** Each identity gets only the permissions needed: source extractor read-only, workflow access to its catalog/schema/volume, publisher stage/load rights, and analysts read-only semantic access. An administrator credential is not a normal runtime credential.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
39. How does CI/CD help this project?

****Answer:**** The free GitHub Actions workflow checks structure, SQL dialect labels, notebook syntax/JSON, secret patterns, relative links, linting, and local tests. It cannot execute SQL Server or Databricks without paid/secured infrastructure, so those gates remain documented run evidence.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
40. What would change at 100 times the data volume?

****Answer:**** I would profile bottlenecks before redesigning. Likely changes include source-side partitioned/incremental extraction, bulk copy, optimized file sizes, partition/clustering choices, incremental Gold, stronger orchestration observability, scalable serving mode, and cost/performance testing. I would not simply use the portfolio scale setting.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
41. Which parts are batch and what could stream?

****Answer:**** All required phases are daily batch. Web events, inventory movements, and shipment tracking could become streaming, but dimensions, late data, deduplication, watermarks, cost, and exactly-once expectations would need a separate design.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
42. What are the limitations of the local-first track?

****Answer:**** Manual handoffs, no real-time operation, local service availability, no production identity/network design, simplified attribution, USD-only reporting, and incomplete production observability. Those are explicitly documented rather than presented as enterprise production readiness.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
43. How would you add CDC?

****Answer:**** Choose source-supported change data capture, retain operation type and source ordering metadata in Bronze, deduplicate by source position, apply deletes deliberately, and test replay/idempotency. It requires database configuration and privileges beyond the beginner path.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
44. How would you handle schema drift?

****Answer:**** Bronze may permit intentional additive columns while recording schema; Silver uses explicit selections/casts and fails or quarantines unsupported changes. A schema-drift test alerts before Gold and semantic contracts silently change.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.
45. What did you personally implement versus use as reference material?

****Answer:**** The workbook provides curriculum, boilerplate, patterns, TODO skeletons, tests, and reference answers. I only claim tasks after I type/adapt them, run them in my environment, capture evidence, and explain them. The final definition-of-done distinguishes reference material from personal verification.

****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer.

Defend every design choice worksheet

For each decision, complete all columns before an interview.

Decision	Requirement it serves	Alternative	Why not chosen now	Failure mode	Test/evidence	What would change at scale
Manual volume handoff	Low-cost local sources	Direct JDBC	No safe route to localhost	Wrong/missing batch	Manifest + environment check	Cloud-managed sources/network
SCD2 customer	Historical attributes	Type 1	Would rewrite history	Overlap/multiple current rows	SCD tests	Identity resolution/MDM
[ADD DECISION]						

STAR troubleshooting story template

- ****Situation:**** What business/pipeline context existed? Include phase and batch, not confidential details.
- ****Task:**** What trustworthy outcome were you responsible for?
- ****Action:**** What evidence did you inspect, how did you isolate the cause, what code/config/test changed, and how did you protect data?
- ****Result:**** What verified test/reconciliation passed? Use real numbers only after your run.
- ****Reflection:**** What prevention or monitoring did you add?

Final self-assessment rubric

Score each 0–3: 0 not started, 1 can follow, 2 can explain and reproduce, 3 can adapt and defend.

Area	0	1	2	3
Requirements/KPIs	Cannot define	Reads definitions	Reproduces and validates	Resolves ambiguity with stakeholders
SQL/source design	Cannot navigate	Runs provided DDL	Explains keys/indexes/dialects	Adapts model safely
Python/generation	Cannot run	Runs quick mode	Explains functions/seed/FKS	Adds a domain and tests
Ingestion/manifest	Cannot trace batch	Follows steps	Explains checksums/watermarks	Repairs/replays safely
Spark/Delta	Cannot identify layers	Runs notebooks	Explains transformations/merge	Tunes and handles drift
Dimensional model	Cannot state grain	Names facts/dims	Defends grain/SCD/unknowns	Evolves model without breaking history
Quality/reconciliation	Relies on screenshots	Runs tests	Explains gates and tie-outs	Designs new risk-based checks
Jobs/operations	Clicks run	Follows graph	Repairs failed run safely	Designs alerts and operational SLOs
Power BI	Builds visuals only	Adds measures	Validates context/relationships	Diagnoses complex totals/roles
Security/cost/Git	Uses defaults	Follows checklist	Explains least privilege/CI	Designs production controls
Communication	Recites tools	Gives overview	Tells evidence-based story	Handles follow-up tradeoffs calmly

Line-by-line explanation

- **Line 1:** # Interview and Teach-Back Preparation is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** ## 30-second explanation is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** Northstar Retail is a fictional omnichannel data platform I am building from local SQL Server, PostgreSQL, and file feeds. I generate reproducible synthetic data, extract it in traceable batches, process Bronze, Silver, and Gold Delta layers in Databricks Jobs, publish a dimensional warehouse back to SQL Server, and define Power BI measures. I only present results after reconciliation and data-quality gates pass. — Northstar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Retail is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fictional is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; omnichannel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; platform is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; I is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; am is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; building is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; from is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** ## 2-minute explanation is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 8:** blank line used only to separate logical sections for readability.
- **Line 9:** Leadership's reports disagree because operational sales and inventory, digital behavior, marketing, shipping, and returns are separated. The local-first architecture keeps SQL Server and PostgreSQL on my Windows computer, uses Python for generation, load, manifest-driven extraction, and publication, and uses a manual Unity Catalog volume boundary because cloud Databricks cannot reach localhost. Bronze preserves source evidence, Silver standardizes and quarantines controlled defects, and Gold uses conformed dimensions, SCD Type 2 for customers/products, and facts with explicit grain. A Lakeflow Job enforces dependencies and blocks publication on failed tests. The validated Gold export is loaded transactionally into \NorthstarRetail_DW`, and Power BI uses a single-direction star schema and documented DAX measures. My proof is the batch manifest, job run, quality results, SQL reconciliation, and matching Power BI values—not file existence alone.—Leadership is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reports is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; disagree is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; because is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; operational is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; inventory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; digital is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; behavior is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 10:** blank line used only to separate logical sections for readability.
- **Line 11:** ## 5-minute architecture walkthrough is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 12:** blank line used only to separate logical sections for readability.
- **Line 13:** 1. ****Requirements:**** define stakeholders, grains, KPI formulas, date roles, quality expectations, and exclusions first. — 1 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Requirements is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; define is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stakeholders is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; grains is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; KPI is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; formulas is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 14:** 2. ****Sources:**** normalized ERP in SQL Server, digital model in PostgreSQL, and file feeds. All records are fictional and reproducible. — 2 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Sources is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; normalized is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ERP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; digital is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 15:** 3. ****Local orchestration boundary:**** Python extracts full or incremental batches with batch ID, row count, and checksum; I manually upload one immutable folder. — 3 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Local is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; orchestration is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; boundary is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Python is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; extracts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; full is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; incremental is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 16:** 4. ****Lakehouse:**** Bronze appends/replaces the batch with audit metadata; Silver casts, cleans, deduplicates, conforms, and quarantines; Gold resolves SCD-aware dimensions and fact grains. — 4 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Lakehouse is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Bronze is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; appends is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; replaces is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the

surrounding explanation; with is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 17:** 5. ****Workflow:**** environment, parallel Bronze tasks, dependent Silver domains, dimensions, facts, data-quality gate, publish, and final reconciliation. — 5 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Workflow is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; environment is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; parallel is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Bronze is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tasks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dependent is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 18:** 6. ****Serving:**** download validated Gold, stage and load SQL Server warehouse in a transaction, then refresh Power BI. — 6 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Serving is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; download is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; validated is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stage is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; load is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 19:** 7. ****Trust:**** reconcile source-to-Bronze, Bronze-to-Silver, Silver-to-Gold, revenue, duplicate reruns, and semantic measures. — 7 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Trust is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reconcile is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source-to-Bronze is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Bronze-to-Silver is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Silver-to-Gold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 20:** 8. ****Limitations:**** manual handoffs and simplified attribution are intentional Track A constraints; Track B covers managed networking and direct JDBC as an extension. — 8 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Limitations is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; manual is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; handoffs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; simplified is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; attribution is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; intentional is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 21:** blank line used only to separate logical sections for readability.
- **Line 22:** ## 10-minute technical walkthrough outline is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 23:** blank line used only to separate logical sections for readability.
- **Line 24:** - Minute 0-1: business problem and definitions. — - is a command option or flag; its exact behavior is defined beside the command; Minute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; 1 is a numeric literal used by the command or expression; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; business is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; problem is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; definitions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 25:** - Minute 1-2: source ERDs and generated quality fixtures. — - is a command option or flag; its exact behavior is defined beside the command; Minute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; 2 is a numeric literal used by the command or expression; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ERDs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; generated is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quality is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fixtures is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 26:** - Minute 2-3: repository, configuration, credentials, and batch manifest. — - is a command option or flag; its exact behavior is defined beside the command; Minute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 2 is a numeric literal used by the command or expression; 3 is a numeric literal used by the command or expression; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; repository is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; configuration is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; credentials is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 27:** - Minute 3-5: medallion contracts, example customer cleanup, and quarantine. — - is a command option or flag; its exact behavior is defined beside the command; Minute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 3 is a numeric literal used by the command or expression; 5 is a numeric literal used by the command or expression; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; medallion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; contracts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; example is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cleanup is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 28:** - Minute 5–6: SCD2 timeline and event-time dimension lookup. — - is a command option or flag; its exact behavior is defined beside the command; Minute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 5 is a numeric literal used by the command or expression; 6 is a numeric literal used by the command or expression; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SCD2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timeline is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; event-time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dimension is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; lookup is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 29:** - Minute 6–7: fact_sales grain, revenue arithmetic, unknown keys. — - is a command option or flag; its exact behavior is defined beside the command; Minute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 6 is a numeric literal used by the command or expression; 7 is a numeric literal used by the command or expression; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact_sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; grain is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; arithmetic is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; unknown is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 30:** - Minute 7–8: Jobs dependency and failure/repair behavior. — - is a command option or flag; its exact behavior is defined beside the command; Minute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 7 is a numeric literal used by the command or expression; 8 is a numeric literal used by the command or expression; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Jobs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dependency is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; failure is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; repair is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; behavior is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 31:** - Minute 8–9: SQL Server publication and Power BI relationships/measures. — - is a command option or flag; its exact behavior is defined beside the command; Minute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 8 is a numeric literal used by the command or expression; 9 is a numeric literal used by the command or expression; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; publication is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Power is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BI is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; relationships is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 32:** - Minute 9–10: verification evidence, limitations, and next scaling step. — - is a command option or flag; its exact behavior is defined beside the command; Minute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 9 is a numeric literal used by the command or expression; 10 is a numeric literal used by the command or expression; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; verification is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; evidence is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; limitations is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; next is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 33:** blank line used only to separate logical sections for readability.
- **Line 34:** # Interview questions and beginner-friendly answers is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 35:** blank line used only to separate logical sections for readability.
- **Line 36:** ## 1. What business problem does the project solve? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 37:** blank line used only to separate logical sections for readability.
- **Line 38:** **Answer:** It brings operational sales, digital behavior, batch files, inventory, shipping, returns, and marketing into a governed daily platform so different teams use the same definitions. The technical success criterion is not merely moving data; it is producing traceable, reconciled facts and reusable semantic measures. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; It is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; brings is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; operational is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sales is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; digital is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; behavior is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 39:** blank line used only to separate logical sections for readability.
- **Line 40:** **Likely follow-up:** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 41:** ## 2. Why use Bronze, Silver, and Gold? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 42:** blank line used only to separate logical sections for readability.
- **Line 43:** **Answer:** Each layer has a different contract. Bronze preserves the source and batch evidence, Silver applies explicit quality and conformance rules, and Gold presents stable dimensional grains. Separating them makes reruns, debugging, lineage, and ownership clearer than one giant transformation. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Each is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; layer is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; has is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; different is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; contract is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Bronze is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; preserves is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 44:** blank line used only to separate logical sections for readability.
- **Line 45:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **:**** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 46:** **## 3. Why use both SQL Server and PostgreSQL?** is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 47:** blank line used only to separate logical sections for readability.
- **Line 48:** ****Answer:**** The fictional company has separate operational domains and the project demonstrates heterogeneous-source integration. It also teaches that SQL dialects, drivers, tools, and operational design differ; it is not an argument that every real company needs both. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **:**** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; The is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fictional is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; company is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; has is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; separate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; operational is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; domains is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 49:** blank line used only to separate logical sections for readability.
- **Line 50:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **:**** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific

role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 51:** ## 4. Why is the source model normalized but the warehouse dimensional? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 52:** blank line used only to separate logical sections for readability.
- **Line 53:** **Answer:** Operational systems normalize data to support correct transactional updates and reduce repeated facts. Analytics favors dimensions and facts because users repeatedly filter, group, and aggregate; a star schema reduces join ambiguity and makes business grain visible. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Operational is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; systems is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; normalize is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; to is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; support is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; correct is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; transactional is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; updates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 54:** blank line used only to separate logical sections for readability.
- **Line 55:** **Likely follow-up:** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 56:** ## 5. What is the grain of fact_sales? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 57:** blank line used only to separate logical sections for readability.
- **Line 58:** **Answer:** One valid order item, identified by order_item_id. Quantity, revenue, discounts, cost, and profit are additive at that grain, and order-level metrics use distinct order_id rather than counting fact rows. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; One is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; item is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; identified is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; by is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order_item_id is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.

- **Line 59:** blank line used only to separate logical sections for readability.
- **Line 60:** `**Likely follow-up:**` Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Likely` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `follow-up` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Give` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `concrete` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 61:** `## 6. What is grain and why declare it first?` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 62:** blank line used only to separate logical sections for readability.
- **Line 63:** `**Answer:**` Grain is exactly what one row represents. Without it, keys and measures can silently double-count. Declaring grain determines uniqueness tests, dimension lookups, additive behavior, and valid joins. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Answer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Grain` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exactly` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `what` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `represents` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; `verify every character before running it`; `Without` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 64:** blank line used only to separate logical sections for readability.
- **Line 65:** `**Likely follow-up:**` Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Likely` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `follow-up` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Give` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `concrete` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 66:** `## 7. What is a surrogate key?` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 67:** blank line used only to separate logical sections for readability.
- **Line 68:** `**Answer:**` A warehouse-controlled key such as `customer_key`. It is separate from the source `customer_number` and allows multiple historical versions, unknown members, and source-system integration without changing the natural business identifier. — `**` is a name, keyword, argument, identifier, or literal whose

project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; A is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; warehouse-controlled is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; such is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; as is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer_key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; It is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 69:** blank line used only to separate logical sections for readability.
- **Line 70:** **Likely follow-up:** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 71:** ## 8. How did you prevent duplicate loads? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 72:** blank line used only to separate logical sections for readability.
- **Line 73:** **Answer:** Each task receives one batch ID. Bronze replaces the same batch before appending; Silver uses batch replacement or merge; Gold enforces deterministic grain and uniqueness checks; the Job permits only one concurrent run. I verify a repair rerun leaves counts stable. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Each is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; task is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; receives is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ID is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Bronze is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; replaces is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 74:** blank line used only to separate logical sections for readability.
- **Line 75:** **Likely follow-up:** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a

name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 76:** ## 9. What makes a pipeline idempotent? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 77:** blank line used only to separate logical sections for readability.
- **Line 78:** **Answer:** Running the same input and batch again produces the same trusted state rather than an extra copy. Idempotency requires stable identities, controlled overwrite/merge scopes, transactions where appropriate, and tests—not merely a statement in documentation. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Running is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; same is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; input is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; again is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; produces is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 79:** blank line used only to separate logical sections for readability.
- **Line 80:** **Likely follow-up:** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 81:** ## 10. How did you handle SCD Type 2? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 82:** blank line used only to separate logical sections for readability.
- **Line 83:** **Answer:** I compare a hash of tracked attributes on the current dimension row. A change expires the old version with an end date and current flag false, then inserts a new current version. Facts look up the version whose effective dates contain the event date. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; I is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; compare is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; hash is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; of is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

role is described in the surrounding explanation; tracked is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; attributes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 84:** blank line used only to separate logical sections for readability.
- **Line 85:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **: **** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 86:** **## 11.** Why Type 2 for customer and product? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 87:** blank line used only to separate logical sections for readability.
- **Line 88:** ****Answer:**** Historical customer tier/address attributes and product category/cost/descriptions can change, and past reports may need the attributes that were true at the event time. Type 1 would overwrite history. I choose tracked columns deliberately because Type 2 on every field creates noise. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **: **** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Historical is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tier is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; address is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; attributes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; category is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 89:** blank line used only to separate logical sections for readability.
- **Line 90:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **: **** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 91:** ## 12. What is the unknown member? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 92:** blank line used only to separate logical sections for readability.
- **Line 93:** ****Answer:**** Dimension key 0 represents missing, invalid, or late-arriving dimension context. It keeps fact foreign keys valid and makes the problem measurable. It is not permission to ignore source quality; unknown-key rates are quality checks. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Dimension is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; represents is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; missing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; invalid is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 94:** blank line used only to separate logical sections for readability.
- **Line 95:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 96:** ## 13. How do you handle late-arriving dimensions? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 97:** blank line used only to separate logical sections for readability.
- **Line 98:** ****Answer:**** The fact can temporarily use key 0 while retaining the natural key and batch evidence. A later repair resolves the dimension and updates the fact under a controlled rule. Alternatively, a minimal inferred member can be created, but that policy must be explicit. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; The is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; can is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; temporarily is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; use is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; key is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; while is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; retaining is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 99:** blank line used only to separate logical sections for readability.
- **Line 100:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 101:** ## 14. How did you validate revenue? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 102:** blank line used only to separate logical sections for readability.
- **Line 103:** **Answer:** I use the same valid-line population and formula-quantity times unit price minus line discount—from Silver through Gold, SQL Server, and Power BI. I compare row counts and sums, define exclusions, and accept only documented rounding tolerance. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; I is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; use is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; same is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid-line is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; population is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; formula is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quantity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 104:** blank line used only to separate logical sections for readability.
- **Line 105:** **Likely follow-up:** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 106:** ## 15. What does a data-quality gate do? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 107:** blank line used only to separate logical sections for readability.
- **Line 108:** **Answer:** It stores test results and raises a task failure when a required rule fails. Publication depends on that task, so a bad batch cannot be exported just because a user clicks the next notebook. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-

specific role is described in the surrounding explanation; It is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stores is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; results is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; raises is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; task is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; failure is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 109:** blank line used only to separate logical sections for readability.
- **Line 110:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **:**** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 111:** **## 16.** What kinds of dirty data are included? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 112:** blank line used only to separate logical sections for readability.
- **Line 113:** ****Answer:**** Controlled duplicate business keys, null/malformed emails, whitespace/case issues, invalid codes, impossible quantities, unknown references, mixed date formats, late records, out-of-order events, currency differences, and time-zone variations. A fixture records configured injections. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **:**** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Controlled is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; duplicate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; business is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; keys is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; null is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; malformed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; emails is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 114:** blank line used only to separate logical sections for readability.
- **Line 115:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **:**** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding ex-

planation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 116:** ## 17. Why intentionally generate dirty data? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 117:** blank line used only to separate logical sections for readability.
- **Line 118:** **Answer:** Real engineering is mostly about behavior when inputs are imperfect. Controlled defects make quality rules testable and repeatable without using real personal data. Clean-only fixtures would let transformations appear correct without proving rejection and quarantine paths. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Real is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; engineering is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mostly is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; about is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; behavior is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; when is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; inputs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 119:** blank line used only to separate logical sections for readability.
- **Line 120:** **Likely follow-up:** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 121:** ## 18. What is quarantine? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 122:** blank line used only to separate logical sections for readability.
- **Line 123:** **Answer:** A governed table or path for records that cannot enter a trusted layer under current rules. It retains the source values, rule, batch, and timestamp so owners can investigate. Quarantine counts must reconcile with layer input. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; A is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; governed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; path is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; records is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; that is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cannot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 124:** blank line used only to separate logical sections for readability.
- **Line 125:** `**Likely follow-up:**` Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Likely` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `follow-up` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Give` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `concrete` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 126:** `## 19. What is the difference between ETL and ELT here?` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 127:** blank line used only to separate logical sections for readability.
- **Line 128:** `**Answer:**` Local source loading and extraction include ETL-like movement, while Databricks primarily follows ELT: land raw data first, then transform in the lakehouse. The important point is where transformation occurs and which copy is preserved. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Answer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Local` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `loading` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `extraction` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `include` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ETL-like` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `movement` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 129:** blank line used only to separate logical sections for readability.
- **Line 130:** `**Likely follow-up:**` Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Likely` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `follow-up` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Give` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `concrete` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 131:** `## 20. What happens when a Databricks task fails?` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 132:** blank line used only to separate logical sections for readability.
- **Line 133:** ****Answer:**** Dependent tasks do not run. I inspect the exact task log and quality/quarantine evidence, fix the cause in version control, rerun local checks, then use a repair run from the earliest affected task. Same-batch behavior must not duplicate data. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **;** ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Dependent is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tasks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; do is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **.** is a path, relative location, branch-style name, or file argument; verify every character before running it; I is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; inspect is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 134:** blank line used only to separate logical sections for readability.
- **Line 135:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **;** ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **,** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **,** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 136:** **##** 21. Why can't Databricks connect to localhost? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 137:** blank line used only to separate logical sections for readability.
- **Line 138:** ****Answer:**** The Databricks compute runs in a remote environment; its localhost is not my Windows computer. Track A therefore uses an uploaded file batch. Track B needs a network-accessible managed source, firewall/private networking, supported driver, and secret-managed identity. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **;** ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; The is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Databricks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; compute is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; runs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remote is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; environment is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; its is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 139:** blank line used only to separate logical sections for readability.
- **Line 140:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-

specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 141:** ## 22. Why is Track A partly manual? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 142:** blank line used only to separate logical sections for readability.
- **Line 143:** **Answer:** It is a low-cost, honest learning architecture. The manual boundary isolates local systems from cloud networking while still teaching source extraction, manifests, lakehouse layers, orchestration, publication, and consumption. I label the operational limitation rather than disguising it. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; It is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; low-cost is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; honest is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; learning is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; architecture is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.
- **Line 144:** blank line used only to separate logical sections for readability.
- **Line 145:** **Likely follow-up:** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 146:** ## 23. What is a high-water mark? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 147:** blank line used only to separate logical sections for readability.
- **Line 148:** **Answer:** The latest source update value that a successful extraction has fully processed. The next incremental run requests rows greater than that value. It advances only after success, and tie-breaker keys may be needed when timestamps are not unique. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; The is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; latest is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; update is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; that is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; successful is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; extraction is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 149:** blank line used only to separate logical sections for readability.
- **Line 150:** `**Likely follow-up:**` Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Likely` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `follow-up` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Give` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `concrete` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 151:** `## 24. How do you handle a failure halfway through extraction?` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 152:** blank line used only to separate logical sections for readability.
- **Line 153:** `**Answer:**` The extractor writes to a unique batch folder and records status in a manifest. I do not advance the watermark until the batch is complete. A failed partial batch is not treated as trusted input; I rerun with the same logical bounds or a new controlled batch. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Answer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `The` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `extractor` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `writes` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `to` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `unique` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `folder` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 154:** blank line used only to separate logical sections for readability.
- **Line 155:** `**Likely follow-up:**` Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Likely` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `follow-up` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Give` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `concrete` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the

surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 156:** ## 25. Why use checksums? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 157:** blank line used only to separate logical sections for readability.
- **Line 158:** **Answer:** A checksum helps prove that the file uploaded or processed is the same bytes the extractor wrote. It detects accidental change or corruption; it does not prove the data is semantically correct. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; A is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; checksum is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; helps is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; prove is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; that is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; uploaded is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 159:** blank line used only to separate logical sections for readability.
- **Line 160:** **Likely follow-up:** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 161:** ## 26. What is Delta Lake adding? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 162:** blank line used only to separate logical sections for readability.
- **Line 163:** **Answer:** Delta tables provide transactional table behavior over lake files, schema metadata, versioned changes, and merge/update patterns. In this project they support batch replacement, merge, reproducible table access, and governed medallion layers. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Delta is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tables is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; provide is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; transactional is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; behavior is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; over is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; lake is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; files is a name, keyword,

argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 164:** blank line used only to separate logical sections for readability.
- **Line 165:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **Likely** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **follow-up** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **:**** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **Give** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **one** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **concrete** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **file** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **,** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **query** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **,** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **test** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 166:** **## 27. Why use a star schema?** is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 167:** blank line used only to separate logical sections for readability.
- **Line 168:** ****Answer:**** It gives each business event a clear fact grain and shared descriptive dimensions. BI relationships are predictable, SQL is understandable, and measures aggregate correctly when keys and relationships are designed well. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **Answer** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **:**** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **It** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **gives** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **each** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **business** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **event** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **a** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **clear** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **fact** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **grain** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 169:** blank line used only to separate logical sections for readability.
- **Line 170:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **Likely** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **follow-up** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **:**** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **Give** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **one** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **concrete** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **file** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **,** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **query** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **,** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **test** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 171:** **## 28. What are additive and non-additive measures?** is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 172:** blank line used only to separate logical sections for readability.

- **Line 173:** ****Answer:**** Revenue and quantity are generally additive across all relevant dimensions. Ratios such as margin percentage are recalculated from aggregated numerator and denominator. Inventory snapshots are not additive across time because summing daily stock double-counts the same inventory. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **;** ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quantity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; generally is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; additive is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; across is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; all is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; relevant is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 174:** blank line used only to separate logical sections for readability.
- **Line 175:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **;** ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **,** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **,** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 176:** **##** 29. How do you define return rate? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 177:** blank line used only to separate logical sections for readability.
- **Line 178:** ****Answer:**** Returned units divided by sold units for a compatible population. I must state whether date filtering is by purchase date or return date, and I do not divide return rows by orders unless the business explicitly wants an order-return incidence metric. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **;** ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Returned is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; units is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; divided is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; by is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; units is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; compatible is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 179:** blank line used only to separate logical sections for readability.
- **Line 180:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **;** ****** is a name, keyword, argument, identifier,

or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 181:** ## 30. How do you define on-time delivery? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 182:** blank line used only to separate logical sections for readability.
- **Line 183:** **Answer:** Delivered shipments with actual delivery date on or before promised date divided by delivered shipments with an actual date. The grain is shipment, not order, because one order may split into multiple shipments. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Delivered is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; shipments is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; with is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; actual is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; delivery is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; before is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 184:** blank line used only to separate logical sections for readability.
- **Line 185:** **Likely follow-up:** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 186:** ## 31. How is ROAS limited? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 187:** blank line used only to separate logical sections for readability.
- **Line 188:** **Answer:** The beginner model links converting sessions to order IDs and is essentially simplified last-session attribution. It does not establish causality or allocate multi-touch credit. I check duplicate links and present the attribution assumption beside the metric. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; The is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; beginner is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; model is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; links is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; converting is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sessions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; to is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; IDs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 189:** blank line used only to separate logical sections for readability.
- **Line 190:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 191:** **## 32. Why use a semantic layer?** is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 192:** blank line used only to separate logical sections for readability.
- **Line 193:** ****Answer:**** It centralizes relationships, date behavior, measure formulas, formatting, descriptions, and reusable business definitions. Otherwise each report author can compute a different version of revenue or return rate. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; It is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; centralizes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; relationships is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; behavior is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; measure is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; formulas is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 194:** blank line used only to separate logical sections for readability.
- **Line 195:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific

role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 196:** ## 33. Why prefer explicit DAX measures? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 197:** blank line used only to separate logical sections for readability.
- **Line 198:** **Answer:** They are named, documented, format-controlled, reusable, and testable. Implicit sums hide business logic and can encourage users to aggregate technical columns incorrectly. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; They is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; named is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; documented is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; format-controlled is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reusable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 199:** blank line used only to separate logical sections for readability.
- **Line 200:** **Likely follow-up:** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 201:** ## 34. Import or DirectQuery for Track A? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 202:** blank line used only to separate logical sections for readability.
- **Line 203:** **Answer:** Import is the default because the local warehouse is modest, interactive performance is predictable, and refresh can be controlled after publication. DirectQuery has freshness advantages but adds source load, latency, feature, and modeling constraints. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Import is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; default is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; because is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; local is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; warehouse is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 204:** blank line used only to separate logical sections for readability.
- **Line 205:** `**Likely follow-up:**` Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Likely` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `follow-up` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Give` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `concrete` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 206:** `## 35. How do you avoid ambiguous Power BI relationships?` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 207:** blank line used only to separate logical sections for readability.
- **Line 208:** `**Answer:**` I use one-to-many, single-direction dimension-to-fact relationships. Secondary shipment date roles are inactive and activated in specific measures if needed. I avoid direct fact-to-fact joins and bidirectional filtering without a tested reason. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Answer` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `I` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `use` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `one-to-many` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `single-direction` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dimension-to-fact` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `relationships` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; `verify every character before running it`; `Secondary` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 209:** blank line used only to separate logical sections for readability.
- **Line 210:** `**Likely follow-up:**` Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Likely` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `follow-up` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Give` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `one` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `concrete` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `query` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `test` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 211:** `## 36. What is data lineage?` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 212:** blank line used only to separate logical sections for readability.
- **Line 213:** `**Answer:**` Evidence connecting a report value back through a measure, serving table, Gold fact/dimension, Silver transformation, Bronze record, source file/table, and batch. This project uses

mappings, batch IDs, source/file metadata, task order, and reconciliation results. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Evidence is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; connecting is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; report is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; value is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; back is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; through is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; measure is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 214:** blank line used only to separate logical sections for readability.
- **Line 215:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 216:** **## 37.** How do you protect secrets? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 217:** blank line used only to separate logical sections for readability.
- **Line 218:** ****Answer:**** Passwords stay in ignored \.env` files; Databricks uses secret management or identities; logs/screenshots are redacted; CI scans common patterns. If a secret is committed, I revoke or rotate it before cleaning history.—is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Passwords is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stay is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ignored is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is a path, relative location, branch-style name, or file argument; verify every character before running it; env is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; files is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Databricks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; uses` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 219:** blank line used only to separate logical sections for readability.
- **Line 220:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument,

identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 221:** ## 38. What does least privilege mean here? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 222:** blank line used only to separate logical sections for readability.
- **Line 223:** **Answer:** Each identity gets only the permissions needed: source extractor read-only, workflow access to its catalog/schema/volume, publisher stage/load rights, and analysts read-only semantic access. An administrator credential is not a normal runtime credential. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Each is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; identity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gets is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; only is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; permissions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; needed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 224:** blank line used only to separate logical sections for readability.
- **Line 225:** **Likely follow-up:** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 226:** ## 39. How does CI/CD help this project? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 227:** blank line used only to separate logical sections for readability.
- **Line 228:** **Answer:** The free GitHub Actions workflow checks structure, SQL dialect labels, notebook syntax/JSON, secret patterns, relative links, linting, and local tests. It cannot execute SQL Server or Databricks without paid/secured infrastructure, so those gates remain documented run evidence. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; The is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; free is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; GitHub is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; Actions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; workflow is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; checks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; structure is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 229:** blank line used only to separate logical sections for readability.
- **Line 230:** `**Likely follow-up:**` Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 231:** `## 40. What would change at 100 times the data volume?` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 232:** blank line used only to separate logical sections for readability.
- **Line 233:** `**Answer:**` I would profile bottlenecks before redesigning. Likely changes include source-side partitioned/incremental extraction, bulk copy, optimized file sizes, partition/clustering choices, incremental Gold, stronger orchestration observability, scalable serving mode, and cost/performance testing. I would not simply use the portfolio scale setting. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; I is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; would is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; profile is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; bottlenecks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; before is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; redesigning is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; changes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 234:** blank line used only to separate logical sections for readability.
- **Line 235:** `**Likely follow-up:**` Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific

role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 236:** ## 41. Which parts are batch and what could stream? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 237:** blank line used only to separate logical sections for readability.
- **Line 238:** **Answer:** All required phases are daily batch. Web events, inventory movements, and shipment tracking could become streaming, but dimensions, late data, deduplication, watermarks, cost, and exactly-once expectations would need a separate design. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; All is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; required is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; phases is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; are is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; daily is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Web is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; events is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 239:** blank line used only to separate logical sections for readability.
- **Line 240:** **Likely follow-up:** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 241:** ## 42. What are the limitations of the local-first track? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 242:** blank line used only to separate logical sections for readability.
- **Line 243:** **Answer:** Manual handoffs, no real-time operation, local service availability, no production identity/network design, simplified attribution, USD-only reporting, and incomplete production observability. Those are explicitly documented rather than presented as enterprise production readiness. — ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Manual is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; handoffs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; no is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; real-time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; operation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; local is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding

explanation; service is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 244:** blank line used only to separate logical sections for readability.
- **Line 245:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **:**** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 246:** **## 43.** How would you add CDC? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 247:** blank line used only to separate logical sections for readability.
- **Line 248:** ****Answer:**** Choose source-supported change data capture, retain operation type and source ordering metadata in Bronze, deduplicate by source position, apply deletes deliberately, and test replay/idempotency. It requires database configuration and privileges beyond the beginner path. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **:**** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Choose is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source-supported is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; change is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; capture is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; retain is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; operation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; type is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 249:** blank line used only to separate logical sections for readability.
- **Line 250:** ****Likely follow-up:**** Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — ****** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; **:**** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 251:** **## 44.** How would you handle schema drift? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 252:** blank line used only to separate logical sections for readability.

- **Line 253:** `**Answer:**` Bronze may permit intentional additive columns while recording schema; Silver uses explicit selections/casts and fails or quarantines unsupported changes. A schema-drift test alerts before Gold and semantic contracts silently change. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Bronze is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; may is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; permit is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; intentional is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; additive is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; columns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; while is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; recording is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 254:** blank line used only to separate logical sections for readability.
- **Line 255:** `**Likely follow-up:**` Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 256:** `## 45.` What did you personally implement versus use as reference material? is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 257:** blank line used only to separate logical sections for readability.
- **Line 258:** `**Answer:**` The workbook provides curriculum, boilerplate, patterns, TODO skeletons, tests, and reference answers. I only claim tasks after I type/adapt them, run them in my environment, capture evidence, and explain them. The final definition-of-done distinguishes reference material from personal verification. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Answer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; The is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; workbook is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; provides is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; curriculum is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; boilerplate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; patterns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 259:** blank line used only to separate logical sections for readability.
- **Line 260:** `**Likely follow-up:**` Give one concrete file, query, test, or screenshot from your verified run that proves the answer. — `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Likely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal

whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Give is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concrete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 261:** blank line used only to separate logical sections for readability.
- **Line 262:** blank line used only to separate logical sections for readability.
- **Line 263:** # Defend every design choice worksheet is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 264:** blank line used only to separate logical sections for readability.
- **Line 265:** For each decision, complete all columns before an interview. — For is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; each is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; decision is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; complete is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; all is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; columns is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; before is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; an is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; interview is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 266:** blank line used only to separate logical sections for readability.
- **Line 267:** | Decision | Requirement it serves | Alternative | Why not chosen now | Failure mode | Test/evidence | What would change at scale | — Decision is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Requirement is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; it is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; serves is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Alternative is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Why is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; chosen is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; now is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Failure is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 268:** | ---|---|---|---|---|---|---| — --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command
- **Line 269:** | Manual volume handoff | Low-cost local sources | Direct JDBC | No safe route to localhost | Wrong/missing batch | Manifest + environment check | Cloud-managed sources/network | — Manual is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; volume is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; handoff is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Low-cost is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; local is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sources is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Direct is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; JDBC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; No is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; safe is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; route is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; to is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 270:** | SCD2 customer | Historical attributes | Type 1 | Would rewrite history | Overlap/multiple current rows | SCD tests | Identity resolution/MDM | — SCD2 is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Historical is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; attributes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Type is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; Would is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; rewrite is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; history is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Overlap is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; multiple is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 271:** | [ADD DECISION] | | | | | | — [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ADD is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DECISION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 272:** blank line used only to separate logical sections for readability.
- **Line 273:** # STAR troubleshooting story template is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 274:** blank line used only to separate logical sections for readability.
- **Line 275:** - **Situation:** What business/pipeline context existed? Include phase and batch, not confidential details. — - is a command option or flag; its exact behavior is defined beside the command; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Situation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; What is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; business is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; pipeline is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; context is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; existed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Include is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; phase is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 276:** - **Task:** What trustworthy outcome were you responsible for? — - is a command option or flag; its exact behavior is defined beside the command; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Task is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; What is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; trustworthy is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; outcome is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; were is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; you is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; responsible is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

ject-specific role is described in the surrounding explanation; for is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 277:** - ****Action:**** What evidence did you inspect, how did you isolate the cause, what code/config/test changed, and how did you protect data? — - is a command option or flag; its exact behavior is defined beside the command; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Action is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; What is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; evidence is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; did is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; you is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; inspect is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; how is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; did is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 278:** - ****Result:**** What verified test/reconciliation passed? Use real numbers only after your run. — - is a command option or flag; its exact behavior is defined beside the command; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Result is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; What is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; verified is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; reconciliation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; passed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Use is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; real is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 279:** - ****Reflection:**** What prevention or monitoring did you add? — - is a command option or flag; its exact behavior is defined beside the command; ** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Reflection is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; :** is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; What is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; prevention is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; monitoring is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; did is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; you is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; add moves selected changes into the staging area
- **Line 280:** blank line used only to separate logical sections for readability.
- **Line 281:** # Final self-assessment rubric is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 282:** blank line used only to separate logical sections for readability.
- **Line 283:** Score each 0–3: 0 not started, 1 can follow, 2 can explain and reproduce, 3 can adapt and defend. — Score is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; each is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; 3 is a numeric literal used by the command or expression; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; started is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 1 is a numeric literal used by the command or expression; can is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 284:** blank line used only to separate logical sections for readability.
- **Line 285:** | Area | 0 | 1 | 2 | 3 | — Area is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; 0 is a numeric literal used by the command or expression; 1 is a numeric literal used by the command or expression; 2 is a numeric literal used by the command or expression; 3 is a numeric literal used by the command or expression
- **Line 286:** | --- | --- | --- | --- | --- | — --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command
- **Line 287:** | Requirements/KPIs | Cannot define | Reads definitions | Reproduces and validates | Resolves ambiguity with stakeholders | — Requirements is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; KPIs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Cannot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; define is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Reads is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; definitions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Reproduces is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; validates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Resolves is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ambiguity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 288:** | SQL/source design | Cannot navigate | Runs provided DDL | Explains keys/indexes/dialects | Adapts model safely | — SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; design is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Cannot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; navigate is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Runs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; provided is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DDL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Explains is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; keys is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.
- **Line 289:** | Python/generation | Cannot run | Runs quick mode | Explains functions/seed/Fks | Adds a domain and tests | — Python is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; generation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Cannot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Runs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quick is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; mode is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Explains is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; functions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; seed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 290:** | Ingestion/manifest | Cannot trace batch | Follows steps | Explains checksums/watermarks | Repairs/replays safely | — Ingestion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify

every character before running it; manifest is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Cannot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; trace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Follows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; steps is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Explains is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; checksums is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; watermarks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 291:** | Spark/Delta | Cannot identify layers | Runs notebooks | Explains transformations/merge | Tunes and handles drift | — Spark is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; Delta is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Cannot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; identify is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; layers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Runs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; notebooks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Explains is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; transformations is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; merge is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 292:** | Dimensional model | Cannot state grain | Names facts/dims | Defends grain/SCD/unknowns | Evolves model without breaking history | — Dimensional is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; model is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Cannot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; state is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; grain is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Names is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; facts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; dims is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Defends is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; grain is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.
- **Line 293:** | Quality/reconciliation | Relies on screenshots | Runs tests | Explains gates and tie-outs | Designs new risk-based checks | — Quality is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; reconciliation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Relies is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; on is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; screenshots is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Runs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tests is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Explains is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; gates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tie-outs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 294:** | Jobs/operations | Clicks run | Follows graph | Repairs failed run safely | Designs alerts and operational SLOs | — Jobs is a name, keyword, argument, identifier, or literal whose project-specific role is described

in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; operations is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Clicks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Follows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; graph is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Repairs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; failed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; safely is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Designs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 295:** | Power BI | Builds visuals only | Adds measures | Validates context/relationships | Diagnoses complex totals/roles | — Power is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BI is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Builds is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; visuals is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; only is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Adds is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; measures is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Validates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; context is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; relationships is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Diagnoses is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 296:** | Security/cost/Git | Uses defaults | Follows checklist | Explains least privilege/CI | Designs production controls | — Security is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; cost is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; Git is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Uses is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; defaults is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Follows is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; checklist is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Explains is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; least is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; privilege is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 297:** | Communication | Recites tools | Gives overview | Tells evidence-based story | Handles follow-up tradeoffs calmly | — Communication is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Recites is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tools is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Gives is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; overview is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Tells is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; evidence-based is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; story is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Handles is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; follow-up is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; tradeoffs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; calmly is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

File introduction — docs/claim_to_evidence_map.md

Exact filename: claim_to_evidence_map.md

Filename extension: .md. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: docs relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 21.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in docs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: claim_to_evidence_map.md.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled claim_to_evidence_map.md and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 21.F2.1 — Create the file docs/claim_to_evidence_map.md from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.

4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\claim_to_evidence_map.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\docs\claim_to_evidence_map.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs\claim_to_evidence_map.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\docs\claim_to_evidence_map.md" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/claim_to_evidence_map.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODD(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under docs.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `docs/claim_to_evidence_map.md`.
3. In PowerShell at the repository root, run `Test-Path ".\docs\claim_to_evidence_map.md"` and require True.
4. Run `git status --short -- "docs/claim_to_evidence_map.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for docs/claim_to_evidence_map.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: `docs/claim_to_evidence_map.md`

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 21.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Claim-to-Evidence Map

Do not convert a reference capability into a personal achievement until the evidence column is complete.

| Claim | Status | Evidence type | Exact file/run/query | Batch/date/commit | Reviewer note |
|---|---|---|---|---|---|
| Quick synthetic fixture is deterministic | Reference only; user verification required | pytest + manifest | `tests/unit/test_generator.py`; `datasets/sample/generation_manifest.json` | Seed 20260815 | Re-run locally |
| SQL Server ERP is loaded | Not started | SSMS row counts + loader artifact | `[ADD]` | `[ADD]` | Do not claim yet |
| PostgreSQL digital source is loaded | Not started | pgAdmin/psql counts + artifact | `[ADD]` | `[ADD]` | Do not claim yet |
| Databricks Job completed and quality passed | Not started | Job run ID + quality table | `[ADD]` | `[ADD]` | Do not claim yet |
| Gold reconciles to SQL Server warehouse | Not started | export manifest + audit SQL | `[ADD]` | `[ADD]` | Do not claim yet |
| Power BI measures match SQL | Not started | validation table/screenshot | `[ADD]` | `[ADD]` | Do not claim yet |
| GitHub Actions validates repository | Not started | Actions run URL/checks | `[ADD]` | `[ADD]` | Do not claim yet |
```

Line-by-line explanation

- **Line 1:** `# Claim-to-Evidence Map` is a comment for a human reader; it normally does not change data or execute business logic.

- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** Do not convert a reference capability into a personal achievement until the evidence column is complete. — Do is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; convert is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; reference is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; capability is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; into is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; personal is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; achievement is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; until is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** | Claim | Status | Evidence type | Exact file/run/query | Batch/date/commit | Reviewer note | — Claim is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Status is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Evidence is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; type is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Exact is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.
- **Line 6:** |---|---|---|---|---|---| — --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command
- **Line 7:** | Quick synthetic fixture is deterministic | Reference only; user verification required | pytest + manifest | \tests/unit/test_generator.py; datasets/sample/generation_manifest.json | Seed 20260815 | Re-run locally |—Quick is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; synthetic is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fixture is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; deterministic is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Reference is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; only is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; user is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; verification is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; required is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; pytest is the Python test runner; + is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 8:** | SQL Server ERP is loaded | Not started | SSMS row counts + loader artifact | \[ADD] | \[ADD] | Do not claim yet |—SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Server is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ERP is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;loadedis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Notis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;startedis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;SSMSis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;rowis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;countsis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;+is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;loader` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 9:** | PostgreSQL digital source is loaded | Not started | pgAdmin/psql counts + artifact | \[ADD]` | \[ADD]` | Do not claim yet | -PostgreSQLis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;digitalis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;sourceis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;isis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;loadedis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Notis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;startedis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;pgAdminis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;/is a path, relative location, branch-style name, or file argument; verify every character before running it;psqlis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;countsis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 10:** | Databricks Job completed and quality passed | Not started | Job run ID + quality table | \[ADD]` | \[ADD]` | Do not claim yet | -Databricksis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Jobis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;completedis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;andis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;qualityis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;passedis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Notis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;startedis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Jobis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;runis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;IDis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;+` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 11:** | Gold reconciles to SQL Server warehouse | Not started | export manifest + audit SQL | \[ADD]` | \[ADD]` | Do not claim yet | -Goldis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;reconcilesis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;tois a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;SQLis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Serveris a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;warehouseis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Notis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;startedis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;exportis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;manifestis a name, keyword, ar-

gument, identifier, or literal whose project-specific role is described in the surrounding explanation; +is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; audit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 12:** | Power BI measures match SQL | Not started | validation table/screenshot | \[ADD]` | `[ADD]` | Do not claim yet | -Power is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BI is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; measures is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; match is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; started is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; validation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; table is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; screenshot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 13:** | GitHub Actions validates repository | Not started | Actions run URL/checks | \[ADD]` | `[ADD]` | Do not claim yet | -GitHub is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Actions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; validates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; repository is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; started is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Actions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; run is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; URL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; checks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

[Back to the Phase 21 lesson index](#)

File introduction — docs/data_lineage.md

Exact filename: data_lineage.md

Filename extension: .md. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: docs relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 21.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.

2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in docs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `data_lineage.md`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `data_lineage.md` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 21.F3.1 — Create the file `docs/data_lineage.md` from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\data_lineage.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\docs\data_lineage.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name,

keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; -Path is a command option or flag; its exact behavior is defined beside the command; ".\docs\data_lineage.md" is a path, relative location, branch-style name, or file argument; verify every character before running it; -ErrorAction is a command option or flag; its exact behavior is defined beside the command; Stop is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\docs\data_lineage.md" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add -Force, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/data_lineage.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with Ctrl+S.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use TODO(USER) where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under docs.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with docs/data_lineage.md.
3. In PowerShell at the repository root, run Test-Path ".\docs\data_lineage.md" and require True.
4. Run git status --short -- "docs/data_lineage.md" to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for docs/data_lineage.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: docs/data_lineage.md

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 21.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Data Lineage

Lineage answers, "Where did this number come from?" The project records technical lineage through source names, file paths, batch IDs, load
timestamps, record hashes, task order, source-to-target mapping, and reconciliation results.

## Example: Total Revenue

1. `SQLSERVER_ERP.erp.order_items.quantity`, `unit_price`, and `discount_amount` are extracted to a batch Parquet file.
2. Bronze preserves source values and adds `source_file`, `batch_id`, `ingested_at_utc`, and `record_hash`.
3. Silver casts numeric fields, quarantines quantity `<= 0`, joins valid order context, and derives `net_sales_amount`.
4. Gold stores one row per valid order item in `fact_sales` and resolves dimension keys.
5. The local publisher stages and transactionally loads `dw.fact_sales`.
6. Power BI measure `[Total Revenue]` sums `fact_sales[net_sales_amount]` under the current filter context.
7. The SQL validation query independently sums the same column.

![[Lineage diagram](images/lineage.svg)]

## Evidence to retain per run

- Local extraction `batch_manifest.json` with row counts and SHA-256 checksums.
- Databricks job run ID and task logs.
- Bronze/Silver/Gold row-count and rejection results.
- `quality.test_results` rows.
- Gold export manifest.
- SQL Server `audit.etl_batch` and `audit.reconciliation_result` rows.
- Power BI refresh timestamp and validation screenshot.
```

Line-by-line explanation

- **Line 1:** # Data Lineage is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** Lineage answers, "Where did this number come from?" The project records technical lineage through source names, file paths, batch IDs, load timestamps, record hashes, task order, source-to-target mapping, and reconciliation results. — Lineage is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; answers is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Where is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; did is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; this is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; number is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; come is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; from is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; The is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; project is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; records is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** `##` Example: Total Revenue is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 6:** blank line used only to separate logical sections for readability.
- **Line 7:** 1. `\SQLSERVER_ERP.erp.order_items.quantity`, `unit_price`, and `discount_amount`` are extracted to a batch Parquet file.–1 is a numeric literal used by the command or expression; is a path, relative location, branch-style name, or file argument; verify every character before running it; `SQLSERVER_ERP.erp.order_items.quantity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `unit_price` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `discount_amount` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `are` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `extracted` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `to` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a`` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 8:** 2. Bronze preserves source values and adds `\_source_file`, `_batch_id`, `_ingested_at_utc`, and `_record_hash``.–2 is a numeric literal used by the command or expression; is a path, relative location, branch-style name, or file argument; verify every character before running it; `Bronze` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `preserves` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `source` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `values` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `adds` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `_source_file` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `_batch_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ``` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 9:** 3. Silver casts numeric fields, quarantines quantity `\<= 0``, joins valid order context, and derives ``net_sales_amount``.–3 is a numeric literal used by the command or expression; is a path, relative location, branch-style name, or file argument; verify every character before running it; `Silver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `casts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `numeric` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fields` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quarantines` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quantity` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<=` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `0` is a numeric literal used by the command or expression; ``` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 10:** 4. Gold stores one row per valid order item in `\fact_sales` and resolves dimension keys.—4 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Gold is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Stores is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; one is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; row is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; per is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; valid is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; order is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; item is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; in is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 11:** 5. The local publisher stages and transactionally loads `\dw.fact_sales`.—5 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; The is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; local is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; publisher is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stages is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; transactionally is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; loads is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `dw.fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 12:** 6. Power BI measure `\[Total Revenue]` sums `\fact_sales[net_sales_amount]` under the current filter context.—6 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; Power is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BI is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; measure is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Total is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Revenue is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sums is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `fact_sales` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; [is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; net_sales_amount is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;] is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 13:** 7. The SQL validation query independently sums the same column. — 7 is a numeric literal used by the command or expression; . is a path, relative location, branch-style name, or file argument; verify every character before running it; The is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; validation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; query is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; independently is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sums is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; the is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; same is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; column is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 14:** blank line used only to separate logical sections for readability.
- **Line 15:** `![Lineage diagram](images/lineage.svg)` — `[` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Lineage` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `diagram` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `]` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `images` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `lineage.svg` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 16:** blank line used only to separate logical sections for readability.
- **Line 17:** `## Evidence to retain per run` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 18:** blank line used only to separate logical sections for readability.
- **Line 19:** `- Local extraction \batch_manifest.json` with row counts and SHA-256 checksums.--` is a command option or flag; its exact behavior is defined beside the command; `Local` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `extraction` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `batch_manifest.json` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `with` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `counts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SHA-256` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `checksums` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ``` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 20:** `- Databricks job run ID and task logs. —` is a command option or flag; its exact behavior is defined beside the command; `Databricks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `job` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `task` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `logs` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 21:** `- Bronze/Silver/Gold row-count and rejection results. —` is a command option or flag; its exact behavior is defined beside the command; `Bronze` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Silver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Gold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row-count` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rejection` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 22:** `- \quality.test_results`rows.--` is a command option or flag; its exact behavior is defined beside the command; `quality.test_results` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rows` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ``` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 23:** `- Gold export manifest. —` is a command option or flag; its exact behavior is defined beside the command; `Gold` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the

surrounding explanation; export is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; manifest is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it

- **Line 24:** - SQL Server \audit.etl_batch` and `audit.reconciliation_result` rows.—is a command option or flag; its exact behavior is defined beside the command;SQLis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;Serveris a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;audit.etl_batchis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;andis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;audit.reconciliation_resultis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;rowsis a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;.` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 25:** - Power BI refresh timestamp and validation screenshot. — - is a command option or flag; its exact behavior is defined beside the command; Power is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; BI is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; refresh is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; timestamp is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; validation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; screenshot is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; . is a path, relative location, branch-style name, or file argument; verify every character before running it

[Back to the Phase 21 lesson index](#)

What your screen should look like now

- Every Phase 21 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

Lesson 21.3 completion checklist

☐

docs/interview_walkthrough.md was created by me, saved, located correctly, and inspected — [STATUS]

☐

docs/claim_to_evidence_map.md was created by me, saved, located correctly, and inspected — [STATUS]

☐

docs/data_lineage.md was created by me, saved, located correctly, and inspected — [STATUS]

Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 21.4](#) and recheck the root and branch.

Lesson 21.4 — Build and validate required objects

What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

Action 21.4.G1 — Inspect the current state without changing it

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Inspect the current state without changing it. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location

with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.4.G2 — Create the smallest required object or decision

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Create the smallest required object or decision. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.4.G3 — Run the phase-specific verification

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run the phase-specific verification. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.4.G4 — Demonstrate one safe failure or alternate-result branch

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Demonstrate one safe failure or alternate-result branch. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

Action 21.4.G5 — Record evidence and prepare the downstream handoff

Application:

VS Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Record evidence and prepare the downstream handoff. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 21 lesson index](#)

What your screen should look like now

- The new Phase 21 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

Lesson 21.4 completion checklist

☐

Every required object was built separately — [STATUS]

☐

Every object has a declared responsibility/grain — [STATUS]

☐

Inputs and outputs were inspected — [STATUS]

☐

A rerun or alternate-result branch was tested where applicable — [STATUS]

☐

Counts, keys, measures, relationships, or settings were independently validated — [STATUS]

☐

I can explain one failure and its safest recovery — [STATUS]

Safe stopping point

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 21.5](#).

Lesson 21.5 — Verify, troubleshoot, commit, teach back, and stop safely

What your screen should look like before starting

The Phase 21 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

Design decision — Verification before automation

Question:

Should the phase prioritize a visible manual verification or jump directly to automation?

Options considered:

- Manual verification first, then automation.
- Automation without a known-good baseline.
- Manual operation forever.

Beginner path selected:

Establish one verified manual baseline and then automate repeatable checks.

Why this path was selected:

A known-good example makes failures interpretable and prevents automation from hiding a wrong assumption.

Tradeoff:

The first run takes longer.

What a company might do instead:

Teams often pair infrastructure as code and automated tests with controlled bootstrap and acceptance procedures.

How to explain this in an interview:

I established a verified baseline, then encoded the same contract in tests and orchestration so automation preserved meaning rather than just speed.

If you see Result A

Every interview claim points to a verified object, result, commit, and tradeoff you can explain. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

If you see Result B

A statement depends only on reference code or an unverified screenshot and cannot be claimed as completed. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

Phase-specific verification gate

☐

Required files/objects exist in the exact location.

☐

I personally created or configured them.

☐

I ran the phase commands in the documented context.

☐

I inspected the output rather than relying on a green icon alone.

☐

Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.

☐

Same-batch/rerun behavior is known where applicable.

☐

A controlled failure or realistic alternate-result branch was tested safely where applicable.

☐

No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.

☐

Actual verified result: [ENTER VERIFIED RESULT].

Point-of-failure troubleshooting record

Exact or representative error message:

[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]

What it means in plain language:

[WRITE YOUR INTERPRETATION]

Most likely cause:

[WRITE ONE TESTABLE HYPOTHESIS]

What not to do:

Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

First diagnostic action:

[ENTER THE SMALLEST OBSERVATIONAL CHECK]

How to read the diagnostic result:

[RESULT A MEANS... / RESULT B MEANS...]

Safest fix:

[ENTER THE NARROW FIX]

Alternative fix when applicable:

[ENTER OR N/A]

How to verify the fix:

[ENTER THE INDEPENDENT RECHECK]

How to return to the main tutorial:

Resume at the exact action number that failed; do not skip its verification gate.

Screenshot checkpoint

Application(s) that should be visible: VS Code, the completed workbook journal, repository evidence, and a timer.

Window/panel: Show claim-to-evidence worksheet and timed explanation notes.

Sensitive information to hide: Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.

Include: The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.

Crop out: Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.

Recommended filename: docs/images/phase21_prepare-interview-explanations-and-defend-every-design-_verified.png.

Recommended repository folder: docs/images/.

README placement: The section describing Phase 21's verified result; do not use the image before the result is actually verified.

What it proves: The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

[ADD YOUR VERIFIED SCREENSHOT HERE]

Git checkpoint after the verified Phase 21 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

Action 21.5.G1 — Verify the repository root before Git operations

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.

4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git rev-parse --show-toplevel
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

Action 21.5.G2 — Verify the active Git branch

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git branch --show-current
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

Action 21.5.G3 — Inspect all current Git changes

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.

4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git status --short
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

Interpret `git status --short`

- ?? means untracked: Git sees a new path but it is not staged.
- M means modified in the working tree but not staged.
- M means the staged version differs from the last commit.
- MM means a staged version exists and the file has additional unstaged edits.
- A means staged as a new file.
- D means deletion; stop and investigate unless deletion is deliberate.
- R means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

Action 21.5.G4 — Inspect the unstaged diff for `docs/interview_walkthrough.md`

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff -- "docs/interview_walkthrough.md"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff -- "docs/interview_walkthrough.md"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"docs/interview_walkthrough.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

Action 21.5.G5 — Stage only the reviewed file docs/interview_walkthrough.md

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git add -- "docs/interview_walkthrough.md"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git add -- "docs/interview_walkthrough.md"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"docs/interview_walkthrough.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

Action 21.5.G6 — Inspect the staged diff

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git diff --cached
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

Action 21.5.G7 — Create the Phase 21 commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git commit -m "docs: add interview design-defense guide"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git commit -m "docs: add interview design-defense guide"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"docs: add interview design-defense guide"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

Action 21.5.G8 — Verify the new commit

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git log -1 --oneline --decorate
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

Action 21.5.G9 — Push the feature branch and set upstream

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
git push -u origin "$(git branch --show-current)"
```

Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row

count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 21 lesson index](#)

Verify on GitHub

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message docs: add interview design-defense guide and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

Explain it in your own words

Without copying the model answer, explain:

1. What you created or configured in Phase 21.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

OPTIONAL REFERENCE ANSWER — PHASE 21 TEACH-BACK MODEL

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

"Phase 21 created the verified boundary described by **Prepare interview explanations and defend every design choice**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed."

Mini quiz

1. Multiple choice: Why does Phase 21 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
 2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
 3. Fill in the blank: The statement that defines what one row represents is the table or entity _____.
 4. What command would you use first to confirm the PowerShell current folder before a repository command?
 5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
 6. Design choice: Explain one tradeoff of Track A and one production alternative.
 7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
 8. Ownership: Which status may you select only after personally performing and inspecting the verification?
-

Mini quiz answer key and explanations

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.
4. **Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
5. Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
6. Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
7. Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
8. **Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

My Phase 21 Learning Journal

Date completed:

[ENTER DATE]

What I created myself:

[WRITE YOUR ANSWER]

Commands I ran:

[WRITE YOUR ANSWER]

What confused me:

[WRITE YOUR ANSWER]

What error occurred:

[WRITE YOUR ANSWER]

What the error meant:

[WRITE YOUR ANSWER]

How I fixed it:

[WRITE YOUR ANSWER]

What I learned:

[WRITE YOUR ANSWER]

What I can now explain without help:

[WRITE YOUR ANSWER]

What I still need to practice:

[WRITE YOUR ANSWER]

What your screen should look like now

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

Lesson 21.5 completion checklist

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

Safe stopping point

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

How this phase connects to the next phase

Phase 22 evaluates advanced alternatives only after the required path is stable.

Phase 21 completion checklist

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]



Output independently inspected — [STATUS]



Failure/alternate branch understood — [STATUS]



Screenshot captured and redacted — [STATUS]



Targeted Git commit exists — [STATUS]



Branch pushed and verified on GitHub — [STATUS]



Quiz completed before answer key — [STATUS]



Journal completed — [STATUS]



Final phase status manually set — [] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

Phase 22 — Evaluate optional extensions without disrupting the required path

[Back to the master table of contents](#)

Track: Optional extensions only after Track A is verified

Status when first opened: PLANNED

Completion rule: The phase is not complete because its instructions exist. Mark it VERIFIED BY USER only after you perform the work, inspect the output, and save evidence.

Why this phase happens now

Earlier work this phase depends on: The complete required beginner path.

What would go wrong if it happened earlier: You would create objects without a verified requirement, valid upstream input, protected repository state, or known execution environment.

Later work that depends on it: No later required phase. Each extension has its own prerequisite and rollback decision.

Decision being made: Choose and verify the simplest local-first method that teaches the professional concept without pretending that local services are publicly reachable.

Alternatives: A company could use managed cloud databases, infrastructure as code, service principals, direct JDBC publication, deployment bundles, or streaming. Those options are discussed as tradeoffs, not substituted silently.

Beginner path selected: Track A: local SQL Server and PostgreSQL, Python batch extracts, manual governed upload to Databricks, Delta Bronze/Silver/Gold, manual verified export, local SQL Server warehouse, and Power BI Import.

Reason for the order: Requirements become source decisions; sources become generated data; source data becomes extraction batches; batches become Bronze; Bronze becomes Silver; Silver becomes Gold; Gold becomes warehouse and Power BI input.

Phase 22 lesson index

1. [Lesson 22.1 — Understand the phase and starting screen](#)
2. [Lesson 22.2 — Perform the notebook-guided actions one at a time](#)
3. [Lesson 22.3 — Create and understand the phase files](#)
4. [Lesson 22.4 — Build and validate required objects](#)
5. [Lesson 22.5 — Verify, troubleshoot, commit, teach back, and stop safely](#)

Lesson 22.1 — Understand the phase and starting screen

What your screen should look like before starting

- **Application open:** VS Code, architecture notes, cost notes, and whichever optional platform is being evaluated.
- **Window in front:** The application named by the first action; do not keep an installer, database editor, notebook, and Power BI dialog stacked ambiguously.
- **Repository folder:** C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project when repository work is required. Verify User on your computer.
- **Git branch:** A feature branch, not a detached commit. Run `git branch --show-current` and record the exact result.
- **Database/workspace connection:** The connection named in the action, not merely the most recent connection.
- **Files that should already exist:** Only files from verified earlier phases. Do not pre-create the full tree.
- **Files that should not exist yet:** New Phase 22 files remain absent until their creation lesson.
- **Terminal prompt:** It should end in the expected repository path or another explicitly named folder.
- **Previous verification:** Phase 21 must be verified when this phase has a prerequisite.

Three-level explanation of the phase

Level 1 — Child-simple explanation

Imagine building a store from labeled boxes. This phase handles one box, checks that the label is correct, and makes sure the next box receives the right thing. You do not pour all boxes together and hope the result is understandable.

Level 2 — Northstar Retail project explanation

Phase 22 creates the specific Northstar Retail files, database objects, transformations, tests, model objects, or evidence described in the lessons. Every output keeps a visible connection to its source and batch so you can trace an order, customer, product, campaign, return, shipment, or inventory record.

Level 3 — Professional explanation

This phase establishes a controlled boundary in an end-to-end batch platform. It defines contracts, execution context, idempotency behavior, quality expectations, evidence, and handoff artifacts so the downstream layer can be reproduced and reviewed independently.

Applications to open

VS Code, architecture notes, cost notes, and whichever optional platform is being evaluated. Open only the application needed for the immediate action. Extra windows increase the chance of typing a command into the wrong context.

Phase status gate

☐

I changed this phase from [] Not started to [-] In progress in the master tracker.

☐

I verified the repository root when repository work is required.

☐

I verified the current branch.

☐

I identified the exact server/database, Databricks workspace/catalog/schema, or Power BI file.

☐

I read the phase ordering explanation.

What your screen should look like now

- The Phase 22 lesson index, starting-screen checklist, and three explanation levels remain visible in this workbook.
- Only the applications required for the next lesson are open; no installer, database transaction, notebook run, or Power BI refresh was started merely to read orientation material.
- The intended repository path, active branch, server/database, Databricks workspace, or Power BI file has been written down when it applies.

- No new Phase 22 project file or object has been created yet unless the lesson explicitly instructed you to create it.
- The Phase 22 master-tracker row is marked [-] In progress only after you personally updated it.

Lesson 22.1 completion checklist

☐

I read why Phase 22 happens now — [STATUS]

☐

I can give the child-simple, Northstar-specific, and professional explanations — [STATUS]

☐

I identified the exact applications and execution context required next — [STATUS]

☐

I verified prerequisites instead of assuming they are complete — [STATUS]

☐

I updated the progress tracker myself — [STATUS]

Safe stopping point

No command, database transaction, Databricks run, loader, or Power BI refresh should be active after this orientation lesson. Save any journal note you created. You may close the workbook viewer and applications. When returning, reopen [Lesson 22.2](#), re-read its starting-screen checklist, and verify the repository root, branch, connection, and workspace before performing an action.

[Back to the Phase 22 lesson index](#)

Lesson 22.2 — Perform the notebook-guided actions one at a time

What your screen should look like before starting

The Phase 22 start gate is checked, only the needed application is in front, and the current context has been verified. The following retained lesson material preserves the Northstar terminology and exercise framing; the action cards then turn each hands-on step into a written screen-recording sequence.

Retained phase lesson, exercises, rules, and expected results

Track: Optional advanced extensions — not required for definition of done

Ownership status at start: Workbook instructions exist; your execution is not yet verified.

Evidence rule: Replace placeholders only after the verification steps pass.

1. What you are building

A decision guide and isolated backlog for streaming, CDC, Auto Loader, Lakeflow Declarative Pipelines, dbt, data contracts, data-quality frameworks, Infrastructure as Code, Declarative Automation Bundles, Azure Data Factory, Kafka, machine learning, forecasting, segmentation, slowly changing facts, row/role-level security, and cloud deployment.

2. Why it matters

Adding every popular technology makes a beginner project harder to finish and explain. Extensions should solve a measured limitation, preserve existing contracts, add tests/evidence, and remain clearly separate from the verified required path.

3. Words to know

Term	Plain-language definition
Extension	An optional capability added after the required platform is stable.
CDC	Change Data Capture: reading inserts, updates, and deletes from source change information.
Streaming	Continuously processing arriving events rather than scheduled finite batches.

Term	Plain-language definition
Auto Loader	Databricks cloudFiles ingestion for incrementally discovering files in cloud storage/volumes.
Lakeflow Declarative Pipelines	Databricks managed declarative pipeline framework; older materials may use Delta Live Tables naming.
dbt	A transformation/testing/documentation tool centered on SQL models and adapters.
Data contract	An agreed schema, semantics, quality, ownership, and change policy between producers and consumers.
Infrastructure as Code	Version-controlled definitions for infrastructure instead of manual-only creation.
Declarative Automation Bundles	Current Databricks project packaging/deployment approach; formerly called Asset Bundles.
Kafka	A distributed event-streaming platform used for durable event logs and real-time integration.
RLS	Row-level security: filtering rows based on the viewer identity/role.
Proof of concept	A bounded experiment that tests feasibility, not a production claim.
Exit criterion	A measurable condition for stopping, adopting, or rejecting an extension.

4. What must already be completed

Required Phases 0–21 are verified or accurately status-labeled, the current limitation is measured, and the extension will be built on a separate branch without breaking the batch baseline.

5. Applications to open

Application	How to open it	Why it is open
VS Code	Open docs/optional_extensions.md, risk register, and architecture.	Write the decision record and isolated design.
Official documentation in browser	Use links in the official source register near the beginning of this workbook and verify current account support/cost.	Avoid outdated names or unsupported tier assumptions.
Git/PowerShell	Create an extension branch and run baseline tests.	Keep optional experiments reversible.
Provider workspace only when chosen	Databricks/Azure/dbt/Kafka/Power BI as applicable.	Run one bounded proof of concept with cost/security controls.

6. Exact click-by-click steps

Beginner rhythm: Read the whole step. Confirm the application and location. Perform only that step. Verify the visible result. Record an error before attempting a fix.

7. Exact commands, one code block at a time

8. Where each command runs

Never infer the execution location from the code language. The command headings above identify the application and folder/database/catalog. Before every PowerShell command, run Get-Location; before SQL, verify the selected database; before Spark, print the catalog/schema/batch widgets.

TEACHING EXAMPLE

The following small example or reference pattern explains the concept before you attempt Northstar-specific work. Read it, then complete **MY TASK** without copying the answer.

Line-by-line code explanation

Line or action	Meaning	Why it belongs here
git switch main	Creates an isolated branch and records a green baseline.	An optional experiment must not silently break the core project.

Line or action	Meaning	Why it belongs here
# OPTIONAL POC — do not replace required Bronze until exit criteria pass.	Shows a bounded Auto Loader available-now pattern with explicit state locations.	The extension should first prove incremental discovery and replay behavior on one feed.
EXTENSION: TODO(USER)	Creates an evidence-based extension decision record.	Technology selection should be tied to a requirement and exit criteria.
python -m scripts.validation.check_required_files	Rechecks the baseline contract, security, tests, and extension scope.	Optional work must not reduce core reliability.

Current naming and selection notes (accessed 2026-08-16)

Databricks documentation describes Auto Loader as the `cloudFiles` source for incremental file ingestion and recommends it for many cloud-object/volume ingestion scenarios. Managed declarative pipelines are currently documented under Lakeflow Declarative Pipelines, while older learning material may say Delta Live Tables. Databricks project deployment documentation uses Declarative Automation Bundles, formerly Asset Bundles. These optional tools have account/cloud-specific features and costs, so re-check official documentation before implementation.

Extension priority rubric

Score business value, learning value, measured need, implementation effort, operating cost, security/network complexity, testability, and explainability. A low-score fashionable tool should wait. The required project is complete without any extension.

MY TASK

Select at most one extension, write a requirement and exit criteria, verify current official support, run a small isolated proof with baseline/replay/failure/cost tests, and document an adopt/revise/reject decision without changing the required project status prematurely.

STARTER SKELETON

```
PROBLEM (not tool): TODO(USER)
CURRENT BASELINE/EVIDENCE: TODO(USER)
TARGET SLA/BEHAVIOR: TODO(USER)
CANDIDATES: TODO(USER)
CHOSEN POC AND WHY: TODO(USER)
IN SCOPE: one source/table/path
OUT OF SCOPE: TODO(USER)
TESTS: initial, rerun, duplicate, failure, recovery, cost, security
EXIT CRITERIA: TODO(USER)
ROLLBACK: TODO(USER)
FINAL DECISION: TODO(USER)
```

Acceptance criteria: The extension starts from a measured problem; current official support/cost is recorded; baseline remains green; POC is isolated and bounded; replay/failure/security/cost tests run; decision is evidence-based; adopted architecture is documented or rejected experiment is closed cleanly.

HINT 1 — CONCEPT

Choose the smallest limitation with the clearest measurable success criterion.

HINT 2 — PSEUDOCODE

Baseline -> one-source POC -> initial run -> no-new-data rerun -> duplicate/schema/failure tests -> cost/security review -> decision.

HINT 3 — NEAR-COMPLETE PATTERN

For an Auto Loader POC, use one nested JSON feed, separate input/schema/checkpoint/table locations, `availableNow`, and reconcile every run to a controlled file set before considering adoption.

13. Expected files or output

- Completed extension decision record.
- Official-source/access-date notes.
- Green pre-change baseline.

- Bounded POC code and isolated resources.
- Replay/failure/cost/security evidence.
- Adopt/revise/reject conclusion and rollback outcome.

14. How to verify

- The core Track A path still works.
- No optional feature is described as required or complete without evidence.
- Feature name, tier, and status were checked on the start date.
- State/checkpoint/schema/network/secret locations are explicit.
- Duplicate/replay/late/change behavior is tested.
- Costs and cleanup are recorded.
- Documentation/lineage updates match the final decision.

15. Automated tests

- Core regression suite before and after.
- Initial and no-new-data rerun.
- Duplicate/replay behavior.
- Schema/change/order/late-data test appropriate to extension.
- Failure and recovery.
- Security/secret/network review.
- Cost/quota and resource cleanup check.

16. Common errors and exact fixes

Message/symptom	Likely cause	Exact diagnostic	Safe fix	Verify
Feature unavailable in work-space	Tier, cloud, region, preview status, or permissions differ.	Check current official docs and account capability page.	Do not work around with insecure assumptions; choose an available POC or record blocked status.	Decision record states actual support.
Auto Loader reprocesses/ignores unexpected file version	Overwrite policy, checkpoint state, or source path changed.	Inspect checkpoint, file metadata/state, and allow-overwrites policy.	Use immutable landing files by default; explicitly test/document overwrite semantics.	Controlled file-set reconciliation passes.
CDC misses deletes or changes order	Source capture mode/retention/sequence semantics are incomplete.	Inspect source log position, operation type, and ordering key.	Define snapshot plus ordered changes, delete handling, retention, and replay tests.	Target reflects all controlled operations once.
POC creates unexpected cost	Compute/storage/event resources were not bounded or stopped.	Open billing/quota/resource inventory and run history.	Stop/delete resources, lower scale, add auto-stop/budget/exit limit.	Resource cleanup and cost evidence recorded.
Core tests fail after extension	Optional code changed shared contracts without migration.	Compare baseline and diff; run first failing test.	Isolate/revert or create an explicit versioned migration with compatibility tests.	Baseline and adopted extension tests pass.

For deeper tool-specific diagnosis, use [the troubleshooting index near the end of this workbook](#).

17. Explain it in your own words

1. Why should an extension start from a limitation rather than a tool name?
2. What state does Auto Loader need for safe incremental processing?
3. What extra semantics must CDC handle beyond append-only extraction?
4. Why is a rejected POC still useful portfolio evidence?

Write your answers in the learning-journal fields in this phase. Do not copy the wording above.

18. Mini quiz

1. Are optional extensions required for the project definition of done?
2. What current Databricks name replaces older Asset Bundles wording?

3. Name three tests required before adopting an incremental ingestion POC.

Record your answers before opening the answer key.

19. Screenshot checkpoint

Capture only the selected POC: decision record, official feature/tier evidence, isolated resource/path, controlled run history, test results, cost cleanup, and final decision. Do not imply all listed extensions were built.

Save the image under docs/images/ with a descriptive filename. Check that no credential, token, password, or unnecessary personal account detail is visible.

20. Git checkpoint and suggested commit message

Run `git status`, review the Source Control diff in VS Code, stage only the intended files, then commit:

```
experiment: evaluate one optional platform extension
```

Push the feature branch only after tests pass.

21. Completion checklist

Use one status per item: Not started, In progress, Verified by me, or Needs troubleshooting.

☐

Measured limitation selected — [STATUS]

☐

Official support/cost verified — [STATUS]

☐

Baseline green — [STATUS]

☐

Isolated POC built — [STATUS]

☐

Replay/failure/security tests complete — [STATUS]

☐

Cost/resources cleaned — [STATUS]

☐

Adopt/revise/reject documented — [STATUS]

☐

Core Track A remains valid — [STATUS]

Attempt the task and record what failed before reading the optional reference answer in this phase: the optional reference-answer section later in this workbook#phase-22. The quiz answer is in the answer-key section immediately after the quiz in this workbook#phase-22.

Next phase: Return to the master checklist. The required project is ready only when its verification gates—not optional technology count—are marked Verified by me.

Appendices and supporting documents

- [Optional reference answers](the optional reference-answer section later in this workbook)
- [Mini quiz answer key](the answer-key section immediately after the quiz in this workbook)
- [Official sources](the official source register near the beginning of this workbook)
- [Business requirements](#)
- [KPI definitions](#)
- [Data dictionary](#)
- [Source-to-target mapping](#)
- [Architecture and diagrams](#)
- [Data lineage](#)
- [Testing strategy](#)
- [Complete troubleshooting index](#)
- [Runbook](#)
- [Security and cost](#)
- Screenshot checkpoints in this workbook
- [Claim-to-evidence map](#)
- Learning-journal sections in this workbook
- [Interview walkthrough](#)
- [Optional extensions](#)

Definition of done

Files existing is not completion. Open ../the final definition-of-done section in this workbook and mark a gate Verified by me only after its exact command/query/test/evidence succeeds. Required gates cover repository setup, installations, both source databases, deterministic generation, source loading/reconciliation, extraction manifests, Bronze/Silver/Gold, SCD2, duplicate reruns, blocking quality failure, Job order/repair, SQL Server warehouse publication, Power BI relationships/DAX, CI, documentation, screenshots, and interview teach-back.

The required project does not depend on Phase 22. One small well-tested extension is more valuable than many unsupported technology names.

End of the guided build notebook

You now have a complete learning notebook rather than a short showcase notebook. The project is not considered finished merely because these instructions exist. Your own screenshots, local database objects, Databricks run IDs, test results, warehouse reconciliation, Power BI model, and Git history are the proof of completion.

Final honesty check

- Keep [ADD YOUR VERIFIED . . .] placeholders until you have evidence.
- Never publish reference code as though you designed every line without understanding it.
- Be ready to explain the grain, keys, quality rules, idempotency strategy, SCD behavior, reconciliation, orchestration dependencies, and local-first limitations in your own words.

Required action cards

Action 22.2.1 — Choose one measured limitation, not one technology. Example: manual upload misses newly arrived files; report

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Choose one measured limitation, not one technology. Example: manual upload misses newly arrived files; report . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Choose one measured limitation, not one technology. Example: manual upload misses newly arrived files; report latency target is now under 15 minutes; source corrections/deletes are not captured.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 1

Action

Choose one measured limitation, not one technology. Example: manual upload misses newly arrived files; report latency target is now under 15 minutes; source corrections/deletes are not captured.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#)

Action 22.2.2 — Open docs/optional_extensions.md and complete the decision table: problem, requirement, candidate, dependenc

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Open docs/optional_extensions.md and complete the decision table: problem, requirement, candidate, dependenc. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Open docs/optional_extensions.md and complete the decision table: problem, requirement, candidate, dependencies, cost, security, contract changes, test plan, rollback, and exit criteria.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 2****Action**

Open docs/optional_extensions.md and complete the decision table: problem, requirement, candidate, dependencies, cost, security, contract changes, test plan, rollback, and exit criteria.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#)

Action 22.2.3 — Read current official documentation on the exact feature and cloud. Confirm availability, account tier, naming

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Read current official documentation on the exact feature and cloud. Confirm availability, account tier, naming. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Read current official documentation on the exact feature and cloud. Confirm availability, account tier, naming, pricing/quota, permissions, network, and preview/GA status on the date you start.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 3

Action

Read current official documentation on the exact feature and cloud. Confirm availability, account tier, naming, pricing/quota, permissions, network, and preview/GA status on the date you start.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#)

Action 22.2.4 — Create a dedicated branch such as feature/auto-loader-poc. Run and record the complete baseline suite before

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Create a dedicated branch such as feature/auto-loader-poc. Run and record the complete baseline suite before. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Create a dedicated branch such as feature/auto-loader-poc. Run and record the complete baseline suite before changing architecture.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 4****Action**

Create a dedicated branch such as feature/auto-loader-poc. Run and record the complete baseline suite before changing architecture.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#)

Action 22.2.5 — Design the smallest proof: one source/table, one controlled input batch/event, one checkpoint/state location,

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Design the smallest proof: one source/table, one controlled input batch/event, one checkpoint/state location, . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Design the smallest proof: one source/table, one controlled input batch/event, one checkpoint/state location, one quality rule, one replay/rerun test, one cost boundary, and one rollback.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 5

Action

Design the smallest proof: one source/table, one controlled input batch/event, one checkpoint/state location, one quality rule, one replay/rerun test, one cost boundary, and one rollback.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#)

Action 22.2.6 — For Auto Loader, preserve source metadata and checkpoint/schema locations; test duplicate/replay/schema evolution

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

For Auto Loader, preserve source metadata and checkpoint/schema locations; test duplicate/replay/schema evolution. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: For Auto Loader, preserve source metadata and checkpoint/schema locations; test duplicate/replay/schema evolution intentionally. Do not convert every Bronze ingestion before the proof succeeds.
4. Pause while the application finishes the operation; do not click the same control twice.

5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 6****Action**

For Auto Loader, preserve source metadata and checkpoint/schema locations; test duplicate/replay/schema evolution intentionally. Do not convert every Bronze ingestion before the proof succeeds.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#)

Action 22.2.7 — For CDC, define how inserts, updates, deletes, ordering, source log retention, and initial snapshot map to Sil

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

For CDC, define how inserts, updates, deletes, ordering, source log retention, and initial snapshot map to Sil. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: For CDC, define how inserts, updates, deletes, ordering, source log retention, and initial snapshot map to Silver/Gold and SCD behavior.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 7

Action

For CDC, define how inserts, updates, deletes, ordering, source log retention, and initial snapshot map to Silver/Gold and SCD behavior.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#)

Action 22.2.8 — For streaming/Kafka, define event keys, ordering, event time/watermarks, late data, exactly-once expectations,

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

For streaming/Kafka, define event keys, ordering, event time/watermarks, late data, exactly-once expectations,. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: For streaming/Kafka, define event keys, ordering, event time/watermarks, late data, exactly-once expectations, dead-letter/quarantine, retention, and backfill.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 8****Action**

For streaming/Kafka, define event keys, ordering, event time/watermarks, late data, exactly-once expectations, dead-letter/quarantine, retention, and backfill.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#)

Action 22.2.9 — For dbt or a quality framework, avoid duplicating all existing logic. Select a bounded set of models/tests and

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

For dbt or a quality framework, avoid duplicating all existing logic. Select a bounded set of models/tests and. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: For dbt or a quality framework, avoid duplicating all existing logic. Select a bounded set of models/tests and compare maintainability, lineage, runtime, and team fit.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 9****Action**

For dbt or a quality framework, avoid duplicating all existing logic. Select a bounded set of models/tests and compare maintainability, lineage, runtime, and team fit.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#)

Action 22.2.10 — For IaC/Bundles/cloud deployment, separate development and production variables, use identity/secret management

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

For IaC/Bundles/cloud deployment, separate development and production variables, use identity/secret management. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: For IaC/Bundles/cloud deployment, separate development and production variables, use identity/secret management, plan/dry-run review, and deletion/cost safeguards.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 10

Action

For IaC/Bundles/cloud deployment, separate development and production variables, use identity/secret management, plan/dry-run review, and deletion/cost safeguards.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#)

Action 22.2.11 — For ML/forecasting/segmentation, first create a leakage-safe business question, training/validation split, bas

Application:

Databricks workspace in a web browser

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

For ML/forecasting/segmentation, first create a leakage-safe business question, training/validation split, bas. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: For ML/forecasting/segmentation, first create a leakage-safe business question, training/validation split, baseline, metrics, feature lineage, ethics, and monitoring plan. A notebook chart alone is not a deployed use case.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Databricks workspace in a web browser. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 11****Action**

For ML/forecasting/segmentation, first create a leakage-safe business question, training/validation split, baseline, metrics, feature lineage, ethics, and monitoring plan. A notebook chart alone is not a deployed use case.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#)

Action 22.2.12 — Run the extension test matrix, compare results to exit criteria, and decide adopt, revise, or reject. Document

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run the extension test matrix, compare results to exit criteria, and decide adopt, revise, or reject. Document. This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Run the extension test matrix, compare results to exit criteria, and decide adopt, revise, or reject. Document a rejected proof honestly—it is still engineering evidence.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:

Step 12

Action

Run the extension test matrix, compare results to exit criteria, and decide adopt, revise, or reject. Document a rejected proof honestly—it is still engineering evidence.

Before continuing

☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#)

Action 22.2.13 — Update architecture/lineage/cost/security/runbook only if the extension is adopted. Keep the original Track A

Application:

Visual Studio Code

Window or panel:

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

Starting condition:

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Update architecture/lineage/cost/security/runbook only if the extension is adopted. Keep the original Track A . This is one bounded construction or inspection task.

Why this action comes now:

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

Exact physical actions:

1. Bring the application named above to the front and confirm that no unrelated window has focus.
2. Read the starting condition and compare it with what is visible on your screen.
3. Perform only this immediate task: Update architecture/lineage/cost/security/runbook only if the extension is adopted. Keep the original Track A path runnable.
4. Pause while the application finishes the operation; do not click the same control twice.
5. Inspect the visible result before changing another file, object, or setting.
6. Record the actual result in the verification field for this action.

Command or code:

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

Where to enter it:

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

What each part means:**Step 13****Action**

Update architecture/lineage/cost/security/runbook only if the extension is adopted. Keep the original Track A path runnable.

Before continuing☐

I used the application named in this phase.

☐

I confirmed the current folder/database/catalog/schema.

☐

I can describe what changed on the screen or filesystem.

☐

I know which later verification proves this step worked.

What should happen:

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

Expected example:

Your exact values may differ. The expected result is the same *type* of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

What may look different:

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

How to verify success:

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

Do not continue unless:

You can point to the visible result and explain what it proves. Mark the action VERIFIED BY USER only after that check.

Common mistake:

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

Exact recovery:

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

What you just accomplished:

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

How this connects to the project:

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#)

Action 22.2.C1 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
git switch main
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git switch main` — `git` calls the Git version-control program; `switch` changes the active branch; `main` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

Action 22.2.C2 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
git pull
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git pull` — `git` calls the Git version-control program; `pull` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

Action 22.2.C3 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
git switch -c feature/optional-extension-poc
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git switch -c feature/optional-extension-poc` — `git` calls the Git version-control program; `switch` changes the active branch; `-c` creates the named branch before switching; `feature` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-

style name, or file argument; verify every character before running it; optional-extension-poc is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

Action 22.2.C4 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m pytest -q
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m pytest -q` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `pytest` the Python test runner; `-q` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

Action 22.2.C5 — Databricks notebook (PySpark)

Application:

Databricks workspace in a web browser

Window or panel:

Databricks notebook (PySpark)

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Python instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Databricks workspace in a web browser** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Python):

```
# OPTIONAL POC – do not replace required Bronze until exit criteria pass.
stream = (spark.readStream
    .format('cloudFiles')
    .option('cloudFiles.format', 'json')
    .option('cloudFiles.schemaLocation', '<POC_SCHEMA_LOCATION>')
    .load('<POC_INPUT_PATH>'))

(stream.writeStream
    .option('checkpointLocation', '<POC_CHECKPOINT_LOCATION>')
    .trigger(availableNow=True)
    .toTable('<POC_CATALOG>.<POC_SCHEMA>.bronze_tracking_events'))
```

Where to enter it:

Databricks notebook (PySpark). Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `# OPTIONAL POC — do not replace required Bronze until exit criteria pass.` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** `stream = (spark.readStream — stream is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spark.readStream is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation`
- **Line 3:** `.format('cloudFiles')` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `format` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'cloudFiles'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 4:** `.option('cloudFiles.format', 'json')` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'cloudFiles.format'` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'json'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 5:** `.option('cloudFiles.schemaLocation', '<POC_SCHEMA_LOCATION>')` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'cloudFiles.schemaLocation'` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'<POC_SCHEMA_LOCATION>'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 6:** `.load('<POC_INPUT_PATH>')` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `load` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'<POC_INPUT_PATH>'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 7:** blank line used only to separate logical sections for readability.
- **Line 8:** `(stream.writeStream — ( is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; stream.writeStream is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation`
- **Line 9:** `.option('checkpointLocation', '<POC_CHECKPOINT_LOCATION>')` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `option` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'checkpointLocation'` is a quoted literal; the quotation marks preserve spaces or mark text; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `'<POC_CHECKPOINT_LOCATION>'` is a quoted literal; the quotation marks preserve spaces or mark text; `)` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 10:** `.trigger(availableNow=True)` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `trigger` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `availableNow` is a name, keyword, argument, identifier, or lit-

eral whose project-specific role is described in the surrounding explanation; = is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; True is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 11:** `.toTable(' <POC_CATALOG>.<POC_SCHEMA>.bronze_tracking_events' ))` — `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `toTable` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `(` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `' <POC_CATALOG>.<POC_SCHEMA>.bronze_tracking_events'` is a quoted literal; the quotation marks preserve spaces or mark text; `) )` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

Action 22.2.C6 — VS Code Markdown

Application:

Visual Studio Code

Window or panel:

VS Code Markdown

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Text instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Visual Studio Code** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Text):

```
EXTENSION: TODO(USER)
MEASURED LIMITATION: TODO(USER)
SUCCESS/EXIT CRITERIA: TODO(USER)
CURRENT OFFICIAL SOURCE + ACCESSED DATE: TODO(USER)
COST/SECURITY/NETWORK DEPENDENCIES: TODO(USER)
BASELINE RESULT: TODO(USER)
POC RESULT: TODO(USER)
DECISION: adopt / revise / reject
ROLLBACK: TODO(USER)
```

Where to enter it:

VS Code Markdown. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** EXTENSION: TODO(USER) — EXTENSION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; TODO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; USER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation
- **Line 2:** MEASURED LIMITATION: TODO(USER) — MEASURED is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; LIMITATION is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; TODO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; USER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

tion, branch-style name, or file argument; verify every character before running it; reject is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 9:** ROLLBACK: TODO(USER) — ROLLBACK is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; : is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; TODO is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; (is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; USER is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation;) is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

Action 22.2.C7 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m scripts.validation.check_required_files
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m scripts.validation.check_required_files` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.check_required_files` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

Action 22.2.C8 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m scripts.validation.scan_secrets
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m scripts.validation.scan_secrets` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.validation.scan_secrets` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

Action 22.2.C9 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
python -m pytest -q
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `python -m pytest -q` — `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `pytest` the Python test runner; `-q` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

Action 22.2.C10 — Windows PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

Windows PowerShell

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained Powershell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (Powershell):

```
git diff --stat
```

Where to enter it:

Windows PowerShell. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `git diff --stat` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--stat` is a command option or flag; its exact behavior is defined beside the command

Does it change anything?

This is intended to inspect or validate state rather than change it.

Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

What your screen should look like now

Each action you actually performed has a visible result and a recorded ownership status. You have not marked unrun reference commands as complete. The branch, connection, batch, and application context remain known.

Lesson 22.2 completion checklist

☐

I performed actions in order — [STATUS]

☐

I verified each command location — [STATUS]



I distinguished typed text from observed output — [STATUS]



I recorded differences caused by versions/tier — [STATUS]



I stopped at the first unexplained error — [STATUS]

Safe stopping point

Save open work and record the next action number. Close an application only after its operation is complete. On return, re-open the Phase 22 starting-screen checklist and re-verify the active context.

Lesson 22.3 — Create and understand the phase files

What your screen should look like before starting

VS Code Explorer shows the repository root, the intended feature branch is active, and no Phase 22 file is assumed to exist merely because a reference answer appears below. Create each folder only when its first file is needed.

Folder introduction — docs/

Exact folder name: docs

Exact parent: repository root

Why it is needed now: Phase 22 is about to place its first responsibility-specific file here; creating it earlier would have produced an unexplained empty structure.

What will eventually be inside: Only files whose purpose matches this folder's name.

Why another folder is not appropriate: Separation keeps source SQL, transformations, tests, configuration, documentation, and generated artifacts reviewable.

Naming rules: Preserve lowercase/uppercase exactly as shown. Hyphens, underscores, spaces, and plural forms create different paths. This project avoids spaces in technical folder names.

VS Code method

1. CLICK: Open Explorer in VS Code.
2. LOOK FOR: Confirm the repository root is the top folder.
3. Expand the parent path repository root one level at a time.
4. RIGHT-CLICK: Right-click the immediate parent folder.
5. CLICK: **New Folder**.
6. TYPE THIS: docs.
7. PRESS: Enter.
8. LOOK FOR: The folder appears once, at the correct indentation level.

PowerShell method

Action 22.D1.1 — Create the folder docs

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType Directory -Path ".\docs" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `Directory` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

Folder-creation decision branches**If the folder is created**

Run `Test-Path -LiteralPath ".\docs" -PathType Container` and require True.

If PowerShell says the folder already exists

Do not delete or recreate it. Run `Get-ChildItem -LiteralPath ".\docs" -Force`, inspect its existing files, and preserve them. Continue only when the existing folder is the intended path.

If the parent path does not exist

Return to the parent-folder tutorial. Create and verify one parent level at a time; do not create the entire future tree in one command.

Verify the folder

1. Run `Test-Path ".\docs" -PathType Container`.
2. Require the output True before creating a child file.
3. Remember: Git does not track an empty folder by itself; the folder becomes visible in Git history when a tracked file is added.

File introduction — docs/optional_extensions.md

Exact filename: optional_extensions.md

Filename extension: .md. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: docs relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 22.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in docs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.

6. CLICK: Select **New File**.
7. TYPE THIS: `optional_extensions.md`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `optional_extensions.md` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 22.F1.1 — Create the file `docs/optional_extensions.md` from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\optional_extensions.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\docs\optional_extensions.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `.\docs\optional_extensions.md` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

File-creation decision branches**If the command creates the file**

Run `Test-Path -LiteralPath ".\docs\optional_extensions.md" -PathType Leaf`. Continue only when it returns True and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/optional_extensions.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.

5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use TODO(USER) where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under docs.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with docs/optional_extensions.md.
3. In PowerShell at the repository root, run Test-Path ".\docs\optional_extensions.md" and require True.
4. Run git status --short -- "docs/optional_extensions.md" to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for docs/optional_extensions.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: docs/optional_extensions.md

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 22.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Optional Extensions – Not Required for the Beginner Definition of Done

| Extension | New concept | Prerequisite | Main new risk | Evidence before claiming completion |
|---|---|---|---|---|
| Streaming ingestion | Event-time, checkpoints, watermarks | Stable batch model | Duplicates/late data/state cost | Restart and late-event tests |
| Change Data Capture | Operation logs and source positions | Source privileges | Missed deletes/order | Replay and delete reconciliation |
| Auto Loader | Incremental file discovery | Cloud storage/UC setup | Schema drift/quota | Backfill and rescued-data tests |
| Lakeflow Declarative Pipelines/current equivalent | Managed declarative dependencies | Workspace feature | Tier/support changes | Current official docs and run evidence |
| dbt | SQL model DAGs/tests/docs | Reachable target/adaptor | Duplicate orchestration | CI and lineage evidence |
| Data contracts | Producer-consumer schema/quality agreement | Stable domains | Governance overhead | Version/change simulation |
| Great Expectations or similar | External quality framework | Existing test strategy | Tool complexity | Same risk coverage and stored results |
| Infrastructure as Code | Repeatable cloud resources | Cloud account and budget | Accidental spend/destruction | Plan/review/destroy evidence |
| Databricks Declarative Automation Bundles | Versioned workspace deployment | CLI/auth/workspace | Environment mismatch | Dev deployment and rollback |
| Azure Data Factory | Managed movement/orchestration | Azure account/network | Cost/duplicate orchestration | Monitored pipeline and rerun proof |
| Kafka | Distributed event streaming | Streaming design | Operational complexity | Partition/replay/exactness tests |
| Machine learning | Forecast/segmentation use | Trusted features and evaluation | Leakage/unfair interpretation | Baseline, holdout, monitoring |
| Forecasting | Time-series demand prediction | Consistent date/product grain | Leakage/seasonality | Backtest and business baseline |
| Customer segmentation | Feature engineering/clustering | Identity/conformed customer | Unstable/unexplainable groups | Stability and interpretation review |
| Slowly changing facts | Correct fact updates over time | Stable fact grain | Double-count/history confusion | Versioning and as-of queries |
| Role/row-level security | User-specific visibility | Identity/governance | Unauthorized access | Positive and negative access tests |
| Cloud deployment | Managed sources/storage/BI | Budget/network/secrets | Security/cost | Architecture review and teardown plan |

Treat every extension as a new project decision, not a badge to add automatically.
```

Line-by-line explanation

- **Line 1:** # Optional Extensions – Not Required for the Beginner Definition of Done is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** | Extension | New concept | Prerequisite | Main new risk | Evidence before claiming completion | — Extension is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; New is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; concept is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Prerequisite is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Main is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; new is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; risk is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Evidence is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; before is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; claiming is a name, keyword, argument, identifier, or literal whose project-specific role is

described in the surrounding explanation; completion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

- **Line 4:** | --- | --- | --- | --- | --- | — --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command; --- is a command option or flag; its exact behavior is defined beside the command
- **Line 5:** | Streaming ingestion | Event-time, checkpoints, watermarks | Stable batch model | Duplicates/late data/state cost | Restart and late-event tests | — Streaming is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; ingestion is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Event-time is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; checkpoints is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; watermarks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Stable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; batch is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; model is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Duplicates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.
- **Line 6:** | Change Data Capture | Operation logs and source positions | Source privileges | Missed deletes/order | Replay and delete reconciliation | — Change is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Capture is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Operation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; logs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; positions is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Source is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; privileges is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Missed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; deletes is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 7:** | Auto Loader | Incremental file discovery | Cloud storage/UC setup | Schema drift/quota | Backfill and rescued-data tests | — Auto is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Loader is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Incremental is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; file is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; discovery is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Cloud is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; storage is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; UC is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; setup is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; drift is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 8:** | Lakeflow Declarative Pipelines/current equivalent | Managed declarative dependencies | Workspace feature | Tier/support changes | Current official docs and run evidence | — Lakeflow is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Declarative is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Pipelines is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character

before running it; current is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; equivalent is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Managed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; declarative is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; dependencies is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Workspace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; feature is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Tier is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 9:** | dbt | SQL model DAGs/tests/docs | Reachable target/adapters | Duplicate orchestration | CI and lineage evidence | — dbt is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; SQL is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; model is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; DAGs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; tests is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; docs is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Reachable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; target is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; adapters is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 10:** | Data contracts | Producer-consumer schema/quality agreement | Stable domains | Governance overhead | Version/change simulation | — Data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; contracts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Producer-consumer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; schema is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; quality is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; agreement is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Stable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; domains is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Governance is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; overhead is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Version is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 11:** | Great Expectations or similar | External quality framework | Existing test strategy | Tool complexity | Same risk coverage and stored results | — Great is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Expectations is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; or is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; similar is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; External is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; quality is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; framework is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Existing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; test is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; strategy is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Tool is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; complexity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 12:** | Infrastructure as Code | Repeatable cloud resources | Cloud account and budget | Accidental spend/destruction | Plan/review/destroy evidence | — Infrastructure is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; as is a name, keyword, argument,

identifier, or literal whose project-specific role is described in the surrounding explanation; Code is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Repeatable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; cloud is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; resources is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Cloud is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; account is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; budget is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Accidental is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; spend is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 13:** | Databricks Declarative Automation Bundles | Versioned workspace deployment | CLI/auth/workspace | Environment mismatch | Dev deployment and rollback | — Databricks is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Declarative is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Automation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Bundles is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Versioned is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; workspace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; deployment is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; CLI is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; auth is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; workspace is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 14:** | Azure Data Factory | Managed movement/orchestration | Azure account/network | Cost/duplicate orchestration | Monitored pipeline and rerun proof | — Azure is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Data is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Factory is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Managed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; movement is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; orchestration is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Azure is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; account is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; network is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Cost is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 15:** | Kafka | Distributed event streaming | Streaming design | Operational complexity | Partition/replay/exactness tests | — Kafka is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Distributed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; event is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; streaming is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Streaming is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; design is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Operational is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; complexity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Partition is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; replay is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file

argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.

- **Line 16:** | Machine learning | Forecast/segmentation use | Trusted features and evaluation | Leakage/unfair interpretation | Baseline, holdout, monitoring | — Machine is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; learning is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Forecast is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; segmentation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; use is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Trusted is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; features is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; and is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; evaluation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Leakage is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.
- **Line 17:** | Forecasting | Time-series demand prediction | Consistent date/product grain | Leakage/seasonality | Backtest and business baseline | — Forecasting is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Time-series is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; demand is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; prediction is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Consistent is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; date is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; product is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; grain is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Leakage is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; seasonality is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 18:** | Customer segmentation | Feature engineering/clustering | Identity/conformed customer | Unstable/unexplainable groups | Stability and interpretation review | — Customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; segmentation is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Feature is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; engineering is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; clustering is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Identity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; conformed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; customer is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Unstable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.
- **Line 19:** | Slowly changing facts | Correct fact updates over time | Stable fact grain | Double-count/history confusion | Versioning and as-of queries | — Slowly is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; changing is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; facts is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Correct is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; updates is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; over is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; time is a name, keyword, argument, identifier, or literal whose project-

specific role is described in the surrounding explanation; Stable is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; fact is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; grain is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Double-count is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 20:** | Role/row-level security | User-specific visibility | Identity/governance | Unauthorized access | Positive and negative access tests | — Role is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; row-level is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; security is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; User-specific is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; visibility is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Identity is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; governance is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Unauthorized is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; access is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Positive is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 21:** | Cloud deployment | Managed sources/storage/BI | Budget/network/secrets | Security/cost | Architecture review and teardown plan | — Cloud is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; deployment is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Managed is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; sources is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; storage is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; BI is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; Budget is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; network is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; / is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.
- **Line 22:** blank line used only to separate logical sections for readability.
- **Line 23:** Treat every extension as a new project decision, not a badge to add automatically. — Treat is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; every is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; extension is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; as is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; new is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; project is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; decision is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; , is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; not is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; a is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; badge is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

[Back to the Phase 22 lesson index](#)

File introduction — docs/runbook.md

Exact filename: runbook.md

Filename extension: .md. In this project it identifies project documentation and helps the correct application choose syntax highlighting or execution behavior.

Exact parent folder: docs relative to the repository root.

Why this file belongs there: Files with the same responsibility are grouped so a reviewer can find configuration, source SQL, transformations, tests, documentation, and report assets without guessing.

What the file will contain: project documentation for Phase 22.

Later dependencies: Later phase commands, tests, GitHub Actions, Databricks tasks, warehouse loads, Power BI work, or portfolio evidence may depend on its exact path.

Should Git track it? Yes, unless the lesson explicitly identifies it as generated data, a local secret, a log, or a machine-specific artifact.

Could it contain secrets? It should not contain live passwords, tokens, connection strings, or account identifiers.

What happens if it is missing: The corresponding phase cannot be reproduced, validated, or explained from the repository alone.

VS Code click path

1. CLICK: In VS Code, open **Explorer** from the left Activity Bar.
2. LOOK FOR: Confirm the top-level folder name is SQL-Data-Warehouse-Project and the status bar shows the intended Git branch.
3. CLICK: Expand each parent folder in docs one level at a time.
4. If a parent folder does not exist, use the folder tutorial in this phase before creating the file.
5. RIGHT-CLICK: Right-click the immediate parent folder.
6. CLICK: Select **New File**.
7. TYPE THIS: `runbook.md`.
8. PRESS: Enter.
9. LOOK FOR: A new editor tab labeled `runbook.md` and an empty file under the correct parent folder.
10. PRESS: Ctrl+S after adding each explained section.

PowerShell path

Action 22.F2.1 — Create the file docs/runbook.md from PowerShell

Application:

Windows PowerShell or the VS Code PowerShell terminal

Window or panel:

VS Code integrated PowerShell terminal opened at the repository root

Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

Current folder or location:

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

Command or code (PowerShell):

```
New-Item -ItemType File -Path ".\docs\runbook.md" -ErrorAction Stop
```

Where to enter it:

VS Code integrated PowerShell terminal opened at the repository root. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

What each part means:

- **Line 1:** `New-Item -ItemType File -Path ".\docs\runbook.md" -ErrorAction Stop` — `New-Item` creates a file or folder; `-ItemType` is a command option or flag; its exact behavior is defined beside the command; `File` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-Path` is a command option or flag; its exact behavior is defined beside the command; `".\docs\runbook.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it; `-ErrorAction` is a command option or flag; its exact behavior is defined beside the command; `Stop` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation

Does it change anything?

This can change files, Git state, packages, database objects, or data.

Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

File-creation decision branches

If the command creates the file

Run `Test-Path -LiteralPath ".\docs\runbook.md" -PathType Leaf`. Continue only when it returns `True` and the file is empty because you just created it.

If PowerShell says the file already exists

Do not add `-Force`, do not clear the file, and do not overwrite it. Open the existing file in VS Code, run `git status --short -- "docs/runbook.md"`, inspect its contents and Git history, and decide whether this lesson should edit the preserved file rather than create a new one.

If PowerShell says part of the path does not exist

Stop. Create only the missing parent folder through the earlier folder tutorial, verify it with `Test-Path -PathType Container`, and rerun this one file command.

File-content construction

1. Start with the smallest header, comment, or first section shown in the phase lesson.
2. Read the purpose before typing the section.
3. Add one logical section at a time.
4. Save with `Ctrl+S`.
5. Run the syntax or content validation named in the phase.
6. Correct the first error before adding another section.
7. Use `TODO(USER)` where the workbook requires your own decision or verified value.

File verification

1. In the VS Code Explorer, confirm the file appears under `docs`.
2. Right-click the editor tab and choose **Copy Path**; compare the ending with `docs/runbook.md`.
3. In PowerShell at the repository root, run `Test-Path ".\docs\runbook.md"` and require `True`.
4. Run `git status --short -- "docs/runbook.md"` to see only this file's Git state.
5. Run the phase-specific validator before committing.

Optional reference answer for docs/runbook.md

Attempt the file construction first. This reference is learning material, not evidence that you created or ran the file.

Language: Markdown

Exact file: `docs/runbook.md`

Purpose: project documentation.

Inputs: The requirements, configuration, source data, or upstream objects described in Phase 22.

Expected outputs: The file's declared objects, logs, tables, checks, or documentation when the user later runs or reads it.

```
# Daily Batch Runbook – Track A

## Before the run

1. **PowerShell, repository root:** activate `.env` and run `git status`. Do not run from an unknown folder.
2. Confirm SQL Server and PostgreSQL services are running.
3. Confirm `.env` exists locally and is not tracked: `git check-ignore .env` should print `.env`.
4. Record the last successful high-water marks and any unresolved quality exceptions.

## Extract

```powershell
python -m scripts.extraction.extract_local_sources --mode incremental --watermark "[LAST_VERIFIED_UTC_TIMESTAMP]"
```

Expected: a new `datasets/raw/EXT-.../batch` folder with `batch_manifest.json`. Do not rename individual files.

## Handoff to Databricks

Upload the entire `EXT-...` folder beneath `/Volumes/<catalog>/landing/track_a_files/`. Verify the remote folder name exactly matches the local batch ID.

## Process

Run the Lakeflow Job with `batch_id=EXT-...`. Monitor tasks in dependency order. A red quality task is a stop condition, not permission to run publication manually.

## Repair

1. Open the failed task output and the relevant quarantine/test table.
2. Decide whether the failure is input quality, code, configuration, permission, or transient infrastructure.
3. Correct the cause in a branch and rerun automated tests.
4. Use **Repair run** from the earliest affected task. Same-batch logic must replace/merge safely.
5. Confirm row counts did not double.

## Publish

After the quality gate passes, download /gold\_exports/<batch\_id>/ and run:

```
python -m scripts.loading.load_gold_to_sqlserver --input-dir "C:\path\to\downloaded\gold_exports\<batch_id>" --mode full-dev
```

Then run sql/sqlserver/validation/02\_validate\_warehouse.sql in SSMS.

## Consume

Refresh Power BI only after SQL Server audit and reconciliation checks pass. Validate the KPI cards against kpi\_validation\_queries.sql, then capture the batch ID and refresh timestamp.

## Incident evidence

Preserve the local manifest, Databricks run URL/ID, failed message, quarantine counts, repair commit, final quality results, SQL audit rows, and Power BI validation. Redact secrets.

#### Line-by-line explanation

```
- **Line 1:** `# Daily Batch Runbook – Track A` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 2:** blank line used only to separate logical sections for readability.
- **Line 3:** `## Before the run` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 4:** blank line used only to separate logical sections for readability.
- **Line 5:** `1. **PowerShell, repository root:** activate \\.venv\ and run \git status\`. Do not run from an unknown folder.` – `1` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `**` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PowerShell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `repository` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `root` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `:` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `activate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `venv` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.
- **Line 6:** `2. Confirm SQL Server and PostgreSQL services are running.` – `2` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Confirm` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Server` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `PostgreSQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `services` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `are` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `running` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it
- **Line 7:** `3. Confirm \\.env\ exists locally and is not tracked: \git check-ignore .env\ should print \\.env\.` – `3` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Confirm` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `env` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `exists` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `locally` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `is` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tracked` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.
- **Line 8:** `4. Record the last successful high-water marks and any unresolved quality exceptions.` – `4` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Record` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `last` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `successful` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `high-water` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `marks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument,
```



gument, identifier, or literal whose project-specific role is described in the surrounding explanation; `code` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- \*\*Line 30:\*\* `3. Correct the cause in a branch and rerun automated tests.` - `3` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Correct` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `cause` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `a` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `rerun` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `automated` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `tests` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- \*\*Line 31:\*\* `4. Use \*\*Repair run\*\* from the earliest affected task. Same-batch logic must replace/merge safely.` - `4` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Use` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\*\*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Repair` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `\*\*` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `from` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `earliest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `affected` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `task` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- \*\*Line 32:\*\* `5. Confirm row counts did not double.` - `5` is a numeric literal used by the command or expression; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `Confirm` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `row` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `counts` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `did` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `not` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `double` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it.

- \*\*Line 33:\*\* blank line used only to separate logical sections for readability.

- \*\*Line 34:\*\* `## Publish` is a comment for a human reader; it normally does not change data or execute business logic.

- \*\*Line 35:\*\* blank line used only to separate logical sections for readability.

- \*\*Line 36:\*\* `After the quality gate passes, download \gold\_exports\<batch\_id>\ and run:` - `After` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `quality` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `gate` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `passes` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `gold\_exports` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `<` is a path, relative location, branch-style name, or file argument; verify every character before running it; `batch\_id` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `>` is a path, relative location, branch-style name, or file argument; verify every character before running it; remaining punctuation and identifiers continue the same statement.

- \*\*Line 37:\*\* blank line used only to separate logical sections for readability.

- \*\*Line 38:\*\* `\\powershell` - `powershell` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.

- \*\*Line 39:\*\* `python -m scripts.loading.load\_gold\_to\_sqlserver --input-dir "C:\path\to\downloaded\gold\_exports\<batch\_id>" --mode full-dev` - `python` invokes Python when that command name is registered; `-m` asks Python to run an installed module; `scripts.loading.load\_gold\_to\_sqlserver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--input` is a command option or flag; its exact behavior is defined beside the command; `-dir` is a command option or flag; its exact behavior is defined beside the command; `C:\path\to\downloaded\gold\_exports\<batch\_id>` is a path, relative location, branch-style name, or file argument; verify every character before running it; `--mode` is a command option or flag; its exact behavior is defined beside the command; `full-dev` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation.

- \*\*Line 40:\*\* `\\` -

- \*\*Line 41:\*\* blank line used only to separate logical sections for readability.

- \*\*Line 42:\*\* `Then run \sql\sqlserver\validation\02\_validate\_warehouse.sql\` in SSMS.` - `Then` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `sqlserver` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `validation` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it; `02\_validate\_warehouse.sql` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `in` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `SSMS` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `.` is a path, relative location, branch-style name, or file argument; verify every character before running it.

- \*\*Line 43:\*\* blank line used only to separate logical sections for readability.

- \*\*Line 44:\*\* `## Consume` is a comment for a human reader; it normally does not change data or execute business logic.

- \*\*Line 45:\*\* blank line used only to separate logical sections for readability.

- \*\*Line 46:\*\* `Refresh Power BI only after SQL Server audit and reconciliation checks pass. Validate the KPI cards against \kpi\_validation\_queries.sql\, then capture the batch ID and refresh timestamp.` - `Refresh` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Power` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `BI` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `only` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `after` is a name, keyword, argument, identifier, or literal whose project-specific role is de-

scribed in the surrounding explanation; `SQL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Server` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `audit` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `and` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `reconciliation` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `checks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `pass` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

- **Line 47:** blank line used only to separate logical sections for readability.
- **Line 48:** `## Incident evidence` is a comment for a human reader; it normally does not change data or execute business logic.
- **Line 49:** blank line used only to separate logical sections for readability.
- **Line 50:** `Preserve the local manifest, Databricks run URL/ID, failed message, quarantine counts, repair commit, final quality results, SQL audit rows, and Power BI validation. Redact secrets.` – `Preserve` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `the` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `local` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `manifest` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `Databricks` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `run` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `URL` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `/` is a path, relative location, branch-style name, or file argument; verify every character before running it; `ID` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `,` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `failed` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; remaining punctuation and identifiers continue the same statement.

[Back to the Phase 22 lesson index](#phase-22-lesson-index)

#### ## What your screen should look like now

- Every Phase 22 file you actually created appears under the correct parent folder.
- Your open editor tab matches the file you are validating.
- The Git branch has not changed unexpectedly.
- `git status --short` lists only expected work.
- Reference answers remain reference material; your status fields are not automatically completed.

#### ## Lesson 22.3 completion checklist

- [ ] `docs/optional\_extensions.md` was created by me, saved, located correctly, and inspected – [STATUS]
- [ ] `docs/runbook.md` was created by me, saved, located correctly, and inspected – [STATUS]

#### ## Safe stopping point

Save every open file with Ctrl+S. You may close VS Code only after `git status --short` is recorded in your journal. The terminal may be closed when no installer, transaction, Python process, Databricks run, warehouse load, or Power BI refresh is active. Resume at [Lesson 22.4] (#phase-22-lesson-4) and recheck the root and branch.

<a id="phase-22-lesson-4"></a>

#### ## Lesson 22.4 – Build and validate required objects

##### ## What your screen should look like before starting

The phase files are saved, the correct application and connection are visible, and the upstream input has passed its verification gate. Build one object at a time; do not create many objects and inspect them only at the end.

<a id="action-22-4-g1"></a>

##### ## Action 22.4.G1 – Inspect the current state without changing it

**Application:**  
VS Code

**Window or panel:**  
Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**Starting condition:**  
The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**Current folder or location:**  
The repository root `C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.

**What you are about to do:**  
Inspect the current state without changing it. This is one bounded construction or inspection task.

**Why this action comes now:**  
It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**Exact physical actions:**

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**\*\*Command or code:\*\***

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**\*\*Where to enter it:\*\***  
VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**\*\*What each part means:\*\***

1. Record the exact starting folders, branch, versions, connections, settings, counts, or evidence.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**\*\*What should happen:\*\***  
A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**\*\*Expected example:\*\***

> Your exact values may differ. The expected result is the same *\*type\** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**\*\*What may look different:\*\***  
Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**\*\*How to verify success:\*\***  
Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**\*\*Do not continue unless:\*\***  
You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

**\*\*Common mistake:\*\***  
Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***  
Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***  
You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***  
The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#phase-22-lesson-index)

<a id="action-22-4-g2"></a>  
## Action 22.4.G2 – Create the smallest required object or decision

**\*\*Application:\*\***  
Visual Studio Code

**\*\*Window or panel:\*\***  
Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**\*\*Starting condition:\*\***  
The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**\*\*Current folder or location:\*\***  
The repository root `C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.

**\*\*What you are about to do:\*\***  
Create the smallest required object or decision. This is one bounded construction or inspection task.

**\*\*Why this action comes now:\*\***  
It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**\*\*Exact physical actions:\*\***

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.



**\*\*Command or code:\*\***

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**\*\*Where to enter it:\*\***

Visual Studio Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**\*\*What each part means:\*\***

1. Use the phase file tutorial and one bounded action card.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**\*\*What should happen:\*\***

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**\*\*Expected example:\*\***

> Your exact values may differ. The expected result is the same *\*type\** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**\*\*What may look different:\*\***

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**\*\*How to verify success:\*\***

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**\*\*Do not continue unless:\*\***

You can point to the visible result and explain what it proves. Mark the action ``VERIFIED BY USER`` only after that check.

**\*\*Common mistake:\*\***

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#phase-22-lesson-index)

<a id="action-22-4-g3"></a>

## Action 22.4.G3 – Run the phase-specific verification

**\*\*Application:\*\***

VS Code

**\*\*Window or panel:\*\***

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**\*\*Starting condition:\*\***

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**\*\*Current folder or location:\*\***

The repository root ``C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project``. The ``User`` segment is an example; verify your real Windows username.

**\*\*What you are about to do:\*\***

Run the phase-specific verification. This is one bounded construction or inspection task.

**\*\*Why this action comes now:\*\***

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**\*\*Exact physical actions:\*\***

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**\*\*Command or code:\*\***

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**\*\*Where to enter it:\*\***

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar

field.

**\*\*What each part means:\*\***

1. Compare actual output with the required condition and record differences.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**\*\*What should happen:\*\***

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**\*\*Expected example:\*\***

> Your exact values may differ. The expected result is the same *\*type\** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**\*\*What may look different:\*\***

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**\*\*How to verify success:\*\***

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**\*\*Do not continue unless:\*\***

You can point to the visible result and explain what it proves. Mark the action ``VERIFIED BY USER`` only after that check.

**\*\*Common mistake:\*\***

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#phase-22-lesson-index)

<a id="action-22-4-g4"></a>

## Action 22.4.G4 – Demonstrate one safe failure or alternate-result branch

**\*\*Application:\*\***

VS Code

**\*\*Window or panel:\*\***

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**\*\*Starting condition:\*\***

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**\*\*Current folder or location:\*\***

The repository root ``C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project``. The ``User`` segment is an example; verify your real Windows username.

**\*\*What you are about to do:\*\***

Demonstrate one safe failure or alternate-result branch. This is one bounded construction or inspection task.

**\*\*Why this action comes now:\*\***

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**\*\*Exact physical actions:\*\***

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**\*\*Command or code:\*\***

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**\*\*Where to enter it:\*\***

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**\*\*What each part means:\*\***

1. Use a toy/disposable case and prove recovery without damaging verified work.
2. Inspect the result.
3. Record the actual value.

4. Continue only after verification.

**\*\*What should happen:\*\***

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**\*\*Expected example:\*\***

> Your exact values may differ. The expected result is the same *\*type\** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**\*\*What may look different:\*\***

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**\*\*How to verify success:\*\***

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**\*\*Do not continue unless:\*\***

You can point to the visible result and explain what it proves. Mark the action ``VERIFIED BY USER`` only after that check.

**\*\*Common mistake:\*\***

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#phase-22-lesson-index)

<a id="action-22-4-g5"></a>

## Action 22.4.G5 – Record evidence and prepare the downstream handoff

**\*\*Application:\*\***

VS Code

**\*\*Window or panel:\*\***

Use the foreground window, sidebar, tab, dialog, query editor, terminal, notebook, or Power BI pane named in the action text. Close no unrelated window merely to make the screen match an example.

**\*\*Starting condition:\*\***

The previous verification gate has passed. Any file you edited is saved. The correct branch, database connection, Databricks workspace, or Power BI file is still active.

**\*\*Current folder or location:\*\***

The repository root ``C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project``. The ``User`` segment is an example; verify your real Windows username.

**\*\*What you are about to do:\*\***

Record evidence and prepare the downstream handoff. This is one bounded construction or inspection task.

**\*\*Why this action comes now:\*\***

It creates or verifies the exact input required by the next action; doing it later would make errors harder to localize.

**\*\*Exact physical actions:\*\***

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**\*\*Command or code:\*\***

No command is required in this action. Use only the clicks, keystrokes, observations, or writing task below.

**\*\*Where to enter it:\*\***

VS Code. When the action is click-only, do not invent a command. When code is shown, use the named editor rather than a visually similar field.

**\*\*What each part means:\*\***

1. Save the verified input that the next phase consumes.
2. Inspect the result.
3. Record the actual value.
4. Continue only after verification.

**\*\*What should happen:\*\***

A visible file, folder, branch, connection, table, notebook, task, relationship, measure, visual, or verification result should appear exactly where the action predicts.

**\*\*Expected example:\*\***

> Your exact values may differ. The expected result is the same *\*type\** of object and the same required status, not necessarily the same timestamps, row counts, colors, generated IDs, or screen dimensions.

**\*\*What may look different:\*\***

Menu wording can vary by product release or account tier. Use the official-source register and look for the described object rather than guessing from an old screenshot.

**\*\*How to verify success:\*\***

Reopen or refresh the relevant panel, locate the named object, and inspect the property or output that proves it exists and is in the intended location.

**\*\*Do not continue unless:\*\***

You can point to the visible result and explain what it proves. Mark the action `VERIFIED BY USER` only after that check.

**\*\*Common mistake:\*\***

Combining several clicks or decisions, choosing a similarly named object, or continuing because no error dialog appeared.

**\*\*Exact recovery:\*\***

Stop at the current screen. Save a screenshot of the error if useful, hide sensitive values, and compare the selected location with the starting condition. Use Back or Cancel rather than Delete or Overwrite. Correct one mismatch and repeat only this action.

**\*\*What you just accomplished:\*\***

You completed one observable piece of the Northstar Retail platform and established a checkpoint that can be audited later.

**\*\*How this connects to the project:\*\***

The resulting object, decision, or evidence becomes an input to the next stage in the source-to-consumption chain.

[Back to the Phase 22 lesson index](#phase-22-lesson-index)

**## What your screen should look like now**

- The new Phase 22 objects are visible in their owning application.
- The current branch remains the intended feature branch.
- The current database, catalog, schema, Power BI model, or GitHub run is still identifiable.
- Verification output is visible or recorded; no object is marked complete solely because creation code was displayed.
- Any failed toy demonstration has been repaired or isolated.

**## Lesson 22.4 completion checklist**

- [ ] Every required object was built separately – [STATUS]
- [ ] Every object has a declared responsibility/grain – [STATUS]
- [ ] Inputs and outputs were inspected – [STATUS]
- [ ] A rerun or alternate-result branch was tested where applicable – [STATUS]
- [ ] Counts, keys, measures, relationships, or settings were independently validated – [STATUS]
- [ ] I can explain one failure and its safest recovery – [STATUS]

**## Safe stopping point**

Stop only after the active command, database transaction, Databricks cell/Job, Python process, warehouse load, or Power BI refresh finishes. Save all files and the Power BI project. Record uncommitted Git changes and exact resume action. Resume at [Lesson 22.5](#phase-22-lesson-5).

<a id="phase-22-lesson-5"></a>

**## Lesson 22.5 – Verify, troubleshoot, commit, teach back, and stop safely**

**## What your screen should look like before starting**

The Phase 22 outputs are visible, all runs are finished, open files are saved, and you can state the active branch and execution context. Any error you encountered has a recorded message and has not been hidden by deleting evidence.

**## Design decision – Batch versus streaming extension**

**\*\*Question:\*\***

Should optional streaming replace the verified batch path?

**\*\*Options considered:\*\***

- Replace batch immediately.
- Add streaming as a separate extension after batch works.
- Never consider streaming.

**\*\*Beginner path selected:\*\***

Keep batch as the baseline and add streaming only with a distinct requirement and cost plan.

**\*\*Why this path was selected:\*\***

The stable batch path remains a rollback and comparison point.

**\*\*Tradeoff:\*\***

Two paths increase maintenance.

**\*\*What a company might do instead:\*\***

Companies select patterns by latency, event semantics, operational maturity, and cost.

**\*\*How to explain this in an interview:\*\***

I treated streaming as an optional architecture decision, not a résumé keyword; I would add it only for a justified latency requirement.

## If you see Result A

Prerequisites, cost, rollback, isolation, and success criteria are written before optional work starts. Record the exact result, perform the named independent verification, and continue only when the gate is satisfied.

## If you see Result B

A prerequisite is missing or the extension would disrupt the verified required path. Treat this as information, not permission to delete or overwrite. Follow the branch-specific diagnostic in the current action, preserve evidence, and continue only after the state is understood.

## If you see an error

Read the first meaningful error line, redact secrets, and verify the application, path, branch, server/database, catalog/schema, batch ID, parameters, version/tier, and object name. Run the smallest observational check first. Do not reset Git history, drop data, disable a rule, force-push, or replace the target merely to make the error disappear.

## Phase-specific verification gate

- [ ] Required files/objects exist in the exact location.
- [ ] I personally created or configured them.
- [ ] I ran the phase commands in the documented context.
- [ ] I inspected the output rather than relying on a green icon alone.
- [ ] Counts, keys, relationships, measures, checksums, statuses, or settings meet the declared contract.
- [ ] Same-batch/rerun behavior is known where applicable.
- [ ] A controlled failure or realistic alternate-result branch was tested safely where applicable.
- [ ] No password, token, or secret appears in tracked files, screenshots, notebook output, terminal history pasted into documentation, or staged changes.
- [ ] Actual verified result: `[ENTER VERIFIED RESULT]`.

## Point-of-failure troubleshooting record

**\*\*Exact or representative error message:\*\***  
`[PASTE THE FIRST MEANINGFUL ERROR, WITH SECRETS REDACTED]`

**\*\*What it means in plain language:\*\***  
`[WRITE YOUR INTERPRETATION]`

**\*\*Most likely cause:\*\***  
`[WRITE ONE TESTABLE HYPOTHESIS]`

**\*\*What not to do:\*\***  
Do not delete data, drop databases, reset Git history, force-push, disable constraints, edit raw evidence, change a quality threshold, or expose credentials merely to make the symptom disappear.

**\*\*First diagnostic action:\*\***  
`[ENTER THE SMALLEST OBSERVATIONAL CHECK]`

**\*\*How to read the diagnostic result:\*\***  
`[RESULT A MEANS... / RESULT B MEANS...]`

**\*\*Safest fix:\*\***  
`[ENTER THE NARROW FIX]`

**\*\*Alternative fix when applicable:\*\***  
`[ENTER OR N/A]`

**\*\*How to verify the fix:\*\***  
`[ENTER THE INDEPENDENT RECHECK]`

**\*\*How to return to the main tutorial:\*\***  
Resume at the exact action number that failed; do not skip its verification gate.

## Screenshot checkpoint

**\*\*Application(s) that should be visible:\*\*** VS Code, architecture notes, cost notes, and whichever optional platform is being evaluated.  
**\*\*Window/panel:\*\*** Show extension decision record showing prerequisites, cost, risks, and rollback path.  
**\*\*Sensitive information to hide:\*\*** Passwords, tokens, connection strings, email addresses, cloud account IDs, unnecessary Windows profile details, and any secret revealed by an error.  
**\*\*Include:\*\*** The object name, batch/run ID when relevant, result/status, and enough surrounding UI to prove which application and layer is shown.  
**\*\*Crop out:\*\*** Unrelated desktop windows, notification previews, bookmarks, chat, and empty background.  
**\*\*Recommended filename:\*\*** `docs/images/phase22\_evaluate-optional-extensions-without-disrupting-the-req\_verified.png`.  
**\*\*Recommended repository folder:\*\*** `docs/images/`.  
**\*\*README placement:\*\*** The section describing Phase 22's verified result; do not use the image before the result is actually verified.  
**\*\*What it proves:\*\*** The visible state existed under the shown context. It does not prove hidden rows, other batches, every filter combination, or a later run.

`[ADD YOUR VERIFIED SCREENSHOT HERE]`

## Git checkpoint after the verified Phase 22 changes

Do not use `git add .` until you understand that it stages every matching change under the current directory, including files you did not intend. Prefer targeted staging after inspecting status and diffs.

### Save open work

1. PRESS: In VS Code, press Ctrl+K, then S if you use **Save All**, or press Ctrl+S in every modified editor.
2. In Power BI, press Ctrl+S and confirm the local file location.
3. Wait for database, Python, Databricks, and refresh operations to finish; Git does not capture uncommitted database/cloud state.

<a id="action-22-5-g1"></a>

## Action 22.5.G1 – Verify the repository root before Git operations

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches `The repository root `C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project`. The `User` segment is an example; verify your real Windows username.` or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
``powershell
git rev-parse --show-toplevel
```

### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git rev-parse --show-toplevel` — `git` calls the Git version-control program; `rev-parse` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-toplevel` is a command option or flag; its exact behavior is defined beside the command

### Does it change anything?

This is intended to inspect or validate state rather than change it.

### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

## Action 22.5.G2 — Verify the active Git branch

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.

5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

#### Command or code (PowerShell):

```
git branch --show-current
```

#### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

#### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

#### What each part means:

- **Line 1:** `git branch --show-current` — `git` calls the Git version-control program; `branch` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--show` is a command option or flag; its exact behavior is defined beside the command; `-current` is a command option or flag; its exact behavior is defined beside the command

#### Does it change anything?

This is intended to inspect or validate state rather than change it.

#### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

#### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

#### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

#### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

#### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

#### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

#### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

#### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

#### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

#### How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)



## Action 22.5.G3 — Inspect all current Git changes

---

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

### Current folder or location:

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

### What you are about to do:

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
git status --short
```

### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git status --short` — `git` calls the Git version-control program; `status` shows tracked, staged, modified, and untracked work; `--short` is a command option or flag; its exact behavior is defined beside the command

### Does it change anything?

This is intended to inspect or validate state rather than change it.

### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

**Interpret `git status --short`**

- `??` means untracked: Git sees a new path but it is not staged.
- `M` means modified in the working tree but not staged.
- `M` means the staged version differs from the last commit.
- `MM` means a staged version exists and the file has additional unstaged edits.
- `A` means staged as a new file.
- `D` means deletion; stop and investigate unless deletion is deliberate.
- `R` means rename; inspect both old and new paths.
- A blank result means the working tree is clean. It does not prove database, Databricks, or Power BI work occurred.

## Action 22.5.G4 — Inspect the unstaged diff for `docs/optional_extensions.md`

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

### Why this action comes now:

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

### Exact physical actions:

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

### Command or code (PowerShell):

```
git diff -- "docs/optional_extensions.md"
```

### Where to enter it:

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

### Plain-English translation:

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git diff -- "docs/optional_extensions.md"` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--` is a command option or flag; its exact behavior is defined beside the command; `"docs/optional_extensions.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

### Does it change anything?

This is intended to inspect or validate state rather than change it.

### Is it safe to rerun?

Normally safe to rerun because it is observational, but still confirm the execution location first.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use Get-Location or Test-Path; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

## Action 22.5.G5 — Stage only the reviewed file docs/optional\_extensions.md

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git add -- "docs/optional_extensions.md"
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

### What each part means:

- **Line 1:** `git add -- "docs/optional_extensions.md"` — `git` calls the Git version-control program; `add` moves selected changes into the staging area; `--` is a command option or flag; its exact behavior is defined beside the command; `"docs/optional_extensions.md"` is a path, relative location, branch-style name, or file argument; verify every character before running it

### Does it change anything?

This can change files, Git state, packages, database objects, or data.

### Is it safe to rerun?

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

### What should happen:

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

### Expected example:

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

### What may look different:

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

### How to verify success:

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

### Do not continue unless:

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

### Common mistake:

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

### Exact recovery:

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

### What you just accomplished:

You executed and checked one controlled operation rather than treating a long script as a black box.

### How this connects to the project:

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

For every additional intended path, run a separate targeted `git add -- "path"` after its diff is reviewed. Never stage `.env`, generated raw batches, credentials, tokens, local database files, or an unverified Power BI artifact.

## Action 22.5.G6 — Inspect the staged diff

### Application:

Windows PowerShell or the VS Code PowerShell terminal

### Window or panel:

VS Code integrated PowerShell terminal

### Starting condition:

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git diff --cached
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git diff --cached` — `git` calls the Git version-control program; `diff` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `--cached` is a command option or flag; its exact behavior is defined beside the command

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as RUN BY USER or VERIFIED BY USER. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

## Action 22.5.G7 — Create the Phase 22 commit

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git commit -m "docs: evaluate optional platform extensions"
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git commit -m "docs: evaluate optional platform extensions"` — `git` calls the Git version-control program; `commit` records the staged snapshot; `-m` asks Python to run an installed module; `"docs: evaluate optional platform extensions"` is a quoted literal; the quotation marks preserve spaces or mark text

**Does it change anything?**

This can change files, Git state, packages, database objects, or data.

**Is it safe to rerun?**

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

## Action 22.5.G8 — Verify the new commit

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.



**Current folder or location:**

The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root C:\Users\User\Desktop\Northstar\_Retail\SQL-Data-Warehouse-Project. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git log -1 --oneline --decorate
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git log -1 --oneline --decorate` — `git` calls the Git version-control program; `log` is a name, keyword, argument, identifier, or literal whose project-specific role is described in the surrounding explanation; `-1` is a command option or flag; its exact behavior is defined beside the command; `--oneline` is a command option or flag; its exact behavior is defined beside the command; `--decorate` is a command option or flag; its exact behavior is defined beside the command

**Does it change anything?**

This is intended to inspect or validate state rather than change it.

**Is it safe to rerun?**

Normally safe to rerun because it is observational, but still confirm the execution location first.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use

`git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

## Action 22.5.G9 — Push the feature branch and set upstream

---

**Application:**

Windows PowerShell or the VS Code PowerShell terminal

**Window or panel:**

VS Code integrated PowerShell terminal

**Starting condition:**

The preceding action is verified, open work is saved, and you have not assumed that a similarly named folder, branch, server, database, catalog, schema, table, or file is the correct one.

**Current folder or location:**

The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username.

**What you are about to do:**

Run one explained PowerShell instruction and inspect its result before any next command is introduced.

**Why this action comes now:**

The next action depends on this command's observed state or created object. Separating commands prevents an early failure from being hidden by later output.

**Exact physical actions:**

1. CLICK: Bring **Windows PowerShell or the VS Code PowerShell terminal** to the front.
2. LOOK FOR: Confirm the current location matches The repository root `C:\Users\User\Desktop\Northstar_Retail\SQL-Data-Warehouse-Project`. The User segment is an example; verify your real Windows username. or the more specific location stated below.
3. CLICK: Place the text cursor in the terminal, query editor, notebook cell, DAX formula bar, or GitHub field named below.
4. TYPE THIS: Enter the command exactly as shown in the code block. Do not type the prompt or expected output.
5. PRESS: Press **Enter** for a one-line terminal command, **F5/Execute** for the named database editor, or the notebook cell Run control as stated.
6. LOOK FOR: Wait until the busy indicator stops and a new prompt, result grid, cell output, or status badge appears.
7. DO NOT CONTINUE UNTIL: Compare the result with the expected shape and complete the verification action.

**Command or code (PowerShell):**

```
git push -u origin "$(git branch --show-current)"
```

**Where to enter it:**

VS Code integrated PowerShell terminal. Do not enter terminal commands into a source-code file and do not paste expected output back into the editor.

**Plain-English translation:**

This instruction asks the selected application to perform exactly the operation named by its leading keyword, using the following names, paths, clauses, flags, and values as inputs.

**What each part means:**

- **Line 1:** `git push -u origin "$(git branch --show-current)"` — `git` calls the Git version-control program; `push` sends local commits to a remote repository; `-u` is a command option or flag; its exact behavior is defined beside the command; `origin` the conventional name of the GitHub remote; `"$(git branch --show-current)"` is a quoted literal; the quotation marks preserve spaces or mark text

**Does it change anything?**

This can change files, Git state, packages, database objects, or data.

**Is it safe to rerun?**

Safe to rerun only after reading the idempotency note and confirming that the same batch, file, or object will not be duplicated.

**What should happen:**

The application should finish without an unhandled error and display a result that corresponds to the command's stated purpose.

**Expected example:**

Your exact values may differ. The exact values may differ. Expect either a version/path/status line, a query result grid, or a clearly labeled completion message that matches the command purpose.

**What may look different:**

Your Windows username, installation version, server instance, database owner, generated identifier, batch timestamp, row count, Databricks interface label, Power BI ribbon position, and terminal theme can differ. A different object name, wrong branch, wrong database, missing target, unexpected duplicate, or failed quality status is not harmless.

**How to verify success:**

Run or perform the smallest independent check named in the lesson. For a path, use `Get-Location` or `Test-Path`; for Git, use `git status` and `git branch --show-current`; for SQL, run a metadata or row-count query; for Databricks, inspect the target object and cell output; for Power BI, compare a filtered DAX result to SQL.

**Do not continue unless:**

The result is visible, understandable, and recorded as `RUN BY USER` or `VERIFIED BY USER`. Displaying this workbook does not count as execution.

**Common mistake:**

The most common mistake is running the right text in the wrong application, folder, database, catalog, schema, branch, or notebook language.

**Exact recovery:**

Do not delete, reset, overwrite, or force-push. Read the first error line. Re-run the location check for this application. Confirm the current folder, branch, server/database, or Databricks catalog. Correct only the mismatched location or spelling, then retry this single command.

**What you just accomplished:**

You executed and checked one controlled operation rather than treating a long script as a black box.

**How this connects to the project:**

This verified state becomes a dependable input to the next Northstar Retail construction action.

[Back to the Phase 22 lesson index](#)

**Verify on GitHub**

1. CLICK: Open the repository page in a browser.
2. CLICK: Open the branch selector above the file list.
3. TYPE THIS: The exact branch name returned locally.
4. CLICK: Select that branch.
5. LOOK FOR: The latest commit message `docs: evaluate optional platform extensions` and the intended changed paths.
6. CLICK: Open each changed file and confirm the GitHub version matches the staged diff.
7. Do not merge solely because the push succeeded; CI and review must pass.

## Push-rejected recovery

If Git says the remote contains work you do not have, do not use `--force`. Run `git fetch origin`, inspect `git log --oneline --graph --decorate --all -20`, then rebase or merge only after understanding the divergence. Resolve conflicts one file at a time, run tests again, and push normally. A rejection protects remote work; it is not permission to overwrite it.

## Explain it in your own words

---

Without copying the model answer, explain:

1. What you created or configured in Phase 22.
2. Why it exists.
3. Where it is located on your computer, in a database, in Databricks, in Power BI, or on GitHub.
4. What information enters it.
5. What information leaves it.
6. How it connects to the next phase.
7. How you verified it.
8. What could cause it to fail.
9. Which decision/tradeoff you could defend in an interview.

## My answer

[WRITE YOUR ANSWER BEFORE READING THE REFERENCE]

## OPTIONAL REFERENCE ANSWER — PHASE 22 TEACH-BACK MODEL

---

Read this only after completing **MY TASK** and using the three hints in order. The model is a learning reference, not evidence of your execution.

“Phase 22 created the verified boundary described by **Evaluate optional extensions without disrupting the required path**. It consumes only the outputs accepted from earlier phases, produces a named and auditable handoff, and records enough metadata and tests to detect wrong context, missing data, duplicates, failed rules, or unsafe reruns. I verified the result in its owning application and connected the evidence to Git without claiming that displayed instructions were executed.”

## Mini quiz

---

1. Multiple choice: Why does Phase 22 occur at this point in the source-to-consumption chain? A) Because menu order is arbitrary. B) Because its verified upstream input now exists. C) Because later phases do not depend on it. D) To create portfolio screenshots before validation.
  2. True or false: A command that displays no error is automatically verified even when it ran in the wrong folder, database, catalog, schema, branch, or filter context.
  3. Fill in the blank: The statement that defines what one row represents is the table or entity \_\_\_\_\_.
  4. What command would you use first to confirm the PowerShell current folder before a repository command?
  5. Troubleshooting: Your output differs from the example. What should you check before deleting, overwriting, resetting, force-pushing, or weakening a test?
  6. Design choice: Explain one tradeoff of Track A and one production alternative.
  7. Code reading: Why are input, accepted, rejected, and deduplicated counts recorded separately?
  8. Ownership: Which status may you select only after personally performing and inspecting the verification?
- 

## Mini quiz answer key and explanations

---

1. **B.** This phase is ordered after its prerequisites so its output has a trustworthy input. A is wrong because order is intentional; C is wrong because later phases depend on the handoff; D is wrong because screenshots follow verification.
2. **False.** Lack of an error proves only that the application accepted something; context and independent verification still matter.
3. **Grain.** Grain is the precise meaning of one row. A key identifies a row, but it does not by itself state the row's business meaning.

- 4. Get-Location.** It shows the shell's current folder. `git rev-parse --show-toplevel` separately verifies the repository root when inside a clone.
- 5.** Check application, folder/path, branch, server/database, catalog/schema, batch ID, parameters, source/target grain, filter context, version/tier differences, and the first meaningful error. Destructive changes are not a diagnostic step.
- 6.** Track A has manual upload/download boundaries and batch latency. A production alternative is routable private connectivity with cloud object storage/managed warehouse, identities, and automated orchestration.
- 7.** Separate counts make every row outcome explainable. A single final count can hide silent drops, duplicates, or records that moved to quarantine.
- 8. Verified by me / VERIFIED BY USER.** `PLANNED`, `CREATED BY USER`, and `RUN BY USER` describe earlier states; only inspected evidence supports verification.

## My Phase 22 Learning Journal

---

**Date completed:**

[ENTER DATE]

**What I created myself:**

[WRITE YOUR ANSWER]

**Commands I ran:**

[WRITE YOUR ANSWER]

**What confused me:**

[WRITE YOUR ANSWER]

**What error occurred:**

[WRITE YOUR ANSWER]

**What the error meant:**

[WRITE YOUR ANSWER]

**How I fixed it:**

[WRITE YOUR ANSWER]

**What I learned:**

[WRITE YOUR ANSWER]

**What I can now explain without help:**

[WRITE YOUR ANSWER]

**What I still need to practice:**

[WRITE YOUR ANSWER]

## What your screen should look like now

---

- The phase output and final validation are saved or visible.
- The intended feature branch is still active.
- `git status` is either clean after the commit or lists only deliberately deferred work recorded in the journal.
- The branch exists on GitHub after a successful push.
- No database transaction, Databricks run, loader, test process, or Power BI refresh is still active.
- The next phase's required input is identifiable.

## Lesson 22.5 completion checklist

---

☐

I completed the verification gate with my actual results — [STATUS]

☐

I recorded at least one realistic failure, alternate result, or diagnostic path — [STATUS]

☐

I captured and redacted the required screenshot only after verification — [STATUS]

☐

I reviewed, staged, committed, pushed, and verified only the intended Git changes — [STATUS]

☐

I completed the teach-back, quiz, answer review, and learning journal — [STATUS]

☐

I can name the exact verified handoff to the next phase — [STATUS]

## Safe stopping point

---

All files must be saved. The terminal may be closed after the Git result is recorded. SSMS/pgAdmin may be closed after queries finish and transactions are committed/rolled back. Databricks compute may be stopped when no Job or notebook is running. Power BI may be closed after saving. Resume from the next phase's screen-start checklist and re-verify root, branch, connection, and latest accepted batch.

## How this phase connects to the next phase

---

No later required phase. Each extension has its own prerequisite and rollback decision.

## Phase 22 completion checklist

---

☐

Phase goal explained in my own words — [STATUS]

☐

Required files/objects created by me — [STATUS]

☐

Commands run by me — [STATUS]

☐

Output independently inspected — [STATUS]

☐

Failure/alternate branch understood — [STATUS]

☐

Screenshot captured and redacted — [STATUS]

☐

Targeted Git commit exists — [STATUS]

☐

Branch pushed and verified on GitHub — [STATUS]

☐

Quiz completed before answer key — [STATUS]

☐

Journal completed — [STATUS]

☐

Final phase status manually set — [ ] Not started / [-] In progress / [x] Verified by me / [!] Needs troubleshooting

[Back to the master table of contents](#)

# Complete glossary

[Back to the master table of contents](#)

## Accepted value

---

**Plain-language definition:** A value inside an approved set, such as STORE or ONLINE.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** A value inside an approved set, such as STORE or ONLINE. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Additive measure

---

**Plain-language definition:** A measure safe to sum across the relevant dimensions, such as sales amount.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** A measure safe to sum across the relevant dimensions, such as sales amount. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Audit column

---

**Plain-language definition:** Metadata such as batch ID, source system, or load timestamp.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Metadata such as batch ID, source system, or load timestamp. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Batch

---

**Plain-language definition:** A named set of records processed together.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** A named set of records processed together. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Batch ID

---

**Plain-language definition:** A stable identifier connecting one extraction, load, tests, and publication.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** A stable identifier connecting one extraction, load, tests, and publication. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Branch

---

**Plain-language definition:** A named line of development within a repository.

**Everyday analogy:** Like making changes on a photocopy while the approved original remains untouched.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 1 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 1 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 1.

**Interview-ready definition:** A named line of development within a repository. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Bronze

---

**Plain-language definition:** Raw, append-oriented lakehouse layer with source evidence.

**Everyday analogy:** Like moving merchandise from receiving, to inspection/cleaning, to labeled shelves ready for shoppers.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Raw, append-oriented lakehouse layer with source evidence. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Business key

---

**Plain-language definition:** A meaningful source identifier such as customer\_number or SKU.

**Everyday analogy:** Like an ID number on a library card: it lets systems point to the intended record without relying on a person's name.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 4 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 4 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 4.

**Interview-ready definition:** A meaningful source identifier such as customer\_number or SKU. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Cardinality

---

**Plain-language definition:** The relationship pattern between key values, such as one-to-many.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 16 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.



**Related terms:** See the Phase 16 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 16.

**Interview-ready definition:** The relationship pattern between key values, such as one-to-many. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Catalog

---

**Plain-language definition:** Top-level governed namespace for Databricks data objects.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 8 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 8 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 8.

**Interview-ready definition:** Top-level governed namespace for Databricks data objects. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Change Data Capture

---

**Plain-language definition:** A method of capturing inserts, updates, and deletes from a source.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** A method of capturing inserts, updates, and deletes from a source. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Checksum

---

**Plain-language definition:** A fingerprint used to detect changed file bytes.

**Everyday analogy:** Like a tamper-evident fingerprint for the exact bytes of a file.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 7 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 7 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 7.

**Interview-ready definition:** A fingerprint used to detect changed file bytes. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## CI/CD

---

**Plain-language definition:** Automated integration checks and, optionally, controlled deployment.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 19 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 19 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 19.

**Interview-ready definition:** Automated integration checks and, optionally, controlled deployment. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Clone

---

**Plain-language definition:** A local repository copied from and connected to a remote repository.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 1 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 1 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 1.

**Interview-ready definition:** A local repository copied from and connected to a remote repository. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Column

---

**Plain-language definition:** One named attribute or measure stored for every row where applicable.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** One named attribute or measure stored for every row where applicable. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Commit

---

**Plain-language definition:** A recorded snapshot of staged changes with an identifier and message.

**Everyday analogy:** Like sealing and labeling a snapshot of the pages you intentionally selected.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 1 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 1 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 1.

**Interview-ready definition:** A recorded snapshot of staged changes with an identifier and message. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Constraint

---

**Plain-language definition:** A database rule that rejects data or schema states that violate an invariant.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 4 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 4 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 4.

**Interview-ready definition:** A database rule that rejects data or schema states that violate an invariant. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Consumption layer

---

**Plain-language definition:** Where users query or view data, such as Power BI or SQL.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 17 to connect source records, batch evidence,

lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 17 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 17.

**Interview-ready definition:** Where users query or view data, such as Power BI or SQL. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Current row

---

**Plain-language definition:** The active SCD2 version for a business key, usually marked by a flag and open-ended end date.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 11 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 11 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 11.

**Interview-ready definition:** The active SCD2 version for a business key, usually marked by a flag and open-ended end date. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Data contract

---

**Plain-language definition:** Explicit agreement about schema, meaning, quality, and change behavior.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 3 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 3 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 3.

**Interview-ready definition:** Explicit agreement about schema, meaning, quality, and change behavior. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Data lake

---

**Plain-language definition:** Storage for files and varied data, often before relational modeling.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Storage for files and varied data, often before relational modeling. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Data lineage

---

**Plain-language definition:** Evidence of how data moves and transforms from source to output.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Evidence of how data moves and transforms from source to output. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

## Data quality

---

**Plain-language definition:** Fitness of data for its intended use, measured by rules and evidence.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Fitness of data for its intended use, measured by rules and evidence. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Data type

---

**Plain-language definition:** A rule describing how a value is stored and validated, such as integer, date, text, or decimal.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 4 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 4 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 4.

**Interview-ready definition:** A rule describing how a value is stored and validated, such as integer, date, text, or decimal. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Data warehouse

---

**Plain-language definition:** Analytical database organized for stable reporting and history.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Analytical database organized for stable reporting and history. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## DataFrame

---

**Plain-language definition:** A tabular programming object with named columns and rows used by pandas or Spark.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 5 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 5 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 5.

**Interview-ready definition:** A tabular programming object with named columns and rows used by pandas or Spark. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## DDL

---

**Plain-language definition:** Data Definition Language: statements that create or alter database objects.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 4 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 4 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 4.

**Interview-ready definition:** Data Definition Language: statements that create or alter database objects. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Deduplication

---

**Plain-language definition:** Choosing or merging one trusted representation of repeated records.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Choosing or merging one trusted representation of repeated records. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Delta Lake

---

**Plain-language definition:** Transactional table layer over lake files used by Databricks.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 8 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 8 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 8.

**Interview-ready definition:** Transactional table layer over lake files used by Databricks. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Delta table

---

**Plain-language definition:** A table backed by Delta Lake transaction metadata and data files.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 8 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 8 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 8.

**Interview-ready definition:** A table backed by Delta Lake transaction metadata and data files. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Dimension

---

**Plain-language definition:** Descriptive context such as customer, product, store, or date.

**Everyday analogy:** Like a catalog card that describes who, what, where, or when.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence,

lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Descriptive context such as customer, product, store, or date. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Dimensional model

---

**Plain-language definition:** Analytics model organized around facts and dimensions.

**Everyday analogy:** Like a catalog card that describes who, what, where, or when.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Analytics model organized around facts and dimensions. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## DirectQuery

---

**Plain-language definition:** A Power BI storage mode that sends queries to a source instead of importing all model data.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 16 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 16 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 16.

**Interview-ready definition:** A Power BI storage mode that sends queries to a source instead of importing all model data. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## DML

---

**Plain-language definition:** Data Manipulation Language: statements that insert, update, delete, or merge rows.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 4 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 4 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 4.

**Interview-ready definition:** Data Manipulation Language: statements that insert, update, delete, or merge rows. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Drill-through

---

**Plain-language definition:** Power BI navigation that opens a detail page filtered to the selected context.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 17 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 17 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 17.

**Interview-ready definition:** Power BI navigation that opens a detail page filtered to the selected context. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)



## Effective date

---

**Plain-language definition:** The date/time from which a dimension version or business rule is valid.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 11 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 11 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 11.

**Interview-ready definition:** The date/time from which a dimension version or business rule is valid. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## ELT

---

**Plain-language definition:** Extract and load first, then transform in the target platform.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Extract and load first, then transform in the target platform. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Entity relationship diagram

---

**Plain-language definition:** Diagram of tables/entities and their relationships.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Diagram of tables/entities and their relationships. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Environment variable

---

**Plain-language definition:** A process-level named value used for configuration without hard-coding it in source code.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 6 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 6 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 6.

**Interview-ready definition:** A process-level named value used for configuration without hard-coding it in source code. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## ETL

---

**Plain-language definition:** Extract, transform, and load.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it

connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Extract, transform, and load. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

---

## Fact

**Plain-language definition:** A measurable business event or snapshot at declared grain.

**Everyday analogy:** Like a receipt line or daily stock count that records a measurable event or state.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** A measurable business event or snapshot at declared grain. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

---

## Filter context

**Plain-language definition:** The set of filters under which a DAX measure is evaluated.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 16 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 16 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 16.

**Interview-ready definition:** The set of filters under which a DAX measure is evaluated. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

---

## Foreign key

**Plain-language definition:** Column that references a key in another table.

**Everyday analogy:** Like an ID number on a library card: it lets systems point to the intended record without relying on a person's name.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 4 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 4 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 4.

**Interview-ready definition:** Column that references a key in another table. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

---

## Full refresh

**Plain-language definition:** Replace/rebuild the entire target scope.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.



**Interview-ready definition:** Replace/rebuild the entire target scope. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## GitHub Actions

---

**Plain-language definition:** GitHub's event-driven workflow automation service.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 19 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 19 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 19.

**Interview-ready definition:** GitHub's event-driven workflow automation service. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Gold

---

**Plain-language definition:** Business-ready dimensional and aggregate layer.

**Everyday analogy:** Like moving merchandise from receiving, to inspection/cleaning, to labeled shelves ready for shoppers.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Business-ready dimensional and aggregate layer. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Grain

---

**Plain-language definition:** What exactly one row represents.

**Everyday analogy:** Like deciding whether one receipt line, one whole receipt, or one day is represented by one row before adding numbers.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** What exactly one row represents. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## High-water mark

---

**Plain-language definition:** Latest source position fully processed by a successful incremental load.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 7 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 7 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 7.

**Interview-ready definition:** Latest source position fully processed by a successful incremental load. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Idempotency key

---

**Plain-language definition:** A stable identifier used to recognize a repeated request, batch, or business event.

**Everyday analogy:** Like an ID number on a library card: it lets systems point to the intended record without relying on a person's name.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 7 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 7 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 7.

**Interview-ready definition:** A stable identifier used to recognize a repeated request, batch, or business event. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Idempotent

---

**Plain-language definition:** Safe to repeat without changing the trusted result or duplicating data.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 7 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 7 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 7.

**Interview-ready definition:** Safe to repeat without changing the trusted result or duplicating data. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Import mode

---

**Plain-language definition:** A Power BI storage mode that copies source data into the in-memory semantic model during refresh.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 16 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 16 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 16.

**Interview-ready definition:** A Power BI storage mode that copies source data into the in-memory semantic model during refresh. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Incremental load

---

**Plain-language definition:** Process only new or changed data since a known point.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 7 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 7 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 7.

**Interview-ready definition:** Process only new or changed data since a known point. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Index

---

**Plain-language definition:** Database structure that can speed selected access patterns at write/storage cost.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it

connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 4 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 4 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 4.

**Interview-ready definition:** Database structure that can speed selected access patterns at write/storage cost. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Ingestion

---

**Plain-language definition:** Bringing data from a source into the platform.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Bringing data from a source into the platform. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## JDBC

---

**Plain-language definition:** Java-based database connectivity commonly used by Spark.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 8 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 8 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 8.

**Interview-ready definition:** Java-based database connectivity commonly used by Spark. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Lakeflow Jobs

---

**Plain-language definition:** Databricks orchestration for tasks, dependencies, schedules, and runs.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 14 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 14 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 14.

**Interview-ready definition:** Databricks orchestration for tasks, dependencies, schedules, and runs. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Lakehouse

---

**Plain-language definition:** Platform combining lake flexibility with warehouse-style table reliability.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Platform combining lake flexibility with warehouse-style table reliability. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Late-arriving data

---

**Plain-language definition:** A record that reaches the platform after related data or expected time.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** A record that reaches the platform after related data or expected time. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Late-arriving dimension

---

**Plain-language definition:** A dimension business key that is unavailable when a related fact first arrives.

**Everyday analogy:** Like a catalog card that describes who, what, where, or when.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 11 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 11 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 11.

**Interview-ready definition:** A dimension business key that is unavailable when a related fact first arrives. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Late-arriving fact

---

**Plain-language definition:** A fact/event that arrives after the period or related records normally expected for it.

**Everyday analogy:** Like a receipt line or daily stock count that records a measurable event or state.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 12 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 12 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 12.

**Interview-ready definition:** A fact/event that arrives after the period or related records normally expected for it. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Local repository

---

**Plain-language definition:** The Git-tracked project copy on your computer.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** The Git-tracked project copy on your computer. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Low-water mark

---

**Plain-language definition:** The starting boundary for an incremental range, often the prior successful high-water mark.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 7 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 7 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 7.

**Interview-ready definition:** The starting boundary for an incremental range, often the prior successful high-water mark. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Manifest

---

**Plain-language definition:** File describing batch contents, row counts, checksums, and metadata.

**Everyday analogy:** Like a packing slip listing every box, quantity, and seal number in one shipment.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 7 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 7 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 7.

**Interview-ready definition:** File describing batch contents, row counts, checksums, and metadata. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Measure

---

**Plain-language definition:** Reusable calculation evaluated under a filter context.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 16 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 16 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 16.

**Interview-ready definition:** Reusable calculation evaluated under a filter context. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Medallion architecture

---

**Plain-language definition:** Bronze/Silver/Gold layering pattern.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Bronze/Silver/Gold layering pattern. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Merge conflict

---

**Plain-language definition:** A situation where Git cannot automatically combine competing changes to the same content.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 1 to connect source records, batch evidence,

lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 1 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 1.

**Interview-ready definition:** A situation where Git cannot automatically combine competing changes to the same content. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Natural key

---

**Plain-language definition:** Source/business identifier; often the same as business key.

**Everyday analogy:** Like an ID number on a library card: it lets systems point to the intended record without relying on a person's name.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 4 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 4 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 4.

**Interview-ready definition:** Source/business identifier; often the same as business key. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Non-additive measure

---

**Plain-language definition:** A ratio, percentage, or other measure that should not be summed across rows or groups.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** A ratio, percentage, or other measure that should not be summed across rows or groups. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Normalized model

---

**Plain-language definition:** Operational design that separates entities to control update consistency.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Operational design that separates entities to control update consistency. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## ODBC

---

**Plain-language definition:** A standard database-connectivity interface used by drivers and applications.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 2 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 2 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 2.

**Interview-ready definition:** A standard database-connectivity interface used by drivers and applications. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.



## Orchestration

---

**Plain-language definition:** Coordinating tasks, order, parameters, retries, schedules, and failure handling.

**Everyday analogy:** Like a stage manager deciding which actor enters, what must finish first, and what happens after a mis-take.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Coordinating tasks, order, parameters, retries, schedules, and failure handling. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Pipeline

---

**Plain-language definition:** A connected sequence that moves and transforms data from source to consumption.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** A connected sequence that moves and transforms data from source to consumption. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Primary key

---

**Plain-language definition:** Column or set of columns uniquely identifying a table row.

**Everyday analogy:** Like an ID number on a library card: it lets systems point to the intended record without relying on a person's name.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 4 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 4 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 4.

**Interview-ready definition:** Column or set of columns uniquely identifying a table row. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Pull

---

**Plain-language definition:** Fetch remote changes and integrate them into the current branch.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 1 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 1 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 1.

**Interview-ready definition:** Fetch remote changes and integrate them into the current branch. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Pull request

---

**Plain-language definition:** A proposed set of branch changes submitted for review before merge.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 1 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 1 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 1.

**Interview-ready definition:** A proposed set of branch changes submitted for review before merge. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Push

---

**Plain-language definition:** Send local commits and branch references to a remote repository.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 1 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 1 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 1.

**Interview-ready definition:** Send local commits and branch references to a remote repository. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## PySpark

---

**Plain-language definition:** Python interfaces for Apache Spark distributed data processing.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 9 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 9 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 9.

**Interview-ready definition:** Python interfaces for Apache Spark distributed data processing. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Quarantine

---

**Plain-language definition:** Retained records excluded from trusted output with a documented rule.

**Everyday analogy:** Like a clearly labeled inspection area that keeps questionable goods without pretending they disappeared.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 10 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 10 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 10.

**Interview-ready definition:** Retained records excluded from trusted output with a documented rule. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Query

---

**Plain-language definition:** A statement that asks a database/lakehouse to return or change data.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.



**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 4 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 4 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 4.

**Interview-ready definition:** A statement that asks a database/lakehouse to return or change data. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Reconciliation

---

**Plain-language definition:** Comparing independently derived counts/amounts across boundaries.

**Everyday analogy:** Like counting packages at shipping and receiving and investigating every difference.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 7 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 7 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 7.

**Interview-ready definition:** Comparing independently derived counts/amounts across boundaries. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Record hash

---

**Plain-language definition:** Hash of selected attributes used for change detection or duplicate evidence.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Hash of selected attributes used for change detection or duplicate evidence. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Remote repository

---

**Plain-language definition:** The network-hosted project copy, such as the GitHub repository named origin.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** The network-hosted project copy, such as the GitHub repository named origin. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Repository

---

**Plain-language definition:** A project folder whose files and history are tracked by Git.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 1 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 1 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 1.

**Interview-ready definition:** A project folder whose files and history are tracked by Git. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

## Row

---

**Plain-language definition:** One record in a table at the table's declared grain.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** One record in a table at the table's declared grain. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Runner

---

**Plain-language definition:** The machine or environment that executes a GitHub Actions job.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 19 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 19 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 19.

**Interview-ready definition:** The machine or environment that executes a GitHub Actions job. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Schema

---

**Plain-language definition:** Namespace and structural definition of database objects.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Namespace and structural definition of database objects. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Schema drift

---

**Plain-language definition:** Unexpected or evolving source columns/types.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Unexpected or evolving source columns/types. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Secret

---

**Plain-language definition:** A credential or sensitive value that must not be printed or committed.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 18 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 18 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 18.

**Interview-ready definition:** A credential or sensitive value that must not be printed or committed. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Semantic layer

---

**Plain-language definition:** Relationships, measures, formatting, descriptions, and business definitions shared by reports.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 16 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 16 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 16.

**Interview-ready definition:** Relationships, measures, formatting, descriptions, and business definitions shared by reports. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Semantic model

---

**Plain-language definition:** The Power BI layer of tables, relationships, measures, formatting, descriptions, and security used by reports.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 16 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 16 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 16.

**Interview-ready definition:** The Power BI layer of tables, relationships, measures, formatting, descriptions, and security used by reports. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Semi-additive measure

---

**Plain-language definition:** A measure that can be summed across some dimensions but not all, such as inventory balance across products but not across snapshot dates.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** A measure that can be summed across some dimensions but not all, such as inventory balance across products but not across snapshot dates. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Serverless compute

---

**Plain-language definition:** Provider-managed compute that starts and scales without a user-managed cluster, subject to account features and quotas.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 8 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 8 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 8.

**Interview-ready definition:** Provider-managed compute that starts and scales without a user-managed cluster, subject to account features and quotas. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Silver

---

**Plain-language definition:** Cleaned, typed, conformed, deduplicated trusted-detail layer.

**Everyday analogy:** Like moving merchandise from receiving, to inspection/cleaning, to labeled shelves ready for shoppers.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Cleaned, typed, conformed, deduplicated trusted-detail layer. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Slicer

---

**Plain-language definition:** A Power BI visual control that applies a selected filter to other visuals according to interactions and relationships.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 17 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 17 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 17.

**Interview-ready definition:** A Power BI visual control that applies a selected filter to other visuals according to interactions and relationships. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Slowly Changing Dimension Type 1

---

**Plain-language definition:** Overwrite a dimension attribute without preserving the old value.

**Everyday analogy:** Like a catalog card that describes who, what, where, or when.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 11 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 11 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 11.

**Interview-ready definition:** Overwrite a dimension attribute without preserving the old value. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Slowly Changing Dimension Type 2

---

**Plain-language definition:** Insert dated versions to preserve attribute history.

**Everyday analogy:** Like a catalog card that describes who, what, where, or when.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 11 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 11 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 11.

**Interview-ready definition:** Insert dated versions to preserve attribute history. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Snapshot fact

---

**Plain-language definition:** Fact capturing state at a point in time, such as daily inventory.

**Everyday analogy:** Like a receipt line or daily stock count that records a measurable event or state.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Fact capturing state at a point in time, such as daily inventory. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Staging area

---

**Plain-language definition:** The selected file versions that will enter the next Git commit.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 1 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 1 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 1.

**Interview-ready definition:** The selected file versions that will enter the next Git commit. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Star schema

---

**Plain-language definition:** Fact table connected to descriptive dimensions.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Fact table connected to descriptive dimensions. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Surrogate key

---

**Plain-language definition:** Warehouse-controlled key independent of source business identifier.

**Everyday analogy:** Like an ID number on a library card: it lets systems point to the intended record without relying on a person's name.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 11 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 11 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 11.

**Interview-ready definition:** Warehouse-controlled key independent of source business identifier. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

## Tooltip page

---

**Plain-language definition:** A Power BI report page designed to appear as rich hover detail for another visual.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 17 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 17 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 17.

**Interview-ready definition:** A Power BI report page designed to appear as rich hover detail for another visual. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Transaction

---

**Plain-language definition:** All-or-nothing database unit that commits or rolls back.

**Everyday analogy:** Like a checkout that either completes every charge/receipt step or cancels the whole attempt.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 4 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 4 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 4.

**Interview-ready definition:** All-or-nothing database unit that commits or rolls back. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Transformation

---

**Plain-language definition:** Changing structure, type, quality, or meaning under explicit rules.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** Changing structure, type, quality, or meaning under explicit rules. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Unity Catalog volume

---

**Plain-language definition:** Governed file storage namespace accessible through Databricks.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 8 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 8 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 8.

**Interview-ready definition:** Governed file storage namespace accessible through Databricks. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Unknown member

---

**Plain-language definition:** Dimension row, usually key 0, for unresolved context.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it

connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 11 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 11 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 11.

**Interview-ready definition:** Dimension row, usually key 0, for unresolved context. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Watermark

---

**Plain-language definition:** See high-water mark.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 0 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 0 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 0.

**Interview-ready definition:** See high-water mark. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)

## Working tree

---

**Plain-language definition:** The checked-out files and current unstaged/staged state on your computer.

**Everyday analogy:** Like a labeled part of a store or delivery process: the label tells you what belongs there and how it connects to other parts.

**Northstar Retail example:** In Northstar Retail, this term is applied in Phase 1 to connect source records, batch evidence, lakehouse tables, warehouse objects, Power BI results, or Git history.

**Related terms:** See the Phase 1 concept and object walkthroughs, plus neighboring terms in this glossary.

**First phase where it appears:** Phase 1.

**Interview-ready definition:** The checked-out files and current unstaged/staged state on your computer. In a design review, I would connect it to the exact Northstar object, grain, validation, and tradeoff.

[Back to the glossary top](#)



# Complete troubleshooting index

[Back to the master table of contents](#)

This index points to detailed repair instructions and does not replace the troubleshooting beside each action.

Tool/area	Error or symptom	Most likely phase	First diagnostic	Detailed repair anchor
Windows paths	<code>`Test-Path`</code> returns <code>`False`</code> or a file opens from the wrong similarly named folder	1	<code>`Get-Location`</code> ; <code>`Test-Path`</code> ; copy path from File Explorer; compare with <code>`git rev-parse --show-toplevel`</code>	<a href="#">Repair</a>
PowerShell	Prompt does not begin with <code>`PS`</code> or command syntax behaves differently	1	Read the window title/prompt; run <code>`\$PSVersionTable.PSVersion`</code>	<a href="#">Repair</a>
PowerShell	Script execution is blocked	2	Read the execution-policy error; run <code>`Get-ExecutionPolicy -List`</code>	<a href="#">Repair</a>
Git	<code>`fatal: not a git repository`</code>	1	<code>`Get-Location`</code> ; <code>`Get-ChildItem -Force`</code> ; <code>`git rev-parse --show-toplevel`</code>	<a href="#">Repair</a>
Git	Working tree contains unexpected modified/untracked files	1	<code>`git status --short`</code> ; targeted <code>`git diff -- path`</code>	<a href="#">Repair</a>
Git	Push rejected because remote contains work	1	<code>`git fetch origin`</code> ; graph log	<a href="#">Repair</a>
GitHub	Branch or commit is not visible on the repository page	1	Compare remote URL, branch name, push output, and GitHub branch selector	<a href="#">Repair</a>
VS Code	<code>`code .`</code> is not recognized	2	Restart terminal; run <code>`Get-Command code -ErrorAction SilentlyContinue`</code>	<a href="#">Repair</a>
VS Code	Wrong Python interpreter selected	5	Command Palette > Python: Select Interpreter; compare path with <code>`.venv`</code>	<a href="#">Repair</a>
Python	<code>`py`</code> or <code>`python`</code> is not recognized	2	Run <code>`Get-Command py,python -ErrorAction SilentlyContinue`</code>	<a href="#">Repair</a>
Virtual environment	Activation script blocked or prompt does not show <code>`.venv`</code>	5	Test <code>`.venv\Scripts\Activate.ps1`</code> ; inspect policy and interpreter path	<a href="#">Repair</a>
Package installation	<code>`ModuleNotFoundError`</code> after <code>`pip install`</code>	5	Run <code>`python -m pip --version`</code> and <code>`python -c "import sys; print(sys.executable)"`</code>	<a href="#">Repair</a>
SQL Server	Server/instance cannot be found	2	Inspect SQL Server Configuration Manager/services; SSMS Browse; record exact instance	<a href="#">Repair</a>
SSMS	Login failed for user	2	Identify Windows vs SQL authentication and exact server; check service status	<a href="#">Repair</a>
SSMS	Object already exists	4	Inspect Object Explorer and metadata definition	<a href="#">Repair</a>
PostgreSQL	Connection refused on port 5432	2	Check PostgreSQL service, host, port, and <code>`pg_isready`</code> if available	<a href="#">Repair</a>
pgAdmin	Server appears but remains disconnected	2	Right-click server > Connect Server; inspect host/port/user and first error	<a href="#">Repair</a>
ODBC	Data source name/driver not found	2	Python: <code>`pyodbc.drivers()`</code> ; compare exact driver string	<a href="#">Repair</a>
Database authentication	Python loader authenticates differently from GUI	6	Print non-secret connection mode/server/database; test minimal query with same account	<a href="#">Repair</a>



Tool/area	Error or symptom	Most likely phase	First diagnostic	Detailed repair anchor
Synthetic generator	Foreign-key generation fails or child references are missing	5	Inspect generation order, parent ID sets, seed/config, first invalid row	<a href="#">Repair</a>
Synthetic generator	Rerun produces different results with same configuration	5	Compare seed, package versions, config, timestamps, unordered sets	<a href="#">Repair</a>
Source loading	Primary-key violation on rerun	6	Inspect load audit/batch ID and existing row counts	<a href="#">Repair</a>
Extraction	Watermark advances after a failed/partial batch	7	Inspect attempt status, manifest completion, and saved watermark	<a href="#">Repair</a>
Extraction	Manifest count or checksum differs	7	Recount source/output; recompute checksum; compare file path and byte size	<a href="#">Repair</a>
Databricks	Workspace menu/label differs	8	Identify cloud, workspace tier, permissions, and current official docs	<a href="#">Repair</a>
Databricks	Catalog/schema/volume permission denied	8	Inspect current user, object owner, grants, writable default catalog/My Files	<a href="#">Repair</a>
Databricks	Compute option unavailable	8	Check serverless/classic availability, quotas, entitlement, workspace status	<a href="#">Repair</a>
PySpark	'AnalysisException' table or column not found	9	Print computed table name, 'SHOW TABLES', and 'printSchema()'	<a href="#">Repair</a>
Delta Lake	Same-batch rerun duplicates rows	9	Count by batch and business-event key before/after rerun	<a href="#">Repair</a>
Silver	Input count does not equal accepted plus rejected	10	Break out parse failures, duplicates, filters, joins, and null behavior	<a href="#">Repair</a>
SCD2	Multiple current dimension rows per business key	11	Group by business key where 'is_current'; inspect merge match condition	<a href="#">Repair</a>
SCD2	Effective date windows overlap	11	Self-join versions or use lag/lead validation	<a href="#">Repair</a>
Gold facts	Unknown key 0 rate is unexpectedly high	12	Group unresolved lookups by source key, dimension, and event date	<a href="#">Repair</a>
Quality gate	Test is green but reads the wrong batch/table	13	Print fully qualified object and batch parameter in result record	<a href="#">Repair</a>
Databricks Jobs	Task failed and dependent tasks skipped	14	Open task output and first caused-by exception; inspect parameters and compute	<a href="#">Repair</a>
Databricks Jobs	Retry repeats deterministic quality failure	14	Compare attempts and identical failure data	<a href="#">Repair</a>
Warehouse publication	Downloaded Gold file belongs to wrong run	15	Compare batch ID, manifest, filename, checksum, export timestamp	<a href="#">Repair</a>
SQL Server warehouse	String truncation or decimal/date conversion failure	15	Load staging with raw value visibility; inspect max lengths and invalid examples	<a href="#">Repair</a>
Power BI connections	SQL Server certificate/encryption warning or connection failure	16	Verify server/database/authentication and Microsoft connector guidance	<a href="#">Repair</a>
Power BI relationships	Duplicate values on the one side prevent relationship	16	Profile candidate dimension key and count duplicates/nulls	<a href="#">Repair</a>
DAX	Measure differs from SQL	16	Match date role, filters, relationships, distinct grain, numerator/denominator and rounding	<a href="#">Repair</a>
Power BI report	Slicer does not affect a visual	17	Use Edit interactions and inspect relationship path	<a href="#">Repair</a>

Tool/area	Error or symptom	Most likely phase	First diagnostic	Detailed repair anchor
Power BI report	Snapshot inventory is overstated across dates	17	Inspect visual axis and aggregation of snapshot balance	<a href="#">Repair</a>
Secrets	<code>.env`</code> or token appears in <code>`git status`/diff</code>	18	<code>`git check-ignore -v`</code> ; inspect staged diff and history	<a href="#">Repair</a>
Cost	Databricks compute continues running	18	Open Compute and Job run pages; inspect active resources	<a href="#">Repair</a>
GitHub Actions	Workflow file is ignored or not listed	19	Confirm path <code>.github/workflows/*.yaml`</code> , branch, YAML syntax, Actions permissions	<a href="#">Repair</a>
GitHub Actions	Tests pass locally but fail on runner	19	Compare Python version, OS, paths/case, dependency lock, environment assumptions	<a href="#">Repair</a>
Markdown	README links or anchors are broken	20	Run link validator; preview in VS Code and GitHub	<a href="#">Repair</a>
Portfolio	Claim has no verified evidence	20	Use claim-to-evidence map and locate commit/run/query/screenshot	<a href="#">Repair</a>
Interview	Answer sounds memorized and cannot survive follow-up	21	Trace answer to one actual file, object, test, error, and tradeoff	<a href="#">Repair</a>
Optional extension	Extension breaks required path	22	Compare branch, config, costs, dependencies, rollback plan	<a href="#">Repair</a>

## Troubleshooting 1 — Windows paths: Test-Path returns False or a file opens from the wrong similarly named folder

### Exact or representative message/symptom:

``Test-Path`returnsFalse` or a file opens from the wrong similarly named folder

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 1.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

`Get-Location``; ``Test-Path``; copy path from File Explorer; compare with ``git rev-parse --show-toplevel``

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Correct only the path/name. Do not copy or delete until the real root is proven.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 1, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 2 — PowerShell: Prompt does not begin with PS or command syntax behaves differently

---

**Exact or representative message/symptom:**

Prompt does not begin with PS or command syntax behaves differently

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 1.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Read the window title/prompt; run `$PSVersionTable.PSVersion`

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Open Windows PowerShell/PowerShell explicitly and rerun the observational check.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 1, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 3 — PowerShell: Script execution is blocked

---

**Exact or representative message/symptom:**

Script execution is blocked

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 2.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Read the execution-policy error; run `Get-ExecutionPolicy -List``

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Use the narrow process-scope policy only when required and authorized; do not weaken machine policy globally.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 2, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 4 — Git: fatal: not a git repository

---

**Exact or representative message/symptom:**

`fatal: not a git repository`

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 1.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

`Get-Location`; `Get-ChildItem -Force`; `git rev-parse --show-toplevel`

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Move to the actual clone; do not initialize a second repository inside it.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 1, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 5 — Git: Working tree contains unexpected modified/untracked files

---

### Exact or representative message/symptom:

Working tree contains unexpected modified/untracked files

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 1.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

```
git status --short`; targeted `git diff -- path
```

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Preserve and inspect. Stage only intended files; do not reset or clean blindly.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 1, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 6 — Git: Push rejected because remote contains work

---

### Exact or representative message/symptom:

Push rejected because remote contains work

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 1.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

```
``git fetch origin; graph log
```

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Integrate remote changes safely; never force-push an unexplained divergence.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 1, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 7 — GitHub: Branch or commit is not visible on the repository page

---

### Exact or representative message/symptom:

Branch or commit is not visible on the repository page

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 1.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Compare remote URL, branch name, push output, and GitHub branch selector

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Push the correct branch to the correct remote; verify account/repository before retrying.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 1, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 8 — VS Code: `code .` is not recognized

---

### Exact or representative message/symptom:

`code .` is not recognized

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 2.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Restart terminal; run `Get-Command code -ErrorAction SilentlyContinue`

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Install/repair VS Code PATH option or open the folder through File > Open Folder.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 2, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 9 — VS Code: Wrong Python interpreter selected

---

### Exact or representative message/symptom:

Wrong Python interpreter selected

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 5.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Command Palette > Python: Select Interpreter; compare path with `.venv`

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.

- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Select the repository `.venv` interpreter and reopen terminal.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 5, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 10 — Python: `py` or `python` is not recognized

---

**Exact or representative message/symptom:**

```
``pyorpython is not recognized
```

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 2.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Run `Get-Command py,python -ErrorAction SilentlyContinue`

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Install/repair the official Python Windows installation and restart the console.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 2, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 11 — Virtual environment: Activation script blocked or prompt does not show `.venv`

---

**Exact or representative message/symptom:**

```
Activation script blocked or prompt does not show .venv``
```



**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 5.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Test `.venv\Scripts\Activate.ps1`; inspect policy and interpreter path

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Use process-scope policy if permitted or call `.venv\Scripts\python.exe` directly.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 5, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 12 — Package installation: `ModuleNotFoundError` after `pip install`

---

**Exact or representative message/symptom:**

`ModuleNotFoundError` after `pip install`

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 5.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Run `python -m pip --version` and `python -c "import sys; print(sys.executable)"`

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Install with the same interpreter: `python -m pip install ...`; do not mix global and venv pip.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 5, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 13 — SQL Server: Server/instance cannot be found

---

**Exact or representative message/symptom:**

Server/instance cannot be found

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 2.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Inspect SQL Server Configuration Manager/services; SSMS Browse; record exact instance

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Start the intended service or use the exact installed instance name; do not guess `localhost` variants indefinitely.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 2, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 14 — SSMS: Login failed for user

---

**Exact or representative message/symptom:**

Login failed for user

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 2.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Identify Windows vs SQL authentication and exact server; check service status

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Use the authentication mode configured during install; do not expose passwords in screenshots/config.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 2, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 15 — SSMS: Object already exists

---

**Exact or representative message/symptom:**

Object already exists

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 4.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Inspect Object Explorer and metadata definition

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Compare existing definition and preserve data; run idempotent script or documented reset only when safe.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 4, find the exact action that produced the symptom, and complete its `Do not continue unless` gate.

[Back to the troubleshooting index](#)

## Troubleshooting 16 — PostgreSQL: Connection refused on port 5432

---

**Exact or representative message/symptom:**

Connection refused on port 5432

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 2.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Check PostgreSQL service, host, port, and `pg_isready` if available

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Start the intended service or correct the port; do not reinstall before checking service/config.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 2, find the exact action that produced the symptom, and complete its `Do not continue unless` gate.

[Back to the troubleshooting index](#)

## Troubleshooting 17 — pgAdmin: Server appears but remains disconnected

---

**Exact or representative message/symptom:**

Server appears but remains disconnected

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 2.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Right-click server > Connect Server; inspect host/port/user and first error

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Correct the connection property or password; avoid saving credentials on shared machines.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 2, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 18 — ODBC: Data source name/driver not found

---

**Exact or representative message/symptom:**

Data source name/driver not found

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 2.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Python: `pyodbc.drivers()`; compare exact driver string

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Install current 64-bit Microsoft ODBC driver and restart applications; use exact returned name.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 2, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 19 — Database authentication: Python loader authenticates differently from GUI

---

### Exact or representative message/symptom:

Python loader authenticates differently from GUI

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 6.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Print non-secret connection mode/server/database; test minimal query with same account

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Align driver/authentication settings and environment variables; do not hard-code credentials.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 6, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 20 — Synthetic generator: Foreign-key generation fails or child references are missing

---

### Exact or representative message/symptom:

Foreign-key generation fails or child references are missing

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 5.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Inspect generation order, parent ID sets, seed/config, first invalid row

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Generate parent entities before children and rerun to a new clean output batch.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 5, find the exact action that produced the symptom, and complete its `Do not continue unless` gate.

[Back to the troubleshooting index](#)

## Troubleshooting 21 — Synthetic generator: Rerun produces different results with same configuration

---

### Exact or representative message/symptom:

Rerun produces different results with same configuration

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 5.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Compare seed, package versions, config, timestamps, unordered sets

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Stabilize seeds and deterministic ordering; separate expected run timestamps from deterministic business data.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 5, find the exact action that produced the symptom, and complete its `Do not continue unless` gate.

[Back to the troubleshooting index](#)

## Troubleshooting 22 — Source loading: Primary-key violation on rerun

---

### Exact or representative message/symptom:

Primary-key violation on rerun

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 6.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Inspect load audit/batch ID and existing row counts

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Use documented reset/upsert/duplicate-batch policy; never silently append.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 6, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 23 — Extraction: Watermark advances after a failed/partial batch

---

### Exact or representative message/symptom:

Watermark advances after a failed/partial batch

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 7.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Inspect attempt status, manifest completion, and saved watermark

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.



- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Restore prior successful watermark; advance only in the same success transaction/atomic state change.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 7, find the exact action that produced the symptom, and complete its `Do not continue unless` gate.

[Back to the troubleshooting index](#)

## Troubleshooting 24 — Extraction: Manifest count or checksum differs

---

**Exact or representative message/symptom:**

Manifest count or checksum differs

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 7.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Recount source/output; recompute checksum; compare file path and byte size

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Discard or quarantine failed batch and re-extract; do not edit manifest to match a bad file.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 7, find the exact action that produced the symptom, and complete its `Do not continue unless` gate.

[Back to the troubleshooting index](#)

## Troubleshooting 25 — Databricks: Workspace menu/label differs

---

**Exact or representative message/symptom:**

Workspace menu/label differs

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 8.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Identify cloud, workspace tier, permissions, and current official docs

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Use equivalent object-based path; record variance rather than claiming feature absence.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 8, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 26 — Databricks: Catalog/schema/volume permission denied

---

**Exact or representative message/symptom:**

Catalog/schema/volume permission denied

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 8.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Inspect current user, object owner, grants, writable default catalog/My Files

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Choose an authorized location or request permission; do not switch to an unknown shared location.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 8, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 27 — Databricks: Compute option unavailable

---

**Exact or representative message/symptom:**

Compute option unavailable

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 8.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Check serverless/classic availability, quotas, entitlement, workspace status

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Use available supported compute or document blocker; do not create costly resources blindly.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 8, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 28 — PySpark: AnalysisException table or column not found

---

**Exact or representative message/symptom:**

`AnalysisException table or column not found`

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 9.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Print computed table name, SHOW TABLES, and printSchema()

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Correct catalog/schema/table/column mapping; do not add fake columns to bypass the error.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 9, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 29 — Delta Lake: Same-batch rerun duplicates rows

---

**Exact or representative message/symptom:**

Same-batch rerun duplicates rows

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 9.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Count by batch and business-event key before/after rerun

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Implement batch rejection, replaceWhere, merge, or deterministic dedupe and test again.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 9, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 30 — Silver: Input count does not equal accepted plus rejected

---

**Exact or representative message/symptom:**

Input count does not equal accepted plus rejected

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 10.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Break out parse failures, duplicates, filters, joins, and null behavior

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Add explicit accounting category/rejection records; eliminate silent drops.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 10, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 31 — SCD2: Multiple current dimension rows per business key

---

**Exact or representative message/symptom:**

Multiple current dimension rows per business key

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 11.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Group by business key where is\_current; inspect merge match condition

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Close prior current row and enforce deterministic one-current-row invariant.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 11, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 32 — SCD2: Effective date windows overlap

---

### Exact or representative message/symptom:

Effective date windows overlap

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 11.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Self-join versions or use lag/lead validation

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Correct boundary convention and rerun the affected business keys/batch safely.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 11, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 33 — Gold facts: Unknown key 0 rate is unexpectedly high

---

### Exact or representative message/symptom:

Unknown key 0 rate is unexpectedly high

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 12.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Group unresolved lookups by source key, dimension, and event date

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Repair conformance/late-arriving logic; keep facts countable and monitor rather than dropping.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 12, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 34 — Quality gate: Test is green but reads the wrong batch/table

---

### Exact or representative message/symptom:

Test is green but reads the wrong batch/table

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 13.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Print fully qualified object and batch parameter in result record

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.

- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Bind test to explicit parameters and store object/batch metadata with actual/expected values.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 13, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 35 — Databricks Jobs: Task failed and dependent tasks skipped

**Exact or representative message/symptom:**

Task failed and dependent tasks skipped

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 14.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Open task output and first caused-by exception; inspect parameters and compute

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Fix root cause, then use Repair run for failed/dependent tasks.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 14, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)



## Troubleshooting 36 — Databricks Jobs: Retry repeats deterministic quality failure

---

**Exact or representative message/symptom:**

Retry repeats deterministic quality failure

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 14.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Compare attempts and identical failure data

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Stop automatic retries for deterministic failures; fix data/code/contract first.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 14, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 37 — Warehouse publication: Downloaded Gold file belongs to wrong run

---

**Exact or representative message/symptom:**

Downloaded Gold file belongs to wrong run

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 15.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Compare batch ID, manifest, filename, checksum, export timestamp

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Locate/export the verified batch; do not rename an older file to match.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 15, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 38 — SQL Server warehouse: String truncation or decimal/date conversion failure

---

### Exact or representative message/symptom:

String truncation or decimal/date conversion failure

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 15.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Load staging with raw value visibility; inspect max lengths and invalid examples

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Correct mapping/staging type or reject invalid rows; keep transaction atomic.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 15, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 39 — Power BI connections: SQL Server certificate/encryption warning or connection failure

---

### Exact or representative message/symptom:

SQL Server certificate/encryption warning or connection failure

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 16.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Verify server/database/authentication and Microsoft connector guidance

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Use trusted local configuration or documented certificate choice; never expose credentials.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 16, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 40 — Power BI relationships: Duplicate values on the one side prevent relationship

---

### Exact or representative message/symptom:

Duplicate values on the one side prevent relationship

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 16.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Profile candidate dimension key and count duplicates/nulls

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Fix the dimension key/conformance upstream; do not force many-to-many to hide bad modeling.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 16, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 41 — DAX: Measure differs from SQL

---

### Exact or representative message/symptom:

Measure differs from SQL

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 16.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Match date role, filters, relationships, distinct grain, numerator/denominator and rounding

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Align semantic definition and SQL context before changing expression.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 16, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 42 — Power BI report: Slicer does not affect a visual

---

### Exact or representative message/symptom:

Slicer does not affect a visual

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 17.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Use Edit interactions and inspect relationship path

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

### Safest fix:

Enable intended interaction or document intentional isolation; verify no disconnected field.

### Alternative fix when applicable:

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

### How to verify the fix:

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

### How to return to the main tutorial:

Return to Phase 17, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 43 — Power BI report: Snapshot inventory is overstated across dates

---

### Exact or representative message/symptom:

Snapshot inventory is overstated across dates

### What it means in plain language:

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

### Most likely cause:

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 17.

### What not to do:

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

### First diagnostic action:

Inspect visual axis and aggregation of snapshot balance

### How to read the diagnostic result:

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.

- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Use latest-date or period-end logic; never sum balances across snapshot dates without meaning.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 17, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 44 — Secrets: .env or token appears in git status/diff

---

**Exact or representative message/symptom:**

``.env or token appears in git status/diff`

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 18.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

`git check-ignore -v; inspect staged diff and history`

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Unstage, add ignore rule, rotate exposed secret, and follow sensitive-data removal guidance if committed.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 18, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 45 — Cost: Databricks compute continues running

---

**Exact or representative message/symptom:**

Databricks compute continues running

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 18.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Open Compute and Job run pages; inspect active resources

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Stop unused compute/runs and record policy; do not assume browser close stops compute.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 18, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 46 — GitHub Actions: Workflow file is ignored or not listed

---

**Exact or representative message/symptom:**

Workflow file is ignored or not listed

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 19.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Confirm path `.github/workflows/*.yml`, branch, YAML syntax, Actions permissions

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Correct path/syntax and push; inspect repository Actions settings.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 19, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 47 — GitHub Actions: Tests pass locally but fail on runner

---

**Exact or representative message/symptom:**

Tests pass locally but fail on runner

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 19.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Compare Python version, OS, paths/case, dependency lock, environment assumptions

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Make test deterministic and infrastructure-free or explicitly configure required dependency.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 19, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

## Troubleshooting 48 — Markdown: README links or anchors are broken

---

**Exact or representative message/symptom:**

README links or anchors are broken

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 20.



**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Run link validator; preview in VS Code and GitHub

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Correct relative path/anchor and preserve case; do not link to local absolute paths.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 20, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 49 — Portfolio: Claim has no verified evidence

---

**Exact or representative message/symptom:**

Claim has no verified evidence

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 20.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Use claim-to-evidence map and locate commit/run/query/screenshot

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Rewrite as planned/implemented-but-unverified or gather real evidence; never fabricate.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 20, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 50 — Interview: Answer sounds memorized and cannot survive follow-up

---

**Exact or representative message/symptom:**

Answer sounds memorized and cannot survive follow-up

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 21.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Trace answer to one actual file, object, test, error, and tradeoff

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Use requirement → decision → implementation → validation → limitation structure.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 21, find the exact action that produced the symptom, and complete its Do not continue unless gate.

[Back to the troubleshooting index](#)

## Troubleshooting 51 — Optional extension: Extension breaks required path

---

**Exact or representative message/symptom:**

Extension breaks required path

**What it means in plain language:**

The observed application state does not match the expected context, object, data contract, permission, parameter, or verification condition.

**Most likely cause:**

A wrong path/context, missing prerequisite, version/tier difference, invalid source value, non-idempotent rerun, or unsafe assumption in Phase 22.

**What not to do:**

Do not delete, reset, overwrite, force-push, disable integrity rules, weaken a quality threshold, edit raw evidence, or expose a secret merely to remove the symptom.

**First diagnostic action:**

Compare branch, config, costs, dependencies, rollback plan

**How to read the diagnostic result:**

- **If it identifies the expected object/context:** Inspect the next closest dependency, parameter, data grain, or permission.
- **If it identifies a different object/context:** Stop. Correct the location before rerunning anything that changes state.
- **If the diagnostic itself fails:** Verify the tool/service is installed/running and use its official documentation entry in this workbook.

**Safest fix:**

Isolate extension on a branch/environment and retain verified batch baseline.

**Alternative fix when applicable:**

Use a documented, isolated development reset or fallback only after preserving the current state and confirming that no needed work/data will be lost.

**How to verify the fix:**

Repeat the original smallest verification, then rerun only the failed action. Compare actual/expected values and record the result.

**How to return to the main tutorial:**

Return to Phase 22, find the exact action that produced the symptom, and complete its `Do not continue unless gate`.

[Back to the troubleshooting index](#)

# Reference-code and file responsibility index

[Back to the master table of contents](#)

This index covers the actual project-file source inventory used to assemble the manual construction lessons. It excludes prior workbook exports, separate answer keys, separate journals, separate glossaries, separate troubleshooting files, separate screenshot checklists, and separate file manifests because those instructional systems are contained in this one workbook. The index is not a second deliverable and does not create any file. Your repository may differ; preserve and inspect current work before applying any instruction.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
1	docs/architecture.md	0	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 0 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
2	.gitignore	1	Repository safety/configuration file.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
3	config/logging.yml	1	YAML configuration for generation, logging, Databricks Jobs, or GitHub Actions.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
4	databricks/notebooks/11_publish_gold.py	1	Python module, script, Databricks source notebook, or automated test.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
5	databricks/notebooks/12_demo_queries.py	1	Python module, script, Databricks source notebook, or automated test.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
6	datasets/raw/files/.gitkeep	1	Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.	Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.	Ignore generated raw/log/local-secret artifacts; track reviewed source, tests, docs, workflow, and small fictional fixtures.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
7	datasets/raw/postgres/.gitkeep	1	Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.	Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.	Ignore generated raw/log/local-secret artifacts; track reviewed source, tests, docs, workflow, and small fictional fixtures.
8	datasets/raw/sqlserver/.gitkeep	1	Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.	Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.	Ignore generated raw/log/local-secret artifacts; track reviewed source, tests, docs, workflow, and small fictional fixtures.
9	docs/images/cloud_connected_flow.dot	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
10	docs/images/cloud_connected_flow.mmd	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
11	docs/images/cloud_connected_flow.svg	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
12	docs/images/gold_star_schema.dot	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
13	docs/images/gold_star_schema.mmd	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
14	docs/images/gold_star_schema.svg	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
15	docs/images/lineage.dot	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
16	docs/images/lineage.mmd	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
17	docs/images/lineage.svg	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
18	docs/images/local_first_flow.dot	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
19	docs/images/local_first_flow.mmd	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
20	docs/images/local_first_flow.svg	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
21	docs/images/overall_architecture.dot	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
22	docs/images/overall_architecture.mmd	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
23	docs/images/overall_architecture.svg	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
24	docs/images/postgres_erd.dot	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
25	docs/images/postgres_erd.mmd	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
26	docs/images/postgres_erd.svg	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
27	docs/images/sqlserver_erd.dot	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
28	docs/images/sqlserver_erd.mmd	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
29	docs/images/sqlserv-er_erd.svg	1	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
30	docs/repository_current_state.md	1	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
31	docs/repository_migration_plan.md	1	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
32	docs/testing_strategy.md	1	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
33	LICENSE	1	Project configuration, documentation, source, fixture, or support asset.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
34	logs/.gitkeep	1	Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.	Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.	Ignore generated raw/log/local-secret artifacts; track reviewed source, tests, docs, workflow, and small fictional fixtures.
35	Makefile-or-PowerShell-Commands.md	1	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.



#	Path	Phase	Responsibility	Creation/ownership route	Git rule
36	README.md	1	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
37	run_artifacts/.gitkeep	1	Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.	Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
38	scripts/__init__.py	1	Python module, script, Databricks source notebook, or automated test.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
39	scripts/loading/__init__.py	1	Python module, script, Databricks source notebook, or automated test.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
40	scripts/setup/__init__.py	1	Python module, script, Databricks source notebook, or automated test.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
41	scripts/setup/create_project_folders.ps1	1	Project configuration, documentation, source, fixture, or support asset.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
42	scripts/utilities/__init__.py	1	Python module, script, Databricks source notebook, or automated test.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
43	scripts/utilities/config.py	1	Python module, script, Databricks source notebook, or automated test.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
44	scripts/utilities/logging_utils.py	1	Python module, script, Databricks source notebook, or automated test.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
45	scripts/validation/__init__.py	1	Python module, script, Databricks source notebook, or automated test.	Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
46	config/dev.example.yml	2	YAML configuration for generation, logging, Databricks Jobs, or GitHub Actions.	Use the Phase 2 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
47	requirements-dev.txt	2	Project configuration, documentation, source, fixture, or support asset.	Use the Phase 2 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
48	requirements.txt	2	Project configuration, documentation, source, fixture, or support asset.	Use the Phase 2 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
49	scripts/setup/verify_local_tools.ps1	2	Project configuration, documentation, source, fixture, or support asset.	Use the Phase 2 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
50	docs/assumption_log.md	3	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 3 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
51	docs/business_requirements.md	3	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 3 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
52	docs/risk_register.md	3	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 3 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
53	docs/data_dictionary.csv	4	Structured mapping, definition, metadata, or small fictional data fixture.	Build rows in VS Code or a spreadsheet-like editor while preserving UTF-8 CSV structure; validate headers, row counts, duplicates, and Git diff.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
54	docs/data_dictionary.md	4	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
55	docs/source_to_target_mapping.csv	4	Structured mapping, definition, metadata, or small fictional data fixture.	Build rows in VS Code or a spreadsheet-like editor while preserving UTF-8 CSV structure; validate headers, row counts, duplicates, and Git diff.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
56	docs/source_to_target_mapping.md	4	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
57	sql/postgres/source/00_create_database.sql	4	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
58	sql/postgres/source/01_create_digital_schema.sql	4	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
59	sql/postgres/source/90_development_reset.sql	4	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
60	sql/sqlserver/source/01_create_erp_database.sql	4	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
61	sql/sqlserver/source/02_create_erp_tables.sql	4	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
62	sql/sqlserver/source/90_development_reset.sql	4	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
63	config/data_generation.yml	5	YAML configuration for generation, logging, Databricks Jobs, or GitHub Actions.	Use the Phase 5 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
64	datasets/sample/files/promotion_calendar.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
65	datasets/sample/files/returns.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
66	datasets/sample/files/shipment_tracking_events.jsonl	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
67	datasets/sample/files/supplier_cost_updates.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
68	datasets/sample/metadata/batch_manifest.json	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
69	datasets/sample/metadata/expected_quality_issues.json	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
70	datasets/sample/metadata/generation_summary.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
71	datasets/sample/metadata/validation_results.json	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
72	datasets/sample/postgres/campaign_touchpoints.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
73	datasets/sample/postgres/campaigns.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
74	datasets/sample/postgres/market-ing_spend.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
75	datasets/sample/postgres/web_events.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
76	datasets/sample/postgres/web_sessions.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
77	datasets/sample/postgres/web_users.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
78	datasets/sample/sqlserver/addresses.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
79	datasets/sample/sqlserver/customers.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
80	datasets/sample/sqlserver/employees.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
81	datasets/sample/sqlserver/inventory_snapshots.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
82	datasets/sample/sqlserver/inventory_transactions.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
83	datasets/sample/sqlserver/order_items.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
84	datasets/sample/sqlserver/orders.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
85	datasets/sample/sqlserver/payments.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
86	datasets/sample/sqlserver/product_categories.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
87	datasets/sample/sqlserver/products.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
88	datasets/sample/sqlserver/shipments.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
89	datasets/sample/sqlserver/stores.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
90	datasets/sample/sqlserver/suppliers.csv	5	Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
91	scripts/data_generation/__init__.py	5	Python module, script, Databricks source notebook, or automated test.	Use the Phase 5 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
92	scripts/data_generation/generate_northstar_data.py	5	Python module, script, Databricks source notebook, or automated test.	Use the Phase 5 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
93	scripts/data_generation/README.md	5	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 5 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
94	scripts/validation/validate_generated_data.py	5	Python module, script, Databricks source notebook, or automated test.	Use the Phase 5 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
95	tests/unit/test_generator.py	5	Python module, script, Databricks source notebook, or automated test.	Use the Phase 5 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
96	.env.example	6	Repository safety/configuration file.	Use the Phase 6 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Ignore generated raw/log/local-secret artifacts; track reviewed source, tests, docs, workflow, and small fictional fixtures.
97	scripts/loading/load_postgres_source.py	6	Python module, script, Databricks source notebook, or automated test.	Use the Phase 6 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.



#	Path	Phase	Responsibility	Creation/ownership route	Git rule
98	scripts/loading/load_sqlserv-er_source.py	6	Python module, script, Databricks source notebook, or automated test.	Use the Phase 6 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
99	sql/postgres/validation/01_validate_digital_source.sql	6	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 6 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
100	sql/sqlserver/validation/01_validate_erp_source.sql	6	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 6 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
101	scripts/extraction/__init__.py	7	Python module, script, Databricks source notebook, or automated test.	Use the Phase 7 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
102	scripts/extraction/extract_local_sources.py	7	Python module, script, Databricks source notebook, or automated test.	Use the Phase 7 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
103	scripts/utilities/checksums.py	7	Python module, script, Databricks source notebook, or automated test.	Use the Phase 7 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
104	scripts/utilities/database.py	7	Python module, script, Databricks source notebook, or automated test.	Use the Phase 7 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
105	databricks/notebooks/00_environment_check.py	8	Python module, script, Databricks source notebook, or automated test.	Use the Phase 8 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
106	databricks/README.md	8	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 8 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
107	databricks/notebooks/01_bronze_sqlserver.py	9	Python module, script, Databricks source notebook, or automated test.	Use the Phase 9 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
108	databricks/notebooks/02_bronze_postgres.py	9	Python module, script, Databricks source notebook, or automated test.	Use the Phase 9 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
109	databricks/notebooks/03_bronze_files.py	9	Python module, script, Databricks source notebook, or automated test.	Use the Phase 9 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
110	sql/databricks/bronze/01_bronze_audit_pattern.sql	9	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 9 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
111	databricks/notebooks/04_silver_customers.py	10	Python module, script, Databricks source notebook, or automated test.	Use the Phase 10 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
112	databricks/notebooks/05_silver_products.py	10	Python module, script, Databricks source notebook, or automated test.	Use the Phase 10 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
113	databricks/notebooks/06_silver_sales.py	10	Python module, script, Databricks source notebook, or automated test.	Use the Phase 10 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
114	databricks/notebooks/07_silver_digital.py	10	Python module, script, Databricks source notebook, or automated test.	Use the Phase 10 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
115	sql/databricks/silver/01_customer_quality_patterns.sql	10	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 10 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
116	sql/databricks/silver/02_sales_quality_patterns.sql	10	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 10 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
117	databricks/notebooks/08_gold_dimensions.py	11	Python module, script, Databricks source notebook, or automated test.	Use the Phase 11 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
118	docs/images/scd2_timeline.dot	11	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 11 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
119	docs/images/ scd2_timeline.mmd	11	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 11 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
120	docs/images/ scd2_timeline.svg	11	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
121	sql/databricks/gold/ 01_scd2_merge_pattern.sql	11	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 11 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
122	databricks/notebooks/ 09_gold_facts.py	12	Python module, script, Databricks source notebook, or automated test.	Use the Phase 12 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
123	sql/databricks/gold/ 02_kpi_views.sql	12	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 12 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
124	databricks/notebooks/ 10_data_quality.py	13	Python module, script, Databricks source notebook, or automated test.	Use the Phase 13 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
125	datasets/expected_quality_issues/.gitkeep	13	Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.	Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
126	sql/databricks/quality/01_quality_gate.sql	13	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 13 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
127	tests/data_quality/test_sample_quality_fixture.py	13	Python module, script, Databricks source notebook, or automated test.	Use the Phase 13 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
128	tests/reconciliation/.gitkeep	13	Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.	Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
129	tests/reconciliation/test_sample_reconciliation.py	13	Python module, script, Databricks source notebook, or automated test.	Use the Phase 13 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
130	databricks/jobs/job_definition.example.yml	14	YAML configuration for generation, logging, Databricks Jobs, or GitHub Actions.	Use the Phase 14 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
131	databricks/jobs/job_task_map.md	14	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 14 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
132	docs/images/jobs_dependency.dot	14	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 14 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
133	docs/images/jobs_dependency.mmd	14	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Use the Phase 14 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
134	docs/images/jobs_dependency.svg	14	Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.	Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
135	scripts/loading/load_gold_to_sqlserver.py	15	Python module, script, Databricks source notebook, or automated test.	Use the Phase 15 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
136	sql/sqlserver/validation/02_validate_warehouse.sql	15	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 15 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
137	sql/sqlserver/warehouse/01_create_warehouse.sql	15	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 15 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
138	sql/sqlserver/warehouse/02_create_staging_tables.sql	15	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 15 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
139	sql/sqlserver/warehouse/03_semantic_views.sql	15	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 15 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
140	docs/kpi_definitions.csv	16	Structured mapping, definition, metadata, or small fictional data fixture.	Build rows in VS Code or a spreadsheet-like editor while preserving UTF-8 CSV structure; validate headers, row counts, duplicates, and Git diff.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
141	docs/kpi_definitions.md	16	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 16 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
142	powerbi/measures.dax	16	Power BI DAX measure definitions.	Use the Phase 16 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
143	powerbi/semantic_model.md	16	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 16 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
144	powerbi/dashboard_requirements.md	17	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 17 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
145	powerbi/README.md	17	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 17 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
146	sql/sqlserver/analytics/20_analyst_questions.sql	17	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 17 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
147	sql/sqlserver/analyt-ics/kpi_validation_queries.sql	17	SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.	Use the Phase 17 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
148	docs/security_and_cost.md	18	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 18 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
149	scripts/validation/scan_secrets.py	18	Python module, script, Databricks source notebook, or automated test.	Use the Phase 18 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
150	.github/ISSUE_TEMPLATE/bug_report.md	19	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
151	.github/ISSUE_TEMPLATE/learning_task.md	19	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
152	.github/pull_request_template.md	19	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
153	.github/workflows/validate_project.yml	19	YAML configuration for generation, logging, Databricks Jobs, or GitHub Actions.	Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.



#	Path	Phase	Responsibility	Creation/ownership route	Git rule
154	pyproject.toml	19	Project configuration, documentation, source, fixture, or support as-set.	Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
155	scripts/validation/check_required_files.py	19	Python module, script, Databricks source notebook, or automated test.	Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
156	scripts/validation/validate_markdown_links.py	19	Python module, script, Databricks source notebook, or automated test.	Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
157	scripts/validation/validate_notebooks.py	19	Python module, script, Databricks source notebook, or automated test.	Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
158	scripts/validation/validate_sql_dialects.py	19	Python module, script, Databricks source notebook, or automated test.	Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
159	tests/integration/.gitkeep	19	Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.	Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
160	tests/integration/test_repository_contract.py	19	Python module, script, Databricks source notebook, or automated test.	Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
161	datasets/README.md	20	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 20 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
162	docs/claim_to_evidence_map.md	20	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 20 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
163	docs/demo_video_script.md	20	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 20 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
164	docs/images/README.md	20	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 20 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
165	docs/images/screenshots/.gitkeep	20	Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.	Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
166	docs/linkedin_post.md	20	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 20 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
167	docs/repository_baseline/README.original.md	20	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 20 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

#	Path	Phase	Responsibility	Creation/ownership route	Git rule
168	docs/re-sume_bullets.md	20	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 20 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
169	docs/data_lineage.md	21	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 21 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
170	docs/interview_walkthrough.md	21	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 21 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
171	docs/optional_extensions.md	22	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 22 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.
172	docs/runbook.md	22	Markdown documentation, requirement, mapping, runbook, evidence, or guide.	Use the Phase 22 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.	Track when the file is reviewed, non-secret, and intentionally part of the reproducible project.

## File responsibility 1 — docs/architecture.md

**Phase introduced:** Phase 0.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 0 folder/file tutorial: create the parent only when needed, create the exact filename in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing

anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 0, Lesson 0.3.

[Back to the file responsibility index](#)

## File responsibility 2 — .gitignore

---

**Phase introduced:** Phase 1.

**Parent folder:** ..

**Extension/type:** (no extension).

**Purpose:** Repository safety/configuration file.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 1, Lesson 1.3.

[Back to the file responsibility index](#)

## File responsibility 3 — config/logging.yml

---

**Phase introduced:** Phase 1.

**Parent folder:** config.

**Extension/type:** .yml.

**Purpose:** YAML configuration for generation, logging, Databricks Jobs, or GitHub Actions.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 4 — databricks/notebooks/11\_publish\_gold.py

---

**Phase introduced:** Phase 1.

**Parent folder:** databricks/notebooks.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing

anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 5 — databricks/notebooks/12\_demo\_queries.py

---

**Phase introduced:** Phase 1.

**Parent folder:** databricks/notebooks.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 6 — datasets/raw/files/.gitkeep

---

**Phase introduced:** Phase 1.

**Parent folder:** datasets/raw/files.

**Extension/type:** (no extension).

**Purpose:** Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.

**Creation/ownership route:** Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 7 — datasets/raw/postgres/.gitkeep

---

**Phase introduced:** Phase 1.

**Parent folder:** datasets/raw/postgres.

**Extension/type:** (no extension).

**Purpose:** Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.

**Creation/ownership route:** Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate

the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 8 — datasets/raw/sqlserver/.gitkeep

---

**Phase introduced:** Phase 1.

**Parent folder:** datasets/raw/sqlserver.

**Extension/type:** (no extension).

**Purpose:** Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.

**Creation/ownership route:** Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 9 — docs/images/cloud\_connected\_flow.dot

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .dot.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 10 — docs/images/cloud\_connected\_flow.mmd

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .mmd.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 11 — docs/images/cloud\_connected\_flow.svg

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .svg.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 12 — docs/images/gold\_star\_schema.dot

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .dot.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 13 — docs/images/gold\_star\_schema.mmd

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .mmd.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture

text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 14 — docs/images/gold\_star\_schema.svg

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .svg.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 15 — docs/images/lineage.dot

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .dot.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)



## File responsibility 16 — docs/images/lineage.mmd

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .mmd.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 17 — docs/images/lineage.svg

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .svg.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 18 — docs/images/local\_first\_flow.dot

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .dot.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 19 — docs/images/local\_first\_flow.mmd

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .mmd.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 20 — docs/images/local\_first\_flow.svg

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .svg.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 21 — docs/images/overall\_architecture.dot

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .dot.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 22 — docs/images/overall\_architecture.mmd

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .mmd.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 23 — docs/images/overall\_architecture.svg

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .svg.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 24 — docs/images/postgres\_erd.dot

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .dot.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified

work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 25 — docs/images/postgres\_erd.mmd

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .mmd.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 26 — docs/images/postgres\_erd.svg

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .svg.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 27 — docs/images/sqlserver\_erd.dot

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .dot.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 28 — docs/images/sqlserver\_erd.mmd

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .mmd.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 29 — docs/images/sqlserver\_erd.svg

---

**Phase introduced:** Phase 1.

**Parent folder:** docs/images.

**Extension/type:** .svg.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 30 — docs/repository\_current\_state.md

---

**Phase introduced:** Phase 1.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 1, Lesson 1.3.

[Back to the file responsibility index](#)

## File responsibility 31 — docs/repository\_migration\_plan.md

---

**Phase introduced:** Phase 1.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 1, Lesson 1.3.

[Back to the file responsibility index](#)

## File responsibility 32 — docs/testing\_strategy.md

---

**Phase introduced:** Phase 1.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)



## File responsibility 33 — LICENSE

---

**Phase introduced:** Phase 1.

**Parent folder:** ..

**Extension/type:** (no extension).

**Purpose:** Project configuration, documentation, source, fixture, or support asset.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 34 — logs/.gitkeep

---

**Phase introduced:** Phase 1.

**Parent folder:** logs.

**Extension/type:** (no extension).

**Purpose:** Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.

**Creation/ownership route:** Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 35 — Makefile-or-PowerShell-Commands.md

---

**Phase introduced:** Phase 1.

**Parent folder:** ..

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 36 — README.md

---

**Phase introduced:** Phase 1.

**Parent folder:** ..

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 1, Lesson 1.3.

[Back to the file responsibility index](#)

## File responsibility 37 — run\_artifacts/.gitkeep

---

**Phase introduced:** Phase 1.

**Parent folder:** run\_artifacts.

**Extension/type:** (no extension).

**Purpose:** Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.

**Creation/ownership route:** Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 38 — scripts/\_\_init\_\_.py

---

**Phase introduced:** Phase 1.

**Parent folder:** scripts.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)



## File responsibility 39 — scripts/loading/\_\_init\_\_.py

---

**Phase introduced:** Phase 1.

**Parent folder:** scripts/loading.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 40 — scripts/setup/\_\_init\_\_.py

---

**Phase introduced:** Phase 1.

**Parent folder:** scripts/setup.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 41 — scripts/setup/create\_project\_folders.ps1

---

**Phase introduced:** Phase 1.

**Parent folder:** scripts/setup.

**Extension/type:** .ps1.

**Purpose:** Project configuration, documentation, source, fixture, or support asset.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

## File responsibility 42 — scripts/utilities/\_\_init\_\_.py

---

**Phase introduced:** Phase 1.

**Parent folder:** scripts/utilities.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 43 — scripts/utilities/config.py

---

**Phase introduced:** Phase 1.

**Parent folder:** scripts/utilities.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 44 — scripts/utilities/logging\_utils.py

---

**Phase introduced:** Phase 1.

**Parent folder:** scripts/utilities.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 45 — scripts/validation/\_\_init\_\_.py

---

**Phase introduced:** Phase 1.

**Parent folder:** scripts/validation.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 1 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 46 — config/dev.example.yml

---

**Phase introduced:** Phase 2.

**Parent folder:** config.

**Extension/type:** .yml.

**Purpose:** YAML configuration for generation, logging, Databricks Jobs, or GitHub Actions.

**Creation/ownership route:** Use the Phase 2 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 2, Lesson 2.3.

[Back to the file responsibility index](#)

## File responsibility 47 — requirements-dev.txt

---

**Phase introduced:** Phase 2.

**Parent folder:** ..

**Extension/type:** .txt.

**Purpose:** Project configuration, documentation, source, fixture, or support asset.

**Creation/ownership route:** Use the Phase 2 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing

anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 2, Lesson 2.3.

[Back to the file responsibility index](#)

## File responsibility 48 — requirements.txt

---

**Phase introduced:** Phase 2.

**Parent folder:** ..

**Extension/type:** .txt.

**Purpose:** Project configuration, documentation, source, fixture, or support asset.

**Creation/ownership route:** Use the Phase 2 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 2, Lesson 2.3.

[Back to the file responsibility index](#)

## File responsibility 49 — scripts/setup/verify\_local\_tools.ps1

---

**Phase introduced:** Phase 2.

**Parent folder:** scripts/setup.

**Extension/type:** .ps1.

**Purpose:** Project configuration, documentation, source, fixture, or support asset.

**Creation/ownership route:** Use the Phase 2 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 2, Lesson 2.3.

[Back to the file responsibility index](#)

## File responsibility 50 — docs/assumption\_log.md

---

**Phase introduced:** Phase 3.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 3 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing

anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 3, Lesson 3.3.

[Back to the file responsibility index](#)

## File responsibility 51 — docs/business\_requirements.md

---

**Phase introduced:** Phase 3.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 3 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 3, Lesson 3.3.

[Back to the file responsibility index](#)

## File responsibility 52 — docs/risk\_register.md

---

**Phase introduced:** Phase 3.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 3 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 3, Lesson 3.3.

[Back to the file responsibility index](#)

## File responsibility 53 — docs/data\_dictionary.csv

---

**Phase introduced:** Phase 4.

**Parent folder:** docs.

**Extension/type:** .csv.

**Purpose:** Structured mapping, definition, metadata, or small fictional data fixture.

**Creation/ownership route:** Build rows in VS Code or a spreadsheet-like editor while preserving UTF-8 CSV structure; validate headers, row counts, duplicates, and Git diff.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 4, Lesson 4.3.

## File responsibility 54 — docs/data\_dictionary.md

---

**Phase introduced:** Phase 4.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 55 — docs/source\_to\_target\_mapping.csv

---

**Phase introduced:** Phase 4.

**Parent folder:** docs.

**Extension/type:** .csv.

**Purpose:** Structured mapping, definition, metadata, or small fictional data fixture.

**Creation/ownership route:** Build rows in VS Code or a spreadsheet-like editor while preserving UTF-8 CSV structure; validate headers, row counts, duplicates, and Git diff.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 4, Lesson 4.3.

[Back to the file responsibility index](#)

## File responsibility 56 — docs/source\_to\_target\_mapping.md

---

**Phase introduced:** Phase 4.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.



## File responsibility 57 — sql/postgres/source/00\_create\_database.sql

---

**Phase introduced:** Phase 4.

**Parent folder:** sql/postgres/source.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 4, Lesson 4.3.

[Back to the file responsibility index](#)

## File responsibility 58 — sql/postgres/source/01\_create\_digital\_schema.sql

---

**Phase introduced:** Phase 4.

**Parent folder:** sql/postgres/source.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 4, Lesson 4.3.

[Back to the file responsibility index](#)

## File responsibility 59 — sql/postgres/source/90\_development\_reset.sql

---

**Phase introduced:** Phase 4.

**Parent folder:** sql/postgres/source.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

## File responsibility 60 — sql/sqlserver/source/01\_create\_erp\_database.sql

---

**Phase introduced:** Phase 4.

**Parent folder:** sql/sqlserver/source.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 4, Lesson 4.3.

[Back to the file responsibility index](#)

## File responsibility 61 — sql/sqlserver/source/02\_create\_erp\_tables.sql

---

**Phase introduced:** Phase 4.

**Parent folder:** sql/sqlserver/source.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 4, Lesson 4.3.

[Back to the file responsibility index](#)

## File responsibility 62 — sql/sqlserver/source/90\_development\_reset.sql

---

**Phase introduced:** Phase 4.

**Parent folder:** sql/sqlserver/source.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 4 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.



## File responsibility 63 — config/data\_generation.yml

---

**Phase introduced:** Phase 5.

**Parent folder:** config.

**Extension/type:** .yaml.

**Purpose:** YAML configuration for generation, logging, Databricks Jobs, or GitHub Actions.

**Creation/ownership route:** Use the Phase 5 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 5, Lesson 5.3.

[Back to the file responsibility index](#)

## File responsibility 64 — datasets/sample/files/promotion\_calendar.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/files.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 65 — datasets/sample/files/returns.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/files.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 66 — datasets/sample/files/shipment\_tracking\_events.jsonl

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/files.

**Extension/type:** .jsonl.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 67 — datasets/sample/files/supplier\_cost\_updates.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/files.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 68 — datasets/sample/metadata/batch\_manifest.json

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/metadata.

**Extension/type:** .json.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 69 — datasets/sample/metadata/expected\_quality\_issues.json

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/metadata.

**Extension/type:** .json.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 70 — datasets/sample/metadata/generation\_summary.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/metadata.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 71 — datasets/sample/metadata/validation\_results.json

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/metadata.

**Extension/type:** .json.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 72 — datasets/sample/postgres/campaign\_touchpoints.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/postgres.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 73 — datasets/sample/postgres/campaigns.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/postgres.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 74 — datasets/sample/postgres/marketing\_spend.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/postgres.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 75 — datasets/sample/postgres/web\_events.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/postgres.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 76 — datasets/sample/postgres/web\_sessions.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/postgres.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 77 — datasets/sample/postgres/web\_users.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/postgres.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 78 — datasets/sample/sqlserver/addresses.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/sqlserver.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 79 — datasets/sample/sqlserver/customers.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/sqlserver.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)



## File responsibility 80 — datasets/sample/sqlserver/employees.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/sqlserver.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 81 — datasets/sample/sqlserver/inventory\_snapshots.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/sqlserver.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 82 — datasets/sample/sqlserver/inventory\_transactions.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/sqlserver.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 83 — datasets/sample/sqlserver/order\_items.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/sqlserver.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 84 — datasets/sample/sqlserver/orders.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/sqlserver.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 85 — datasets/sample/sqlserver/payments.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/sqlserver.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)



## File responsibility 86 — datasets/sample/sqlserver/product\_categories.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/sqlserver.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 87 — datasets/sample/sqlserver/products.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/sqlserver.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 88 — datasets/sample/sqlserver/shipments.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/sqlserver.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 89 — datasets/sample/sqlserver/stores.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/sqlserver.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 90 — datasets/sample/sqlserver/suppliers.csv

---

**Phase introduced:** Phase 5.

**Parent folder:** datasets/sample/sqlserver.

**Extension/type:** .csv.

**Purpose:** Small fictional sample fixture used by tests and examples; generated/maintained separately from full raw batches.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 91 — scripts/data\_generation/\_\_init\_\_.py

---

**Phase introduced:** Phase 5.

**Parent folder:** scripts/data\_generation.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 5 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 92 — scripts/data\_generation/generate\_northstar\_data.py

---

**Phase introduced:** Phase 5.

**Parent folder:** scripts/data\_generation.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 5 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 5, Lesson 5.3.

[Back to the file responsibility index](#)

## File responsibility 93 — scripts/data\_generation/README.md

---

**Phase introduced:** Phase 5.

**Parent folder:** scripts/data\_generation.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 5 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 94 — scripts/validation/validate\_generated\_data.py

---

**Phase introduced:** Phase 5.

**Parent folder:** scripts/validation.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 5 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 5, Lesson 5.3.

[Back to the file responsibility index](#)

## File responsibility 95 — tests/unit/test\_generator.py

---

**Phase introduced:** Phase 5.

**Parent folder:** tests/unit.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 5 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 5, Lesson 5.3.

[Back to the file responsibility index](#)

## File responsibility 96 — .env.example

---

**Phase introduced:** Phase 6.

**Parent folder:** ..

**Extension/type:** .example.

**Purpose:** Repository safety/configuration file.

**Creation/ownership route:** Use the Phase 6 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 6, Lesson 6.3.

[Back to the file responsibility index](#)

## File responsibility 97 — scripts/loading/load\_postgres\_source.py

---

**Phase introduced:** Phase 6.

**Parent folder:** scripts/loading.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 6 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 6, Lesson 6.3.

[Back to the file responsibility index](#)

## File responsibility 98 — scripts/loading/load\_sqlserver\_source.py

---

**Phase introduced:** Phase 6.

**Parent folder:** scripts/loading.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 6 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 6, Lesson 6.3.

[Back to the file responsibility index](#)

## File responsibility 99 — sql/postgres/validation/01\_validate\_digital\_source.sql

---

**Phase introduced:** Phase 6.

**Parent folder:** sql/postgres/validation.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 6 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 6, Lesson 6.3.

[Back to the file responsibility index](#)

## File responsibility 100 — sql/sqlserver/validation/01\_validate\_erp\_source.sql

---

**Phase introduced:** Phase 6.

**Parent folder:** sql/sqlserver/validation.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 6 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 6, Lesson 6.3.

[Back to the file responsibility index](#)

## File responsibility 101 — scripts/extraction/\_\_init\_\_.py

---

**Phase introduced:** Phase 7.

**Parent folder:** scripts/extraction.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 7 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 102 — scripts/extraction/extract\_local\_sources.py

---

**Phase introduced:** Phase 7.

**Parent folder:** scripts/extraction.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 7 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 7, Lesson 7.3.

[Back to the file responsibility index](#)

## File responsibility 103 — scripts/utilities/checksums.py

---

**Phase introduced:** Phase 7.

**Parent folder:** scripts/utilities.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 7 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 7, Lesson 7.3.

[Back to the file responsibility index](#)



## File responsibility 104 — scripts/utilities/database.py

---

**Phase introduced:** Phase 7.

**Parent folder:** scripts/utilities.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 7 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 7, Lesson 7.3.

[Back to the file responsibility index](#)

## File responsibility 105 — databricks/notebooks/00\_environment\_check.py

---

**Phase introduced:** Phase 8.

**Parent folder:** databricks/notebooks.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 8 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 8, Lesson 8.3.

[Back to the file responsibility index](#)

## File responsibility 106 — databricks/README.md

---

**Phase introduced:** Phase 8.

**Parent folder:** databricks.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 8 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 8, Lesson 8.3.

[Back to the file responsibility index](#)

## File responsibility 107 — databricks/notebooks/01\_bronze\_sqlserver.py

---

**Phase introduced:** Phase 9.

**Parent folder:** databricks/notebooks.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 9 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 9, Lesson 9.3.

[Back to the file responsibility index](#)

## File responsibility 108 — databricks/notebooks/02\_bronze\_postgres.py

---

**Phase introduced:** Phase 9.

**Parent folder:** databricks/notebooks.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 9 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 9, Lesson 9.3.

[Back to the file responsibility index](#)

## File responsibility 109 — databricks/notebooks/03\_bronze\_files.py

---

**Phase introduced:** Phase 9.

**Parent folder:** databricks/notebooks.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 9 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 9, Lesson 9.3.

[Back to the file responsibility index](#)



## File responsibility 110 — sql/databricks/bronze/01\_bronze\_audit\_pattern.sql

---

**Phase introduced:** Phase 9.

**Parent folder:** sql/databricks/bronze.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 9 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 9, Lesson 9.3.

[Back to the file responsibility index](#)

## File responsibility 111 — databricks/notebooks/04\_silver\_customers.py

---

**Phase introduced:** Phase 10.

**Parent folder:** databricks/notebooks.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 10 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 10, Lesson 10.3.

[Back to the file responsibility index](#)

## File responsibility 112 — databricks/notebooks/05\_silver\_products.py

---

**Phase introduced:** Phase 10.

**Parent folder:** databricks/notebooks.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 10 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 10, Lesson 10.3.

[Back to the file responsibility index](#)

## File responsibility 113 — databricks/notebooks/06\_silver\_sales.py

---

**Phase introduced:** Phase 10.

**Parent folder:** databricks/notebooks.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 10 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 10, Lesson 10.3.

[Back to the file responsibility index](#)

## File responsibility 114 — databricks/notebooks/07\_silver\_digital.py

---

**Phase introduced:** Phase 10.

**Parent folder:** databricks/notebooks.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 10 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 10, Lesson 10.3.

[Back to the file responsibility index](#)

## File responsibility 115 — sql/databricks/silver/01\_customer\_quality\_patterns.sql

---

**Phase introduced:** Phase 10.

**Parent folder:** sql/databricks/silver.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 10 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 10, Lesson 10.3.

[Back to the file responsibility index](#)

## File responsibility 116 — sql/databricks/silver/02\_sales\_quality\_patterns.sql

---

**Phase introduced:** Phase 10.

**Parent folder:** sql/databricks/silver.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 10 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 10, Lesson 10.3.

[Back to the file responsibility index](#)

## File responsibility 117 — databricks/notebooks/08\_gold\_dimensions.py

---

**Phase introduced:** Phase 11.

**Parent folder:** databricks/notebooks.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 11 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 11, Lesson 11.3.

[Back to the file responsibility index](#)

## File responsibility 118 — docs/images/scd2\_timeline.dot

---

**Phase introduced:** Phase 11.

**Parent folder:** docs/images.

**Extension/type:** .dot.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 11 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 119 — docs/images/scd2\_timeline.mmd

---

**Phase introduced:** Phase 11.

**Parent folder:** docs/images.

**Extension/type:** .mmd.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 11 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 120 — docs/images/scd2\_timeline.svg

---

**Phase introduced:** Phase 11.

**Parent folder:** docs/images.

**Extension/type:** .svg.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 121 — sql/databricks/gold/01\_scd2\_merge\_pattern.sql

---

**Phase introduced:** Phase 11.

**Parent folder:** sql/databricks/gold.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 11 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 11, Lesson 11.3.

[Back to the file responsibility index](#)

## File responsibility 122 — databricks/notebooks/09\_gold\_facts.py

---

**Phase introduced:** Phase 12.

**Parent folder:** databricks/notebooks.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 12 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 12, Lesson 12.3.

[Back to the file responsibility index](#)

## File responsibility 123 — sql/databricks/gold/02\_kpi\_views.sql

---

**Phase introduced:** Phase 12.

**Parent folder:** sql/databricks/gold.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 12 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 12, Lesson 12.3.

[Back to the file responsibility index](#)

## File responsibility 124 — databricks/notebooks/10\_data\_quality.py

---

**Phase introduced:** Phase 13.

**Parent folder:** databricks/notebooks.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 13 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 13, Lesson 13.3.

[Back to the file responsibility index](#)

## File responsibility 125 — datasets/expected\_quality\_issues/.gitkeep

---

**Phase introduced:** Phase 13.

**Parent folder:** datasets/expected\_quality\_issues.

**Extension/type:** (no extension).

**Purpose:** Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.

**Creation/ownership route:** Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 126 — sql/databricks/quality/01\_quality\_gate.sql

---

**Phase introduced:** Phase 13.

**Parent folder:** sql/databricks/quality.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 13 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 13, Lesson 13.3.

[Back to the file responsibility index](#)

## File responsibility 127 — tests/data\_quality/test\_sample\_quality\_fixture.py

---

**Phase introduced:** Phase 13.

**Parent folder:** tests/data\_quality.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 13 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 13, Lesson 13.3.

[Back to the file responsibility index](#)



## File responsibility 128 — tests/reconciliation/.gitkeep

---

**Phase introduced:** Phase 13.

**Parent folder:** tests/reconciliation.

**Extension/type:** (no extension).

**Purpose:** Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.

**Creation/ownership route:** Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 129 — tests/reconciliation/test\_sample\_reconciliation.py

---

**Phase introduced:** Phase 13.

**Parent folder:** tests/reconciliation.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 13 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 13, Lesson 13.3.

[Back to the file responsibility index](#)

## File responsibility 130 — databricks/jobs/job\_definition.example.yml

---

**Phase introduced:** Phase 14.

**Parent folder:** databricks/jobs.

**Extension/type:** .yml.

**Purpose:** YAML configuration for generation, logging, Databricks Jobs, or GitHub Actions.

**Creation/ownership route:** Use the Phase 14 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 14, Lesson 14.3.

[Back to the file responsibility index](#)

## File responsibility 131 — databricks/jobs/job\_task\_map.md

---

**Phase introduced:** Phase 14.

**Parent folder:** databricks/jobs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 14 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 14, Lesson 14.3.

[Back to the file responsibility index](#)

## File responsibility 132 — docs/images/jobs\_dependency.dot

---

**Phase introduced:** Phase 14.

**Parent folder:** docs/images.

**Extension/type:** .dot.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 14 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 133 — docs/images/jobs\_dependency.mmd

---

**Phase introduced:** Phase 14.

**Parent folder:** docs/images.

**Extension/type:** .mmd.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Use the Phase 14 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.



## File responsibility 134 — docs/images/jobs\_dependency.svg

---

**Phase introduced:** Phase 14.

**Parent folder:** docs/images.

**Extension/type:** .svg.

**Purpose:** Architecture or lineage diagram source/render used for documentation evidence; create from verified architecture text and never fabricate execution results.

**Creation/ownership route:** Generated/exported by the phase tool or preserved from existing work; do not manually type generated bytes. Verify generator/export source, path, size, and content.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 135 — scripts/loading/load\_gold\_to\_sqlserver.py

---

**Phase introduced:** Phase 15.

**Parent folder:** scripts/loading.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 15 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 15, Lesson 15.3.

[Back to the file responsibility index](#)

## File responsibility 136 — sql/sqlserver/validation/02\_validate\_warehouse.sql

---

**Phase introduced:** Phase 15.

**Parent folder:** sql/sqlserver/validation.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 15 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 15, Lesson 15.3.

## File responsibility 137 — sql/sqlserver/warehouse/01\_create\_warehouse.sql

---

**Phase introduced:** Phase 15.

**Parent folder:** sql/sqlserver/warehouse.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 15 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 15, Lesson 15.3.

[Back to the file responsibility index](#)

## File responsibility 138 — sql/sqlserver/warehouse/02\_create\_staging\_tables.sql

---

**Phase introduced:** Phase 15.

**Parent folder:** sql/sqlserver/warehouse.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 15 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 15, Lesson 15.3.

[Back to the file responsibility index](#)

## File responsibility 139 — sql/sqlserver/warehouse/03\_semantic\_views.sql

---

**Phase introduced:** Phase 15.

**Parent folder:** sql/sqlserver/warehouse.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 15 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 15, Lesson 15.3.

[Back to the file responsibility index](#)

## File responsibility 140 — docs/kpi\_definitions.csv

---

**Phase introduced:** Phase 16.

**Parent folder:** docs.

**Extension/type:** .csv.

**Purpose:** Structured mapping, definition, metadata, or small fictional data fixture.

**Creation/ownership route:** Build rows in VS Code or a spreadsheet-like editor while preserving UTF-8 CSV structure; validate headers, row counts, duplicates, and Git diff.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 16, Lesson 16.3.

[Back to the file responsibility index](#)

## File responsibility 141 — docs/kpi\_definitions.md

---

**Phase introduced:** Phase 16.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 16 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 142 — powerbi/measures.dax

---

**Phase introduced:** Phase 16.

**Parent folder:** powerbi.

**Extension/type:** .dax.

**Purpose:** Power BI DAX measure definitions.

**Creation/ownership route:** Use the Phase 16 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 16, Lesson 16.3.

[Back to the file responsibility index](#)

## File responsibility 143 — powerbi/semantic\_model.md

---

**Phase introduced:** Phase 16.

**Parent folder:** powerbi.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 16 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 16, Lesson 16.3.

[Back to the file responsibility index](#)

## File responsibility 144 — powerbi/dashboard\_requirements.md

---

**Phase introduced:** Phase 17.

**Parent folder:** powerbi.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 17 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 17, Lesson 17.3.

[Back to the file responsibility index](#)

## File responsibility 145 — powerbi/README.md

---

**Phase introduced:** Phase 17.

**Parent folder:** powerbi.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 17 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 146 — sql/sqlserver/analytics/20\_analyst\_questions.sql

---

**Phase introduced:** Phase 17.

**Parent folder:** sql/sqlserver/analytics.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 17 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 17, Lesson 17.3.

[Back to the file responsibility index](#)

## File responsibility 147 — sql/sqlserver/analytics/kpi\_validation\_queries.sql

---

**Phase introduced:** Phase 17.

**Parent folder:** sql/sqlserver/analytics.

**Extension/type:** .sql.

**Purpose:** SQL definition, transformation pattern, validation query, or analyst query for the named dialect/location.

**Creation/ownership route:** Use the Phase 17 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 17, Lesson 17.3.

[Back to the file responsibility index](#)

## File responsibility 148 — docs/security\_and\_cost.md

---

**Phase introduced:** Phase 18.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 18 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 18, Lesson 18.3.

[Back to the file responsibility index](#)

## File responsibility 149 — scripts/validation/scan\_secrets.py

---

**Phase introduced:** Phase 18.

**Parent folder:** scripts/validation.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 18 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 18, Lesson 18.3.

[Back to the file responsibility index](#)

## File responsibility 150 — .github/ISSUE\_TEMPLATE/bug\_report.md

---

**Phase introduced:** Phase 19.

**Parent folder:** .github/ISSUE\_TEMPLATE.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 151 — .github/ISSUE\_TEMPLATE/learning\_task.md

---

**Phase introduced:** Phase 19.

**Parent folder:** .github/ISSUE\_TEMPLATE.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)



## File responsibility 152 — .github/pull\_request\_template.md

---

**Phase introduced:** Phase 19.

**Parent folder:** .github.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 153 — .github/workflows/validate\_project.yml

---

**Phase introduced:** Phase 19.

**Parent folder:** .github/workflows.

**Extension/type:** .yml.

**Purpose:** YAML configuration for generation, logging, Databricks Jobs, or GitHub Actions.

**Creation/ownership route:** Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 19, Lesson 19.3.

[Back to the file responsibility index](#)

## File responsibility 154 — pyproject.toml

---

**Phase introduced:** Phase 19.

**Parent folder:** ..

**Extension/type:** .toml.

**Purpose:** Project configuration, documentation, source, fixture, or support asset.

**Creation/ownership route:** Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 19, Lesson 19.3.

[Back to the file responsibility index](#)

## File responsibility 155 — scripts/validation/check\_required\_files.py

---

**Phase introduced:** Phase 19.

**Parent folder:** scripts/validation.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 19, Lesson 19.3.

[Back to the file responsibility index](#)

## File responsibility 156 — scripts/validation/validate\_markdown\_links.py

---

**Phase introduced:** Phase 19.

**Parent folder:** scripts/validation.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 19, Lesson 19.3.

[Back to the file responsibility index](#)

## File responsibility 157 — scripts/validation/validate\_notebooks.py

---

**Phase introduced:** Phase 19.

**Parent folder:** scripts/validation.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 19, Lesson 19.3.

[Back to the file responsibility index](#)



## File responsibility 158 — scripts/validation/validate\_sql\_dialects.py

---

**Phase introduced:** Phase 19.

**Parent folder:** scripts/validation.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 19, Lesson 19.3.

[Back to the file responsibility index](#)

## File responsibility 159 — tests/integration/.gitkeep

---

**Phase introduced:** Phase 19.

**Parent folder:** tests/integration.

**Extension/type:** (no extension).

**Purpose:** Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.

**Creation/ownership route:** Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 160 — tests/integration/test\_repository\_contract.py

---

**Phase introduced:** Phase 19.

**Parent folder:** tests/integration.

**Extension/type:** .py.

**Purpose:** Python module, script, Databricks source notebook, or automated test.

**Creation/ownership route:** Use the Phase 19 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 19, Lesson 19.3.

[Back to the file responsibility index](#)

## File responsibility 161 — datasets/README.md

---

**Phase introduced:** Phase 20.

**Parent folder:** datasets.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 20 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 162 — docs/claim\_to\_evidence\_map.md

---

**Phase introduced:** Phase 20.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 20 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 20, Lesson 20.3.

[Back to the file responsibility index](#)

## File responsibility 163 — docs/demo\_video\_script.md

---

**Phase introduced:** Phase 20.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 20 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 20, Lesson 20.3.

[Back to the file responsibility index](#)

## File responsibility 164 — docs/images/README.md

---

**Phase introduced:** Phase 20.

**Parent folder:** docs/images.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 20 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 165 — docs/images/screenshots/.gitkeep

---

**Phase introduced:** Phase 20.

**Parent folder:** docs/images/screenshots.

**Extension/type:** (no extension).

**Purpose:** Placeholder that allows Git to retain an otherwise empty directory; remove only when a real tracked file makes it unnecessary.

**Creation/ownership route:** Create only when the folder must exist before a real tracked file; verify with Git status and remove after a meaningful tracked child exists.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 166 — docs/linkedin\_post.md

---

**Phase introduced:** Phase 20.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 20 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 20, Lesson 20.3.

[Back to the file responsibility index](#)

## File responsibility 167 — docs/repository\_baseline/README.original.md

---

**Phase introduced:** Phase 20.

**Parent folder:** docs/repository\_baseline.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 20 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** This responsibility card is the construction record for a support/generated file not treated as a core hand-written source file.

[Back to the file responsibility index](#)

## File responsibility 168 — docs/resume\_bullets.md

---

**Phase introduced:** Phase 20.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 20 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 20, Lesson 20.3.

[Back to the file responsibility index](#)

## File responsibility 169 — docs/data\_lineage.md

---

**Phase introduced:** Phase 21.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 21 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 21, Lesson 21.3.

[Back to the file responsibility index](#)

## File responsibility 170 — docs/interview\_walkthrough.md

---

**Phase introduced:** Phase 21.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 21 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 21, Lesson 21.3.

[Back to the file responsibility index](#)

## File responsibility 171 — docs/optional\_extensions.md

---

**Phase introduced:** Phase 22.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 22 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 22, Lesson 22.3.

[Back to the file responsibility index](#)

## File responsibility 172 — docs/runbook.md

---

**Phase introduced:** Phase 22.

**Parent folder:** docs.

**Extension/type:** .md.

**Purpose:** Markdown documentation, requirement, mapping, runbook, evidence, or guide.

**Creation/ownership route:** Use the Phase 22 folder/file tutorial: create the parent only when needed, create the exact file-name in VS Code or PowerShell, build content in explained sections, save, validate, inspect Git diff, and commit only verified work.

**Dependencies:** Follow the phase order and do not create this path before its first real input exists.

**Verification:** Confirm exact path, extension, expected producer/editor, content shape, non-secret status, and Git behavior. For generated content, compare manifest/checksum or export source rather than hand-editing it.

**If missing:** The related test, transformation, documentation, job, report, or reproducibility claim is incomplete; investigate the producer and phase gate rather than inventing an empty placeholder.

**If already present:** Preserve and inspect it. Compare current Git history and contents with the workbook before replacing anything.

**Detailed route:** Detailed file introduction and optional reference answer appear in Phase 22, Lesson 22.3.

[Back to the file responsibility index](#)

# Final definition of done

[Back to the master table of contents](#)

Do not fill these statuses automatically. For every row, require: object exists; you personally created/configured it; code was reviewed; command/run occurred; output was inspected; tests passed; error handling was checked; screenshot exists; Git commit exists; remote branch contains the work; documentation is accurate; no credentials are committed; and you can explain it in your own words.

## Repository and environment audit

Component	Exists	Created/configured by me	Verified output	Failure tested	Screenshot	Commit/push	No secrets	I can explain it
Correct local clone and remote	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Feature branch and targeted history	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Verified tool versions	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
.venv and dependencies	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Secret/ignore controls	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

## SQL Server source tables

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
product_categories	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
suppliers	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
products	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
stores	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
employees	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
customers	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
addresses	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
orders	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
order_items	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
payments	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
shipments	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
inventory_transactions	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
inventory_snapshots	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

## PostgreSQL digital source tables

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
web_users	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
campaigns	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
web_sessions	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
web_events	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
campaign_touchpoints	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
marketing_spend	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

## File feeds

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
returns.csv	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
supplier_cost_updates.csv	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
promotion_calendar.csv	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
shipment_tracking_events.jsonl	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

## Bronze Delta tables

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
bronze.erp_product_categories	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.erp_suppliers	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.erp_products	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.erp_stores	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.erp_employees	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.erp_customers	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.erp_addresses	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.erp_orders	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.erp_order_items	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.erp_payments	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.erp_shipments	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.erp_inventory_transactions	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.erp_inventory_snapshots	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
bronze.digital_web_users	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.digital_campaigns	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.digital_web_sessions	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.digital_web_events	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.digital_campaign_touchpoints	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.digital_marketing_spend	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.file_returns	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.file_supplier_cost_updates	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.file_promotion_calendar	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze.file_shipping_tracking_events	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

## Silver conformed entities and quarantines

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
silver.addresses	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
silver.campaign_touchpoints	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
silver.campaigns	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
silver.customers	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
silver.employees	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
silver.inventory_snapshots	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
silver.inventory_transactions	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]



Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
sil-ver.market-ing_spend	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.order_items	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.orders	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.payments	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.product_categories	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.products	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.promotions	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.re-returns	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.sales	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.shipment_tracking_events	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.shipments	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.stores	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.supplier_cost_updates	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.suppliers	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.web_events	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.web_sessions	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
sil-ver.web_users	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

## Gold dimensions

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
gold.dim_date	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
gold.dim_customer	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
gold.dim_product	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
gold.dim_store	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
gold.dim_employee	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
gold.dim_supplier	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
gold.dim_promotion	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
gold.dim_channel	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
gold.dim_campaign	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

## Gold facts

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
gold.fact_sales	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
gold.fact_returns	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
gold.fact_inventory_snapshot	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
gold.fact_shipments	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
gold.fact_web_sessions	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
gold.fact_marketing_spend	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

## SQL Server warehouse dimensions/facts

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
dw.dim_date	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
dw.dim_customer	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
dw.dim_product	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
dw.dim_store	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
dw.dim_employee	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
dw.dim_supplier	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
dw.dim_promotion	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
dw.dim_channel	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
dw.dim_campaign	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
dw.fact_sales	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
dw.fact_returns	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
dw.fact_inventory_snapshot	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
dw.fact_shipments	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
dw.fact_web_sessions	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
dw.fact_marketing_spend	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

## Power BI measures

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
Total Revenue	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Gross Profit	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Gross Margin %	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Promotion Gross Profit	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Orders	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Units Sold	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Average Order Value	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Returned Units	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Return Rate	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Delivered Shipments	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
On-Time Shipments	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
On-Time Delivery %	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Marketing Spend	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Web Sessions	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Converted Sessions	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Conversion Rate	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Attributed Revenue	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Return on Ad Spend	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Purchasing Customers	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
New Customers	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Returning Customers	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
Inventory On Hand	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Stockout Risk Count	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

## Power BI report pages

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
Executive Overview	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Sales Performance	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Product and Category	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Store and Region	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Customer	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Marketing Funnel	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Inventory	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Shipping and Returns	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Data Quality	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

## Databricks Job tasks

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
environment_check	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze_sql_server	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze_postgres	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
bronze_files	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
silver_customers	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
silver_products	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
silver_sales	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
silver_digital	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
gold_dimensions	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
gold_facts	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
data_quality	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
publish_gold	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Object	Exists	Created by me	Code/design reviewed	Run/loaded	Output inspected	Test passed	Failure checked	Screenshot	Git/GitHub	Explain
demo_queries	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

## Cross-layer reconciliation audit

Boundary	Count reconciled	Amount/quantity reconciled	Checksum/manifest verified	Batch/run identified	Difference explained	Status
Generator → source databases	[ ]	[ ]	[ ]	[ ]	[ ]	[STATUS]
Source databases → local extracts	[ ]	[ ]	[ ]	[ ]	[ ]	[STATUS]
Extracts → Bronze	[ ]	[ ]	[ ]	[ ]	[ ]	[STATUS]
Bronze → Silver accepted/rejected	[ ]	[ ]	[ ]	[ ]	[ ]	[STATUS]
Silver → Gold dimensions/facts	[ ]	[ ]	[ ]	[ ]	[ ]	[STATUS]
Gold → exported files	[ ]	[ ]	[ ]	[ ]	[ ]	[STATUS]
Exported files → SQL Server warehouse	[ ]	[ ]	[ ]	[ ]	[ ]	[STATUS]
SQL Server warehouse → Power BI	[ ]	[ ]	n/a	[ ]	[ ]	[STATUS]

## Security, cost, CI, and documentation audit

- ☐ .env and live credentials are untracked and absent from history/evidence.
- ☐ Secret scanning and ignored-file checks pass.
- ☐ Databricks compute was stopped when not needed and cost-sensitive settings were reviewed.
- ☐ GitHub Actions ran on the intended branch/event and every log was read.
- ☐ A controlled CI failure was diagnosed, fixed, and rerun.
- ☐ README claims link to real evidence and use honest status words.
- ☐ Screenshots are redacted, named, committed deliberately, and placed in relevant sections.
- ☐ Architecture diagrams match the implemented path and show manual Track A boundaries.
- ☐ Limitations include synthetic data, batch latency, manual handoffs, tier variance, and infrastructure-free CI scope.
- ☐ Resume/LinkedIn/interview statements do not describe planned or reference code as completed work.

## Final user sign-off

**Date:** [ENTER]  
**Verified branch:** [ENTER]  
**Latest verified commit:** [ENTER]  
**Verified Databricks Job run ID:** [ENTER]  
**Verified publication batch ID:** [ENTER]  
**Power BI file/project path:** [ENTER]  
**Outstanding troubleshooting items:** [ENTER OR NONE]

**Statement in my own words:** [EXPLAIN THE FULL SYSTEM FROM SOURCE TO CONSUMPTION, INCLUDING TWO TRADEOFFS AND ONE FAILURE YOU FIXED]

# Final portfolio and interview audit

[Back to the master table of contents](#)

## Thirty-second explanation worksheet

**Business problem:** [ENTER]  
**Sources:** [ENTER]  
**Pipeline:** [ENTER]  
**Warehouse/semantic model:** [ENTER]  
**Quality evidence:** [ENTER]  
**Honest status/limitation:** [ENTER]

## Two-minute architecture explanation worksheet

Use: requirement → source placement → extraction/batch evidence → Bronze → Silver → Gold → quality gates → Job → warehouse publication → Power BI → Git/CI → limitation/next scale step.  
[WRITE AND PRACTICE YOUR ANSWER]

## Defend-every-decision worksheet

Decision	Alternatives	Why selected	Tradeoff	Evidence	What changes at 100× scale
Two source engines	[ENTER]	[ENTER]	[ENTER]	[ENTER]	[ENTER]
CSV/JSON Track A hand-off	[ENTER]	[ENTER]	[ENTER]	[ENTER]	[ENTER]
Deterministic synthetic data	[ENTER]	[ENTER]	[ENTER]	[ENTER]	[ENTER]
Bronze/Silver/Gold	[ENTER]	[ENTER]	[ENTER]	[ENTER]	[ENTER]
SCD2 customer/product	[ENTER]	[ENTER]	[ENTER]	[ENTER]	[ENTER]
Unknown member key 0	[ENTER]	[ENTER]	[ENTER]	[ENTER]	[ENTER]
Batch idempotency	[ENTER]	[ENTER]	[ENTER]	[ENTER]	[ENTER]
Blocking quality gate	[ENTER]	[ENTER]	[ENTER]	[ENTER]	[ENTER]
Manual Gold publication	[ENTER]	[ENTER]	[ENTER]	[ENTER]	[ENTER]
Power BI Import	[ENTER]	[ENTER]	[ENTER]	[ENTER]	[ENTER]
Infrastructure-free CI	[ENTER]	[ENTER]	[ENTER]	[ENTER]	[ENTER]

## Truthful résumé bullet gate

A bullet may use a completed-action verb only when the referenced component is VERIFIED BY USER and the claim-to-evidence map points to a real commit plus execution evidence. Planned or reference-only work must be described as planned, designed, prototyped, or omitted until verified.

## Final safe stopping point

The required path is complete only after every applicable definition-of-done row is manually updated. Preserve the verified branch and evidence. Optional extensions must begin from a new branch or isolated environment with prerequisites, cost, rollback, and success criteria written before implementation.